



Facultad de
Ciencias
UNAM

Almacenes y Minería de Datos

Diego Villaba

Tabla de contenidos

Prefacio	1
Temario	1
Requisitos previos	1
Agradecimientos	1
Autor	1
Cómo citar	1
 I. Entendimiento de Negocio	 3
 II. Fundamentos y Stakeholders	 5
1. Gestión de Requerimientos y Stakeholders	7
1.1. Identificación de Partes Interesadas (<i>Stakeholders</i>)	8
1.2. Análisis de Stakeholders: La Matriz de Poder e Interés	10
1.3. Definición de alcance y traducción de preguntas de negocio	10
1.4. Refinamiento iterativo mediante preguntas	11
1.5. Ejemplos de traducción de preguntas de negocio	13
1.6. El proceso de solicitud de datos (<i>Data Request</i>)	13
1.7. Cierre	14
1.8. Caso de Estudio Práctico: Alineación en TheLook eCommerce	14
1.9. Referencias	15
2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos	25
2.1. El Ciclo de Vida de los Datos	26
2.2. Fases Críticas de la Gestión de Datos	27
2.3. Actividad Guiada: Diseñar un Plan de Ciclo de Vida para TheLook eCommerce	28
2.4. Referencias	29
 III. Planeación del Proyecto	 39
3. Planeación Real de un Proyecto: El Business Understanding Canvas	41
3.1. Paso 1: La Bitácora de la Reunión con el Stakeholder	42
3.2. Paso 2: El Business Understanding Canvas (Una Página)	43
3.3. Paso 3: Preguntas de Investigación Priorizadas	44
3.4. Paso 4: Criterio de Éxito en Términos de Negocio	44
3.5. Paso 5: Supuestos y riesgos	45
3.6. Actividad: Construye tu Propio Business Understanding Canvas	48
3.7. Referencias	48

IV. Almacenes de Datos	59
V. Arquitectura de almacenes de Datos	61
4. Introducción	63
4.1. La Arquitectura en Ciencias de la Computación	63
4.2. Eje 1: El costo de leer datos — filas vs. columnas	63
4.3. Eje 2: El costo de la rigidez — datos homogéneos vs. heterogéneos	65
4.4. Eje 3: El costo de la consistencia — ACID vs. BASE	66
4.5. El sentido de ser de las arquitecturas de almacenamiento	67
5. Data Warehouse	69
5.1. Datos para la toma de decisiones	69
5.2. Data Warehouse	71
6. Arquitectura de un Data Warehouse	73
6.1. Modelo de 3 Capas	73
6.2. Data Mart	74
6.3. Data Warehouse vs Data Lake	75
7. Enfoques de implementación: Kimball vs Inmon	77
7.1. Enfoque Kimball (bottom-up, bus architecture)	77
7.2. Enfoque Inmon (top-down, EDW primero)	80
7.3. Cómo elegir adecuadamente entre Kimball e Inmon	80
8. Ejemplo práctico: Facultad de Ciencias	83
8.1. Fuentes de datos	83
8.2. Procesos de integración (ETL)	83
8.3. Consolidación en el Data Warehouse	84
8.4. Diagrama arquitectónico del DW académico	86
9. Conclusión	89
9.1. Recapitulación visual: del registro a la analítica	89
10. Data Lakehouse	91
10.1. Pregunta rectora	91
10.2. El problema antes de la definición	92
10.3. Dos caminos que deben representar el mismo negocio	92
10.4. Caso Fintech, fase 1: el Warehouse era suficiente	92
10.5. Año 4: el punto de quiebre	93
11. La base técnica del Lakehouse	95
11.1. Vocabulario mínimo	95
11.2. Una base física compartida todavía no es una tabla	96
11.3. La idea que debe sobrevivir	96
11.4. Responsabilidades lógicas de la arquitectura	96
11.5. Archivo, tabla y catálogo no son sinónimos	97
11.6. Cómo un snapshot oculta una escritura incompleta	98
11.7. Evolución de esquema sin reescribir toda la historia	98
11.8. Catálogo y gobierno: mapa, versión y acceso	98

12. Integrar calidad, cargas y productos de datos	99
12.1. Medallion: Bronze, Silver y Gold	99
12.2. Fase 2: una base gobernada y varios motores	99
12.3. Tecnologías abiertas y servicios administrados	99
13. Implementaciones en Google Cloud, AWS y Azure	101
13.1. Google Cloud: Cloud Storage, Iceberg y varios motores	101
13.2. AWS: S3, Iceberg, Glue y Lake Formation	101
13.3. Azure y Microsoft Fabric: OneLake, Delta y Medallion	102
14. El trabajo que no desaparece	103
14.1. El impuesto de los archivos pequeños	103
14.2. Una fuente compartida comparte el radio de impacto	103
15. Decidir, no evangelizar	105
15.1. Preguntas frecuentes y errores de recuperación	106
15.2. Exit ticket	107
16. Cheatsheet	109
16.1. Modelo mental en una página	109
16.2. Diagnóstico rápido	109
16.3. Lista mínima de implementación	110
17. Bibliografía y guías oficiales de implementación	111
17.1. Google Cloud	111
17.2. Amazon Web Services	111
17.3. Microsoft Azure y Fabric	111
VI. Modelado multidimensional	113
18. Modelado Multidimensional	115
18.1. Hechos y Dimensiones	115
18.2. Granularidad: la decisión de diseño más importante	117
18.3. Jerarquías en dimensiones y navegación analítica	119
18.4. Conclusión	119
19. Esquemas Dimensionales y Redundancia	121
19.1. Esquema de Estrella	121
19.2. Esquema de Copo de Nieve (Snowflake)	123
19.3. Redundancia controlada: qué cuesta realmente desnormalizar	126
19.4. Costo de consulta: joins y motores en la nube	126
19.5. Esquema de Constelación (Fact Constellation / Galaxy Schema)	127
19.6. Patrones de diseño para gestionar redundancia	129
19.7. Conclusión	129
20. Dimensiones Lentamente Cambiantes (SCD)	131
20.1. Tipos de SCD	131
20.2. Implementación con dbt	133
20.3. Pruebas de integridad	134
20.4. Comparación y criterio de elección	135

21. Ejemplo Completo: Café Norte — Análisis y Construcción de Data Marts (Kimball)	137
21.1. Propósito y supuestos	137
21.2. 1. Estructura esperada de archivos CSV	137
21.3. 2. Carga de datos y validación de esquema	138
21.4. 3. Homologación e integración de fuentes (Staging Integrado)	140
21.5. 4. Construcción de dimensiones conformadas	142
21.6. 5. Construcción de la tabla de hechos (fact_linea_pedido)	145
21.7. 6. Data Marts y respuestas a preguntas estratégicas	146
21.8. 7. Exportación de tablas: Dimensiones y Hechos (Data Mart físico)	149
21.9. 8. Discusión técnica: decisiones de diseño y extensiones	150
21.10. 9. Reproducibilidad y control de versiones	151
 VII. Integración ETL/ELT	 153
22. Introducción a PySpark y Fundamentos ETL	155
22.1. 1. ¿Qué es el ETL y por qué importa?	155
22.2. 2. Entorno de prácticas: Databricks Community Edition	156
22.3. 3. PySpark: Fundamentos	157
22.4. Referencias de este capítulo	159
23. Extracción y Fuentes Heterogéneas	161
23.1. 1. Extracción (<i>Extract</i>)	161
23.2. 2. Integración de fuentes heterogéneas: Data Lake → DW	163
23.3. Referencias de este capítulo	174
24. Transformación, Carga y Pipeline ETL	175
24.1. 1. Transformación (<i>Transform</i>)	175
24.2. 2. Transformaciones SQL con Spark	178
24.3. 3. Carga (<i>Load</i>)	179
24.4. 4. Pipeline ETL completo	180
24.5. Referencias de este capítulo	182
25. Prácticas ETL en Databricks	183
25.1. 1. Ejercicios	183
25.2. Referencias de este capítulo	189
26. Orquestación y Herramientas Modernas	191
26.1. 1. ETL moderno: dbt + PySpark	191
26.2. 2. Orquestación con Apache Airflow	192
26.3. 3. Resumen y mapa de herramientas	193
26.4. 4. Referencias y recursos adicionales	193
 VIII. Modelado para consumo	 195
27. Computación de cubos de datos OLAP	197
27.1. Del almacén de datos al análisis OLAP	197
27.2. Modelo multidimensional	198
27.3. Operaciones fundamentales en cubos OLAP	199
27.4. Cálculo manual de un cubo OLAP en un dataset pequeño	201

27.5. Ejemplo reproducible en Python	206
27.6. Implementación de operaciones OLAP	209
27.7. Exploración interactiva del cubo	211
27.8. Agregaciones y medidas derivadas	211
27.9. Ventajas, limitaciones y criterios de diseño	213
27.10 Buenas prácticas para documentación reproducible	213
27.11 Conclusiones	214
27.12 Referencias	215
28. Data Marts: del Almacén de Datos al Consumo por Área de Negocio	225
28.1. ¿Qué es un data mart?	225
28.2. Arquitecturas: dependiente, independiente e híbrida	226
28.3. Dimensiones conformadas y la matriz de bus	227
28.4. Diseño dimensional de un data mart	228
28.5. Ejemplo reproducible: construir el data mart en Python	228
28.6. El riesgo de los data marts independientes: dimensiones no conformadas	231
28.7. Ventajas, limitaciones y buenas prácticas	232
28.8. De data marts a productos de datos	232
28.9. Conclusiones	233
28.10 Referencias	233
29. Consumo Moderno de Data Marts: APIs y Capas Semánticas	243
29.1. Del acceso directo al contrato de servicio	243
29.2. REST sobre un data mart	243
29.3. GraphQL sobre un data mart	246
29.4. Capa semántica: métricas como código	249
29.5. Un mismo dato, tres contratos	250
29.6. Seguridad y gobierno en la capa de API	250
29.7. Ventajas, limitaciones y buenas prácticas	251
29.8. Conclusiones	251
29.9. Referencias	252
IX. Marco Metodológico de Minería de Datos	263
30. Introducción a la Minería de Datos y sus Frameworks de Proceso	265
30.1. La pregunta que abre el capítulo	265
30.2. Recuperación de datos versus minería de datos	265
30.3. Correlación y causalidad: la primera trampa	266
30.4. Las tareas centrales de la minería de datos	266
30.5. Frameworks de proceso: cuatro formas de responder la misma pregunta	268
30.6. Comparativa de frameworks	269
30.7. Los frameworks en la nube: de la metodología a la herramienta	270
30.8. Consideraciones éticas y de implementación	271
30.9. Resumen final	271
30.10 Preguntas de repaso	272
Recursos y lecturas adicionales	272
Referencias	273
31. Business Understanding - Caso BigMart Sales	285
Escenario ficticio de negocio	285

Situación problemática	286
Diccionario de Variables	286
Necesidades del negocio que motivan el análisis	287
Del problema de negocio al Data Understanding	289
X. Análisis Exploratorio de Datos (EDA)	291
XI. Fundamentos del EDA	293
32. Fundamentos del análisis exploratorio de datos	295
32.1. Introducción	295
32.2. Preguntas que orientan el EDA	295
32.3. Unidad de observación y granularidad	296
32.4. Tipos de variables	296
32.5. Frecuencias y proporciones	297
32.6. Frecuencias acumuladas y ECDF	297
32.7. Ejemplo de calidad categórica	297
32.8. Ejemplo discreto y acumulado	298
32.9. Datos faltantes	298
32.10. Valores atípicos y anomalías	299
32.11. Ruta descriptiva del libro	299
32.12. Checklist de salida	299
32.13. Ejercicios	299
Referencias	300
XII. Medidas de localización	311
33. Medidas de localización	313
33.1. Introducción	313
33.2. Media aritmética	313
33.3. Media ponderada y media recortada	313
33.4. Mediana	314
33.5. Moda	314
33.6. Cuantiles y percentiles	314
33.7. Cálculo manual paso a paso	314
33.8. Cuantiles como función de la distribución	317
33.9. Selección de la medida	318
33.10. Puente al caso BigMart	318
33.11. Ejercicios	318
Referencias	318
34. Medidas de localización en Big Mart Sales	329
34.1. Preparación del entorno	329
34.2. Inspección inicial	330
34.3. Selección de variables para el análisis de localización	331
34.4. Medidas de localización básicas	334
34.5. Interpretación estadística de la media y la mediana	335
34.6. Cuantiles y lectura ampliada del centro	337

34.7. Impacto de eliminar faltantes en <code>Item_Weight</code>	338
34.8. Ejercicio sugerido	342

XIIIMedidas de variabilidad 343

35.Medidas de variabilidad 345

35.1. Introducción	345
35.2. Rango	345
35.3. Varianza y desviación estándar	345
35.4. Cuartiles e intervalo intercuartílico	346
35.5. Desviación absoluta mediana	346
35.6. Coeficiente de variación	346
35.7. Cálculo manual paso a paso	346
35.8. Misma media, distinta dispersión	349
35.9. Sensibilidad frente a valores extremos	349
35.10Boxplot interactivo	350
35.11Selección de la medida	350
35.12Puente al caso BigMart	350
35.13Ejercicios	350
Referencias	351

36.Medidas de variabilidad en Big Mart Sales 361

36.1. Preparación del entorno	361
36.2. De la localización a la dispersión	362
36.3. Cálculo de medidas de variabilidad	362
36.4. Visualización de dispersión con boxplots	363
36.5. Rango frente a IQR: medidas sensibles y robustas	364
36.6. Comparación de dispersión entre grupos	365
36.7. Coeficiente de variación y comparación relativa	367
36.8. Histograma con bandas de dispersión	367
36.9. Ejercicio sugerido	369

XIVMedidas de forma 371

37.Medidas de forma 373

37.1. Introducción	373
37.2. Momentos centrales	373
37.3. Asimetría	373
37.4. Curtosis	374
37.5. Cálculo manual paso a paso	374
37.6. Asimetría visible e interactiva	376
37.7. Misma media y varianza, colas diferentes	376
37.8. Gráfico Q-Q	377
37.9. Transformaciones y forma	377
37.10Multimodalidad	378
37.11Puente al caso BigMart	378
37.12Ejercicios	378
Referencias	378

38. Medidas de forma en Big Mart Sales	389
38.1. Introducción	389
38.2. Preparación del entorno	389
38.3. Fundamentos teóricos	391
38.4. Exploración gráfica inicial de la variable de ventas	391
38.5. Cálculo de asimetría y curtosis	394
38.6. Comparación de forma en varias variables numéricas	394
38.7. Transformaciones y su efecto sobre la forma	395
38.8. Gráfico cuantil-cuantil como apoyo visual	397
38.9. Mini-dashboard de forma	398
 XV. Medidas de heterogeneidad	 401
39. Medidas de heterogeneidad	403
39.1. El problema: ¿qué significa que unos datos sean heterogéneos?	403
39.2. Heterogeneidad categórica I: Gini–Simpson	404
39.3. Heterogeneidad categórica II: de la sorpresa a la entropía	406
39.4. Comparar Gini–Simpson y Shannon	409
39.5. Una escala común: números de Hill	409
39.6. Heterogeneidad de una variable numérica entre grupos	410
39.7. Heterogeneidad cruzada entre variables categóricas	412
39.8. Puentes con algoritmos de minería de datos	412
39.9. Implementación reproducible	413
39.10 Guía de decisión	414
39.11 Puente al caso BigMart	415
39.12 Ejercicios formativos	415
Referencias	415
 40. Medidas de heterogeneidad en Big Mart Sales	 427
40.1. Introducción	427
40.2. Preparación del entorno	427
40.3. Fundamentos teóricos	429
40.4. Variable objetivo y grupos de interés	429
40.5. Heterogeneidad por tipo de outlet	431
40.6. Heterogeneidad por tamaño del outlet	432
40.7. Heterogeneidad por ubicación del outlet	434
40.8. Heterogeneidad por tipo de producto	435
40.9. Tablas comparativas con múltiples indicadores	436
40.10 Heterogeneidad cruzada	437
40.11 Entropía de Shannon como medida de heterogeneidad	439
40.12 Mini-dashboard de heterogeneidad	442
40.13 Conclusiones del capítulo	443
 XVI. Medidas de concentración	 445
41. Medidas de concentración	447
41.1. Introducción	447
41.2. Razones de concentración	447
41.3. Índice Herfindahl–Hirschman	447

41.4. Curva de Lorenz	448
41.5. Coeficiente de Gini	448
41.6. Cálculo manual paso a paso	448
41.7. Pareto y participación acumulada	450
41.8. Lorenz y Gini	451
41.9. Concentración al crecer una unidad dominante	452
41.10Guía de decisión	452
41.11Puente al caso BigMart	452
41.12Ejercicios	452
Referencias	453
42. Medidas de concentración en Big Mart Sales	463
42.1. Preparación del entorno	463
43. Fundamento conceptual	465
43.1. Participación relativa	465
43.2. Participación acumulada	465
43.3. Curva de concentración	465
44. Concentración de ventas por categoría de producto	467
44.1. Ranking de contribución	467
44.2. Gráfico de Pareto	468
44.3. Interpretación	469
45. Concentración de ventas por outlet	471
45.1. Comentario analítico	472
46. Concentración por tipo de outlet	473
47. Curva de concentración simplificada	475
47.1. Lectura de la curva	476
48. Indicadores resumidos de concentración	477
49. Mini-dashboard de concentración	479
49.1. KPIs principales	479
50. Discusión final	481
51. Ejercicios sugeridos	483
XVI Medidas de correlación	485
52. Medidas de correlación	487
52.1. Introducción	487
52.2. Covarianza y Pearson	487
52.3. Spearman	487
52.4. Kendall	487
52.5. Cálculo manual paso a paso	488
52.6. Lineal, monótona y no monótona	490
52.7. Sensibilidad a un valor extremo	491

52.8. Cuarteto de Anscombe	491
52.9. Matriz de correlación	492
52.10. Guía de decisión	492
52.11. Puente al caso BigMart	492
52.12. Ejercicios	492
Referencias	492
53. Medidas de correlación en Big Mart Sales	503
54. Introducción	505
55. Objetivos del capítulo	507
56. Carga y preparación de datos	509
57. Fundamento conceptual	511
57.1. Correlación de Pearson	511
57.2. Correlación de Spearman	511
57.3. Advertencia importante	511
58. Revisión de valores faltantes en variables numéricas	513
59. Matriz de correlación de Pearson	515
59.1. Mapa de calor	515
59.2. Interpretación inicial	515
60. Correlación de Spearman	517
60.1. Comparación entre Pearson y Spearman	517
61. Correlación con la variable objetivo	519
61.1. Comentario analítico	519
62. Diagramas de dispersión	521
62.1. Ventas vs precio de lista	521
62.2. Ventas vs visibilidad	521
62.3. Ventas vs peso	522
62.4. Qué observar en los diagramas	522
63. Correlación por subgrupos	523
63.1. Ejemplo: correlación entre Item_MRP y ventas por tipo de outlet	523
63.2. Lectura	524
64. Tabla resumen de asociaciones	525
65. Mini-dashboard de correlación	527
65.1. KPIs de correlación	527
66. Discusión final	529
67. Ejercicios sugeridos	531

XVIII	Síntesis del EDA	533
68.	Dashboard global	535
69.	Introducción	537
70.	Objetivos del capítulo	539
71.	Carga y preparación de datos	541
72.	Arquitectura conceptual del dashboard	543
72.1.	Capa 1. Localización	543
72.2.	Capa 2. Variabilidad	543
72.3.	Capa 3. Forma	543
72.4.	Capa 4. Heterogeneidad	543
72.5.	Capa 5. Concentración	543
72.6.	Capa 6. Correlación	543
73.	Selección de variable principal	545
74.	Panel de indicadores globales	547
75.	Panel de localización y forma	549
75.1.	Interpretación	549
76.	Panel de variabilidad	551
77.	Panel de heterogeneidad	553
77.1.	Ventas por tipo de outlet	553
77.2.	Ventas por tipo de producto	553
77.3.	Tabla comparativa por grupo	554
78.	Panel de concentración	555
78.1.	Indicadores breves de concentración	556
79.	Panel de correlación	557
80.	Ensamblaje del dashboard global	559
80.1.	Ejemplo de lectura integrada	559
81.	Propuesta de cierre visual para el book	561
82.	Conclusiones	563
83.	Sugerencias de extensión	565
84.	Integración al libro de Quarto	567
XIX	Análisis Multivariado	569
85.	Análisis Exploratorio de Datos Multivariado: Teoría y Visualización	571
86.	La maldición de la dimensionalidad	573

87. Fundamentos matemáticos del PCA	575
87.1. Formulación Matemática y Procedimiento	575
87.2. Ejemplo Práctico: Cálculo Manual Paso a Paso	576
87.3. Ejemplo Práctico: Visualización paso a paso	577
88. PCA y la Descomposición en Valores Singulares (SVD)	581
89. Ejemplo práctico: Implementación paso a paso	583
Dataset: Iris	583
Paso 1: Estandarización	584
Paso 2: Matriz de covarianza	584
Paso 3: Eigendescomposición vía SVD	585
Paso 4: Scree plot	586
Paso 5: Biplot	587
Paso 6: Loadings detallados	588
90. Ejemplo avanzado: Dataset con ruido	591
Paso 1 — La trayectoria real y el ruido de medición	591
Paso 2 — Tres sensores, tres perspectivas distintas	592
Paso 3 — Construir la matriz de datos multi-sensor	596
Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?	597
Paso 5 — Proyección en 2D: la hélice emerge del ruido	598
Paso 6 — Proyección en 3D: confirmando la estructura helicoidal	599
91. Limitaciones del PCA	601
91.1. Linealidad	601
91.2. Sensibilidad a outliers	602
91.3. PCA no captura estructura de clases	602
91.4. Asunción de normalidad (implícita)	603
92. Más allá del PCA: técnicas no lineales	605
92.1. t-SNE (t-distributed Stochastic Neighbor Embedding)	605
92.2. UMAP (Uniform Manifold Approximation and Projection)	606
92.3. Cuándo usar cada método	607
93. Resumen y conexiones	609
94. Ejercicios propuestos	611
94.1. Recursos recomendados: EDA multivariado	611
95. Análisis Exploratorio de Datos Multivariado: Ejemplos prácticos	613
96. Ejemplo práctico: Implementación paso a paso	615
Dataset: Iris	615
Paso 1: Estandarización	616
Paso 2: Matriz de covarianza	616
Paso 3: Eigendescomposición vía SVD	617
Paso 4: Scree plot	618
Paso 5: Biplot	619
Paso 6: Loadings detallados	620

97. Ejemplo avanzado: Dataset con ruido	623
97.1. El problema: una partícula en movimiento que no podemos ver directamente	623
97.2. Paso 1 — La trayectoria real y el ruido de medición	623
97.3. Paso 2 — Tres sensores, tres perspectivas distintas	624
97.4. Paso 3 — Construir la matriz de datos multi-sensor	628
97.5. Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?	629
97.6. Paso 5 — Proyección en 2D: la hélice emerge del ruido	630
97.7. Paso 6 — Proyección en 3D: confirmando la estructura helicoidal	631
 XX. Preprocesamiento	 635
98. Introducción a Preprocesamiento de Datos	637
99. Conversión y Transformación de Datos	639
100. Reducción de ruido	641
101. Procesamiento de Datos Faltantes	643
101.1. Objetivos de la clase	643
102. Introducción	645
102.1. Idea central	645
102.2. Motivación	645
102.3. Ejemplo visual:	645
102.4. Preguntas guía	646
102.5. Tipos estructurales de datos faltantes	646
103. Mecanismos de ausencia	647
103.1. MCAR — Missing Completely At Random	647
103.2. Ejemplo sintético: MCAR en acción	647
103.3. MAR — Missing At Random	648
103.4. Ejemplo sintético: MAR en acción	648
103.5. Efecto de MAR sobre la muestra observada	648
103.6. MNAR — Missing Not At Random	649
103.7. Ejemplo sintético: MNAR en acción	649
103.8. Clasificación Global del Mecanismo de Datos Faltantes	650
104. Estrategias para manejar datos faltantes	653
104.1. De la teoría a la práctica	653
104.2. Estrategias prácticas	653
104.3. Eliminación Listwise	653
104.4. Imputación simple	654
104.5. Ejemplo sintético: imputación simple	655
104.6. Imputación múltiple	655
104.7. Ejemplo sintético: imputación múltiple	656
104.8. Imputación con KNN / modelos	657
104.9. Ejemplo sintético: imputación con KNN	657
104.10. Recomendaciones para práctica profesional	658
104.11. Mini ejercicio en vivo	658
104.12. Para discutir	658

104.1	Cierre	659
104.1	Próximo paso sugerido	659
105	Conclusiones	661
106	Reducción de Dimensionalidad con PCA	663
107	La maldición de la dimensionalidad	665
108	Fundamentos matemáticos del PCA	667
108.1	Formulación Matemática y Procedimiento	667
108.2	Ejemplo Práctico: Cálculo Manual Paso a Paso	668
108.3	Ejemplo Práctico: Visualización paso a paso	669
109	PCA y la Descomposición en Valores Singulares (SVD)	673
110	Ejemplo práctico: Implementación paso a paso	675
	Dataset: Iris	675
	Paso 1: Estandarización	676
	Paso 2: Matriz de covarianza	676
	Paso 3: Eigendescomposición vía SVD	677
	Paso 4: Scree plot	678
	Paso 5: Biplot	679
	Paso 6: Loadings detallados	680
111	Ejemplo avanzado: Dataset con ruido	683
	Paso 1 — La trayectoria real y el ruido de medición	683
	Paso 2 — Tres sensores, tres perspectivas distintas	684
	Paso 3 — Construir la matriz de datos multi-sensor	688
	Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?	689
	Paso 5 — Proyección en 2D: la hélice emerge del ruido	690
	Paso 6 — Proyección en 3D: confirmando la estructura helicoidal	691
112	Limitaciones del PCA	693
112.1	Linealidad	693
112.2	Sensibilidad a outliers	694
112.3	PCA no captura estructura de clases	694
112.4	Asunción de normalidad (implícita)	695
113	Más allá del PCA: técnicas no lineales	697
113.1	t-SNE (t-distributed Stochastic Neighbor Embedding)	697
113.2	UMAP (Uniform Manifold Approximation and Projection)	698
113.3	Cuándo usar cada método	699
114	Resumen y conexiones	701
115	Ejercicios propuestos	703
115.1	Recursos recomendados: EDA multivariado	703
116	Reducción de variables categóricas con ²	705
116.1	Principios estadísticos	705
116.2	Definición: la prueba ²	712

116.3Ejemplo: Tabaquismo × Ejercicio	718
116.4. ² y reducción de datos	722
116.5Ejemplo: Encuesta de satisfacción — Aerolínea	722
116.6Conclusión	729
XXI Extracción de Patrones	731
XXI Reglas de Asociación	733
117 Reglas de Asociación: Fundamentos, Interpretación y Selección	735
117.1Introducción	735
117.2Definiciones Fundamentales	735
117.3Generación de Reglas Frecuentes	738
117.4El Problema de las Demasiadas Reglas	740
117.5Métricas de Evaluación: Interpretación Profunda	741
117.6Interpretación Visual del Espacio de Reglas	743
117.7Selección de Reglas Útiles	745
117.8Limitaciones del Enfoque	751
117.9Resumen Conceptual	752
Referencias	753
118 Más Allá del Lift: Métricas Avanzadas, Sensibilidad y Estructura de Reglas	763
118.1Introducción	763
118.2Las Limitaciones Estructurales del Lift	763
118.3Métricas Complementarias al Lift	767
118.4Sensibilidad a los Parámetros del Algoritmo	771
118.5La Estructura Global del Espacio de Reglas: Representación como Grafo	776
118.6Un Marco Integrado de Evaluación	781
118.7Resumen Conceptual	784
Referencias	785
119 FP-Growth y minería de patrones frecuentes	797
120 FP-Growth y minería de patrones frecuentes	799
120.1Introducción	799
120.2Conceptos básicos	799
120.3De Apriori a FP-Growth	800
120.4Ejemplo base: transacciones de cesta de compra	800
120.5Ordenamiento por frecuencia	801
120.6Construcción del FP-tree	802
120.7Caminos prefijo y base condicional	804
120.8Árbol condicional	805
120.9Patrones frecuentes del ejemplo	806
120.10Implementación práctica con <code>mlxtend</code>	807
120.11Comparación entre FP-Growth y Apriori	809
120.12Aplicaciones	810
120.13Ventajas y limitaciones	810
120.14Buenas prácticas de análisis	811
120.15Conclusiones	811

XXI Clasificación	813
XXII Introducción y Evaluación	815
121 Clasificación y árboles de decisión	817
121.1 Introducción	817
121.2 Clasificación como problema supervisado	817
121.3 Entrenamiento, prueba y generalización	818
121.4 Preparación de datos antes de construir un clasificador	818
121.5 Árboles de decisión	819
121.6 Partición de datos en árboles de decisión	820
121.7 Algoritmo de Hunt	820
121.8 Criterios de parada	821
121.9 ID3: selección de atributos mediante ganancia de información	821
121.10 Entropía como medida de impureza	821
121.11 Ganancia de información	822
121.12 Ejemplo: construir un árbol para decidir si se juega béisbol	823
121.13 Construcción de ramas después de elegir la raíz	825
121.14 Representación del árbol final	826
121.15 Extracción de reglas de clasificación	827
121.16 Implementación con <code>scikit-learn</code>	828
121.17 Visualización del árbol	829
121.18 Complejidad computacional	830
121.19 Ventajas y limitaciones	831
121.20 Conclusiones	831
122 Evaluación de Clasificadores: Métricas Esenciales y su Interpretación	833
122.1 Introducción	833
122.2 Errores	833
122.3 Tipo de errores y su costo	834
122.4 Métricas de desempeño: más allá del error rate	840
122.5 F1-Score y sus Variantes Multiclase	849
122.6 Curva ROC y el Área Bajo la Curva (AUC)	852
122.7 Curva Precision-Recall y Average Precision	856
122.8 Optimización del Umbral de Decisión	860
122.9 Comparación Final de Modelos	863
122.10 Errores Comunes en la Interpretación de Métricas	865
122.11 Guía de Selección de Métricas	866
122.12 Conclusiones	867
122.13 Referencias	867
123 Evaluación de un modelo de clasificación	869
123.1 Introducción	869
123.2 Aprendizaje supervisado y evaluación	869
123.3 Partición de datos	870
123.4 Ejemplo en Python: partición holdout y validación cruzada	872
123.5 Matriz de confusión	874
123.6 Métricas básicas de clasificación	874
123.7 Ejemplo en Python: matriz de confusión y métricas	876
123.8 Selección de métricas según la necesidad del problema	877

123.9	Umbral de decisión	878
123.10	Curva ROC y AUC	879
123.11	Ejemplo en Python: curva ROC y AUC	880
123.12	Comparación con la curva Precision–Recall	880
123.13	Flujo recomendado para evaluar clasificadores	881
123.14	Ejemplo final: evaluación completa en Python	882
123.15	Conclusiones	883

XXV Árboles y Ensamblés **885**

124 Árboles CART: árboles de clasificación y regresión **887**

124.1	Introducción	887
124.2	De los árboles de decisión a CART	887
124.3	Clasificación, pureza e impureza	888
124.4	Índice de Gini	888
124.5	Evaluación de una partición binaria	889
124.6	Particiones binarias con variables categóricas	890
124.7	Particiones binarias con variables continuas	891
124.8	Ejemplo completo: compra de computadora portátil	892
124.9	Reducción de impureza e interpretación	894
124.10	Criterios de parada	895
124.11	Poda de costo-complejidad	895
124.12	Visualización de un árbol entrenado	896
124.13	Extracción de reglas de clasificación	897
124.14	CART para regresión	898
124.15	Complejidad computacional	898
124.16	Ventajas y limitaciones	899
124.17	Conclusiones	899

125 Árboles de regresión lineal, ensambles y vecinos cercanos **901**

125.1	De la regresión lineal a modelos por regiones	901
125.2	Clasificadores no lineales y modelos por partes	902
125.3	Funciones por escalones	903
125.4	Predicción dentro de una región	903
125.5	Error total y criterio de división	904
125.6	Particionamiento recursivo	904
125.7	Ejemplo manual: precio de laptops según RAM	905
125.8	Implementación en Python	906
125.9	Sobreaajuste y poda	907
125.10	Evaluación de árboles de regresión	909
125.11	Ventajas y limitaciones de los árboles	909
125.12	Bagging: agregación por bootstrap	910
125.13	Random forests	911
125.14	Boosting	912
125.15	Bagging frente a boosting	913
125.16	Clasificación del vecino más cercano	913
125.17	Métricas de distancia para vecinos cercanos	914
125.18	De 1-NN a k-NN	915
125.19	Error asintótico de vecinos cercanos	917

125.2Maldición de la dimensionalidad	917
125.2Métodos mejorados para vecinos cercanos	918
125.2Cuándo usar KNN	920
125.2Comparación conceptual: árboles y vecinos cercanos	920
125.2Buenas prácticas de modelado	920
125.2Conclusiones	921
126Gradient Boosting y XGBoost	923
126.1De los arboles de decision al boosting	923
126.2El objetivo de XGBoost	929
126.3Hiperparametros principales	935
126.4Importancia de variables	940
126.5Regularizacion y poda	942
126.6Contribuciones individuales (SHAP simplificado)	944
126.7XGBoost para clasificacion	947
126.8Buenas practicas y comparacion con alternativas	949
Referencias	950
127Clasificadores Supervisados: Experimento Práctico	961
128Introducción	963
129Dataset: Breast Cancer Wisconsin	965
130Flujo general de un experimento de clasificación	967
131Preparación de datos	969
131.1Separación entrenamiento/prueba	969
131.2Pipelines para preprocesamiento sin leakage	969
131.3Validación cruzada	970
132Clasificadores: intuición, sesgos y hiperparámetros	971
132.1Regresión Logística (baseline)	971
132.2Árboles de decisión: CART e ID3	971
132.3Random Forest (Breiman 2001)	972
132.4Gradient Boosting (friedman2001gradient?)	973
132.5Red neuronal multicapa (MLP)	974
133Recordatorio de métricas	977
133.1Curva ROC y AUC	977
133.2Curva Precision-Recall y Average Precision	978
133.3Selección de métricas según el contexto	978
134Ajuste de thresholds de decisión	979
134.1Motivación	979
134.2predict_proba y decision_function	979
134.3Comparación de thresholds	979
134.4Consideraciones prácticas sobre el threshold	981
135Comparación entre modelos	983
136Buenas prácticas para evaluación	985

137	Errores comunes en experimentos de clasificación	987
138	Retos prácticos del flujo de clasificación	989
139	Conclusión	991
	Referencias	993
XX	Modelos Lineales	1003
140	Regresión y clasificadores lineales	1005
140.1	Regresión: predicción de variables continuas	1005
140.2	Representación matricial de la regresión lineal	1006
140.3	Ejemplo práctico: regresión lineal múltiple en Python	1009
140.4	Métricas de evaluación para regresión	1010
140.5	Ventajas y limitaciones de la regresión lineal	1012
140.6	Más allá de la linealidad: árboles de regresión	1013
140.7	De regresión a clasificación lineal	1013
140.8	Discriminante lineal de Fisher	1014
140.9	El perceptrón	1018
140.10	Comparación conceptual: regresión lineal, Fisher y perceptrón	1021
140.11	Buenas prácticas en documentación reproducible con Quarto	1022
140.12	Conclusiones	1022
141	Regresión Logística	1023
141.1	Motivación: el problema de clasificar con regresión lineal	1023
141.2	La función logística (sigmoide)	1025
141.3	El modelo de regresión logística	1028
141.4	Estimación por máxima verosimilitud	1031
141.5	Interpretación de coeficientes: odds ratio	1034
141.6	Regularización	1037
141.7	Extensión multiclase	1041
141.8	Calibración de probabilidades	1044
141.9	Ventajas y limitaciones	1047
141.10	Recomendaciones prácticas	1048
141.11	Resumen del capítulo	1049
141.12	Referencias	1049
142	Máquinas de Soporte Vectorial	1061
142.1	Motivación: clasificar con margen máximo	1061
142.2	SVM de margen duro (datos linealmente separables)	1064
142.3	SVM de margen suave (datos no separables)	1069
142.4	El kernel trick	1073
142.5	Efecto de los hiperparámetros C y γ (kernel RBF)	1077
142.6	Comparación de kernels en datos reales	1080
142.7	SVM para regresión (SVR)	1083
142.8	Aspectos computacionales	1086
142.9	Ventajas y limitaciones	1087
142.10	Obtener probabilidades de SVM: calibración de Platt	1088
142.11	Recomendaciones prácticas	1089

142.1	Referencias	1090
XXV	Clasificadores Probabilísticos	1101
143	Clasificadores bayesianos	1103
143.1	Vista probabilística de la clasificación	1108
143.2	Clasificador bayesiano de error mínimo	1114
143.3	Clasificador Naïve Bayes	1117
143.4	Clasificadores bayesianos gaussianos	1125
143.5	¿Cómo evaluar el supuesto gaussiano?	1132
143.6	Clasificación bayesiana de secuencias discretas	1134
143.7	Clasificadores bayesianos de riesgo mínimo	1136
143.8	Minimizar un tipo de error condicionado al otro	1139
143.9	Criterio minimax cuando los priors son desconocidos	1140
143.10	Recomendaciones prácticas	1141
143.11	Conclusiones	1141
144	Modelos de Mezcla Gaussiana	1143
144.1	Motivación: ¿por qué un solo gaussiano no basta?	1143
144.2	El modelo de mezcla gaussiana	1145
144.3	El algoritmo EM	1148
144.4	Tipos de covarianza	1155
144.5	Selección del número de componentes	1157
144.6	GMM como clasificador vs. K-Means	1160
144.7	GMM como modelo generativo	1163
144.8	Consideraciones prácticas	1167
	Referencias	1168
XXVI	Redes Neuronales	1179
145	Redes Neuronales para Clasificación	1181
145.1	El Perceptron	1181
145.2	Funciones de Activación	1187
145.3	Red Neuronal Multicapa (MLP)	1189
145.4	Retropropagación del Error	1192
145.5	Optimización	1196
145.6	Implementación desde Cero	1200
145.7	Redes Convolucionales (CNN)	1203
145.8	Regularización en Redes Neuronales	1208
145.9	Más allá: Tipos de Redes	1214
145.10	Resumen	1215
	Referencias	1216
146	Implementación Práctica: PyTorch y JAX	1227
146.1	El ecosistema de aprendizaje profundo	1227
146.2	Dataset a explorar	1230
146.3	Preparación de datos	1231
146.4	PyTorch: fundamentos	1232
146.5	Clasificación con PyTorch	1238

146.6	CNN con PyTorch	1251
146.7	JAX: fundamentos	1256
146.8	Clasificación con JAX puro	1262
146.9	CNN con Flax	1274
146.10	Comparativa PyTorch vs JAX	1277
146.11	Ejercicios	1286
146.12	Resumen	1290
	Referencias	1291
XXI	Aprendizaje No Supervisado	1293
XXX	Clustering	1295
147	Introducción al clustering y métricas de evaluación	1297
147.1	El problema de clustering	1297
147.2	Medidas de distancia y similitud	1297
147.3	Evaluación de clustering sin etiquetas verdaderas	1299
147.4	Evaluación con etiquetas verdaderas	1301
147.5	Preprocesamiento antes de clustering	1301
147.6	Selección del número de grupos	1302
148	K-Means y K-Medoids	1303
148.1	El objetivo de K-Means	1303
148.2	Algoritmo de Lloyd	1303
148.3	Inicialización K-Means++	1304
148.4	Limitaciones de K-Means	1304
148.5	Método del codo	1305
148.6	K-Medoids (PAM)	1305
148.7	Implementación con scikit-learn	1307
148.8	Consideraciones prácticas	1307
149	Clustering jerárquico	1309
149.1	Método aglomerativo	1309
149.2	Criterios de enlace	1309
149.3	El dendrograma	1310
149.4	Correlación cofenética	1310
149.5	Complejidad y escalabilidad	1312
149.6	Aplicaciones del clustering jerárquico	1313
150	DBSCAN y HDBSCAN	1315
150.1	Conceptos fundamentales	1315
150.2	Alcanzabilidad y conectividad por densidad	1315
150.3	Algoritmo DBSCAN	1316
150.4	Selección de parámetros	1316
150.5	Ventajas y limitaciones	1318
150.6	HDBSCAN	1318
150.7	Aplicaciones	1319

XXX Reducción de Dimensionalidad	1321
151t-SNE, UMAP y reducción de dimensionalidad no lineal	1323
151.1 Limitaciones de PCA para visualización	1323
151.2 SNE y t-SNE	1324
151.3 UMAP	1326
151.4 Comparación t-SNE vs UMAP	1328
151.5 Autoencoder básico para reducción de dimensionalidad	1329
151.6 Precauciones y buenas prácticas	1331
151.7 Cuando usar cada método	1332
Apéndices	1333
A. Presentaciones de clase	1333
B. Banco de Proyectos	1337
B.1. 1. Value bets en Liga MX	1337
B.2. 2. Priorización de contratos públicos sospechosos	1338
B.3. 3. Detección temprana de réplicas sísmicas anómalas	1338
B.4. 4. Calidad del aire y “días de contingencia” en CDMX	1339
B.5. 5. Fraude en reembolsos/transacciones fintech	1339
B.6. 6. Segmentación de incidentes de inseguridad por alcaldía/colonia	1340
B.7. 7. Reseñas falsas o manipuladas en e-commerce	1340
B.8. 8. Predicción de retrasos y saturación en el Metro/transporte público	1341
B.9. 9. Deserción escolar en educación media superior	1341
B.10.10. Discurso de odio/polarización en debates electorales	1342
C. Glosario Unificado de Términos de Datos y Computación en la Nube	1343
C.1. A	1343
C.2. B	1343
C.3. C	1344
C.4. D	1344
C.5. N	1344
C.6. S	1344
D. Preparación para Entrevistas Analíticas	1345
D.1. 1. La Importancia del Proceso de Pensamiento	1345
D.2. 2. El Marco de Resolución de Problemas en 4 Pasos	1345
D.3. 3. Preguntas de Práctica Comunes y Guías de Respuesta	1346
E. Bibliografía y Recursos Adicionales	1349
E.1. 1. Referencias Bibliográficas	1349
E.2. 2. Documentación Oficial y Guías de Tecnologías	1349
E.3. 3. Conjuntos de Datos y Casos de Estudio	1350

Prefacio

Este curso nació originalmente en el semestre 2026-II y se ha ido actualizando y enriqueciendo con el paso del tiempo. En la actualidad, está fuertemente orientado a sentar las bases sólidas del *Data Engineering* y *Data Science*, con el objetivo primordial de dotar a los estudiantes de herramientas que son directamente aplicables a la industria. A lo largo del texto encontrarás ejemplos basados en casos de uso reales y prácticas guiadas con tecnologías modernas como PySpark y BigQuery.

En las tarjetas a continuación podrás explorar el contenido general del curso. Aunque la estructura está diseñada con un orden de aprendizaje progresivo sugerido, los capítulos están contruidos de manera autocontenida, lo que te permitirá consultar directamente los temas que más te interesen.

Temario

Requisitos previos

Orientado a estudiantes de Ciencias de la Computación, Matemáticas, Física y carreras afines.

Agradecimientos

Quiero expresar mi profundo agradecimiento a la profesora **Jessica Santizo** por su invaluable invitación a participar de manera inicial en este curso, así como por sus invaluables contribuciones y discusiones que han sido fundamentales para el mejoramiento continuo de este material.

Un agradecimiento muy especial a **las y los alumnos** que han cursado la materia; sus dudas, comentarios y retroalimentación constante han fortalecido enormemente la calidad y claridad del contenido en estas notas.

A la **Universidad Nacional Autónoma de México (UNAM)**, nuestra gran casa de estudios; un baluarte invaluable del conocimiento libre y accesible para todos.

Finalmente, extendiendo mi gratitud a los creadores de los diversos cursos y materiales en línea de múltiples entidades, comunidades y empresas tecnológicas. Gracias a su libre disposición y compromiso con el conocimiento abierto, ha sido posible la estructuración y realización de estas notas.

Autor

Cómo citar

Si utilizas este material en tus trabajos, proyectos o investigaciones, por favor utiliza la siguiente cita en formato BibTeX:

```
@misc{villalba2026almacenes,  
  title={Almacenes y Miner\ '{i}a de Datos},  
  author={Villalba, Diego},  
  year={2026},  
  howpublished={Repositorio de GitHub / GitHub Pages},  
  note={Notas de curso de autor\ '{i}a propia},  
  url={https://diegoviillalba.github.io/almacenamienes-y-mineria-de-datos/}  
}
```

Parte I.

Entendimiento de Negocio

Parte II.

Fundamentos y Stakeholders

1. Gestión de Requerimientos y Stakeholders

En la mayoría de los cursos de minería de datos y almacenes de datos, el punto de partida suele ser una base de datos limpia o un algoritmo de aprendizaje supervisado como árboles de decisión o regresión logística. Sin embargo, en el mundo real, los proyectos analíticos no comienzan con datos listos para ser procesados. Comienzan con una necesidad de negocio, una pregunta vaga o un problema operativo expresado por personas que no necesariamente entienden de algoritmos o de modelado dimensional.



El éxito de un proyecto de datos no se mide por la precisión del modelo (por ejemplo, un AUC de 0.95), sino por el **impacto económico o estratégico** que genera en la organización. Por ello, la fase inicial de cualquier proyecto de datos exitoso, el **Entendimiento del Negocio** (fase inicial de la metodología CRISP-DM que estudiaremos más adelante en el curso (Chapman et al. 2000)) es la más crítica. Antes de escribir la primera línea de código o importar librerías de modelado, debemos comprender en profundidad qué busca solucionar la organización. Para ejecutar esto con éxito, es fundamental dominar la comunicación con los **stakeholders** (partes interesadas) y traducir sus necesidades cualitativas en problemas técnicos cuantitativos estructurados.

1.1. Identificación de Partes Interesadas (*Stakeholders*)

Definición (stakeholder o parte interesada). De acuerdo con el marco del *Community Tool Box* de la Universidad de Kansas (Center for Community Health and Development, University of Kansas 2023), una parte interesada es cualquier persona, grupo u organización que puede verse afectada por un proyecto, o que puede influir directa o indirectamente en su desarrollo y éxito. Esta definición retoma la teoría clásica de los *stakeholders* propuesta por Freeman (1984) y ampliamente adoptada en la gestión moderna de proyectos.

En proyectos de analítica y minería de datos, identificar a las partes interesadas permite comprender quiénes utilizarán los resultados, quiénes podrían verse afectados y quiénes tienen la capacidad de orientar o decidir el rumbo del proyecto. Siguiendo las técnicas de identificación y análisis descritas por Bryson (2004), las partes interesadas pueden estudiarse mediante dos criterios complementarios: su relación con la organización y su papel dentro del proyecto.

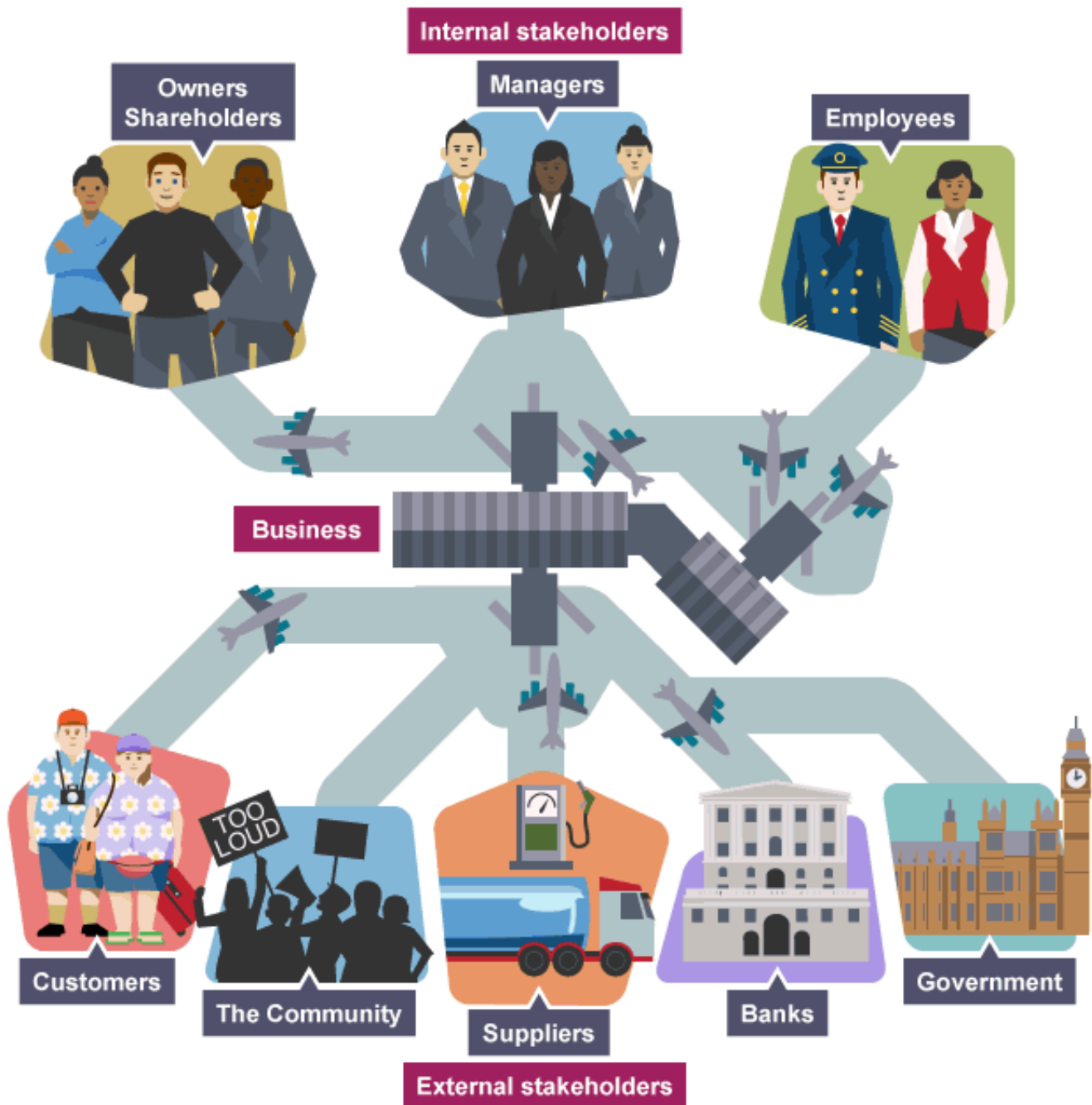
1.1.1. Clasificación según su relación con la organización

Una manera de clasificar a los stakeholders es según su relación con la organización responsable del proyecto.

1.1.2. Clasificación según su papel en el proyecto

Esta segunda clasificación no sustituye a la anterior. Una misma parte interesada puede ser interna o externa y, al mismo tiempo, primaria, secundaria o clave.

1.1.3. Ejemplo: un aeropuerto internacional



En la operación de un **aeropuerto internacional** pueden identificarse partes interesadas internas, como la autoridad aeroportuaria, el personal de seguridad y los responsables operativos; y partes interesadas externas, como los pasajeros, las aerolíneas, los proveedores, las empresas de transporte, los organismos gubernamentales y los inversionistas.

Si se desarrolla un sistema analítico para predecir retrasos, los pasajeros, pilotos y operadores podrían considerarse partes interesadas primarias; los proveedores de servicios y las empresas de transporte terrestre, partes interesadas secundarias; y la autoridad aeroportuaria, los organismos reguladores y los patrocinadores del proyecto, partes interesadas clave.

1. Gestión de Requerimientos y Stakeholders

La clasificación dependerá del alcance del proyecto y de la manera en que cada actor resulte afectado o pueda influir en sus decisiones.

1.2. Análisis de Stakeholders: La Matriz de Poder e Interés

Para gestionar las expectativas de manera eficiente y evitar que la falta de alineación descarrile un proyecto de datos, se utiliza la **Matriz de Poder vs. Interés** (también conocida como Matriz de Influencia/Interés), originalmente propuesta por Mendelow (1981) y adoptada como práctica estándar de gestión de interesados por el *PMBOK Guide* (Project Management Institute 2017). Esta herramienta clasifica a los stakeholders en una cuadrícula basada en dos dimensiones clave:

1. **Poder (Influencia)**: La capacidad de cambiar la dirección del proyecto, asignar recursos, aprobar entregables o cancelarlo por completo.
2. **Interés**: El nivel de atención y preocupación que tiene el stakeholder sobre el impacto del proyecto en su área.

A continuación se presenta el diagrama de la matriz de alineación estratégica para proyectos de análisis de datos:

1.2.1. Estrategias de Gestión por Cuadrante

- **Alto Poder, Alto Interés (Gestionar de Cerca)**: Deben participar activamente en las fases de planificación del proyecto, validar los entregables intermedios y recibir reportes de avance periódicos.
- **Alto Poder, Bajo Interés (Mantener Satisfecho)**: Involucrarlos en puntos críticos. Es fundamental garantizar que el proyecto cumple con las normativas de seguridad, gobernanza y gobernanza de nube.
- **Bajo Poder, Alto Interés (Mantener Informado)**: Mantenerlos entusiasmados e informados sobre cómo el modelo les facilitará el trabajo. Es vital pedir su retroalimentación temprana.
- **Bajo Poder, Bajo Interés (Monitorear)**: Monitorear pasivamente con comunicación general.

1.3. Definición de alcance y traducción de preguntas de negocio

Uno de los errores más frecuentes en los proyectos de analítica consiste en aceptar una pregunta de negocio tal como fue formulada y comenzar a trabajar con los datos de inmediato. Expresiones como “*queremos evitar que los clientes se vayan*” o “*necesitamos aumentar las ventas*” comunican una preocupación legítima, pero todavía no constituyen un objetivo analítico.

La función del analista no es únicamente extraer datos o construir modelos. También debe actuar como **traductor entre las necesidades del negocio y las capacidades analíticas de la organización**. Esto implica transformar una inquietud general en una pregunta concreta, medible y vinculada con una decisión.

La definición de alcance —o *scoping*— es el proceso mediante el cual se aclaran:

- el problema que se desea resolver;
- la decisión que se tomará con el resultado;
- la población o unidad de análisis;
- el periodo relevante;

- las métricas de éxito;
- las restricciones operativas;
- y el formato de entrega esperado.

Solo después de establecer estos elementos tiene sentido evaluar qué datos están disponibles y qué metodología podría utilizarse.

Esta traducción constituye el punto de partida de todo proyecto analítico. Una vez comprendida la necesidad del negocio, el equipo puede estudiar los datos disponibles, prepararlos, realizar el análisis, evaluar si los resultados responden a la pregunta original y, cuando sea necesario, desplegar una solución en producción. Por tanto, la definición de alcance no es una formalidad previa al análisis: es la etapa que orienta todas las decisiones posteriores.

1.3.1. Del problema de negocio al objetivo analítico

Una pregunta cualitativa debe atravesar varias capas de definición antes de convertirse en un objetivo técnico. El propósito no es reemplazar el lenguaje del stakeholder por terminología estadística, sino construir una especificación compartida que conecte el análisis con una acción concreta.

En este ejemplo, el resultado del proceso no es todavía la selección de un algoritmo. Decidir anticipadamente que se utilizará XGBoost, Random Forest o cualquier otra técnica puede limitar el análisis antes de comprender el problema. La metodología debe elegirse después de confirmar el objetivo, revisar los datos y establecer cómo se evaluará la utilidad de la solución.

1.4. Refinamiento iterativo mediante preguntas

La primera pregunta del stakeholder rara vez contiene todo el contexto necesario. Por ello, la definición de alcance debe tratarse como una **conversación iterativa** y no como un cuestionario que se responde una sola vez.

Una estrategia útil consiste en avanzar por capas. Cada respuesta permite formular una pregunta más precisa, detectar supuestos y regresar al stakeholder para validar el entendimiento. El ciclo termina cuando ambas partes pueden expresar con claridad qué se analizará, para qué se utilizará y cómo se reconocerá un resultado útil.

1.4.1. Estrategias para formular mejores preguntas

Durante este ciclo pueden combinarse varias estrategias de indagación:

1. **Preguntar por el contexto antes que por la solución.**

En lugar de comenzar con “¿qué dashboard necesitas?”, conviene preguntar “¿qué situación motivó esta solicitud?”. Esto evita confundir el formato de entrega con el problema que debe resolverse.

2. **Conectar el análisis con una decisión.**

Preguntas como “¿qué harías de manera diferente si tuvieras esta información?” permiten determinar si el análisis tendrá un uso concreto. Si no existe una decisión asociada, es posible que la solicitud todavía sea demasiado general.

1. Gestión de Requerimientos y Stakeholders

3. Convertir conceptos ambiguos en definiciones operativas.

Términos como *cliente activo*, *abandono*, *éxito*, *rentabilidad* o *bajo desempeño* pueden tener significados diferentes entre áreas. El analista debe preguntar cómo se observan y calculan en la práctica.

4. Delimitar población, periodo y nivel de detalle.

Es necesario precisar si el análisis se realizará por cliente, transacción, producto, sucursal o región, así como definir el periodo histórico y el horizonte futuro relevantes.

5. Explorar restricciones y capacidad de acción.

No basta con detectar un comportamiento si la organización no puede intervenir sobre él. Conviene preguntar qué acciones están disponibles, cuál es su costo y sobre cuántos casos puede actuar el equipo.

6. Reformular y validar.

Después de escuchar las respuestas, el analista debe resumir el entendimiento alcanzado: “*Entonces, lo que necesitamos es...*”. Esta reformulación permite que el stakeholder corrija interpretaciones antes de invertir tiempo en el análisis.

1.4.2. Ejemplo de una conversación de refinamiento

Supongamos que un stakeholder plantea la siguiente necesidad:

“Queremos evitar que los clientes se vayan.”

La conversación podría avanzar de esta manera:

- **Contexto:** ¿Qué comportamiento reciente indica que existe un problema de abandono?
- **Definición:** ¿Qué significa exactamente que un cliente “se vaya”: cancelar, dejar de comprar o permanecer inactivo?
- **Población:** ¿El problema afecta a todos los clientes o a un segmento específico?
- **Horizonte:** ¿Con cuánta anticipación necesitamos identificar el riesgo?
- **Decisión:** ¿Qué acción se realizará cuando se detecte un cliente en riesgo?
- **Capacidad:** ¿A cuántos clientes puede contactar el equipo de retención cada semana?
- **Éxito:** ¿Cómo se medirá el resultado: cancelaciones evitadas, ingresos retenidos o retorno de la campaña?
- **Validación:** ¿La prioridad es detectar la mayor cantidad posible de clientes que podrían abandonar, aunque existan algunos falsos positivos?

Después de esta iteración, la pregunta original podría reformularse así:

Objetivo analítico: identificar semanalmente a los clientes de suscripción mensual con mayor probabilidad de cancelar durante los próximos 30 días, utilizando su comportamiento de los últimos 90 días, para priorizar hasta 2,000 contactos del equipo de retención. La solución se evaluará por su capacidad para identificar cancelaciones reales y por los ingresos retenidos mediante la intervención.

Esta formulación ya establece una población, una ventana de observación, un horizonte de predicción, una capacidad operativa y una medida de valor. A partir de ella, el equipo puede investigar la disponibilidad de los datos y comparar distintas metodologías.

1.5. Ejemplos de traducción de preguntas de negocio

Pregunta del stakeholder	Traducción analítica	Posible metodología
<i>¿Cómo podemos aumentar el ticket promedio de la tienda en línea?</i>	Identificar productos que tienden a comprarse juntos y evaluar qué combinaciones podrían utilizarse en recomendaciones durante el proceso de compra.	Reglas de asociación —Apriori o FP-Growth— evaluadas mediante soporte, confianza y <i>lift</i> .
<i>¿Cómo evitamos la fuga de clientes de suscripción mensual?</i>	Estimar la probabilidad de cancelación durante los próximos 30 días a partir de la actividad reciente de cada cliente, con el fin de priorizar intervenciones.	Modelos de clasificación supervisada, evaluados con métricas coherentes con la capacidad y el costo de intervención.
<i>Queremos campañas personalizadas, pero no sabemos cómo agrupar a los clientes.</i>	Identificar grupos de clientes con patrones semejantes de recencia, frecuencia, valor monetario y comportamiento de compra.	Segmentación RFM y técnicas de agrupamiento como K-Means o DBSCAN, acompañadas de interpretación de perfiles.
<i>Necesitamos saber si los descuentos de fin de año funcionaron.</i>	Estimar el efecto de los descuentos sobre el margen neto y distinguirlo de cambios estacionales u otros factores simultáneos.	Análisis causal o cuasiexperimental, según la disponibilidad de grupos de comparación y datos históricos.

Estas metodologías son puntos de partida, no decisiones automáticas. La técnica definitiva dependerá de la calidad de los datos, los supuestos que puedan sostenerse y la forma en que el resultado será utilizado.

1.6. El proceso de solicitud de datos (*Data Request*)

Una vez validado el objetivo, el entendimiento alcanzado debe registrarse. El *Data Request* o solicitud de datos es el mecanismo formal mediante el cual las áreas operativas comunican una necesidad al equipo analítico y documentan su contexto, alcance y criterios de éxito.

Definición — Solicitud de datos (*Data Request*): proceso formal, generalmente representado mediante un formulario o *intake ticket*, que captura la necesidad de negocio y la información necesaria para evaluar, priorizar y ejecutar un proyecto analítico.

El ticket no sustituye la conversación con el stakeholder. Su función es conservar los acuerdos alcanzados durante el proceso de refinamiento y reducir ambigüedades posteriores.

1.6.1. Elementos esenciales de un *Data Request Ticket*

1. Problema o pregunta principal

¿Qué situación se desea comprender, resolver o mejorar?

2. Contexto de negocio

¿Por qué surge la necesidad y por qué es relevante en este momento?

1. Gestión de Requerimientos y Stakeholders

3. Decisión o acción esperada

¿Qué se hará de manera diferente a partir del resultado?

4. Población y alcance

¿Qué clientes, productos, transacciones, regiones o periodos se incluirán?

5. Métricas de éxito

¿Cómo se determinará si el análisis fue útil para el negocio?

6. Fuentes de datos conocidas o sugeridas

¿Existen tablas, archivos, sistemas o proveedores externos que podrían contener la información necesaria?

7. Formato de entrega

¿Se requiere un análisis exploratorio, un reporte, un dashboard, una lista priorizada, un experimento o un modelo integrado en un proceso operativo?

8. Responsable de validación

¿Quién confirmará las definiciones, los resultados y las decisiones derivadas del análisis?

9. Restricciones y riesgos

¿Existen límites de tiempo, privacidad, presupuesto, capacidad operativa o calidad de datos?

10. Fecha de decisión

¿Cuándo debe estar disponible el resultado para que todavía sea útil?

1.7. Cierre

La calidad de un proyecto analítico depende, en gran medida, de la calidad de la pregunta que lo origina. Un objetivo ambiguo puede producir un análisis técnicamente correcto pero irrelevante; un objetivo bien definido, en cambio, permite seleccionar datos, métricas y métodos de forma coherente.

Definir el alcance significa construir un entendimiento compartido entre el stakeholder y el equipo de datos. Para lograrlo, el analista debe preguntar, escuchar, reformular y validar de manera iterativa. El resultado de este proceso no es simplemente una pregunta más técnica, sino un vínculo explícito entre los datos, una decisión y un resultado de negocio.

1.8. Caso de Estudio Práctico: Alineación en TheLook eCommerce

Imagina que eres contratado como líder de datos en **TheLook eCommerce** (el caso de estudio que utilizaremos a lo largo del curso). El Director de Mercadotecnia te aborda con el siguiente requerimiento:

“Necesito que me entregues un reporte con todos los clientes y lo que compran porque las ventas de ropa de mujer han bajado este mes y queremos lanzar un descuento.”

Si actúas únicamente de manera reactiva, extraerás una consulta SQL pesada, generarás un archivo CSV de gigabytes con millones de filas y se lo enviarás por correo electrónico. **Este es un camino directo al fracaso del proyecto:** el stakeholder no podrá abrir el archivo debido a su tamaño, o pasará horas filtrándolo en Excel sin extraer ninguna conclusión clara.

1.8.1. La Conversación de Alineación Estratégica

- **Tú (Analista):** “Para afinar la extracción de datos, ¿cuál es el objetivo de la campaña de descuento? ¿Quieren recuperar clientes inactivos o incentivar a las clientas recurrentes a comprar categorías más caras?”
- **Director (Stakeholder):** “Queremos incentivar a las clientas que ya compran regularmente vestidos de diseñador a que exploren nuestra nueva línea de accesorios de lujo de alto margen.”
- **Tú (Analista):** “Excelente. En lugar de todos los clientes, extraeré un reporte enfocado en el segmento con alta frecuencia de compra en la categoría de mujer/vestidos. Además, aplicaré un modelo de asociación para identificar qué accesorios compran junto con los vestidos para sugerir combinaciones específicas en la campaña.”

Resultado: Redujiste el esfuerzo técnico de extraer bases de datos enteras, entregaste un análisis de alto impacto enfocado en conversión de alto margen, y alineaste las expectativas de negocio desde el primer día.

1.9. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.

1. Gestión de Requerimientos y Stakeholders

- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.

- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Thesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.

1. Gestión de Requerimientos y Stakeholders

- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.

- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.

1. Gestión de Requerimientos y Stakeholders

- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied

- to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

1. Gestión de Requerimientos y Stakeholders

- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.

- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03>.

1. Gestión de Requerimientos y Stakeholders

002.

- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

En organizaciones modernas, los proyectos analíticos raramente son desarrollados por una sola persona. A medida que el volumen de datos crece y las tecnologías se especializan, surge la necesidad de contar con equipos multidisciplinarios. De acuerdo con el marco de gobierno de datos DAMA-DMBOK (DAMA International 2017) y con la caracterización del rol del científico de datos hecha por Davenport y Patil (2012), existen cuatro perfiles principales, cada uno con responsabilidades, herramientas y productos específicos.

2.0.1. Comparativa de Roles en la Ingeniería y Ciencia de Datos

Rol	Definición y Objetivos	Responsabilidades Principales	Herramientas Comunes
Data Engineer (<i>Ingeniero de Datos</i>)	Construye y mantiene la infraestructura física y lógica necesaria para que los datos fluyan de forma segura y eficiente en la empresa.	• Diseña tuberías de datos (pipelines ETL/ELT). • Administra el Data Warehouse y Data Lake. • Optimiza bases de datos.	PySpark, SQL, dbt, Apache Airflow, Docker, Snowflake, BigQuery.
Data Analyst (<i>Analista de Datos</i>)	Limpia, organiza e interpreta datos históricos para extraer conclusiones operativas y apoyar la toma de decisiones del negocio.	• Diseña reportes y tableros interactivos (dashboards). • Realiza análisis exploratorios básicos. • Responde a solicitudes de datos operativas (tickets).	SQL, Tableau, Power BI, Python (Pandas/Polars), Excel, BigQuery.
Data Scientist (<i>Científico de Datos</i>)	Utiliza matemáticas, estadística y algoritmos de Machine Learning para realizar predicciones o automatizar decisiones complejas basadas en datos.	• Desarrolla modelos de regresión, clasificación y clustering. • Realiza experimentos de validación estadística (Pruebas A/B). • Optimiza hiperparámetros.	Python (Scikit-Learn, PyTorch, XGBoost), R, MLflow, Jupyter Notebooks.

Rol	Definición y Objetivos	Responsabilidades Principales	Herramientas Comunes
Data Architect (<i>Arquitecto de Datos</i>)	Define la estructura de datos general de la organización, creando el mapa (blueprint) de integración de datos entre sistemas de la empresa.	• Diseña el esquema conceptual de la base de datos empresarial. • Define políticas de modelado dimensional y gobierno. • Alinea la infraestructura con la estrategia corporativa.	Herramientas de modelado de datos (Erwin, dbdiagram.io), Cloud Architectures.

Las prácticas de modelado dimensional que emplea el Arquitecto de Datos, y que retomaremos en la Parte I del curso, siguen la metodología clásica de Kimball y Ross (2013).

A continuación se detalla gráficamente el flujo de trabajo lógico entre los distintos perfiles de datos:

2.1. El Ciclo de Vida de los Datos

Definición (Ciclo de Vida de los Datos). Secuencia de fases por las que pasa la información dentro de una organización, desde su planificación inicial hasta su eventual eliminación segura, pasando por su ingesta, almacenamiento y análisis (DAMA International 2017).

El ciclo de vida de los datos describe la secuencia de fases por las que pasa la información dentro de una organización. Este ciclo es fundamental para estructurar el desarrollo del curso: la **Parte I (Almacenes de Datos)** abarca principalmente las fases de captura y almacenamiento, mientras que la **Parte II (Minería de Datos)** se enfoca en el análisis estratégico y predictivo.

Tanto la administración de la infraestructura (fase de almacenamiento) como la analítica aplicada (fase de exploración y modelado) no ocurren de manera aislada. Se integran dentro de un **proceso secuencial e iterativo para proyectos de datos**. Antes de construir cualquier modelo o base de datos, debemos pasar por etapas que nos ayudan a comprender la viabilidad del proyecto: el entendimiento del negocio, la recolección e inspección de datos, el preprocesamiento, y la posterior evaluación de resultados para garantizar el éxito de su despliegue real en producción. Esta aproximación estructurada es la que diferencia a los proyectos analíticos profesionales de las pruebas de concepto académicas.

El ciclo consta de **seis etapas clave** (Google 2022):

1. **Planificar (Plan):** Determinar los requisitos de datos para resolver el problema de negocio, definir fuentes necesarias e identificar el equipo necesario.
2. **Capturar (Capture):** Recolectar datos de fuentes internas y externas (bases de datos transaccionales, logs web, APIs). En esta fase los datos son extraídos y preparados.
3. **Gestionar (Manage):** Almacenar los datos de manera estructurada en bases de datos relacionales o almacenes de datos analíticos. Aquí se aplican políticas de seguridad, limpieza inicial y linaje de datos.
4. **Analizar (Analyze):** Explorar los datos mediante estadística y minería de datos (EDA), y entrenar modelos analíticos para responder preguntas clave o automatizar procesos.

5. **Archivar (Archive):** Mover los datos históricos que ya no se consultan frecuentemente a sistemas de almacenamiento pasivos de bajo costo (ej. Glacier o Google Cloud Coldline Storage), protegiéndolos contra modificaciones accidentales.
 6. **Destruir (Destroy):** Eliminar de manera definitiva los datos una vez que ha expirado el periodo requerido de retención de datos, garantizando que no queden residuos ni copias inseguras de datos personales (PII) bajo políticas de borrado seguro.
-

2.2. Fases Críticas de la Gestión de Datos

Independientemente de en qué etapa del ciclo de vida nos encontremos, dos preguntas atraviesan todo el proceso: **qué tan sensibles son los datos que estamos manejando** y **quién tiene permitido acceder a ellos**. Ninguna de las dos se resuelve de forma improvisada; ambas se gobiernan mediante políticas explícitas que el equipo de datos debe conocer antes de tocar una sola tabla.

2.2.1. Datos que Requieren Tratamiento Especial

No todos los datos representan el mismo nivel de riesgo si se exponen o se usan indebidamente. Ciertas categorías —conocidas como **datos sensibles** o de categoría especial— exigen controles adicionales de recolección, almacenamiento y acceso:

- **Información de identificación personal (PII):** nombre, dirección, número de identificación oficial, correo electrónico o cualquier dato que permita identificar a una persona de forma directa o indirecta.
- **Datos de salud:** historiales clínicos, diagnósticos o resultados de laboratorio; en México, protegidos explícitamente como datos personales sensibles por la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Congreso de los Estados Unidos Mexicanos 2010).
- **Datos financieros y biométricos:** números de tarjeta, historial crediticio, huellas dactilares o reconocimiento facial.

Por ello, antes de capturar cualquiera de estos tipos de datos se aplica el **principio de minimización**: recolectar únicamente lo estrictamente necesario para resolver la pregunta de negocio, en lugar de acumular información sensible “por si acaso” que solo incrementa el riesgo sin aportar valor analítico.

2.2.2. Gobernanza, Política de Uso y Control de Acceso

La forma más común de mitigar estos riesgos es trabajar con una **política de uso de datos** (*data usage policy*): un documento formal que define qué datos existen, quién puede consultarlos, bajo qué condiciones y con qué finalidad. En las empresas de hoy en día, esta política suele ser diseñada por el **Data Architect** en conjunto con las áreas legal y de seguridad, y se traduce operativamente en:

- **Control de acceso de mínimo privilegio:** cada usuario o rol recibe una capacidad de acceso específica, limitada únicamente a los datos indispensables para su función —un *Data Analyst* puede consultar ventas agregadas por región, pero no necesariamente el número de tarjeta de un cliente individual— implementado mediante políticas de Gestión de Identidad y Accesos (IAM) y control de acceso basado en roles (RBAC).
- **Encriptación:** de la información en reposo y en tránsito.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

- **Catálogo de datos:** un inventario actualizado que documenta qué campos son sensibles, quién es su responsable (*data owner*) y qué reglas de retención les aplican, evitando interpretaciones erróneas entre equipos.

Este principio de mínimo privilegio —otorgar a cada usuario el acceso indispensable, ni uno más— es hoy el estándar de facto en la industria: es tan relevante para el Data Engineer que configura permisos en Snowflake o BigQuery como para el Data Analyst que solicita acceso a una tabla del Data Warehouse.

2.3. Actividad Guiada: Diseñar un Plan de Ciclo de Vida para TheLook eCommerce

Como actividad introductoria práctica para este curso, diseñarás el plan inicial de datos para la empresa ficticia **TheLook eCommerce**, basándote en la solicitud de requerimientos de la mesa directiva de los departamentos de Mercadotecnia y Ventas.

2.3.1. Escenario de Negocio

TheLook eCommerce ha crecido rápidamente gracias a su oferta multicanal. Sin embargo, su infraestructura de datos actual es caótica. El equipo de Mercadotecnia ha identificado cuatro preguntas de negocio principales que necesitan responder para lanzar campañas efectivas el próximo trimestre.

2.3.2. Ejercicio de Alineación de Objetivos y Planificación de Datos

Para cada pregunta del negocio, asociaremos su objetivo analítico e identificaremos los datos necesarios que deben capturarse y gestionarse en nuestro futuro Data Warehouse.

#	Pregunta de Negocio (Stakeholder)	Objetivo Analítico	Datos Necesarios a Capturar
1	<i>“¿Qué productos deberíamos promocionar juntos en nuestra página principal?”</i>	Descubrir reglas de asociación entre categorías y productos comprados de forma conjunta por los usuarios.	• Historial de pedidos (<code>orders</code> y <code>order_items</code>). • Identificador de producto e historial de clics en la sesión de navegación.
2	<i>“¿Cómo podemos optimizar los niveles de inventario de vestidos de mujer en cada bodega regional?”</i>	Predecir la demanda futura semanal de productos específicos por ubicación geográfica.	• Registros de stock diario en almacén (<code>inventory_items</code>). • Dirección de entrega de compras históricas y códigos postales.

#	Pregunta de Negocio (Stakeholder)	Objetivo Analítico	Datos Necesarios a Capturar
3	“¿Cuál es el valor acumulado a largo plazo de los clientes adquiridos a través de anuncios pagados en Google?”	Calcular el Valor de Vida del Cliente (<i>Customer Lifetime Value</i> - CLV) agrupado por canal de adquisición.	• Tablas de usuarios (users) con fecha de registro y canal de procedencia. • Transacciones históricas totales asociadas a cada identificador único de usuario.
4	“¿Cómo detectamos y prevenimos devoluciones fraudulentas de mercancía de alto costo?”	Desarrollar un clasificador de transacciones sospechosas antes del procesamiento de la devolución.	• Historial de devoluciones por usuario. • Métodos de pago y tiempos transcurridos entre entrega y solicitud de devolución.

2.4. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.

- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.

- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.

- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03>.

002.

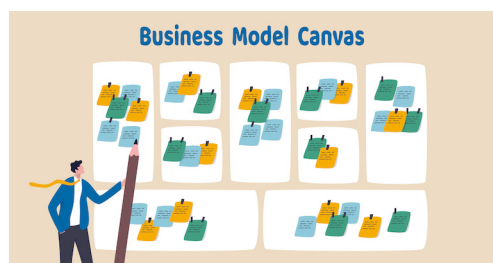
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte III.

Planeación del Proyecto

3. Planeación Real de un Proyecto: El Business Understanding Canvas

En el capítulo anterior aprendiste a identificar stakeholders, ubicarlos en la matriz de poder e interés, y a traducir preguntas cualitativas en objetivos analíticos. Sin embargo, en un proyecto real esa traducción no ocurre de forma espontánea dentro de tu cabeza: se **documenta**. Un equipo de datos profesional deja evidencia escrita de cada conversación con el negocio, y condensa esa evidencia en un artefacto de una sola página que cualquier miembro del equipo —o cualquier stakeholder nuevo— pueda leer en cinco minutos y entender exactamente qué se va a hacer, por qué, y cómo se sabrá si funcionó.



Ese artefacto es el **Business Understanding Canvas**. Es la salida tangible de la primera fase de cualquier metodología estructurada de minería de datos (formalizarás esto más adelante como la fase de *Business Understanding* de **CRISP-DM** (Chapman et al. 2000; Shearer 2000; Wirth y Hipp 2000)). Aquí lo construiremos de forma práctica, aplicado a un caso real, para que cuando llegues al marco metodológico completo ya tengas la intuición de qué problema resuelve cada pieza.

Definición (Business Understanding Canvas). Documento de una página que resume el objetivo de negocio, los criterios de éxito, los supuestos y riesgos, y el plan de un proyecto de datos, sirviendo como contrato de entendimiento entre el equipo analítico y los stakeholders antes de tocar una sola fuente de datos. El formato de una página se inspira en el lenguaje visual de los *canvas* estratégicos popularizado por Osterwalder y Pigneur (2010), aplicado aquí a la fase de entendimiento del negocio de CRISP-DM.

En este capítulo construirás las cinco piezas que componen la documentación de entendimiento de negocio de un proyecto:

1. Una **bitácora de reunión** con el stakeholder.
2. El **Business Understanding Canvas** de una página.
3. De **3 a 4 preguntas de investigación priorizadas**.
4. El **criterio de éxito en términos de negocio**.
5. Una **lista de supuestos y riesgos** (con al menos dos riesgos reales y su mitigación).

Lo haremos continuando el caso de **TheLook eCommerce** introducido en el capítulo anterior, donde el Director de Mercadotecnia solicitó impulsar la nueva línea de accesorios de lujo entre las clientas frecuentes de vestidos de diseñador.

3.1. Paso 1: La Bitácora de la Reunión con el Stakeholder

Antes de sintetizar nada, se documenta la conversación tal como ocurrió. La bitácora (*meeting log*) no es un resumen embellecido: es un registro fiel de quién dijo qué, qué se acordó y qué quedó pendiente. Sirve como fuente de verdad si más adelante hay desacuerdos sobre el alcance del proyecto.

3.1.1. Elementos mínimos de una bitácora

- **Metadatos:** fecha, hora, duración, modalidad y lugar.
- **Participantes:** nombre, rol y su cuadrante en la matriz de poder/interés.
- **Contexto:** qué disparó la reunión.
- **Preguntas y respuestas:** el intercambio real de la conversación de descubrimiento.
- **Decisiones y acuerdos:** qué quedó explícitamente aceptado por ambas partes.
- **Pendientes y próximos pasos:** con dueño y fecha.

3.1.2. Ejemplo aplicado: Kickoff del proyecto “Accesorios de Lujo”

Bitácora de Reunión — Proyecto Accesorios de Lujo (TheLook eCommerce)

Fecha: 3 de agosto de 2026 · **Hora:** 10:00–10:40 (40 min) · **Modalidad:** Videollamada
Participantes:

Nombre	Rol	Cuadrante (Poder/Interés)
Director de Mercadotecnia	Patrocinador del proyecto	Alto Poder / Alto Interés
Gerente de Inventario	Provee datos de stock por bodega	Alto Poder / Bajo Interés
Analista de Datos (tú)	Traductor técnico	—

Contexto: Seguimiento a la solicitud inicial de reporte de clientes; se formaliza como proyecto tras la conversación de alineación del capítulo anterior.

Preguntas formuladas y respuestas:

- “¿Qué significa ‘éxito’ para ustedes en este proyecto: más ingresos, más margen, o retención de clientas VIP?” → **Director:** “Margen. Los accesorios tienen 3x el margen de los vestidos; nos interesa que las clientas que ya gastan mucho añadan accesorios al carrito, no solo captar clientas nuevas.”
- “¿Con qué frecuencia necesitan actualizar las recomendaciones: por campaña, semanal, en tiempo real?” → **Director:** “Por campaña está bien para el piloto. No necesitamos tiempo real todavía.”
- “¿Hay restricciones de inventario que debemos considerar antes de recomendar un accesorio?” → **Gerente de Inventario:** “Sí. Varias piezas de la nueva línea tienen stock limitado en algunas bodegas regionales; no queremos promocionar algo que se agote en 48 horas.”
- “¿Existe un presupuesto o fecha límite fija?” → **Director:** “Necesitamos resultados del piloto antes de la campaña de fin de año, así que en 6 semanas debemos tener algo accionable.”

Decisiones y acuerdos:

- El piloto se enfoca en clientas con alta frecuencia de compra en la categoría vestidos/diseñador (no en toda la base de clientes).
- El éxito se mide en **margen incremental**, no en volumen de ventas ni en métricas de modelo.
- El Gerente de Inventario comparte semanalmente el archivo de stock por bodega para filtrar recomendaciones de productos con baja disponibilidad.

Pendientes / Próximos pasos:

Acción	Dueño	Fecha límite
Compartir acceso a tabla <code>inventory_items</code>	Gerente de Inventario	5 de agosto de 2026
Entregar canvas de entendimiento de negocio para validación	Analista de Datos	7 de agosto de 2026
Validar canvas y dar luz verde a extracción de datos	Director de Mercadotecnia	10 de agosto de 2026

3.2. Paso 2: El Business Understanding Canvas (Una Página)

El canvas condensa la bitácora en seis bloques que cualquier persona del negocio o del equipo técnico puede revisar en minutos. Estos seis bloques no son arbitrarios: corresponden directamente a las tareas que la guía CRISP-DM (Chapman et al. 2000) define para la fase de Business Understanding —determinar objetivos de negocio, evaluar la situación, determinar objetivos de minería de datos y producir un plan de proyecto—, desdobladas aquí en sus criterios de éxito de negocio y técnico. A diferencia de la bitácora —que es cronológica y detallada— el canvas es **atemporal y sintético**: es el documento que se archiva y se consulta durante todo el proyecto.

Observa que el bloque 5 (Éxito Técnico) usa una métrica de modelo (*lift*, confianza), mientras que el bloque 2 (Criterio de Éxito de Negocio) usa una métrica financiera (margen incremental). Esta distinción

es intencional y se profundiza en el siguiente paso.

3.3. Paso 3: Preguntas de Investigación Priorizadas

Rara vez existe una sola pregunta analítica detrás de un proyecto. Es tarea del analista descomponer el objetivo de negocio en varias preguntas de investigación y **priorizarlas** según su impacto en el negocio y la factibilidad de responderlas con los datos disponibles, siguiendo la misma lógica de la matriz de poder/interés del capítulo anterior, ahora aplicada a preguntas en vez de personas.

#	Pregunta de Investigación	Impacto de Negocio	Factibilidad de Datos	Prioridad
1	¿Qué accesorios se compran junto con vestidos de diseñador con mayor frecuencia?	Alto	Alto	1 (Piloto)
2	¿Qué accesorios recomendar cuando el stock regional es limitado, sin frustrar la demanda?	Alto	Medio	2
3	¿El efecto de venta cruzada varía según la región o bodega de origen del pedido?	Medio	Alto	3
4	¿Las clientas que reciben la recomendación aumentan su frecuencia de compra a largo plazo (no solo el ticket)?	Alto	Bajo (requiere datos longitudinales aún no disponibles)	4 (Fuera de alcance del piloto)

La pregunta 1 concentra el esfuerzo del piloto porque combina alto impacto con datos ya disponibles en el Data Warehouse. La pregunta 4, aunque valiosa, requiere una ventana de observación más larga y queda documentada para una segunda iteración, no descartada.

3.4. Paso 4: Criterio de Éxito en Términos de Negocio

Un error común de analistas junior es definir el éxito exclusivamente con métricas de modelo (precisión, *recall*, AUC, *lift*). Esas métricas son necesarias pero **no son el criterio de éxito del proyecto**: son el

criterio de éxito del *modelo*. El stakeholder no puede evaluar si un AUC de 0.83 fue una buena inversión; sí puede evaluar si el margen subió. Esta distinción entre el objetivo de negocio y el objetivo de minería de datos es, según Provost y Fawcett (2013), la fuente más frecuente de proyectos analíticos que producen modelos técnicamente correctos pero comercialmente irrelevantes.

Tipo de Criterio	Ejemplo en este proyecto	¿Quién lo evalúa?
Negocio (el que importa para aprobar o cancelar el proyecto)	+8 % de margen incremental en el segmento piloto vs. grupo de control, en 6 semanas	Director de Mercadotecnia
Técnico (el que usa el equipo de datos para iterar el modelo)	<i>Lift</i> > 1.2 y confianza > 30 % en las reglas de asociación	Analista de Datos

Regla práctica. Todo proyecto debe poder resumir su éxito en una sola oración que un directivo sin conocimientos técnicos pueda repetir de memoria. Si esa oración contiene un término estadístico (AUC, *lift*, silueta), probablemente estás describiendo el éxito técnico, no el de negocio.

3.5. Paso 5: Supuestos y riesgos

Todo canvas simplifica la complejidad del proyecto. Por ello, debe complementarse con una lista de **supuestos** y un registro de **riesgos**.

Un supuesto es una condición que se considera verdadera, aunque todavía no se haya verificado. Un riesgo es un evento incierto que podría afectar el desarrollo o los resultados del proyecto. Los supuestos deben validarse; los riesgos, evaluarse y mitigarse.

3.5.1. Supuestos

Para el piloto se consideran los siguientes supuestos:

- El comportamiento histórico de compra conjunta —por ejemplo, vestido y accesorio— representa razonablemente el comportamiento futuro durante la campaña.
- El archivo de inventario se actualizará semanalmente durante las seis semanas del piloto.
- El segmento de clientas frecuentes contiene suficientes transacciones para generar reglas de asociación confiables.
- El equipo de Mercadotecnia podrá incorporar las recomendaciones en su calendario de campaña.

Los supuestos más importantes deben comprobarse durante las primeras etapas. Si alguno resulta falso, puede ser necesario modificar los datos, el método o el alcance del proyecto.

3.5.2. Clasificación de los riesgos

Clasificar los riesgos permite asignarlos al equipo responsable y evita reducir el análisis únicamente a problemas técnicos. En un proyecto analítico pueden distinguirse cinco categorías principales:

- **Datos:** problemas de calidad, disponibilidad, actualización o representatividad.
- **Analíticos:** sesgo, estacionalidad, sobreajuste, asociaciones espurias o pérdida de desempeño.

3. Planeación Real de un Proyecto: El Business Understanding Canvas

- **Operativos:** falta de inventario, capacidad insuficiente o procesos que no pueden utilizar los resultados.
- **Tecnológicos y de seguridad:** fallas de integración, permisos excesivos, accesos no autorizados o interrupciones de los sistemas.
- **Legales, éticos y reputacionales:** uso indebido de datos personales, discriminación o personalización percibida como invasiva.

También pueden existir riesgos de adopción cuando los usuarios no comprenden, no confían o no utilizan la solución.

3.5.3. Evaluación de los riesgos

Cada riesgo debe evaluarse según su **probabilidad** y su **impacto**. Esta valoración permite priorizar los riesgos que requieren atención inmediata.

Además de la mitigación, conviene registrar:

- la persona responsable;
- la señal que indicaría que el riesgo está ocurriendo;
- el control preventivo;
- y el plan de contingencia.

3.5.4. Registro de riesgos del proyecto

#	Clasificación	Riesgo	Probabilidad	Impacto	Mitigación
1	Privacidad	La extracción de información de clientas VIP podría exponer datos personales fuera del equipo autorizado, incumpliendo la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Congreso de los Estados Unidos Mexicanos 2010).	Media	Alto	Eliminar identificadores innecesarios, aplicar controles IAM/RBAC y mantener registros de acceso.

#	Clasificación	Riesgo	Probabilidad	Impacto	Mitigación
2	Operativo	El modelo podría recomendar accesorios sin inventario suficiente.	Alta	Medio	Validar las recomendaciones contra <code>inventory_items</code> antes de cada campaña y establecer un nivel mínimo de existencias.
3	Analítico	Las asociaciones podrían estar influenciadas por una temporada específica y no repetirse durante el piloto.	Media	Medio	Comparar las reglas en distintas ventanas temporales antes de utilizarlas.
4	Datos	El archivo de inventario podría estar incompleto o desactualizado.	Alta	Alto	Automatizar controles de actualidad y suspender el envío si la fuente no supera las validaciones.
5	Adopción	Mercadotecnia podría no utilizar las recomendaciones si no comprende su origen o utilidad.	Media	Medio	Explicar cada recomendación, capacitar a los usuarios y recopilar retroalimentación durante el piloto.

3.5.5. Tratamiento y seguimiento

Después de evaluar un riesgo, el equipo puede:

1. **Evitarlo:** modificar el diseño para eliminar su causa.
2. **Mitigarlo:** reducir su probabilidad o impacto.
3. **Transferirlo:** compartir su gestión con un proveedor o área especializada.
4. **Aceptarlo:** reconocerlo y preparar una contingencia.

El registro debe revisarse durante el piloto y actualizarse cuando cambien los datos, el modelo o las condiciones operativas.

Un modelo puede ser técnicamente correcto y, aun así, resultar inutilizable si emplea datos sin autorización, recomienda productos agotados o genera resultados que la operación no puede atender. Por ello, los riesgos deben identificarse desde el entendimiento del negocio y revisarse durante todo el proyecto.

3.5.6. Resumen de la gestión de riesgos

3.6. Actividad: Construye tu Propio Business Understanding Canvas

Usando la plantilla en blanco a continuación, elige un escenario de negocio (puede ser uno de los cuatro casos de TheLook eCommerce del capítulo anterior, u otro que definas con tu instructor) y completa las cinco piezas de este capítulo:

1. Escribe una bitácora simulada de una reunión de 30-40 minutos con el stakeholder, incluyendo al menos tres preguntas de descubrimiento y sus respuestas.
2. Completa el canvas de seis bloques.
3. Prioriza entre 3 y 4 preguntas de investigación usando la tabla de impacto/factibilidad.
4. Redacta el criterio de éxito de negocio en una sola oración, y su contraparte técnica.
5. Documenta al menos dos riesgos reales (no genéricos) con su mitigación.

Este documento será la base sobre la que construirás el proyecto de datos a lo largo del curso: cada decisión posterior de modelado en las Partes III a V deberá poder justificarse trazándola de vuelta a alguno de estos cinco elementos.

3.7. Referencias

Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.

Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.

Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.

Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.

Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.

Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.

Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.

- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en*

3. Planeación Real de un Proyecto: El Business Understanding Canvas

- Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.

- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

3. Planeación Real de un Proyecto: El Business Understanding Canvas

- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.

- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2:

3. Planeación Real de un Proyecto: El Business Understanding Canvas

1137-43.

Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.

Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.

Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.

Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.

Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.

MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.

McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.

McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.

McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.

McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.

McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.

Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).

Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP),

- archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin.

3. Planeación Real de un Proyecto: El Business Understanding Canvas

2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal*

- 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.

3. Planeación Real de un Proyecto: El Business Understanding Canvas

- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte IV.

Almacenes de Datos

Parte V.

Arquitectura de almacenes de Datos

4. Introducción

4.1. La Arquitectura en Ciencias de la Computación

En Ciencias de la Computación (CS) y en la Ingeniería de Software, el término **arquitectura** no se refiere al diseño estético de un edificio físico, sino a la definición y diseño de las estructuras que componen un sistema digital complejo.

Definición (Arquitectura de Sistemas de Datos). La organización estructural de los sistemas de hardware, software y almacenamiento que permiten capturar, transformar, organizar y consultar datos a gran escala, definiendo los flujos de información y los límites operativos entre cada componente del sistema.

Una buena arquitectura de datos garantiza que la información esté disponible para quien la necesita, en el formato adecuado, de forma segura y con un rendimiento predecible. Pero esa definición esconde una pregunta más incómoda: **¿por qué no basta con “guardar los datos en un disco y ya”?** ¿Por qué la industria ha invertido cuatro décadas diseñando y rediseñando la forma en que se persisten los datos analíticos?

La respuesta tiene poco que ver con estética o con moda tecnológica, y mucho con tres restricciones físicas muy concretas:

1. **Cómo está organizada la información sobre el disco/memoria determina qué tan rápido (y qué tan caro) es responder una pregunta sobre ella.**
2. **Qué tan rígido o flexible es el sistema de persistencia determina qué tipos de datos puede almacenar de manera eficiente, y qué tanto puede crecer.**
3. **Qué tan estrictas son las garantías de consistencia que ofrece el sistema determina qué tan caro es coordinar escrituras concurrentes.**

Toda arquitectura de almacenamiento —Data Warehouse, Data Lake o Data Lakehouse, que estudiaremos en los siguientes capítulos— es, en el fondo, una decisión de diseño distinta sobre estos tres ejes. Antes de nombrarlas, vale la pena entender *por qué* existen: cuál es su **sentido de ser** desde una perspectiva técnica.

4.2. Eje 1: El costo de leer datos — filas vs. columnas

Cuando un motor de base de datos ejecuta una consulta, el cuello de botella casi nunca es la CPU: es la cantidad de datos que hay que **leer desde disco** para responderla. La forma en que los registros se distribuyen físicamente sobre el almacenamiento —fila por fila o columna por columna— cambia radicalmente cuánto hay que leer.

Definición (Almacenamiento orientado a filas / *Row-oriented*). Modelo de persistencia en el que todos los valores de un mismo registro (fila) se guardan de forma contigua en disco. Es el modelo clásico de los sistemas transaccionales (OLTP).

Definición (Almacenamiento orientado a columnas / *Column-oriented*). Modelo de persistencia en el que todos los valores de un mismo atributo (columna) se guardan de forma contigua en disco, separando cada columna de la tabla en su propio bloque físico. Es el modelo dominante en los sistemas analíticos (OLAP).

4.2.1. Visualización: mismo dato, dos disposiciones físicas distintas

Esta diferencia, que parece un detalle de implementación, tiene consecuencias enormes a escala:

- **Menos I/O:** si una tabla tiene 50 columnas y una consulta analítica solo necesita 3, un motor columnar lee, en teoría, el 6 % de los datos que leería un motor por filas.
- **Mejor compresión:** una columna contiene valores del mismo tipo y con baja variabilidad (por ejemplo, un mismo **área** se repite miles de veces), lo que permite algoritmos de compresión mucho más agresivos que comprimir filas heterogéneas.
- **Ejecución vectorizada:** al tener los valores de una columna contiguos en memoria, el CPU puede aplicar operaciones (sumar, comparar, filtrar) sobre bloques completos usando instrucciones SIMD, en lugar de procesar registro por registro.

Por eso los sistemas transaccionales (bases de datos operacionales de un e-commerce, un banco, una universidad) son **row-oriented** —necesitan escribir y leer registros completos rápidamente— mientras que los sistemas analíticos (Data Warehouse, Data Lakehouse) son **column-oriented**, o usan formatos de archivo columnares como **Parquet** u **ORC**, porque su carga de trabajo dominante son agregaciones sobre volúmenes masivos de filas.

4.2.2. OLTP vs. OLAP: dos cargas de trabajo distintas

La elección entre almacenamiento por filas o por columnas no es arbitraria: responde a que, desde los años 90, la industria reconoce dos tipos de carga de trabajo con necesidades opuestas.

Definición (OLTP / *Online Transaction Processing*). Carga de trabajo dominada por operaciones de escritura y lectura de registros individuales, con alta concurrencia y baja latencia por operación. Es el patrón de las aplicaciones operacionales.

Definición (OLAP / *Online Analytical Processing*). Carga de trabajo dominada por consultas de agregación y exploración sobre grandes volúmenes de filas, típicamente sobre pocas columnas a la vez, con menor concurrencia y tolerancia a mayor latencia por consulta.

Criterio	OLTP	OLAP
Objetivo	Registrar la operación del negocio	Analizar el histórico del negocio
Operación típica	INSERT / UPDATE de un registro	SELECT ... GROUP BY sobre millones de filas
Volumen por operación	Pocas filas, todas sus columnas	Muchas filas, pocas columnas

Criterio	OLTP	OLAP
Almacenamiento natural	Orientado a filas	Orientado a columnas
Usuarios	Aplicación operacional (miles de transacciones concurrentes)	Analistas/BI (pocas consultas, pero pesadas)
Ejemplo	Registrar una venta en un POS	Calcular ventas promedio por región del último año

Esta tabla es la razón última por la que un **Data Warehouse** nunca se alimenta directamente de la base transaccional de producción: son dos cargas de trabajo que compiten por los mismos recursos y que, además, favorecen disposiciones físicas de almacenamiento opuestas.

4.3. Eje 2: El costo de la rigidez — datos homogéneos vs. heterogéneos

El segundo eje no tiene que ver con *cómo* se organiza el dato dentro de la tabla, sino con **qué tan dispuesto está el sistema de persistencia a aceptar datos que no encajan en una tabla**.

Una base de datos relacional clásica exige que, antes de insertar un dato, este se ajuste a un esquema fijo: columnas, tipos, longitudes. Esto es excelente para garantizar consistencia, pero es una barrera de entrada para datos que **no nacen tabulares**: un archivo de log, una imagen de un producto, un JSON anidado que cambia de estructura entre un evento y otro, una transcripción de audio.

Durante décadas, la única opción para ese tipo de datos era descartarlos, transformarlos forzosamente en tablas (perdiendo información en el proceso) o almacenarlos en sistemas de archivos completamente aislados de la infraestructura analítica.

El avance que habilitó a los sistemas modernos de persistencia a resolver esto fue la combinación de dos ideas:

- **Almacenamiento de objetos** (*object storage*, como Amazon S3 o Google Cloud Storage): un sistema de persistencia que no exige un esquema para aceptar un archivo, es económico por naturaleza (paga por byte, no por rendimiento de disco de alta gama) y escala de forma prácticamente ilimitada.
- **Formatos de archivo abiertos y autodescriptivos** (Parquet, Avro, JSON, ORC): archivos que llevan su propio esquema embebido, permitiendo que datos estructurados, semi-estructurados y no estructurados convivan en el mismo almacén sin perder la posibilidad de ser leídos e interpretados después.

Esta combinación es lo que permite que un **Data Lake** —y, más adelante, un **Data Lakehouse**— almacene en el mismo lugar una tabla de transacciones, un conjunto de imágenes de productos y un registro de eventos de clics de una aplicación web, sin que el sistema de almacenamiento le exija a ninguno de ellos ajustarse a una estructura predefinida antes de guardarse. El esquema, si se necesita, se interpreta **en el momento de la lectura**, no al momento de la escritura.

4.3.1. Escalar verticalmente vs. escalar horizontalmente

La flexibilidad de un sistema de persistencia también depende de cómo crece cuando el volumen de datos supera lo que un solo servidor puede manejar. Existen dos estrategias:

Definición (Escalamiento vertical / *Scale-up*). Estrategia que aumenta la capacidad de un sistema añadiendo más recursos (CPU, RAM, disco) a una única máquina. Es el patrón tradicional de las bases de datos relacionales.

Definición (Escalamiento horizontal / *Scale-out*). Estrategia que aumenta la capacidad de un sistema añadiendo más máquinas (nodos), distribuyendo los datos y el trabajo entre ellas. Es el patrón nativo del almacenamiento de objetos y del cómputo distribuido.

El escalamiento vertical tiene un techo físico y económico: existe un límite de CPU/RAM que un solo servidor puede alojar, y el costo de esas máquinas de gama alta crece mucho más rápido que linealmente conforme se acercan a ese límite. Es una restricción que las bases de datos relacionales de un Data Warehouse clásico heredan directamente.

El escalamiento horizontal, en cambio, permite crecer añadiendo nodos de hardware relativamente económico (*commodity hardware*), lo cual es lo que hace viable, en términos de costo, almacenar petabytes de datos heterogéneos en un Data Lake: el sistema no necesita un servidor cada vez más costoso, sino simplemente más espacio distribuido entre miles de discos comunes.

4.4. Eje 3: El costo de la consistencia — ACID vs. BASE

El tercer eje no tiene que ver con la velocidad de lectura ni con qué tipos de datos se aceptan, sino con **qué tan estrictamente garantiza el sistema que los datos siempre reflejen un estado correcto**, incluso cuando ocurren escrituras simultáneas o fallas a mitad de una operación.

Definición (ACID). Conjunto de garantías —*Atomicity* (atomicidad), *Consistency* (consistencia), *Isolation* (aislamiento) y *Durability* (durabilidad)— que aseguran que cada transacción se ejecute por completo o no se ejecute en absoluto, y que el sistema nunca quede en un estado intermedio o corrupto.

Definición (BASE). Conjunto de garantías más laxas —*Basically Available* (básicamente disponible), *Soft state* (estado flexible) y *Eventual consistency* (consistencia eventual)— que priorizan la disponibilidad y la capacidad de seguir operando bajo fallas o alta distribución, aceptando que distintos lectores puedan ver temporalmente versiones ligeramente distintas del dato.

Garantizar ACID tiene un costo real: exige coordinación (bloqueos, registros de transacciones, verificación de reglas) que se vuelve cada vez más cara conforme el sistema se distribuye en más nodos. Por eso los sistemas relacionales clásicos —diseñados para escalar verticalmente, en una sola máquina o un clúster pequeño y fuertemente acoplado— pueden ofrecer ACID de forma económica, mientras que los sistemas de almacenamiento de objetos masivamente distribuidos, pensados para escalar horizontalmente a miles de nodos, adoptaron tradicionalmente garantías BASE: aceptan una ventana de inconsistencia a cambio de disponibilidad y escala casi ilimitada.

Este es precisamente el problema que resolverá más adelante el **Data Lakehouse**: introducir una capa de metadatos transaccionales que reintroduce garantías ACID *sobre* un almacenamiento de objetos que, por diseño, nació BASE.

4.5. El sentido de ser de las arquitecturas de almacenamiento

Ningún sistema optimiza simultáneamente los tres ejes al máximo: **velocidad de consulta** (filas vs. columnas), **flexibilidad para persistir datos heterogéneos** (rigidez del esquema y del escalamiento) y **garantías de consistencia** (ACID vs. BASE) compiten entre sí, porque cada garantía adicional tiene un costo de rigidez o de cómputo.

Esa tensión de tres vías es exactamente el problema que cada arquitectura de datos intenta resolver con un balance distinto:

- Un sistema puede priorizar el **eje 1** y el **eje 3** al extremo —almacenamiento columnar y ACID estricto— aceptando datos únicamente estructurados a cambio de un rendimiento de consulta extraordinario y consistencia total.
- Un sistema puede priorizar el **eje 2** al extremo —escalamiento horizontal y BASE— aceptando cualquier tipo de dato y creciendo casi sin límite, a cambio de renunciar a garantías de consistencia inmediata.
- Un sistema puede intentar obtener lo mejor de los tres mundos, incorporando una capa adicional que reintroduzca control transaccional (ACID) sobre un almacenamiento de objetos flexible y escalado horizontalmente.

En los próximos capítulos estudiaremos, respectivamente, el **Data Warehouse**, el **Data Lake** y el **Data Lakehouse**: tres materializaciones concretas de estas decisiones de diseño. Lo importante, antes de entrar en el detalle de cada una, es no perder de vista que ninguna de estas arquitecturas es una moda: cada una existe porque resuelve, con distintas concesiones, el mismo trío de restricciones físicas de eficiencia que acabamos de describir.

5. Data Warehouse

5.1. Datos para la toma de decisiones

Las organizaciones modernas generan grandes volúmenes de datos diariamente: ventas, transacciones financieras, registros académicos, consultas médicas, interacciones digitales, entre otros.

Sin embargo, operar el negocio no es lo mismo que analizarlo. Los sistemas operacionales están diseñados para registrar transacciones de manera eficiente, pero no para responder preguntas estratégicas.

Por ejemplo:

- Un sistema de ventas permite registrar compras.
- Pero responder preguntas como:
 - ¿Qué producto se vende más en el último año?
 - ¿Cómo han evolucionado las ventas en los últimos cinco años?
 - ¿Qué sucursal tiene mejor desempeño?

requiere una estructura distinta, optimizada para análisis histórico, agregación y comparación temporal.

La diferencia puede resumirse como un cambio de perspectiva: los sistemas operacionales observan cada evento mientras ocurre; un Data Warehouse reúne esos eventos para explicar cómo cambia el negocio a lo largo del tiempo. El primero responde «¿qué acaba de pasar?»; el segundo ayuda a responder «¿qué patrón se está formando y qué decisión conviene tomar?».

En este contexto, resulta fundamental comprender que el análisis de datos no comienza directamente con la tecnología, sino con la definición clara del problema de negocio. Una de las metodologías más utilizadas para estructurar este proceso es **CRISP-DM** (*Cross Industry Standard Process for Data Mining*).

CRISP-DM propone un marco sistemático compuesto por seis fases:

1. **Comprensión del negocio:** identificar los objetivos estratégicos y traducirlos en preguntas analíticas concretas.
2. **Comprensión de los datos:** explorar las fuentes disponibles, evaluar calidad y detectar posibles inconsistencias.
3. **Preparación de los datos:** limpiar, transformar e integrar la información.
4. **Modelado:** aplicar técnicas analíticas o modelos estadísticos.
5. **Evaluación:** verificar si los resultados responden adecuadamente a los objetivos iniciales.
6. **Despliegue:** implementar los hallazgos en la operación o en sistemas de soporte a decisiones.

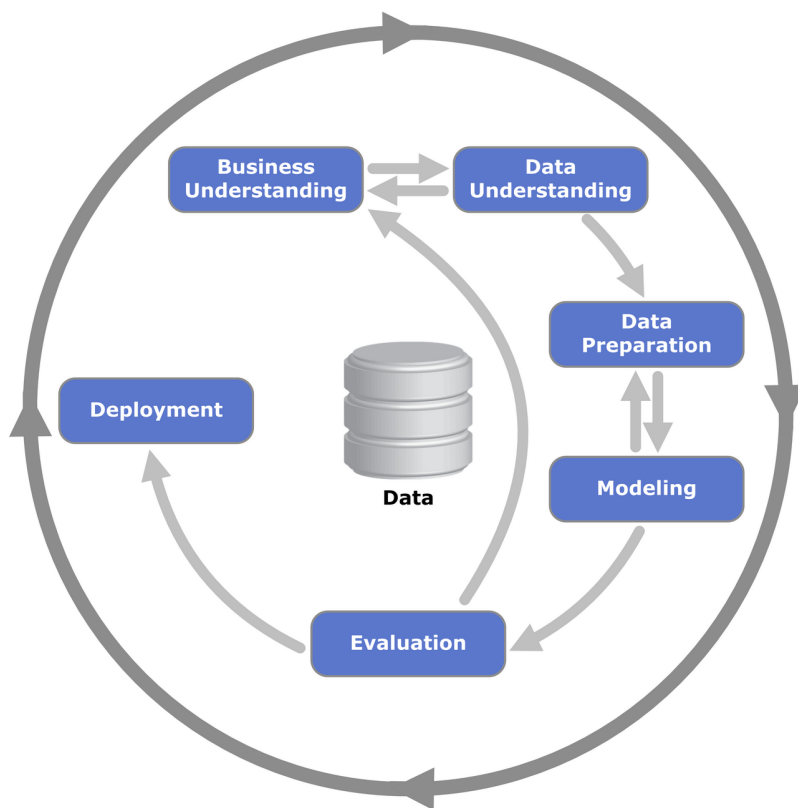


Figura 5.1.: Figura 1. Proceso CRISP-DM

Figura 1. Diagrama del proceso CRISP-DM. Fuente: Wikimedia Commons, “CRISP-DM Process Diagram”, dominio público.

Es importante destacar que CRISP-DM **no es un proceso lineal**, sino un modelo iterativo. En la práctica, las fases se retroalimentan constantemente. Durante el modelado pueden detectarse problemas en la calidad de los datos que obliguen a regresar a la fase de preparación. En la evaluación puede descubrirse que el problema de negocio fue formulado de manera incompleta, lo que implica volver a la primera fase. Incluso después del despliegue, los resultados obtenidos suelen generar nuevas preguntas estratégicas, reiniciando el ciclo.

Esta naturaleza iterativa es coherente con la realidad organizacional: la toma de decisiones es un proceso continuo de refinamiento y aprendizaje. Así, antes de construir tablas, definir dimensiones o elegir herramientas tecnológicas, es necesario responder preguntas como:

- ¿Qué decisiones desea mejorar la organización?
- ¿Qué métricas son críticas?
- ¿Qué horizonte temporal se requiere analizar?

La arquitectura de datos debe ser flexible y escalable, permitiendo incorporar nuevas fuentes, nuevas métricas y nuevas dimensiones conforme evolucionan las necesidades del negocio. En otras palabras, la arquitectura de datos no es un fin en sí mismo; es el soporte técnico que permite convertir datos operacionales en conocimiento útil dentro de un ciclo continuo de mejora y toma de decisiones.

Desde esta perspectiva, CRISP-DM define el ciclo de trabajo analítico, mientras que el Data Warehouse aporta una memoria organizacional confiable para recorrerlo. No todos los proyectos de CRISP-DM

requieren un Data Warehouse, pero cuando las preguntas exigen integrar varias fuentes, comparar periodos y reutilizar métricas consistentes, el almacén de datos se vuelve una pieza central.

5.2. Data Warehouse

William H. Inmon (1996) define un Data Warehouse como:

“A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management’s decision-making process.”

En español, un Almacén de Datos es una colección de datos orientada a temas, integrada, variante en el tiempo y no volátil, diseñada para apoyar la toma de decisiones.

Las cuatro propiedades de la definición no son características aisladas. Juntas explican por qué un Data Warehouse puede ofrecer una versión coherente y durable del negocio:

Un Data Warehouse es **orientado a temas**, lo que significa que no se organiza por aplicaciones o sistemas operacionales, sino por entidades relevantes para el negocio. Por ejemplo, en una compañía de seguros, el almacén de datos se estructura alrededor de conceptos como cliente, póliza o prima, y no por productos específicos como seguro de automóvil o seguro de vida. El enfoque está en analizar entidades del negocio y no módulos técnicos.

Es también **variante en el tiempo**, ya que mantiene información histórica. Los datos no se sobrescriben ni se actualizan como en un sistema transaccional; la nueva información se agrega al repositorio. Esto permite analizar tendencias a lo largo de cinco, diez años o más, y responder preguntas como cómo ha evolucionado el número de diagnósticos en los últimos años o qué patrones se observan en el crecimiento de ventas.

Además, es **integrado**, porque consolida información proveniente de múltiples sistemas. Por ejemplo, si un sistema registra el género como “m” y “f” y otro como 0 y 1, antes de integrarlos deben transformarse para mantener consistencia. Lo mismo ocurre con formatos de fecha distintos, monedas diferentes o nombres de productos inconsistentes. La integración implica procesos de limpieza, transformación y estandarización de datos.

Finalmente, es **no volátil**, lo que significa que se trata de un repositorio físicamente separado de los sistemas operacionales. En él se realizan principalmente operaciones de carga y consulta de datos, pero no actualizaciones frecuentes, eliminaciones ni transacciones. Su propósito es analítico y no operativo.

En conjunto, estas propiedades permiten que una métrica conserve el mismo significado al atravesar áreas y periodos. Por ejemplo, comparar la tasa de abandono de dos generaciones solo es válido si la identidad del estudiante, la definición de abandono y el calendario académico fueron integrados con reglas comunes, y si los valores históricos se conservaron.

Referencia:

Inmon, W. H. *Building the Data Warehouse*, 2nd ed., 1996.

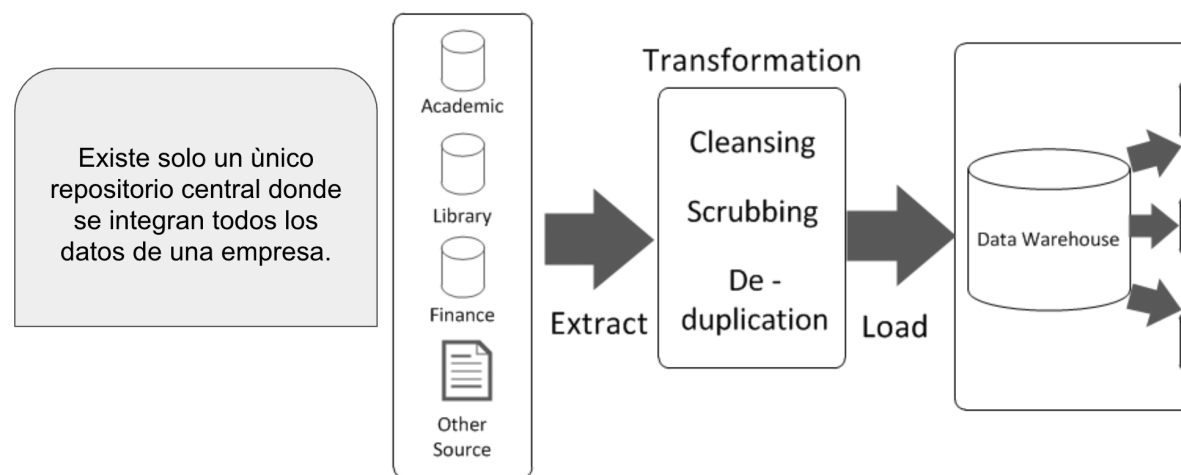
6. Arquitectura de un Data Warehouse

Un Data Warehouse no debe entenderse únicamente como un conjunto de tablas organizadas para consulta. En realidad, es una arquitectura integral de datos cuyo propósito es separar claramente la operación diaria del análisis estratégico. Esta arquitectura permite integrar información proveniente de múltiples sistemas, conservar historia y ofrecer estructuras optimizadas para la toma de decisiones.

La separación también protege ambos tipos de trabajo: las transacciones pueden continuar con baja latencia y las consultas analíticas pueden recorrer millones de registros sin competir directamente con la operación. Entre ambos mundos existe una cadena de transformación que documenta de dónde proviene cada dato y qué reglas se le aplicaron.

6.1. Modelo de 3 Capas

La arquitectura clásica de un Data Warehouse suele representarse mediante un modelo de tres capas lógicas. Estas capas no necesariamente corresponden a servidores físicos distintos, sino a niveles funcionales dentro



del ecosistema de datos.

1. Capa de fuentes (Operational Layer).

En este nivel residen los sistemas que soportan la actividad diaria de la organización. En una empresa pueden incluirse sistemas ERP que registran ventas y facturación, CRM que gestionan clientes, plataformas académicas que almacenan calificaciones o sistemas hospitalarios que documentan consultas médicas. También forman parte de esta capa archivos planos como CSV o Excel, APIs externas que proveen información pública y registros de aplicaciones (logs).

El objetivo de esta capa es transaccional: registrar eventos de manera rápida, consistente y segura. No está diseñada para análisis histórico profundo ni para responder preguntas agregadas complejas. Por ejemplo, un sistema hospitalario puede indicar cuántos pacientes están siendo atendidos hoy, pero no está optimizado para analizar la evolución de diagnósticos durante los últimos diez años.

2. Capa de integración (ETL / Data Integration Layer).

Esta capa constituye el núcleo técnico del Data Warehouse. En ella se ejecutan los procesos de extracción, limpieza, transformación e integración de datos. Es aquí donde se resuelven inconsistencias entre sistemas: un género registrado como “M/F” en un sistema y como “0/1” en otro debe unificarse; formatos de fecha distintos deben homogenizarse; monedas diferentes deben convertirse a un estándar común.

Asimismo, se eliminan duplicados, se generan claves sustitutas y se aplican reglas de negocio que permiten consolidar información proveniente de múltiples fuentes. Dependiendo del enfoque arquitectónico adoptado, esta capa puede materializarse como un **Enterprise Data Warehouse (EDW)** central integrado, en el que se consolida toda la información corporativa antes de exponerla a los usuarios, o bien puede consistir en procesos que alimentan directamente modelos dimensionales específicos para cada área del negocio. En cualquier caso, su función es garantizar coherencia, calidad e integración de datos a nivel empresarial.

3. Capa de consumo (Presentation / Analytics Layer).

En este nivel los datos ya han sido integrados y estructurados para facilitar el análisis. Aquí se implementan data marts, modelos dimensionales (como esquemas estrella), tableros de Business Intelligence y herramientas de visualización. Es el punto donde analistas, directivos y responsables de negocio interactúan con la información para responder preguntas estratégicas.

Por ejemplo, en un grupo de clínicas esta capa permitiría consultar la evolución anual de pacientes por clínica, el tiempo promedio de espera por especialidad o la distribución de diagnósticos por región. La característica esencial es que los datos están optimizados para consulta, agregación y comparación temporal, no para registrar nuevas transacciones.

6.2. Data Mart

Un **Data Mart** es un subconjunto especializado del Data Warehouse orientado a un área específica del negocio. Mientras que el Data Warehouse integra información a nivel corporativo, el Data Mart concentra datos relevantes para un dominio particular, como ventas, finanzas, recursos humanos o trayectoria académica.

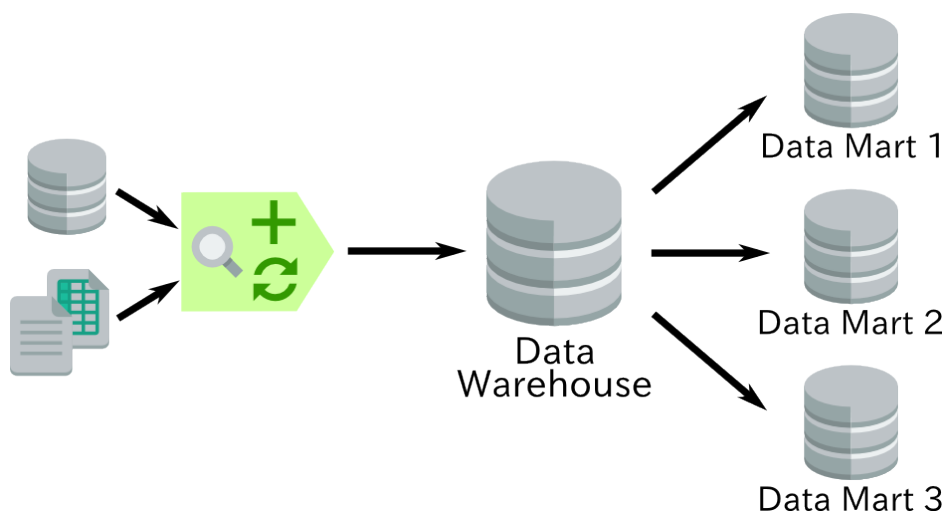


Figura 6.1.: Data Marts dentro de la arquitectura de un Data Warehouse

Por ejemplo, en una universidad puede existir un Data Mart académico que almacene hechos relacionados con inscripciones, calificaciones y créditos, junto con dimensiones como estudiante, asignatura y tiempo. En una empresa comercial, un Data Mart de ventas podría contener hechos de transacciones y dimensiones como producto, cliente y región.

El objetivo principal de un Data Mart es facilitar el análisis focalizado y mejorar el rendimiento de consultas dentro de un área concreta. Generalmente se implementa utilizando modelado multidimensional (por ejemplo, esquemas estrella), ya que este formato está optimizado para agregaciones y reportes.

Dependiendo de la estrategia arquitectónica adoptada, los Data Marts pueden construirse como derivaciones de un Data Warehouse central (enfoque top-down) o desarrollarse inicialmente por proceso de negocio e integrarse posteriormente mediante dimensiones compartidas (enfoque bottom-up).

Por tanto, un Data Mart no es simplemente un almacén pequeño. Su valor depende de que sus métricas y dimensiones mantengan coherencia con el resto de la organización. Un Data Mart de ventas y otro de inventario, por ejemplo, solo pueden analizarse juntos si ambos entienden «producto», «sucursal» y «tiempo» de la misma manera.

6.3. Data Warehouse vs Data Lake

En el contexto actual de gestión de datos, es importante mencionar brevemente el concepto de **Data Lake**, ya que suele confundirse con un Data Warehouse.

Un Data Lake es un repositorio centralizado que almacena datos en su formato original, ya sean estructurados, semiestructurados o no estructurados. A diferencia del Data Warehouse, que requiere procesos de limpieza e integración antes de almacenar la información, el Data Lake adopta un enfoque más flexible: primero almacena los datos y posteriormente los transforma cuando se necesitan para análisis.

Por ejemplo, un Data Warehouse almacenará ventas consolidadas y normalizadas listas para consulta. En cambio, un Data Lake puede almacenar registros crudos de sensores, imágenes médicas, archivos JSON, logs de aplicaciones o grandes volúmenes de texto sin procesar.

La diferencia clave radica en su propósito:

- El **Data Warehouse** está diseñado para análisis estructurado y toma de decisiones empresariales.
- El **Data Lake** está orientado al almacenamiento masivo y flexible de datos, especialmente útil en escenarios de Big Data y ciencia de datos.

En arquitecturas modernas, ambos pueden coexistir. El Data Lake actúa como repositorio de datos crudos, mientras que el Data Warehouse ofrece estructuras optimizadas para análisis empresarial.

7. Enfoques de implementación: Kimball vs Inmon

Existen dos enfoques clásicos para construir un DW completo: Kimball y Inmon. Ambos aparecen como referencias bibliográficas en el material del curso: Kimball como referente del modelado dimensional e Inmon como referente de la arquitectura de Data Warehouse.

Ambos enfoques persiguen información integrada y útil, pero comienzan en extremos diferentes. Kimball parte de procesos de negocio concretos y los conecta gradualmente; Inmon parte de un núcleo empresarial integrado y luego publica estructuras de consumo. La dirección del recorrido determina qué valor aparece primero y dónde se concentra la complejidad.

7.1. Enfoque Kimball (bottom-up, bus architecture)

El enfoque Kimball prioriza entregar valor temprano mediante data marts dimensionales (usualmente en estrella) construidos por procesos de negocio. La idea central es que distintos data marts se integren a través de dimensiones conformadas (conformed dimensions), es decir, dimensiones compartidas y coherentes (por ejemplo, la misma definición de “Tiempo” y “Producto” para ventas y devoluciones).

Rasgos característicos:

- Implementación incremental: se construye un data mart (p. ej., Ventas) y se amplía gradualmente (p. ej., Inventario, Marketing).
- Modelo objetivo: dimensional, orientado a consulta.
- Énfasis en la capa semántica: definiciones de métricas consistentes (por ejemplo, qué significa “venta neta”).
- Alta utilidad para BI/analítica desde etapas tempranas.

Ventajas:

- Entrega rápida de resultados a usuarios.
- Consultas simples y rendimiento alto.
- Ajuste natural a preguntas de negocio, que constituyen el punto de partida recomendado en el curso.

Riesgos o retos:

- Requiere disciplina fuerte para definir dimensiones conformadas; si no, se terminan construyendo “silos” (varios data marts incompatibles).
- El gobierno de datos y la integración pueden volverse complejos si se crece sin estándares.

Metodología Multidimensional de Ralph Kimball

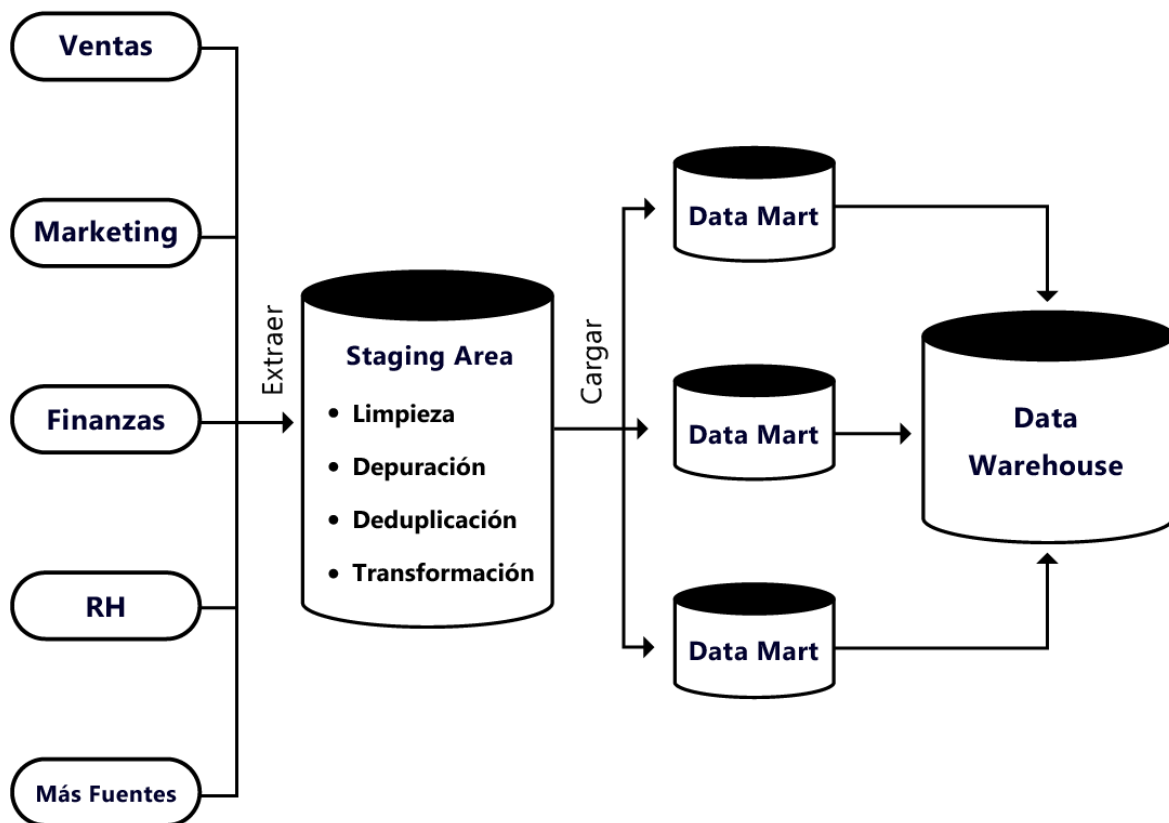


Figura 7.1.: Enfoque Kimball para construcción de Data Warehouse

Metodología Relacional Bill Inmon

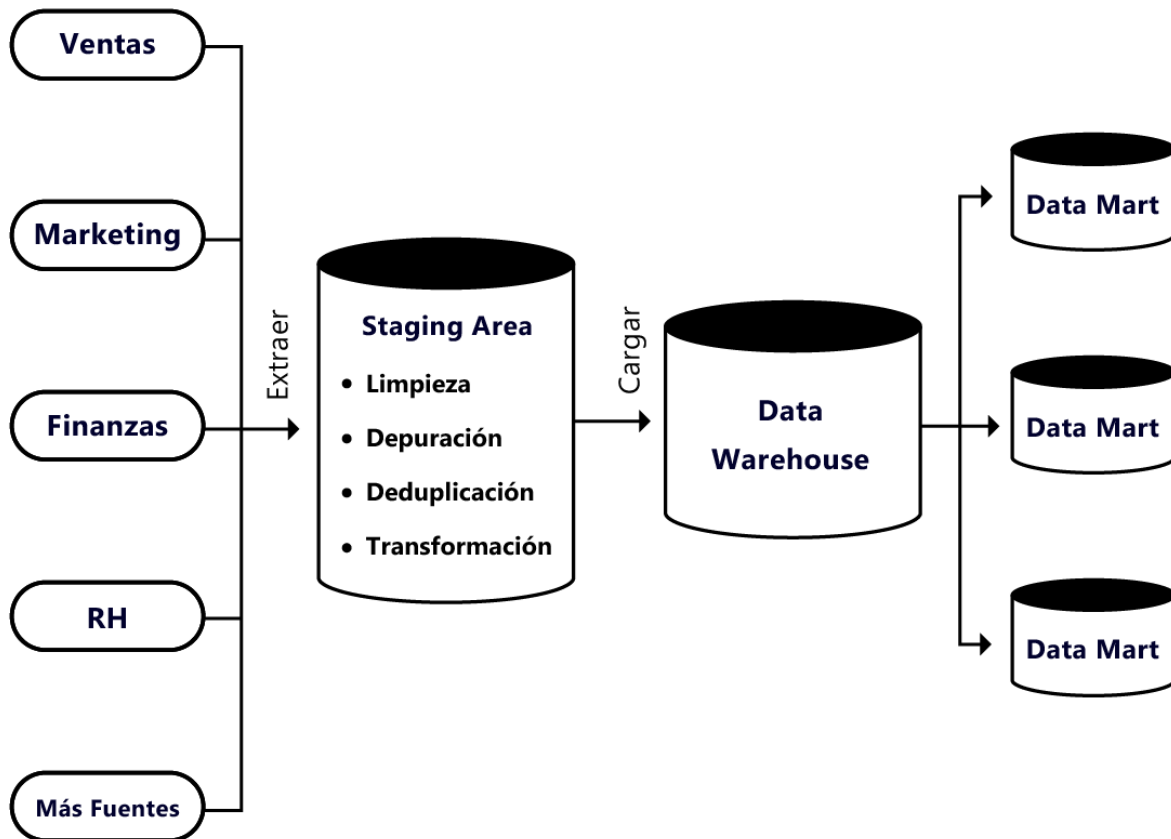


Figura 7.2.: Enfoque Inmon para construcción de Data Warehouse

7.2. Enfoque Inmon (top-down, EDW primero)

El enfoque Inmon propone construir primero un Enterprise Data Warehouse (EDW) central e integrado (frecuentemente en un modelo más normalizado), y después derivar data marts o estructuras dimensionales para consumo analítico.

Rasgos característicos:

- La integración empresarial es prioritaria: definiciones corporativas únicas desde el inicio.
- Se enfatiza un “repositorio central” donde se armonizan entidades y reglas (cliente, producto, organización).
- Los data marts o modelos dimensionales se construyen a partir del EDW, como vistas o capas derivadas.

Ventajas:

- Alta consistencia y control de definiciones a nivel corporativo.
- Favorece gobierno de datos, auditoría, linaje y estándares.

Riesgos o retos:

- Mayor tiempo hasta generar valor visible si el EDW se vuelve un proyecto grande.
- Es más sensible a cambios de requerimientos porque “primero se integra todo” y luego se habilitan casos de uso.

7.3. Cómo elegir adecuadamente entre Kimball e Inmon

La elección no debería plantearse como “cuál es mejor”, sino “cuál es más adecuado” al contexto institucional, al horizonte de tiempo y a la madurez del gobierno de datos. Criterios prácticos:

1. Urgencia de valor analítico y recursos disponibles

- Si se necesita entregar reportes y tableros funcionales rápidamente, y el equipo es pequeño/mediano, Kimball suele ser más viable: se construye un primer data mart y se itera.
- Si la organización puede sostener una inversión inicial grande y su prioridad es consolidar definiciones corporativas desde el inicio, Inmon puede ser más apropiado.

2. Grado de heterogeneidad y fragmentación de fuentes

- Cuando existen muchas fuentes dispares y hay conflictos fuertes de definición (por ejemplo, “cliente activo” significa cosas distintas en áreas distintas), un enfoque tipo Inmon ayuda a forzar una integración central.
- Si las fuentes son manejables y el problema principal es acelerar el análisis por proceso de negocio (ventas, trayectoria académica, atención médica), Kimball suele funcionar bien.

3. Gobierno de datos, auditoría y trazabilidad

- Si hay requerimientos estrictos de auditoría, linaje, cumplimiento o trazabilidad (por ejemplo, regulaciones y auditorías frecuentes), el enfoque top-down facilita un marco centralizado.
- Si el gobierno de datos aún está en formación, un enfoque incremental puede ser más realista, siempre y cuando se establezcan estándares mínimos (dimensiones conformadas, glosario de métricas, versionado de reglas).

4. Naturaleza de los casos de uso

- Casos muy orientados a métricas agregadas y navegación dimensional (tendencias por tiempo, comparativos por región, cohortes) se benefician de dimensionalidad desde el inicio, favoreciendo Kimball.
- Casos de integración empresarial transversal (master data robusto, conciliación entre sistemas, integración de dominios complejos) favorecen Inmon o, al menos, una capa central fuerte.

En la práctica moderna es común un enfoque híbrido: se puede construir una capa integrada (tipo EDW o “core”) y, en paralelo o después, exponer modelos dimensionales para BI. El punto esencial es mantener coherencia: definir claramente qué capa es “fuente de verdad” para definiciones y cómo se publican métricas y dimensiones para consumo analítico.

La elección, entonces, no termina en una etiqueta arquitectónica. Debe traducirse en decisiones verificables: quién define cada métrica, cómo se comparten las dimensiones, dónde se conserva la historia, cómo se rastrea el linaje y qué producto analítico se entregará primero.

8. Ejemplo práctico: Facultad de Ciencias

En un contexto universitario, el sistema de calificaciones registra eventos puntuales como la inscripción de un estudiante en una asignatura o la obtención de una nota al finalizar el semestre. Sin embargo, analizar la trayectoria académica completa de un estudiante requiere una visión mucho más amplia: integrar múltiples periodos, generaciones, asignaturas, planes de estudio y, en muchos casos, información administrativa y financiera asociada.

Los sistemas operacionales académicos están diseñados para soportar procesos diarios —inscripciones, captura de calificaciones, emisión de constancias— pero no para realizar análisis longitudinales o comparativos complejos. Por ejemplo, responder preguntas como:

- ¿Cómo ha evolucionado el desempeño promedio por generación en los últimos diez años?
- ¿Qué cohortes presentan mayores tasas de abandono?
- ¿Existe relación entre modalidad de ingreso y desempeño académico?

implica integrar información dispersa en distintos sistemas y conservar historia de manera estructurada.

Un **Data Warehouse académico** permite consolidar esta información en un repositorio histórico, integrado y consistente, diseñado específicamente para análisis estratégico.

8.1. Fuentes de datos

En una Facultad de Ciencias, la información relevante para análisis institucional suele encontrarse distribuida en múltiples sistemas independientes. Entre ellos pueden incluirse:

- Sistemas de control escolar que registran inscripciones, calificaciones y créditos.
- Sistemas de admisiones que almacenan información de ingreso, generación y antecedentes académicos.
- Sistemas administrativos y financieros que gestionan pagos, becas y apoyos.
- Sistemas de recursos humanos con información sobre profesores y carga docente.
- Archivos externos como encuestas institucionales o evaluaciones académicas.

Cada uno de estos sistemas cumple una función operativa específica. Sin embargo, ninguno por sí mismo ofrece una visión integral del desempeño académico institucional a lo largo del tiempo. La fragmentación de fuentes dificulta el análisis global y puede generar inconsistencias en métricas y definiciones.

8.2. Procesos de integración (ETL)

Para transformar datos operacionales en información estratégica es necesario un proceso de integración sistemático.

En esta etapa se extrae periódicamente la información de los distintos sistemas, se validan y corrigen inconsistencias, se homologan formatos y se consolidan registros que pertenecen a la misma entidad institucional. Por ejemplo, un estudiante puede aparecer con variaciones en su nombre entre sistemas

8. Ejemplo práctico: Facultad de Ciencias

distintos; el proceso de integración debe identificar que se trata de la misma persona y unificar su información.

Asimismo, se normalizan conceptos institucionales: nombres de carreras, códigos de asignatura, periodos académicos y categorías administrativas deben definirse de manera coherente para todo el repositorio.

El resultado de estos procesos no es simplemente una copia de los sistemas originales, sino un conjunto de datos consolidados, limpios y preparados para análisis histórico.

8.3. Consolidación en el Data Warehouse

Una vez integrados, los datos se almacenan en un repositorio central que conserva información histórica y garantiza consistencia institucional. Este repositorio no está orientado a registrar nuevas transacciones, sino a permitir consultas analíticas eficientes y comparaciones longitudinales.

Desde el punto de vista arquitectónico, el Data Warehouse académico se convierte en la base para construir vistas analíticas especializadas —como un data mart académico— desde donde pueden generarse reportes, tableros y estudios institucionales.

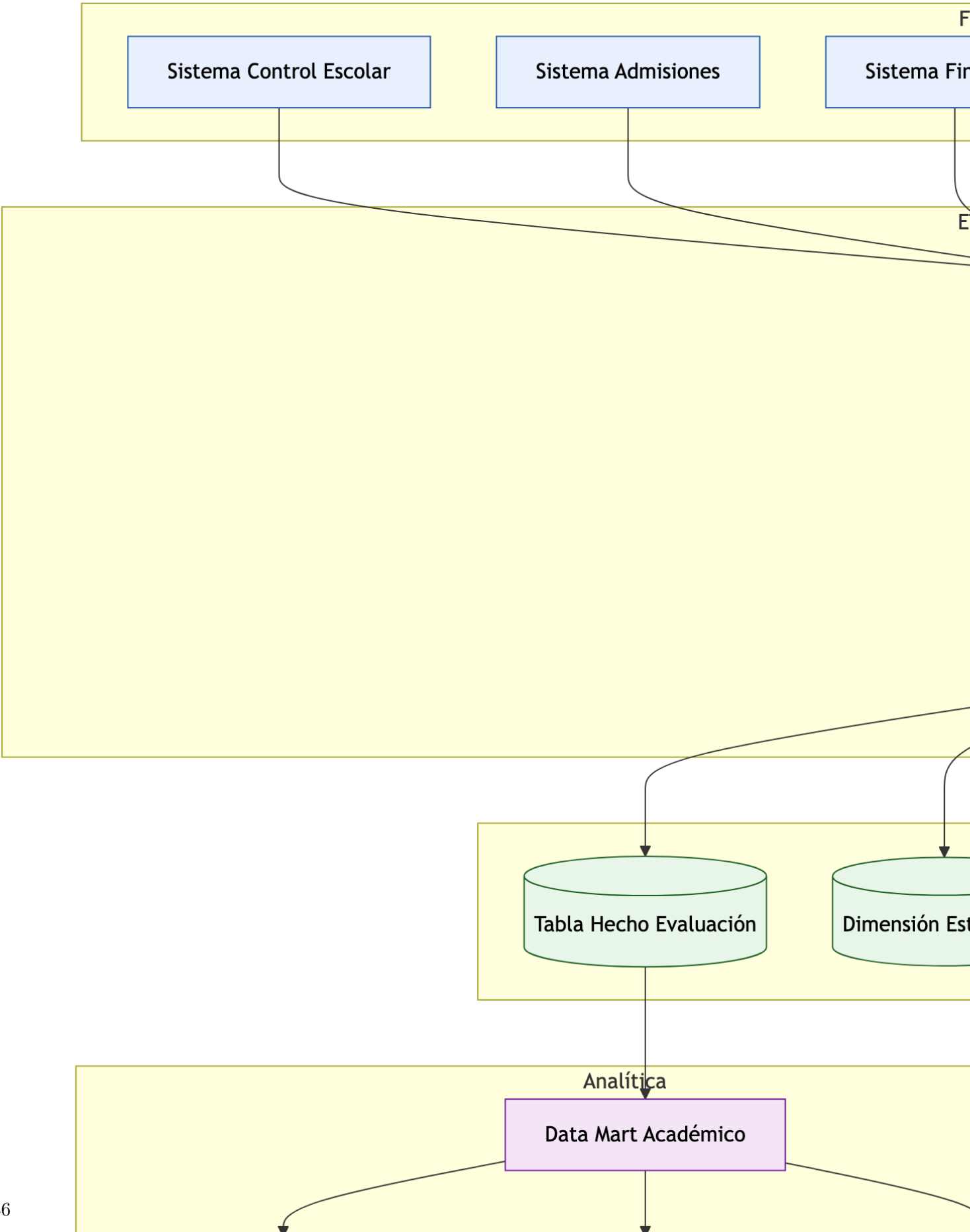
Gracias a esta consolidación, la Facultad puede:

- Analizar cohortes completas desde ingreso hasta egreso.
- Medir tasas de reprobación y abandono por generación.
- Identificar asignaturas críticas.
- Evaluar tendencias de desempeño a lo largo del tiempo.
- Integrar indicadores académicos con variables administrativas o financieras.

La diferencia fundamental respecto a los sistemas transaccionales es que el Data Warehouse ofrece una visión histórica, integrada y consistente del desempeño institucional, permitiendo transformar datos operacionales en conocimiento estratégico.

El diseño específico de las estructuras analíticas —como modelos dimensionales, tablas de hechos y dimensiones— se abordará en el siguiente capítulo, donde se detallará cómo se organizan técnicamente estos datos para optimizar su consulta.

8.4. Diagrama arquitectónico del DW académico



Con esta arquitectura, la Facultad podría analizar cohortes completas desde el ingreso hasta el egreso, medir las tasas de abandono por generación, detectar asignaturas críticas con alta reprobación, evaluar el impacto de las becas en el desempeño académico e identificar patrones longitudinales de rendimiento. A diferencia del sistema transaccional, el Data Warehouse integra información que se encuentra dispersa en distintos sistemas, conserva el historial de los datos a lo largo del tiempo, permite realizar análisis multidimensionales y está optimizado para procesos de agregación y consulta. En este contexto, el Data Warehouse no reemplaza los sistemas académicos, sino que los complementa, transformando los datos operacionales en información estratégica que apoya la toma de decisiones institucionales.

9. Conclusión

Un Data Warehouse no sustituye a los sistemas operacionales ni registra transacciones cotidianas. Su función es integrar información dispersa, conservar historia, romper silos organizacionales y facilitar análisis complejos orientados a la toma de decisiones estratégicas.

En la siguiente sección se abordará el modelado multidimensional, que define cómo se estructuran las tablas analíticas dentro de esta arquitectura y cómo se optimizan para consultas agregadas.

9.1. Recapitulación visual: del registro a la analítica

Para cerrar el recorrido, retomemos el caso de la Facultad de Ciencias y observemos de forma compacta cómo un evento operacional adquiere contexto analítico al atravesar la arquitectura.

En un contexto universitario, el sistema de calificaciones registra eventos puntuales como la inscripción de un estudiante o la obtención de una nota. Sin embargo, analizar la trayectoria académica completa de un estudiante requiere una visión mucho más amplia: integrar múltiples periodos, generaciones, asignaturas, planes de estudio y, en muchos casos, información administrativa asociada.

Un **Data Warehouse académico** permite consolidar esta información en un repositorio histórico, integrado y consistente, diseñado específicamente para análisis estratégico.

9.1.1. Diagrama Arquitectónico del Data Warehouse Académico

El siguiente diagrama muestra cómo fluye la información escolar, financiera y docente para conformar la analítica académica:

Con esta arquitectura, la Facultad puede analizar cohortes completas desde el ingreso hasta el egreso, medir las tasas de abandono por generación, detectar asignaturas críticas con alta reprobación e identificar patrones longitudinales de rendimiento.

10. Data Lakehouse

De silos a una base abierta y gobernada

10.1. Pregunta rectora

¿En qué momento compartir una base abierta y gobernada para BI, *streaming* y aprendizaje automático justifica asumir la operación adicional de un Lakehouse?

Un **Data Lakehouse** no es el destino obligatorio de toda plataforma de datos. En una organización pequeña, un Data Warehouse administrado puede entregar resultados con menos componentes y menos personal. El Lakehouse se vuelve razonable cuando la duplicación, el movimiento de datos, la variedad de cargas y la necesidad de reutilizar contratos comunes superan el costo de operar tablas, metadatos, catálogo, gobierno y mantenimiento.

Esta lectura sigue un caso de una *fintech* desde sus primeros reportes hasta una plataforma compartida. El propósito no es memorizar productos, sino aprender a separar responsabilidades y argumentar una decisión arquitectónica.

i Ruta de lectura

Tiempo estimado: 35–45 minutos. Conviene haber leído antes el capítulo de Data Warehouse. Los diagramas usan siempre la misma convención: azul para datos y almacenamiento, morado para metadatos y gobierno, naranja para ingesta, gris para cómputo, verde para consumo confiable y rojo para riesgo o duplicación.

10.1.1. Objetivos de aprendizaje

Al terminar, podrás:

1. explicar por qué un Data Warehouse fue una elección correcta durante la primera etapa de la *fintech*;
2. localizar duplicación, latencia y controles dobles en una arquitectura con silos;
3. distinguir objeto, *bucket*, formato de archivo, formato de tabla, catálogo, gobierno y motor de cómputo;
4. narrar cómo un *snapshot* evita que una escritura incompleta se vuelva visible;
5. justificar una transición usando volumen, variedad, velocidad, cargas de trabajo, costo y capacidad operativa;
6. reconocer que compactación, evolución de esquemas, observabilidad y radio de impacto siguen siendo responsabilidades del equipo.

10.2. El problema antes de la definición

A las 10:02, la *fintech* recibe la transacción tx_8472 por \$3,200. Finanzas la incluye en los ingresos del día, pero el sistema de fraude la presenta como bloqueada. Ambos equipos ejecutaron correctamente su propio proceso. La discrepancia está en las rutas y contratos que utilizaron.

La pregunta útil no es todavía «¿qué producto compraremos?», sino «¿qué trabajo se duplica para sostener estas dos rutas?».

10.3. Dos caminos que deben representar el mismo negocio

Históricamente, el Data Warehouse y el Data Lake resolvieron problemas legítimos:

- El **Data Warehouse** ofreció tablas controladas, SQL, concurrencia y reportes comprensibles para el negocio.
- El **Data Lake** permitió conservar grandes volúmenes y tipos diversos en almacenamiento de objetos económico y flexible.

El costo aparece cuando cada uno se convierte en una autoridad separada.

Si Producto agrega `device_risk_score`, el cambio debe capturarse, aterrizarse, transformarse, incorporarse a la tabla analítica, probarse, publicarse, autorizarse y comunicarse a consumidores. Cuando las rutas están separadas, varias de esas decisiones se repiten. La deuda no está sólo en los bytes duplicados: está en las decisiones duplicadas.

10.4. Caso Fintech, fase 1: el Warehouse era suficiente

Durante los años 0–3, la empresa tenía datos relacionales, unas 10,000 transacciones diarias, alrededor de 500 GB al año y reportes por lotes. Su necesidad dominante era conocer ingresos, morosidad y crecimiento con rapidez.

Un servicio administrado, dbt y una persona analista podían entregar valor en una semana y operar por cientos de dólares mensuales dentro del supuesto didáctico. Contratar un equipo especializado para un Lakehouse habría agregado observabilidad, integración y mantenimiento sin un retorno claro.

! Principio de decisión

Elegir un Data Warehouse en esta fase no fue falta de visión. Fue ajustar arquitectura, costo y capacidad del equipo al problema real.

10.4.1. Qué hace cada producto en esa fase

- **PostgreSQL, MySQL o una base operacional administrada** registra usuarios, cuentas y pagos de la aplicación. Está optimizada para operar el negocio, no para escanear años de historia.
- **Fivetran o Airbyte** extrae o replica datos desde las fuentes hacia un destino analítico.
- **BigQuery, Snowflake, Amazon Redshift o Fabric Warehouse** puede cumplir el rol de Data Warehouse. Son alternativas; no es necesario contratar todos.

- **dbt** ejecuta transformaciones y pruebas principalmente mediante SQL sobre el Warehouse; no es el lugar principal donde viven los datos.
- **Metabase, Tableau, Power BI o Looker** presenta métricas y tableros a usuarios de negocio.

La marca no determina por sí sola la arquitectura. BigQuery puede ser un Warehouse de tablas nativas y también un motor que consulta tablas abiertas. Siempre pregunta: **¿qué almacena, qué cataloga y qué procesa?**

Un Data Warehouse moderno también puede ser elástico, admitir datos semiestructurados y consultar almacenamiento externo. La decisión se toma por requisitos, economía y capacidad operativa, no por presentar al Warehouse como una tecnología rígida por definición.

10.5. Año 4: el punto de quiebre

La arquitectura inicial no falla por recibir un JSON aislado. El cambio aparece por una combinación de presiones.

Los 4 TB/mes equivalen aproximadamente a 48 TB/año. Llegar a petabytes requeriría crecimiento y retención durante varios años; no debe afirmarse que cuatro terabytes mensuales ya sean un petabyte. Los montos del caso son supuestos docentes, no cotizaciones de proveedores.

10.5.1. Actividad de clasificación

Síntoma	Fuerza principal
Millones de clics móviles	Volumen y velocidad
PDFs de contratos	Variedad
Modelo de fraude que necesita segundos	Velocidad y nuevos consumidores
Científicos que descargan terabytes	Movimiento
Dos equipos calculan la misma característica de riesgo	Coordinación
La factura de transformaciones se dispara	Economía y operación

La pregunta de diseño es: **¿escalamos el Warehouse, añadimos un Lake separado o rediseñamos una base compartida?** La respuesta debe empezar por requisitos, no por productos.

11. La base técnica del Lakehouse

11.1. Vocabulario mínimo

Término	Explicación cotidiana	Ejemplo en la fintech	Qué no significa
Objeto	Unidad guardada en la nube: bytes, nombre o <i>key</i> y metadatos.	PDF, imagen o archivo Parquet.	No es necesariamente una fila ni una tabla.
Bucket	Contenedor con nombre donde un proveedor guarda objetos.	<code>s3://fintech-datos-prod/</code> .	No es motor SQL, base relacional ni todo el Data Lake.
Key	Localizador único de un objeto dentro del bucket; puede parecer una ruta.	<code>bronze/kafka/2026/08/25/part-0001.json</code> .	5 carpetas / no siempre representan carpetas tradicionales.
Data Lake	Arquitectura para conservar y procesar datos diversos sobre almacenamiento económico, con ingesta, organización, seguridad y gobierno.	Objetos, políticas, catálogo y procesos.	Un bucket aislado no se vuelve automáticamente un Data Lake.
Data Warehouse	Sistema analítico orientado a tablas, SQL y datos preparados para decisiones.	Tablas de ingresos y morosidad.	No es la base operacional que autoriza cada pago.
Motor de cómputo	Software que consulta, transforma, agrega o entrena.	BigQuery, Trino, Spark o Databricks SQL.	El almacenamiento de objetos no procesa consultas por sí solo.
Formato de archivo	Organización de bytes dentro de un archivo.	Parquet organiza columnas y compresión.	No coordina por sí mismo una tabla de muchos archivos.
Formato de tabla	Protocolo que decide qué archivos forman una tabla y conserva cambios, esquema y snapshots.	Iceberg, Delta Lake o Hudi.	No sustituye almacenamiento, catálogo ni motor.
Catálogo	Mapa que relaciona un nombre con ubicación, esquema, propietario y metadatos vigentes.	<code>fintech.silver.transactions</code> .	Un sistema no garantiza calidad sin procesos y responsables.

Una analogía breve ayuda a recordar las responsabilidades:

11. La base técnica del Lakehouse

El bucket es el contenedor; los objetos son los paquetes; la *key* es la etiqueta. Parquet organiza el contenido de ciertos paquetes. Iceberg, Delta Lake o Hudi mantienen el inventario versionado que dice qué paquetes forman una tabla. El catálogo permite pedir la tabla por nombre y verificar el acceso. El motor abre los paquetes y realiza el cálculo.

11.1.1. Anatomía de un bucket

La URI `s3://fintech-datos-prod/documents/loan-8472.pdf` se interpreta así: `s3` identifica el servicio, `fintech-datos-prod` es el bucket y el resto es la *key*. En Google Cloud se vería `gs://...`; en ADLS Gen2 se usa un contenedor dentro de una cuenta de almacenamiento.

Tres distinciones deben quedar explícitas:

1. **Bucket archivo.** El bucket contiene objetos; el PDF o Parquet es el objeto.
2. **Bucket Data Lake.** Un Data Lake incluye ingesta, organización, seguridad, catálogo, cómputo y operación.
3. **Bucket Lakehouse.** El Lakehouse añade tablas transaccionales, catálogo, gobierno y motores coordinados.

11.2. Una base física compartida todavía no es una tabla

El primer paso del Lakehouse es desacoplar los bytes del motor: guardar datos en objetos y asignar cómputo distinto a SQL, ML o *streaming*. Esa base ofrece escala, formatos abiertos y economía de capacidad. Sin embargo, una carpeta puede contener:

```
transactions/  
  part-0001.parquet  
  part-0002.parquet  
  part-0003.parquet  
  part-0003-retry.parquet  
  _temporary-part-0004.parquet
```

El nombre de la carpeta no decide de forma confiable cuáles archivos pertenecen a la tabla, cuál fue un reintento y si la escritura terminó.

11.3. La idea que debe sobrevivir

No es «un lago junto a un almacén» ni «guardar Parquet en S3». Es hacer que conjuntos de archivos se comporten como tablas confiables sin perder apertura y separación entre almacenamiento y cómputo.

11.4. Responsabilidades lógicas de la arquitectura

Una implementación puede agrupar varias responsabilidades en un servicio administrado o distribuirlas entre herramientas. Las capas son lógicas, no una lista obligatoria de cinco productos.

11.4.1. 1. Ingesta

Conecta datos estructurados, semiestructurados y no estructurados. Las cargas **batch** mueven datos acumulados a intervalos; **CDC** replica cambios de una fuente; *streaming* procesa eventos a medida que llegan. La modalidad determina frescura, tamaño de archivos, reintentos y costo operativo.

11.4.2. 2. Almacenamiento

S3, Cloud Storage, ADLS Gen2 u OneLake conservan objetos a gran escala. Parquet y ORC son formatos columnares eficientes para datos tabulares; JSON, PDFs e imágenes pueden permanecer como objetos. Usar un formato abierto reduce dependencia, pero no garantiza que todos los motores soporten las mismas operaciones.

11.4.3. 3. Formato de tabla, catálogo y gobierno

Iceberg, Delta Lake y Hudi coordinan archivos como tablas versionadas. El catálogo resuelve nombres, ubicaciones, esquemas y metadatos vigentes. El gobierno añade identidad, clasificación, permisos, auditoría y responsabilidades. Son tareas relacionadas, pero distintas.

11.4.4. 4. Interfaces y motores

SQL, APIs de DataFrame y APIs de metadatos permiten que motores como BigQuery, Trino, Spark o Databricks SQL procesen datos sin convertir el almacenamiento en el motor. La compatibilidad se verifica por operación, versión del formato y configuración.

11.4.5. 5. Consumo

BI necesita métricas certificadas; ciencia de datos necesita entidades reutilizables y datos autorizados; fraude necesita baja latencia; auditoría necesita historia y evidencias. Compartir la base no obliga a compartir un único motor ni a consultar la misma capa de calidad.

11.5. Archivo, tabla y catálogo no son sinónimos

Parquet no publica una transacción sobre veinte archivos. El formato de tabla mantiene la lista autorizada, el esquema, las particiones y los snapshots. El catálogo permite que un nombre estable apunte a esos metadatos. Iceberg, Delta Lake y Hudi comparten esa intención general, pero no son idénticos.

11.6. Cómo un snapshot oculta una escritura incompleta

El reto no es que S3, Cloud Storage o Azure Blob Storage necesariamente «pierdan consistencia». Una modificación de tabla puede crear muchos objetos; hace falta un **commit de tabla atómico** que publique el conjunto completo.

La atomicidad se logra porque los lectores consultan una lista autorizada. Si el job falla antes de publicar, el puntero sigue en el snapshot 17. Si termina, un commit mueve el puntero a la versión 18 completa. Un lector que resolvió la versión antes del commit continúa con una vista consistente según el aislamiento y el motor.

Las garantías ACID suelen expresarse por tabla. No debe suponerse automáticamente una transacción atómica entre muchas tablas; hay que verificar el formato y el motor.

11.7. Evolución de esquema sin reescribir toda la historia

Los metadatos pueden separar la identidad de una columna de su nombre visible.

Esto habilita evolución de esquema y viaje en el tiempo, pero no vuelve gratuitos todos los cambios. Modificar tipos incompatibles, semántica, particiones o unidades puede exigir migración, *backfill*, vistas de transición y pruebas.

11.8. Catálogo y gobierno: mapa, versión y acceso

El catálogo convierte un nombre de negocio en ubicación, esquema y metadatos vigentes. El gobierno responde otra pregunta: aunque alguien conozca la ruta, ¿tiene derecho a verla y con qué detalle? Un catálogo no mueve datos ni garantiza calidad por existir. Requiere propietarios, políticas, clasificación, auditoría y credenciales con privilegio mínimo.

11.8.1. Comprobación rápida

Asigna cada responsabilidad:

1. «Organiza columnas y compresión dentro de `part-0007`» → **formato de archivo**.
2. «Decide que A, B y C forman la versión 17» → **formato de tabla**.
3. «Resuelve dónde vive `silver.transactions` y verifica permisos» → **catálogo y gobierno**.
4. «Conserva el PDF original y los bytes de Parquet» → **almacenamiento de objetos**.

12. Integrar calidad, cargas y productos de datos

12.1. Medallion: Bronze, Silver y Gold

La arquitectura Medallion es una forma frecuente, no una ley universal, de representar estados de calidad. Los nombres no designan productos ni ubicaciones físicas obligatorias.

Bronze preserva; Silver integra; Gold traduce al lenguaje del negocio. Finanzas no debería calcular ingresos directamente desde JSON Bronze. Una organización puede publicar varias tablas Gold porque cada producto responde a una decisión y granularidad distintas.

PDFs, imágenes y otros binarios permanecen como objetos. Una tabla Silver puede registrar `object_uri`, hash, propietario, clasificación, estado de OCR y relaciones. No conviene afirmar que una fotografía «se convierte en Iceberg».

12.2. Fase 2: una base gobernada y varios motores

La *fintech* no cambió sólo de contenedor. Cambió el contrato:

- **Finanzas** consulta productos Gold certificados.
- **ML** utiliza entidades Silver autorizadas y resultados derivados de documentos sin descargar terabytes a almacenamiento local.
- **Fraude** procesa eventos en tiempo real y consulta o actualiza señales gobernadas según las garantías del pipeline.

Separar almacenamiento y cómputo permite dimensionar y apagar motores por carga, pero no garantiza costos menores. Formato, estadísticas, tamaño de archivos, partición, cachés y consultas siguen determinando desempeño y gasto.

12.3. Tecnologías abiertas y servicios administrados

Responsabilidad	Ejemplos frecuentes	Función en el caso
Fuente operacional	PostgreSQL, MySQL, SQL Server, Cloud SQL, RDS, Azure SQL	Usuarios, cuentas, cobros y transferencias.
Ingesta batch/CDC	Fivetran, Airbyte, AWS DMS, Datastream, Azure Data Factory	Replica cambios operacionales.
Eventos	Kafka, Confluent, Kinesis, Pub/Sub, Event Hubs	Clics, telemetría y fraude.
Almacenamiento de objetos	S3, Cloud Storage, ADLS Gen2, OneLake	Conserva JSON, Parquet, PDFs e imágenes.

12. Integrar calidad, cargas y productos de datos

Responsabilidad	Ejemplos frecuentes	Función en el caso
Formato de archivo	Parquet, ORC, Avro, JSON	Organiza bytes dentro de cada archivo.
Formato de tabla	Iceberg, Delta Lake, Hudi	Publica snapshots, esquema e inventario de archivos.
Catálogo y gobierno	Glue + Lake Formation, Lakehouse runtime catalog + Knowledge Catalog, Unity Catalog, Fabric + Purview	Descubrimiento, permisos, linaje y auditoría.
Motores	BigQuery, Athena, Redshift, Trino, Spark, Databricks SQL, Fabric SQL	Ejecuta la carga analítica o de ingeniería.
Consumo	Looker, QuickSight, Power BI, Tableau, notebooks, modelos	Convierte datos en decisiones o productos.

Parquet, Iceberg, Delta Lake, Hudi, Spark, Kafka y Trino son estándares o proyectos abiertos. S3, BigQuery, Redshift, Databricks, Fabric y Fivetran son servicios o plataformas administradas. Una plataforma comercial puede administrar tecnologías abiertas.

13. Implementaciones en Google Cloud, AWS y Azure

No hay traducción exacta entre servicios. La comparación debe mantener fijas las responsabilidades: almacenamiento, tabla, catálogo, gobierno, cómputo, consumo y operación.

13.1. Google Cloud: Cloud Storage, Iceberg y varios motores

La arquitectura actual de Google Cloud combina Cloud Storage, Apache Iceberg, un *runtime catalog*, gobierno y motores como BigQuery, Spark, Flink y Trino, según su documentación de arquitectura Lakehouse. Las tablas Iceberg administradas por BigQuery permanecen en buckets controlados por la organización y añaden evolución de esquema, *time travel* y optimización automática, como detalla la guía de tablas Iceberg administradas.

Una ruta de implementación razonable es:

1. elegir región, proyecto y bucket con políticas de retención y cifrado;
2. crear un Lakehouse runtime catalog y definir namespaces;
3. usar Iceberg V2 para nuevas tablas abiertas y decidir quién administra sus metadatos;
4. configurar IAM y, cuando corresponda, credenciales delegadas con privilegio mínimo;
5. conectar BigQuery y los motores abiertos que realmente utilizará el equipo;
6. probar lectura, escritura, evolución, recuperación y costos antes de declarar interoperabilidad.

El *quickstart* oficial recorre la creación del catálogo Iceberg REST, permisos, tablas y consultas. BigQuery también puede operar como un Data Warehouse moderno sin que la arquitectura deba llamarse Lakehouse.

13.2. AWS: S3, Iceberg, Glue y Lake Formation

La arquitectura Lakehouse de SageMaker unifica datos en S3, S3 Tables y Redshift mediante catálogos y acceso gobernado. En una pila modular, S3 conserva objetos, Iceberg coordina tablas, Glue Data Catalog registra metadatos, Lake Formation aplica permisos y Athena, Redshift, EMR o Glue ejecutan las cargas.

Una ruta de implementación es:

1. definir cuentas, regiones, buckets, cifrado, retención y límites de red;
2. registrar ubicaciones y tablas en Glue Data Catalog;
3. aplicar Lake Formation e IAM con separación entre administración, escritura y consumo;
4. crear tablas Iceberg y seleccionar motores compatibles por operación;
5. probar *compaction*, snapshots, limpieza y evolución de esquema;
6. publicar productos de datos y observar frescura, uso y costo.

AWS ofrece un recorrido inicial para crear proyecto, explorar, cargar, consultar y publicar datos. Su guía prescriptiva de Iceberg cubre servicios compatibles y prácticas para costo, rendimiento y operación a escala.

13.3. Azure y Microsoft Fabric: OneLake, Delta y Medallion

Microsoft ofrece al menos dos rutas frecuentes. **Fabric Lakehouse** usa OneLake, tablas Delta, Spark, un endpoint SQL y Power BI. **Azure Databricks** combina ADLS Gen2, Delta Lake, Unity Catalog, Spark y Databricks SQL. La elección depende de integración, gobierno, habilidades y modelo operativo.

Para Fabric, una ruta de implementación es:

1. decidir dominios, workspaces y fronteras de seguridad;
2. crear el Lakehouse y diseñar Bronze, Silver y Gold por calidad y propietario;
3. ingerir o enlazar datos, conservando el original cuando sea necesario;
4. publicar Silver y Gold como tablas Delta;
5. conectar Spark, endpoint SQL y Power BI según la carga;
6. automatizar pruebas, mantenimiento, **OPTIMIZE**, retención y observabilidad.

La guía de Medallion en OneLake explica modelos de despliegue, formatos, tamaño de archivos, retención y acceso. El tutorial de extremo a extremo cubre workspace, ingesta, transformación, Delta, endpoint SQL y consumo. Para una ruta empresarial con Data Factory y Azure Databricks, Microsoft organiza el diseño desde una base inicial hasta endurecimiento y misión crítica.

 No confundas una demostración con una arquitectura de producción

Un tutorial valida el flujo funcional. Producción exige además identidad, red, cifrado, continuidad, RPO/RTO, segregación de entornos, CI/CD, contratos, observabilidad, presupuestos, soporte y responsables.

14. El trabajo que no desaparece

14.1. El impuesto de los archivos pequeños

El *streaming* puede crear muchos archivos válidos y pequeños. El motor debe enumerarlos, abrirllos y planificarlos. La compactación reescribe el mismo contenido en menos archivos y publica el reemplazo mediante un nuevo snapshot.

14.2. Una fuente compartida comparte el radio de impacto

14.2.1. Responsabilidades operativas

- compactación y dimensionamiento de archivos;
- retención de snapshots y limpieza de archivos huérfanos;
- calidad, contratos y evolución compatible;
- permisos, auditoría y prevención de accesos directos no gobernados;
- observabilidad de latencia, frescura, costo y fallas;
- pruebas reales de interoperabilidad para cada formato y motor;
- presupuestos, cuotas y apagado o escalamiento del cómputo;
- continuidad: *time travel* es útil, pero no sustituye por sí solo respaldos, aislamiento y recuperación.

15. Decidir, no evangelizar

Criterio	Warehouse administrado	Lake + Warehouse separados	Lakehouse
Datos dominantes	Relacionales y preparados	Crudos diversos con usos separados	Diversos y compartidos entre BI, ML y streaming
Prioridad	Rapidez de adopción y SQL	Aislar economías o equipos	Reducir copias con contratos comunes
Cómputo	Principalmente SQL/BI	Motores y autoridades separadas	Motores distintos sobre tablas compartidas
Operación	Menor carga para un equipo pequeño	Coordinación y sincronización dobles	Metadatos, compactación y gobierno rigurosos
Señal económica	El servicio administrado sigue alineado	La separación tiene fronteras claras	Movimiento, copias o almacenamiento propietario dominan el costo
Riesgo principal	Rigidez o costo al crecer	Dos verdades y latencia	Complejidad y radio de impacto compartido

Para la *fintech*:

- **Años 0–3:** Warehouse administrado; datos relacionales, objetivo de reporting y equipo pequeño.
- **Año 5:** Lakehouse; BI, ML y fraude reutilizan datos diversos a escala, y el movimiento ya domina parte del costo.
- **Otra empresa:** puede seguir en Warehouse o usar una arquitectura híbrida. No hay ganador universal.

Usa esta forma de argumento:

Elegimos ____ porque ____ y ; *aceptamos el costo o riesgo de* .

Ejemplo inicial: «Elegimos un Warehouse administrado porque los datos son relacionales y necesitamos reporting rápido; aceptamos menor flexibilidad futura a cambio de una operación sencilla».

Ejemplo a escala: «Elegimos Lakehouse porque BI, ML y fraude necesitan reutilizar datos diversos y el movimiento de terabytes domina el costo; aceptamos operar compactación, contratos y gobierno con un equipo especializado».

15.1. Preguntas frecuentes y errores de recuperación

15.1.1. ¿Un bucket es una carpeta gigante?

Es una aproximación inicial útil, pero incompleta. El bucket contiene objetos identificados por keys. Los prefijos pueden mostrarse como carpetas aunque el modelo base no sea un sistema de archivos tradicional.

15.1.2. ¿Puedo hacer SQL directamente sobre un bucket?

El bucket no ejecuta SQL. Athena, BigQuery, Trino, Spark u otro motor puede leer archivos. Para tratarlos como tablas confiables necesita esquema, lista vigente de archivos, formato de tabla y catálogo.

15.1.3. ¿S3 es un Data Lake?

S3 suele ser su base física. El Data Lake incluye también ingesta, organización, seguridad, catálogo, cómputo y operación.

15.1.4. ¿Entonces el Data Warehouse ya no sirve?

No. En la fase inicial fue la opción más eficiente y puede seguir atendiendo productos Gold o cargas de BI. La transición se evalúa cuando el beneficio de compartir datos diversos supera la carga operativa.

15.1.5. ¿Guardar Parquet en S3 ya crea un Lakehouse?

No. Resuelve almacenamiento y formato de archivo. Faltan commits consistentes, formato de tabla, catálogo, permisos, calidad, mantenimiento y motores compatibles.

15.1.6. ¿Iceberg, Delta Lake y Hudi son bases de datos?

Son formatos o protocolos de tabla. Un catálogo y motores completan la experiencia; una plataforma puede empaquetarlos como servicio.

15.1.7. ¿El catálogo guarda todos los datos?

Normalmente guarda o resuelve metadatos: nombres, ubicaciones, esquemas, propietarios, políticas y punteros. Los archivos permanecen en almacenamiento de objetos.

15.1.8. ¿La fuente única de verdad es una sola tabla?

No. Significa autoridades y contratos coherentes. Bronze, Silver y Gold pueden coexistir, con productos adecuados a distintas decisiones.

15.1.9. ¿Formato abierto significa compatibilidad perfecta?

No. Deben probarse operaciones, versión del formato, extensiones, escrituras concurrentes, evolución de esquema y mantenimiento en cada motor.

15.1.10. ¿Separar cómputo siempre abarata?

No. Permite escalar por carga, pero consultas ineficientes, recursos permanentes o falta de límites pueden elevar el gasto.

15.1.11. ¿Time travel sustituye los respaldos?

No necesariamente. Depende de retención, eliminación de objetos, aislamiento de cuentas y requisitos de recuperación. Es versionado útil, no una estrategia completa de continuidad.

15.2. Exit ticket

Completa sin consultar el capítulo:

1. Un bucket con Parquet no es todavía una tabla porque...
2. El catálogo sirve para..., mientras el formato de tabla sirve para...
3. La migración de la *fintech* se justifica cuando..., pero obliga a...

Respuestas mínimas esperadas

1. Falta una lista versionada y transaccional que determine qué archivos forman la tabla.
2. El catálogo permite encontrar y gobernar; el formato coordina snapshots, esquema y commits.
3. Varias cargas comparten datos diversos a escala y las copias o costos se vuelven dominantes; a cambio se operan mantenimiento, gobierno y contratos.

15.2.1. Criterio de logro

El objetivo se alcanza si puedes reconstruir la cadena **silos** → **objetos** → **snapshots** → **catálogo/gobierno** → **motores especializados** → **operación** → **decisión** y, además, explicar por qué el Warehouse fue correcto en la fase inicial y por qué el Lakehouse sólo se volvió razonable al cambiar los requisitos.

16. Cheatsheet

16.1. Modelo mental en una página

Si ves o escuchas...	Piensa en...	Pregunta de verificación
s3://, gs://, ADLS, OneLake	Almacenamiento de objetos	¿Dónde viven los bytes y quién controla la ubicación?
Parquet u ORC	Formato de archivo	¿Cómo se organizan columnas dentro de cada archivo?
Iceberg, Delta o Hudi	Formato de tabla	¿Cómo se publican commits, snapshots y evolución de esquema?
Glue, Lakehouse runtime catalog, Unity Catalog, Fabric/Purview	Catálogo y gobierno	¿Quién resuelve nombres, metadatos, permisos, linaje y propietario?
BigQuery, Athena, Trino, Spark, Redshift, Databricks SQL	Motor de cómputo	¿Qué procesa y qué operaciones soporta sobre el formato?
Bronze	Evidencia recibida	¿Puede reconstruirse el proceso y respetarse la retención?
Silver	Entidades validadas	¿Existe contrato, calidad, deduplicación y responsable?
Gold	Producto de datos	¿Qué decisión, granularidad y definición de negocio sirve?
Muchos archivos pequeños	Deuda operativa	¿Quién compacta, actualiza estadísticas y limpia huérfanos?
«Una copia para cada equipo»	Impuesto de coordinación	¿Pueden compartir tabla, catálogo y política sin compartir motor?

16.2. Diagnóstico rápido

Todavía es un Data Lake consultable si hay objetos, Parquet, catálogo básico y motor, pero no existe un protocolo transaccional que publique versiones confiables de tabla.

Hay un Lakehouse funcional si los datos tabulares comparten formato de tabla, catálogo, gobierno y motores compatibles, con operación de calidad, mantenimiento y costo.

Conviene un Warehouse administrado cuando domina SQL sobre datos preparados, el equipo es pequeño y la simplicidad ofrece más retorno que la flexibilidad adicional.

Conviene evaluar Lakehouse cuando BI, ML y streaming necesitan reutilizar datos diversos; las copias, el movimiento o el almacenamiento propietario son el cuello de botella; y existe capacidad para operarlo.

16.3. Lista mínima de implementación

1. Definir decisiones y SLO de frescura, calidad, costo y recuperación.
2. Elegir almacenamiento, regiones, cifrado, retención y fronteras de red.
3. Elegir formato de tabla y probar compatibilidad real con cada motor.
4. Establecer catálogo, identidad, clasificación, permisos, linaje y propietarios.
5. Diseñar contratos y estados de calidad; Medallion es una opción.
6. Automatizar pruebas, compactación, estadísticas, retención y limpieza.
7. Observar frescura, fallas, uso, rendimiento y costo por producto de datos.
8. Preparar evolución compatible, rollback y continuidad.

La síntesis final es:

```
silos y copias
→ base común en objetos
→ archivos coordinados por snapshots
→ catálogo y permisos
→ motores distintos sobre contratos compartidos
→ mantenimiento y control del radio de impacto
→ elección condicionada por el caso
```

17. Bibliografía y guías oficiales de implementación

Las referencias siguientes son documentación primaria de los proveedores. Fueron consultadas el 30 de agosto de 2026.

17.1. Google Cloud

- Google Cloud. *How Lakehouse works*. Arquitectura de almacenamiento, catálogo, gobierno y motores sobre Apache Iceberg.
- Google Cloud. *Quickstart: Use the Iceberg REST catalog with Lakehouse*. Implementación paso a paso del catálogo, permisos, tablas y consultas.
- Google Cloud. *Apache Iceberg managed tables*. Capacidades y límites de las tablas Iceberg administradas por BigQuery.

17.2. Amazon Web Services

- Amazon Web Services. *What is the lakehouse architecture of Amazon SageMaker?*. Componentes y funcionamiento de la arquitectura administrada.
- Amazon Web Services. *Getting started with the lakehouse architecture of Amazon SageMaker*. Recorrido inicial para proyecto, datos, consultas y publicación.
- Garcia Garcia, D. y Sandona, S. *Using Apache Iceberg on AWS*. AWS Prescriptive Guidance para servicios compatibles, costo, desempeño y operación.

17.3. Microsoft Azure y Fabric

- Microsoft. *Understand medallion architecture for Fabric with OneLake*. Diseño de capas, despliegue, Delta, archivos y mantenimiento.
- Microsoft. *Lakehouse end-to-end scenario: overview and architecture*. Implementación de ingesta, transformación, endpoint SQL y consumo.
- Microsoft. *Design a medallion lakehouse with Azure Data Factory*. Ruta desde la implementación base hasta endurecimiento empresarial y misión crítica.

Parte VI.

Modelado multidimensional

18. Modelado Multidimensional

El modelado multidimensional constituye la base conceptual de los almacenes de datos (Data Warehouses, DW). A diferencia del modelo relacional tradicional altamente normalizado (diseñado para sistemas transaccionales (OLTP) donde predominan inserciones y actualizaciones frecuentes) el modelo multidimensional está optimizado para el análisis (OLAP), es decir, para responder preguntas complejas de negocio de manera rápida, clara y comprensible para los usuarios finales.

Como se enfatiza en el material del curso, el diseño de un DW no comienza por la base de datos ni por la tecnología, sino por las decisiones que el negocio desea tomar y la información necesaria para sustentarlas. El modelo se construye entonces alrededor de preguntas analíticas, no alrededor de procesos operativos.

A diferencia del modelo relacional normalizado, aquí se utilizan estructuras optimizadas para consultas complejas:

- **Tabla de Hechos (Fact Table):** Contiene las métricas cuantitativas o medidas del negocio.
- **Tablas de Dimensiones:** Contienen los atributos descriptivos que contextualizan los hechos (Tiempo, Geografía, Producto, Estudiante, etc.).

18.1. Hechos y Dimensiones

Un hecho representa un evento medible del negocio. Debe cumplir tres características fundamentales: ocurre en un momento específico, se puede medir y sucede repetidamente. En lugar de describirlo solo de forma conceptual, puede entenderse mejor mediante una representación estructural.

A continuación se muestra un ejemplo de una **tabla de hechos académica**, donde cada fila representa la evaluación de un estudiante en una asignatura durante un periodo determinado.

HECHO_EVALUACION_ACADEMICA	
int	id_estudiante
int	id_asignatura
int	id_tiempo
decimal	calificacion
int	creditos
int	indicador_reprobado

En esta tabla:

- Cada registro corresponde a un evento académico concreto.
- Las columnas `calificacion`, `creditos` e `indicador_reprobado` son medidas.
- Las claves (`id_estudiante`, `id_asignatura`, `id_tiempo`) conectan el hecho con su contexto.

Las **dimensiones** proporcionan precisamente ese contexto. Permiten responder preguntas como: ¿quién?, ¿cuándo?, ¿dónde? y ¿qué? ocurrió el evento.

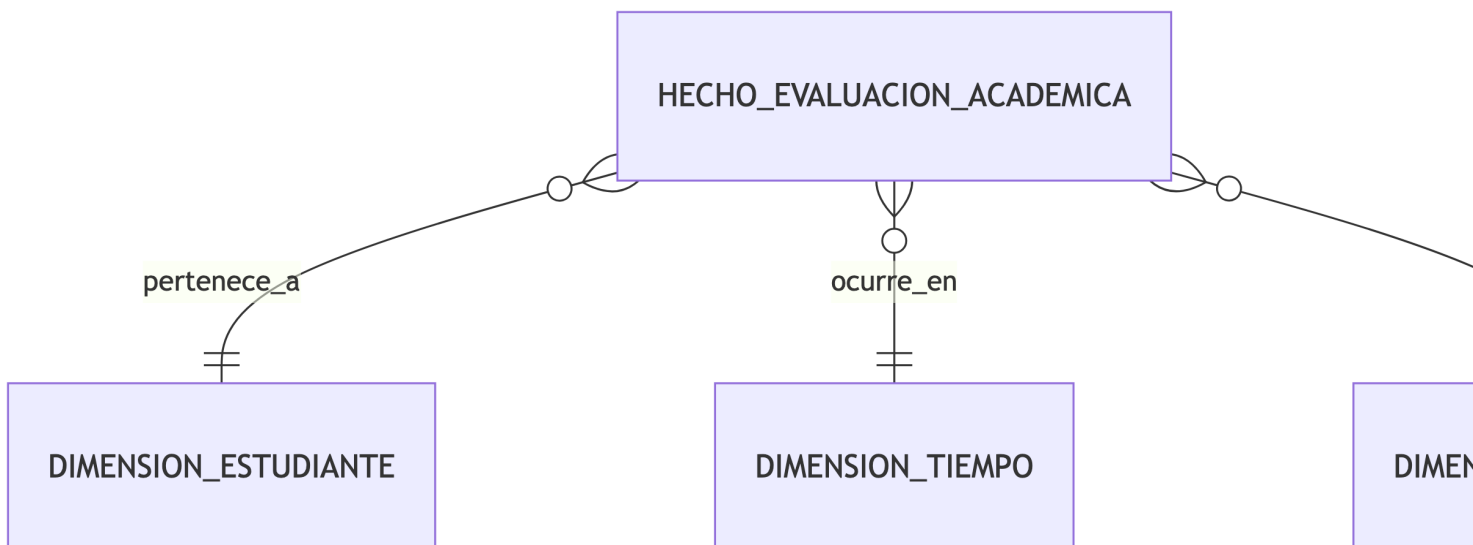
A continuación se muestra una representación simplificada de algunas dimensiones académicas:

DIMENSION_ESTUDIANTE	
int	id_estudiante
string	nombre
string	sexo
int	generacion
string	carrera
string	facultad

DIMENSION_TIEMPO	
int	id_tiempo
int	semestre
int	anio
string	periodo

DIMENSION...	
int	
string	
string	

El hecho adquiere significado únicamente cuando se conecta con estas dimensiones. Conceptualmente, la relación puede representarse así:



De esta forma, una fila del hecho puede interpretarse como:

“El estudiante X cursó la asignatura Y en el periodo Z y obtuvo determinada calificación”.

Un hecho aislado es poco informativo; adquiere significado al conectarse con dimensiones. Esta relación también guía el diseño del modelo: si una medida no puede explicarse adecuadamente con las dimensiones disponibles, suele indicar que falta una dimensión relevante, como campus, plan de estudios, modalidad o grupo.

18.2. Granularidad: la decisión de diseño más importante

La granularidad define qué representa exactamente una fila en la tabla de hechos.

Impacta directamente el volumen de datos, el costo de infraestructura, la complejidad del ETL y el tipo de análisis que puede realizarse.

18.2.1. Comparación de niveles de granularidad

HECHO_TRAYECTORIA_SEMESTRAL	
int	id_estudiante
int	id_tiempo
int	total_materias
int	total_creditos
decimal	promedio_semestre

HECHO_EVALUACION_ACADEMICA	
int	id_estudiante
int	id_asignatura
int	id_tiempo
decimal	calificacion
int	creditos
int	indicador_reprobado

Izquierda: Granularidad baja — una fila por alumno por semestre.

Cada registro resume el desempeño global del estudiante en un periodo.

No se almacenan materias individuales, sino resultados agregados del semestre.

Con 200 alumnos y 2 semestres por año se generan aproximadamente 400 filas por año (2,000 en cinco años).

Este enfoque reduce volumen y complejidad, pero limita el análisis detallado por asignatura o profesor.

Derecha: Granularidad alta — una fila por alumno por materia por semestre.

Cada registro representa una inscripción específica en una asignatura.

Permite analizar reprobación por materia, desempeño por docente y evolución por plan de estudios.

Con 200 alumnos, 6 materias promedio y 2 semestres por año se generan aproximadamente 4,800 filas por año (24,000 en cinco años).

Este enfoque aumenta capacidad analítica, pero también volumen, costo y complejidad operativa.

18.2.2. Hecho aditivo

Un hecho es aditivo cuando puede sumarse correctamente a través de todas las dimensiones del modelo (incluida la dimensión tiempo). En el material del curso se menciona como ejemplos típicos: número de inscritos, créditos inscritos y número de reprobados.

Ejemplos claros:

- Ventas: sumar “monto vendido” por día, por tienda, por producto, por ciudad. La suma conserva sentido en cualquier nivel de agregación.
- Académico: sumar “créditos inscritos” por estudiante, por carrera, por semestre o por año; y sumar “indicador_reprobado” (0/1) para obtener el total de reprobados en un conjunto dado (por asignatura, periodo, generación, etc.).

Implicación de diseño: si una medida es aditiva, suele ser una buena candidata para almacenarse directamente en la tabla de hechos, porque habilita muchas consultas rápidas y robustas.

18.2.3. Hecho semi-aditivo

Un hecho es semi-aditivo cuando puede sumarse en algunas dimensiones, pero no en todas; típicamente no se suma válidamente a través del tiempo. En el material se ejemplifica con “estudiantes activos al finalizar semestre” o “créditos acumulados”.

Ejemplo:

- “Estudiantes activos” por carrera en un mismo semestre: sumar por carrera para obtener el total de activos en ese semestre puede tener sentido (si las carreras son particiones disjuntas).
- “Estudiantes activos” entre semestres: sumar activos del semestre 1 más activos del semestre 2 suele perder interpretación porque muchas personas se repiten; el resultado puede inflarse y no representa una entidad real.

Implicación de diseño: medidas semi-aditivas a menudo se consultan con funciones como MAX, MIN o “valor al cierre del periodo”, o bien se modelan como “snapshots” (cortes periódicos), dependiendo del caso.

18.2.4. Hecho no aditivo

Un hecho es no aditivo cuando no puede sumarse en ninguna dimensión; requiere cálculo (promedio, razón, porcentaje). En el curso se mencionan: promedio de calificaciones, tasa de reprobación y porcentaje de deserción.

Ejemplo:

- La tasa de reprobación se calcula como (reprobados / inscritos) bajo un filtro o un grupo; no se almacena como tal en la tabla de hechos porque debe recomponerse según el contexto de la consulta.
- El promedio de calificaciones depende de la población (y a veces de ponderaciones). “Sumar promedios” rara vez es significativo.

Implicación de diseño: en general se almacenan los componentes base (numeradores/denominadores, conteos, sumas) y se calculan métricas derivadas en la capa semántica o en vistas/consultas.

18.3. Jerarquías en dimensiones y navegación analítica

Las jerarquías permiten analizar de lo general a lo particular (roll-up y drill-down). Ejemplos típicos:

- Tiempo: Año → Semestre → Periodo.
- Geografía: País → Estado → Ciudad.
- Académico: Facultad → Carrera → Estudiante.

Este diseño es clave para reportes: permite cambiar el “nivel” de agregación sin rediseñar la consulta, solo cambiando el `GROUP BY` o el nivel de la dimensión utilizado.

Con hechos, dimensiones, granularidad y aditividad definidos, el siguiente paso es decidir *cómo* se estructuran físicamente esas dimensiones alrededor del hecho, y qué costo real tiene esa decisión — el tema del siguiente capítulo.

18.4. Conclusión

El modelado multidimensional organiza los datos alrededor de hechos medibles y dimensiones descriptivas, priorizando claridad y desempeño para análisis. Para que el diseño sea correcto, debe fijarse adecuadamente la granularidad y entender la aditividad de las medidas, ya que esto determina qué agregaciones son válidas y qué métricas deben calcularse.

19. Esquemas Dimensionales y Redundancia

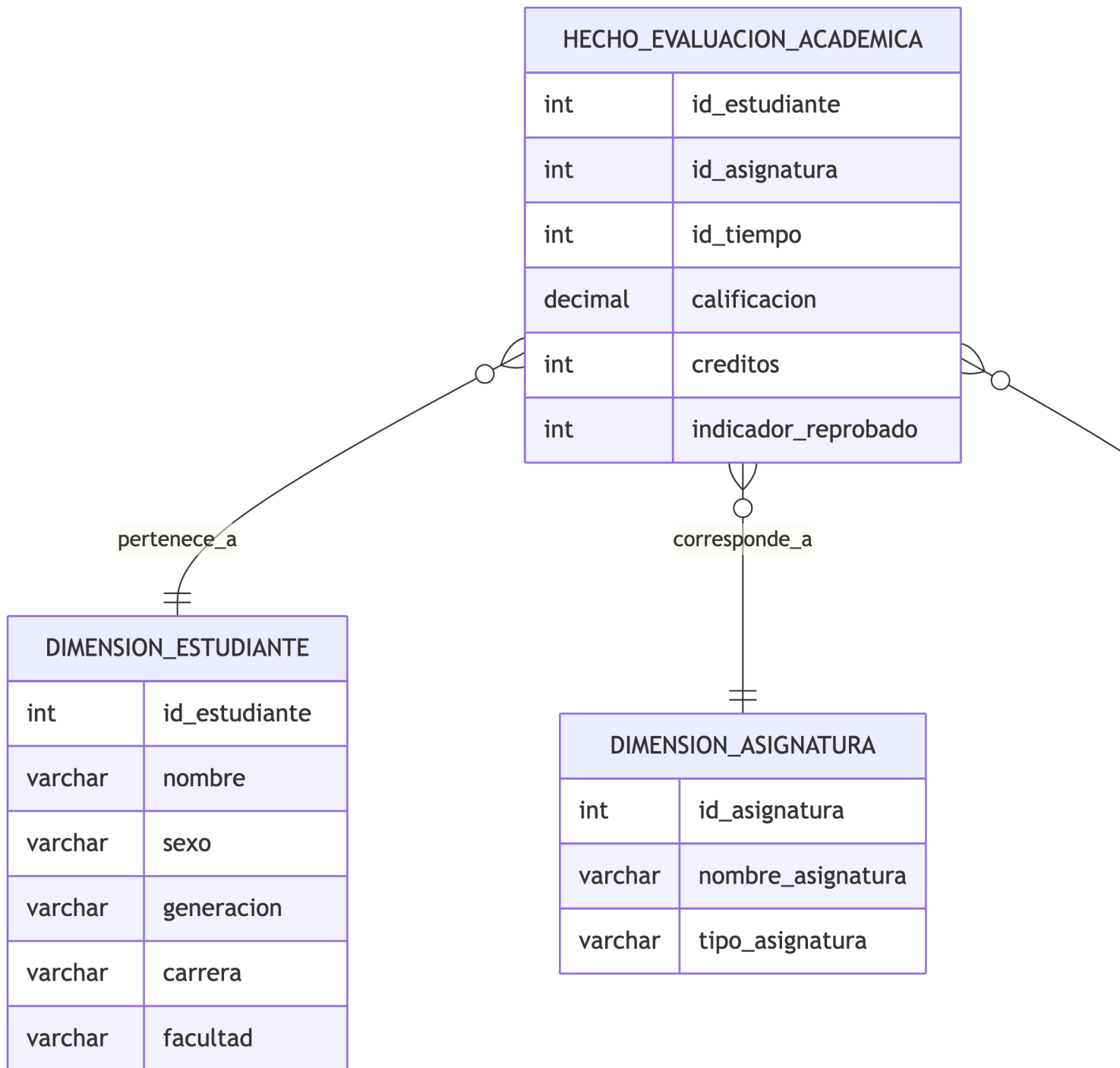
En el capítulo anterior se definieron hechos, dimensiones, granularidad y aditividad. Ahora corresponde decidir *cómo* se estructuran físicamente las dimensiones alrededor del hecho: qué tan normalizadas están, cuánta redundancia se acepta y qué impacto real tiene esa decisión sobre el costo y la velocidad de consulta.

19.1. Esquema de Estrella

El esquema estrella consiste en una tabla de hechos central conectada directamente a varias dimensiones. Sus dimensiones suelen estar desnormalizadas (redundancia controlada) para reducir joins y optimizar consultas analíticas.

En el ejemplo académico, la dimensión estudiante incluye atributos como carrera y facultad en la misma tabla, lo que simplifica consultas típicas del estilo “promedio por carrera y por semestre”. El costo es aceptar repetición (por ejemplo, repetir “ciencias” como facultad en muchos registros).

Desde el punto de vista estructural, el modelo puede representarse así:

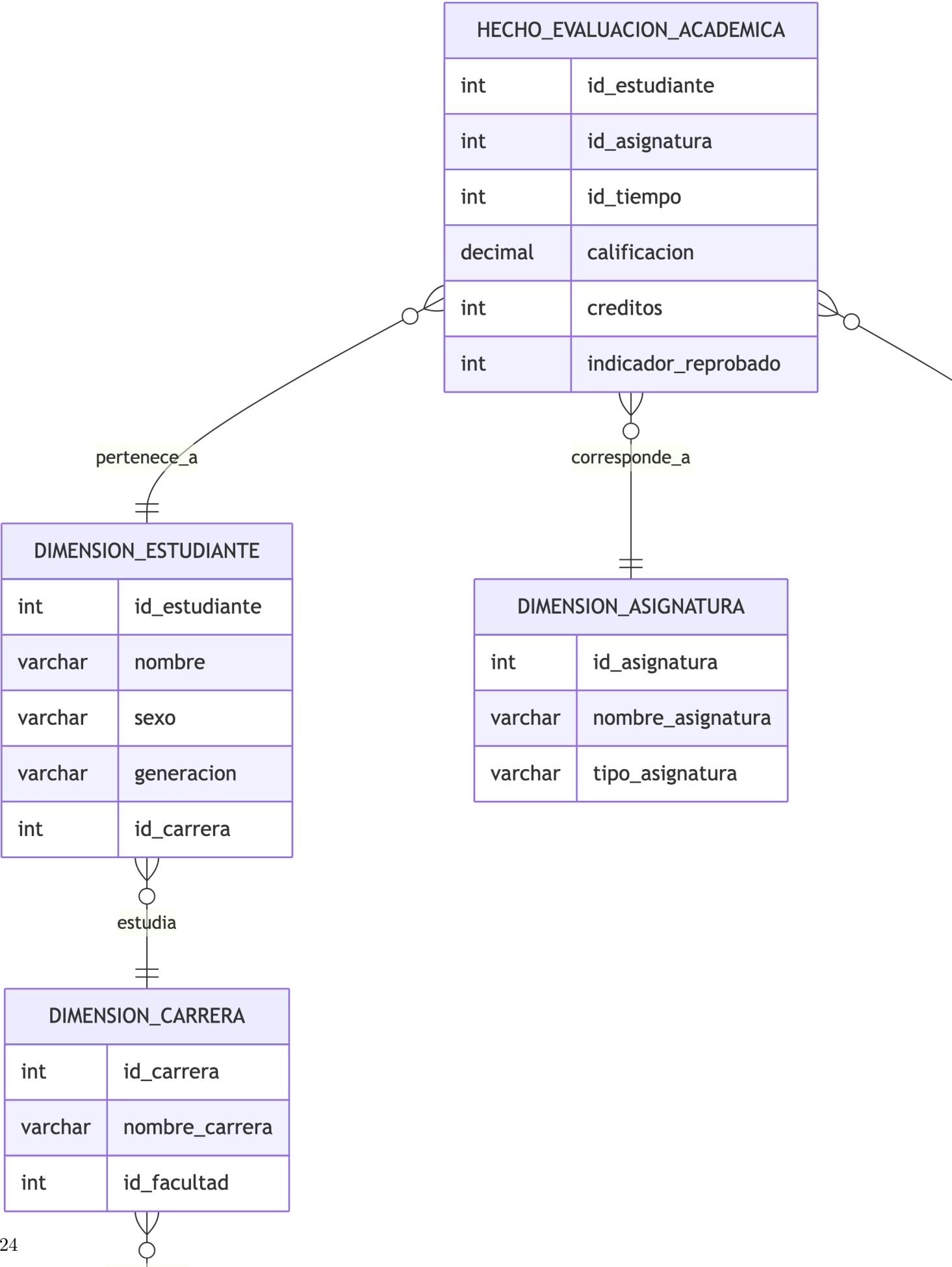


En este esquema:

- La tabla de hechos está al centro.
- Todas las dimensiones se conectan directamente a ella.
- No existen relaciones entre dimensiones.
- La información jerárquica (por ejemplo, carrera → facultad) se almacena dentro de la misma dimensión.

19.2. Esquema de Copo de Nieve (Snowflake)

El esquema de copo de nieve normaliza las dimensiones en subdimensiones. Por ejemplo, separar “carrera” y “facultad” en tablas propias relacionadas jerárquicamente, evitando redundancia, pero aumentando joins y complejidad de consulta.



En este esquema:

- La dimensión estudiante ya no almacena directamente la facultad.
- Carrera y facultad se modelan como entidades independientes.
- Se elimina redundancia.
- Se incrementa el número de joins necesarios para consultas que involucren facultad.

19.2.1. Comparación conceptual

Aspecto	Estrella	Copo de nieve
Normalización	Baja (desnormalizada)	Alta
Redundancia	Controlada	Reducida
Número de joins	Menor	Mayor
Complejidad conceptual	Más simple	Más estructurada
Rendimiento OLAP	Generalmente mejor	Puede disminuir
Gobierno de datos	Más flexible	Más controlado

19.2.2. Lo mismo, en SQL

Con un dominio distinto (ventas), la diferencia entre ambos esquemas se ve directamente en la consulta:

```
-- Estrella: un solo join por dimensión
select
  d.anio,
  p.categoria,
  sum(f.importe) as ventas
from fact_ventas as f
join dim_fecha as d using (fecha_key)
join dim_producto as p using (producto_key)
group by d.anio, p.categoria;
```

```
-- Copo de nieve: dim_producto no tiene "categoria", hay que subir un nivel más
select
  d.anio,
  c.categoria,
  sum(f.importe) as ventas
from fact_ventas as f
join dim_fecha as d using (fecha_key)
join dim_producto as p using (producto_key)
join dim_categoria as c using (categoria_key)
group by d.anio, c.categoria;
```

Criterio	Estrella	Copo de nieve
Consultas	Simples, pocas uniones	Más uniones
Redundancia	Mayor	Menor
Mantenimiento	Directo	Más entidades y claves

Criterio	Estrella	Copo de nieve
Uso recomendado	BI y autoservicio	Jerarquías grandes o compartidas

19.3. Redundancia controlada: qué cuesta realmente desnormalizar

La comparación anterior suele quedarse en “el copo de nieve reduce redundancia”, como si repetir datos fuera automáticamente malo. En un almacén de datos columnar (Parquet, Delta, BigQuery, Snowflake) esa redundancia casi no cuesta lo que parece.

Un motor columnar no almacena cada valor de texto tal cual: aplica **codificación por diccionario** (cada valor distinto se guarda una sola vez y cada fila solo referencia un código corto) y **run-length encoding** (RLE) sobre columnas ordenadas o de baja cardinalidad. Si `dim_producto` tiene 500,000 filas y la columna `categoria` solo tiene 20 valores distintos, el motor no almacena “Electrónica” 25,000 veces: almacena el diccionario de 20 categorías una vez y, por fila, un índice de unos cuantos bits.

En la práctica esto significa que:

- Guardar `categoria` directamente en `dim_producto` (estrella) cuesta, en almacenamiento, un orden de magnitud similar a guardar `categoria_key` (copo de nieve) — ambos terminan siendo, a nivel de disco, un entero pequeño por fila.
- El ahorro de espacio que promete la normalización es, en un motor columnar moderno, marginal frente a lo que cuesta en 1990 con motores orientados a fila.
- Lo que sí cambia entre ambos esquemas no es tanto el almacenamiento, sino el **costo de consulta**.

19.4. Costo de consulta: joins y motores en la nube

El argumento a favor de la estrella no es “ocupa menos espacio”, es “cada consulta hace menos trabajo”. Esto importa especialmente cuando las dimensiones no son pequeñas.

Considera una dimensión cliente con 5 millones de filas, normalizada como `dim_cliente` → `dim_ciudad` → `dim_estado` (jerarquía geográfica en copo de nieve) contra la misma dimensión desnormalizada donde `estado` vive directamente en `dim_cliente`.

Para responder “ventas totales por estado”:

- **Estrella:** `fact_ventas` (100M filas) hace un solo join contra `dim_cliente` (5M filas), que ya trae `estado`.
- **Copo de nieve:** `fact_ventas` hace join contra `dim_cliente` (5M filas), y ese resultado intermedio vuelve a hacer join contra `dim_ciudad` (5K filas) y luego contra `dim_estado` (32 filas).

El segundo camino no es más caro por las tablas pequeñas (`dim_ciudad`, `dim_estado`) — es más caro porque agrega **etapas de shuffle** sobre un resultado intermedio ya grande (la salida del primer join, del tamaño de `fact_ventas`). Cada hop adicional en la cadena de joins vuelve a mover ese volumen de datos por la red del clúster.

Esto se traduce directamente en costo facturable:

- **BigQuery** cobra por bytes escaneados/procesados; más joins sobre resultados intermedios grandes significan más bytes movidos y más slots ocupados.

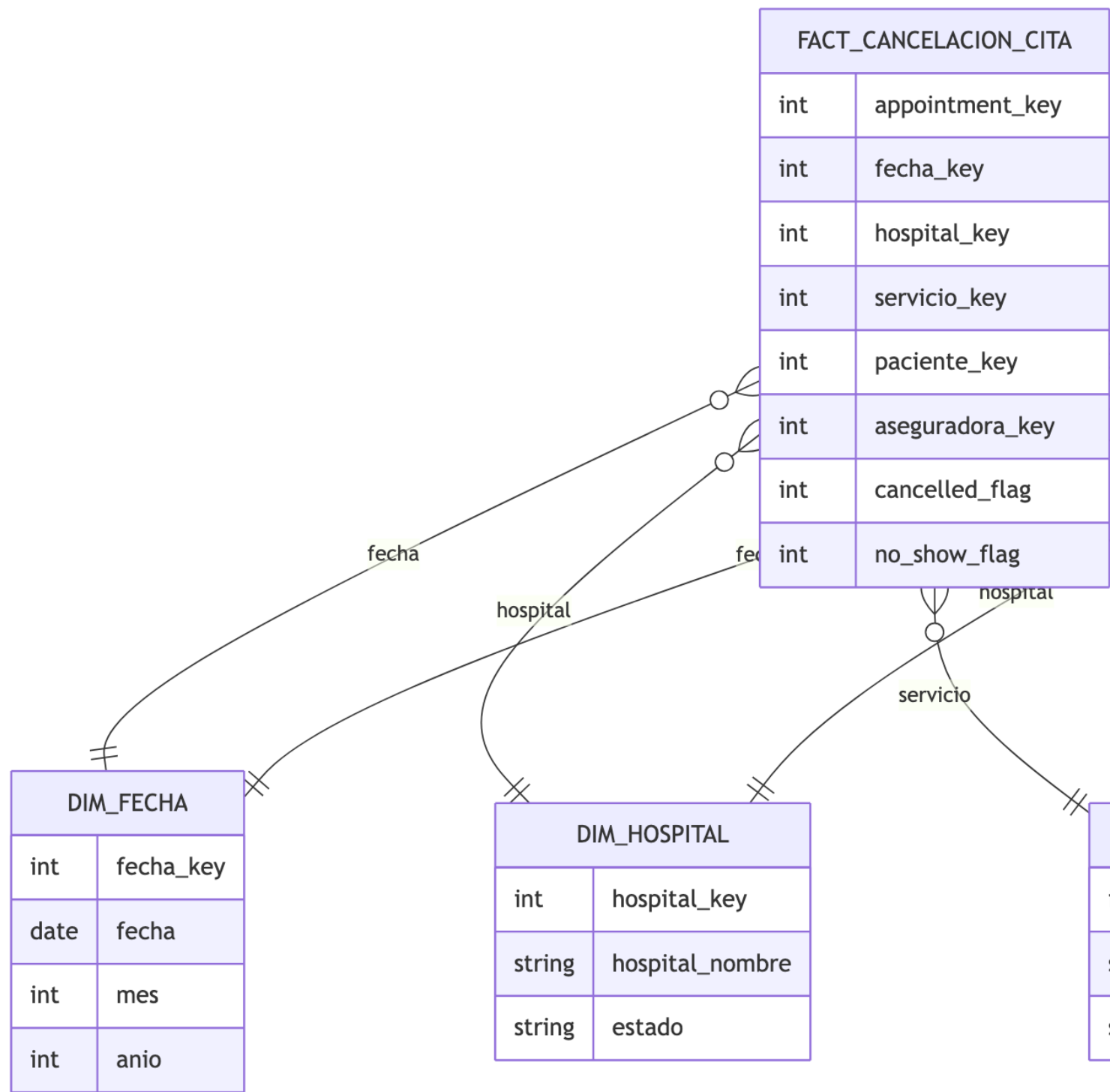
- **Snowflake** cobra por tiempo de cómputo del warehouse (créditos); una cadena de joins más larga generalmente implica más segundos de warehouse activo para la misma pregunta.

Regla práctica: la redundancia de un esquema estrella es barata cuando vive en una dimensión (se paga una vez, al cargar). El costo de un join extra se paga en **cada consulta**, multiplicado por cuántas veces se ejecuta esa consulta al día. Por eso los data marts orientados a BI casi siempre prefieren estrella, y el copo de nieve se reserva para jerarquías muy grandes, compartidas entre muchos data marts, donde mantener una sola versión de la verdad importa más que el costo por consulta.

19.5. Esquema de Constelación (Fact Constellation / Galaxy Schema)

Cuando dos o más procesos de negocio comparten las mismas dimensiones, tiene sentido que compartan también las tablas físicas: eso es un **esquema de constelación**. En vez de una estrella aislada por proceso, varias tablas de hechos se conectan a un conjunto de **dimensiones conformadas** (misma clave, mismo grano, mismo significado en todos los hechos que la usan).

Retomando el dominio hospitalario: además de FACT_CANCELACION_CITA (una fila por cita programada), el hospital también factura por cada servicio prestado. En vez de crear dimensiones nuevas y redundantes para ese segundo proceso, se reutilizan DIM_FECHA, DIM_HOSPITAL, DIM_SERVICIO, DIM_PACIENTE y DIM_ASEGURADORA:



Esto resuelve exactamente el tipo de redundancia más caro: sin dimensiones conformadas, cada equipo que construye un nuevo data mart terminaría creando su propia versión de “paciente” o “servicio”, con distinta granularidad, distintos códigos y sin forma de cruzar ambos hechos (“¿cuántas citas canceladas generaron facturación parcial de cualquier forma?”). Con dimensiones conformadas, ambos hechos son comparables y combinables por las mismas llaves.

19.6. Patrones de diseño para gestionar redundancia

Además de estrella, copo de nieve y constelación, hay patrones puntuales para decidir *dónde* vive cada dato — evitando tanto la redundancia descontrolada como los joins innecesarios.

Dimensiones conformadas. Una misma dimensión (mismo grano, mismas llaves) reutilizada por varios hechos, como DIM_FECHA en el ejemplo anterior. Es la base de cualquier esquema de constelación y evita que cada data mart reinvente “cliente” o “producto” con reglas distintas.

Dimensiones degeneradas. Un atributo que identifica al hecho (folio de orden, número de ticket) pero que no tiene atributos propios que ameriten una tabla de dimensión. Se guarda como columna directa en la tabla de hechos en vez de crear una dimensión de una sola columna.

Junk dimensions. Varios indicadores booleanos o de baja cardinalidad (*es_promocion*, *metodo_pago*, *canal*) se combinan en una sola dimensión “basurero” con una fila por combinación posible, en vez de agregar media docena de columnas sueltas al hecho o crear media docena de dimensiones diminutas.

Role-playing dimensions. La misma tabla física de dimensión se usa varias veces con distinto significado — por ejemplo, DIM_FECHA referenciada como fecha de pedido, fecha de envío y fecha de pago dentro del mismo hecho, mediante alias en la consulta (*fecha_pedido_key*, *fecha_envio_key*, *fecha_pago_key*, todas apuntando a la misma DIM_FECHA).

Mini-dimensiones. Cuando algunos atributos de una dimensión grande cambian con mucha frecuencia (ingresos estimados de un cliente, score de riesgo) separarlos en una dimensión pequeña propia evita que esos cambios disparen versiones nuevas de SCD tipo 2 en toda la fila del cliente — tema que se retoma en el siguiente capítulo.

Bridge tables. Cuando la relación entre el hecho y una dimensión es de muchos a muchos (un estudiante con varias becas activas, un diagnóstico con múltiples médicos tratantes) una tabla puente resuelve la relación sin duplicar filas del hecho ni forzar una cardinalidad que no existe en el negocio.

19.7. Conclusión

Estrella, copo de nieve y constelación no son alternativas puramente estéticas: cada una mueve el costo de un lugar a otro (almacenamiento vs. cómputo por consulta vs. complejidad de mantenimiento). En un motor columnar moderno, la redundancia dentro de una dimensión suele ser la opción más barata. Lo que todavía no se ha cubierto es qué pasa cuando esas dimensiones **cambian** — un cliente se muda de ciudad, un producto cambia de categoría. Ese es el tema del siguiente capítulo: dimensiones lentamente cambiantes (SCD).

20. Dimensiones Lentamente Cambiantes (SCD)

El capítulo anterior asumió, implícitamente, que las dimensiones son estáticas: un estudiante siempre pertenece a la misma carrera y un producto siempre a la misma categoría. En la práctica no es así: un cliente cambia de ciudad, un producto cambia de categoría y un empleado cambia de puesto.

Las **Slowly Changing Dimensions (SCD)** definen cómo tratar esos cambios de atributos descriptivos sin romper la relación entre un hecho y el contexto que tenía cuando ocurrió. La **clave natural** identifica a la entidad en el sistema fuente; la **clave sustituta** identifica una fila o versión concreta dentro del data warehouse.

20.1. Tipos de SCD

20.1.1. Tipo 1: sobrescribir

SCD Tipo 1 actualiza la fila existente. El valor nuevo sustituye al anterior y todas las consultas, incluso las que analizan hechos antiguos, ven la descripción corregida o más reciente.

Ventajas

- Es el patrón más sencillo de implementar, consultar y mantener.
- No incrementa el número de filas de la dimensión.
- Corrige errores de captura para todo el historial de manera inmediata.
- No requiere columnas de vigencia ni lógica temporal en los joins.

Desventajas

- Destruye el valor anterior y no permite reconstruir cómo se veía la entidad en una fecha pasada.
- Puede reinterpretar hechos históricos con atributos que todavía no existían cuando ocurrieron.
- No es adecuado para auditoría, trazabilidad ni análisis de evolución.

Conviene usarlo cuando el cambio es una corrección ortográfica, una normalización o un atributo cuyo historial carece de valor analítico. No conviene usarlo para cambios de puesto, categoría, segmento o ubicación cuando esos cambios alteran la interpretación del hecho.

```
merge into dim_cliente as destino
using stg_cliente as fuente
on destino.cliente_id = fuente.cliente_id
when matched then update set
    nombre = fuente.nombre,
    ciudad = fuente.ciudad
when not matched then insert (cliente_id, nombre, ciudad)
values (fuente.cliente_id, fuente.nombre, fuente.ciudad);
```

20.1.2. Tipo 2: crear una versión

SCD Tipo 2 cierra la versión vigente e inserta una fila nueva. La clave natural se mantiene, pero cada versión recibe una clave sustituta distinta y un intervalo de vigencia.

Ventajas

- Conserva el historial completo de los atributos seleccionados.
- Mantiene el contexto correcto de cada hecho sin depender de joins por rango de fecha, siempre que el proceso de carga asigne la clave sustituta vigente.
- Permite auditoría, reconstrucción histórica y análisis de transiciones.
- Hace explícita la vigencia mediante `valido_desde`, `valido_hasta` y `es_actual`.

Desventajas

- Aumenta el volumen de la dimensión: una entidad puede producir muchas filas.
- La carga es más compleja porque debe detectar cambios, cerrar la versión anterior e insertar la nueva de forma consistente.
- Requiere reglas claras para cambios tardíos, correcciones retroactivas y límites de vigencia.
- Puede generar versiones duplicadas, solapamientos o más de una fila vigente si el proceso no es idempotente o no se ejecuta de forma atómica.

Conviene usarlo cuando es necesario responder preguntas como “¿qué categoría tenía el producto cuando se realizó la venta?” o “¿en qué ciudad estaba registrado el cliente cuando ocurrió la transacción?”.

```
update dim_cliente as d
set valido_hasta = current_date - interval '1 day',
    es_actual = false
from stg_cliente as s
where d.cliente_id = s.cliente_id
    and d.es_actual
    and (d.nombre is distinct from s.nombre
        or d.ciudad is distinct from s.ciudad);

insert into dim_cliente
(cliente_id, nombre, ciudad, valido_desde, valido_hasta, es_actual)
select s.cliente_id,
    s.nombre,
    s.ciudad,
    current_date,
    date '9999-12-31',
    true
from stg_cliente as s
left join dim_cliente as d
on d.cliente_id = s.cliente_id
and d.es_actual
where d.cliente_id is null
    or d.nombre is distinct from s.nombre
    or d.ciudad is distinct from s.ciudad;
```


En producción, el cierre y la inserción deberían ejecutarse dentro de una misma transacción cuando el motor lo permita. La clave sustituta puede generarse mediante una columna `identity`, una secuencia, un hash estable de la versión o la estrategia definida por la plataforma.

20.1.3. Tipo 3: guardar el valor anterior

SCD Tipo 3 conserva el valor vigente y un valor previo en la misma fila. No crea una historia ilimitada: al llegar otro cambio, el valor anterior también se sobrescribe.

Ventajas

- Permite comparar fácilmente el estado actual con el inmediatamente anterior.
- Mantiene una sola fila por entidad y evita el crecimiento de SCD Tipo 2.
- Las consultas son sencillas porque no necesitan intervalos de vigencia ni múltiples versiones.

Desventajas

- Sólo conserva una cantidad fija y limitada de cambios.
- Requiere agregar columnas específicas para cada atributo histórico.
- Pierde utilidad cuando varios atributos cambian de forma independiente.
- No permite reconstruir el estado de la dimensión para una fecha arbitraria.

Conviene usarlo cuando el negocio sólo compara el valor actual contra el anterior, por ejemplo, segmento actual frente a segmento previo. No sustituye a SCD Tipo 2 cuando existe un requisito de auditoría completa.

```
update dim_cliente as d
set ciudad_anterior = d.ciudad_actual,
    ciudad_actual = s.ciudad,
    fecha_cambio = current_date
from stg_cliente as s
where d.cliente_id = s.cliente_id
    and d.ciudad_actual is distinct from s.ciudad;
```

20.2. Implementación con dbt

dbt materializa el modelo dimensional desde tablas de *staging*. Un modelo incremental puede cubrir SCD Tipo 1; para SCD Tipo 2, un *snapshot* registra automáticamente las versiones detectadas.

```
-- snapshots/clientes.sql
{% snapshot clientes_snapshot%}
{{
    config(
        target_schema='snapshots',
        unique_key='cliente_id',
        strategy='check',
        check_cols=['nombre', 'ciudad']
    )
}}
```

```
select cliente_id, nombre, ciudad
from {{ ref('stg_clientes') }}
{% endsnapshot %}
```

El *snapshot* agrega `dbt_valid_from`, `dbt_valid_to` y `dbt_scd_id`. La dimensión final usa esas columnas para exponer la vigencia y la clave sustituta.

```
-- models/marts/dim_cliente.sql
select
  dbt_scd_id as cliente_key,
  cliente_id,
  nombre,
  ciudad,
  dbt_valid_from as valido_desde,
  coalesce(dbt_valid_to, timestamp '9999-12-31') as valido_hasta,
  dbt_valid_to is null as es_actual
from {{ ref('clientes_snapshot') }}
```

20.2.1. Ventajas de usar snapshots de dbt

- Centralizan la detección y el versionado de cambios.
- Reducen la cantidad de SQL procedural escrito manualmente.
- Generan metadatos de vigencia de forma consistente.
- Se integran con dependencias, documentación y pruebas del proyecto dbt.

20.2.2. Desventajas y precauciones

- El comportamiento depende de elegir correctamente `unique_key`, `strategy` y `check_cols`.
- Incluir columnas volátiles, como una marca de tiempo de carga, puede crear versiones innecesarias en cada ejecución.
- Un snapshot conserva lo que observa durante sus ejecuciones; no puede reconstruir cambios que ocurrieron y desaparecieron entre dos corridas.
- Los cambios retroactivos y las eliminaciones requieren una política explícita.

20.3. Pruebas de integridad

Las pruebas declarativas verifican que la clave sustituta sea única y no nula, y que la clave natural esté presente.

```
models:
- name: dim_cliente
  columns:
    - name: cliente_key
      tests: [unique, not_null]
    - name: cliente_id
      tests: [not_null]
```

Esas pruebas no garantizan por sí solas que exista una sola versión vigente. Esa regla puede verificarse con una prueba singular adicional:

```
-- tests/una_version_vigente_por_cliente.sql
select cliente_id
from {{ ref('dim_cliente') }}
where es_actual
group by cliente_id
having count(*) > 1
```

También conviene comprobar que los intervalos no se solapen y que `valido_desde` sea menor que `valido_hasta`.

20.4. Comparación y criterio de elección

Tipo	Historial	Filas por entidad	Complejidad	Uso principal
Tipo 1	Ninguno	Una	Baja	Correcciones y atributos sin valor histórico
Tipo 2	Completo	Una por versión	Alta	Auditoría y contexto histórico preciso
Tipo 3	Limitado	Una	Media	Comparar el estado actual con el previo

La elección no tiene que hacerse para toda la dimensión. En una misma tabla es posible aplicar Tipo 1 a atributos correctivos, Tipo 2 a atributos cuyo historial importa y, de forma excepcional, Tipo 3 a comparaciones limitadas. Lo importante es definir la política **por atributo**, documentar por qué se conserva o descarta el historial y diseñar las pruebas que protejan esa decisión.

21. Ejemplo Completo: Café Norte — Análisis y Construcción de Data Marts (Kimball)

21.1. Propósito y supuestos

Este ejemplo documenta, con un enfoque **Kimball**, la construcción de dimensiones conformadas y una tabla de hechos a granularidad **línea de pedido** para evaluar el impacto del lanzamiento de la app (enero de 2023) y del programa de lealtad. Es el cierre práctico de la sección: aplica granularidad y aditividad, esquema estrella y redundancia controlada, y dimensiones conformadas, todo sobre un caso end-to-end.

Hito de intervención (cutoff): 2023-01-01. Se define:

- Periodo **pre**: fechas menores a 2023-01-01.
- Periodo **post**: fechas mayores o iguales a 2023-01-01.

Se asume que el lector **ya cuenta con CSVs** provenientes de sistemas operacionales (POS y App) y catálogos maestros.

21.2. 1. Estructura esperada de archivos CSV

Coloque los archivos en un directorio `data/` al mismo nivel que este `.qmd`.

21.2.1. 1.1 Catálogos maestros (dimensiones)

- `data/clientes.csv` Campos mínimos: `cliente_id`, `fecha_alta`, `tipo_cliente`, `nivel_lealtad` (u otros atributos).
- `data/productos.csv` Campos mínimos: `sku`, `nombre`, `categoria`, `subcategoria`, `tamano`, `activo`.
- `data/sucursales.csv` Campos mínimos: `sucursal_id`, `nombre`, `zona`, `formato`.
- (Opcional) `data/promociones.csv` Campos mínimos: `promo_codigo`, `campania`, `descripcion`.

21.2.2. 1.2 Transacciones (hechos)

Se recomienda conservar el encabezado del pedido y las líneas separadas.

- data/pedidos_pos.csv y data/pedidos_app.csv Campos mínimos: order_id, fecha, cliente_id, sucursal_id, canal (POS o APP).
- data/lineas_pos.csv y data/lineas_app.csv Campos mínimos: order_id, line_num, sku, cantidad, precio_unitario, promo_codigo, metodo_pago, cancelado_flag.

Nota: si en origen el descuento ya viene desglosado, incluya también descuento_importe. Si no, puede derivarse en el proceso (ver más adelante).

21.3. 2. Carga de datos y validación de esquema

En un pipeline reproducible, **la primera etapa** consiste en cargar datos y aplicar validaciones: tipos, llaves no nulas, dominios controlados y rangos.

```
import pandas as pd
import numpy as np
from pathlib import Path

DATA_DIR = Path("../data")

def read_csv(path: Path) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"No se encontró el archivo: {path.resolve()}")
    return pd.read_csv(path)

clientes = read_csv(DATA_DIR / "clientes.csv")
productos = read_csv(DATA_DIR / "productos.csv")
sucursales = read_csv(DATA_DIR / "sucursales.csv")

ped_pos = read_csv(DATA_DIR / "pedidos_pos.csv")
ped_app = read_csv(DATA_DIR / "pedidos_app.csv")
lin_pos = read_csv(DATA_DIR / "lineas_pos.csv")
lin_app = read_csv(DATA_DIR / "lineas_app.csv")

# Promociones es opcional
prom_path = DATA_DIR / "promociones.csv"
promociones = read_csv(prom_path) if prom_path.exists() else None

# Visualización rápida
clientes.head(), productos.head(), sucursales.head()
```

```
( cliente_id fecha_alta tipo_cliente nivel_lealtad
0 C000001 2021-09-10 lealtad Plata
1 C000002 2022-02-15 sin_lealtad NaN
2 C000003 2022-06-01 lealtad Bronce
3 C000004 2022-11-20 sin_lealtad NaN
4 C000005 2023-01-05 lealtad Oro,

sku nombre categoria subcategoria tamano activo
0 SKU0001 Americano Bebidas Calientes Americano M 1
1 SKU0002 Latte Bebidas Calientes Latte G 1
2 SKU0003 Capuccino Bebidas Calientes Capuccino M 1
3 SKU0004 Cold Brew Bebidas Frias Cold Brew M 1
4 SKU0005 Frappé Vainilla Bebidas Frías Frappé G 1,

sucursal_id nombre zona formato
0 S001 Centro 1 Centro Tienda
1 S002 Norte 1 Norte Tienda
2 S003 Sur 1 Sur Tienda
3 S004 Oriente 1 Oriente Kiosco
4 S005 Poniente 1 Poniente Kiosco)
```

21.3.1. 2.1 Chequeos mínimos de calidad

A continuación se implementan reglas mínimas. En un DW real, estas reglas alimentan un sistema de monitoreo (por ejemplo, Great Expectations) y generan reportes de calidad.

```
def assert_columns(df: pd.DataFrame, required: list[str], name: str):
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"[{name}] Faltan columnas requeridas: {missing}")

# Esquema mínimo
assert_columns(clientes, ["cliente_id", "fecha_alta"], "clientes")
assert_columns(productos, ["sku", "nombre", "categoria"], "productos")
assert_columns(sucursales, ["sucursal_id", "nombre", "zona"], "sucursales")

assert_columns(ped_pos, ["order_id", "fecha", "cliente_id", "sucursal_id", "canal"], "pedidos_pos")
assert_columns(ped_app, ["order_id", "fecha", "cliente_id", "sucursal_id", "canal"], "pedidos_app")

assert_columns(lin_pos, ["order_id", "line_num", "sku", "cantidad", "precio_unitario"], "lineas_pos")
assert_columns(lin_app, ["order_id", "line_num", "sku", "cantidad", "precio_unitario"], "lineas_app")

# Tipos: fechas
for df, col, name in [
    (clientes, "fecha_alta", "clientes"),
    (ped_pos, "fecha", "pedidos_pos"),
    (ped_app, "fecha", "pedidos_app"),
]:
    df[col] = pd.to_datetime(df[col], errors="coerce")
    if df[col].isna().any():
        bad = df[df[col].isna()].head(5)
```

```

        raise ValueError(f"[{name}] Fechas inválidas en '{col}'. Ejemplos:\n{bad}")

# No nulos en llaves operacionales
def assert_not_null(df: pd.DataFrame, cols: list[str], name: str):
    for c in cols:
        if df[c].isna().any():
            raise ValueError(f"[{name}] Hay nulos en la columna clave: {c}")

assert_not_null(ped_pos, ["order_id", "cliente_id", "sucursal_id"], "pedidos_pos")
assert_not_null(ped_app, ["order_id", "cliente_id", "sucursal_id"], "pedidos_app")
assert_not_null(lin_pos, ["order_id", "sku"], "lineas_pos")
assert_not_null(lin_app, ["order_id", "sku"], "lineas_app")

# Dominios controlados
canales_validos = {"POS", "APP"}
if not set(ped_pos["canal"].unique()).issubset(canales_validos):
    raise ValueError(f"[pedidos_pos] Canal fuera de dominio: {set(ped_pos['canal'].unique()) - canales_validos}")
if not set(ped_app["canal"].unique()).issubset(canales_validos):
    raise ValueError(f"[pedidos_app] Canal fuera de dominio: {set(ped_app['canal'].unique()) - canales_validos}")

# Rangos numéricos
for df, name in [(lin_pos, "lineas_pos"), (lin_app, "lineas_app")]:
    if (df["cantidad"] <= 0).any():
        raise ValueError(f"[{name}] Se encontraron cantidades <= 0")
    if (df["precio_unitario"] < 0).any():
        raise ValueError(f"[{name}] Se encontraron precios negativos")

print("Validaciones mínimas: OK")

```

Validaciones mínimas: OK

21.4. 3. Homologación e integración de fuentes (Staging Integrado)

En Kimball, la integración lógica se garantiza mediante **dimensiones conformadas** y una **bus matrix**. Antes de construir dimensiones y hechos, se unifican las fuentes POS y App.

21.4.1. 3.1 Integración de encabezados y líneas

```

# Unificar encabezados y líneas
pedidos = pd.concat([ped_pos, ped_app], ignore_index=True)
lineas = pd.concat([lin_pos, lin_app], ignore_index=True)

# Normalizaciones

```



```

pedidos["canal"] = pedidos["canal"].str.upper()

# --- CLAVE ÚNICA GLOBAL (DW) ---
# Si order_id puede repetirse entre POS y APP, usamos (canal, order_id).
# Creamos un identificador único: order_uid
pedidos["order_uid"] = pedidos["canal"].astype(str) + "-" + pedidos["order_id"].astype(str)

# Las líneas no traen canal; se lo pegamos desde el header con un join previo (por order_id y canal)
# Si tus líneas sí traen canal, úsalo; si no, asumimos que lineas_pos -> POS y lineas_app -> APP
lin_pos_ = lin_pos.copy()
lin_pos_["canal"] = "POS"
lin_app_ = lin_app.copy()
lin_app_["canal"] = "APP"

lineas = pd.concat([lin_pos_, lin_app_], ignore_index=True)
lineas["canal"] = lineas["canal"].str.upper()
lineas["order_uid"] = lineas["canal"].astype(str) + "-" + lineas["order_id"].astype(str)

# Flags
lineas["promo_codigo"] = lineas.get("promo_codigo", pd.Series(["NA"]*len(lineas))).fillna("NA")
lineas["cancelado_flag"] = lineas.get("cancelado_flag", 0)
lineas["cancelado_flag"] = lineas["cancelado_flag"].fillna(0).astype(int)

# Encabezados duplicados (reenvíos desde origen): se conserva el más reciente.
# En una extracción real, el mismo pedido puede llegar más de una vez (reintentos,
# recargas parciales); quedarse con la versión más reciente por 'fecha' es la
# convención más simple, documentada aquí para que sea auditable.
if pedidos["order_uid"].duplicated().any():
    n_dup = pedidos["order_uid"].duplicated().sum()
    print(f"Aviso: {n_dup} encabezados de pedido duplicados (reenvíos); se conserva el más reciente")
    pedidos = pedidos.sort_values("fecha").drop_duplicates(subset=["order_uid"], keep="last")

# Integración header + líneas a granularidad línea usando order_uid
# (order_id y canal ya vienen en 'lineas'; se descartan de 'pedidos' para evitar sufijos _x/_y)
stg = lineas.merge(pedidos.drop(columns=["order_id", "canal"]), on=["order_uid"], how="left", validate="one-to-one")

# Chequeo de integridad
if stg["fecha"].isna().any():
    bad = stg[stg["fecha"].isna()][["order_id", "order_uid", "line_num", "canal"]].head(10)
    raise ValueError(f"Existen líneas sin encabezado de pedido asociado. Ejemplos:\n{bad}")

```

Aviso: 13 encabezados de pedido duplicados (reenvíos); se conserva el más reciente por 'fecha'.

21.4.2. 3.2 Derivación de métricas transaccionales

Se recomienda almacenar importes en el hecho para evitar dependencia histórica de catálogos (p. ej., cambios de precio de lista).

```

# Medidas base
stg["importe_bruto"] = (stg["cantidad"] * stg["precio_unitario"]).astype(float)

# Descuento: si no viene explícito, se asume 0 (ajuste si aplica)
if "descuento_importe" not in stg.columns:
    stg["descuento_importe"] = 0.0
else:
    stg["descuento_importe"] = stg["descuento_importe"].fillna(0.0).astype(float)

stg["importe_netto"] = stg["importe_bruto"] - stg["descuento_importe"]

# Convención simple para cancelaciones:
# - Importe neto se fuerza a 0 para líneas canceladas.
stg.loc[stg["cancelado_flag"].eq(1), "importe_netto"] = 0.0

# Cupón aplicado (bandera)
stg["cupon_aplicado_flag"] = (stg["promo_codigo"].fillna("NA") != "NA").astype(int)

stg[["order_id", "line_num", "cantidad", "precio_unitario", "importe_bruto", "descuento_importe", "importe_netto", "cancelado_flag", "promo_codigo", "cupon_aplicado_flag"]].head()

```

	order_id	line_num	cantidad	precio_unitario	importe_bruto	descuento_importe	importe_netto	cancelado_flag
0	P000007	1	2	31.47	62.94	0.0	62.94	0
1	P000007	2	1	36.67	36.67	0.0	36.67	0
2	P000001	1	3	51.26	153.78	0.0	153.78	0
3	P000001	2	3	78.11	234.33	0.0	234.33	0
4	P000010	1	3	72.02	216.06	0.0	216.06	0

21.5. 4. Construcción de dimensiones conformadas

En un DW tipo estrella, todas las dimensiones deben tener:

- **llave sustituta** (*_key, entero),
- **llave natural** (clave operacional como cliente_id, sku, etc.),
- atributos descriptivos.

Para un libro y un ejemplo didáctico, se implementa SCD Tipo 1 (sobrescritura). En producción, dim_cliente y dim_producto suelen requerir SCD Tipo 2, como se vio en el capítulo anterior.

21.5.1. 4.1 dim_tiempo

```
# Rango de fechas basado en staging
min_date = stg["fecha"].min().normalize()
max_date = stg["fecha"].max().normalize()

dim_tiempo = pd.DataFrame({"fecha": pd.date_range(min_date, max_date, freq="D")})
dim_tiempo["anio"] = dim_tiempo["fecha"].dt.year
dim_tiempo["mes"] = dim_tiempo["fecha"].dt.month
dim_tiempo["trimestre"] = dim_tiempo["fecha"].dt.quarter
dim_tiempo["semana_iso"] = dim_tiempo["fecha"].dt.isocalendar().week.astype(int)
dim_tiempo["dia_semana"] = dim_tiempo["fecha"].dt.dayofweek + 1
dim_tiempo["es_fin_semana"] = dim_tiempo["dia_semana"].isin([6,7]).astype(int)

dim_tiempo.insert(0, "tiempo_key", np.arange(1, len(dim_tiempo)+1, dtype=int))
dim_tiempo.head()
```

	tiempo_key	fecha	anio	mes	trimestre	semana_iso	dia_semana	es_fin_semana
0	1	2022-07-11	2022	7	3	28	1	0
1	2	2022-07-12	2022	7	3	28	2	0
2	3	2022-07-13	2022	7	3	28	3	0
3	4	2022-07-14	2022	7	3	28	4	0
4	5	2022-07-15	2022	7	3	28	5	0

21.5.2. 4.2 Función auxiliar para dimensiones (Tipo 1)

```
def build_dim_type1(df: pd.DataFrame, natural_key: str, attrs: list[str], sk_name: str) -> pd.DataFrame:
    out = df[[natural_key] + attrs].drop_duplicates().reset_index(drop=True)
    out.insert(0, sk_name, np.arange(1, len(out)+1, dtype=int))
    return out

# Dimensiones básicas
dim_canal = pd.DataFrame({"canal": ["POS", "APP"]})
dim_canal.insert(0, "canal_key", [1,2])

# Metodo de pago: si no existe en staging, se crea "NA"
if "metodo_pago" not in stg.columns:
    stg["metodo_pago"] = "NA"
dim_metodo_pago = pd.DataFrame({"metodo_pago": sorted(stg["metodo_pago"].fillna("NA").unique())})
dim_metodo_pago.insert(0, "metodo_pago_key", np.arange(1, len(dim_metodo_pago)+1, dtype=int))

# Promoción
if promociones is None:
    # Mínimo viable: solo códigos observados
    dim_promocion = pd.DataFrame({"promo_codigo": sorted(stg["promo_codigo"].fillna("NA").unique())})
else:
    # Con catálogo maestro: se prioriza catálogo, pero garantizamos NA
    dim_promocion = promociones.copy()
```

```

if "promo_codigo" not in dim_promocion.columns:
    raise ValueError("promociones.csv debe incluir 'promo_codigo'")
if "NA" not in set(dim_promocion["promo_codigo"].astype(str).unique()):
    dim_promocion = pd.concat([pd.DataFrame({"promo_codigo": ["NA"]})], dim_promocion), ignore_

dim_promocion["promo_codigo"] = dim_promocion["promo_codigo"].astype(str).fillna("NA")
dim_promocion = dim_promocion.drop_duplicates(subset=["promo_codigo"]).reset_index(drop=True)
dim_promocion.insert(0, "promocion_key", np.arange(1, len(dim_promocion)+1, dtype=int))
dim_promocion.head()

```

	promocion_key	promo_codigo	campania	descripcion
0	1	NA	NaN	NaN
1	2	nan	Sin promo	Sin descuento
2	3	APP10	Lanzamiento App	10 % en app
3	4	2X1FRIO	Bebidas frías	2x1 bebidas frías
4	5	DULCE15	Postres	15 % en postres

21.5.3. 4.3 dim_cliente, dim_producto, dim_sucursal

```

# Ajustes básicos de tipos
if "nivel_lealtad" not in clientes.columns:
    clientes["nivel_lealtad"] = "NA"
if "tipo_cliente" not in clientes.columns:
    clientes["tipo_cliente"] = "NA"

dim_cliente = build_dim_type1(
    clientes,
    natural_key="cliente_id",
    attrs=["tipo_cliente", "nivel_lealtad", "fecha_alta"],
    sk_name="cliente_key"
)

# Productos
for col in ["subcategoria", "tamano", "activo"]:
    if col not in productos.columns:
        productos[col] = "NA" if col != "activo" else 1

dim_producto = build_dim_type1(
    productos,
    natural_key="sku",
    attrs=["nombre", "categoria", "subcategoria", "tamano", "activo"],
    sk_name="producto_key"
)

# Sucursales
for col in ["formato"]:

```

```

if col not in sucursales.columns:
    sucursales[col] = "NA"

dim_sucursal = build_dim_type1(
    sucursales,
    natural_key="sucursal_id",
    attrs=["nombre", "zona", "formato"],
    sk_name="sucursal_key"
)

dim_cliente.head(), dim_producto.head(), dim_sucursal.head()

```

```

(  cliente_key cliente_id tipo_cliente nivel_lealtad fecha_alta
0          1    C000001      lealtad      Plata 2021-09-10
1          2    C000002  sin_lealtad      NaN 2022-02-15
2          3    C000003      lealtad    Bronce 2022-06-01
3          4    C000004  sin_lealtad      NaN 2022-11-20
4          5    C000005      lealtad      Oro 2023-01-05,
    producto_key sku      nombre      categoria subcategoria \
0          1  SKU0001    Americano  Bebidas Calientes    Americano
1          2  SKU0002      Latte  Bebidas Calientes      Latte
2          3  SKU0003    Capuccino  Bebidas Calientes    Capuccino
3          4  SKU0004    Cold Brew  Bebidas Frías    Cold Brew
4          5  SKU0005  Frappé Vainilla  Bebidas Frías    Frappé

    tamano activo
0      M      1
1      G      1
2      M      1
3      M      1
4      G      1 ,
    sucursal_key sucursal_id      nombre      zona formato
0          1      S001    Centro 1    Centro Tienda
1          2      S002    Norte 1    Norte Tienda
2          3      S003      Sur 1      Sur Tienda
3          4      S004    Oriente 1    Oriente Kiosco
4          5      S005  Poniente 1  Poniente Kiosco)

```

21.6. 5. Construcción de la tabla de hechos (fact_linea_pedido)

El hecho se define a granularidad **línea de pedido**, con:

- Llaves sustitutas hacia dimensiones conformadas.
- Medidas aditivas (cantidad, importe_*) y banderas (cancelado_flag, cupon_aplicado_flag).
- Llave degenerada order_id para trazabilidad.

```

fact = (stg
    .merge(dim_tiempo[["tiempo_key","fecha"]], on="fecha", how="left")
    .merge(dim_cliente[["cliente_key","cliente_id"]], on="cliente_id", how="left")
    .merge(dim_sucursal[["sucursal_key","sucursal_id"]], on="sucursal_id", how="left")
    .merge(dim_producto[["producto_key","sku"]], on="sku", how="left")
    .merge(dim_canal[["canal_key","canal"]], on="canal", how="left")
    .merge(dim_metodo_pago[["metodo_pago_key","metodo_pago"]], on="metodo_pago", how="left")
    .merge(dim_promocion[["promocion_key","promo_codigo"]], on="promo_codigo", how="left"))

# Integridad dimensional (unknown members)
for col in ["tiempo_key","cliente_key","sucursal_key","producto_key","canal_key","metodo_pago_key"]:
    if fact[col].isna().any():
        # En DW real: asignar key=0 a Unknown y registrar warning.
        bad = fact[fact[col].isna()][["order_id","line_num"]].head(10)
        raise ValueError(f"Hay filas sin mapeo dimensional para '{col}'. Ejemplos:\n{bad}")

fact_linea_pedido = fact[[
    "order_id","line_num",
    "tiempo_key","cliente_key","sucursal_key","producto_key",
    "canal_key","metodo_pago_key","promocion_key",
    "cantidad","precio_unitario","importe_bruto","descuento_importe","importe_netto",
    "cancelado_flag","cupon_aplicado_flag"
]].copy()

fact_linea_pedido.head()

```

	order_id	line_num	tiempo_key	cliente_key	sucursal_key	producto_key	canal_key	metodo_pago_key
0	P000007	1	245	8	1	7	1	2
1	P000007	2	245	8	1	7	1	2
2	P000001	1	528	1	2	3	1	1
3	P000001	2	528	1	2	5	1	2
4	P000010	1	356	5	4	5	1	1

21.7. 6. Data Marts y respuestas a preguntas estratégicas

21.7.1. 6.1 Pregunta 1: Pedidos mensuales desde enero 2023, segmentado por canal

Se define como **número de pedidos únicos** (COUNT DISTINCT order_id) por mes.

```

APP_LAUNCH = pd.Timestamp("2023-01-01")

fact_time = fact_linea_pedido.merge(
    dim_tiempo[["tiempo_key","fecha","anio","mes"]],
    on="tiempo_key", how="left"
)

```

```

)
fact_time["yyyymm"] = fact_time["fecha"].dt.to_period("M").astype(str)

# Desde enero 2023
mask_2023 = fact_time["fecha"] >= APP_LAUNCH

q1 = (fact_time.loc[mask_2023]
      .groupby("yyyymm")
      .agg(pedidos=("order_id", "nunique"),
          lineas=("order_id", "size"),
          ingreso_neto=("importe_neto", "sum"))
      .reset_index()
      .sort_values("yyyymm"))

# Segmentación por canal
fact_time_canal = fact_time.merge(dim_canal, on="canal_key", how="left")
q1_canal = (fact_time_canal.loc[mask_2023]
            .groupby(["yyyymm", "canal"])
            .agg(pedidos=("order_id", "nunique"),
                ingreso_neto=("importe_neto", "sum"))
            .reset_index()
            .sort_values(["yyyymm", "canal"]))

q1.head(), q1_canal.head()

```

```

(   yyyymm  pedidos  lineas  ingreso_neto
0  2023-02         1       3         389.25
1  2023-03         1       3         257.86
2  2023-04         2       4         533.00
3  2023-05         2       7         797.46
4  2023-06         3       7        1129.59,
   yyyymm canal  pedidos  ingreso_neto
0  2023-02   APP         1         389.25
1  2023-03   POS         1         257.86
2  2023-04   APP         2         533.00
3  2023-05   APP         1         360.25
4  2023-05   POS         1         437.21)

```

21.7.2. 6.2 Pregunta 2: Categorías con mayor demanda en el periodo post

Se recomienda reportar tanto unidades (volumen) como ingreso neto.

```

fact_prod = fact_time.merge(dim_producto[["producto_key", "categoria", "subcategoria"]], on="producto_key")
q2 = (fact_prod.loc[fact_prod["fecha"] >= APP_LAUNCH]
      .groupby("categoria")
      .agg(unidades=("cantidad", "sum"),
          ingreso_neto=("importe_neto", "sum"),
          pedidos=("order_id", "nunique"))

```

```
.reset_index()
.sort_values(["ingreso_neto","unidades"], ascending=False))

q2.head(10)
```

	categoria	unidades	ingreso_neto	pedidos
1	Bebidas Calientes	49	2545.44	13
2	Bebidas Frías	28	2037.89	8
4	Postres	35	1692.36	12
0	Accesorios	15	1386.63	7
3	Panadería	27	894.85	11

21.7.3. 6.3 Pregunta 3: Variaciones por zona y tipo de cliente (pre vs post)

Se propone comparar **delta** (post - pre) en pedidos e ingreso neto.

```
fact_zona = fact_prod.merge(dim_sucursal[["sucursal_key","zona"]], on="sucursal_key", how="left")
fact_zona = fact_zona.merge(dim_cliente[["cliente_key","tipo_cliente"]], on="cliente_key", how="left")

fact_zona["periodo"] = np.where(fact_zona["fecha"] >= APP_LAUNCH, "post", "pre")

q3 = (fact_zona
      .groupby(["periodo","zona","tipo_cliente"])
      .agg(pedidos=("order_id","nunique"),
           ingreso_neto=("importe_neto","sum"))
      .reset_index())

# Pivot para deltas
q3_pivot = (q3.pivot_table(index=["zona","tipo_cliente"], columns="periodo",
                           values=["pedidos","ingreso_neto"], aggfunc="sum")
            .fillna(0))

q3_pivot[("pedidos","delta")] = q3_pivot[("pedidos","post")] - q3_pivot[("pedidos","pre")]
q3_pivot[("ingreso_neto","delta")] = q3_pivot[("ingreso_neto","post")] - q3_pivot[("ingreso_neto","pre")]

q3_delta = q3_pivot.reset_index()
q3_delta.columns = ["_".join([c for c in col if c]) if isinstance(col, tuple) else col for col in q3_delta.columns]

q3_delta.sort_values("ingreso_neto_delta", ascending=False).head(15)
```

	zona	tipo_cliente	ingreso_neto_post	ingreso_neto_pre	pedidos_post	pedidos_pre	pedidos_delta
8	Sur	sin_lealtad	1919.22	0.00	5.0	0.0	5.0
0	Centro	lealtad	1700.87	201.77	5.0	1.0	4.0
4	Oriente	lealtad	1208.98	272.56	2.0	1.0	1.0
7	Sur	lealtad	858.78	0.00	2.0	0.0	2.0
3	Norte	sin_lealtad	854.44	0.00	2.0	0.0	2.0

	zona	tipo_cliente	ingreso_netto_post	ingreso_netto_pre	pedidos_post	pedidos_pre	pedidos_delta
2	Norte	lealtad	1006.74	426.19	2.0	1.0	1.0
5	Oriente	sin_lealtad	477.21	137.77	2.0	1.0	1.0
6	Poniente	lealtad	104.19	0.00	1.0	0.0	1.0
1	Centro	sin_lealtad	426.74	407.78	1.0	1.0	0.0

21.7.4. 6.4 Pregunta 4: Cancelación y cupones (tasas) por mes

Las tasas son **no aditivas**, por lo que deben calcularse como promedios o cocientes.

```
q4 = (fact_time
      .groupby("yyyymm")
      .agg(
          tasa_cancelacion=("cancelado_flag", "mean"),
          tasa_cupon=("cupon_aplicado_flag", "mean"),
          pedidos=("order_id", "nunique"),
          lineas=("order_id", "size"),
          ingreso_netto=("importe_netto", "sum"),
      )
      .reset_index()
      .sort_values("yyyymm"))

q4.tail(12)
```

	yyyymm	tasa_cancelacion	tasa_cupon	pedidos	lineas	ingreso_netto
3	2022-12	0.000000	0.333333	1	3	272.56
4	2023-02	0.000000	0.333333	1	3	389.25
5	2023-03	0.000000	0.666667	1	3	257.86
6	2023-04	0.000000	0.000000	2	4	533.00
7	2023-05	0.000000	0.285714	2	7	797.46
8	2023-06	0.000000	0.714286	3	7	1129.59
9	2023-07	0.000000	0.000000	1	5	822.09
10	2023-08	0.166667	0.166667	1	6	696.85
11	2023-09	0.125000	0.375000	2	8	653.90
12	2023-10	0.142857	0.428571	5	14	1545.01
13	2023-11	0.000000	0.428571	3	7	961.61
14	2023-12	0.000000	0.000000	1	5	770.55

21.8. 7. Exportación de tablas: Dimensiones y Hechos (Data Mart físico)

En una implementación típica, estas tablas se materializan como CSV/Parquet en un *lakehouse* o como tablas físicas en un motor analítico (PostgreSQL, BigQuery, Snowflake, DuckDB, etc.). Aquí se exportan a out/.

```

OUT_DIR = Path("out")
OUT_DIR.mkdir(exist_ok=True)

dim_tiempo.to_csv(OUT_DIR / "dim_tiempo.csv", index=False)
dim_cliente.to_csv(OUT_DIR / "dim_cliente.csv", index=False)
dim_producto.to_csv(OUT_DIR / "dim_producto.csv", index=False)
dim_sucursal.to_csv(OUT_DIR / "dim_sucursal.csv", index=False)
dim_canal.to_csv(OUT_DIR / "dim_canal.csv", index=False)
dim_metodo_pago.to_csv(OUT_DIR / "dim_metodo_pago.csv", index=False)
dim_promocion.to_csv(OUT_DIR / "dim_promocion.csv", index=False)

fact_linea_pedido.to_csv(OUT_DIR / "fact_linea_pedido.csv", index=False)

# Data marts de salida (consultas)
q1.to_csv(OUT_DIR / "mart_pedidos_mensuales.csv", index=False)
q1_canal.to_csv(OUT_DIR / "mart_pedidos_mensuales_canal.csv", index=False)
q2.to_csv(OUT_DIR / "mart_demanda_categorias_post.csv", index=False)
q3_delta.to_csv(OUT_DIR / "mart_variacion_zona_cliente_pre_post.csv", index=False)
q4.to_csv(OUT_DIR / "mart_tasas_cancelacion_cupon_mes.csv", index=False)

sorted([p.name for p in OUT_DIR.glob("*.csv")])

['dim_canal.csv',
 'dim_cliente.csv',
 'dim_metodo_pago.csv',
 'dim_producto.csv',
 'dim_promocion.csv',
 'dim_sucursal.csv',
 'dim_tiempo.csv',
 'fact_linea_pedido.csv',
 'mart_demanda_categorias_post.csv',
 'mart_pedidos_mensuales.csv',
 'mart_pedidos_mensuales_canal.csv',
 'mart_tasas_cancelacion_cupon_mes.csv',
 'mart_variacion_zona_cliente_pre_post.csv']

```

21.9. 8. Discusión técnica: decisiones de diseño y extensiones

1. **Dimensiones conformadas:** `dim_tiempo`, `dim_cliente`, `dim_producto`, `dim_sucursal`, `dim_canal` permiten integrar *Ventas POS* y *Pedidos App* en una misma semántica de consulta (bus architecture).
2. **Granularidad:** al modelar a nivel de línea, se preserva el análisis de mezcla de productos y el efecto de promociones en el importe.

3. **SCD Tipo 2 (extensión recomendada)**: si se desea preservar historia de cambios en `categoria` o en `nivel_lealtad`, se debe:
 - añadir `vigencia_inicio`, `vigencia_fin`, `vigente_flag`,
 - resolver la clave dimensional “as-of” en la fecha del hecho.
4. **Late Arriving Dimensions**: en escenarios reales, pueden llegar transacciones cuyo cliente aún no existe en el catálogo. La práctica estándar es asignar **Unknown Member** (`*_key = 0`) y corregir al cierre del ciclo de carga.
5. **Prueba estadística pre/post (opcional)**: para un tratamiento académico, se sugiere formalizar el cambio con pruebas no paramétricas o bootstrap sobre series mensuales (p. ej., pedidos o tasa de cancelación).

21.10. 9. Reproducibilidad y control de versiones

Para un libro académico, se recomienda:

- fijar versiones de dependencias (p. ej., `pandas==2.x`, `numpy==2.x`),
- mantener `data/` como directorio externo (no versionar datos sensibles),
- versionar el pipeline y el `.qmd` (Git),
- documentar el diccionario de datos final (metadatos de cada dimensión y hecho).

Parte VII.

Integración ETL/ELT

22. Introducción a PySpark y Fundamentos ETL

22.1. 1. ¿Qué es el ETL y por qué importa?

ETL (*Extract, Transform, Load*) es el proceso que mueve datos desde los sistemas operacionales (OLTP, APIs, logs, archivos) hacia el Data Warehouse, transformándolos en el camino para que sean consistentes, limpios y aptos para el análisis.

i Nota

Distinción con el Preprocesamiento (Parte II) ETL es ingeniería de datos: opera sobre flujos de producción, terabytes, esquemas fijos, y su salida es reproducible y automatizada. El preprocesamiento analítico (Parte II) toma una *vista ya limpia* del DW y la transforma para un experimento concreto. El DW es el punto de transferencia entre ambos.

22.1.1. 1.1 ETL clásico vs ELT moderno

	ETL clásico	ELT moderno
Orden	Extraer → Transformar → Cargar	Extraer → Cargar → Transformar
Dónde transforma	Motor ETL externo	Dentro del DW / lakehouse
Herramientas	Informatica, SSIS, Talend	dbt + BigQuery / Snowflake / DuckDB
Ventaja	Datos siempre limpios al llegar	Aprovechar la potencia del DW columnares
Desventaja	Cuello de botella en el motor	El DW necesita ser potente y barato

22.1.2. 1.2 ¿Por qué PySpark?

Cuando los datos superan la memoria RAM de una sola máquina, pandas deja de ser suficiente. **Apache Spark** divide el trabajo en particiones que se procesan en paralelo sobre un clúster de nodos, usando un plan de ejecución optimizado (DAG — *Directed Acyclic Graph*).

PySpark es la API Python de Spark. Con ella puedes escribir transformaciones que escalan de un laptop (modo local) a cientos de nodos (AWS EMR, Databricks, GCP Dataproc) sin cambiar el código.

22.2. 2. Entorno de prácticas: Databricks Community Edition

Todas las prácticas de este bloque se realizan en **Databricks Community Edition**, que proporciona un clúster Spark gratuito sin necesidad de instalación local.

22.2.1. 2.1 Configurar la cuenta

1. Ir a community.cloud.databricks.com y crear una cuenta gratuita.
2. En el menú lateral: **Compute** → **Create compute** → elegir el runtime **Databricks Runtime 14.x LTS (Spark 3.5, Python 3.11)**.
3. Crear un **Notebook** (lenguaje: Python).

! Importante

En Databricks, `spark` ya existe como variable global — **no** crees un `SparkSession` manual. Simplemente empieza con `spark.read...`

22.2.2. 2.2 Cargar datasets a DBFS

```
# Databricks File System (DBFS) - sistema de archivos del clúster
# Opción A: subir archivo desde la UI (File → Upload data)

# Opción B: descargar directamente desde el notebook con dbutils
dbutils.fs.mkdirs("dbfs:/datasets/taxi/")

# Descargar NYC Taxi (Parquet, ~500 MB por mes)
import urllib.request
url = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet"
urllib.request.urlretrieve(url, "/tmp/yellow_2023_01.parquet")
dbutils.fs.mv("file:/tmp/yellow_2023_01.parquet", "dbfs:/datasets/taxi/yellow_2023_01.parquet")

# Verificar
display(dbutils.fs.ls("dbfs:/datasets/taxi/"))
```

```
# Leer desde DBFS en Databricks
df = spark.read.parquet("dbfs:/datasets/taxi/yellow_2023_01.parquet")
display(df.limit(5)) # display() es el equivalente de show() en Databricks
```

22.2.3. 2.3 Comandos específicos de Databricks

Comando	Descripción
<code>display(df)</code>	Tabla interactiva con gráficos integrados
<code>dbutils.fs.ls("dbfs:/ruta/")</code>	Listar archivos en DBFS
<code>dbutils.fs.mkdirs("dbfs:/ruta/")</code>	Crear directorio en DBFS

Comando	Descripción
<code>dbutils.fs.rm("dbfs:/ruta/", True)</code>	Eliminar archivo o directorio
<code>%sql SELECT ...</code>	Celda SQL sobre tablas registradas
<code>spark.catalog.listTables()</code>	Ver tablas disponibles en el metastore

💡 Tip

Delta Lake está preinstalado en Databricks. No necesitas `pip install delta-spark` — simplemente usa `.format("delta")` directamente.

22.2.4. 2.4 Datasets para las prácticas

Dataset	Tamaño	Dónde obtenerlo en Databricks
NYC Taxi (1 mes)	~500 MB	Descargar con <code>urllib</code> (ver §2.2)
NYC Taxi (todo 2023)	~6 GB	Bucle sobre los 12 meses
TPC-H sintético	Configurable	<code>%pip install tpch + python -m tpch generate</code>
Databricks sample data	Varios	<code>dbfs:/databricks-datasets/</code> (preinstalado)

```
# Explorar los datasets de muestra de Databricks
display(dbutils.fs.ls("dbfs:/databricks-datasets/"))
# Incluye: nyctaxi, airlines, wine-quality, iot-stream, retail-org, etc.

# Leer el dataset de ejemplo de taxis (ya disponible)
df_sample = spark.read.format("delta")\
    .load("dbfs:/databricks-datasets/nyctaxi/tables/nyctaxi_yellow")
display(df_sample.limit(10))
```

22.3. 3. PySpark: Fundamentos

22.3.1. 3.1 SparkSession

En **Databricks** el `SparkSession` ya existe como `spark`. En entorno local:

```
from pyspark.sql import SparkSession

# Solo necesario fuera de Databricks
spark = (SparkSession.builder
    .appName("ETL-Mineria-Datos")
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())
```

22.3.2. 3.2 El DataFrame de Spark: lazy vs eager

```
# Transformaciones (lazy) - solo construyen el plan, no ejecutan nada
df = spark.read.parquet("dbfs:/datasets/taxi/yellow_2023_01.parquet")
df2 = df.filter(df.trip_distance > 1.0)
df3 = df2.select("tpep_pickup_datetime", "trip_distance", "total_amount")

# Acción - dispara la ejecución del DAG completo
df3.show(5)      # o display(df3) en Databricks
df3.count()
```

Concepto	Significado
Transformación	filter, select, groupBy, join — construyen el DAG
Acción	show, count, collect, write — ejecutan el DAG
Partición	Unidad de paralelismo; un archivo Parquet por partición típicamente
RDD	Capa de bajo nivel; usa DataFrame salvo casos especiales

22.3.3. 3.3 Esquema explícito (mejor práctica)

```
from pyspark.sql.types import *

schema = StructType([
    StructField("tpep_pickup_datetime", TimestampType(), True),
    StructField("tpep_dropoff_datetime", TimestampType(), True),
    StructField("passenger_count", IntegerType(), True),
    StructField("trip_distance", DoubleType(), True),
    StructField("fare_amount", DoubleType(), True),
    StructField("tip_amount", DoubleType(), True),
    StructField("total_amount", DoubleType(), True),
    StructField("payment_type", IntegerType(), True),
])

df = spark.read.parquet("dbfs:/datasets/taxi/yellow_2023_01.parquet")
# Parquet guarda el schema; con CSV/JSON conviene declararlo explícito:
# df = spark.read.csv("dbfs:/ruta.csv", header=True, schema=schema)
```

inferSchema=True hace una pasada completa del archivo; con esquema explícito la lectura es más rápida y sin sorpresas de tipos.

22.4. Referencias de este capítulo

- Chambers & Zaharia. *Spark: The Definitive Guide*. O'Reilly, 2018.
- PySpark Documentation
- Databricks Community Edition
- Databricks Free Training — cursos oficiales gratuitos.

23. Extracción y Fuentes Heterogéneas

Nota

Este capítulo asume que tienes Databricks Community Edition configurado y los datasets cargados en DBFS según el capítulo anterior.

23.1. 1. Extracción (*Extract*)

23.1.1. 4.1 Lectura de formatos comunes

```
# CSV
df_csv = spark.read.csv("data/trips.csv", header=True, schema=schema)

# Parquet (columnar, comprimido - el formato preferido para DW)
df_pq = spark.read.parquet("data/trips.parquet")

# JSON (líneas)
df_json = spark.read.json("data/reviews.jsonl")

# Directorio completo de Parquet (Spark los une automáticamente)
df_all = spark.read.parquet("data/trips/year=2023/")

# JDBC (base de datos relacional)
df_jdbc = (spark.read
    .format("jdbc")
    .option("url", "jdbc:postgresql://host:5432/ventas")
    .option("dbtable", "public.pedidos")
    .option("user", "etl_user")
    .option("password", "secret")
    .option("numPartitionColumns", "pedido_id") # paralelismo
    .option("lowerBound", "1")
    .option("upperBound", "10000000")
    .option("numPartitions", "16")
    .load())
```

23.1.2. 4.2 Particionamiento y paralelismo en la lectura

```
# Ver cuántas particiones tiene el DataFrame
print(f"Particiones: {df_pq.rdd.getNumPartitions()}")

# Reparticionar para balancear carga
df_balanced = df_pq.repartition(32)

# Coalesce - reduce particiones sin shuffle (para escribir pocos archivos)
df_small = df_balanced.coalesce(4)
```

23.1.3. 4.3 Ejercicio E-1 — Ingestar NYC Taxi

Importante

Ejercicio E-1: Lectura y exploración básica

```
# Descargar 1 mes de datos ( 500 MB)
wget https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet \
-O data/yellow_tripdata_2023-01.parquet
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, mean, min, max, stddev

spark = SparkSession.builder.appName("E1-NYC").master("local[*]").getOrCreate()

df = spark.read.parquet("data/yellow_tripdata_2023-01.parquet")

# 1. ¿Cuántos registros y columnas tiene el archivo?
print(f"Filas: {df.count():,} Columnas: {len(df.columns)}")

# 2. Muestra el esquema inferido
df.printSchema()

# 3. Estadísticas descriptivas de las variables numéricas
df.select("trip_distance", "fare_amount", "tip_amount", "total_amount")\
    .describe().show()

# 4. ¿Cuántos viajes por tipo de pago?
df.groupBy("payment_type").count().orderBy("count", ascending=False).show()

# 5. ¿Cuántas particiones usa Spark al leer el Parquet?
print(f"Particiones: {df.rdd.getNumPartitions()}")
```

Entregable: tabla con estadísticas y respuesta a las 5 preguntas.

23.2. 2. Integración de fuentes heterogéneas: Data Lake → DW

En la práctica, los datos llegan en formatos distintos, con nombres de columnas inconsistentes, tipos incompatibles y niveles de anidamiento variables. Esta sección cubre el conjunto de técnicas que permiten **homologar** esas fuentes antes de cargarlas al DW.

23.2.1. 5.1 El problema: fuentes heterogéneas en la Zona RAW

Cuando un Data Lake acumula datos de múltiples fuentes, los problemas típicos son:

Problema	Ejemplo real
Formatos distintos	ventas en CSV, eventos en JSON, catálogo en XML
Nombres de columna inconsistentes	customer_id vs cust_id vs clienteId
Tipos incompatibles	"2024-01-15" (string) vs 1705276800 (epoch) vs 20240115 (int)
Anidamiento en JSON	{"usuario": {"nombre": "Ana", "dir": {"ciudad": "CDMX"}}}
Encodings distintos	UTF-8 vs Latin-1 vs CP1252
Delimitadores no estándar	; en CSV europeo, en dumps de bases de datos
Nulos representados distinto	",", "NULL", "N/A", "\\N", None
Unidades distintas	distancia en km vs millas, precios en MXN vs USD

23.2.2. 5.2 JSON anidado: aplanar para el DW

El JSON es el formato más común en APIs y event streams, pero el DW necesita una tabla **plana** (filas × columnas escalares). PySpark ofrece tres herramientas para esto:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, get_json_object, from_json, to_json,
    explode, explode_outer, inline, flatten,
    struct, to_timestamp
)
from pyspark.sql.types import *

spark = SparkSession.builder.appName("ETL-JSON").master("local[*]").getOrCreate()

# Ejemplo: eventos de compra (JSON lines)
# Cada línea es un JSON con objetos anidados y arrays
# {"event_id":"e1","ts":"2024-01-15T10:30:00Z",
#  "user":{"id":42,"city":"CDMX","segment":"premium"},
#  "items":[{"sku":"A01","qty":2,"price":199.0},
#           {"sku":"B03","qty":1,"price":89.5}]}

df_raw = spark.read.json("data/lake/eventos_compra.jsonl")
df_raw.printSchema()
# root
```

```
# |-- event_id: string
# |-- ts: string
# |-- user: struct
# |   |-- id: long
# |   |-- city: string
# |   |-- segment: string
# |-- items: array
# |   |-- element: struct
# |       |-- sku: string
# |       |-- qty: long
# |       |-- price: double
```

Técnica 1: acceder a campos anidados con notación punto

```
# Struct → columnas individuales con notación punto
df_flat = df_raw.select(
    col("event_id"),
    to_timestamp(col("ts")).alias("evento_ts"),
    col("user.id").alias("usuario_id"),
    col("user.city").alias("ciudad"),
    col("user.segment").alias("segmento"),
    col("items") # el array queda como columna de tipo array
)
```

Técnica 2: explode — un array → una fila por elemento

```
# Cada item del array se convierte en una fila independiente
df_items = (df_raw
    .select(
        col("event_id"),
        to_timestamp(col("ts")).alias("evento_ts"),
        col("user.id").alias("usuario_id"),
        explode(col("items")).alias("item") # <- desdobra el array
    )
    .select(
        col("event_id"),
        col("evento_ts"),
        col("usuario_id"),
        col("item.sku").alias("sku"),
        col("item.qty").alias("cantidad"),
        col("item.price").alias("precio_unitario"),
        (col("item.qty") * col("item.price")).alias("subtotal")
    )
)
df_items.show(5)
# +-----+-----+-----+-----+-----+-----+-----+
# |event_id|evento_ts|usuario_id|sku|cantidad|precio_unitario|subtotal|
# +-----+-----+-----+-----+-----+-----+-----+
```


#	e1	2024-01-15 10:30:00 42	A01 2	199.0	398.0	
#	e1	2024-01-15 10:30:00 42	B03 1	89.5	89.5	

i Nota

explode elimina filas con array vacío o nulo. Usa `explode_outer` si quieres conservarlas como fila con `item = null`.

Técnica 3: JSON embebido como string → parsear con esquema explícito

En muchos pipelines el JSON llega como una columna de texto dentro de otro CSV o tabla (columna tipo `STRING` con JSON serializado):

```
# CSV con una columna "payload" que es un JSON serializado como string
# event_id,payload
# e1,"{\"user_id\":42,\"amount\":288.5,\"currency\":\"MXN\"}"

df_csv = spark.read.csv("data/lake/events_flat.csv", header=True)

# Definir el esquema del JSON embebido
payload_schema = StructType([
    StructField("user_id", LongType(), True),
    StructField("amount", DoubleType(), True),
    StructField("currency", StringType(), True),
])

# Parsear la columna string → struct tipado
df_parsed = (df_csv
    .withColumn("payload_struct",
        from_json(col("payload"), payload_schema))
    .select(
        col("event_id"),
        col("payload_struct.user_id").alias("usuario_id"),
        col("payload_struct.amount").alias("monto"),
        col("payload_struct.currency").alias("moneda")
    )
)
```

Técnica 4: get_json_object — extracción selectiva con JSONPath

Cuando solo necesitas algunos campos de un JSON muy profundo sin parsear el esquema completo:

```
df_select = df_csv.select(
    col("event_id"),
    get_json_object(col("payload"), "$.user_id").cast("long").alias("usuario_id"),
    get_json_object(col("payload"), "$.amount").cast("double").alias("monto"),
    get_json_object(col("payload"), "$.currency").alias("moneda"),
)
```

23.2.3. 5.3 CSV heterogéneo: normalización de esquema entre fuentes

Un escenario muy común: el mismo tipo de dato llega de tres sistemas distintos con nombres de columna y tipos diferentes. El ETL debe producir una tabla unificada.

```
# Fuente 1: sistema ERP (México)
# cust_id,fecha_pedido,monto_mxn,desc_pct
# 1001,15/01/2024,3500.00,0.10

# Fuente 2: sistema CRM (EE.UU.)
# CustomerID,OrderDate,AmountUSD,Discount
# 1001,2024-01-15,175.00,10

# Fuente 3: event log (JSON lines)
# {"cliente":1001,"ts":"2024-01-15T10:30:00Z","total":{"mxn":3500,"usd":175},"dcto":0.10}

from pyspark.sql.functions import (
    col, to_date, to_timestamp, lit, round as spark_round
)

TIPO_CAMBIO_MXN_USD = 17.0 # tasa de ejemplo

# Fuente 1: ERP
df_erp = (spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .csv("data/lake/erp_pedidos.csv")
    .select(
        col("cust_id").cast("long").alias("cliente_id"),
        to_date(col("fecha_pedido"), "dd/MM/yyyy").alias("fecha"),
        col("monto_mxn").cast("double").alias("monto_mxn"),
        spark_round(col("monto_mxn").cast("double") / TIPO_CAMBIO_MXN_USD, 2).alias("monto_usd"),
        col("desc_pct").cast("double").alias("descuento"),
        lit("ERP").alias("fuente")
    )
)

# Fuente 2: CRM
df_crm = (spark.read
    .option("header", True)
    .option("encoding", "Latin-1") # encoding diferente
    .csv("data/lake/crm_orders.csv")
    .select(
        col("CustomerID").cast("long").alias("cliente_id"),
        to_date(col("OrderDate"), "yyyy-MM-dd").alias("fecha"),
        spark_round(col("AmountUSD").cast("double") * TIPO_CAMBIO_MXN_USD, 2).alias("monto_mxn"),
        col("AmountUSD").cast("double").alias("monto_usd"),
        (col("Discount").cast("double") / 100).alias("descuento"), # % → fracción
        lit("CRM").alias("fuente")
    )
)
```

```

)

# Fuente 3: Event log JSON
payload_schema_ev = StructType([
    StructField("cliente", LongType(), True),
    StructField("ts", StringType(), True),
    StructField("total", StructType([
        StructField("mxn", DoubleType(), True),
        StructField("usd", DoubleType(), True),
    ]), True),
    StructField("dcto", DoubleType(), True),
])

df_events = (spark.read.schema(payload_schema_ev).json("data/lake/events.jsonl")
    .select(
        col("cliente").alias("cliente_id"),
        to_date(to_timestamp(col("ts"))).alias("fecha"),
        col("total.mxn").alias("monto_mxn"),
        col("total.usd").alias("monto_usd"),
        col("dcto").alias("descuento"),
        lit("EVENT").alias("fuente")
    )
)

# Unión de las tres fuentes con esquema homologado
df_unificado = df_erp.unionByName(df_crm).unionByName(df_events)

print(f"ERP:      {df_erp.count():>8,} registros")
print(f"CRM:      {df_crm.count():>8,} registros")
print(f"Events: {df_events.count():>8,} registros")
print(f"Total:   {df_unificado.count():>8,} registros")

df_unificado.printSchema()
df_unificado.groupBy("fuente").count().show()

```

Advertencia

`unionByName` une columnas **por nombre**, no por posición. Si una fuente tiene una columna extra que las otras no, usa `unionByName(df2, allowMissingColumns=True)` para rellenarla con `null`.

23.2.4. 5.4 Normalización de tipos y valores nulos heterogéneos

```

from pyspark.sql.functions import (
    when, trim, upper, lower, regexp_replace,
    coalesce, nanvl, col
)

```

```

# Nulos representados como strings
NULLS_COMO_STRING = ["NULL", "N/A", "\\N", "None", "nan", "", " "]

def limpiar_nulos(df):
    for c in df.columns:
        if dict(df.dtypes)[c] == "string":
            df = df.withColumn(c,
                               when(trim(col(c)).isin(NULLS_COMO_STRING), None)
                               .otherwise(trim(col(c))))
    )
    return df

df_limpio = limpiar_nulos(df_unificado)

# Normalización de texto
df_texto = (df_limpio
            # quitar acentos y caracteres especiales en columnas de texto
            .withColumn("ciudad",
                        regexp_replace(upper(trim(col("ciudad"))),
                                      "[ÁÉÍÓÚÜ]", lambda m: {"Á": "A", "É": "E", "Í": "I",
                                                                "Ó": "O", "Ú": "U", "Ü": "U"}
                                      .get(m.group(), m.group()))
            # normalizar segmentos a minúsculas sin espacios extra
            .withColumn("segmento", lower(trim(col("segmento"))))
            )

# Conversión de unidades heterogéneas
# Distancias en distintas unidades según la fuente
df_distancias = df_raw.withColumn("distancia_km",
                                   when(col("unidad") == "mi", col("distancia") * 1.60934)
                                   .when(col("unidad") == "km", col("distancia"))
                                   .when(col("unidad") == "m", col("distancia") / 1000.0)
                                   .otherwise(None)
                                   )

# Fechas en múltiples formatos → timestamp estándar
from pyspark.sql.functions import coalesce, to_timestamp

def parse_fecha_multiple(col_name):
    formatos = [
        "yyyy-MM-dd'T'HH:mm:ssX", # ISO 8601 con timezone
        "yyyy-MM-dd HH:mm:ss",    # datetime sin timezone
        "dd/MM/yyyy",             # fecha europea
        "MM/dd/yyyy",             # fecha americana
        "yyyyMMdd",               # compacto
    ]
    return coalesce(*[to_timestamp(col(col_name), fmt) for fmt in formatos])

df_fechas = df_raw.withColumn("fecha_std", parse_fecha_multiple("fecha_raw"))

```

23.2.5. 5.5 CSV con delimitadores no estándar y encodings

```
# Archivo con delimitador | y encoding Latin-1 (típico de dumps de SAP/Oracle)
df_sap = (spark.read
    .option("sep", "|")
    .option("encoding", "ISO-8859-1")
    .option("header", True)
    .option("quote", "'")
    .option("escape", "\\")
    .option("multiLine", False)
    .option("nullValue", "")
    .csv("data/lake/sap_export.txt")
)

# Archivo XML - requiere la librería spark-xml
# spark-submit --packages com.databricks:spark-xml_2.12:0.17.0
df_xml = (spark.read
    .format("com.databricks.spark.xml")
    .option("rowTag", "pedido")      # tag que define una fila
    .load("data/lake/pedidos.xml")
)

# Avro - requiere spark-avro (incluido en Spark 3.x)
df_avro = spark.read.format("avro").load("data/lake/eventos.avro")
```

23.2.6. 5.6 Conversión JSON (Data Lake) → Parquet (DW): pipeline completo

Este es el patrón más frecuente en la industria: el Data Lake almacena eventos en JSON; el DW los consume como Parquet particionado y tipado.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, explode, to_timestamp, to_date,
    lit, trim, when, coalesce
)
from pyspark.sql.types import *

spark = (SparkSession.builder
    .appName("lake-to-dw")
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())

# EXTRACT
# Data Lake: JSON semiestructurado con anidamiento
schema_raw = StructType([
    StructField("event_id", StringType(), False),
    StructField("ts", StringType(), True),
```

```

StructField("version", IntegerType(), True),
StructField("user", StructType([
    StructField("id", LongType(), True),
    StructField("city", StringType(), True),
    StructField("segment", StringType(), True),
    StructField("country", StringType(), True),
]), True),
StructField("items", ArrayType(StructType([
    StructField("sku", StringType(), True),
    StructField("category", StringType(), True),
    StructField("qty", IntegerType(), True),
    StructField("price_usd", DoubleType(), True),
])), True),
StructField("payment", StructType([
    StructField("method", StringType(), True),
    StructField("currency", StringType(), True),
    StructField("fx_rate", DoubleType(), True), # tasa vs USD
]), True),
])

df_raw = spark.read.schema(schema_raw).json("data/lake/raw/eventos/")

print(f"Registros en lago: {df_raw.count():,}")

# TRANSFORM: aplanar + normalizar + explode
df_items = (df_raw
    # Explotar el array de items (1 evento → N filas)
    .withColumn("item", explode(col("items")))
    # Aplanar structs anidados
    .select(
        col("event_id"),
        to_timestamp(col("ts")).alias("evento_ts"),
        to_date(to_timestamp(col("ts"))).alias("fecha"),
        col("user.id").cast("long").alias("usuario_id"),
        trim(col("user.city")).alias("ciudad"),
        trim(col("user.segment")).alias("segmento"),
        coalesce(col("user.country"), lit("MX")).alias("pais"),
        col("item.sku").alias("sku"),
        col("item.category").alias("categoria"),
        col("item.qty").cast("integer").alias("cantidad"),
        col("item.price_usd").alias("precio_usd"),
        # Convertir a moneda local usando la tasa del evento
        (col("item.price_usd") * col("payment.fx_rate")).alias("precio_local"),
        col("payment.method").alias("metodo_pago"),
        col("payment.currency").alias("moneda"),
        # Subtotales
        (col("item.qty") * col("item.price_usd")).alias("subtotal_usd"),
        (col("item.qty") * col("item.price_usd") *

```

```

        col("payment.fx_rate")).alias("subtotal_local"),
    )
    # Filtros de calidad
    .filter(col("cantidad") > 0)
    .filter(col("precio_usd") > 0)
    .filter(col("evento_ts").isNotNull())
)

# Añadir columnas de particionamiento derivadas del timestamp
from pyspark.sql.functions import year, month, dayofmonth
df_final = (df_items
    .withColumn("anio", year("fecha"))
    .withColumn("mes", month("fecha"))
    .withColumn("dia", dayofmonth("fecha"))
)

# LOAD: Parquet particionado en el DW
DW_PATH = "output/dw/hechos_ventas_items/"

(df_final
    .write
    .mode("overwrite")
    .partitionBy("anio", "mes", "dia")
    .parquet(DW_PATH))

# Verificar
df_dw = spark.read.parquet(DW_PATH)
print(f"\nRegistros en DW: {df_dw.count():,}")
print(f"Columnas: {len(df_dw.columns)}")
df_dw.printSchema()

# Vista rápida de calidad
df_dw.createOrReplaceTempView("hechos")
spark.sql("""
    SELECT
        pais, segmento, moneda,
        COUNT(*) AS n_items,
        SUM(subtotal_usd) AS ingreso_usd,
        AVG(precio_usd) AS precio_prom_usd
    FROM hechos
    GROUP BY pais, segmento, moneda
    ORDER BY ingreso_usd DESC
""").show(truncate=False)

spark.stop()

```

23.2.7. 5.7 Reconciliación de esquemas con mergeSchema

Cuando el JSON evoluciona con el tiempo y los archivos del lago tienen esquemas distintos (una columna nueva apareció en febrero, otra se renombró en marzo), Spark puede reconciliarlos automáticamente:

```
# Leer múltiples versiones de Parquet con esquemas que evolucionaron
df_historico = (spark.read
    .option("mergeSchema", "true")    # une el superset de columnas
    .parquet("data/lake/parquet/historico/"))

# Las columnas que no existían en versiones antiguas aparecen como null
df_historico.printSchema()

# Para Delta Lake: evolución de esquema está integrada
(df_nuevo.write
    .format("delta")
    .option("mergeSchema", "true")    # permite agregar columnas nuevas
    .mode("append")
    .save("output/delta/hechos/"))
```

23.2.8. 5.8 Ejercicio E-1b — JSON a Parquet del Data Lake al DW

Importante

Ejercicio E-1b: Integración de fuentes heterogéneas

Descarga los datos de GitHub Archive (eventos de un día, 2 GB en JSON comprimido) y conviértelos a una tabla de hechos Parquet en el DW.

```
# Descargar un día de eventos de GitHub (JSON gzippeado por hora)
for HORA in $(seq -w 0 23); do
    wget "https://data.gharchive.org/2024-01-15-#{HORA}.json.gz" \
        -P data/lake/gharchive/
done
# Total: ~2 GB, ~3-4 M eventos
```



```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_timestamp, get_json_object, explode_outer

spark = SparkSession.builder.appName("E1b-GitHub").master("local[*]")\
    .config("spark.sql.shuffle.partitions","8").getOrCreate()

# GitHub Archive: cada línea es un JSON de evento
# Campos clave: id, type, actor.login, repo.name, created_at, payload (objeto)
df_raw = spark.read.json("data/lake/gharchive/*.json.gz")

# TODO 1: ¿Cuántos tipos de evento hay? ¿Cuál es el más frecuente?
tipos = ... # ← completa

# TODO 2: Extrae una tabla plana con:
# evento_id, tipo, actor, repo, fecha (date), hora (int)
df_plano = (df_raw.select(
    col("id").alias("evento_id"),
    col("type").alias("tipo"),
    col("actor.login").alias("actor"),
    col("repo.name").alias("repo"),
    to_timestamp(col("created_at")).alias("evento_ts"),
    # ← añade fecha y hora derivados del timestamp
)) # completa

# TODO 3: Filtra solo eventos "PushEvent" y extrae el número de commits
# del payload (payload.commits es un array; usa size() o explode_outer)
df_push = ... # ← completa

# TODO 4: Escribe df_plano como Parquet particionado por tipo y fecha
... # ← completa

# TODO 5: Consulta con Spark SQL:
# Los 10 repos con más pushes en el día, con el total de commits
... # ← completa

spark.stop()

```

Preguntas: a) ¿Cuántos eventos totales hay en el día? ¿Qué fracción son PushEvents? b) El campo `payload` es diferente para cada tipo de evento. ¿Cómo manejarías esto si quisieras modelar cada tipo como su propia tabla de hechos? c) ¿Qué ventaja tiene Parquet vs JSON para consultas repetidas sobre estos datos? Compara el tiempo de `COUNT(*)` sobre el JSON original vs el Parquet.

23.3. Referencias de este capítulo

- NYC TLC Trip Record Data
- GitHub Archive

24. Transformación, Carga y Pipeline ETL

Nota

Continuación del capítulo anterior. Asume que tienes el DataFrame `df` cargado desde DBFS.

24.1. 1. Transformación (*Transform*)

24.1.1. 1.1 Funciones de columna (`pyspark.sql.functions`)

```
from pyspark.sql.functions import (  
    col, lit, when, coalesce,  
    to_timestamp, date_format, hour, dayofweek, month,  
    regexp_replace, trim, upper, lower,  
    round as spark_round, abs as spark_abs,  
    log1p, sqrt,  
    sum as spark_sum, avg, count, countDistinct,  
    lag, lead, rank, dense_rank,  
    window  
)
```

24.1.2. 1.2 Limpieza de datos a escala

```
# Filtrar viajes imposibles (distancia negativa, tarifa negativa, etc.)  
df_clean = (df  
    .filter(col("trip_distance") > 0)  
    .filter(col("fare_amount") > 0)  
    .filter(col("total_amount") > 0)  
    .filter(col("passenger_count").between(1, 6))  
    # Eliminar outliers extremos (99.9 percentil 120 USD total)  
    .filter(col("total_amount") < 500)  
)  
  
# Imputar nulos con valor por defecto  
df_clean = df_clean.fillna({  
    "passenger_count": 1,  
    "trip_distance": 0.0,  
})
```

```
# Crear columnas derivadas
df_feat = (df_clean
    .withColumn("hora_recogida", hour("tpep_pickup_datetime"))
    .withColumn("dia_semana", dayofweek("tpep_pickup_datetime"))
    .withColumn("mes", month("tpep_pickup_datetime"))
    .withColumn("duracion_min",
        (col("tpep_dropoff_datetime").cast("long") -
         col("tpep_pickup_datetime").cast("long")) / 60)
    .withColumn("tarifa_por_km",
        when(col("trip_distance") > 0,
            spark_round(col("fare_amount") / col("trip_distance"), 2))
        .otherwise(None))
    .withColumn("propina_pct",
        spark_round(col("tip_amount") / col("fare_amount") * 100, 1))
)
```

24.1.3. 1.3 Agregaciones y *Window Functions*

```
from pyspark.sql import Window

# Agregación simple: ingresos por hora del día
ingresos_hora = (df_feat
    .groupBy("hora_recogida")
    .agg(
        count("*").alias("n_viajes"),
        avg("total_amount").alias("ticket_promedio"),
        spark_sum("total_amount").alias("ingreso_total")
    )
    .orderBy("hora_recogida")
)
ingresos_hora.show(24)

# Window function: ranking de zonas por ingreso diario
wspec = Window.partitionBy("mes", "dia_semana").orderBy(col("ingreso_total").desc())

ranking = (df_feat
    .groupBy("mes", "dia_semana", "PULocationID")
    .agg(spark_sum("total_amount").alias("ingreso_total"))
    .withColumn("rank", dense_rank().over(wspec))
    .filter(col("rank") <= 5)
    .orderBy("mes", "dia_semana", "rank")
)
ranking.show(20)
```

24.1.4. 1.4 Joins a escala

```
# Cargar tabla de zonas (pequeña - cabe en memoria → broadcast join)
from pyspark.sql.functions import broadcast

zonas = spark.read.csv("data/taxi_zone_lookup.csv",
                      header=True, inferSchema=True)

# Broadcast join: Spark envía la tabla pequeña a todos los executors
df_con_zona = df_feat.join(
    broadcast(zonas),
    df_feat.PULocationID == zonas.LocationID,
    "left"
)

# Para tablas grandes: sort-merge join (default automático en Spark)
df_join_grande = df_feat.join(otra_tabla_grande, on="clave", how="inner")
```

Advertencia

Evitar el Cartesian Product. Un join sin condición sobre DataFrames grandes puede generar $N \times M$ filas y agotar la memoria del clúster. Spark lanzará un error si intentas un cross join sin `.crossJoin()` explícito.

24.1.5. 1.5 Ejercicio E-2 — Pipeline de limpieza completo

Importante

Ejercicio E-2: Transformación y calidad
Usando el Parquet del Ejercicio E-1:

```

from pyspark.sql.functions import *

# TODO: Completa cada paso
# 1. Elimina filas con nulos en pickup_datetime o dropoff_datetime
df_sin_nulos = df.dropna(subset=["tpep_pickup_datetime", "tpep_dropoff_datetime"])

# 2. Crea la columna `duracion_min` (diferencia en minutos)
#     y filtra viajes con duración entre 1 y 180 minutos
df_duracion = ... # ← completa

# 3. Añade columnas: hora_recogida, dia_semana, es_fin_de_semana (boolean)
df_enriquecido = ... # ← completa

# 4. Calcula la mediana de total_amount por hora del día
#     (usa approxQuantile sobre el grupo - los percentiles exactos son costosos)
mediana_por_hora = ... # ← completa

# 5. ¿Qué porcentaje de viajes tienen propina = 0?
#     ¿Difiere este porcentaje por día de la semana?
pct_sin_propina = ... # ← completa

# 6. Escribe el resultado limpio como Parquet particionado por mes y día
df_enriquecido.write.mode("overwrite")\
    .partitionBy("mes", "dia_semana")\
    .parquet("output/taxi_limpio/")

```

Preguntas de análisis: a) ¿Cuántas filas se perdieron en cada paso de limpieza? b) ¿En qué hora del día el ticket promedio es más alto? c) ¿Los viernes tienen más propinas que los lunes?

24.2. 2. Transformaciones SQL con Spark

Spark SQL permite usar sentencias SQL estándar sobre DataFrames:

```

# Registrar el DataFrame como vista temporal
df_feat.createOrReplaceTempView("viajes")

# Consulta SQL - idéntica a lo que harías en BigQuery o Snowflake
resultado = spark.sql("""
    SELECT
        hora_recogida,
        dia_semana,
        COUNT(*)                AS n_viajes,
        ROUND(AVG(total_amount), 2) AS ticket_promedio,
        ROUND(AVG(propina_pct), 1)  AS propina_pct_promedio,
        ROUND(SUM(total_amount), 0) AS ingreso_total

```

```
FROM viajes
WHERE total_amount BETWEEN 2.5 AND 200
GROUP BY hora_recogida, dia_semana
ORDER BY hora_recogida, dia_semana
""")
resultado.show(48)
```

Tip

Si ya conoces SQL, puedes empezar con Spark SQL y aprender la API DataFrame gradualmente. Ambas generan exactamente el mismo plan de ejecución optimizado.

24.3. 3. Carga (*Load*)

24.3.1. 3.1 Escritura en Parquet (formato estándar del lakehouse)

```
# Escritura simple
df_feat.write.mode("overwrite").parquet("output/viajes_limpios/")

# Particionado - clave para consultas eficientes (pruning de particiones)
df_feat.write\
    .mode("overwrite")\
    .partitionBy("mes", "dia_semana")\
    .parquet("output/viajes_por_fecha/")

# Controlar el número de archivos de salida
df_feat.coalesce(8).write.mode("overwrite").parquet("output/viajes_8archivos/")
```

24.3.2. 3.2 Delta Lake — el formato lakehouse con ACID

```
# pip install delta-spark
from delta import configure_spark_with_delta_pip, DeltaTable

spark = configure_spark_with_delta_pip(
    SparkSession.builder.appName("ETL-Delta")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
).getOrCreate()

# Escritura inicial
df_feat.write.format("delta").mode("overwrite").save("output/delta/viajes/")
```

```
# Upsert (MERGE) - fundamental para cargas incrementales
delta_table = DeltaTable.forPath(spark, "output/delta/viajes/")

nuevos = spark.read.parquet("data/yellow_tripdata_2023-02.parquet")

(delta_table.alias("viejo")
 .merge(nuevos.alias("nuevo"),
        "viejo.VendorID = nuevo.VendorID AND "
        "viejo.tpep_pickup_datetime = nuevo.tpep_pickup_datetime")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())

# Time travel - leer versión anterior de la tabla
df_ayer = spark.read.format("delta").option("versionAsOf", 0)\
    .load("output/delta/viajes/")
```

24.3.3. 3.3 Escritura a una base de datos (JDBC)

```
(df_feat.write
 .format("jdbc")
 .option("url",      "jdbc:postgresql://dw-host:5432/datawarehouse")
 .option("dbtable",  "hechos.viajes_taxi")
 .option("user",     "etl_user")
 .option("password", "secret")
 .option("batchsize", 10000)      # filas por batch INSERT
 .option("numPartitions", 8)      # conexiones paralelas
 .mode("append")
 .save())
```

24.4. 4. Pipeline ETL completo

Ahora unimos Extract + Transform + Load en una función reutilizable:

```
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import (
    col, hour, dayofweek, month, when,
    spark_round as _round
)
import logging

logging.basicConfig(level=logging.INFO)
log = logging.getLogger("ETL-Taxi")
```



```

def extract(spark: SparkSession, path: str) -> DataFrame:
    log.info(f"Leyendo {path}")
    return spark.read.parquet(path)

def transform(df: DataFrame) -> DataFrame:
    log.info("Transformando...")
    return (df
        .filter(col("trip_distance") > 0)
        .filter(col("fare_amount").between(2.5, 500))
        .filter(col("passenger_count").between(1, 6))
        .fillna({"passenger_count": 1})
        .withColumn("hora", hour("tpep_pickup_datetime"))
        .withColumn("dia", dayofweek("tpep_pickup_datetime"))
        .withColumn("mes", month("tpep_pickup_datetime"))
        .withColumn("duracion",
            (col("tpep_dropoff_datetime").cast("long") -
             col("tpep_pickup_datetime").cast("long")) / 60)
        .filter(col("duracion").between(1, 180))
        .withColumn("propina_pct",
            when(col("fare_amount") > 0,
                _round(col("tip_amount")/col("fare_amount")*100, 1))
            .otherwise(0.0))
    )

def load(df: DataFrame, output_path: str) -> None:
    log.info(f"Escribiendo en {output_path}")
    (df.write
        .mode("overwrite")
        .partitionBy("mes", "dia")
        .parquet(output_path))
    log.info(";ETL completo!")

if __name__ == "__main__":
    spark = (SparkSession.builder
        .appName("ETL-NYC-Taxi")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "16")
        .getOrCreate())

    raw = extract(spark, "data/yellow_tripdata_2023-01.parquet")
    clean = transform(raw)
    load(clean, "output/taxi_etl/")

    spark.stop()

```

24.5. Referencias de este capítulo

- Delta Lake Documentation
- PySpark SQL Functions

25. Prácticas ETL en Databricks

Este capítulo contiene los ejercicios de aplicación. Todos se ejecutan en **Databricks Community Edition** usando el clúster configurado en el primer capítulo del bloque.



Tip

Workflow sugerido por ejercicio: 1. Crear un notebook nuevo en Databricks con el nombre del ejercicio. 2. En la primera celda: **# Ejercicio E-X - Título** 3. Ejecutar celda por celda, observar el Spark UI para ver el plan de ejecución. 4. Adjuntar capturas del resultado como evidencia.

25.1. 1. Ejercicios

25.1.1. 1.1 Ejercicio E-3 — Pipeline incremental con Delta Lake

Importante

Ejercicio E-3: Carga incremental

Simula un pipeline de producción que carga datos mes a mes sin duplicar registros.

```

from pyspark.sql import SparkSession
from delta import DeltaTable, configure_spark_with_delta_pip

spark = configure_spark_with_delta_pip(
    SparkSession.builder.appName("E3-Incremental")
    .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .master("local[*]")
).getOrCreate()

DELTA_PATH = "output/delta_taxi/"
MESES = [
    "data/yellow_tripdata_2023-01.parquet",
    "data/yellow_tripdata_2023-02.parquet",
    "data/yellow_tripdata_2023-03.parquet",
]

def transform(df):
    from pyspark.sql.functions import col, hour, dayofweek, month
    return (df
        .filter(col("trip_distance") > 0)
        .filter(col("fare_amount").between(2.5, 500))
        .withColumn("hora", hour("tpep_pickup_datetime"))
        .withColumn("mes", month("tpep_pickup_datetime"))
    )

for path in MESES:
    nuevos = transform(spark.read.parquet(path))

    if DeltaTable.isDeltaTable(spark, DELTA_PATH):
        # UPSERT: insertar solo lo que no existe
        dt = DeltaTable.forPath(spark, DELTA_PATH)
        (dt.alias("viejo")
         .merge(nuevos.alias("nuevo"),
                "viejo.VendorID = nuevo.VendorID AND "
                "viejo.tpep_pickup_datetime = nuevo.tpep_pickup_datetime")
         .whenNotMatchedInsertAll()
         .execute())
    else:
        # Primera carga
        nuevos.write.format("delta").mode("overwrite").save(DELTA_PATH)

    print(f" {path.split('/')[-1]} cargado. Total filas: "
          f"{spark.read.format('delta').load(DELTA_PATH).count():,}")

spark.stop()

```

Preguntas: a) ¿Qué pasa si cargas el mismo mes dos veces? ¿Por qué? b) Modifica el MERGE para que si un viaje ya existe pero tiene distinto `total_amount`, actualice el registro (simula una corrección tardía). c) Usa `DESCRIBE HISTORY delta.\output/delta_taxi/` para ver el log de transacciones.

25.1.2. 1.2 Ejercicio E-4 — ETL multi-fuente con join a escala

Importante

Ejercicio E-4: Enriquecimiento con join

Descarga el catálogo de zonas y úsalo para enriquecer los viajes con el nombre del barrio.

```
# Descargar catálogo de zonas (26 KB)
wget https://d37ci6vzurychx.cloudfront.net/misc/taxi+_zone_lookup.csv \
-O data/taxi_zone_lookup.csv
```

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast, col, count, avg, desc

spark = SparkSession.builder.appName("E4-Join").master("local[*]").getOrCreate()

viajes = spark.read.parquet("data/yellow_tripdata_2023-01.parquet")
zonas = spark.read.csv("data/taxi_zone_lookup.csv",
                      header=True, inferSchema=True)

# TODO 1: Renombra columnas de zonas para evitar ambigüedad
zonas_pu = zonas.select(
    col("LocationID").alias("pu_location_id"),
    col("Zone").alias("pu_zone"),
    col("Borough").alias("pu_borough")
)

# TODO 2: Haz un broadcast join de viajes con zonas_pu
#         usando PULocationID == pu_location_id
df_enriquecido = ... # ← completa

# TODO 3: Calcula el top-10 de zonas de recogida por ingreso total
top_zonas = (df_enriquecido
    .groupBy("pu_zone", "pu_borough")
    .agg(
        count("*").alias("n_viajes"),
        avg("total_amount").alias("ticket_prom")
    )
    .orderBy(desc("n_viajes"))
    .limit(10)
)
top_zonas.show(truncate=False)

# TODO 4: ¿Qué borough tiene la propina promedio más alta?
#         Añade la columna propina_pct y agrupa por pu_borough
propina_borough = ... # ← completa

spark.stop()

```

Preguntas: a) ¿Por qué usamos `broadcast` para el join con zonas? b) ¿Qué ocurriría si quitamos el `broadcast` y el DataFrame de zonas fuera de 10 GB en vez de 26 KB? c) Extiende el enriquecimiento para incluir también la zona de destino (`DOLocationID`).

25.1.3. 1.3 Ejercicio E-5 — Detección de calidad con Great Expectations

Importante

Ejercicio E-5: Validación automatizada

```
pip install great_expectations
```

```
import great_expectations as gx
from great_expectations.dataset import SparkDFDataset

spark = SparkSession.builder.appName("E5-GX").master("local[*]").getOrCreate()
df = spark.read.parquet("data/yellow_tripdata_2023-01.parquet")

# Convertir a Great Expectations dataset
gx_df = SparkDFDataset(df)

# Definir expectativas de calidad
resultados = []
resultados.append(gx_df.expect_column_values_to_not_be_null("tpep_pickup_datetime"))
resultados.append(gx_df.expect_column_values_to_be_between(
    "trip_distance", min_value=0, max_value=500))
resultados.append(gx_df.expect_column_values_to_be_between(
    "fare_amount", min_value=0, max_value=1000))
resultados.append(gx_df.expect_column_values_to_be_in_set(
    "payment_type", [1, 2, 3, 4, 5, 6]))
resultados.append(gx_df.expect_column_values_to_not_be_null("VendorID"))

# Reporte
for r in resultados:
```

25.1.4. 1.4 Ejercicio E-6 — Proyecto integrador TPC-H

Importante

Ejercicio E-6: ETL sobre esquema estrella

Genera datos sintéticos con TPC-H (escala 1 GB) y construye un DW en Parquet.

```
pip install tpch # o clonar https://github.com/electrum/tpch-dbggen
python -m tpch generate --scale 1 # 1 GB de datos relacionales
```

```
# Esquema TPC-H (simplificado):
# ORDERS (pedido_id, cliente_id, fecha, prioridad, total)
# LINEITEM (pedido_id, producto_id, cantidad, precio, descuento)
# CUSTOMER (cliente_id, nombre, segmento, nacion_id)
# NATION (nacion_id, nombre, region_id)

spark = SparkSession.builder.appName("E6-TPCH").master("local[*]").getOrCreate()

orders = spark.read.csv("tpch/orders.tbl", sep="|", inferSchema=True)
lineitem = spark.read.csv("tpch/lineitem.tbl", sep="|", inferSchema=True)
customer = spark.read.csv("tpch/customer.tbl", sep="|", inferSchema=True)
nation = spark.read.csv("tpch/nation.tbl", sep="|", inferSchema=True)

# TODO 1: Renombra columnas (TPC-H usa _c0, _c1, ...)
# Consulta el diccionario en: http://www.tpc.org/tpc_documents_current_versions/pdf/tpc

# TODO 2: Construye la tabla de hechos `hechos_ventas` con estas columnas:
# pedido_id, cliente_id, producto_id, nacion, segmento,
# fecha_pedido, cantidad, ingreso_netto (precio*cantidad*(1-descuento))

# TODO 3: Escribe hechos_ventas como Parquet particionado por año y mes

# TODO 4: Responde con Spark SQL:
# Q1: Ingreso total por segmento de cliente y año
# Q2: Top-5 naciones por ingreso en el último trimestre disponible
# Q3: ¿Qué porcentaje de pedidos son de prioridad "URGENT"?

spark.stop()
```

Este ejercicio simula exactamente un pipeline ETL empresarial: extracción de tablas OLTP
→ joins dimensionales → tabla de hechos → Parquet.

25.2. Referencias de este capítulo

- NYC TLC Trip Record Data
- Great Expectations
- TPC-H Benchmark
- Delta Lake Documentation

26. Orquestación y Herramientas Modernas

Una vez dominado PySpark, el siguiente paso es automatizar los pipelines y adoptar las herramientas que usa la industria para transformaciones declarativas y scheduling de producción.

26.1. 1. ETL moderno: dbt + PySpark

dbt (*data build tool*) separa la *transformación* del pipeline técnico: defines las transformaciones como modelos SQL, y dbt se encarga de ejecutarlos en el orden correcto, con pruebas automáticas y documentación.

```
pip install dbt-spark
dbt init mi_proyecto_etl
```

```
-- models/staging/stg_viajes.sql
-- dbt materializa esto como una tabla o vista en Spark/DW
SELECT
    VendorID                AS vendor_id,
    tpep_pickup_datetime    AS pickup_at,
    trip_distance,
    fare_amount,
    tip_amount,
    total_amount,
    payment_type
FROM {{ source('raw', 'yellow_taxi') }}
WHERE trip_distance > 0
    AND fare_amount > 2.5
```

```
-- models/marts/fct_viajes_diarios.sql
SELECT
    DATE(pickup_at)          AS fecha,
    HOUR(pickup_at)          AS hora,
    COUNT(*)                 AS n_viajes,
    AVG(total_amount)        AS ticket_prom,
    SUM(total_amount)        AS ingreso_total
FROM {{ ref('stg_viajes') }}
GROUP BY 1, 2
```

```
# models/schema.yml - pruebas automáticas de calidad
models:
  - name: stg_viajes
    columns:
```

```

- name: vendor_id
  tests: [not_null]
- name: total_amount
  tests:
    - not_null
    - dbt_utils.accepted_range:
        min_value: 0
        max_value: 500

```

```

dbt run      # ejecuta los modelos
dbt test     # corre las pruebas de calidad
dbt docs generate && dbt docs serve  # abre documentación interactiva

```

26.2. 2. Orquestación con Apache Airflow

En producción, los pipelines ETL se ejecutan automáticamente según un calendario. **Airflow** define estos flujos como DAGs de Python:

```

# dags/etl_taxi_mensual.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

def extraer(**ctx):
    mes = ctx['logical_date'].strftime("%Y-%m")
    print(f"Extrayendo mes {mes}...")
    # aquí va el código de extracción

def transformar(**ctx): ...
def cargar(**ctx): ...
def validar(**ctx): ...

with DAG(
    dag_id="etl_nyc_taxi_mensual",
    start_date=datetime(2023, 1, 1),
    schedule_interval="0 3 1 * *", # el día 1 de cada mes a las 3 AM
    catchup=False,
    default_args={"retries": 2, "retry_delay": timedelta(minutes=5)},
) as dag:

    t_extract = PythonOperator(task_id="extraer", python_callable=extraer)
    t_transform = PythonOperator(task_id="transformar", python_callable=transformar)
    t_load = PythonOperator(task_id="cargar", python_callable=cargar)
    t_validate = PythonOperator(task_id="validar", python_callable=validar)

    t_extract >> t_transform >> t_load >> t_validate

```

 Tip

Alternativas a Airflow: - **Prefect** — más Python-nativo, sin XML/YAML, fácil para equipos pequeños. - **Dagster** — fuerte en observabilidad y gestión de assets de datos. - Para empezar: **Prefect + PySpark local** → luego migrar a Airflow en producción.

26.3. 3. Resumen y mapa de herramientas

Etapas	Herramienta	Cuándo usarla
Extract	PySpark	Archivos grandes (>1 GB), clústeres, Parquet/JDBC
Extract	Kafka + Debezium	Streaming en tiempo real, CDC
Transform	Spark SQL / DataFrame API	Joins complejos, window functions, ML features
Transform	dbt	Transformaciones SQL versionadas sobre DW cloud
Quality	Great Expectations	Validación declarativa antes de la carga
Load	Delta Lake / Iceberg	ACID, upsert, time travel en el lakehouse
Load	BigQuery / Snowflake	DW cloud sin gestión de infraestructura
Load	DuckDB	Notebooks, sin clúster, prototipos rápidos
Orquestar	Airflow	Pipelines de producción complejos con SLA
Orquestar	Prefect	Equipos pequeños, Python-native, simplicidad

26.4. 4. Referencias y recursos adicionales

- Chambers & Zaharia. *Spark: The Definitive Guide*. O'Reilly, 2018.
- PySpark Documentation — API reference completa.
- Delta Lake Documentation — ACID y time travel.
- dbt Documentation — transformaciones SQL versionadas.
- Great Expectations — validación de calidad.
- NYC TLC Trip Record Data — dataset de ejercicios.
- TPC-H Benchmark — esquema de referencia para DW.

Parte VIII.

Modelado para consumo

27. Computación de cubos de datos OLAP

Un enfoque técnico y reproducible

Los cubos de datos son una forma de organizar información multidimensional para análisis resumido, exploración interactiva y toma de decisiones. En una arquitectura típica de *data warehouse*, los datos se extraen de sistemas transaccionales, pasan por procesos de limpieza e integración, se almacenan en un repositorio analítico y se presentan mediante reportes, minería de datos o cubos OLAP. Esta secuencia aparece en la presentación base, donde se recuerda el flujo general desde fuentes de datos hasta almacenamiento, *data marts* y presentación analítica (Santizo Galicia 2026).

El objetivo de este capítulo es convertir esa idea en una formulación técnica y reproducible. Se presentará el concepto de procesamiento analítico en línea, la estructura matemática de un cubo de datos, sus principales operaciones y una implementación práctica en Python. La discusión se apoya en la tradición de almacenes de datos (Inmon 2005; Kimball y Ross 2013), en la literatura clásica sobre OLAP y cubos de datos (Chaudhuri y Dayal 1997; Gray et al. 1996), y en herramientas modernas de análisis tabular como **pandas** (McKinney 2010; The pandas development team 2024).

Un cubo OLAP no debe entenderse únicamente como una figura tridimensional. En realidad, es una abstracción para representar medidas agregadas sobre varias dimensiones. Por ejemplo, una empresa puede analizar ventas por tiempo, región y producto. Cada combinación específica de esas dimensiones define una celda del cubo, y esa celda contiene una medida, como ventas totales, unidades vendidas, costo promedio o margen de ganancia.

27.1. Del almacén de datos al análisis OLAP

Un sistema transaccional u OLTP (*Online Transaction Processing*) está optimizado para registrar operaciones individuales: compras, pagos, inventarios, altas de clientes o movimientos bancarios. En contraste, un sistema OLAP (*Online Analytical Processing*) está optimizado para consultar datos agregados y responder preguntas de negocio. Esta distinción es importante porque el análisis multidimensional requiere estructuras distintas a las de una base de datos operacional.

i Nota

Definición. Un sistema OLAP es una estructura analítica que organiza medidas cuantitativas mediante dimensiones categóricas o jerárquicas para permitir consultas agregadas desde distintas perspectivas.

$$C : D_1 \times D_2 \times \cdots \times D_k \longrightarrow \mathbb{R}^m$$

En esta definición, $D_1, \dots, D_k$ representan dimensiones como tiempo, región, producto o sucursal. El valor $C(d_1, \dots, d_k)$ contiene una o varias medidas agregadas. Por ejemplo, si D_1 es tiempo, D_2 es región y D_3 es

producto, entonces una celda puede representar las ventas de computadoras en la región Centro durante el trimestre T1.

La presentación original destaca que en un *data warehouse* los datos se organizan en una estructura multidimensional, que puede implementarse como un cubo OLAP y analizarse desde diferentes perspectivas (Santizo Galicia 2026). Esta idea se puede conectar con tres componentes esenciales:

1. **Dimensiones:** ejes de análisis, como tiempo, región o producto.
2. **Jerarquías:** niveles dentro de una dimensión, como día → mes → trimestre → año.
3. **Medidas:** variables numéricas agregables, como ventas, unidades, utilidad o presupuesto.

Una ventaja central de OLAP es que permite adaptar el nivel de detalle a cada usuario. Un gerente general puede consultar ventas totales por país; una dirección regional puede revisar ventas por estado; y un supervisor puede analizar ventas por tienda. La misma base de datos analítica soporta múltiples perspectivas mediante operaciones de navegación sobre el cubo.

27.2. Modelo multidimensional

Supongamos un conjunto de transacciones $\mathcal{T}$ donde cada registro contiene atributos de dimensión y una medida de ventas. Una transacción puede escribirse como

$$t_i = (r_i, \tau_i, p_i, v_i),$$

donde r_i indica la región, τ_i el periodo, p_i el producto y v_i el valor de venta. El cubo agregado por suma se obtiene agrupando transacciones que comparten las mismas coordenadas dimensionales.

i Nota

Definición. Dado un conjunto de transacciones $\mathcal{T}$, el valor de una celda de un cubo de ventas se define como la suma de las ventas de todas las transacciones que coinciden con una combinación de dimensiones:

$$C(r, \tau, p) = \sum_{i: r_i=r, \tau_i=\tau, p_i=p} v_i$$

Esta formulación deja claro que un cubo es una función agregada. La representación visual tridimensional es útil para enseñar el concepto, pero en la práctica pueden existir muchas más dimensiones. Un cubo de ventas real puede incluir canal de venta, tipo de cliente, vendedor, tienda, campaña publicitaria y categoría fiscal.

27.2.1. Dimensiones, medidas y granularidad

La **granularidad** indica el nivel de detalle con que se almacenan o consultan los datos. Una tabla con ventas diarias por tienda y producto tiene mayor granularidad que una tabla con ventas mensuales por región y categoría. Diseñar correctamente la granularidad es una decisión crítica en un almacén de datos, porque afecta el almacenamiento, la velocidad de consulta y la flexibilidad analítica (Kimball y Ross 2013).

i Nota

Definición. La granularidad de un cubo es el nivel mínimo de detalle con el que se representan sus dimensiones.

$$G(C) = \{\ell_1, \ell_2, \dots, \ell_k\}$$

Aquí ℓ_j representa el nivel usado en la dimensión D_j . Por ejemplo, $G(C) = \{\text{mes}, \text{estado}, \text{producto}\}$ indica que el cubo se consulta por mes, estado y producto. Si se cambia de mes a semana, la granularidad aumenta; si se cambia de estado a país, la granularidad disminuye.

27.3. Operaciones fundamentales en cubos OLAP

Las operaciones OLAP permiten navegar entre distintos niveles y subconjuntos del cubo. La presentación base introduce cinco operaciones principales: *drill-down*, *roll-up*, *slice*, *dice* y *pivot* o *rotate* (Santizo Galicia 2026). A continuación se formalizan y se muestran ejemplos reproducibles.

27.3.1. Drill-down

El *drill-down* permite pasar de datos resumidos a datos más detallados. Por ejemplo, si una empresa observa que en 2025 vendió 1,000 productos, puede preguntar cuánto se vendió por trimestre, por mes, por estado o por producto. En términos de jerarquías, el *drill-down* se mueve hacia niveles más específicos.

i Nota

Definición. Una operación de drill-down reemplaza un nivel agregado ℓ de una dimensión por un nivel más detallado ℓ' dentro de la misma jerarquía.

$$\ell' \prec \ell$$

Por ejemplo, en la jerarquía temporal

$$\text{año} \rightarrow \text{trimestre} \rightarrow \text{mes} \rightarrow \text{semana} \rightarrow \text{día},$$

pasar de año a trimestre o de mes a semana es una operación de *drill-down*. En la presentación, esta operación se ilustra al pasar de una vista mensual a una vista semanal para el mes de enero (Santizo Galicia 2026).

27.3.2. Roll-up

El *roll-up* es la operación inversa. Permite agrupar datos detallados para obtener resúmenes de mayor nivel. Por ejemplo, las ventas diarias pueden agregarse como ventas mensuales, trimestrales o anuales; las ventas por tienda pueden agregarse como ventas por estado o por país.

i Nota

Definición. Una operación de roll-up reemplaza un nivel detallado ℓ' de una dimensión por un nivel más agregado ℓ y combina las celdas mediante una función de agregación A .

$$C_\ell(d) = A(\{C_{\ell'}(d') : d' \mapsto d\})$$

La función A puede ser una suma, un promedio, un máximo, un conteo o una medida derivada. En ventas suele usarse la suma; en precios puede ser más adecuado usar promedio ponderado; en inventarios puede usarse el valor final de periodo.

27.3.3. Slice

La operación *slice* fija un valor específico en una dimensión y devuelve un subcubo con una dimensión menos. Si el cubo original tiene dimensiones Región, Tiempo y Producto, fijar Producto = Tablets produce una vista Región $\times$ Tiempo.

i Nota

Definición. Dado un cubo $C : D_1 \times \dots \times D_k \rightarrow \mathbb{R}$, una operación slice fija una dimensión D_j en un valor $a \in D_j$.

$$C_{D_j=a}(d_1, \dots, d_{j-1}, d_{j+1}, \dots, d_k) = C(d_1, \dots, a, \dots, d_k)$$

La presentación ejemplifica esta operación al seleccionar un solo estado, un solo producto o un solo mes (Santizo Galicia 2026).

27.3.4. Dice

La operación *dice* selecciona subconjuntos en dos o más dimensiones al mismo tiempo. A diferencia de *slice*, que fija un único valor en una dimensión, *dice* puede conservar varios valores por dimensión.

i Nota

Definición. Una operación dice restringe el cubo a subconjuntos $S_j \subseteq D_j$ en una o varias dimensiones.

$$C_{S_1, \dots, S_k} = C|_{S_1 \times \dots \times S_k}$$

Por ejemplo, se puede seleccionar Tiempo $\in \{\text{Enero, Febrero}\}$ y Producto $\in \{\text{Smartphones, Tablets}\}$. También se puede restringir Región $\in \{\text{Jalisco, Estado de México}\}$ y Tiempo $\in \{\text{Enero, Febrero}\}$.

27.3.5. Pivot o rotate

La operación *pivot* cambia la orientación de la vista sin alterar los datos subyacentes. En una tabla, esto equivale a intercambiar filas y columnas o a reordenar dimensiones para responder otra pregunta. Por ejemplo, una tabla con semanas en filas y productos en columnas responde: “¿cuánto se vendió de cada producto en cada semana?”. Al pivotar, una tabla con productos en filas y semanas en columnas responde: “¿cómo se comportó cada producto a lo largo de las semanas?” (Santizo Galicia 2026).

i Nota

Definición. Una operación pivot aplica una permutación π al orden de las dimensiones de un cubo.

$$C^\pi(d_{\pi(1)}, \dots, d_{\pi(k)}) = C(d_1, \dots, d_k)$$

Esta operación es especialmente útil para visualización. No cambia la granularidad ni los valores agregados; cambia la forma en que se presentan los ejes.

27.4. Cálculo manual de un cubo OLAP en un dataset pequeño

Antes de implementar operaciones OLAP con Python, conviene resolver un ejemplo pequeño a mano. El objetivo didáctico es que el estudiante pueda reconocer las dimensiones, identificar las celdas del cubo, calcular totales y ejecutar operaciones básicas como *slice* y *roll-up* sin depender todavía de una biblioteca computacional.

Consideremos una empresa con ventas trimestrales por **tiempo**, **ubicación** y **departamento**. Los departamentos son

$$(M, P, D, E) = (\text{Madera}, \text{Plomería}, \text{Decoración}, \text{Eléctrico}).$$

La medida del cubo será el valor de ventas. Las dimensiones son:

- $D_1 = \text{Ubicación} = \{\text{CDMX}, \text{Guadalajara}, \text{Monterrey}, \text{Mérida}\}.$
- $D_2 = \text{Tiempo} = \{T1, T2\}.$
- $D_3 = \text{Departamento} = \{M, P, D, E\}.$

Por tanto, el cubo puede representarse como una función

$$C : \text{Ubicación} \times \text{Tiempo} \times \text{Departamento} \longrightarrow \mathbb{R}.$$

La siguiente tabla contiene los datos originales.

Tiempo	CDMX (M, P, D, E)	Guadalajara (M, P, D, E)
T1	(70, 40, 20, 30)	(60, 35, 25, 40)
T2	(50, 60, 22, 28)	(45, 50, 30, 35)

Tiempo	Monterrey (M, P, D, E)	Mérida (M, P, D, E)
T1	(48, 38, 18, 26)	(52, 42, 15, 37)
T2	(36, 55, 25, 30)	(44, 33, 20, 28)

27.4.1. Paso 1: representar las celdas del cubo

Cada número de la tabla corresponde a una celda del cubo. Por ejemplo,

$$C(\text{CDMX}, T1, \text{Madera}) = 70,$$

$$C(\text{CDMX}, T1, \text{Eléctrico}) = 30,$$

$$C(\text{Mérida}, T2, \text{Plomería}) = 33.$$

En este caso hay

$$4 \text{ ubicaciones} \times 2 \text{ trimestres} \times 4 \text{ departamentos} = 32$$

celdas elementales. La representación del cubo consiste en organizar esas 32 celdas de forma que puedan consultarse por cualquiera de sus dimensiones.

27.4.2. Paso 2: calcular totales por ubicación

El total por ubicación se obtiene sumando todos los trimestres y todos los departamentos de una misma ubicación:

$$\text{Total}(u) = \sum_{t \in \{T1, T2\}} \sum_{d \in \{M, P, D, E\}} C(u, t, d).$$

Para CDMX:

$$\begin{aligned} \text{Total}(\text{CDMX}) &= (70 + 40 + 20 + 30) + (50 + 60 + 22 + 28) \\ &= 160 + 160 \\ &= 320. \end{aligned}$$

Para Guadalajara:

$$\begin{aligned} \text{Total}(\text{Guadalajara}) &= (60 + 35 + 25 + 40) + (45 + 50 + 30 + 35) \\ &= 160 + 160 \\ &= 320. \end{aligned}$$

Para Monterrey:

$$\begin{aligned} \text{Total}(\text{Monterrey}) &= (48 + 38 + 18 + 26) + (36 + 55 + 25 + 30) \\ &= 130 + 146 \\ &= 276. \end{aligned}$$

Para Mérida:

$$\begin{aligned}
\text{Total(Mérida)} &= (52 + 42 + 15 + 37) + (44 + 33 + 20 + 28) \\
&= 146 + 125 \\
&= 271.
\end{aligned}$$

La tabla de totales por ubicación queda:

Ubicación	Total
CDMX	320
Guadalajara	320
Monterrey	276
Mérida	271
Total general	1187

27.4.3. Paso 3: calcular totales por tiempo

El total por tiempo se obtiene sumando todas las ubicaciones y departamentos para cada trimestre:

$$\text{Total}(t) = \sum_u \sum_d C(u, t, d).$$

Para $T1$:

$$\begin{aligned}
\text{Total}(T1) &= (160) + (160) + (130) + (146) \\
&= 596.
\end{aligned}$$

Para $T2$:

$$\begin{aligned}
\text{Total}(T2) &= (160) + (160) + (146) + (125) \\
&= 591.
\end{aligned}$$

Tiempo	Total
T1	596
T2	591
Total general	1187

27.4.4. Paso 4: calcular totales por departamento

Ahora se suman todas las ubicaciones y tiempos para cada departamento:

$$\text{Total}(d) = \sum_u \sum_t C(u, t, d).$$

Para Madera:

$$70 + 50 + 60 + 45 + 48 + 36 + 52 + 44 = 405.$$

Para Plomería:

$$40 + 60 + 35 + 50 + 38 + 55 + 42 + 33 = 353.$$

Para Decoración:

$$20 + 22 + 25 + 30 + 18 + 25 + 15 + 20 = 175.$$

Para Eléctrico:

$$30 + 28 + 40 + 35 + 26 + 30 + 37 + 28 = 254.$$

Departamento	Total
Madera	405
Plomería	353
Decoración	175
Eléctrico	254
Total general	1187

27.4.5. Paso 5: operación slice a mano

Supongamos que se pide aplicar un *slice* con las siguientes condiciones:

$$\text{ubicación} \in \{\text{CDMX}, \text{Mérida}\}, \quad \text{tiempo} = T1,$$

$$\text{departamento} \in \{\text{Madera}, \text{Eléctrico}\}.$$

Aunque en sentido estricto un *slice* fija una dimensión en un valor, en ejercicios introductorios suele combinarse con una selección de subconjuntos. En una terminología más precisa, fijar $T1$ es un *slice*, mientras que seleccionar varias ubicaciones y varios departamentos es una restricción tipo *dice* sobre el subcubo resultante.

Se buscan únicamente las celdas:

$$C(\text{CDMX}, T1, \text{Madera}) = 70,$$

$$C(\text{CDMX}, T1, \text{Eléctrico}) = 30,$$

$$C(\text{Mérida}, T1, \text{Madera}) = 52,$$

$$C(\text{Mérida}, T1, \text{Eléctrico}) = 37.$$

Por tanto, el resultado del subcubo es:

Ubicación	Tiempo	Madera	Eléctrico	Total
CDMX	T1	70	30	100
Mérida	T1	52	37	89
Total		122	67	189

27.4.6. Paso 6: operación roll-up a mano

Ahora agrupemos las ubicaciones en regiones. Una posible correspondencia es:

Ubicación	Región
Monterrey	Norte
CDMX	Centro
Guadalajara	Occidente
Mérida	Sureste

El *roll-up* por región se calcula reemplazando la dimensión ubicación por una dimensión más agregada llamada región:

$$C_{\text{región}}(r, t, d) = \sum_{u: g(u)=r} C(u, t, d),$$

donde $g(u)$ asigna cada ubicación a su región. In this example, each region has only one location, so the numerical values are the same as the original values, but the conceptual operation is important: the cube changes from the level *ubicación* to the level *región*.

Región	Tiempo	Madera	Plomería	Decoración	Eléctrico	Total
Centro	T1	70	40	20	30	160
Centro	T2	50	60	22	28	160
Norte	T1	48	38	18	26	130
Norte	T2	36	55	25	30	146
Occidente	T1	60	35	25	40	160
Occidente	T2	45	50	30	35	160
Sureste	T1	52	42	15	37	146
Sureste	T2	44	33	20	28	125
Total		405	353	175	254	1187

Si se desea un *roll-up* regional más agregado, también se pueden definir zonas. Por ejemplo,

$$\text{Zona Norte-Centro} = \{\text{Monterrey, CDMX}\},$$

$$\text{Zona Occidente-Sureste} = \{\text{Guadalajara, Mérida}\}.$$

Entonces, para la Zona Norte-Centro en $T1$:

$$C(\text{Zona Norte-Centro}, T1, M) = 48 + 70 = 118,$$

$$C(\text{Zona Norte-Centro}, T1, P) = 38 + 40 = 78,$$

$$C(\text{Zona Norte-Centro}, T1, D) = 18 + 20 = 38,$$

$$C(\text{Zona Norte-Centro}, T1, E) = 26 + 30 = 56.$$

El resultado completo por zonas es:

Zona	Tiempo	Madera	Plomería	Decoración	Eléctrico	Total
Norte-Centro	T1	118	78	38	56	290
Norte-Centro	T2	86	115	47	58	306
Occidente-Sureste	T1	112	77	40	77	306
Occidente-Sureste	T2	89	83	50	63	285
Total		405	353	175	254	1187

Este ejemplo muestra que calcular un cubo a mano consiste en tres acciones básicas: identificar coordenadas dimensionales, sumar celdas según la agregación solicitada y conservar claramente el nivel de granularidad usado en cada resultado.

27.5. Ejemplo reproducible en Python

La presentación propone un ejercicio con ventas trimestrales de una empresa mexicana de electrónica. Las dimensiones son Región, Tiempo y Producto, y la medida son ventas expresadas en cientos de miles de pesos mexicanos (Santizo Galicia 2026). A continuación se construye el cubo de datos en Python.

```
import pandas as pd

records = [
    # Norte
    ("Norte", "T1", "Televisores", 35), ("Norte", "T1", "Computadoras", 22),
    ("Norte", "T1", "Lavadoras", 10), ("Norte", "T1", "Cámaras", 14),
    ("Norte", "T2", "Televisores", 28), ("Norte", "T2", "Computadoras", 26),
    ("Norte", "T2", "Lavadoras", 13), ("Norte", "T2", "Cámaras", 17),
    ("Norte", "T3", "Televisores", 18), ("Norte", "T3", "Computadoras", 15),
    ("Norte", "T3", "Lavadoras", 8), ("Norte", "T3", "Cámaras", 12),
    ("Norte", "T4", "Televisores", 30), ("Norte", "T4", "Computadoras", 33),
    ("Norte", "T4", "Lavadoras", 16), ("Norte", "T4", "Cámaras", 15),

    # Centro
    ("Centro", "T1", "Televisores", 40), ("Centro", "T1", "Computadoras", 30),
    ("Centro", "T1", "Lavadoras", 18), ("Centro", "T1", "Cámaras", 11),
    ("Centro", "T2", "Televisores", 32), ("Centro", "T2", "Computadoras", 28),
```

```

("Centro", "T2", "Lavadoras", 20), ("Centro", "T2", "Cámaras", 14),
("Centro", "T3", "Televisores", 25), ("Centro", "T3", "Computadoras", 19),
("Centro", "T3", "Lavadoras", 15), ("Centro", "T3", "Cámaras", 10),
("Centro", "T4", "Televisores", 38), ("Centro", "T4", "Computadoras", 35),
("Centro", "T4", "Lavadoras", 22), ("Centro", "T4", "Cámaras", 16),

# Occidente
("Occidente", "T1", "Televisores", 20), ("Occidente", "T1", "Computadoras", 27),
("Occidente", "T1", "Lavadoras", 9), ("Occidente", "T1", "Cámaras", 6),
("Occidente", "T2", "Televisores", 24), ("Occidente", "T2", "Computadoras", 31),
("Occidente", "T2", "Lavadoras", 11), ("Occidente", "T2", "Cámaras", 8),
("Occidente", "T3", "Televisores", 29), ("Occidente", "T3", "Computadoras", 36),
("Occidente", "T3", "Lavadoras", 7), ("Occidente", "T3", "Cámaras", 9),
("Occidente", "T4", "Televisores", 19), ("Occidente", "T4", "Computadoras", 34),
("Occidente", "T4", "Lavadoras", 10), ("Occidente", "T4", "Cámaras", 5),

# Sureste
("Sureste", "T1", "Televisores", 26), ("Sureste", "T1", "Computadoras", 17),
("Sureste", "T1", "Lavadoras", 12), ("Sureste", "T1", "Cámaras", 9),
("Sureste", "T2", "Televisores", 21), ("Sureste", "T2", "Computadoras", 20),
("Sureste", "T2", "Lavadoras", 16), ("Sureste", "T2", "Cámaras", 11),
("Sureste", "T3", "Televisores", 23), ("Sureste", "T3", "Computadoras", 18),
("Sureste", "T3", "Lavadoras", 14), ("Sureste", "T3", "Cámaras", 10),
("Sureste", "T4", "Televisores", 27), ("Sureste", "T4", "Computadoras", 24),
("Sureste", "T4", "Lavadoras", 19), ("Sureste", "T4", "Cámaras", 12),
]

df = pd.DataFrame(records, columns=["region", "tiempo", "producto", "ventas"])
df.head()

```

	region	tiempo	producto	ventas
0	Norte	T1	Televisores	35
1	Norte	T1	Computadoras	22
2	Norte	T1	Lavadoras	10
3	Norte	T1	Cámaras	14
4	Norte	T2	Televisores	28

27.5.1. Representación como cubo

En *pandas*, una forma natural de representar el cubo es usar un índice múltiple. Cada celda queda identificada por una tupla (region, tiempo, producto).

```

cubo = df.set_index(["region", "tiempo", "producto"]).sort_index()
cubo.head(10)

```

region	tiempo	producto	ventas
Centro	T1	Computadoras	30
		Cámaras	11
		Lavadoras	18
		Televisores	40
	T2	Computadoras	28
		Cámaras	14
		Lavadoras	20
		Televisores	32
	T3	Computadoras	19
		Cámaras	10

También se puede construir una vista tabular con productos como columnas. Esta vista es conveniente para reportes, aunque internamente sigue representando el mismo cubo.

```
vista = df.pivot_table(
    index=["region", "tiempo"],
    columns="producto",
    values="ventas",
    aggfunc="sum",
    fill_value=0,
)
vista
```

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Centro	T1	30	11	18	40
	T2	28	14	20	32
	T3	19	10	15	25
	T4	35	16	22	38
Norte	T1	22	14	10	35
	T2	26	17	13	28
	T3	15	12	8	18
	T4	33	15	16	30
Occidente	T1	27	6	9	20
	T2	31	8	11	24
	T3	36	9	7	29
	T4	34	5	10	19
Sureste	T1	17	9	12	26
	T2	20	11	16	21
	T3	18	10	14	23
	T4	24	12	19	27

27.6. Implementación de operaciones OLAP

27.6.1. Slice: fijar un trimestre

El ejercicio pide realizar una operación *slice* fijando Tiempo = T2. El resultado conserva las dimensiones Región y Producto.

```
slice_t2 = df[df["tiempo"] == "T2"]
slice_t2.pivot_table(
    index="region",
    columns="producto",
    values="ventas",
    aggfunc="sum",
    fill_value=0,
)
```

	Computadoras	Cámaras	Lavadoras	Televisores
region				
Centro	28	14	20	32
Norte	26	17	13	28
Occidente	31	8	11	24
Sureste	20	11	16	21

Esta salida permite comparar las ventas del segundo trimestre entre regiones y productos.

27.6.2. Dice: seleccionar regiones, tiempos y productos

El ejercicio también solicita una operación *dice* con Región = Norte o Centro, Tiempo = T1 o T4 y Producto = Televisores o Cámaras.

```
dice = df[
    df["region"].isin(["Norte", "Centro"])
    & df["tiempo"].isin(["T1", "T4"])
    & df["producto"].isin(["Televisores", "Cámaras"])
]

dice.pivot_table(
    index=["region", "tiempo"],
    columns="producto",
    values="ventas",
    aggfunc="sum",
    fill_value=0,
)
```

region	producto tiempo	Cámaras	Televisores
Centro	T1	11	40
	T4	16	38
Norte	T1	14	35
	T4	15	30

La operación reduce el cubo original a un subcubo con dos regiones, dos periodos y dos productos. El resultado es más pequeño, pero conserva la estructura multidimensional.

27.6.3. Roll-up: agrupar regiones en zonas

El ejercicio pide crear dos categorías regionales: Zona Norte-Centro y Zona Occidente-Sureste. Esto es un *roll-up* sobre la dimensión Región.

```
mapa_zonas = {
    "Norte": "Zona Norte-Centro",
    "Centro": "Zona Norte-Centro",
    "Occidente": "Zona Occidente-Sureste",
    "Sureste": "Zona Occidente-Sureste",
}

df_zonas = df.assign(zona=df["region"].map(mapa_zonas))

rollup_zonas = df_zonas.pivot_table(
    index=["zona", "tiempo"],
    columns="producto",
    values="ventas",
    aggfunc="sum",
    fill_value=0,
)

rollup_zonas
```

zona	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Zona Norte-Centro	T1	52	25	28	75
	T2	54	31	33	60
	T3	34	22	23	43
	T4	68	31	38	68
Zona Occidente-Sureste	T1	44	15	21	46
	T2	51	19	27	45
	T3	54	19	21	52
	T4	58	17	29	46

En este caso la función de agregación es la suma. Por ejemplo, las ventas de Televisores en T1 para Zona Norte-Centro se obtienen como Norte-T1-Televisores más Centro-T1-Televisores.

27.6.4. Pivot: cambiar la orientación de la consulta

Para estudiar la evolución temporal de cada producto, se puede pivotar la tabla y colocar productos en filas y trimestres en columnas.

```
pivot_producto_tiempo = df.pivot_table(
    index="producto",
    columns="tiempo",
    values="ventas",
    aggfunc="sum",
    fill_value=0,
)

pivot_producto_tiempo
```

tiempo	T1	T2	T3	T4
producto				
Computadoras	96	105	88	126
Cámaras	40	50	41	48
Lavadoras	49	60	44	67
Televisores	121	105	95	114

Esta vista responde una pregunta distinta: ya no se enfatiza la región, sino el comportamiento temporal agregado de cada producto.

27.7. Exploración interactiva del cubo

Esta sección usa widgets interactivos (OJS) que solo funcionan en la versión web del libro. Consúltala en línea para explorar las cinco operaciones OLAP (slice, dice, roll-up, drill-down, pivot) sobre el cubo de ejemplo.

27.8. Agregaciones y medidas derivadas

Las medidas de un cubo pueden ser simples o derivadas. Una medida simple es la suma de ventas; una medida derivada puede ser participación porcentual, crecimiento trimestral o margen. En OLAP conviene distinguir entre medidas aditivas, semi-aditivas y no aditivas (Kimball y Ross 2013).

Nota

Definición. Una medida aditiva puede agregarse mediante suma a lo largo de todas las dimensiones relevantes del cubo.

$$M(A \cup B) = M(A) + M(B), \quad A \cap B = \emptyset$$

Las ventas son usualmente aditivas: se pueden sumar por producto, región y tiempo. En cambio, una tasa de conversión no debe agregarse sumando porcentajes; debe recalcularse a partir de numeradores y denominadores.

El siguiente ejemplo calcula ventas totales por producto y participación porcentual dentro del total.

```
ventas_producto = (
    df.groupby("producto", as_index=False)["ventas"]
    .sum()
    .assign(participacion=lambda x: x["ventas"] / x["ventas"].sum())
)

ventas_producto
```

	producto	ventas	participacion
0	Computadoras	415	0.332266
1	Cámaras	179	0.143315
2	Lavadoras	220	0.176141
3	Televisores	435	0.348279

También se puede calcular el crecimiento entre trimestres para cada producto. En este caso, el cubo se reorganiza por producto y tiempo.

```
ventas_tiempo = (
    df.groupby(["producto", "tiempo"], as_index=False)["ventas"]
    .sum()
    .sort_values(["producto", "tiempo"])
)

ventas_tiempo["crecimiento"] = ventas_tiempo.groupby("producto")["ventas"].pct_change()
ventas_tiempo
```

	producto	tiempo	ventas	crecimiento
0	Computadoras	T1	96	NaN
1	Computadoras	T2	105	0.093750
2	Computadoras	T3	88	-0.161905
3	Computadoras	T4	126	0.431818
4	Cámaras	T1	40	NaN
5	Cámaras	T2	50	0.250000
6	Cámaras	T3	41	-0.180000
7	Cámaras	T4	48	0.170732
8	Lavadoras	T1	49	NaN
9	Lavadoras	T2	60	0.224490
10	Lavadoras	T3	44	-0.266667
11	Lavadoras	T4	67	0.522727
12	Televisores	T1	121	NaN
13	Televisores	T2	105	-0.132231
14	Televisores	T3	95	-0.095238
15	Televisores	T4	114	0.200000

27.9. Ventajas, limitaciones y criterios de diseño

La presentación enumera como ventajas de OLAP el análisis rápido de grandes datos, la visión multidimensional, la identificación de patrones y tendencias, el soporte para decisiones y la comparación entre periodos, regiones o productos (Santizo Galicia 2026). Estas ventajas se explican por la preagregación, el modelado dimensional y la separación entre cargas transaccionales y cargas analíticas.

Sin embargo, OLAP también tiene limitaciones. Puede requerir mucho almacenamiento, no siempre se actualiza en tiempo real, el diseño del cubo puede ser complejo y depende de una buena calidad de datos (Santizo Galicia 2026). En un entorno profesional, estas limitaciones se traducen en decisiones concretas:

- Definir qué dimensiones realmente aportan valor analítico.
- Elegir la granularidad mínima necesaria.
- Evitar jerarquías ambiguas o inconsistentes.
- Validar calidad de datos antes de construir agregados.
- Documentar las reglas de negocio usadas en cada medida.

Un error común consiste en pensar que un cubo OLAP resuelve automáticamente la calidad de los datos. En realidad, el cubo amplifica los problemas de integración: categorías mal escritas, fechas incompletas, productos duplicados o regiones inconsistentes pueden producir agregaciones incorrectas.

27.10. Buenas prácticas para documentación reproducible

Cuando un cubo se usa en un reporte técnico o curso de ciencia de datos, conviene documentar el proceso completo: fuente de datos, transformaciones, reglas de agregación y consultas realizadas. Quarto permite combinar explicación, matemáticas, código ejecutable y resultados en un mismo documento. Para que el análisis sea reproducible, se recomienda:

1. Cargar los datos desde una fuente controlada.
2. Construir el cubo mediante código, no manualmente.
3. Separar definiciones conceptuales de operaciones computacionales.
4. Mostrar consultas representativas: *slice*, *dice*, *roll-up*, *drill-down* y *pivot*.
5. Verificar que las agregaciones coincidan con las reglas de negocio.

El siguiente bloque resume una función reutilizable para construir vistas OLAP simples a partir de una tabla larga.

```
def vista_olap(datos, filas, columnas, medida="ventas", agregacion="sum"):
    """Construye una vista tabular tipo OLAP con pandas.

    Parameters
    -----
    datos : pandas.DataFrame
        Tabla en formato largo.
    filas : str or list[str]
        Dimensiones colocadas en las filas.
    columnas : str or list[str]
        Dimensiones colocadas en las columnas.
    medida : str
        Variable numérica que se agregará.
```

```

agregacion : str or callable
    Función de agregación, por ejemplo "sum", "mean" o "count".
"""
return datos.pivot_table(
    index=filas,
    columns=columnas,
    values=medida,
    aggfunc=agregacion,
    fill_value=0,
)

vista_olap(df, filas=["region", "tiempo"], columnas="producto")

```

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Centro	T1	30	11	18	40
	T2	28	14	20	32
	T3	19	10	15	25
	T4	35	16	22	38
Norte	T1	22	14	10	35
	T2	26	17	13	28
	T3	15	12	8	18
	T4	33	15	16	30
Occidente	T1	27	6	9	20
	T2	31	8	11	24
	T3	36	9	7	29
	T4	34	5	10	19
Sureste	T1	17	9	12	26
	T2	20	11	16	21
	T3	18	10	14	23
	T4	24	12	19	27

27.11. Conclusiones

Los cubos de datos OLAP son una herramienta central para organizar, resumir y explorar información multidimensional. Su valor no está solo en la representación visual de un cubo, sino en el modelo conceptual que conecta dimensiones, jerarquías, medidas y operaciones de navegación. A partir de una tabla transaccional, un cubo permite responder preguntas como: cuánto se vendió, dónde se vendió, cuándo ocurrió, qué productos dominaron y cómo cambian los patrones al modificar el nivel de agregación.

Las operaciones *drill-down*, *roll-up*, *slice*, *dice* y *pivot* constituyen el lenguaje básico para interactuar con un cubo. En conjunto, permiten pasar de resúmenes ejecutivos a análisis detallados, restringir subconjuntos relevantes y reorientar la información según la pregunta de negocio. Implementarlas en Python con **pandas** facilita la reproducibilidad y ayuda a conectar la teoría de OLAP con prácticas modernas de ciencia de datos.

Finalmente, el diseño de un cubo exige decisiones cuidadosas sobre granularidad, calidad de datos, jerarquías y medidas. Un cubo bien diseñado mejora la interpretación y la toma de decisiones; un cubo mal diseñado puede producir conclusiones engañosas aunque las consultas sean técnicamente correctas.

27.12. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.

- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.

- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals*

- of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD

- '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global

- Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.

- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.

- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.

- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

28. Data Marts: del Almacén de Datos al Consumo por Área de Negocio

Arquitectura, diseño dimensional y ejemplo reproducible

En el capítulo anterior se definió un cubo OLAP como una función que agrega medidas sobre varias dimensiones, $C : D_1 \times \dots \times D_k \rightarrow \mathbb{R}^m$. Esa función necesita datos organizados de forma dimensional para poder calcularse de manera eficiente. La pregunta natural es: ¿de dónde salen exactamente esas tablas de hechos y dimensiones que el cubo consulta? La respuesta, en la mayoría de las organizaciones, es un **data mart**: un recorte del almacén de datos empresarial, diseñado para un área de negocio específica, sobre el cual se construyen los cubos, los reportes y —como se verá en el siguiente capítulo— las APIs de consumo analítico.

Este capítulo formaliza el concepto de data mart, distingue sus arquitecturas (dependiente, independiente e híbrida), presenta el proceso de diseño dimensional de Kimball y lo aplica al mismo caso de la ferretería multiestado usado en el capítulo de cubos OLAP (Kimball y Ross 2013; Inmon 2005). Así, el data mart que se construye aquí es exactamente la fuente que alimentaría el cubo Ubicación $\times$ Tiempo $\times$ Departamento discutido previamente.

28.1. ¿Qué es un data mart?

Un **data warehouse** (DW) empresarial integra datos de múltiples procesos de negocio —ventas, inventario, recursos humanos, finanzas— en un repositorio único, histórico y consistente. Sin embargo, pocas veces un analista de un área específica necesita consultar *todo* el DW: al equipo de Ventas le interesan las transacciones de venta; a Recursos Humanos, la nómina y la rotación de personal. Consultar directamente un DW completo, con decenas de tablas de hechos y cientos de atributos, resulta ineficiente y difícil de entender para usuarios de negocio.

Definición. Un **data mart** es un subconjunto especializado de un data warehouse, delimitado por un área funcional o proceso de negocio, diseñado para responder las preguntas analíticas de un grupo específico de usuarios.

$$DM_j \subseteq DW, \quad DM_j = \pi_{A_j}(\sigma_{P_j}(DW))$$

En esta notación, σ_{P_j} selecciona los hechos correspondientes al proceso de negocio P_j (por ejemplo, ventas) y π_{A_j} proyecta únicamente los atributos y dimensiones relevantes para esa área A_j . El resultado es una porción del almacén, más pequeña, más rápida de consultar y más fácil de entender para su audiencia (Devlin y Murphy 1988).

Tres características distinguen a un data mart de un data warehouse completo:

1. **Alcance acotado.** Cubre un proceso o departamento, no la organización entera.

2. **Audiencia definida.** Se diseña pensando en las preguntas de un grupo de usuarios de negocio concreto.
3. **Modelo dimensional explícito.** Casi siempre se implementa como un esquema estrella o copo de nieve, optimizado para consultas de agregación, no como una réplica normalizada de sistemas operacionales.

28.2. Arquitecturas: dependiente, independiente e híbrida

La forma en que un data mart obtiene sus datos —directamente del DW o directamente de las fuentes— define su arquitectura y tiene consecuencias importantes sobre la consistencia de la información en toda la organización.

28.2.1. Data marts dependientes (enfoque Inmon)

En el enfoque de Bill Inmon, el DW corporativo se construye primero como un repositorio normalizado (tercera forma normal) que integra y depura todas las fuentes (Inmon 2005). Los data marts se derivan *después*, como vistas dimensionales de ese DW ya integrado.

Definición. Un data mart es **dependiente** cuando su única fuente de datos es el data warehouse corporativo ya integrado.

$$DM_j = f_j(DW), \quad \text{con } DW \text{ como única fuente}$$

La ventaja principal es la consistencia: como todos los data marts parten de la misma fuente integrada, un mismo cliente, producto o empleado tiene exactamente la misma definición en cualquier data mart. El costo es tiempo: el DW corporativo debe existir y estar completo antes de poder entregar el primer data mart, lo cual puede tardar meses o años en organizaciones grandes.

28.2.2. Data marts independientes (enfoque bottom-up)

En el enfoque bottom-up asociado a Ralph Kimball, no es necesario esperar a un DW normalizado completo. Cada data mart se construye directamente desde las fuentes operacionales de su propio proceso de negocio, con su propio proceso ETL.

Definición. Un data mart es **independiente** cuando se construye directamente desde sistemas fuente, sin pasar por un data warehouse integrado previo.

$$DM_j = g_j(S_1, S_2, \dots, S_n), \quad S_i = \text{sistemas fuente}$$

La ventaja es velocidad de entrega: un equipo puede construir y poner en producción el data mart de Ventas en semanas, sin depender de que otras áreas terminen su integración. El riesgo, si no se gestiona con cuidado, es la aparición de **silos de datos** (*stovepipes*): si el data mart de Ventas y el de Marketing definen “cliente activo” de formas distintas, sus reportes no van a coincidir, y reconciliar esa diferencia después es costoso.

28.2.3. Arquitectura híbrida: el bus de Kimball

Kimball resuelve el dilema de velocidad contra consistencia con la **arquitectura de bus**: los data marts se construyen de forma incremental, proceso por proceso, pero comparten un conjunto de **dimensiones conformadas** —definiciones únicas de Tiempo, Producto, Cliente, Ubicación, etc.— acordadas de antemano entre equipos (Kimball y Ross 2013). El conjunto de data marts conformados, visto en conjunto, constituye un DW empresarial *virtual*, sin necesidad de construir primero un modelo normalizado centralizado.

La siguiente tabla resume las diferencias prácticas entre las tres arquitecturas.

Criterio	Dependiente (Inmon)	Independiente	Bus / Híbrida (Kimball)
Fuente de datos	EDW integrado	Sistemas fuente directos	Sistemas fuente directos
Consistencia entre áreas	Alta	Baja si no hay gobierno	Alta, vía dimensiones conformadas
Tiempo de entrega inicial	Largo (requiere EDW previo)	Corto	Corto por data mart
Riesgo principal	Costo y tiempo de construir el EDW	Silos de datos, definiciones divergentes	Requiere disciplina de gobierno desde el inicio
Escalabilidad organizacional	Alta una vez construido el EDW	Baja sin coordinación	Alta, crece data mart por data mart

28.3. Dimensiones conformadas y la matriz de bus

Definición. Una **dimensión conformada** es una dimensión con la misma clave, atributos y significado de negocio en dos o más data marts, de manera que puede reutilizarse sin transformación adicional entre ellos.

La herramienta principal para planear qué dimensiones deben conformarse es la **matriz de bus**: una tabla donde las filas son procesos de negocio y las columnas son dimensiones candidatas. Una marca indica que ese proceso usa esa dimensión con la definición conformada.

Proceso de negocio	Tiempo	Ubicación	Producto/Departamento	Cliente	Empleado
Ventas					
Inventario					
Recursos Humanos					
Finanzas					

La matriz de bus revela que **Tiempo** y **Ubicación** son candidatas naturales a conformarse primero, porque las usan todos los procesos: conviene diseñarlas una sola vez, con una sola clave y una sola jerarquía, y reutilizarlas en cada data mart que se construya después.

28.4. Diseño dimensional de un data mart

Kimball propone cuatro decisiones, en orden, para diseñar cualquier data mart (Kimball y Ross 2013):

1. **Elegir el proceso de negocio.** Un evento medible y repetible: una venta, un movimiento de inventario, una nómina.
2. **Declarar la granularidad.** ¿Qué representa exactamente una fila de la tabla de hechos? (Ver el capítulo de modelado multidimensional para la definición formal de granularidad.)
3. **Identificar las dimensiones.** Los ejes de análisis relevantes para ese proceso: quién, qué, cuándo, dónde.
4. **Identificar los hechos.** Las medidas numéricas que se desean agregar.

28.4.1. Ejemplo integrador: el data mart detrás del cubo de la ferretería

En el capítulo anterior se trabajó con una cadena de ferreterías con sucursales en CDMX, Guadalajara, Monterrey y Mérida, departamentos Madera, Plomería, Decoración y Eléctrico, y ventas trimestrales como medida. Ese cubo no aparece de la nada: es el resultado de consultar un **data mart de Ventas** diseñado explícitamente con las cuatro decisiones anteriores.

1. **Proceso de negocio:** ventas en tienda.
2. **Granularidad:** una fila por ticket de venta, departamento y trimestre.
3. **Dimensiones:** `dim_tiempo` (trimestre, año), `dim_ubicacion` (tienda, ciudad, región), `dim_departamento` (departamento, categoría).
4. **Hechos:** `monto_venta` (medida aditiva), `unidades` (medida aditiva).

El esquema estrella resultante se representa así:

28.5. Ejemplo reproducible: construir el data mart en Python

A continuación se construye este data mart en Python usando `pandas`. En un entorno de producción, estas mismas tablas suelen vivir en un motor analítico como DuckDB, BigQuery o Snowflake, y las consultas se expresan en SQL en lugar de operaciones de `DataFrame`; aquí se usa `pandas` para mantener continuidad con el resto del curso, y las consultas se escriben primero en SQL (para mostrar la estructura relacional) y luego se resuelven con `merge/groupby`. Se reutilizan los mismos datos del capítulo de cubos OLAP para que ambos capítulos sean consistentes entre sí.

```
import pandas as pd

dim_tiempo = pd.DataFrame({
    "id_tiempo": [1, 2],
    "trimestre": ["T1", "T2"],
    "anio": [2025, 2025],
})

dim_ubicacion = pd.DataFrame({
    "id_ubicacion": [1, 2, 3, 4],
    "ciudad": ["CDMX", "Guadalajara", "Monterrey", "Mérida"],
    "region": ["Centro", "Occidente", "Norte", "Sureste"],
```



```

})

dim_departamento = pd.DataFrame({
    "id_departamento": [1, 2, 3, 4],
    "departamento": ["Madera", "Plomería", "Decoración", "Eléctrico"],
})

# fact_ventas: (id_tiempo, id_ubicacion, id_departamento, monto_venta)
fact_ventas = pd.DataFrame([
    (1, 1, 1, 70), (1, 1, 2, 40), (1, 1, 3, 20), (1, 1, 4, 30),
    (2, 1, 1, 50), (2, 1, 2, 60), (2, 1, 3, 22), (2, 1, 4, 28),
    (1, 2, 1, 60), (1, 2, 2, 35), (1, 2, 3, 25), (1, 2, 4, 40),
    (2, 2, 1, 45), (2, 2, 2, 50), (2, 2, 3, 30), (2, 2, 4, 35),
    (1, 3, 1, 48), (1, 3, 2, 38), (1, 3, 3, 18), (1, 3, 4, 26),
    (2, 3, 1, 36), (2, 3, 2, 55), (2, 3, 3, 25), (2, 3, 4, 30),
    (1, 4, 1, 52), (1, 4, 2, 42), (1, 4, 3, 15), (1, 4, 4, 37),
    (2, 4, 1, 44), (2, 4, 2, 33), (2, 4, 3, 20), (2, 4, 4, 28),
], columns=["id_tiempo", "id_ubicacion", "id_departamento", "monto_venta"])

fact_ventas.head()

```

	id_tiempo	id_ubicacion	id_departamento	monto_venta
0	1	1	1	70
1	1	1	2	40
2	1	1	3	20
3	1	1	4	30
4	2	1	1	50

El data mart queda expresado como cuatro tablas relacionadas por llaves foráneas: exactamente la estructura de un esquema estrella. La consulta analítica equivalente en SQL —el equivalente a un *roll-up* del capítulo anterior— sería:

```

SELECT
    u.region,
    t.trimestre,
    d.departamento,
    SUM(f.monto_venta) AS total_ventas
FROM fact_ventas f
JOIN dim_tiempo      t ON f.id_tiempo = t.id_tiempo
JOIN dim_ubicacion   u ON f.id_ubicacion = u.id_ubicacion
JOIN dim_departamento d ON f.id_departamento = d.id_departamento
GROUP BY u.region, t.trimestre, d.departamento
ORDER BY u.region, t.trimestre, d.departamento

```

En pandas, el mismo JOIN seguido de GROUP BY se expresa encadenando `merge` y `groupby`, en lugar de operar sobre un cubo ya materializado:

```

reporte = (
    fact_ventas
    .merge(dim_tiempo, on="id_tiempo")
    .merge(dim_ubicacion, on="id_ubicacion")
    .merge(dim_departamento, on="id_departamento")
    .groupby(["region", "trimestre", "departamento"], as_index=False)["monto_venta"]
    .sum()
    .rename(columns={"monto_venta": "total_ventas"})
    .sort_values(["region", "trimestre", "departamento"])
)

reporte

```

	region	trimestre	departamento	total_ventas
0	Centro	T1	Decoración	20
1	Centro	T1	Eléctrico	30
2	Centro	T1	Madera	70
3	Centro	T1	Plomería	40
4	Centro	T2	Decoración	22
5	Centro	T2	Eléctrico	28
6	Centro	T2	Madera	50
7	Centro	T2	Plomería	60
8	Norte	T1	Decoración	18
9	Norte	T1	Eléctrico	26
10	Norte	T1	Madera	48
11	Norte	T1	Plomería	38
12	Norte	T2	Decoración	25
13	Norte	T2	Eléctrico	30
14	Norte	T2	Madera	36
15	Norte	T2	Plomería	55
16	Occidente	T1	Decoración	25
17	Occidente	T1	Eléctrico	40
18	Occidente	T1	Madera	60
19	Occidente	T1	Plomería	35
20	Occidente	T2	Decoración	30
21	Occidente	T2	Eléctrico	35
22	Occidente	T2	Madera	45
23	Occidente	T2	Plomería	50
24	Sureste	T1	Decoración	15
25	Sureste	T1	Eléctrico	37
26	Sureste	T1	Madera	52
27	Sureste	T1	Plomería	42
28	Sureste	T2	Decoración	20
29	Sureste	T2	Eléctrico	28
30	Sureste	T2	Madera	44
31	Sureste	T2	Plomería	33

Esta salida es, cifra por cifra, la misma información que el cubo del capítulo anterior representaba como

$C(\text{región, tiempo, departamento})$. La diferencia es de capa: el data mart es el modelo de almacenamiento relacional; el cubo es la forma de consultarlo y navegarlo interactivamente.

28.5.1. Consultar el mismo data mart con la sintaxis de un roll-up regional

El siguiente ejemplo agrupa `dim_ubicacion` en zonas antes del `GROUP BY`, ilustrando que un *roll-up* no es más que sustituir el nivel de agrupación de una dimensión conformada, sin tocar la tabla de hechos.

```
zona_map = pd.DataFrame({
    "region": ["Centro", "Occidente", "Norte", "Sureste"],
    "zona": ["Zona Norte-Centro", "Zona Occidente-Sureste",
            "Zona Norte-Centro", "Zona Occidente-Sureste"],
})

reporte_zona = (
    fact_ventas
    .merge(dim_tiempo, on="id_tiempo")
    .merge(dim_ubicacion, on="id_ubicacion")
    .merge(zona_map, on="region")
    .groupby(["zona", "trimestre"], as_index=False)["monto_venta"]
    .sum()
    .rename(columns={"monto_venta": "total_ventas"})
    .sort_values(["zona", "trimestre"])
)

reporte_zona
```

	zona	trimestre	total_ventas
0	Zona Norte-Centro	T1	290
1	Zona Norte-Centro	T2	306
2	Zona Occidente-Sureste	T1	306
3	Zona Occidente-Sureste	T2	285

28.6. El riesgo de los data marts independientes: dimensiones no conformadas

Para ilustrar por qué la conformidad de dimensiones importa, supongamos que el equipo de Marketing construyó su propio data mart de manera independiente, sin coordinarse con Ventas, y definió “región” con una granularidad distinta (por zona comercial, no por ciudad).

Data mart	Definición de “región”	Valores
Ventas (este capítulo)	Ciudad sede de la sucursal	CDMX, Guadalajara, Monterrey, Mérida
Marketing (hipotético, independiente)	Zona comercial agregada	Zona Norte-Centro, Zona Occidente-Sureste

Ambas definiciones son legítimas por separado, pero si un directivo intenta cruzar un reporte de Ventas por “región” con uno de Marketing por “región” sin saber que la granularidad difiere, llegará a conclusiones incorrectas o a totales que no cuadran. La solución de gobierno de datos es simple de enunciar y difícil de ejecutar en la práctica: acordar la dimensión `dim_ubicacion` (y su jerarquía ciudad → zona → región) **una sola vez**, en la matriz de bus, antes de que cada equipo construya su propio data mart.

28.7. Ventajas, limitaciones y buenas prácticas

Los data marts existen porque resuelven un problema real: un DW corporativo completo puede ser demasiado grande, lento y genérico para las preguntas cotidianas de un área específica. Entre sus ventajas destacan:

- **Consultas más rápidas**, al reducir el volumen de datos y el número de tablas relevantes.
- **Modelo más simple**, alineado al vocabulario de negocio del área (menos jerga técnica de todo el DW).
- **Entrega incremental**, especialmente en la arquitectura de bus, sin depender de un proyecto monolítico.
- **Gobernanza más manejable**, al delimitar quién es dueño de qué datos.

Sus limitaciones más comunes son:

- Riesgo de **silos** cuando se construyen de forma independiente sin dimensiones conformadas.
- **Redundancia de almacenamiento**, al replicar dimensiones compartidas en varios data marts.
- Necesidad de mantener **sincronizados** los procesos ETL/ELT de cada data mart conforme cambian las fuentes.

Buenas prácticas recomendadas al diseñar un nuevo data mart:

1. Revisar la matriz de bus antes de crear una dimensión nueva; reutilizar una conformada si ya existe.
2. Documentar la granularidad exacta de la tabla de hechos desde el día uno.
3. Versionar el esquema (nombres de columnas, tipos, claves) igual que se versiona código.
4. Validar que las medidas declaradas sean aditivas, semi-aditivas o no aditivas, y documentarlo explícitamente (Kimball y Ross 2013).
5. Definir desde el diseño quién consumirá el data mart y cómo (reportes, cubos OLAP, o —tema del siguiente capítulo— una API).

28.8. De data marts a productos de datos

En años recientes, el concepto de data mart ha evolucionado hacia el de **producto de datos** dentro de arquitecturas de *data mesh*: cada dominio de negocio no solo publica su data mart, sino que lo trata como un producto con dueño, documentación, garantías de calidad y, sobre todo, una interfaz de consumo estándar en lugar de acceso directo a tablas (Dehghani 2022). Esa interfaz de consumo es, cada vez con más frecuencia, una API. El siguiente capítulo retoma exactamente el data mart de Ventas construido aquí y muestra cómo exponerlo mediante REST, GraphQL y una capa semántica moderna.

28.9. Conclusiones

Un data mart es el punto de encuentro entre el modelo dimensional (hechos, dimensiones, granularidad) y las necesidades concretas de un área de negocio. La decisión de construirlo de forma dependiente, independiente o mediante una arquitectura de bus no es solo técnica: determina qué tan rápido se entrega valor y qué tan consistentes serán los números entre departamentos. El ejemplo de la ferretería mostró que un cubo OLAP no es una estructura aislada, sino la forma de consultar un data mart bien diseñado; y que las mismas operaciones de *roll-up* estudiadas antes corresponden, a nivel relacional, a sustituir el nivel de una dimensión conformada dentro de una consulta SQL.

28.10. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.

- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics

Publications.

- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinowler.com. <https://martinowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.

Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.

Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.

Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.

Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).

Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.

Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.

Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).

Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.

Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.

Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.

Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.

GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.

Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan

- Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology

derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.

Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.

James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.

Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.

Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.

Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.

Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.

Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.

Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.

Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.

Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.

Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.

Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.

- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.

- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.

- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

29. Consumo Moderno de Data Marts: APIs y Capas Semánticas

De la consulta directa al contrato de servicio

El capítulo anterior terminó con el data mart de Ventas ya construido: cuatro tablas —`dim_tiempo`, `dim_ubicacion`, `dim_departamento`, `fact_ventas`— listas para responder preguntas de negocio mediante JOIN y GROUP BY, o mediante las operaciones de un cubo OLAP. Esa forma de consumo —un analista con acceso directo al motor de base de datos o a una herramienta de BI conectada por ODBC/JDBC— sigue siendo válida, pero no es la única, ni siempre la más adecuada. Una aplicación móvil de ventas, un panel embebido en el sitio de un cliente, otro microservicio de la organización o un agente automatizado no deberían necesitar credenciales de base de datos ni conocer el esquema interno del data mart para obtener una cifra de ventas. Este capítulo formaliza esa idea: exponer el data mart mediante un **contrato de servicio** —una API— en lugar de acceso directo a las tablas.

29.1. Del acceso directo al contrato de servicio

En el modelo de consumo tradicional, cada cliente (una hoja de cálculo, un dashboard, un script) se conecta directamente al data mart y ejecuta su propia consulta SQL. Este modelo es simple, pero acopla a cada cliente con los detalles internos del esquema: si `dim_ubicacion` cambia una columna de nombre, todos los clientes que la referencian se rompen. Además, es difícil aplicar de forma centralizada control de acceso, límites de uso o versionado.

Definición. Un **contrato de servicio de datos** desacopla la representación interna de almacenamiento DM_j de la interfaz que ven los clientes, mediante una función de acceso A que expone solo las operaciones y campos autorizados:

$$A : \text{Petición} \longrightarrow \text{Respuesta}, \quad A \text{ no revela el esquema interno de } DM_j$$

Esta separación entre almacenamiento y consumo es la misma idea de independencia de datos que ya aparece en el diseño de un DW (Kimball y Ross 2013), llevada un nivel más arriba: así como el modelo dimensional independiza el análisis de los sistemas operacionales, la capa de API independiza a los clientes del modelo dimensional.

29.2. REST sobre un data mart

REST (*Representational State Transfer*) es el estilo arquitectónico definido por Roy Fielding para sistemas distribuidos basados en recursos, representaciones y operaciones sin estado sobre HTTP (Fielding 2000).

En el contexto de un data mart, un recurso natural es una consulta agregada: por ejemplo, “ventas por región y trimestre”.

Definición. Un endpoint REST sobre un data mart mapea una combinación de filtros y agrupaciones a una representación (usualmente JSON):

$$\text{GET } /ventas?region = r \& \text{ trimestre} = t \mapsto \{\text{departamento} : v \mid v = C(r, t, \cdot)\}$$

Nótese el paralelismo con la función de cubo C del primer capítulo: el endpoint REST es, en esencia, una forma de invocar esa misma función agregada a través de HTTP, con los filtros codificados como parámetros de consulta.

Martin Fowler resume la madurez de una API REST en el **Richardson Maturity Model**: nivel 0 (un solo endpoint tipo RPC), nivel 1 (recursos individuales), nivel 2 (verbos HTTP y códigos de estado correctos) y nivel 3 (HATEOAS, enlaces de navegación embebidos en la respuesta) (Fowler 2010). La mayoría de las APIs analíticas en producción se detienen en el nivel 2, que ya resuelve el problema práctico de exponer datos de forma predecible.

29.2.1. Diseño del endpoint

Un endpoint de nivel 2 razonable para el data mart de Ventas podría verse así (código ilustrativo, no ejecutado en este documento, para evitar añadir dependencias nuevas al entorno del curso):

```
# app.py - ilustrativo, requiere fastapi (no incluido en requirements.txt del curso)
from fastapi import FastAPI, Query

app = FastAPI(title="API de Ventas")

@app.get("/v1/ventas")
def obtener_ventas(
    region: str | None = Query(None),
    trimestre: str | None = Query(None),
    departamento: str | None = Query(None),
):
    """Devuelve ventas agregadas filtradas por región, trimestre y/o departamento."""
    resultado = consultar_metrica(
        "total_ventas",
        group_by=["region", "trimestre", "departamento"],
        filtros={"region": region, "trimestre": trimestre, "departamento": departamento},
    )
    return {"data": resultado}
```

El detalle importante aquí no es la sintaxis de FastAPI (Ramírez, Sebastián 2024), sino la separación de responsabilidades: el endpoint solo traduce parámetros HTTP en una llamada a `consultar_metrica`, una función de capa semántica que se define y prueba una sola vez (se implementa más adelante en este capítulo) y que oculta por completo los JOIN y GROUP BY del data mart.

29.2.2. Simulación reproducible de la petición y respuesta

Sin necesidad de levantar un servidor, se puede reproducir exactamente lo que ese endpoint haría: recibir un diccionario de parámetros y devolver un diccionario JSON. Primero se reconstruyen las tablas del data mart del capítulo anterior.

```
import pandas as pd

dim_tiempo = pd.DataFrame({
    "id_tiempo": [1, 2], "trimestre": ["T1", "T2"], "anio": [2025, 2025],
})
dim_ubicacion = pd.DataFrame({
    "id_ubicacion": [1, 2, 3, 4],
    "ciudad": ["CDMX", "Guadalajara", "Monterrey", "Mérida"],
    "region": ["Centro", "Occidente", "Norte", "Sureste"],
})
dim_departamento = pd.DataFrame({
    "id_departamento": [1, 2, 3, 4],
    "departamento": ["Madera", "Plomería", "Decoración", "Eléctrico"],
})
fact_ventas = pd.DataFrame([
    (1, 1, 1, 70), (1, 1, 2, 40), (1, 1, 3, 20), (1, 1, 4, 30),
    (2, 1, 1, 50), (2, 1, 2, 60), (2, 1, 3, 22), (2, 1, 4, 28),
    (1, 2, 1, 60), (1, 2, 2, 35), (1, 2, 3, 25), (1, 2, 4, 40),
    (2, 2, 1, 45), (2, 2, 2, 50), (2, 2, 3, 30), (2, 2, 4, 35),
    (1, 3, 1, 48), (1, 3, 2, 38), (1, 3, 3, 18), (1, 3, 4, 26),
    (2, 3, 1, 36), (2, 3, 2, 55), (2, 3, 3, 25), (2, 3, 4, 30),
    (1, 4, 1, 52), (1, 4, 2, 42), (1, 4, 3, 15), (1, 4, 4, 37),
    (2, 4, 1, 44), (2, 4, 2, 33), (2, 4, 3, 20), (2, 4, 4, 28),
], columns=["id_tiempo", "id_ubicacion", "id_departamento", "monto_venta"])

data_mart = (
    fact_ventas
    .merge(dim_tiempo, on="id_tiempo")
    .merge(dim_ubicacion, on="id_ubicacion")
    .merge(dim_departamento, on="id_departamento")
)
data_mart.head()
```

	id_tiempo	id_ubicacion	id_departamento	monto_venta	trimestre	anio	ciudad	region	departame
0	1	1	1	70	T1	2025	CDMX	Centro	Madera
1	1	1	2	40	T1	2025	CDMX	Centro	Plomería
2	1	1	3	20	T1	2025	CDMX	Centro	Decoración
3	1	1	4	30	T1	2025	CDMX	Centro	Eléctrico
4	2	1	1	50	T2	2025	CDMX	Centro	Madera

Ahora se define, en Python puro, la función que un endpoint REST invocaría internamente, y se simula una petición GET `/v1/ventas?region=Centro&trimestre=T2`:

```
def obtener_ventas(region=None, trimestre=None, departamento=None):
    df = data_mart
    if region:
        df = df[df["region"] == region]
    if trimestre:
        df = df[df["trimestre"] == trimestre]
    if departamento:
        df = df[df["departamento"] == departamento]

    agregado = (
        df.groupby(["region", "trimestre", "departamento"], as_index=False)["monto_venta"]
        .sum()
        .rename(columns={"monto_venta": "total_ventas"})
    )
    return {"data": agregado.to_dict(orient="records")}

peticion = {"region": "Centro", "trimestre": "T2"}
respuesta = obtener_ventas(**peticion)
respuesta
```

```
{'data': [{'region': 'Centro',
            'trimestre': 'T2',
            'departamento': 'Decoración',
            'total_ventas': 22},
          {'region': 'Centro',
            'trimestre': 'T2',
            'departamento': 'Eléctrico',
            'total_ventas': 28},
          {'region': 'Centro',
            'trimestre': 'T2',
            'departamento': 'Madera',
            'total_ventas': 50},
          {'region': 'Centro',
            'trimestre': 'T2',
            'departamento': 'Plomería',
            'total_ventas': 60}]}
```

La respuesta trae **todos** los campos del recurso “ventas”, independientemente de si el cliente necesita solo `total_ventas` o también `departamento` y `trimestre`. Esto es exactamente la limitación que motiva GraphQL.

29.3. GraphQL sobre un data mart

GraphQL es un lenguaje de consulta en el que el cliente, no el servidor, decide la forma exacta de la respuesta: qué campos quiere y con qué nivel de anidamiento, en una sola petición a un único endpoint (GraphQL Foundation 2021). Esto evita dos problemas comunes en REST: el *over-fetching* (recibir campos

que no se necesitan) y el *under-fetching* (necesitar varias peticiones encadenadas para reunir toda la información).

Definición. Una consulta GraphQL especifica un subconjunto de campos $F \subseteq \text{campos}(R)$ de un recurso R ; el resolutor devuelve únicamente F , sin importar cuántos campos adicionales exponga el esquema.

Un esquema típico sobre el data mart de Ventas (código ilustrativo; requeriría una librería como `strawberry-graphql` o `graphene`, no incluida en el entorno del curso):

```
# schema.py - ilustrativo, requiere strawberry-graphql
import strawberry

@strawberry.type
class VentaAgregada:
    region: str
    trimestre: str
    departamento: str
    total_ventas: float

@strawberry.type
class Query:
    @strawberry.field
    def ventas(self, region: str | None = None, trimestre: str | None = None) -> list[VentaAgregada]:
        resultado = consultar_metrica(
            "total_ventas",
            group_by=["region", "trimestre", "departamento"],
            filtros={"region": region, "trimestre": trimestre},
        )
        return [VentaAgregada(**fila) for fila in resultado]

schema = strawberry.Schema(query=Query)
```

Un cliente que solo necesita el total por departamento, sin región ni trimestre, escribiría:

```
query {
  ventas(region: "Centro", trimestre: "T2") {
    departamento
    totalVentas
  }
}
```

29.3.1. Simulación reproducible: el cliente decide la forma de la respuesta

El siguiente código, ejecutado con `pandas` puro, reproduce el comportamiento esencial de un resolutor GraphQL: recibe la misma consulta y los mismos filtros que en el ejemplo REST, pero un parámetro adicional —los campos solicitados por el cliente— determina qué columnas trae la respuesta.

```
def resolver_ventas(campos_solicitados, region=None, trimestre=None, departamento=None):
    df = data_mart
    if region:
        df = df[df["region"] == region]
    if trimestre:
        df = df[df["trimestre"] == trimestre]
    if departamento:
        df = df[df["departamento"] == departamento]

    agregado = (
        df.groupby(["region", "trimestre", "departamento"], as_index=False)["monto_venta"]
        .sum()
        .rename(columns={"monto_venta": "total_ventas"})
    )
    return agregado[campos_solicitados].to_dict(orient="records")

# El cliente REST recibía region, trimestre, departamento y total_ventas.
# El cliente GraphQL pide solo lo que necesita:
resolver_ventas(["departamento", "total_ventas"], region="Centro", trimestre="T2")
```

```
[{'departamento': 'Decoración', 'total_ventas': 22},
 {'departamento': 'Eléctrico', 'total_ventas': 28},
 {'departamento': 'Madera', 'total_ventas': 50},
 {'departamento': 'Plomería', 'total_ventas': 60}]
```

La misma pregunta de negocio —ventas del Centro en T2— produce una respuesta más pequeña porque el cliente, no el servidor, definió la forma de la salida. Esta flexibilidad es la principal ventaja de GraphQL; su costo es una capa de resolución más compleja y, en resolutores mal diseñados, el riesgo del problema $N+1$ (una consulta adicional por cada elemento anidado) (Kleppmann 2017).

29.3.2. REST y GraphQL en comparación

Criterio	REST	GraphQL
Forma de la respuesta	Fija, la define el servidor	Flexible, la define el cliente
Número de endpoints	Uno por recurso (o pocos genéricos)	Uno único para todo el esquema
Riesgo típico	Over-/under-fetching	Consultas anidadas costosas (N+1)
Caché HTTP estándar	Sencilla (por URL)	Requiere estrategia propia
Curva de aprendizaje para el cliente	Baja	Media (requiere conocer el esquema)
Adecuado para	Integraciones simples, servicios públicos	Apps con necesidades de datos muy variables

29.4. Capa semántica: métricas como código

Tanto el ejemplo REST como el de GraphQL delegaron el cálculo real a una función llamada `consultar_metrica`. Esa función es la pieza más importante de una arquitectura de consumo moderna: la **capa semántica**. Su función es definir cada métrica de negocio (qué se suma, con qué filtros válidos, con qué nombre) **una sola vez**, para que REST, GraphQL, un notebook y una herramienta de BI calculen exactamente el mismo número (dbt Labs 2024; Cube Dev, Inc. 2024).

Definición. Una **métrica semántica** es una función registrada $M : (\text{agrupación}, \text{filtros}) \rightarrow \mathbb{R}$ sobre el data mart, independiente de cualquier consumidor específico.

$$M_{\text{total_ventas}}(G, F) = \sum_{i: i \models F} \text{monto_venta}_i \quad \text{agrupado por } G$$

Esta idea es el equivalente, a nivel de métricas, de lo que las **dimensiones conformadas** son a nivel de atributos (capítulo anterior): en lugar de que cada equipo reimplemente su propio cálculo de “ventas totales” dentro de cada API o cada dashboard, existe una sola definición autorizada. A esta arquitectura —métricas versionadas como código, consumidas por múltiples herramientas de presentación— se le conoce como **headless BI**.

29.4.1. Implementación reproducible de un registro de métricas

```
METRICAS = {
    "total_ventas": {
        "columna": "monto_venta",
        "agregacion": "sum",
    },
    "ticket_promedio": {
        "columna": "monto_venta",
        "agregacion": "mean",
    },
}

def consultar_metrica(nombre_metrica, group_by, filtros=None):
    if nombre_metrica not in METRICAS:
        raise ValueError(f"Métrica no registrada: {nombre_metrica}")

    definicion = METRICAS[nombre_metrica]
    df = data_mart

    for campo, valor in (filtros or {}).items():
        if valor is not None:
            df = df[df[campo] == valor]

    agregado = (
        df.groupby(group_by, as_index=False)[definicion["columna"]]
        .agg(definicion["agregacion"])
        .rename(columns={definicion["columna"]: nombre_metrica})
    )
```

```

    )
    return agregado.to_dict(orient="records")

consultar_metrica(
    "total_ventas",
    group_by=["region", "trimestre"],
    filtros={"trimestre": "T2"},
)

```

```

[{'region': 'Centro', 'trimestre': 'T2', 'total_ventas': 160},
 {'region': 'Norte', 'trimestre': 'T2', 'total_ventas': 146},
 {'region': 'Occidente', 'trimestre': 'T2', 'total_ventas': 160},
 {'region': 'Sureste', 'trimestre': 'T2', 'total_ventas': 125}]

```

```

consultar_metrica(
    "ticket_promedio",
    group_by=["departamento"],
    filtros={"region": "Centro"},
)

```

```

[{'departamento': 'Decoración', 'ticket_promedio': 21.0},
 {'departamento': 'Eléctrico', 'ticket_promedio': 29.0},
 {'departamento': 'Madera', 'ticket_promedio': 60.0},
 {'departamento': 'Plomería', 'ticket_promedio': 50.0}]

```

Con este registro, tanto `obtener_ventas` (REST) como `resolver_ventas` (GraphQL) podrían reescribirse para llamar únicamente a `consultar_metrica`, garantizando que ambos protocolos devuelvan cifras consistentes entre sí y con cualquier otra herramienta —incluido el cubo OLAP del primer capítulo— que también consulte el mismo registro.

29.5. Un mismo dato, tres contratos

El siguiente panel resume, para la misma pregunta de negocio (ventas del Centro en T2, por departamento), cómo se ve la petición y la respuesta bajo cada uno de los tres contratos discutidos: REST, GraphQL y una llamada directa a la capa semántica.

29.6. Seguridad y gobierno en la capa de API

Exponer un data mart mediante una API centraliza también el lugar natural para aplicar controles que serían difíciles de garantizar si cada cliente consultara la base de datos directamente:

- **Autenticación y autorización.** Cada petición debe identificar quién la hace (token, llave de API) y qué puede ver.
- **Seguridad a nivel de fila (*row-level security*).** Por ejemplo, un gerente regional solo debería poder consultar `region = "su región"`, sin importar qué parámetros envíe.

- **Límites de uso (*rate limiting*).** Evita que una consulta costosa mal escrita degrade el servicio para el resto de los clientes.
- **Versionado explícito del contrato.** El prefijo /v1/ en los ejemplos anteriores permite evolucionar el esquema sin romper clientes existentes.
- **Documentación ejecutable.** Un esquema OpenAPI (para REST) o el esquema introspectable de GraphQL permiten generar documentación siempre sincronizada con el código.

29.7. Ventajas, limitaciones y buenas prácticas

Exponer un data mart mediante APIs y una capa semántica tiene ventajas claras frente al acceso directo:

- **Desacoplamiento:** cambios en el esquema físico del data mart no rompen a los clientes si el contrato se mantiene.
- **Reutilización:** un mismo conjunto de métricas alimenta apps, dashboards y otros servicios sin duplicar lógica de agregación.
- **Consistencia:** una sola definición de cada métrica evita que dos equipos reporten cifras distintas para la misma pregunta.
- **Gobierno centralizado:** autenticación, autorización y límites de uso se aplican en un solo lugar.

También introduce costos que conviene anticipar:

- Una capa adicional de infraestructura que hay que operar y monitorear.
- Riesgo de latencia si cada petición recalcula agregaciones costosas en lugar de usar resultados pre-agregados o cacheados.
- En GraphQL, riesgo de consultas anidadas ineficientes si los resolutores no se diseñan con cuidado.

Buenas prácticas recomendadas:

1. Definir las métricas en la capa semántica antes de escribir cualquier endpoint, nunca al revés.
2. Versionar el contrato de la API igual que se versiona el esquema del data mart.
3. Pre-agregar o cachear las consultas más frecuentes en lugar de recalcularlas en cada petición.
4. Aplicar seguridad a nivel de fila en la capa semántica, no confiar en que cada cliente filtre correctamente.
5. Tratar cada data mart expuesto como un **producto de datos**, con dueño, documentación y garantías de calidad (Dehghani 2022).

29.8. Conclusiones

Este capítulo cerró el recorrido de la sección de modelado para consumo: partiendo de un data mart bien diseñado, existen (al menos) dos formas complementarias de consumirlo. La primera, cubierta en el primer capítulo, es la navegación interactiva mediante un cubo OLAP: *slice*, *dice*, *roll-up*, *drill-down* y *pivot* sobre una vista multidimensional, pensada para un analista humano explorando datos. La segunda, cubierta aquí, es la integración programática mediante un contrato de servicio —REST o GraphQL— respaldado por una capa semántica que garantiza que la definición de cada métrica sea única, sin importar cuántas aplicaciones distintas la consulten.

Ambas formas de consumo parten exactamente del mismo data mart. La elección entre exponer un cubo, una API o ambos no es una decisión tecnológica aislada: depende de quién necesita los datos y cómo los va a usar. Un director que explora tendencias trimestre a trimestre se beneficia de un cubo interactivo;

una aplicación móvil que muestra el total de ventas del día se beneficia de un endpoint REST ligero; un ecosistema de aplicaciones internas con necesidades de datos heterogéneas se beneficia de una capa semántica expuesta vía GraphQL. Diseñar bien el data mart y sus dimensiones conformadas es lo que hace posible que las tres alternativas coexistan sin contradecirse entre sí.

29.9. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.

- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Calínski, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.

- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals*

- of *Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD

- '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global

- Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.

- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.

- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.

- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte IX.

Marco Metodológico de Minería de Datos

30. Introducción a la Minería de Datos y sus Frameworks de Proceso

30.1. La pregunta que abre el capítulo

Imagina que acabas de incorporarte como analista de datos a **BigMart**, una cadena minorista con outlets de distinto tamaño y formato en varias ciudades — el mismo caso que se retoma en detalle a lo largo de esta unidad (Analytics Vidhya 2016). En tu primera semana, la directora comercial te muestra un reporte y te hace una pregunta que suena simple:

“Aquí está Item_DT_1985, un paquete de galletas. En el outlet OUT027 se vende como pan caliente; en el OUT010, casi nadie lo compra. Es el mismo producto, el mismo precio de lista. ¿Por qué se vende tan distinto? Y más importante: ¿qué deberíamos hacer al respecto?”

Esta pregunta —aparentemente trivial— es en realidad la puerta de entrada a toda la disciplina que estudia este capítulo. Note que la directora **no** preguntó “muéstrame las ventas de Item_DT_1985 por outlet” (eso ya lo tiene, en el reporte que sostiene). Preguntó *por qué*, y qué hacer. Responder eso requiere algo que ninguna consulta a una base de datos puede entregar por sí sola: un modelo de las relaciones que gobiernan el fenómeno.

Este capítulo construye, paso a paso, el vocabulario y las herramientas de proceso necesarias para convertir una pregunta como esta en un proyecto ejecutable. La ruta es la siguiente: primero se distingue la minería de datos de la simple recuperación de datos; después se advierte sobre la trampa más común del oficio —confundir correlación con causalidad—; luego se catalogan las tareas centrales de la minería de datos; y finalmente se presentan los **frameworks de proceso** (KDD, CRISP-DM, SEMMA, TDSP y sus evoluciones modernas) que estructuran cómo, en la práctica, un equipo pasa de la pregunta de la directora a una decisión de negocio. A lo largo del capítulo volveremos varias veces a Item_DT_1985 y a BigMart para anclar cada idea nueva en un caso concreto.

30.2. Recuperación de datos versus minería de datos

Antes de convocar a un equipo de ciencia de datos, vale la pena preguntarse si la pregunta de la directora puede responderse con una simple consulta. La respuesta revela la diferencia central de este capítulo.

La **recuperación de datos** responde preguntas cuya forma ya está completamente definida por quien pregunta, antes de tocar el sistema. Es una transformación de la forma “dame los registros que satisfacen la condición *C*”, donde *C* es conocida de antemano y el resultado es un subconjunto del repositorio. Si la directora hubiera preguntado “¿cuántas unidades de Item_DT_1985 se vendieron en OUT027 el trimestre pasado?”, la respuesta es una consulta SQL: agregar Item_Outlet_Sales filtrando por Item_Identifier y Outlet_Identifier. No hay nada que aprender; solo hay que filtrar y sumar.

La **minería de datos**, en cambio, busca responder preguntas donde la forma de la relación es desconocida. El objetivo no es filtrar, sino **aprender**: estimar funciones, reglas o estructuras que expliquen o anticipen

un fenómeno. La pregunta real de la directora — *¿qué hace que un producto rote distinto entre outlets, y qué deberíamos hacer?* — no tiene una columna que la responda directamente. Obliga a definir variables candidatas (`Outlet_Type`, `Outlet_Size`, `Item_Visibility`, `Item_MRP...`), construir un conjunto de datos adecuado, elegir un criterio de evaluación y cuantificar qué tan confiable es la respuesta.

En la práctica, todo proyecto de minería de datos empieza con muchas consultas de recuperación (explorar, filtrar, contar): son el insumo del análisis exploratorio de datos que se desarrolla en la siguiente unidad. La diferencia importa porque confundir ambas actividades lleva a un error común: tratar un hallazgo exploratorio (un filtro, un promedio) como si fuera un modelo validado, sin haber pasado por el proceso de aprendizaje y validación que la minería de datos exige.

30.3. Correlación y causalidad: la primera trampa

Supón que, al explorar los datos de BigMart, encuentras que `Item_Visibility` (el porcentaje de espacio en anaquel que ocupa un producto) está correlacionada negativamente con `Item_Outlet_Sales`: los productos con más visibilidad tienden a vender *menos*, no más. Es tentador concluir: “*hay que reducir la visibilidad de los productos para aumentar ventas*” — una conclusión que, llevada a producción, sería un despropósito.

Una asociación estadística (correlación) no garantiza un vínculo causal. La razón técnica es que los datos observacionales pueden estar dominados por variables omitidas, selección no aleatoria, confusores o retroalimentaciones entre variables (Pearl 2009). En el caso de BigMart, una explicación mucho más plausible es un **confusor**: los productos de baja rotación (por ejemplo, artículos de nicho o de temporada) se colocan deliberadamente en espacios más visibles precisamente *porque* no se venden solos, mientras que los productos de alta rotación (leche, pan) no necesitan visibilidad extra porque el cliente ya los busca activamente. La demanda base del producto está causando tanto la visibilidad asignada como, indirectamente, el patrón de ventas — no es la visibilidad la que causa las ventas bajas.

Para hablar de causalidad con solidez, típicamente se requiere un diseño experimental —idealmente aleatorizado. El experimento por excelencia en sistemas digitales es la **prueba A/B**: se asigna aleatoriamente una intervención (por ejemplo, mover un subconjunto de productos a estantes más visibles en una muestra aleatoria de outlets) a un grupo de tratamiento y se compara contra un grupo control bajo una métrica previamente definida (Kohavi et al. 2020). Empresas con alto volumen de experimentación, como Netflix, documentan públicamente cómo combinan experimentación *offline* (entrenar y validar modelos con datos históricos) con experimentación *online* (A/B testing en producción) antes de lanzar un cambio a todos sus usuarios (Netflix Technology Blog 2016). Incluso con un buen diseño A/B, sigue siendo necesario vigilar violaciones: interferencia entre unidades (*spillover*, por ejemplo si mover un producto en un outlet afecta las ventas de otro producto en el mismo estante), cambios temporales (estacionalidad, como en la Navidad) y sesgos de medición.

30.4. Las tareas centrales de la minería de datos

Una vez que la pregunta de negocio se reconoce como un problema de minería de datos —y no de simple recuperación—, el siguiente paso es identificar *qué tipo* de tarea es. La literatura clásica (Han et al. 2011; Tan et al. 2006) organiza estas tareas en dos grandes familias: **supervisadas**, cuando existe una variable objetivo que se quiere predecir o explicar, y **no supervisadas / de descubrimiento**, cuando el objetivo es encontrar estructura sin una etiqueta previa.

30.4.1. Clasificación (aprendizaje supervisado)

La clasificación modela una variable objetivo categórica. Dado un vector de atributos $\mathbf{x} \in \mathbb{R}^p$, se aprende una función $f(\mathbf{x}) \in \{1, \dots, K\}$ que minimiza una pérdida esperada (por ejemplo, entropía cruzada o error 0-1 aproximado). El desempeño se valida con esquemas como *train/validation/test* o *k-fold cross-validation*, analizando sensibilidad a desbalance de clases, umbrales de decisión y costos asimétricos.

En BigMart: transformar `Item_Outlet_Sales` en una etiqueta binaria (“rotación alta” / “rotación baja” por outlet) y predecirla a partir de `Outlet_Type`, `Outlet_Size` e `Item_Type`, para decidir qué outlets necesitan intervención comercial prioritaria.

30.4.2. Regresión (aprendizaje supervisado)

La regresión se usa cuando la variable objetivo es numérica. El problema consiste en aproximar una función $y \approx g(\mathbf{x})$ bajo un criterio como MSE o MAE. La selección del modelo depende de la estructura esperada: linealidad, heterocedasticidad, colinealidad y presencia de *outliers*.

En BigMart: esta es, literalmente, la tarea central del dataset — predecir `Item_Outlet_Sales` (numérica) a partir de los atributos del producto y del outlet. Es el ejercicio de modelado que se retoma con detalle en la unidad de clasificadores y regresión.

30.4.3. Clustering (aprendizaje no supervisado)

El clustering busca particiones naturales sin etiquetas. El caso prototípico es *k-means*, que minimiza la suma de distancias cuadráticas a centroides. Exige decisiones técnicas: estandarización de variables, selección de *k*, sensibilidad a inicialización y validez del supuesto de clusters aproximadamente esféricos.

En BigMart: segmentar los outlets (no los productos) según su perfil de ventas, tamaño y tipo, para descubrir si existen “arquetipos” de tienda —por ejemplo, supermercados urbanos grandes contra tiendas de conveniencia pequeñas— que expliquen buena parte de la variación observada.

30.4.4. Reglas de asociación

Las reglas tipo “si A entonces B” formalizan co-ocurrencias, típicamente en análisis de canasta de mercado. Las métricas clave son soporte, confianza y *lift*, discutidas en profundidad en la unidad de reglas de asociación.

Nota honesta sobre BigMart: el dataset de BigMart está agregado a nivel producto-outlet (una fila resume ventas históricas), **no** a nivel transacción individual. Por lo tanto, las reglas de asociación **no aplican directamente** sobre estos datos: no sabemos qué productos se compraron juntos en el mismo ticket. Para ilustrar esta tarea se necesita un dataset transaccional distinto —el clásico ejemplo es descubrir que quienes compren jarabe para la tos tienden a comprar también pastillas para la garganta, lo que habilita *cross-sell* o colocación conjunta en anaquel. Si BigMart quisiera aplicar esta tarea, tendría que capturar primero el detalle de cada ticket de compra, algo que su sistema actual no registra.

30.4.5. Detección de anomalías

Detecta observaciones raras respecto a un patrón normal. Se puede abordar como distancia a vecindarios (LOF), separación por particiones (Isolation Forest), reconstrucción (autoencoders) o modelos probabilísticos. Un punto crítico es que “raro” no siempre significa “problema”: requiere validación operacional y un proceso para manejar falsos positivos.

En BigMart: detectar outlets cuyo patrón de ventas se desvía abruptamente de lo esperado para su tipo y tamaño —una señal temprana de un problema operativo (desabasto, error de captura, cierre parcial) o de una oportunidad no explicada que vale la pena investigar.

30.5. Frameworks de proceso: cuatro formas de responder la misma pregunta

Con las tareas centrales identificadas, falta resolver un problema distinto: **¿cómo se organiza el trabajo** desde que la directora hace la pregunta hasta que existe una decisión sustentada en datos? A medida que la disciplina creció en volumen, variedad y velocidad de datos, se hizo necesaria la estandarización de prácticas; de ahí emergen los *frameworks* de proceso: guías end-to-end para conducir proyectos de analítica y minería de datos de manera consistente y repetible.

No todos los frameworks enmarcan la misma pregunta de la misma forma. El siguiente comparador interactivo aplica **la pregunta de la directora de BigMart** a través de cuatro frameworks —KDD, CRISP-DM, SEMMA y TDSP— para mostrar en qué difieren.

30.5.1. KDD como modelo de descubrimiento

KDD (*Knowledge Discovery in Databases*) describe el flujo completo desde datos crudos hacia conocimiento: selección, preprocesamiento, transformación, minería y evaluación/interpretación, culminando en conocimiento accionable (Fayyad et al. 1996). Es especialmente útil cuando el objetivo es **entender** y **extraer patrones explicables**, además de modelar — la minería de datos propiamente dicha es solo una de sus cinco etapas, no el proceso completo.

30.5.2. CRISP-DM como marco de proyecto end-to-end

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) organiza el proyecto en seis fases iterativas: comprensión del negocio, comprensión de los datos, preparación, modelado, evaluación y despliegue (Chapman et al. 2000; Shearer 2000). Su fortaleza es gestionar el ciclo completo con énfasis explícito en llevar resultados a operación (producción, monitoreo, impacto). Es, con diferencia, el framework más adoptado en la industria desde su publicación en 1999–2000, y es el que se desarrolla con mayor detalle en el resto de esta unidad, aplicado al caso BigMart.

30.5.3. SEMMA como pipeline práctico para iterar modelado

SEMMA (*Sample, Explore, Modify, Model, Assess*), originalmente asociado a las herramientas de SAS Institute (SAS Institute 2017), enfatiza el ciclo de modelado: muestrear, explorar, modificar (ingeniería de variables), modelar y evaluar. Es particularmente útil cuando ya existe un dataset razonablemente definido y el objetivo es optimizar desempeño comparando alternativas rápidamente — pero, a diferencia de CRISP-DM, no dice nada sobre cómo se llegó a ese dataset ni sobre qué pasa después de elegir el mejor modelo.

30.5.4. TDSP como alternativa moderna orientada a ingeniería

TDSP (*Team Data Science Process*), documentado y mantenido por Microsoft (Microsoft 2023), es un proceso de equipo que combina gestión, reproducibilidad e implementación con prácticas de ingeniería de software (versionado, artefactos, CI/CD). En comparación con CRISP-DM, enfatiza más explícitamente el “cómo se construye” en ambientes productivos — pipelines, repositorios, roles de equipo, trazabilidad —, aunque su esencia sigue siendo end-to-end y reconoce abiertamente su deuda con CRISP-DM.

30.6. Comparativa de frameworks

Framework	Enfoque principal	¿Cuándo usarlo?	Lo que menos enfatiza	Entregable típico	Aplicado a BigMart
CRISP-DM	Proceso end-to-end orientado a negocio y despliegue	Alinear problema, datos, modelo y operación con métricas de impacto	Técnicas específicas (no dicta algoritmos)	Modelo + plan de despliegue + monitoreo	Objetivo → datos → modelo de ventas → despliegue de alertas por outlet
SEMMA	Pipeline de modelado iterativo (centrado en desempeño)	Ya se tienen datos y se quiere iterar rápido para mejorar métricas	Contexto de negocio y despliegue operacional	Comparativo de modelos y reporte de evaluación	Probar 3–4 modelos para predecir <code>Item_Outlet_Sales</code> , elegir el mejor por RMSE
KDD	Descubrimiento de conocimiento y patrones interpretables	Extraer patrones/insights con interpretación validada	Gestión del proyecto y despliegue	Patrones/reglas + interpretación	Descubrir qué combinación producto-outlet explica mejor la dispersión de ventas
TDSP	End-to-end con énfasis en ingeniería y trabajo en equipo	El resultado debe vivir como sistema (pipelines, CI/CD, reproducibilidad)	Marco teórico de descubrimiento; más prescriptivo en implementación	Pipelines + artefactos + despliegue versionado	Servicio de predicción de ventas: ingesta → features → entrenamiento → registro → API

Framework	Enfoque principal	¿Cuándo usarlo?	Lo que menos enfatiza	Entregable típico	Aplicado a BigMart
CRISP-ML(Q) (Studer et al. 2021)	Evolución de CRISP-DM con aseguramiento de calidad explícito	El modelo es crítico y necesita monitoreo formal de calidad en cada fase	No es (aún) tan adoptado como CRISP-DM	Checklist de calidad por fase + modelo monitoreado	Definir umbrales de calidad de datos y de desempeño antes de desplegar el modelo de BigMart
ASUM-DM (IBM 2015)	Evolución de CRISP-DM de IBM, con guías de gobierno e IA responsable	La organización ya usa herramientas de IBM o necesita gobierno formal	Es una extensión propietaria, menos documentada públicamente	Artefactos de gobierno + modelo + despliegue	—

30.6.1. Cómo se combinan en un proyecto real

En proyectos reales no es necesario elegir un único marco. Una estrategia común es usar **CRISP-DM** para gobernar el proyecto completo, aplicar **SEMMA** dentro de la fase de modelado para iterar eficientemente, y apoyarse en la lógica de **KDD** cuando el objetivo incluye extraer patrones explicables o reglas interpretables para la directora comercial.

30.7. Los frameworks en la nube: de la metodología a la herramienta

Un framework de proceso dice *qué* hacer en cada fase, pero no *con qué*. En la última década, los tres grandes proveedores de nube —además de plataformas como Databricks— han construido servicios administrados que se alinean, fase por fase, con CRISP-DM y TDSP, reduciendo la fricción de ingeniería que antes exigía cada etapa.

Fase (CRISP-DM / TDSP)	AWS	Azure	Google Cloud	Databricks / código abierto
Comprensión de negocio y datos	Amazon QuickSight, Athena	Power BI, Synapse Analytics	Looker, BigQuery	Databricks SQL, dbt
Preparación de datos	AWS Glue / Glue DataBrew	Azure Data Factory	Dataflow, Dataprep	Apache Spark (Databricks), dbt
Modelado	Amazon SageMaker (Amazon Web Services 2024)	Azure Machine Learning (Microsoft 2024)	Vertex AI (Google Cloud 2024)	Databricks ML + MLflow

Despliegue y monitoreo	SageMaker Endpoints + Model Monitor	Azure ML Endpoints + Model Monitoring	Vertex AI Prediction + Model Monitoring	MLflow Model Registry + Model Serving
------------------------	---	---	---	---

Estas plataformas no reemplazan al framework: **lo operacionalizan**. TDSP, por ejemplo, nació explícitamente pensado para integrarse con Azure Machine Learning (Microsoft 2023): sus plantillas de estructura de proyecto y control de versiones están diseñadas para conectarse directamente con esa plataforma. De forma análoga, si el equipo de BigMart decidiera desplegar el modelo de predicción de ventas como un servicio que se actualiza automáticamente cada mes, necesitaría un registro de modelos, un pipeline de entrenamiento reproducible y un endpoint versionado — exactamente lo que ofrecen SageMaker, Azure ML, Vertex AI o Databricks, independientemente de si el equipo gobierna el proyecto con CRISP-DM o TDSP.

Dos consecuencias prácticas de esta convergencia entre frameworks y nube:

1. **La elección de framework y la elección de plataforma son decisiones distintas.** Un equipo puede seguir fielmente CRISP-DM y desplegar en cualquiera de las cuatro columnas de la tabla anterior; el framework no impone el proveedor.
2. **Las plataformas modernas incorporan piezas que los frameworks clásicos dejaban implícitas**, en particular el monitoreo continuo de *data drift* y desempeño del modelo en producción — la motivación detrás de evoluciones como **CRISP-ML(Q)** (Studer et al. 2021), que añade aseguramiento de calidad explícito en cada fase, y **ASUM-DM** (IBM 2015), la evolución de IBM hacia gobierno de datos e IA responsable.

30.8. Consideraciones éticas y de implementación

En minería de datos, el modelo aprende patrones del pasado; si el pasado está sesgado, el modelo puede amplificar sesgos. Si el histórico de ventas de BigMart refleja, por ejemplo, que ciertos outlets recibieron menos inversión en surtido por decisiones administrativas pasadas —no por falta de demanda real—, un modelo entrenado ciegamente sobre esos datos aprenderá y perpetuará esa asignación desigual. Por eso, además de métricas predictivas, se requiere evaluación de equidad (por grupo o segmento), explicación razonable del modelo y gobernanza de los datos usados.

En implementación, también importa la estabilidad operacional: un modelo “bueno” en validación puede fallar en producción si la distribución de los datos cambia (*data drift*), si el proceso de captura se modifica (un nuevo sistema de punto de venta), o si aparece variación estacional no vista en el entrenamiento. Por ello, el despliegue debe incluir monitoreo continuo: estadísticos de entrada, tasas de error, calibración y alertas, además de un protocolo explícito para reentrenamiento y auditoría — precisamente el vacío que frameworks como CRISP-ML(Q) buscan cerrar formalmente.

30.9. Resumen final

- La **recuperación de datos** responde preguntas con forma conocida (C definida de antemano); la **minería de datos** aprende funciones, reglas o estructuras cuando la forma de la relación es desconocida.
- Una correlación fuerte (por ejemplo, visibilidad vs. ventas en BigMart) no implica causalidad: siempre hay que preguntarse si un confusor no observado explica la asociación. Los diseños experimentales (A/B testing) son la vía más robusta hacia afirmaciones causales.

- Las **tareas centrales** de la minería de datos son clasificación y regresión (supervisadas), clustering (no supervisada), reglas de asociación y detección de anomalías — no todas aplican a cualquier dataset: BigMart, por ejemplo, no tiene la granularidad transaccional que las reglas de asociación requieren.
- **KDD**, **CRISP-DM**, **SEMMA** y **TDSP** son cuatro formas distintas de organizar el trabajo frente a la misma pregunta de negocio: KDD prioriza el descubrimiento interpretable, CRISP-DM gobierna el proyecto completo hasta despliegue, SEMMA acelera la iteración de modelado, y TDSP añade ingeniería de software y trabajo en equipo.
- En la práctica, estos frameworks se **combinan** (CRISP-DM como gobierno, SEMMA/KDD dentro del modelado) en lugar de competir entre sí.
- Las plataformas de nube (AWS SageMaker, Azure ML, Vertex AI, Databricks) **operacionalizan** las fases de estos frameworks, pero la elección de framework y la elección de plataforma son decisiones independientes.
- Evoluciones recientes como **CRISP-ML(Q)** y **ASUM-DM** incorporan explícitamente lo que los frameworks clásicos dejaban implícito: aseguramiento de calidad, monitoreo de drift y gobierno responsable de los modelos en producción.

30.10. Preguntas de repaso

1. ¿Cuál es la diferencia esencial entre una consulta de recuperación de datos y una tarea de minería de datos? Ilustra con un ejemplo de BigMart distinto al usado en el capítulo.
2. Explica por qué una correlación fuerte entre `Item_Visibility` e `Item_Outlet_Sales` no implica que reducir la visibilidad de un producto vaya a aumentar sus ventas. ¿Qué confusor podría explicar la correlación observada?
3. La directora de BigMart pregunta “¿por qué el mismo producto vende distinto en cada tienda?”. Redacta cómo **KDD** y cómo **CRISP-DM** formularían esta pregunta como un problema abordable, y señala al menos una diferencia clave en su enfoque.
4. Da un ejemplo —de BigMart o de otro dominio— de cada una de las cinco tareas centrales de minería de datos: clasificación, regresión, clustering, reglas de asociación y detección de anomalías.
5. ¿Por qué las reglas de asociación no se pueden aplicar directamente al dataset de BigMart tal como se describe en este capítulo? ¿Qué tendría que cambiar en los datos para que sí aplicaran?
6. Compara SEMMA y CRISP-DM: ¿en qué escenario de proyecto elegirías apoyarte más en uno que en el otro?
7. ¿Qué aporta TDSP que CRISP-DM no especifica explícitamente? Menciona al menos dos elementos concretos.
8. Elige una fase de CRISP-DM o TDSP y menciona dos herramientas de nube, de proveedores distintos, que podrían usarse para implementarla.
9. ¿Qué problema de los frameworks clásicos busca resolver CRISP-ML(Q)? ¿Por qué ese problema se volvió más urgente al llevar modelos a producción?
10. Un modelo de BigMart que funcionaba bien en 2013 empieza a fallar en 2015. Menciona dos causas técnicas posibles y qué práctica de monitoreo, de las discutidas en el capítulo, ayudaría a detectarlas a tiempo.

Recursos y lecturas adicionales

Lecturas fundamentales

- Fayyad et al. (1996) — el artículo original que define el proceso KDD y distingue explícitamente la minería de datos (una etapa) del descubrimiento de conocimiento (el proceso completo).
- Chapman et al. (2000) y Shearer (2000) — la guía metodológica original de CRISP-DM y el artículo que la presentó como “el nuevo blueprint” de la industria.
- Han et al. (2011) y Tan et al. (2006) — libros de texto de referencia para las tareas centrales de minería de datos (clasificación, regresión, clustering, reglas de asociación, anomalías).
- Provost y Fawcett (2013) — una introducción muy accesible al pensamiento analítico de negocio detrás de la minería de datos, sin asumir formación matemática previa.
- Pearl (2009) — la referencia canónica sobre inferencia causal y por qué la correlación no basta.
- Kohavi et al. (2020) — guía práctica y rigurosa sobre el diseño de pruebas A/B en producción.

Documentación oficial de frameworks y plataformas

- Microsoft Learn: *Team Data Science Process* — learn.microsoft.com/azure/architecture/data-science-process/overview (Microsoft 2023).
- Amazon SageMaker Documentation — docs.aws.amazon.com/sagemaker (Amazon Web Services 2024).
- Vertex AI Documentation (Google Cloud) — cloud.google.com/vertex-ai/docs (Google Cloud 2024).
- Azure Machine Learning Documentation — learn.microsoft.com/azure/machine-learning (Microsoft 2024).
- Databricks — databricks.com (Databricks 2024), para la perspectiva de *lakehouse* sobre este mismo ciclo.

Videos y cursos

- Canal de YouTube **IBM Technology** (youtube.com/@IBMTechnology): explicaciones cortas y visuales sobre CRISP-DM, ciencia de datos y MLOps.
- Canal de YouTube **StatQuest with Josh Starmer** (youtube.com/@statquest): la mejor referencia en video para repasar los fundamentos estadísticos detrás de clasificación, regresión y clustering antes de las unidades siguientes.
- Coursera: *IBM Data Science Professional Certificate* (coursera.org/professional-certificates/ibm-data-science) — incluye un módulo dedicado a la metodología CRISP-DM aplicada paso a paso.
- Coursera: *Google Data Analytics Professional Certificate* (coursera.org/professional-certificates/google-data-analytics) — buen complemento sobre el ciclo de vida de los datos previo al modelado.

Caso de estudio de esta unidad

- Analytics Vidhya (2016) — el dataset BigMart Sales, usado como hilo conductor en este capítulo y desarrollado en detalle en el resto de la unidad (comprensión de negocio, EDA, preprocesamiento y modelado).

Referencias

Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.

Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.

- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM*

- Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.

- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. [martinfowler.com](http://martinfowler.com/articles/richardsonMaturityModel.html). <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal*

- of *Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem*

- for JAX. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.

- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in

- Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.

[//doi.org/10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432).

Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.

SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.

Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.

Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.

Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.

Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.

Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.

Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.

Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.

Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.

Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.

Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.

Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.

Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.

The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.

- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

31. Business Understanding - Caso BigMart Sales

Un error común en proyectos de analítica y minería de datos es iniciar el análisis directamente sobre la estructura del dataset, sin detenerse antes en el contexto que da sentido a la información disponible. En entornos reales, los datos forman parte de procesos operativos, comerciales o estratégicos, por lo que su exploración debe estar guiada por una comprensión inicial del problema de negocio. En este sentido, el análisis exploratorio de datos (EDA) no solo permite describir variables y detectar patrones, sino también traducir necesidades del negocio en preguntas analíticas investigables.

Por ello, en este ejercicio se utilizará una breve narrativa de negocio como punto de partida para orientar el EDA y establecer:

- quién genera los datos;
- con qué propósito se recolectan;
- qué representa cada observación del conjunto de datos;
- qué relaciones o comportamientos conviene explorar;
- y qué preguntas del negocio pueden examinarse a través del análisis exploratorio.

Escenario ficticio de negocio

Supongamos que **BigMart** es una cadena nacional de tiendas minoristas con presencia en varias ciudades y con distintos formatos de establecimiento. Algunos outlets son supermercados grandes, otros son tiendas de conveniencia y otros funcionan como puntos comerciales de escala intermedia. La empresa comercializa múltiples categorías de productos de consumo frecuente: alimentos, bebidas, productos de hogar, artículos perecederos y no perecederos, entre otros.

Durante 2013, la dirección comercial de BigMart detectó que el desempeño en ventas no era homogéneo entre productos ni entre tiendas. Algunos artículos tenían una rotación muy alta en ciertos outlets, pero un comportamiento mediocre en otros. Del mismo modo, tiendas con características aparentemente similares mostraban resultados muy distintos en ingresos por producto.

La empresa contaba con sistemas operativos que registraban transacciones, atributos del catálogo de productos y características básicas de los establecimientos. Sin embargo, esa información se utilizaba sobre todo para fines administrativos y no se había integrado de forma sistemática para apoyar decisiones analíticas.

Ante ello, la dirección regional decidió solicitar al área de analítica un estudio con dos propósitos iniciales:

1. **entender mejor la estructura de las ventas** a nivel producto-tienda;
2. **identificar factores asociados con un mayor desempeño comercial.**

Este encargo es el origen del dataset con el que trabajaremos.

Situación problemática

La preocupación de la empresa no surge de manera abstracta. En este escenario ficticio, BigMart enfrenta cuatro tensiones operativas:

- **Planeación de inventario** No siempre resulta claro cuánto surtido asignar a cada tipo de producto en cada outlet. Una mala estimación puede producir sobreinventario o desabasto.
- **Diferencias entre tiendas** Outlets de distinto tamaño, ubicación o formato comercial parecen comportarse de manera desigual. La empresa necesita saber si esas diferencias son consistentes y medibles.
- **Diseño de promociones y surtido** La gerencia comercial quiere distinguir qué tipos de producto tienen mejor desempeño y si ciertas características, como categoría o precio de lista, parecen asociarse con mayores ventas.
- **Falta de evidencia integrada** Los datos existen, pero todavía no se han convertido en evidencia útil para la toma de decisiones. Se requiere transformar registros operativos en conocimiento analítico.

En consecuencia, el problema inicial de negocio puede formularse así:

BigMart necesita comprender cómo se distribuyen y de qué dependen las ventas por producto y por outlet, para mejorar decisiones de surtido, operación comercial y planeación.

Diccionario de Variables

En la siguiente tabla se detalla la naturaleza y la interpretación de negocio de cada una de las variables presentes en el conjunto de datos.

Tabla 31.1.: Diccionario de variables del dataset de ventas

Categoría	Variable	Descripción e Interpretación de Negocio
Producto	<code>Item_Identifier</code>	Identificador único del producto. Permite el rastreo y vinculación con otras fuentes de información.
Producto	<code>Item_Weight</code>	Peso del producto. Relevante para la logística, el empaque y la presentación comercial.
Producto	<code>Item_Fat_Content</code>	Clasificación del contenido graso. Útil para segmentar subgrupos de alimentos y bebidas.
Producto	<code>Item_Visibility</code>	Porcentaje de exposición en anaquel. Indica la prominencia comercial del artículo en la tienda.
Producto	<code>Item_Type</code>	Categoría general del producto. Clave para analizar diferencias estructurales entre líneas de negocio.

Categoría	Variable	Descripción e Interpretación de Negocio
Producto	Item_MRP	Precio máximo de venta (precio de lista). Variable principal para explicar variaciones en ventas monetarias.
Outlet	Outlet_Identifier	Identificador único del establecimiento. Permite estudiar el desempeño comparativo entre tiendas.
Outlet	Outlet_Establishment_Year	Año de apertura. Funciona como indicador de antigüedad y madurez operativa de la tienda.
Outlet	Outlet_Size	Tamaño del local. Se asocia con la capacidad de surtido, tráfico de clientes y volumen de exhibición.
Outlet	Outlet_Location_Type	Tipo de ubicación geográfica. Resume el entorno socioeconómico y el perfil de la demanda.
Outlet	Outlet_Type	Formato comercial (ej. Supermercado, Tienda de conveniencia). Define la lógica operativa del punto de venta.
Resultado	Item_Outlet_Sales	Variable Objetivo. Ventas totales del producto en la tienda. Es el eje central del análisis de desempeño comercial.

Necesidades del negocio que motivan el análisis

En la práctica, las preguntas analíticas actúan como una guía para seleccionar las herramientas del EDA. Para esto es necesario traducir las preguntas no técnicas de nuestro cliente en preguntas que, como equipo de análisis sabemos que os permitirán realizar un mejor trabajo, en este ejemplo tras una conversación con BigMart se plantean varias necesidades.

Necesidad 1: entender el comportamiento global de las ventas

“Antes de entrar en detalles, queremos entender cómo se están comportando las ventas en general. ¿Normalmente se mantienen en un rango parecido o hay mucha variación? ¿Existen casos excepcionalmente altos o bajos que debamos revisar?”

Esta es una forma típica en la que un cliente o la dirección expresa una inquietud de negocio. Aunque no utiliza lenguaje estadístico, como equipo de analítica podemos traducirla en preguntas exploratorias más concretas:

- ¿cuál es el nivel típico de ventas por combinación producto-outlet?

- ¿la distribución de ventas es homogénea o presenta alta dispersión?
- ¿hay valores extremos particularmente altos o bajos?

Desde la perspectiva del EDA, estas preguntas orientan el estudio de la variable de ventas mediante **medidas de localización, variabilidad y forma**, así como la identificación de posibles valores atípicos.

Necesidad 2: comparar desempeño entre segmentos

“Queremos saber si algunos tipos de productos o algunas tiendas están funcionando mejor que otros. En particular, nos interesa ver si el desempeño cambia según la categoría del producto, el tipo de outlet, el tamaño de la tienda o su ubicación.”

Esta es una inquietud típica del cliente cuando busca identificar diferencias relevantes entre grupos. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas de análisis exploratorio como:

- ¿hay categorías de producto que venden sistemáticamente más?
- ¿ciertos tipos de outlet concentran mejores resultados?
- ¿las ventas cambian según tamaño o localización de la tienda?

Estas preguntas orientan un análisis de **heterogeneidad entre subgrupos**, mediante comparaciones entre categorías y segmentos relevantes del negocio.

Necesidad 3: identificar concentración comercial

“Nos interesa entender si el negocio está bien diversificado o si una parte importante de las ventas depende de muy pocas categorías o de muy pocos outlets.”

Esta es una preocupación estratégica habitual en contextos comerciales, ya que permite evaluar qué tan diversificada o dependiente es la estructura de ingresos. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas exploratorias como:

- ¿unas pocas categorías explican gran parte del ingreso?
- ¿hay outlets particularmente dominantes?
- ¿la estructura comercial está diversificada o concentrada?

Estas preguntas orientan un análisis de **concentración**, enfocado en identificar si las ventas se distribuyen de manera equilibrada o si están fuertemente concentradas en un conjunto reducido de categorías o establecimientos.

Necesidad 4: explorar asociaciones entre atributos y ventas

“Queremos una primera idea de qué variables podrían estar más relacionadas con las ventas, para entender mejor el negocio antes de avanzar hacia análisis predictivos.”

Esta es una inquietud frecuente en etapas tempranas del análisis, cuando el cliente busca una primera comprensión de los factores que podrían estar vinculados con las ventas. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas exploratorias como:

- ¿el precio de lista está asociado con mayores ventas?

- ¿la visibilidad parece relacionarse con el desempeño?
- ¿algunas variables del outlet muestran asociación con mejores resultados?

Estas preguntas orientan el estudio de **correlación** y de **relaciones bivariadas**, con el fin de detectar patrones iniciales entre atributos y ventas que luego puedan examinarse con mayor profundidad.

Del problema de negocio al Data Understanding

La transición entre Business Understanding y Data Understanding ocurre cuando las preguntas del negocio se convierten en preguntas sobre los datos disponibles. A partir del escenario anterior, como equipo analítico debemos formular preguntas como las siguientes:

Sobre estructura general del dataset

- ¿cuántos registros hay?
- ¿cuántas variables numéricas y categóricas existen?
- ¿hay faltantes, inconsistencias o codificaciones problemáticas?

Sobre la variable de ventas

- ¿cómo se distribuye `Item_Outlet_Sales`?
- ¿qué tan sesgada está?
- ¿qué tan dispersa es respecto a su centro?

Sobre variables categóricas

- ¿cuántas categorías de producto existen?
- ¿cómo se distribuyen los registros entre tipos de outlet?
- ¿hay clases raras o escasamente representadas?

Sobre calidad de datos

- ¿existen valores faltantes en peso o tamaño del outlet?
- ¿hay categorías inconsistentes, como variantes textuales de una misma clase?
- ¿hay variables que realmente son identificadores y no rasgos analíticos?

Estas preguntas ya no pertenecen solo al negocio: son preguntas de **Data Understanding** y abren el camino al EDA.

Una vez que se ha establecido el escenario de negocio, la unidad de análisis, el diccionario de variables y las necesidades comerciales, es posible formular preguntas concretas que el EDA deberá responder. Estas preguntas constituyen el puente natural entre la fase de Business Understanding y la fase de Data Understanding.

Preguntas sobre ventas

- ¿Cuál es el nivel típico de `Item_Outlet_Sales`?
- ¿Qué tan alejadas están las ventas de ese nivel típico?
- ¿La distribución es simétrica o presenta cola derecha?

Preguntas sobre productos

- ¿Qué categorías de producto aparecen con mayor frecuencia?
- ¿Qué categorías concentran mayores ventas?
- ¿El precio del producto parece jugar un papel importante?

Preguntas sobre outlets

- ¿Existen diferencias notables entre tipos de outlet?
- ¿El tamaño del outlet parece importar?
- ¿La antigüedad del outlet podría estar asociada con su desempeño?

Preguntas sobre calidad y preparación de datos

- ¿Qué variables presentan faltantes?
- ¿Qué categorías deben homogenizarse?
- ¿Qué transformaciones podrían ser necesarias antes de modelar?

Estas preguntas constituyen el puente natural hacia el Data Understanding y el EDA.

Parte X.

Análisis Exploratorio de Datos (EDA)

Parte XI.

Fundamentos del EDA

32. Fundamentos del análisis exploratorio de datos

32.1. Introducción

El análisis exploratorio de datos (EDA) es el proceso iterativo mediante el cual se conoce un conjunto de datos antes de construir modelos o comunicar resultados definitivos. Combina resúmenes numéricos y visuales para descubrir estructura, evaluar calidad y formular preguntas más precisas (Tukey 1977; Heckert et al. 2012).

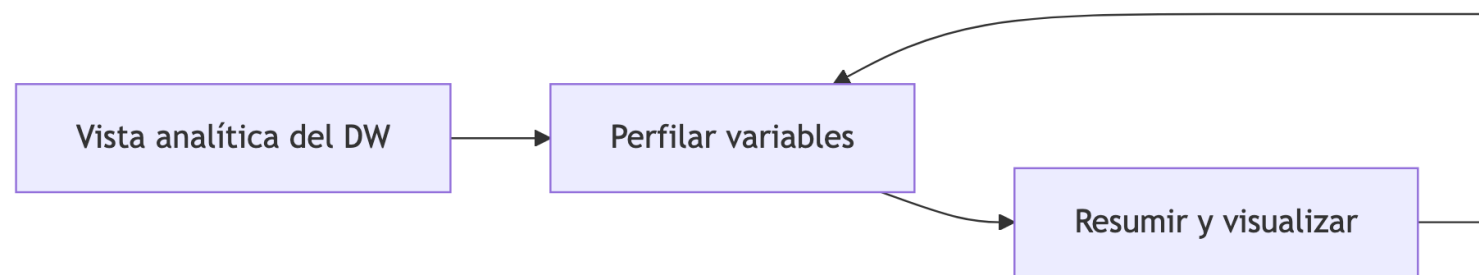
Dentro de CRISP-DM, el EDA forma parte del entendimiento de los datos y mantiene una relación iterativa con su preparación. Un hallazgo exploratorio puede exigir corregir una variable; después de corregirla, la distribución debe examinarse nuevamente.

Definición. El EDA es una estrategia de investigación que utiliza estadística descriptiva y visualización para reconocer patrones, anomalías, relaciones y limitaciones antes de elegir un procedimiento de modelado.

32.2. Preguntas que orientan el EDA

- ¿Cuál es la unidad de observación y cuántos registros existen?
- ¿Qué significa cada variable y en qué escala fue medida?
- ¿Qué valores faltan y por qué podrían faltar?
- ¿Qué valores son imposibles, inconsistentes o duplicados?
- ¿Dónde se localizan los valores y cuánto varían?
- ¿Qué forma tienen las distribuciones?
- ¿Existen diferencias entre grupos o concentración del total?
- ¿Qué asociaciones aparecen entre variables?

```
flowchart LR
  A["Vista analítica del DW"] --> B["Perfilar variables"]
  B --> C["Resumir y visualizar"]
  C --> D["Formular hallazgos"]
  D --> E["Decidir tratamiento"]
  E --> B
  D --> F["Preparar modelado"]
```



La salida del EDA no es una colección de gráficos. Es una bitácora de hallazgos, evidencia, decisiones y preguntas pendientes.

32.3. Unidad de observación y granularidad

Antes de calcular cualquier estadístico debe identificarse qué representa una fila: una venta, un producto por tienda, un cliente, una visita o una medición temporal. Mezclar granularidades produce duplicaciones y resúmenes engañosos.

Ejemplo. Si una tabla contiene una fila por producto y outlet, sumar ventas puede ser válido; contar productos sin distinguir el outlet puede contar el mismo producto varias veces.

32.4. Tipos de variables

32.4.1. Variables cualitativas

- **Nominales:** categorías sin orden intrínseco, como tipo de outlet o método de pago.
- **Ordinales:** niveles ordenados sin distancias métricas garantizadas, como satisfacción baja, media y alta.

32.4.2. Variables cuantitativas

- **Discretas:** conteos con valores separados, como número de compras.
- **Continuas:** mediciones en un intervalo, como peso, ingreso o duración.

Tipo	Resúmenes adecuados	Visualizaciones iniciales
Nominal	Frecuencias, proporciones, moda, diversidad	Barras
Ordinal	Frecuencias ordenadas, mediana, cuantiles	Barras ordenadas, acumulada
Discreta	Frecuencias, centro, dispersión	Barras o histograma entero
Continua	Centro, dispersión, forma	Histograma, ECDF, boxplot

El tipo almacenado no siempre coincide con el tipo estadístico. Un código postal numérico sigue siendo nominal y una fecha guardada como texto sigue representando tiempo.

32.5. Frecuencias y proporciones

Para valores $v_1, \dots, v_K$, la frecuencia absoluta y la proporción son

$$f_k = \#\{i : x_i = v_k\}, \quad p_k = \frac{f_k}{n}, \quad \sum_{k=1}^K p_k = 1. \quad (1)$$

La colección $(p_1, \dots, p_K)$ es la distribución empírica de una variable categórica o discreta.

32.6. Frecuencias acumuladas y ECDF

Cuando existe un orden natural,

$$F_k = \sum_{j \leq k} f_j, \quad P_k = \sum_{j \leq k} p_j. \quad (2)$$

Para una variable cuantitativa, la función de distribución empírica es

$$\widehat{F}_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(x_i \leq x). \quad (3)$$

Definición. La ECDF indica la proporción observada de datos menores o iguales que cada umbral. A diferencia del histograma, no requiere elegir intervalos.

32.7. Ejemplo de calidad categórica

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

resultado = pd.Series([
    "Asistió", "No asistió", "ASISTÍÓ", "Canceló", "Asistió",
    "NA", "No asistió", "No asistió", None, "Asistió",
    "Canceló", "ASISTÍÓ", "No asistió", "NA", "Asistió"
], name="resultado_cita")

conteos = resultado.fillna("<faltante>").value_counts().rename_axis("Categoría").reset_index(name="conteo")
```

```
conteos["Proporción"] = conteos["Frecuencia"]/conteos["Frecuencia"].sum()
fig = px.bar(conteos, x="Categoría", y="Frecuencia", color="Categoría", text="Frecuencia",
             title="Frecuencias que revelan duplicados semánticos y faltantes")
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Hallazgo. Asistió y ASISTIÓ, así como No asistió y No asistió, son duplicados semánticos. NA y <faltante> podrían representar la misma ausencia, pero debe confirmarse con el diccionario de datos antes de recodificar.

32.8. Ejemplo discreto y acumulado

```
rng = np.random.default_rng(2026)
espera = np.r_[rng.poisson(12, 600), [-4, -1, 95, 120]]
tabla = pd.Series(espera).value_counts().sort_index().rename_axis("Días").reset_index(name="Frecuencia")
tabla["Acumulada"] = tabla["Frecuencia"].cumsum()/tabla["Frecuencia"].sum()

fig = make_subplots(specs=[[{"secondary_y": True}]]))
fig.add_bar(x=tabla["Días"], y=tabla["Frecuencia"], name="Frecuencia", secondary_y=False)
fig.add_scatter(x=tabla["Días"], y=tabla["Acumulada"], mode="lines+markers",
               name="Proporción acumulada", secondary_y=True)
fig.update_yaxes(title_text="Frecuencia", secondary_y=False)
fig.update_yaxes(title_text="Proporción acumulada", tickformat=".0%", secondary_y=True)
fig.update_layout(title="Tiempo de espera: distribución y acumulación")
fig.show()
```

Unable to display output for mime type(s): text/html

Los tiempos negativos son imposibles bajo la definición habitual y requieren revisión. Los valores de 95 y 120 días son extremos, pero podrían ser casos reales; no deben eliminarse sin evidencia.

32.9. Datos faltantes

El porcentaje faltante debe analizarse por variable y por grupos relevantes. Un promedio global puede ocultar que la ausencia se concentra en una fuente, fecha o categoría. La decisión de imputar pertenece al preprocesamiento, pero el EDA debe documentar patrón, magnitud y posibles mecanismos.

32.10. Valores atípicos y anomalías

Un valor atípico es una observación alejada según una regla estadística; una anomalía es un caso incompatible con el proceso esperado. No son sinónimos de error.

Evidencia	Pregunta	Acción posible
Valor imposible	¿Contradice el dominio?	Corregir, marcar o excluir con trazabilidad
Valor raro pero posible	¿Es un segmento real?	Conservar y analizar por separado
Cola extensa	¿La escala es adecuada?	Transformar o usar medidas robustas
Categoría infrecuente	¿Es error o nivel real?	Estandarizar o agrupar con criterio

32.11. Ruta descriptiva del libro

Los siguientes capítulos desarrollan seis perspectivas complementarias:

1. Medidas de localización: ¿dónde están los valores?
2. Medidas de variabilidad: ¿cuánto cambian?
3. Medidas de forma: ¿cómo son sus colas y simetría?
4. Medidas de heterogeneidad: ¿cambian entre categorías o grupos?
5. Medidas de concentración: ¿quién acumula el total?
6. Medidas de correlación: ¿cómo se asocian dos variables?

Cada capítulo teórico está seguido por una práctica reproducible con BigMart.

32.12. Checklist de salida

Un EDA reproducible debe dejar: diccionario y granularidad, perfil de faltantes, dominios validados, resúmenes con unidades, visualizaciones pertinentes, hallazgos redactados, decisiones justificadas y preguntas pendientes.

32.13. Ejercicios

1. Clasifique cada variable de BigMart por escala estadística y justifique los casos ambiguos.
2. Proponga una regla de validación para tres columnas del dataset.
3. Compare qué información aporta un histograma frente a una ECDF.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.

- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. [martinfowler.com](http://martinfowler.com/articles/richardsonMaturityModel.html). <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best

- Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data*

- Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbj1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine

- Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte XII.

Medidas de localización

33. Medidas de localización

33.1. Introducción

Las medidas de localización resumen la región donde se sitúa una distribución. No existe un único “centro” universal: la media utiliza todas las magnitudes, la mediana utiliza el orden y la moda utiliza la frecuencia. La elección debe responder al tipo de variable, a la forma de la distribución y al propósito del análisis (Tukey 1977; Freedman et al. 2007).

Definición. Una medida de localización es un estadístico que identifica una posición representativa de la distribución. Debe interpretarse junto con una medida de variabilidad y una gráfica de la distribución.

33.2. Media aritmética

Para una muestra $x_1, \dots, x_n$, la media es

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (1)$$

La media es el punto que hace cero la suma de desviaciones y minimiza la suma de errores cuadrados. También es equivariante ante transformaciones afines: $a\bar{X} + b = \overline{aX + b}$. Su desventaja es una sensibilidad elevada a valores extremos (Huber y Ronchetti 2009).

Ejemplo. Para $x = (2, 3, 3, 4, 8)$, $\bar{x} = 4$. Si 8 se reemplaza por 80, la media aumenta a 18.4, aunque solo cambió una observación.

33.3. Media ponderada y media recortada

Cuando cada observación representa una cantidad diferente, se usa

$$\bar{x}_w = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad w_i \geq 0. \quad (2)$$

La media recortada elimina la misma proporción de observaciones en ambas colas antes de promediar. Es un compromiso entre eficiencia y robustez; debe declararse siempre el porcentaje recortado (Wilcox 2012).

33.4. Mediana

Si $x_{(1)} \leq \dots \leq x_{(n)}$ son los datos ordenados, la mediana es el cuantil 0.5.

$$\tilde{x} = \begin{cases} x_{((n+1)/2)}, & n \text{ impar}, \\ \frac{x_{(n/2)} + x_{(n/2+1)}}{2}, & n \text{ par}. \end{cases} \quad (3)$$

Interpretación. La mediana divide la muestra ordenada en dos mitades. Es preferible a la media para ingresos, tiempos y ventas muy sesgadas, porque su punto de ruptura es mucho mayor.

33.5. Moda

Para una variable discreta o categórica,

$$\text{Moda}(X) = \arg \max_v f(v). \quad (4)$$

Puede no ser única. En variables continuas, la “moda” calculada con un histograma depende del ancho y del origen de los intervalos; conviene describirla como *clase modal* o estimarla mediante densidad.

No debe calcularse una media de códigos nominales. Si rojo=1, azul=2 y verde=3, el promedio de los códigos no posee interpretación métrica.

33.6. Cuantiles y percentiles

El cuantil $Q(p)$ es un valor que deja aproximadamente una proporción p de observaciones por debajo. Los percentiles son cuantiles expresados de 0 a 100. Existen varios métodos de interpolación muestral; por eso conviene documentar el método del software cuando el tamaño de muestra es pequeño (Hyndman y Fan 1996).

33.7. Cálculo manual paso a paso

Usaremos el conjunto pequeño y ya ordenado

$$x = (2, 3, 3, 5, 7, 8, 9, 11), \quad n = 8.$$

33.7.1. Media aritmética

Paso 1. Sumar todos los valores:

$$2 + 3 + 3 + 5 + 7 + 8 + 9 + 11 = 48.$$

Paso 2. Dividir entre el número de observaciones:

$$\bar{x} = \frac{48}{8} = 6.$$

Por tanto, el centro de gravedad de los datos es 6.

33.7.2. Mediana

Paso 1. Ordenar los datos. Ya están ordenados.

Paso 2. Localizar las posiciones centrales. Como $n = 8$ es par, son las posiciones 4 y 5:

$$x_{(4)} = 5, \quad x_{(5)} = 7.$$

Paso 3. Promediar los dos valores centrales:

$$\tilde{x} = \frac{5 + 7}{2} = \frac{12}{2} = 6.$$

33.7.3. Moda

Contamos cuántas veces aparece cada valor:

$$f(2) = 1, \quad f(3) = 2, \quad f(5) = 1, \quad f(7) = 1, \quad f(8) = 1, \quad f(9) = 1, \quad f(11) = 1.$$

La frecuencia mayor es 2 y corresponde al valor 3. Entonces

$$\text{Moda}(x) = 3.$$

33.7.4. Cuartiles

Para este ejemplo usaremos el método de las medianas de las dos mitades. Otros programas pueden interpolar de otra manera (Hyndman y Fan 1996).

Paso 1. Separar las mitades:

$$L = (2, 3, 3, 5), \quad U = (7, 8, 9, 11).$$

Paso 2. Calcular Q_1 como la mediana de L :

$$Q_1 = \frac{3 + 3}{2} = 3.$$

Paso 3. Calcular Q_3 como la mediana de U :

$$Q_3 = \frac{8 + 9}{2} = 8.5.$$

Así, 25 % de la muestra queda aproximadamente por debajo de 3 y 75 % por debajo de 8.5.

33.7.5. Media recortada

Recortaremos una observación de cada extremo, equivalente a $1/8 = 12.5\%$ por cola.

Paso 1. Retirar el mínimo y el máximo:

$$(2, 3, 3, 5, 7, 8, 9, 11) \rightarrow (3, 3, 5, 7, 8, 9).$$

Paso 2. Sumar los valores restantes:

$$3 + 3 + 5 + 7 + 8 + 9 = 35.$$

Paso 3. Dividir entre los seis valores conservados:

$$\bar{x}_{\text{rec}} = \frac{35}{6} = 5.8333.$$

33.7.6. Media ponderada

Supongamos precios 10, 20 y 30, comprados en cantidades 2, 3 y 1.

Paso 1. Multiplicar cada valor por su peso:

$$10(2) = 20, \quad 20(3) = 60, \quad 30(1) = 30.$$

Paso 2. Sumar los productos y los pesos:

$$20 + 60 + 30 = 110, \quad 2 + 3 + 1 = 6.$$

Paso 3. Dividir:

$$\bar{x}_w = \frac{110}{6} = 18.3333.$$

El resultado no es $(10 + 20 + 30)/3 = 20$ porque el precio 20 aparece con mayor cantidad.

Del cálculo al programa. Un algoritmo debe: recibir los datos, contar n , acumular la suma para la media, ordenar para mediana y cuantiles, construir una tabla de frecuencias para la moda y, cuando existan pesos, acumular simultáneamente $\sum w_i x_i$ y $\sum w_i$.

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

rng = np.random.default_rng(42)
base = rng.lognormal(mean=4.5, sigma=0.42, size=700)
extremos = rng.lognormal(mean=7.0, sigma=0.25, size=12)
compras = np.concatenate([base, extremos])

media = compras.mean()
```



```

mediana = np.median(compras)
recortada = np.mean(np.sort(compras)[36:-36])

fig = px.histogram(
    x=compras, nbins=55, marginal="box",
    labels={"x": "Monto de compra (MXN)"},
    title="Media, mediana y media recortada ante una cola derecha"
)
for valor, nombre, color in [
    (media, "Media", "#D62728"),
    (mediana, "Mediana", "#1F77B4"),
    (recortada, "Media recortada 10%", "#2CA02C")
]:
    fig.add_vline(x=valor, line_width=3, line_dash="dash", line_color=color,
                  annotation_text=f"{nombre}: {valor:,.0f}")
fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Lectura de la gráfica. La media se desplaza hacia la cola de montos altos. La mediana y la media recortada describen mejor una compra habitual, mientras que la media conserva utilidad para estimar ingreso total por operación.

33.8. Cuantiles como función de la distribución

```

x_ord = np.sort(compras)
ecdf = np.arange(1, len(x_ord) + 1) / len(x_ord)
fig = go.Figure(go.Scatter(x=x_ord, y=ecdf, mode="lines", line_shape="hv",
                           hovertemplate="Monto={x:,.0f}<br>F(x)={y:.3f}<extra></extra>"))
for p, color in [(0.25, "#2CA02C"), (0.50, "#1F77B4"), (0.90, "#FF7F0E")]:
    q = np.quantile(compras, p)
    fig.add_hline(y=p, line_dash="dot", line_color=color)
    fig.add_vline(x=q, line_dash="dot", line_color=color,
                  annotation_text=f"P{int(100*p)}={q:,.0f}")
fig.update_layout(title="ECDF interactiva para leer cuantiles",
                  xaxis_title="Monto de compra (MXN)", yaxis_title="Proporción acumulada")
fig.show()

```

Unable to display output for mime type(s): text/html

33.9. Selección de la medida

Situación	Medida principal	Complemento
Distribución aproximadamente simétrica	Media	Mediana y desviación estándar
Distribución sesgada o con extremos	Mediana	IQR y media recortada
Variable ordinal	Mediana y cuantiles	Moda y frecuencias
Variable nominal	Moda	Proporciones y entropía
Observaciones con pesos distintos	Media ponderada	Distribución de pesos

Media y mediana cercanas no demuestran normalidad. Una distribución multimodal o simétrica con colas pesadas puede producir ambas medidas casi iguales.

33.10. Puente al caso BigMart

El capítulo Medidas de localización en Big Mart Sales aplica estas decisiones a `Item_Weight`, `Item_Visibility`, `Item_MRP` e `Item_Outlet_Sales`.

33.11. Ejercicios

1. Construya un conjunto donde media y mediana coincidan, pero la distribución sea bimodal.
2. Compare media, mediana y media recortada antes y después de agregar dos valores extremos.
3. Explique qué medida usaría para informar el precio típico y cuál para proyectar ingreso total.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.

- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Calínski, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.

33. Medidas de localización

- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for

- discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di*

33. Medidas de localización

Cagliari 3: 1-158.

Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.

Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.

GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.

Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.

Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.

Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.

Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.

Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.

Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.

Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.

Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.

Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8):

1735-80.

- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.

- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.

- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness,

- Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.

- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in*

Neural Information Processing Systems (NeurIPS) 30.

Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.

Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.

Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.

Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.

Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

34. Medidas de localización en Big Mart Sales

Este capítulo aplica los conceptos desarrollados en Medidas de localización. Conviene revisar primero las diferencias entre media, mediana, moda y cuantiles.

Desde la perspectiva del análisis exploratorio de datos, las medidas de localización responden a una pregunta fundamental: **¿en qué región de valores tiende a concentrarse el fenómeno observado?**. En términos prácticos, esto permite identificar valores típicos, comparar distribuciones y comenzar a interpretar el comportamiento general del sistema antes de pasar a medidas de dispersión, forma o segmentación.

El dataset incluye variables numéricas como `Item_Weight`, `Item_Visibility`, `Item_MRP` y `Item_Outlet_Sales`, junto con variables categóricas asociadas a tipo de producto, establecimiento y ubicación del outlet. Para este primer análisis nos enfocaremos en el comportamiento univariado de las variables numéricas, usando medidas como la **media**, la **mediana**, la **moda** y algunos **cuantiles**.

Definición operativa. En esta práctica, una medida de localización resume un valor típico o una posición relevante de una variable numérica; ninguna medida se interpreta sin observar también su distribución (Tukey 1977; Hyndman y Fan 1996).

34.1. Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

```
import pandas as pd
import numpy as np
import plotly.express as px

DATA_PATH = "../../data/bigmart_sales.csv"

df = pd.read_csv(DATA_PATH)
df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	C
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	C
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	C
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	C
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	C

34.2. Inspección inicial

Antes de calcular medidas de localización conviene identificar el tipo de variables disponibles y confirmar qué columnas numéricas serán analizadas.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Item_Identifier                       8523 non-null   object
1   Item_Weight                           7060 non-null   float64
2   Item_Fat_Content                      8523 non-null   object
3   Item_Visibility                       8523 non-null   float64
4   Item_Type                             8523 non-null   object
5   Item_MRP                             8523 non-null   float64
6   Outlet_Identifier                     8523 non-null   object
7   Outlet_Establishment_Year             8523 non-null   int64
8   Outlet_Size                           6113 non-null   object
9   Outlet_Location_Type                  8523 non-null   object
10  Outlet_Type                           8523 non-null   object
11  Item_Outlet_Sales                     8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

```
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()

num_cols, cat_cols[:10]
```

```
(['Item_Weight',
  'Item_Visibility',
  'Item_MRP',
  'Outlet_Establishment_Year',
  'Item_Outlet_Sales'],
 ['Item_Identifier',
  'Item_Fat_Content',
  'Item_Type',
  'Outlet_Identifier',
  'Outlet_Size',
  'Outlet_Location_Type',
  'Outlet_Type'])
```

En este punto es útil destacar que no todas las variables numéricas tienen el mismo significado. Por ejemplo, `Item_MRP` representa el precio máximo sugerido del producto, mientras que `Item_Outlet_Sales` corresponde a las ventas del producto en un outlet particular. La interpretación del “centro” de cada variable, por tanto, depende del contexto económico de la medición.

34.3. Selección de variables para el análisis de localización

Para esta etapa trabajaremos con un subconjunto de variables numéricas especialmente interpretables:

- Item_Weight
- Item_Visibility
- Item_MRP
- Item_Outlet_Sales

Crearemos un nuevo dataframe con estas columnas para facilitar el análisis, identificando además el número de datos faltantes en las mismas.

```
vars_localizacion = [
    "Item_Weight",
    "Item_Visibility",
    "Item_MRP",
    "Item_Outlet_Sales",
]

missing_report = df[vars_localizacion].isna().sum().to_frame("missing")
missing_report["missing_pct"] = 100 * missing_report["missing"] / len(df)
missing_report
```

	missing	missing_pct
Item_Weight	1463	17.165317
Item_Visibility	0	0.000000
Item_MRP	0	0.000000
Item_Outlet_Sales	0	0.000000

34.3.1. Datos faltantes y su impacto en el análisis de localización

La presencia de valores faltantes en la variable `Item_Weight` no impide comenzar el análisis descriptivo de localización, **sin embargo, eliminarlos sin analizar su origen puede introducir sesgos en los resultados**. Por ello, antes de calcular medidas descriptivas es necesario examinar la magnitud y el posible patrón de los datos ausentes. Extrayendo un dataframe con los casos faltantes en `Item_Weight` tenemos:

```
missing_weight = df[df["Item_Weight"].isna()]
missing_weight.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet
7	FDP10	NaN	Low Fat	0.127470	Snack Foods	107.7622	OUT02
18	DRI11	NaN	Low Fat	0.034238	Hard Drinks	113.2834	OUT02
21	FDW12	NaN	Regular	0.035400	Baking Goods	144.5444	OUT02
23	FDC37	NaN	Low Fat	0.057557	Baking Goods	107.6938	OUT01
29	FDC14	NaN	Regular	0.072222	Canned	43.6454	OUT01

Si bien no es el objetivo de esta sección resolver el problema de los datos faltantes, es importante tener una noción clara de su impacto. Para esto existen tres preguntas clave que podemos hacernos en estos casos:

1. ¿Qué porcentaje de datos faltantes hay en la variable?

Este paso permite dimensionar la magnitud del problema. Calcular la proporción de valores ausentes respecto al total de observaciones ayuda a evaluar si los faltantes representan una fracción pequeña del dataset o si son lo suficientemente numerosos como para afectar la representatividad del análisis.

```
print(f"Porcentaje de datos faltantes en Item_Weight: {missing_report.loc['Item_Weight', 'missing']}
```

Porcentaje de datos faltantes en Item_Weight: 17.17%

En nuestro caso la variable presenta aproximadamente un 17 % de valores faltantes, lo que implica que cerca de una sexta parte de las observaciones no contiene información para este atributo. Esta proporción no es despreciable, por lo que antes de excluir estos registros conviene examinar si los datos faltantes siguen algún patrón sistemático y evaluar el posible impacto de su eliminación en el análisis exploratorio.

2. ¿Existen patrones de ausencia relacionados con otras variables?

Aquí se busca identificar si los datos faltantes aparecen de manera aleatoria o si se concentran en ciertos grupos (por ejemplo, en determinadas categorías de producto, tipos de outlet o rangos de precio). Detectar patrones de ausencia es importante porque podría indicar un posible sesgo en los datos.

Analicemos el promedio de valores faltantes en Item_Weight por tipo de producto y tipo de outlet:

```
df['Item_Weight'].isna().groupby(df['Item_Type']).mean()
```

```
Item_Type
Baking Goods      0.172840
Breads            0.187251
Breakfast         0.190909
Canned            0.169492
Dairy             0.170088
Frozen Foods      0.161215
Fruits and Vegetables 0.172890
Hard Drinks       0.144860
Health and Hygiene 0.173077
Household         0.165934
Meat              0.207059
Others            0.189349
Seafood           0.203125
Snack Foods       0.176667
Soft Drinks       0.159551
Starchy Foods     0.121622
Name: Item_Weight, dtype: float64
```

```
df['Item_Weight'].isna().groupby(df['Outlet_Type']).mean()
```



```

Outlet_Type
Grocery Store      0.487535
Supermarket Type1  0.000000
Supermarket Type2  0.000000
Supermarket Type3  1.000000
Name: Item_Weight, dtype: float64

```

Podemos observar como la falta de datos en `Item_Weight` se concentra principalmente en 2 tipos de outlets: `Grocery Store` y `Supermarket Type1`. Esto sugiere que la ausencia de información no es aleatoria en estos casos, mientras que tenemos una distribución más equilibrada en `Item_Type`.

Este patrón de ausencia podría estar relacionado con la naturaleza de los productos comercializados en esos outlets o con prácticas de registro de datos específicas, lo que implica que eliminar estos casos sin un análisis previo podría sesgar el análisis descriptivo.

```

import pandas as pd
import plotly.express as px

# Proporción de missing de Item_Weight por Outlet_Type
missing_outlet = (
    df['Item_Weight']
    .isna()
    .groupby(df['Outlet_Type'])
    .mean()
    .mul(100)
    .reset_index(name='pct_missing')
    .sort_values('pct_missing', ascending=False)
)

fig = px.bar(
    missing_outlet,
    x='Outlet_Type',
    y='pct_missing',
    color='pct_missing',
    color_continuous_scale='Reds',
    text='pct_missing',
    labels={
        'Outlet_Type': 'Tipo de outlet',
        'pct_missing': '% de faltantes en Item_Weight'
    },
    title='Porcentaje de valores faltantes de Item_Weight por Outlet_Type'
)

fig.update_traces(
    texttemplate='%{text:.1f}%',
    textposition='outside'
)

fig.update_layout(
    yaxis_title='% de faltantes',

```

```

    xaxis_title='Tipo de outlet',
    coloraxis_colorbar_title='% faltante'
)

fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

3. ¿Cuál es el impacto potencial de eliminar casos con datos faltantes en el análisis?

En este punto, la pregunta clave ya no es solo si existen datos faltantes, sino si su eliminación cambia nuestra lectura general de los datos. Por ello, una forma natural de continuar el EDA es comparar medidas de localización antes y después de excluir los casos incompletos. Si estas medidas permanecen estables, la omisión de dichos registros tendría un impacto limitado; si cambian de manera apreciable, entonces la ausencia de datos sí afecta la interpretación.

Con lo cual procederemos en las siguientes secciones a evaluar estas medidas de localización asociadas.

34.4. Medidas de localización básicas

Como vimos en el capítulo pasado las medidas más usadas en esta etapa son:

- **Media:**

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- **Mediana:** valor que deja 50 % de observaciones a cada lado.
- **Moda:** valor más frecuente. En variables continuas muchas veces es más útil hablar de la región modal observada en un histograma.
- **Cuantiles:** percentiles como $Q_{0.25}$, $Q_{0.50}$ y $Q_{0.75}$.

Estas se pueden conocer de manera rápida en pandas mediante los métodos `mean()`, `median()`, `mode()` y `quantile()`. Para facilitar la lectura comparativa, conviene construir una tabla resumen con estas medidas para cada variable numérica clave.

```

def mode_series(s):
    m = s.mode(dropna=True)
    if len(m) == 0:
        return np.nan
    if len(m) == 1:
        return m.iloc[0]
    return m.iloc[:5].tolist()

summary_loc = pd.DataFrame({
    "media": df[vars_localizacion].mean(numeric_only=True),
    "mediana": df[vars_localizacion].median(numeric_only=True),
    "moda": [mode_series(df[c]) for c in vars_localizacion],

```

```

"q25": df[vars_localizacion].quantile(0.25),
"q75": df[vars_localizacion].quantile(0.75),
"min": df[vars_localizacion].min(numeric_only=True),
"max": df[vars_localizacion].max(numeric_only=True),
})
summary_loc

```

	media	mediana	moda	q25	q75	min	max
Item_Weight	12.857645	12.600000	12.1500	8.773750	16.850000	4.555	21.350000
Item_Visibility	0.066132	0.053931	0.0000	0.026989	0.094585	0.000	0.328391
Item_MRP	140.992782	143.012800	172.0422	93.826500	185.643700	31.290	266.888400
Item_Outlet_Sales	2181.288914	1794.331000	958.7520	834.247400	3101.296400	33.290	13086.964800

Esta tabla permite una primera lectura comparativa. La relación entre media y mediana es particularmente informativa. Si ambas están muy próximas, la distribución podría ser relativamente simétrica o al menos no estar muy afectada por colas extremas. Si difieren de forma notable, conviene anticipar que más adelante aparecerán señales de asimetría.

34.5. Interpretación estadística de la media y la mediana

En datos de ventas es común encontrar distribuciones con sesgo a la derecha. Esto significa que la mayor parte de las observaciones se agrupan en niveles bajos o medios, pero existe una cola de valores altos que empuja la media hacia arriba. En ese caso, la media deja de representar un valor “típico” tan bien como la mediana.

Podemos ilustrarlo directamente con histogramas sobre los que se tracen ambas medidas.

```

from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=vars_localizacion
)

for i, col in enumerate(vars_localizacion):
    s = df[col].dropna()
    row = i // 2 + 1
    col_pos = i % 2 + 1

    media = s.mean()
    mediana = s.median()

    # Histograma
    fig.add_trace(
        go.Histogram(

```

```

        x=s,
        nbinsx=30,
        name=col,
        showlegend=False
    ),
    row=row, col=col_pos
)

# Línea de la media
fig.add_vline(
    x=media,
    line_dash="dash",
    line_width=2,
    annotation_text=f"Media = {media:.2f}",
    annotation_position="top right",
    row=row, col=col_pos
)

# Línea de la mediana
fig.add_vline(
    x=mediana,
    line_dash="dot",
    line_width=2,
    annotation_text=f"Mediana = {mediana:.2f}",
    annotation_position="top left",
    row=row, col=col_pos
)

fig.update_xaxes(title_text=col, row=row, col=col_pos)
fig.update_yaxes(title_text="Frecuencia", row=row, col=col_pos)

fig.update_layout(
    # title="Distribuciones con media y mediana",
    height=700,
    width=1000,
    bargap=0.05
)

fig.show()

```

Unable to display output for mime type(s): text/html

La superposición de estas líneas verticales ofrece una interpretación inmediata. En particular, si en *Item_Outlet_Sales* la media aparece sensiblemente a la derecha de la mediana, puede concluirse que las ventas no se distribuyen de forma simétrica y que existen casos de ventas altas elevan el promedio. En cambio, si en *Item_MRP* ambas medidas son relativamente cercanas, la estructura de precios podría ser más estable o menos sesgada.

34.6. Cuantiles y lectura ampliada del centro

Cuando hablamos de **localización**, no siempre basta con resumir una variable mediante un único valor, como la media o la mediana. En muchos casos, resulta más útil describir la **zona central** donde se concentra la mayor parte de las observaciones. Para ello, los **cuantiles** son especialmente importantes.

En particular, los percentiles 25, 50 y 75 permiten identificar el tramo donde se encuentra el **50 % central de los datos**. Esta información ayuda a responder preguntas más ricas que un simple “¿cuál es el promedio?”, por ejemplo:

- ¿entre qué valores se concentra la mitad central de los casos?
- ¿qué tan alejados están los valores altos respecto del centro?
- ¿la mediana representa bien a la distribución o existen asimetrías importantes?

```
quantiles = df[vars_localizacion].quantile([0.10, 0.25, 0.50, 0.75, 0.90])
quantiles
```

	Item_Weight	Item_Visibility	Item_MRP	Item_Outlet_Sales
0.10	6.69500	0.012042	52.79560	343.5528
0.25	8.77375	0.026989	93.82650	834.2474
0.50	12.60000	0.053931	143.01280	1794.3310
0.75	16.85000	0.094585	185.64370	3101.2964
0.90	19.35000	0.139514	231.20048	4570.0512

La tabla de cuantiles permite ampliar la lectura del centro. Mientras la media resume la localización en un solo número, los cuantiles muestran cómo se distribuyen los datos alrededor de esa zona central. Así, el percentil 25 marca el punto por debajo del cual se encuentra el 25 % de las observaciones, la mediana divide la distribución en dos partes iguales, y el percentil 75 indica el límite superior del 50 % central.

Para complementar esta lectura, es útil recurrir a los boxplots, ya que representan visualmente la mediana, los cuartiles, la dispersión central y la posible presencia de valores atípicos.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=vars_localizacion
)

for i, var in enumerate(vars_localizacion):
    row = i // 2 + 1
    col = i % 2 + 1

    fig.add_trace(
        go.Box(
            y=df[var].dropna(),
            name=var,
            boxpoints="outliers",
```

```

        showlegend=False
    ),
    row=row, col=col
)

fig.update_layout(
    title="Boxplots de variables seleccionadas",
    height=700,
    width=900
)

fig.show()

```

Unable to display output for mime type(s): text/html

A partir de estos gráficos, la interpretación ya no se limita a un valor “promedio”, sino que puede centrarse en la estructura de la distribución. En particular, los boxplots permiten observar:

- dónde se ubica la mediana;
- qué tan amplio es el rango intercuartílico;
- si la distribución parece simétrica o sesgada;
- y si existen valores atípicos que puedan influir en la media.

De esta forma, los cuantiles y los boxplots enriquecen el análisis de localización, porque muestran no solo un centro numérico, sino también la región en la que efectivamente se concentra la mayor parte de los datos.

34.7. Impacto de eliminar faltantes en Item_Weight

Para evaluar si eliminar las observaciones con valores faltantes en `Item_Weight` introduce sesgos en el análisis, compararemos el comportamiento del dataset antes y después de excluir dichos casos. La idea es verificar si esta reducción de la muestra modifica de forma importante la localización de otras variables relevantes y la composición general de los datos.

Primero construimos dos subconjuntos:

- el dataset original;
- el dataset filtrado, conservando solo las observaciones con `Item_Weight` disponible.

```

df_complete_weight = df[df["Item_Weight"].notna()].copy()

n_total = len(df)
n_complete = len(df_complete_weight)
n_removed = n_total - n_complete

print(f"Observaciones totales: {n_total}")
print(f"Observaciones con Item_Weight disponible: {n_complete}")

```

```
print(f"Observaciones eliminadas: {n_removed}")
print(f"Porcentaje eliminado: {100 * n_removed / n_total:.2f}%")
```

```
Observaciones totales: 8523
Observaciones con Item_Weight disponible: 7060
Observaciones eliminadas: 1463
Porcentaje eliminado: 17.17%
```

Una primera forma de evaluar el posible sesgo consiste en comparar medidas de localización de variables relevantes antes y después del filtrado.

```
vars_compare = ["Item_MRP", "Item_Outlet_Sales", "Item_Visibility"]

summary_compare = pd.DataFrame({
    "media_original": df[vars_compare].mean(),
    "media_filtrado": df_complete_weight[vars_compare].mean(),
    "mediana_original": df[vars_compare].median(),
    "mediana_filtrado": df_complete_weight[vars_compare].median()
})

summary_compare["dif_media_abs"] = summary_compare["media_filtrado"] - summary_compare["media_ori
summary_compare["dif_media_pct"] = 100 * summary_compare["dif_media_abs"] / summary_compare["medi

summary_compare["dif_mediana_abs"] = summary_compare["mediana_filtrado"] - summary_compare["media
summary_compare["dif_mediana_pct"] = 100 * summary_compare["dif_mediana_abs"] / summary_compare["

summary_compare.round(3)
```

	media_original	media_filtrado	mediana_original	mediana_filtrado	dif_media_abs	d
Item_MRP	140.993	141.241	143.013	142.730	0.248	0.
Item_Outlet_Sales	2181.289	2118.627	1794.331	1789.670	-62.662	-2
Item_Visibility	0.066	0.064	0.054	0.052	-0.002	-3

Además de la comparación numérica, es útil observar gráficamente si las distribuciones cambian al eliminar los registros con Item_Weight faltante.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

vars_compare = ["Item_MRP", "Item_Outlet_Sales", "Item_Visibility"]

fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=vars_compare
)
```

```

for i, var in enumerate(vars_compare, start=1):
    fig.add_trace(
        go.Histogram(
            x=df[var].dropna(),
            name="Original",
            opacity=0.55,
            nbinsx=30,
            histnorm="probability density",
            showlegend=(i == 1)
        ),
        row=1, col=i
    )

    fig.add_trace(
        go.Histogram(
            x=df_complete_weight[var].dropna(),
            name="Sin faltantes en Item_Weight",
            opacity=0.55,
            nbinsx=30,
            histnorm="probability density",
            showlegend=(i == 1)
        ),
        row=1, col=i
    )

fig.update_layout(
    title="Distribuciones antes y después de eliminar faltantes en Item_Weight",
    barmode="overlay",
    height=450,
    width=1100
)

fig.show()

```

Unable to display output for mime type(s): text/html

Para reforzar la lectura de localización, también podemos comparar boxplots en ambos escenarios.

```

fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=vars_compare
)

for i, var in enumerate(vars_compare, start=1):
    fig.add_trace(
        go.Box(
            y=df[var].dropna(),
            name="Original",

```



```

        boxpoints=False,
        showlegend=(i == 1)
    ),
    row=1, col=i
)

fig.add_trace(
    go.Box(
        y=df_complete_weight[var].dropna(),
        name="Sin faltantes en Item_Weight",
        boxpoints=False,
        showlegend=(i == 1)
    ),
    row=1, col=i
)

fig.update_layout(
    title="Comparación de boxplots antes y después del filtrado",
    height=450,
    width=1100
)

fig.show()

```

Unable to display output for mime type(s): text/html

Finalmente, también conviene revisar si la eliminación afecta la composición de algunos grupos del negocio.

```

comp_outlet = pd.concat([
    df["Outlet_Type"].value_counts(normalize=True).rename("original"),
    df_complete_weight["Outlet_Type"].value_counts(normalize=True).rename("filtrado")
], axis=1)

comp_outlet["dif_pct_points"] = 100 * (comp_outlet["filtrado"] - comp_outlet["original"])
comp_outlet.round(4)

```

	original	filtrado	dif_pct_points
Outlet_Type			
Supermarket Type1	0.6543	0.7899	13.5596
Grocery Store	0.1271	0.0786	-4.8456
Supermarket Type3	0.1097	NaN	NaN
Supermarket Type2	0.1089	0.1314	2.2563

Los resultados muestran que la eliminación de observaciones con *Item_Weight* faltante produce cambios pequeños en las medidas de localización de las variables analizadas. Si bien en algunos casos las diferencias porcentuales superan ligeramente el 2 %, especialmente en *Item_Outlet_Sales* e *Item_Visibility*, la

magnitud de estas variaciones no parece suficiente para modificar de manera sustantiva la interpretación general del centro de las distribuciones. Por ello, dentro del contexto de este EDA, trabajar con los casos completos puede considerarse una decisión razonable y con bajo riesgo de sesgo en términos de localización.

	Variable	N válidos	Media	Mediana	Q1	Q3
0	Item_Weight	7060	12.857645	12.600000	8.773750	16.850000
1	Item_Visibility	8523	0.066132	0.053931	0.026989	0.094585
2	Item_MRP	8523	140.992782	143.012800	93.826500	185.643700
3	Item_Outlet_Sales	8523	2181.288914	1794.331000	834.247400	3101.296400

34.8. Ejercicio sugerido

Construya una versión inicial de dashboard en Quarto que incluya:

- una tabla resumen con media, mediana y cuantiles de las variables numéricas seleccionadas;
- un histograma con líneas para media y mediana de `Item_Outlet_Sales`;
- un gráfico de barras con la mediana de ventas por `Outlet_Type`;
- un breve comentario interpretativo donde se compare el uso de la media frente a la mediana.

Parte XIII.

Medidas de variabilidad

35. Medidas de variabilidad

35.1. Introducción

Una medida de centro sin una medida de dispersión es incompleta. Dos grupos pueden compartir la misma media y diferir radicalmente en estabilidad, riesgo o predictibilidad. La variabilidad cuantifica cuánto cambian las observaciones entre sí o respecto de una referencia (Tukey 1977; Heckert et al. 2012).

Definición. Una medida de variabilidad resume la extensión de una distribución. Puede expresarse en unidades cuadradas, en las unidades originales o de manera relativa y adimensional.

35.2. Rango

$$R = x_{(n)} - x_{(1)}. \quad (1)$$

El rango es fácil de interpretar, pero depende solo de dos observaciones y crece con el tamaño de la muestra.

35.3. Varianza y desviación estándar

La varianza poblacional y su estimador muestral usual son

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2. \quad (2)$$

La desviación estándar $s = \sqrt{s^2}$ recupera las unidades originales. El divisor $n-1$ corrige el sesgo al estimar la varianza poblacional a partir de una muestra (Freedman et al. 2007).

Ejemplo. Los conjuntos $A = (8, 9, 10, 11, 12)$ y $B = (0, 5, 10, 15, 20)$ tienen media 10. Sus desviaciones estándar muestrales son aproximadamente 1.58 y 7.91: el centro no distingue su estabilidad.

35.4. Cuartiles e intervalo intercuartílico

$$IQR = Q_3 - Q_1. \quad (3)$$

El IQR describe la amplitud del 50 % central. La regla de Tukey marca como candidatos a valores atípicos los puntos fuera de $[Q_1 - 1.5IQR, Q_3 + 1.5IQR]$. Es una regla de exploración, no una orden automática para borrar datos (Tukey 1977).

35.5. Desviación absoluta mediana

$$MAD = \text{mediana}_i\{|x_i - \tilde{x}|\}. \quad (4)$$

Para hacerla comparable con σ bajo normalidad suele multiplicarse por 1.4826. El MAD es resistente a contaminación y complementa al IQR (Rousseeuw y Croux 1993).

Un punto fuera de los bigotes de un boxplot no es necesariamente un error. Puede representar un evento real, un segmento distinto o una cola legítima. Primero debe investigarse su procedencia.

35.6. Coeficiente de variación

$$CV = \frac{s}{|\bar{x}|}. \quad (5)$$

El CV permite comparar dispersión relativa entre variables de razón con unidades o escalas diferentes.

El CV pierde sentido cuando la media es cero o cercana a cero, y no debe usarse en escalas con origen arbitrario, como temperatura en grados Celsius.

35.7. Cálculo manual paso a paso

Usaremos el conjunto clásico

$$x = (2, 4, 4, 4, 5, 5, 7, 9), \quad n = 8.$$

35.7.1. Rango

El mínimo es 2 y el máximo es 9. Restamos:

$$R = 9 - 2 = 7.$$

35.7.2. Media necesaria para calcular varianza

$$\bar{x} = \frac{2 + 4 + 4 + 4 + 5 + 5 + 7 + 9}{8} = \frac{40}{8} = 5.$$

35.7.3. Varianza poblacional y desviación estándar poblacional

Paso 1. Restar la media a cada observación:

$$x_i - 5 = (-3, -1, -1, -1, 0, 0, 2, 4).$$

Paso 2. Elevar cada desviación al cuadrado:

$$(x_i - 5)^2 = (9, 1, 1, 1, 0, 0, 4, 16).$$

Paso 3. Sumar los cuadrados:

$$9 + 1 + 1 + 1 + 0 + 0 + 4 + 16 = 32.$$

Paso 4. Si los ocho datos constituyen la población, dividir entre $N = 8$:

$$\sigma^2 = \frac{32}{8} = 4.$$

Paso 5. Extraer la raíz cuadrada:

$$\sigma = \sqrt{4} = 2.$$

35.7.4. Varianza y desviación estándar muestrales

Si los mismos datos son una muestra de una población mayor, se divide entre $n - 1$.

$$s^2 = \frac{32}{8 - 1} = \frac{32}{7} = 4.5714,$$

$$s = \sqrt{4.5714} = 2.1381.$$

La suma de cuadrados es la misma; solo cambia el divisor según la pregunta estadística.

35.7.5. Cuartiles, IQR y límites de Tukey

Usaremos nuevamente el método de medianas de las mitades.

Mitad inferior y superior:

$$L = (2, 4, 4, 4), \quad U = (5, 5, 7, 9).$$

Cuartiles:

$$Q_1 = \frac{4+4}{2} = 4, \quad Q_3 = \frac{5+7}{2} = 6.$$

Intervalo intercuartílico:

$$IQR = Q_3 - Q_1 = 6 - 4 = 2.$$

Límites de Tukey:

$$L_i = 4 - 1.5(2) = 4 - 3 = 1,$$

$$L_s = 6 + 1.5(2) = 6 + 3 = 9.$$

Ningún valor queda fuera de $[1, 9]$; el 9 está exactamente en el límite superior.

35.7.6. MAD

La mediana del conjunto es $(4 + 5)/2 = 4.5$.

Paso 1. Calcular distancias absolutas respecto de 4.5:

$$|x_i - 4.5| = (2.5, 0.5, 0.5, 0.5, 0.5, 0.5, 2.5, 4.5).$$

Paso 2. Ordenarlas:

$$(0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 2.5, 2.5, 4.5).$$

Paso 3. Promediar las posiciones 4 y 5:

$$MAD = \frac{0.5 + 0.5}{2} = 0.5.$$

MAD escalado para comparación aproximada con una desviación normal:

$$MAD_N = 1.4826(0.5) = 0.7413.$$

35.7.7. Coeficiente de variación

Si tratamos los datos como población:

$$CV = \frac{\sigma}{|\mu|} = \frac{2}{5} = 0.40 = 40 \%.$$

Si se tratan como muestra:

$$CV_m = \frac{s}{|\bar{x}|} = \frac{2.1381}{5} = 0.4276 = 42.76 \%.$$

Del cálculo al programa. Primero se obtiene el centro; después se recorre la colección para acumular desviaciones cuadráticas o absolutas. Las medidas basadas en orden —IQR y MAD—

exigen ordenar o seleccionar posiciones. El programa debe recibir un parámetro que indique si la varianza es poblacional o muestral.

35.8. Misma media, distinta dispersión

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

rng = np.random.default_rng(23)
n = 1500
datos = pd.DataFrame({
    "valor": np.r_[rng.normal(50, 3, n), rng.normal(50, 12, n)],
    "escenario": ["Baja variabilidad"] * n + ["Alta variabilidad"] * n
})
fig = px.histogram(datos, x="valor", color="escenario", nbins=70,
                   barmode="overlay", histnorm="probability density",
                   opacity=0.55, title="Mismo centro, diferente dispersión")
fig.add_vline(x=50, line_dash="dash", annotation_text="Media común")
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

35.9. Sensibilidad frente a valores extremos

```
base = rng.normal(100, 10, 300)
filas = []
for extremo in range(100, 501, 25):
    x = np.r_[base, extremo]
    q1, q3 = np.quantile(x, [0.25, 0.75])
    filas.extend([
        {"Extremo agregado": extremo, "Medida": "Desviación estándar", "Valor": x.std(ddof=1)},
        {"Extremo agregado": extremo, "Medida": "IQR", "Valor": q3-q1},
        {"Extremo agregado": extremo, "Medida": "MAD escalado", "Valor": 1.4826*np.median(np.abs(
    ]))
sensibilidad = pd.DataFrame(filas)
fig = px.line(sensibilidad, x="Extremo agregado", y="Valor", color="Medida",
              markers=True, title="Respuesta de medidas clásicas y robustas a un extremo")
fig.update_traces(hovertemplate="Extremo=%{x}<br>Medida=%{y:.2f}<extra></extra>")
fig.show()
```

Unable to display output for mime type(s): text/html

Interpretación. La desviación estándar responde con fuerza a la magnitud del extremo. IQR y MAD cambian poco porque dependen del orden y del centro de la muestra.

35.10. Boxplot interactivo

```
fig = px.box(datos, x="escenario", y="valor", color="escenario", points="outliers",
             title="IQR, mediana y candidatos a valores atípicos")
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

35.11. Selección de la medida

Objetivo	Medida recomendada	Precaución
Describir dispersión alrededor de la media	Desviación estándar	Sensible a extremos
Describir el 50 % central	IQR	No usa toda la muestra
Resistencia ante contaminación	MAD	Puede ser cero en datos discretos
Comparar escalas positivas	CV	Requiere escala de razón y media no nula
Comunicar amplitud total	Rango	Muy sensible al tamaño muestral

35.12. Puente al caso BigMart

El capítulo Medidas de variabilidad en Big Mart Sales compara estas medidas en ventas, precios, peso y visibilidad.

35.13. Ejercicios

1. Construya dos conjuntos con igual media y desviación estándar, pero distinto IQR.
2. Investigue cómo cambia el rango al aumentar el tamaño de una muestra normal.
3. Justifique si el CV es apropiado para comparar `Item_MRP` e `Item_Outlet_Sales`.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.

- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Thesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. [martinfowler.com](http://martinfowler.com/articles/richardsonMaturityModel.html). <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best

- Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data*

- Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbj1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine

- Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

36. Medidas de variabilidad en Big Mart Sales

Este caso continúa el desarrollo teórico de Medidas de variabilidad y lo aplica al dataset BigMart.

Una vez identificada la ubicación o región central de las principales variables del conjunto de datos, el siguiente paso natural es estudiar su **variabilidad**. Mientras las medidas de localización responden a la pregunta “¿dónde se ubican los valores típicos?”, las medidas de dispersión responden a “¿qué tan separados o concentrados están los datos alrededor de ese centro?”. Esta distinción es crucial porque dos distribuciones pueden tener la misma media o la misma mediana y, sin embargo, exhibir comportamientos muy distintos en términos de estabilidad, riesgo o heterogeneidad interna.

En el contexto de **Big Mart Sales**, la variabilidad tiene una lectura operativa inmediata. Una variable con baja dispersión sugiere un comportamiento relativamente estable entre productos u outlets, mientras que una alta dispersión puede indicar mezcla de segmentos, presencia de valores extremos o una dinámica comercial desigual. Para variables como `Item_Outlet_Sales`, esto es especialmente relevante porque no basta con conocer el nivel promedio de ventas: también interesa saber si las ventas se mantienen relativamente homogéneas o si existen diferencias muy amplias entre observaciones.

Definición operativa. La variabilidad cuantifica cuánto se separan las observaciones del centro o entre sí; aquí se contrasta la desviación estándar con medidas robustas como el IQR (Tukey 1977; Rousseeuw y Croux 1993).

36.1. Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

```
import pandas as pd
import numpy as np

DATA_PATH = "../../../data/bigmart_sales.csv"

df = pd.read_csv(DATA_PATH)

vars_variabilidad = [
    "Item_Weight",
    "Item_Visibility",
    "Item_MRP",
    "Item_Outlet_Sales",
]
```

36.2. De la localización a la dispersión

Si en el capítulo anterior se observó que la media y la mediana resumen el centro, ahora interesa medir qué tanto se alejan los datos de esos valores representativos. Las medidas más comunes son:

- **Rango:**

$$R = x_{\text{máx}} - x_{\text{mín}}$$

- **Varianza muestral:**

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

- **Desviación estándar muestral:**

$$s = \sqrt{s^2}$$

- **Recorrido intercuartílico:**

$$IQR = Q_3 - Q_1$$

- **Coefficiente de variación:**

$$CV = \frac{s}{\bar{x}}$$

siempre que la media sea positiva y tenga interpretación de escala.

Estas medidas no son intercambiables. El rango depende exclusivamente de los valores extremos, la desviación estándar resume el alejamiento promedio respecto a la media y el IQR se concentra en el 50 % central de la distribución, por lo que resulta más robusto ante observaciones atípicas.

36.3. Cálculo de medidas de variabilidad

Para el cálculo de las medidas de interés podemos utilizar los métodos de pandas `.mean()`, `.var()`, `.std()`, `.min()`, `.quantile()`, `.median()` y `.max()`.

```
summary_var = pd.DataFrame({
    "media": df[vars_variabilidad].mean(),
    "varianza": df[vars_variabilidad].var(),
    "desv_std": df[vars_variabilidad].std(),
    "min": df[vars_variabilidad].min(),
    "q1": df[vars_variabilidad].quantile(0.25),
    "mediana": df[vars_variabilidad].median(),
    "q3": df[vars_variabilidad].quantile(0.75),
    "max": df[vars_variabilidad].max(),
})

summary_var["rango"] = summary_var["max"] - summary_var["min"]
summary_var["iqr"] = summary_var["q3"] - summary_var["q1"]
summary_var["cv"] = summary_var["desv_std"] / summary_var["media"]
summary_var
```

	media	varianza	desv_std	min	q1	mediana	q3
Item_Weight	12.857645	2.156169e+01	4.643456	4.555	8.773750	12.600000	16.850000

	media	varianza	desv_std	min	q1	mediana	q3
Item_Visibility	0.066132	2.662335e-03	0.051598	0.000	0.026989	0.053931	0.094585
Item_MRP	140.992782	3.878184e+03	62.275067	31.290	93.826500	143.012800	185.643700
Item_Outlet_Sales	2181.288914	2.912141e+06	1706.499616	33.290	834.247400	1794.331000	3101.296400

La interpretación de estas medidas debe hacerse con cuidado. Tanto la varianza como la desviación estándar dependen de la escala en la que se mide cada variable. Por esta razón, no es conveniente comparar de forma directa la desviación estándar de `Item_MRP` con la de `Item_Visibility`, ya que ambas variables están expresadas en magnitudes muy distintas. Para una comparación más justa, resulta más útil el **coeficiente de variación (cv)**, porque relaciona la dispersión con el tamaño del promedio.

A partir del coeficiente de variación, se observa que `Item_Outlet_Sales` ($cv = 0.782$) y `Item_Visibility` ($cv = 0.780$) son las variables con mayor dispersión relativa. Esto significa que, en ambos casos, los valores se encuentran bastante separados de su media y, por tanto, presentan una alta heterogeneidad. En términos prácticos, las ventas y la visibilidad no se comportan de manera uniforme en el dataset, sino que muestran diferencias importantes entre observaciones.

Por otro lado, `Item_MRP` presenta un coeficiente de variación intermedio ($cv = 0.442$). Aunque sus valores se distribuyen en un rango amplio, su variabilidad relativa es menor que la observada en ventas y visibilidad. Finalmente, `Item_Weight` es la variable menos dispersa en términos relativos ($cv = 0.361$), lo que sugiere que sus valores están más concentrados alrededor de la media.

Esta lectura puede reforzarse con el **rango intercuartílico (iqr)**, que muestra la amplitud del 50 % central de los datos. En `Item_Outlet_Sales`, el valor de $iqr = 2267.049$ indica que incluso la parte central de la distribución presenta una dispersión considerable. En `Item_MRP`, el $iqr = 91.817$ también sugiere variabilidad, aunque menos extrema. En `Item_Weight` e `Item_Visibility`, estos valores deben interpretarse dentro de su propia escala, evitando comparaciones absolutas entre variables distintas.

En conjunto, estas medidas muestran que `Item_Outlet_Sales` y `Item_Visibility` son las variables más variables del conjunto, mientras que `Item_Weight` presenta un comportamiento relativamente más estable. Para el EDA, esto es importante porque indica que algunas variables requieren una exploración más cuidadosa de su dispersión, posibles valores extremos y diferencias entre grupos.

36.4. Visualización de dispersión con boxplots

Los **boxplots** o diagramas de caja son una herramienta muy útil en EDA porque permiten resumir, en una sola figura, varios aspectos importantes de una variable: su centro, su dispersión y la posible presencia de valores atípicos.

En cada boxplot:

- la **línea dentro de la caja** representa la **mediana**;
- la **base inferior** de la caja corresponde al **primer cuartil** (Q_1);
- la **base superior** corresponde al **tercer cuartil** (Q_3);
- la **altura de la caja** representa el **rango intercuartílico** ($IQR = Q_3 - Q_1$), es decir, la amplitud del 50 % central de los datos;
- los **bigotes** muestran hasta dónde se extienden los valores que todavía pueden considerarse dentro del comportamiento general de la variable;
- los puntos que quedan fuera suelen interpretarse como **posibles valores atípicos**.

Esto hace que el boxplot sea especialmente valioso, porque no solo muestra dónde está el centro de la distribución, sino también qué tan concentrados o dispersos están los datos alrededor de ese centro.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=vars_variabilidad
)

for i, col in enumerate(vars_variabilidad):
    row = i // 2 + 1
    col_pos = i % 2 + 1

    fig.add_trace(
        go.Box(
            y=df[col].dropna(),
            name=col,
            boxpoints="outliers",
            showlegend=False
        ),
        row=row, col=col_pos
    )

    fig.update_yaxes(title_text=col, row=row, col=col_pos)

fig.update_layout(
    title="Boxplots de variables seleccionadas",
    height=700,
    width=950
)

fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Esta visualización permite notar rápidamente qué variables presentan mayor amplitud central, qué tan alejados aparecen los extremos y si existen observaciones fuera del patrón principal. En contextos de EDA, el boxplot es una transición natural entre el estudio de localización y el análisis de forma, porque revela tanto la posición de la mediana como la anchura del bloque central.

36.5. Rango frente a IQR: medidas sensibles y robustas

El rango es intuitivo, pero muy sensible a valores extremos. Si una sola observación toma un valor muy grande o muy pequeño, el rango cambia de forma drástica. El IQR, en cambio, describe la anchura del

bloque central de observaciones y por ello suele ser una mejor medida cuando se desea resumir dispersión sin que la descripción dependa demasiado de extremos aislados.

Para visualizar esta idea, puede compararse el rango con el IQR en una tabla compacta.

```
robustez = summary_var[["rango", "iqr", "desv_std", "cv"]].sort_values("cv", ascending=False)
robustez
```

	rango	iqr	desv_std	cv
Item_Outlet_Sales	13053.674800	2267.049000	1706.499616	0.782335
Item_Visibility	0.328391	0.067596	0.051598	0.780224
Item_MRP	235.598400	91.817200	62.275067	0.441690
Item_Weight	16.795000	8.076250	4.643456	0.361144

A partir de los resultados observados, se nota que en todas las variables el **rango es considerablemente mayor que el IQR**, lo cual indica que los valores más extremos amplían mucho la dispersión total respecto de la dispersión que presenta la zona central de los datos.

Por ejemplo, en `Item_Outlet_Sales` el rango es de **13053.67**, mientras que el IQR es de **2267.05**. Esto significa que, aunque el total de valores cubre un intervalo muy amplio, la mitad central de las observaciones se concentra en una región mucho más reducida. En otras palabras, la dispersión total de las ventas está fuertemente influida por valores extremos o muy alejados del centro.

Algo similar ocurre con `Item_Visibility`, donde el rango (**0.3284**) es casi cinco veces el IQR (**0.0676**). Esto sugiere que, aunque la mayor parte de los productos tiene niveles de visibilidad concentrados en una zona relativamente estrecha, existen algunos casos con valores bastante más alejados que expanden notablemente el rango.

En `Item_MRP`, el rango (**235.60**) también supera claramente al IQR (**91.82**), lo que indica que el precio de lista presenta variación importante, aunque la dispersión del bloque central es más moderada que la sugerida por los extremos. Por último, en `Item_Weight`, el rango (**16.80**) es aproximadamente el doble del IQR (**8.08**), lo que refleja una distribución relativamente más estable en comparación con las demás variables.

En conjunto, los resultados sugieren que `Item_Outlet_Sales` y `Item_Visibility` son variables donde la diferencia entre dispersión total y dispersión central es más marcada, mientras que `Item_Weight` presenta una estructura comparativamente más compacta. Por ello, en el EDA resulta recomendable usar el IQR junto con otras medidas robustas cuando se quiere describir la variabilidad sin que los extremos dominen la interpretación.

36.6. Comparación de dispersión entre grupos

La variabilidad también puede cambiar entre subpoblaciones. En Big Mart Sales una comparación razonable consiste en estudiar la dispersión de `Item_Outlet_Sales` según `Outlet_Type`.

```
by_outlet = (
    df.groupby("Outlet_Type")["Item_Outlet_Sales"]
      .agg(["count", "mean", "median", "std"])
      .rename(columns={"count": "n", "mean": "media", "median": "mediana", "std": "desv_std"})
      .sort_values("desv_std", ascending=False)
)

by_outlet
```

Outlet_Type	n	media	mediana	desv_std
Supermarket Type3	935	3694.038558	3364.9532	2127.760054
Supermarket Type1	5577	2316.181148	1990.7420	1515.965558
Supermarket Type2	928	1995.498739	1655.1788	1375.932889
Grocery Store	1083	339.828500	256.9988	260.851582

```
import plotly.express as px

fig = px.box(
    df,
    x="Outlet_Type",
    y="Item_Outlet_Sales",
    points="outliers",
    title="Dispersión de Item_Outlet_Sales por Outlet_Type",
    labels={
        "Outlet_Type": "Tipo de Outlet",
        "Item_Outlet_Sales": "Ventas por producto"
    }
)

fig.update_layout(
    xaxis_tickangle=45,
    height=500,
    width=900
)

fig.show()
```

Unable to display output for mime type(s): text/html

Esta figura permite observar que no solo cambia el centro de las ventas según tipo de outlet, sino también su grado de dispersión. Algunos grupos pueden presentar ventas más consistentes, mientras otros muestran una distribución más extendida. Esta observación será clave cuando más adelante se discuta heterogeneidad.

36.7. Coeficiente de variación y comparación relativa

En muchas aplicaciones interesa comparar dispersión entre variables con escalas diferentes. La desviación estándar por sí sola no siempre permite hacerlo de manera justa. El coeficiente de variación proporciona una medida adimensional que expresa la dispersión como proporción del promedio.

```
cv_table = summary_var[["media", "desv_std", "cv"]].sort_values("cv", ascending=False)
cv_table
```

	media	desv_std	cv
Item_Outlet_Sales	2181.288914	1706.499616	0.782335
Item_Visibility	0.066132	0.051598	0.780224
Item_MRP	140.992782	62.275067	0.441690
Item_Weight	12.857645	4.643456	0.361144

Si una variable exhibe un coeficiente de variación alto, ello sugiere que su dispersión es grande en relación con su propio nivel medio. En el caso de ventas o visibilidad, este tipo de medida puede ser más informativa que la desviación estándar absoluta cuando se comparan fenómenos de distinta escala.

36.8. Histograma con bandas de dispersión

Otra visualización útil consiste en representar la media junto con bandas de una desviación estándar. No debe interpretarse como intervalo de confianza ni como regla universal de cobertura, pero sí como una guía visual de la amplitud típica alrededor del promedio.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=vars_variabilidad
)

for i, col in enumerate(vars_variabilidad):
    s = df[col].dropna()
    mu = s.mean()
    sigma = s.std()

    row = i // 2 + 1
    col_pos = i % 2 + 1

    # Histograma
    fig.add_trace(
        go.Histogram(
            x=s,
            nbinsx=30,
```

```

        name=col,
        showlegend=False
    ),
    row=row, col=col_pos
)

# Línea de la media
fig.add_vline(
    x=mu,
    line_dash="dash",
    line_width=2,
    annotation_text=f"Media = {mu:.2f}",
    annotation_position="top right",
    row=row, col=col_pos
)

# Línea media - s
fig.add_vline(
    x=mu - sigma,
    line_dash="dot",
    line_width=2,
    annotation_text="Media - s",
    annotation_position="top left",
    row=row, col=col_pos
)

# Línea media + s
fig.add_vline(
    x=mu + sigma,
    line_dash="dot",
    line_width=2,
    annotation_text="Media + s",
    annotation_position="top right",
    row=row, col=col_pos
)

fig.update_xaxes(title_text=col, row=row, col=col_pos)
fig.update_yaxes(title_text="Frecuencia", row=row, col=col_pos)

fig.update_layout(
    title="Histogramas con media y desviación estándar",
    height=700,
    width=1000,
    bargap=0.05
)

fig.show()

```

Unable to display output for mime type(s): text/html

Esta visualización resulta especialmente útil para introducir una idea intuitiva: distribuciones con la misma media pueden diferir porque sus observaciones se encuentran más o menos alejadas de ese centro.

36.9. Ejercicio sugerido

Construya un segundo dashboard en Quarto que incluya:

- una tabla resumen con rango, desviación estándar, IQR y coeficiente de variación;
- boxplots para `Item_Weight`, `Item_Visibility`, `Item_MRP` e `Item_Outlet_Sales`;
- una comparación de la dispersión de `Item_Outlet_Sales` por `Outlet_Type`;
- un comentario breve donde se explique por qué el IQR es más robusto que el rango frente a valores extremos.

Parte XIV.

Medidas de forma

37. Medidas de forma

37.1. Introducción

La localización y la variabilidad no determinan la geometría de una distribución. Las medidas de forma describen asimetría, peso de las colas y multimodalidad, propiedades que influyen en la elección de resúmenes robustos y transformaciones (Heckert et al. 2012; Joanes y Gill 1998).

37.2. Momentos centrales

$$m_r = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^r. \quad (1)$$

El segundo momento se relaciona con la varianza; los momentos estandarizados tercero y cuarto originan medidas de asimetría y curtosis.

Definición. La forma es la estructura global de una distribución una vez considerados su centro y escala: simetría, colas, picos y número de modos.

37.3. Asimetría

La asimetría poblacional se expresa como

$$\gamma_1 = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3}. \quad (2)$$

- $\gamma_1 > 0$: cola derecha más larga.
- $\gamma_1 < 0$: cola izquierda más larga.
- $\gamma_1 \approx 0$: ausencia de asimetría marcada, pero no necesariamente normalidad.

Las implementaciones muestrales aplican correcciones distintas; debe documentarse la convención usada (Joanes y Gill 1998).

37.4. Curtosis

$$\beta_2 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4}, \quad \gamma_2 = \beta_2 - 3. \quad (3)$$

γ_2 es el exceso de curtosis, que vale cero para una distribución normal. La curtosis se interpreta principalmente como sensibilidad a valores alejados y peso de colas, no solo como “altura del pico”.

Reportar “curtosis = 3” o “curtosis = 0” puede describir la misma muestra bajo convenciones diferentes. Indique siempre si se usa curtosis de Pearson o exceso de Fisher.

37.5. Cálculo manual paso a paso

Para visualizar cada operación usaremos

$$x = (0, 0, 0, 0, 5), \quad n = 5.$$

Es un conjunto pequeño con cuatro observaciones en cero y una observación hacia la derecha.

37.5.1. Media y desviaciones

Paso 1. Calcular la media:

$$\bar{x} = \frac{0 + 0 + 0 + 0 + 5}{5} = \frac{5}{5} = 1.$$

Paso 2. Restar la media:

$$x_i - \bar{x} = (-1, -1, -1, -1, 4).$$

37.5.2. Segundo momento central

Elevamos las desviaciones al cuadrado:

$$(-1)^2 = 1, \quad (-1)^2 = 1, \quad (-1)^2 = 1, \quad (-1)^2 = 1, \quad 4^2 = 16.$$

Sumamos y dividimos entre 5:

$$m_2 = \frac{1 + 1 + 1 + 1 + 16}{5} = \frac{20}{5} = 4.$$

La escala usada para estandarizar los momentos será

$$\sqrt{m_2} = \sqrt{4} = 2.$$

37.5.3. Asimetría por momentos

Paso 1. Elevar las desviaciones al cubo:

$$(-1)^3 = -1, \quad (-1)^3 = -1, \quad (-1)^3 = -1, \quad (-1)^3 = -1, \quad 4^3 = 64.$$

Paso 2. Calcular el tercer momento:

$$m_3 = \frac{-1 - 1 - 1 - 1 + 64}{5} = \frac{60}{5} = 12.$$

Paso 3. Estandarizar:

$$\gamma_1 = \frac{m_3}{m_2^{3/2}} = \frac{12}{4^{3/2}} = \frac{12}{(\sqrt{4})^3} = \frac{12}{2^3} = \frac{12}{8} = 1.5.$$

El resultado positivo coincide con la cola situada a la derecha.

37.5.4. Curtosis y exceso de curtosis

Paso 1. Elevar las desviaciones a la cuarta potencia:

$$(-1)^4 = 1, \quad (-1)^4 = 1, \quad (-1)^4 = 1, \quad (-1)^4 = 1, \quad 4^4 = 256.$$

Paso 2. Calcular el cuarto momento:

$$m_4 = \frac{1 + 1 + 1 + 1 + 256}{5} = \frac{260}{5} = 52.$$

Paso 3. Estandarizar para obtener curtosis de Pearson:

$$\beta_2 = \frac{m_4}{m_2^2} = \frac{52}{4^2} = \frac{52}{16} = 3.25.$$

Paso 4. Restar 3 para obtener exceso de Fisher:

$$\gamma_2 = \beta_2 - 3 = 3.25 - 3 = 0.25.$$

37.5.5. Comparación con una distribución simétrica

Para $z = (-2, -1, 0, 1, 2)$ la media es cero. Los cubos se cancelan:

$$(-2)^3 + (-1)^3 + 0^3 + 1^3 + 2^3 = -8 - 1 + 0 + 1 + 8 = 0.$$

Por tanto, $m_3 = 0/5 = 0$ y su asimetría por momentos es cero.

Estos cálculos usan momentos con divisor n para construir intuición. Funciones como `pandas.Series.skew()` o `scipy.stats.skew()` pueden aplicar correcciones de sesgo muestral y producir valores distintos en muestras pequeñas (Joanes y Gill 1998).

Del cálculo al programa. El algoritmo realiza dos recorridos: en el primero calcula la media; en el segundo acumula $(x_i - \bar{x})^2$, $(x_i - \bar{x})^3$ y $(x_i - \bar{x})^4$. Finalmente divide entre n y estandariza los momentos tercero y cuarto con m_2 .

37.6. Asimetría visible e interactiva

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from scipy import stats

rng = np.random.default_rng(31)
n = 4000
sim = rng.normal(0, 1, n)
derecha = rng.lognormal(0, 0.75, n)
izquierda = -rng.lognormal(0, 0.75, n)
forma = pd.DataFrame({
    "valor": np.r_[izquierda, sim, derecha],
    "distribución": ["Cola izquierda"]*n + ["Simétrica"]*n + ["Cola derecha"]*n
})
fig = px.histogram(forma, x="valor", color="distribución", facet_row="distribución",
                    histnorm="probability density", nbins=80, opacity=.75,
                    title="Tres direcciones de asimetría")
fig.update_layout(height=720, showlegend=False)
fig.for_each_annotation(lambda a: a.update(text=a.text.split("=")[-1]))
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

37.7. Misma media y varianza, colas diferentes

```
x_n = rng.normal(size=n)
x_l = rng.laplace(scale=1/np.sqrt(2), size=n)
x_t = rng.standard_t(df=5, size=n) * np.sqrt(3/5)
colas = pd.DataFrame({
    "valor": np.r_[x_n, x_l, x_t],
    "distribución": ["Normal"]*n + ["Laplace"]*n + ["t(5)"]*n
})
fig = px.histogram(colas, x="valor", color="distribución", nbins=100,
                    histnorm="probability density", barmode="overlay", opacity=.45,
```

```
range_x=[-6, 6], title="Escala comparable, comportamiento distinto en las colas")
fig.show()
```

Unable to display output for mime type(s): text/html

Interpretación. Superponer distribuciones revela diferencias que media y varianza ocultan. Al pasar el cursor se pueden inspeccionar regiones centrales y colas sin perder la identidad de cada distribución.

37.8. Gráfico Q-Q

Un gráfico Q-Q compara cuantiles observados con cuantiles de una distribución de referencia. Una línea aproximadamente recta indica compatibilidad visual; desviaciones sistemáticas en los extremos revelan asimetría o colas diferentes.

```
muestra = rng.lognormal(0, .7, 700)
osm, osr = stats.probplot(muestra, dist="norm", fit=False)
pendiente, intercepto, _ = stats.probplot(muestra, dist="norm", fit=True)[1]
fig = go.Figure()
fig.add_trace(go.Scatter(x=osm, y=osr, mode="markers", name="Cuantiles observados"))
xline = np.array([min(osm), max(osm)])
fig.add_trace(go.Scatter(x=xline, y=intercepto + pendiente*xline, mode="lines",
                        name="Referencia normal", line=dict(dash="dash"))))
fig.update_layout(title="Q-Q normal: curvatura causada por asimetría",
                  xaxis_title="Cuantiles teóricos", yaxis_title="Cuantiles observados")
fig.show()
```

Unable to display output for mime type(s): text/html

37.9. Transformaciones y forma

```
filas = []
for lam in [1.0, 0.5, 0.25, 0.0]:
    transformado = np.log1p(muestra) if lam == 0 else ((muestra + 1)**lam - 1)/lam
    etiqueta = "log" if lam == 0 else f"={lam}"
    filas.extend({"valor": v, "transformación": etiqueta} for v in transformado)
anim = pd.DataFrame(filas)
fig = px.histogram(anim, x="valor", animation_frame="transformación", nbins=55,
                  histnorm="probability density", range_y=[0, 1.8],
                  title="Animación: efecto de transformaciones de potencia")
fig.show()
```

Unable to display output for mime type(s): text/html

Transformar no “arregla” automáticamente los datos. La nueva escala debe conservar una interpretación útil y aplicarse dentro del pipeline para evitar inconsistencias.

37.10. Multimodalidad

La asimetría y la curtosis no indican por sí solas cuántos subgrupos contiene una distribución. Histogramas, densidades, ECDF y comparaciones por grupo son indispensables. Una forma bimodal puede sugerir mezcla de poblaciones, estacionalidad o categorías omitidas.

37.11. Puente al caso BigMart

El capítulo Medidas de forma en Big Mart Sales estudia ventas, asimetría, curtosis, transformaciones y Q-Q plots.

37.12. Ejercicios

1. Construya una distribución simétrica y bimodal con asimetría cercana a cero.
2. Compare exceso de curtosis antes y después de agregar dos extremos.
3. Anime una transformación logarítmica gradual y describa qué cambia en la escala.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.

- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Calínski, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.

- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.

- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinowler.com. <https://martinowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.

- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide*

to *A/B Testing*. Cambridge University Press.

LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.

Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.

Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.

Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.

Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.

MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.

McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.

McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.

McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.

McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.

McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.

Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).

Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.

Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en->

us/azure/machine-learning/.

- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.

- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data*

Warehousing 5 (4): 13-22.

- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE*

Transactions on Systems, Man, and Cybernetics SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.

Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

38. Medidas de forma en Big Mart Sales

Este caso aplica los conceptos presentados en Medidas de forma. Allí se distinguen asimetría, curtosis, multimodalidad y gráficos Q-Q.

38.1. Introducción

En una etapa inicial del análisis exploratorio de datos se estudian las **medidas de localización** y las **medidas de variabilidad** con el fin de identificar, respectivamente, el comportamiento central de una variable y el grado en que sus observaciones se dispersan alrededor de un valor representativo. Sin embargo, una descripción estadística rigurosa no se agota en estas dos dimensiones. Dos distribuciones pueden compartir una media y una desviación estándar semejantes, y aun así diferir de manera importante en su estructura global. Por ello es necesario incorporar el estudio de la **forma** de la distribución.

Las medidas de forma permiten caracterizar aspectos tales como la **asimetría** y el grado de **concentración de masa alrededor del centro o en las colas**, usualmente descrito mediante la **curtosis**. En términos aplicados, estas medidas ayudan a responder preguntas como las siguientes: ¿la mayor parte de los productos tiene ventas moderadas con unos cuantos casos excepcionalmente altos?, ¿las observaciones se reparten de forma aproximadamente simétrica?, ¿aparecen colas pesadas o valores extremos frecuentes?, ¿una transformación puede facilitar la interpretación visual y estadística del fenómeno? Además, estas medidas orientan decisiones posteriores del EDA, como la revisión de valores atípicos, la conveniencia de usar medidas robustas o la necesidad de transformar variables para facilitar su análisis.

En este capítulo se estudian las **medidas de forma** en el dataset de BigMart, utilizando como variable principal `Item_Outlet_Sales`. No obstante, varias de las herramientas presentadas pueden aplicarse también a otras variables numéricas del conjunto de datos, como `Item_MRP`, `Item_Visibility` y `Item_Weight`.

Definición operativa. La forma describe asimetría, colas y estructura modal después de considerar el centro y la escala de la distribución (Joanes y Gill 1998).

38.2. Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy, SciPy y Plotly.

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from scipy import stats
```

```
DATA_PATH = "../../../data/bigmart_sales.csv"
```

```
df = pd.read_csv(DATA_PATH)
df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	C
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	C
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	C
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	C
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	C

Antes de proceder conviene verificar el tipo de las variables y concentrarnos en aquellas que sean cuantitativas.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8523 entries, 0 to 8522
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	Item_Identifier	8523 non-null	object
1	Item_Weight	7060 non-null	float64
2	Item_Fat_Content	8523 non-null	object
3	Item_Visibility	8523 non-null	float64
4	Item_Type	8523 non-null	object
5	Item_MRP	8523 non-null	float64
6	Outlet_Identifier	8523 non-null	object
7	Outlet_Establishment_Year	8523 non-null	int64
8	Outlet_Size	6113 non-null	object
9	Outlet_Location_Type	8523 non-null	object
10	Outlet_Type	8523 non-null	object
11	Item_Outlet_Sales	8523 non-null	float64

```
dtypes: float64(4), int64(1), object(7)
```

```
memory usage: 799.2+ KB
```

```
num_cols = df.select_dtypes(include=np.number).columns.tolist()
num_cols
```

```
['Item_Weight',
 'Item_Visibility',
 'Item_MRP',
 'Outlet_Establishment_Year',
 'Item_Outlet_Sales']
```

38.3. Fundamentos teóricos

38.3.1. Asimetría

La asimetría describe el grado en que una distribución se desvía de la simetría respecto de su centro. Intuitivamente:

- una distribución **simétrica** presenta masas comparables a la izquierda y derecha del centro;
- una distribución con **asimetría positiva** presenta una cola más larga hacia valores altos;
- una distribución con **asimetría negativa** presenta una cola más larga hacia valores bajos.

Una medida común de asimetría es el coeficiente:

$$g_1 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^{3/2}}.$$

Cuando $g_1 > 0$, la cola derecha tiende a ser más extensa; cuando $g_1 < 0$, domina la cola izquierda; y cuando $g_1 \approx 0$, la distribución puede considerarse aproximadamente simétrica, aunque esta interpretación siempre debe acompañarse de una inspección gráfica.

38.3.2. Curtosis

La curtosis resume el peso relativo de las colas y la concentración de observaciones alrededor del centro. Una expresión frecuente es:

$$g_2 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^2} - 3.$$

La resta de 3 permite comparar la distribución con la normal estándar:

- si $g_2 \approx 0$, la curtosis es similar a la de una normal;
- si $g_2 > 0$, la distribución exhibe colas relativamente pesadas y mayor propensión a valores extremos;
- si $g_2 < 0$, las colas son relativamente ligeras y la distribución resulta más plana.

La curtosis no debe leerse únicamente como “pico” central: en la práctica, suele ser más útil interpretarla en términos de **presencia de colas pesadas y extremos**.

38.4. Exploración gráfica inicial de la variable de ventas

Comenzamos con la variable de mayor interés económico: `Item_Outlet_Sales`.

```
sales = df["Item_Outlet_Sales"].dropna()

sales.describe()
```

38. Medidas de forma en Big Mart Sales

```
count      8523.000000
mean       2181.288914
std        1706.499616
min         33.290000
25%        834.247400
50%       1794.331000
75%       3101.296400
max       13086.964800
Name: Item_Outlet_Sales, dtype: float64
```

```
fig = px.histogram(
    df,
    x="Item_Outlet_Sales",
    nbins=50,
    marginal="box",
    title="Distribución de Item_Outlet_Sales"
)
fig.update_layout(bargap=0.05)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

A partir del histograma y el boxplot, se pueden responder varias preguntas sobre la forma de la distribución.

En primer lugar, **no se observa simetría visual**. La distribución presenta una clara **asimetría positiva (sesgo a la derecha)**: la mayor parte de las observaciones se concentra en valores relativamente bajos, mientras que una cola se extiende hacia valores de ventas más altos.

En segundo lugar, **sí aparecen colas largas**, especialmente hacia la derecha. Esto indica que, aunque la mayoría de los productos presenta niveles de ventas moderados, existen algunos casos con ventas considerablemente mayores que el resto.

En tercer lugar, **se observan posibles valores extremos**, lo cual también se confirma en el boxplot superior, donde varios puntos aparecen fuera de los bigotes. Estos puntos corresponden a observaciones con ventas muy superiores al comportamiento típico.

Finalmente, la gráfica muestra que **la masa principal de los datos se concentra en una región relativamente acotada**, aproximadamente en el rango bajo y medio de la distribución, mientras que un número reducido de observaciones alcanza valores de ventas mucho más altos. Esto sugiere que el comportamiento típico de las ventas está dominado por valores moderados, pero con algunos productos o combinaciones producto-tienda que generan ventas excepcionalmente elevadas.

Para reforzar la interpretación puede añadirse una curva de densidad aproximada.

```

import numpy as np
import plotly.graph_objects as go
from scipy import stats

xs = np.linspace(sales.min(), sales.max(), 400)
kde = stats.gaussian_kde(sales)

fig = go.Figure()

# Histograma en densidad
fig.add_trace(
    go.Histogram(
        x=sales,
        nbinsx=40,
        histnorm="probability density",
        opacity=0.6,
        name="Histograma"
    )
)

# Curva KDE
fig.add_trace(
    go.Scatter(
        x=xs,
        y=kde(xs),
        mode="lines",
        name="Densidad estimada",
        line=dict(width=2)
    )
)

fig.update_layout(
    title="Histograma y densidad estimada de Item_Outlet_Sales",
    xaxis_title="Item_Outlet_Sales",
    yaxis_title="Densidad",
    bargap=0.02,
    height=500,
    width=900
)

fig.show()

```

Unable to display output for mime type(s): text/html

La superposición del histograma con la curva de densidad permite observar con mayor claridad la forma general de la distribución. Mientras el histograma muestra la frecuencia relativa por intervalos, la curva KDE suaviza esa información y ayuda a identificar la concentración principal de los datos, la presencia de asimetría y la posible existencia de colas largas.

38.5. Cálculo de asimetría y curtosis

```
skew_sales = sales.skew()
kurt_sales = sales.kurt()

pd.DataFrame({
    "medida": ["asimetría", "curtosis_exceso"],
    "valor": [skew_sales, kurt_sales]
})
```

	medida	valor
0	asimetría	1.177531
1	curtosis_exceso	1.615877

Una interpretación orientativa puede apoyarse en criterios simples:

- $|asimetría| < 0.5$: asimetría baja;
- $0.5 \leq |asimetría| < 1$: asimetría moderada;
- $|asimetría| \geq 1$: asimetría marcada.

Estos umbrales no son absolutos, pero ayudan a construir una primera lectura del fenómeno. En datasets comerciales es frecuente observar asimetría positiva: muchos productos exhiben ventas moderadas, mientras un grupo reducido alcanza ventas muy elevadas.

38.6. Comparación de forma en varias variables numéricas

La forma no tiene por qué ser igual en todas las variables del conjunto de datos. Conviene comparar la asimetría y curtosis de todas las columnas numéricas disponibles.

```
shape_table = pd.DataFrame({
    "variable": num_cols,
    "asimetria": [df[c].dropna().skew() for c in num_cols],
    "curtosis_exceso": [df[c].dropna().kurt() for c in num_cols]
}).sort_values("asimetria", key=np.abs, ascending=False)

shape_table
```

	variable	asimetria	curtosis_exceso
4	Item_Outlet_Sales	1.177531	1.615877
1	Item_Visibility	1.167091	1.679445
3	Outlet_Establishment_Year	-0.396641	-1.205694
2	Item_MRP	0.127202	-0.889769
0	Item_Weight	0.082426	-1.227766

```
fig = px.bar(
    shape_table,
    x="variable",
    y="asimetria",
    title="Asimetría por variable numérica",
    text_auto=True
)
fig.show()
```

Unable to display output for mime type(s): text/html

```
fig = px.bar(
    shape_table,
    x="variable",
    y="curtosis_exceso",
    title="Curtosis (exceso) por variable numérica",
    text_auto=True
)
fig.show()
```

Unable to display output for mime type(s): text/html

La comparación de **asimetría** y **curtosis en exceso** muestra que no todas las variables numéricas presentan la misma estructura de forma. En particular, `Item_Outlet_Sales` exhibe una asimetría positiva de **1.177531** y una curtosis en exceso de **1.615877**, mientras que `Item_Visibility` presenta una asimetría de **1.167091** y una curtosis en exceso de **1.679445**. Estos valores sugieren distribuciones con **colas derechas largas** y una posible presencia de **valores extremos**, por lo que en ambas variables conviene reforzar el análisis con medidas robustas, boxplots y, si resulta útil, transformaciones como la logarítmica.

En cambio, `Item_MRP` muestra una asimetría mucho más baja (**0.127202**) y una curtosis en exceso negativa (**-0.889769**), mientras que `Item_Weight` presenta una asimetría de **0.082426** y una curtosis en exceso de **-1.227766**. Esto sugiere que ambas variables tienen una forma más cercana a la simetría y una distribución relativamente menos influida por colas pesadas.

Por su parte, `Outlet_Establishment_Year` registra una asimetría negativa de **-0.396641** y una curtosis en exceso de **-1.205694**. Aunque su forma no parece especialmente extrema, esta variable requiere una interpretación adicional debido a su naturaleza temporal, ya que su análisis no debe basarse únicamente en criterios de distribución continua.

En conjunto, estos resultados indican que el EDA debe adaptarse a la forma específica de cada variable y no aplicar una misma lectura descriptiva a todas por igual.

38.7. Transformaciones y su efecto sobre la forma

Cuando una variable presenta fuerte asimetría positiva, una transformación logarítmica puede facilitar tanto la visualización como el modelado posterior. En el caso de las ventas, una opción práctica es usar:

$$y^* = \log(1 + y),$$

que evita problemas cuando existen observaciones cercanas a cero.

```
df["log_Item_Outlet_Sales"] = np.log1p(df["Item_Outlet_Sales"])
log_sales = df["log_Item_Outlet_Sales"].dropna()

pd.DataFrame({
    "variable": ["Item_Outlet_Sales", "log_Item_Outlet_Sales"],
    "asimetria": [sales.skew(), log_sales.skew()],
    "curtosis_exceso": [sales.kurt(), log_sales.kurt()]
})
```

	variable	asimetria	curtosis_exceso
0	Item_Outlet_Sales	1.177531	1.615877
1	log_Item_Outlet_Sales	-0.882266	0.532920

```
fig = px.histogram(
    df,
    x="log_Item_Outlet_Sales",
    nbins=50,
    marginal="box",
    title="Distribución de log(1 + Item_Outlet_Sales)"
)
fig.show()
```

Unable to display output for mime type(s): text/html

```
xs_log = np.linspace(log_sales.min(), log_sales.max(), 400)
kde_log = stats.gaussian_kde(log_sales)
fig = go.Figure()
fig.add_trace(go.Histogram(
    x=log_sales, nbinsx=40, histnorm="probability density",
    opacity=0.55, name="Histograma"
))
fig.add_trace(go.Scatter(
    x=xs_log, y=kde_log(xs_log), mode="lines",
    line=dict(width=3), name="Densidad KDE"
))
fig.update_layout(
    title="Histograma y densidad de log(1 + Item_Outlet_Sales)",
    xaxis_title="log(1 + Item_Outlet_Sales)", yaxis_title="Densidad",
    barmode="overlay"
)
fig.show()
```


Unable to display output for mime type(s): text/html

La comparación entre la variable original y su versión transformada permite mostrar un punto importante del EDA: las medidas de forma no son meramente descriptivas, también orientan decisiones analíticas posteriores. Si la asimetría disminuye sustancialmente bajo la transformación, esto sugiere una representación más equilibrada del fenómeno.

En particular, una **transformación logarítmica** puede ser útil cuando la distribución presenta una **asimetría positiva marcada**, es decir, cuando existen valores muy altos que alargan la cola derecha de la distribución. Al aplicar el logaritmo, los valores grandes se comprimen relativamente más que los valores pequeños, lo que tiende a reducir la asimetría y a concentrar mejor los datos alrededor del centro.

Desde el punto de vista del EDA, esta transformación ayuda a:

- **visualizar mejor la estructura de la distribución**, evitando que unos pocos valores muy grandes dominen el gráfico;
- **comparar observaciones en términos relativos**, ya que las diferencias en escala logarítmica representan cambios proporcionales;
- **reducir la influencia de valores extremos**, facilitando una lectura más clara del comportamiento típico de los datos.

Además, esta transformación puede ser útil para **análisis posteriores**, especialmente cuando se emplean métodos estadísticos o modelos que funcionan mejor con distribuciones más simétricas o con variabilidad más estable. En contextos de modelado, por ejemplo, trabajar con una variable transformada puede mejorar la interpretación de los resultados, estabilizar la varianza y facilitar el ajuste de modelos lineales o regresiones.

En síntesis, el uso del logaritmo en el EDA no solo mejora la interpretación visual de distribuciones muy asimétricas, sino que también prepara los datos para etapas posteriores del análisis, donde una representación más equilibrada de la variable puede resultar analíticamente más conveniente.

38.8. Gráfico cuantil-cuantil como apoyo visual

Otra herramienta útil para estudiar la forma es el gráfico Q-Q frente a la distribución normal. No debe interpretarse como una prueba definitiva, pero sí como un recurso visual para detectar desviaciones sistemáticas respecto a simetría y colas gaussianas.

```
def qq_plotly(x, titulo):
    (teoricos, observados), (pendiente, intercepto, _) = stats.probplot(x, dist="norm")
    linea_x = np.array([teoricos.min(), teoricos.max()])
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=teoricos, y=observados, mode="markers", name="Cuantiles"))
    fig.add_trace(go.Scatter(
        x=linea_x, y=intercepto + pendiente * linea_x,
        mode="lines", name="Referencia normal", line=dict(dash="dash")
    ))
    fig.update_layout(title=titulo, xaxis_title="Cuantiles teóricos",
                      yaxis_title="Cuantiles observados")
    return fig

qq_plotly(sales, "Gráfico Q-Q de Item_Outlet_Sales").show()
```

Unable to display output for mime type(s): text/html

```
qq_plotly(log_sales, "Gráfico Q-Q de log(1 + Item_Outlet_Sales)").show()
```

Unable to display output for mime type(s): text/html

Si la versión transformada se alinea mejor con la recta de referencia que la variable original, ello aporta evidencia visual de una mejora en la regularidad de la forma, aunque no implica normalidad perfecta.

38.9. Mini-dashboard de forma

La siguiente sección integra un dashboard simple para resumir la forma de la variable de ventas y su transformación.

```
from plotly.subplots import make_subplots

fig = make_subplots(
    rows=2,
    cols=2,
    subplot_titles=(
        "Ventas originales",
        "Ventas transformadas con log(1+y)",
        "Asimetría comparada",
        "Curtosis comparada"
    )
)

fig.add_trace(
    go.Histogram(x=sales, nbinsx=40, name="Ventas originales"),
    row=1, col=1
)
fig.add_trace(
    go.Histogram(x=log_sales, nbinsx=40, name="Ventas log"),
    row=1, col=2
)
fig.add_trace(
    go.Bar(
        x=["Original", "Log"],
        y=[sales.skew(), log_sales.skew()],
        name="Asimetría"
    ),
    row=2, col=1
)
fig.add_trace(
    go.Bar(
        x=["Original", "Log"],
        y=[sales.kurt(), log_sales.kurt()],
```

```
        name="Curtosis"  
    ),  
    row=2, col=2  
)  
  
fig.update_layout(  
    title="Dashboard básico de medidas de forma",  
    height=700,  
    showlegend=False  
)  
fig.show()
```

Unable to display output for mime type(s): text/html

Parte XV.

Medidas de heterogeneidad

39. Medidas de heterogeneidad

39.1. El problema: ¿qué significa que unos datos sean heterogéneos?

Decir que unos datos son **heterogéneos** significa que no están concentrados en una sola posibilidad. Sin embargo, la pregunta concreta depende del tipo de variable:

Tipo de datos	Pregunta de heterogeneidad	Poca heterogeneidad	Mucha heterogeneidad
Catégoricos	¿Cómo se reparte la frecuencia entre las clases?	Casi todas las observaciones pertenecen a una sola clase.	Las observaciones se reparten de forma equilibrada entre varias clases.
Núméricos agrupados	¿Cuánto cambia la distribución de una respuesta entre grupos?	Los grupos tienen centros y formas semejantes.	La pertenencia al grupo ayuda a anticipar el valor numérico.

Esta distinción evita un error frecuente: una variable catégorica no tiene distancias numéricas naturales entre sus etiquetas, mientras que una variable numérica sí permite medir desviaciones respecto a una media. Por eso se necesitan construcciones diferentes.

En la primera mitad del capítulo seguiremos un único ejemplo: una bolsa contiene seis canicas de tres clases, cada clase identificada por un color.

Clase	Color	Número de canicas	Proporción
<i>A</i>	azul	3	$p_A = 3/6 = 1/2$
<i>B</i>	naranja	1	$p_B = 1/6$
<i>C</i>	verde	2	$p_C = 2/6 = 1/3$
Total		6	1

La variable aleatoria¹ X registra la clase de una canica extraída. Sus valores posibles son $\{A, B, C\}$ y su distribución es

$$p = (p_A, p_B, p_C) = \left(\frac{1}{2}, \frac{1}{6}, \frac{1}{3}\right).$$

Las clases son mutuamente excluyentes y exhaustivas:² cada extracción produce exactamente una de las tres etiquetas. La heterogeneidad catégorica dependerá de estas proporciones, no del nombre ni del color asignado a cada clase.

¹Una **variable aleatoria** es una regla que asigna un valor a cada resultado de un experimento. Aquí no significa que las etiquetas sean números: X asigna la etiqueta A , B o C a la canica observada.

²**Mutuamente excluyentes** significa que una canica no puede pertenecer simultáneamente a dos clases. **Exhaustivas** significa que las tres clases cubren todos los resultados posibles.

Dos casos que servirán como brújula

- Si las seis canicas fueran de clase A , conocer la clase de una extracción sería trivial: la heterogeneidad debería valer **cero**.
- Si hubiera dos canicas de cada clase, ninguna clase dominaría: con tres clases, la heterogeneidad debería ser **máxima**.

Toda medida que usemos deberá respetar estos dos extremos.

39.2. Heterogeneidad categórica I: Gini–Simpson**39.2.1. De una extracción a dos extracciones**

Al extraer una canica de la bolsa completa, la probabilidad³ de obtener cada clase es su fracción dentro del total:

$$P(X = A) = \frac{3}{6}, \quad P(X = B) = \frac{1}{6}, \quad P(X = C) = \frac{2}{6}.$$

Una sola extracción informa cuál clase apareció, pero todavía no produce una noción directa de diversidad. Para obtenerla, planteemos otro experimento:

Se extrae una canica, se registra su clase, se devuelve a la bolsa, se mezcla y se extrae una segunda canica. ¿Cuál es la probabilidad de que las dos clases sean diferentes?

Devolver la primera canica significa muestrear **con reemplazo**.⁴ Gracias a ello, la composición de la bolsa es la misma antes de ambas extracciones. Supondremos además que las extracciones son **independientes**:⁵ saber qué ocurrió en la primera no cambia las probabilidades de la segunda.

La independencia permite multiplicar probabilidades. Por ejemplo,

$$P(X_1 = A, X_2 = A) = P(X_1 = A)P(X_2 = A) = \frac{3}{6} \frac{3}{6} = \frac{9}{36}.$$

Del mismo modo,

$$P(B, B) = \left(\frac{1}{6}\right)^2 = \frac{1}{36}, \quad P(C, C) = \left(\frac{2}{6}\right)^2 = \frac{4}{36}.$$

La tabla contiene los $6 \times 6 = 36$ pares ordenados de canicas que pueden obtenerse. Las celdas diagonales corresponden a coincidencias de clase.

³En este ejemplo equiprobable a nivel de canica, la probabilidad de una clase es el número de canicas de esa clase dividido entre el total. En una interpretación frecuentista, es también la proporción a la que tendería la frecuencia de esa clase al repetir muchas veces el experimento bajo las mismas condiciones.

⁴Muestrear **con reemplazo** consiste en devolver cada unidad antes de la siguiente selección. Así, las probabilidades permanecen constantes. Sin reemplazo, después de extraer una canica quedarían cinco y la segunda probabilidad dependería de la primera extracción.

⁵Dos variables aleatorias X_1 y X_2 son **independientes** si conocer el valor de una no modifica la distribución de la otra. Formalmente, $P(X_1 = j, X_2 = k) = P(X_1 = j)P(X_2 = k)$ para todo par (j, k) .

Primera \ segunda	A	B	C	Total de la fila
A	9/36	3/36	6/36	18/36
B	3/36	1/36	2/36	6/36
C	6/36	2/36	4/36	12/36
Total de la columna	18/36	6/36	12/36	36/36

Por tanto, la probabilidad de obtener **la misma clase** es

$$P(X_1 = X_2) = \frac{9 + 1 + 4}{36} = \frac{14}{36} = \frac{7}{18}.$$

Los eventos “misma clase” y “clases diferentes” son complementarios,⁶ de modo que

$$P(X_1 \neq X_2) = 1 - P(X_1 = X_2) = 1 - \frac{7}{18} = \frac{11}{18} \approx 0.6111.$$

Ya tenemos una medida interpretable: en aproximadamente el 61.1 % de los pares extraídos de manera independiente y con reemplazo, las clases serán diferentes.

39.2.2. Definición formal

Sea $p = (p_1, \dots, p_K)$ una distribución sobre K categorías. El índice de **Gini–Simpson** (Simpson 1949) es

$$D(p) = 1 - \sum_{k=1}^K p_k^2. \quad (1)$$

La fórmula se obtiene directamente del experimento anterior:

$$\underbrace{\sum_{k=1}^K p_k^2}_{P(X_1=X_2)} + \underbrace{D(p)}_{P(X_1 \neq X_2)} = 1.$$

También puede escribirse como

$$D(p) = \sum_{j \neq k} p_j p_k,$$

que suma todos los pares ordenados de clases distintas. Esta es la razón probabilística —no solo algebraica— del cuadrado en p_k^2 .

⁶Si E es un evento, su complemento E^c reúne todos los resultados en los que E no ocurre. Como son mutuamente excluyentes y cubren todo el espacio, $P(E^c) = 1 - P(E)$.

39.2.3. El cuadrado de probabilidades

Imagine un cuadrado de área uno. El eje horizontal representa la segunda extracción y el vertical la primera. Al dividir ambos ejes en segmentos de longitudes p_A , p_B y p_C , aparecen nueve rectángulos:

- los tres bloques diagonales tienen áreas p_A^2 , p_B^2 y p_C^2 y representan coincidencias;
- los seis bloques restantes tienen áreas $p_j p_k$ con $j \neq k$ y representan diferencias.

El laboratorio interactivo de la versión HTML permite cambiar los conteos de A , B y C . En la distribución inicial $(3, 1, 2)$, el área de coincidencia es $7/18$ y el área complementaria es $D = 11/18$.

39.2.4. Cotas e interpretación

Si una sola clase concentra toda la probabilidad, entonces $\sum p_k^2 = 1$ y $D = 0$. Por la desigualdad de Cauchy–Schwarz, $\sum_k p_k^2 \geq 1/K$, con igualdad cuando todas las probabilidades son iguales. Por tanto, si las K clases son equiprobables, $p_k = 1/K$, se alcanza

$$D_{\max} = 1 - K \left(\frac{1}{K} \right)^2 = 1 - \frac{1}{K}. \quad (2)$$

Así, con tres clases $D \in [0, 2/3]$. El valor 0.6111 de nuestras canicas está cerca del máximo 0.6667, pero no debe interpretarse como “61.11 % del máximo”. Para esa lectura se normaliza:

$$D_{\text{norm}} = \frac{D}{1 - 1/K}.$$

En el ejemplo, $D_{\text{norm}} = (11/18)/(2/3) = 11/12 \approx 0.9167$.

No confundir dos índices llamados Gini

La diversidad de Gini–Simpson mide la probabilidad de desacuerdo entre categorías. El **coeficiente de Gini** asociado con la curva de Lorenz mide desigualdad en una variable numérica, como ingreso o riqueza. Comparten parte del nombre, pero responden preguntas diferentes.

39.3. Heterogeneidad categórica II: de la sorpresa a la entropía

Gini–Simpson construye diversidad comparando dos extracciones. Shannon toma otro camino: mide cuánta información aporta observar una sola extracción.

39.3.1. La sorpresa de un resultado

Antes de extraer una canica, compare dos resultados:

- obtener A no sería muy sorprendente, porque ocurre con probabilidad $1/2$;
- obtener B sería más sorprendente, porque ocurre con probabilidad $1/6$.

La **información propia** o sorpresa de un evento con probabilidad p se define como

$$I_b(p) = -\log_b(p), \quad b > 1. \quad (3)$$

Esta elección cumple propiedades deseables: un evento seguro tiene sorpresa cero, los eventos menos probables producen mayor sorpresa y la sorpresa de dos eventos independientes se suma.⁷

En bits, es decir, usando base 2,

$$I_2(A) = 1, \quad I_2(B) = \log_2(6) \approx 2.585, \quad I_2(C) = \log_2(3) \approx 1.585.$$

39.3.2. De la sorpresa individual a la sorpresa promedio

No basta con promediar esos tres números dando el mismo peso a cada clase: A aparece más veces que B . La sorpresa media debe ponderar cada resultado por la probabilidad con que ocurre. Ese valor esperado⁸ es la **entropía de Shannon** (Shannon 1948):

$$H_b(p) = \mathbb{E}[I_b(X)] = -\sum_{k=1}^K p_k \log_b(p_k). \quad (4)$$

Para las seis canicas y base 2:

Clase	p_k	Sorpresa $-\log_2 p_k$	Aporte promedio $-p_k \log_2 p_k$
A	0.5000	1.0000	0.5000
B	0.1667	2.5850	0.4308
C	0.3333	1.5850	0.5283
Total	1		$H_2 = 1.4591$ bits

Observe la diferencia entre **sorpresa** y **aporte a la entropía**. B es el resultado más sorprendente, pero aparece poco; C aporta más al promedio porque combina sorpresa considerable con mayor frecuencia.

Cuando $p_k = 0$, la expresión contiene formalmente $0 \log 0$. Se adopta la extensión continua

$$0 \log 0 := \lim_{p \rightarrow 0^+} p \log p = 0,$$

porque una clase imposible nunca llega a aportar sorpresa al promedio.

⁷Para eventos independientes con probabilidades p y q , $I_b(pq) = -\log_b(pq) = -\log_b p - \log_b q = I_b(p) + I_b(q)$.

⁸El **valor esperado** de una variable discreta es su promedio teórico ponderado: $\mathbb{E}[Z] = \sum_z zP(Z = z)$. No asigna el mismo peso a resultados frecuentes y raros.

39.3.3. ¿Dónde está la entropía máxima?

Con K fijo, la entropía satisface

$$0 \leq H_b(p) \leq \log_b K. \quad (5)$$

El mínimo ocurre cuando una sola clase tiene probabilidad uno. El máximo ocurre en la distribución uniforme $p_k = 1/K$. Puede verificarse con multiplicadores de Lagrange: al maximizar $-\sum p_k \ln p_k$ sujeto a $\sum p_k = 1$, las condiciones de primer orden exigen que todos los p_k sean iguales; la restricción obliga a que valgan $1/K$. Como la matriz hessiana tiene diagonal $-1/p_k < 0$, la entropía es estrictamente cóncava en el interior del simplex y ese punto crítico es el máximo global único.

Para comparar variables con distinto número de clases conviene usar

$$H_{\text{norm}} = \frac{H_b(p)}{\log_b K} \in [0, 1]. \quad (6)$$

En nuestro ejemplo,

$$H_{\text{norm}} = \frac{1.4591}{\log_2 3} = 0.9206.$$

Dos máximos que no deben confundirse

La contribución de **una sola clase**, $f(p) = -p \ln p$, alcanza su máximo en $p = 1/e \approx 0.3679$. En cambio, la entropía **de toda la distribución**, sujeta a que las probabilidades sumen uno, alcanza su máximo en $p_k = 1/K$. Para tres clases, el máximo global está en $(1/3, 1/3, 1/3)$, no en tres probabilidades iguales a $1/e$, pues estas últimas ni siquiera sumarían uno.

En efecto,

$$f'(p) = -\ln p - 1 = 0 \iff p = 1/e, \quad f''(p) = -1/p < 0.$$

39.3.4. Cambiar la base cambia la escala, no la conclusión

Por la fórmula de cambio de base,

$$H_b(p) = \frac{H_e(p)}{\ln b}.$$

- base 2: bits;
- base e : nats;
- base 10: hartleys.

La base multiplica todas las entropías por una constante positiva. Por tanto, no cambia qué distribución es más heterogénea. Tampoco cambia H_{norm} , porque numerador y denominador se escalan por el mismo factor.

En la versión HTML, un diagrama ternario permite desplazar la distribución (p_A, p_B, p_C) por todo el simplex. La entropía normalizada aumenta al acercarse al centro $(1/3, 1/3, 1/3)$ y disminuye al aproximarse a cualquiera de los vértices.

39.4. Comparar Gini–Simpson y Shannon

Los dos índices miden heterogeneidad, pero observan la distribución desde experimentos distintos:

Índice	Experimento mental	Fórmula	Mayor sensibilidad relativa
Gini–Simpson	¿Dos extracciones tendrán clases distintas?	$1 - \sum p_k^2$	Clases dominantes; las probabilidades se elevan al cuadrado.
Shannon	¿Cuánta sorpresa produce, en promedio, una extracción?	$-\sum p_k \log p_k$	Clases raras; una probabilidad pequeña conserva aporte mediante $-p \log p$.

No conviene comparar directamente D y H , porque tienen escalas y máximos diferentes. Con K fijo, compare D_{norm} con H_{norm} :

Distribución (p_A, p_B, p_C)	D	D_{norm}	H (nats)	H_{norm}
(1, 0, 0)	0.0000	0.0000	0.0000	0.0000
(0.90, 0.05, 0.05)	0.1850	0.2775	0.3944	0.3590
(0.60, 0.30, 0.10)	0.5400	0.8100	0.8979	0.8173
(0.50, 0.1667, 0.3333)	0.6111	0.9167	1.0114	0.9206
(1/3, 1/3, 1/3)	0.6667	1.0000	1.0986	1.0000

Ambos ordenan estos ejemplos de la misma manera. Sin embargo, Shannon asigna relativamente más diversidad al escenario (0.90, 0.05, 0.05) porque reconoce la información aportada por las dos clases raras.

Regla de elección

- Use **Gini–Simpson** cuando la interpretación por pares o la probabilidad de desacuerdo sea natural.
- Use **Shannon** cuando la incertidumbre, codificación o sorpresa sea central.
- Si necesita comunicar “cuántas categorías igualmente frecuentes equivalen a esta distribución”, transforme ambos a **números efectivos de Hill**.

39.5. Una escala común: números de Hill

Los índices anteriores no están expresados en “número de categorías”. La familia de Hill (Hill 1973; Jost 2006) los convierte a esa escala:

$${}^qD = \left(\sum_{k=1}^K p_k^q \right)^{1/(1-q)}, \quad q \geq 0, \quad q \neq 1. \quad (7)$$

El parámetro q controla cuánto pesan las clases raras:

39. Medidas de heterogeneidad

1. $q = 0$, **riqueza observada**: ${}^0D = K$; solo cuenta las clases presentes.
2. $q \rightarrow 1$, **diversidad de Shannon**: ${}^1D = e^{H_e}$.
3. $q = 2$, **diversidad de Simpson**: ${}^2D = 1 / \sum p_k^2 = 1 / (1 - D)$.

Para las canicas,

$${}^0D = 3, \quad {}^1D = e^{1.0114} = 2.750, \quad {}^2D = \frac{1}{7/18} = \frac{18}{7} = 2.571.$$

La interpretación de ${}^1D = 2.750$ es concreta: la distribución $(3, 1, 2)$ tiene la misma entropía que una distribución ideal con 2.750 categorías equiprobables. En general,

$${}^0D \geq {}^1D \geq {}^2D, \quad (8)$$

y los tres valores coinciden con K únicamente bajo uniformidad perfecta.

39.6. Heterogeneidad de una variable numérica entre grupos

Ahora cambia la pregunta. Sea Y una respuesta numérica y sea G una variable categórica que identifica grupos. Ya no preguntamos si dos etiquetas son distintas, sino:

¿Qué parte de la variación numérica de Y se asocia con diferencias entre los grupos?

Usaremos seis observaciones:

$$A = (4, 5), \quad B = (8, 9), \quad C = (5, 5).$$

La media global y las medias de grupo son

$$\bar{y} = 6, \quad \bar{y}_A = 4.5, \quad \bar{y}_B = 8.5, \quad \bar{y}_C = 5.$$

39.6.1. La identidad que separa dos fuentes de variación

Para una observación y_{gi} del grupo g ,

$$y_{gi} - \bar{y} = \underbrace{(y_{gi} - \bar{y}_g)}_{\text{desviación dentro del grupo}} + \underbrace{(\bar{y}_g - \bar{y})}_{\text{desviación entre grupos}}.$$

Al elevar al cuadrado y sumar todas las observaciones, el término cruzado desaparece porque $\sum_i (y_{gi} - \bar{y}_g) = 0$ dentro de cada grupo. Se obtiene la descomposición exacta de ANOVA:

$$\underbrace{\sum_g \sum_i (y_{gi} - \bar{y})^2}_{SS_T} = \underbrace{\sum_g n_g (\bar{y}_g - \bar{y})^2}_{SS_B} + \underbrace{\sum_g \sum_i (y_{gi} - \bar{y}_g)^2}_{SS_W}. \quad (9)$$

- SS_T : variación total respecto a la media global;
- SS_B : variación **entre** medias grupales (*between*);
- SS_W : variación **dentro** de los grupos (*within*).

39.6.2. Cálculo completo

La variación total es

$$SS_T = (4 - 6)^2 + (5 - 6)^2 + (8 - 6)^2 + (9 - 6)^2 + (5 - 6)^2 + (5 - 6)^2 = 20.$$

La parte entre grupos es

$$\begin{aligned} SS_B &= 2(4.5 - 6)^2 + 2(8.5 - 6)^2 + 2(5 - 6)^2 \\ &= 4.5 + 12.5 + 2 = 19. \end{aligned}$$

La parte interna es

$$\begin{aligned} SS_W &= [(4 - 4.5)^2 + (5 - 4.5)^2] \\ &\quad + [(8 - 8.5)^2 + (9 - 8.5)^2] + [(5 - 5)^2 + (5 - 5)^2] \\ &= 0.5 + 0.5 + 0 = 1. \end{aligned}$$

La identidad se comprueba: $20 = 19 + 1$.

39.6.3. Eta cuadrada y omega cuadrada

La proporción descriptiva de variación explicada por las diferencias de medias es

$$\eta^2 = \frac{SS_B}{SS_T} \in [0, 1]. \quad (10)$$

En el ejemplo, $\eta^2 = 19/20 = 0.95$: conocer el grupo explica el 95 % de la variación cuadrática observada. Esto no significa que el grupo “cause” el resultado; es una descripción de asociación en estos datos.

En muestras pequeñas, η^2 tiende a sobreestimar el efecto poblacional. Una corrección habitual es

$$\omega^2 = \frac{SS_B - (G - 1)MS_W}{SS_T + MS_W}, \quad MS_W = \frac{SS_W}{N - G}. \quad (11)$$

Aquí, $MS_W = 1/(6 - 3) = 1/3$ y

$$\omega^2 = \frac{19 - 2(1/3)}{20 + 1/3} = 0.9016.$$

La corrección reduce el optimismo de 0.9500 a 0.9016. En muestras reales, ω^2 puede resultar ligeramente negativa; eso se interpreta como ausencia de evidencia de variación explicada, aunque algunas implementaciones la truncan a cero para presentación.

El laboratorio de la versión HTML representa cada grupo mediante dos observaciones, “centro $\pm$ separación”. Al alejar las medias de los grupos aumenta SS_B ; al separar las observaciones dentro de cada grupo aumenta SS_W .

39.6.4. Lo que η^2 sí ve y lo que no ve

η^2 resume diferencias de **medias**. No detecta por sí sola todos los tipos de heterogeneidad distributiva. Dos grupos pueden tener la misma media y, aun así, diferir en:

- dispersión;
- asimetría;
- número o severidad de valores atípicos;
- multimodalidad.

Por ello, la descomposición ANOVA debe acompañarse con tamaños muestrales y gráficos de las distribuciones: diagramas de caja, violines, densidades o funciones de distribución empírica. Una cifra alta tampoco demuestra causalidad; puede reflejar confusión, selección o diseño muestral.

39.7. Heterogeneidad cruzada entre variables categóricas

Cuando se cruzan dos variables categóricas, la pregunta pasa a ser si la composición de una cambia entre niveles de la otra. Para cada grupo g , compare las probabilidades condicionales⁹

$$p_{k|g} = P(X = k \mid G = g).$$

Una tabla de conteos responde cuánto soporte tiene cada combinación; una tabla de porcentajes por fila permite comparar composiciones. El mapa de calor debe mostrar ambas cosas o, como mínimo, acompañar cada porcentaje con su n_{kg} .

Celdas pequeñas

Las combinaciones con pocos casos generan porcentajes inestables. No existe un umbral universal que vuelva confiable una celda; la regla práctica $n < 10$ solo funciona como alerta inicial. Revise conteos, incertidumbre y mecanismo de muestreo antes de interpretar una diferencia como estructura real.

Si se necesita un resumen formal de asociación, una prueba χ^2 contrasta independencia y la V de Cramér cuantifica intensidad. Estas herramientas responden una pregunta relacional distinta de la diversidad marginal: una variable puede ser muy diversa y, al mismo tiempo, ser independiente de la otra.

39.8. Puentes con algoritmos de minería de datos

Algoritmo	Medida de heterogeneidad	Uso
Árboles CART	Impureza de Gini $1 - \sum p_k^2$	Elige el corte que produce nodos hijos más puros mediante la reducción ponderada de impureza.

⁹La probabilidad condicional $P(X = k \mid G = g)$ restringe el universo a los casos del grupo g y calcula dentro de él la proporción que pertenece a la clase k .

Algoritmo	Medida de heterogeneidad	Uso
ID3 / C4.5	Entropía de Shannon	ID3 maximiza ganancia de información; C4.5 la ajusta con <i>gain ratio</i> para evitar cortes con demasiadas ramas.
k -means	SS_W o inercia	Busca centroides que minimicen la variación dentro de los grupos. Con datos fijos, minimizar SS_W equivale a maximizar SS_B .
Calinski–Harabasz	Razón tipo ANOVA	Compara separación entre <i>clusters</i> con compactación interna: $CH = (SS_B/(k-1))/(SS_W/(n-k))$.

En un árbol, “impureza de Gini” es precisamente Gini–Simpson aplicado a la distribución de clases dentro de un nodo. En *clustering*, los grupos no vienen dados: el algoritmo intenta construirlos para reducir heterogeneidad interna.

39.9. Implementación reproducible

Las siguientes funciones reproducen los cálculos del capítulo y hacen explícitas las convenciones para probabilidades nulas.

```
import numpy as np
import pandas as pd

def indices_categoricos(conteos, base=np.e):
    conteos = np.asarray(conteos, dtype=float)
    if np.any(conteos < 0) or conteos.sum() == 0:
        raise ValueError("Los conteos deben ser no negativos y sumar más de cero.")

    p = conteos / conteos.sum()
    p_positivas = p[p > 0]
    k = len(p_positivas)
    h = -(p_positivas * (np.log(p_positivas) / np.log(base))).sum()
    gini_simpson = 1 - np.square(p).sum()

    return pd.Series({
        "K observado": k,
        "Gini-Simpson": gini_simpson,
        "Gini-Simpson normalizado": gini_simpson / (1 - 1 / k) if k > 1 else 0,
        "Entropía": h,
        "Entropía normalizada": h / (np.log(k) / np.log(base)) if k > 1 else 0,
        "Hill q=1": base ** h,
        "Hill q=2": 1 / np.square(p).sum(),
    })
```

```
indices_categoricos([3, 1, 2], base=np.e).round(4)
```

```
K observado          3.0000
Gini-Simpson          0.6111
Gini-Simpson normalizado 0.9167
Entropía              1.0114
Entropía normalizada  0.9206
Hill q=1              2.7495
Hill q=2              2.5714
dtype: float64
```

```
def descomposicion_anova(valores, grupos):
    datos = pd.DataFrame({"y": valores, "grupo": grupos})
    media_global = datos["y"].mean()
    medias_grupo = datos.groupby("grupo")["y"].transform("mean")
    ss_t = np.square(datos["y"] - media_global).sum()
    ss_w = np.square(datos["y"] - medias_grupo).sum()
    ss_b = ss_t - ss_w
    n, g = len(datos), datos["grupo"].nunique()
    ms_w = ss_w / (n - g)
    eta2 = ss_b / ss_t if ss_t > 0 else np.nan
    omega2 = (ss_b - (g - 1) * ms_w) / (ss_t + ms_w) if ss_t + ms_w > 0 else np.nan
    return pd.Series({"SS_T": ss_t, "SS_B": ss_b, "SS_W": ss_w,
                      "eta²": eta2, "omega²": omega2})

descomposicion_anova(
    valores=[4, 5, 8, 9, 5, 5],
    grupos=["A", "A", "B", "B", "C", "C"],
).round(4)
```

```
SS_T      20.0000
SS_B      19.0000
SS_W       1.0000
eta²       0.9500
omega²     0.9016
dtype: float64
```

39.10. Guía de decisión

Pregunta	Herramienta principal	Comprobación necesaria
¿Qué tan repartida está una variable categórica?	Barras, Gini-Simpson, Shannon	Reportar K , conteos y versión normalizada si se comparan cardinalidades.
¿Cuántas clases efectivas hay?	Números de Hill	Indicar el orden q y su sensibilidad a clases raras.

Pregunta	Herramienta principal	Comprobación necesaria
¿Cambian las medias numéricas entre grupos?	$SS_B, SS_W, \eta^2, \omega^2$	Inspeccionar tamaños, dispersión y forma de cada grupo.
¿Cambia una composición categórica entre grupos?	Proporciones condicionales, mapa de calor, χ^2 , V de Cramér	Mostrar soporte de cada celda.
¿Hay heterogeneidad causal?	Ningún índice descriptivo basta	Se requiere diseño causal y control de variables de confusión.

39.11. Puente al caso BigMart

El capítulo práctico Medidas de heterogeneidad en Big Mart Sales traslada estas ideas a datos de *retail*:

- diversidad de categorías de producto mediante Shannon, Gini–Simpson y números de Hill;
- heterogeneidad de ventas entre formatos de establecimiento mediante η^2 y ω^2 ;
- composiciones cruzadas entre ubicación, tipo y antigüedad del establecimiento.

La secuencia recomendada es siempre la misma: **ver los conteos o distribuciones, formular el experimento que da significado al índice, calcularlo y finalmente interpretar su escala.**

39.12. Ejercicios formativos

1. **Recupere el significado probabilístico.** Para $p = (0.7, 0.2, 0.1)$, construya la matriz de probabilidades $p_j p_k$. Sume la diagonal y su complemento. Explique el resultado como frecuencia esperada de desacuerdos en 100 pares.
2. **Sorpresa frente a aporte.** Para las mismas probabilidades, calcule $I_2(p_k)$ y $p_k I_2(p_k)$. Identifique qué clase sorprende más y cuál aporta más a la entropía. ¿Son necesariamente la misma?
3. **Escala y base.** Calcule H en bits y nats. Compruebe que H_{norm} es el mismo en ambas bases.
4. **Números efectivos.** Calcule 0D , 1D y 2D para $p = (0.7, 0.2, 0.1)$ y verifique ${}^0D \geq {}^1D \geq {}^2D$.
5. **Descomposición numérica.** Para $G_1 = (10, 12)$ y $G_2 = (20, 22)$, calcule SS_T , SS_B , SS_W , η^2 y ω^2 .
6. **Límite de η^2 .** Construya dos grupos con la misma media y dispersiones muy distintas. Muestre que $\eta^2 = 0$ y explique por qué eso no demuestra que las distribuciones sean iguales.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.

- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.

- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.

- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*.

- Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2):

427-32. <https://doi.org/10.2307/1934352>.

Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.

Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.

Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.

Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.

Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.

Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.

Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.

Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.

IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.

Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.

James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.

Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.

Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.

Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.

- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.

- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.

- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias,

Universidad Nacional Autónoma de México.

- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.

- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

40. Medidas de heterogeneidad en Big Mart Sales

40.1. Introducción

Este caso aplica la diversidad categórica y la comparación entre grupos desarrolladas en Medidas de heterogeneidad.

Después de estudiar la localización, la variabilidad y la forma de las distribuciones, el análisis exploratorio de datos debe avanzar hacia una pregunta más estructural: **¿el comportamiento observado es homogéneo en toda la población o cambia de manera importante entre grupos?**. Esta cuestión introduce el concepto de **heterogeneidad**, una dimensión central del EDA cuando el dataset contiene segmentaciones naturales, como categorías de producto, tipos de tienda, tamaños de outlet o ubicaciones geográficas.

En términos estadísticos, la heterogeneidad alude a la existencia de diferencias sistemáticas entre subpoblaciones. Tales diferencias pueden expresarse en el centro de la distribución, en su dispersión, en su forma o incluso en la presencia diferencial de valores extremos. En un problema de negocio como BigMart, detectar heterogeneidad es indispensable, pues permite identificar si ciertos tipos de tiendas venden consistentemente más, si algunas categorías presentan patrones de ventas más variables, o si la estructura comercial cambia según el entorno del outlet.

En este capítulo se trabaja la heterogeneidad como una extensión natural del análisis univariado. La idea principal es que una variable numérica no debe interpretarse de manera aislada, sino también en relación con variables categóricas relevantes.

Definición operativa. La heterogeneidad aparece cuando la distribución de ventas cambia entre subgrupos o cuando la composición categórica presenta diversidad relevante (Shannon 1948; Simpson 1949).

40.2. Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

DATA_PATH = "../data/bigmart_sales.csv"
df = pd.read_csv(DATA_PATH)
df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	C
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	C
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	C
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	C
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	C

Conviene verificar las variables categóricas y numéricas del conjunto de datos.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Item_Identifier                       8523 non-null   object
1   Item_Weight                           7060 non-null   float64
2   Item_Fat_Content                      8523 non-null   object
3   Item_Visibility                       8523 non-null   float64
4   Item_Type                             8523 non-null   object
5   Item_MRP                             8523 non-null   float64
6   Outlet_Identifier                     8523 non-null   object
7   Outlet_Establishment_Year             8523 non-null   int64
8   Outlet_Size                           6113 non-null   object
9   Outlet_Location_Type                  8523 non-null   object
10  Outlet_Type                           8523 non-null   object
11  Item_Outlet_Sales                     8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

```
num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(exclude=np.number).columns.tolist()
```

```
num_cols, cat_cols
```

```
(['Item_Weight',
 'Item_Visibility',
 'Item_MRP',
 'Outlet_Establishment_Year',
 'Item_Outlet_Sales'],
 ['Item_Identifier',
 'Item_Fat_Content',
 'Item_Type',
 'Outlet_Identifier',
 'Outlet_Size',
 'Outlet_Location_Type',
 'Outlet_Type'])
```


40.3. Fundamentos teóricos

La heterogeneidad puede analizarse desde varias perspectivas. Si Y es una variable numérica de interés y G una variable categórica que define grupos, pueden estudiarse diferencias en:

40.3.1. Centro por grupo

Se comparan medidas como:

$$\bar{y}_g = \frac{1}{n_g} \sum_{i \in g} y_i,$$

así como la mediana y otros cuantiles dentro de cada grupo g .

40.3.2. Dispersión por grupo

La variabilidad interna de cada subpoblación puede resumirse mediante la desviación estándar, el rango intercuartílico o el coeficiente de variación. Dos grupos pueden tener medias semejantes y, sin embargo, diferir de forma sustancial en estabilidad o consistencia.

40.3.3. Forma por grupo

La heterogeneidad también puede aparecer en la asimetría o en la presencia de colas pesadas. Algunos grupos pueden concentrarse en rangos estrechos y otros exhibir ventas con distribución muy extendida.

40.3.4. Participación relativa de cada grupo

Finalmente, también es útil estudiar cuánto contribuye cada grupo al total de observaciones o al total de ventas. Esta dimensión conecta la heterogeneidad con el estudio posterior de la concentración.

40.4. Variable objetivo y grupos de interés

En este capítulo tomaremos nuevamente como variable principal a `Item_Outlet_Sales`. Como ejes de segmentación se consideran, al menos, las siguientes variables categóricas:

- `Outlet_Type`,
- `Outlet_Size`,
- `Outlet_Location_Type`,
- `Item_Type`.

Antes de comparar grupos, es útil revisar su tamaño.

```
for col in ["Outlet_Type", "Outlet_Size", "Outlet_Location_Type", "Item_Type"]:
    print(f"\n{col}")
    print(df[col].value_counts(dropna=False))
```

40. Medidas de heterogeneidad en Big Mart Sales

```
Outlet_Type
Outlet_Type
Supermarket Type1    5577
Grocery Store        1083
Supermarket Type3     935
Supermarket Type2     928
Name: count, dtype: int64
```

```
Outlet_Size
Outlet_Size
Medium    2793
NaN       2410
Small     2388
High       932
Name: count, dtype: int64
```

```
Outlet_Location_Type
Outlet_Location_Type
Tier 3    3350
Tier 2    2785
Tier 1    2388
Name: count, dtype: int64
```

```
Item_Type
Item_Type
Fruits and Vegetables  1232
Snack Foods            1200
Household              910
Frozen Foods           856
Dairy                  682
Canned                 649
Baking Goods           648
Health and Hygiene     520
Soft Drinks            445
Meat                   425
Breads                 251
Hard Drinks            214
Others                 169
Starchy Foods          148
Breakfast              110
Seafood                64
Name: count, dtype: int64
```

Este paso es importante porque la interpretación de diferencias entre grupos debe considerar el tamaño muestral. Un grupo muy pequeño puede mostrar estadísticas inestables o aparentes extremos debidos a pocos casos.

40.5. Heterogeneidad por tipo de outlet

Comenzamos con una variable de alto interés de negocio: `Outlet_Type`.

```
outlet_type_summary = (
    df.groupby("Outlet_Type")["Item_Outlet_Sales"]
      .agg(["count", "mean", "median", "std", "min", "max"])
      .sort_values("mean", ascending=False)
)
outlet_type_summary
```

	count	mean	median	std	min	max
Outlet_Type						
Supermarket Type3	935	3694.038558	3364.9532	2127.760054	241.6854	13086.9648
Supermarket Type1	5577	2316.181148	1990.7420	1515.965558	73.2380	10256.6490
Supermarket Type2	928	1995.498739	1655.1788	1375.932889	69.2432	6768.5228
Grocery Store	1083	339.828500	256.9988	260.851582	33.2900	1775.6886

```
fig = px.box(
    df,
    x="Outlet_Type",
    y="Item_Outlet_Sales",
    color="Outlet_Type",
    title="Distribución de ventas por tipo de outlet"
)
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

```
fig = px.bar(
    outlet_type_summary.reset_index(),
    x="Outlet_Type",
    y="mean",
    error_y="std",
    title="Media y desviación estándar de ventas por tipo de outlet",
    text_auto=True
)
fig.show()
```

Unable to display output for mime type(s): text/html

A partir de este análisis se observa una **heterogeneidad clara por tipo de outlet**, lo que indica que **Outlet_Type** sí está asociado con diferencias relevantes en el nivel de ventas.

En primer lugar, **Supermarket Type3** destaca como el segmento con **mayor nivel de ventas**, tanto en la **media (3694.039)** como en la **mediana (3364.953)**. Además, su boxplot se ubica claramente por encima del resto, lo que sugiere un desempeño consistentemente superior.

En segundo lugar, **Supermarket Type1** y **Supermarket Type2** presentan niveles de ventas **intermedios**, aunque **Supermarket Type1** muestra valores centrales algo mayores (**media = 2316.181**, **mediana = 1990.742**) que **Supermarket Type2** (**media = 1995.499**, **mediana = 1655.179**). Esto sugiere que, aunque ambos formatos son comparables, no tienen exactamente el mismo comportamiento comercial.

Por otro lado, **Grocery Store** aparece como el grupo con **menor nivel de ventas**, con una **media de 339.829** y una **mediana de 256.999**, muy por debajo de los supermercados. Esto revela una diferencia estructural importante entre este tipo de outlet y los demás.

En términos de **variabilidad**, **Supermarket Type3** también presenta la mayor dispersión (**std = 2127.760**), lo que indica que no solo vende más, sino que sus ventas son más heterogéneas. **Supermarket Type1** y **Supermarket Type2** también muestran dispersión considerable, mientras que **Grocery Store** tiene una variabilidad mucho menor en términos absolutos (**std = 260.852**), coherente con su escala de ventas más baja.

Los boxplots también muestran la presencia de **valores atípicos** en todos los grupos, especialmente en los supermercados. Esto sugiere que dentro de cada tipo de outlet existen productos o combinaciones producto-tienda con desempeños excepcionalmente altos.

En conjunto, este resultado tiene varias implicaciones para el EDA:

- **Outlet_Type** debe considerarse una **variable clave de segmentación**;
- no conviene analizar **Item_Outlet_Sales** como si proviniera de una sola población homogénea;
- es recomendable profundizar con comparaciones entre grupos, distribuciones segmentadas y medidas robustas;
- en etapas posteriores, **Outlet_Type** probablemente será una variable explicativa importante en cualquier análisis predictivo o inferencial.

En síntesis, el análisis muestra que el tipo de outlet introduce diferencias sustanciales tanto en el **nivel** como en la **dispersión** de las ventas, por lo que esta variable merece un lugar central dentro del EDA.

40.6. Heterogeneidad por tamaño del outlet

El tamaño del outlet puede influir tanto en el volumen típico de ventas como en la dispersión observada.

```
outlet_size_summary = (
    df.groupby("Outlet_Size")["Item_Outlet_Sales"]
      .agg(["count", "mean", "median", "std"])
      .sort_values("mean", ascending=False)
)
outlet_size_summary
```

	count	mean	median	std
Outlet_Size				
Medium	2793	2681.603542	2251.0698	1855.210528
High	932	2298.995256	2050.6640	1533.531664
Small	2388	1912.149161	1544.6560	1582.370364

```
fig = px.violin(
    df,
    x="Outlet_Size",
    y="Item_Outlet_Sales",
    color="Outlet_Size",
    box=True,
    points=False,
    title="Ventas por tamaño de outlet"
)
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

Los violin plots son especialmente útiles aquí porque combinan información sobre dispersión y forma dentro de cada grupo. Si dos grupos tienen medianas parecidas pero densidades internas distintas, la heterogeneidad va más allá del simple promedio.

Este análisis también muestra **heterogeneidad por tamaño del outlet**, aunque con diferencias menos extremas que las observadas por `Outlet_Type`. Aun así, `Outlet_Size` aporta información relevante para el EDA porque parece asociarse tanto con el nivel típico de ventas como con su dispersión.

En primer lugar, los outlets de tamaño `Medium` presentan el mayor nivel de ventas, con una **media de 2681.604** y una **mediana de 2251.070**. Les siguen los outlets `High`, con **media = 2298.995** y **mediana = 2050.664**, mientras que los outlets `Small` muestran los valores más bajos (media = 1912.149, mediana = 1544.656*). Esto sugiere que el tamaño del outlet sí introduce diferencias sistemáticas en el comportamiento comercial.

En términos de dispersión, `Medium` también presenta la mayor variabilidad (**std = 1855.211**), seguido por `Small` (**std = 1582.370**) y `High` (**std = 1533.532**). Los violin plots refuerzan esta idea al mostrar que no solo cambian los niveles centrales, sino también la forma interna de la distribución en cada grupo.

Para el EDA, esto tiene varias consecuencias:

- `Outlet_Size` debe considerarse una **variable relevante de segmentación**, ya que las ventas no parecen distribuirse igual entre tamaños de tienda;
- no basta con comparar solo promedios: la forma de los violines sugiere diferencias en **concentración, dispersión y densidad interna** entre grupos;
- conviene cruzar `Outlet_Size` con otras variables categóricas, especialmente `Outlet_Type`, para ver si parte de esta heterogeneidad se explica por la combinación entre ambas;
- en análisis posteriores, `Outlet_Size` puede funcionar como una **variable explicativa útil**, aunque probablemente con menor poder discriminante que `Outlet_Type`;
- dado que esta variable tenía valores faltantes en etapas previas, su papel analítico resulta todavía más importante, porque ayuda a decidir si conviene imputarla y preservarla en lugar de descartarla.

En síntesis, el tamaño del outlet no solo afecta el nivel típico de ventas, sino también la estructura de su distribución. Por ello, en el EDA conviene tratarlo como un factor de heterogeneidad relevante y no como una simple característica secundaria del establecimiento.

40.7. Heterogeneidad por ubicación del outlet

Otra dimensión natural del análisis es `Outlet_Location_Type`, que puede reflejar diferencias territoriales o de mercado.

```
location_summary = (
    df.groupby("Outlet_Location_Type")["Item_Outlet_Sales"]
      .agg(["count", "mean", "median", "std"])
      .sort_values("mean", ascending=False)
)
location_summary
```

	count	mean	median	std
Outlet_Location_Type				
Tier 2	2785	2323.990559	2004.0580	1520.543543
Tier 3	3350	2279.627651	1812.3076	1912.451333
Tier 1	2388	1876.909159	1487.3972	1561.649293

```
fig = px.box(
    df,
    x="Outlet_Location_Type",
    y="Item_Outlet_Sales",
    color="Outlet_Location_Type",
    title="Ventas por tipo de localización del outlet"
)
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

El análisis por `Outlet_Location_Type` revela también **heterogeneidad territorial en las ventas**, aunque con diferencias más moderadas que las observadas por tipo de outlet.

En términos de nivel central, **Tier 2** presenta el mayor promedio (**mean = 2323.991**, **median = 2004.058**), seguido muy de cerca por **Tier 3** (**mean = 2279.628**, **median = 1812.308**). En contraste, **Tier 1** muestra los valores más bajos (**mean = 1876.909**, **median = 1487.397**), lo que sugiere que las ventas tienden a ser menores en este tipo de localización.

En cuanto a la dispersión, **Tier 3** presenta la mayor variabilidad (**std = 1912.451**), lo que indica una mayor heterogeneidad en el desempeño de ventas dentro de este grupo. Los boxplots también muestran la presencia de **valores extremos** en todas las ubicaciones, especialmente en **Tier 3**.

Para el EDA, esto sugiere que:

- Outlet_Location_Type introduce **diferencias territoriales en el nivel de ventas**;
- Tier 2 y Tier 3 tienen comportamientos relativamente similares en el centro de la distribución, aunque Tier 3 muestra mayor dispersión;
- Tier 1 parece corresponder a ubicaciones con **menor desempeño promedio**;
- conviene explorar **interacciones entre ubicación, tipo y tamaño del outlet**, ya que parte de la heterogeneidad observada podría explicarse por la combinación de estas variables.

En síntesis, la localización del outlet añade otra dimensión de segmentación en el EDA y refuerza la idea de que las ventas no provienen de una población homogénea, sino de **subgrupos con comportamientos distintos**.

40.8. Heterogeneidad por tipo de producto

La variable Item_Type suele contener más niveles que las anteriores, por lo que conviene resumir y ordenar cuidadosamente la información.

```
item_type_summary = (
    df.groupby("Item_Type")["Item_Outlet_Sales"]
      .agg(["count", "mean", "median", "std"])
      .sort_values("mean", ascending=False)
)
item_type_summary.head(10)
```

	count	mean	median	std
Item_Type				
Starchy Foods	148	2374.332773	1968.1048	1773.945328
Seafood	64	2326.065928	2055.3246	1842.988719
Fruits and Vegetables	1232	2289.009592	1830.9500	1799.503459
Snack Foods	1200	2277.321739	1944.1360	1705.121755
Household	910	2258.784300	1981.4208	1692.245757
Dairy	682	2232.542597	1650.8511	1884.404698
Canned	649	2225.194904	1860.2452	1645.235638
Breads	251	2204.132226	1860.2452	1644.235914
Meat	425	2158.977911	1829.6184	1695.231081
Hard Drinks	214	2139.221622	1816.6353	1606.191587

```
fig = px.bar(
    item_type_summary.reset_index(),
    x="mean",
    y=item_type_summary.reset_index()["Item_Type"],
    orientation="h",
    title="Media de ventas por tipo de producto",
    labels={"mean": "Media de ventas", "y": "Tipo de producto"}
)
fig.show()
```

Unable to display output for mime type(s): text/html

```

top_item_types = item_type_summary.head(8).index.tolist()
fig = px.box(
    df[df["Item_Type"].isin(top_item_types)],
    x="Item_Type",
    y="Item_Outlet_Sales",
    color="Item_Type",
    title="Distribución de ventas en los tipos de producto con mayor media"
)
fig.update_layout(showlegend=False)
fig.show()

```

Unable to display output for mime type(s): text/html

El análisis por `Item_Type` muestra que también existe **heterogeneidad por categoría de producto**, aunque menos marcada que la observada por `Outlet_Type`. Aun así, las diferencias son suficientes para justificar una exploración segmentada.

En términos de promedio, las categorías con mayores ventas son `Starchy Foods` (**mean = 2374.333**), `Seafood` (**mean = 2326.066**) y `Fruits and Vegetables` (**mean = 2289.010**). Sin embargo, estas dos primeras tienen tamaños muestrales mucho menores (**148** y **64** observaciones, respectivamente), por lo que sus medias deben interpretarse con más cautela.

Entre las categorías con mayor presencia en el dataset, `Fruits and Vegetables` (**1232** casos), `Snack Foods` (**1200**) y `Household` (**910**) combinan **niveles altos de ventas** con una base de observaciones más sólida, lo que las vuelve especialmente relevantes para el EDA.

Los boxplots sugieren además que estas categorías no solo difieren en el nivel central, sino también en su **dispersión** y en la presencia de **valores extremos**. Por ejemplo, `Household`, `Fruits and Vegetables` y `Snack Foods` muestran colas superiores amplias y varios valores altos, lo que sugiere comportamientos heterogéneos dentro de cada tipo de producto.

Para el EDA, esto tiene varias implicaciones:

- `Item_Type` debe considerarse una **variable de segmentación importante**;
- conviene distinguir entre categorías con **muchas observaciones** y categorías con **poca representación**, para no sobredimensionar diferencias basadas en muestras pequeñas;
- resulta útil cruzar `Item_Type` con variables como `Outlet_Type` u `Outlet_Size`, ya que parte de la heterogeneidad podría depender de la interacción entre producto y tienda;
- en etapas posteriores, `Item_Type` puede aportar valor explicativo, pero probablemente con efecto desigual según la categoría.

En síntesis, el tipo de producto introduce diferencias reales en las ventas, pero su interpretación debe combinar **nivel de ventas**, **dispersión** y **tamaño muestral** para evitar conclusiones apresuradas.

40.9. Tablas comparativas con múltiples indicadores

Una forma útil de sistematizar la heterogeneidad es construir tablas resumen que integren varias medidas por grupo.


```
def group_profile(df, group_col, target="Item_Outlet_Sales"):
    out = (
        df.groupby(group_col)[target]
        .agg(
            n="count",
            media="mean",
            mediana="median",
            desviacion="std",
            q1=lambda x: x.quantile(0.25),
            q3=lambda x: x.quantile(0.75)
        )
        .reset_index()
    )
    out["iqr"] = out["q3"] - out["q1"]
    out["cv"] = out["desviacion"] / out["media"]
    return out.sort_values("media", ascending=False)

profile_outlet_type = group_profile(df, "Outlet_Type")
profile_outlet_type
```

	Outlet_Type	n	media	mediana	desviacion	q1	q3	iqr	cv
3	Supermarket Type3	935	3694.038558	3364.9532	2127.760054	2044.33890	4975.52340	2931.1845	0.5
1	Supermarket Type1	5577	2316.181148	1990.7420	1515.965558	1151.16820	3135.91800	1984.7498	0.6
2	Supermarket Type2	928	1995.498739	1655.1788	1375.932889	981.55565	2702.64865	1721.0930	0.6
0	Grocery Store	1083	339.828500	256.9988	260.851582	153.79980	458.73620	304.9364	0.7

```
profile_location = group_profile(df, "Outlet_Location_Type")
profile_location
```

	Outlet_Location_Type	n	media	mediana	desviacion	q1	q3	iqr
1	Tier 2	2785	2323.990559	2004.0580	1520.543543	1171.80800	3110.6176	1938.80960
2	Tier 3	3350	2279.627651	1812.3076	1912.451333	731.38130	3307.6944	2576.31310
0	Tier 1	2388	1876.909159	1487.3972	1561.649293	593.72715	2803.0180	2209.29085

Estas tablas permiten pasar de una observación visual a una descripción más precisa: no solo se identifican grupos “más altos” o “más bajos”, sino también grupos más o menos variables y con distinta estabilidad relativa.

40.10. Heterogeneidad cruzada

En muchos problemas reales, la segmentación relevante no depende de una sola variable categórica. Una comparación cruzada, por ejemplo entre tipo de outlet y localización, ayuda a identificar patrones más finos.

```

pivot_mean = pd.pivot_table(
    df,
    values="Item_Outlet_Sales",
    index="Outlet_Type",
    columns="Outlet_Location_Type",
    aggfunc="mean"
)
pivot_mean

```

Outlet_Location_Type	Tier 1	Tier 2	Tier 3
Outlet_Type			
Grocery Store	340.329723	NaN	339.351662
Supermarket Type1	2313.099451	2323.990559	2298.995256
Supermarket Type2	NaN	NaN	1995.498739
Supermarket Type3	NaN	NaN	3694.038558

```

fig = px.imshow(
    pivot_mean,
    text_auto=True,
    aspect="auto",
    title="Media de ventas: tipo de outlet vs localización"
)
fig.show()

```

Unable to display output for mime type(s): text/html

La heterogeneidad cruzada muestra que una sola variable categórica no siempre basta para describir el comportamiento de las ventas. En este caso, la combinación entre `Outlet_Type` y `Outlet_Location_Type` permite detectar patrones más específicos que no se observan con la misma claridad al analizar cada variable por separado.

El resultado más visible es que `Supermarket Type3` en `Tier 3` presenta la media de ventas más alta (**3694.039**), mientras que `Grocery Store` mantiene niveles muy bajos tanto en `Tier 1` (**340.330**) como en `Tier 3` (**339.352**). Por su parte, `Supermarket Type1` muestra un comportamiento bastante estable entre ubicaciones, con medias de **2313.099** en `Tier 1`, **2323.991** en `Tier 2` y **2298.995** en `Tier 3`.

Para el EDA, esto tiene varias implicaciones:

- la variación en ventas no depende solo del tipo de outlet ni solo de la ubicación, sino de su **combinación**;
- `Supermarket Type1` parece ser un formato relativamente estable entre territorios, mientras que otros tipos están mucho más concentrados en ciertas ubicaciones;
- existen combinaciones ausentes (NaN), lo que indica que no todos los formatos de outlet están presentes en todas las localizaciones, y eso también forma parte de la estructura real del negocio;
- conviene seguir explorando **interacciones** entre variables categóricas, porque parte de la heterogeneidad puede estar oculta cuando se estudian los factores por separado;
- en etapas posteriores, estas combinaciones pueden ser útiles para construir variables explicativas más informativas que cada factor individual por sí solo.

En síntesis, este análisis refuerza una idea central del EDA: las ventas no provienen de grupos simples, sino de **segmentos combinados** donde el contexto del outlet importa tanto como su categoría individual.

40.11. Entropía de Shannon como medida de heterogeneidad

Hasta ahora, la heterogeneidad se ha estudiado comparando promedios, medianas, dispersiones y distribuciones entre grupos. Sin embargo, también es útil contar con una medida **global** que resuma qué tan **diversificada** o **concentrada** está una variable categórica. Para ello puede utilizarse la **entropía de Shannon**.

Si una categoría concentra casi toda la masa de observaciones o de ventas, la entropía será baja. En cambio, si la distribución está más repartida entre varios grupos, la entropía será mayor. En este sentido, la entropía ofrece una medida sintética de diversidad.

En términos generales, para una variable categórica con proporciones $p_1, p_2, \dots, p_k$, la entropía se define como

$$H = - \sum_{i=1}^k p_i \log(p_i)$$

donde valores más altos indican mayor heterogeneidad. Como el valor máximo depende del número de categorías, también conviene trabajar con una **entropía normalizada**:

$$H_{\text{norm}} = \frac{H}{\log(k)}$$

De esta forma, el resultado queda entre 0 y 1:

- valores cercanos a **0** indican **alta concentración**;
- valores cercanos a **1** indican **alta diversidad**.

40.11.1. Entropía de ventas por variable categórica

En esta sección calcularemos la entropía sobre la **distribución de ventas** agregadas por categoría, para responder preguntas como: ¿las ventas están repartidas entre muchos grupos o dependen de unos pocos?

```
import numpy as np
import pandas as pd

vars_cat_heterogeneidad = [
    "Outlet_Type",
    "Outlet_Size",
    "Outlet_Location_Type",
    "Item_Type"
]

def shannon_entropy_from_sales(df, cat_var, sales_var="Item_Outlet_Sales"):
    sales_by_group = df.groupby(cat_var)[sales_var].sum().dropna()
    p = sales_by_group / sales_by_group.sum()
```

```

h = -(p * np.log(p)).sum()
h_norm = h / np.log(len(p)) if len(p) > 1 else np.nan

return pd.Series({
    "n_categorias": len(p),
    "entropia": h,
    "entropia_normalizada": h_norm
})

entropy_table = pd.DataFrame({
    var: shannon_entropy_from_sales(df, var)
    for var in vars_cat_heterogeneidad
}).T.reset_index(names="variable")

entropy_table.sort_values("entropia_normalizada", ascending=False).round(4)

```

	variable	n_categorias	entropia	entropia_normalizada
2	Outlet_Location_Type	3.0	1.0758	0.9792
3	Item_Type	16.0	2.5168	0.9078
1	Outlet_Size	3.0	0.9876	0.8990
0	Outlet_Type	4.0	0.8731	0.6298

La interpretación debe hacerse con cuidado. Una entropía alta no significa necesariamente “mejor”, sino simplemente que las ventas están más repartidas entre las categorías de esa variable. Una entropía baja, en cambio, sugiere que el negocio está más concentrado en unos pocos segmentos.

```

import plotly.express as px

entropy_plot = entropy_table.sort_values("entropia_normalizada", ascending=True)

fig = px.bar(
    entropy_plot,
    x="entropia_normalizada",
    y="variable",
    orientation="h",
    text="entropia_normalizada",
    title="Entropía normalizada de Shannon por variable categórica",
    labels={
        "entropia_normalizada": "Entropía normalizada",
        "variable": "Variable categórica"
    },
    range_x=[0, 1]
)

fig.update_traces(texttemplate="%{text:.3f}", textposition="outside")
fig.update_layout(height=450, width=900)

fig.show()

```

Unable to display output for mime type(s): text/html

Si se desea profundizar en una variable concreta, también puede visualizarse directamente la distribución de las ventas agregadas por categoría. Por ejemplo, para `Item_Type`:

```
sales_item_type = (
    df.groupby("Item_Type")["Item_Outlet_Sales"]
        .sum()
        .sort_values(ascending=False)
        .reset_index()
)

sales_item_type["proporcion"] = (
    sales_item_type["Item_Outlet_Sales"] /
    sales_item_type["Item_Outlet_Sales"].sum()
)

fig = px.bar(
    sales_item_type,
    x="Item_Type",
    y="proporcion",
    text="proporcion",
    title="Proporción de ventas por tipo de producto",
    labels={
        "Item_Type": "Tipo de producto",
        "proporcion": "Proporción de ventas"
    }
)

fig.update_traces(texttemplate="%{text:.3f}", textposition="outside")
fig.update_layout(xaxis_tickangle=45, height=500, width=950)

fig.show()
```

Unable to display output for mime type(s): text/html

Los resultados de entropía sugieren que no todas las variables categóricas aportan el mismo tipo de segmentación para el EDA. En particular, `Outlet_Location_Type` presenta la **mayor entropía normalizada (0.9792)**, seguido por `Item_Type` (**0.9078**) y `Outlet_Size` (**0.8990**). Esto indica que, en estas variables, las ventas están **bastante repartidas** entre sus categorías y no dependen fuertemente de un solo grupo.

En cambio, `Outlet_Type` muestra una entropía normalizada claramente menor (**0.6298**), lo que sugiere una **mayor concentración** de las ventas en unos pocos tipos de outlet. Esto es coherente con los análisis previos, donde se observó que ciertos formatos, especialmente `Supermarket Type3` y `Supermarket Type1`, concentran gran parte del volumen de ventas.

Para el EDA, esto tiene varias implicaciones:

- `Outlet_Type` aparece como una variable especialmente útil para detectar **concentración comercial**, ya que distingue grupos con pesos muy desiguales en el negocio;

- `Outlet_Location_Type`, `Outlet_Size` e `Item_Type` reflejan una estructura más **diversificada**, por lo que su utilidad analítica está menos en la concentración y más en comparar patrones internos entre categorías;
- una entropía alta no elimina la heterogeneidad, sino que indica que el negocio está más repartido entre grupos, por lo que conviene profundizar con medidas de nivel, dispersión y forma dentro de cada categoría;
- el gráfico de proporciones por `Item_Type` muestra que, aunque la distribución es bastante diversa, algunas categorías como **Fruits and Vegetables (0.152)** y **Snack Foods (0.147)** tienen un peso algo mayor que otras, por lo que pueden merecer atención especial en análisis posteriores.

En síntesis, la entropía complementa el EDA porque ayuda a distinguir entre variables que describen un negocio **más concentrado** y variables que describen un negocio **más distribuido**. En este caso, `Outlet_Type` parece ser la variable más útil para estudiar concentración, mientras que `Item_Type`, `Outlet_Size` y `Outlet_Location_Type` son más útiles para explorar diversidad y segmentación interna.

La entropía anterior describe diversidad de participaciones. El coeficiente de Gini basado en la curva de Lorenz responde específicamente a desigualdad y se desarrolla, con su fórmula corregida, en Medidas de concentración en Big Mart Sales.

40.12. Mini-dashboard de heterogeneidad

A continuación se construye un dashboard compacto que integra varias vistas segmentadas.

```
from plotly.subplots import make_subplots

fig = make_subplots(
    rows=2,
    cols=2,
    subplot_titles=(
        "Media por tipo de outlet",
        "Media por tamaño de outlet",
        "Media por localización",
        "Top 8 tipos de producto"
    )
)

summary1 = group_profile(df, "Outlet_Type")
summary2 = group_profile(df.dropna(subset=["Outlet_Size"]), "Outlet_Size")
summary3 = group_profile(df, "Outlet_Location_Type")
summary4 = group_profile(df, "Item_Type").head(8)

fig.add_trace(
    go.Bar(x=summary1["Outlet_Type"], y=summary1["media"], name="Outlet Type"),
    row=1, col=1
)

fig.add_trace(
    go.Bar(x=summary2["Outlet_Size"], y=summary2["media"], name="Outlet Size"),
    row=1, col=2
```

```

)
fig.add_trace(
    go.Bar(x=summary3["Outlet_Location_Type"], y=summary3["media"], name="Location"),
    row=2, col=1
)
fig.add_trace(
    go.Bar(x=summary4["Item_Type"], y=summary4["media"], name="Item Type"),
    row=2, col=2
)

fig.update_layout(
    title="Dashboard básico de heterogeneidad en BigMart",
    height=750,
    showlegend=False
)
fig.show()

```

Unable to display output for mime type(s): text/html

40.13. Conclusiones del capítulo

En este capítulo se mostró que la heterogeneidad constituye una dimensión esencial del análisis exploratorio de datos. Comparar grupos permite ampliar la descripción estadística de una variable y pasar de una lectura global a una interpretación segmentada y contextual.

En el dataset de BigMart, la heterogeneidad se manifiesta en diferencias entre tipos de outlet, tamaños, localizaciones y categorías de producto. Estas comparaciones enriquecen el EDA porque permiten distinguir subestructuras internas del fenómeno, identificar patrones consistentes y detectar segmentos con comportamientos contrastantes.

Este capítulo prepara de manera natural el siguiente paso del proyecto: el estudio de la **concentración**, donde se analizará si una fracción reducida de grupos, productos o outlets explica una proporción sustancial del total de ventas.

Parte XVI.

Medidas de concentración

41. Medidas de concentración

41.1. Introducción

La concentración estudia cómo se reparte una masa no negativa —ventas, ingreso, consumo o participación— entre unidades. Puede existir una media estable y, aun así, depender el total de unas pocas categorías o establecimientos.

Definición. Si $s_i = x_i / \sum_j x_j$ es la participación de la unidad i , una distribución está más concentrada cuando pocas participaciones reúnen gran parte del total.

41.2. Razones de concentración

Ordenando $s_{(1)} \geq \dots \geq s_{(K)}$, la razón de concentración de las k unidades principales es

$$CR_k = \sum_{i=1}^k s_{(i)}. \quad (1)$$

CR_k comunica con claridad cuánto aporta el *top k*, aunque ignora cómo se distribuye la masa dentro y fuera de ese grupo.

41.3. Índice Herfindahl–Hirschman

$$HHI = \sum_{i=1}^K s_i^2. \quad (2)$$

El índice vale $1/K$ bajo reparto uniforme y 1 bajo concentración total. A veces se reporta en escala 0–10,000 multiplicando por 10,000 (Hirschman 1945). Para comparar distintos K puede normalizarse:

$$HHI^* = \frac{HHI - 1/K}{1 - 1/K}. \quad (3)$$

41.4. Curva de Lorenz

Ordenando $x_{(1)} \leq \dots \leq x_{(n)}$, la curva de Lorenz relaciona la fracción acumulada de unidades con la fracción acumulada de masa (Lorenz 1905):

$$L_i = \frac{\sum_{j=1}^i x_{(j)}}{\sum_{j=1}^n x_{(j)}}. \quad (4)$$

41.5. Coeficiente de Gini

El coeficiente de Gini es dos veces el área entre la diagonal de igualdad y la curva de Lorenz (Gini 1912). Para observaciones no negativas ordenadas:

$$G = \frac{2 \sum_{i=1}^n i x_{(i)}}{n \sum_{i=1}^n x_{(i)}} - \frac{n+1}{n}. \quad (5)$$

Esta fórmula requiere masa no negativa y total positivo. El coeficiente de Gini de desigualdad no es la impureza de Gini usada para diversidad o árboles de decisión.

41.6. Cálculo manual paso a paso

Supongamos cuatro outlets con ventas

$$x = (10, 20, 30, 40).$$

41.6.1. Participaciones y razón de concentración

Paso 1. Calcular el total:

$$T = 10 + 20 + 30 + 40 = 100.$$

Paso 2. Dividir cada venta entre el total:

$$s_1 = \frac{10}{100} = 0.10, \quad s_2 = \frac{20}{100} = 0.20, \quad s_3 = \frac{30}{100} = 0.30, \quad s_4 = \frac{40}{100} = 0.40.$$

Paso 3. Ordenar las participaciones de mayor a menor:

$$(0.40, 0.30, 0.20, 0.10).$$

Paso 4. Sumar las dos mayores:

$$CR_2 = 0.40 + 0.30 = 0.70 = 70\%.$$

Los dos outlets principales aportan 70 % de las ventas.

41.6.2. HHI y HHI normalizado

Paso 1. Elevar cada participación al cuadrado:

$$0.10^2 = 0.01, \quad 0.20^2 = 0.04, \quad 0.30^2 = 0.09, \quad 0.40^2 = 0.16.$$

Paso 2. Sumar:

$$HHI = 0.01 + 0.04 + 0.09 + 0.16 = 0.30.$$

En escala 0–10,000:

$$HHI_{10000} = 0.30(10000) = 3000.$$

Paso 3. Normalizar considerando $K = 4$:

$$HHI^* = \frac{HHI - 1/K}{1 - 1/K} = \frac{0.30 - 1/4}{1 - 1/4} = \frac{0.30 - 0.25}{0.75} = \frac{0.05}{0.75} = 0.0667.$$

41.6.3. Puntos de la curva de Lorenz

Los valores ya están ordenados de menor a mayor.

La fracción acumulada de unidades avanza en cuartos:

$$u = (0, 0.25, 0.50, 0.75, 1.00).$$

Acumulamos las ventas y dividimos entre 100:

$$L_0 = 0,$$

$$L_1 = \frac{10}{100} = 0.10,$$

$$L_2 = \frac{10 + 20}{100} = \frac{30}{100} = 0.30,$$

$$L_3 = \frac{10 + 20 + 30}{100} = \frac{60}{100} = 0.60,$$

$$L_4 = \frac{10 + 20 + 30 + 40}{100} = 1.00.$$

Los puntos de Lorenz son

$$(0, 0), (0.25, 0.10), (0.50, 0.30), (0.75, 0.60), (1, 1).$$

41.6.4. Coeficiente de Gini

Paso 1. Multiplicar cada valor ordenado por su posición:

$$1(10) = 10, \quad 2(20) = 40, \quad 3(30) = 90, \quad 4(40) = 160.$$

Paso 2. Sumar los productos:

$$\sum_{i=1}^4 ix_{(i)} = 10 + 40 + 90 + 160 = 300.$$

Paso 3. Sustituir $n = 4$ y $\sum x_i = 100$:

$$G = \frac{2(300)}{4(100)} - \frac{4+1}{4}.$$

Paso 4. Resolver ambas fracciones:

$$G = \frac{600}{400} - \frac{5}{4} = 1.50 - 1.25 = 0.25.$$

El valor 0.25 expresa desigualdad moderada en este ejemplo; no significa que “25 % de las unidades concentre las ventas”.

Del cálculo al programa. Validar primero que $x_i \geq 0$ y que la suma sea positiva. Después: normalizar para obtener participaciones, ordenar descendientemente para CR_k , elevar y sumar para HHI, ordenar ascendentemente y acumular para Lorenz, y reutilizar ese orden para Gini.

41.7. Pareto y participación acumulada

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

rng = np.random.default_rng(77)
ventas = rng.lognormal(mean=3.5, sigma=1.1, size=35)
tabla = pd.DataFrame({"Unidad": [f"U{i:02d}" for i in range(1, 36)], "Ventas": ventas})
tabla = tabla.sort_values("Ventas", ascending=False)
tabla["Participación acumulada"] = tabla["Ventas"].cumsum()/tabla["Ventas"].sum()

fig = make_subplots(specs=[[{"secondary_y": True}]]
fig.add_bar(x=tabla["Unidad"], y=tabla["Ventas"], name="Ventas", secondary_y=False)
fig.add_scatter(x=tabla["Unidad"], y=tabla["Participación acumulada"],
                mode="lines+markers", name="Participación acumulada", secondary_y=True)
fig.add_hline(y=.80, line_dash="dash", line_color="#D62728", secondary_y=True,
              annotation_text="80%")
fig.update_yaxes(title_text="Ventas", secondary_y=False)
```

```
fig.update_yaxes(title_text="Participación acumulada", tickformat=".0%", secondary_y=True)
fig.update_layout(title="Diagrama de Pareto interactivo")
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

41.8. Lorenz y Gini

```
def lorenz_gini(x):
    x = np.asarray(x, dtype=float)
    if np.any(x < 0) or x.sum() <= 0:
        raise ValueError("Se requieren valores no negativos y suma positiva")
    x = np.sort(x)
    n = len(x)
    poblacion = np.arange(n + 1)/n
    masa = np.r_[0, np.cumsum(x)/x.sum()]
    gini = 2*np.sum(np.arange(1, n+1)*x)/(n*x.sum())-(n+1)/n
    return poblacion, masa, gini

escenarios = {
    "Reparto cercano": rng.uniform(.8, 1.2, 600),
    "Concentración media": rng.exponential(1, 600),
    "Concentración alta": rng.lognormal(0, 1.4, 600)
}

fig = go.Figure()
fig.add_trace(go.Scatter(x=[0,1], y=[0,1], mode="lines", name="Igualdad",
                        line=dict(dash="dash", color="gray"))))
for nombre, x in escenarios.items():
    u, L, g = lorenz_gini(x)
    fig.add_trace(go.Scatter(x=u, y=L, mode="lines", name=f"{nombre} · G={g:.2f}"))
fig.update_layout(title="Curvas de Lorenz para tres niveles de concentración",
                  xaxis_title="Fracción acumulada de unidades",
                  yaxis_title="Fracción acumulada de masa")
fig.show()
```

Unable to display output for mime type(s): text/html

Interpretación. Cuanto más se aleja la curva de la diagonal, mayor es la desigualdad. El hover permite responder preguntas operativas como “¿qué proporción de unidades produce 50 % del total?”.

41.9. Concentración al crecer una unidad dominante

```

filas = []
for dominante in np.linspace(.20, .92, 13):
    resto = (1-dominante)/4
    shares = np.array([dominante] + [resto]*4)
    for i, s in enumerate(shares, 1):
        filas.append({"Dominancia": f"{dominante:.2f}", "Unidad": f"U{i}", "Participación": s})
anim = pd.DataFrame(filas)
fig = px.bar(anim, x="Unidad", y="Participación", color="Unidad",
             animation_frame="Dominancia", range_y=[0,1],
             title="Animación: formación de una estructura concentrada")
fig.update_layout(showlegend=False)
fig.show()

```

Unable to display output for mime type(s): text/html

41.10. Guía de decisión

Pregunta	Indicador
¿Cuánto explica el top 3 o top 5?	CR_k
¿Qué tan concentradas están todas las participaciones?	HHI / HHI normalizado
¿Cómo se acumula el total?	Pareto o Lorenz
¿Cuál es la desigualdad global?	Gini

41.11. Puente al caso BigMart

El capítulo Medidas de concentración en Big Mart Sales aplica estas herramientas a tipos de producto y outlets.

41.12. Ejercicios

1. Compare dos distribuciones con el mismo CR_1 y diferente HHI.
2. Verifique los límites del Gini con una distribución uniforme y otra casi concentrada.
3. Calcule cuántas categorías producen 80 % de las ventas.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.

- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Thesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. [martinfowler.com](http://martinfowler.com/articles/richardsonMaturityModel.html). <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best

- Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data*

- Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.

Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.

Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.

Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.

Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.

Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.

Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.

Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.

Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.

Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.

Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.

Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.

Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbj1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine

- Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

42. Medidas de concentración en Big Mart Sales

Este caso aplica las razones de concentración, HHI, Pareto, Lorenz y Gini desarrollados en Medidas de concentración.

Hasta este punto del análisis exploratorio ya hemos estudiado la localización, la variabilidad, la forma y la heterogeneidad de las variables principales del conjunto de datos **Big Mart Sales**. Falta ahora una dimensión especialmente útil cuando se trabaja con datos económicos, comerciales y administrativos: la **concentración**.

La idea central es simple. No basta con saber cuánto venden, en promedio, los productos o los outlets; también interesa conocer **cómo se reparte** el total de ventas entre categorías, tipos de tienda o establecimientos concretos. Dos escenarios pueden tener la misma venta total y la misma media, pero diferir radicalmente en su estructura interna: en uno las ventas pueden estar distribuidas de manera relativamente equilibrada, mientras que en otro una pequeña fracción de categorías o sucursales puede concentrar una porción muy grande del ingreso total.

Desde una perspectiva de EDA, las medidas y visualizaciones de concentración ayudan a responder preguntas como las siguientes:

- ¿Qué proporción del ingreso total proviene de unas cuantas categorías?
- ¿Los outlets contribuyen de forma equilibrada al volumen de ventas o existe dominancia de unos pocos?
- ¿Se observa una estructura tipo **Pareto**, donde una fracción pequeña de elementos explica gran parte del fenómeno?

En este capítulo se desarrollan medidas descriptivas y representaciones gráficas para estudiar la concentración de las ventas en **Big Mart Sales**, concluyendo con un mini-dashboard de concentración que después se integrará al dashboard global del libro.

Definición operativa. La concentración cuantifica si pocas categorías u outlets reúnen una fracción grande de las ventas totales (Lorenz 1905; Gini 1912).

42.1. Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

DATA_PATH = "../data/bigmart_sales.csv"
```

```
df = pd.read_csv(DATA_PATH)

df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	O
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	O
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	O
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	O
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	O

Como en capítulos anteriores, conviene revisar que las variables categóricas relevantes estén limpias y listas para agruparse.

```
# Estandarización mínima de una variable categórica clásica en Big Mart
if "Item_Fat_Content" in df.columns:
    df["Item_Fat_Content"] = (
        df["Item_Fat_Content"]
        .replace({
            "LF": "Low Fat",
            "low fat": "Low Fat",
            "reg": "Regular"
        })
    )

# Vista rápida de columnas útiles para concentración
candidate_cols = [
    "Item_Identifier",
    "Item_Type",
    "Outlet_Identifier",
    "Outlet_Type",
    "Outlet_Location_Type",
    "Outlet_Size",
    "Item_Outlet_Sales",
]

existing = [c for c in candidate_cols if c in df.columns]
df[existing].head()
```

	Item_Identifier	Item_Type	Outlet_Identifier	Outlet_Type	Outlet_Location_Type	Outlet_Size
0	FDA15	Dairy	OUT049	Supermarket Type1	Tier 1	Medium
1	DRC01	Soft Drinks	OUT018	Supermarket Type2	Tier 3	Medium
2	FDN15	Meat	OUT049	Supermarket Type1	Tier 1	Medium
3	FDX07	Fruits and Vegetables	OUT010	Grocery Store	Tier 3	Natural
4	NCD19	Household	OUT013	Supermarket Type1	Tier 3	High

43. Fundamento conceptual

43.1. Participación relativa

Sea x_i el total de ventas asociado a la categoría o unidad i . La participación relativa de esa unidad respecto del total se define por

$$p_i = \frac{x_i}{\sum_{j=1}^n x_j}.$$

Esta cantidad permite medir el peso de cada categoría en el total agregado. En términos porcentuales:

$$p_i^{(\%) } = 100 \cdot p_i.$$

43.2. Participación acumulada

Si ordenamos las unidades de mayor a menor según su contribución, la participación acumulada hasta la posición k está dada por

$$P_k = \sum_{i=1}^k p_{(i)},$$

donde $p_{(i)}$ representa la participación de la i -ésima unidad en el orden descendente.

Esta medida es la base del análisis de **Pareto**, que permite estudiar cuántas categorías explican cierto porcentaje del total.

43.3. Curva de concentración

Otra idea útil consiste en comparar la acumulación observada con una situación ideal de reparto uniforme. Si unas pocas unidades concentran gran parte del total, la curva acumulada crecerá rápidamente al inicio y se separará de la diagonal de reparto uniforme.

Aunque en cursos introductorios puede evitarse una formalización excesiva, esta representación es la antesala conceptual de herramientas más avanzadas como la **curva de Lorenz** y el **índice de Gini**.

44. Concentración de ventas por categoría de producto

Comenzamos agregando la venta total por tipo de producto.

```
sales_by_item_type = (  
    df.groupby("Item_Type", dropna=False)["Item_Outlet_Sales"]  
        .sum()  
        .sort_values(ascending=False)  
        .reset_index()  
)  
  
sales_by_item_type["share"] = (  
    sales_by_item_type["Item_Outlet_Sales"] /  
    sales_by_item_type["Item_Outlet_Sales"].sum()  
)  
  
sales_by_item_type["cum_share"] = sales_by_item_type["share"].cumsum()  
sales_by_item_type["share_pct"] = 100 * sales_by_item_type["share"]  
sales_by_item_type["cum_share_pct"] = 100 * sales_by_item_type["cum_share"]  
  
sales_by_item_type.head(10)
```

	Item_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Fruits and Vegetables	2.820060e+06	0.151688	0.151688	15.168849	15.168849
1	Snack Foods	2.732786e+06	0.146994	0.298683	14.699412	29.868261
2	Household	2.055494e+06	0.110563	0.409246	11.056317	40.924578
3	Frozen Foods	1.825735e+06	0.098205	0.507450	9.820464	50.745042
4	Dairy	1.522594e+06	0.081899	0.589349	8.189897	58.934939
5	Canned	1.444151e+06	0.077680	0.667029	7.767962	66.702901
6	Baking Goods	1.265525e+06	0.068071	0.735100	6.807148	73.510048
7	Health and Hygiene	1.045200e+06	0.056220	0.791321	5.622038	79.132086
8	Meat	9.175656e+05	0.049355	0.840676	4.935503	84.067590
9	Soft Drinks	8.928977e+05	0.048028	0.888704	4.802817	88.870407

44.1. Ranking de contribución

Una primera lectura descriptiva consiste en ordenar las categorías por ventas y observar su peso relativo.

```
fig = px.bar(
    sales_by_item_type,
    x="Item_Type",
    y="share_pct",
    title="Participación porcentual de ventas por tipo de producto",
    labels={"share_pct": "Participación (%)", "Item_Type": "Tipo de producto"}
)
fig.update_layout(xaxis_tickangle=-45)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Este gráfico responde a una pregunta básica: **qué categorías pesan más en la estructura total de ventas**. En un contexto de negocio, esto ayuda a distinguir líneas estratégicas, categorías troncales y segmentos con menor incidencia.

44.2. Gráfico de Pareto

El gráfico de Pareto combina barras de contribución individual con una curva acumulada. Es una de las formas más didácticas de introducir la idea de concentración.

```
pareto_df = sales_by_item_type.copy()

fig = go.Figure()
fig.add_bar(
    x=pareto_df["Item_Type"],
    y=pareto_df["share_pct"],
    name="Participación individual (%)"
)
fig.add_scatter(
    x=pareto_df["Item_Type"],
    y=pareto_df["cum_share_pct"],
    mode="lines+markers",
    name="Participación acumulada (%)",
    yaxis="y2"
)

fig.update_layout(
    title="Gráfico de Pareto de ventas por tipo de producto",
    xaxis=dict(title="Tipo de producto"),
    yaxis=dict(title="Participación individual (%)",
    yaxis2=dict(
        title="Participación acumulada (%)",
        overlaying="y",
        side="right",
```



```

        range=[0, 105]
    ),
    legend=dict(orientation="h"),
)
fig.update_xaxes(tickangle=-45)
fig.show()

```

Unable to display output for mime type(s): text/html

44.3. Interpretación

Cuando la curva acumulada crece muy rápido al inicio, ello indica que unas pocas categorías concentran una gran parte de las ventas. Si, en cambio, la acumulación es más gradual, la estructura es relativamente más equilibrada.

Una buena práctica pedagógica consiste en cuantificar preguntas como:

- ¿Cuántas categorías explican el 50 % de las ventas?
- ¿Cuántas categorías explican el 80 %?
- ¿Qué fracción del total se concentra en las tres categorías principales?

```

def categories_to_reach_threshold(table, threshold=0.8):
    idx = np.argmax(table["cum_share"].to_numpy() >= threshold)
    return idx + 1

n50 = categories_to_reach_threshold(sales_by_item_type, 0.50)
n80 = categories_to_reach_threshold(sales_by_item_type, 0.80)

top3_share = sales_by_item_type["share"].head(3).sum() * 100

summary_concentration_item = pd.DataFrame({
    "Indicador": [
        "Categorías para alcanzar 50% de ventas",
        "Categorías para alcanzar 80% de ventas",
        "Participación acumulada de las 3 principales (%)"
    ],
    "Valor": [n50, n80, round(top3_share, 2)]
})

summary_concentration_item

```

	Indicador	Valor
0	Categorías para alcanzar 50 % de ventas	4.00
1	Categorías para alcanzar 80 % de ventas	9.00
2	Participación acumulada de las 3 principales (%)	40.92

45. Concentración de ventas por outlet

La concentración también puede analizarse desde el punto de vista de las tiendas o outlets. Esta mirada es importante porque distingue entre un portafolio equilibrado de establecimientos y una estructura dominada por unas pocas unidades.

```
sales_by_outlet = (
    df.groupby("Outlet_Identifier", dropna=False)["Item_Outlet_Sales"]
        .sum()
        .sort_values(ascending=False)
        .reset_index()
)

sales_by_outlet["share"] = (
    sales_by_outlet["Item_Outlet_Sales"] /
    sales_by_outlet["Item_Outlet_Sales"].sum()
)

sales_by_outlet["cum_share"] = sales_by_outlet["share"].cumsum()
sales_by_outlet["share_pct"] = 100 * sales_by_outlet["share"]
sales_by_outlet["cum_share_pct"] = 100 * sales_by_outlet["cum_share"]

sales_by_outlet
```

	Outlet_Identifier	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	OUT027	3.453926e+06	0.185784	0.185784	18.578359	18.578359
1	OUT035	2.268123e+06	0.122000	0.307784	12.200030	30.778389
2	OUT049	2.183970e+06	0.117474	0.425258	11.747378	42.525768
3	OUT017	2.167465e+06	0.116586	0.541844	11.658602	54.184370
4	OUT013	2.142664e+06	0.115252	0.657096	11.525196	65.709565
5	OUT046	2.118395e+06	0.113947	0.771042	11.394658	77.104223
6	OUT045	2.036725e+06	0.109554	0.880596	10.955364	88.059587
7	OUT018	1.851823e+06	0.099608	0.980204	9.960789	98.020377
8	OUT010	1.883402e+05	0.010131	0.990334	1.013065	99.033442
9	OUT019	1.796941e+05	0.009666	1.000000	0.966558	100.000000

```
fig = go.Figure()
fig.add_bar(
    x=sales_by_outlet["Outlet_Identifier"],
    y=sales_by_outlet["share_pct"],
    name="Participación individual (%)"
)
fig.add_scatter(
```

```

x=sales_by_outlet["Outlet_Identifier"],
y=sales_by_outlet["cum_share_pct"],
mode="lines+markers",
name="Participación acumulada (%)",
yaxis="y2"
)
fig.update_layout(
    title="Concentración de ventas por outlet",
    xaxis=dict(title="Outlet"),
    yaxis=dict(title="Participación individual (%)"),
    yaxis2=dict(
        title="Participación acumulada (%)",
        overlaying="y",
        side="right",
        range=[0, 105]
    ),
    legend=dict(orientation="h")
)
fig.show()

```

Unable to display output for mime type(s): text/html

45.1. Comentario analítico

A diferencia del análisis por tipo de producto, aquí suele haber menos unidades agregadas, de modo que la concentración puede hacerse visible con mayor claridad. Si pocas tiendas explican una fracción grande de las ventas, ello puede tener implicaciones operativas, logísticas o estratégicas.

46. Concentración por tipo de outlet

Además de estudiar outlets individuales, conviene analizar el reparto de ventas según tipo de tienda.

```
sales_by_outlet_type = (
    df.groupby("Outlet_Type", dropna=False)["Item_Outlet_Sales"]
        .sum()
        .sort_values(ascending=False)
        .reset_index()
)

sales_by_outlet_type["share"] = (
    sales_by_outlet_type["Item_Outlet_Sales"] /
    sales_by_outlet_type["Item_Outlet_Sales"].sum()
)
sales_by_outlet_type["cum_share"] = sales_by_outlet_type["share"].cumsum()
sales_by_outlet_type["share_pct"] = 100 * sales_by_outlet_type["share"]
sales_by_outlet_type["cum_share_pct"] = 100 * sales_by_outlet_type["cum_share"]

sales_by_outlet_type
```

	Outlet_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Supermarket Type1	1.291734e+07	0.694812	0.694812	69.481228	69.481228
1	Supermarket Type3	3.453926e+06	0.185784	0.880596	18.578359	88.059587
2	Supermarket Type2	1.851823e+06	0.099608	0.980204	9.960789	98.020377
3	Grocery Store	3.680343e+05	0.019796	1.000000	1.979623	100.000000

```
fig = px.bar(
    sales_by_outlet_type,
    x="Outlet_Type",
    y="share_pct",
    title="Participación de ventas por tipo de outlet",
    labels={"share_pct": "Participación (%)", "Outlet_Type": "Tipo de outlet"}
)
fig.update_layout(xaxis_tickangle=-20)
fig.show()
```

Unable to display output for mime type(s): text/html

47. Curva de concentración simplificada

Para construir una curva de concentración simple, ordenamos las unidades por contribución creciente o decreciente y comparamos la acumulación observada contra una referencia uniforme.

Aquí haremos una versión descriptiva, suficiente para fines pedagógicos.

```
def concentration_curve(values):
    values = np.asarray(values, dtype=float)
    values = np.sort(values) # ascendente
    cum_values = np.cumsum(values)
    cum_values = cum_values / cum_values[-1]
    cum_units = np.arange(1, len(values) + 1) / len(values)
    return cum_units, cum_values

x_units, y_conc = concentration_curve(sales_by_item_type["Item_Outlet_Sales"])

curve_df = pd.DataFrame({
    "Fraccion_unidades": x_units,
    "Fraccion_ventas": y_conc
})

fig = go.Figure()
fig.add_scatter(
    x=curve_df["Fraccion_unidades"],
    y=curve_df["Fraccion_ventas"],
    mode="lines",
    name="Curva de concentración"
)
fig.add_scatter(
    x=[0, 1],
    y=[0, 1],
    mode="lines",
    name="Reparto uniforme"
)
fig.update_layout(
    title="Curva de concentración de ventas por tipo de producto",
    xaxis_title="Fracción acumulada de categorías",
    yaxis_title="Fracción acumulada de ventas"
)
fig.show()
```

Unable to display output for mime type(s): text/html

47.1. Lectura de la curva

Cuanto más separada esté la curva respecto de la diagonal de reparto uniforme, mayor será la concentración. En un reparto perfectamente equilibrado, ambas coincidirían. En cambio, una curva muy hundida al inicio y muy empinada al final refleja que pocas categorías explican gran parte del volumen total.

48. Indicadores resumidos de concentración

A modo de síntesis, conviene construir una tabla con indicadores descriptivos que resuman la estructura observada.

Para participaciones s_i , el HHI es $\sum_i s_i^2$. Para valores no negativos ordenados, el Gini se calcula como $2 \sum_i i x_{(i)} / (n \sum_i x_{(i)}) - (n + 1) / n$. Este coeficiente no debe confundirse con Gini-Simpson.

```
top1_item = sales_by_item_type["share"].iloc[0] * 100
top5_item = sales_by_item_type["share"].head(5).sum() * 100

top1_outlet = sales_by_outlet["share"].iloc[0] * 100
top3_outlet = sales_by_outlet["share"].head(3).sum() * 100

def hhi(shares):
    shares = np.asarray(shares, dtype=float)
    return np.sum(shares ** 2)

def gini_nonnegative(values):
    x = np.sort(np.asarray(values, dtype=float))
    if np.any(x < 0) or x.sum() <= 0:
        raise ValueError("Gini requiere valores no negativos y suma positiva")
    n = x.size
    return 2 * np.sum(np.arange(1, n + 1) * x) / (n * x.sum()) - (n + 1) / n

hhi_item = hhi(sales_by_item_type["share"])
gini_item = gini_nonnegative(sales_by_item_type["Item_Outlet_Sales"])

concentration_summary = pd.DataFrame({
    "Ámbito": [
        "Productos",
        "Productos",
        "Productos",
        "Outlets",
        "Outlets",
        "Productos",
        "Productos"
    ],
    "Indicador": [
        "Participación de la categoría líder (%)",
        "Participación acumulada del top 5 (%)",
        "Categorías necesarias para 80% de ventas",
        "Participación del outlet líder (%)",

```

```

    "Participación acumulada de los 3 principales outlets (%)",
    "HHI por tipo de producto",
    "Gini por tipo de producto"
  ],
  "Valor": [
    round(top1_item, 2),
    round(top5_item, 2),
    n80,
    round(top1_outlet, 2),
    round(top3_outlet, 2),
    round(hhi_item, 4),
    round(gini_item, 4)
  ]
})

concentration_summary

```

	Ámbito	Indicador	Valor
0	Productos	Participación de la categoría líder (%)	15.1700
1	Productos	Participación acumulada del top 5 (%)	58.9300
2	Productos	Categorías necesarias para 80 % de ventas	9.0000
3	Outlets	Participación del outlet líder (%)	18.5800
4	Outlets	Participación acumulada de los 3 principales o...	42.5300
5	Productos	HHI por tipo de producto	0.0941
6	Productos	Gini por tipo de producto	0.3968

49. Mini-dashboard de concentración

El objetivo final del capítulo es reunir varias perspectivas en un panel compacto. En un book de Quarto, una estrategia práctica consiste en combinar tablas de KPIs con gráficos interactivos.

49.1. KPIs principales

```
total_sales = df["Item_Outlet_Sales"].sum()
n_item_types = df["Item_Type"].nunique() if "Item_Type" in df.columns else np.nan
n_outlets = df["Outlet_Identifier"].nunique() if "Outlet_Identifier" in df.columns else np.nan

dashboard_kpis = pd.DataFrame({
    "Métrica": [
        "Ventas totales",
        "Tipos de producto",
        "Outlets",
        "Top 3 categorías (%)",
        "Top 3 outlets (%)"
    ],
    "Valor": [
        round(total_sales, 2),
        n_item_types,
        n_outlets,
        round(sales_by_item_type["share"].head(3).sum() * 100, 2),
        round(sales_by_outlet["share"].head(3).sum() * 100, 2)
    ]
})
```

dashboard_kpis

	Métrica	Valor
0	Ventas totales	18591125.41
1	Tipos de producto	16.00
2	Outlets	10.00
3	Top 3 categorías (%)	40.92
4	Top 3 outlets (%)	42.53

50. Discusión final

La concentración es un componente esencial del EDA porque complementa lo aprendido en capítulos previos. Mientras la localización resume el centro, la variabilidad describe la dispersión, la forma estudia la geometría de la distribución y la heterogeneidad compara subgrupos, la concentración revela **cómo se reparte el total entre las unidades que componen el sistema**.

En **Big Mart Sales**, esta dimensión permite identificar si el negocio depende fuertemente de unas pocas categorías o unos pocos outlets, o si el ingreso se distribuye de manera más equilibrada. Esta información es valiosa tanto para fines descriptivos como para una primera interpretación operativa.

51. Ejercicios sugeridos

1. Construye el gráfico de Pareto para `Outlet_Location_Type` y compara su concentración con la observada en `Outlet_Type`.
2. Calcula qué porcentaje de ventas acumula el 20 % de las categorías con mayores ventas.
3. Repite el análisis utilizando el número de registros por categoría en lugar del total de ventas, y compara ambos resultados.
4. Describe, con lenguaje de negocio, si la estructura de ventas parece diversificada o concentrada.

Parte XVII.

Medidas de correlación

52. Medidas de correlación

52.1. Introducción

La correlación resume la dirección y fuerza de ciertos tipos de asociación entre dos variables. Debe acompañarse siempre de una gráfica: conjuntos con el mismo coeficiente pueden tener geometrías completamente diferentes (Anscombe 1973).

Definición. Una correlación es una medida estandarizada de asociación. Pearson describe relación lineal; Spearman y Kendall describen concordancia por orden.

52.2. Covarianza y Pearson

$$s_{XY} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad r = \frac{s_{XY}}{s_X s_Y}. \quad (1)$$

$r \in [-1, 1]$ y es invariante ante cambios positivos de unidad. Un valor cercano a cero significa ausencia de relación lineal fuerte, no independencia (Pearson 1895).

52.3. Spearman

$$\rho_s = \text{corr}(R_X, R_Y), \quad (2)$$

donde R_X y R_Y son rangos. Spearman detecta relaciones monótonas y es menos sensible a la magnitud de extremos, aunque los empates requieren rangos promedio (Spearman 1904).

52.4. Kendall

Si C y D son los números de pares concordantes y discordantes, sin empates,

$$\tau = \frac{C - D}{\binom{n}{2}}. \quad (3)$$

Kendall tiene una interpretación probabilística basada en concordancia y suele ser útil con muestras pequeñas u ordinales (Kendall 1938).

Correlación no implica causalidad. Confusión, selección, tendencias temporales y variables omitidas pueden generar asociaciones sin un mecanismo causal directo.

52.5. Cálculo manual paso a paso

Usaremos cinco pares de observaciones:

$$X = (1, 2, 3, 4, 5), \quad Y = (2, 1, 4, 3, 5), \quad n = 5.$$

52.5.1. Covarianza muestral

Paso 1. Calcular las medias:

$$\bar{x} = \frac{1 + 2 + 3 + 4 + 5}{5} = \frac{15}{5} = 3,$$

$$\bar{y} = \frac{2 + 1 + 4 + 3 + 5}{5} = \frac{15}{5} = 3.$$

Paso 2. Calcular desviaciones:

$$x_i - \bar{x} = (-2, -1, 0, 1, 2),$$

$$y_i - \bar{y} = (-1, -2, 1, 0, 2).$$

Paso 3. Multiplicar desviaciones pareja por pareja:

$$(-2)(-1) = 2, \quad (-1)(-2) = 2, \quad (0)(1) = 0,$$

$$(1)(0) = 0, \quad (2)(2) = 4.$$

Paso 4. Sumar y dividir entre $n - 1 = 4$:

$$s_{XY} = \frac{2 + 2 + 0 + 0 + 4}{4} = \frac{8}{4} = 2.$$

52.5.2. Correlación de Pearson

Paso 1. Sumar cuadrados de las desviaciones de X :

$$(-2)^2 + (-1)^2 + 0^2 + 1^2 + 2^2 = 4 + 1 + 0 + 1 + 4 = 10.$$

Paso 2. Hacer lo mismo para Y :

$$(-1)^2 + (-2)^2 + 1^2 + 0^2 + 2^2 = 1 + 4 + 1 + 0 + 4 = 10.$$

Paso 3. Calcular ambas desviaciones estándar muestrales:

$$s_X = \sqrt{\frac{10}{4}} = \sqrt{2.5} = 1.5811,$$

$$s_Y = \sqrt{\frac{10}{4}} = \sqrt{2.5} = 1.5811.$$

Paso 4. Estandarizar la covarianza:

$$r = \frac{s_{XY}}{s_X s_Y} = \frac{2}{(1.5811)(1.5811)} = \frac{2}{2.5} = 0.8.$$

Existe una asociación lineal positiva fuerte, aunque no perfecta.

52.5.3. Correlación de Spearman

No hay empates. Los rangos de X son $(1, 2, 3, 4, 5)$. Como $Y = (2, 1, 4, 3, 5)$, sus rangos son $(2, 1, 4, 3, 5)$.

Paso 1. Restar los rangos:

$$d_i = R_{X_i} - R_{Y_i} = (-1, 1, -1, 1, 0).$$

Paso 2. Elevar al cuadrado y sumar:

$$\sum d_i^2 = (-1)^2 + 1^2 + (-1)^2 + 1^2 + 0^2 = 1 + 1 + 1 + 1 + 0 = 4.$$

Paso 3. Aplicar la fórmula sin empates:

$$\rho_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} = 1 - \frac{6(4)}{5(5^2 - 1)}.$$

$$\rho_s = 1 - \frac{24}{5(24)} = 1 - \frac{24}{120} = 1 - 0.2 = 0.8.$$

52.5.4. Correlación de Kendall

Con cinco observaciones existen

$$\binom{5}{2} = \frac{5(4)}{2} = 10$$

pares posibles.

Como X ya está ordenada, basta examinar el orden de $Y = (2, 1, 4, 3, 5)$. Hay dos inversiones o pares discordantes:

$$(2, 1) \quad \text{y} \quad (4, 3).$$

Por tanto,

$$D = 2, \quad C = 10 - 2 = 8.$$

Finalmente,

$$\tau = \frac{C - D}{10} = \frac{8 - 2}{10} = \frac{6}{10} = 0.6.$$

Kendall es menor que Pearson y Spearman porque cuenta concordancias de pares, no distancias numéricas.

La fórmula abreviada de Spearman y la cuenta simple de Kendall usadas aquí suponen ausencia de empates. Con empates se requieren rangos promedio y las correcciones correspondientes.

Del cálculo al programa. Pearson requiere acumular medias, desviaciones, productos y cuadrados. Spearman transforma primero los valores en rangos y reutiliza Pearson. Kendall compara todos los pares (i, j) y aumenta un contador de concordancia o discordancia; su versión directa tiene costo cuadrático.

52.6. Lineal, monótona y no monótona

```
import numpy as np
import pandas as pd
import plotly.express as px
from scipy import stats

rng = np.random.default_rng(9)
n = 600
x = rng.uniform(-2, 2, n)
casos = {
    "Lineal": 2*x+rng.normal(0,.7,n),
    "Monótona curva": np.exp(x)+rng.normal(0,.35,n),
    "Parabólica": x**2+rng.normal(0,.25,n)
}
filas=[]
for caso, y in casos.items():
    r=stats.pearsonr(x,y).statistic
    rho=stats.spearmanr(x,y).statistic
    for xi, yi in zip(x,y):
        filas.append({"X":xi,"Y":yi,"Relación":caso,"Pearson":r,"Spearman":rho})
df=pd.DataFrame(filas)
fig=px.scatter(df,x="X",y="Y",facet_col="Relación",color="Relación",opacity=.6,
               hover_data={"Pearson":":.3f","Spearman":":.3f"},
               title="La geometría determina qué correlación resulta informativa")
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Interpretación. Pearson reconoce la recta; Spearman conserva una asociación alta en la curva monótona. Ambas pueden acercarse a cero en una parábola perfectamente visible.

52.7. Sensibilidad a un valor extremo

```
x0=rng.normal(size=100)
y0=.7*x0+rng.normal(scale=.6,size=100)
filas=[]
for extremo in np.linspace(0,20,21):
    xx=np.r_[x0,extremo]
    yy=np.r_[y0,-extremo]
    filas.extend([
        {"Magnitud del extremo":extremo,"Coeficiente":"Pearson","Valor":stats.pearsonr(xx,yy).sta
        {"Magnitud del extremo":extremo,"Coeficiente":"Spearman","Valor":stats.spearmanr(xx,yy).s
        {"Magnitud del extremo":extremo,"Coeficiente":"Kendall","Valor":stats.kendalltau(xx,yy).s
    ])
sens=pd.DataFrame(filas)
fig=px.line(sens,x="Magnitud del extremo",y="Valor",color="Coeficiente",markers=True,
            range_y=[-1,1],title="Un solo punto puede invertir la correlación de Pearson")
fig.add_hline(y=0,line_dash="dot")
fig.show()
```

Unable to display output for mime type(s): text/html

52.8. Cuarteto de Anscombe

```
anscombe = pd.DataFrame({
    "x1": [10,8,13,9,11,14,6,4,12,7,5],
    "x2": [10,8,13,9,11,14,6,4,12,7,5],
    "x3": [10,8,13,9,11,14,6,4,12,7,5],
    "x4": [8,8,8,8,8,8,19,8,8,8],
    "y1": [8.04,6.95,7.58,8.81,8.33,9.96,7.24,4.26,10.84,4.82,5.68],
    "y2": [9.14,8.14,8.74,8.77,9.26,8.10,6.13,3.10,9.13,7.26,4.74],
    "y3": [7.46,6.77,12.74,7.11,7.81,8.84,6.08,5.39,8.15,6.42,5.73],
    "y4": [6.58,5.76,7.71,8.84,8.47,7.04,5.25,12.50,5.56,7.91,6.89]
})
anscombe = pd.concat([
    pd.DataFrame({"x": anscombe[f"x{i}"], "y": anscombe[f"y{i}"], "dataset": f"Serie {i}"})
    for i in range(1, 5)
], ignore_index=True)
fig=px.scatter(anscombe,x="x",y="y",facet_col="dataset",color="dataset",
               trendline="ols",title="Mismos resúmenes, patrones visuales distintos")
fig.update_layout(showlegend=False)
fig.show()
```

Unable to display output for mime type(s): text/html

52.9. Matriz de correlación

Una matriz resume muchos pares, pero puede ocultar no linealidad, subgrupos y tamaños muestrales diferentes. Si se usa eliminación por pares, cada celda puede calcularse con observaciones distintas.

Antes de interpretar un heatmap, documente el tratamiento de faltantes, inspeccione variables casi constantes y revise diagramas de dispersión para los pares relevantes.

52.10. Guía de decisión

Relación esperada	Medida	Gráfica mínima
Lineal, cuantitativa	Pearson	Scatterplot con tendencia
Monótona u ordinal	Spearman	Scatterplot/rangos
Concordancia ordinal	Kendall	Scatterplot o tabla ordinal
No monótona	Ninguna de las anteriores basta	Scatterplot y métodos no lineales

52.11. Puente al caso BigMart

El capítulo Medidas de correlación en Big Mart Sales compara Pearson y Spearman, construye matrices y estudia relaciones por tipo de outlet.

52.12. Ejercicios

1. Construya una relación fuerte con Pearson cercano a cero.
2. Compare los tres coeficientes antes y después de agregar un valor extremo.
3. Explique por qué una matriz calculada por pares puede no ser coherente como conjunto.

Referencias

Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.

Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.

Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.

- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.

- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.

- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*.

Friedrich Perthes; I. H. Besser.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.

Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.

Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.

Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.

GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.

Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.

Han, Jiawei, Jian Pei, y Yiwon Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.

Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.

Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.

Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.

Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.

Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2):

427-32. <https://doi.org/10.2307/1934352>.

- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.

- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.

- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.

- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias,

Universidad Nacional Autónoma de México.

- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1nj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.

Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.

Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.

Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.

Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.

Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.

Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.

Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

53. Medidas de correlación en Big Mart Sales

54. Introducción

Este caso aplica Pearson, Spearman y la inspección visual descritos en Medidas de correlación.

Después de estudiar la localización, la variabilidad, la forma, la heterogeneidad y la concentración, resulta natural incorporar una dimensión adicional del análisis exploratorio: la **correlación**. Mientras los capítulos anteriores se enfocaron principalmente en la descripción univariada o en comparaciones entre grupos, el análisis de correlación permite explorar **relaciones entre variables numéricas**.

En términos generales, la correlación busca responder preguntas como las siguientes:

- ¿Las ventas aumentan cuando aumenta el precio de lista del producto?
- ¿Existe asociación entre la visibilidad del producto y sus ventas?
- ¿El peso del producto se relaciona con otras variables cuantitativas?
- ¿Las asociaciones observadas son lineales, monotónicas o prácticamente inexistentes?

Este capítulo se centra en el estudio de las relaciones entre variables numéricas del conjunto de datos **Big Mart Sales**, con énfasis en la variable objetivo de interés:

`Item_Outlet_Sales`.

Además de calcular coeficientes de correlación, construiremos representaciones gráficas que ayuden a interpretar dichas asociaciones dentro de un contexto de EDA. La meta no es inferir causalidad, sino **detectar patrones de dependencia, redundancia o independencia aproximada** que puedan enriquecer la comprensión del conjunto de datos.

Definición operativa. Pearson resume asociación lineal y Spearman asociación monótona por rangos; ambos requieren inspección gráfica (Pearson 1895; Spearman 1904; Anscombe 1973).

55. Objetivos del capítulo

Al finalizar este capítulo, el lector debería ser capaz de:

1. distinguir entre correlación positiva, negativa y cercana a cero;
2. interpretar la magnitud y el signo de un coeficiente de correlación;
3. comparar correlaciones de Pearson y Spearman;
4. construir una matriz de correlación para variables numéricas;
5. integrar los hallazgos de correlación al dashboard global del libro.

56. Carga y preparación de datos

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

DATA_PATH = "../../../data/bigmart_sales.csv" # Ajustar según tu proyecto
df = pd.read_csv(DATA_PATH)

if "Item_Fat_Content" in df.columns:
    df["Item_Fat_Content"] = (
        df["Item_Fat_Content"]
        .replace({
            "LF": "Low Fat",
            "low fat": "Low Fat",
            "reg": "Regular"
        })
    )

df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	C
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	C
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	C
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	C
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	C

Para estudiar correlación nos concentraremos primero en las variables numéricas.

```
num_df = df.select_dtypes(include=[np.number]).copy()
num_df.columns.tolist()
```

```
['Item_Weight',
 'Item_Visibility',
 'Item_MRP',
 'Outlet_Establishment_Year',
 'Item_Outlet_Sales']
```

En muchos conjuntos de datos de tipo comercial, la cantidad de variables numéricas es moderada. En **Big Mart Sales**, típicamente aparecen variables como:

- `Item_Weight`
- `Item_Visibility`
- `Item_MRP`
- `Outlet_Establishment_Year`
- `Item_Outlet_Sales`

Dependiendo de la versión del dataset, puede haber más o menos variables numéricas. El análisis se adaptará automáticamente a las columnas detectadas.

57. Fundamento conceptual

57.1. Correlación de Pearson

La correlación de Pearson entre dos variables aleatorias X e Y se define como

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}.$$

En la práctica, con datos muestrales, se utiliza su estimador muestral. Este coeficiente mide la intensidad y dirección de una **asociación lineal**.

Sus valores están acotados por

$$-1 \leq r \leq 1.$$

- Si $r \approx 1$, la asociación lineal es positiva fuerte.
- Si $r \approx -1$, la asociación lineal es negativa fuerte.
- Si $r \approx 0$, no hay evidencia de asociación lineal fuerte.

57.2. Correlación de Spearman

La correlación de Spearman se basa en los rangos de los datos. Es especialmente útil cuando la relación entre dos variables no es estrictamente lineal, pero sí **monotónica**.

En el EDA, Pearson y Spearman son complementarias:

- **Pearson**: sensible a relaciones lineales y a valores extremos.
- **Spearman**: más robusta frente a no linealidad monotónica y menos sensible a outliers severos.

57.3. Advertencia importante

La correlación **no implica causalidad**. Dos variables pueden estar correlacionadas por múltiples razones: dependencia directa, mediación por otra variable, estructura del proceso de muestreo o simple coincidencia en muestras pequeñas.

58. Revisión de valores faltantes en variables numéricas

Antes de calcular correlaciones, conviene revisar valores faltantes.

```
missing_num = num_df.isna().sum().sort_values(ascending=False).reset_index()
missing_num.columns = ["Variable", "Faltantes"]
missing_num
```

	Variable	Faltantes
0	Item_Weight	1463
1	Item_Visibility	0
2	Item_MRP	0
3	Outlet_Establishment_Year	0
4	Item_Outlet_Sales	0

Si existen valores faltantes, **pandas** calculará correlaciones utilizando observaciones disponibles por pares. Para EDA esto suele ser suficiente, aunque conviene documentarlo.

59. Matriz de correlación de Pearson

Comenzamos con la matriz de correlación de Pearson para todas las variables numéricas.

```
corr_pearson = num_df.corr(method="pearson")
corr_pearson
```

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Item_Outlet_Sales
Item_Weight	1.000000	-0.014048	0.027141	-0.011588	0.014123
Item_Visibility	-0.014048	1.000000	-0.001315	-0.074834	-0.128625
Item_MRP	0.027141	-0.001315	1.000000	0.005020	0.567574
Outlet_Establishment_Year	-0.011588	-0.074834	0.005020	1.000000	-0.049135
Item_Outlet_Sales	0.014123	-0.128625	0.567574	-0.049135	1.000000

59.1. Mapa de calor

```
fig = px.imshow(
    corr_pearson,
    text_auto=True,
    aspect="auto",
    title="Matriz de correlación de Pearson",
    color_continuous_scale="RdBu",
    zmin=-1,
    zmax=1
)
fig.update_layout(coloraxis_colorbar_title="r")
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

59.2. Interpretación inicial

Esta matriz permite una lectura rápida de las asociaciones lineales entre variables numéricas. En particular, interesa observar:

- qué variables están más asociadas con `Item_Outlet_Sales`;

59. Matriz de correlación de Pearson

- si existen pares de variables numéricas altamente correlacionados entre sí;
- si la mayoría de relaciones son débiles, moderadas o fuertes.

En datasets como **Big Mart Sales**, suele ser común encontrar que **Item_MRP** tiene una asociación positiva visible con las ventas, mientras que otras variables presentan correlaciones más débiles.

60. Correlación de Spearman

Para contrastar con Pearson, calculamos ahora la matriz de Spearman.

```
corr_spearman = num_df.corr(method="spearman")
corr_spearman
```

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Item_Outlet_Sales
Item_Weight	1.000000	-0.014879	0.030822	-0.009393	0.015062
Item_Visibility	-0.014879	1.000000	0.005688	-0.054924	-0.115076
Item_MRP	0.030822	0.005688	1.000000	0.003782	0.562986
Outlet_Establishment_Year	-0.009393	-0.054924	0.003782	1.000000	0.042947
Item_Outlet_Sales	0.015062	-0.115076	0.562986	0.042947	1.000000

```
fig = px.imshow(
    corr_spearman,
    text_auto=True,
    aspect="auto",
    title="Matriz de correlación de Spearman",
    color_continuous_scale="RdBu",
    zmin=-1,
    zmax=1
)
fig.update_layout(coloraxis_colorbar_title=" ")
fig.show()
```

Unable to display output for mime type(s): text/html

60.1. Comparación entre Pearson y Spearman

Cuando Pearson y Spearman tienen valores similares, suele haber evidencia de una asociación aproximadamente lineal o, al menos, de una estructura monótonica estable. Cuando Spearman es claramente mayor que Pearson, puede ser una señal de relación monótonica no lineal o de sensibilidad de Pearson a valores extremos.

Una forma compacta de comparar ambas matrices es centrarse en la variable de interés.

```
target = "Item_Outlet_Sales"

if target in num_df.columns:
    corr_compare = pd.DataFrame({
        "Pearson": corr_pearson[target],
        "Spearman": corr_spearman[target]
    }).sort_values(by="Pearson", ascending=False)

    corr_compare
else:
    print(f"La variable {target} no está presente en las columnas numéricas.")
```

61. Correlación con la variable objetivo

Dado que el fenómeno central del conjunto de datos son las ventas, es útil ordenar las correlaciones con `Item_Outlet_Sales`.

```
if target in num_df.columns:
    sales_corr = (
        corr_pearson[target]
        .drop(labels=[target], errors="ignore")
        .sort_values(ascending=False)
        .reset_index()
    )
    sales_corr.columns = ["Variable", "Correlacion_Pearson"]
    sales_corr
```

```
if target in num_df.columns:
    fig = px.bar(
        sales_corr,
        x="Variable",
        y="Correlacion_Pearson",
        title="Correlación de Pearson con Item_Outlet_Sales",
        labels={"Correlacion_Pearson": "Correlación", "Variable": "Variable"}
    )
    fig.update_layout(xaxis_tickangle=-30)
    fig.show()
```

Unable to display output for mime type(s): text/html

61.1. Comentario analítico

Esta visualización resume qué variables numéricas muestran una asociación lineal más fuerte con las ventas. Una correlación positiva moderada entre `Item_MRP` y `Item_Outlet_Sales`, por ejemplo, puede interpretarse como evidencia de que los productos con mayor precio de lista tienden también a registrar mayores ventas monetarias, aunque esto no implica necesariamente que “vender más caro” cause mayores ventas en sentido causal.

62. Diagramas de dispersión

Las matrices de correlación son útiles, pero pueden ocultar detalles estructurales. Por ello, es indispensable acompañarlas con gráficos de dispersión.

62.1. Ventas vs precio de lista

```
if {"Item_MRP", "Item_Outlet_Sales"}.issubset(df.columns):
    fig = px.scatter(
        df,
        x="Item_MRP",
        y="Item_Outlet_Sales",
        opacity=0.5,
        title="Relación entre Item_MRP e Item_Outlet_Sales",
        trendline="ols"
    )
    fig.show()
```

Unable to display output for mime type(s): text/html

62.2. Ventas vs visibilidad

```
if {"Item_Visibility", "Item_Outlet_Sales"}.issubset(df.columns):
    fig = px.scatter(
        df,
        x="Item_Visibility",
        y="Item_Outlet_Sales",
        opacity=0.5,
        title="Relación entre Item_Visibility e Item_Outlet_Sales",
        trendline="ols"
    )
    fig.show()
```

Unable to display output for mime type(s): text/html

62.3. Ventas vs peso

```
if {"Item_Weight", "Item_Outlet_Sales"}.issubset(df.columns):  
    fig = px.scatter(  
        df,  
        x="Item_Weight",  
        y="Item_Outlet_Sales",  
        opacity=0.5,  
        title="Relación entre Item_Weight e Item_Outlet_Sales",  
        trendline="ols"  
    )  
    fig.show()
```

Unable to display output for mime type(s): text/html

62.4. Qué observar en los diagramas

Los scatter plots permiten responder preguntas que una matriz de correlación no resuelve por sí sola:

- ¿La relación parece lineal?
- ¿Hay subgrupos o clústeres?
- ¿Existen outliers que afecten la correlación?
- ¿La nube de puntos sugiere heterocedasticidad?
- ¿Hay curvatura o estructura por segmentos?

En EDA, esta lectura visual es esencial, porque una correlación numérica puede ser engañosa si la relación no es homogénea.

63. Correlación por subgrupos

Una extensión particularmente rica consiste en estudiar si la correlación cambia según el tipo de outlet o el tipo de producto. Esto conecta el análisis de correlación con la heterogeneidad.

63.1. Ejemplo: correlación entre Item_MRP y ventas por tipo de outlet

```
if {"Outlet_Type", "Item_MRP", "Item_Outlet_Sales"}.issubset(df.columns):
    corr_by_outlet = (
        df.groupby("Outlet_Type")[["Item_MRP", "Item_Outlet_Sales"]]
        .corr(method="pearson")
        .reset_index()
    )

    corr_by_outlet = corr_by_outlet[
        (corr_by_outlet["level_1"] == "Item_MRP")
    ][["Outlet_Type", "Item_Outlet_Sales"]]

    corr_by_outlet.columns = ["Outlet_Type", "Corr_Pearson_MRP_Sales"]
    corr_by_outlet
```

```
if {"Outlet_Type", "Item_MRP", "Item_Outlet_Sales"}.issubset(df.columns):
    fig = px.bar(
        corr_by_outlet,
        x="Outlet_Type",
        y="Corr_Pearson_MRP_Sales",
        title="Correlación MRP vs ventas por tipo de outlet",
        labels={
            "Outlet_Type": "Tipo de outlet",
            "Corr_Pearson_MRP_Sales": "Correlación de Pearson"
        }
    )
    fig.update_layout(xaxis_tickangle=-20)
    fig.show()
```

Unable to display output for mime type(s): text/html

63.2. **Lectura**

Si la intensidad de la correlación cambia notablemente entre subgrupos, eso sugiere que la relación entre variables no es uniforme en todo el dataset. Este hallazgo es importante porque un coeficiente global puede ocultar comportamientos diferenciados por segmento.

64. Tabla resumen de asociaciones

Para cerrar el análisis, conviene construir una tabla corta con las correlaciones más relevantes respecto de la variable objetivo.

```
if target in num_df.columns:
    corr_summary = pd.DataFrame({
        "Variable": [c for c in num_df.columns if c != target],
        "Pearson": [corr_pearson.loc[c, target] for c in num_df.columns if c != target],
        "Spearman": [corr_spearman.loc[c, target] for c in num_df.columns if c != target]
    }).sort_values(by="Pearson", ascending=False)

    corr_summary
```

Podemos incluso añadir una etiqueta cualitativa de intensidad para facilitar la lectura didáctica.

```
def strength_label(r):
    a = abs(r)
    if a < 0.10:
        return "Muy débil"
    elif a < 0.30:
        return "Débil"
    elif a < 0.50:
        return "Moderada"
    elif a < 0.70:
        return "Fuerte"
    else:
        return "Muy fuerte"

if target in num_df.columns:
    corr_summary_labeled = corr_summary.copy()
    corr_summary_labeled["Intensidad_Pearson"] = corr_summary_labeled["Pearson"].apply(strength_label)
    corr_summary_labeled["Intensidad_Spearman"] = corr_summary_labeled["Spearman"].apply(strength_label)
    corr_summary_labeled
```


65. Mini-dashboard de correlación

Para integrarlo al libro, este capítulo puede cerrarse con un panel compacto compuesto por:

1. tabla de correlaciones con la variable objetivo;
2. mapa de calor de Pearson;
3. scatter plot de la relación más relevante;
4. comparación por subgrupos si procede.

65.1. KPIs de correlación

```
if target in num_df.columns:
    strongest_var = corr_summary.iloc[0]["Variable"] if len(corr_summary) > 0 else np.nan
    strongest_corr = corr_summary.iloc[0]["Pearson"] if len(corr_summary) > 0 else np.nan

    corr_kpis = pd.DataFrame({
        "Indicador": [
            "Número de variables numéricas",
            "Variable más correlacionada con ventas",
            "Correlación de Pearson asociada",
            "Correlación de Spearman asociada"
        ],
        "Valor": [
            num_df.shape[1],
            strongest_var,
            round(strongest_corr, 4) if pd.notnull(strongest_corr) else np.nan,
            round(
                corr_summary.loc[corr_summary["Variable"] == strongest_var, "Spearman"].iloc[0],
                4) if pd.notnull(strongest_var) and len(corr_summary) > 0 else np.nan
        ]
    })

corr_kpis
```


66. Discusión final

El análisis de correlación amplía la mirada del EDA al introducir relaciones entre variables. En **Big Mart Sales**, esta dimensión permite identificar qué atributos numéricos están más asociados con las ventas y si dicha asociación es débil, moderada o fuerte.

Sin embargo, el uso de correlaciones debe acompañarse siempre de lectura visual y criterio analítico. Una matriz numérica puede sugerir asociación, pero solo los diagramas de dispersión permiten ver si la relación es lineal, si hay segmentos diferenciados o si algunos valores extremos dominan el comportamiento.

Además, comparar correlaciones globales con correlaciones por subgrupos conecta este capítulo con la idea de heterogeneidad: no todas las relaciones son homogéneas en toda la población.

67. Ejercicios sugeridos

1. Repite el análisis usando solo outlets de un tipo específico y compara los resultados con la correlación global.
2. Calcula la matriz de correlación para un subconjunto de categorías de producto y comenta las diferencias.
3. Investiga si la transformación logarítmica de `Item_Outlet_Sales` cambia de forma importante las correlaciones.
4. Identifica un par de variables con baja correlación lineal pero cuya nube de puntos sugiera alguna estructura no trivial.

Parte XVIII.

Síntesis del EDA

68. Dashboard global

69. Introducción

A lo largo de los capítulos anteriores se desarrolló una ruta progresiva para el análisis exploratorio del conjunto de datos **Big Mart Sales**. La lógica seguida no fue arbitraria: se construyó una descripción estadística de las ventas a partir de seis dimensiones complementarias.

Primero se estudió la **localización**, para responder dónde se sitúan los valores típicos. Después se introdujo la **variabilidad**, para cuantificar la dispersión alrededor de esos centros. Más adelante se abordó la **forma**, con el fin de entender la asimetría, la curtosis y la estructura general de las distribuciones. Luego se examinó la **heterogeneidad**, es decir, las diferencias entre categorías, tipos de outlet y otros subgrupos. La **concentración** permitió analizar cómo se reparte el total entre las unidades del sistema y la **correlación** añadió la exploración de asociaciones entre variables numéricas.

El propósito de este capítulo es integrar todas esas perspectivas en un **dashboard global**, útil tanto para la enseñanza del EDA como para la comunicación de hallazgos en un entorno de análisis aplicado. Este dashboard no sustituye el análisis detallado de los capítulos previos; más bien, actúa como una síntesis estructurada que permite recorrer, de manera compacta, la construcción progresiva de la medida.

70. Objetivos del capítulo

Este capítulo busca que el lector sea capaz de:

1. integrar múltiples dimensiones del EDA en una misma narrativa analítica;
2. construir indicadores resumidos y visualizaciones consistentes entre sí;
3. diseñar un dashboard reproducible en Quarto compatible con un libro;
4. interpretar de forma conjunta las dimensiones de localización, variabilidad, forma, heterogeneidad, concentración y correlación.

71. Carga y preparación de datos

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

DATA_PATH = "../data/bigmart_sales.csv" # Ajustar según tu proyecto
df = pd.read_csv(DATA_PATH)

if "Item_Fat_Content" in df.columns:
    df["Item_Fat_Content"] = (
        df["Item_Fat_Content"]
        .replace({
            "LF": "Low Fat",
            "low fat": "Low Fat",
            "reg": "Regular"
        })
    )

df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	C
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	C
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	C
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	C
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	C

72. Arquitectura conceptual del dashboard

El dashboard global puede entenderse como una composición de seis capas analíticas:

72.1. Capa 1. Localización

Resume el centro de la distribución mediante indicadores como media, mediana, moda y cuantiles.

72.2. Capa 2. Variabilidad

Introduce la amplitud y dispersión mediante rango, desviación estándar, IQR y coeficiente de variación.

72.3. Capa 3. Forma

Permite describir asimetría, curtosis y presencia de colas pesadas o valores atípicos.

72.4. Capa 4. Heterogeneidad

Compara la estructura de ventas entre subgrupos relevantes, como tipos de producto o tipos de outlet.

72.5. Capa 5. Concentración

Evalúa qué tan equilibrada o dominada está la estructura total de ventas.

72.6. Capa 6. Correlación

Explora asociaciones lineales y monótonas entre variables numéricas sin atribuir causalidad.

En un dashboard pedagógico, estas seis capas deben coexistir de forma coherente, de manera que el lector pueda pasar del nivel global al comparativo sin perder el hilo interpretativo.

73. Selección de variable principal

Para articular el dashboard usaremos como variable principal:

```
[ Item_Outlet_Sales, ]
```

ya que representa el fenómeno central del conjunto de datos: las ventas observadas por producto y outlet.

```
sales = df["Item_Outlet_Sales"].dropna()
sales.describe()
```

```
count      8523.000000
mean       2181.288914
std        1706.499616
min         33.290000
25%        834.247400
50%       1794.331000
75%       3101.296400
max       13086.964800
Name: Item_Outlet_Sales, dtype: float64
```


74. Panel de indicadores globales

El primer componente del dashboard debe consistir en un bloque de KPIs generales que resuman el estado del conjunto de datos.

```
def safe_mode(series):
    m = series.mode(dropna=True)
    if len(m) == 0:
        return np.nan
    return m.iloc[0]

mean_sales = sales.mean()
median_sales = sales.median()
mode_sales = safe_mode(sales)
std_sales = sales.std(ddof=1)
iqr_sales = sales.quantile(0.75) - sales.quantile(0.25)
cv_sales = std_sales / mean_sales if mean_sales != 0 else np.nan
skew_sales = sales.skew()
kurt_sales = sales.kurt()

global_kpis = pd.DataFrame({
    "Indicador": [
        "Número de registros",
        "Ventas totales",
        "Media",
        "Mediana",
        "Moda",
        "Desviación estándar",
        "IQR",
        "Coeficiente de variación",
        "Asimetría",
        "Curtosis"
    ],
    "Valor": [
        len(df),
        round(sales.sum(), 2),
        round(mean_sales, 2),
        round(median_sales, 2),
        round(mode_sales, 2) if pd.notnull(mode_sales) else np.nan,
        round(std_sales, 2),
        round(iqr_sales, 2),
        round(cv_sales, 4),
        round(skew_sales, 4),
        round(kurt_sales, 4)
    ]
})
```

```
    ]
  })

global_kpis
```

	Indicador	Valor
0	Número de registros	8.523000e+03
1	Ventas totales	1.859113e+07
2	Media	2.181290e+03
3	Mediana	1.794330e+03
4	Moda	9.587500e+02
5	Desviación estándar	1.706500e+03
6	IQR	2.267050e+03
7	Coefficiente de variación	7.823000e-01
8	Asimetría	1.177500e+00
9	Curtosis	1.615900e+00

Este bloque de KPIs cumple una función importante: reúne en una sola tabla las medidas fundamentales de centro, dispersión y forma que fueron desarrolladas por separado en capítulos previos.

75. Panel de localización y forma

Una forma natural de combinar localización y forma es mediante un histograma enriquecido con líneas de referencia para la media y la mediana.

```
fig = px.histogram(
    df,
    x="Item_Outlet_Sales",
    nbins=50,
    marginal="box",
    title="Distribución global de Item_Outlet_Sales"
)

fig.add_vline(x=mean_sales, line_dash="dash", annotation_text="Media")
fig.add_vline(x=median_sales, line_dash="dot", annotation_text="Mediana")

fig.update_layout(
    xaxis_title="Ventas",
    yaxis_title="Frecuencia"
)
fig.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

75.1. Interpretación

Este panel sintetiza dos preguntas esenciales:

- ¿Dónde se localizan los valores típicos?
- ¿La distribución es aproximadamente simétrica o presenta sesgo?

Cuando la media y la mediana difieren de manera visible, y cuando la cola derecha es más larga, suele haber evidencia de asimetría positiva. En un entorno como Big Mart Sales, esto es compatible con la existencia de observaciones con ventas muy elevadas respecto del grueso de la distribución.

76. Panel de variabilidad

La variabilidad puede representarse con un boxplot acompañado de una tabla resumida.

```
variability_kpis = pd.DataFrame({
    "Medida": ["Mínimo", "Q1", "Mediana", "Q3", "Máximo", "Rango", "IQR", "Desv. estándar", "CV"],
    "Valor": [
        round(sales.min(), 2),
        round(sales.quantile(0.25), 2),
        round(sales.median(), 2),
        round(sales.quantile(0.75), 2),
        round(sales.max(), 2),
        round(sales.max() - sales.min(), 2),
        round(iqr_sales, 2),
        round(std_sales, 2),
        round(cv_sales, 4)
    ]
})
variability_kpis
```

	Medida	Valor
0	Mínimo	33.2900
1	Q1	834.2500
2	Mediana	1794.3300
3	Q3	3101.3000
4	Máximo	13086.9600
5	Rango	13053.6700
6	IQR	2267.0500
7	Desv. estándar	1706.5000
8	CV	0.7823

```
fig = px.box(
    df,
    y="Item_Outlet_Sales",
    points="suspectedoutliers",
    title="Variabilidad global de Item_Outlet_Sales"
)
fig.update_layout(yaxis_title="Ventas")
fig.show()
```

Unable to display output for mime type(s): text/html

76. *Panel de variabilidad*

Este panel muestra de forma visual la amplitud de la distribución, la caja intercuartílica y la presencia de observaciones extremas.

77. Panel de heterogeneidad

La heterogeneidad requiere comparar subgrupos. Una opción muy expresiva es contrastar las ventas por tipo de outlet y por tipo de producto.

77.1. Ventas por tipo de outlet

```
fig = px.box(
    df,
    x="Outlet_Type",
    y="Item_Outlet_Sales",
    color="Outlet_Type",
    title="Heterogeneidad de ventas por tipo de outlet"
)
fig.update_layout(
    xaxis_title="Tipo de outlet",
    yaxis_title="Ventas",
    xaxis_tickangle=-20,
    showlegend=False
)
fig.show()
```

Unable to display output for mime type(s): text/html

77.2. Ventas por tipo de producto

Para evitar saturación visual, puede ser útil mostrar solo las categorías con mayor venta total.

```
top_item_types = (
    df.groupby("Item_Type")["Item_Outlet_Sales"]
      .sum()
      .sort_values(ascending=False)
      .head(8)
      .index
)

df_top_items = df[df["Item_Type"].isin(top_item_types)].copy()

fig = px.box(
```

```

df_top_items,
x="Item_Type",
y="Item_Outlet_Sales",
color="Item_Type",
title="Heterogeneidad de ventas en las principales categorías de producto"
)
fig.update_layout(
    xaxis_title="Tipo de producto",
    yaxis_title="Ventas",
    xaxis_tickangle=-45,
    showlegend=False
)
fig.show()

```

Unable to display output for mime type(s): text/html

77.3. Tabla comparativa por grupo

```

heterogeneity_summary = (
    df.groupby("Outlet_Type")["Item_Outlet_Sales"]
    .agg(["count", "mean", "median", "std"])
    .reset_index()
    .rename(columns={
        "count": "n",
        "mean": "media",
        "median": "mediana",
        "std": "desv_std"
    })
)

heterogeneity_summary

```

	Outlet_Type	n	media	mediana	desv_std
0	Grocery Store	1083	339.828500	256.9988	260.851582
1	Supermarket Type1	5577	2316.181148	1990.7420	1515.965558
2	Supermarket Type2	928	1995.498739	1655.1788	1375.932889
3	Supermarket Type3	935	3694.038558	3364.9532	2127.760054

Este bloque permite observar que la heterogeneidad no se limita a diferencias de media: también pueden cambiar la mediana, la dispersión y la presencia de valores extremos.

78. Panel de concentración

Para el dashboard global conviene incorporar una visualización compacta de concentración. El análisis por tipo de producto suele ser especialmente ilustrativo.

```
sales_by_item_type = (
    df.groupby("Item_Type")["Item_Outlet_Sales"]
      .sum()
      .sort_values(ascending=False)
      .reset_index()
)

sales_by_item_type["share"] = (
    sales_by_item_type["Item_Outlet_Sales"] /
    sales_by_item_type["Item_Outlet_Sales"].sum()
)

sales_by_item_type["cum_share"] = sales_by_item_type["share"].cumsum()
sales_by_item_type["share_pct"] = 100 * sales_by_item_type["share"]
sales_by_item_type["cum_share_pct"] = 100 * sales_by_item_type["cum_share"]

sales_by_item_type.head()
```

	Item_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Fruits and Vegetables	2.820060e+06	0.151688	0.151688	15.168849	15.168849
1	Snack Foods	2.732786e+06	0.146994	0.298683	14.699412	29.868261
2	Household	2.055494e+06	0.110563	0.409246	11.056317	40.924578
3	Frozen Foods	1.825735e+06	0.098205	0.507450	9.820464	50.745042
4	Dairy	1.522594e+06	0.081899	0.589349	8.189897	58.934939

```
fig = go.Figure()
fig.add_bar(
    x=sales_by_item_type["Item_Type"],
    y=sales_by_item_type["share_pct"],
    name="Participación (%)"
)
fig.add_scatter(
    x=sales_by_item_type["Item_Type"],
    y=sales_by_item_type["cum_share_pct"],
    mode="lines+markers",
    name="Acumulado (%)",
    yaxi="y2"
)
```

```
fig.update_layout(
    title="Concentración de ventas por tipo de producto",
    yaxis=dict(title="Participación (%)",
    yaxis2=dict(
        title="Acumulado (%)",
        overlaying="y",
        side="right",
        range=[0, 105]
    ),
    xaxis=dict(title="Tipo de producto", tickangle=-45)
)
fig.show()
```

Unable to display output for mime type(s): text/html

78.1. Indicadores breves de concentración

```
def categories_to_reach_threshold(table, threshold=0.8):
    idx = np.argmax(table["cum_share"].to_numpy() >= threshold)
    return idx + 1

n80 = categories_to_reach_threshold(sales_by_item_type, 0.80)
top3_item_share = sales_by_item_type["share"].head(3).sum() * 100

concentration_kpis = pd.DataFrame({
    "Indicador": [
        "Top 3 categorías (%)",
        "Categorías para 80% de ventas"
    ],
    "Valor": [
        round(top3_item_share, 2),
        n80
    ]
})

concentration_kpis
```

	Indicador	Valor
0	Top 3 categorías (%)	40.92
1	Categorías para 80 % de ventas	9.00

79. Panel de correlación

La síntesis se completa con Pearson y Spearman. La comparación ayuda a detectar asociaciones monotónicas que no son estrictamente lineales.

```
num_cols = df.select_dtypes(include=np.number).columns
pearson = df[num_cols].corr(method="pearson")
spearman = df[num_cols].corr(method="spearman")

fig = px.imshow(
    pearson,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    title="Matriz de correlación de Pearson"
)
fig.update_layout(height=650)
fig.show()
```

Unable to display output for mime type(s): text/html

```
corr_target = pd.DataFrame({
    "Variable": [c for c in num_cols if c != "Item_Outlet_Sales"],
    "Pearson": [pearson.loc[c, "Item_Outlet_Sales"] for c in num_cols if c != "Item_Outlet_Sales"],
    "Spearman": [spearman.loc[c, "Item_Outlet_Sales"] for c in num_cols if c != "Item_Outlet_Sales"]
}).melt(id_vars="Variable", var_name="Coeficiente", value_name="Correlación")

fig = px.bar(
    corr_target,
    x="Variable",
    y="Correlación",
    color="Coeficiente",
    barmode="group",
    range_y=[-1, 1],
    title="Asociaciones con Item_Outlet_Sales"
)
fig.add_hline(y=0, line_dash="dot")
fig.show()
```

Unable to display output for mime type(s): text/html

El panel de correlación sirve para generar hipótesis, no para afirmar causalidad. Cada coeficiente relevante debe contrastarse con un diagrama de dispersión y con el patrón de faltantes.

80. Ensamblaje del dashboard global

En un libro de Quarto, el dashboard puede presentarse como una secuencia ordenada de paneles. Una estructura práctica es la siguiente:

1. **Bloque inicial de KPIs:** resumen numérico de alto nivel.
2. **Distribución global:** histograma y boxplot para localización, forma y variabilidad.
3. **Comparación entre grupos:** heterogeneidad por outlet y por producto.
4. **Concentración:** Pareto y tabla de acumulación.
5. **Correlación:** heatmap y comparación Pearson–Spearman.
6. **Texto interpretativo:** síntesis narrativa de hallazgos.

80.1. Ejemplo de lectura integrada

Desde una perspectiva estadística, el dashboard global permite articular una lectura de este tipo:

- La distribución global de ventas presenta un centro identificable, pero también una dispersión importante.
- La separación entre media y mediana, junto con la cola derecha, sugiere asimetría positiva.
- Los boxplots muestran variabilidad considerable y presencia de valores atípicos.
- Las comparaciones por tipo de outlet y tipo de producto revelan heterogeneidad entre subgrupos.
- El análisis de concentración muestra que una fracción limitada de categorías contribuye de forma desproporcionada al total.

Esta lectura resume precisamente la lógica del EDA progresivo: la medida no se agota en un único número, sino que se construye mediante varias capas complementarias.

81. Propuesta de cierre visual para el book

En un proyecto de Quarto Book, este capítulo puede funcionar como capítulo de cierre analítico. Una buena práctica consiste en terminar con una tabla compacta de hallazgos.

```
final_findings = pd.DataFrame({
    "Dimensión": [
        "Localización",
        "Variabilidad",
        "Forma",
        "Heterogeneidad",
        "Concentración"
    ],
    "Pregunta guía": [
        "¿Dónde se ubican los valores típicos?",
        "¿Qué tan dispersas son las ventas?",
        "¿La distribución es simétrica o sesgada?",
        "¿Cambian las ventas entre grupos?",
        "¿Se reparte el total de manera equilibrada?"
    ],
    "Indicador / Visual principal": [
        "Media, mediana, histograma",
        "IQR, desviación estándar, boxplot",
        "Asimetría, curtosis, histograma",
        "Boxplots por grupo, tablas comparativas",
        "Pareto, shares acumulados"
    ]
})

final_findings
```

	Dimensión	Pregunta guía	Indicador / Visual principal
0	Localización	¿Dónde se ubican los valores típicos?	Media, mediana, histograma
1	Variabilidad	¿Qué tan dispersas son las ventas?	IQR, desviación estándar, boxplot
2	Forma	¿La distribución es simétrica o sesgada?	Asimetría, curtosis, histograma
3	Heterogeneidad	¿Cambian las ventas entre grupos?	Boxplots por grupo, tablas comparativas
4	Concentración	¿Se reparte el total de manera equilibrada?	Pareto, shares acumulados

82. Conclusiones

El dashboard global no debe entenderse como una simple colección de gráficas, sino como una **síntesis argumentada del EDA**. Su valor reside en que integra distintos niveles de descripción en una sola narrativa:

- el centro de la distribución,
- su amplitud,
- su geometría,
- sus diferencias entre subgrupos,
- y su estructura de reparto.

En el caso de **Big Mart Sales**, esta integración permite comunicar de forma compacta cómo se comportan las ventas desde varios ángulos complementarios. Para fines docentes, el dashboard muestra que la estadística descriptiva es una construcción progresiva de la medida; para fines aplicados, ofrece un instrumento claro de exploración y comunicación.

83. Sugerencias de extensión

1. Añadir filtros interactivos por `Outlet_Type`, `Outlet_Size` o `Item_Fat_Content`.
2. Integrar una sección de correlación entre variables numéricas.
3. Incorporar mapas si se dispone de georreferenciación de outlets.
4. Conectar el dashboard con la fase posterior de modelado predictivo.

84. Integración al libro de Quarto

Si tu libro ya incluye los capítulos anteriores, puedes agregar este cierre junto con el capítulo de concentración en el archivo `_quarto.yml` con una estructura como la siguiente:

```
book:
  chapters:
    - index.qmd
    - 01_localizacion_bigmart.qmd
    - 02_variabilidad_bigmart.qmd
    - 03_forma_bigmart.qmd
    - 04_heterogeneidad_bigmart.qmd
    - 05_concentracion_bigmart.qmd
    - 06_dashboard_global_bigmart.qmd
```

Con ello quedará completa la secuencia del EDA progresivo en formato de book.

Parte XIX.

Análisis Multivariado

85. Análisis Exploratorio de Datos Multivariado: Teoría y Visualización

El análisis exploratorio de datos (EDA) ha evolucionado significativamente en los últimos años. Mientras que el enfoque clásico se centraba en estadísticas descriptivas y visualizaciones básicas, el EDA multivariable moderno reconoce que los datos reales son intrínsecamente de alta dimensión y que su comprensión requiere integrar múltiples niveles de análisis de manera estructurada.

86. La maldición de la dimensionalidad

Cuando el número de variables p crece, la geometría del espacio se vuelve profundamente contraintuitiva. El fenómeno conocido como **maldición de la dimensionalidad** (*curse of dimensionality*, Bellman, 1957) describe cómo el volumen del espacio crece tan rápidamente que los datos se vuelven inevitablemente dispersos.

Consideremos una hipersfera de radio r inscrita en un hipercubo de lado $2r$ en $\mathbb{R}^d$. La proporción del volumen de la esfera respecto al cubo es:

$$\frac{V_{\text{esfera}}}{V_{\text{cubo}}} = \frac{\pi^{d/2} r^d}{\Gamma(d/2 + 1) \cdot (2r)^d} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)}$$

En 2 y en 3 dimensiones, esta proporción es relativamente alta (~78 % y ~52 % respectivamente), como se puede apreciar en la siguiente visualización interactiva donde $r = 1$.

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Sin embargo, esta proporción **tiende a cero** cuando $d \rightarrow \infty$. Esto significa que la mayoría del volumen del hipercubo se concentra en las esquinas, y la esfera (que representa la región de interés) ocupa una fracción cada vez menor del espacio total.

Unable to display output for mime type(s): text/html

Esto implica que para el **análisis de datos en alta dimensión**, mantener una densidad de muestreo fija ρ en $\mathbb{R}^d$, el número de observaciones necesario escala como:

$$n \sim \rho \cdot L^d$$

Con $p = 20$ variables, el número de correlaciones posibles es $\binom{20}{2} = 190$. Con $p = 100$, son 4,950. Revisar e interpretar cada relación manualmente deja de ser practicable. Por ello necesitamos métodos que resuman la información sin perder lo esencial.

87. Fundamentos matemáticos del PCA

El Análisis de Componentes Principales es una técnica de reducción de dimensionalidad que transforma un conjunto de datos con variables correlacionadas en un nuevo sistema de variables no correlacionadas llamadas **componentes principales**. Cada componente es una combinación lineal de las variables originales y representa una dirección de máxima varianza, preservando así la mayor cantidad de información posible de los datos.

87.1. Formulación Matemática y Procedimiento

Dado un conjunto de datos $X \in \mathbb{R}^{n \times p}$, donde n es el número de muestras y p el número de variables, el proceso se rige por la siguiente secuencia lógica:

1. Estandarización y Centrado

Para evitar que las unidades de medida distorsionen el análisis, los datos deben transformarse a una escala común. Esto se realiza en dos pasos:

- **Centrado:** restar la media de cada variable
- **Escalado:** dividir entre su desviación estándar

De esta forma, cada variable tiene media cero y varianza unitaria.

$$\tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}$$

donde:

- μ_j es la media de la variable j
- σ_j es su desviación estándar

2. **Cálculo de la Matriz de Covarianza:** Se construye la matriz S (o $\mathbf{C}$ dependiendo de la notación, $\mathbf{C}$ es común en ML), que resume la variabilidad de cada variable y la relación entre ellas:

$$S = \frac{1}{n-1} \tilde{X}^T \tilde{X}$$

De forma elemento a elemento:

$$s_{jk} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)(x_{ik} - \mu_k)$$

donde:

- s_{jj} representa la **varianza** de la variable j
- s_{jk} representa la **covarianza** entre las variables j y k

Esta matriz es simétrica y semidefinida positiva, lo que garantiza que sus eigenvalores sean reales y no negativos.

3. **Descomposición Espectral:** Se resuelve el problema de autovalores para encontrar las direcciones principales:

$$S\mathbf{v} = \lambda\mathbf{v}$$

Aquí, λ representa los eigenvalores (la varianza explicada en esa dirección) y $\mathbf{v}$ los eigenvectores (las direcciones o *loadings*).

4. **Optimización y Ordenamiento:** El primer componente principal w_1 es el eigenvector asociado al eigenvalor máximo λ_1 . Los componentes siguientes se obtienen maximizando la varianza restante bajo la condición de ser ortogonales a los anteriores. Se clasifican de forma descendente: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$.
5. **Proyección y Reducción:** Finalmente, se proyectan los datos originales sobre el subespacio generado por los primeros k componentes:

$$\mathbf{Z} = \tilde{X}\mathbf{V}_k$$

Donde $\mathbf{V}_k$ es la matriz que contiene los primeros k eigenvectores ordenados.

Cada componente captura una parte de la variabilidad total de los datos, la cual equivale a la traza de la matriz de covarianza ($\sum \lambda_j$). La importancia de cada componente se mide a través de la **Proporción de Varianza Explicada (PVE)**:

$$\text{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}$$

Esto permite reducir la dimensionalidad manteniendo, por ejemplo, el 95 % de la varianza acumulada original.

87.2. Ejemplo Práctico: Cálculo Manual Paso a Paso

A continuación, se desarrolla el proceso completo partiendo de una matriz de datos crudos de dos variables ($p = 2$):

1. **Definición de la matriz de datos original:** Supongamos $n = 3$ observaciones (ej. mediciones de dos sensores) representados como filas y $p = 2$ variables representadas como columnas:

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{pmatrix}$$

2. **Centrado de los datos:** Calculamos la media de cada columna: $\bar{x}_1 = 3$ y $\bar{x}_2 = 4$. Restamos la media a cada fila para obtener $\tilde{X}$:

$$\tilde{X} = \begin{pmatrix} 2-3 & 3-4 \\ 3-3 & 4-4 \\ 4-3 & 5-4 \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

3. **Cálculo de la matriz de covarianza (S):** Aplicamos $S = \frac{1}{n-1} \tilde{X}^T \tilde{X}$ con $n-1 = 2$:

$$S = \frac{1}{2} \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

4. **Obtención de eigenvalores (λ):** Resolvemos el determinante $\det(S - \lambda I) = 0$:

$$\det \begin{pmatrix} 1-\lambda & 1 \\ 1 & 1-\lambda \end{pmatrix} = (1-\lambda)^2 - 1 = 0 \implies \lambda^2 - 2\lambda = 0$$

Los resultados son $\lambda_1 = 2$ y $\lambda_2 = 0$. La varianza total es $2 + 0 = 2$. El primer componente explica el 100 % de la varianza.

5. **Obtención de eigenvectores ($\mathbf{v}$):** Para $\lambda_1 = 2$, resolvemos $(S - 2I)\mathbf{v}_1 = 0$:

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{21} \end{pmatrix} = 0 \implies v_{11} = v_{21}$$

Normalizando para que $\|v\| = 1$: $v_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \approx \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$.

6. **Proyección de los datos ($k = 1$):** Transformamos los datos centrados a la nueva dimensión $Z = \tilde{X}v_1$:

$$Z = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix} = \begin{pmatrix} -1.414 \\ 0 \\ 1.414 \end{pmatrix}$$

7. **Conclusión:** Dado que los datos originales estaban perfectamente alineados ($x_2 = x_1 + 1$), el PCA ha logrado reducir la dimensionalidad de 2 a 1 sin ninguna pérdida de información, capturando toda la estructura de los datos en un solo eje.

87.3. Ejemplo Práctico: Visualización paso a paso

A continuación, se desarrolla el proceso completo de nuevo incluyendo una visualización interactiva en cada paso para ilustrar cómo se transforman los datos a lo largo del proceso de PCA.

1. **Definición de la matriz de datos original**

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{pmatrix}$$

```
import numpy as np
import plotly.graph_objects as go

X = np.array([[2,3],[3,4],[4,5]])

fig = go.Figure()
fig.add_trace(go.Scatter(
```

```

x=X[:,0], y=X[:,1],
mode='markers+text',
text=['P1','P2','P3'],
textposition='top center'
))
fig.update_layout(title="Datos originales")
fig.show()

```

Unable to display output for mime type(s): text/html

2. Centrado de los datos

$$\tilde{X} = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

```

X_centered = X - X.mean(axis=0)

fig = go.Figure()
fig.add_hline(y=0)
fig.add_vline(x=0)

fig.add_trace(go.Scatter(
    x=X_centered[:,0], y=X_centered[:,1],
    mode='markers+text',
    text=['P1','P2','P3'],
    textposition='top center'
))

fig.update_layout(title="Datos centrados")
fig.show()

```

Unable to display output for mime type(s): text/html

3. Cálculo de la matriz de covarianza

$$S = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

```

S = np.cov(X_centered, rowvar=False)

fig = go.Figure(data=go.Heatmap(
    z=S,
    text=S.round(2),
))

```

```
fig.update_layout(title="Matriz de covarianza")
fig.show()
```

Unable to display output for mime type(s): text/html

4. Obtención de eigenvalores

$$\lambda_1 = 2, \quad \lambda_2 = 0$$

```
eigvals, eigvecs = np.linalg.eig(S)

fig = go.Figure()
fig.add_trace(go.Bar(
    x=['1', '2'],
    y=eigvals
))

fig.update_layout(title="Varianza por componente")
fig.show()
```

Unable to display output for mime type(s): text/html

5. Obtención de eigenvectores

$$v_1 \approx \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$$

```
v1 = eigvecs[:, np.argmax(eigvals)]
t = np.linspace(-2, 2, 100)

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=X_centered[:, 0], y=X_centered[:, 1],
    mode='markers'
))

fig.add_trace(go.Scatter(
    x=t*v1[0],
    y=t*v1[1],
    mode='lines',
    name='PC1'
))

fig.update_layout(title="Dirección principal (PC1)")
fig.show()
```

Unable to display output for mime type(s): text/html

6. Proyección de los datos

$$Z = \begin{pmatrix} -1.414 \\ 0 \\ 1.414 \end{pmatrix}$$

```
Z = X_centered @ v1

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=Z,
    y=[0,0,0],
    mode='markers+text',
    text=['P1', 'P2', 'P3'],
    textposition='top center'
))

fig.update_layout(title="Datos proyectados en PC1")
fig.show()
```

Unable to display output for mime type(s): text/html

7. Conclusión

Dado que los datos están perfectamente alineados:

- Toda la varianza se concentra en una dirección
- PCA reduce de 2D a 1D sin pérdida
- La interpretación es clara: la dirección de máxima varianza coincide con la línea que conecta los puntos, y el segundo componente no aporta información adicional.

88. PCA y la Descomposición en Valores Singulares (SVD)

En la práctica, en las paqueterías modernas PCA **nunca** se calcula mediante la eigendescomposición de S directamente (pueden ver el ejemplo de la documentación de scikitlearn). En cambio, se utiliza la **Descomposición en Valores Singulares (SVD)** de la matriz centrada $\tilde{X}$, que es numéricamente más estable y eficiente.

Toda matriz $\tilde{X} \in \mathbb{R}^{n \times p}$ admite la descomposición:

$$\tilde{X} = U\Sigma V^T$$

donde:

- $U \in \mathbb{R}^{n \times n}$: matriz ortogonal de **vectores singulares izquierdos** (columnas = eigenvectores de $\tilde{X}\tilde{X}^T$)
- $\Sigma \in \mathbb{R}^{n \times p}$: matriz diagonal de **valores singulares** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(n,p)} \geq 0$
- $V \in \mathbb{R}^{p \times p}$: matriz ortogonal de **vectores singulares derechos** (columnas = eigenvectores de $\tilde{X}^T\tilde{X}$)

La relación entre SVD y PCA es directa:

$$S = \frac{1}{n-1} \tilde{X}^T \tilde{X} = \frac{1}{n-1} (U\Sigma V^T)^T (U\Sigma V^T) = \frac{1}{n-1} V\Sigma^2 V^T$$

Por tanto:

- Las **columnas de V** son los **eigenvectores de S** (componentes principales)
- Los **eigenvalores de S** son $\lambda_k = \sigma_k^2 / (n-1)$
- Los **scores** (proyecciones) son $Z = \tilde{X}V = U\Sigma$

¿Por qué SVD y no eigendescomposición directa?

La eigendescomposición de S requiere calcular primero $S = \tilde{X}^T \tilde{X} / (n-1)$, lo cual:

1. **Pierde precisión numérica:** los números de condición se elevan al cuadrado. Si $\tilde{X}$ tiene número de condición κ , S tiene κ^2 .
2. **Es ineficiente cuando $n \ll p$:** la SVD puede trabajar directamente con la matrix de $n \times p$ usando la *thin SVD*, evitando la costosa matrix $p \times p$.
3. **El algoritmo de Golub-Reinsch** para SVD es más estable numéricamente que los algoritmos de eigendescomposición para matrices no simétricas.

88. PCA y la Descomposición en Valores Singulares (SVD)

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

X = load_iris().data
X_std = StandardScaler().fit_transform(X)

# Método 1: eigendecomposición de S
S = np.cov(X_std.T)
eigenvalues, eigenvectors = np.linalg.eigh(S)
# eigh ordena de menor a mayor, invertir
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# Método 2: SVD directa sobre X_std
U, sigma, Vt = np.linalg.svd(X_std, full_matrices=False)
lambda_svd = sigma**2 / (len(X_std) - 1)

print("Eigenvalores via S: ", np.round(eigenvalues, 6))
print("Eigenvalores via SVD:", np.round(lambda_svd, 6))
print("Son idénticos:", np.allclose(eigenvalues, lambda_svd))
```

```
Eigenvalores via S: [2.938085 0.920165 0.147742 0.020854]
Eigenvalores via SVD: [2.938085 0.920165 0.147742 0.020854]
Son idénticos: True
```

89. Ejemplo práctico: Implementación paso a paso

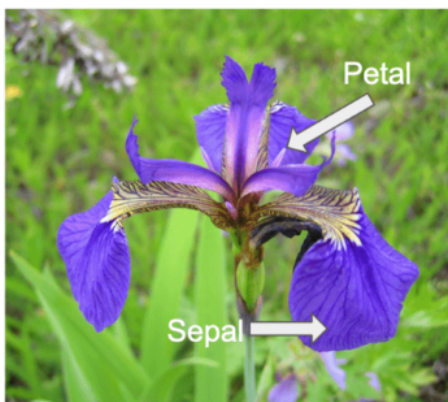
Dataset: Iris

Para este ejemplo usaremos Scikit-learn, mediante el clásico dataset *Iris* (flores de lirio de este lado del charco), que contiene mediciones de flores de tres especies (*setosa*, *versicolor* y *virginica*). Cada observación está descrita por cuatro variables numéricas:

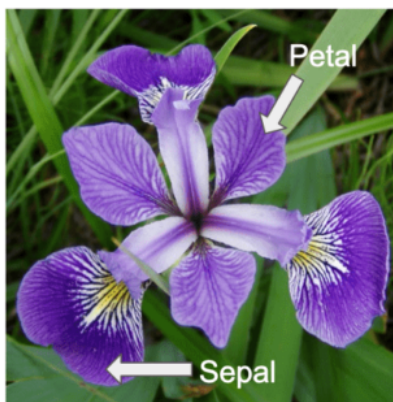
- longitud del sépalo
- ancho del sépalo
- longitud del pétalo
- ancho del pétalo

Estas variables capturan características morfológicas de las flores y presentan correlaciones entre sí, lo que lo convierte en un ejemplo ideal para ilustrar cómo PCA identifica estructura y reduce dimensionalidad.

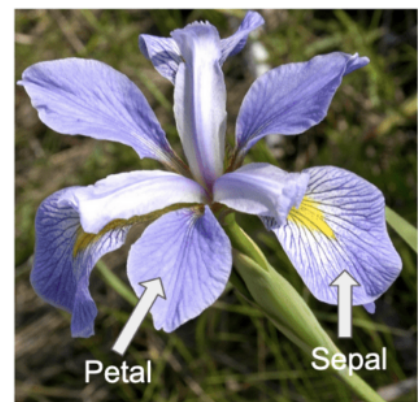
Iris setosa



Iris versicolor



Iris virginica



```
iris = load_iris()
X = iris.data
y = iris.target
nombres = iris.target_names
feature_names = ['Sépalo largo', 'Sépalo ancho', 'Pétalo largo', 'Pétalo ancho']

df_iris = pd.DataFrame(X, columns=feature_names)
df_iris['Especie'] = [nombres[i] for i in y]
df_iris.describe().round(3)
```

	Sépalo largo	Sépalo ancho	Pétalo largo	Pétalo ancho
count	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.300	2.000	1.000	0.100
25 %	5.100	2.800	1.600	0.300
50 %	5.800	3.000	4.350	1.300
75 %	6.400	3.300	5.100	1.800
max	7.900	4.400	6.900	2.500

Paso 1: Estandarización

La estandarización es **obligatoria** cuando las variables tienen escalas distintas. Sin ella, las variables con mayor varianza absoluta dominarían el PCA independientemente de su relevancia.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Después de estandarizar: $\bar{z}_j = 0$ y $s_{z_j} = 1$ para toda variable j .

```
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# Verificar
df_check = pd.DataFrame({
    'Variable': feature_names,
    'Media original': X.mean(axis=0).round(3),
    'Desv original': X.std(axis=0).round(3),
    'Media estand.': X_std.mean(axis=0).round(10),
    'Desv estand.': X_std.std(axis=0).round(3)
})
df_check
```

	Variable	Media original	Desv original	Media estand.	Desv estand.
0	Sépalo largo	5.843	0.825	-0.0	1.0
1	Sépalo ancho	3.057	0.434	-0.0	1.0
2	Pétalo largo	3.758	1.759	-0.0	1.0
3	Pétalo ancho	1.199	0.760	-0.0	1.0

Paso 2: Matriz de covarianza

```

S = np.cov(X_std.T)

fig = go.Figure(data=go.Heatmap(
    z=S,
    x=feature_names,
    y=feature_names,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(S, 3),
    texttemplate='%{text}',
    textfont=dict(size=13),
    hovertemplate='%{y}  %{x}<br>cov =%{z:.3f}<extra></extra>',
    colorbar=dict(title='Covarianza')
))
fig.update_layout(
    title="Matriz de covarianza - datos estandarizados (= matriz de correlación)",
    height=420
)
fig.show()

```

Unable to display output for mime type(s): text/html

i Nota

Cuando los datos están estandarizados, la matriz de covarianza es idéntica a la **matriz de correlación de Pearson**. Por eso PCA sobre datos estandarizados equivale a hacer PCA sobre correlaciones.

Paso 3: Eigendecomposición vía SVD

```

pca = PCA()
scores = pca.fit_transform(X_std)

eigenvalues = pca.explained_variance_
pve = pca.explained_variance_ratio_ * 100
pve_acum = np.cumsum(pve)

df_pca = pd.DataFrame({
    'Componente': [f'PC{i+1}' for i in range(len(eigenvalues))],
    'Eigenvalue ( )': eigenvalues.round(4),
    '% Varianza': pve.round(2),
    '% Acumulado': pve_acum.round(2)
})
df_pca

```

	Componente	Eigenvalue ()	% Varianza	% Acumulado
0	PC1	2.9381	72.96	72.96
1	PC2	0.9202	22.85	95.81
2	PC3	0.1477	3.67	99.48
3	PC4	0.0209	0.52	100.00

Paso 4: Scree plot

```
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=df_pca['Componente'], y=df_pca['% Varianza'],
    name='% Varianza',
    text=[f"{v:.1f}%" for v in pve],
    textposition='outside'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=df_pca['Componente'], y=pve_acum,
    mode='lines+markers',
    line=dict(width=2.5),
    marker=dict(size=9),
    name='% Acumulado'
), secondary_y=True)

fig.add_hline(y=80, line_dash="dash", line_color="gray",
              annotation_text="80%", secondary_y=True)

fig.update_yaxes(title_text="% Varianza por componente", secondary_y=False)
fig.update_yaxes(title_text="% Varianza acumulada", range=[0, 110], secondary_y=True)
fig.update_layout(
    title="Scree Plot - Regla de Kaiser: retener componentes con  $\lambda_k > 1$ ",
    height=380
)
fig.show()
```

Unable to display output for mime type(s): text/html

💡 Tip

Criterios para elegir k :

- **Regla de Kaiser:** retener componentes con $\lambda_k > 1$ (solo aplica con datos estandarizados)
- **Varianza acumulada:** retener los primeros k que superen el 80 % (o 90 %)
- **Scree plot:** buscar el “codo” donde la pendiente cae abruptamente
- **Interpretabilidad de dominio:** ¿los componentes tienen sentido en el problema?

Paso 5: Biplot

El **biplot** superpone los scores de las observaciones y los vectores de los loadings en el mismo espacio reducido, permitiendo interpretar simultáneamente la estructura de las observaciones y la contribución de las variables.

```
loadings = pca.components_ # shape: (n_components, n_features)
scores_df = pd.DataFrame(scores[:, :2], columns=['PC1', 'PC2'])
scores_df['Especie'] = [nombres[i] for i in y]

fig = go.Figure()

# Scores
for especie in nombres:
    mask = scores_df['Especie'] == especie
    fig.add_trace(go.Scatter(
        x=scores_df.loc[mask, 'PC1'],
        y=scores_df.loc[mask, 'PC2'],
        mode='markers',
        marker=dict(size=7, opacity=0.7),
        name=especie
    ))

# Vectores de loadings escalados
escala = 3.0
for i, feat in enumerate(feature_names):
    fig.add_annotation(
        x=loadings[0, i] * escala,
        y=loadings[1, i] * escala,
        ax=0, ay=0,
        arrowhead=3, arrowwidth=2,
        xref='x', yref='y', axref='x', ayref='y',
        showarrow=True
    )
    fig.add_annotation(
        x=loadings[0, i] * escala * 1.15,
        y=loadings[1, i] * escala * 1.15,
        text=f"<b>{feat}</b>",
        showarrow=False,
        font=dict(size=11)
    )

fig.update_xaxes(title_text=f"PC1 ({pve[0]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text=f"PC2 ({pve[1]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title="Biplot: scores + loadings - Iris",
    height=480,
```

```

    legend=dict(title='Especie')
)
fig.show()

```

Unable to display output for mime type(s): text/html

Interpretación del biplot:

- Las flechas (loadings) representan los **vectores de las variables proyectados** sobre el nuevo sistema de coordenadas definido por los componentes principales. Es decir, cada flecha muestra cómo una variable original se expresa en la nueva base.
- Flechas largas → la variable tiene **gran proyección (norma)** sobre ese plano, por lo que contribuye significativamente a la variabilidad capturada por los componentes.
- Flechas apuntando en la misma dirección → sus vectores forman un **ángulo pequeño**, lo que indica alta correlación (producto punto positivo grande).
Flechas opuestas → correlación negativa.
Flechas ortogonales → variables no correlacionadas.
- La posición de los puntos corresponde a la **proyección de las observaciones** en el subespacio generado por los primeros componentes.
La separación entre grupos refleja qué tan bien este subespacio (de menor dimensión) preserva la estructura original de los datos.

En conjunto, el biplot muestra simultáneamente: - un **cambio de base** (álgebra lineal)

- y una **proyección de los datos** en un subespacio de menor dimensión

Paso 6: Loadings detallados

```

df_loadings = pd.DataFrame(
    loadings[:2].T,
    index=feature_names,
    columns=['PC1', 'PC2']
).round(4)

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=[
        "<b>Componente Principal 1 (PC1)</b>",
        "<b>Componente Principal 2 (PC2)</b>"
    ]
)

# PC1
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[0],

```



```

        text=[f"{v:.3f}" for v in loadings[0]],
        textposition='outside'
    ), row=1, col=1)

# PC2
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[1],
    text=[f"{v:.3f}" for v in loadings[1]],
    textposition='outside'
), row=1, col=2)

# Ejes
for c in [1, 2]:
    fig.update_yaxes(
        range=[-1.1, 1.1],
        zeroline=True,
        zerolinecolor='lightgray',
        row=1, col=c
    )
    fig.update_xaxes(
        tickangle=-25,
        row=1, col=c
    )

# Hacer más visibles los títulos de subplots
for ann in fig['layout']['annotations']:
    ann['font'] = dict(size=14)

# Nota interpretativa
fig.update_layout(
    showlegend=False,
    height=420,
    annotations=list(fig.layout.annotations) + [
        dict(
            x=0.5, y=-0.2,
            xref='paper', yref='paper',
            text="PC1  tamaño general | PC2  contraste estructural del sépalo",
            showarrow=False,
            font=dict(size=12, color='gray'),
            xanchor='center'
        )
    ]
)

fig.show()

```

Unable to display output for mime type(s): text/html

Interpretación de los loadings:

Cada componente principal es una **combinación lineal de las variables originales**. Es decir:

$$PC_k = w_{1k}x_1 + w_{2k}x_2 + \cdots + w_{pk}x_p$$

donde los *loadings* w_{jk} indican el peso de cada variable en ese componente.

- **PC1** combina fuertemente *pétalo largo*, *pétalo ancho* y *sépalo largo* (coeficientes positivos similares), por lo que representa un eje de **tamaño general de la flor**. El *sépalo ancho* contribuye negativamente, ajustando ligeramente esa dirección.
- **PC2** está dominado por *sépalo ancho*, con poca contribución del resto, por lo que captura un **contraste específico en la forma del sépalo**.

En términos geométricos, cada componente define una **dirección en el espacio original**, y los loadings son las **coordenadas de esa dirección** en la base de variables originales.

90. Ejemplo avanzado: Dataset con ruido

El problema: una partícula en movimiento que no podemos ver directamente

Imagina que una partícula traza una trayectoria helicoidal en el espacio tridimensional. Nosotros no podemos observarla directamente — solo tenemos acceso a **sensores**, y cada sensor introduce su propio ruido de medición.

El desafío central es: dado un conjunto de lecturas ruidosas de múltiples sensores, ¿podemos recuperar la trayectoria original?

PCA responde exactamente a esa pregunta. Pero antes de aplicarlo, construyamos el experimento desde cero.

Paso 1 — La trayectoria real y el ruido de medición

Generamos 500 puntos sobre una hélice paramétrica:

$$x(t) = t, \quad y(t) = \cos(t), \quad z(t) = \sin(t), \quad t \in [-10, 10]$$

Luego añadimos ruido gaussiano independiente en y y z para simular la imprecisión de los instrumentos de medida.

```
n_points = 500
t = np.linspace(-10, 10, n_points)

# Trayectoria limpia
x_clean = t
y_clean = np.cos(t)
z_clean = np.sin(t)

# Observaciones ruidosas (lo que miden los sensores)
sigma = 0.5
y_noise = y_clean + np.random.normal(0, sigma, n_points)
z_noise = z_clean + np.random.normal(0, sigma, n_points)

points = np.column_stack([x_clean, y_noise, z_noise])

# Visualización
fig = go.Figure()

fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers',
```

```

    marker=dict(size=3, color='steelblue', opacity=0.5),
    name='Mediciones ruidosas'
))

fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines',
    line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

fig.update_layout(
    title='Trayectoria helicoidal: señal real vs. mediciones ruidosas',
    scene=dict(
        xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
        yaxis=dict(range=[-5, 5]),
        xaxis=dict(range=[-11, 11])
    ),
    legend=dict(x=0.01, y=0.99),
    height=520
)
fig.show()

```

Unable to display output for mime type(s): text/html

Los puntos azules son lo que los sensores registran. La línea roja es la realidad que queremos recuperar. A simple vista ya es difícil distinguir la estructura subyacente del ruido.

Paso 2 — Tres sensores, tres perspectivas distintas

En un experimento real, distintos sensores observan el mismo fenómeno desde **ángulos diferentes**. Cada sensor proyecta la nube de puntos sobre su propio plano de referencia y mide coordenadas locales (u, v) en ese plano — más, inevitablemente, su propio ruido adicional.

Simulamos tres sensores con planos de proyección distintos.

```

# Funciones auxiliares

def projection(points, plane_point, plane_normal):
    """Proyecta cada punto sobre un plano definido por un punto y su normal."""
    n = plane_normal / np.linalg.norm(plane_normal)
    d = points - plane_point
    dist = d.dot(n)
    return points - np.outer(dist, n)

def plane_axes(plane_normal):
    """Devuelve dos vectores ortogonales que forman la base del plano."""

```

```

n = plane_normal / np.linalg.norm(plane_normal)
# Vector arbitrario no paralelo a n
arb = np.array([1, 0, 0]) if abs(n[0]) < 0.9 else np.array([0, 1, 0])
u = np.cross(n, arb)
u /= np.linalg.norm(u)
v = np.cross(n, u)
v /= np.linalg.norm(v)
return u, v

def sensor_reading(points, plane_point, plane_normal, noise_std=0.5):
    """
    Proyecta puntos sobre el plano del sensor y devuelve:
    - coordenadas 2D locales (s, t) con ruido adicional
    - la proyección 3D (para visualización)
    - los vectores de la base del plano (u, v)
    """
    proj_3d = projection(points, plane_point, plane_normal)
    u, v = plane_axes(plane_normal)

    proj_vectors = proj_3d - plane_point
    s = proj_vectors.dot(u) + np.random.normal(0, noise_std, len(points))
    t = proj_vectors.dot(v) + np.random.normal(0, noise_std, len(points))

    return s, t, proj_3d, u, v

def plane_mesh(plane_point, plane_normal, size=12, res=20):
    """Genera una malla rectangular para visualizar el plano del sensor."""
    u, v = plane_axes(plane_normal)
    lin = np.linspace(-size, size, res)
    uu, vv = np.meshgrid(lin, lin)
    X = plane_point[0] + uu * u[0] + vv * v[0]
    Y = plane_point[1] + uu * u[1] + vv * v[1]
    Z = plane_point[2] + uu * u[2] + vv * v[2]
    return X, Y, Z

# Definición de los tres sensores
sensors = [
    dict(point=np.array([5., -6., 0.]), normal=np.array([1., -1., 0.]),
        color='royalblue', name='Sensor A'),
    dict(point=np.array([10., 0., 0.]), normal=np.array([10., 0., 0.1]),
        color='seagreen', name='Sensor B'),
    dict(point=np.array([3., 3., 3.]), normal=np.array([1., 1., 1.]),
        color='darkorange', name='Sensor C'),
]

readings = []
for s in sensors:

```

```

sc, tc, proj3d, u, v = sensor_reading(
    points, s['point'], s['normal'], noise_std=0.5
)
readings.append({'s': sc, 't': tc, 'proj3d': proj3d,
                 'u': u, 'v': v, **s})

```

Perspectiva 3D: sensores y sus proyecciones

```

fig = go.Figure()

# Trayectoria real
fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines', line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

# Puntos originales
fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers', marker=dict(size=2, color='lightgray', opacity=0.4),
    name='Mediciones originales'
))

for r in readings:
    # Plano del sensor
    px_m, py_m, pz_m = plane_mesh(r['point'], r['normal'], size=10)
    fig.add_trace(go.Surface(
        x=px_m, y=py_m, z=pz_m,
        colorscale=[[0, r['color']], [1, r['color']]],
        opacity=0.15, showscale=False,
        name=f"Plano {r['name']}", showlegend=False
    ))
    # Proyección sobre el plano
    fig.add_trace(go.Scatter3d(
        x=r['proj3d'][:, 0], y=r['proj3d'][:, 1], z=r['proj3d'][:, 2],
        mode='markers', marker=dict(size=2.5, color=r['color'], opacity=0.7),
        name=r['name']
    ))

fig.update_layout(
    title='Los tres sensores proyectan la nube sobre sus planos respectivos',
    scene=dict(xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
               xaxis=dict(range=[-12, 12]),
               yaxis=dict(range=[-8, 8])),
    legend=dict(x=0.01, y=0.99),
    height=560
)

```

```
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor “aplana” la nube sobre su propio plano. Ninguno captura la hélice perfectamente, pero cada uno aporta información parcial y complementaria.

Lo que registra cada sensor: coordenadas 2D

```
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=[r['name'] for r in readings],
    shared_yaxes=False
)

for i, r in enumerate(readings, 1):
    fig.add_trace(go.Scatter(
        x=r['s'], y=r['t'],
        mode='markers',
        marker=dict(size=3, color=r['color'], opacity=0.6),
        name=r['name'],
        showlegend=False
    ), row=1, col=i)
    fig.update_xaxes(title_text='u (eje 1 del plano)', row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')
    fig.update_yaxes(title_text='v (eje 2 del plano)' if i == 1 else '',
                     row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')

fig.update_layout(
    title='Lecturas brutas de cada sensor (coordenadas 2D locales)',
    height=400
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor produce una imagen 2D distorsionada de la misma hélice. El sensor A la ve casi como una elipse, el B como una banda horizontal, el C como una nube más compacta. Ninguno por sí solo revela la hélice original.

Paso 3 — Construir la matriz de datos multi-sensor

Apilamos las seis coordenadas (2 por cada sensor) para formar una **matriz de datos** $X \in \mathbb{R}^{500 \times 6}$. Cada fila es una muestra; cada columna es una variable medida por algún sensor.

Esta es exactamente la estructura con la que trabaja PCA: muchas variables correlacionadas que comparten una estructura latente de baja dimensión.

```
# Matriz de datos: 500 muestras × 6 variables (2 por sensor)
data_matrix = np.column_stack([r['s'] for r in readings] +
                               [r['t'] for r in readings])
# Reordenamos: s_A, t_A, s_B, t_B, s_C, t_C
data_matrix = np.column_stack([
    readings[0]['s'], readings[0]['t'],
    readings[1]['s'], readings[1]['t'],
    readings[2]['s'], readings[2]['t'],
])

col_labels = ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']

print(f"Dimensión de la matriz de datos: {data_matrix.shape}")
print(f"Variables: {col_labels}")
```

Dimensión de la matriz de datos: (500, 6)

Variables: ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']

Matriz de correlación entre sensores

```
corr = np.corrcoef(data_matrix.T)

fig = go.Figure(data=go.Heatmap(
    z=corr,
    x=col_labels, y=col_labels,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(corr, 2),
    texttemplate='%{text}',
    textfont=dict(size=11),
    colorbar=dict(title='Correlación')
))

fig.update_layout(
    title='Correlación entre las 6 variables medidas por los sensores',
    height=420
)
fig.show()
```

Unable to display output for mime type(s): text/html

La matriz de correlación revela la redundancia. Variables del mismo sensor están fuertemente correlacionadas entre sí, y también con variables de otros sensores — evidencia de que todas miden el mismo fenómeno subyacente. PCA explotará precisamente esa redundancia.

Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?

PCA descompone la varianza total en **componentes ortogonales ordenados por importancia**. Si el fenómeno real vive en pocas dimensiones, la mayor parte de la varianza se concentrará en los primeros componentes.

```
pca_full = PCA(n_components=6)
components_full = pca_full.fit_transform(data_matrix)

explained = pca_full.explained_variance_ratio_
cumulative = np.cumsum(explained)

# Scree plot
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=[f'PC{i+1}' for i in range(6)],
    y=explained * 100,
    name='% Varianza individual',
    text=[f'{v*100:.1f}%' for v in explained],
    textposition='outside',
    marker_color='steelblue'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=[f'PC{i+1}' for i in range(6)],
    y=cumulative * 100,
    mode='lines+markers',
    name='% Acumulado',
    line=dict(color='crimson', width=2.5),
    marker=dict(size=9)
), secondary_y=True)

fig.add_hline(y=90, line_dash='dash', line_color='gray',
              annotation_text='90%', secondary_y=True)

fig.update_yaxes(title_text='% Varianza por componente', secondary_y=False)
fig.update_yaxes(title_text='% Varianza acumulada', range=[0, 110],
                 secondary_y=True)
fig.update_layout(
    title='Scree plot: ¿cuántos componentes capturan la señal?',
    height=400
)
fig.show()
```

```
print("Varianza explicada por componente:")
for i, (v, c) in enumerate(zip(explained, cumulative)):
    print(f"  PC{i+1}: {v*100:5.1f}%    acumulado: {c*100:5.1f}%")
```

Unable to display output for mime type(s): text/html

Varianza explicada por componente:

PC1:	89.3%	acumulado:	89.3%
PC2:	5.3%	acumulado:	94.6%
PC3:	3.7%	acumulado:	98.4%
PC4:	0.6%	acumulado:	98.9%
PC5:	0.6%	acumulado:	99.5%
PC6:	0.5%	acumulado:	100.0%

Los primeros **2–3 componentes** concentran la gran mayoría de la varianza. Esto confirma lo que esperábamos: el sistema tiene **2 grados de libertad reales** (la hélice es una curva en 3D, parametrizada por un solo parámetro t), y el resto es ruido.

Paso 5 — Proyección en 2D: la hélice emerge del ruido

Con solo los dos primeros componentes principales recuperamos la estructura esencial del movimiento.

```
pca_2d = PCA(n_components=2)
proj_2d = pca_2d.fit_transform(data_matrix)

# Línea de referencia: sin(t) proyectada para comparación
t_ref = np.linspace(-12, 12, 300)
ref_x = t_ref / t_ref.max() * proj_2d[:, 0].max()

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=proj_2d[:, 0], y=proj_2d[:, 1],
    mode='markers',
    marker=dict(size=4, color='steelblue', opacity=0.6),
    name='Muestras (PC1 vs PC2)'
))

# Colorear por el parámetro t para mostrar la continuidad
fig.add_trace(go.Scatter(
    x=proj_2d[:, 0], y=proj_2d[:, 1],
    mode='markers',
    marker=dict(
        size=4, opacity=0.8,
        color=t,
        colorscale='Viridis',
```

```

        colorbar=dict(title='t (tiempo)', len=0.6)
    ),
    name='Coloreado por t'
))

fig.update_xaxes(title_text='Componente Principal 1',
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text='Componente Principal 2',
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title=f'PC1 vs PC2 - {pca_2d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    height=480
)
fig.show()

```

Unable to display output for mime type(s): text/html

Al colorear los puntos por el parámetro t (el “tiempo” de la trayectoria), vemos que el orden temporal se preserva perfectamente: PCA ha recuperado la parametrización natural de la hélice a partir de seis variables ruidosas.

Paso 6 — Proyección en 3D: confirmando la estructura helicoidal

Agregando el tercer componente recuperamos la geometría completa.

```

pca_3d = PCA(n_components=3)
proj_3d = pca_3d.fit_transform(data_matrix)

fig = go.Figure()

# Muestras coloreadas por t
fig.add_trace(go.Scatter3d(
    x=proj_3d[:, 0], y=proj_3d[:, 1], z=proj_3d[:, 2],
    mode='markers',
    marker=dict(
        size=3, opacity=0.7,
        color=t,
        colorscale='Viridis',
        colorbar=dict(title='t', len=0.5)
    ),
    name='Muestras'
))

# Trayectoria original reescalada para superposición visual
scale = proj_3d[:, 0].std() / x_clean.std()
fig.add_trace(go.Scatter3d(

```

```

x=x_clean * scale,
y=y_clean * proj_3d[:, 1].std(),
z=z_clean * proj_3d[:, 2].std(),
mode='lines',
line=dict(color='crimson', width=4),
name='Trayectoria original (reescalada)'
))

fig.update_layout(
    title=f'Proyección 3D - {pca_3d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    scene=dict(
        xaxis_title='PC 1',
        yaxis_title='PC 2',
        zaxis_title='PC 3',
        xaxis=dict(range=[-15, 15]),
        yaxis=dict(range=[-7, 7]),
        zaxis=dict(range=[-7, 7])
    ),
    legend=dict(x=0.01, y=0.99),
    height=560
)
fig.show()

```

Unable to display output for mime type(s): text/html

La nube de puntos reconstruida en el espacio PCA sigue fielmente la hélice original. PCA ha separado la señal del ruido sin conocer a priori la geometría de la trayectoria.

i La idea central

PCA no sabe nada sobre hélices ni sobre física. Solo busca las direcciones de **máxima varianza** en los datos. El hecho de que esas direcciones coincidan con la geometría real de la trayectoria es una consecuencia de que **la señal real tiene más varianza que el ruido**.

Cuando $\text{SNR} = \sigma_{\text{señal}} / \sigma_{\text{ruido}} \gg 1$, los primeros componentes principales capturan la señal; los últimos, el ruido.

⚠ Limitación importante

PCA funciona bien aquí porque la hélice es una curva **linealizable** a través de sus coordenadas en el espacio de observaciones. Para estructuras genuinamente no lineales (manifolds curvos), se necesitan técnicas como **UMAP** o **t-SNE**.

91. Limitaciones del PCA

91.1. Linealidad

PCA busca direcciones de máxima varianza **lineales**. No puede capturar estructuras no lineales en los datos.

```
from sklearn.datasets import make_swiss_roll

X_swiss, t_swiss = make_swiss_roll(n_samples=800, noise=0.1, random_state=42)
X_swiss_std = StandardScaler().fit_transform(X_swiss)

pca_swiss = PCA(n_components=2)
scores_swiss = pca_swiss.fit_transform(X_swiss_std)

fig = make_subplots(1, 2,
                    subplot_titles=['Swiss Roll (3D original)',
                                    'Proyección PCA (2D) - estructura perdida'],
                    specs=[[{'type': 'scatter3d'}, {'type': 'scatter'}]])

fig.add_trace(go.Scatter3d(
    x=X_swiss[:,0], y=X_swiss[:,1], z=X_swiss[:,2],
    mode='markers',
    marker=dict(size=3, color=t_swiss, colorscale='Viridis', opacity=0.7),
    showlegend=False
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=scores_swiss[:,0], y=scores_swiss[:,1],
    mode='markers',
    marker=dict(size=4, color=t_swiss, colorscale='Viridis', opacity=0.7),
    showlegend=False
), row=1, col=2)

fig.update_layout(height=420,
                  annotations=[dict(
                      x=0.5, y=-0.08, xref='paper', yref='paper',
                      text="PCA colapsa el Swiss Roll: puntos lejanos en la variedad quedan superpuestos",
                      showarrow=False, font=dict(size=12, color='gray'), xanchor='center'
                  )])
fig.show()
```

Unable to display output for mime type(s): text/html

Este es un ejemplo clásico: el Swiss Roll es una variedad no lineal que PCA no puede “desenrollar”. Técnicas como **Isomap** o **UMAP** sí pueden recuperar la estructura subyacente.

91.2. Sensibilidad a outliers

PCA minimiza la varianza de la reconstrucción, que es equivalente a minimizar el error cuadrático medio. Por tanto, es **muy sensible a outliers**, ya que un solo punto alejado puede dominar la dirección del primer componente.

Unable to display output for mime type(s): text/html

La alternativa robusta es **Robust PCA** (Candès et al., 2011), que descompone $X = L + S$ donde L es la componente de bajo rango y S es una matriz dispersa de outliers.

91.3. PCA no captura estructura de clases

PCA es un método **no supervisado** — maximiza varianza global sin considerar etiquetas. Puede ocurrir que la dirección de máxima varianza no sea la que mejor separa las clases.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X_iris_std = StandardScaler().fit_transform(load_iris().data)
y_iris = load_iris().target
nombres_iris = load_iris().target_names

# PCA
pca_comp = PCA(n_components=2)
scores_pca = pca_comp.fit_transform(X_iris_std)

# LDA
lda = LinearDiscriminantAnalysis(n_components=2)
scores_lda = lda.fit_transform(X_iris_std, y_iris)

fig = make_subplots(1, 2,
                    subplot_titles=['PCA (no supervisado)', 'LDA (supervisado)'])

colors = px.colors.qualitative.Set1[:3]
for i, esp in enumerate(nombres_iris):
    mask = y_iris == i
    fig.add_trace(go.Scatter(
        x=scores_pca[mask, 0], y=scores_pca[mask, 1],
        mode='markers', marker=dict(color=colors[i], size=7, opacity=0.7),
        name=esp, legendgroup=esp
    ), row=1, col=1)
    fig.add_trace(go.Scatter(
        x=scores_lda[mask, 0], y=scores_lda[mask, 1],
```

```

        mode='markers', marker=dict(color=colors[i], size=7, opacity=0.7),
        name=esp, legendgroup=esp, showlegend=False
    ), row=1, col=2)

fig.update_layout(height=400,
    title="PCA vs LDA en Iris - LDA optimiza separación entre clases")
fig.show()

```

Unable to display output for mime type(s): text/html

91.4. Asunción de normalidad (implícita)

PCA es óptimo en el sentido de máxima varianza bajo **cualquier distribución**. Sin embargo, su interpretación probabilística (PCA probabilístico, Tipping & Bishop 1999) asume normalidad:

$$x = Wz + \mu + \varepsilon, \quad z \sim \mathcal{N}(0, I), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Para datos con distribuciones muy asimétricas o discretas, otras técnicas pueden ser más apropiadas.

92. Más allá del PCA: técnicas no lineales

Cuando los datos tienen estructuras no lineales, PCA falla. Las siguientes técnicas abordan esta limitación. Analizaremos dos de las más populares: **t-SNE** y **UMAP**.

Para esta sección emplearemos el MNIST dataset, un dataset de imágenes de dígitos escritos a mano que tiene una estructura no lineal compleja con las siguientes características:

- 70,000 imágenes de dígitos manuscritos (28x28 píxeles → 784 dimensiones)
- 10 clases (dígitos 0–9)
- Desafío clásico para reducción de dimensionalidad

Unable to display output for mime type(s): text/html

Notemos que si realizamos PCA sobre este dataset, los dígitos no se separan claramente en el espacio de componentes principales. Esto se debe a que la estructura de los datos es altamente no lineal: dígitos similares pueden estar cerca en el espacio original pero no necesariamente en el espacio PCA.

Unable to display output for mime type(s): text/html

A continuación, veremos cómo técnicas no lineales como t-SNE y UMAP pueden revelar la estructura subyacente de los dígitos, agrupando visualmente los dígitos similares y separando claramente las clases, a pesar de la complejidad no lineal de los datos.

92.1. t-SNE (t-distributed Stochastic Neighbor Embedding)

t-SNE (van der Maaten & Hinton, 2008) construye una distribución de probabilidad sobre pares de puntos en el espacio original y minimiza la divergencia KL con la distribución en el espacio reducido:

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

donde p_{ij} captura similitudes en el espacio original y q_{ij} en el espacio reducido usando una distribución t de Student.

Características:

- Excelente para visualización en 2D/3D
- Preserva estructura **local** (vecinos cercanos)
- **No preserva distancias globales** — la distancia entre clusters no es interpretable
- No determinista: ejecuciones distintas dan resultados distintos
- No produce una transformación reutilizable para nuevos datos
- Hiperparámetro clave: **perplexity** (típicamente 5–50)

La *perplexity* controla cuántos vecinos considera relevantes cada punto al construir la representación. Formalmente, está relacionada con la **entropía de una distribución de probabilidad local**:

$$\text{Perplexity} = 2^{H(P_i)}$$

donde P_i describe qué tan probable es que otros puntos sean vecinos de x_i . Ajustar la perplexity es crucial: valores muy bajos pueden fragmentar la estructura, mientras que valores muy altos pueden mezclar clusters distintos.

Unable to display output for mime type(s): text/html

92.2. UMAP (Uniform Manifold Approximation and Projection)

UMAP (McInnes et al., 2018) está fundamentado en la **teoría de categorías** y la geometría Riemanniana. El algoritmo construye una representación del manifold topológico de los datos y optimiza una representación en baja dimensión que preserve esa topología.

Matemáticamente, UMAP modela cada punto x_i como el centro de una bola geodésica de radio ρ_i (distancia al vecino más cercano), construyendo un grafo difuso ponderado con pesos:

$$w(x_i, x_j) = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

y simetrizado como:

$$B(x_i, x_j) = w(x_i, x_j) + w(x_j, x_i) - w(x_i, x_j) \cdot w(x_j, x_i)$$

La proyección minimiza la divergencia de entropía cruzada fuzzy entre el grafo original y el proyectado.

Ventajas sobre t-SNE:

Característica	t-SNE	UMAP
Velocidad	Lento ($O(n^2)$)	Rápido ($O(n^{1.14})$)
Estructura global	No preserva	Preserva mejor
Nuevos datos	No (sin extensión)	Sí (<code>transform()</code>)
Determinismo	No	Sí (con <code>random_state</code>)
Escalabilidad	Hasta ~10k puntos	Hasta millones

Unable to display output for mime type(s): text/html

Situación	Método recomendado
-----------	--------------------

92.3. Cuándo usar cada método

Situación	Método recomendado
Preprocesamiento antes de modelado	PCA
Visualización exploratoria	t-SNE o UMAP
Dataset grande (>50k puntos)	UMAP
Necesito transformar nuevos datos	PCA o UMAP
Interpretabilidad de componentes	PCA (o Sparse PCA)
Estructura no lineal compleja	UMAP
Análisis supervisado + reducción	LDA

La elección del método de reducción de dimensionalidad depende del objetivo del análisis y de la naturaleza de los datos. No existe una técnica universalmente superior: cada método enfatiza distintos aspectos de la estructura.

PCA es adecuado cuando se busca una representación lineal, interpretable y reutilizable, especialmente en contextos de preprocesamiento para modelos. En contraste, t-SNE y UMAP están diseñados principalmente para exploración visual, priorizando la preservación de relaciones locales, aunque sacrifican interpretabilidad y capacidad de generalización.

UMAP ofrece un equilibrio interesante entre estructura local y global, además de escalar mejor a datasets grandes. Finalmente, métodos como LDA incorporan información de clases, siendo más apropiados cuando el objetivo es discriminación supervisada.

En conjunto, la elección del método debe responder a la pregunta central del análisis: ¿buscamos interpretar, visualizar o modelar?

93. Resumen y conexiones

El PCA es el punto de partida del análisis multivariante moderno. Sus conexiones van más allá de la estadística descriptiva:

- **Álgebra lineal:** eigendescomposición, SVD, espacios vectoriales
- **Estadística:** modelo probabilístico gaussiano, máxima verosimilitud
- **Optimización:** problema de Rayleigh quotient, multiplicadores de Lagrange
- **Aprendizaje automático:** preprocesamiento, regularización, autoencoders lineales
- **Geometría diferencial:** UMAP extiende PCA al caso de variedades Riemannianas

La familia de métodos de reducción de dimensionalidad puede verse como un espectro continuo desde lo lineal (PCA) hasta lo fuertemente no lineal (t-SNE, UMAP), con un trade-off entre interpretabilidad, preservación de estructura global y escalabilidad.

94. Ejercicios propuestos

1. **PCA desde cero:** implementa PCA usando únicamente `numpy.linalg.svd` sin usar `sklearn`. Verifica que tus eigenvalores y eigenvectores coincidan con los de `sklearn.decomposition.PCA`.
 2. **Efecto de la estandarización:** aplica PCA al dataset Iris con y sin estandarización. ¿Cambian los componentes? ¿Cambia la varianza explicada? Justifica matemáticamente.
 3. **Reconstrucción:** dado un dataset X y su proyección PCA a k componentes, reconstruye la aproximación $\hat{X} = Z_k W_k^T + \mu$. Grafica el error de reconstrucción $\|X - \hat{X}\|_F$ en función de k .
 4. **Comparación t-SNE vs UMAP:** aplica ambas técnicas al dataset de dígitos MNIST (disponible en `sklearn.datasets.load_digits`). Compara visualmente la separación de clases y el tiempo de cómputo.
 5. **Regla de Kaiser:** ¿por qué la regla de Kaiser ($\lambda > 1$) solo aplica cuando los datos están estandarizados? Demuéstralo con un ejemplo donde viola la regla sin estandarizar.
-

94.1. Recursos recomendados: EDA multivariado

A continuación se listan libros y recursos clave para profundizar en análisis exploratorio de datos multivariado, reducción de dimensionalidad y visualización. Los pongo en mi orden de preferencia, pero todos son excelentes y complementarios.

94.1.1. Análisis multivariado (nivel intermedio)

- **Brunton & Kutz — *Data-Driven Science and Engineering***
<https://databookuw.com/>
→ Enfoque moderno: PCA, SVD, sistemas dinámicos. (Libre)
 - **Trevor Hastie, Robert Tibshirani, Jerome Friedman — *The Elements of Statistical Learning***
<https://web.stanford.edu/~hastie/ElemStatLearn/>
→ PCA, LDA, reducción de dimensionalidad y aprendizaje estadístico. (Libre)
 - **Gareth James et al. — *An Introduction to Statistical Learning***
<https://www.statlearning.com/>
→ Versión más accesible del anterior, incluye PCA y LDA. (Libre)
 - **Kevin Murphy — *Probabilistic Machine Learning: An Introduction***
<https://probml.github.io/pml-book/book1.html>
→ Enfoque moderno probabilístico, incluye PCA y modelos latentes. (Libre)
-

94.1.2. Geometría y fundamentos (nivel más teórico)

- Gilbert Strang — *Linear Algebra and Learning from Data*
<https://math.mit.edu/~gs/learningfromdata/>
→ Conexión entre álgebra lineal, PCA y datos. (Parcialmente libre)
-

94.1.3. Visualización y alta dimensión

- Claus O. Wilke — *Fundamentals of Data Visualization*
<https://clauswilke.com/dataviz/>
→ Principios de visualización (muy útil para EDA). (Libre)
- Distill.pub (artículos interactivos)
<https://distill.pub/>
→ Explicaciones visuales de t-SNE, embeddings y ML. (Libre)

95. Análisis Exploratorio de Datos Multivariado: Ejemplos prácticos

96. Ejemplo práctico: Implementación paso a paso

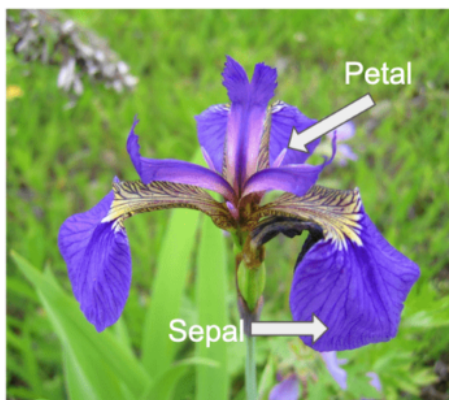
Dataset: Iris

Para este ejemplo usaremos Scikit-learn, mediante el clásico dataset *Iris* (flores de lirio de este lado del charco), que contiene mediciones de flores de tres especies (*setosa*, *versicolor* y *virginica*). Cada observación está descrita por cuatro variables numéricas:

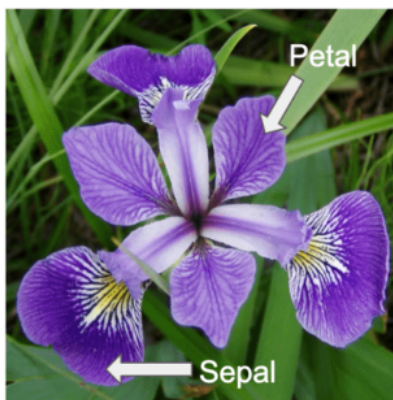
- longitud del sépalo
- ancho del sépalo
- longitud del pétalo
- ancho del pétalo

Estas variables capturan características morfológicas de las flores y presentan correlaciones entre sí, lo que lo convierte en un ejemplo ideal para ilustrar cómo PCA identifica estructura y reduce dimensionalidad.

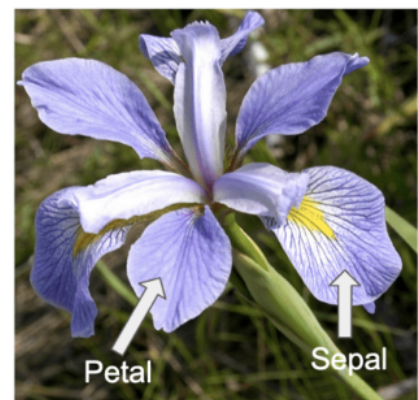
Iris setosa



Iris versicolor



Iris virginica



```
iris = load_iris()
X = iris.data
y = iris.target
nombres = iris.target_names
feature_names = ['Sépalo largo', 'Sépalo ancho', 'Pétalo largo', 'Pétalo ancho']

df_iris = pd.DataFrame(X, columns=feature_names)
df_iris['Especie'] = [nombres[i] for i in y]
df_iris.describe().round(3)
```

	Sépalo largo	Sépalo ancho	Pétalo largo	Pétalo ancho
count	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.300	2.000	1.000	0.100
25 %	5.100	2.800	1.600	0.300
50 %	5.800	3.000	4.350	1.300
75 %	6.400	3.300	5.100	1.800
max	7.900	4.400	6.900	2.500

Paso 1: Estandarización

La estandarización es **obligatoria** cuando las variables tienen escalas distintas. Sin ella, las variables con mayor varianza absoluta dominarían el PCA independientemente de su relevancia.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Después de estandarizar: $\bar{z}_j = 0$ y $s_{z_j} = 1$ para toda variable j .

```
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# Verificar
df_check = pd.DataFrame({
    'Variable': feature_names,
    'Media original': X.mean(axis=0).round(3),
    'Desv original': X.std(axis=0).round(3),
    'Media estand.': X_std.mean(axis=0).round(10),
    'Desv estand.': X_std.std(axis=0).round(3)
})
df_check
```

	Variable	Media original	Desv original	Media estand.	Desv estand.
0	Sépalo largo	5.843	0.825	-0.0	1.0
1	Sépalo ancho	3.057	0.434	-0.0	1.0
2	Pétalo largo	3.758	1.759	-0.0	1.0
3	Pétalo ancho	1.199	0.760	-0.0	1.0

Paso 2: Matriz de covarianza

```

S = np.cov(X_std.T)

fig = go.Figure(data=go.Heatmap(
    z=S,
    x=feature_names,
    y=feature_names,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(S, 3),
    texttemplate='%{text}',
    textfont=dict(size=13),
    hovertemplate='%{y}  %{x}<br>cov =%{z:.3f}<extra></extra>',
    colorbar=dict(title='Covarianza')
))
fig.update_layout(
    title="Matriz de covarianza - datos estandarizados (= matriz de correlación)",
    height=420
)
fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

i Nota

Cuando los datos están estandarizados, la matriz de covarianza es idéntica a la **matriz de correlación de Pearson**. Por eso PCA sobre datos estandarizados equivale a hacer PCA sobre correlaciones.

Paso 3: Eigendescomposición vía SVD

```

pca = PCA()
scores = pca.fit_transform(X_std)

eigenvalues = pca.explained_variance_
pve = pca.explained_variance_ratio_ * 100
pve_acum = np.cumsum(pve)

df_pca = pd.DataFrame({
    'Componente': [f'PC{i+1}' for i in range(len(eigenvalues))],
    'Eigenvalue ( )': eigenvalues.round(4),
    '% Varianza': pve.round(2),
    '% Acumulado': pve_acum.round(2)
})
df_pca

```

	Componente	Eigenvalue ()	% Varianza	% Acumulado
0	PC1	2.9381	72.96	72.96
1	PC2	0.9202	22.85	95.81
2	PC3	0.1477	3.67	99.48
3	PC4	0.0209	0.52	100.00

Paso 4: Scree plot

```
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=df_pca['Componente'], y=df_pca['% Varianza'],
    name='% Varianza',
    text=[f"{v:.1f}%" for v in pve],
    textposition='outside'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=df_pca['Componente'], y=pve_acum,
    mode='lines+markers',
    line=dict(width=2.5),
    marker=dict(size=9),
    name='% Acumulado'
), secondary_y=True)

fig.add_hline(y=80, line_dash="dash", line_color="gray",
              annotation_text="80%", secondary_y=True)

fig.update_yaxes(title_text="% Varianza por componente", secondary_y=False)
fig.update_yaxes(title_text="% Varianza acumulada", range=[0, 110], secondary_y=True)
fig.update_layout(
    title="Scree Plot - Regla de Kaiser: retener componentes con  $\lambda > 1$ ",
    height=380
)
fig.show()
```

Unable to display output for mime type(s): text/html

💡 Tip

Criterios para elegir k :

- **Regla de Kaiser:** retener componentes con $\lambda_k > 1$ (solo aplica con datos estandarizados)
- **Varianza acumulada:** retener los primeros k que superen el 80 % (o 90 %)
- **Scree plot:** buscar el “codo” donde la pendiente cae abruptamente
- **Interpretabilidad de dominio:** ¿los componentes tienen sentido en el problema?

Paso 5: Biplot

El **biplot** superpone los scores de las observaciones y los vectores de los loadings en el mismo espacio reducido, permitiendo interpretar simultáneamente la estructura de las observaciones y la contribución de las variables.

```
loadings = pca.components_ # shape: (n_components, n_features)
scores_df = pd.DataFrame(scores[:, :2], columns=['PC1', 'PC2'])
scores_df['Especie'] = [nombres[i] for i in y]

fig = go.Figure()

# Scores
for especie in nombres:
    mask = scores_df['Especie'] == especie
    fig.add_trace(go.Scatter(
        x=scores_df.loc[mask, 'PC1'],
        y=scores_df.loc[mask, 'PC2'],
        mode='markers',
        marker=dict(size=7, opacity=0.7),
        name=especie
    ))

# Vectores de loadings escalados
escala = 3.0
for i, feat in enumerate(feature_names):
    fig.add_annotation(
        x=loadings[0, i] * escala,
        y=loadings[1, i] * escala,
        ax=0, ay=0,
        arrowhead=3, arrowwidth=2,
        xref='x', yref='y', axref='x', ayref='y',
        showarrow=True
    )
    fig.add_annotation(
        x=loadings[0, i] * escala * 1.15,
        y=loadings[1, i] * escala * 1.15,
        text=f"<b>{feat}</b>",
        showarrow=False,
        font=dict(size=11)
    )

fig.update_xaxes(title_text=f"PC1 ({pve[0]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text=f"PC2 ({pve[1]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title="Biplot: scores + loadings - Iris",
    height=480,
```

```
    legend=dict(title='Especie')
)
fig.show()
```

Unable to display output for mime type(s): text/html

Interpretación del biplot:

- Las flechas (loadings) representan los **vectores de las variables proyectados** sobre el nuevo sistema de coordenadas definido por los componentes principales. Es decir, cada flecha muestra cómo una variable original se expresa en la nueva base.
- Flechas largas → la variable tiene **gran proyección (norma)** sobre ese plano, por lo que contribuye significativamente a la variabilidad capturada por los componentes.
- Flechas apuntando en la misma dirección → sus vectores forman un **ángulo pequeño**, lo que indica alta correlación (producto punto positivo grande).
Flechas opuestas → correlación negativa.
Flechas ortogonales → variables no correlacionadas.
- La posición de los puntos corresponde a la **proyección de las observaciones** en el subespacio generado por los primeros componentes.
La separación entre grupos refleja qué tan bien este subespacio (de menor dimensión) preserva la estructura original de los datos.

En conjunto, el biplot muestra simultáneamente: - un **cambio de base** (álgebra lineal)

- y una **proyección de los datos** en un subespacio de menor dimensión

Paso 6: Loadings detallados

```
df_loadings = pd.DataFrame(
    loadings[:2].T,
    index=feature_names,
    columns=['PC1', 'PC2']
).round(4)

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=[
        "<b>Componente Principal 1 (PC1)</b>",
        "<b>Componente Principal 2 (PC2)</b>"
    ]
)

# PC1
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[0],
```



```

        text=[f"{v:.3f}" for v in loadings[0]],
        textposition='outside'
    ), row=1, col=1)

# PC2
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[1],
    text=[f"{v:.3f}" for v in loadings[1]],
    textposition='outside'
), row=1, col=2)

# Ejes
for c in [1, 2]:
    fig.update_yaxes(
        range=[-1.1, 1.1],
        zeroline=True,
        zerolinecolor='lightgray',
        row=1, col=c
    )
    fig.update_xaxes(
        tickangle=-25,
        row=1, col=c
    )

# Hacer más visibles los títulos de subplots
for ann in fig['layout']['annotations']:
    ann['font'] = dict(size=14)

# Nota interpretativa
fig.update_layout(
    showlegend=False,
    height=420,
    annotations=list(fig.layout.annotations) + [
        dict(
            x=0.5, y=-0.2,
            xref='paper', yref='paper',
            text="PC1  tamaño general | PC2  contraste estructural del sépalos",
            showarrow=False,
            font=dict(size=12, color='gray'),
            xanchor='center'
        )
    ]
)

fig.show()

```

Unable to display output for mime type(s): text/html

Interpretación de los loadings:

Cada componente principal es una **combinación lineal de las variables originales**. Es decir:

$$PC_k = w_{1k}x_1 + w_{2k}x_2 + \cdots + w_{pk}x_p$$

donde los *loadings* w_{jk} indican el peso de cada variable en ese componente.

- **PC1** combina fuertemente *pétalo largo*, *pétalo ancho* y *sépalo largo* (coeficientes positivos similares), por lo que representa un eje de **tamaño general de la flor**. El *sépalo ancho* contribuye negativamente, ajustando ligeramente esa dirección.
- **PC2** está dominado por *sépalo ancho*, con poca contribución del resto, por lo que captura un **contraste específico en la forma del sépalo**.

En términos geométricos, cada componente define una **dirección en el espacio original**, y los loadings son las **coordenadas de esa dirección** en la base de variables originales.

97. Ejemplo avanzado: Dataset con ruido

97.1. El problema: una partícula en movimiento que no podemos ver directamente

Imagina que una partícula traza una trayectoria helicoidal en el espacio tridimensional. Nosotros no podemos observarla directamente — solo tenemos acceso a **sensores**, y cada sensor introduce su propio ruido de medición.

El desafío central es: dado un conjunto de lecturas ruidosas de múltiples sensores, ¿podemos recuperar la trayectoria original?

PCA responde exactamente a esa pregunta. Pero antes de aplicarlo, construyamos el experimento desde cero.

97.2. Paso 1 — La trayectoria real y el ruido de medición

Generamos 500 puntos sobre una hélice paramétrica:

$$x(t) = t, \quad y(t) = \cos(t), \quad z(t) = \sin(t), \quad t \in [-10, 10]$$

Luego añadimos ruido gaussiano independiente en y y z para simular la imprecisión de los instrumentos de medida.

```
n_points = 500
t = np.linspace(-10, 10, n_points)

# Trayectoria limpia
x_clean = t
y_clean = np.cos(t)
z_clean = np.sin(t)

# Observaciones ruidosas (lo que miden los sensores)
sigma = 0.5
y_noise = y_clean + np.random.normal(0, sigma, n_points)
z_noise = z_clean + np.random.normal(0, sigma, n_points)

points = np.column_stack([x_clean, y_noise, z_noise])

# Visualización
```

```

fig = go.Figure()

fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers',
    marker=dict(size=3, color='steelblue', opacity=0.5),
    name='Mediciones ruidosas'
))

fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines',
    line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

fig.update_layout(
    title='Trayectoria helicoidal: señal real vs. mediciones ruidosas',
    scene=dict(
        xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
        yaxis=dict(range=[-5, 5]),
        xaxis=dict(range=[-11, 11])
    ),
    legend=dict(x=0.01, y=0.99),
    height=520
)
fig.show()

```

Unable to display output for mime type(s): text/html

Los puntos azules son lo que los sensores registran. La línea roja es la realidad que queremos recuperar. A simple vista ya es difícil distinguir la estructura subyacente del ruido.

97.3. Paso 2 — Tres sensores, tres perspectivas distintas

En un experimento real, distintos sensores observan el mismo fenómeno desde **ángulos diferentes**. Cada sensor proyecta la nube de puntos sobre su propio plano de referencia y mide coordenadas locales (u, v) en ese plano — más, inevitablemente, su propio ruido adicional.

Simulamos tres sensores con planos de proyección distintos.

```

# Funciones auxiliares

def projection(points, plane_point, plane_normal):
    """Proyecta cada punto sobre un plano definido por un punto y su normal."""

```

```

n = plane_normal / np.linalg.norm(plane_normal)
d = points - plane_point
dist = d.dot(n)
return points - np.outer(dist, n)

def plane_axes(plane_normal):
    """Devuelve dos vectores ortogonales que forman la base del plano."""
    n = plane_normal / np.linalg.norm(plane_normal)
    # Vector arbitrario no paralelo a n
    arb = np.array([1, 0, 0]) if abs(n[0]) < 0.9 else np.array([0, 1, 0])
    u = np.cross(n, arb)
    u /= np.linalg.norm(u)
    v = np.cross(n, u)
    v /= np.linalg.norm(v)
    return u, v

def sensor_reading(points, plane_point, plane_normal, noise_std=0.5):
    """
    Proyecta puntos sobre el plano del sensor y devuelve:
    - coordenadas 2D locales (s, t) con ruido adicional
    - la proyección 3D (para visualización)
    - los vectores de la base del plano (u, v)
    """
    proj_3d = projection(points, plane_point, plane_normal)
    u, v = plane_axes(plane_normal)

    proj_vectors = proj_3d - plane_point
    s = proj_vectors.dot(u) + np.random.normal(0, noise_std, len(points))
    t = proj_vectors.dot(v) + np.random.normal(0, noise_std, len(points))

    return s, t, proj_3d, u, v

def plane_mesh(plane_point, plane_normal, size=12, res=20):
    """Genera una malla rectangular para visualizar el plano del sensor."""
    u, v = plane_axes(plane_normal)
    lin = np.linspace(-size, size, res)
    uu, vv = np.meshgrid(lin, lin)
    X = plane_point[0] + uu * u[0] + vv * v[0]
    Y = plane_point[1] + uu * u[1] + vv * v[1]
    Z = plane_point[2] + uu * u[2] + vv * v[2]
    return X, Y, Z

# Definición de los tres sensores
sensors = [
    dict(point=np.array([5., -6., 0.]), normal=np.array([1., -1., 0.]),
        color='royalblue', name='Sensor A'),

```

```

dict(point=np.array([10., 0., 0.]), normal=np.array([10., 0., 0.1]),
    color='seagreen', name='Sensor B'),
dict(point=np.array([3., 3., 3.]), normal=np.array([1., 1., 1.]),
    color='darkorange', name='Sensor C'),
]

readings = []
for s in sensors:
    sc, tc, proj3d, u, v = sensor_reading(
        points, s['point'], s['normal'], noise_std=0.5
    )
    readings.append({'s': sc, 't': tc, 'proj3d': proj3d,
                    'u': u, 'v': v, **s})

```

97.3.1. Perspectiva 3D: sensores y sus proyecciones

```

fig = go.Figure()

# Trayectoria real
fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines', line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

# Puntos originales
fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers', marker=dict(size=2, color='lightgray', opacity=0.4),
    name='Mediciones originales'
))

for r in readings:
    # Plano del sensor
    px_m, py_m, pz_m = plane_mesh(r['point'], r['normal'], size=10)
    fig.add_trace(go.Surface(
        x=px_m, y=py_m, z=pz_m,
        colorscale=[[0, r['color']], [1, r['color']]],
        opacity=0.15, showscale=False,
        name=f"Plano {r['name']}", showlegend=False
    ))
    # Proyección sobre el plano
    fig.add_trace(go.Scatter3d(
        x=r['proj3d'][:, 0], y=r['proj3d'][:, 1], z=r['proj3d'][:, 2],
        mode='markers', marker=dict(size=2.5, color=r['color'], opacity=0.7),
        name=r['name']
    ))

```

```
fig.update_layout(
    title='Los tres sensores proyectan la nube sobre sus planos respectivos',
    scene=dict(xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
               xaxis=dict(range=[-12, 12]),
               yaxis=dict(range=[-8, 8])),
    legend=dict(x=0.01, y=0.99),
    height=560
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor “aplana” la nube sobre su propio plano. Ninguno captura la hélice perfectamente, pero cada uno aporta información parcial y complementaria.

97.3.2. Lo que registra cada sensor: coordenadas 2D

```
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=[r['name'] for r in readings],
    shared_yaxes=False
)

for i, r in enumerate(readings, 1):
    fig.add_trace(go.Scatter(
        x=r['s'], y=r['t'],
        mode='markers',
        marker=dict(size=3, color=r['color'], opacity=0.6),
        name=r['name'],
        showlegend=False
    ), row=1, col=i)
    fig.update_xaxes(title_text='u (eje 1 del plano)', row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')
    fig.update_yaxes(title_text='v (eje 2 del plano)' if i == 1 else '',
                     row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')

fig.update_layout(
    title='Lecturas brutas de cada sensor (coordenadas 2D locales)',
    height=400
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor produce una imagen 2D distorsionada de la misma hélice. El sensor A la ve casi como una elipse, el B como una banda horizontal, el C como una nube más compacta. Ninguno por sí solo revela la hélice original.

97.4. Paso 3 — Construir la matriz de datos multi-sensor

Apilamos las seis coordenadas (2 por cada sensor) para formar una **matriz de datos** $X \in \mathbb{R}^{500 \times 6}$. Cada fila es una muestra; cada columna es una variable medida por algún sensor.

Esta es exactamente la estructura con la que trabaja PCA: muchas variables correlacionadas que comparten una estructura latente de baja dimensión.

```
# Matriz de datos: 500 muestras × 6 variables (2 por sensor)
data_matrix = np.column_stack([r['s'] for r in readings] +
                               [r['t'] for r in readings])
# Reordenamos: s_A, t_A, s_B, t_B, s_C, t_C
data_matrix = np.column_stack([
    readings[0]['s'], readings[0]['t'],
    readings[1]['s'], readings[1]['t'],
    readings[2]['s'], readings[2]['t'],
])

col_labels = ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']

print(f"Dimensión de la matriz de datos: {data_matrix.shape}")
print(f"Variables: {col_labels}")
```

```
Dimensión de la matriz de datos: (500, 6)
Variables: ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']
```

97.4.1. Matriz de correlación entre sensores

```
corr = np.corrcoef(data_matrix.T)

fig = go.Figure(data=go.Heatmap(
    z=corr,
    x=col_labels, y=col_labels,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(corr, 2),
    texttemplate='%{text}',
    textfont=dict(size=11),
    colorbar=dict(title='Correlación')
))
```



```
fig.update_layout(
    title='Correlación entre las 6 variables medidas por los sensores',
    height=420
)
fig.show()
```

Unable to display output for mime type(s): text/html

La matriz de correlación revela la redundancia. Variables del mismo sensor están fuertemente correlacionadas entre sí, y también con variables de otros sensores — evidencia de que todas miden el mismo fenómeno subyacente. PCA explotará precisamente esa redundancia.

97.5. Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?

PCA descompone la varianza total en **componentes ortogonales ordenados por importancia**. Si el fenómeno real vive en pocas dimensiones, la mayor parte de la varianza se concentrará en los primeros componentes.

```
pca_full = PCA(n_components=6)
components_full = pca_full.fit_transform(data_matrix)

explained = pca_full.explained_variance_ratio_
cumulative = np.cumsum(explained)

# Scree plot
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=[f'PC{i+1}' for i in range(6)],
    y=explained * 100,
    name='% Varianza individual',
    text=[f'{v*100:.1f}%' for v in explained],
    textposition='outside',
    marker_color='steelblue'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=[f'PC{i+1}' for i in range(6)],
    y=cumulative * 100,
    mode='lines+markers',
    name='% Acumulado',
    line=dict(color='crimson', width=2.5),
    marker=dict(size=9)
), secondary_y=True)
```

```

fig.add_hline(y=90, line_dash='dash', line_color='gray',
              annotation_text='90%', secondary_y=True)

fig.update_yaxes(title_text='% Varianza por componente', secondary_y=False)
fig.update_yaxes(title_text='% Varianza acumulada', range=[0, 110],
                 secondary_y=True)
fig.update_layout(
    title='Scree plot: ¿cuántos componentes capturan la señal?',
    height=400
)
fig.show()

print("Varianza explicada por componente:")
for i, (v, c) in enumerate(zip(explained, cumulative)):
    print(f"  PC{i+1}: {v*100:5.1f}%    acumulado: {c*100:5.1f}%")

```

Unable to display output for mime type(s): text/html

Varianza explicada por componente:

PC1:	89.4%	acumulado:	89.4%
PC2:	4.7%	acumulado:	94.1%
PC3:	4.1%	acumulado:	98.2%
PC4:	0.7%	acumulado:	98.9%
PC5:	0.5%	acumulado:	99.5%
PC6:	0.5%	acumulado:	100.0%

Los primeros **2–3 componentes** concentran la gran mayoría de la varianza. Esto confirma lo que esperábamos: el sistema tiene **2 grados de libertad reales** (la hélice es una curva en 3D, parametrizada por un solo parámetro t), y el resto es ruido.

97.6. Paso 5 — Proyección en 2D: la hélice emerge del ruido

Con solo los dos primeros componentes principales recuperamos la estructura esencial del movimiento.

```

pca_2d = PCA(n_components=2)
proj_2d = pca_2d.fit_transform(data_matrix)

# Línea de referencia: sin(t) proyectada para comparación
t_ref = np.linspace(-12, 12, 300)
ref_x = t_ref / t_ref.max() * proj_2d[:, 0].max()

fig = go.Figure()

fig.add_trace(go.Scatter(

```

```

x=proj_2d[:, 0], y=proj_2d[:, 1],
mode='markers',
marker=dict(size=4, color='steelblue', opacity=0.6),
name='Muestras (PC1 vs PC2)'
))

# Colorear por el parámetro t para mostrar la continuidad
fig.add_trace(go.Scatter(
    x=proj_2d[:, 0], y=proj_2d[:, 1],
    mode='markers',
    marker=dict(
        size=4, opacity=0.8,
        color=t,
        colorscale='Viridis',
        colorbar=dict(title='t (tiempo)', len=0.6)
    ),
    name='Coloreado por t'
))

fig.update_xaxes(title_text='Componente Principal 1',
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text='Componente Principal 2',
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title=f'PC1 vs PC2 - {pca_2d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    height=480
)
fig.show()

```

Unable to display output for mime type(s): text/html

Al colorear los puntos por el parámetro t (el “tiempo” de la trayectoria), vemos que el orden temporal se preserva perfectamente: PCA ha recuperado la parametrización natural de la hélice a partir de seis variables ruidosas.

97.7. Paso 6 — Proyección en 3D: confirmando la estructura helicoidal

Agregando el tercer componente recuperamos la geometría completa.

```

pca_3d = PCA(n_components=3)
proj_3d = pca_3d.fit_transform(data_matrix)

fig = go.Figure()

```

```

# Muestras coloreadas por t
fig.add_trace(go.Scatter3d(
    x=proj_3d[:, 0], y=proj_3d[:, 1], z=proj_3d[:, 2],
    mode='markers',
    marker=dict(
        size=3, opacity=0.7,
        color=t,
        colorscale='Viridis',
        colorbar=dict(title='t', len=0.5)
    ),
    name='Muestras'
))

# Trayectoria original reescalada para superposición visual
scale = proj_3d[:, 0].std() / x_clean.std()
fig.add_trace(go.Scatter3d(
    x=x_clean * scale,
    y=y_clean * proj_3d[:, 1].std(),
    z=z_clean * proj_3d[:, 2].std(),
    mode='lines',
    line=dict(color='crimson', width=4),
    name='Trayectoria original (reescalada)'
))

fig.update_layout(
    title=f'Proyección 3D - {pca_3d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    scene=dict(
        xaxis_title='PC 1',
        yaxis_title='PC 2',
        zaxis_title='PC 3',
        xaxis=dict(range=[-15, 15]),
        yaxis=dict(range=[-7, 7]),
        zaxis=dict(range=[-7, 7])
    ),
    legend=dict(x=0.01, y=0.99),
    height=560
)
fig.show()

```

Unable to display output for mime type(s): text/html

La nube de puntos reconstruida en el espacio PCA sigue fielmente la hélice original. PCA ha separado la señal del ruido sin conocer a priori la geometría de la trayectoria.

i La idea central

PCA no sabe nada sobre hélices ni sobre física. Solo busca las direcciones de **máxima varianza** en los datos. El hecho de que esas direcciones coincidan con la geometría real de la trayectoria es una consecuencia de que **la señal real tiene más varianza que el ruido**.

Cuando $\text{SNR} = \sigma_{\text{señal}}/\sigma_{\text{ruido}} \gg 1$, los primeros componentes principales capturan la señal; los últimos, el ruido.

 Limitación importante

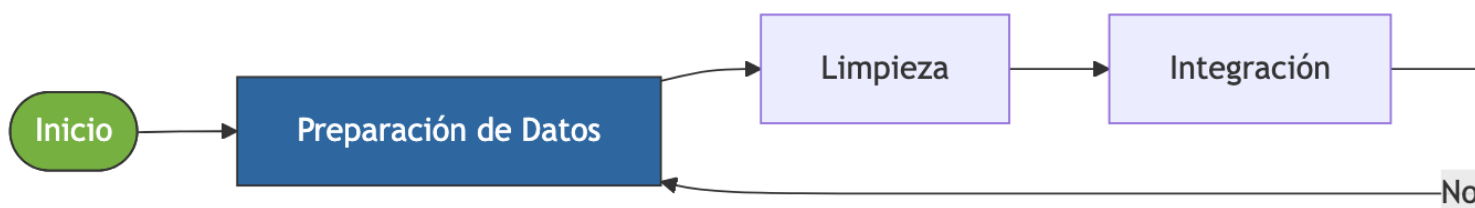
PCA funciona bien aquí porque la hélice es una curva **linealizable** a través de sus coordenadas en el espacio de observaciones. Para estructuras genuinamente no lineales (manifolds curvos), se necesitan técnicas como **UMAP** o **t-SNE**.

Parte XX.

Preprocesamiento

98. Introducción a Preprocesamiento de Datos

En cualquier proceso de minería de datos o analítica avanzada, la calidad de los resultados depende directamente de la calidad de los datos de entrada. En la práctica, los datos crudos rara vez se encuentran en condiciones adecuadas para su análisis: pueden contener valores faltantes, ruido, inconsistencias, formatos heterogéneos o incluso redundancias que dificultan la extracción de conocimiento.



La **preparación de datos** constituye una de las etapas más críticas dentro del flujo de un proyecto analítico. Antes de aplicar cualquier algoritmo, es necesario evaluar si los datos presentan problemas y, en caso afirmativo, ejecutar una serie de transformaciones que permitan obtener un conjunto de datos consistente, interpretable y útil.

Este proceso no debe entenderse como una secuencia de pasos aislados, sino como componentes interdependientes en un flujo iterativo que transforma datos crudos en conocimiento accionable.

Siguiendo las mejores prácticas de la minería de datos moderna, el orden lógico de estas tareas es:

- **Integración de datos:** Permite combinar múltiples fuentes (bases de datos, archivos, APIs) en una estructura única y coherente.
- **Limpieza de datos:** Donde se abordan problemas como valores faltantes, ruido, errores de captura e inconsistencias sobre el conjunto ya integrado.
- **Transformación de datos:** Orientada a adaptar los datos al formato requerido por los modelos (por ejemplo, mediante normalización o discretización).
- **Reducción de datos:** Cuyo objetivo es disminuir la complejidad del conjunto (filas o columnas) sin perder la información relevante para el modelo final.

Es importante destacar que la preparación de datos no es un proceso meramente mecánico, sino una **fase analítica fundamental**. Las decisiones tomadas en esta etapa —como el tratamiento de valores faltantes, la selección de variables o la reducción de dimensionalidad— influyen directamente en:

- La validez estadística

- La interpretabilidad
- El desempeño de los modelos

El objetivo es proporcionar un marco que permita transformar datos crudos en estructuras consistentes, robustas y útiles para la generación de conocimiento y la toma de decisiones.

99. Conversión y Transformación de Datos

La conversión de datos es una etapa fundamental en la preparación de datos, cuyo objetivo es transformar las variables a formatos adecuados para su análisis y modelado.

En muchos casos, los algoritmos de minería de datos requieren variables numéricas y formatos consistentes, por lo que es necesario aplicar distintas transformaciones.

99.0.1. Imputación de valores faltantes

Consiste en reemplazar valores ausentes para evitar la pérdida de información. Algunas estrategias incluyen:

- Eliminación de registros incompletos (*listwise*)
- Imputación con media o mediana
- Imputación por grupo
- Estimación mediante modelos

99.0.2. Estandarización de formatos

Los datos pueden presentar inconsistencias en su representación. Por ejemplo:

- Fechas en distintos formatos
- Unidades de medida distintas
- Codificaciones inconsistentes

La estandarización permite garantizar coherencia y comparabilidad.

99.0.3. Conversión de variables categóricas

Muchas variables son de tipo nominal o categórico y deben convertirse a formato numérico:

- Variables binarias:
Ejemplo: género = {M, F} $\rightarrow$ {1, 0}
- Variables ordinales:
Se asignan valores respetando el orden
Ejemplo: {bajo, medio, alto} $\rightarrow$ {1, 2, 3}
- Variables nominales (one-hot encoding):
Cada categoría se convierte en una variable binaria

$$\text{Color} \in \{\text{rojo}, \text{azul}\} \Rightarrow (1, 0), (0, 1)$$

Nota: Los identificadores únicos no deben utilizarse como variables predictoras.

99.0.4. Conclusión

La conversión de datos permite transformar información heterogénea en una representación estructurada, facilitando su uso en algoritmos de análisis y aprendizaje automático.

100. Reducción de ruido

El ruido en los datos corresponde a variaciones aleatorias, errores de medición o inconsistencias en el registro, que pueden afectar negativamente el análisis y los modelos.

100.0.1. Detección de ruido

Algunas estrategias para identificar ruido incluyen:

- Análisis de rangos válidos
- Detección de outliers
- Visualización (histogramas, boxplots)
- Comparación con reglas del dominio

100.0.2. Técnicas de reducción de ruido

100.0.2.1. Binning

Consiste en agrupar datos en intervalos (bins) y reemplazar los valores por estadísticas representativas.

El ancho del intervalo se define como:

$$W = \frac{B - A}{N}$$

donde: - A = valor mínimo

- B = valor máximo

- N = número de bins

Los valores pueden reemplazarse por:

- La media del bin
- El valor extremo más cercano

100.0.2.2. Regresión lineal

La regresión lineal modela la relación entre una variable predictora X y una variable respuesta Y , ajustando una recta que captura la tendencia general:

$$y = a + bx$$

donde: - a es el intercepto

- b es la pendiente

El modelo se obtiene minimizando la suma de los errores cuadráticos:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

100.0.3. Interpretación

La regresión permite reemplazar valores ruidosos por estimaciones más estables sobre la recta ajustada, reduciendo la variabilidad aleatoria sin perder la tendencia global.

100.0.4. Conclusión

La reducción de ruido mejora la calidad de los datos al eliminar variaciones irrelevantes, permitiendo que los modelos capturen patrones reales en lugar de artefactos aleatorios.

101. Procesamiento de Datos Faltantes

101.1. Objetivos de la clase

- Entender por qué los datos faltantes importan.
- Diferenciar **MCAR**, **MAR** y **MNAR**.
- Ver cómo cambian sesgo, varianza y distribución.
- Comparar estrategias:
 - `dropna`
 - imputación por media
 - KNN / imputación basada en modelo
- Revisar aplicaciones en BigMart.

102. Introducción

102.1. Idea central

No basta con saber **cuántos** datos faltan.
Hay que pensar **por qué** faltan.

Porque el mecanismo de ausencia determina:

- si nuestras **estimaciones son sesgadas**
 - si la **imputación es válida**
 - si el modelo aprende de una **muestra representativa**
-

102.2. Motivación

Los datos faltantes pueden producir:

- pérdida de potencia estadística
- sesgo en los estimadores
- errores estándar incorrectos
- intervalos de confianza engañosos
- peor desempeño predictivo

102.3. Ejemplo visual:

Pérdida más probable en valores altos

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Media verdadera: 12.043, promedio verdadero: 12.043
Media observada: 11.119, promedio observado: 11.119
Proporción missing: 28.650%

102.4. Preguntas guía

Antes de emplear cualquier técnica, conviene preguntar:

1. ¿Cuál es el **porcentaje** de faltantes?
 2. ¿Cuál es el **patrón** de ausencia?
 3. ¿Cual es el **mecanismo** de ausencia?
 4. ¿Modelamos una **variable objetivo** o un **predictor**?¹
-

102.5. Tipos estructurales de datos faltantes

102.5.0.1. Unit nonresponse

Falta toda la observación

X1	X2	Y
?	?	?

Ejemplo:

una persona no responde ninguna parte de una encuesta.

102.5.0.2. Item nonresponse

Faltan solo algunas variables

X1	X2	Y
	?	

Ejemplo:

un registro sí existe, pero una de sus variables no fue capturada.

¹Un modelo puede aprender con predictores imperfectos, pero no puede aprender sin conocer la respuesta correcta.

103. Mecanismos de ausencia

103.1. MCAR — Missing Completely At Random

La probabilidad de missing **no** depende de ninguna variable.

$$f(R | Y, X, \theta) = f(R | \theta)$$

Se puede entender como que “Se perdieron filas al azar.”

Ejemplo: falla aleatoria en el sistema de captura de datos.

103.2. Ejemplo sintético: MCAR en acción

Unable to display output for mime type(s): text/html

```
Proporción missing: 20.57%
Media verdadera:    12.0425
Media observada:    12.0411
Desv. estándar verdadera: 2.1935
Desv. estándar observada: 2.1811
```

103.2.1. Interpretación

- la ausencia es independiente de los datos observados
- también es independiente de los datos no observados
- es el caso más simple desde el punto de vista estadístico

103.2.2. Consecuencia práctica

Si ignoramos los faltantes bajo MCAR:

- no necesariamente introducimos sesgo
 - pero sí perdemos información y potencia estadística
-

103.3. MAR — Missing At Random

La probabilidad de missing depende **solo de variables observadas**.

$$f(R | Y, X, \theta) = f(R | Y_{obs}, X, \theta)$$

Se entiende como “Faltan más datos en ciertos grupos observables.”

Ejemplo: `Item_Weight` falta más en ciertos `Outlet_Type` que sí conocemos.

103.4. Ejemplo sintético: MAR en acción

Unable to display output for mime type(s): text/html

Proporción missing por grupo:

	Outlet_Type	missing
0	Grocery Store	0.763
1	Mini Market	0.173
2	Supermarket	0.031

Media verdadera: 12.177 Media observada: 13.693

103.5. Efecto de MAR sobre la muestra observada

Unable to display output for mime type(s): text/html

103.5.1. Interpretación

- la ausencia puede explicarse con variables que sí vemos
- condicionado en lo observado, el missing ya no depende del valor faltante
- es el supuesto más común en análisis aplicados

103.5.2. Consecuencia práctica

Si ignoramos los faltantes bajo MAR:

- podemos introducir sesgo
- pero una imputación que use variables observadas puede corregirlo parcialmente o bien

103.6. MNAR — Missing Not At Random

La probabilidad de missing depende **del propio valor faltante**.

$$f(R | Y, X, \theta) = f(R | Y_{obs}, Y_{mis}, X, \theta)$$

Es decir “Los valores faltan precisamente por su valor.”

Ejemplo: `Item_Weight` falta más en productos muy pesados, justamente por su peso.

103.7. Ejemplo sintético: MNAR en acción

Unable to display output for mime type(s): text/html

Media verdadera: 12.012
 Media observada: 10.716
 Proporción missing: 62.44%

103.7.1. Interpretación

- el mecanismo de ausencia depende de información no observada
- el dato faltante es parte de la causa de su propia ausencia
- es el caso más difícil de manejar

103.7.2. Consecuencia práctica

Si ignoramos los faltantes bajo MNAR:

- aparece sesgo estructural
- ni una buena imputación estándar garantiza corregir completamente el problema

	Region	Promo	Item_Weight	Item_MRP
0	Centro	0	12.107466	65.073799
1	Norte	0	10.533381	71.460484
2	Sur	0	13.092823	68.802480
3	Centro	0	9.452396	57.931334
4	Centro	0	16.762764	85.996319

	Completo	Después_dropna
Region		
Centro	12.046	12.063
Norte	12.014	11.998
Sur	12.005	11.982

103.8. Clasificación Global del Mecanismo de Datos Faltantes

En un conjunto de datos, distintas variables pueden presentar mecanismos de datos faltantes diferentes, como MCAR, MAR o MNAR. Sin embargo, la clasificación del mecanismo de datos faltantes debe realizarse de manera global, considerando el comportamiento conjunto del vector de indicadores de missingness.

Formalmente, el mecanismo global se define a partir de la distribución:

$$P(R \mid X_{\text{obs}}, X_{\text{mis}})$$

donde R indica la presencia de datos faltantes.

Aunque algunas variables puedan cumplir con los supuestos de MCAR o MAR, la presencia de al menos una variable con mecanismo MNAR implica que la probabilidad de datos faltantes depende de valores no observados, es decir:

$$P(R \mid X_{\text{obs}}, X_{\text{mis}}) \neq P(R \mid X_{\text{obs}})$$

En consecuencia, el conjunto de datos completo debe clasificarse como **MNAR**.

103.8.1. Ejemplo ilustrativo

Considérese una base de datos con tres variables:

- X_1 : **Edad** presenta un mecanismo MCAR (faltantes completamente aleatorios).
- X_2 : **Ingreso** presenta un mecanismo MAR (los faltantes dependen de la educación, que sí está observada).
- X_3 : **Nivel de deuda** presenta un mecanismo MNAR (los individuos con mayor deuda tienden a no reportarla).

En este caso:

- X_1 no introduce dependencia.
- X_2 depende solo de variables observadas.
- Pero X_3 introduce dependencia en valores no observados (X_{mis}).

Por lo tanto, el mecanismo conjunto satisface:

$$P(R \mid X_{\text{obs}}, X_{\text{mis}}) \neq P(R \mid X_{\text{obs}})$$

y el conjunto de datos debe clasificarse globalmente como **MNAR**.

Conclusión: La presencia de cualquier componente MNAR domina la clasificación global del conjunto de datos.

104. Estrategias para manejar datos faltantes

104.1. De la teoría a la práctica

Una vez que entendemos **cómo** y **por qué** aparecen los datos faltantes, la siguiente pregunta natural es:

¿Qué estrategias existen para tratarlos?

No todas las técnicas son igual de adecuadas: su validez depende del **mecanismo de ausencia**, de la **cantidad de missing values** y del papel que juega la variable en el análisis.

104.2. Estrategias prácticas

- **Eliminación Listwise (Casos completos)**
eliminar observaciones incompletas; simple, pero pierde información.
- **Imputación simple**
media, mediana o moda; rápida, pero subestima varianza.
- **Imputación múltiple**
genera varias imputaciones y combina resultados; incorpora incertidumbre.
- **Modelos de regresión / KNN**
usan variables observadas para imputar; útiles especialmente bajo **MAR**.

104.3. Eliminación Listwise

Una de las estrategias más simples para manejar datos faltantes es la **eliminación listwise** (o eliminación completa de casos), la cual consiste en eliminar cualquier observación que contenga al menos un valor faltante.

Formalmente, si una observación i presenta algún valor faltante en cualquiera de sus variables, entonces dicha observación es descartada completamente del análisis.

104.3.1. Impacto en el tamaño de muestra

Aunque esta técnica es fácil de implementar, tiene una desventaja importante: puede reducir significativamente el tamaño efectivo de la muestra.

Si la probabilidad de que una observación esté completamente observada es $P(\text{completa})$, entonces el tamaño efectivo se reduce a:

$$n_{\text{efectivo}} = n \cdot P(\text{completa})$$

En presencia de múltiples variables con datos faltantes, esta probabilidad puede decrecer rápidamente, provocando una pérdida considerable de información.

104.3.2. Consideraciones

- Es válida principalmente bajo el supuesto de **MCAR (Missing Completely At Random)**.
- Puede introducir sesgo si los datos no son MCAR.
- Reduce la potencia estadística al disminuir el tamaño de muestra.

Advertencia

Es importante destacar que la eliminación listwise no debe confundirse con una técnica de reducción de ruido. Mientras que la reducción de ruido busca suavizar variaciones aleatorias en los datos, la eliminación listwise simplemente descarta observaciones incompletas sin modificar los valores existentes.

104.4. Imputación simple

Consiste en reemplazar valores faltantes por un valor representativo de la variable.

Es decir, si no conocemos un valor, podemos **sustituirlo por una estimación simple** calculada a partir de los datos observados.

- media
- mediana
- moda
- constante

104.4.1. Imputación por media

Sea una variable Y con valores faltantes, la imputación por media reemplaza:

$$Y_{mis} \leftarrow \bar{Y}_{obs}$$

donde

$$\bar{Y}_{obs} = \frac{1}{n_{obs}} \sum_{i=1}^{n_{obs}} Y_i$$

104.5. Ejemplo sintético: imputación simple

Unable to display output for mime type(s): text/html

104.5.1. Consecuencia estadística

La imputación simple suele:

- preservar parcialmente la **tendencia central**
 - **reducir artificialmente la varianza**
 - introducir una **acumulación artificial** en el valor imputado, porque muchos faltantes se reemplazan por el mismo número
-

104.6. Imputación múltiple

En lugar de crear **un solo dataset imputado**, generamos **varias versiones plausibles**. Esto porque cada valor faltante puede tener **varias estimaciones posibles**.

La imputación múltiple:

1. genera **m datasets imputados**
 2. ajusta el modelo en cada uno
 3. **combina los resultados**
-

104.6.1. Formulación matemática

Sea q_i la estimación del parámetro en el dataset imputado i . la estimación final y su varianza:

$$\bar{q} = \frac{1}{m} \sum_{i=1}^m q_i \quad T = \bar{U} + \left(1 + \frac{1}{m}\right) B$$

donde

- $\bar{U}$ = varianza promedio dentro de imputaciones
 - B = varianza entre imputaciones
-

104.7. Ejemplo sintético: imputación múltiple

Unable to display output for mime type(s): text/html

104.7.1. Ventajas

- **incorpora la incertidumbre** del valor faltante
- preserva mejor la **variabilidad** que la imputación simple
- permite obtener inferencias más **rigurosas**

104.7.2. Desventajas

- es más **costosa computacionalmente**
 - requiere **varios datasets imputados**
 - depende de supuestos de modelado, típicamente cercanos a **MAR**
-

104.8. Imputación con KNN / modelos

En lugar de usar un único valor global, estimamos los faltantes usando **información de observaciones similares**.

Es decir, si un valor falta, buscamos observaciones **parecidas en otras variables** y usamos sus valores para estimarlo.

Ejemplo:

- productos con **precio, categoría y tienda similares**
 - probablemente tendrán **peso similar**
-

104.8.1. Formulación matemática (KNN)

Para una observación con valor faltante x_i :

1. se identifican los k vecinos más cercanos
2. se estima el valor faltante usando sus valores

$$\hat{x}_i = \frac{1}{k} \sum_{j \in N_k(i)} x_j$$

donde $N_k(i)$ son los **k vecinos más cercanos**.

104.9. Ejemplo sintético: imputación con KNN

Unable to display output for mime type(s): text/html

104.9.1. Ventajas

- preserva mejor la **estructura de los datos**
- utiliza **relaciones entre variables**
- suele funcionar bien bajo **MAR**

104.9.2. Desventajas

- depende de la **métrica de distancia**
 - puede ser **costoso computacionalmente** en datasets grandes
 - su desempeño disminuye si hay **muchos valores faltantes**
-
-

104.10. Recomendaciones para práctica profesional

- Visualiza siempre el patrón de faltantes.
 - No uses imputación por media como solución automática.
 - Piensa si la ausencia puede ser informativa.
 - Si asumes MAR, incluye buenas variables auxiliares.
 - Haz análisis de sensibilidad cuando el riesgo de MNAR sea alto.
-

104.11. Mini ejercicio en vivo

Pregunta: si un patrón de missing cambia por **Region**, ¿qué mecanismo parece más plausible?

	Region	Promo	missing
0	Centro	0	0.171785
1	Centro	1	0.198907
2	Norte	0	0.091124
3	Norte	1	0.075691
4	Sur	0	0.338710
5	Sur	1	0.320567

104.12. Para discutir

- ¿Se parece a MCAR?
 - ¿Qué variables usarías para imputar?
 - ¿dropna sería aceptable aquí?
-

104.13. Cierre

Manejar datos faltantes no es una tarea de limpieza menor.
Es una decisión estadística que afecta inferencia, predicción y credibilidad.

104.14. Próximo paso sugerido

Tomar un dataset real como **BigMart** y comparar cómo cambian las predicciones de ventas bajo distintas imputaciones.

105. Conclusiones

Los datos faltantes son inevitables en proyectos reales, pero su manejo inadecuado puede invalidar cualquier análisis posterior. Los puntos clave de este notebook son:

- 1. El mecanismo importa más que el porcentaje.** Un 5% de MNAR puede causar más daño que un 25% de MCAR.
- 2. Siempre diagnostica antes de imputar.** Visualiza si el missingness está asociado con otras variables. Un heatmap y un boxplot por grupo son el primer paso obligatorio.
- 3. La imputación por media es simple pero costosa.** Contrae la varianza y sesga los estimadores cuando el mecanismo no es MCAR. Úsala solo como línea base.
- 4. La imputación estocástica preserva la distribución.** Si la variable imputada será usada como predictor o se analizará su variabilidad, añadir ruido proporcional al MSE del modelo de imputación es esencial.
- 5. La variable indicadora de missingness es gratuita y poderosa.** Siempre créala. Si el mecanismo es MNAR, puede ser el predictor más informativo que tienes.
- 6. Usa pipelines para evitar *data leakage*.** El imputador debe ajustarse *solo sobre el conjunto de entrenamiento* y transformar el de prueba. `sklearn.pipeline.Pipeline` garantiza esto automáticamente.

Basado en: Protopapas, Rader & Tanner — CS109A Introduction to Data Science, Lecture 19: Dealing with Missing Data.

106. Reducción de Dimensionalidad con PCA

El análisis exploratorio de datos (EDA) ha evolucionado significativamente en los últimos años. Mientras que el enfoque clásico se centraba en estadísticas descriptivas y visualizaciones básicas, el EDA multivariable moderno reconoce que los datos reales son intrínsecamente de alta dimensión y que su comprensión requiere integrar múltiples niveles de análisis de manera estructurada.

107. La maldición de la dimensionalidad

Cuando el número de variables p crece, la geometría del espacio se vuelve profundamente contraintuitiva. El fenómeno conocido como **maldición de la dimensionalidad** (*curse of dimensionality*, Bellman, 1957) describe cómo el volumen del espacio crece tan rápidamente que los datos se vuelven inevitablemente dispersos.

Consideremos una hiperesfera de radio r inscrita en un hipercubo de lado $2r$ en $\mathbb{R}^d$. La proporción del volumen de la esfera respecto al cubo es:

$$\frac{V_{\text{esfera}}}{V_{\text{cubo}}} = \frac{\pi^{d/2} r^d}{\Gamma(d/2 + 1) \cdot (2r)^d} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)}$$

En 2 y en 3 dimensiones, esta proporción es relativamente alta (~78 % y ~52 % respectivamente), como se puede apreciar en la siguiente visualización interactiva donde $r = 1$.

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Sin embargo, esta proporción **tiende a cero** cuando $d \rightarrow \infty$. Esto significa que la mayoría del volumen del hipercubo se concentra en las esquinas, y la esfera (que representa la región de interés) ocupa una fracción cada vez menor del espacio total.

Unable to display output for mime type(s): text/html

Esto implica que para el **análisis de datos en alta dimensión**, mantener una densidad de muestreo fija ρ en $\mathbb{R}^d$, el número de observaciones necesario escala como:

$$n \sim \rho \cdot L^d$$

Con $p = 20$ variables, el número de correlaciones posibles es $\binom{20}{2} = 190$. Con $p = 100$, son 4,950. Revisar e interpretar cada relación manualmente deja de ser practicable. Por ello necesitamos métodos que resuman la información sin perder lo esencial.

108. Fundamentos matemáticos del PCA

El Análisis de Componentes Principales es una técnica de reducción de dimensionalidad que transforma un conjunto de datos con variables correlacionadas en un nuevo sistema de variables no correlacionadas llamadas **componentes principales**. Cada componente es una combinación lineal de las variables originales y representa una dirección de máxima varianza, preservando así la mayor cantidad de información posible de los datos.

108.1. Formulación Matemática y Procedimiento

Dado un conjunto de datos $X \in \mathbb{R}^{n \times p}$, donde n es el número de muestras y p el número de variables, el proceso se rige por la siguiente secuencia lógica:

1. Estandarización y Centrado

Para evitar que las unidades de medida distorsionen el análisis, los datos deben transformarse a una escala común. Esto se realiza en dos pasos:

- **Centrado:** restar la media de cada variable
- **Escalado:** dividir entre su desviación estándar

De esta forma, cada variable tiene media cero y varianza unitaria.

$$\tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}$$

donde:

- μ_j es la media de la variable j
- σ_j es su desviación estándar

2. **Cálculo de la Matriz de Covarianza:** Se construye la matriz S (o $\mathbf{C}$ dependiendo de la notación, $\mathbf{C}$ es común en ML), que resume la variabilidad de cada variable y la relación entre ellas:

$$S = \frac{1}{n-1} \tilde{X}^T \tilde{X}$$

De forma elemento a elemento:

$$s_{jk} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)(x_{ik} - \mu_k)$$

donde:

- s_{jj} representa la **varianza** de la variable j
- s_{jk} representa la **covarianza** entre las variables j y k

Esta matriz es simétrica y semidefinida positiva, lo que garantiza que sus eigenvalores sean reales y no negativos.

3. **Descomposición Espectral:** Se resuelve el problema de autovalores para encontrar las direcciones principales:

$$S\mathbf{v} = \lambda\mathbf{v}$$

Aquí, λ representa los eigenvalores (la varianza explicada en esa dirección) y $\mathbf{v}$ los eigenvectores (las direcciones o *loadings*).

4. **Optimización y Ordenamiento:** El primer componente principal w_1 es el eigenvector asociado al eigenvalor máximo λ_1 . Los componentes siguientes se obtienen maximizando la varianza restante bajo la condición de ser ortogonales a los anteriores. Se clasifican de forma descendente: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$.
5. **Proyección y Reducción:** Finalmente, se proyectan los datos originales sobre el subespacio generado por los primeros k componentes:

$$\mathbf{Z} = \tilde{X}\mathbf{V}_k$$

Donde $\mathbf{V}_k$ es la matriz que contiene los primeros k eigenvectores ordenados.

Cada componente captura una parte de la variabilidad total de los datos, la cual equivale a la traza de la matriz de covarianza ($\sum \lambda_j$). La importancia de cada componente se mide a través de la **Proporción de Varianza Explicada (PVE)**:

$$\text{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}$$

Esto permite reducir la dimensionalidad manteniendo, por ejemplo, el 95 % de la varianza acumulada original.

108.2. Ejemplo Práctico: Cálculo Manual Paso a Paso

A continuación, se desarrolla el proceso completo partiendo de una matriz de datos crudos de dos variables ($p = 2$):

1. **Definición de la matriz de datos original:** Supongamos $n = 3$ observaciones (ej. mediciones de dos sensores) representados como filas y $p = 2$ variables representadas como columnas:

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{pmatrix}$$

2. **Centrado de los datos:** Calculamos la media de cada columna: $\bar{x}_1 = 3$ y $\bar{x}_2 = 4$. Restamos la media a cada fila para obtener $\tilde{X}$:

$$\tilde{X} = \begin{pmatrix} 2-3 & 3-4 \\ 3-3 & 4-4 \\ 4-3 & 5-4 \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

3. **Cálculo de la matriz de covarianza (S):** Aplicamos $S = \frac{1}{n-1} \tilde{X}^T \tilde{X}$ con $n-1 = 2$:

$$S = \frac{1}{2} \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

4. **Obtención de eigenvalores (λ):** Resolvemos el determinante $\det(S - \lambda I) = 0$:

$$\det \begin{pmatrix} 1-\lambda & 1 \\ 1 & 1-\lambda \end{pmatrix} = (1-\lambda)^2 - 1 = 0 \implies \lambda^2 - 2\lambda = 0$$

Los resultados son $\lambda_1 = 2$ y $\lambda_2 = 0$. La varianza total es $2 + 0 = 2$. El primer componente explica el 100 % de la varianza.

5. **Obtención de eigenvectores ($\mathbf{v}$):** Para $\lambda_1 = 2$, resolvemos $(S - 2I)\mathbf{v}_1 = 0$:

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{21} \end{pmatrix} = 0 \implies v_{11} = v_{21}$$

Normalizando para que $\|v\| = 1$: $v_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \approx \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$.

6. **Proyección de los datos ($k = 1$):** Transformamos los datos centrados a la nueva dimensión $Z = \tilde{X}v_1$:

$$Z = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix} = \begin{pmatrix} -1.414 \\ 0 \\ 1.414 \end{pmatrix}$$

7. **Conclusión:** Dado que los datos originales estaban perfectamente alineados ($x_2 = x_1 + 1$), el PCA ha logrado reducir la dimensionalidad de 2 a 1 sin ninguna pérdida de información, capturando toda la estructura de los datos en un solo eje.

108.3. Ejemplo Práctico: Visualización paso a paso

A continuación, se desarrolla el proceso completo de nuevo incluyendo una visualización interactiva en cada paso para ilustrar cómo se transforman los datos a lo largo del proceso de PCA.

1. **Definición de la matriz de datos original**

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{pmatrix}$$

```
import numpy as np
import plotly.graph_objects as go

X = np.array([[2,3],[3,4],[4,5]])

fig = go.Figure()
fig.add_trace(go.Scatter(
```

```

x=X[:,0], y=X[:,1],
mode='markers+text',
text=['P1','P2','P3'],
textposition='top center'
))
fig.update_layout(title="Datos originales")
fig.show()

```

Unable to display output for mime type(s): text/html

2. Centrado de los datos

$$\tilde{X} = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

```

X_centered = X - X.mean(axis=0)

fig = go.Figure()
fig.add_hline(y=0)
fig.add_vline(x=0)

fig.add_trace(go.Scatter(
    x=X_centered[:,0], y=X_centered[:,1],
    mode='markers+text',
    text=['P1','P2','P3'],
    textposition='top center'
))

fig.update_layout(title="Datos centrados")
fig.show()

```

Unable to display output for mime type(s): text/html

3. Cálculo de la matriz de covarianza

$$S = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

```

S = np.cov(X_centered, rowvar=False)

fig = go.Figure(data=go.Heatmap(
    z=S,
    text=S.round(2),
))

```

```
fig.update_layout(title="Matriz de covarianza")
fig.show()
```

Unable to display output for mime type(s): text/html

4. Obtención de eigenvalores

$$\lambda_1 = 2, \quad \lambda_2 = 0$$

```
eigvals, eigvecs = np.linalg.eig(S)

fig = go.Figure()
fig.add_trace(go.Bar(
    x=[' 1', ' 2'],
    y=eigvals
))

fig.update_layout(title="Varianza por componente")
fig.show()
```

Unable to display output for mime type(s): text/html

5. Obtención de eigenvectores

$$v_1 \approx \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$$

```
v1 = eigvecs[:, np.argmax(eigvals)]
t = np.linspace(-2, 2, 100)

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=X_centered[:, 0], y=X_centered[:, 1],
    mode='markers'
))

fig.add_trace(go.Scatter(
    x=t*v1[0],
    y=t*v1[1],
    mode='lines',
    name='PC1'
))

fig.update_layout(title="Dirección principal (PC1)")
fig.show()
```

Unable to display output for mime type(s): text/html

6. Proyección de los datos

$$Z = \begin{pmatrix} -1.414 \\ 0 \\ 1.414 \end{pmatrix}$$

```
Z = X_centered @ v1

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=Z,
    y=[0,0,0],
    mode='markers+text',
    text=['P1', 'P2', 'P3'],
    textposition='top center'
))

fig.update_layout(title="Datos proyectados en PC1")
fig.show()
```

Unable to display output for mime type(s): text/html

7. Conclusión

Dado que los datos están perfectamente alineados:

- Toda la varianza se concentra en una dirección
- PCA reduce de 2D a 1D sin pérdida
- La interpretación es clara: la dirección de máxima varianza coincide con la línea que conecta los puntos, y el segundo componente no aporta información adicional.

109. PCA y la Descomposición en Valores Singulares (SVD)

En la práctica, en las paqueterías modernas PCA **nunca** se calcula mediante la eigendescomposición de S directamente (pueden ver el ejemplo de la documentación de scikitlearn). En cambio, se utiliza la **Descomposición en Valores Singulares (SVD)** de la matriz centrada $\tilde{X}$, que es numéricamente más estable y eficiente.

Toda matriz $\tilde{X} \in \mathbb{R}^{n \times p}$ admite la descomposición:

$$\tilde{X} = U\Sigma V^T$$

donde:

- $U \in \mathbb{R}^{n \times n}$: matriz ortogonal de **vectores singulares izquierdos** (columnas = eigenvectores de $\tilde{X}\tilde{X}^T$)
- $\Sigma \in \mathbb{R}^{n \times p}$: matriz diagonal de **valores singulares** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(n,p)} \geq 0$
- $V \in \mathbb{R}^{p \times p}$: matriz ortogonal de **vectores singulares derechos** (columnas = eigenvectores de $\tilde{X}^T\tilde{X}$)

La relación entre SVD y PCA es directa:

$$S = \frac{1}{n-1} \tilde{X}^T \tilde{X} = \frac{1}{n-1} (U\Sigma V^T)^T (U\Sigma V^T) = \frac{1}{n-1} V\Sigma^2 V^T$$

Por tanto:

- Las **columnas de V** son los **eigenvectores de S** (componentes principales)
- Los **eigenvalores de S** son $\lambda_k = \sigma_k^2 / (n-1)$
- Los **scores** (proyecciones) son $Z = \tilde{X}V = U\Sigma$

¿Por qué SVD y no eigendescomposición directa?

La eigendescomposición de S requiere calcular primero $S = \tilde{X}^T \tilde{X} / (n-1)$, lo cual:

1. **Pierde precisión numérica:** los números de condición se elevan al cuadrado. Si $\tilde{X}$ tiene número de condición κ , S tiene κ^2 .
2. **Es ineficiente cuando $n \ll p$:** la SVD puede trabajar directamente con la matrix de $n \times p$ usando la *thin SVD*, evitando la costosa matrix $p \times p$.
3. **El algoritmo de Golub-Reinsch** para SVD es más estable numéricamente que los algoritmos de eigendescomposición para matrices no simétricas.

```

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

X = load_iris().data
X_std = StandardScaler().fit_transform(X)

# Método 1: eigendescomposición de S
S = np.cov(X_std.T)
eigenvalues, eigenvectors = np.linalg.eigh(S)
# eigh ordena de menor a mayor, invertir
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# Método 2: SVD directa sobre X_std
U, sigma, Vt = np.linalg.svd(X_std, full_matrices=False)
lambda_svd = sigma**2 / (len(X_std) - 1)

print("Eigenvalores via S: ", np.round(eigenvalues, 6))
print("Eigenvalores via SVD:", np.round(lambda_svd, 6))
print("Son idénticos:", np.allclose(eigenvalues, lambda_svd))

```

```

Eigenvalores via S:  [2.938085 0.920165 0.147742 0.020854]
Eigenvalores via SVD: [2.938085 0.920165 0.147742 0.020854]
Son idénticos: True

```

110. Ejemplo práctico: Implementación paso a paso

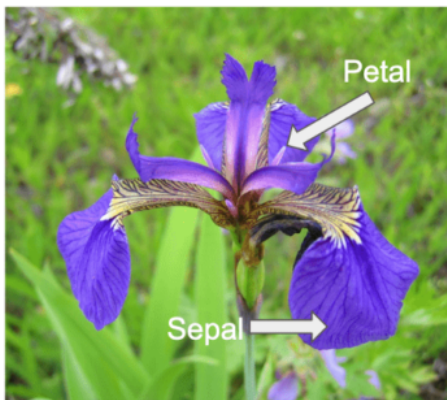
Dataset: Iris

Para este ejemplo usaremos Scikit-learn, mediante el clásico dataset *Iris* (flores de lirio de este lado del charco), que contiene mediciones de flores de tres especies (*setosa*, *versicolor* y *virginica*). Cada observación está descrita por cuatro variables numéricas:

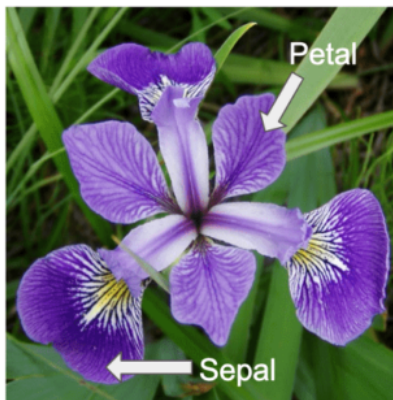
- longitud del sépalo
- ancho del sépalo
- longitud del pétalo
- ancho del pétalo

Estas variables capturan características morfológicas de las flores y presentan correlaciones entre sí, lo que lo convierte en un ejemplo ideal para ilustrar cómo PCA identifica estructura y reduce dimensionalidad.

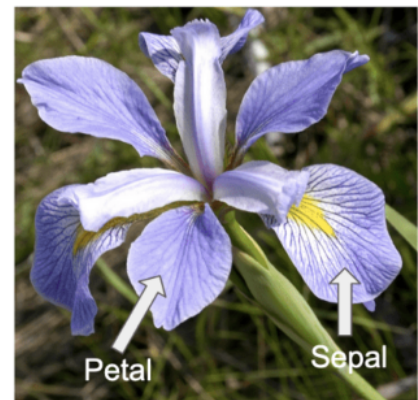
Iris setosa



Iris versicolor



Iris virginica



```
iris = load_iris()
X = iris.data
y = iris.target
nombres = iris.target_names
feature_names = ['Sépalo largo', 'Sépalo ancho', 'Pétalo largo', 'Pétalo ancho']

df_iris = pd.DataFrame(X, columns=feature_names)
df_iris['Especie'] = [nombres[i] for i in y]
df_iris.describe().round(3)
```

	Sépalo largo	Sépalo ancho	Pétalo largo	Pétalo ancho
count	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.300	2.000	1.000	0.100
25 %	5.100	2.800	1.600	0.300
50 %	5.800	3.000	4.350	1.300
75 %	6.400	3.300	5.100	1.800
max	7.900	4.400	6.900	2.500

Paso 1: Estandarización

La estandarización es **obligatoria** cuando las variables tienen escalas distintas. Sin ella, las variables con mayor varianza absoluta dominarían el PCA independientemente de su relevancia.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Después de estandarizar: $\bar{z}_j = 0$ y $s_{z_j} = 1$ para toda variable j .

```
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# Verificar
df_check = pd.DataFrame({
    'Variable': feature_names,
    'Media original': X.mean(axis=0).round(3),
    'Desv original': X.std(axis=0).round(3),
    'Media estand.': X_std.mean(axis=0).round(10),
    'Desv estand.': X_std.std(axis=0).round(3)
})
df_check
```

	Variable	Media original	Desv original	Media estand.	Desv estand.
0	Sépalo largo	5.843	0.825	-0.0	1.0
1	Sépalo ancho	3.057	0.434	-0.0	1.0
2	Pétalo largo	3.758	1.759	-0.0	1.0
3	Pétalo ancho	1.199	0.760	-0.0	1.0

Paso 2: Matriz de covarianza


```

S = np.cov(X_std.T)

fig = go.Figure(data=go.Heatmap(
    z=S,
    x=feature_names,
    y=feature_names,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(S, 3),
    texttemplate='%{text}',
    textfont=dict(size=13),
    hovertemplate='%{y}  %{x}<br>cov =%{z:.3f}<extra></extra>',
    colorbar=dict(title='Covarianza')
))
fig.update_layout(
    title="Matriz de covarianza - datos estandarizados (= matriz de correlación)",
    height=420
)
fig.show()

```

Unable to display output for mime type(s): text/html

i Nota

Cuando los datos están estandarizados, la matriz de covarianza es idéntica a la **matriz de correlación de Pearson**. Por eso PCA sobre datos estandarizados equivale a hacer PCA sobre correlaciones.

Paso 3: Eigendecomposición vía SVD

```

pca = PCA()
scores = pca.fit_transform(X_std)

eigenvalues = pca.explained_variance_
pve = pca.explained_variance_ratio_ * 100
pve_acum = np.cumsum(pve)

df_pca = pd.DataFrame({
    'Componente': [f'PC{i+1}' for i in range(len(eigenvalues))],
    'Eigenvalue ( )': eigenvalues.round(4),
    '% Varianza': pve.round(2),
    '% Acumulado': pve_acum.round(2)
})
df_pca

```

	Componente	Eigenvalue ()	% Varianza	% Acumulado
0	PC1	2.9381	72.96	72.96
1	PC2	0.9202	22.85	95.81
2	PC3	0.1477	3.67	99.48
3	PC4	0.0209	0.52	100.00

Paso 4: Scree plot

```
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=df_pca['Componente'], y=df_pca['% Varianza'],
    name='% Varianza',
    text=[f"{v:.1f}%" for v in pve],
    textposition='outside'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=df_pca['Componente'], y=pve_acum,
    mode='lines+markers',
    line=dict(width=2.5),
    marker=dict(size=9),
    name='% Acumulado'
), secondary_y=True)

fig.add_hline(y=80, line_dash="dash", line_color="gray",
              annotation_text="80%", secondary_y=True)

fig.update_yaxes(title_text="% Varianza por componente", secondary_y=False)
fig.update_yaxes(title_text="% Varianza acumulada", range=[0, 110], secondary_y=True)
fig.update_layout(
    title="Scree Plot - Regla de Kaiser: retener componentes con  $\lambda_k > 1$ ",
    height=380
)
fig.show()
```

Unable to display output for mime type(s): text/html

💡 Tip

Criterios para elegir k :

- **Regla de Kaiser:** retener componentes con $\lambda_k > 1$ (solo aplica con datos estandarizados)
- **Varianza acumulada:** retener los primeros k que superen el 80 % (o 90 %)
- **Scree plot:** buscar el “codo” donde la pendiente cae abruptamente
- **Interpretabilidad de dominio:** ¿los componentes tienen sentido en el problema?

Paso 5: Biplot

El **biplot** superpone los scores de las observaciones y los vectores de los loadings en el mismo espacio reducido, permitiendo interpretar simultáneamente la estructura de las observaciones y la contribución de las variables.

```
loadings = pca.components_ # shape: (n_components, n_features)
scores_df = pd.DataFrame(scores[:, :2], columns=['PC1', 'PC2'])
scores_df['Especie'] = [nombres[i] for i in y]

fig = go.Figure()

# Scores
for especie in nombres:
    mask = scores_df['Especie'] == especie
    fig.add_trace(go.Scatter(
        x=scores_df.loc[mask, 'PC1'],
        y=scores_df.loc[mask, 'PC2'],
        mode='markers',
        marker=dict(size=7, opacity=0.7),
        name=especie
    ))

# Vectores de loadings escalados
escala = 3.0
for i, feat in enumerate(feature_names):
    fig.add_annotation(
        x=loadings[0, i] * escala,
        y=loadings[1, i] * escala,
        ax=0, ay=0,
        arrowhead=3, arrowwidth=2,
        xref='x', yref='y', axref='x', ayref='y',
        showarrow=True
    )
    fig.add_annotation(
        x=loadings[0, i] * escala * 1.15,
        y=loadings[1, i] * escala * 1.15,
        text=f"<b>{feat}</b>",
        showarrow=False,
        font=dict(size=11)
    )

fig.update_xaxes(title_text=f"PC1 ({pve[0]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text=f"PC2 ({pve[1]:.1f}%)",
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title="Biplot: scores + loadings - Iris",
    height=480,
```

```

    legend=dict(title='Especie')
)
fig.show()

```

Unable to display output for mime type(s): text/html

Interpretación del biplot:

- Las flechas (loadings) representan los **vectores de las variables proyectados** sobre el nuevo sistema de coordenadas definido por los componentes principales. Es decir, cada flecha muestra cómo una variable original se expresa en la nueva base.
- Flechas largas → la variable tiene **gran proyección (norma)** sobre ese plano, por lo que contribuye significativamente a la variabilidad capturada por los componentes.
- Flechas apuntando en la misma dirección → sus vectores forman un **ángulo pequeño**, lo que indica alta correlación (producto punto positivo grande).
Flechas opuestas → correlación negativa.
Flechas ortogonales → variables no correlacionadas.
- La posición de los puntos corresponde a la **proyección de las observaciones** en el subespacio generado por los primeros componentes.
La separación entre grupos refleja qué tan bien este subespacio (de menor dimensión) preserva la estructura original de los datos.

En conjunto, el biplot muestra simultáneamente: - un **cambio de base** (álgebra lineal)

- y una **proyección de los datos** en un subespacio de menor dimensión

Paso 6: Loadings detallados

```

df_loadings = pd.DataFrame(
    loadings[:2].T,
    index=feature_names,
    columns=['PC1', 'PC2']
).round(4)

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=[
        "<b>Componente Principal 1 (PC1)</b>",
        "<b>Componente Principal 2 (PC2)</b>"
    ]
)

# PC1
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[0],

```

```

        text=[f"{v:.3f}" for v in loadings[0]],
        textposition='outside'
    ), row=1, col=1)

# PC2
fig.add_trace(go.Bar(
    x=feature_names,
    y=loadings[1],
    text=[f"{v:.3f}" for v in loadings[1]],
    textposition='outside'
), row=1, col=2)

# Ejes
for c in [1, 2]:
    fig.update_yaxes(
        range=[-1.1, 1.1],
        zeroline=True,
        zerolinecolor='lightgray',
        row=1, col=c
    )
    fig.update_xaxes(
        tickangle=-25,
        row=1, col=c
    )

# Hacer más visibles los títulos de subplots
for ann in fig['layout']['annotations']:
    ann['font'] = dict(size=14)

# Nota interpretativa
fig.update_layout(
    showlegend=False,
    height=420,
    annotations=list(fig.layout.annotations) + [
        dict(
            x=0.5, y=-0.2,
            xref='paper', yref='paper',
            text="PC1  tamaño general | PC2  contraste estructural del sépalo",
            showarrow=False,
            font=dict(size=12, color='gray'),
            xanchor='center'
        )
    ]
)

fig.show()

```

Unable to display output for mime type(s): text/html

Interpretación de los loadings:

Cada componente principal es una **combinación lineal de las variables originales**. Es decir:

$$PC_k = w_{1k}x_1 + w_{2k}x_2 + \cdots + w_{pk}x_p$$

donde los *loadings* w_{jk} indican el peso de cada variable en ese componente.

- **PC1** combina fuertemente *pétalo largo*, *pétalo ancho* y *sépalo largo* (coeficientes positivos similares), por lo que representa un eje de **tamaño general de la flor**. El *sépalo ancho* contribuye negativamente, ajustando ligeramente esa dirección.
- **PC2** está dominado por *sépalo ancho*, con poca contribución del resto, por lo que captura un **contraste específico en la forma del sépalo**.

En términos geométricos, cada componente define una **dirección en el espacio original**, y los loadings son las **coordenadas de esa dirección** en la base de variables originales.

111. Ejemplo avanzado: Dataset con ruido

El problema: una partícula en movimiento que no podemos ver directamente

Imagina que una partícula traza una trayectoria helicoidal en el espacio tridimensional. Nosotros no podemos observarla directamente — solo tenemos acceso a **sensores**, y cada sensor introduce su propio ruido de medición.

El desafío central es: dado un conjunto de lecturas ruidosas de múltiples sensores, ¿podemos recuperar la trayectoria original?

PCA responde exactamente a esa pregunta. Pero antes de aplicarlo, construyamos el experimento desde cero.

Paso 1 — La trayectoria real y el ruido de medición

Generamos 500 puntos sobre una hélice paramétrica:

$$x(t) = t, \quad y(t) = \cos(t), \quad z(t) = \sin(t), \quad t \in [-10, 10]$$

Luego añadimos ruido gaussiano independiente en y y z para simular la imprecisión de los instrumentos de medida.

```
n_points = 500
t = np.linspace(-10, 10, n_points)

# Trayectoria limpia
x_clean = t
y_clean = np.cos(t)
z_clean = np.sin(t)

# Observaciones ruidosas (lo que miden los sensores)
sigma = 0.5
y_noise = y_clean + np.random.normal(0, sigma, n_points)
z_noise = z_clean + np.random.normal(0, sigma, n_points)

points = np.column_stack([x_clean, y_noise, z_noise])

# Visualización
fig = go.Figure()

fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers',
```

```

    marker=dict(size=3, color='steelblue', opacity=0.5),
    name='Mediciones ruidosas'
))

fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines',
    line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

fig.update_layout(
    title='Trayectoria helicoidal: señal real vs. mediciones ruidosas',
    scene=dict(
        xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
        yaxis=dict(range=[-5, 5]),
        xaxis=dict(range=[-11, 11])
    ),
    legend=dict(x=0.01, y=0.99),
    height=520
)
fig.show()

```

Unable to display output for mime type(s): text/html

Los puntos azules son lo que los sensores registran. La línea roja es la realidad que queremos recuperar. A simple vista ya es difícil distinguir la estructura subyacente del ruido.

Paso 2 — Tres sensores, tres perspectivas distintas

En un experimento real, distintos sensores observan el mismo fenómeno desde **ángulos diferentes**. Cada sensor proyecta la nube de puntos sobre su propio plano de referencia y mide coordenadas locales (u, v) en ese plano — más, inevitablemente, su propio ruido adicional.

Simulamos tres sensores con planos de proyección distintos.

```

# Funciones auxiliares

def projection(points, plane_point, plane_normal):
    """Proyecta cada punto sobre un plano definido por un punto y su normal."""
    n = plane_normal / np.linalg.norm(plane_normal)
    d = points - plane_point
    dist = d.dot(n)
    return points - np.outer(dist, n)

def plane_axes(plane_normal):
    """Devuelve dos vectores ortogonales que forman la base del plano."""

```



```

n = plane_normal / np.linalg.norm(plane_normal)
# Vector arbitrario no paralelo a n
arb = np.array([1, 0, 0]) if abs(n[0]) < 0.9 else np.array([0, 1, 0])
u = np.cross(n, arb)
u /= np.linalg.norm(u)
v = np.cross(n, u)
v /= np.linalg.norm(v)
return u, v

def sensor_reading(points, plane_point, plane_normal, noise_std=0.5):
    """
    Proyecta puntos sobre el plano del sensor y devuelve:
    - coordenadas 2D locales (s, t) con ruido adicional
    - la proyección 3D (para visualización)
    - los vectores de la base del plano (u, v)
    """
    proj_3d = projection(points, plane_point, plane_normal)
    u, v = plane_axes(plane_normal)

    proj_vectors = proj_3d - plane_point
    s = proj_vectors.dot(u) + np.random.normal(0, noise_std, len(points))
    t = proj_vectors.dot(v) + np.random.normal(0, noise_std, len(points))

    return s, t, proj_3d, u, v

def plane_mesh(plane_point, plane_normal, size=12, res=20):
    """Genera una malla rectangular para visualizar el plano del sensor."""
    u, v = plane_axes(plane_normal)
    lin = np.linspace(-size, size, res)
    uu, vv = np.meshgrid(lin, lin)
    X = plane_point[0] + uu * u[0] + vv * v[0]
    Y = plane_point[1] + uu * u[1] + vv * v[1]
    Z = plane_point[2] + uu * u[2] + vv * v[2]
    return X, Y, Z

# Definición de los tres sensores
sensors = [
    dict(point=np.array([5., -6., 0.]), normal=np.array([1., -1., 0.]),
        color='royalblue', name='Sensor A'),
    dict(point=np.array([10., 0., 0.]), normal=np.array([10., 0., 0.1]),
        color='seagreen', name='Sensor B'),
    dict(point=np.array([3., 3., 3.]), normal=np.array([1., 1., 1.]),
        color='darkorange', name='Sensor C'),
]

readings = []
for s in sensors:

```

```

sc, tc, proj3d, u, v = sensor_reading(
    points, s['point'], s['normal'], noise_std=0.5
)
readings.append({'s': sc, 't': tc, 'proj3d': proj3d,
                 'u': u, 'v': v, **s})

```

Perspectiva 3D: sensores y sus proyecciones

```

fig = go.Figure()

# Trayectoria real
fig.add_trace(go.Scatter3d(
    x=x_clean, y=y_clean, z=z_clean,
    mode='lines', line=dict(color='crimson', width=4),
    name='Trayectoria real'
))

# Puntos originales
fig.add_trace(go.Scatter3d(
    x=points[:, 0], y=points[:, 1], z=points[:, 2],
    mode='markers', marker=dict(size=2, color='lightgray', opacity=0.4),
    name='Mediciones originales'
))

for r in readings:
    # Plano del sensor
    px_m, py_m, pz_m = plane_mesh(r['point'], r['normal'], size=10)
    fig.add_trace(go.Surface(
        x=px_m, y=py_m, z=pz_m,
        colorscale=[[0, r['color']], [1, r['color']]],
        opacity=0.15, showscale=False,
        name=f"Plano {r['name']}", showlegend=False
    ))
    # Proyección sobre el plano
    fig.add_trace(go.Scatter3d(
        x=r['proj3d'][:, 0], y=r['proj3d'][:, 1], z=r['proj3d'][:, 2],
        mode='markers', marker=dict(size=2.5, color=r['color'], opacity=0.7),
        name=r['name']
    ))

fig.update_layout(
    title='Los tres sensores proyectan la nube sobre sus planos respectivos',
    scene=dict(xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
               xaxis=dict(range=[-12, 12]),
               yaxis=dict(range=[-8, 8])),
    legend=dict(x=0.01, y=0.99),
    height=560
)

```

```
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor “aplana” la nube sobre su propio plano. Ninguno captura la hélice perfectamente, pero cada uno aporta información parcial y complementaria.

Lo que registra cada sensor: coordenadas 2D

```
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=[r['name'] for r in readings],
    shared_yaxes=False
)

for i, r in enumerate(readings, 1):
    fig.add_trace(go.Scatter(
        x=r['s'], y=r['t'],
        mode='markers',
        marker=dict(size=3, color=r['color'], opacity=0.6),
        name=r['name'],
        showlegend=False
    ), row=1, col=i)
    fig.update_xaxes(title_text='u (eje 1 del plano)', row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')
    fig.update_yaxes(title_text='v (eje 2 del plano)' if i == 1 else '',
                     row=1, col=i,
                     zeroline=True, zerolinecolor='lightgray')

fig.update_layout(
    title='Lecturas brutas de cada sensor (coordenadas 2D locales)',
    height=400
)
fig.show()
```

Unable to display output for mime type(s): text/html

Cada sensor produce una imagen 2D distorsionada de la misma hélice. El sensor A la ve casi como una elipse, el B como una banda horizontal, el C como una nube más compacta. Ninguno por sí solo revela la hélice original.

Paso 3 — Construir la matriz de datos multi-sensor

Apilamos las seis coordenadas (2 por cada sensor) para formar una **matriz de datos** $X \in \mathbb{R}^{500 \times 6}$. Cada fila es una muestra; cada columna es una variable medida por algún sensor.

Esta es exactamente la estructura con la que trabaja PCA: muchas variables correlacionadas que comparten una estructura latente de baja dimensión.

```
# Matriz de datos: 500 muestras × 6 variables (2 por sensor)
data_matrix = np.column_stack([r['s'] for r in readings] +
                               [r['t'] for r in readings])
# Reordenamos: s_A, t_A, s_B, t_B, s_C, t_C
data_matrix = np.column_stack([
    readings[0]['s'], readings[0]['t'],
    readings[1]['s'], readings[1]['t'],
    readings[2]['s'], readings[2]['t'],
])

col_labels = ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']

print(f"Dimensión de la matriz de datos: {data_matrix.shape}")
print(f"Variables: {col_labels}")
```

Dimensión de la matriz de datos: (500, 6)

Variables: ['A_u', 'A_v', 'B_u', 'B_v', 'C_u', 'C_v']

Matriz de correlación entre sensores

```
corr = np.corrcoef(data_matrix.T)

fig = go.Figure(data=go.Heatmap(
    z=corr,
    x=col_labels, y=col_labels,
    colorscale='RdBu_r',
    zmid=0, zmin=-1, zmax=1,
    text=np.round(corr, 2),
    texttemplate='%{text}',
    textfont=dict(size=11),
    colorbar=dict(title='Correlación')
))

fig.update_layout(
    title='Correlación entre las 6 variables medidas por los sensores',
    height=420
)
fig.show()
```

Unable to display output for mime type(s): text/html

La matriz de correlación revela la redundancia. Variables del mismo sensor están fuertemente correlacionadas entre sí, y también con variables de otros sensores — evidencia de que todas miden el mismo fenómeno subyacente. PCA explotará precisamente esa redundancia.

Paso 4 — Aplicar PCA: ¿cuánta información hay realmente?

PCA descompone la varianza total en **componentes ortogonales ordenados por importancia**. Si el fenómeno real vive en pocas dimensiones, la mayor parte de la varianza se concentrará en los primeros componentes.

```
pca_full = PCA(n_components=6)
components_full = pca_full.fit_transform(data_matrix)

explained = pca_full.explained_variance_ratio_
cumulative = np.cumsum(explained)

# Scree plot
fig = make_subplots(specs=[[{"secondary_y": True}]]))

fig.add_trace(go.Bar(
    x=[f'PC{i+1}' for i in range(6)],
    y=explained * 100,
    name='% Varianza individual',
    text=[f'{v*100:.1f}%' for v in explained],
    textposition='outside',
    marker_color='steelblue'
), secondary_y=False)

fig.add_trace(go.Scatter(
    x=[f'PC{i+1}' for i in range(6)],
    y=cumulative * 100,
    mode='lines+markers',
    name='% Acumulado',
    line=dict(color='crimson', width=2.5),
    marker=dict(size=9)
), secondary_y=True)

fig.add_hline(y=90, line_dash='dash', line_color='gray',
              annotation_text='90%', secondary_y=True)

fig.update_yaxes(title_text='% Varianza por componente', secondary_y=False)
fig.update_yaxes(title_text='% Varianza acumulada', range=[0, 110],
                 secondary_y=True)
fig.update_layout(
    title='Scree plot: ¿cuántos componentes capturan la señal?',
    height=400
)
fig.show()
```

```
print("Varianza explicada por componente:")
for i, (v, c) in enumerate(zip(explained, cumulative)):
    print(f"  PC{i+1}: {v*100:5.1f}%    acumulado: {c*100:5.1f}%")
```

Unable to display output for mime type(s): text/html

Varianza explicada por componente:

PC1:	89.1%	acumulado:	89.1%
PC2:	5.2%	acumulado:	94.3%
PC3:	4.0%	acumulado:	98.3%
PC4:	0.6%	acumulado:	98.9%
PC5:	0.6%	acumulado:	99.5%
PC6:	0.5%	acumulado:	100.0%

Los primeros **2–3 componentes** concentran la gran mayoría de la varianza. Esto confirma lo que esperábamos: el sistema tiene **2 grados de libertad reales** (la hélice es una curva en 3D, parametrizada por un solo parámetro t), y el resto es ruido.

Paso 5 — Proyección en 2D: la hélice emerge del ruido

Con solo los dos primeros componentes principales recuperamos la estructura esencial del movimiento.

```
pca_2d = PCA(n_components=2)
proj_2d = pca_2d.fit_transform(data_matrix)

# Línea de referencia: sin(t) proyectada para comparación
t_ref = np.linspace(-12, 12, 300)
ref_x = t_ref / t_ref.max() * proj_2d[:, 0].max()

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=proj_2d[:, 0], y=proj_2d[:, 1],
    mode='markers',
    marker=dict(size=4, color='steelblue', opacity=0.6),
    name='Muestras (PC1 vs PC2)'
))

# Colorear por el parámetro t para mostrar la continuidad
fig.add_trace(go.Scatter(
    x=proj_2d[:, 0], y=proj_2d[:, 1],
    mode='markers',
    marker=dict(
        size=4, opacity=0.8,
        color=t,
        colorscale='Viridis',
```

```

        colorbar=dict(title='t (tiempo)', len=0.6)
    ),
    name='Coloreado por t'
))

fig.update_xaxes(title_text='Componente Principal 1',
                  zeroline=True, zerolinecolor='lightgray')
fig.update_yaxes(title_text='Componente Principal 2',
                  zeroline=True, zerolinecolor='lightgray',
                  scaleanchor='x')
fig.update_layout(
    title=f'PC1 vs PC2 - {pca_2d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    height=480
)
fig.show()

```

Unable to display output for mime type(s): text/html

Al colorear los puntos por el parámetro t (el “tiempo” de la trayectoria), vemos que el orden temporal se preserva perfectamente: PCA ha recuperado la parametrización natural de la hélice a partir de seis variables ruidosas.

Paso 6 — Proyección en 3D: confirmando la estructura helicoidal

Agregando el tercer componente recuperamos la geometría completa.

```

pca_3d = PCA(n_components=3)
proj_3d = pca_3d.fit_transform(data_matrix)

fig = go.Figure()

# Muestras coloreadas por t
fig.add_trace(go.Scatter3d(
    x=proj_3d[:, 0], y=proj_3d[:, 1], z=proj_3d[:, 2],
    mode='markers',
    marker=dict(
        size=3, opacity=0.7,
        color=t,
        colorscale='Viridis',
        colorbar=dict(title='t', len=0.5)
    ),
    name='Muestras'
))

# Trayectoria original reescalada para superposición visual
scale = proj_3d[:, 0].std() / x_clean.std()
fig.add_trace(go.Scatter3d(

```

```

x=x_clean * scale,
y=y_clean * proj_3d[:, 1].std(),
z=z_clean * proj_3d[:, 2].std(),
mode='lines',
line=dict(color='crimson', width=4),
name='Trayectoria original (reescalada)'
))

fig.update_layout(
    title=f'Proyección 3D - {pca_3d.explained_variance_ratio_.sum()*100:.1f}% de la varianza total',
    scene=dict(
        xaxis_title='PC 1',
        yaxis_title='PC 2',
        zaxis_title='PC 3',
        xaxis=dict(range=[-15, 15]),
        yaxis=dict(range=[-7, 7]),
        zaxis=dict(range=[-7, 7])
    ),
    legend=dict(x=0.01, y=0.99),
    height=560
)
fig.show()

```

Unable to display output for mime type(s): text/html

La nube de puntos reconstruida en el espacio PCA sigue fielmente la hélice original. PCA ha separado la señal del ruido sin conocer a priori la geometría de la trayectoria.

i La idea central

PCA no sabe nada sobre hélices ni sobre física. Solo busca las direcciones de **máxima varianza** en los datos. El hecho de que esas direcciones coincidan con la geometría real de la trayectoria es una consecuencia de que **la señal real tiene más varianza que el ruido**.

Cuando $\text{SNR} = \sigma_{\text{señal}} / \sigma_{\text{ruido}} \gg 1$, los primeros componentes principales capturan la señal; los últimos, el ruido.

⚠ Limitación importante

PCA funciona bien aquí porque la hélice es una curva **linealizable** a través de sus coordenadas en el espacio de observaciones. Para estructuras genuinamente no lineales (manifolds curvos), se necesitan técnicas como **UMAP** o **t-SNE**.

112. Limitaciones del PCA

112.1. Linealidad

PCA busca direcciones de máxima varianza **lineales**. No puede capturar estructuras no lineales en los datos.

```
from sklearn.datasets import make_swiss_roll

X_swiss, t_swiss = make_swiss_roll(n_samples=800, noise=0.1, random_state=42)
X_swiss_std = StandardScaler().fit_transform(X_swiss)

pca_swiss = PCA(n_components=2)
scores_swiss = pca_swiss.fit_transform(X_swiss_std)

fig = make_subplots(1, 2,
                    subplot_titles=['Swiss Roll (3D original)',
                                    'Proyección PCA (2D) - estructura perdida'],
                    specs=[[{'type': 'scatter3d'}, {'type': 'scatter'}]])

fig.add_trace(go.Scatter3d(
    x=X_swiss[:,0], y=X_swiss[:,1], z=X_swiss[:,2],
    mode='markers',
    marker=dict(size=3, color=t_swiss, colorscale='Viridis', opacity=0.7),
    showlegend=False
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=scores_swiss[:,0], y=scores_swiss[:,1],
    mode='markers',
    marker=dict(size=4, color=t_swiss, colorscale='Viridis', opacity=0.7),
    showlegend=False
), row=1, col=2)

fig.update_layout(height=420,
                  annotations=[dict(
                      x=0.5, y=-0.08, xref='paper', yref='paper',
                      text="PCA colapsa el Swiss Roll: puntos lejanos en la variedad quedan superpuestos",
                      showarrow=False, font=dict(size=12, color='gray'), xanchor='center'
                  )])
fig.show()
```

Unable to display output for mime type(s): text/html

Este es un ejemplo clásico: el Swiss Roll es una variedad no lineal que PCA no puede “desenrollar”. Técnicas como **Isomap** o **UMAP** sí pueden recuperar la estructura subyacente.

112.2. Sensibilidad a outliers

PCA minimiza la varianza de la reconstrucción, que es equivalente a minimizar el error cuadrático medio. Por tanto, es **muy sensible a outliers**, ya que un solo punto alejado puede dominar la dirección del primer componente.

Unable to display output for mime type(s): text/html

La alternativa robusta es **Robust PCA** (Candès et al., 2011), que descompone $X = L + S$ donde L es la componente de bajo rango y S es una matriz dispersa de outliers.

112.3. PCA no captura estructura de clases

PCA es un método **no supervisado** — maximiza varianza global sin considerar etiquetas. Puede ocurrir que la dirección de máxima varianza no sea la que mejor separa las clases.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X_iris_std = StandardScaler().fit_transform(load_iris().data)
y_iris = load_iris().target
nombres_iris = load_iris().target_names

# PCA
pca_comp = PCA(n_components=2)
scores_pca = pca_comp.fit_transform(X_iris_std)

# LDA
lda = LinearDiscriminantAnalysis(n_components=2)
scores_lda = lda.fit_transform(X_iris_std, y_iris)

fig = make_subplots(1, 2,
                    subplot_titles=['PCA (no supervisado)', 'LDA (supervisado)'])

colors = px.colors.qualitative.Set1[:3]
for i, esp in enumerate(nombres_iris):
    mask = y_iris == i
    fig.add_trace(go.Scatter(
        x=scores_pca[mask, 0], y=scores_pca[mask, 1],
        mode='markers', marker=dict(color=colors[i], size=7, opacity=0.7),
        name=esp, legendgroup=esp
    ), row=1, col=1)
    fig.add_trace(go.Scatter(
        x=scores_lda[mask, 0], y=scores_lda[mask, 1],
```

```

        mode='markers', marker=dict(color=colors[i], size=7, opacity=0.7),
        name=esp, legendgroup=esp, showlegend=False
    ), row=1, col=2)

fig.update_layout(height=400,
    title="PCA vs LDA en Iris - LDA optimiza separación entre clases")
fig.show()

```

Unable to display output for mime type(s): text/html

112.4. Asunción de normalidad (implícita)

PCA es óptimo en el sentido de máxima varianza bajo **cualquier distribución**. Sin embargo, su interpretación probabilística (PCA probabilístico, Tipping & Bishop 1999) asume normalidad:

$$x = Wz + \mu + \varepsilon, \quad z \sim \mathcal{N}(0, I), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Para datos con distribuciones muy asimétricas o discretas, otras técnicas pueden ser más apropiadas.

113. Más allá del PCA: técnicas no lineales

Cuando los datos tienen estructuras no lineales, PCA falla. Las siguientes técnicas abordan esta limitación. Analizaremos dos de las más populares: **t-SNE** y **UMAP**.

Para esta sección emplearemos el MNIST dataset, un dataset de imágenes de dígitos escritos a mano que tiene una estructura no lineal compleja con las siguientes características:

- 70,000 imágenes de dígitos manuscritos (28x28 píxeles → 784 dimensiones)
- 10 clases (dígitos 0–9)
- Desafío clásico para reducción de dimensionalidad

Unable to display output for mime type(s): text/html

Notemos que si realizamos PCA sobre este dataset, los dígitos no se separan claramente en el espacio de componentes principales. Esto se debe a que la estructura de los datos es altamente no lineal: dígitos similares pueden estar cerca en el espacio original pero no necesariamente en el espacio PCA.

Unable to display output for mime type(s): text/html

A continuación, veremos cómo técnicas no lineales como t-SNE y UMAP pueden revelar la estructura subyacente de los dígitos, agrupando visualmente los dígitos similares y separando claramente las clases, a pesar de la complejidad no lineal de los datos.

113.1. t-SNE (t-distributed Stochastic Neighbor Embedding)

t-SNE (van der Maaten & Hinton, 2008) construye una distribución de probabilidad sobre pares de puntos en el espacio original y minimiza la divergencia KL con la distribución en el espacio reducido:

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

donde p_{ij} captura similitudes en el espacio original y q_{ij} en el espacio reducido usando una distribución t de Student.

Características:

- Excelente para visualización en 2D/3D
- Preserva estructura **local** (vecinos cercanos)
- **No preserva distancias globales** — la distancia entre clusters no es interpretable
- No determinista: ejecuciones distintas dan resultados distintos
- No produce una transformación reutilizable para nuevos datos
- Hiperparámetro clave: **perplexity** (típicamente 5–50)

La *perplexity* controla cuántos vecinos considera relevantes cada punto al construir la representación. Formalmente, está relacionada con la **entropía de una distribución de probabilidad local**:

$$\text{Perplexity} = 2^{H(P_i)}$$

donde P_i describe qué tan probable es que otros puntos sean vecinos de x_i . Ajustar la perplexity es crucial: valores muy bajos pueden fragmentar la estructura, mientras que valores muy altos pueden mezclar clusters distintos.

Unable to display output for mime type(s): text/html

113.2. UMAP (Uniform Manifold Approximation and Projection)

UMAP (McInnes et al., 2018) está fundamentado en la **teoría de categorías** y la geometría Riemanniana. El algoritmo construye una representación del manifold topológico de los datos y optimiza una representación en baja dimensión que preserve esa topología.

Matemáticamente, UMAP modela cada punto x_i como el centro de una bola geodésica de radio ρ_i (distancia al vecino más cercano), construyendo un grafo difuso ponderado con pesos:

$$w(x_i, x_j) = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

y simetrizado como:

$$B(x_i, x_j) = w(x_i, x_j) + w(x_j, x_i) - w(x_i, x_j) \cdot w(x_j, x_i)$$

La proyección minimiza la divergencia de entropía cruzada fuzzy entre el grafo original y el proyectado.

Ventajas sobre t-SNE:

Característica	t-SNE	UMAP
Velocidad	Lento ($O(n^2)$)	Rápido ($O(n^{1.14})$)
Estructura global	No preserva	Preserva mejor
Nuevos datos	No (sin extensión)	Sí (<code>transform()</code>)
Determinismo	No	Sí (con <code>random_state</code>)
Escalabilidad	Hasta ~10k puntos	Hasta millones

Unable to display output for mime type(s): text/html

Situación	Método recomendado
-----------	--------------------

113.3. Cuándo usar cada método

Situación	Método recomendado
Preprocesamiento antes de modelado	PCA
Visualización exploratoria	t-SNE o UMAP
Dataset grande (>50k puntos)	UMAP
Necesito transformar nuevos datos	PCA o UMAP
Interpretabilidad de componentes	PCA (o Sparse PCA)
Estructura no lineal compleja	UMAP
Análisis supervisado + reducción	LDA

La elección del método de reducción de dimensionalidad depende del objetivo del análisis y de la naturaleza de los datos. No existe una técnica universalmente superior: cada método enfatiza distintos aspectos de la estructura.

PCA es adecuado cuando se busca una representación lineal, interpretable y reutilizable, especialmente en contextos de preprocesamiento para modelos. En contraste, t-SNE y UMAP están diseñados principalmente para exploración visual, priorizando la preservación de relaciones locales, aunque sacrifican interpretabilidad y capacidad de generalización.

UMAP ofrece un equilibrio interesante entre estructura local y global, además de escalar mejor a datasets grandes. Finalmente, métodos como LDA incorporan información de clases, siendo más apropiados cuando el objetivo es discriminación supervisada.

En conjunto, la elección del método debe responder a la pregunta central del análisis: ¿buscamos interpretar, visualizar o modelar?

114. Resumen y conexiones

El PCA es el punto de partida del análisis multivariante moderno. Sus conexiones van más allá de la estadística descriptiva:

- **Álgebra lineal:** eigendescomposición, SVD, espacios vectoriales
- **Estadística:** modelo probabilístico gaussiano, máxima verosimilitud
- **Optimización:** problema de Rayleigh quotient, multiplicadores de Lagrange
- **Aprendizaje automático:** preprocesamiento, regularización, autoencoders lineales
- **Geometría diferencial:** UMAP extiende PCA al caso de variedades Riemannianas

La familia de métodos de reducción de dimensionalidad puede verse como un espectro continuo desde lo lineal (PCA) hasta lo fuertemente no lineal (t-SNE, UMAP), con un trade-off entre interpretabilidad, preservación de estructura global y escalabilidad.

115. Ejercicios propuestos

1. **PCA desde cero:** implementa PCA usando únicamente `numpy.linalg.svd` sin usar `sklearn`. Verifica que tus eigenvalores y eigenvectores coincidan con los de `sklearn.decomposition.PCA`.
 2. **Efecto de la estandarización:** aplica PCA al dataset Iris con y sin estandarización. ¿Cambian los componentes? ¿Cambia la varianza explicada? Justifica matemáticamente.
 3. **Reconstrucción:** dado un dataset X y su proyección PCA a k componentes, reconstruye la aproximación $\hat{X} = Z_k W_k^T + \mu$. Grafica el error de reconstrucción $\|X - \hat{X}\|_F$ en función de k .
 4. **Comparación t-SNE vs UMAP:** aplica ambas técnicas al dataset de dígitos MNIST (disponible en `sklearn.datasets.load_digits`). Compara visualmente la separación de clases y el tiempo de cómputo.
 5. **Regla de Kaiser:** ¿por qué la regla de Kaiser ($\lambda > 1$) solo aplica cuando los datos están estandarizados? Demuéstralo con un ejemplo donde viola la regla sin estandarizar.
-

115.1. Recursos recomendados: EDA multivariado

A continuación se listan libros y recursos clave para profundizar en análisis exploratorio de datos multivariado, reducción de dimensionalidad y visualización. Los pongo en mi orden de preferencia, pero todos son excelentes y complementarios.

115.1.1. Análisis multivariado (nivel intermedio)

- **Brunton & Kutz — *Data-Driven Science and Engineering***
<https://databookuw.com/>
→ Enfoque moderno: PCA, SVD, sistemas dinámicos. (Libre)
 - **Trevor Hastie, Robert Tibshirani, Jerome Friedman — *The Elements of Statistical Learning***
<https://web.stanford.edu/~hastie/ElemStatLearn/>
→ PCA, LDA, reducción de dimensionalidad y aprendizaje estadístico. (Libre)
 - **Gareth James et al. — *An Introduction to Statistical Learning***
<https://www.statlearning.com/>
→ Versión más accesible del anterior, incluye PCA y LDA. (Libre)
 - **Kevin Murphy — *Probabilistic Machine Learning: An Introduction***
<https://probml.github.io/pml-book/book1.html>
→ Enfoque moderno probabilístico, incluye PCA y modelos latentes. (Libre)
-

115.1.2. Geometría y fundamentos (nivel más teórico)

- Gilbert Strang — *Linear Algebra and Learning from Data*
<https://math.mit.edu/~gs/learningfromdata/>
→ Conexión entre álgebra lineal, PCA y datos. (Parcialmente libre)
-

115.1.3. Visualización y alta dimensión

- Claus O. Wilke — *Fundamentals of Data Visualization*
<https://clauswilke.com/dataviz/>
→ Principios de visualización (muy útil para EDA). (Libre)
- Distill.pub (artículos interactivos)
<https://distill.pub/>
→ Explicaciones visuales de t-SNE, embeddings y ML. (Libre)

116. Reducción de variables categóricas con ²

En capítulos anteriores se introdujo la correlación como una herramienta para analizar la relación entre variables numéricas, permitiendo cuantificar su dependencia mediante medidas como el coeficiente de Pearson.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Sin embargo, muchos problemas reales involucran variables categóricas, para las cuales estas herramientas no son aplicables. En este contexto surge la necesidad de medir la **asociación estadística**, es decir, evaluar si dos variables categóricas son independientes o están relacionadas.

Ejemplos de estas preguntas incluyen:

- ¿El **sexo biológico** está relacionado con la **preferencia de marca**?
- ¿El **nivel educativo** influye en el **partido político** votado?
- ¿El **tipo de tratamiento** médico afecta el **resultado clínico**?

Para ello se utilizan tablas de contingencia y la prueba ² de independencia, que permite detectar relaciones, junto con medidas como el V de Cramér para cuantificar su intensidad.

Más allá del análisis, estas herramientas son fundamentales en la **reducción de datos categóricos**, ya que permiten identificar variables redundantes y simplificar el conjunto de datos sin perder información relevante.

En este capítulo se desarrollan estos conceptos, mostrando cómo ² no solo detecta asociaciones, sino que también guía decisiones sobre selección y reducción de variables.

116.1. Principios estadísticos

Tip

Idea clave: independencia estadística

Dos variables categóricas X e Y son **independientes** si:

$$P(X = x_i, Y = y_j) = P(X = x_i) P(Y = y_j) \quad \forall i, j$$

Esto significa que conocer el valor de una variable **no cambia la probabilidad** de la otra. En términos prácticos, la información de Y no aporta nada adicional sobre X , y viceversa.

Para ilustrar esta idea, imaginemos el lanzamiento de dos dados justos. Sea X el resultado del primer dado y Y el resultado del segundo. Entonces:

$$P(X = 2) = \frac{1}{6}, \quad P(Y = 3) = \frac{1}{6}$$

Como los lanzamientos son independientes, la probabilidad conjunta se obtiene multiplicando las probabilidades marginales¹:

$$P(X = 2, Y = 3) = \frac{1}{6} \times \frac{1}{6} = \frac{1}{36}$$

Si esta factorización no se cumpliera, diríamos que las variables están relacionadas.

Para ilustrar esta idea, imaginemos el lanzamiento de dos dados justos. Sea X el resultado del primer dado y Y el resultado del segundo. Entonces:

$$P(X = 2) = \frac{1}{6}, \quad P(Y = 3) = \frac{1}{6}$$

Como los lanzamientos son independientes, la probabilidad conjunta se obtiene multiplicando las probabilidades marginales²:

$$P(X = 2, Y = 3) = \frac{1}{6} \times \frac{1}{6} = \frac{1}{36}$$

Si esta factorización no se cumpliera, diríamos que las variables están relacionadas.

Para visualizar la independencia de forma dinámica, simulamos múltiples lanzamientos de dos dados y observamos cómo evoluciona la probabilidad conjunta de obtener $X = 2$ e $Y = 3$ a medida que aumentamos el número de lanzamientos. En un escenario justo, esta probabilidad debería converger a $1/36 \approx 0.0278$.

```
import numpy as np
import pandas as pd
import plotly.express as px

np.random.seed(42)
n = 5000

# Dados justos (independientes)
X_justo = np.random.choice([1,2,3,4,5,6], size=n)
Y_justo = np.random.choice([1,2,3,4,5,6], size=n)

# Dados sesgados
probs = [0.1, 0.1, 0.1, 0.1, 0.1, 0.5]
X_sesgado = np.random.choice([1,2,3,4,5,6], size=n, p=probs)
Y_sesgado = np.random.choice([1,2,3,4,5,6], size=n, p=probs)

# Evento: X=2, Y=3
evento_justo = (X_justo == 2) & (Y_justo == 3)
```

¹Las probabilidades marginales corresponden a la distribución de cada variable por separado, ignorando la otra. En una tabla de contingencia, se obtienen a partir de los totales por fila o por columna.

²Las probabilidades marginales corresponden a la distribución de cada variable por separado, ignorando la otra. En una tabla de contingencia, se obtienen a partir de los totales por fila o por columna.

```

evento_sesgado = (X_sesgado == 2) & (Y_sesgado == 3)

# Frecuencia acumulada (convergencia)
prob_justo = np.cumsum(evento_justo) / np.arange(1, n+1)
prob_sesgado = np.cumsum(evento_sesgado) / np.arange(1, n+1)

df_plot = pd.DataFrame({
    "Tiros": np.arange(1, n+1),
    "Justo": prob_justo,
    "Sesgado": prob_sesgado
}).melt(id_vars="Tiros", var_name="Tipo", value_name="Probabilidad")

fig = px.line(
    df_plot,
    x="Tiros",
    y="Probabilidad",
    color="Tipo",
    title="Convergencia de la probabilidad conjunta P(X=2, Y=3)"
)

# Línea teórica 1/36
fig.add_hline(
    y=1/36,
    line_dash="dash",
    annotation_text="1/36 (valor teórico)",
    annotation_position="bottom right"
)

fig.update_layout(
    template="plotly_white",
    legend=dict(
        orientation="h",
        yanchor="top",
        y=-0.2,
        xanchor="center",
        x=0.5
    )
)
fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

En la práctica no observamos probabilidades, sino **conteos** organizados en tablas de contingencia³.

³Una tabla de contingencia es una matriz que resume la frecuencia de ocurrencia de combinaciones entre dos variables categóricas, mostrando cómo se distribuyen conjuntamente sus categorías.

Sea o_{ij} el número de observaciones donde $X = x_i$ y $Y = y_j$. Bajo el supuesto de independencia, la frecuencia esperada se define como:

$$e_{ij} = \frac{(\text{total fila}_i)(\text{total columna}_j)}{n}$$

Esta expresión se obtiene al estimar las probabilidades marginales como proporciones:

$$P(X = i) \approx \frac{n_i}{n}, \quad P(Y = j) \approx \frac{n_j}{n}$$

y asumir independencia:

$$P(X = i, Y = j) \approx \frac{n_i}{n} \cdot \frac{n_j}{n}$$

Multiplicando por el tamaño total de la muestra n , se obtiene la frecuencia esperada e_{ij} .

Así, e_{ij} representa lo que **esperaríamos observar si no hubiera relación**, mientras que o_{ij} refleja lo que realmente ocurre en los datos. La diferencia entre ambos constituye la base para detectar asociación.

Bajo este enfoque, la independencia funciona como un modelo de referencia: si los datos se alejan significativamente de él, surge la evidencia de una relación entre las variables. Esta lógica conduce directamente al estadístico χ^2 , configurado como una prueba de hipótesis. En ella, partimos de la hipótesis nula (H_0) de independencia y evaluamos si la discrepancia entre las frecuencias observadas (o_{ij}) y las esperadas (e_{ij}) es lo suficientemente grande como para rechazar dicha premisa.

En última instancia, la prueba χ^2 actúa como una balanza: cuantifica la desviación acumulada y permite determinar si las diferencias observadas son meras fluctuaciones del azar o si, por el contrario, representan una relación estructural y estadísticamente significativa entre las variables.

116.1.1. De independencia a frecuencias esperadas

Consideremos una encuesta realizada a $n = 100$ personas en la que se registran dos variables categóricas: el género y la bebida preferida.

- X : Género $\rightarrow \{\text{Hombre, Mujer}\}$
- Y : Bebida $\rightarrow \{\text{Café, Té}\}$

El objetivo es entender cómo el supuesto de **independencia** entre estas variables se refleja en una tabla de datos.

Al realizar un análisis exploratorio inicial, lo primero que observamos son los **marginales**, es decir, los totales de cada variable por separado:

- Total de hombres: **50** | Total de mujeres: **50**
- Total que prefieren café: **60** | Total que prefieren té: **40**


```

fig = go.Figure()

fig.add_trace(go.Bar(
    name="P(Género)",
    x=["Hombre", "Mujer"],
    y=[0.5, 0.5],
    marker_color=["#457b9d", "#e63946"],
    text=["50%", "50%"], textposition="outside",
    width=0.4
))

fig.add_trace(go.Bar(
    name="P(Bebida)",
    x=["Café", "Té"],
    y=[0.6, 0.4],
    marker_color=["#2a9d8f", "#e9c46a"],
    text=["60%", "40%"], textposition="outside",
    width=0.4
))

fig.update_layout(
    title="Probabilidades marginales - lo único que sabemos antes de asumir independencia",
    yaxis_title="Probabilidad",
    yaxis_range=[0, 0.85],
    # template="plotly_white",
    # barmode="group",
    # height=400
)
fig.show()

```

Unable to display output for mime type(s): text/html

Podemos estimar probabilidades directamente de frecuencias relativas:

$$P(X = \text{Hombre}) = \frac{50}{100} = 0.5, \quad P(Y = \text{Café}) = \frac{60}{100} = 0.6$$

Si fueran independientes, la probabilidad conjunta **se factorizaría** tomando el valor:

$$P(\text{Hombre}|\text{Café}) = P(\text{Hombre}) \times P(\text{Café}) = 0.5 \times 0.6 = 0.30$$

Para obtener la frecuencia esperada multiplicamos la probabilidad conjunta por n :

$$e_{11} = P(\text{Hombre}) \cdot P(\text{Café}) \cdot n = 0.5 \times 0.6 \times 100 = 30$$

Es decir, si no hubiera relación entre género y bebida, esperaríamos que **30 hombres** prefieran café.

De manera general, para cualquier celda (i, j) de la tabla la frecuencia esperada es:

$$e_{ij} = \frac{(\text{total fila}_i)(\text{total columna}_j)}{n}$$

Calculando la tabla esperada completa:

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import numpy as np

labels_x = ["Café", "Té"]
labels_y = ["Hombre", "Mujer"]
esp = np.array([[30, 20], [30, 20]])

fig = go.Figure(go.Heatmap(
    z=esp, x=labels_x, y=labels_y,
    colorscale="Blues", showscale=False,
    text=esp, texttemplate="%{text}",
    zmin=0, zmax=40
))

# Anotación de totales
for i, (row, rtot) in enumerate(zip(esp, [50, 50])):
    fig.add_annotation(x=2.35, y=labels_y[i], text=f"<b>{rtot}</b>",
                       showarrow=False, font=dict(size=13, color="#457b9d"),
                       xref="x", yref="y")
for j, (col, ctot) in enumerate(zip(labels_x, [60, 40])):
    fig.add_annotation(x=labels_x[j], y=-0.65, text=f"<b>{ctot}</b>",
                       showarrow=False, font=dict(size=13, color="#457b9d"),
                       xref="x", yref="y")

fig.update_layout(
    title="Tabla esperada bajo independencia - cada celda = (fila × col) / n",
    # template="plotly_white", height=600,
    # xaxis=dict(range=[-0.5, 2.6]),
    # yaxis=dict(range=[-1, 1.5]),
    # margin=dict(l=20, r=20, t=50, b=40)
)
fig.show()
```

Unable to display output for mime type(s): text/html

Así, la tabla de frecuencias esperadas puede interpretarse como un modelo de referencia que describe cómo se verían los datos si no existiera relación entre las variables. Si los datos observados o_{ij} se parecen a este modelo, no hay evidencia de asociación; si se alejan, surge la necesidad de explicarlo.

Matemáticamente, para cuantificar esta discrepancia entre lo observado (o_{ij}) y lo esperado (e_{ij}) se emplea el estadístico ², definido como:

$$\chi^2 = \sum_{i,j} \frac{(o_{ij} - e_{ij})^2}{e_{ij}}$$

Apliquemoslo a nuestro ejemplo. Recordemos que bajo independencia se espera $e_{11} = 30$ para la celda $(i = 1, j = 1)$, pero supongamos que se observa $o_{11} = 40$. La contribución de esta celda al estadístico es:

$$\frac{(40 - 30)^2}{30} = \frac{100}{30} \approx 3.33$$

De manera análoga, si en otra celda se tiene $e_{12} = 20$ y $o_{12} = 10$, su contribución es:

$$\frac{(10 - 20)^2}{20} = \frac{100}{20} = 5$$

El estadístico χ^2 se obtiene sumando estas contribuciones sobre todas las celdas. En este sentido, χ^2 puede interpretarse como una medida global de la **distancia** entre lo observado y lo esperado: cuanto mayores sean las diferencias $o_{ij} - e_{ij}$, mayor será el valor de χ^2 .

Valores pequeños indican que las discrepancias pueden atribuirse al azar, mientras que valores grandes sugieren una desviación significativa respecto al modelo de independencia.

Veamos ahora cómo evoluciona χ^2 a medida que variamos una de las celdas, manteniendo los marginales fijos.

```
import numpy as np
import plotly.graph_objects as go
from scipy.stats import chi2_contingency

k_vals = np.arange(30, 56, 1)
chi2_vals = []

for k in k_vals:
    tabla = np.array([[k, 50 - k], [60 - k, 50 - (60 - k)]])
    if tabla.min() >= 0:
        chi2_v, *_ = chi2_contingency(tabla)
        chi2_vals.append(chi2_v)
    else:
        chi2_vals.append(np.nan)

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=k_vals, y=chi2_vals,
    mode="lines+markers",
    line=dict(color="#457b9d", width=2.5),
    marker=dict(size=5),
    name="χ²"
))

fig.add_vline(x=30, line_dash="dash", line_color="#2a9d8f",
```

```

        annotation_text="k=30: independencia exacta ( $\chi^2=0$ )",
        annotation_position="top right",
        annotation_font_color="#2a9d8f")

fig.add_hline(y=3.841, line_dash="dot", line_color="#e63946",
              annotation_text="Valor crítico  $\alpha=0.05$  (gl=1)",
              annotation_position="bottom right",
              annotation_font_color="#e63946")

fig.update_layout(
    title="  $\chi^2$  crece mientras más se alejan los datos de la independencia",
    xaxis_title="Hombres que toman café (k) - marginales fijos en 50/50 y 60/40",
    yaxis_title="Estadístico  $\chi^2$ ",
)
fig.show()

```

Unable to display output for mime type(s): text/html

Para un número de hombres que toman café $k = 30$, las frecuencias observadas coinciden con las esperadas y se tiene $\chi^2 = 0$. A medida que k se aleja de este valor, las diferencias entre o_{ij} y e_{ij} aumentan, y χ^2 crece en consecuencia.

Este comportamiento refleja que χ^2 cuantifica qué tan lejos están los datos del modelo de independencia: valores pequeños indican cercanía, mientras que valores grandes sugieren una desviación significativa. La línea horizontal marca el valor crítico, a partir del cual la discrepancia ya no puede atribuirse al azar.

116.2. Definición: la prueba χ^2

La prueba χ^2 mide qué tan lejos están las frecuencias observadas de lo que esperaríamos si no hubiera relación entre las variables.

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

donde O_{ij} es la frecuencia observada y E_{ij} la frecuencia esperada bajo independencia. La suma se realiza sobre todas las celdas de la tabla de contingencia.

La división entre E_{ij} permite **normalizar la discrepancia**, de modo que se mide el error relativo y no el absoluto. Por ejemplo, una diferencia de 10 es relevante si se esperaban 12, pero poco significativa si se esperaban 1000.

Cada término puede interpretarse como la contribución de una celda al estadístico, y es aproximadamente equivalente a un valor normal estandarizado al cuadrado:

$$\frac{(O_{ij} - E_{ij})^2}{E_{ij}} \approx Z_{ij}^2$$

Bajo la hipótesis nula H_0 de independencia, estas desviaciones se deben únicamente al azar, y el estadístico ² puede interpretarse como la suma de cantidades aproximadamente normales al cuadrado, lo que explica que siga una distribución ².

Finalmente, la forma de esta distribución depende de los **grados de libertad**, dados por:

$$(r - 1)(c - 1)$$

los cuales reflejan cuántas celdas pueden variar libremente una vez que los totales por fila y columna están fijados.

Para entender los grados de libertad en una tabla 2×2 , consideremos una tabla con marginales fijos. Fijamos libremente **una sola celda** (p. ej. $X, A = 30$); el resto queda determinado por los totales:

	A	B	Total
X	30 ← libre	20 ← fila X	50
Y	30 ← col A	20 ← ambos	50
Total	60	40	100

A partir de una sola elección, toda la tabla queda fijada: en una tabla 2×2 existe **un solo grado de libertad**.

En general, los grados de libertad $(r - 1)(c - 1)$ representan cuántos valores pueden elegirse libremente antes de que los marginales determinen el resto.

116.2.1. La distribución ²

Recordemos que cada celda de la tabla aporta un término $\approx Z_{ij}^2$ al estadístico. Sumar cuadrados de normales es precisamente cómo nace la distribución ²: si lanzas una moneda justa muchas veces y mides qué tan lejos cayeron los resultados de lo esperado, esa acumulación de discrepancias al cuadrado sigue una distribución ².

Esto explica directamente sus propiedades:

- **Solo toma valores positivos** — acumula cuadrados, así que nunca puede ser negativa. Un ² = 0 significaría que observado y esperado coinciden perfectamente en cada celda.
- **Se desplaza a la derecha con más g.l.** — cada grado de libertad adicional suma un cuadrado más, por lo que el total tiende a crecer. De hecho, la media es exactamente igual a los g.l.
- **Es asimétrica con pocos g.l.** — cuando solo sumas uno o dos cuadrados, los valores pequeños dominan y la cola derecha es larga. Conforme sumas más términos, por el teorema central del límite, la distribución se va pareciendo a una normal.
- **Es aditiva** — si tienes dos tablas independientes con sus propios ², puedes sumar los estadísticos y los g.l. directamente. Esto es útil, por ejemplo, para combinar resultados de varios estudios.

Con esto claro, la forma de la distribución varía según los grados de libertad:

```

import numpy as np
import plotly.graph_objects as go
from scipy.stats import chi2

x = np.linspace(0, 30, 500)

fig = go.Figure()
for df_, color in zip([1, 2, 4, 6, 9],
                      ["#e63946", "#457b9d", "#2a9d8f", "#e9c46a", "#f4a261"]):
    fig.add_trace(go.Scatter(
        x=x, y=chi2.pdf(x, df_),
        mode="lines", name=f"gl = {df_}",
        line=dict(color=color, width=2.5)
    ))

fig.update_layout(
    title="Distribución  $\chi^2$  para distintos grados de libertad",
    xaxis_title="x", yaxis_title="Densidad",
    legend_title="g.l.",
    height=400
)
fig.show()

```

Unable to display output for mime type(s): text/html

Observa en la gráfica cómo se manifiestan estas propiedades: con $gl = 1$ la masa está concentrada cerca de cero y cae abruptamente; conforme aumentan los g.l. la distribución se desplaza a la derecha, la cola se alarga y la curva se va achatando. Esto tiene una consecuencia práctica directa: **un mismo valor de χ^2 observado corresponde a p-values muy distintos dependiendo de los g.l.**, por lo que nunca debe interpretarse el estadístico sin conocer los grados de libertad de la prueba.

Más g.l. = más formas en las que los datos pueden variar

116.2.2. Decisión estadística: p-value

Una vez que calculamos el estadístico χ^2 , necesitamos saber qué tan inusual es ese valor. Para eso usamos la distribución χ^2 como referencia: nos dice cómo se distribuirían los estadísticos si repitiéramos el muestreo muchas veces bajo un mundo donde H_0 fuera cierta, es decir, donde no hubiera ninguna relación entre las variables.

Intuitivamente, la curva representa todos los valores de χ^2 que **cabría esperar por puro azar**. Los valores pequeños son frecuentes (poca discrepancia, nada raro), mientras que los valores grandes son raros (mucha discrepancia, difícil de explicar solo por azar). El p-value es simplemente el **área a la derecha** de nuestro valor observado:

$$p = P(\chi^2 \geq \chi_{\text{obs}}^2 \mid H_0)$$

Esa área responde a la pregunta: *¿qué fracción de los resultados posibles bajo H_0 serían tan extremos o más que el nuestro?* Un área grande significa que nuestro resultado es común bajo H_0 — no hay razón

para dudar de ella. Un área pequeña significa que nuestro resultado sería muy raro si H_0 fuera cierta — evidencia en su contra.

```
from scipy.stats import chi2
import numpy as np
import plotly.graph_objects as go

gl = 4
chi2_obs = 11.5
x = np.linspace(0, 25, 500)

fig = go.Figure()

# Distribución completa
fig.add_trace(go.Scatter(
    x=x, y=chi2.pdf(x, gl),
    fill="tozeroy", fillcolor="rgba(69,123,157,0.2)",
    line=dict(color="#457b9d", width=2),
    name="Distribución  $\chi^2(4)$ "
))

# Área de rechazo
x_tail = x[x >= chi2_obs]
fig.add_trace(go.Scatter(
    x=np.concatenate([[chi2_obs], x_tail, [x_tail[-1]]]),
    y=np.concatenate([[0], chi2.pdf(x_tail, gl), [0]]),
    fill="toself", fillcolor="rgba(230,57,70,0.5)",
    line=dict(color="#e63946", width=2),
    name=f"p-value = {chi2.sf(chi2_obs, gl):.4f}"
))

fig.add_vline(x=chi2_obs, line_dash="dash", line_color="#e63946",
    annotation_text=f" $\alpha$ ={chi2_obs}",
    annotation_position="top right")

fig.update_layout(
    title="Región de rechazo (  $\alpha$  = 0.05, gl = 4)",
    xaxis_title=" $\chi^2$ ", yaxis_title="Densidad",
)
fig.show()
```

Unable to display output for mime type(s): text/html

El área roja es el p-value: la porción de la distribución que queda a la derecha de nuestro χ^2 observado. Cuanto más a la derecha cae el valor, más pequeña es esa área y más difícil de explicar el resultado por azar.

La decisión se toma comparando esa área con un umbral predefinido (comúnmente $\alpha = 0.05$):

- Si $p\text{-value} < 0.05 \rightarrow$ el área es tan pequeña que rechazamos H_0

- Si $p\text{-value} \geq 0.05 \rightarrow$ el área es suficientemente grande; no hay evidencia para rechazar H_0

Aquí:

- $p \approx 0.0215 < 0.05$

La desviación observada es **poco probable bajo independencia**

Sin embargo, saber que existe una relación no dice nada sobre su magnitud. Antes de interpretar cualquier resultado, conviene recordar que la prueba tiene supuestos que deben cumplirse:

! Importante

La prueba **NO es válida** si se violan estos supuestos.

1. **Independencia** de las observaciones — cada persona o unidad debe aparecer una sola vez en la tabla. Si el mismo individuo aporta múltiples filas, las celdas ya no son comparables entre sí.
2. **Tamaño de muestra** suficiente: $n \geq 20$ — la distribución χ^2 es una aproximación que solo funciona bien con muestras grandes. Con n pequeño, el estadístico puede no seguir esa distribución y el $p\text{-value}$ sería poco confiable.
3. **Frecuencias esperadas** ≥ 5 en al menos el 80% de las celdas — si una celda tiene muy pocos casos esperados, su término $\frac{(O-E)^2}{E}$ se vuelve inestable: una pequeña diferencia absoluta produce un valor enorme solo por dividir entre un número muy pequeño.
4. **Ninguna celda** con $E_{ij} = 0$ — una frecuencia esperada de cero hace la división imposible. Ocurre cuando alguna combinación de categorías es estructuralmente imposible en los datos.

116.2.3. Fuerza de asociación: V de Cramér

Un χ^2 significativo solo dice *que algo existe* — no dice *qué tan grande es*. Con muestras muy grandes, hasta una relación trivial produce un χ^2 enorme y un $p\text{-value}$ pequeñísimo. La V de Cramér corrige esto: divide el estadístico entre el tamaño de muestra y lo escala según el tamaño de la tabla, para que el resultado siempre quede entre 0 y 1 independientemente de n .

$$V = \sqrt{\frac{\chi^2/n}{\min(r-1, c-1)}} \in [0, 1]$$

Intuitivamente, $V = 0$ significa que conocer la fila no te dice nada sobre la columna; $V = 1$ significa que una variable determina completamente a la otra. En la práctica:

V	Interpretación
0.00 – 0.10	Asociación negligible
0.10 – 0.20	Asociación débil
0.20 – 0.40	Asociación moderada
> 0.40	Asociación fuerte

La siguiente gráfica simula tablas de contingencia 2×2 con distintos niveles de V para que puedas ver cómo luce la asociación en los datos reales:


```

import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Tablas representativas para cada nivel de V
# Cada tabla está construida para producir aproximadamente el V indicado
tables = {
    "Negligible (V 0.05)": np.array([[26, 24], [24, 26]]),
    "Débil (V 0.15)": np.array([[35, 15], [25, 25]]),
    "Moderada (V 0.30)": np.array([[40, 10], [20, 30]]),
    "Fuerte (V 0.55)": np.array([[45, 5], [10, 40]]),
}

colors = {
    "Negligible (V 0.05)": ["#d9f0f7", "#a8d8ea"],
    "Débil (V 0.15)": ["#c8e6c9", "#66bb6a"],
    "Moderada (V 0.30)": ["#fff3cd", "#ffc107"],
    "Fuerte (V 0.55)": ["#ffd6d6", "#e63946"],
}

fig = make_subplots(
    rows=1, cols=4,
    subplot_titles=list(tables.keys()),
    horizontal_spacing=0.08
)

for col_idx, (label, table) in enumerate(tables.items(), start=1):
    n = table.sum()
    total = table / n * 100 # porcentajes

    cats_col = ["A", "B"]
    cats_row = ["X", "Y"]

    for r_idx, row_label in enumerate(cats_row):
        fig.add_trace(go.Bar(
            name=row_label,
            x=cats_col,
            y=total[r_idx],
            marker_color=colors[label][r_idx],
            text=[f"{v:.0f}%" for v in total[r_idx]],
            textposition="inside",
            showlegend=(col_idx == 1),
        ), row=1, col=col_idx)

fig.update_layout(
    barmode="stack",
    legend_title="Categoría",
)

```

```
fig.update_yaxes(range=[0, 100], ticksuffix="%")
fig.show()
```

Unable to display output for mime type(s): text/html

Observa cómo cambia la forma de las barras: cuando la asociación es negligible, las proporciones de X e Y son casi idénticas en A y B — conocer la columna no te dice nada sobre la fila. Conforme V crece, las barras se vuelven más disparejas: en la asociación fuerte, A está dominada por X y B por Y, lo que indica que las dos variables se mueven juntas de manera sistemática.

Un ² significativo sin una V de Cramér grande puede ser estadísticamente real pero prácticamente irrelevante.

116.3. Ejemplo: Tabaquismo × Ejercicio

Para ilustrar todo lo anterior, analizamos la relación entre el hábito de fumar y la frecuencia de ejercicio en una muestra de 300 personas. La pregunta es simple: ¿las personas que fuman más tienden a ejercitarse menos?

```
import pandas as pd

np.random.seed(42)
n = 300

fumar_base = np.random.choice([0, 1, 2, 3], size=n, p=[0.35, 0.30, 0.20, 0.15])
ejercicio = []
for f in fumar_base:
    if f == 0:
        ejercicio.append(np.random.choice(["Nunca", "Ocasional", "Frecuente"], p=[0.15, 0.35, 0.5]))
    elif f == 1:
        ejercicio.append(np.random.choice(["Nunca", "Ocasional", "Frecuente"], p=[0.30, 0.45, 0.25]))
    elif f == 2:
        ejercicio.append(np.random.choice(["Nunca", "Ocasional", "Frecuente"], p=[0.55, 0.35, 0.1]))
    else:
        ejercicio.append(np.random.choice(["Nunca", "Ocasional", "Frecuente"], p=[0.70, 0.25, 0.05]))

fumar_labels = {0: "No fuma", 1: "Ocasional", 2: "Regular", 3: "Exceso"}
df = pd.DataFrame({
    "Fumar": [fumar_labels[f] for f in fumar_base],
    "Ejercicio": ejercicio
})

tabla = pd.crosstab(df["Fumar"], df["Ejercicio"])
tabla
```

Ejercicio Fumar	Frecuente	Nunca	Ocasional
Exceso	4	38	6
No fuma	65	18	31
Ocasional	26	22	35
Regular	5	34	16

Ya en la tabla se percibe un patrón: los no fumadores tienden a aparecer más en la columna “Frecuente”, mientras que los fumadores en exceso se concentran en “Nunca”. Pero ¿es esta diferencia suficientemente grande como para no atribuirla al azar?

```
from scipy.stats import chi2_contingency

chi2_val, p_val, dof, expected = chi2_contingency(tabla)

print(f" $\chi^2$  = {chi2_val:.4f}")
print(f"p-value = {p_val:.6f}")
print(f"Grados de libertad = {dof}")
print(f"\nFrecuencias esperadas (mínima = {expected.min():.2f})")
print(f"¿Todas 5? {' Sí' if expected.min() >= 5 else ' No - verificar supuesto'}")
```

χ^2 = 95.5706

p-value = 0.000000

Grados de libertad = 6

Frecuencias esperadas (mínima = 14.08)

¿Todas 5? Sí

El estadístico χ^2 resulta muy alto y el p-value es prácticamente cero: la probabilidad de observar estas diferencias en un mundo donde fumar y ejercicio fueran independientes es negligible. Además, todas las frecuencias esperadas superan 5, por lo que los supuestos de la prueba se cumplen.

Pero, ¿qué tan grande es la relación?

Fuerza de asociación: V de Cramér

```
n_total = tabla.values.sum()
min_dim = min(tabla.shape[0] - 1, tabla.shape[1] - 1)
cramer_v = np.sqrt(chi2_val / (n_total * min_dim))

print(f"V de Cramér = {cramer_v:.4f}")
if cramer_v < 0.10:
    interp = "Asociación negligible"
elif cramer_v < 0.20:
    interp = "Asociación débil"
elif cramer_v < 0.40:
```

```

    interp = "Asociación moderada"
else:
    interp = "Asociación fuerte"
print(f"Interpretación: {interp}")

```

V de Cramér = 0.3991

Interpretación: Asociación moderada

La V de Cramér confirma que no se trata solo de un efecto estadístico: la asociación es **moderada-fuerte**, lo que significa que el hábito de fumar explica una porción relevante de la variación en el nivel de ejercicio.

Visualización: Observadas vs Esperadas

Para entender de dónde viene el ², comparamos lo que vimos en los datos con lo que habríamos esperado si no hubiera ninguna relación. Si las dos tablas fueran idénticas, el ² sería cero.

```

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

obs = tabla.values
exp = expected

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Frecuencias Observadas", "Frecuencias Esperadas")
)

fig.add_trace(go.Heatmap(
    z=obs, x=tabla.columns.tolist(), y=tabla.index.tolist(),
    colorscale="Blues", text=obs, texttemplate="%{text}",
    showscale=False, name="Observadas"
), row=1, col=1)

fig.add_trace(go.Heatmap(
    z=np.round(exp, 1), x=tabla.columns.tolist(), y=tabla.index.tolist(),
    colorscale="Oranges", text=np.round(exp, 1), texttemplate="%{text}",
    showscale=False, name="Esperadas"
), row=1, col=2)

fig.update_layout(
    title="Comparación: Observadas vs Esperadas bajo H ",
    template="plotly_white", height=420
)
fig.show()

```

Unable to display output for mime type(s): text/html

Las diferencias entre ambas tablas son visibles a simple vista: los fumadores en exceso ejercen mucho menos de lo que se esperaría si fumar y ejercicio fueran independientes, y los no fumadores ejercen mucho más. Esas discrepancias son las que inflan el χ^2 .

Residuos estandarizados: ¿qué celdas explican el χ^2 ?

El χ^2 global nos dice que *algo* pasa, pero no *dónde*. Los residuos estandarizados responden esa pregunta: miden cuánto se desvía cada celda de lo esperado, en unidades comparables. Un valor por encima de 2 en términos absolutos indica que esa celda contribuye de forma significativa al estadístico.

$$r_{ij} = \frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}(1 - p_{i.})(1 - p_{.j})}}$$

```
row_sums = obs.sum(axis=1, keepdims=True)
col_sums = obs.sum(axis=0, keepdims=True)
n = obs.sum()

pi = row_sums / n
pj = col_sums / n
std_res = (obs - exp) / np.sqrt(exp * (1 - pi) * (1 - pj))

fig = px.imshow(
    np.round(std_res, 2),
    x=tabla.columns.tolist(),
    y=tabla.index.tolist(),
    color_continuous_scale="RdBu_r",
    color_continuous_midpoint=0,
    text_auto=True,
    title="Residuos estandarizados (|r| > 2 → desviación significativa)"
)
fig.update_layout(template="plotly_white", height=420)
fig.show()
```

Unable to display output for mime type(s): text/html

El mapa de calor lo hace inmediato: el rojo intenso en *Exceso* × *Nunca* indica que hay muchos más fumadores sedentarios de lo esperado, mientras que el azul en *No fuma* × *Frecuente* señala que los no fumadores se ejercitan con una frecuencia mucho mayor a la que predice la independencia. Esas dos celdas son las principales responsables del χ^2 elevado.

En conjunto, la prueba confirma que el hábito de fumar y el nivel de ejercicio **no son independientes**: a mayor consumo de tabaco, menor frecuencia de actividad física, y esta relación es lo suficientemente fuerte como para ser prácticamente relevante, no solo estadísticamente significativa.

116.4. ² y reducción de datos

Hasta ahora usamos ² para detectar **dependencia** entre variables categóricas y evaluar su significancia. Pero hay una consecuencia práctica que vale la pena explorar: si dos variables están fuertemente asociadas, ¿tiene sentido incluir ambas en un modelo?

116.4.1. Redundancia en variables categóricas

Imagina que tienes dos variables en tu base de datos: *nivel de actividad física* y *frecuencia de visitas al gimnasio*. Intuitivamente, quien va mucho al gimnasio probablemente también reporta alta actividad física — y viceversa. Incluir ambas en un modelo es como preguntarle dos veces lo mismo a los datos.

Formalmente, dos variables son redundantes cuando no son independientes:

$$P(X, Y) \neq P(X)P(Y) \quad (\text{dependencia fuerte})$$

Esto significa que conocer X ya te da información sobre Y . Cuanto más fuerte es la asociación, más se solapan los contenidos de ambas variables y menos aporta tener las dos.

En la práctica, mantener variables redundantes tiene costos concretos:

- **Ruido** — si una variable ya está capturada por otra, la versión adicional solo introduce variabilidad innecesaria en las estimaciones.
- **Sobreajuste** — el modelo aprende patrones espurios que no generalizan a datos nuevos.
- **Interpretabilidad** — es más difícil entender qué está explicando cada variable cuando varias dicen esencialmente lo mismo.

La lógica es la misma que en correlación entre variables numéricas, pero aplicada al mundo categórico: así como no incluirías altura y talla de zapato como predictores independientes, tampoco deberías incluir dos variables categóricas que midan casi lo mismo.

² + V de Cramér permiten detectar esta redundancia: un p-value significativo señala que la dependencia existe; una V alta indica que es lo suficientemente fuerte como para considerar eliminar una de las dos variables.

116.5. Ejemplo: Encuesta de satisfacción — Aerolínea

Una aerolínea quiere predecir qué tan satisfechos quedan sus pasajeros. Tiene 7 variables disponibles, pero no todas son igualmente útiles — algunas explican la satisfacción, otras son irrelevantes, y algunas repiten información que ya captura otra variable. El objetivo es quedarse solo con las que realmente aportan.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

np.random.seed(42)
```

```

n = 1000

clase = np.random.choice(["Económica","Business","Primera"],
                          size=n, p=[0.65, 0.25, 0.10])

tipo_map = {
    "Económica": [0.30, 0.70],
    "Business":  [0.85, 0.15],
    "Primera":   [0.70, 0.30]
}
tipo = np.array([np.random.choice(["Negocio","Vacaciones"],
                                   p=tipo_map[c]) for c in clase])

freq_map = {
    "Económica": [0.15, 0.45, 0.40],
    "Business":  [0.65, 0.30, 0.05],
    "Primera":   [0.75, 0.20, 0.05]
}
frecuencia = np.array([np.random.choice(["Frecuente","Ocasional","Primera vez"],
                                          p=freq_map[c]) for c in clase])

retraso_map = {
    "Negocio":    [0.50, 0.30, 0.20],
    "Vacaciones": [0.45, 0.35, 0.20]
}
retraso = np.array([np.random.choice(["Sin retraso","Leve","Grave"],
                                       p=retraso_map[t]) for t in tipo])

satisf = []
for c, r in zip(clase, retraso):
    if c == "Primera" and r == "Sin retraso":
        satisf.append(np.random.choice(["Baja","Media","Alta"], p=[0.05,0.15,0.80]))
    elif c == "Business" and r != "Grave":
        satisf.append(np.random.choice(["Baja","Media","Alta"], p=[0.10,0.35,0.55]))
    elif c == "Económica" and r == "Grave":
        satisf.append(np.random.choice(["Baja","Media","Alta"], p=[0.60,0.30,0.10]))
    elif c == "Económica" and r == "Sin retraso":
        satisf.append(np.random.choice(["Baja","Media","Alta"], p=[0.20,0.45,0.35]))
    else:
        satisf.append(np.random.choice(["Baja","Media","Alta"], p=[0.30,0.40,0.30]))
satisf = np.array(satisf)

asiento = np.random.choice(["Ventana","Pasillo","Centro"],
                            size=n, p=[0.45, 0.40, 0.15])

def p_norm():
    p = np.array([0.50, 0.50]) + np.random.uniform(-0.03, 0.03, 2)
    return p / p.sum()

```

```

momento = np.array([np.random.choice(["Anticipado", "Último momento"],
                                     p=p_norm()) for _ in range(n)])

df = pd.DataFrame({
    "Clase": clase, "Tipo_viaje": tipo, "Frecuencia": frecuencia,
    "Retraso": retraso, "Satisfacción": satisf,
    "Asiento": asiento, "Momento_compra": momento
})

```

Paso 1: Explorar los datos

Antes de correr cualquier prueba, miramos la tabla. Ya a simple vista aparecen patrones: los pasajeros de Primera clase con vuelos puntuales tienden a reportar satisfacción alta, mientras que los de Económica con retrasos graves se concentran en satisfacción baja. Pero la intuición no es suficiente — necesitamos cuantificar.

Clase	Tipo_viaje	Frecuencia	Retraso	Satisfacción	Asiento	Momento_compra
Económica	Negocio	Ocasional	Leve	Media	Ventana	Último momento
Primera	Negocio	Frecuente	Leve	Alta	Pasillo	Último momento
Business	Vacaciones	Ocasional	Sin retraso	Alta	Centro	Último momento
Económica	Vacaciones	Ocasional	Leve	Baja	Ventana	Último momento
Económica	Vacaciones	Ocasional	Leve	Baja	Centro	Anticipado
Económica	Vacaciones	Primera vez	Grave	Baja	Ventana	Último momento
Económica	Vacaciones	Ocasional	Grave	Baja	Pasillo	Anticipado
Business	Negocio	Ocasional	Sin retraso	Media	Pasillo	Anticipado

Paso 2: ¿Qué tan asociada está cada variable con Satisfacción?

Calculamos ² y V de Cramér entre cada variable predictora y la satisfacción. Esto nos da un ranking objetivo de utilidad.

```

predictoras = ["Clase", "Tipo_viaje", "Frecuencia", "Retraso", "Asiento", "Momento_compra"]
resultados = []

for var in predictoras:
    tabla = pd.crosstab(df[var], df["Satisfacción"])
    chi2_v, p_v, dof_v, exp_v = chi2_contingency(tabla)
    n_tot = tabla.values.sum()
    min_dim = min(tabla.shape[0]-1, tabla.shape[1]-1)
    cramer = np.sqrt(chi2_v / (n_tot * min_dim))

```



```

if cramer > 0.3:
    fuerza = "Fuerte"
elif cramer > 0.1:
    fuerza = "Moderada"
else:
    fuerza = "Débil"

resultados.append({
    "Variable": var, "²": round(chi2_v,2),
    "p-value": round(p_v, 4), "V de Cramér": round(cramer, 3),
    "Fuerza": fuerza
})

res_df = pd.DataFrame(resultados).sort_values("V de Cramér", ascending=False)

```

Variable	²	p-value	V de Cramér	Fuerza
Retraso	108.79	0.0000	0.233	Moderada
Clase	74.03	0.0000	0.192	Moderada
Tipo_viaje	9.79	0.0075	0.099	Débil
Frecuencia	16.60	0.0023	0.091	Débil
Asiento	2.05	0.7265	0.032	Débil
Momento_compra	0.60	0.7401	0.025	Débil

La tabla cuenta una historia clara: *Retraso* y *Clase* son las variables más informativas. *Tipo_viaje* y *Frecuencia* tienen una asociación estadísticamente significativa pero débil en magnitud. Y *Asiento* y *Momento_compra* tienen p-values altos y V cercana a cero — son prácticamente independientes de la satisfacción, ruido puro.

Paso 3: Visualizar fuerza de asociación con Satisfacción

```

colores = {"Fuerte": "#e63946", "Moderada": "#e9c46a", "Débil": "#2a9d8f"}

fig = go.Figure()
fig.add_trace(go.Bar(
    x=res_df["Variable"],
    y=res_df["V de Cramér"],
    marker_color=[colores[f] for f in res_df["Fuerza"]],
    text=res_df["V de Cramér"].astype(str),
    textposition="outside"
))

fig.add_hline(y=0.1, line_dash="dot", line_color="#e9c46a",
    annotation_text="Umbral débil/moderada (0.1)",
    annotation_position="top left",
    annotation_font_color="#b5850a")
fig.add_hline(y=0.3, line_dash="dot", line_color="#e63946",

```

```

        annotation_text="Umbral moderada/fuerte (0.3)",
        annotation_position="top left",
        annotation_font_color="#e63946")

fig.update_layout(
    title="V de Cramér de cada variable vs Satisfacción",
    yaxis_title="V de Cramér", yaxis_range=[0, 1.0],
    template="plotly_white", height=430, showlegend=False
)
fig.show()

```

Unable to display output for mime type(s): text/html

La gráfica hace evidente la separación: hay un grupo de variables que claramente están por encima del umbral y otro que apenas despegas del cero. Las dos últimas barras (*Asiento* y *Momento_compra*) son candidatas directas a eliminación.

Paso 4: ¿Hay redundancia entre las predictoras útiles?

Eliminar ruido es fácil. Lo más delicado es detectar si entre las variables útiles alguna repite información de otra. Para eso calculamos la V de Cramér entre todas las predictoras entre sí — no contra la satisfacción, sino entre ellas.

```

vars_utiles = ["Clase", "Tipo_viaje", "Frecuencia", "Retraso"]
m = len(vars_utiles)
matriz = pd.DataFrame(np.zeros((m, m)), index=vars_utiles, columns=vars_utiles)

for i, v1 in enumerate(vars_utiles):
    for j, v2 in enumerate(vars_utiles):
        if i == j:
            matriz.loc[v1, v2] = 1.0
        elif i < j:
            t = pd.crosstab(df[v1], df[v2])
            chi2_v, _, _, _ = chi2_contingency(t)
            n_tot = t.values.sum()
            min_d = min(t.shape[0]-1, t.shape[1]-1)
            v = round(np.sqrt(chi2_v / (n_tot * min_d)), 3)
            matriz.loc[v1, v2] = v
            matriz.loc[v2, v1] = v

fig = px.imshow(
    matriz.values,
    x=vars_utiles, y=vars_utiles,
    color_continuous_scale="RdYlGn_r",
    zmin=0, zmax=1,
    text_auto=True,
    title="V de Cramér entre predictoras - ¿hay redundancia interna?"
)

```

```
fig.update_layout(
    template="plotly_white", height=400,
    coloraxis_colorbar_title="V de Cramér"
)
fig.show()
```

Unable to display output for mime type(s): text/html

La diagonal siempre es 1 (cada variable es perfectamente dependiente de sí misma). Lo que nos interesa son los valores fuera de ella. Aquí *Clase* y *Frecuencia* muestran una asociación moderada — tiene sentido, porque los pasajeros de Business y Primera tienden a volar con más regularidad. Sin embargo, ningún par alcanza valores tan altos como para justificar eliminar una variable por redundancia con otra. Podemos conservarlas todas con cierta tranquilidad.

Paso 5: ¿Cómo luce la satisfacción en las variables clave?

Antes de tomar decisiones finales, vale la pena ver el patrón real en los datos. Si *Clase* y *Retraso* realmente explican la satisfacción, deberíamos ver distribuciones muy distintas entre sus categorías.

```
fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Satisfacción por Clase", "Satisfacción por Retraso")
)

orden_sat = ["Baja", "Media", "Alta"]
colores_sat = {"Baja": "#e63946", "Media": "#e9c46a", "Alta": "#2a9d8f"}

for sat in orden_sat:
    sub = df[df["Satisfacción"]==sat]
    counts_clase = sub["Clase"].value_counts().reindex(["Económica", "Business", "Primera"], fill_value=0)
    totales_clase = df["Clase"].value_counts().reindex(["Económica", "Business", "Primera"])
    pct_clase = (counts_clase / totales_clase * 100).round(1)

    fig.add_trace(go.Bar(
        name=sat, x=["Económica", "Business", "Primera"],
        y=pct_clase.values,
        marker_color=colores_sat[sat],
        text=pct_clase.astype(str)+"%",
        textposition="inside",
        showlegend=(True if sat=="Baja" else False)
    ), row=1, col=1)

    counts_ret = sub["Retraso"].value_counts().reindex(["Sin retraso", "Leve", "Grave"], fill_value=0)
    totales_ret = df["Retraso"].value_counts().reindex(["Sin retraso", "Leve", "Grave"])
    pct_ret = (counts_ret / totales_ret * 100).round(1)

    fig.add_trace(go.Bar(
        name=sat, x=["Sin retraso", "Leve", "Grave"],
```

```

        y=pct_ret.values,
        marker_color=colores_sat[sat],
        text=pct_ret.astype(str)+"%",
        textposition="inside",
        showlegend=True
    ), row=1, col=2)

fig.update_layout(
    barmode="stack",
    template="plotly_white", height=430,
    legend_title="Satisfacción"
)
fig.show()

```

Unable to display output for mime type(s): text/html

El patrón es contundente. En la gráfica de *Clase*, la barra de Primera está dominada por verde (satisfacción alta), mientras que Económica tiene una porción roja mucho mayor. En la gráfica de *Retraso*, el efecto es igualmente claro: un vuelo con retraso grave casi duplica la proporción de pasajeros insatisfechos respecto a uno puntual. Estas son exactamente las variables que el ² identificó como más informativas.

Paso 6: ¿Cuánto importa la selección en el modelo?

La prueba final es práctica: ¿eliminar las variables de bajo V de Cramér realmente mejora o al menos mantiene el desempeño predictivo? Entrenamos un Random Forest con tres configuraciones distintas.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder

df_enc = df.copy()
for col in df_enc.columns:
    df_enc[col] = LabelEncoder().fit_transform(df_enc[col])

y = df_enc["Satisfacción"]

configs = {
    "Todas (6 vars)": ["Clase", "Tipo_viaje", "Frecuencia", "Retraso", "Asiento", "Momento_comp"],
    "Sin ruido (4 vars)": ["Clase", "Tipo_viaje", "Frecuencia", "Retraso"],
    "Solo fuertes (2 vars)": ["Clase", "Retraso"],
}

scores, stds, nombres = [], [], []
for nombre, cols in configs.items():
    cv = cross_val_score(
        RandomForestClassifier(n_estimators=200, random_state=42),
        df_enc[cols], y, cv=5, scoring="accuracy"
    )

```

```

scores.append(cv.mean())
stds.append(cv.std())
nombres.append(nombre)

fig = go.Figure()
fig.add_trace(go.Bar(
    x=nombres, y=[round(s,4) for s in scores],
    error_y=dict(type="data", array=[round(s,4) for s in stds], visible=True),
    marker_color=["#457b9d", "#2a9d8f", "#e9c46a"],
    text=[f"{s:.1%}" for s in scores],
    textposition="outside", width=0.4
))

fig.update_layout(
    title="Accuracy (CV=5) según conjunto de variables seleccionadas por 2",
    yaxis_title="Accuracy promedio ± std",
    yaxis_range=[0, 1.1],
    template="plotly_white", height=420
)
fig.show()

```

Unable to display output for mime type(s): text/html

El resultado confirma la intuición: el modelo con todas las variables no supera al que usa solo las 4 relevantes — y en algunos casos lo hace peor, porque *Asiento* y *Momento_compra* introducen ruido que el modelo intenta aprender sin éxito. El modelo con solo las 2 variables más fuertes (*Clase* y *Retraso*) pierde algo de precisión, pero es notablemente más simple e interpretable.

Decisión final

² y V de Cramér nos permitieron pasar de 6 variables a 2 con criterio estadístico, sin necesidad de entrenar ningún modelo primero. Las variables *Asiento* y *Momento_compra* se eliminan por ser independientes de la satisfacción; las 4 restantes se conservan porque aportan información real, aunque *Clase* y *Retraso* concentran la mayor parte del poder predictivo.

116.6. Conclusión

i Lo que la prueba ² nos da

- **Decisión formal:** rechazar o no la independencia con control del error tipo I
- **Diagnóstico celda a celda:** residuos estandarizados revelan dónde está la relación
- **Magnitud:** V de Cramér cuantifica qué tan fuerte es la asociación

- Herramienta **indispensable** en EDA categórico, estudios clínicos, ciencias sociales y ML
- Siempre verificar **supuestos** antes de interpretar
- Combinar con **visualizaciones** para comunicar los hallazgos

Parte XXI.

Extracción de Patrones

Parte XXII.

Reglas de Asociación

117. Reglas de Asociación: Fundamentos, Interpretación y Selección

117.1. Introducción

En la mayoría de los contextos donde se recopilan datos transaccionales —ventas minoristas, registros médicos, historiales de navegación web, interacciones en plataformas digitales— existe una pregunta recurrente que antecede a cualquier modelo predictivo: *¿qué elementos tienden a aparecer juntos?* Esta pregunta, aparentemente simple, da lugar a uno de los problemas más estudiados en la minería de datos: el descubrimiento de reglas de asociación.

El problema fue formalizado por Agrawal y Srikant (1994) en el contexto del análisis de canastas de mercado (*market basket analysis*), donde cada transacción de un supermercado registra el conjunto de productos adquiridos por un cliente en una visita. La motivación original era directamente comercial: identificar productos que se compren juntos con frecuencia permite optimizar la disposición física de los artículos en tienda, diseñar estrategias de promoción cruzada y personalizar recomendaciones. El mismo formalismo, sin embargo, se ha aplicado desde entonces en dominios tan distintos como la bioinformática —para identificar genes que se co-expresan—, la epidemiología —para detectar síntomas que co-ocurren en pacientes—, y la ciberseguridad —para reconocer patrones de acceso que preceden a intrusiones (Han et al. 2011).

Lo que hace al problema pedagógicamente valioso no es su aplicabilidad, sino la tensión que revela entre dos tareas que parecen equivalentes pero no lo son: *generar* reglas es computacionalmente sencillo; *interpretar y seleccionar* reglas útiles requiere juicio estadístico genuino. Un analista que genere reglas sin comprensión crítica de las métricas que las respaldan puede llegar a conclusiones erróneas con aparente rigor cuantitativo. Este capítulo busca construir, de manera progresiva, los instrumentos conceptuales necesarios para evitar ese error.

La exposición está organizada de lo concreto a lo formal. Se comienza con definiciones motivadas por ejemplos del mundo real, se desarrolla el proceso de generación de reglas, y se dedica la mayor parte del capítulo al problema que resulta ser el más difícil: no cómo obtener reglas, sino cómo decidir cuáles valen la pena.

117.2. Definiciones Fundamentales

117.2.1. El Modelo de Datos: Transacciones e Ítems

El punto de partida es un conjunto de transacciones $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$, donde cada transacción T_i es un subconjunto de un universo finito de ítems $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$. En el contexto de un supermercado, $\mathcal{I}$ es el catálogo completo de productos y cada T_i es el conjunto de productos comprados en una visita particular. Es importante notar que este modelo no registra cantidades ni precios: únicamente la presencia o ausencia de cada ítem en cada transacción (Tan et al. 2006).

Un *itemset* es cualquier subconjunto $X \subseteq \mathcal{I}$. Un itemset de cardinalidad k se denomina k -itemset. Por ejemplo, {leche, pan} es un 2-itemset, y {leche, pan, mantequilla} es un 3-itemset.

117.2.2. Soporte

La primera métrica fundamental es el soporte, que mide la prevalencia de un patrón en el dataset.

Definición. El *soporte* de un itemset X en $\mathcal{D}$ es la fracción de transacciones que contienen a X :

$$\text{supp}(X) = \frac{|\{T_i \in \mathcal{D} : X \subseteq T_i\}|}{|\mathcal{D}|} \quad (117.1)$$

En términos probabilísticos, $\text{supp}(X) = P(X)$, es decir, la probabilidad empírica de observar el itemset X en una transacción elegida al azar. Para una regla $A \Rightarrow B$, el soporte es $P(A \cup B)$: la probabilidad de que tanto el antecedente como el consecuente estén presentes en la misma transacción.

El soporte tiene una propiedad formal relevante conocida como la *propiedad antimonotona* o principio Apriori: si un itemset X tiene soporte menor que un umbral $\sigma_{\min}$, entonces cualquier superconjunto $Y \supset X$ también tendrá soporte menor que $\sigma_{\min}$. Esta propiedad, demostrada formalmente en Agrawal y Srikant (1994), es la base algorítmica que hace tractable la búsqueda de patrones frecuentes en datasets con miles de ítems.

Interpretación e intuición. El soporte responde a la pregunta: *¿qué tan común es este patrón en los datos?* Un soporte bajo indica que el patrón es raro, lo que puede deberse a una relación genuinamente infrecuente o simplemente a que ambos ítems son raros por separado. Un soporte alto es condición necesaria —pero no suficiente— para que una regla sea útil en la práctica.

117.2.3. Confianza

La confianza mide el poder predictivo de una regla: dado que ocurre el antecedente, ¿con qué frecuencia ocurre también el consecuente?

Definición. La *confianza* de la regla $A \Rightarrow B$ es la probabilidad condicional empírica de B dado A :

$$\text{conf}(A \Rightarrow B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)} = P(B \mid A) \quad (117.2)$$

Una confianza de 0.80 para la regla $A \Rightarrow B$ significa que, en el 80 % de las transacciones que contienen a A , también aparece B . Esta interpretación como probabilidad condicional hace que la confianza sea intuitiva: es la tasa de acierto de la regla como regla de predicción.

La limitación fundamental de la confianza. A pesar de su intuitividad, la confianza presenta una limitación crítica que con frecuencia pasa desapercibida: *ignora la frecuencia base del consecuente*. Considérese la siguiente situación: en un supermercado donde el 65 % de todas las transacciones incluyen leche entera, cualquier regla cuyo consecuente sea leche entera tendrá automáticamente una confianza de al menos 65 %, independientemente de cuál sea el antecedente. Una confianza de 0.70 para la regla $\{\text{pan}\} \Rightarrow \{\text{leche entera}\}$ no es impresionante si la leche entera aparece en el 65 % de las transacciones: el pan añade menos de 10 puntos porcentuales sobre la tasa base. La regla tiene poder predictivo mínimo.

Este problema —confundir alta confianza con relación fuerte— es el error más común en el análisis de reglas de asociación, y motiva la necesidad del lift como métrica complementaria.

117.2.4. Lift

El lift cuantifica cuánto se aleja la relación entre A y B de la independencia estadística.

Definición. El *lift* de la regla $A \Rightarrow B$ es el cociente entre la confianza observada y la confianza esperada bajo independencia:

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{P(B)} = \frac{P(A \cup B)}{P(A) \cdot P(B)} \quad (117.3)$$

La interpretación probabilística es directa: si A y B fueran estadísticamente independientes, la probabilidad de su co-ocurrencia sería $P(A) \cdot P(B)$. El lift es precisamente el factor por el cual la co-ocurrencia observada supera (o cae por debajo de) esta expectativa bajo independencia (Tan et al. 2006).

- $\text{lift} = 1$: A y B son estadísticamente independientes. La presencia de A no aporta información sobre B .
- $\text{lift} > 1$: A y B co-ocurren más frecuentemente de lo esperado bajo independencia. La regla señala una asociación positiva.
- $\text{lift} < 1$: A y B co-ocurren menos frecuentemente de lo esperado. La presencia de A inhibe la presencia de B .

Una propiedad algebraica importante es que el lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Esto tiene implicaciones prácticas significativas que se discuten en la Sección 118.2.

117.2.5. Un Ejemplo Concreto

Para anclar las definiciones, considérese el siguiente dataset de cinco transacciones:

Transacción	Ítems
T_1	leche, pan, mantequilla
T_2	leche, pan
T_3	leche, mantequilla
T_4	pan, mantequilla
T_5	leche, pan, queso

Para la regla $\{\text{pan}\} \Rightarrow \{\text{leche}\}$:

$$\text{supp}(\{\text{pan}, \text{leche}\}) = \frac{3}{5} = 0.60$$

$$\text{conf}(\{\text{pan}\} \Rightarrow \{\text{leche}\}) = \frac{\text{supp}(\{\text{pan}, \text{leche}\})}{\text{supp}(\{\text{pan}\})} = \frac{3/5}{4/5} = 0.75$$

$$\text{lift}(\{\text{pan}\} \Rightarrow \{\text{leche}\}) = \frac{0.75}{P(\{\text{leche}\})} = \frac{0.75}{4/5} = 0.9375$$

El lift de 0.9375 — menor que 1 — revela que, a pesar de una confianza aparentemente alta de 0.75, la presencia de pan en realidad *reduce ligeramente* la probabilidad de encontrar leche en la misma transacción, en relación con la tasa base de leche (que es 0.80). La confianza del 75 % era engañosa porque la leche ya era muy frecuente.

117.3. Generación de Reglas Frecuentes

117.3.1. De Itemsets Frecuentes a Reglas

El proceso de descubrimiento de reglas de asociación se divide en dos etapas (Agrawal y Srikant 1994; Han et al. 2011):

1. **Descubrimiento de itemsets frecuentes:** identificar todos los itemsets $X \subseteq \mathcal{I}$ cuyo soporte supere un umbral mínimo $\sigma_{\min}$ fijado por el analista.
2. **Generación de reglas a partir de itemsets frecuentes:** para cada itemset frecuente X de cardinalidad $k \geq 2$, generar todas las reglas $A \Rightarrow B$ donde $A \cup B = X$, $A \cap B = \emptyset$, $A \neq \emptyset$ y $B \neq \emptyset$, y retener solo aquellas cuya confianza supere un umbral mínimo $\gamma_{\min}$.

La separación entre estas dos etapas no es arbitraria: la primera es computacionalmente costosa —el espacio de todos los subconjuntos posibles de $\mathcal{I}$ tiene cardinalidad $2^{|\mathcal{I}|}$ — mientras que la segunda, una vez conocidos los itemsets frecuentes, es relativamente rápida.

117.3.2. El Algoritmo Apriori

El algoritmo Apriori, propuesto en Agrawal y Srikant (1994), fue el primero en resolver eficientemente la primera etapa mediante una estrategia de búsqueda en anchura que explota la propiedad antimonotona del soporte. La idea central es que no tiene sentido evaluar el soporte de un k -itemset si alguno de sus $(k - 1)$ -subconjuntos ya es infrecuente: por antimonotonía, el k -itemset también será infrecuente. Esto permite podar el espacio de búsqueda de manera drástica.

El funcionamiento a alto nivel es el siguiente: en cada iteración k , el algoritmo genera candidatos de tamaño k a partir de los itemsets frecuentes de tamaño $k - 1$ (fase de unión), descarta aquellos que contienen subconjuntos infrecuentes (fase de poda) y escanea el dataset para calcular el soporte de los candidatos restantes. El proceso termina cuando no se pueden generar nuevos candidatos frecuentes.

Tabla 117.1.

Trans.	Ítems
--------	-------

Ítem	Conteo	Soporte
------	--------	---------

No es objetivo de este capítulo detallar la implementación del algoritmo — para ello puede consultarse Han et al. (2011), capítulo 6, o Tan et al. (2006), capítulo 5. Lo que sí importa aquí es comprender sus parámetros y sus consecuencias sobre los resultados.

117.3.3. Los Parámetros como Decisiones del Analista

La ejecución de Apriori requiere especificar al menos dos umbrales:

- $\sigma_{\min}$ (**soporte mínimo**): controla cuán frecuente debe ser un patrón para ser considerado. Un valor bajo produce más itemsets frecuentes —y potencialmente más reglas— pero también más ruido. Un valor alto filtra agresivamente y puede eliminar patrones genuinos pero poco frecuentes.
- $\gamma_{\min}$ (**confianza mínima**): controla cuán predictiva debe ser una regla para ser reportada. Como se discutirá en la Sección 117.4, este umbral no tiene interpretación estadística directa: su valor apropiado depende del dominio.

Un aspecto crucial que los textos de introducción rara vez enfatizan es que *los resultados dependen directamente de los parámetros elegidos*. El número de reglas obtenidas puede variar en un orden de magnitud al cambiar $\sigma_{\min}$ en unos pocos puntos porcentuales. Esto implica que reportar reglas sin reportar los parámetros con los que fueron generadas es una práctica metodológicamente deficiente.

El siguiente bloque de código ilustra esta sensibilidad de manera empírica:

```
soportes = [0.06, 0.08, 0.10, 0.12, 0.14, 0.16, 0.18]
resultados_sens = []

for s in soportes:
    its = apriori(df, min_support=s, use_colnames=True)
    if len(it) == 0:
        resultados_sens.append({"min_support": s, "n_reglas": 0, "lift_med": np.nan})
        continue
    r = association_rules(it, metric="confidence", min_threshold=0.3)
    resultados_sens.append({
        "min_support": round(s, 2),
        "n_reglas": len(r),
        "lift_med": round(r["lift"].median(), 3) if len(r) > 0 else np.nan,
    })

df_sens = pd.DataFrame(resultados_sens)

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=df_sens["min_support"], y=df_sens["n_reglas"],
    mode="lines+markers", name="N° de reglas",
    line=dict(color="#e63946", width=2), marker=dict(size=8),
    yaxis="y1",
))
fig.add_trace(go.Scatter(
    x=df_sens["min_support"], y=df_sens["lift_med"],
    mode="lines+markers", name="Lift mediano",
    line=dict(color="#457b9d", width=2, dash="dash"), marker=dict(size=8),
    yaxis="y2",
))
fig.update_layout(
    xaxis=dict(title="Soporte mínimo (_min)"),
```

```

yaxis=dict(title="N° de reglas generadas", side="left"),
yaxis2=dict(title="Lift mediano", overlaying="y", side="right"),
legend=dict(x=0.65, y=0.95),
height=400,
title="Sensibilidad de los resultados al soporte mínimo",
)
fig.show()

```

Unable to display output for mime type(s): text/html

Número de reglas generadas en función del soporte mínimo (confianza mínima fija en 0.3). La caída es pronunciada: pasar de soporte 0.08 a 0.14 puede reducir las reglas a la mitad.

Unable to display output for mime type(s): text/html

Nótese el comportamiento opuesto de ambas curvas: al aumentar $\sigma_{\min}$, el número de reglas disminuye mientras que el lift mediano tiende a aumentar. Esto no es una coincidencia: los itemsets frecuentes con soporte muy bajo son los más susceptibles de ser artefactos estadísticos, y al filtrarlos se conservan los patrones con mayor señal relativa.

117.4. El Problema de las Demasiadas Reglas

117.4.1. La Explosión Combinatoria

Supóngase un dataset con 100 ítems distintos y $\sigma_{\min} = 0.01$. El número de posibles itemsets es $2^{100} - 1$, un número astronómico. Incluso con la poda del algoritmo Apriori, y con un soporte mínimo razonablemente restrictivo, es posible generar decenas de miles de reglas a partir de datasets de tamaño moderado. El dataset Groceries original, con 9835 transacciones y 169 ítems, puede generar entre varios cientos y varios miles de reglas según los parámetros elegidos (Han et al. 2011).

Este volumen de resultados crea un problema de segundo orden: la proliferación de reglas hace que la revisión manual sea inviable, y la selección automatizada requiere criterios que, si son mal elegidos, pueden introducir un sesgo sistemático.

117.4.2. Redundancia e Irrelevancia

Parte del problema es estructural. Dado un itemset frecuente $X = \{A, B, C\}$, el algoritmo genera todas las particiones posibles como reglas: $\{A\} \Rightarrow \{B, C\}$, $\{B\} \Rightarrow \{A, C\}$, $\{C\} \Rightarrow \{A, B\}$, $\{A, B\} \Rightarrow \{C\}$, $\{A, C\} \Rightarrow \{B\}$, $\{B, C\} \Rightarrow \{A\}$. Muchas de estas reglas comparten la misma información de co-ocurrencia y son, en cierta medida, redundantes entre sí. La elección del consecuente más adecuado depende del contexto de aplicación, no del algoritmo.

Una capa adicional de dificultad es la irrelevancia. Una regla puede ser formalmente válida —soporte y confianza por encima de los umbrales— y al mismo tiempo no añadir ninguna información accionable. Esto ocurre sistemáticamente cuando el consecuente es un ítem muy frecuente: la regla parece predictiva

porque su confianza es alta, pero en realidad refleja la mera ubicuidad del consecuente, no una relación genuina con el antecedente.

Este es el problema central que toda metodología rigurosa de análisis de reglas de asociación debe abordar, y que las métricas de evaluación descritas en la siguiente sección intentan resolver.

117.5. Métricas de Evaluación: Interpretación Profunda

117.5.1. Soporte: Prevalencia, No Interés

El soporte, tal como fue definido en Sección 117.2, responde exclusivamente a la pregunta *¿qué tan frecuente es este patrón?* No dice nada sobre si el patrón es inesperado, interesante o accionable. Una regla con soporte muy alto puede ser trivial: si el 80 % de los clientes compra agua embotellada, la regla $\{X\} \Rightarrow \{\text{agua}\}$ tendrá soporte alto para prácticamente cualquier X , sin revelar nada útil.

El soporte sí tiene un papel importante como **umbral de confiabilidad estadística**: un patrón respaldado por muy pocas observaciones es inestable y puede desaparecer o cambiar sustancialmente al agregar nuevos datos. En un dataset de 50 transacciones, un soporte de 0.10 corresponde a apenas 5 observaciones — insuficiente para afirmaciones estadísticas robustas. Con datasets más grandes (del orden de miles de transacciones), los umbrales de soporte pueden y deben ser más exigentes.

117.5.2. Confianza: Predictividad Condicional con un Sesgo Estructural

La confianza mide $P(B | A)$: la probabilidad de que el consecuente esté presente dado el antecedente. Su interpretación como probabilidad condicional es clara y directamente útil en sistemas de recomendación: si el 80 % de los clientes que compran pan también compran mantequilla, esa confianza justifica directamente recomendar mantequilla a alguien que acaba de poner pan en su carrito.

Sin embargo, la confianza tiene un sesgo estructural bien documentado (Tan et al. 2006): es monótonamente creciente en $P(B)$. Para un antecedente fijo A :

$$\text{conf}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A)} \leq \frac{P(B)}{P(A)} \cdot P(A) = P(B) \cdot \frac{1}{P(A)}$$

La confianza depende de $P(B)$ de manera directa. Un ítem B muy frecuente producirá, ceteris paribus, reglas de alta confianza para cualquier antecedente A , simplemente porque B aparece en casi todas las transacciones. Esta dependencia hace que la confianza no sea comparable entre reglas con consecuentes de frecuencias muy distintas, lo que introduce una fuente de sesgo sistemática si se usa como único criterio de selección.

La consecuencia práctica es la siguiente: antes de declarar que una regla tiene “alta confianza”, es necesario comparar esa confianza con $P(B)$. Solo si $\text{conf}(A \Rightarrow B) \gg P(B)$ puede afirmarse que la presencia de A aporta genuinamente a la predicción de B .

! Importante

Umbral de confianza no-trivial. Para que la regla $A \Rightarrow B$ tenga $\text{lift} > k$, se requiere:

$$\text{conf}(A \Rightarrow B) > k \cdot P(B)$$

En particular, para que la regla sea simplemente no-trivial ($\text{lift} > 1$), es necesario que la confianza

supere la frecuencia base del consecuente: $\text{conf}(A \Rightarrow B) > P(B)$.

117.5.3. Lift: La Métrica Central de Interés

El lift, definido en Ecuación 120.4, corrige el sesgo de la confianza al normalizar por $P(B)$. Su interpretación como razón de verosimilitud hace que sea directamente comparable entre reglas con distintos consecuentes.

Desde el punto de vista de la probabilidad, el lift es el cociente entre la probabilidad conjunta observada y la probabilidad conjunta esperada bajo la hipótesis de independencia:

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)}$$

Un lift significativamente mayor que 1 constituye evidencia en favor de una asociación genuina entre A y B . Un lift cercano a 1 indica que conocer A no modifica la probabilidad de B , lo que hace que la regla sea informacionalmente vacía (Tan et al. 2006).

La asimetría del lift. Como se mencionó al final de Sección 117.2, el lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Esto puede resultar sorprendente, pues la confianza sí es asimétrica. La razón es algebraica: el lift compara $P(A \cup B)$ con $P(A) \cdot P(B)$, y esta expresión trata a A y a B de manera equivalente.

La asimetría del lift tiene implicaciones prácticas: cuando se generan las reglas $A \Rightarrow B$ y $B \Rightarrow A$, ambas tienen exactamente el mismo lift pero pueden tener confianzas muy diferentes. La dirección que debe preferirse en una aplicación real depende del contexto: si el objetivo es recomendar B dado que se observó A , la confianza de $A \Rightarrow B$ es la métrica relevante; si el objetivo es recomendar A dado B , lo es $\text{conf}(B \Rightarrow A)$.

Limitaciones del lift. A pesar de su superioridad sobre la confianza para detectar asociaciones no triviales, el lift tiene dos limitaciones importantes:

1. *Inestabilidad con soporte bajo.* En datasets pequeños o con ítems raros, el lift puede inflarse artificialmente. Si A y B son ítems raros — $P(A) = P(B) = 0.01$ — y co-ocurren una sola vez, su soporte es $P(A \cup B) = 0.01$ y su lift es $1/0.01 = 100$. Este valor es enorme pero está respaldado por una sola observación, lo que lo hace estadísticamente no significativo.
2. *Techo variable.* El valor máximo que puede alcanzar el lift no es universal: depende de los soportes individuales de A y B . El lift máximo teórico es $1/\max(P(A), P(B))$, lo que implica que no es válido comparar lifts entre reglas con distintos niveles de frecuencia sin esta corrección.

Estas limitaciones motivan el uso de métricas complementarias, que se introducen en la Sección 117.7.

117.5.4. Resumen Comparativo

La Tabla 117.2 sintetiza las propiedades de las tres métricas fundamentales.

Tabla 117.2.: Propiedades de las métricas fundamentales de reglas de asociación.

Métrica	Pregunta que responde	Escala	Limitación principal
Soporte	¿Qué tan común es el patrón?	$[0, 1]$	No mide interés

Métrica	Pregunta que responde	Escala	Limitación principal
Confianza	¿Qué tan predecible es B dado A ?	$[0, 1]$	Sesgada por $P(B)$
Lift	¿Es la co-ocurrencia mayor que el azar?	$(0, +\infty)$	Inestable con soporte bajo

Ninguna de las tres métricas es suficiente por sí sola. La práctica correcta consiste en utilizarlas en conjunto y privilegiar reglas que resulten sólidas según múltiples criterios.

117.6. Interpretación Visual del Espacio de Reglas

117.6.1. Representación en el Espacio Soporte-Confianza-Lift

El conjunto de reglas generadas por Apriori puede visualizarse como una nube de puntos en un espacio tridimensional definido por soporte, confianza y lift. Esta representación permite identificar de manera inmediata los distintos tipos de reglas —las útiles, las engañosas y las triviales— y orienta la selección de manera más informada que la simple inspección de tablas.

La forma más conveniente de reducir esta representación tridimensional a un gráfico plano es proyectar confianza y lift sobre los ejes cartesianos, y codificar el soporte mediante el tamaño de los marcadores. El uso de color para representar el lift añade redundancia visual que facilita la identificación de los cuadrantes de interés.

```
fig = px.scatter(
    reglas,
    x="confidence",
    y="lift",
    size="support",
    hover_data={
        "regla": True,
        "support": ":.3f",
        "confidence": ":.3f",
        "lift": ":.3f",
    },
    color="lift",
    color_continuous_scale="RdYlGn",
    labels={
        "confidence": "Confianza",
        "lift": "Lift",
        "support": "Soporte",
    },
    size_max=28,
)

fig.add_hline(
    y=1.0,
    line_dash="dash",
```

```

    line_color="crimson",
    annotation_text="Lift = 1 (independencia estadística)",
    annotation_position="top left",
    annotation_font_color="crimson",
)

fig.update_layout(
    height=500,
    xaxis=dict(range=[0, 1.05], title="Confianza"),
    yaxis=dict(title="Lift"),
    coloraxis_colorbar=dict(title="Lift"),
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 117.1.: Espacio de reglas: confianza (eje horizontal) vs. lift (eje vertical), con el soporte codificado en el tamaño del marcador y el lift en el color. La línea roja discontinua marca $\text{lift} = 1$, la frontera de independencia estadística. Las reglas por debajo de esta línea son espurias o triviales. Pasar el cursor sobre cualquier punto revela la regla completa.

117.6.2. Lectura del Gráfico: Cuatro Zonas

La Figura 117.1 define implícitamente cuatro zonas de interpretación:

Zona I — Cuadrante deseable (lift alto, confianza alta, marcador grande). Las reglas en esta zona son las candidatas más sólidas: tienen poder predictivo (confianza alta), representan una asociación genuina no explicada por las frecuencias base ($\text{lift} > 1$), y están respaldadas por una cantidad razonable de transacciones (soporte alto). Son los patrones más accionables.

Zona II — Alta confianza, lift bajo (debajo de la línea roja). Esta es la zona de mayor riesgo para el analista no crítico. Las reglas aquí pueden tener confianzas impresionantes —0.70, 0.80 o más— pero un lift cercano o inferior a 1. En la mayoría de los casos, esto se debe al problema de la frecuencia base: el consecuente es tan ubicuo que aparece independientemente del antecedente. Confiar en estas reglas para tomar decisiones es, en el mejor de los casos, inútil y, en el peor, contraproducente.

Zona III — Lift alto, soporte bajo (marcadores pequeños por encima de la línea). Estas reglas señalan asociaciones potencialmente fuertes, pero están respaldadas por pocas observaciones. Su lift puede ser un artefacto estadístico: en un dataset de 50 transacciones, dos ítems que co-ocurren apenas 5 veces pueden mostrar un lift elevado por pura aleatoriedad. La regla puede ser genuina, pero requiere cautela y, de ser posible, validación con datos adicionales.

Zona IV — Bajo en todo. Reglas sin soporte, sin confianza relevante y con lift cercano a 1. No hay argumento para retenerlas.

117.6.3. Co-ocurrencia Bruta: La Matriz de Transacciones

Antes de interpretar las métricas formales, una visualización complementaria de utilidad es la matriz de co-ocurrencia entre los ítems más frecuentes. Esta matriz registra, para cada par de ítems, el número absoluto de transacciones en las que ambos aparecen simultáneamente.

```
freq = df.sum().sort_values(ascending=False)
top_items = freq.head(10).index.tolist()
df_top = df[top_items]
cooc = df_top.T.dot(df_top)
np.fill_diagonal(cooc.values, 0)

fig = px.imshow(
    cooc,
    color_continuous_scale="Oranges",
    labels={"color": "Co-ocurrencias"},
    aspect="auto",
    text_auto=True,
)
fig.update_layout(height=430)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 117.2.: Matriz de co-ocurrencia entre los 10 ítems más frecuentes del dataset. La diagonal está fijada a cero (un ítem no co-ocurre consigo mismo). Las celdas de intensidad naranja alta deben interpretarse con precaución cuando involucran ítems de alta frecuencia base.

Una observación importante sobre la Figura 117.2: las celdas con mayor intensidad tienden a corresponder a pares que involucran los ítems más frecuentes —en este caso, *whole milk* y *bread*. Esto no es evidencia de una asociación especialmente fuerte entre ellos; es, en gran medida, consecuencia de sus altas frecuencias individuales. La co-ocurrencia bruta es el numerador de $P(A \cup B)$, pero no incorpora los denominadores $P(A)$ y $P(B)$ que son necesarios para evaluar si la relación es no trivial. Las celdas más informativas son aquellas con alta intensidad entre ítems de frecuencia moderada, como el clúster *granola-yogurt-fruit*.

117.7. Selección de Reglas Útiles

117.7.1. Criterios Prácticos de Filtrado

El proceso de selección de reglas útiles no es un procedimiento automatizable en su totalidad: requiere decisiones del analista que deben estar informadas por el contexto del problema. Sin embargo, es posible enunciar un conjunto de criterios generales que sirven como punto de partida (Tan et al. 2006; Han et al. 2011):

Criterio 1: Lift sustancialmente mayor que 1. Una regla cuyo lift no supera de manera apreciable el valor de referencia de la independencia (1.0) no revela ninguna asociación genuina entre el antecedente y el consecuente. Un umbral pragmático es $\text{lift} \geq 1.2$ para relaciones débiles y $\text{lift} \geq 1.5$ para relaciones

moderadas. En datasets grandes y en aplicaciones donde el costo de una decisión incorrecta es alto, valores más exigentes ($\text{lift} \geq 2.0$) son más apropiados.

Criterio 2: Soporte respaldado por suficientes observaciones. El soporte mínimo debe traducirse a un número absoluto de transacciones que resulte estadísticamente sensato. Con un dataset de 50 transacciones, $\text{soporte} = 0.10$ corresponde a apenas 5 observaciones — un valor muy inestable. Con un dataset de 9000 transacciones, ese mismo soporte corresponde a 900 observaciones, lo cual es perfectamente robusto.

Criterio 3: Confianza superior a la frecuencia base del consecuente. Como se derivó en Sección 117.5, la condición $\text{conf}(A \Rightarrow B) > P(B)$ es equivalente a $\text{lift} > 1$. Una buena práctica es calcular explícitamente el “umbral de confianza no-trivial” para cada posible consecuente antes de analizar las reglas, tal como ilustra la siguiente tabla:

```
umbrales = []
for item in df.columns:
    p = df[item].mean()
    umbrales.append({
        "Ítem": item,
        "P(ítem)": round(p, 3),
        "conf mín para lift > 1.0": round(p, 3),
        "conf mín para lift > 1.5": round(1.5 * p, 3),
    })

pd.DataFrame(umbrales).sort_values("P(ítem)", ascending=False).head(10)
```

Tabla 117.3.: Umbral mínimo de confianza necesario para que una regla sea no-trivial ($\text{lift} > 1$) o sustancialmente no-trivial ($\text{lift} > 1.5$), calculado a partir de la frecuencia base de cada ítem.

	Ítem	P(ítem)	conf mín para lift > 1.0	conf mín para lift > 1.5
13	whole milk	0.52	0.52	0.78
1	bread	0.40	0.40	0.60
7	eggs	0.28	0.28	0.42
14	yogurt	0.28	0.28	0.42
2	butter	0.26	0.26	0.39
8	fruit	0.20	0.20	0.30
12	vegetables	0.18	0.18	0.27
6	coffee	0.16	0.16	0.24
10	soda	0.16	0.16	0.24
4	cheese	0.14	0.14	0.21

Esta tabla es una herramienta diagnóstica directamente útil: antes de entusiasmarse con una regla cuyo consecuente es *whole milk*, conviene consultar que la confianza supera 0.58 (para $\text{lift} > 1$) o 0.87 (para $\text{lift} > 1.5$). Si la confianza observada no alcanza esos valores, la regla no aporta nada.

117.7.2. Métricas Complementarias

La literatura de minería de datos propone numerosas métricas alternativas al lift para evaluar el interés de una regla (Tan et al. 2006). Tres de las más utilizadas en la práctica son las siguientes.

Leverage (apalancamiento).

$$\text{lev}(A \Rightarrow B) = P(A \cup B) - P(A) \cdot P(B)$$

El leverage es la diferencia absoluta entre la co-ocurrencia observada y la esperada bajo independencia. Su interpretación es más directa que la del lift: mide cuántas transacciones *adicionales*, sobre el baseline de independencia, tienen tanto A como B . Un leverage de 0.05 en un dataset de 1000 transacciones significa que aproximadamente 50 transacciones tienen tanto A como B más allá de lo que esperaríamos si fueran independientes. El rango del leverage es $[-P(A) \cdot P(B), \min(P(A), P(B)) - P(A) \cdot P(B)]$.

Conviction.

$$\text{conv}(A \Rightarrow B) = \frac{1 - P(B)}{1 - \text{conf}(A \Rightarrow B)}$$

La conviction mide con qué frecuencia la regla $A \Rightarrow B$ está “equivocada” — es decir, con qué frecuencia A ocurre sin B — en relación con lo esperado bajo independencia. A diferencia del lift, la conviction es asimétrica: $\text{conv}(A \Rightarrow B) \neq \text{conv}(B \Rightarrow A)$ en general, lo que la hace más apropiada para evaluar reglas dirigidas. Cuando la confianza tiende a 1, la conviction tiende a infinito, lo cual puede interpretarse como que la regla es cuasi-determinista. Conviction = 1 indica independencia.

Zhang’s metric.

$$Z(A, B) = \frac{P(A \cup B) - P(A) \cdot P(B)}{\max\{P(A \cup B)(1 - P(A)), P(A)(P(B) - P(A \cup B))\}}$$

La métrica de Zhang, propuesta para superar algunas limitaciones del lift, tiene rango $[-1, 1]$ y es simétrica. Su principal ventaja es que normaliza correctamente tanto en el caso de co-ocurrencia alta como en el de co-ocurrencia baja, siendo más robusta ante ítems muy frecuentes. Un valor de Zhang cercano a 0 indica independencia; valores positivos cercanos a 1 indican asociación fuerte.

```
cols_mostrar = ["regla", "support", "confidence", "lift", "leverage", "conviction", "zhang"]
reglas.sort_values("lift", ascending=False).head(8)[cols_mostrar].round(3)
```

Tabla 117.4.: Comparación de métricas para las 8 reglas con mayor lift. Las métricas adicionales —leverage, conviction y Zhang— permiten corroborar o cuestionar lo que el lift sugiere.

	regla	support	confidence	lift	leverage	conviction	zhang
15	chips $\rightarrow$ soda	0.14	1.000	6.250	0.118	inf	0.977
14	soda $\rightarrow$ chips	0.14	0.875	6.250	0.118	6.880	1.000
23	yogurt $\rightarrow$ granola	0.10	0.357	3.571	0.072	1.400	1.000
22	granola $\rightarrow$ yogurt	0.10	1.000	3.571	0.072	inf	0.800
13	cheese $\rightarrow$ eggs	0.10	0.714	2.551	0.061	2.520	0.707
12	eggs $\rightarrow$ cheese	0.10	0.357	2.551	0.061	1.338	0.844
21	fruit $\rightarrow$ yogurt	0.12	0.600	2.143	0.064	1.800	0.667
20	yogurt $\rightarrow$ fruit	0.12	0.429	2.143	0.064	1.400	0.741

117.7.3. Reglas Fuertes vs. Reglas Débiles: Un Contraste

Para ilustrar la diferencia entre una regla bien respaldada y una engañosa, considérense los dos extremos del espacio de reglas:

```
print("=== REGLAS CON ALTA CONFIANZA PERO LIFT BAJO ===")
print("(apariencia de fortaleza, realidad de trivialidad)\n")
debiles = reglas[reglas["confidence"] > 0.45].sort_values("lift").head(3)
print(debiles[["regla", "support", "confidence", "lift", "zhang"]].round(3).to_string(index=False))

print("\n=== REGLAS CON LIFT ALTO Y SOPORTE RAZONABLE ===")
print("(señal genuina respaldada por múltiples métricas)\n")
fuertes = reglas[
    (reglas["lift"] > 1.3) & (reglas["support"] >= 0.10)
].sort_values("lift", ascending=False).head(3)
print(fuertes[["regla", "support", "confidence", "lift", "zhang"]].round(3).to_string(index=False))
```

```
=== REGLAS CON ALTA CONFIANZA PERO LIFT BAJO ===
(apariencia de fortaleza, realidad de trivialidad)
```

	regla	support	confidence	lift	zhang
butter → whole milk		0.14	0.538	1.036	0.046
yogurt → whole milk		0.16	0.571	1.099	0.125
coffee → whole milk		0.10	0.625	1.202	0.200

```
=== REGLAS CON LIFT ALTO Y SOPORTE RAZONABLE ===
(señal genuina respaldada por múltiples métricas)
```

	regla	support	confidence	lift	zhang
soda → chips		0.14	0.875	6.250	1.000
chips → soda		0.14	1.000	6.250	0.977
granola → yogurt		0.10	1.000	3.571	0.800

Las reglas del primer grupo tienen confianzas aparentemente razonables, pero sus lifts cercanos a 1 y sus valores de Zhang bajos revelan que no hay asociación genuina entre antecedente y consecuente. Las reglas del segundo grupo, aunque puedan tener confianzas más modestas en términos absolutos, son más informativas porque representan desviaciones reales del comportamiento esperado bajo independencia.

117.7.4. Un Procedimiento de Selección Sistemático

El siguiente procedimiento condensa los criterios anteriores en una función de evaluación que puede aplicarse a cualquier conjunto de reglas:

```
def evaluar_regla(row, n_transacciones=50, verbose=True):
    """
    Evalúa el interés de una regla de asociación según cuatro criterios.
    Retorna una clasificación cualitativa: ROBUSTA, CANDIDATA o DESCARTAR.
    """
```



```

observaciones = []
puntos = 0

# Criterio 1: Lift
if row["lift"] > 1.5:
    observaciones.append("Lift fuerte (> 1.5)")
    puntos += 3
elif row["lift"] > 1.1:
    observaciones.append("Lift moderado (1.1 - 1.5)")
    puntos += 1
else:
    observaciones.append("Lift débil ( 1.1) - no trivial")

# Criterio 2: Soporte absoluto
n_obs = int(row["support"] * n_transacciones)
if n_obs >= 10:
    observaciones.append(f"Soporte sólido ({n_obs} transacciones)")
    puntos += 2
elif n_obs >= 5:
    observaciones.append(f"Soporte mínimo ({n_obs} transacciones)")
    puntos += 1
else:
    observaciones.append(f"Soporte insuficiente ({n_obs} transacciones)")

# Criterio 3: Zhang
if row["zhang"] > 0.3:
    observaciones.append("Zhang robusto (> 0.3)")
    puntos += 2
elif row["zhang"] > 0.1:
    observaciones.append("Zhang moderado (0.1 - 0.3)")
    puntos += 1
else:
    observaciones.append("Zhang débil ( 0.1)")

# Criterio 4: Conviction
if row["conviction"] > 1.5:
    observaciones.append("Conviction alta (> 1.5)")
    puntos += 1

clasificacion = (
    "ROBUSTA" if puntos >= 6
    else "CANDIDATA" if puntos >= 3
    else "DESCARTAR"
)

if verbose:
    print(f"Regla: {row['regla']}")
    for obs in observaciones:
        print(f"    • {obs}")

```

```

        print(f" → Clasificación: {clasificacion} (puntos: {puntos}/8)\n")
    return clasificacion

# Aplicar a las 5 reglas con mayor lift
print("Evaluación de las 5 reglas con mayor lift:\n")
for _, row in reglas.sort_values("lift", ascending=False).head(5).iterrows():
    evaluar_regla(row)

```

Evaluación de las 5 reglas con mayor lift:

Regla: chips → soda

- Lift fuerte (> 1.5)
 - Soporte mínimo (7 transacciones)
 - Zhang robusto (> 0.3)
 - Conviction alta (> 1.5)
- Clasificación: ROBUSTA (puntos: 7/8)

Regla: soda → chips

- Lift fuerte (> 1.5)
 - Soporte mínimo (7 transacciones)
 - Zhang robusto (> 0.3)
 - Conviction alta (> 1.5)
- Clasificación: ROBUSTA (puntos: 7/8)

Regla: yogurt → granola

- Lift fuerte (> 1.5)
 - Soporte mínimo (5 transacciones)
 - Zhang robusto (> 0.3)
- Clasificación: ROBUSTA (puntos: 6/8)

Regla: granola → yogurt

- Lift fuerte (> 1.5)
 - Soporte mínimo (5 transacciones)
 - Zhang robusto (> 0.3)
 - Conviction alta (> 1.5)
- Clasificación: ROBUSTA (puntos: 7/8)

Regla: cheese → eggs

- Lift fuerte (> 1.5)
 - Soporte mínimo (5 transacciones)
 - Zhang robusto (> 0.3)
 - Conviction alta (> 1.5)
- Clasificación: ROBUSTA (puntos: 7/8)

Este procedimiento no debe interpretarse como un algoritmo definitivo: los umbrales y los pesos son propuestas que deben ajustarse al dominio. Su valor pedagógico reside en hacer explícita la multidimensionalidad del juicio sobre la calidad de una regla, en contraste con la práctica de usar una única métrica como criterio absoluto.

117.8. Limitaciones del Enfoque

117.8.1. El Problema de la Causalidad

Quizás la limitación más importante del análisis de reglas de asociación, y la más frecuentemente ignorada, es que las reglas son *descriptivas* y *correlacionales*, nunca *causales* (Pearl 2009). La regla $\{\text{pan}\} \Rightarrow \{\text{mantequilla}\}$ con lift alto nos dice que quienes compren pan tienden a comprar mantequilla con más frecuencia de la esperada. No nos dice que comprar pan *causa* la compra de mantequilla, ni que poner pan al lado de la mantequilla *causará* que más personas los compren juntos. La confusión entre correlación y causalidad en el análisis de datos ha sido extensamente documentada como una fuente de errores costosos en decisiones de negocio y de política (Pearl 2009).

La distinción es especialmente relevante en aplicaciones de alto impacto como medicina o epidemiología. Identificar que ciertos síntomas co-ocurren con cierta diagnosis no equivale a entender el mecanismo causal que los conecta. Para inferencias causales son necesarios diseños experimentales o marcos formales de causalidad como los grafos causales de Pearl (2009).

117.8.2. Ítems Dominantes y Sesgo por Frecuencia Base

Como se ha analizado en detalle a lo largo de este capítulo, la presencia de ítems con soporte muy alto introduce un sesgo estructural en el análisis. En el dataset Groceries, *whole milk* aparece en aproximadamente el 26 % de las 9835 transacciones originales —y en el 58 % de la muestra de 50 transacciones utilizada en los ejemplos computacionales. Este dominio hace que *whole milk* aparezca como consecuente en un gran número de reglas con alta confianza, la mayoría de las cuales no representa asociaciones genuinas.

Una estrategia práctica para mitigar este problema es eliminar o tratar de manera especial los ítems cuyo soporte supere un umbral alto (por ejemplo, 40–50 %) antes de ejecutar el análisis, o analizar subgrupos del dataset donde la distribución de ítems sea más homogénea.

117.8.3. Ausencia de Estructura Temporal

El modelo de datos de las reglas de asociación —conjuntos sin orden— es incapaz de capturar relaciones temporales. Si una secuencia de comportamiento es relevante (por ejemplo, los clientes compran primero un producto básico y luego un complemento en visitas posteriores), Apriori la ignorará. Para este tipo de análisis existen algoritmos de descubrimiento de patrones secuenciales, como GSP (*Generalized Sequential Patterns*) y PrefixSpan, que extienden el formalismo de Apriori a secuencias ordenadas de itemsets (Han et al. 2011).

117.8.4. Inestabilidad con Datasets Pequeños

Los ejemplos computacionales de este capítulo utilizan 50 transacciones, lo cual es suficiente para ilustrar los conceptos pero insuficiente para afirmaciones estadísticas robustas. Con datasets pequeños, el soporte es muy sensible a fluctuaciones individuales: agregar o eliminar una sola transacción puede cambiar el soporte de un itemset en 2 puntos porcentuales (1/50), lo que puede hacer que un patrón cruce el umbral mínimo en una dirección u otra. En la práctica, el análisis de reglas de asociación requiere datasets con al menos varios cientos de transacciones para ser confiable, y preferiblemente miles.

117.8.5. Sensibilidad a los Parámetros

Como se demostró en Sección 117.3, el número y la naturaleza de las reglas obtenidas dependen fuertemente de los parámetros elegidos. Esta sensibilidad tiene dos consecuencias metodológicas: (1) los resultados no son objetivos en el sentido de ser independientes de las decisiones del analista, y (2) dos análisis del mismo dataset con parámetros distintos pueden producir conclusiones superficialmente contradictorias. La buena práctica consiste en explorar el espacio de parámetros sistemáticamente y reportar la robustez de los hallazgos ante variaciones razonables.

117.9. Resumen Conceptual

Este capítulo ha desarrollado los fundamentos del análisis de reglas de asociación con énfasis en la interpretación crítica. Los puntos clave que el lector debe retener son los siguientes:

Sobre las definiciones fundamentales:

- El soporte mide la prevalencia de un patrón; la confianza mide su poder predictivo condicional; el lift mide su desviación respecto a la independencia estadística.
- Ninguna de las tres métricas es suficiente por sí sola: soporte alto no implica interés, confianza alta no implica relación genuina, lift alto con soporte bajo puede ser ruido.

Sobre la generación de reglas:

- El proceso de generación es automático y produce grandes volúmenes de reglas; el proceso de selección requiere juicio estadístico y conocimiento del dominio.
- Los parámetros de Apriori ($\sigma_{\min}$, $\gamma_{\min}$) son decisiones del analista sin fundamento estadístico intrínseco; siempre deben reportarse junto con los resultados.

Sobre el problema central:

- Alta confianza no equivale a relación fuerte cuando el consecuente es frecuente. El lift corrige este sesgo.
- El umbral mínimo de confianza para que una regla sea no-trivial es $\text{conf}(A \Rightarrow B) > P(B)$.

Sobre la selección de reglas útiles:

- El scatter lift-vs-confianza con soporte codificado en tamaño es la visualización más informativa para explorar el espacio de reglas.
- Métricas complementarias como leverage, conviction y Zhang's metric permiten corroborar o cuestionar lo que el lift sugiere. Las reglas robustas son aquellas que resultan sólidas según múltiples criterios simultáneamente.
- La dirección de la regla importa para la toma de decisiones: el lift es simétrico pero la confianza no.

Sobre las limitaciones:

- Las reglas de asociación son herramientas descriptivas y correlacionales; no permiten inferir causalidad.
- El enfoque no captura estructura temporal; para secuencias ordenadas se requieren algoritmos especializados.
- Los resultados son inestables con datasets pequeños y sensibles a la elección de parámetros.

El objetivo final no es maximizar el número de reglas obtenidas, sino identificar el subconjunto pequeño de reglas que representan patrones genuinamente no triviales, respaldados por suficientes observaciones y consistentes con múltiples métricas de evaluación. Alcanzar este objetivo requiere, más que dominio algorítmico, una comprensión sólida de los fundamentos estadísticos que subyacen a cada métrica.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.

- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.

- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals*

- of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD

- '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global

- Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.

- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.

- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.

- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

118. Más Allá del Lift: Métricas Avanzadas, Sensibilidad y Estructura de Reglas

118.1. Introducción

El capítulo anterior estableció los fundamentos del análisis de reglas de asociación: las definiciones de soporte, confianza y lift, el proceso de generación mediante el algoritmo Apriori, y los criterios básicos para distinguir reglas genuinas de reglas triviales. El argumento central fue que generar reglas es algorítmicamente sencillo, pero que la evaluación crítica de su validez exige ir más allá de la confianza como criterio único.

Este capítulo continúa ese argumento y profundiza en cuatro direcciones que el tratamiento introductorio deja abiertas. En primer lugar, se analizan con rigor las *limitaciones estructurales del lift* como métrica de evaluación: aunque el lift corrige el sesgo de la confianza, tiene sus propias deficiencias que la literatura especializada ha documentado con detalle, y que son frecuentemente ignoradas en presentaciones introductorias. En segundo lugar, se introduce un conjunto de *métricas complementarias* —leverage, conviction y la métrica de Zhang— cuyas propiedades matemáticas permiten evaluar aspectos de las reglas que el lift no captura, y cuya combinación produce evaluaciones más robustas. En tercer lugar, se aborda el problema de la *sensibilidad a parámetros*: se demuestra empíricamente que el número, la naturaleza y la calidad de las reglas obtenidas dependen de manera no trivial de los umbrales que el analista fija al invocar el algoritmo, lo cual tiene consecuencias metodológicas directas. Finalmente, se propone una representación del conjunto de reglas como *grafo de relaciones* entre ítems, que permite identificar la estructura global del espacio de reglas y distinguir patrones centrales de patrones periféricos.

El hilo conductor de todas estas secciones es el mismo que motivó el capítulo anterior: el objetivo no es maximizar el número de reglas reportadas, sino identificar el subconjunto pequeño de reglas que representan patrones no triviales, respaldados por evidencia consistente desde múltiples perspectivas.

118.2. Las Limitaciones Estructurales del Lift

118.2.1. La Simetría del Lift y la Asimetría de la Decisión

Una propiedad algebraica del lift que pocas presentaciones introductorias discuten en profundidad es su *simetría*: para cualquier par de itemsets A y B ,

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)} = \frac{P(A \cup B)}{P(B) \cdot P(A)} = \text{lift}(B \Rightarrow A) \quad (118.1)$$

La simetría se sigue directamente de la conmutatividad del producto en el denominador. El lift mide la desviación de la probabilidad conjunta $P(A \cup B)$ respecto al valor esperado bajo independencia $P(A) \cdot P(B)$, y esta desviación es la misma independientemente de cuál de los dos conjuntos se designe como antecedente y cuál como consecuente.

Esta simetría tiene una consecuencia práctica importante que con frecuencia pasa inadvertida. Cuando el algoritmo genera las dos reglas $A \Rightarrow B$ y $B \Rightarrow A$ a partir del mismo itemset frecuente $\{A, B\}$, ambas reglas tendrán exactamente el mismo lift. Sin embargo, sus confianzas serán, en general, distintas:

$$\text{conf}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A)}, \quad \text{conf}(B \Rightarrow A) = \frac{P(A \cup B)}{P(B)}$$

La regla con confianza más alta es aquella cuyo antecedente tiene *menor* soporte individual. Si $P(A) < P(B)$, entonces $\text{conf}(A \Rightarrow B) > \text{conf}(B \Rightarrow A)$: predecir el ítem más frecuente a partir del menos frecuente es más “fácil” en el sentido de la confianza, precisamente porque el consecuente ocurre con mayor frecuencia de base.

La Tabla 118.1 ilustra este fenómeno con datos empíricos sobre el par yogurt — whole milk:

```
par = reglas_base[
    (reglas_base["antecedents_str"].isin(["whole milk", "yogurt"])) &
    (reglas_base["consequents_str"].isin(["whole milk", "yogurt"])) &
    (reglas_base["antecedents"].apply(len) == 1)
][["regla", "antecedent support", "consequent support", "support",
    "confidence", "lift"]].round(3)
par
```

Tabla 118.1.: Las dos direcciones de la relación yogurt — whole milk tienen el mismo lift pero confianzas distintas. La dirección $\{\text{yogurt}\} \rightarrow \{\text{whole milk}\}$ tiene mayor confianza porque whole milk es el ítem más frecuente de los dos.

	regla	antecedent support	consequent support	support	confidence	lift
24	whole milk $\rightarrow$ yogurt	0.52	0.28	0.16	0.308	1.099
25	yogurt $\rightarrow$ whole milk	0.28	0.52	0.16	0.571	1.099

La implicación práctica es directa: cuando el objetivo es tomar una acción condicionada a la observación de un ítem —“¿qué recomiendo a alguien que compra X ?”— la dirección de la regla importa, y el lift por sí solo no permite elegirla. La confianza, a pesar de sus limitaciones ya documentadas, es la métrica relevante para guiar esa elección. Un sistema de recomendación que ordene reglas exclusivamente por lift y tome la primera dirección que encuentre puede estar recomendando la dirección subóptima.

118.2.2. El Techo Teórico del Lift y la Comparabilidad entre Reglas

Una segunda limitación del lift, igualmente importante y menos conocida, es que su escala no es universal: el valor máximo que el lift puede alcanzar para un par dado depende de las frecuencias individuales de los ítems involucrados. Esta propiedad hace que comparar lifts entre reglas con distintos niveles de soporte individual sea, en sentido estricto, metodológicamente incorrecto.

Para derivar el techo teórico, obsérvese que el soporte de la co-ocurrencia $P(A \cup B)$ está acotado superiormente por el soporte del ítem menos común de los dos: si $P(A) \leq P(B)$, entonces $P(A \cup B) \leq P(A)$ (no pueden co-ocurrir más transacciones de las que contienen al ítem más raro). Por lo tanto:

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)} \leq \frac{P(A)}{P(A) \cdot P(B)} = \frac{1}{P(B)} = \frac{1}{\max(P(A), P(B))} \quad (118.2)$$

El lift máximo teórico para el par (A, B) es:

$$\text{lift}_{\text{máx}}(A, B) = \frac{1}{\text{máx}(P(A), P(B))} \quad (118.3)$$

Este resultado tiene una consecuencia inmediata: dos ítems frecuentes —digamos, con soporte 0.50 cada uno— tienen un lift máximo teórico de apenas 2.0. Dos ítems raros —con soporte 0.05 cada uno— tienen un lift máximo teórico de 20. Un lift observado de 1.8 para el primer par representa el 90 % del máximo alcanzable; el mismo lift de 1.8 para el segundo par representa solo el 9 %. La misma cifra señala una regla casi óptima en un caso y una regla débil en el otro.

Para corregir esta asimetría de escala se puede construir el *lift relativo*, definido como la fracción del máximo teórico que alcanza el lift observado:

$$\text{lift}_{\text{rel}}(A \Rightarrow B) = \frac{\text{lift}(A \Rightarrow B)}{\text{lift}_{\text{máx}}(A, B)} = \text{lift}(A \Rightarrow B) \cdot \text{máx}(P(A), P(B)) \quad (118.4)$$

El lift relativo tiene rango $(0, 1]$ y es directamente comparable entre reglas con distintos niveles de frecuencia. Un lift relativo cercano a 1 indica que los ítems co-ocurren con la máxima frecuencia posible dado sus soportes individuales.

```
freq_rel = df.mean()

def lift_max_teorico(row):
    pa = freq_rel.get(row["antecedents_str"], np.nan)
    pb = freq_rel.get(row["consequents_str"], np.nan)
    if pd.isna(pa) or pd.isna(pb):
        return np.nan
    return 1.0 / max(pa, pb)

reglas_simples = reglas_base[
    reglas_base["antecedents"].apply(len) == 1
].copy()
reglas_simples["lift_max_teorico"] = reglas_simples.apply(lift_max_teorico, axis=1)
reglas_simples["lift_relativo"] = (
    reglas_simples["lift"] / reglas_simples["lift_max_teorico"]
).round(3)

reglas_simples[["regla", "lift", "lift_max_teorico", "lift_relativo"]].round(3).sort_values(
    "lift_relativo", ascending=False
).head(10)
```

Tabla 118.2.: Lift observado, lift máximo teórico y lift relativo para las reglas con antecedentes simples. Las columnas lift y lift_max_teorico no son directamente comparables entre filas; el lift_relativo sí lo es.

	regla	lift	lift_max_teorico	lift_relativo
14	chips $\rightarrow$ soda	6.250	6.250	1.000
23	yogurt $\rightarrow$ granola	3.571	3.571	1.000

Tabla 118.2.: Lift observado, lift máximo teórico y lift relativo para las reglas con antecedentes simples. Las columnas lift y lift_max_teorico no son directamente comparables entre filas; el lift_relativo sí lo es.

	regla	lift	lift_max_teorico	lift_relativo
22	granola → yogurt	3.571	3.571	1.000
15	soda → chips	6.250	6.250	1.000
13	eggs → cheese	2.551	3.571	0.714
12	cheese → eggs	2.551	3.571	0.714
2	bread → coffee	1.562	2.500	0.625
3	coffee → bread	1.562	2.500	0.625
17	coffee → whole milk	1.202	1.923	0.625
16	whole milk → coffee	1.202	1.923	0.625

```
fig = px.scatter(
    reglas_simples.dropna(subset=["lift_relativo"]),
    x="lift",
    y="lift_relativo",
    size="support",
    color="confidence",
    hover_data={
        "regla": True,
        "lift": ":{.3f}",
        "lift_max_teorico": ":{.3f}",
        "lift_relativo": ":{.3f}",
    },
    color_continuous_scale="Viridis",
    labels={
        "lift": "Lift observado",
        "lift_relativo": "Lift relativo (fracción del máximo teórico)",
        "confidence": "Confianza",
    },
    size_max=25,
)
fig.add_hline(
    y=0.5,
    line_dash="dot",
    line_color="darkorange",
    annotation_text="50% del máximo teórico",
    annotation_position="bottom right",
)
fig.update_layout(height=460)
fig.show()
```

La Figura 118.1 revela una estructura que la simple ordenación por lift absoluto oculta. Las reglas en la esquina superior izquierda —lift observado modesto, pero lift relativo alto— son asociaciones entre ítems poco frecuentes que co-ocurren con una fracción significativa de su máximo teórico posible: son reglas *específicas* y sólidas, aunque no frecuentes. Las reglas en la esquina inferior derecha —lift absoluto más

Unable to display output for mime type(s): text/html

(a) Lift observado (eje horizontal) versus lift relativo —fracción del máximo teórico— (eje vertical). El tamaño del marcador indica el soporte; el color indica la confianza. La línea punteada naranja marca el 50 % del máximo teórico. Puntos en la esquina inferior derecha tienen lift alto en términos absolutos pero están lejos de su potencial máximo, lo que indica que se trata de ítems frecuentes cuya relación es en realidad moderada.

Unable to display output for mime type(s): text/html

(b)

Figura 118.1.

alto, pero lift relativo bajo— involucran ítems frecuentes cuya co-ocurrencia, aunque superior al azar, está lejos de ser tan estrecha como el lift absoluto podría sugerir.

118.3. Métricas Complementarias al Lift

El catálogo de métricas propuestas en la literatura para evaluar el interés de una regla de asociación es extenso: Tan et al. (2006) enumeran y comparan sistemáticamente más de veinte medidas distintas. Ninguna de ellas es universalmente superior; cada una captura una faceta distinta de lo que intuitivamente llamamos “interés”. Este apartado desarrolla en detalle tres métricas que, por sus propiedades matemáticas y su complementariedad con el lift, son especialmente útiles en la práctica.

118.3.1. Leverage: La Diferencia Absoluta sobre el Azar

El leverage de una regla $A \Rightarrow B$ mide la diferencia *absoluta* entre la frecuencia de co-ocurrencia observada y la esperada bajo la hipótesis de independencia estadística:

$$\text{lev}(A \Rightarrow B) = P(A \cup B) - P(A) \cdot P(B) \quad (118.5)$$

Esta definición es idéntica en estructura al numerador del lift, pero sin la normalización por $P(A) \cdot P(B)$. La consecuencia es que el leverage conserva las unidades de probabilidad, lo que le otorga una interpretación directamente operacional: un leverage de 0.05, en un dataset de 1000 transacciones, significa que aproximadamente 50 transacciones contienen tanto A como B más allá de lo esperado bajo independencia. Este es el número de transacciones adicionales atribuibles a la asociación.

El rango teórico del leverage es $[-\min(P(A), P(B)) \cdot (1 - \max(P(A), P(B))), \min(P(A), P(B)) \cdot (1 - \max(P(A), P(B)))]$, centrado en cero. Un leverage positivo indica asociación positiva; negativo, inhibición; cero, independencia —exactamente como el lift, pero medido en una escala absoluta en lugar de relativa.

La diferencia fundamental entre leverage y lift es de escala. El lift normaliza la desviación respecto a la independencia y produce un número adimensional comparable entre reglas con distintos consecuentes. El leverage da la magnitud absoluta de esa desviación, que es más interpretable cuando se quiere cuantificar el *impacto* de una regla: en un sistema de recomendación, el leverage es proporcional al número de transacciones adicionales que la regla puede generar si se actúa sobre ella. Sin embargo, esta misma dependencia de las frecuencias absolutas hace que el leverage sea sistemáticamente bajo para ítems raros, incluso cuando la relación entre ellos es muy fuerte en términos relativos —exactamente el problema opuesto al del lift absoluto.

118.3.2. Conviction: Asimetría y Cuasi-Determinismo

La conviction de una regla $A \Rightarrow B$ fue introducida con la motivación de crear una métrica dirigida —asimétrica, como la confianza— que al mismo tiempo refleje el alejamiento de la independencia estadística (Tan et al. 2006):

$$\text{conv}(A \Rightarrow B) = \frac{1 - P(B)}{1 - \text{conf}(A \Rightarrow B)} = \frac{P(A) \cdot (1 - P(B))}{P(A) - P(A \cup B)} \quad (118.6)$$

La interpretación de la conviction es menos intuitiva que la del lift o el leverage, pero merece un análisis cuidadoso. El numerador $1 - P(B)$ es la frecuencia esperada de los “errores” de la regla —las transacciones en las que B no ocurre— bajo la hipótesis de independencia: si A y B fueran independientes, una fracción $1 - P(B)$ de las transacciones que contienen A no contendría B . El denominador $1 - \text{conf}(A \Rightarrow B)$ es la frecuencia observada de esos mismos errores. El cociente mide, por tanto, cuántas veces *menos* errores comete la regla en comparación con lo esperado bajo independencia.

De esta definición se siguen varias propiedades notables:

- Cuando A y B son independientes, $\text{conf}(A \Rightarrow B) = P(B)$, y $\text{conviction} = (1 - P(B))/(1 - P(B)) = 1$. La conviction vale 1 si y solo si hay independencia —igual que el lift vale 1 en independencia.
- Cuando la confianza se aproxima a 1, el denominador se aproxima a 0, y la conviction tiende a infinito. Una regla con conviction muy alta es cuasi-determinista: la presencia de A predice B casi con certeza.
- Cuando la confianza es menor que $P(B)$ (es decir, cuando hay una asociación negativa), la conviction es menor que 1.
- A diferencia del lift, la conviction es asimétrica: $\text{conv}(A \Rightarrow B) \neq \text{conv}(B \Rightarrow A)$ en general, lo que la hace adecuada para evaluar la fuerza de una regla en su dirección específica.

Esta asimetría es particularmente valiosa en contextos donde la dirección de la regla tiene significado operacional: en sistemas de recomendación, en diagnóstico médico, o en cualquier aplicación donde se actúa sobre el antecedente para predecir el consecuente.

118.3.3. La Métrica de Zhang: Rango Acotado y Robustez

La métrica de Zhang, propuesta para superar algunas de las limitaciones del lift, está definida como (Tan et al. 2006):

$$Z(A, B) = \frac{P(A \cup B) - P(A) \cdot P(B)}{\max\{P(A \cup B) \cdot [1 - P(A)], P(A) \cdot [P(B) - P(A \cup B)]\}} \quad (118.7)$$

El numerador es el mismo que el del leverage: la diferencia entre co-ocurrencia observada e independencia. El denominador es una normalización que garantiza que $Z \in [-1, 1]$ para cualquier par de ítems, independientemente de sus frecuencias individuales. Esta propiedad —que el lift no tiene— hace a la métrica de Zhang directamente comparable entre reglas con distintos niveles de frecuencia.

La normalización en el denominador se elige como el máximo entre dos términos: $P(A \cup B) \cdot [1 - P(A)]$, que domina cuando la co-ocurrencia es alta (asociación positiva fuerte), y $P(A) \cdot [P(B) - P(A \cup B)]$, que domina cuando la co-ocurrencia es baja (asociación negativa o inhibición). Este diseño garantiza que la escala sea simétrica: $Z = 1$ corresponde a co-ocurrencia máxima (A y B co-ocurren siempre que ocurre el más raro de los dos), y $Z = -1$ corresponde a co-ocurrencia mínima (exclusión mutua).

La métrica de Zhang es simétrica —al igual que el lift— y especialmente robusta ante ítems muy frecuentes, que son precisamente los que generan el sesgo más severo en la confianza y el leverage. Por estas razones, Tan et al. (2006) la recomiendan como una de las métricas más fiables para ordenar reglas por interés general cuando no hay información a priori sobre el dominio.

118.3.4. Cálculo y Comparación de las Cuatro Métricas

```
cols = ["regla", "support", "confidence", "lift",
        "leverage", "conviction", "zhang"]
reglas_base[cols].sort_values("zhang", ascending=False).head(8).round(3)
```

Tabla 118.3.: Las seis métricas de evaluación para las ocho reglas con mayor valor de Zhang. Observar cómo el ranking por Zhang difiere del ranking por lift: las reglas que Zhang considera más interesantes no siempre son las de mayor lift absoluto.

	regla	support	confidence	lift	leverage	conviction	zhang
23	yogurt → granola	0.10	0.357	3.571	0.072	1.400	1.000
15	soda → chips	0.14	0.875	6.250	0.118	6.880	1.000
14	chips → soda	0.14	1.000	6.250	0.118	inf	0.977
13	eggs → cheese	0.10	0.357	2.551	0.061	1.338	0.844
22	granola → yogurt	0.10	1.000	3.571	0.072	inf	0.800
21	yogurt → fruit	0.12	0.429	2.143	0.064	1.400	0.741
12	cheese → eggs	0.10	0.714	2.551	0.061	2.520	0.707
20	fruit → yogurt	0.12	0.600	2.143	0.064	1.800	0.667

118.3.5. La Correlación entre Métricas: Dónde Coinciden y Dónde Discrepan

Una pregunta natural ante la multiplicidad de métricas es si son redundantes: si lift, leverage y Zhang siempre coinciden en ordenar las reglas, da igual cuál se use. El análisis de correlación entre métricas responde esta pregunta de manera empírica.

```
metricas_cols = ["support", "confidence", "lift", "leverage", "conviction", "zhang"]
corr_matrix = reglas_base[metricas_cols].corr().round(2)

fig = px.imshow(
    corr_matrix,
    text_auto=True,
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    aspect="auto",
    labels={"color": "Correlación"},
)
fig.update_layout(height=430)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 118.2.: Matriz de correlación de Pearson entre las seis métricas de evaluación calculadas sobre el conjunto completo de reglas. Los valores se muestran en el interior de cada celda. Escala rojo–blanco–azul: rojo intenso = correlación positiva alta, azul intenso = correlación negativa alta, blanco = sin correlación.

La Figura 118.2 revela varias relaciones estructurales de interés. En primer lugar, lift y Zhang muestran correlación positiva alta: ambas métricas miden esencialmente la misma desviación de la independencia, la diferencia es de escala y de comportamiento en los extremos. En segundo lugar, leverage y support muestran correlación positiva moderada: ítems más frecuentes producen leverages más altos en términos absolutos, reflejando la dependencia del leverage en las frecuencias base. En tercer lugar, conviction puede tener correlación baja con las demás métricas en ciertos datasets, precisamente porque es sensible a valores de confianza cercanos a 1 —donde una pequeña variación en confianza produce cambios muy grandes en conviction— mientras que el resto de métricas varía suavemente.

Las zonas de correlación baja o negativa en esta matriz son las más informativas desde el punto de vista analítico: donde dos métricas discrepan, el analista necesita decidir a cuál le concede mayor peso según el contexto del problema. Una regla con lift alto pero leverage bajo involucra ítems frecuentes cuya relación relativa es fuerte pero cuyo impacto absoluto es limitado. Una regla con Zhang alto pero conviction moderada puede ser robusta en términos de asociación general pero no cuasi-determinista en su dirección específica.

118.3.6. El Perfil Multimétrico de una Regla

Una manera de sintetizar visualmente el comportamiento de una regla en todas las métricas simultáneamente es el gráfico de radar normalizado. Para cada regla, se normalizan las métricas al rango $[0, 1]$ mediante una transformación min-max —exclusivamente con fines de visualización, sin implicar que todas las métricas sean igualmente importantes— y se traza el polígono resultante.

```
metricas_viz = ["support", "confidence", "lift", "leverage", "zhang"]
scaler = MinMaxScaler()
reglas_norm = reglas_base.copy()
reglas_norm[metricas_viz] = scaler.fit_transform(reglas_base[metricas_viz])

top5_idx = reglas_base["lift"].nlargest(5).index
top5 = reglas_norm.loc[top5_idx]

fig = go.Figure()
for _, row in top5.iterrows():
    fig.add_trace(go.Scatterpolar(
        r=[row[m] for m in metricas_viz] + [row[metricas_viz[0]]],
        theta=metricas_viz + [metricas_viz[0]],
        fill="toself",
        name=row["regla"][:38],
        opacity=0.55,
    ))

fig.update_layout(
```

```
polar=dict(radialaxis=dict(visible=True, range=[0, 1])),
height=500,
showlegend=True,
legend=dict(x=1.05, y=1.0),
)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 118.3.: Perfil normalizado de las cinco reglas con mayor lift. Cada eje corresponde a una métrica, normalizada al rango [0,1]. Un polígono más grande y más regular indica una regla más robusta según múltiples criterios simultáneamente. La normalización es solo para visualización: no implica que todas las métricas tengan igual peso.

La Figura 118.3 permite comparar simultáneamente cinco reglas en cinco dimensiones. Una regla cuyo polígono es grande y aproximadamente regular es robusta: todas las métricas coinciden en señalarla como interesante. Una regla con un polígono asimétrico —grande en un eje y pequeño en los demás— es una regla que parece buena según un criterio particular pero débil según los demás, lo que debe interpretarse con cautela.

La Tabla 118.4 resume la interpretación cualitativa de los patrones de perfil más frecuentes:

Tabla 118.4.: Interpretación de perfiles multimétricos.

Patrón del perfil	Interpretación
Polígono grande y regular Lift alto, leverage pequeño	Regla robusta: coincidencia entre múltiples métricas Asociación relativa fuerte entre ítems raros; impacto absoluto limitado
Conviction muy grande, resto moderado	Regla cuasi-determinista en su dirección; confianza cercana a 1
Zhang pequeño, lift moderado	El lift puede estar inflado por las frecuencias base; Zhang lo corrige
Support pequeño, todo lo demás pequeño	Patrón raro posiblemente espurio; insuficiente evidencia empírica

118.4. Sensibilidad a los Parámetros del Algoritmo

118.4.1. Los Umbrales como Decisiones sin Fundamento Estadístico Intrínseco

Cuando se invoca el algoritmo Apriori con parámetros específicos —por ejemplo, `min_support = 0.10` y un umbral mínimo de lift de 1.0— se están tomando decisiones que, a diferencia de los parámetros en modelos estadísticos formales, no tienen una justificación estadística intrínseca. No existe una teoría que establezca que el soporte mínimo “correcto” es 0.10 en lugar de 0.08 o 0.12. El analista elige estos umbrales en función de consideraciones prácticas —tiempo de cómputo, número manejable de resultados, conocimiento del dominio— pero el algoritmo los acepta sin cuestionarlos.

Esta circunstancia tiene una consecuencia metodológica seria: el conjunto de reglas obtenido no es una propiedad objetiva del dataset, sino una función conjunta del dataset y de los parámetros elegidos. Dos

analistas que trabajan con el mismo dataset pero con umbrales ligeramente distintos pueden obtener conjuntos de reglas sustancialmente diferentes y, si no son conscientes de esta dependencia, pueden llegar a conclusiones que parecen contradictorias. La buena práctica metodológica exige, como mínimo, explorar la estabilidad de los resultados ante variaciones razonables en los parámetros —lo que se denomina *análisis de sensibilidad*— y reportar esos parámetros junto con los resultados.

118.4.2. Sensibilidad al Soporte Mínimo

El primer experimento de sensibilidad consiste en variar sistemáticamente $\sigma_{\min}$ mientras se mantiene fijo un umbral de lift de 1.0, y registrar cómo cambia el número de reglas obtenidas y la calidad mediana del conjunto resultante.

```
soportes_grid = [0.06, 0.08, 0.10, 0.12, 0.14, 0.16, 0.18, 0.20]
resultados_sens = []

for s in soportes_grid:
    its = apriori(df, min_support=s, use_colnames=True)
    if len(it) == 0:
        resultados_sens.append({
            "_min": s, "Itemsets frecuentes": 0,
            "Reglas": 0, "Lift mediano": np.nan, "Lift máximo": np.nan
        })
        continue
    r = association_rules(it, metric="lift", min_threshold=1.0)
    resultados_sens.append({
        "_min": round(s, 2),
        "Itemsets frecuentes": len(it),
        "Reglas": len(r),
        "Lift mediano": round(r["lift"].median(), 3) if len(r) > 0 else np.nan,
        "Lift máximo": round(r["lift"].max(), 3) if len(r) > 0 else np.nan,
    })

pd.DataFrame(resultados_sens)
```

Tabla 118.5.: Número de itemsets frecuentes, número de reglas generadas, y estadísticos de calidad del conjunto de reglas en función del soporte mínimo. El umbral de lift se mantiene fijo en 1.0.

	<code>_min</code>	Itemsets frecuentes	Reglas	Lift mediano	Lift máximo
0	0.06	37	36	1.896	6.250
1	0.08	28	22	1.648	6.250
2	0.10	26	20	1.605	6.250
3	0.12	18	12	1.374	6.250
4	0.14	15	6	1.099	6.250
5	0.16	11	2	1.099	1.099
6	0.18	7	0	NaN	NaN
7	0.20	6	0	NaN	NaN


```

df_sens = pd.DataFrame(resultados_sens)

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=df_sens["_min"], y=df_sens["Reglas"],
    mode="lines+markers", name="N° de reglas",
    line=dict(color="#e63946", width=2.5), marker=dict(size=9),
    yaxis="y1",
))
fig.add_trace(go.Scatter(
    x=df_sens["_min"], y=df_sens["Lift mediano"],
    mode="lines+markers", name="Lift mediano",
    line=dict(color="#457b9d", width=2.5, dash="dash"), marker=dict(size=9),
    yaxis="y2",
))
fig.update_layout(
    xaxis=dict(title="Soporte mínimo (_min)"),
    yaxis=dict(title="N° de reglas generadas", side="left", showgrid=True),
    yaxis2=dict(title="Lift mediano del conjunto", overlaying="y", side="right", showgrid=False),
    legend=dict(x=0.60, y=0.92),
    height=430,
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 118.4.: Comportamiento del número de reglas (eje izquierdo, rojo) y del lift mediano del conjunto (eje derecho, azul discontinuo) en función del soporte mínimo. Las dos curvas se mueven en sentidos opuestos: más soporte implica menos reglas pero de mayor calidad mediana.

El comportamiento mostrado en la Figura 118.4 no es accidental ni específico de este dataset: es una propiedad estructural del proceso de generación de reglas. Al elevar $\sigma_{\min}$, se excluyen los itemsets menos frecuentes. Muchos de estos itemsets involucran ítems raros que co-ocurren por casualidad en el dataset disponible —son, precisamente, el tipo de patrón más vulnerable a la inestabilidad estadística. Al eliminarlos, el conjunto de reglas restante tiene, en promedio, un soporte mayor y un lift mediano más alto. En términos informales: los patrones que sobreviven filtros más estrictos son, en promedio, más robustos.

La tensión entre ambas curvas refleja el *compromiso fundamental del análisis de reglas de asociación*: no es posible maximizar simultáneamente el número de reglas y su calidad promedio. Todo umbral de soporte es una elección en este espacio de trade-off, y la elección correcta depende del costo relativo de los dos tipos de error: incluir reglas espurias (falsos positivos) versus omitir patrones genuinos pero poco frecuentes (falsos negativos).

118.4.3. Sensibilidad al Umbral de Lift

El segundo experimento mantiene el soporte fijo en 0.10 y varía el umbral mínimo de lift sobre un rango amplio. El resultado muestra la curva de supervivencia del conjunto de reglas: para cada umbral, cuántas reglas lo superan.

```

its_fijo = apriori(df, min_support=0.10, use_colnames=True)
lifts_umbral = np.round(np.arange(0.5, 2.6, 0.1), 1)
curva_supervivencia = []

for lt in lifts_umbral:
    r = association_rules(its_fijo, metric="lift", min_threshold=float(lt))
    curva_supervivencia.append({
        "Umbral mínimo de lift": lt,
        "Reglas supervivientes": len(r),
    })

df_surv = pd.DataFrame(curva_supervivencia)

fig = px.line(
    df_surv,
    x="Umbral mínimo de lift",
    y="Reglas supervivientes",
    markers=True,
    color_discrete_sequence=["#2a9d8f"],
)
fig.add_vline(
    x=1.0, line_dash="dash", line_color="crimson",
    annotation_text="lift = 1 (independencia)",
    annotation_position="top right",
    annotation_font_color="crimson",
)
fig.update_layout(height=390)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 118.5.: Curva de supervivencia del conjunto de reglas en función del umbral mínimo de lift. Cada punto responde a la pregunta: si exijo que lift x , ¿cuántas reglas quedan? Una caída abrupta en algún valor x indica que muchas reglas se concentran justo por encima de ese umbral.

La forma de la curva en la Figura 118.5 contiene información sobre la distribución de lifts en el dataset. Una caída pronunciada en el intervalo $[1.0, 1.3]$ indica que muchas reglas tienen lifts apenas superiores a la independencia —son reglas formalmente no-triviales pero de interés marginal. Una caída más gradual sugiere que los lifts están distribuidos de manera más uniforme sobre un rango amplio. En ambos casos, la curva permite identificar el umbral de lift a partir del cual el conjunto de reglas se estabiliza en un tamaño manejable sin perder los patrones más sólidos.

118.4.4. El Mapa Completo del Espacio de Parámetros

El análisis de sensibilidad más completo consiste en explorar simultáneamente los dos parámetros principales: soporte mínimo y confianza mínima. El resultado es una superficie bidimensional que muestra el número de reglas generadas para cada combinación de valores.

```

grid_support = np.round(np.arange(0.06, 0.22, 0.02), 2)
grid_conf     = np.round(np.arange(0.20, 1.00, 0.10), 2)
heatmap_data = []

for s in grid_support:
    for c in grid_conf:
        its = apriori(df, min_support=float(s), use_colnames=True)
        if len(its) == 0:
            heatmap_data.append({"min_support": s, "min_confidence": c, "n_reglas": 0})
            continue
        r = association_rules(its, metric="confidence", min_threshold=float(c))
        heatmap_data.append({
            "min_support": s,
            "min_confidence": c,
            "n_reglas": len(r),
        })

df_hm = pd.DataFrame(heatmap_data)
pivot = df_hm.pivot(index="min_confidence", columns="min_support", values="n_reglas")

fig = px.imshow(
    pivot,
    color_continuous_scale="YlOrRd",
    labels={"x": "Soporte mínimo", "y": "Confianza mínima", "color": "N° reglas"},
    aspect="auto",
    text_auto=True,
)
fig.update_layout(height=450)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 118.6.: Mapa de calor del número de reglas generadas en función del soporte mínimo (eje horizontal) y la confianza mínima (eje vertical). La zona inferior izquierda —umbrales laxos— produce el máximo de reglas con la menor calidad promedio. La zona superior derecha —umbrales exigentes— produce pocas reglas o ninguna, pero de alta confiabilidad. El analista debe elegir conscientemente su posición en este mapa.

La Figura 118.6 es el resumen más completo del espacio de decisión del analista. Cada celda responde a la pregunta: *si elijo estos umbrales, ¿cuántas reglas obtengo?* La zona de alta densidad (esquina inferior izquierda, color rojo intenso) corresponde a umbrales laxos y produce el mayor número de reglas —pero también el mayor número de reglas espurias. La zona de baja densidad (esquina superior derecha, amarillo pálido o blanco) corresponde a umbrales exigentes y produce pocas reglas, pero cada una de ellas está respaldada por alta frecuencia y alto poder predictivo.

La implicación metodológica más importante de este análisis es la siguiente: no existe un punto en este mapa que sea correcto en abstracto. La elección de umbrales es siempre relativa al contexto del problema. En una aplicación de retail con miles de transacciones diarias, un soporte mínimo de 0.01 puede estar justificado porque incluso el 1 % de transacciones representa cientos de observaciones. En un dataset

médico con 200 pacientes, el mismo umbral correspondería a apenas 2 casos, lo cual es estadísticamente insuficiente para cualquier afirmación. La elección correcta requiere traducir el umbral de soporte a un número absoluto de observaciones y preguntarse si ese número es suficiente para respaldar la afirmación que se quiere hacer.

118.5. La Estructura Global del Espacio de Reglas: Representación como Grafo

118.5.1. Motivación para una Representación en Red

Los análisis de los capítulos anteriores han representado el conjunto de reglas como tablas y como nubes de puntos en el espacio de métricas. Ambas representaciones son útiles para evaluar reglas individuales o comparar pequeños subconjuntos, pero pierden información sobre la *estructura de relaciones* que existe entre los ítems a nivel global. Un ítem puede participar en muchas reglas, algunas como antecedente y otras como consecuente; dos ítems pueden estar conectados directamente mediante una regla, o indirectamente a través de un ítem intermediario. Esta estructura topológica es invisible en una tabla.

La representación en grafo hace explícita esta estructura. Formalmente, se construye un grafo $G = (V, E)$ donde el conjunto de vértices V es el conjunto de ítems únicos que participan en al menos una regla (con los filtros de calidad aplicados), y el conjunto de aristas E contiene una arista (A, B) por cada regla $A \Rightarrow B$ que supera los umbrales mínimos de lift y soporte. Para evitar la proliferación de aristas y mantener la legibilidad, se consideran solo reglas con antecedentes y consecuentes simples (1-itemsets).

118.5.2. Centralidad, Hubs y Nodos Puente

En la teoría de grafos, la *centralidad* de un nodo mide su importancia relativa en la red. Existen múltiples definiciones de centralidad; en el contexto de las reglas de asociación, una medida natural es la suma de los lifts de todas las reglas en las que el ítem participa, como antecedente o como consecuente. Este criterio prioriza los ítems que están involucrados en muchas asociaciones *fuertes*, no simplemente en muchas asociaciones.

```
reglas_grafo = reglas_base[
    (reglas_base["lift"] > 1.1) &
    (reglas_base["support"] >= 0.10) &
    (reglas_base["antecedents"].apply(len) == 1) &
    (reglas_base["consequents"].apply(len) == 1)
].copy()

nodos = list(set(
    reglas_grafo["antecedents_str"].tolist() +
    reglas_grafo["consequents_str"].tolist()
))
n_nodos = len(nodos)
nodo_idx = {nd: i for i, nd in enumerate(nodos)}

angulos = [2 * math.pi * i / n_nodos for i in range(n_nodos)]
xs = [math.cos(a) for a in angulos]
ys = [math.sin(a) for a in angulos]
```

```

# Centralidad por suma de lifts
centralidad = {}
for nd in nodos:
    r_sub = reglas_grafo[
        (reglas_grafo["antecedents_str"] == nd) |
        (reglas_grafo["consequents_str"] == nd)
    ]
    centralidad[nd] = r_sub["lift"].sum()

edge_x, edge_y = [], []
for _, row in reglas_grafo.iterrows():
    x0 = xs[nodo_idx[row["antecedents_str"]]]
    y0 = ys[nodo_idx[row["antecedents_str"]]]
    x1 = xs[nodo_idx[row["consequents_str"]]]
    y1 = ys[nodo_idx[row["consequents_str"]]]
    edge_x += [x0, x1, None]
    edge_y += [y0, y1, None]

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=edge_x, y=edge_y,
    mode="lines",
    line=dict(width=1.2, color="#cccccc"),
    hoverinfo="none",
    showlegend=False,
))

fig.add_trace(go.Scatter(
    x=xs, y=ys,
    mode="markers+text",
    text=nodos,
    textposition="top center",
    textfont=dict(size=10),
    marker=dict(
        size=[max(12, centralidad.get(nd, 1) * 5) for nd in nodos],
        color=[centralidad.get(nd, 0) for nd in nodos],
        colorscale="Plasma",
        showscale=True,
        colorbar=dict(title="Centralidad<br>(Σ lifts)"),
        line=dict(color="white", width=1),
    ),
    hovertext=[
        f"<b>{nd}</b><br>Centralidad: {centralidad.get(nd, 0):.2f}<br>"
        f"Reglas conectadas: {len(reglas_grafo[(reglas_grafo['antecedents_str']==nd)|(reglas_graf"
        for nd in nodos
    ]),
    hoverinfo="text",
    showlegend=False,

```

```

))

fig.update_layout(
    xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    yaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    height=540,
    plot_bgcolor="white",
    margin=dict(t=30, b=30, l=30, r=30),
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 118.7.: Grafo de reglas de asociación para ítems simples con $\text{lift} > 1.1$ y soporte > 0.10 . Cada nodo es un ítem; cada arista es una regla que supera los umbrales. El tamaño y el color de cada nodo representan su centralidad, definida como la suma de los lifts de todas las reglas en las que el ítem participa. Los nodos grandes y brillantes son ítems que participan en muchas asociaciones fuertes; los pequeños y oscuros son ítems periféricos.

La Figura 118.7 hace visible la topología del espacio de reglas. Los ítems con mayor centralidad son aquellos que participan en múltiples asociaciones fuertes, no simplemente en muchas asociaciones. Un ítem puede ser *hub* por dos razones distintas: porque es genuinamente central en la red de co-compras —sus combinaciones con otros ítems son sorprendentemente frecuentes dado lo que sus frecuencias individuales predecirían— o porque es tan frecuente que aparece en casi todas las transacciones y, por tanto, co-ocurre con casi todo, aunque con lifts bajos.

Esta distinción entre centralidad genuina y centralidad espúrea por frecuencia es directamente análoga al problema de la confianza: igual que una confianza alta puede ser un artefacto de la frecuencia base del consecuente, una alta centralidad en el grafo puede ser un artefacto de la alta frecuencia del ítem. El uso del lift —en lugar del conteo de co-ocurrencias— como peso de la centralidad es precisamente el mecanismo que intenta corregir este sesgo.

Un concepto igualmente importante es el de *nodo puente* o *broker*: un ítem con pocos vecinos en el grafo, pero cuyas aristas conectan grupos de ítems que de otro modo estarían desconectados. En el contexto del análisis de canastas de mercado, un nodo puente puede corresponder a un ítem de nicho que funciona como “gatillo” para una cadena de compras específica. Estos nodos son invisibles en un análisis basado exclusivamente en frecuencias, pero pueden ser muy accionables desde el punto de vista del marketing.

118.5.3. El “Whole Milk Problem”: Un Caso de Estudio

El análisis del ítem *whole milk* en el dataset Groceries ilustra de manera concreta los problemas discutidos en las secciones anteriores. Con un soporte de aproximadamente 0.58 en la muestra utilizada —es decir, presente en más de la mitad de todas las transacciones— *whole milk* es un ítem dominante cuya alta frecuencia distorsiona sistemáticamente las métricas de todas las reglas en las que participa como consecuente.

```

p_wm = df["whole milk"].mean()
print(f"Soporte de whole milk: {p_wm:.3f}")
print(f"Aparece en {int(p_wm * len(df))} de {len(df)} transacciones\n")

```

```

wm_consecuente = reglas_base[
    reglas_base["consequents_str"] == "whole milk"
][["regla", "support", "confidence", "lift", "zhang"]].sort_values("lift", ascending=False)

print("Todas las reglas con whole milk como consecuente:")
print(wm_consecuente.round(3).to_string(index=False))

```

Soporte de whole milk: 0.520
Aparece en 26 de 50 transacciones

Todas las reglas con whole milk como consecuente:

	regla	support	confidence	lift	zhang
coffee → whole milk		0.10	0.625	1.202	0.200
yogurt → whole milk		0.16	0.571	1.099	0.125
butter → whole milk		0.14	0.538	1.036	0.046
bread → whole milk		0.16	0.400	0.769	-0.333
eggs → whole milk		0.10	0.357	0.687	-0.388

```

print(f"P(whole milk) = {p_wm:.3f}\n")
print(f"Para lift > 1.0:  conf > {p_wm:.3f}")
print(f"Para lift > 1.2:  conf > {1.2 * p_wm:.3f}")
print(f"Para lift > 1.5:  conf > {1.5 * p_wm:.3f}")
print(f"Para lift > 2.0:  conf > {2.0 * p_wm:.3f}\n")

candidatas = reglas_base[
    (reglas_base["consequents_str"] == "whole milk") &
    (reglas_base["lift"] > 1.2)
]
if len(candidatas) > 0:
    print("Reglas con whole milk como consecuente y lift > 1.2:")
    print(candidatas[["regla", "confidence", "lift"]].round(3).to_string(index=False))
else:
    print("Ninguna regla con whole milk como consecuente supera lift > 1.2.")
    print("Conclusión: whole milk no es un consecuente útil en este dataset.")

```

$P(\text{whole milk}) = 0.520$

Para lift > 1.0: conf > 0.520
 Para lift > 1.2: conf > 0.624
 Para lift > 1.5: conf > 0.780
 Para lift > 2.0: conf > 1.040

Reglas con whole milk como consecuente y lift > 1.2:

	regla	confidence	lift
coffee → whole milk		0.625	1.202

El análisis confirma una conclusión que la teoría predice: cuando $P(B)$ es muy alto, el espacio para que una regla $A \Rightarrow B$ sea genuinamente no-trivial se estrecha dramáticamente. Para que el lift sea mayor que

1.5, se requiere $\text{conf}(A \Rightarrow B) > 1.5 \times 0.58 \approx 0.87$: prácticamente 9 de cada 10 transacciones que contienen A también contienen leche entera. Esta condición es casi imposible de cumplir en un supermercado general donde la leche ya es omnipresente. La conclusión no es que el dataset sea deficiente, sino que *whole milk* es un mal consecuente para el análisis de reglas de asociación: su ubicuidad impide que ninguna regla hacia él sea informativa.

La generalización es inmediata: para cualquier ítem B , el umbral de confianza necesario para que una regla sea no-trivial con factor k es:

$$\text{conf}(A \Rightarrow B) > k \cdot P(B) \quad \text{equivalentemente,} \quad \text{lift}(A \Rightarrow B) > k \quad (118.8)$$

Esta relación permite construir, de manera sistemática, una tabla de umbrales específicos para cada posible consecuente, que el analista puede consultar antes de interpretar cualquier regla.

```
umbrales = []
for item in sorted(df.columns):
    p = df[item].mean()
    umbrales.append({
        "Ítem (consecuente)": item,
        "P(ítem)": round(p, 3),
        "conf mín (lift > 1.0)": round(p, 3),
        "conf mín (lift > 1.5)": round(min(1.0, 1.5 * p), 3),
    })

pd.DataFrame(umbrales).sort_values("P(ítem)", ascending=False)
```

Tabla 118.6.: Umbrales mínimos de confianza necesarios para que una regla sea no-trivial ($\text{lift} > 1.0$) o moderadamente interesante ($\text{lift} > 1.5$), calculados para cada ítem como posible consecuente. Ítems con $P(\text{ítem})$ alto requieren confianzas muy elevadas para generar reglas genuinamente informativas.

	Ítem (consecuente)	P(ítem)	conf mín (lift > 1.0)	conf mín (lift > 1.5)
13	whole milk	0.52	0.52	0.78
1	bread	0.40	0.40	0.60
7	eggs	0.28	0.28	0.42
14	yogurt	0.28	0.28	0.42
2	butter	0.26	0.26	0.39
8	fruit	0.20	0.20	0.30
12	vegetables	0.18	0.18	0.27
6	coffee	0.16	0.16	0.24
10	soda	0.16	0.16	0.24
4	cheese	0.14	0.14	0.21
5	chips	0.14	0.14	0.21
9	granola	0.10	0.10	0.15
11	sugar	0.10	0.10	0.15
0	beer	0.04	0.04	0.06
3	candy	0.04	0.04	0.06

118.6. Un Marco Integrado de Evaluación

118.6.1. De la Fragmentación a la Coherencia

Los conceptos desarrollados en este capítulo —limitaciones del lift, métricas complementarias, sensibilidad a parámetros, estructura de grafo— no son independientes entre sí. Forman un marco coherente para responder una única pregunta: *¿cuánta confianza merece una regla particular?*

La respuesta a esa pregunta tiene múltiples dimensiones:

1. **Dimensión relativa:** ¿cuánto se aleja el lift observado de la independencia, en términos del máximo alcanzable dado los soportes de los ítems? El lift relativo responde esta pregunta.
2. **Dimensión absoluta:** ¿cuántas transacciones adicionales, sobre el baseline de independencia, exhiben la co-ocurrencia? El leverage responde esta pregunta.
3. **Dimensión predictiva direccional:** ¿cuán determinista es la regla en su dirección específica? La conviction responde esta pregunta.
4. **Dimensión de robustez normalizada:** ¿es la asociación fuerte en una escala comparable entre ítems de frecuencias distintas? Zhang responde esta pregunta.
5. **Dimensión de estabilidad paramétrica:** ¿la regla sobrevive con cualquier elección razonable de umbrales, o desaparece si se endurece ligeramente $\sigma_{\min}$? El análisis de sensibilidad responde esta pregunta.

Una regla que es sólida en todas estas dimensiones merece alta confianza. Una regla que solo lo es en una o dos puede ser válida pero debe interpretarse con mayor cautela. La siguiente función implementa esta evaluación multidimensional de manera sistemática:

```
def evaluar_regla_completa(row, n_transacciones=50, verbose=True):
    """
    Evaluación multidimensional de una regla de asociación.
    Devuelve clasificación: ROBUSTA / CANDIDATA / DESCARTAR
    """
    criterios = []
    puntos = 0

    # Dimensión 1: Lift
    if row["lift"] > 1.5:
        criterios.append("Lift fuerte (> 1.5)")
        puntos += 3
    elif row["lift"] > 1.1:
        criterios.append("Lift moderado (1.1 - 1.5)")
        puntos += 1
    else:
        criterios.append("Lift débil ( 1.1) - relación no distinguible del azar")

    # Dimensión 2: Soporte absoluto
    n_obs = int(row["support"] * n_transacciones)
    if n_obs >= 10:
        criterios.append(f"Soporte sólido ({n_obs} transacciones de {n_transacciones})")
```

```

    puntos += 2
elif n_obs >= 5:
    criterios.append(f"Soporte mínimo  ({n_obs} transacciones de {n_transacciones})")
    puntos += 1
else:
    criterios.append(f"Soporte insuficiente  ({n_obs} transacciones de {n_transacciones})")

# Dimensión 3: Zhang (robustez normalizada)
if row["zhang"] > 0.3:
    criterios.append("Zhang robusto  (> 0.3)")
    puntos += 2
elif row["zhang"] > 0.1:
    criterios.append("Zhang moderado  (0.1 - 0.3)")
    puntos += 1
else:
    criterios.append("Zhang débil  ( 0.1) - posiblemente artefacto de frecuencia base")

# Dimensión 4: Conviction (determinismo direccional)
if row["conviction"] > 1.5:
    criterios.append("Conviction alta  (> 1.5) - regla cuasi-determinista")
    puntos += 1

clasificacion = (
    "ROBUSTA"      if puntos >= 6 else
    "CANDIDATA"    if puntos >= 3 else
    "DESCARTAR"
)

if verbose:
    print(f"{' '*55}")
    print(f"Regla:  {row['regla']}")
    print(f"{' '*55}")
    for c in criterios:
        print(f"    • {c}")
    print(f"\n → Clasificación: {clasificacion}  ({puntos}/8 puntos)\n")

return clasificacion

# Aplicar a las 6 reglas con mayor lift
print("Evaluación de las 6 reglas con mayor lift:\n")
for _, row in reglas_base.sort_values("lift", ascending=False).head(6).iterrows():
    evaluar_regla_completa(row)

```

Evaluación de las 6 reglas con mayor lift:

Regla: soda → chips

- Lift fuerte (> 1.5)

- Soporte mínimo (7 transacciones de 50)
- Zhang robusto (> 0.3)
- Conviction alta (> 1.5) - regla cuasi-determinista

→ Clasificación: ROBUSTA (7/8 puntos)

Regla: chips → soda

- Lift fuerte (> 1.5)
- Soporte mínimo (7 transacciones de 50)
- Zhang robusto (> 0.3)
- Conviction alta (> 1.5) - regla cuasi-determinista

→ Clasificación: ROBUSTA (7/8 puntos)

Regla: yogurt → granola

- Lift fuerte (> 1.5)
- Soporte mínimo (5 transacciones de 50)
- Zhang robusto (> 0.3)

→ Clasificación: ROBUSTA (6/8 puntos)

Regla: granola → yogurt

- Lift fuerte (> 1.5)
- Soporte mínimo (5 transacciones de 50)
- Zhang robusto (> 0.3)
- Conviction alta (> 1.5) - regla cuasi-determinista

→ Clasificación: ROBUSTA (7/8 puntos)

Regla: eggs → cheese

- Lift fuerte (> 1.5)
- Soporte mínimo (5 transacciones de 50)
- Zhang robusto (> 0.3)

→ Clasificación: ROBUSTA (6/8 puntos)

Regla: cheese → eggs

- Lift fuerte (> 1.5)
- Soporte mínimo (5 transacciones de 50)
- Zhang robusto (> 0.3)

- Conviction alta (> 1.5) - regla cuasi-determinista
- Clasificación: ROBUSTA (7/8 puntos)

118.6.2. Lo que Cambia con Este Marco

La diferencia entre el análisis de reglas de asociación tratado en los textos introductorios y el análisis desarrollado en estos dos capítulos no reside en el código: las funciones de `mlxtend` son las mismas. La diferencia reside en las preguntas que el analista se hace antes de interpretar los resultados y en los instrumentos conceptuales que usa para responderlas.

La Tabla 126.3 resume esta diferencia de manera esquemática:

Tabla 118.7.: Comparación entre enfoque elemental y enfoque crítico del análisis de reglas de asociación.

Dimensión	Enfoque elemental	Enfoque crítico
Métricas consideradas	Soporte, confianza	Lift, lift relativo, leverage, conviction, Zhang
Dirección de la regla	Ignorada	Evaluada mediante la asimetría de confianza
Comparabilidad del lift	Asumida universal	Corregida con lift relativo
Umbral de los parámetros	Fijos por convención	Explorados mediante análisis de sensibilidad
Visualización	Tablas ordenadas	Scatter de métricas + radar + grafo
Criterio de selección final	Confianza o lift únicos	Convergencia entre múltiples métricas

La tabla pone en evidencia que el análisis crítico no es simplemente más trabajo —es un análisis fundamentalmente diferente en su estructura epistémica. El analista que usa el marco crítico genera, en la mayoría de los casos, un conjunto de reglas reportadas más pequeño que el analista que usa el enfoque elemental. Pero ese conjunto más pequeño está respaldado por evidencia más robusta, es más estable ante variaciones en los parámetros, y es más accionable en la práctica.

118.7. Resumen Conceptual

Este capítulo ha profundizado en el análisis de reglas de asociación más allá de las métricas fundamentales introducidas en el capítulo anterior. Los puntos clave que el lector debe retener son los siguientes:

Sobre las limitaciones estructurales del lift:

- El lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Para elegir la dirección apropiada de una regla en una aplicación concreta, la confianza —métrica asimétrica— es el criterio relevante.
- El lift tiene un techo teórico que depende de los soportes individuales: $\text{lift}_{\text{máx}} = 1 / \max(P(A), P(B))$. Comparar lifts absolutos entre reglas con distintos soportes puede ser engañoso. El lift relativo —cociente entre lift observado y lift máximo— es una escala más justa para la comparación.

Sobre las métricas complementarias:

- El leverage mide la diferencia absoluta entre co-ocurrencia observada e independencia. Es interpretable como el número de transacciones adicionales atribuibles a la asociación, pero es sistemáticamente bajo para ítems raros.
- La conviction es asimétrica y mide el cuasi-determinismo de la regla en su dirección: valores muy altos indican confianza cercana a 1.
- La métrica de Zhang tiene rango $[-1, 1]$, es simétrica y robusta ante ítems frecuentes. Es la métrica recomendada para comparaciones entre reglas con distintos niveles de frecuencia.
- Las métricas no son redundantes: sus correlaciones son imperfectas, y las discrepancias entre ellas son la señal más valiosa para el análisis crítico.

Sobre la sensibilidad a parámetros:

- Los umbrales del algoritmo Apriori no tienen justificación estadística intrínseca. El conjunto de reglas obtenido es función conjunta del dataset y de los parámetros elegidos.
- El análisis de sensibilidad —exploración sistemática del espacio de parámetros— es una práctica metodológica necesaria, no opcional.
- El mapa soporte-confianza visualiza el espacio completo de decisión y permite al analista elegir conscientemente su posición en el compromiso entre número y calidad de reglas.

Sobre la representación en grafo:

- El grafo de reglas hace visible la estructura topológica del espacio de co-ocurrencias: hubs, nodos puente y clústeres de ítems relacionados.
- La centralidad basada en lift —no en conteo de aristas— permite distinguir ítems genuinamente centrales de ítems centrales por mera frecuencia.
- El “Whole Milk Problem” ilustra cómo la alta frecuencia de un ítem como consecuente hace prácticamente imposible encontrar reglas genuinamente informativas hacia él, independientemente de los umbrales elegidos.

Sobre el marco integrado:

- La evaluación robusta de una regla requiere examinar simultáneamente al menos cuatro dimensiones: lift relativo, leverage, Zhang y conviction.
- Las reglas robustas son aquellas en las que múltiples métricas convergen en señalarlas como interesantes; las reglas candidatas son aquellas donde hay convergencia parcial; las reglas descartables son aquellas donde las métricas más exigentes las rechazan.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.

- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.

- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.

- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*.

- Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwon Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2):

427-32. <https://doi.org/10.2307/1934352>.

Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.

Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.

Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.

Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.

Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.

Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.

Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.

Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.

IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.

Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.

James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.

Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.

Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.

Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.

- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.

- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.

- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias,

Universidad Nacional Autónoma de México.

- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.

- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

119. FP-Growth y minería de patrones frecuentes

120. FP-Growth y minería de patrones frecuentes

120.1. Introducción

La minería de reglas de asociación busca descubrir regularidades en bases de datos transaccionales: productos que suelen comprarse juntos, páginas que suelen visitarse en una misma sesión, palabras que coaparecen en documentos o eventos que tienden a registrarse en conjunto. En este contexto, una transacción representa una observación compuesta por varios ítems; por ejemplo, una compra de supermercado puede contener {Pan, Leche, Pañales}. El objetivo no es predecir una etiqueta, sino identificar estructuras de coocurrencia que permitan extraer conocimiento interpretable a partir de datos discretos (Agrawal y Srikant 1994; Han et al. 2012).

El algoritmo **FP-Growth** (*Frequent Pattern Growth*) fue propuesto como una alternativa eficiente a Apriori para encontrar conjuntos frecuentes de ítems. Su idea central es evitar la generación explícita de candidatos y comprimir la base de transacciones en una estructura llamada **FP-tree** (*Frequent Pattern Tree*). A partir de ese árbol, el algoritmo explora patrones frecuentes mediante bases condicionales y árboles condicionales (Han et al. 2000; Borgelt 2005).

Este capítulo desarrolla FP-Growth de manera progresiva. Primero se revisan las nociones de transacción, itemset, soporte y frecuencia. Después se muestra cómo construir un FP-tree usando un ejemplo pequeño de cesta de compra. Finalmente, se explica cómo extraer patrones frecuentes mediante caminos prefijo condicionales, se compara FP-Growth con Apriori y se implementa un flujo práctico en Python.

120.2. Conceptos básicos

En minería de asociaciones, trabajamos con una colección de transacciones. Cada transacción contiene un subconjunto de ítems tomado de un universo finito de productos, términos o eventos posibles. Esta representación es natural en problemas donde los datos son conjuntos: una compra contiene productos, un documento contiene palabras, una sesión web contiene páginas visitadas.

Definición. Sea $I = \{i_1, i_2, \dots, i_m\}$ un conjunto finito de ítems. Una transacción T es un subconjunto de I , es decir:

$$T \subseteq I.$$

Una base transaccional es una colección $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$ de transacciones.

Definición. Un itemset es cualquier subconjunto $X \subseteq I$. Decimos que una transacción T_i contiene a X si todos los ítems de X aparecen en T_i :

$$X \subseteq T_i.$$

El concepto clave para decidir si un patrón es relevante es el **soporte**. Existen dos formas comunes de reportarlo: como conteo absoluto o como proporción. El conteo absoluto indica cuántas transacciones contienen el patrón; el soporte relativo divide ese conteo entre el número total de transacciones.

Definición. El soporte absoluto de un itemset X en una base transaccional $\mathcal{D}$ es el número de transacciones que contienen a X :

$$\text{supp}_{\#}(X) = |\{T_i \in \mathcal{D} : X \subseteq T_i\}|.$$

El soporte relativo se define como:

$$\text{supp}(X) = \frac{\text{supp}_{\#}(X)}{|\mathcal{D}|}. \quad (120.1)$$

Definición. Dado un umbral mínimo de soporte min_sup , un itemset X es frecuente si su soporte absoluto cumple:

$$\text{supp}_{\#}(X) \geq \text{min_sup}. \quad (120.2)$$

El umbral min_sup controla la granularidad del análisis. Un valor demasiado bajo puede producir demasiados patrones, muchos de ellos triviales o difíciles de interpretar. Un valor demasiado alto puede eliminar patrones interesantes pero menos comunes. Por ello, en aplicaciones reales suele elegirse mediante exploración, conocimiento del dominio y restricciones computacionales.

120.3. De Apriori a FP-Growth

El algoritmo Apriori explota una propiedad fundamental: si un conjunto de ítems es frecuente, entonces todos sus subconjuntos también son frecuentes. De manera equivalente, si un itemset no es frecuente, ningún superconjunto suyo puede ser frecuente (Agrawal y Srikant 1994). Esta propiedad permite podar el espacio de búsqueda, pero Apriori todavía necesita generar muchos candidatos y escanear la base varias veces.

FP-Growth sigue otra estrategia. En lugar de construir y evaluar candidatos explícitamente, realiza dos recorridos principales sobre la base de datos: uno para calcular los soportes de los ítems individuales y otro para construir un árbol compacto. Después extrae patrones frecuentes desde la estructura comprimida (Han et al. 2000).

La ventaja aparece especialmente cuando muchas transacciones comparten prefijos. Si los ítems se ordenan globalmente por frecuencia decreciente, transacciones parecidas tienden a compartir las primeras ramas del árbol. Así, el FP-tree almacena información agregada en lugar de repetir cada transacción completa.

120.4. Ejemplo base: transacciones de cesta de compra

Considérese la siguiente base de transacciones y un soporte mínimo absoluto $\text{min_sup} = 3$:

Transacción	Ítems
T_1	{Pan, Leche}
T_2	{Pan, Leche, Pañales, Cerveza}

Transacción	Ítems
T_3	{Leche, Pañales, Cerveza}
T_4	{Pan, Leche, Pañales, Cerveza}
T_5	{Pan, Leche, Pañales}

Los soportes absolutos de los ítems individuales son:

Ítem	Transacciones donde aparece	Soporte absoluto
Pan	T_1, T_2, T_4, T_5	4
Leche	T_1, T_2, T_3, T_4, T_5	5
Pañales	T_2, T_3, T_4, T_5	4
Cerveza	T_2, T_3, T_4	3

Como todos los ítems tienen soporte absoluto mayor o igual que 3, ninguno se elimina en esta etapa. Si hubiera ítems con soporte menor que `min_sup`, se descartarían antes de construir el árbol.

```
# Ejemplo base de transacciones
transactions = [
    {"Pan", "Leche"},
    {"Pan", "Leche", "Pañales", "Cerveza"},
    {"Leche", "Pañales", "Cerveza"},
    {"Pan", "Leche", "Pañales", "Cerveza"},
    {"Pan", "Leche", "Pañales"},
]

min_sup = 3

from collections import Counter

item_counts = Counter()
for transaction in transactions:
    item_counts.update(transaction)

item_counts
```

```
Counter({'Leche': 5, 'Pan': 4, 'Pañales': 4, 'Cerveza': 3})
```

120.5. Ordenamiento por frecuencia

El FP-tree depende de un orden global de ítems. La regla usual consiste en ordenar los ítems de mayor a menor frecuencia. En caso de empates, conviene fijar un criterio determinista, por ejemplo orden alfabético. Este ordenamiento hace que transacciones con ítems frecuentes compartan prefijos y que el árbol sea más compacto.

En el ejemplo, el orden global puede tomarse como:

Leche $\succ$ Pan $\succ$ Pañales $\succ$ Cerveza.

Con ese orden, las transacciones se transforman en:

Transacción original	Transacción ordenada
{Pan, Leche}	[Leche, Pan]
{Pan, Leche, Pañales, Cerveza}	[Leche, Pan, Pañales, Cerveza]
{Leche, Pañales, Cerveza}	[Leche, Pañales, Cerveza]
{Pan, Leche, Pañales, Cerveza}	[Leche, Pan, Pañales, Cerveza]
{Pan, Leche, Pañales}	[Leche, Pan, Pañales]

```
# Orden global por frecuencia descendente y desempate alfabético
frequent_items = {item for item, count in item_counts.items() if count >= min_sup}
order = sorted(frequent_items, key=lambda item: (-item_counts[item], item))
rank = {item: position for position, item in enumerate(order)}

ordered_transactions = []
for transaction in transactions:
    filtered = [item for item in transaction if item in frequent_items]
    ordered = sorted(filtered, key=lambda item: rank[item])
    ordered_transactions.append(ordered)

order, ordered_transactions
```

```
(['Leche', 'Pan', 'Pañales', 'Cerveza'],
 [['Leche', 'Pan'],
  ['Leche', 'Pan', 'Pañales', 'Cerveza'],
  ['Leche', 'Pañales', 'Cerveza'],
  ['Leche', 'Pan', 'Pañales', 'Cerveza'],
  ['Leche', 'Pan', 'Pañales']])
```

120.6. Construcción del FP-tree

Un FP-tree es un árbol con raíz nula. Cada nodo almacena un ítem, un contador y enlaces hacia sus hijos. Para insertar una transacción ordenada, se recorre la transacción de izquierda a derecha. Si el siguiente ítem ya existe como hijo del nodo actual, se incrementa su contador. Si no existe, se crea un nuevo nodo con contador inicial igual a uno.

Definición. Un FP-tree es una estructura arbórea $\mathcal{T}$ donde cada nodo no raíz representa un ítem y almacena un contador. El contador de un nodo indica cuántas transacciones comparten el prefijo correspondiente al camino desde la raíz hasta ese nodo:

$$\text{count}(v) = |\{T_i \in \mathcal{D} : \text{path}(v) \subseteq T_i\}|.$$

El procedimiento de construcción puede resumirse así:

1. Crear una raíz `null`.
2. Ordenar cada transacción de acuerdo con la frecuencia global.
3. Insertar cada transacción en el árbol.
4. Incrementar contadores cuando una rama ya existe.
5. Crear nodos nuevos cuando el prefijo todavía no aparece.

```
class FPNode:
    def __init__(self, item=None, count=0, parent=None):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}

    def add_child(self, item):
        child = FPNode(item=item, count=0, parent=self)
        self.children[item] = child
        return child

def insert_transaction(root, transaction):
    current = root
    for item in transaction:
        if item not in current.children:
            current.add_child(item)
        current = current.children[item]
        current.count += 1

def build_fp_tree(ordered_transactions):
    root = FPNode(item="null", count=0)
    for transaction in ordered_transactions:
        insert_transaction(root, transaction)
    return root

def print_tree(node, indent=0):
    label = f"{node.item}: {node.count}" if node.item != "null" else "null"
    print("  " * indent + label)
    for child in node.children.values():
        print_tree(child, indent + 1)

root = build_fp_tree(ordered_transactions)
print_tree(root)
```

```
null
  Leche: 5
    Pan: 4
      Pañales: 3
        Cerveza: 2
```

Pañales: 1
Cerveza: 1

En este ejemplo, la rama más importante inicia con **Leche**, porque aparece en todas las transacciones. Desde **Leche**, varias transacciones continúan con **Pan**; otras continúan directamente con **Pañales**. La compresión proviene de sumar contadores en prefijos compartidos en lugar de almacenar cada transacción de forma independiente.

120.7. Caminos prefijo y base condicional

Una vez construido el FP-tree, FP-Growth extrae patrones frecuentes considerando los ítems desde los menos frecuentes hacia los más frecuentes. Para cada ítem objetivo, se buscan todos los caminos que terminan en ese ítem. Al quitar el ítem objetivo de esos caminos se obtiene su **base de patrones condicional**.

Definición. Dado un ítem a , la base de patrones condicional de a , denotada $\mathcal{B}_a$, es el multiconjunto de caminos prefijo que aparecen antes de a en el FP-tree, ponderados por el contador del nodo asociado a a :

$$\mathcal{B}_a = \{(P_j, c_j) : P_j \text{ es un camino prefijo de } a \text{ y } c_j \text{ es su conteo}\}.$$

Para el ítem **Cerveza**, los caminos relevantes son:

Camino en el árbol	Conteo
[Leche, Pan, Pañales, Cerveza]	2
[Leche, Pañales, Cerveza]	1

Al eliminar **Cerveza**, se obtiene:

Camino prefijo	Conteo
[Leche, Pan, Pañales]	2
[Leche, Pañales]	1

En esta base condicional, los conteos acumulados son:

Ítem	Conteo condicional
Leche	3
Pan	2
Pañales	3

Con $\text{min_sup} = 3$, **Pan** se elimina de la base condicional de **Cerveza**, mientras que **Leche** y **Pañales** permanecen. Por tanto, los patrones frecuentes que incluyen **Cerveza** son:

$\{\text{Cerveza}\}, \quad \{\text{Leche}, \text{Cerveza}\}, \quad \{\text{Pañales}, \text{Cerveza}\}, \quad \{\text{Leche}, \text{Pañales}, \text{Cerveza}\}.$


```
def collect_prefix_paths(node, target_item):
    """Devuelve caminos prefijo hacia target_item dentro de un FP-tree simple."""
    paths = []

    def traverse(current):
        for child in current.children.values():
            if child.item == target_item:
                path = []
                parent = child.parent
                while parent is not None and parent.item != "null":
                    path.append(parent.item)
                    parent = parent.parent
                path.reverse()
                if path:
                    paths.append((path, child.count))
            traverse(child)

    traverse(node)
    return paths

prefix_paths_cerveza = collect_prefix_paths(root, "Cerveza")
prefix_paths_cerveza
```

```
[(['Leche', 'Pan', 'Pañales'], 2), (['Leche', 'Pañales'], 1)]
```

```
def conditional_counts(prefix_paths):
    counts = Counter()
    for path, weight in prefix_paths:
        for item in path:
            counts[item] += weight
    return counts

conditional_counts(prefix_paths_cerveza)
```

```
Counter({'Leche': 3, 'Pañales': 3, 'Pan': 2})
```

120.8. Árbol condicional

Después de obtener la base condicional de un ítem, se eliminan los ítems que no alcanzan el soporte mínimo y se construye un nuevo FP-tree restringido a esa base. Ese árbol se llama **árbol condicional**. En el caso de **Cerveza**, el árbol condicional conserva la estructura:

Leche : 3 $\rightarrow$ Pañales : 3.

Esto significa que, entre las transacciones que contienen **Cerveza**, existen tres ocurrencias donde **Leche** aparece junto con **Cerveza** y tres ocurrencias donde **Pañales** aparece junto con **Cerveza**. Además, el camino conjunto indica que {Leche, Pañales, Cerveza} también es frecuente.

Definición. El árbol condicional de un ítem a es el FP-tree construido a partir de la base condicional $\mathcal{B}_a$ después de eliminar los ítems cuyo conteo condicional es menor que min_sup :

$$\mathcal{T}_a = \text{FPtree}(\{P_j \in \mathcal{B}_a : \text{count}_{\mathcal{B}_a}(i) \geq \text{min_sup}\}).$$

```
def expand_weighted_paths(prefix_paths):
    expanded = []
    for path, weight in prefix_paths:
        expanded.extend([path] * weight)
    return expanded

conditional_item_counts = conditional_counts(prefix_paths_cerveza)
conditional_frequent = {
    item for item, count in conditional_item_counts.items()
    if count >= min_sup
}

conditional_order = sorted(
    conditional_frequent,
    key=lambda item: (-conditional_item_counts[item], item)
)
conditional_rank = {item: pos for pos, item in enumerate(conditional_order)}

conditional_transactions = []
for path in expand_weighted_paths(prefix_paths_cerveza):
    filtered_path = [item for item in path if item in conditional_frequent]
    ordered_path = sorted(filtered_path, key=lambda item: conditional_rank[item])
    if ordered_path:
        conditional_transactions.append(ordered_path)

conditional_root = build_fp_tree(conditional_transactions)
print_tree(conditional_root)
```

```
null
  Leche: 3
  Pañales: 3
```

120.9. Patrones frecuentes del ejemplo

Repitiendo el procedimiento para los ítems restantes en orden ascendente de frecuencia, se obtienen los conjuntos frecuentes:

Ítem base	Patrones frecuentes asociados
Cerveza	{Cerveza}, {Leche, Cerveza}, {Pañales, Cerveza}, {Leche, Pañales, Cerveza}

Ítem base	Patrones frecuentes asociados
Pañales	{Pañales}, {Leche, Pañales}, {Pan, Pañales}, {Leche, Pan, Pañales}
Pan	{Pan}, {Leche, Pan}
Leche	{Leche}

Agrupando y eliminando duplicados, los itemsets frecuentes con `min_sup = 3` son:

{Leche}, {Pan}, {Pañales}, {Cerveza},
 {Leche, Pan}, {Leche, Pañales}, {Leche, Cerveza},
 {Pan, Pañales}, {Pañales, Cerveza},
 {Leche, Pan, Pañales}, {Leche, Pañales, Cerveza}.

Es importante notar que FP-Growth no produce directamente reglas de asociación; primero produce itemsets frecuentes. Las reglas se construyen después a partir de esos itemsets, evaluando métricas como confianza, lift o convicción (Tan et al. 2019; Han et al. 2012).

120.10. Implementación práctica con mlxtend

En Python, una forma sencilla de aplicar FP-Growth es utilizar `mlxtend`, que implementa tanto Apriori como FP-Growth para matrices transaccionales codificadas en formato one-hot (Raschka 2018).

```
# Si es necesario, instalar previamente:
# pip install mlxtend pandas

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

transactions_list = [list(t) for t in transactions]

encoder = TransactionEncoder()
one_hot = encoder.fit(transactions_list).transform(transactions_list)
df = pd.DataFrame(one_hot, columns=encoder.columns_)

df
```

	Cerveza	Leche	Pan	Pañales
0	False	True	True	False
1	True	True	True	True
2	True	True	False	True
3	True	True	True	True
4	False	True	True	True

Como `mlxtend` usa soporte relativo, el umbral absoluto `min_sup = 3` debe transformarse dividiendo entre el número de transacciones:

$$\text{min_sup}_{rel} = \frac{3}{5} = 0.6.$$

```
min_support_relative = min_sup / len(transactions)

frequent_itemsets = fpgrowth(
    df,
    min_support=min_support_relative,
    use_colnames=True
)

frequent_itemsets.sort_values(
    by=["support", "itemsets"],
    ascending=[False, True]
)
```

	support	itemsets
0	1.0	(Leche)
1	0.8	(Pan)
2	0.8	(Pañales)
4	0.8	(Leche, Pan)
7	0.8	(Leche, Pañales)
3	0.6	(Cerveza)
5	0.6	(Pan, Pañales)
6	0.6	(Leche, Pan, Pañales)
8	0.6	(Cerveza, Pañales)
9	0.6	(Leche, Cerveza)
10	0.6	(Leche, Cerveza, Pañales)

A partir de los itemsets frecuentes se pueden generar reglas. Por ejemplo, una regla $X \Rightarrow Y$ se interpreta como: “cuando aparece X , también tiende a aparecer Y ”.

Definición. La confianza de una regla de asociación $X \Rightarrow Y$, con $X \cap Y = \emptyset$, mide la proporción de transacciones que contienen X y también contienen Y :

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}. \quad (120.3)$$

Definición. El lift de una regla $X \Rightarrow Y$ compara la coocurrencia observada de X y Y con la coocurrencia esperada bajo independencia estadística:

$$\text{lift}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X) \text{supp}(Y)}. \quad (120.4)$$

```

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.7
)

selected_columns = [
    "antecedents", "consequents", "support", "confidence", "lift"
]

rules[selected_columns].sort_values(
    by=["lift", "confidence"],
    ascending=False
)

```

	antecedents	consequents	support	confidence	lift
11	(Cerveza)	(Pañales)	0.6	1.00	1.2500
14	(Leche, Cerveza)	(Pañales)	0.6	1.00	1.2500
17	(Cerveza)	(Leche, Pañales)	0.6	1.00	1.2500
12	(Pañales)	(Cerveza)	0.6	0.75	1.2500
15	(Leche, Pañales)	(Cerveza)	0.6	0.75	1.2500
18	(Pañales)	(Leche, Cerveza)	0.6	0.75	1.2500
1	(Pan)	(Leche)	0.8	1.00	1.0000
6	(Pan, Pañales)	(Leche)	0.6	1.00	1.0000
10	(Pañales)	(Leche)	0.8	1.00	1.0000
13	(Cerveza)	(Leche)	0.6	1.00	1.0000
16	(Cerveza, Pañales)	(Leche)	0.6	1.00	1.0000
0	(Leche)	(Pan)	0.8	0.80	1.0000
9	(Leche)	(Pañales)	0.8	0.80	1.0000
2	(Pan)	(Pañales)	0.6	0.75	0.9375
3	(Pañales)	(Pan)	0.6	0.75	0.9375
4	(Leche, Pan)	(Pañales)	0.6	0.75	0.9375
5	(Leche, Pañales)	(Pan)	0.6	0.75	0.9375
7	(Pan)	(Leche, Pañales)	0.6	0.75	0.9375
8	(Pañales)	(Leche, Pan)	0.6	0.75	0.9375

La confianza puede ser alta aun cuando el consecuente sea muy frecuente. Por ello, conviene interpretarla junto con el lift. Un lift mayor que uno sugiere asociación positiva; un lift cercano a uno sugiere que la regla no mejora sustancialmente lo esperado por independencia; un lift menor que uno sugiere asociación negativa.

120.11. Comparación entre FP-Growth y Apriori

Característica	FP-Growth	Apriori
Generación de candidatos	No requerida	Necesaria

Característica	FP-Growth	Apriori
Recorridos de los datos	Usualmente dos recorridos principales	Varios recorridos
Velocidad	Eficiente en bases grandes y densas	Puede ser lento por generación repetida de candidatos
Uso de memoria	Puede ser alto por el FP-tree	Generalmente menor, basado en listas y conteos
Implementación	Más compleja	Más sencilla de explicar e implementar

FP-Growth suele ser preferible cuando la base es grande y contiene muchos patrones frecuentes. Apriori puede ser útil con fines didácticos o cuando se desea una implementación directa basada en candidatos. En la práctica, la elección también depende de la densidad de las transacciones, el umbral de soporte, la memoria disponible y la necesidad de interpretar pasos intermedios.

120.12. Aplicaciones

FP-Growth es aplicable a distintos dominios donde los datos pueden representarse como conjuntos de eventos o ítems:

- **Análisis de cesta de compra:** identificar productos que se compran juntos con frecuencia.
- **Sistemas de recomendación:** sugerir productos, contenidos o servicios a partir de patrones de coocurrencia.
- **Análisis de comportamiento del cliente:** estudiar hábitos de compra en comercio minorista o electrónico.
- **Minería de uso web:** detectar rutas de navegación o secuencias de clics comunes.
- **Minería de texto:** encontrar palabras, términos o frases que aparecen juntas en documentos.

En todos estos casos, los patrones frecuentes deben interpretarse con cuidado. La coocurrencia no implica causalidad. Una regla puede ser estadísticamente fuerte y, aun así, carecer de utilidad práctica si representa un patrón obvio, redundante o inducido por sesgos de captura de datos.

120.13. Ventajas y limitaciones

Entre las principales ventajas de FP-Growth se encuentran:

- Evita la generación explícita de candidatos.
- Reduce el número de escaneos completos de la base de datos.
- Comprime transacciones mediante prefijos compartidos.
- Puede extraer patrones frecuentes complejos de manera eficiente.
- Funciona bien en bases densas donde Apriori produce demasiados candidatos.

Sus limitaciones principales son:

- El FP-tree puede consumir mucha memoria si la base es grande y poco compresible.
- La implementación es más compleja que Apriori.
- La minería condicional puede ser costosa cuando existen muchos patrones frecuentes.

- No es naturalmente incremental: cuando cambian los datos, puede requerir reconstruir el árbol.

Estas limitaciones no invalidan el método, pero sí obligan a seleccionar adecuadamente el umbral de soporte, revisar la calidad de los datos y evaluar la interpretabilidad de los patrones resultantes.

120.14. Buenas prácticas de análisis

Al aplicar FP-Growth en un proyecto real, conviene seguir una estrategia reproducible:

1. Definir qué representa una transacción.
2. Limpiar y normalizar los ítems.
3. Explorar frecuencias individuales antes de minar patrones.
4. Probar varios valores de soporte mínimo.
5. Generar reglas solo después de revisar los itemsets frecuentes.
6. Evaluar confianza, lift y soporte de manera conjunta.
7. Validar los patrones con conocimiento del dominio.
8. Documentar decisiones, umbrales y supuestos.

La etapa más importante no es ejecutar el algoritmo, sino construir una representación transaccional coherente. Por ejemplo, en minería de texto, decidir si los ítems serán palabras, lemas, n-gramas o entidades cambia completamente el significado de los patrones. En comercio electrónico, decidir si una transacción corresponde a una compra, una sesión o un usuario también modifica la interpretación del soporte.

120.15. Conclusiones

FP-Growth es un algoritmo fundamental para la minería de patrones frecuentes. Su principal contribución es reemplazar la generación explícita de candidatos por una estructura compacta, el FP-tree, que permite almacenar prefijos compartidos y extraer patrones mediante bases condicionales. Esta estrategia reduce recorridos sobre la base de datos y suele mejorar el rendimiento frente a Apriori en escenarios grandes o densos.

Desde el punto de vista conceptual, FP-Growth muestra una idea central en ciencia de datos: la eficiencia no depende solo del algoritmo, sino también de la representación. Al transformar transacciones en un árbol comprimido, el problema se vuelve más manejable sin perder la información necesaria para recuperar los itemsets frecuentes.

En aplicaciones reales, el algoritmo debe acompañarse de una interpretación cuidadosa. Los patrones frecuentes son puntos de partida para formular hipótesis, diseñar recomendaciones o comprender comportamientos, pero no constituyen por sí solos evidencia causal. Por ello, la minería de asociaciones debe integrarse con análisis exploratorio, conocimiento del dominio y criterios claros de utilidad práctica.

Parte XXIII.

Clasificación

Parte XXIV.

Introducción y Evaluación

121. Clasificación y árboles de decisión

121.1. Introducción

La clasificación es una de las tareas fundamentales del aprendizaje supervisado. Su objetivo es aprender, a partir de ejemplos previamente observados, una regla de decisión capaz de asignar una etiqueta categórica a nuevas observaciones. A diferencia de la predicción de una variable numérica, donde la salida pertenece a un conjunto continuo o discreto ordenado, en clasificación la salida pertenece a un conjunto finito de clases: por ejemplo, **spam** o **no spam**, **aprobar** o **rechazar**, **positivo**, **negativo** o **neutral** (Hastie et al. 2009; James et al. 2021).

En términos generales, un problema de clasificación parte de un conjunto de datos de entrenamiento compuesto por pares

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n,$$

donde $\mathbf{x}_i \in \mathcal{X}$ representa las variables predictoras u observaciones disponibles, mientras que $y_i \in \mathcal{Y}$ representa la clase asociada. El objetivo es construir una función

$$\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$$

que generalice razonablemente bien a datos no vistos.

En este capítulo se estudian los árboles de decisión como modelos de clasificación interpretables. El desarrollo se concentra en la lógica general de construcción de árboles, el algoritmo de Hunt, el algoritmo ID3, la entropía, la ganancia de información y la extracción de reglas de clasificación. Estos contenidos reorganizan y amplían las ideas centrales de la presentación de clase sobre clasificación y árboles de decisión.

121.2. Clasificación como problema supervisado

En un problema de clasificación se dispone de ejemplos históricos donde la clase correcta ya es conocida. El modelo utiliza estos ejemplos para aprender regularidades entre las variables explicativas y la etiqueta objetivo.

Algunos ejemplos típicos son:

- clasificar si un correo electrónico es **spam** o **no spam**;
- decidir si una solicitud de crédito debe ser **aprobada** o **rechazada**;
- identificar si una persona presenta o no hipertensión;
- clasificar un comentario como **positivo**, **negativo** o **neutral**;
- predecir si un cliente comprará o no una computadora.

La distinción entre clasificación y regresión depende del tipo de variable objetivo. Si la salida es categórica, se trata de clasificación. Si la salida es numérica, se trata de regresión o predicción continua.

Tarea	Tipo de salida	Ejemplo
Clasificación	Categoría	Aprobado / Reprobado
Regresión	Valor numérico	Temperatura de mañana

i Nota

Definición. Un problema de clasificación supervisada consiste en estimar una función $\hat{f}$ que asigna a cada observación $\mathbf{x}$ una clase y dentro de un conjunto finito de etiquetas $\mathcal{Y}$:

$$\hat{f}: \mathcal{X} \rightarrow \mathcal{Y}, \quad |\mathcal{Y}| < \infty. \quad (121.1)$$

121.3. Entrenamiento, prueba y generalización

La clasificación suele dividirse en dos etapas principales: entrenamiento y prueba. Durante el entrenamiento, el algoritmo ajusta sus reglas internas utilizando datos etiquetados. Durante la prueba, el modelo se evalúa sobre datos no utilizados en el ajuste para estimar su capacidad de generalización.

Esta separación es esencial porque un modelo puede memorizar los datos de entrenamiento sin aprender patrones útiles. Este fenómeno se conoce como sobreajuste u *overfitting*. En árboles de decisión, el sobreajuste aparece cuando el árbol crece demasiado y produce particiones muy específicas para casos particulares (Breiman et al. 1984; Quinlan 1986).

i Nota

Definición. Dado un conjunto de entrenamiento $\mathcal{D}_{\text{train}}$ y un conjunto de prueba $\mathcal{D}_{\text{test}}$, la capacidad de generalización de un clasificador se evalúa estimando su desempeño sobre ejemplos que no fueron usados para ajustar el modelo:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset. \quad (121.2)$$

121.4. Preparación de datos antes de construir un clasificador

Antes de entrenar un clasificador, es necesario preparar los datos. La calidad del modelo depende directamente de la calidad de las variables disponibles. Tres tareas frecuentes son la limpieza, la transformación y la reducción de dimensiones.

La limpieza de datos incluye corregir valores imposibles o inconsistentes, como edades negativas, fechas mal formateadas o categorías escritas con variantes ortográficas. También incluye decidir cómo tratar valores faltantes. La transformación de datos incluye codificación de variables categóricas, normalización de variables numéricas y conversión de fechas a representaciones útiles. La reducción de dimensiones consiste en eliminar variables redundantes, irrelevantes o excesivamente ruidosas.

En árboles de decisión, la preparación puede ser menos exigente que en modelos lineales o redes neuronales, porque los árboles toleran variables en escalas distintas. Sin embargo, la calidad de las categorías, la

presencia de valores faltantes y el tratamiento de variables continuas siguen siendo factores importantes (James et al. 2021; Géron 2022).

```
import pandas as pd

clientes = pd.DataFrame({
    "ingreso": ["alto", "bajo", "medio", "bajo", "medio"],
    "historial_credificio": ["bueno", "malo", "bueno", "regular", "malo"],
    "deudas": ["bajas", "altas", "bajas", "altas", "medias"],
    "empleo_estable": ["sí", "no", "sí", "no", "sí"],
    "decision": ["aprobar", "rechazar", "aprobar", "rechazar", "rechazar"]
})

clientes
```

	ingreso	historial_credificio	deudas	empleo_estable	decision
0	alto	bueno	bajas	sí	aprobar
1	bajo	malo	altas	no	rechazar
2	medio	bueno	bajas	sí	aprobar
3	bajo	regular	altas	no	rechazar
4	medio	malo	medias	sí	rechazar

El ejemplo anterior representa una tarea de clasificación binaria: decidir si una solicitud de crédito debe aprobarse o rechazarse. Las variables predictoras son categóricas y la etiqueta objetivo es **decision**.

121.5. Árboles de decisión

Un árbol de decisión es un modelo predictivo que organiza decisiones sucesivas en una estructura jerárquica. Cada nodo interno representa una pregunta sobre una variable, cada rama representa una posible respuesta y cada hoja representa una clase final. La predicción de una observación se obtiene recorriendo el árbol desde la raíz hasta una hoja.

Los árboles son especialmente útiles cuando se busca interpretabilidad. A diferencia de modelos más opacos, un árbol puede leerse como un conjunto de reglas condicionales del tipo “si ocurre cierta combinación de condiciones, entonces se predice cierta clase” (Breiman et al. 1984; Quinlan 1993).

i Nota

Definición. Un árbol de decisión para clasificación es una estructura jerárquica compuesta por nodos internos, ramas y hojas. Cada nodo interno aplica una prueba $h_j(\mathbf{x})$, cada rama corresponde a un resultado de la prueba y cada hoja asigna una clase $c \in \mathcal{Y}$:

$$T(\mathbf{x}) = c, \quad c \in \mathcal{Y}. \quad (121.3)$$

Los elementos principales de un árbol son:

Elemento	Descripción
Nodo raíz	Primer nodo del árbol; contiene todos los ejemplos antes de dividir.
Nodo interno	Nodo donde se evalúa un atributo y se divide el conjunto de datos.
Rama	Resultado posible de una prueba aplicada en un nodo.
Hoja	Nodo terminal que asigna una clase.

121.6. Partición de datos en árboles de decisión

El proceso de construcción de un árbol se basa en particionar el conjunto de datos. En cada nodo se elige una variable y se divide el conjunto de ejemplos según sus valores.

Cuando el atributo es categórico, el árbol puede crear una rama por cada categoría. Por ejemplo, si **pronóstico** toma los valores **soleado**, **nublado** y **lluvia**, puede generarse una rama para cada caso. Si el árbol se restringe a divisiones binarias, una categoría puede evaluarse mediante preguntas del tipo “¿el color es rojo?”.

Cuando el atributo es continuo, se requiere seleccionar un punto de corte. Por ejemplo, una variable como **ingreso** puede dividirse en **ingreso $\leq$ 25000** e **ingreso $>$ 25000**. Aunque ID3 fue propuesto principalmente para atributos categóricos, extensiones como C4.5 incorporan mecanismos más generales para tratar variables continuas (Quinlan 1986, 1993).

121.7. Algoritmo de Hunt

El algoritmo de Hunt describe una estrategia recursiva general para construir árboles de decisión. La idea central es simple: si todos los ejemplos que llegan a un nodo pertenecen a la misma clase, el nodo se convierte en hoja. Si no, se elige un atributo para dividir los datos y se repite el procedimiento en cada subconjunto.

Sea D_t el conjunto de ejemplos que llega al nodo t . El procedimiento puede resumirse así:

1. Si todos los ejemplos en D_t pertenecen a una misma clase y , crear una hoja con etiqueta y .
2. Si D_t está vacío, crear una hoja con una clase predeterminada, usualmente la clase mayoritaria del nodo padre.
3. Si los ejemplos pertenecen a distintas clases, elegir una prueba sobre algún atributo.
4. Dividir D_t en subconjuntos según el resultado de la prueba.
5. Aplicar recursivamente el mismo procedimiento a cada subconjunto.

Este enfoque es *top-down*, porque comienza en la raíz y avanza hacia las hojas, y es *divide and conquer*, porque descompone el problema global en problemas más pequeños.

i Nota

Definición. Sea D_t el conjunto de ejemplos que llega al nodo t . Un nodo es puro si todos los ejemplos

en D_t pertenecen a la misma clase:

$$\exists c \in \mathcal{Y} \text{ tal que } y_i = c \quad \forall (\mathbf{x}_i, y_i) \in D_t. \quad (121.4)$$

121.8. Criterios de parada

La partición del árbol no puede continuar indefinidamente. Las condiciones más comunes para detener el crecimiento son:

- todos los ejemplos del nodo pertenecen a la misma clase;
- no quedan atributos disponibles para seguir dividiendo;
- el número de ejemplos en el nodo es demasiado pequeño;
- la profundidad máxima permitida del árbol ha sido alcanzada;
- la mejora obtenida por una nueva división es menor que un umbral.

En el caso clásico de ID3, si no quedan atributos y todavía hay ejemplos de distintas clases, se asigna la clase más frecuente del nodo. Esta decisión refleja un principio de mayoría local.

121.9. ID3: selección de atributos mediante ganancia de información

ID3, *Iterative Dichotomiser 3*, es un algoritmo clásico para inducir árboles de decisión. Fue propuesto por Quinlan y utiliza ganancia de información para elegir, en cada nodo, el atributo que mejor separa las clases (Quinlan 1986).

La intuición es que un buen atributo debe producir subconjuntos más homogéneos que el conjunto original. Para medir esta homogeneidad, ID3 utiliza la entropía, una medida proveniente de la teoría de la información (Shannon 1948).

El algoritmo procede de la siguiente manera:

1. Comienza con todos los ejemplos en el nodo raíz.
2. Calcula la entropía del conjunto actual.
3. Evalúa cada atributo disponible.
4. Para cada atributo, calcula la entropía promedio después de dividir.
5. Selecciona el atributo con mayor ganancia de información.
6. Repite el proceso recursivamente en cada rama.

121.10. Entropía como medida de impureza

La entropía mide el grado de mezcla o incertidumbre de las clases en un conjunto de datos. Si todos los ejemplos pertenecen a la misma clase, la entropía es cero. Si las clases están muy mezcladas, la entropía es alta. En un problema binario, la entropía máxima ocurre cuando ambas clases tienen la misma proporción.

i Nota

Definición. Sea S un conjunto de ejemplos con c clases y sea p_i la proporción de ejemplos pertenecientes a la clase i . La entropía de S se define como:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i). \quad (121.5)$$

La convención usual es considerar que $0 \log_2(0) = 0$, pues el límite de $p \log_2(p)$ cuando $p \rightarrow 0^+$ es cero.

Si S contiene 15 ejemplos, con 9 respuestas **Sí** y 6 respuestas **No**, entonces

$$p(\text{Sí}) = \frac{9}{15}, \quad p(\text{No}) = \frac{6}{15}.$$

Por tanto,

$$H(S) = -\frac{9}{15} \log_2\left(\frac{9}{15}\right) - \frac{6}{15} \log_2\left(\frac{6}{15}\right) \approx 0.971.$$

```
import numpy as np

def entropy(labels):
    values, counts = np.unique(labels, return_counts=True)
    probs = counts / counts.sum()
    return -np.sum(probs * np.log2(probs))

labels = ["Sí"] * 9 + ["No"] * 6
entropy(labels)
```

```
np.float64(0.9709505944546686)
```

121.11. Ganancia de información

La ganancia de información mide cuánto se reduce la entropía después de dividir un conjunto de datos usando un atributo. Si la división produce subconjuntos puros, la ganancia será alta. Si la división apenas cambia la mezcla de clases, la ganancia será baja.

i Nota

Definición. Sea S un conjunto de datos y sea A un atributo con valores posibles $v \in \text{Values}(A)$. Si S_v denota el subconjunto de ejemplos cuyo atributo A toma el valor v , la ganancia de información se define como:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v). \quad (121.6)$$

El primer término, $H(S)$, representa el desorden antes de dividir. El segundo término representa el desorden promedio después de dividir, ponderado por el tamaño de cada subconjunto. Por ello, la ganancia mide la mejora obtenida al usar el atributo A .

121.12. Ejemplo: construir un árbol para decidir si se juega béisbol

Consideremos un conjunto de datos donde se desea predecir si se juega o no un partido de béisbol. Las variables son Pronóstico, Temperatura, Humedad y Viento. La clase objetivo es Juega.

```
import pandas as pd

df = pd.DataFrame({
    "Pronóstico": [
        "Soleado", "Soleado", "Nublado", "Lluvia", "Lluvia",
        "Lluvia", "Nublado", "Soleado", "Soleado", "Lluvia",
        "Soleado", "Nublado", "Nublado", "Lluvia", "Soleado"
    ],
    "Temperatura": [
        "Caluroso", "Caluroso", "Caluroso", "Templado", "Fresco",
        "Fresco", "Fresco", "Templado", "Fresco", "Templado",
        "Templado", "Templado", "Caluroso", "Templado", "Fresco"
    ],
    "Humedad": [
        "Alta", "Alta", "Alta", "Alta", "Normal",
        "Normal", "Normal", "Alta", "Normal", "Normal",
        "Normal", "Alta", "Normal", "Alta", "Alta"
    ],
    "Viento": [
        "Débil", "Fuerte", "Débil", "Débil", "Débil",
        "Fuerte", "Fuerte", "Débil", "Débil", "Débil",
        "Débil", "Fuerte", "Débil", "Fuerte", "Débil"
    ],
    "Juega": [
        "No", "No", "Sí", "Sí", "Sí",
        "No", "Sí", "No", "Sí", "Sí",
        "Sí", "Sí", "Sí", "No", "No"
    ]
})

df
```

	Pronóstico	Temperatura	Humedad	Viento	Juega
0	Soleado	Caluroso	Alta	Débil	No
1	Soleado	Caluroso	Alta	Fuerte	No
2	Nublado	Caluroso	Alta	Débil	Sí
3	Lluvia	Templado	Alta	Débil	Sí
4	Lluvia	Fresco	Normal	Débil	Sí
5	Lluvia	Fresco	Normal	Fuerte	No
6	Nublado	Fresco	Normal	Fuerte	Sí
7	Soleado	Templado	Alta	Débil	No
8	Soleado	Fresco	Normal	Débil	Sí
9	Lluvia	Templado	Normal	Débil	Sí

	Pronóstico	Temperatura	Humedad	Viento	Juega
10	Soleado	Templado	Normal	Débil	Sí
11	Nublado	Templado	Alta	Fuerte	Sí
12	Nublado	Caluroso	Normal	Débil	Sí
13	Lluvia	Templado	Alta	Fuerte	No
14	Soleado	Fresco	Alta	Débil	No

Primero calculamos la entropía del conjunto completo.

```
H_total = entropy(df["Juega"])
H_total
```

```
np.float64(0.9709505944546686)
```

El conjunto tiene 9 ejemplos con clase **Sí** y 6 ejemplos con clase **No**, por lo que la entropía inicial es aproximadamente 0.971.

Ahora calculamos la ganancia de información para cada atributo.

```
def information_gain(data, attribute, target):
    total_entropy = entropy(data[target])
    weighted_entropy = 0.0
    for value, subset in data.groupby(attribute):
        weight = len(subset) / len(data)
        weighted_entropy += weight * entropy(subset[target])
    return total_entropy - weighted_entropy

attributes = ["Pronóstico", "Temperatura", "Humedad", "Viento"]

gains = {
    attr: information_gain(df, attr, "Juega")
    for attr in attributes
}

pd.Series(gains).sort_values(ascending=False)
```

```
Pronóstico    0.279982
Humedad       0.185805
Viento        0.059773
Temperatura   0.013315
dtype: float64
```

Los valores esperados son aproximadamente:

Atributo	Ganancia de información
Pronóstico	0.280

Atributo	Ganancia de información
Humedad	0.186
Viento	0.145
Temperatura	0.013

Por tanto, el atributo elegido como raíz del árbol es **Pronóstico**, ya que produce la mayor reducción de entropía.

121.13. Construcción de ramas después de elegir la raíz

Una vez elegida la raíz, se construye una rama por cada valor de **Pronóstico**:

- **Pronóstico** = Nublado;
- **Pronóstico** = Soleado;
- **Pronóstico** = Lluvia.

En el subconjunto **Nublado**, todos los ejemplos pertenecen a la clase **Sí**, por lo que esa rama se convierte en una hoja.

```
df[df["Pronóstico"] == "Nublado"]
```

	Pronóstico	Temperatura	Humedad	Viento	Juega
2	Nublado	Caluroso	Alta	Débil	Sí
6	Nublado	Fresco	Normal	Fuerte	Sí
11	Nublado	Templado	Alta	Fuerte	Sí
12	Nublado	Caluroso	Normal	Débil	Sí

Para la rama **Soleado**, todavía hay mezcla de clases, por lo que se evalúan los atributos restantes: **Temperatura**, **Humedad** y **Viento**.

```
soleado = df[df["Pronóstico"] == "Soleado"]
remaining = ["Temperatura", "Humedad", "Viento"]

pd.Series({
    attr: information_gain(soleado, attr, "Juega")
    for attr in remaining
}).sort_values(ascending=False)
```

```
Humedad      0.918296
Temperatura   0.251629
Viento        0.109170
dtype: float64
```

La mayor ganancia se obtiene con **Humedad**. En esta rama se observa que:

121. Clasificación y árboles de decisión

- si Pronóstico = Soleado y Humedad = Alta, entonces Juega = No;
- si Pronóstico = Soleado y Humedad = Normal, entonces Juega = Sí.

Para la rama Lluvia, se evalúan los atributos restantes. El mejor atributo es Viento.

```
lluvia = df[df["Pronóstico"] == "Lluvia"]

pd.Series({
    attr: information_gain(lluvia, attr, "Juega")
    for attr in remaining
}).sort_values(ascending=False)
```

```
Viento      0.970951
Temperatura  0.019973
Humedad     0.019973
dtype: float64
```

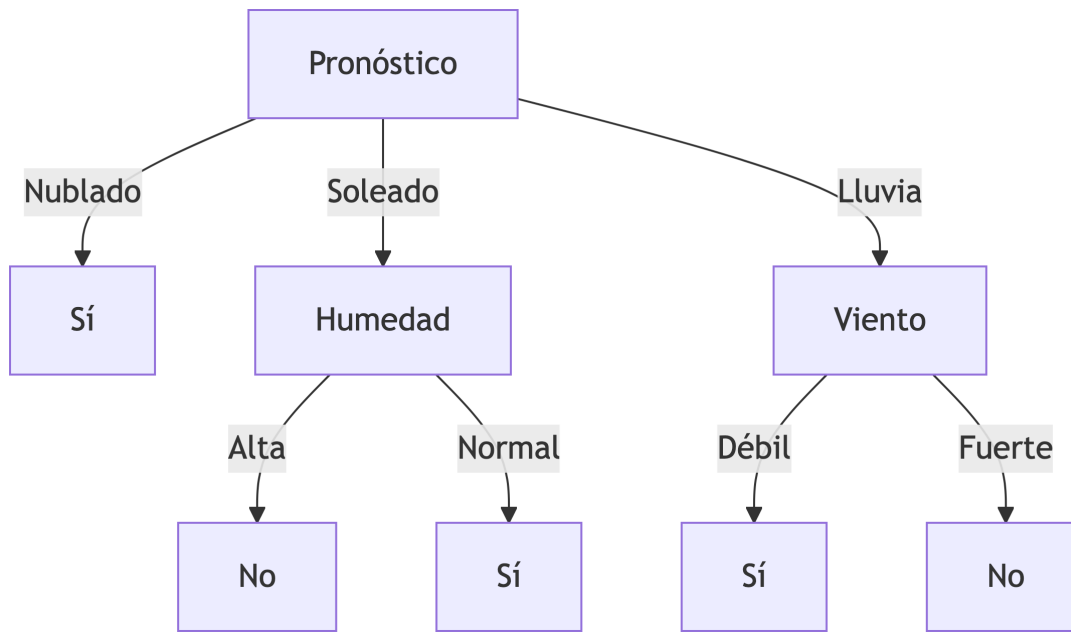
La regla resultante es:

- si Pronóstico = Lluvia y Viento = Débil, entonces Juega = Sí;
- si Pronóstico = Lluvia y Viento = Fuerte, entonces Juega = No.

121.14. Representación del árbol final

El árbol aprendido puede resumirse como:

```
flowchart TD
    A[Pronóstico] -->|Nublado| B[Sí]
    A -->|Soleado| C[Humedad]
    A -->|Lluvia| D[Viento]
    C -->|Alta| E[No]
    C -->|Normal| F[Sí]
    D -->|Débil| G[Sí]
    D -->|Fuerte| H[No]
```



Esta representación muestra que el modelo no utiliza todas las variables en todas las ramas. Por ejemplo, en la rama **Nublado** no se requiere revisar **Temperatura**, **Humedad** ni **Viento**, porque la clase ya queda determinada por los ejemplos de entrenamiento.

121.15. Extracción de reglas de clasificación

Una de las ventajas principales de los árboles de decisión es que pueden transformarse en reglas de clasificación. Cada camino desde la raíz hasta una hoja produce una regla de tipo SI-ENTONCES.

Del árbol anterior se obtienen las siguientes reglas:

1. SI **Pronóstico** = **Soleado** Y **Humedad** = **Alta**, ENTONCES **Juega** = **No**.
2. SI **Pronóstico** = **Soleado** Y **Humedad** = **Normal**, ENTONCES **Juega** = **Sí**.
3. SI **Pronóstico** = **Nublado**, ENTONCES **Juega** = **Sí**.
4. SI **Pronóstico** = **Lluvia** Y **Viento** = **Débil**, ENTONCES **Juega** = **Sí**.
5. SI **Pronóstico** = **Lluvia** Y **Viento** = **Fuerte**, ENTONCES **Juega** = **No**.

i Nota

Definición. Una regla de clasificación extraída de un árbol es una implicación lógica formada por las condiciones de un camino desde la raíz hasta una hoja:

$$\text{SI } (A_1 = v_1) \wedge (A_2 = v_2) \wedge \cdots \wedge (A_k = v_k) \text{ ENTONCES } y = c. \quad (121.7)$$

Estas reglas son interpretables y pueden comunicarse fácilmente a personas no especialistas. Por esta razón, los árboles de decisión se han utilizado en aplicaciones como diagnóstico médico, evaluación de riesgo crediticio, mantenimiento predictivo y clasificación de clientes (Breiman et al. 1984; James et al. 2021).

121.16. Implementación con `scikit-learn`

Aunque el ejemplo anterior calcula manualmente la entropía y la ganancia de información, en la práctica se usan bibliotecas especializadas. En Python, `scikit-learn` proporciona implementaciones eficientes de árboles de decisión (Pedregosa et al. 2011).

Para trabajar con variables categóricas, primero se codifican mediante variables indicadoras. Después se ajusta un árbol usando el criterio `entropy`, que corresponde a la ganancia de información.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(columns="Juega")
y = df["Juega"]

categorical_features = X.columns.tolist()

preprocess = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=3,
    random_state=42
)

clf = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", model)
])

clf.fit(X, y)
clf.predict(X)
```

```
array(['No', 'No', 'Sí', 'No', 'Sí', 'No', 'Sí', 'No', 'Sí', 'Sí', 'Sí',
       'Sí', 'Sí', 'No', 'No'], dtype=object)
```

En conjuntos pequeños, evaluar sobre los mismos datos de entrenamiento solo sirve como verificación técnica. En problemas reales, debe separarse un conjunto de prueba o usarse validación cruzada.


```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

```

Accuracy: 0.6

```

	precision	recall	f1-score	support
No	0.50	1.00	0.67	2
Sí	1.00	0.33	0.50	3
accuracy			0.60	5
macro avg	0.75	0.67	0.58	5
weighted avg	0.80	0.60	0.57	5

121.17. Visualización del árbol

```

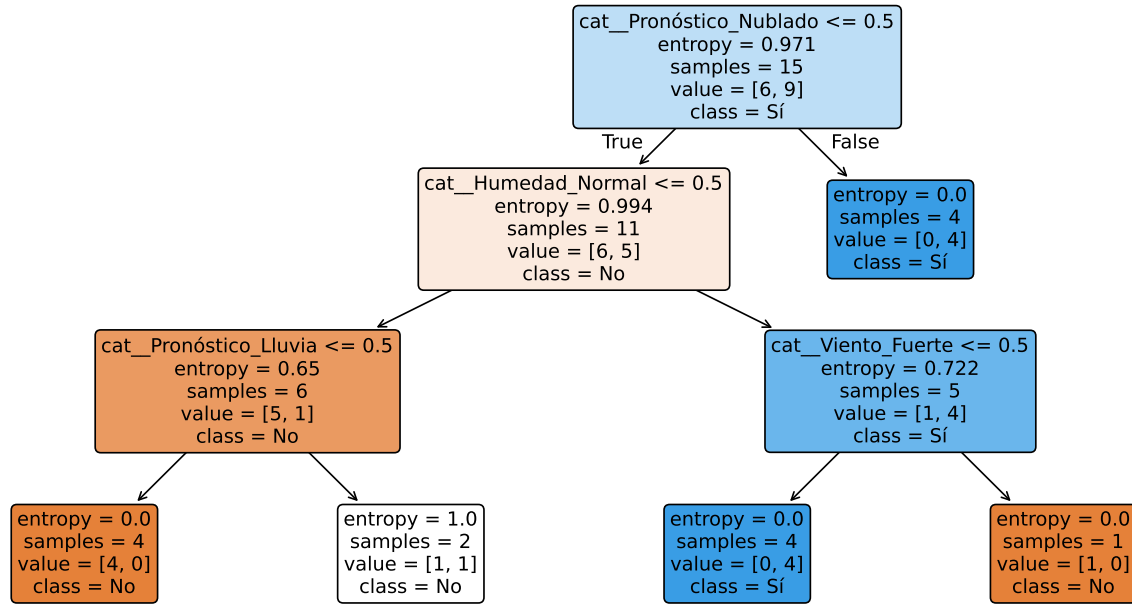
import matplotlib.pyplot as plt

clf.fit(X, y)

feature_names = clf.named_steps["preprocess"].get_feature_names_out()
tree_model = clf.named_steps["model"]

plt.figure(figsize=(14, 7))
plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=tree_model.classes_,
    filled=True,
    rounded=True
)
plt.show()

```



La visualización permite inspeccionar qué variables fueron utilizadas y cómo se organizaron las divisiones. En árboles más grandes, la visualización puede volverse difícil de leer; por ello se recomienda controlar hiperparámetros como `max_depth`, `min_samples_split` y `min_samples_leaf`.

121.18. Complejidad computacional

La complejidad de construir un árbol depende del número de ejemplos n , el número de atributos m , el número de clases y el costo de evaluar divisiones. Para atributos categóricos, una estimación simplificada del costo en cada nodo consiste en revisar los atributos disponibles y calcular la distribución de clases por valor del atributo. Si se hace de forma directa, el costo puede aproximarse por $O(mn)$ en la raíz.

A lo largo del árbol, el costo total depende de cómo se reparten los datos entre nodos. En árboles balanceados, cada nivel procesa en conjunto aproximadamente n ejemplos, por lo que una estimación razonable es

$$O(mnd),$$

donde d es la profundidad del árbol. En el peor caso, la profundidad puede crecer hasta n , produciendo costos mayores y árboles poco generalizables. Para variables continuas, además puede ser necesario ordenar valores o evaluar umbrales candidatos, lo que modifica el costo de entrenamiento (Hastie et al. 2009; Pedregosa et al. 2011).

i Nota

Definición. Si n es el número de ejemplos, m el número de atributos y d la profundidad del árbol, una estimación simplificada del costo de inducción de un árbol categórico es:

$$T(n, m, d) = O(mnd). \quad (121.8)$$

La predicción con un árbol ya entrenado suele ser eficiente. Para clasificar una observación, basta recorrer un camino desde la raíz hasta una hoja, por lo que el costo de inferencia es $O(d)$.

121.19. Ventajas y limitaciones

Los árboles de decisión presentan varias ventajas:

- son fáciles de explicar;
- generan reglas interpretables;
- permiten combinar variables categóricas y numéricas;
- requieren relativamente poca preparación de escalas;
- pueden capturar interacciones no lineales entre variables.

Sin embargo, también tienen limitaciones importantes:

- pueden sobreajustarse si crecen sin restricciones;
- son sensibles a pequeñas variaciones en los datos;
- ID3 favorece atributos con muchos valores posibles;
- la versión clásica de ID3 no maneja directamente variables continuas;
- una decisión local óptima no garantiza un árbol globalmente óptimo.

La naturaleza *greedy* de ID3 implica que el algoritmo elige el mejor atributo en cada nodo según una medida local. Esto hace que el procedimiento sea eficiente, pero no garantiza encontrar el árbol óptimo global. En la práctica, técnicas como poda, validación cruzada y métodos de ensamble ayudan a mejorar la generalización (Breiman et al. 1984; Hastie et al. 2009).

121.20. Conclusiones

La clasificación supervisada busca asignar etiquetas categóricas a nuevas observaciones usando información aprendida a partir de ejemplos históricos. Los árboles de decisión ofrecen una forma clara e interpretable de resolver esta tarea mediante una secuencia de preguntas sobre los atributos.

El algoritmo de Hunt proporciona la estructura recursiva general para construir árboles. ID3 especifica un criterio para elegir atributos usando entropía y ganancia de información. La entropía mide la impureza de un conjunto de ejemplos, mientras que la ganancia de información mide cuánto se reduce esa impureza después de dividir por un atributo.

El ejemplo del partido de béisbol muestra cómo se calcula la entropía inicial, cómo se comparan atributos, cómo se elige la raíz y cómo se construyen ramas sucesivas hasta obtener un árbol final. Además, el árbol puede convertirse en reglas SI-ENTONCES, lo que facilita su interpretación y comunicación.

En aplicaciones reales, los árboles de decisión deben evaluarse cuidadosamente para evitar sobreajuste. Aun así, siguen siendo una herramienta central en ciencia de datos por su equilibrio entre capacidad predictiva, simplicidad e interpretabilidad.

122. Evaluación de Clasificadores: Métricas Esenciales y su Interpretación

122.1. Introducción

La evaluación de modelos de clasificación constituye uno de los problemas más estratégicamente complejos en la Minería de Datos. No es suficiente construir un modelo que “funcione”: es necesario cuantificar de manera precisa y contextualmente apropiada qué tan bien funciona, en qué escenarios falla y qué tipo de errores comete, ya que como veremos mas adelante la calidad de un clasificador esta completamente determiada por el objetivo del modelo y el contexto de aplicación (Han et al. 2011).

Esta distinción tiene consecuencias directas en entornos reales: un modelo de diagnóstico médico que confunde “exactitud estadística” con “utilidad clínica” puede dejar pacientes enfermos sin tratamiento; un sistema de detección de fraude que optimiza la métrica equivocada puede bloquear transacciones legítimas y dañar la relación con el cliente.

De esta manera la literatura especializada ha producido un conjunto robusto de métricas y herramientas de evaluación que van más allá del intuitivo porcentaje de aciertos. Con lo cual comprender estas métricas es competencia central del científico de datos (Provost y Fawcett 2013).

Este capítulo desarrolla sistemáticamente las métricas más importantes para la evaluación de clasificadores binarios y multiclase, integrándolas con el proceso KDD (Knowledge Discovery in Databases) y con la lógica de la toma de decisiones en contextos de alto impacto. Cada sección combina rigor matemático con ejemplos de código ejecutable en Python, siguiendo el principio de que la comprensión profunda de una métrica requiere tanto su derivación formal como su implementación y visualización sobre datos reales.

122.2. Errores

En este capitulo definiremos un **error** como la situación en la que el modelo de clasificación asigna una etiqueta incorrecta a una instancia.

Mientras que definiremos el ratio de error como la proporción de instancias clasificadas incorrectamente sobre el total de instancias evaluadas.

Definición. El *ratio de error* de un clasificador con k errores en un conjunto de prueba de n instancias se define como:

$$\epsilon = \frac{k}{n} \quad (122.1)$$

En primera instnacia esta definición puede parecer suficiente para los propositos de cualquier clasificador, ya que en la mayoría de los casos el objetivo e minimizar el numero de errores que comete el modelo. Sin embargo como veremos a lo largo del capitulo, no todos los errores son iguales, ya que dependiendo del contexto una equivocación puede tener diferentes consecuencias que exploraremos a continuación.

122.3. Tipo de errores y su costo

122.3.1. La matriz de confusión: la base de toda evaluación de clasificadores

En terminos generales decimos que una una clase es **positiva** (1) cuando es la clase de interés, y **negativa** (0) cuando es la clase complementaria. Por ejemplo, en un sistema de detección de fraude, la clase positiva sería “fraude” y la negativa “transacción legítima”. En un sistema de diagnóstico médico, la clase positiva sería “enfermo” y la negativa “sano”.

De esta manera podemos clasificar los errores de un clasificador binario por medio de la matriz de confusión, que es la representación tabular que sintetiza el desempeño de un clasificador binario (Han et al. 2011). Formalmente se define como:

Definición. Para un clasificador con clases positiva (1) y negativa (0), la matriz de confusión se define como:

$$M = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix} \quad (122.2)$$

donde la fila representa la clase real y la columna la clase predicha. Los cuatro elementos son:

- **TP (Verdadero Positivo):** El modelo predijo positivo y el caso real es positivo. Clasificación correcta de la clase de interés.
- **TN (Verdadero Negativo):** El modelo predijo negativo y el caso real es negativo. Clasificación correcta de la clase mayoritaria.
- **FP (Falso Positivo):** El modelo predijo positivo pero el caso real es negativo. También llamado Error Tipo I.
- **FN (Falso Negativo):** El modelo predijo negativo pero el caso real es positivo. También llamado Error Tipo II.

Visualmente podemos visualizar los tipos de errores en la siguiente figura:

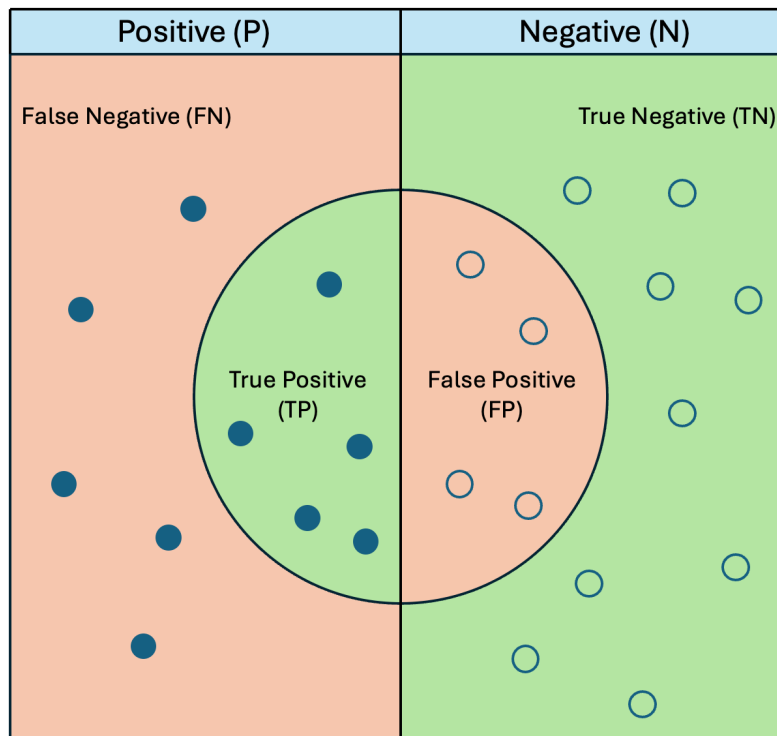


Figura 122.1.: Representación visual de una matriz de confusión y métricas de clasificación binaria. El diagrama ilustra la relación entre la realidad (columnas Positivo/Negativo) y las predicciones del modelo (círculo central). Se identifican los Verdaderos Positivos (TP) y Falsos Positivos (FP) dentro del área de predicción, mientras que los Falsos Negativos (FN) y Verdaderos Negativos (TN) se ubican en el área de exclusión.

Como veremos mas adelante, la matriz de confusión es la base de toda evaluación de clasificadores, ya que a partir de ella se derivan todas las métricas de desempeño. Además, la interpretación de cada elemento de la matriz tiene implicaciones estratégicas directas en la toma de decisiones basada en el modelo.

122.3.2. Generación de los Datasets Sintéticos

A lo largo de este capítulo se utilizan dos datasets sintéticos que replican las características estadísticas de problemas reales.

- El **Dataset Médico** simula la detección de una condición grave con prevalencia del ~8 %, reflejando el desbalance típico en estudios de screening.
- El **Dataset Financiero** simula transacciones con una tasa de fraude del ~3 %, consistente con benchmarks de la industria (Pedregosa et al., 2011).

Ambos datasets se generan con `sklearn.datasets.make_classification`, que permite controlar el número de características informativas, el nivel de ruido en las etiquetas y el desbalance de clases.

```
# Configuración global e importaciones
import numpy as np
import pandas as pd
import plotly.graph_objects as go
```

```

import plotly.figure_factory as ff
from plotly.subplots import make_subplots
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score,
    f1_score, fbeta_score, roc_curve, auc, precision_recall_curve,
    average_precision_score, classification_report,
)

SEED = 42

# Dataset médico: 92% sanos, 8% enfermos
X_med, y_med = make_classification(
    n_samples=1000, n_features=10, n_informative=5,
    weights=[0.92, 0.08], # fuerte desbalance
    flip_y=0.02,          # 2% de ruido en las etiquetas
    random_state=SEED,
)

X_tr_m, X_te_m, y_tr_m, y_te_m = train_test_split(
    X_med, y_med, test_size=0.3, stratify=y_med, random_state=SEED
)

# Dataset financiero: 97% legítimas, 3% fraude
X_fin, y_fin = make_classification(
    n_samples=5000, n_features=15, n_informative=7,
    weights=[0.97, 0.03],
    flip_y=0.01,
    random_state=SEED,
)

X_tr_f, X_te_f, y_tr_f, y_te_f = train_test_split(
    X_fin, y_fin, test_size=0.3, stratify=y_fin, random_state=SEED
)

# Modelo 1: Regresión Logística - dataset médico
modelo_lr = LogisticRegression(random_state=SEED, max_iter=500)
modelo_lr.fit(X_tr_m, y_tr_m)
y_pred_lr = modelo_lr.predict(X_te_m)
y_prob_lr = modelo_lr.predict_proba(X_te_m)[:, 1]

# Modelo 2: Random Forest - dataset médico
modelo_rf = RandomForestClassifier(n_estimators=100, random_state=SEED)
modelo_rf.fit(X_tr_m, y_tr_m)
y_pred_rf = modelo_rf.predict(X_te_m)
y_prob_rf = modelo_rf.predict_proba(X_te_m)[:, 1]

# Modelo 3: Regresión Logística - dataset financiero

```



```

modelo_fin = LogisticRegression(
    random_state=SEED, max_iter=500, class_weight="balanced"
)
modelo_fin.fit(X_tr_f, y_tr_f)
y_pred_fin = modelo_fin.predict(X_te_f)
y_prob_fin = modelo_fin.predict_proba(X_te_f)[: , 1]

print("Distribución de clases - test médico      :",
      dict(zip(*np.unique(y_te_m, return_counts=True))))
print("Distribución de clases - test financiero:",
      dict(zip(*np.unique(y_te_f, return_counts=True))))

```

```
Distribución de clases - test médico      : {np.int64(0): np.int64(275), np.int64(1): np.int64(25)}
```

```
Distribución de clases - test financiero: {np.int64(0): np.int64(1449), np.int64(1): np.int64(51)}
```

Para evaluar a lo largo del capítulo diferentes modelos empleamos tres modelos de clasificación binaria: una **regresión logística** y un **random forest** para el dataset médico, y una **regresión logística con ajuste de clase** para el dataset financiero.

Así como primer ejemplo mostramos la matriz de confusión del modelo de regresión logística sobre el dataset médico:

```

# Función reutilizable: matriz de confusión interactiva
def plot_confusion_matrix(cm,
    titulo=" ",
    etiquetas=None):
    """
    Visualiza una matriz de confusión 2x2 con Plotly.

    Muestra conteos absolutos y porcentajes por fila
    (tasa de acierto/error por clase real).

    Parámetros
    -----
    cm          : np.ndarray - salida de sklearn confusion_matrix
    titulo      : str        - título del gráfico
    etiquetas   : list[str]  - nombres de las clases [neg, pos]
    """
    if etiquetas is None:
        etiquetas = ["Negativo (0)", "Positivo (1)"]

    cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100
    texto = [
        [f"{cm[i,j]}<br>({cm_pct[i,j]:.1f}%)" for j in range(2)]
        for i in range(2)
    ]
    # Invertir filas para que TP quede arriba-derecha (convención visual habitual)
    z_plot = cm[::-1, :]
    texto_plot = texto[::-1]

```

```

y_labels    = etiquetas[:-1]

fig = ff.create_annotated_heatmap(
    z=z_plot,
    x=etiquetas,
    y=y_labels,
    annotation_text=texto_plot,
    colorscale=[[0, "#f8f9fa"], [0.5, "#4a90d9"], [1, "#e74c3c"]],
    showscale=True,
)
fig.update_layout(
    title=titulo,
    xaxis_title="Predicción del modelo",
    yaxis_title="Valor real",
    height=430, width=530,
    font=dict(size=13),
)
# Etiquetas de cuadrante
etq = {(0, 0): ("TN", "gray"),      (0, 1): ("FP", "#e67e22"),
       (1, 0): ("FN", "#e74c3c"),  (1, 1): ("TP", "#27ae60")}
for (ri, ci), (label, color) in etq.items():
    ri_plot = 1 - ri
    fig.add_annotation(
        x=etiquetas[ci], y=y_labels[ri_plot],
        text=f"<b>{label}</b>",
        showarrow=False,
        font=dict(size=11, color=color),
        yshift=-22,
    )
return fig

# Aplicar al clasificador médico
cm_lr = confusion_matrix(y_te_m, y_pred_lr)
TP = int(cm_lr[1, 1]); TN = int(cm_lr[0, 0])
FP = int(cm_lr[0, 1]); FN = int(cm_lr[1, 0])

print("Matriz de confusión - Regresión Logística (médico):")
print(cm_lr)
print(f"\nTP={TP}, TN={TN}, FP={FP}, FN={FN}")

fig_cm = plot_confusion_matrix(
    cm_lr,
    # titulo="Matriz de Confusión - Clasificador Médico (Reg. Logística)",
    etiquetas=["Sano", "Enfermo"],
)
fig_cm.show()

```

Matriz de confusión - Regresión Logística (médico):

```
[[273  2]
 [ 19  6]]
```

TP=6, TN=273, FP=2, FN=19

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

A partir de esta matriz de confusión se derivan todas las métricas de desempeño que analizaremos a lo largo del capítulo. Además, la interpretación de cada elemento de la matriz tiene implicaciones estratégicas directas en la toma de decisiones basada en el modelo.

122.3.3. El Costo Asimétrico del Error: Contextos Reales

La característica más importante de los errores en clasificación no es su frecuencia absoluta, sino su **costo diferencial**. En la mayoría de los problemas reales de minería de datos, FP y FN no tienen el mismo costo, y optimizar hacia uno implica sacrificar el otro (James et al., 2021).

122.3.3.1. Medicina: El Costo Devastador del Falso Negativo

En un sistema de detección temprana de cáncer de pulmón, un **Falso Negativo** significa que el sistema predice “benigno” para un nódulo que en realidad es canceroso. El paciente no recibe tratamiento. Esta asimetría implica que en medicina diagnóstica se prefieren clasificadores con **alto Recall** (baja tasa de FN), incluso a expensas de menor Precision. Un **Falso Positivo** significa que el sistema predice “maligno” para un nódulo benigno: el paciente es referido para biopsia innecesaria —procedimiento invasivo y costoso—, pero comparado con el costo del FN, el FP es el error tolerable.

122.3.3.2. Finanzas: El Costo Operativo del Falso Positivo

En un sistema de detección de fraude, un **Falso Positivo** bloquea una transacción legítima: el cliente no puede completar su compra, con el riesgo de perder la relación comercial. Un **Falso Negativo** deja pasar un fraude real, pero la pérdida suele ser cubierta por seguro o chargeback. Esta asimetría inversa implica que en detección de fraude se prefieren clasificadores con **alta Precision**, aceptando que algunos fraudes pasen sin detectarse (Provost & Fawcett, 2013).

```
# Visualización del costo asimétrico por contexto
contextos = ["Medicina\n(screening)", "Finanzas\n(fraude)"]
costo_fn = [10, 3] # escala relativa normalizada
costo_fp = [2, 8]

fig = go.Figure(data=[
    go.Bar(name="Costo FN (Falso Negativo)", x=contextos, y=costo_fn,
        marker_color="#e74c3c",
        text=["Crítico: paciente\nsin tratamiento",
            "Moderado: pérdida\ncubierta por seguro"],
        textposition="auto"),
```

```

go.Bar(name="Costo FP (Falso Positivo)", x=contextos, y=costo_fp,
       marker_color="#e67e22",
       text=["Moderado: biopsia\ninnecesaria",
            "Alto: cliente\nabandonado"],
       textposition="auto"),
])
fig.update_layout(
    barmode="group",
    title=(
        "Costo Asimétrico de los Errores por Contexto<br>"
        "<sup>Escala relativa normalizada - Provost & Fawcett (2013)</sup>"
    ),
    yaxis_title="Severidad relativa del error",
    height=420,
    # legend=dict(orientation="h", yanchor="bottom", y=1.02),
)
fig.show()

```

Unable to display output for mime type(s): text/html

122.4. Métricas de desempeño: más allá del error rate

Como mencionamos anteriormente, si queremos evaluar de manera adecuada un clasificador es necesario considerar no solo el número de errores, sino también su tipo y costo. Para esto se han desarrollado un conjunto de métricas que se derivan de la matriz de confusión y que permiten cuantificar diferentes aspectos del desempeño del modelo. Estas métricas no son mutuamente excluyentes, sino complementarias, y su interpretación debe hacerse en conjunto para obtener una visión completa del modelo. A continuación se presentan las métricas más importantes para la evaluación de clasificadores binarios, con su definición formal, cálculo detallado e interpretación contextualizada.

122.4.1. Accuracy: Definición, Utilidad y Limitaciones

122.4.1.1. Definición Formal

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

La accuracy mide la proporción de instancias correctamente clasificadas sobre el total (Han et al., 2011).

122.4.1.2. Cálculo e Interpretación Básica

```
# Accuracy del modelo real vs. clasificador trivial
acc_lr      = accuracy_score(y_te_m, y_pred_lr)
y_trivial   = np.zeros_like(y_te_m)           # siempre predice "sano"
acc_trivial = accuracy_score(y_te_m, y_trivial)

print(f"Accuracy - Regresión Logística : {acc_lr:.4f}  ({acc_lr*100:.2f}%)")
print(f"Accuracy - Modelo trivial      : {acc_trivial:.4f}  ({acc_trivial*100:.2f}%)")
print()
print("¿El modelo trivial (siempre 'sano') detecta algún enfermo?")
print(f"Enfermos detectados: {(y_trivial==1) & (y_te_m==1)}.sum() / {y_te_m.sum()}")
```

```
Accuracy - Regresión Logística : 0.9300  (93.00 %)
Accuracy - Modelo trivial      : 0.9167  (91.67 %)
```

```
¿El modelo trivial (siempre 'sano') detecta algún enfermo?
Enfermos detectados: 0 / 25
```

122.4.1.3. La Paradoja de la Exactitud (Accuracy Paradox)

La limitación más crítica de la accuracy emerge cuando el dataset presenta **desbalance de clases**, situación extremadamente común en aplicaciones reales de minería de datos (Saito & Rehmsmeier, 2015). En nuestro dataset financiero con 97 % de transacciones legítimas, un clasificador trivial alcanza ~97 % de accuracy sin detectar ningún fraude:

```
# Paradoja de la exactitud - dataset financiero
y_trivial_fin = np.zeros_like(y_te_f)
acc_trivial_fin = accuracy_score(y_te_f, y_trivial_fin)
acc_modelo_fin = accuracy_score(y_te_f, y_pred_fin)
recall_trivial = recall_score(y_te_f, y_trivial_fin, zero_division=0)
recall_modelo = recall_score(y_te_f, y_pred_fin)

print("Dataset financiero (97 % legítimas, 3 % fraude)")
print(" " * 52)
print(f"Accuracy - modelo trivial : {acc_trivial_fin:.4f}  ({acc_trivial_fin*100:.1f}%)")
print(f"Accuracy - modelo real    : {acc_modelo_fin:.4f}  ({acc_modelo_fin*100:.1f}%)")
print()
print(f"Recall - modelo trivial   : {recall_trivial:.4f}  → 0 fraudes detectados")
print(f"Recall - modelo real     : {recall_modelo:.4f}  → detecta fraudes reales")
print()
print("Conclusión: mayor accuracy  mejor modelo en datos desbalanceados.")
print("(Saito & Rehmsmeier, 2015)")
```

```
Dataset financiero (97 % legítimas, 3 % fraude)
```

```
Accuracy - modelo trivial : 0.9660  (96.6 %)
Accuracy - modelo real    : 0.7820  (78.2 %)
```

```
Recall - modelo trivial   : 0.0000  → 0 fraudes detectados
```

Recall - modelo real : 0.7255 → detecta fraudes reales

Conclusión: mayor accuracy mejor modelo en datos desbalanceados.
(Saito & Rehmsmeier, 2015)

```
# Visualización: Accuracy vs. Recall
modelos = ["Modelo Real\n(LR + class_weight)", "Modelo Trivial\n(siempre negativo)"]
accuracies = [acc_modelo_fin, acc_trivial_fin]
recalls = [recall_modelo, recall_trivial]

fig = go.Figure(data=[
    go.Bar(name="Accuracy", x=modelos, y=accuracies,
           marker_color=["#4a90d9", "#e74c3c"],
           text=[f"{v:.2%}" for v in accuracies], textposition="auto"),
    go.Bar(name="Recall (clase fraude)", x=modelos, y=recalls,
           marker_color=["#27ae60", "#c0392b"],
           text=[f"{v:.2%}" for v in recalls], textposition="auto"),
])
fig.update_layout(
    barmode="group",
    title=(
        "Paradoja de la Exactitud en el Dataset Financiero<br>"
        "<sup>Saito & Rehmsmeier (2015): un modelo con mayor accuracy detecta 0 fraudes</sup>"
    ),
    yaxis=dict(title="Valor", range=[0, 1.1], tickformat="%"),
    height=420,
    # legend=dict(orientation="h", yanchor="bottom", y=1.02),
)
fig.show()
```

Unable to display output for mime type(s): text/html

122.4.2. Precision: Confiabilidad de las Predicciones Positivas

122.4.2.1. Definición Formal

$$\text{Precision} = \frac{TP}{TP + FP}$$

La Precision mide qué proporción de las instancias predichas como positivas son genuinamente positivas (James et al., 2021).

122.4.2.2. Cálculo Detallado

```
# Precision: cálculo manual y con sklearn
precision_manual = TP / (TP + FP) if (TP + FP) > 0 else 0
precision_sk      = precision_score(y_te_m, y_pred_lr, zero_division=0)

print(f"Cálculo manual - Precision = {TP} / ({TP}+{FP}) = {precision_manual:.4f}")
print(f"sklearn      - Precision = {precision_sk:.4f}")
print()
print(f"Interpretación: de cada 10 alarmas de 'enfermo',")
print(f" aproximadamente {precision_sk*10:.1f} corresponden a enfermos reales.")
```

Cálculo manual - Precision = 6 / (6+2) = 0.7500
 sklearn - Precision = 0.7500

Interpretación: de cada 10 alarmas de 'enfermo',
 aproximadamente 7.5 corresponden a enfermos reales.

122.4.2.3. Limitación: Alta Precision Buen Clasificador

Para entenderlo mejor, imagina que la **Precisión** es la “puntería” o calidad de lo que el sistema detecta.

Si un sistema es extremadamente tímido y solo lanza una alerta cuando tiene una certeza absoluta, su **Precisión será perfecta (1.0)**: cada vez que hable, tendrá razón. Sin embargo, el costo de esa perfección es el silencio; el sistema ignorará muchísimos casos reales por miedo a equivocarse. En la práctica, un sistema que casi nunca se atreve a alertar no es útil, aunque nunca mienta.

Por el contrario, si el sistema es demasiado “sensible” y alerta por cualquier cosa, su precisión cae. Fawcett (2006) identifica que cuando la precisión es muy baja, ocurre la **fatiga de alarmas**: el usuario recibe tantos avisos falsos que deja de confiar en el sistema y termina ignorándolo por completo.

122.4.2.4. Ejemplo: El filtro de correos “Spam”

- **Alta Precisión (Certeza absoluta):** Configuras tu correo para que solo marque como Spam los mensajes que digan explícitamente “ESTO ES UNA ESTAFA”. Tendrás una precisión de 1.0 (nunca se equivocará con un correo importante), pero tu bandeja de entrada principal se llenará de basura que el filtro no se atrevió a marcar.
- **Baja Precisión (Fatiga de alarmas):** Configuras el filtro para que marque como Spam cualquier correo que tenga un signo de exclamación o una palabra en mayúsculas. El filtro atrapará todo el spam, pero también enviará a la basura facturas importantes y mensajes de tus amigos. Al final, tendrás que revisar la carpeta de Spam todos los días, anulando la utilidad del filtro.

```
# Modelo "conservador": alta precision, bajo recall
umbral_alto = 0.90
y_conserv   = (y_prob_lr >= umbral_alto).astype(int)

p_c = precision_score(y_te_m, y_conserv, zero_division=0)
r_c = recall_score(y_te_m, y_conserv, zero_division=0)
f1_c = f1_score(y_te_m, y_conserv, zero_division=0)
```

```
print(f"Umbral = {umbral_alto} (modelo muy conservador)")
print(f" Precision : {p_c:.4f} (casi nunca se equivoca cuando alerta)")
print(f" Recall : {r_c:.4f} (pero deja de detectar muchos enfermos)")
print(f" F1-Score : {f1_c:.4f}")
print(f" Alertas emitidas: {y_conserv.sum()} de {y_te_m.sum()} enfermos reales")
```

```
Umbral = 0.9 (modelo muy conservador)
Precision : 1.0000 (casi nunca se equivoca cuando alerta)
Recall : 0.0400 (pero deja de detectar muchos enfermos)
F1-Score : 0.0769
Alertas emitidas: 1 de 25 enfermos reales
```

122.4.3. Recall (Sensibilidad): Cobertura de los Positivos Reales

122.4.3.1. Definición Formal

$$\text{Recall} = \text{Sensibilidad} = \text{TPR} = \frac{TP}{TP + FN}$$

El Recall mide qué proporción de los casos verdaderamente positivos fue correctamente identificada por el clasificador (Fawcett, 2006).

122.4.3.2. Cálculo y Relación con la Tasa de Falsos Negativos

```
# Recall: cálculo y relación con FNR
recall_manual = TP / (TP + FN) if (TP + FN) > 0 else 0
recall_sk      = recall_score(y_te_m, y_pred_lr)
fnr           = FN / (TP + FN) # Tasa de Falsos Negativos = 1 - Recall

print(f"Cálculo manual - Recall = {TP} / ({TP}+{FN}) = {recall_manual:.4f}")
print(f"sklearn - Recall = {recall_sk:.4f}")
print(f"FNR (1 - Recall) = {fnr:.4f}")
print()
print(f"De {y_te_m.sum()} enfermos reales en el conjunto de prueba,")
print(f" el modelo detecta {TP} y deja sin detectar {FN}.")
```

```
Cálculo manual - Recall = 6 / (6+19) = 0.2400
sklearn - Recall = 0.2400
FNR (1 - Recall) = 0.7600
```

De 25 enfermos reales en el conjunto de prueba,
el modelo detecta 6 y deja sin detectar 19.

122.4.3.3. Limitación: Alto Recall Buen Clasificador

Si la Precisión se enfocaba en la “calidad” de las alertas (no mentir), el **Recall** se enfoca en la “cantidad” o cobertura: qué tan buena es la red del sistema para atrapar todos los casos reales.

Un sistema con **Recall = 1.0** es como una red de pesca sin agujeros: atrapa absolutamente todos los peces. Sin embargo, el problema es que, para lograrlo, esa red suele ser tan pesada o grande que termina arrastrando también basura, piedras y algas. En términos operativos, buscar un Recall perfecto suele obligar al sistema a ser menos selectivo, lo que termina dañando la Precisión.

122.4.3.4. Ejemplo: El detector de incendios

- **Recall Alto (Sensibilidad total):** Imagina un detector de humo tan sensible que se activa incluso con el vapor de una ducha caliente o el humo de un cigarrillo a diez metros. Su Recall es de 1.0 porque es imposible que ocurra un incendio real sin que la alarma suene. El problema es que vivirás evacuando el edificio por errores (baja precisión).
- **Recall Bajo (Filtro exigente):** Ahora imagina que el detector solo se activa cuando las llamas tocan directamente el sensor. Su precisión será alta (si suena, es que hay fuego de verdad), pero su Recall será muy bajo: para cuando el sistema avise, el incendio ya habrá consumido media habitación. El sistema fue “preciso” pero llegó demasiado tarde porque no tuvo la capacidad de detectar el problema a tiempo.

122.4.4. Trade-off Precision–Recall: Tabla por Umbral

Como mencionaba Fawcett (2006), ambos extremos son fallas: un sistema con Recall bajo es “ciego” ante la realidad, mientras que uno con Recall alto pero baja precisión genera la ya mencionada fatiga de alarmas. En la práctica, el científico de datos debe encontrar un equilibrio entre ambos, lo que se conoce como el **trade-off Precision–Recall**. Este equilibrio se controla principalmente a través del **umbral de decisión**: el punto de corte sobre la probabilidad predicha que determina cuándo el modelo clasifica una instancia como positiva.

La siguiente tabla es la evidencia empírica del trade-off fundamental entre Recall y Precision que toda decisión de umbral debe considerar (Provost & Fawcett, 2013):

```
# Trade-off P-R según umbral - dataset médico
rows = []
for u in [0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80]:
    y_u = (y_prob_lr >= u).astype(int)
    tp_u = int(((y_u==1) & (y_te_m==1)).sum())
    fp_u = int(((y_u==1) & (y_te_m==0)).sum())
    fn_u = int(((y_u==0) & (y_te_m==1)).sum())
    tn_u = int(((y_u==0) & (y_te_m==0)).sum())
    rows.append({
        "Umbral ": u,
        "TP": tp_u, "FP": fp_u, "FN": fn_u, "TN": tn_u,
        "Recall": round(tp_u/(tp_u+fn_u), 3) if (tp_u+fn_u)>0 else 0,
        "Precision": round(tp_u/(tp_u+fp_u), 3) if (tp_u+fp_u)>0 else 0,
        "F1": round(f1_score(y_te_m, y_u, zero_division=0), 3),
    })
```

```
df_tradeoff = pd.DataFrame(rows)
print("Trade-off Precision-Recall según el umbral de decisión")
print("Dataset Médico - Regresión Logística")
print("(Provost & Fawcett, 2013)\n")
print(df_tradeoff.to_string(index=False))
```

Trade-off Precision-Recall según el umbral de decisión
 Dataset Médico - Regresión Logística
 (Provost & Fawcett, 2013)

Umbral	TP	FP	FN	TN	Recall	Precision	F1
0.2	11	20	14	255	0.44	0.355	0.393
0.3	10	12	15	263	0.40	0.455	0.426
0.4	9	3	16	272	0.36	0.750	0.486
0.5	6	2	19	273	0.24	0.750	0.364
0.6	6	0	19	275	0.24	1.000	0.387
0.7	5	0	20	275	0.20	1.000	0.333
0.8	3	0	22	275	0.12	1.000	0.214

122.4.4.1. Visualización del Trade-off

```
# Curvas Precision y Recall en función del umbral
umbrales_range = np.arange(0.05, 0.95, 0.01)
prec_vals, rec_vals = [], []

for u in umbrales_range:
    y_u = (y_prob_lr >= u).astype(int)
    prec_vals.append(precision_score(y_te_m, y_u, zero_division=0))
    rec_vals.append(recall_score(y_te_m, y_u, zero_division=0))

fig = go.Figure()
fig.add_trace(go.Scatter(x=umbrales_range, y=prec_vals, name="Precision",
                        line=dict(color="#4a90d9", width=2.5)))
fig.add_trace(go.Scatter(x=umbrales_range, y=rec_vals, name="Recall",
                        line=dict(color="#e74c3c", width=2.5)))
fig.add_vline(x=0.5, line_dash="dash", line_color="gray",
              annotation_text=" = 0.5 (default)")
fig.update_layout(
    title=(
        "Trade-off Precision vs. Recall según el Umbral de Decisión<br>"
        "<sup>Al subir : ↑ Precision, ↓ Recall - Fawcett (2006)</sup>"
    ),
    xaxis_title="Umbral de Decisión ( )",
    yaxis_title="Valor de la Métrica",
    yaxis=dict(range=[0, 1.05]),
    height=400,
    # legend=dict(orientation="h", yanchor="bottom", y=1.02),
```

```
)
fig.show()
```

Unable to display output for mime type(s): text/html

122.4.5. Specificity (Especificidad): Protección de los Negativos

122.4.5.1. Definición Formal

$$\text{Specificity} = \text{TN}R = \frac{TN}{TN + FP}$$

La Especificidad mide qué proporción de los casos verdaderamente negativos fue correctamente identificada como negativa por el clasificador (Han et al., 2011).

122.4.5.2. Cálculo y Relación con FPR

```
# Specificity: cálculo y relación con la Tasa de Falsos Positivos
specificity = TN / (TN + FP) if (TN + FP) > 0 else 0
fpr_val     = FP / (TN + FP) # 1 - Specificity = FPR

print(f"Specificity = {TN} / ({TN}+{FP}) = {specificity:.4f}")
print(f"FPR (1 - Specificity) = {fpr_val:.4f}")
print()
print("Tabla resumen de las tres métricas de relevancia:")
print(f" Precision   = {precision_score(y_te_m, y_pred_lr):.4f}  "
      f"- fiabilidad al predecir positivo")
print(f" Recall      = {recall_score(y_te_m, y_pred_lr):.4f}    "
      f"- cobertura de positivos reales")
print(f" Specificity = {specificity:.4f}  "
      f"- protección de los negativos reales")
```

Specificity = 273 / (273+2) = 0.9927

FPR (1 - Specificity) = 0.0073

Tabla resumen de las tres métricas de relevancia:

```
Precision   = 0.7500 - fiabilidad al predecir positivo
Recall      = 0.2400 - cobertura de positivos reales
Specificity = 0.9927 - protección de los negativos reales
```

122.4.6. Valor Predictivo Negativo (VPN): La fiabilidad del silencio

Mientras que la Precisión se enfoca en la confianza de una alerta (“¡Positivo!”), el **Valor Predictivo Negativo (VPN)** mide qué tanto podemos confiar en el “silencio” del sistema. Responde a la pregunta crucial: *Si el modelo dice que el caso es negativo, ¿qué tan seguro es que realmente lo sea?*

122.4.6.1. Definición Formal

$$NPV = \frac{TN}{TN + FN}$$

Un VPN alto permite que el usuario tome una decisión basada en el resultado negativo con total tranquilidad. Si el VPN es bajo, un resultado negativo no es confiable, pues existe un riesgo alto de que se trate de un positivo que el sistema no logró identificar.

122.4.6.2. Cálculo en Código

```
# Valor Predictivo Negativo (VPN)
vpn = TN / (TN + FN) if (TN + FN) > 0 else 0

print(f"VPN = {TN} / ({TN}+{FN}) = {vpn:.4f}")
print("Indica la probabilidad de que un caso sea realmente negativo tras una predicción negativa.
```

VPN = 273 / (273+19) = 0.9349
Indica la probabilidad de que un caso sea realmente negativo tras una predicción negativa.

122.4.7. Resumen de las “Cuatro Clásicas” de Confianza

Para que no te confundas, piensa en esto como un espejo:

Realidad del Modelo	Nombre de la Métrica	Lo que el usuario siente
El modelo dice “¡Positivo!”	Precisión (VPP)	“¿Será verdad o es una falsa alarma?”
El modelo dice “Negativo”	VPN	“¿Puedo estar tranquilo o me estaré confiando?”
El dato es Positivo	Sensibilidad (Recall)	“¿Tendrá el modelo la capacidad de verme?”
El dato es Negativo	Especificidad	“¿Sabrá el modelo dejarme en paz si no pasa nada?”

122.4.8. La Regla SnNout / SpPin en Medicina

James et al. (2021) introducen la regla mnemotécnica clínica:

- **SnNout** (alta **S**nsensibilidad → resultado **N**egativo descarta la enfermedad): las pruebas de **scree-****ning** deben tener alta Sensibilidad para no dejar pasar enfermos.
- **SpPin** (alta **S**pecificidad → resultado **P**ositivo confirma la enfermedad): las pruebas **confirmatorias** deben tener alta Especificidad para no confirmar falsos positivos.

122.5. F1-Score y sus Variantes Multiclase

122.5.1. La Necesidad de una Métrica Compuesta

El dilema entre Precision y Recall genera la necesidad de una métrica que capture ambas dimensiones en un solo número. El F1-Score resuelve esto mediante la media armónica (Han et al., 2011).

122.5.2. Definición Formal

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

122.5.3. Por Qué Media Armónica y No Aritmética

La media aritmética de Precision y Recall puede ocultar valores extremos. La media armónica penaliza asimétricamente a los clasificadores que sacrifican completamente una de las dos métricas:

```
# Demostración: media aritmética vs. armónica
def f1_manual(p, r):
    return 2 * p * r / (p + r) if (p + r) > 0 else 0

def media_aritmetica(p, r):
    return (p + r) / 2

casos = [
    ("Modelo A - balanceado", 0.90, 0.90),
    ("Modelo B - solo Precision", 0.99, 0.01),
    ("Modelo C - solo Recall", 0.01, 0.99),
    ("Modelo D - umbral alto moderado", 0.85, 0.60),
]

print(f"{'Modelo':<35} {'P':>5} {'R':>5} {'Aritmética':>11} {'F1 (armónica)':>14}")
print(" " * 78)
for nombre, p, r in casos:
    print(f"{'nombre':<35} {'p':>5.2f} {'r':>5.2f} "
          f"{'media_aritmetica(p,r)':>11.4f} {'f1_manual(p,r)':>14.4f}")

print()
print("La media armónica domina hacia el valor más bajo: castiga los extremos.")
print("(Han et al., 2011)")
```

Modelo	P	R	Aritmética	F1 (armónica)
Modelo A - balanceado	0.90	0.90	0.9000	0.9000
Modelo B - solo Precision	0.99	0.01	0.5000	0.0198
Modelo C - solo Recall	0.01	0.99	0.5000	0.0198
Modelo D - umbral alto moderado	0.85	0.60	0.7250	0.7034

La media armónica domina hacia el valor más bajo: castiga los extremos.
(Han et al., 2011)

122.5.4. La Generalización: F-Beta Score

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

La elección de β es una declaración explícita sobre la preferencia relativa entre minimizar FP o FN (James et al., 2021):

```
# F-beta: el parámetro como declaración de prioridad
p_lr = precision_score(y_te_m, y_pred_lr, zero_division=0)
r_lr = recall_score(y_te_m, y_pred_lr)

print(f"Modelo LR médico - P = {p_lr:.3f}, R = {r_lr:.3f}\n")
print(f"{' ':>5} {'Énfasis':<40} {'F-beta':>8}")
print(" " * 58)
for beta, desc in [
    (0.5, "Precision pesa 2× más (FP es más costoso)"),
    (1.0, "Precision = Recall (F1 estándar)"),
    (2.0, "Recall pesa 2× más (FN es más costoso)"),
    (5.0, "Recall domina (contexto crítico: FN catastrófico)"),
]:
    fb = fbeta_score(y_te_m, y_pred_lr, beta=beta, zero_division=0)
    print(f"{beta:>5} {desc:<40} {fb:>8.4f}")
```

Modelo LR médico - P = 0.750, R = 0.240

	Énfasis		F-beta
0.5	Precision pesa 2× más (FP es más costoso)		0.5263
1.0	Precision = Recall (F1 estándar)		0.3636
2.0	Recall pesa 2× más (FN es más costoso)		0.2778
5.0	Recall domina (contexto crítico: FN catastrófico)		0.2464

122.5.5. Variantes para Clasificación Multiclase

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c \quad F1_{\text{micro}} = \frac{2 \sum TP_c}{2 \sum TP_c + \sum FP_c + \sum FN_c} \quad F1_{\text{weighted}} = \sum_{c=1}^C w_c \cdot F1_c$$

```
# F1 macro / micro / weighted / binario
print("Variantes del F1-Score:")
for avg in ["macro", "micro", "weighted", "binary"]:
    val = f1_score(y_te_m, y_pred_lr, average=avg, zero_division=0)
```

```

    print(f"  F1 {avg:<10}: {val:.4f}")

print()
print("Reporte completo por clase:")
print(classification_report(
    y_te_m, y_pred_lr,
    target_names=["Sano (0)", "Enfermo (1)"],
))

```

Variantes del F1-Score:

```

F1 macro      : 0.6633
F1 micro      : 0.9300
F1 weighted   : 0.9130
F1 binary     : 0.3636

```

Reporte completo por clase:

	precision	recall	f1-score	support
Sano (0)	0.93	0.99	0.96	275
Enfermo (1)	0.75	0.24	0.36	25
accuracy			0.93	300
macro avg	0.84	0.62	0.66	300
weighted avg	0.92	0.93	0.91	300

122.5.6. Visualización Comparativa de Variantes

```

# Comparación de variantes del F1
variantes = ["macro", "micro", "weighted", "binary\n(clase positiva)"]
valores_f1 = [
    f1_score(y_te_m, y_pred_lr, average="macro"),
    f1_score(y_te_m, y_pred_lr, average="micro"),
    f1_score(y_te_m, y_pred_lr, average="weighted"),
    f1_score(y_te_m, y_pred_lr, average="binary"),
]
descripciones = [
    "Todas las clases\npeso igual",
    "Todas las instancias\npeso igual",
    "Ponderado por\nfrecuencia de clase",
    "Solo clase\npositiva (enfermos)",
]

fig = go.Figure(go.Bar(
    x=variantes, y=valores_f1,
    marker_color=["#4a90d9", "#27ae60", "#e67e22", "#e74c3c"],
    text=[f"{v:.3f}<br><i>{d}</i>" for v, d in zip(valores_f1, descripciones)],
    textposition="auto",

```

```

))
fig.update_layout(
    title=(
        "Variantes del F1-Score - ¿Qué mide cada una?<br>"
        "<sup>Han et al. (2011); Pedregosa et al. (2011)</sup>"
    ),
    yaxis=dict(title="F1", range=[0, 1.1]),
    height=420,
    xaxis_title="Tipo de promedio",
)
fig.show()

```

Unable to display output for mime type(s): text/html

122.6. Curva ROC y el Área Bajo la Curva (AUC)

122.6.1. Motivación

Muchas métricas de clasificación binaria dependen del **umbral de decisión** θ . Por ejemplo, un mismo modelo puede tener alta sensibilidad con un umbral bajo, pero también producir más falsos positivos. La **curva ROC** (*Receiver Operating Characteristic*) permite analizar este intercambio sin fijar un único umbral, evaluando el comportamiento del clasificador a lo largo de **todos los umbrales posibles** (Fawcett, 2006).

122.6.2. Definición formal

Sea $\hat{p}(x)$ el *score* o probabilidad estimada de que la observación x pertenezca a la clase positiva. Para cada umbral θ , el modelo predice:

- clase positiva si $\hat{p}(x) \geq \theta$,
- clase negativa si $\hat{p}(x) < \theta$.

A partir de ello se definen:

$$\text{TPR}(\theta) = \frac{TP(\theta)}{TP(\theta) + FN(\theta)}, \quad \text{FPR}(\theta) = \frac{FP(\theta)}{FP(\theta) + TN(\theta)} = 1 - \text{Specificity}(\theta)$$

donde:

- $\text{TPR}(\theta)$ es la **tasa de verdaderos positivos**,
- $\text{FPR}(\theta)$ es la **tasa de falsos positivos**.

La **curva ROC** es el conjunto de puntos

$$(\text{FPR}(\theta), \text{TPR}(\theta))$$

obtenidos al variar θ desde los valores más estrictos hasta los más flexibles.

122.6.3. Interpretación de sus ejes

122.6.3.1. 1. Tasa de Verdaderos Positivos (TPR)

La TPR coincide con el **Recall** o la **Sensibilidad**. Indica qué proporción de los casos realmente positivos es identificada correctamente por el modelo.

- **Nombre común:** Sensibilidad o Exhaustividad.
- **En lenguaje sencillo:** Si un caso realmente es positivo, ¿qué tan probable es que el modelo lo detecte?

122.6.3.2. 2. Tasa de Falsos Positivos (FPR)

La FPR es el complemento de la **Especificidad**. Indica qué proporción de los casos realmente negativos es clasificada incorrectamente como positiva.

- **Nombre común:** Tasa de falsos positivos o *fall-out*.
- **En lenguaje sencillo:** Si un caso realmente es negativo, ¿qué tan probable es que el modelo genere una falsa alarma?

122.6.4. ¿Qué representa la curva ROC?

La curva ROC muestra el compromiso entre:

- **detectar más positivos** (aumentar TPR),
- **evitar falsas alarmas** (mantener baja FPR).

Un clasificador deseable es aquel cuya curva se acerca a la esquina superior izquierda del plano ROC, pues allí se combina una **alta sensibilidad** con una **baja tasa de falsos positivos**.

Como referencia:

- la diagonal $TPR = FPR$ representa un clasificador sin capacidad discriminativa, equivalente a adivinar al azar;
- cuanto más por encima de esa diagonal se encuentre la curva, mejor será la capacidad del modelo para separar ambas clases.

122.6.5. Área Bajo la Curva (AUC)

El **AUC** (*Area Under the Curve*) resume la curva ROC en un solo número:

$$AUC = \int_0^1 TPR(u) du \quad \text{con } u = FPR$$

En términos prácticos, el AUC mide la capacidad global del modelo para **ordenar correctamente** a los positivos por encima de los negativos, sin depender de un umbral específico.

Sus valores suelen interpretarse así:

- $AUC \approx 0.5$: discriminación aleatoria,

- $AUC > 0.5$: mejor que el azar,
- $AUC \rightarrow 1$: excelente capacidad de discriminación.

122.6.6. Interpretación probabilística del AUC

Bamber (1975) mostró que el AUC tiene una interpretación probabilística muy natural:

$$AUC = P(\hat{p}(x^+) > \hat{p}(x^-))$$

donde x^+ es una instancia positiva elegida al azar y x^- una negativa, también elegida al azar.

Esto significa que el AUC puede entenderse como la probabilidad de que el modelo asigne un *score* mayor a un caso positivo que a uno negativo. Por ejemplo, un valor de $AUC = 0.85$ indica que, en promedio, existe un **85 % de probabilidad** de que el modelo puntúe más alto a un positivo que a un negativo seleccionados aleatoriamente.

122.6.7. Ventajas de la curva ROC y del AUC

- No dependen de un único umbral de decisión.
- Permiten comparar clasificadores de forma global.
- Son especialmente útiles cuando interesa evaluar la **capacidad de discriminación** del modelo más allá de una configuración específica.

122.6.8. Limitación importante

Aunque la curva ROC y el AUC son muy útiles, no siempre reflejan adecuadamente el desempeño en problemas con clases muy desbalanceadas. En esos casos, puede ser más informativa la **curva Precision-Recall**, porque penaliza de forma más directa los falsos positivos en contextos donde la clase positiva es rara.

```
# Curva ROC: Regresión Logística vs. Random Forest
fpr_lr, tpr_lr, thresholds_roc = roc_curve(y_te_m, y_prob_lr)
fpr_rf, tpr_rf, _              = roc_curve(y_te_m, y_prob_rf)

auc_lr = auc(fpr_lr, tpr_lr)
auc_rf = auc(fpr_rf, tpr_rf)

print(f"AUC - Regresión Logística : {auc_lr:.4f}")
print(f"AUC - Random Forest      : {auc_rf:.4f}")
print()
print(f"Interpretación LR: P(score(enfermo) > score(sano)) = {auc_lr:.1%}")
print(f"(Bamber, 1975; Fawcett, 2006)")
```

```
AUC - Regresión Logística : 0.7219
AUC - Random Forest      : 0.9201
```

```
Interpretación LR: P(score(enfermo) > score(sano)) = 72.2%
(Bamber, 1975; Fawcett, 2006)
```

122.6.9. Visualización Interactiva: Curva ROC Comparativa

```
# Curva ROC interactiva con área sombreada
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1], mode="lines",
    name="Aleatorio (AUC = 0.50)",
    line=dict(color="#aaa", width=1.5, dash="dash"),
))
fig.add_trace(go.Scatter(
    x=[0, 0, 1], y=[0, 1, 1], mode="lines",
    name="Perfecto (AUC = 1.00)",
    line=dict(color="#27ae60", width=1.5, dash="dot"),
))
fig.add_trace(go.Scatter(
    x=fpr_lr, y=tpr_lr, mode="lines",
    name=f"Reg. Logística (AUC = {auc_lr:.3f})",
    line=dict(color="#4a90d9", width=2.5),
    fill="tozeroy", fillcolor="rgba(74,144,217,0.10)",
))
fig.add_trace(go.Scatter(
    x=fpr_rf, y=tpr_rf, mode="lines",
    name=f"Random Forest (AUC = {auc_rf:.3f})",
    line=dict(color="#e74c3c", width=2.5),
    fill="tozeroy", fillcolor="rgba(231,76,60,0.08)",
))

fig.update_layout(
    title=(
        "Curva ROC - Comparación de Modelos (Dataset Médico)<br>"
        "<sup>Fawcett (2006); Bamber (1975)</sup>"
    ),
    xaxis_title="Tasa de Falsos Positivos (FPR = 1 - Specificity)",
    yaxis_title="Tasa de Verdaderos Positivos (Recall)",
    xaxis=dict(range=[0, 1]), yaxis=dict(range=[0, 1.02]),
    height=460,
    legend=dict(x=0.55, y=0.15),
)
fig.show()
```

Unable to display output for mime type(s): text/html

122.6.10. Limitaciones del AUC-ROC en Datos Desbalanceados

Saito y Rehmsmeier (2015) demostraron que en datasets con desbalance severo, el AUC-ROC puede sobreestimar el desempeño del clasificador porque incluye los TN en el denominador de la FPR: cuando los

negativos son muy numerosos, incluso muchos FP producen una FPR aparentemente pequeña, haciendo que la curva parezca optimista.

122.7. Curva Precision–Recall y Average Precision

122.7.1. Motivación

En muchos problemas de clasificación, especialmente cuando la clase positiva es poco frecuente, no basta con saber cuántos aciertos globales tiene el modelo. En estos contextos suele ser más importante responder dos preguntas:

- cuando el modelo predice un positivo, ¿qué tan confiable es esa predicción?;
- de todos los positivos reales, ¿cuántos está logrando recuperar?

La **curva Precision–Recall (PR)** se construye precisamente para estudiar este compromiso. A diferencia de la curva ROC, aquí los **verdaderos negativos (TN)** no intervienen de forma explícita. Por ello, esta curva se enfoca en el desempeño del modelo sobre la **clase positiva**, lo cual resulta particularmente útil en escenarios con clases desbalanceadas (Saito & Rehmsmeier, 2015).

122.7.2. Definición formal

Sea $\hat{p}(x)$ el *score* o probabilidad estimada para la clase positiva, y sea θ un umbral de decisión. Para cada valor de θ , se definen:

$$\text{Precision}(\theta) = \frac{TP(\theta)}{TP(\theta) + FP(\theta)}, \quad \text{Recall}(\theta) = \frac{TP(\theta)}{TP(\theta) + FN(\theta)}$$

donde:

- **Precision** mide la proporción de predicciones positivas que realmente son correctas;
- **Recall** mide la proporción de positivos reales que el modelo logra recuperar.

La **curva Precision–Recall** es el conjunto de puntos

$$(\text{Recall}(\theta), \text{Precision}(\theta))$$

obtenidos al variar el umbral θ .

122.7.3. Interpretación de sus ejes

122.7.3.1. 1. Precision

La **Precision** evalúa la pureza de las predicciones positivas. Es alta cuando, entre los casos señalados como positivos por el modelo, predominan los verdaderos positivos.

- **Nombre común:** Precisión.
- **En lenguaje sencillo:** Si el modelo dice “positivo”, ¿qué tan probable es que tenga razón?

122.7.3.2. 2. Recall

El **Recall** o **Sensibilidad** mide la capacidad del modelo para no dejar escapar positivos reales.

- **Nombre común:** Sensibilidad o Exhaustividad.
- **En lenguaje sencillo:** De todos los positivos reales, ¿cuántos logra detectar el modelo?

122.7.4. ¿Qué representa la curva Precision–Recall?

La curva PR muestra el compromiso entre:

- **recuperar más positivos** (aumentar Recall),
- **mantener confiables las alertas positivas** (conservar alta Precision).

En general, al hacer el umbral más flexible, el modelo detecta más positivos y el Recall aumenta, pero también tiende a cometer más falsos positivos, por lo que la Precision puede disminuir. La curva permite visualizar este intercambio de manera completa.

Un modelo deseable es aquel cuya curva se mantiene lo más arriba posible, pues eso indica que conserva una **alta precisión** incluso cuando incrementa su cobertura de positivos.

122.7.5. Baseline de la curva PR

A diferencia de la curva ROC, cuya referencia natural es la diagonal aleatoria, en la curva PR la línea de referencia depende de la **prevalencia** de la clase positiva en el conjunto de datos:

$$\text{Baseline PR} = P(\text{positivo}) = \frac{TP + FN}{N}$$

Esto significa que, si la clase positiva representa por ejemplo el 10 % de los datos, entonces un clasificador sin capacidad real de discriminación tenderá a tener una Precision cercana a 0.10.

Por ello, en problemas desbalanceados, una curva PR y un valor de AP por encima de la prevalencia indican que el modelo está haciendo algo mejor que una asignación aleatoria proporcional.

122.7.6. Average Precision (AP)

El **Average Precision (AP)** resume la curva Precision–Recall en un solo número. Puede entenderse como un promedio ponderado de la precisión a lo largo de distintos niveles de recall:

$$AP = \sum_k (R_k - R_{k-1}) P_k$$

donde R_k representa el Recall en el punto k y P_k la Precision correspondiente.

En términos prácticos, el AP asigna mayor valor a los modelos que no solo recuperan positivos, sino que lo hacen manteniendo una buena precisión a medida que aumenta la cobertura.

122.7.7. Interpretación del AP

- Un valor de **AP** cercano a 1 indica que el modelo logra identificar positivos con alta precisión a lo largo de casi todo el rango de recall.
- Un valor de **AP** cercano a la prevalencia sugiere un comportamiento cercano al azar bajo esta métrica.
- Cuanto mayor sea el AP respecto a la prevalencia, mejor será la capacidad del modelo para priorizar correctamente la clase positiva.

122.7.8. ¿Por qué es útil en clases desbalanceadas?

Cuando la clase positiva es rara, puede ocurrir que un modelo tenga una curva ROC aparentemente buena y, sin embargo, produzca demasiados falsos positivos para que sus predicciones sean útiles en la práctica. La curva PR evita que los TN dominen la evaluación y se concentra en lo que muchas veces realmente importa: la capacidad de encontrar positivos sin generar demasiadas falsas alarmas.

Por esta razón, en aplicaciones médicas, detección de fraude, mantenimiento predictivo o búsqueda de eventos raros, la curva PR y el AP suelen ser más informativos que la curva ROC.

```
# Curva Precision-Recall: LR vs. RF
prec_lr_c, rec_lr_c, _ = precision_recall_curve(y_te_m, y_prob_lr)
prec_rf_c, rec_rf_c, _ = precision_recall_curve(y_te_m, y_prob_rf)

ap_lr = average_precision_score(y_te_m, y_prob_lr)
ap_rf = average_precision_score(y_te_m, y_prob_rf)
prevalencia = y_te_m.mean()

print(f"Average Precision - Reg. Logística: {ap_lr:.4f}")
print(f"Average Precision - Random Forest : {ap_rf:.4f}")
print(f"Baseline (prevalencia clase +)      : {prevalencia:.4f}")
print()
print("Un AP mayor que la prevalencia indica que el modelo supera")
print("una predicción aleatoria proporcional. (Saito & Rehmsmeier, 2015)")
```

```
Average Precision - Reg. Logística: 0.4637
Average Precision - Random Forest : 0.7544
Baseline (prevalencia clase +)      : 0.0833
```

Un AP mayor que la prevalencia indica que el modelo supera una predicción aleatoria proporcional. (Saito & Rehmsmeier, 2015)

122.7.9. Visualización Interactiva: Curva PR Comparativa

```
# Curva Precision-Recall interactiva
fig = go.Figure()

fig.add_hline(
```

```

y=prevalencia, line_dash="dash", line_color="#e74c3c",
annotation_text=f"Baseline aleatoria (prevalencia = {prevalencia:.2f})",
annotation_position="top right",
)
fig.add_trace(go.Scatter(
    x=rec_lr_c, y=prec_lr_c, mode="lines",
    name=f"Reg. Logística (AP = {ap_lr:.3f})",
    line=dict(color="#4a90d9", width=2.5),
    fill="tozeroy", fillcolor="rgba(74,144,217,0.12)",
))
fig.add_trace(go.Scatter(
    x=rec_rf_c, y=prec_rf_c, mode="lines",
    name=f"Random Forest (AP = {ap_rf:.3f})",
    line=dict(color="#e67e22", width=2.5),
    fill="tozeroy", fillcolor="rgba(230,126,34,0.10)",
))
fig.update_layout(
    title=(
        "Curva Precision-Recall - Comparación de Modelos (Dataset Médico)<br>"
        "<sup>Preferible a ROC en clases desbalanceadas - Saito & Rehmsmeier (2015)</sup>"
    ),
    xaxis_title="Recall",
    yaxis_title="Precision",
    xaxis=dict(range=[0, 1]), yaxis=dict(range=[0, 1.05]),
    height=460,
    legend=dict(x=0.45, y=0.95),
)
fig.show()

```

Unable to display output for mime type(s): text/html

122.7.10. Panel Comparativo ROC vs. Precision-Recall

```

# Panel 1x2: ROC vs. PR para el mismo modelo
fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=[
        f"Curva ROC (AUC = {auc_lr:.3f})",
        f"Curva Precision-Recall (AP = {ap_lr:.3f})",
    ],
)
# ROC
fig.add_trace(go.Scatter(x=[0,1], y=[0,1], mode="lines",
    line=dict(color="#aaa", dash="dash"), showlegend=False),
    row=1, col=1)
fig.add_trace(go.Scatter(x=fpr_lr, y=tpr_lr, mode="lines",

```

```

    line=dict(color="#4a90d9", width=2.2), name="LR (ROC)", showlegend=True),
    row=1, col=1)
# PR
fig.add_hline(y=prevalencia, line_dash="dash", line_color="#e74c3c",
              row=1, col=2)
fig.add_trace(go.Scatter(x=rec_lr_c, y=prec_lr_c, mode="lines",
                        line=dict(color="#4a90d9", width=2.2), name="LR (PR)", showlegend=True),
              row=1, col=2)

fig.update_xaxes(range=[0, 1], row=1, col=1, title_text="FPR")
fig.update_yaxes(range=[0, 1.02], row=1, col=1, title_text="TPR (Recall)")
fig.update_xaxes(range=[0, 1], row=1, col=2, title_text="Recall")
fig.update_yaxes(range=[0, 1.05], row=1, col=2, title_text="Precision")
fig.update_layout(
    height=500,
    title=(
        "ROC vs. Precision-Recall: el mismo modelo, perspectivas distintas<br>"
        "<sup>Saito & Rehmsmeier (2015)</sup>"
    ),
)
fig.show()

```

Unable to display output for mime type(s): text/html

122.8. Optimización del Umbral de Decisión

122.8.1. El Umbral como Parámetro Estratégico

La mayoría de los clasificadores probabilísticos producen $\hat{p}(x) \in [0, 1]$ antes de asignar una clase. El umbral por defecto $\theta = 0.5$ es una convención, no una elección óptima (Provost & Fawcett, 2013). Si se conocen los costos de cada tipo de error, el umbral óptimo puede derivarse analíticamente (James et al., 2021):

$$\theta^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- **Medicina** con $C_{FN} = 100 \cdot C_{FP}$: $\theta^* = 1/101 \approx 0.01$ (casi cualquier señal positiva debe investigarse)
- **Finanzas** con $C_{FP} = 10 \cdot C_{FN}$: $\theta^* = 10/11 \approx 0.91$ (se requiere certeza alta antes de bloquear)

122.8.2. Efecto del Umbral sobre Todas las Métricas


```

# Efecto del umbral sobre accuracy, precision, recall y F1
umbrales_demo = np.arange(0.05, 0.95, 0.02)
metricas_umbral = []

for u in umbrales_demo:
    y_u = (y_prob_lr >= u).astype(int)
    tp_u = int(((y_u==1) & (y_te_m==1)).sum())
    tn_u = int(((y_u==0) & (y_te_m==0)).sum())
    fp_u = int(((y_u==1) & (y_te_m==0)).sum())
    fn_u = int(((y_u==0) & (y_te_m==1)).sum())
    n = tp_u + tn_u + fp_u + fn_u
    metricas_umbral.append({
        "umbral": u,
        "accuracy": (tp_u + tn_u) / n if n > 0 else 0,
        "precision": tp_u / (tp_u + fp_u) if (tp_u + fp_u) > 0 else 0,
        "recall": tp_u / (tp_u + fn_u) if (tp_u + fn_u) > 0 else 0,
        "f1": 2*tp_u / (2*tp_u+fp_u+fn_u) if (2*tp_u+fp_u+fn_u)>0 else 0,
    })

df_umb = pd.DataFrame(metricas_umbral)

fig = go.Figure()
for col, color, name in [
    ("accuracy", "#4a90d9", "Accuracy"),
    ("precision", "#27ae60", "Precision"),
    ("recall", "#e74c3c", "Recall"),
    ("f1", "#e67e22", "F1-Score"),
]:
    fig.add_trace(go.Scatter(
        x=df_umb["umbral"], y=df_umb[col],
        name=name, line=dict(color=color, width=2.2), mode="lines",
    ))

idx_best = df_umb["f1"].idxmax()
best_u = df_umb.loc[idx_best, "umbral"]
fig.add_vline(x=0.5, line_dash="dash", line_color="gray",
    annotation_text="=0.5 (default)")
fig.add_vline(x=best_u, line_dash="dot", line_color="#e67e22",
    annotation_text=f"*_F1 = {best_u:.2f}")

fig.update_layout(
    title=(
        "Efecto del Umbral de Decisión sobre las Métricas<br>"
        "<sup>Provost & Fawcett (2013)</sup>"
    ),
    xaxis_title="Umbral de Decisión ( )",
    yaxis_title="Valor",
    yaxis=dict(range=[0, 1.05]),
    height=430,

```

```

    legend=dict(orientation="h", yanchor="bottom", y=1.02),
)
fig.show()

```

Unable to display output for mime type(s): text/html

122.8.3. Búsqueda del Umbral Óptimo según el Criterio

```

# Tres estrategias de selección del umbral
print("Estrategia 1: Maximizar F1-Score")
f1_arr = [f1_score(y_te_m, (y_prob_lr >= u).astype(int), zero_division=0)
           for u in np.arange(0.01, 0.99, 0.01)]
u_best_f1 = np.arange(0.01, 0.99, 0.01)[np.argmax(f1_arr)]
y_best_f1 = (y_prob_lr >= u_best_f1).astype(int)
print(f" * = {u_best_f1:.2f} → F1={max(f1_arr):.4f}, "
      f"P={precision_score(y_te_m, y_best_f1, zero_division=0):.4f}, "
      f"R={recall_score(y_te_m, y_best_f1):.4f}\n")

print("Estrategia 2 (médica): mayor con Recall 0.90")
best_med_u = None
for u in np.arange(0.01, 0.99, 0.01):
    if recall_score(y_te_m, (y_prob_lr >= u).astype(int), zero_division=0) >= 0.90:
        best_med_u = u
if best_med_u is not None:
    y_med_u = (y_prob_lr >= best_med_u).astype(int)
    print(f" * = {best_med_u:.2f} → "
          f"P={precision_score(y_te_m, y_med_u, zero_division=0):.4f}, "
          f"R={recall_score(y_te_m, y_med_u):.4f}\n")

print("Estrategia 3: Punto de Youden (max TPR - FPR)")
J = tpr_lr - fpr_lr
idx_yo = np.argmax(J)
u_youden = thresholds_roc[idx_yo]
y_yo = (y_prob_lr >= u_youden).astype(int)
print(f" * = {u_youden:.2f} → J={J[idx_yo]:.4f}, "
      f"P={precision_score(y_te_m, y_yo, zero_division=0):.4f}, "
      f"R={recall_score(y_te_m, y_yo):.4f}")
print(" (James et al., 2021)")

```

Estrategia 1: Maximizar F1-Score

* = 0.34 → F1=0.5000, P=0.6667, R=0.4000

Estrategia 2 (médica): mayor con Recall 0.90

* = 0.01 → P=0.0924, R=0.9200

Estrategia 3: Punto de Youden (max TPR - FPR)

* = 0.27 → J=0.3927, P=0.4583, R=0.4400

(James et al., 2021)

122.9. Comparación Final de Modelos

Esta sección demuestra que un modelo con **menor accuracy puede ser estratégicamente superior** cuando el contexto así lo demanda (Provost & Fawcett, 2013). Se comparan tres configuraciones del clasificador médico:

```
# Tres configuraciones del clasificador médico
y_base = y_pred_lr # LR =0.50
y_medico = (y_prob_lr >= 0.25).astype(int) # LR =0.25 (médico)
y_rf_b = y_pred_rf # RF =0.50

modelos_cmp = {
    "LR =0.50\n(base)": y_base,
    "LR =0.25\n(médico)": y_medico,
    "RF =0.50\n(base)": y_rf_b,
}

resultados = []
for nombre, y_c in modelos_cmp.items():
    resultados.append({
        "Modelo": nombre,
        "Accuracy": round(accuracy_score(y_te_m, y_c), 4),
        "Precision": round(precision_score(y_te_m, y_c, zero_division=0), 4),
        "Recall": round(recall_score(y_te_m, y_c, zero_division=0), 4),
        "F1": round(f1_score(y_te_m, y_c, zero_division=0), 4),
    })

df_cmp = pd.DataFrame(resultados)
print("Comparación de modelos - Dataset Médico")
print("(Provost & Fawcett, 2013)\n")
print(df_cmp.to_string(index=False))
print()
print(" El modelo 'LR =0.25' tiene menor Accuracy que el base,")
print(" pero detecta más enfermos reales (mayor Recall).")
print(" Para un médico, ese es el modelo correcto.")
```

Comparación de modelos - Dataset Médico
(Provost & Fawcett, 2013)

	Modelo	Accuracy	Precision	Recall	F1
	LR =0.50\n(base)	0.9300	0.7500	0.24	0.3636
	LR =0.25\n(médico)	0.8967	0.3929	0.44	0.4151
	RF =0.50\n(base)	0.9500	1.0000	0.40	0.5714

El modelo 'LR =0.25' tiene menor Accuracy que el base,
pero detecta más enfermos reales (mayor Recall).
Para un médico, ese es el modelo correcto.

```
# Visualización comparativa
metricas_plot = ["Accuracy", "Precision", "Recall", "F1"]
colores_plot = ["#4a90d9", "#27ae60", "#e74c3c", "#e67e22"]

fig = go.Figure()
for metrica, color in zip(metricas_plot, colores_plot):
    fig.add_trace(go.Bar(
        name=metrica,
        x=df_cmp["Modelo"],
        y=df_cmp[metrica],
        marker_color=color,
        text=[f"{v:.3f}" for v in df_cmp[metrica]],
        textposition="auto",
    ))

fig.update_layout(
    barmode="group",
    title=(
        "Comparación de Modelos: ¿Más Accuracy = Mejor Modelo?<br>"
        "<sup>La métrica correcta depende del costo del error - Provost & Fawcett (2013)</sup>"
    ),
    yaxis=dict(title="Valor", range=[0, 1.1]),
    height=450,
    legend=dict(orientation="h", yanchor="bottom", y=1.02),
)
fig.show()
```

Unable to display output for mime type(s): text/html

122.9.1. Tabla-Resumen: ¿Qué Métrica Usar y Cuándo?

Situación	Métrica recomendada	Referencia clave
Dataset balanceado	Accuracy, F1 macro	Han et al. (2011)
Clase positiva rara (< 10 %)	F1 clase positiva, PR-AUC	Saito & Rehmsmeier (2015)
FN muy costoso (medicina, seguridad)	Recall / Sensibilidad	James et al. (2021)
FP muy costoso (spam, fraude leve)	Precision	Fawcett (2006)
FP y FN con costo similar	F1-Score	Han et al. (2011)
Comparar modelos sin fijar umbral	ROC-AUC	Fawcett (2006); Bamber (1975)
Clase escasa + interés en ranking	Precision-Recall AUC	Saito & Rehmsmeier (2015)

Situación	Métrica recomendada	Referencia clave
Multiclase, todas las clases importan igual	F1 macro	Pedregosa et al. (2011)

122.10. Errores Comunes en la Interpretación de Métricas

122.10.1. Reportar Solo Accuracy en Datos Desbalanceados

El error más frecuente y más dañino (Saito & Rehmsmeier, 2015). La paradoja de la exactitud demostrada en la Sección 4.3 hace que la accuracy sea engañosamente alta en datos desbalanceados. **Regla práctica:** siempre reportar Precision, Recall y F1 cuando la proporción entre clases supera 80/20.

122.10.2. Confundir Precision con Recall

Precision mide la fiabilidad de las predicciones positivas; Recall mide la cobertura de los positivos reales. Son métricas ortogonales y complementarias, no intercambiables. Confundirlas lleva a interpretar que un modelo con alta Precision “no falla con los enfermos” cuando en realidad puede estar dejando sin detectar a la mayoría (Fawcett, 2006).

122.10.3. Asumir que Mayor AUC = Mejor Modelo en Producción

```
# AUC alto pero F1 bajo en el umbral operativo
# Modelo bien rankeado pero con probabilidades mal calibradas
np.random.seed(99)
y_fake_prob = np.where(
    y_te_m == 1,
    np.random.beta(3, 1, size=y_te_m.shape),
    np.random.beta(1, 2, size=y_te_m.shape),
)
y_fake_pred = (y_fake_prob >= 0.50).astype(int)

fpr_fk, tpr_fk, _ = roc_curve(y_te_m, y_fake_prob)
auc_fk = auc(fpr_fk, tpr_fk)

print("Modelo con AUC alto pero mal calibrado (F1 bajo al umbral default):")
print(f"  AUC                : {auc_fk:.4f}")
print(f"  F1 ( = 0.50)       : {f1_score(y_te_m, y_fake_pred, zero_division=0):.4f}")
print(f"  Recall              : {recall_score(y_te_m, y_fake_pred, zero_division=0):.4f}")
print()
print("Conclusión: el AUC mide capacidad de ranking, no desempeño operativo.")
print("Validar siempre las métricas en el umbral de producción.")
print("(Fawcett, 2006)")
```

Modelo con AUC alto pero mal calibrado (F1 bajo al umbral default):

```
AUC           : 0.9340
F1 ( = 0.50)  : 0.3962
Recall        : 0.8400
```

Conclusión: el AUC mide capacidad de ranking, no desempeño operativo.

Validar siempre las métricas en el umbral de producción.

(Fawcett, 2006)

122.10.4. Optimizar el Umbral en el Conjunto de Prueba (Data Leakage)

Ajustar el umbral sobre los mismos datos con los que se evalúa constituye *data leakage* y produce estimaciones optimistas del desempeño real (James et al., 2021). El umbral debe seleccionarse en el **conjunto de validación** y evaluarse únicamente en el **conjunto de prueba**.

122.10.5. Usar F1 Macro Ocultando el Desempeño en la Clase Minoritaria

```
# F1 macro oculta mal desempeño en clase positiva
y_casi_trivial      = np.zeros_like(y_te_m)
idx_top2            = np.argsort(y_prob_lr)[-2:]
y_casi_trivial[idx_top2] = 1

f1_macro_mal = f1_score(y_te_m, y_casi_trivial, average="macro")
f1_pos_mal   = f1_score(y_te_m, y_casi_trivial, average="binary")

print("Modelo que casi nunca detecta enfermos (solo 2 predicciones positivas):")
print(f"  F1 macro    = {f1_macro_mal:.4f}  ← parece 'aceptable'")
print(f"  F1 clase +   = {f1_pos_mal:.4f}    ← revela el verdadero problema")
print()
print("Siempre revisar el F1 por clase, no solo el promedio.")
print("(Han et al., 2011)")
```

Modelo que casi nunca detecta enfermos (solo 2 predicciones positivas):

```
F1 macro    = 0.5540  ← parece 'aceptable'
F1 clase +   = 0.1481  ← revela el verdadero problema
```

Siempre revisar el F1 por clase, no solo el promedio.

(Han et al., 2011)

122.11. Guía de Selección de Métricas

La selección de la métrica adecuada debe seguir un proceso de análisis del contexto antes de calcular cualquier número (Provost & Fawcett, 2013):

Paso 1 — Caracterizar el desbalance: Si la proporción entre clases supera 80/20, descartar la accuracy como métrica principal (Saito & Rehmsmeier, 2015).

Paso 2 — Identificar el costo diferencial del error: FN catastróficos → Recall/Sensibilidad. FP muy costosos → Precision. Ambos similares → F1-Score (James et al., 2021).

Paso 3 — Determinar si se comparan modelos o se elige un umbral: Comparación global → ROC-AUC o PR-AUC. Configuración en producción → métricas en el umbral operativo (Fawcett, 2006).

Paso 4 — Naturaleza multiclase: Todas las clases con igual importancia de negocio → F1 Macro. Volumen total de errores → F1 Micro. Ponderación por prevalencia → F1 Weighted (Han et al., 2011; Pedregosa et al., 2011).

Paso 5 — Traducción a impacto de negocio: Las métricas estadísticas deben conectar con decisiones reales: ¿cuántos pacientes se salvan?, ¿cuánto fraude se evita?, ¿cuántos clientes se retienen? (Provost & Fawcett, 2013).

122.12. Conclusiones

La evaluación de clasificadores en Minería de Datos no es un paso mecánico al final del proceso KDD (Fayyad et al., 1996): es una actividad de análisis estratégico que conecta la calidad estadística del modelo con la utilidad práctica del conocimiento descubierto.

Las métricas presentadas en este capítulo forman un sistema complementario (Han et al., 2011). La accuracy provee contexto general pero falla en datos desbalanceados (Saito & Rehmsmeier, 2015). Precision y Recall cuantifican el balance entre los dos tipos de error con implicaciones radicalmente diferentes según el costo asimétrico del dominio (Provost & Fawcett, 2013). El F1-Score sintetiza ese balance mediante la media armónica, penalizando asimétricamente los extremos (James et al., 2021). La curva ROC y el AUC, con su interpretación probabilística directa (Bamber, 1975; Fawcett, 2006), caracterizan la capacidad discriminativa independiente del umbral, pero deben complementarse con la curva Precision-Recall cuando la clase positiva es minoritaria (Saito & Rehmsmeier, 2015). El umbral de decisión, finalmente, es una palanca estratégica derivable del costo diferencial del error (Provost & Fawcett, 2013).

La competencia central del científico de datos en este dominio no es memorizar fórmulas, sino comprender qué declara cada métrica sobre el modelo y el problema, y elegir la que mejor alinea la optimización estadística con el objetivo de negocio.

122.13. Referencias

- Bamber, D. (1975). The area above the ordinal dominance graph and the area below the receiver operating characteristic graph. *Journal of Mathematical Psychology*, 12(4), 387–415. [https://doi.org/10.1016/0022-2496\(75\)90001-2](https://doi.org/10.1016/0022-2496(75)90001-2)
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>

- Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>
- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Provost, F., & Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
-

Fin del Capítulo 7 — Métricas de Clasificación en Minería de Datos

123. Evaluación de un modelo de clasificación

123.1. Introducción

La evaluación de un modelo de clasificación es una etapa central del aprendizaje supervisado. En este tipo de problemas se dispone de un conjunto de observaciones para las cuales se conocen tanto las variables predictoras como la etiqueta o clase verdadera. A partir de estos datos, el objetivo es aprender una regla de decisión capaz de asignar una clase a nuevas observaciones no vistas durante el entrenamiento (Hastie et al. 2009; James et al. 2021).

En términos prácticos, evaluar un clasificador no significa únicamente calcular cuántas predicciones son correctas. La calidad de un modelo depende también de cómo se distribuyen sus errores, del balance entre clases, del coste relativo de falsos positivos y falsos negativos, y del contexto de uso. Por ejemplo, en un sistema de filtrado de spam puede ser aceptable que algunos correos no deseados lleguen a la bandeja principal, pero puede ser mucho más grave clasificar como spam un correo importante. En un sistema de detección médica, en cambio, puede ser preferible generar falsos positivos antes que dejar pasar casos reales de enfermedad (Fawcett 2006; Saito y Rehmsmeier 2015).

Este capítulo desarrolla los elementos fundamentales para evaluar modelos de clasificación. Primero se estudian los métodos de partición de datos, como holdout y validación cruzada K -fold. Después se introducen la matriz de confusión y las principales métricas derivadas de ella: exactitud, tasa de error, precisión, recall, sensibilidad, especificidad y F1-score. Finalmente, se discute la curva ROC, el área bajo la curva y la selección de métricas según la necesidad del problema.

123.2. Aprendizaje supervisado y evaluación

En aprendizaje supervisado se trabaja con un conjunto de datos etiquetado. Cada observación contiene un vector de características $\mathbf{x}_i$ y una etiqueta asociada y_i . En clasificación binaria, la etiqueta suele tomar valores en $\{0, 1\}$, donde 1 representa la clase positiva y 0 la clase negativa.

i Nota

Definición. Un conjunto de datos supervisado es una colección de pares entrada-salida:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^p, \quad y_i \in \mathcal{Y}$$

El modelo de clasificación puede entenderse como una función entrenable

$$f_\theta : \mathbb{R}^p \rightarrow \mathcal{Y},$$

donde θ representa los parámetros aprendidos a partir de los datos. En modelos probabilísticos, el clasificador puede producir una puntuación o probabilidad estimada:

$$\hat{p}_i = P_\theta(y_i = 1 \mid \mathbf{x}_i).$$

Para convertir esta puntuación en una clase, se usa un umbral de decisión τ :

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases}$$

La evaluación se realiza sobre datos no utilizados para entrenar el modelo. Esto busca estimar su capacidad de generalización, es decir, su desempeño ante nuevas observaciones. Si se evalúa el modelo únicamente sobre los datos de entrenamiento, se corre el riesgo de obtener una estimación demasiado optimista debido al sobreajuste (Hastie et al. 2009; Bishop 2006).

123.3. Partición de datos

Una práctica estándar consiste en dividir el conjunto de datos en subconjuntos con funciones distintas. La presentación base distingue entre conjunto de entrenamiento, conjunto de validación y conjunto de prueba.

i Nota

Definición. Una partición entrenamiento–validación–prueba divide el conjunto de datos en tres subconjuntos disjuntos:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_a \cap \mathcal{D}_b = \emptyset \quad (a \neq b)$$

El conjunto de entrenamiento se usa para ajustar los parámetros del modelo. El conjunto de validación se usa para comparar configuraciones, ajustar hiperparámetros o seleccionar entre modelos candidatos. El conjunto de prueba se reserva hasta el final para estimar el desempeño del modelo seleccionado.

Una partición típica puede ser:

Subconjunto	Porcentaje típico	Función principal
Entrenamiento	70 %	Ajustar el modelo
Validación	15 %	Seleccionar hiperparámetros o modelos
Prueba	15 %	Evaluar el modelo final

Otra alternativa, frecuente cuando se desea dedicar más datos a validación y prueba, es dividir en 50 % para entrenamiento, 25 % para validación y 25 % para prueba. La elección depende del tamaño del dataset, del coste computacional y de la estabilidad esperada de la evaluación.

123.3.1. Método holdout

El método holdout, o método de reserva, consiste en dividir el conjunto de datos una sola vez. Por ejemplo, se puede asignar 80 % de las observaciones al conjunto de entrenamiento y 20 % al conjunto de prueba.

i Nota

Definición. En el método holdout se construye una partición única:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$$

Su principal ventaja es la simplicidad. Es fácil de implementar, rápido de ejecutar y escalable a conjuntos de datos grandes. Sin embargo, su principal desventaja es que la evaluación depende fuertemente de la partición realizada. Si por azar el conjunto de prueba contiene ejemplos demasiado fáciles, el rendimiento parecerá mejor de lo real. Si contiene ejemplos particularmente difíciles o atípicos, el rendimiento parecerá peor.

En problemas con clases desbalanceadas, el holdout aleatorio puede producir particiones con representación insuficiente de la clase minoritaria. Por ejemplo, si un dataset contiene 950 registros de pacientes sin enfermedad y solo 50 con enfermedad, una división aleatoria puede dejar pocos casos positivos en validación o prueba. Esto vuelve inestable la estimación de métricas como recall o sensibilidad.

123.3.2. Muestreo aleatorio estratificado

Para reducir el riesgo de particiones no representativas, se usa el muestreo estratificado. La idea es preservar aproximadamente la proporción de clases en cada subconjunto. Si la clase A representa 80 % del dataset y la clase B representa 20 %, entonces entrenamiento, validación y prueba deben conservar proporciones cercanas a 80 % y 20 %.

i Nota

Definición. Una partición estratificada preserva aproximadamente la distribución marginal de clases:

$$P_{\mathcal{D}_{\text{train}}}(Y = y) \approx P_{\mathcal{D}_{\text{val}}}(Y = y) \approx P_{\mathcal{D}_{\text{test}}}(Y = y) \approx P_{\mathcal{D}}(Y = y)$$

La estratificación es especialmente útil cuando hay desbalance de clases. No obstante, no elimina todos los problemas. Aunque preserve proporciones, puede ocurrir que algunos subconjuntos contengan casos más fáciles, más difíciles o más atípicos que otros. En datasets pequeños, incluso la estratificación puede ser insuficiente para obtener una evaluación estable.

123.3.3. Holdout repetido

Una extensión natural del holdout es repetir varias veces el proceso de partición aleatoria y promediar las métricas. Así se reduce la dependencia respecto a una única división.

i Nota

Definición. En holdout repetido se generan R particiones independientes y se reporta el promedio de una métrica M :

$$\bar{M} = \frac{1}{R} \sum_{r=1}^R M^{(r)}$$

Por ejemplo, si cinco repeticiones producen exactitudes de 85 %, 82 %, 88 %, 84 % y 86 %, el promedio es:

$$\bar{M} = \frac{85 + 82 + 88 + 84 + 86}{5} = 85 \%$$

Este método ofrece una evaluación más estable que el holdout simple, pero tiene una limitación: algunos registros pueden aparecer muchas veces en conjuntos de prueba o entrenamiento, mientras que otros pueden aparecer pocas veces o ninguna. Por ello, cuando el tamaño del dataset es reducido, la validación cruzada suele ser una alternativa más sistemática.

123.3.4. Validación cruzada K-fold

La validación cruzada K -fold divide el conjunto de datos en K subconjuntos llamados folds. El modelo se entrena K veces. En cada iteración, un fold se usa como prueba y los $K - 1$ folds restantes se usan como entrenamiento (Stone 1974; Kohavi 1995).

i Nota

Definición. En validación cruzada K -fold, una métrica M se estima como el promedio de las métricas obtenidas en los K folds:

$$\widehat{M}_{CV} = \frac{1}{K} \sum_{k=1}^K M_k$$

Por ejemplo, con $K = 5$, cada fold puede contener 20 % de los datos. En la primera iteración se prueba con el fold 1 y se entrena con los folds 2, 3, 4 y 5. En la segunda iteración se prueba con el fold 2 y se entrena con los demás, y así sucesivamente.

Si las exactitudes obtenidas son 82 %, 80 %, 84 %, 81 % y 83 %, entonces:

$$\widehat{M}_{CV} = \frac{82 + 80 + 84 + 81 + 83}{5} = 82 \%.$$

La validación cruzada usa mejor los datos disponibles porque cada observación se evalúa exactamente una vez y se usa para entrenamiento en $K - 1$ iteraciones. Su principal coste es computacional: el modelo debe entrenarse K veces. Por esta razón, $K = 5$ o $K = 10$ son elecciones frecuentes en la práctica (Kohavi 1995).

123.4. Ejemplo en Python: partición holdout y validación cruzada

El siguiente ejemplo usa `scikit-learn` para entrenar una regresión logística en un problema binario. Se muestra primero una partición holdout estratificada y después validación cruzada K -fold.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_informative=5,
    n_redundant=2,
    weights=[0.85, 0.15],
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```

    test_size=0.20,
    stratify=y,
    random_state=42
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

metricas_holdout = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred),
    "recall": recall_score(y_test, y_pred),
    "f1": f1_score(y_test, y_pred),
}

metricas_holdout

{'accuracy': 0.87,
 'precision': 0.7777777777777778,
 'recall': 0.22580645161290322,
 'f1': 0.35}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
    model,
    X,
    y,
    cv=cv,
    scoring=["accuracy", "precision", "recall", "f1"],
    return_train_score=False
)

{metrica: scores[f"test_{metrica}"].mean()
 for metrica in ["accuracy", "precision", "recall", "f1"]}

{'accuracy': np.float64(0.859),
 'precision': np.float64(0.5909090909090909),
 'recall': np.float64(0.29290322580645156),
 'f1': np.float64(0.3833043294837315)}

```

Este ejemplo también ilustra por qué no basta con observar solo accuracy. En un dataset desbalanceado, un clasificador puede obtener alta exactitud simplemente favoreciendo la clase mayoritaria, pero tener bajo recall sobre la clase positiva.

123.5. Matriz de confusión

La matriz de confusión resume la relación entre las etiquetas reales y las predicciones del modelo. En clasificación binaria se compone de cuatro cantidades: verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

i Nota

Definición. Para una clase positiva 1 y una clase negativa 0, la matriz de confusión se define por:

$$\begin{array}{c|cc} & \hat{y} = 0 & \hat{y} = 1 \\ \hline y = 0 & TN & FP \\ y = 1 & FN & TP \end{array}$$

Las cantidades se interpretan así:

Cantidad	Nombre	Interpretación
TP	Verdadero positivo	El caso era positivo y el modelo predijo positivo
TN	Verdadero negativo	El caso era negativo y el modelo predijo negativo
FP	Falso positivo	El caso era negativo, pero el modelo predijo positivo
FN	Falso negativo	El caso era positivo, pero el modelo predijo negativo

En el ejemplo de detección de spam, un falso positivo ocurre cuando un correo legítimo se clasifica como spam. Un falso negativo ocurre cuando un correo spam se clasifica como no spam.

123.6. Métricas básicas de clasificación

A partir de la matriz de confusión se definen las principales métricas de evaluación. Cada una responde una pregunta distinta, por lo que ninguna métrica es universalmente superior.

123.6.1. Exactitud y tasa de error

i Nota

Definición. La exactitud o accuracy es la proporción total de predicciones correctas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

La accuracy responde: ¿qué porcentaje total de observaciones clasificó correctamente el modelo? Es útil cuando las clases están relativamente balanceadas y cuando los costes de los errores son similares.

i Nota

Definición. La tasa de error es la proporción total de predicciones incorrectas:

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - \text{Accuracy}$$

La tasa de error responde: ¿qué porcentaje de decisiones fueron incorrectas?

123.6.2. Precisión**i** Nota

Definición. La precisión mide qué proporción de las predicciones positivas fueron correctas:

$$\text{Precision} = \frac{TP}{TP + FP}$$

La precisión responde: cuando el modelo predice positivo, ¿qué tan seguido tiene razón? Es una métrica importante cuando los falsos positivos son costosos. Por ejemplo, en un sistema que marca transacciones como fraudulentas, una baja precisión puede generar demasiadas falsas alarmas.

123.6.3. Recall, sensibilidad y tasa de verdaderos positivos**i** Nota

Definición. El recall o sensibilidad mide qué proporción de positivos reales fueron detectados:

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN}$$

El recall responde: de todos los casos positivos reales, ¿cuántos encontró el modelo? Es fundamental cuando los falsos negativos son costosos. En screening médico, por ejemplo, suele priorizarse una alta sensibilidad para reducir el riesgo de dejar pasar casos reales.

123.6.4. Especificidad y tasa de falsos positivos**i** Nota

Definición. La especificidad mide qué proporción de negativos reales fueron identificados correctamente:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

La especificidad responde: de todos los casos negativos reales, ¿cuántos fueron clasificados correctamente como negativos? Es importante cuando se desea evitar falsas alarmas.

i Nota

Definición. La tasa de falsos positivos mide qué proporción de negativos reales fueron clasificados erróneamente como positivos:

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

La tasa de falsos positivos es central para construir la curva ROC.

123.6.5. F1-score**i** Nota

Definición. El F1-score es la media armónica entre precisión y recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

El F1-score es útil cuando se desea balancear precisión y recall en una sola medida. Al ser una media armónica, penaliza valores muy bajos de cualquiera de las dos métricas. Por ello resulta especialmente informativo en problemas con clases desbalanceadas (Sokolova y Lapalme 2009).

123.6.6. Exactitud balanceada**i** Nota

Definición. La exactitud balanceada es el promedio entre sensibilidad y especificidad:

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}$$

Esta métrica es útil cuando las clases están desbalanceadas, porque no permite que el desempeño sobre la clase mayoritaria domine completamente la evaluación.

123.7. Ejemplo en Python: matriz de confusión y métricas

```
from sklearn.metrics import confusion_matrix, classification_report

cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[167,  2],
       [ 24,  7]])
```



```
print(classification_report(y_test, y_pred, target_names=["Clase 0", "Clase 1"]))
```

	precision	recall	f1-score	support
Clase 0	0.87	0.99	0.93	169
Clase 1	0.78	0.23	0.35	31
accuracy			0.87	200
macro avg	0.83	0.61	0.64	200
weighted avg	0.86	0.87	0.84	200

También podemos calcular explícitamente las métricas a partir de la matriz de confusión.

```
tn, fp, fn, tp = cm.ravel()

accuracy = (tp + tn) / (tp + tn + fp + fn)
error_rate = (fp + fn) / (tp + tn + fp + fn)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
specificity = tn / (tn + fp)
f1 = 2 * precision * recall / (precision + recall)
balanced_accuracy = (recall + specificity) / 2

{
    "accuracy": accuracy,
    "error_rate": error_rate,
    "precision": precision,
    "recall": recall,
    "specificity": specificity,
    "f1": f1,
    "balanced_accuracy": balanced_accuracy,
}
```

```
{'accuracy': np.float64(0.87),
 'error_rate': np.float64(0.13),
 'precision': np.float64(0.7777777777777778),
 'recall': np.float64(0.22580645161290322),
 'specificity': np.float64(0.9881656804733728),
 'f1': np.float64(0.35000000000000003),
 'balanced_accuracy': np.float64(0.606986066043138)}
```

123.8. Selección de métricas según la necesidad del problema

La métrica adecuada depende del contexto. Antes de evaluar un modelo conviene preguntar qué tipo de error es más costoso.

Métrica	Necesidad típica	Pregunta de decisión
Accuracy	La mayoría de decisiones deben ser correctas	¿Todos los errores tienen coste similar?
Error rate	Se desea cuantificar el porcentaje de fallos	¿Qué porcentaje de errores es aceptable?
Precision	Se desea que las predicciones positivas sean confiables	Cuando el modelo predice positivo, ¿qué tan seguido acierta?
Recall / Sensitivity	Se desea detectar tantos positivos reales como sea posible	¿Cuántos positivos reales deja pasar el modelo?
Specificity	Se desea evitar falsas alarmas	¿Cuántos negativos reales identifica correctamente?
F1-score	Se desea balancear precisión y recall	¿Hay compromiso entre falsos positivos y falsos negativos?

Un ejemplo clásico es la detección de cáncer en pruebas de screening. Supongamos dos modelos:

Modelo	TPR	FPR
A	0.92	0.35
B	0.81	0.08

El modelo A detecta más casos positivos reales, pero también genera más falsos positivos. En screening médico, la alta sensibilidad puede ser preferible porque es muy importante no dejar pasar casos reales. Sin embargo, si los falsos positivos generan pruebas invasivas, ansiedad o costes elevados, el modelo B podría ser más equilibrado. Esta decisión no puede tomarse solo con una métrica; requiere entender el coste de cada error y el uso final del sistema (Fawcett 2006).

123.9. Umbral de decisión

Muchos clasificadores producen puntuaciones continuas o probabilidades estimadas. La clase final depende del umbral τ . Cambiar el umbral modifica la matriz de confusión y, por tanto, todas las métricas.

i Nota

Definición. Dada una puntuación $\hat{p}(\mathbf{x})$, una regla de clasificación binaria con umbral τ se define como:

$$\hat{y}(\mathbf{x}; \tau) = \mathbb{I}\{\hat{p}(\mathbf{x}) \geq \tau\}$$

Si se baja el umbral de decisión de 0.70 a 0.30, más observaciones serán clasificadas como positivas. En general, esto aumenta el TPR porque se detectan más positivos reales, pero también aumenta el FPR porque más negativos son clasificados erróneamente como positivos.

```
from sklearn.metrics import confusion_matrix

probs = model.predict_proba(X_test)[: , 1]
```

```
for tau in [0.70, 0.50, 0.30]:
    y_pred_tau = (probs >= tau).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred_tau).ravel()
    tpr = tp / (tp + fn)
    fpr = fp / (fp + tn)
    print(f"umbral={tau:.2f} | TPR={tpr:.3f} | FPR={fpr:.3f}")
```

```
umbral=0.70 | TPR=0.129 | FPR=0.000
```

```
umbral=0.50 | TPR=0.226 | FPR=0.012
```

```
umbral=0.30 | TPR=0.613 | FPR=0.041
```

123.10. Curva ROC y AUC

La curva ROC permite analizar el rendimiento del clasificador para todos los posibles umbrales de decisión. En el eje vertical se grafica la tasa de verdaderos positivos, y en el eje horizontal la tasa de falsos positivos (Fawcett 2006).

i Nota

Definición. La curva ROC es el conjunto de puntos:

$$\text{ROC} = \{(\text{FPR}(\tau), \text{TPR}(\tau)) : \tau \in \mathbb{R}\}$$

Un clasificador ideal se acerca a la esquina superior izquierda, donde $\text{TPR} = 1$ y $\text{FPR} = 0$. La línea diagonal representa un clasificador sin valor predictivo. Un modelo cuya curva está cerca de esa diagonal no discrimina bien entre positivos y negativos.

El AUC resume el área bajo la curva ROC. Puede interpretarse como la probabilidad de que el modelo asigne una puntuación mayor a una observación positiva elegida al azar que a una observación negativa elegida al azar (Fawcett 2006).

i Nota

Definición. El área bajo la curva ROC se define como:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

Una interpretación usual de AUC es:

AUC	Interpretación general
0.90–1.00	Sobresaliente
0.80–0.90	Bueno o excelente
0.70–0.80	Aceptable
0.60–0.70	Bajo
0.50–0.60	Sin buena discriminación

Estas categorías deben usarse con cuidado. Un AUC alto no garantiza que el modelo sea adecuado para todos los umbrales ni para todos los costes de error. Además, en problemas fuertemente desbalanceados, la curva Precision–Recall puede ser más informativa que la curva ROC (Saito y Rehmsmeier 2015).

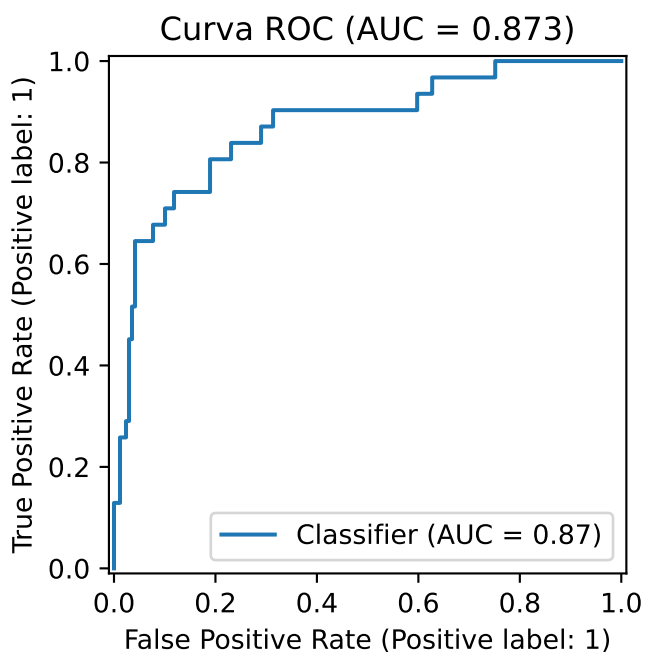
Si un clasificador tiene $AUC = 0.50$, significa que no discrimina mejor que el azar. En un ejemplo médico, si se compara al azar un paciente con cáncer y uno sin cáncer, el modelo solo tendría 50 % de probabilidad de asignar una puntuación mayor al paciente con cáncer.

123.11. Ejemplo en Python: curva ROC y AUC

```
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay, roc_auc_score

auc = roc_auc_score(y_test, probs)

RocCurveDisplay.from_predictions(y_test, probs)
plt.title(f"Curva ROC (AUC = {auc:.3f})")
plt.show()
```



La curva ROC no fija un único umbral. En su lugar, muestra el comportamiento del modelo al variar el umbral. Esto permite elegir un punto de operación de acuerdo con las necesidades del problema.

123.12. Comparación con la curva Precision–Recall

Aunque la presentación se centra en ROC y AUC, es importante mencionar una limitación: cuando la clase positiva es rara, la curva ROC puede presentar una visión demasiado optimista del desempeño. En estos

casos, la curva Precision–Recall suele ser más sensible a los cambios en la detección de la clase minoritaria (Saito y Rehmsmeier 2015).

i Nota

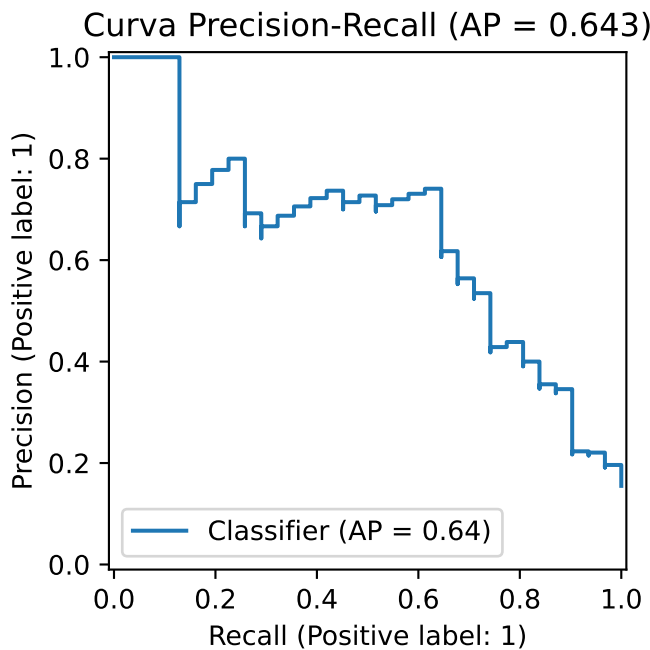
Definición. La curva Precision–Recall representa el conjunto de puntos:

$$PR = \{(\text{Recall}(\tau), \text{Precision}(\tau)) : \tau \in \mathbb{R}\}$$

```
from sklearn.metrics import PrecisionRecallDisplay, average_precision_score

ap = average_precision_score(y_test, probs)

PrecisionRecallDisplay.from_predictions(y_test, probs)
plt.title(f"Curva Precision-Recall (AP = {ap:.3f})")
plt.show()
```



La métrica Average Precision resume el rendimiento bajo la curva Precision–Recall. A diferencia de ROC-AUC, su línea base depende de la prevalencia de la clase positiva. Si solo 15 % de las observaciones son positivas, una curva PR debe compararse contra esa prevalencia como referencia.

123.13. Flujo recomendado para evaluar clasificadores

Una evaluación reproducible puede organizarse como un flujo de trabajo:

1. Definir la clase positiva y el objetivo del modelo.
2. Separar datos de prueba antes de ajustar hiperparámetros.
3. Usar partición estratificada cuando haya desbalance.
4. Entrenar modelos candidatos en entrenamiento.

5. Seleccionar hiperparámetros con validación o validación cruzada.
6. Evaluar una sola vez el modelo final sobre prueba.
7. Reportar matriz de confusión y métricas alineadas con el problema.
8. Analizar umbrales si el modelo produce probabilidades o puntuaciones.
9. Documentar semillas, particiones, versiones de librerías y criterios de selección.

En proyectos reales, esta documentación es tan importante como el valor de las métricas. Sin ella, los resultados son difíciles de reproducir y comparar.

123.14. Ejemplo final: evaluación completa en Python

```
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
)

def evaluar_clasificador(y_true, y_score, threshold=0.5):
    y_pred = (y_score >= threshold).astype(int)

    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    return {
        "threshold": threshold,
        "TP": tp,
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "accuracy": accuracy_score(y_true, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred),
        "specificity": tn / (tn + fp),
        "f1": f1_score(y_true, y_pred),
        "auc": roc_auc_score(y_true, y_score),
    }

for tau in [0.3, 0.5, 0.7]:
    print(evaluar_clasificador(y_test, probs, threshold=tau))
```

```
{'threshold': 0.3, 'TP': np.int64(19), 'TN': np.int64(162), 'FP': np.int64(7), 'FN': np.int64(12)}
{'threshold': 0.5, 'TP': np.int64(7), 'TN': np.int64(167), 'FP': np.int64(2), 'FN': np.int64(24)}
{'threshold': 0.7, 'TP': np.int64(4), 'TN': np.int64(169), 'FP': np.int64(0), 'FN': np.int64(27),
```

Este patrón permite comparar explícitamente cómo cambian las métricas al modificar el umbral. También obliga a distinguir entre evaluación basada en clases predichas y evaluación basada en puntuaciones.

123.15. Conclusiones

Evaluar un modelo de clasificación requiere más que calcular accuracy. La primera decisión importante es cómo separar los datos para estimar la generalización. El holdout es simple y rápido, pero puede ser inestable si la partición no representa adecuadamente la distribución de los datos. La validación cruzada K -fold ofrece una estimación más robusta, especialmente en conjuntos pequeños, aunque incrementa el coste computacional.

La matriz de confusión proporciona la base para entender los errores del clasificador. A partir de ella se derivan métricas como precisión, recall, especificidad, F1-score y exactitud balanceada. Cada métrica responde una pregunta diferente y debe elegirse según el objetivo del problema. En escenarios donde los falsos negativos son graves, como screening médico, puede priorizarse la sensibilidad. En escenarios donde los falsos positivos son costosos, puede priorizarse la precisión o la especificidad.

Finalmente, la curva ROC y el AUC permiten analizar el desempeño del modelo a través de distintos umbrales. Sin embargo, el umbral final debe seleccionarse con base en el coste de los errores y el contexto de aplicación. Una evaluación rigurosa no solo reporta métricas, sino que documenta cómo se obtuvieron, con qué particiones, bajo qué supuestos y con qué criterios de decisión.

Parte XXV.

Árboles y Ensamblajes

124. Árboles CART: árboles de clasificación y regresión

124.1. Introducción

Los árboles de decisión constituyen una familia de modelos supervisados cuyo objetivo es aprender reglas de decisión a partir de datos etiquetados. En clasificación, estas reglas asignan una observación a una categoría; en regresión, estiman un valor numérico. Dentro de esta familia, el método CART (*Classification and Regression Trees*) ocupa un lugar central porque formaliza la construcción de árboles binarios capaces de tratar variables categóricas y numéricas, además de incorporar mecanismos de poda para controlar la complejidad del modelo (Breiman et al. 1984; Hastie et al. 2009).

A diferencia de algoritmos como ID3, que originalmente se formuló para árboles de clasificación con divisiones multi-rama y variables categóricas, CART construye árboles estrictamente binarios. En cada nodo, el algoritmo busca una pregunta de la forma “¿se cumple cierta condición?” y divide los datos en dos subconjuntos: los que satisfacen la condición y los que no. La idea fundamental es que cada partición produzca nodos cada vez más homogéneos respecto a la variable objetivo.

En un problema de clasificación, la homogeneidad de un nodo se mide mediante una función de impureza. CART suele utilizar el índice de Gini, aunque también pueden emplearse otras medidas. En regresión, el criterio más común se basa en la reducción de la varianza o del error cuadrático medio. Este capítulo se enfoca principalmente en CART para clasificación, siguiendo el desarrollo conceptual de árboles binarios, índice de Gini, particiones para variables categóricas y continuas, reducción de impureza, poda y complejidad computacional.

124.2. De los árboles de decisión a CART

Un árbol de decisión representa un conjunto de reglas jerárquicas. Cada nodo interno contiene una condición sobre una variable predictora, cada rama representa el resultado de dicha condición y cada hoja contiene una predicción. Para clasificar una nueva observación, se recorre el árbol desde la raíz hasta una hoja aplicando sucesivamente las reglas aprendidas.

CART fue propuesto por Breiman, Friedman, Olshen y Stone en 1984 como un marco general para construir árboles de clasificación y regresión (Breiman et al. 1984). Sus rasgos principales son los siguientes:

- construye árboles binarios;
- evalúa particiones candidatas mediante criterios de impureza;
- puede trabajar con variables numéricas y categóricas;
- selecciona localmente la mejor partición en cada nodo;
- utiliza poda de costo-complejidad para evitar árboles excesivamente grandes;
- produce modelos interpretables mediante reglas SI-ENTONCES.

i Nota

Definición. . Un árbol CART es un árbol de decisión binario en el que cada nodo interno t particiona el subconjunto de datos D_t en dos subconjuntos disjuntos D_L y D_R mediante una regla de decisión s :

$$D_t = D_L \cup D_R, \quad D_L \cap D_R = \emptyset.$$

La propiedad binaria es esencial: aunque una variable categórica tenga más de dos valores posibles, CART no genera una rama por categoría. En su lugar, agrupa los valores en dos bloques y evalúa qué agrupación produce la mayor mejora en pureza.

124.3. Clasificación, pureza e impureza

En clasificación supervisada se parte de un conjunto de entrenamiento

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n,$$

donde $\mathbf{x}_i \in \mathcal{X}$ representa las variables predictoras de la observación i y $y_i \in \{1, 2, \dots, K\}$ es su clase. El objetivo es aprender una función $f: \mathcal{X} \rightarrow \{1, 2, \dots, K\}$ que prediga la clase de nuevas observaciones.

Un nodo es puro si todas las observaciones que contiene pertenecen a la misma clase. En ese caso, la predicción de la hoja es inequívoca. En cambio, si dentro de un nodo conviven observaciones de distintas clases, el nodo es impuro y conviene seguir dividiéndolo si existen particiones útiles.

i Nota

Definición. . Sea D_t el conjunto de observaciones que llegan al nodo t . La proporción de observaciones de clase k en el nodo se define como:

$$p_{k,t} = \frac{1}{|D_t|} \sum_{(\mathbf{x}_i, y_i) \in D_t} \mathbf{1}(y_i = k).$$

Si $p_{k,t} = 1$ para alguna clase k , entonces el nodo es puro. Si las proporciones están más repartidas entre clases, el nodo presenta mayor impureza. Esta intuición se formaliza mediante el índice de Gini.

124.4. Índice de Gini

El índice de Gini mide la impureza de un nodo en términos de la distribución de clases. Para K clases, se define como:

i Nota

Definición. . El índice de Gini de un nodo D_t con proporciones de clase $p_{1,t}, \dots, p_{K,t}$ es:

$$Gini(D_t) = 1 - \sum_{k=1}^K p_{k,t}^2. \quad (124.1)$$

Esta expresión puede interpretarse como la probabilidad de clasificar incorrectamente una observación si se asignara aleatoriamente una clase de acuerdo con la distribución empírica del nodo (Breiman et al. 1984; James et al. 2021).

En clasificación binaria, con clases “sí” y “no”, si p es la proporción de la clase positiva, entonces

$$Gini(D_t) = 1 - p^2 - (1 - p)^2 = 2p(1 - p).$$

El mínimo valor es 0, alcanzado cuando $p = 0$ o $p = 1$, es decir, cuando el nodo contiene una sola clase. El máximo valor en el caso binario es 0.5, alcanzado cuando $p = 0.5$.

```
import numpy as np
import pandas as pd

def gini_from_counts(counts):
    counts = np.asarray(counts, dtype=float)
    probs = counts / counts.sum()
    return 1 - np.sum(probs**2)

# Ejemplo: 9 registros de clase sí y 5 de clase no
gini_raiz = gini_from_counts([9, 5])
print(f"Gini raíz = {gini_raiz:.3f}")
```

Gini raíz = 0.459

En este ejemplo, la impureza inicial es aproximadamente 0.459, lo cual indica que el nodo raíz todavía mezcla observaciones de ambas clases.

124.5. Evaluación de una partición binaria

En CART, una partición divide el conjunto D_t en dos subconjuntos hijos: D_L y D_R . La calidad de la partición no se mide solo por la pureza individual de cada subconjunto, sino por una suma ponderada de sus impurezas. Esto evita favorecer divisiones que produzcan un nodo pequeño y puro pero de escasa relevancia global.

i Nota

Definición. . La impureza ponderada de una partición binaria s que divide D_t en D_L y D_R es:

$$Gini_s(D_t) = \frac{|D_L|}{|D_t|} Gini(D_L) + \frac{|D_R|}{|D_t|} Gini(D_R). \quad (124.2)$$

CART selecciona, entre las particiones candidatas, aquella que minimiza la impureza ponderada. Equivalentemente, puede seleccionarse la partición que maximiza la reducción de impureza.

i Nota

Definición. . La reducción de impureza producida por una partición s en el nodo D_t es:

$$\Delta Gini(s, D_t) = Gini(D_t) - Gini_s(D_t). \quad (124.3)$$

Ambas formulaciones son equivalentes para comparar particiones dentro del mismo nodo: minimizar $Gini_s(D_t)$ es lo mismo que maximizar $\Delta Gini(s, D_t)$, ya que $Gini(D_t)$ permanece constante durante la comparación.

124.6. Particiones binarias con variables categóricas

Supongamos que una variable categórica A toma valores en un conjunto finito

$$\mathcal{A} = \{a_1, a_2, \dots, a_m\}.$$

En lugar de crear una rama para cada valor, CART construye preguntas binarias del tipo:

$$A \in S_A,$$

donde S_A es un subconjunto propio y no vacío de $\mathcal{A}$. Por ejemplo, si

$$ingreso \in \{bajo, medio, alto\},$$

entonces algunas preguntas candidatas son:

$$ingreso \in \{bajo\}, \quad ingreso \in \{medio\}, \quad ingreso \in \{bajo, medio\}.$$

Cada pregunta separa los datos en dos subconjuntos: los que cumplen la condición y los que no.

i Nota

Definición. . Si un atributo categórico A tiene m valores distintos, el número de particiones binarias no triviales distintas es:

$$\frac{2^m - 2}{2} = 2^{m-1} - 1. \quad (124.4)$$

Se restan dos subconjuntos porque el subconjunto vacío y el conjunto completo no producen una partición válida. Después se divide entre dos porque S_A y su complemento generan la misma partición, solo intercambiando los lados izquierdo y derecho.

```
from itertools import combinations
```

```
def binary_partitions(values):
    values = tuple(values)
    m = len(values)
```

```

partitions = []
# Basta explorar subconjuntos hasta tamaño m-1; luego eliminamos duplicados por complemento.
seen = set()
full = frozenset(values)
for r in range(1, m):
    for comb in combinations(values, r):
        left = frozenset(comb)
        right = full - left
        key = frozenset([left, right])
        if key not in seen:
            seen.add(key)
            partitions.append((set(left), set(right)))
return partitions

for left, right in binary_partitions(["bajo", "medio", "alto"]):
    print(left, "vs", right)

```

```

{'bajo'} vs {'alto', 'medio'}
{'medio'} vs {'alto', 'bajo'}
{'alto'} vs {'medio', 'bajo'}

```

124.7. Particiones binarias con variables continuas

Para variables numéricas, CART busca puntos de corte. Si A es una variable continua, una partición típica tiene la forma

$$A < c \quad \text{frente a} \quad A \geq c.$$

El procedimiento usual consiste en ordenar los valores observados de A , calcular puntos medios entre valores consecutivos y evaluar cada punto como candidato de corte. Esta estrategia permite convertir una variable continua en una pregunta binaria interpretable.

i Nota

Definición. . Dados los valores ordenados $a_{(1)} < a_{(2)} < \dots < a_{(r)}$ de una variable numérica A , los puntos de corte candidatos se definen como:

$$c_j = \frac{a_{(j)} + a_{(j+1)}}{2}, \quad j = 1, \dots, r-1. \quad (124.5)$$

Consideremos el ejemplo:

$$(20, \text{Sí}), \quad (25, \text{Sí}), \quad (30, \text{No}), \quad (40, \text{No}).$$

Los valores ordenados son 20, 25, 30, 40, por lo que los puntos medios candidatos son:

$$22.5, \quad 27.5, \quad 35.$$

Para cada corte c , se forman los subconjuntos

$$D_L = \{A < c\}, \quad D_R = \{A \geq c\},$$

y se calcula el índice de Gini ponderado.

```
import pandas as pd

edad = pd.DataFrame({
    "edad": [20, 25, 30, 40],
    "clase": ["Sí", "Sí", "No", "No"]
})

def gini(labels):
    counts = pd.Series(labels).value_counts().to_numpy()
    return gini_from_counts(counts)

def weighted_gini_for_cut(df, feature, target, c):
    left = df[df[feature] < c]
    right = df[df[feature] >= c]
    n = len(df)
    return (len(left)/n)*gini(left[target]) + (len(right)/n)*gini(right[target])

values = sorted(edad["edad"].unique())
cuts = [(values[i] + values[i+1]) / 2 for i in range(len(values)-1)]

resultados = pd.DataFrame({
    "corte": cuts,
    "gini_ponderado": [weighted_gini_for_cut(edad, "edad", "clase", c) for c in cuts]
})
resultados
```

	corte	gini_ponderado
0	22.5	0.333333
1	27.5	0.000000
2	35.0	0.333333

El mejor corte es $c = 27.5$, porque separa perfectamente las clases: a la izquierda quedan los dos casos “Sí” y a la derecha los dos casos “No”. Por tanto, el índice de Gini ponderado es 0.

124.8. Ejemplo completo: compra de computadora portátil

Consideremos un conjunto de datos en el que se desea predecir si una persona compra o no una computadora portátil. Las variables predictoras son edad, ingreso, si es estudiante y calificación crediticia. La variable objetivo es `compra_lap`.


```

datos = pd.DataFrame({
    "id": range(1, 15),
    "edad": ["joven", "joven", "adulto", "a_mayor", "a_mayor", "a_mayor", "adulto", "joven", "joven", "adulto", "adulto", "adulto", "adulto", "adulto"],
    "ingreso": ["alto", "alto", "alto", "medio", "bajo", "bajo", "bajo", "medio", "bajo", "bajo", "medio", "alto", "alto", "alto"],
    "estudiante": ["no", "no", "no", "no", "si", "si", "si", "no", "si", "si", "si", "no", "si", "si"],
    "calif_credito": ["favorable", "excelente", "favorable", "favorable", "favorable", "favorable", "excelente", "favorable", "favorable", "favorable", "favorable", "favorable", "favorable", "favorable"],
    "compra_lap": ["no", "no", "si", "si", "si", "no", "si", "no", "si", "si", "si", "si", "si", "si"],
})

datos

```

	id	edad	ingreso	estudiante	calif_credito	compra_lap
0	1	joven	alto	no	favorable	no
1	2	joven	alto	no	excelente	no
2	3	adulto	alto	no	favorable	si
3	4	a_mayor	medio	no	favorable	si
4	5	a_mayor	bajo	si	favorable	si
5	6	a_mayor	bajo	si	excelente	no
6	7	adulto	bajo	si	excelente	si
7	8	joven	medio	no	favorable	no
8	9	joven	bajo	si	favorable	si
9	10	a_mayor	medio	si	favorable	si
10	11	joven	medio	si	excelente	si
11	12	adulto	medio	no	excelente	si
12	13	adulto	alto	si	favorable	si
13	14	a_mayor	medio	no	excelente	no

La distribución de la variable objetivo contiene 9 casos “sí” y 5 casos “no”, por lo que la impureza del nodo raíz es:

```

gini_inicial = gini(datos["compra_lap"])
print(f"Gini inicial = {gini_inicial:.3f}")

```

Gini inicial = 0.459

Para evaluar una variable categórica, se consideran todas sus particiones binarias no equivalentes. La siguiente función calcula la mejor partición de una variable categórica usando el índice de Gini ponderado.

```

def weighted_gini_categorical(df, feature, target, left_values):
    left_values = set(left_values)
    left = df[df[feature].isin(left_values)]
    right = df[~df[feature].isin(left_values)]
    n = len(df)
    return (len(left)/n)*gini(left[target]) + (len(right)/n)*gini(right[target])

```

```
def best_categorical_split(df, feature, target):
    values = sorted(df[feature].unique())
    rows = []
    for left, right in binary_partitions(values):
        wg = weighted_gini_categorical(df, feature, target, left)
        rows.append({
            "atributo": feature,
            "particion_izquierda": tuple(sorted(left)),
            "particion_derecha": tuple(sorted(right)),
            "gini_ponderado": wg,
            "reduccion": gini(df[target]) - wg
        })
    return pd.DataFrame(rows).sort_values("gini_ponderado")

resumen = pd.concat([
    best_categorical_split(datos, col, "compra_lap").head(1)
    for col in ["edad", "ingreso", "estudiante", "calif_credito"]
], ignore_index=True)

resumen.sort_values("gini_ponderado")
```

	atributo	particion_izquierda	particion_derecha	gini_ponderado	reduccion
0	edad	(adulto,)	(a_mayor, joven)	0.357143	0.102041
2	estudiante	(no,)	(si,)	0.367347	0.091837
3	calif_credito	(excelente,)	(favorable,)	0.428571	0.030612
1	ingreso	(alto,)	(bajo, medio)	0.442857	0.016327

En este ejemplo, una de las mejores particiones iniciales corresponde a separar la categoría **adulto** del resto en la variable **edad**. Esta partición produce una reducción importante de impureza porque todos los registros con edad **adulto** tienen clase “sí”. Sin embargo, CART continúa evaluando particiones en los nodos hijos para construir reglas más específicas.

124.9. Reducción de impureza e interpretación

La reducción de impureza permite comparar cuánto mejora una partición respecto al nodo original. En el ejemplo anterior, si el nodo raíz tiene

$$Gini(D) = 0.459,$$

y una partición produce

$$Gini_s(D) = 0.357,$$

entonces la reducción es

$$\Delta Gini = 0.459 - 0.357 = 0.102.$$

Una reducción más grande indica que la partición separó mejor las clases. En la práctica, CART evalúa todas las particiones candidatas disponibles en el nodo y selecciona aquella con mayor reducción de impureza. Este procedimiento se repite recursivamente en cada nodo hijo hasta que se satisface algún criterio de parada.

124.10. Criterios de parada

Un árbol no puede crecer indefinidamente. En términos prácticos, la expansión se detiene cuando se cumple alguna de las siguientes condiciones:

- el nodo es puro;
- no quedan variables útiles para dividir;
- el número de observaciones en el nodo es menor que un umbral mínimo;
- la reducción de impureza es demasiado pequeña;
- se alcanza una profundidad máxima especificada;
- todos los posibles cortes producen particiones inválidas o equivalentes.

Estos criterios tienen un papel estadístico importante. Si el árbol crece demasiado, puede memorizar el conjunto de entrenamiento y perder capacidad de generalización. Si se detiene demasiado pronto, puede subajustar los datos y no capturar patrones relevantes.

124.11. Poda de costo-complejidad

Una característica fundamental de CART es la poda de costo-complejidad. La idea consiste en permitir primero que el árbol crezca y luego simplificarlo eliminando ramas que aportan poca mejora predictiva en relación con su complejidad (Breiman et al. 1984).

Para un árbol T , se define una función objetivo que combina el error empírico con una penalización por tamaño:

i Nota

Definición. . El criterio de costo-complejidad para un árbol T se define como:

$$R_{\alpha}(T) = R(T) + \alpha|T|, \quad (124.6)$$

Aquí, $R(T)$ representa el error del árbol, $|T|$ es el número de hojas y $\alpha \geq 0$ controla la penalización por complejidad. Cuando $\alpha = 0$, no se penaliza el tamaño del árbol. A medida que α aumenta, se prefieren árboles más pequeños.

La poda permite obtener modelos más simples, interpretables y menos propensos al sobreajuste. En bibliotecas como `scikit-learn`, este procedimiento se implementa mediante el parámetro `ccp_alpha` (Pedregosa et al. 2011).

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = datos[["edad", "ingreso", "estudiante", "calif_credito"]]
y = datos["compra_lap"]

preprocess = ColumnTransformer(
    transformers=[("cat", OneHotEncoder(handle_unknown="ignore"), X.columns)]
)

modelo = Pipeline(steps=[
    ("preprocess", preprocess),
    ("tree", DecisionTreeClassifier(criterion="gini", random_state=42))
])

modelo.fit(X, y)
pred = modelo.predict(X)
print(f"Exactitud en entrenamiento = {accuracy_score(y, pred):.3f}")

```

Exactitud en entrenamiento = 1.000

El ejemplo anterior usa codificación *one-hot* para representar variables categóricas antes de entrenar un árbol de decisión con `scikit-learn`. Aunque la implementación interna de CART puede tratar particiones binarias de variables categóricas de distintas maneras según el software, este flujo es común en aplicaciones prácticas de ciencia de datos con Python.

124.12. Visualización de un árbol entrenado

```

import matplotlib.pyplot as plt

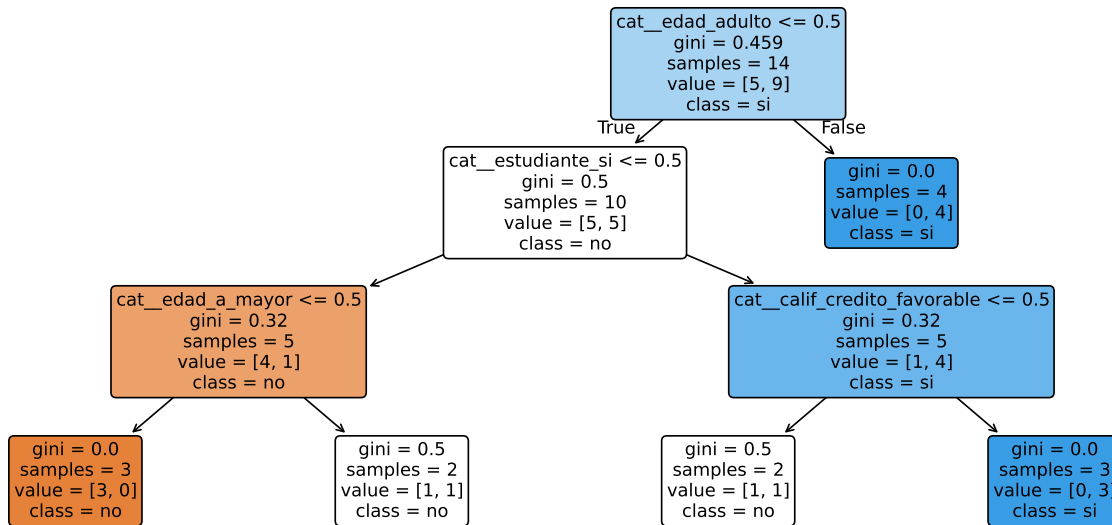
modelo_simple = Pipeline(steps=[
    ("preprocess", preprocess),
    ("tree", DecisionTreeClassifier(
        criterion="gini",
        max_depth=3,
        random_state=42
    ))
])

modelo_simple.fit(X, y)

feature_names = modelo_simple.named_steps["preprocess"].get_feature_names_out()

```

```
plt.figure(figsize=(14, 6))
plot_tree(
    modelo_simple.named_steps["tree"],
    feature_names=feature_names,
    class_names=modelo_simple.named_steps["tree"].classes_,
    filled=True,
    rounded=True
)
plt.show()
```



La visualización permite inspeccionar las reglas aprendidas, las impurezas de cada nodo y la distribución de clases en las hojas. Esta interpretabilidad es una de las principales ventajas de los árboles de decisión frente a otros modelos más opacos.

124.13. Extracción de reglas de clasificación

Un árbol puede traducirse a reglas SI-ENTONCES. Cada regla corresponde a un camino desde la raíz hasta una hoja. Por ejemplo, si un árbol decide primero con base en la edad y luego con base en si la persona es estudiante, una regla posible sería:

SI edad $\neq$ adulto Y estudiante = sí ENTONCES compra_lap = sí.

Esta forma de representación facilita comunicar el modelo a audiencias no técnicas. Además, permite auditar si las reglas son razonables, si dependen de variables sensibles o si reflejan sesgos presentes en los datos.

```
from sklearn.tree import export_text

arbol = modelo_simple.named_steps["tree"]
reglas = export_text(arbol, feature_names=list(feature_names))
print(reglas)
```

```

|--- cat__edad_adulto <= 0.50
|   |--- cat__estudiante_si <= 0.50
|   |   |--- cat__edad_a_mayor <= 0.50
|   |   |   |--- class: no
|   |   |   |--- cat__edad_a_mayor > 0.50
|   |   |   |--- class: no
|   |   |--- cat__estudiante_si > 0.50
|   |   |   |--- cat__calif_credito_favorable <= 0.50
|   |   |   |--- class: no
|   |   |   |--- cat__calif_credito_favorable > 0.50
|   |   |   |--- class: si
|--- cat__edad_adulto > 0.50
|   |--- class: si

```

124.14. CART para regresión

Aunque este capítulo se ha concentrado en clasificación, CART también se utiliza para regresión. En ese caso, las hojas no predicen clases sino valores numéricos. La predicción típica de una hoja es el promedio de los valores de respuesta de las observaciones que llegan a ella.

Para elegir particiones en regresión, se busca reducir la variabilidad dentro de los nodos hijos. Un criterio común es minimizar la suma de errores cuadráticos dentro de los nodos:

i Nota

Definición. . En un árbol de regresión, si una partición s divide D_t en D_L y D_R , el criterio de error cuadrático puede escribirse como:

$$RSS(s) = \sum_{i \in D_L} (y_i - \bar{y}_L)^2 + \sum_{i \in D_R} (y_i - \bar{y}_R)^2. \quad (124.7)$$

Así, CART unifica problemas de clasificación y regresión bajo una misma lógica: dividir recursivamente el espacio de variables para formar regiones internamente homogéneas.

124.15. Complejidad computacional

La complejidad de construir un árbol CART depende del número de observaciones n , el número de variables m , el tipo de variables y la profundidad final del árbol. Para variables numéricas, evaluar cortes requiere ordenar valores o mantener estructuras ordenadas. De manera simplificada, si se ordenan los valores de cada atributo, el costo inicial puede aproximarse por

$$O(mn \log n).$$

Después, en cada nodo, se evalúan cortes candidatos para seleccionar la mejor partición. En implementaciones eficientes, el costo total depende de cómo se reutilicen los ordenamientos y de las restricciones impuestas al crecimiento del árbol (Hastie et al. 2009; Pedregosa et al. 2011).

Una forma práctica de interpretar la complejidad es la siguiente:

- n controla cuántas observaciones deben repartirse y evaluarse;
- m controla cuántos atributos se prueban en cada nodo;
- la profundidad controla cuántas veces se repite el proceso de partición;
- las variables categóricas con muchos niveles pueden ser costosas porque el número de particiones binarias crece como $2^{m-1} - 1$ respecto al número de categorías del atributo.

Por esta razón, en aplicaciones reales se imponen restricciones como `max_depth`, `min_samples_split`, `min_samples_leaf` o poda por `ccp_alpha`.

124.16. Ventajas y limitaciones

CART tiene varias ventajas:

- produce modelos interpretables;
- requiere pocas transformaciones de escala en variables numéricas;
- captura interacciones no lineales entre atributos;
- permite clasificación y regresión;
- puede combinarse con métodos de ensamble como Random Forests y Gradient Boosting (Breiman 2001; Friedman 2001).

Sin embargo, también tiene limitaciones importantes:

- puede sobreajustarse si no se controla su profundidad;
- pequeñas variaciones en los datos pueden producir árboles distintos;
- las decisiones son locales y greedy;
- un árbol individual suele tener menor desempeño predictivo que ensambles de árboles;
- las variables categóricas con muchos niveles pueden inducir particiones complejas o inestables.

Estas limitaciones explican por qué CART suele utilizarse como modelo base dentro de métodos de ensamble. Aun así, su valor pedagógico e interpretativo sigue siendo fundamental.

124.17. Conclusiones

CART es uno de los métodos clásicos más importantes para construir árboles de decisión. Su principio central consiste en dividir recursivamente los datos mediante particiones binarias que reduzcan la impureza de los nodos. En clasificación, el índice de Gini proporciona una medida sencilla y efectiva para evaluar la mezcla de clases; en regresión, se emplean criterios basados en la reducción del error cuadrático.

El enfoque CART combina interpretabilidad, flexibilidad y una formulación matemática clara. La selección de particiones mediante impureza ponderada, la comparación mediante reducción de impureza y la poda de costo-complejidad permiten construir árboles útiles sin perder de vista el riesgo de sobreajuste. Por ello, CART no solo es un modelo por derecho propio, sino también la base conceptual de técnicas modernas de ensamble ampliamente usadas en ciencia de datos.

125. Árboles de regresión lineal, ensambles y vecinos cercanos

Los árboles de decisión constituyen una familia de métodos de aprendizaje supervisado basada en reglas. A diferencia de un modelo lineal global, que intenta describir todo el conjunto de datos mediante una sola ecuación, un árbol divide el espacio de variables en regiones más pequeñas y ajusta una predicción simple en cada una. Esta idea es especialmente útil cuando la relación entre las variables explicativas y la variable respuesta no es lineal, cambia por zonas o involucra interacciones difíciles de especificar manualmente (Breiman et al. 1984; Hastie et al. 2009).

En problemas de regresión, el objetivo es predecir una variable numérica. Por ejemplo, podríamos estimar el precio de una laptop a partir de su memoria RAM, el tamaño de pantalla, el procesador o la marca; o bien estimar el precio de una vivienda con base en su área, ubicación y antigüedad. Un árbol de regresión construye reglas del tipo “si una variable cae por debajo de cierto umbral, sigue una rama; en caso contrario, sigue otra”. Al final de cada rama se encuentra una hoja, donde la predicción suele ser el promedio de las observaciones de entrenamiento que llegaron a esa región.

Este capítulo desarrolla la lógica de los árboles de regresión, su interpretación como funciones por partes, el criterio de error usado para elegir divisiones, la poda para controlar el sobreajuste y las métricas básicas de evaluación. Posteriormente se introducen métodos de ensamble basados en árboles, en particular bagging, random forests y boosting. Finalmente se presenta el algoritmo de vecinos más cercanos como contraste con los árboles: un método supervisado basado en distancia y aprendizaje perezoso.

125.1. De la regresión lineal a modelos por regiones

La regresión lineal múltiple modela una respuesta continua y como combinación lineal de variables explicativas $x_1, \dots, x_p$:

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i.$$

Su principal ventaja es la interpretabilidad de los coeficientes: cada β_j mide el cambio esperado en y ante un incremento unitario de x_j , manteniendo constantes las demás variables. Sin embargo, esta estructura impone una forma funcional global. Cuando la relación entre X e y cambia en distintas regiones del espacio, una sola ecuación puede resultar demasiado rígida.

Los árboles de regresión responden a esta dificultad mediante particionamiento recursivo. En lugar de buscar una única superficie global, dividen el espacio de predictores en regiones y asignan una predicción constante a cada región. Esto produce un modelo no paramétrico, flexible e interpretable (James et al. 2021).

i Nota

Definición. Un árbol de regresión es un modelo que particiona el espacio de predictores en regiones disjuntas $R_1, \dots, R_J$ y predice un valor constante c_h dentro de cada región:

$$\hat{f}(x) = \sum_{h=1}^J c_h \mathbb{I}(x \in R_h). \quad (125.1)$$

En esta expresión, x representa una nueva observación; R_h es una región del espacio de variables; c_h es el valor asignado a esa región, usualmente el promedio de las respuestas observadas dentro de ella; y $\mathbb{I}(x \in R_h)$ es una función indicadora que vale uno si x pertenece a la región R_h y cero en caso contrario.

i Nota

Definición. La función indicadora de una región R_h se define como:

$$\mathbb{I}(x \in R_h) = \begin{cases} 1, & \text{si } x \in R_h, \\ 0, & \text{si } x \notin R_h. \end{cases} \quad (125.2)$$

125.2. Clasificadores no lineales y modelos por partes

En problemas de clasificación, un clasificador lineal separa clases mediante hiperplanos. Si los datos son linealmente separables, esta estrategia puede ser suficiente. Sin embargo, en muchos problemas reales las fronteras entre clases son curvas, fragmentadas o dependen de interacciones entre variables. En estos casos existen dos alternativas: mantener un clasificador lineal y aceptar ciertos errores, o usar clasificadores no lineales capaces de aproximar fronteras más flexibles.

Una forma pedagógica de construir no linealidad es usar piezas lineales. La idea es dividir una clase en subclases o regiones locales y asignar a cada subregión un discriminante lineal. La frontera global resultante deja de ser un solo hiperplano y se convierte en una unión de segmentos o caras lineales. Esta lógica anticipa dos ideas que aparecen más adelante: los árboles, que dividen el espacio mediante cortes recursivos, y los vecinos cercanos, que llevan la idea local al extremo de tratar cada muestra como una referencia de decisión (Duda et al. 2001; Hastie et al. 2009).

i Nota

Definición. Un clasificador lineal por partes asigna a cada clase ω_i un conjunto de subclases $\omega_i^1, \dots, \omega_i^{\ell_i}$, cada una con un discriminante lineal $g_i^\ell(x)$, y define el discriminante de la clase como el máximo local:

$$g_i(x) = \max_{\ell=1, \dots, \ell_i} g_i^\ell(x), \quad \hat{y}(x) = \arg \max_{i=1, \dots, c} g_i(x). \quad (125.3)$$

Si se escribe $g_i^\ell(x) = w_{i\ell}^\top x + w_{i\ell,0}$, cada subclase aporta una frontera lineal. El operador máximo selecciona la subregión de la clase que mejor explica el punto x . Cuando el número de subclases aumenta, la frontera puede aproximar formas no lineales cada vez más complejas. No obstante, esta flexibilidad tiene un costo: se deben decidir cuántas subclases usar, cómo inicializarlas y cómo evitar que el modelo memorice ruido.

Desde esta perspectiva, los árboles de decisión construyen una aproximación por partes, pero con piezas constantes en regresión y fronteras alineadas con los ejes. KNN, por su parte, puede verse como un caso

extremadamente local: en lugar de aprender subclases explícitas, conserva los ejemplos de entrenamiento y decide usando distancias.

125.3. Funciones por escalones

Un árbol de regresión produce una función por escalones. Para una sola variable x , un modelo simple podría definirse como:

$$\hat{f}(x) = \begin{cases} 100, & x < 10, \\ 200, & 10 \leq x < 20, \\ 300, & x \geq 20. \end{cases}$$

Este modelo no intenta ajustar una recta. En su lugar, divide el eje de x en intervalos y asigna una predicción constante a cada intervalo. En dos variables, las regiones suelen ser rectángulos; en p variables, se interpretan como hiperrectángulos. Por ejemplo, con edad, salario y experiencia, una regla podría ser:

$$\text{edad} < 30 \quad \text{y} \quad \text{salario} > 5000.$$

Este tipo de reglas es fácil de leer, lo que explica la popularidad de los árboles en contextos donde la interpretación es importante. No obstante, también implica una limitación geométrica: las fronteras de decisión son paralelas a los ejes, por lo que algunas relaciones oblicuas o altamente correlacionadas pueden necesitar muchos cortes para aproximarse bien (Hastie et al. 2009).

125.4. Predicción dentro de una región

Una vez que el árbol define una región R_h , la predicción óptima bajo pérdida cuadrática es el promedio de los valores reales dentro de esa región. Supongamos que en una región hay observaciones con respuestas $y_1, \dots, y_{n_h}$. La predicción constante c_h se elige minimizando la suma de errores cuadrados:

$$\sum_{i: x_i \in R_h} (y_i - c_h)^2.$$

La solución es el promedio muestral.

i Nota

Definición. La predicción de una hoja R_h en un árbol de regresión, bajo error cuadrático, es el promedio de las respuestas que caen en esa hoja:

$$\hat{c}_h = \frac{1}{|R_h|} \sum_{i: x_i \in R_h} y_i. \quad (125.4)$$

Por ejemplo, si una región contiene los valores reales 20, 40, 100, 120, su predicción antes de dividir es:

$$\hat{c} = \frac{20 + 40 + 100 + 120}{4} = 70.$$

Si esa región se divide en dos grupos, $R_1 = \{20, 40\}$ y $R_2 = \{100, 120\}$, las nuevas predicciones son:

$$\hat{c}_1 = 30, \quad \hat{c}_2 = 110.$$

La división es útil porque cada grupo queda descrito por un promedio más representativo de sus valores.

125.5. Error total y criterio de división

El árbol decide sus cortes evaluando cuánto disminuye el error al dividir una región. Para regresión, el criterio más común es la suma de errores cuadrados dentro de las hojas, también conocida como residual sum of squares (RSS) (Breiman et al. 1984; James et al. 2021).

i Nota

Definición. El error cuadrático total de un árbol con regiones $R_1, \dots, R_J$ es:

$$D(T) = \sum_{h=1}^J \sum_{i: x_i \in R_h} (y_i - \hat{c}_h)^2. \quad (125.5)$$

Cuando se propone dividir una región R en dos subregiones R_L y R_R , el árbol compara el error antes y después de la división. El error después de dividir es:

$$D_{\text{split}} = \sum_{i: x_i \in R_L} (y_i - \hat{c}_L)^2 + \sum_{i: x_i \in R_R} (y_i - \hat{c}_R)^2.$$

i Nota

Definición. La ganancia de una división se define como la reducción de error producida por el corte:

$$\text{Gain} = D_{\text{antes}} - D_{\text{después}}. \quad (125.6)$$

Una división es buena si reduce mucho el error. Si la reducción es pequeña, el corte puede no justificar el aumento de complejidad. Durante el entrenamiento, el algoritmo prueba muchas divisiones candidatas: por ejemplo, cortes en edad, salario o experiencia; y dentro de cada variable, varios umbrales posibles. Se elige la división que produce la mayor reducción de error.

125.6. Particionamiento recursivo

El entrenamiento de un árbol se realiza de forma codiciosa. Primero se busca el mejor corte para todo el conjunto de entrenamiento. Luego, dentro de cada región resultante, se vuelve a buscar el mejor corte local. Este procedimiento se repite hasta que se cumple un criterio de parada: profundidad máxima, número mínimo de observaciones por hoja, reducción mínima de error o tamaño mínimo de nodo.

El algoritmo básico puede resumirse así:

1. Comenzar con todos los datos en una sola región.
2. Para cada variable y cada punto de corte candidato, calcular el error después de dividir.

3. Elegir la división que más reduzca el error.
4. Repetir el proceso dentro de cada subregión.
5. Detener el crecimiento del árbol según reglas de control de complejidad.

El enfoque es eficiente y produce reglas interpretables, pero no garantiza encontrar el árbol globalmente óptimo. La búsqueda exacta del mejor árbol es combinatoriamente costosa, por lo que los algoritmos prácticos usan heurísticas codiciosas (Breiman et al. 1984).

125.7. Ejemplo manual: precio de laptops según RAM

Consideremos un conjunto de datos pequeño donde se desea predecir el precio de una laptop a partir de la memoria RAM.

Laptop	RAM	Precio real
A	4 GB	6500
B	4 GB	7200
C	8 GB	10500
D	8 GB	12000
E	16 GB	21000
F	16 GB	24000

Si no se divide el árbol, la predicción para todas las laptops es el promedio global:

$$\bar{y} = \frac{6500 + 7200 + 10500 + 12000 + 21000 + 24000}{6} = 13533.33.$$

El error sin dividir es:

$$D_{\text{sin dividir}} = \sum_i (y_i - \bar{y})^2 = 266,433,333.33.$$

Ahora se propone el corte:

$$\text{RAM} < 12.$$

Esto genera dos regiones:

$$R_1 = \{6500, 7200, 10500, 12000\}, \quad R_2 = \{21000, 24000\}.$$

Las predicciones de cada hoja son:

$$\hat{c}_1 = \frac{6500 + 7200 + 10500 + 12000}{4} = 9050,$$

$$\hat{c}_2 = \frac{21000 + 24000}{2} = 22500.$$

El error después de dividir es:

$$D_{\text{con división}} = (6500-9050)^2 + (7200-9050)^2 + (10500-9050)^2 + (12000-9050)^2 + (21000-22500)^2 + (24000-22500)^2.$$

Al calcular:

$$D_{\text{con división}} = 25,230,000.$$

La mejora relativa es:

$$\frac{D_{\text{sin dividir}} - D_{\text{con división}}}{D_{\text{sin dividir}}} = \frac{266,433,333.33 - 25,230,000}{266,433,333.33} \approx 0.9053.$$

Por tanto, el corte reduce el error aproximadamente en 90.53 %. El árbol resultante tiene una regla clara: si la RAM es menor que 12 GB, predice 9050; si es mayor o igual que 12 GB, predice 22500.

125.8. Implementación en Python

El siguiente ejemplo reproduce la idea anterior con `scikit-learn`. Aunque el conjunto de datos es muy pequeño, permite observar cómo un árbol de regresión aprende divisiones por umbral.

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, export_text
from sklearn.metrics import mean_squared_error, r2_score

# Datos de ejemplo
X = pd.DataFrame({
    "RAM_GB": [4, 4, 8, 8, 16, 16]
})
y = np.array([6500, 7200, 10500, 12000, 21000, 24000])

# Árbol pequeño de regresión
tree = DecisionTreeRegressor(max_depth=1, random_state=42)
tree.fit(X, y)

pred = tree.predict(X)

print(export_text(tree, feature_names=list(X.columns)))
print("Predicciones:", pred)
print("MSE:", mean_squared_error(y, pred))
print("R2:", r2_score(y, pred))
```

```
|--- RAM_GB <= 12.00
|   |--- value: [9050.00]
|--- RAM_GB > 12.00
|   |--- value: [22500.00]
```

Predicciones: [9050. 9050. 9050. 9050. 22500. 22500.]

MSE: 4205000.0

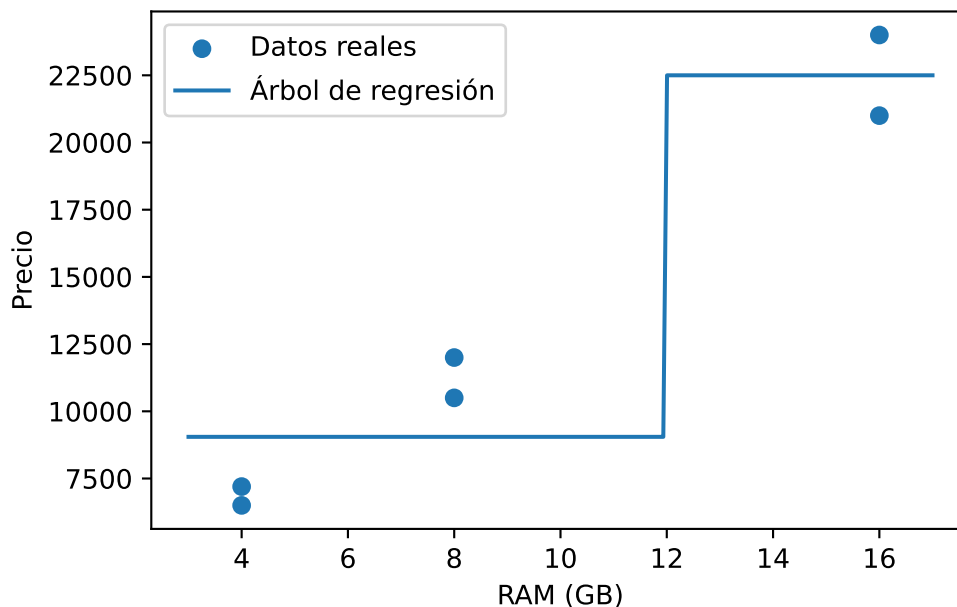
R2: 0.9053046415613661

El árbol de profundidad uno aprende un único corte. En un problema real se usaría una división entrenamiento-prueba o validación cruzada para estimar el desempeño fuera de muestra.

```
import matplotlib.pyplot as plt

ram_grid = np.linspace(3, 17, 200).reshape(-1, 1)
pred_grid = tree.predict(pd.DataFrame(ram_grid, columns=["RAM_GB"]))

plt.figure()
plt.scatter(X["RAM_GB"], y, label="Datos reales")
plt.plot(ram_grid, pred_grid, label="Árbol de regresión")
plt.xlabel("RAM (GB)")
plt.ylabel("Precio")
plt.legend()
plt.show()
```



125.9. Sobreajuste y poda

Un árbol puede seguir dividiendo hasta que cada hoja contenga muy pocas observaciones. En el extremo, podría memorizar los datos de entrenamiento. Esto reduce el error dentro de la muestra, pero aumenta el riesgo de fallar con datos nuevos. Este fenómeno se conoce como sobreajuste.

Para evitarlo, se controla la complejidad del árbol. Una estrategia clásica es la poda por costo-complejidad, que penaliza el número de hojas del árbol (Breiman et al. 1984; Pedregosa et al. 2011).

i Nota

Definición. El criterio de costo-complejidad penaliza el error total del árbol más un término proporcional al número de hojas:

$$C_{\alpha}(T) = \sum_{h=1}^J D_h + \alpha J. \quad (125.7)$$

Aquí D_h es el error de la hoja h , J es el número de hojas y $\alpha \geq 0$ controla la penalización por complejidad. Si α es pequeño, se permite un árbol más grande; si α es grande, se castigan más las divisiones.

Consideremos dos árboles:

Modelo	Error	Número de hojas J	α	Costo total
Árbol pequeño	500	3	20	$500 + 20(3) = 560$
Árbol grande	300	20	20	$300 + 20(20) = 700$

Aunque el árbol grande tiene menor error de entrenamiento, su costo total es mayor debido a la penalización por complejidad. Bajo este criterio, se preferiría el árbol pequeño.

En `scikit-learn`, este control puede implementarse mediante el parámetro `ccp_alpha`.

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_absolute_error

housing = fetch_california_housing(as_frame=True)
X = housing.data
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

for alpha in [0.0, 0.001, 0.01, 0.05]:
    model = DecisionTreeRegressor(random_state=42, ccp_alpha=alpha)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print({
        "ccp_alpha": alpha,
        "n_leaves": model.get_n_leaves(),
        "MAE": mean_absolute_error(y_test, y_pred),
        "R2": r2_score(y_test, y_pred)
    })
```

```
{'ccp_alpha': 0.0, 'n_leaves': np.int64(14853), 'MAE': 0.4647446201550388, 'R2': 0.60660300191292}
{'ccp_alpha': 0.001, 'n_leaves': np.int64(98), 'MAE': 0.45694597870846365, 'R2': 0.67828939325199}
```



```
{'ccp_alpha': 0.01, 'n_leaves': np.int64(11), 'MAE': 0.5813661215307218, 'R2': 0.5457586289530623}
{'ccp_alpha': 0.05, 'n_leaves': np.int64(5), 'MAE': 0.6348311418653407, 'R2': 0.47820743421206036}
```

125.10. Evaluación de árboles de regresión

En regresión, una métrica frecuente es el coeficiente de determinación R^2 . Este compara el error del modelo con el error de un modelo base que siempre predice el promedio de la variable respuesta.

i Nota

Definición. El coeficiente de determinación R^2 se define como:

$$R^2 = 1 - \frac{SSE}{SS_{TOT}},$$

con

$$SSE = \sum_i (y_i - \hat{y}_i)^2, \quad SS_{TOT} = \sum_i (y_i - \bar{y})^2. \quad (125.8)$$

Un valor $R^2 = 0.90$ indica que el modelo explica aproximadamente el 90 % de la variabilidad de la respuesta respecto al promedio. Un valor cercano a cero indica que el modelo no mejora sustancialmente frente a predecir siempre la media. En conjuntos de prueba, un R^2 negativo también es posible: significa que el modelo generaliza peor que el predictor promedio.

Además de R^2 , se suelen reportar el error absoluto medio (MAE), el error cuadrático medio (MSE) y la raíz del error cuadrático medio (RMSE). La selección de métrica depende del problema. Si se desea penalizar mucho los errores grandes, RMSE puede ser adecuado; si se busca una medida más robusta ante valores extremos, MAE suele ser más interpretable.

125.11. Ventajas y limitaciones de los árboles

Los árboles de regresión tienen varias ventajas. Son fáciles de entender porque se parecen a diagramas de decisión; producen reglas explícitas; pueden capturar relaciones no lineales e interacciones; requieren poca preparación de variables; y pueden trabajar con distintos criterios de error (James et al. 2021). Algunos algoritmos de árboles también incorporan estrategias para tratar datos faltantes, como divisiones sustitutas o *surrogate splits* (Breiman et al. 1984).

Sus limitaciones también son importantes. Primero, tienden a sobreajustarse si crecen demasiado. Segundo, sus particiones rectangulares pueden ser ineficientes cuando la verdadera relación entre variables es suave u oblicua. Tercero, son inestables: pequeños cambios en los datos pueden producir árboles muy distintos. Cuarto, pueden ser sensibles al ruido y a valores atípicos. Finalmente, si llegan nuevos datos, normalmente se reentrena el árbol completo.

Estas debilidades motivan el uso de métodos de ensamble, donde se combinan muchos árboles para obtener predicciones más estables y precisas.

125.12. Bagging: agregación por bootstrap

Bagging, abreviatura de *bootstrap aggregating*, fue propuesto por Breiman como una técnica para mejorar la estabilidad y precisión de predictores inestables (Breiman 1996). La idea consiste en entrenar muchos modelos sobre muestras bootstrap del conjunto original y combinar sus predicciones.

i Nota

Definición. Una muestra bootstrap de tamaño n se obtiene seleccionando n observaciones con reemplazo a partir del conjunto de entrenamiento original.

$$\mathcal{D}^{*(b)} = \{(x_1^{*(b)}, y_1^{*(b)}), \dots, (x_n^{*(b)}, y_n^{*(b)})\}. \quad (125.9)$$

El procedimiento general es:

1. Generar muchas muestras bootstrap del conjunto de entrenamiento.
2. Entrenar un árbol independiente en cada muestra.
3. Combinar las predicciones de todos los árboles.

En regresión, la combinación suele ser el promedio:

i Nota

Definición. La predicción bagging para regresión es el promedio de las predicciones de B modelos entrenados sobre muestras bootstrap:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*(b)}(x). \quad (125.10)$$

Bagging reduce la varianza sin aumentar demasiado el sesgo. Es especialmente útil con árboles de decisión, porque los árboles individuales pueden variar mucho ante cambios pequeños en los datos.

```
from sklearn.ensemble import BaggingRegressor

bag = BaggingRegressor(
    estimator=DecisionTreeRegressor(random_state=42),
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

bag.fit(X_train, y_train)
y_pred_bag = bag.predict(X_test)

print("MAE bagging:", mean_absolute_error(y_test, y_pred_bag))
print("R2 bagging:", r2_score(y_test, y_pred_bag))
```

MAE bagging: 0.32921646063953486

R2 bagging: 0.8074171978455695

125.13. Random forests

Random forest extiende bagging introduciendo aleatoriedad adicional en la construcción de cada árbol. Además de entrenar cada árbol con una muestra bootstrap, en cada división se considera solo un subconjunto aleatorio de variables candidatas (Breiman 2001). Esto disminuye la correlación entre árboles y mejora la reducción de varianza.

i Nota

Definición. Un random forest es un ensamble de árboles entrenados sobre muestras bootstrap, donde cada división se elige considerando solo un subconjunto aleatorio de variables.

$$\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x). \quad (125.11)$$

El modelo conserva parte de la interpretabilidad de los árboles, por ejemplo mediante medidas de importancia de variables, pero su estructura global ya no se resume en un único diagrama. A cambio, suele ofrecer mejor desempeño predictivo y menor inestabilidad.

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=300,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print("MAE random forest:", mean_absolute_error(y_test, y_pred_rf))
print("R2 random forest:", r2_score(y_test, y_pred_rf))

importances = pd.Series(rf.feature_importances_, index=X_train.columns)
importances.sort_values(ascending=False).head(10)
```

MAE random forest: 0.32785055025839827

R2 random forest: 0.8157534169907247

```
MedInc      0.355768
Latitude    0.133573
Longitude   0.131074
AveOccup    0.123788
AveRooms    0.119190
HouseAge    0.056062
AveBedrms   0.044804
Population  0.035741
dtype: float64
```

125.14. Boosting

Boosting también combina varios modelos, pero a diferencia de bagging, los entrena de forma secuencial. Cada nuevo modelo intenta corregir los errores cometidos por los modelos anteriores. La intuición es construir un predictor fuerte a partir de muchos aprendices débiles (Freund y Schapire 1997; Schapire 1990).

En clasificación, algunos métodos de boosting asignan mayor peso a las observaciones mal clasificadas. En regresión, una forma muy influyente es gradient boosting, donde cada nuevo árbol se ajusta a los residuos o, más generalmente, a la dirección de descenso del gradiente de una función de pérdida (Friedman 2001).

i Nota

Definición. En gradient boosting para regresión con pérdida cuadrática, el modelo se construye aditivamente:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \nu h_m(x),$$

donde cada h_m se ajusta a los residuos del modelo anterior y ν es la tasa de aprendizaje. \$\$\{\text{\#eq-gradient-boosting}\}

El procedimiento conceptual es:

1. Entrenar un primer modelo simple.
2. Calcular los errores o residuos.
3. Entrenar un nuevo modelo para explicar esos errores.
4. Repetir el proceso varias veces.
5. Combinar los modelos, usualmente con pesos o una tasa de aprendizaje.

Boosting puede alcanzar alta precisión, pero suele ser más sensible al ruido, a valores atípicos y al sobreajuste. Por ello requiere controlar hiperparámetros como número de estimadores, profundidad de los árboles, tasa de aprendizaje y regularización.

```
from sklearn.ensemble import GradientBoostingRegressor

gb = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)

print("MAE gradient boosting:", mean_absolute_error(y_test, y_pred_gb))
print("R2 gradient boosting:", r2_score(y_test, y_pred_gb))
```

MAE gradient boosting: 0.355833880633663

R2 gradient boosting: 0.7956950818163337

Entre las implementaciones modernas de boosting se encuentran XGBoost, LightGBM y CatBoost. Estas bibliotecas agregan optimizaciones computacionales y regularización avanzada, y son comunes en competencias y aplicaciones tabulares de alto rendimiento (Chen y Guestrin 2016; Ke et al. 2017; Prokhorenkova et al. 2018).

125.15. Bagging frente a boosting

Bagging y boosting combinan modelos, pero responden a problemas distintos.

Característica	Bagging	Boosting
Entrenamiento	Modelos independientes	Modelos secuenciales
Objetivo principal	Reducir varianza	Reducir sesgo y mejorar precisión
Manejo de errores	Las muestras se tratan de forma similar	Se enfoca en errores previos
Paralelización	Naturalmente paralelizable	Menos paralelizable por dependencia secuencial
Ruido y atípicos	Más robusto	Más sensible
Ejemplos	Bagging, Random Forest	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost

Como regla práctica, bagging es recomendable cuando el modelo base tiene alta varianza y tendencia al sobreajuste, como ocurre con árboles profundos. Boosting es útil cuando se busca mayor precisión y el modelo base todavía subajusta patrones relevantes, aunque debe regularizarse cuidadosamente.

125.16. Clasificación del vecino más cercano

El algoritmo del vecino más cercano pertenece a otra familia de métodos supervisados. En lugar de construir reglas explícitas como un árbol, almacena los datos de entrenamiento y clasifica un nuevo punto según las observaciones más parecidas. Por ello se considera un método basado en memoria o *lazy learning*: el “aprendizaje” ocurre principalmente al guardar los casos, mientras que el esfuerzo computacional se desplaza al momento de predecir (Cover y Hart 1967; Duda et al. 2001).

Sea un conjunto de entrenamiento

$$S_N = \{(x_1, \theta_1), (x_2, \theta_2), \dots, (x_N, \theta_N)\},$$

donde $x_i \in \mathbb{R}^p$ es el vector de atributos y $\theta_i \in \{1, \dots, c\}$ es la etiqueta de clase. Dada una métrica o función de distancia $d(x, z)$, el clasificador de vecino más cercano busca el punto de entrenamiento más próximo a una nueva observación x .

i Nota

Definición. En el clasificador de vecino más cercano, una nueva observación x recibe la clase de la observación de entrenamiento más cercana bajo una métrica de distancia d :

$$\hat{y}(x) = \theta_{i^*}, \quad i^* = \arg \min_{i=1, \dots, N} d(x, x_i). \quad (125.12)$$

También puede formularse mediante funciones discriminantes. Para cada clase ω_j , se calcula la distancia mínima entre x y los ejemplos de esa clase; se asigna la clase cuya distancia mínima sea menor.

i Nota

Definición. El discriminante de distancia mínima para la clase ω_j es:

$$g_j(x) = \min_{i:\theta_i=j} d(x, x_i), \quad \hat{y}(x) = \arg \min_{j=1,\dots,c} g_j(x). \quad (125.13)$$

Esta formulación muestra la conexión con los clasificadores por partes: si cada muestra de entrenamiento se trata como una subclase, entonces la regla de decisión se reduce a escoger la muestra más cercana.

125.17. Métricas de distancia para vecinos cercanos

La elección de la distancia es crucial. KNN no “entiende” directamente el significado semántico de las variables; solo compara puntos mediante una métrica. Por eso, variables medidas en escalas grandes pueden dominar la decisión si no se normalizan o estandarizan. Además, distintas métricas inducen geometrías distintas: la distancia euclidiana produce vecindades esféricas, mientras que la distancia Manhattan produce vecindades con forma de rombo en dos dimensiones.

i Nota

Definición. La distancia de Minkowski de orden $s \geq 1$ entre $x, z \in \mathbb{R}^p$ se define como:

$$d_s(x, z) = \left(\sum_{j=1}^p |x_j - z_j|^s \right)^{1/s}. \quad (125.14)$$

Casos particulares importantes son la distancia Manhattan ($s = 1$) y la distancia euclidiana ($s = 2$).

i Nota

Definición. La distancia Manhattan o taxicab entre x y z es:

$$d_1(x, z) = \sum_{j=1}^p |x_j - z_j|. \quad (125.15)$$

i Nota

Definición. La distancia euclidiana entre x y z es:

$$d_2(x, z) = \sqrt{\sum_{j=1}^p (x_j - z_j)^2}. \quad (125.16)$$

Otra opción es la distancia de Chebyshev, que mide la máxima diferencia coordenada a coordenada.

i Nota

Definición. La distancia de Chebyshev entre x y z es:

$$d_{\infty}(x, z) = \max_{j=1, \dots, p} |x_j - z_j|. \quad (125.17)$$

También se usan distancias cuadráticas ponderadas, útiles cuando las variables tienen distinta escala o correlación. Si Q es una matriz simétrica definida positiva, entonces

i Nota

Definición. Una distancia cuadrática ponderada entre x y z puede escribirse como:

$$d_Q(x, z) = (x - z)^{\top} Q (x - z). \quad (125.18)$$

Un caso relevante es la distancia de Mahalanobis, donde $Q = \Sigma^{-1}$ y Σ es la matriz de covarianza. Esta distancia corrige por escala y correlación entre variables (Duda et al. 2001).

```
import numpy as np
```

```
x = np.array([30, 25])
```

```
z = np.array([28, 24])
```

```
manhattan = np.sum(np.abs(x - z))
```

```
euclidean = np.sqrt(np.sum((x - z)**2))
```

```
chebyshev = np.max(np.abs(x - z))
```

```
print({
```

```
    "Manhattan": manhattan,
```

```
    "Euclidiana": euclidean,
```

```
    "Chebyshev": chebyshev
```

```
})
```

```
{'Manhattan': np.int64(3), 'Euclidiana': np.float64(2.23606797749979), 'Chebyshev': np.int64(2)}
```

125.18. De 1-NN a k-NN

La regla 1-NN puede generar regiones de clasificación muy fragmentadas, pues una sola observación ruidosa puede modificar la frontera local. Para reducir esa sensibilidad, se generaliza el método considerando los k vecinos más cercanos. En clasificación, cada vecino vota por su clase; en regresión, se promedian sus respuestas.

i Nota

Definición. En k -NN para clasificación, si $N_k(x)$ es el conjunto de índices de los k vecinos más cercanos de x , el discriminante de la clase j es el número de vecinos que pertenecen a esa clase:

$$g_j(x) = \sum_{i \in N_k(x)} \mathbb{I}(\theta_i = j), \quad \hat{y}(x) = \arg \max_{j=1, \dots, c} g_j(x). \quad (125.19)$$

i Nota

Definición. En k -NN para regresión, la predicción es el promedio de las respuestas de los k vecinos más cercanos:

$$\hat{f}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i. \quad (125.20)$$

La elección de k controla el equilibrio sesgo-varianza. Con $k = 1$, el modelo tiene bajo sesgo local, pero alta varianza: puede adaptarse demasiado a puntos individuales. Con un k grande, la frontera se suaviza y la varianza disminuye, pero aumenta el sesgo porque se promedian observaciones que pueden pertenecer a regiones distintas (Hastie et al. 2009). En aplicaciones prácticas, k suele elegirse mediante validación cruzada. Para clasificación binaria, a menudo se usan valores impares de k para reducir empates.

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

clientes = pd.DataFrame({
    "edad": [25, 28, 32, 35, 29, 31, 40, 22],
    "ingreso": [20, 24, 26, 30, 23, 27, 35, 18],
    "compra": ["No compra", "No compra", "Compra", "Compra", "No compra", "Compra", "Compra", "No compra"]
})

X_cls = clientes[["edad", "ingreso"]]
y_cls = clientes["compra"]

knn = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])

param_grid = {"model__n_neighbors": [1, 3, 5]}
search = GridSearchCV(knn, param_grid=param_grid, cv=2)
search.fit(X_cls, y_cls)

nuevo_cliente = pd.DataFrame({"edad": [30], "ingreso": [25]})
print("Mejor k:", search.best_params_)
print("Predicción:", search.predict(nuevo_cliente))
```

```
Mejor k: {'model__n_neighbors': 1}
Predicción: ['No compra']
```


125.19. Error asintótico de vecinos cercanos

Una razón histórica de la importancia de NN es que posee garantías teóricas simples. En el límite de muchas observaciones independientes e idénticamente distribuidas, el error asintótico de 1-NN está acotado por el error de Bayes, que es el menor error posible si se conocieran las distribuciones verdaderas de las clases (Cover y Hart 1967).

i Nota

Definición. Si P^* es el error de Bayes y P_1 es el error asintótico de 1-NN con c clases, entonces:

$$P^* \leq P_1 \leq P^* \left(2 - \frac{c}{c-1} P^* \right). \quad (125.21)$$

Como consecuencia, para dos o más clases se suele resumir la cota como $P_1 \leq 2P^*$. Es decir, bajo los supuestos del resultado, 1-NN no es arbitrariamente malo en el límite: su error asintótico es, a lo más, aproximadamente el doble del error óptimo de Bayes.

Para k -NN, si $N \rightarrow \infty$, $k \rightarrow \infty$ y $k/N \rightarrow 0$, el método puede aproximar el rendimiento óptimo de Bayes. La condición $k/N \rightarrow 0$ expresa que el vecindario debe crecer en número de puntos, pero seguir siendo local en relación con el tamaño total de la muestra (Stone 1977; Hastie et al. 2009).

i Nota

Definición. Una condición clásica de consistencia para k -NN es:

$$N \rightarrow \infty, \quad k \rightarrow \infty, \quad \frac{k}{N} \rightarrow 0. \quad (125.22)$$

Estas conclusiones dependen de que los vecinos cercanos sean realmente cercanos en el espacio de atributos. En alta dimensión, esta condición puede fallar: incluso el vecino más cercano puede quedar lejos en términos geométricos. Este fenómeno forma parte de la llamada maldición de la dimensionalidad.

125.20. Maldición de la dimensionalidad

KNN funciona mejor cuando la dimensión efectiva del problema no es demasiado alta y existe una noción de cercanía significativa. En espacios de alta dimensión, las distancias tienden a concentrarse: la diferencia entre el vecino más cercano y el más lejano puede volverse pequeña en relación con la escala total. Como resultado, la noción de “vecino” pierde poder discriminativo y se requieren muchas más muestras para cubrir el espacio (Hastie et al. 2009; Bellman 1961).

i Nota

Definición. En un hipercubo unitario de dimensión p , una región con fracción lineal r en cada coordenada ocupa una fracción de volumen:

$$V(r, p) = r^p. \quad (125.23)$$

Esta expresión muestra por qué la dimensión importa. Si $r = 0.1$ y $p = 10$, el volumen relativo es 10^{-10} . Para cubrir localmente un espacio de alta dimensión se necesitaría una cantidad enorme de datos. Por

ello, antes de aplicar KNN conviene considerar selección de variables, reducción de dimensionalidad, escalamiento adecuado y validación cuidadosa.

125.21. Métodos mejorados para vecinos cercanos

El método NN básico tiene tres limitaciones principales: debe almacenar todos los datos, debe comparar una consulta contra muchas observaciones y es sensible al ruido. Existen variantes que atacan estas dificultades.

125.21.1. Búsqueda eficiente: branch and bound

Los algoritmos de búsqueda rápida reducen el tiempo de predicción mediante preparación fuera de línea. La idea general es organizar las muestras en una estructura jerárquica y descartar regiones completas cuando no pueden contener al vecino más cercano. Esta lógica aparece en métodos *branch and bound*, árboles k-d y árboles de bolas (Friedman et al. 1977; Fukunaga y Narendra 1975).

Supongamos que un nodo p contiene un subconjunto de muestras $\mathcal{X}_p$, con centro M_p y radio r_p definido como la máxima distancia desde el centro a una muestra del nodo.

i Nota

Definición. Para un nodo p con muestras $\mathcal{X}_p$, su centro y radio pueden definirse como:

$$M_p = \frac{1}{N_p} \sum_{x_i \in \mathcal{X}_p} x_i, \quad r_p = \max_{x_i \in \mathcal{X}_p} d(x_i, M_p). \quad (125.24)$$

Si B es la mejor distancia encontrada hasta el momento para una consulta x , se puede descartar un nodo cuando, por desigualdad triangular, ningún punto del nodo puede mejorar esa distancia. Una forma intuitiva de la regla es: si el centro del nodo está demasiado lejos de x en comparación con su radio, no vale la pena explorar ese nodo.

i Nota

Definición. Una regla de descarte por cota inferior para un nodo p es:

$$d(x, M_p) - r_p > B. \quad (125.25)$$

Si se cumple Ecuación 125.25, entonces incluso el punto más favorable dentro del nodo estaría a una distancia mayor que B , por lo que el nodo puede descartarse.

125.21.2. Edición de vecinos cercanos

La edición de vecinos cercanos busca eliminar muestras ambiguas o ruidosas, especialmente en regiones donde las clases se traslapan. La intuición es que ciertos puntos mal etiquetados o ubicados en zonas de conflicto pueden fragmentar la frontera de decisión y perjudicar la generalización (Wilson 1972).

Un procedimiento típico consiste en preclasificar cada muestra usando sus vecinos; si una muestra es clasificada incorrectamente por el resto del conjunto, se marca como candidata a eliminar. Este proceso

puede repetirse varias rondas si se cuenta con suficientes datos. La edición puede mejorar robustez, aunque también puede eliminar casos difíciles pero legítimos, por lo que debe evaluarse con validación.

i Nota

Definición. La edición por preclasificación elimina una muestra (x_i, y_i) si su etiqueta no coincide con la predicción obtenida al clasificarla usando el conjunto de entrenamiento sin esa muestra:

$$(x_i, y_i) \text{ se elimina si } \hat{y}_{-i}(x_i) \neq y_i. \quad (125.26)$$

125.21.3. Condensed nearest neighbor

El método *condensed nearest neighbor* busca reducir memoria y cómputo conservando solo un subconjunto representativo de muestras (Hart 1968). Se separa el conjunto original en dos partes: un conjunto almacenado $\mathcal{S}$ y un conjunto restante $\mathcal{G}$. Se comienza con pocas muestras en $\mathcal{S}$; luego se revisan las muestras de $\mathcal{G}$. Si una muestra se clasifica correctamente usando $\mathcal{S}$, no se agrega; si se clasifica mal, se incorpora a $\mathcal{S}$. Al final, solo $\mathcal{S}$ se usa para clasificar.

i Nota

Definición. El conjunto condensado $\mathcal{S}$ busca ser un subconjunto del entrenamiento tal que el clasificador 1-NN construido con $\mathcal{S}$ preserve, aproximadamente, las decisiones obtenidas con el conjunto completo:

$$\mathcal{S} \subseteq S_N, \quad \hat{y}_{\mathcal{S}}(x_i) \approx \hat{y}_{S_N}(x_i). \quad (125.27)$$

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X_demo, y_demo = make_moons(n_samples=300, noise=0.25, random_state=42)

scores = []
for k in [1, 3, 5, 11, 21, 51]:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    acc = cross_val_score(model, X_demo, y_demo, cv=5, scoring="accuracy").mean()
    scores.append({"k": k, "accuracy_cv": acc})

pd.DataFrame(scores)
```

	k	accuracy_cv
0	1	0.906667
1	3	0.923333
2	5	0.920000

	k	accuracy_cv
3	11	0.926667
4	21	0.916667
5	51	0.900000

125.22. Cuándo usar KNN

KNN es una buena opción cuando el tamaño de muestra es razonable, la dimensión no es demasiado alta, las variables pueden escalarse de forma significativa y se desea una decisión basada en casos similares. Su interpretación es natural: “este punto se clasifica así porque se parece a estos ejemplos”. Sin embargo, no es ideal cuando hay millones de observaciones, demasiadas variables irrelevantes, datos muy ruidosos o necesidad de predicción en tiempo real sin estructuras de búsqueda especializadas.

125.23. Comparación conceptual: árboles y vecinos cercanos

Los árboles y KNN son métodos no lineales, pero su lógica es distinta. Los árboles construyen particiones explícitas mediante reglas jerárquicas; KNN define vecindades alrededor del punto a predecir. Los árboles generan modelos relativamente rápidos al momento de predecir, porque basta recorrer una ruta desde la raíz hasta una hoja. KNN puede ser más costoso en predicción, porque debe calcular distancias con respecto a muchos puntos de entrenamiento, salvo que se usen estructuras de búsqueda eficientes.

Desde el punto de vista pedagógico, ambos métodos ayudan a entender que el aprendizaje supervisado no se reduce a ajustar rectas. Un modelo puede aproximar una función mediante regiones constantes, mediante promedios locales o mediante combinaciones de muchos modelos simples.

125.24. Buenas prácticas de modelado

Para usar árboles, ensambles o KNN en un flujo reproducible de ciencia de datos, conviene seguir algunas reglas:

1. Separar datos de entrenamiento, validación y prueba, o usar validación cruzada.
2. Comparar contra modelos base simples, como predecir siempre el promedio en regresión.
3. Reportar métricas adecuadas al problema, no solo una métrica global.
4. Controlar hiperparámetros con validación, no con el conjunto de prueba.
5. Documentar semillas aleatorias, versiones de paquetes y criterios de evaluación.
6. Interpretar el modelo considerando sus supuestos geométricos: regiones rectangulares en árboles, distancias en KNN y reducción de varianza o sesgo en ensambles.

El siguiente ejemplo muestra una comparación breve entre árbol individual, random forest y gradient boosting.

```
from sklearn.model_selection import cross_validate

models = {
    "Árbol": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1),
```

```

    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

results = []
for name, model in models.items():
    cv = cross_validate(
        model,
        X,
        y,
        cv=5,
        scoring={"r2": "r2", "neg_mae": "neg_mean_absolute_error"},
        n_jobs=-1
    )
    results.append({
        "modelo": name,
        "R2_promedio": cv["test_r2"].mean(),
        "MAE_promedio": -cv["test_neg_mae"].mean()
    })

pd.DataFrame(results).sort_values("R2_promedio", ascending=False)

```

	modelo	R2_promedio	MAE_promedio
2	Gradient Boosting	0.669829	0.471677
1	Random Forest	0.654549	0.464681
0	Árbol	0.344065	0.620737

125.25. Conclusiones

Los árboles de regresión son modelos supervisados que reemplazan una ecuación global por una colección de reglas locales. Su predicción es constante dentro de cada región y se calcula como el promedio de las observaciones que caen en la hoja correspondiente. El entrenamiento se basa en particionamiento recursivo: se prueban cortes y se elige aquel que reduce más el error, usualmente medido como suma de cuadrados.

Su principal fortaleza es la interpretabilidad, pero también presentan riesgos: sobreajuste, sensibilidad al ruido, inestabilidad y fronteras rectangulares. La poda y el criterio de costo-complejidad ayudan a controlar árboles individuales. Los métodos de ensamble, como bagging, random forests y boosting, buscan superar estas limitaciones combinando muchos árboles. Bagging reduce varianza mediante entrenamiento paralelo sobre muestras bootstrap; boosting entrena modelos secuencialmente para corregir errores y suele obtener alta precisión bajo una buena regularización.

Finalmente, el vecino más cercano muestra otra forma de razonamiento no lineal: predecir a partir de casos similares. Mientras los árboles dividen el espacio mediante reglas, KNN usa distancias. En conjunto, estos métodos ofrecen una introducción poderosa al modelado predictivo moderno y preparan el terreno para técnicas más avanzadas de aprendizaje automático.

126. Gradient Boosting y XGBoost

XGBoost (*eXtreme Gradient Boosting*) fue propuesto por Chen y Guestrin en 2016 (Chen y Guestrin 2016) y desde entonces ha dominado competencias de machine learning en Kaggle y otros benchmarks de referencia. Se trata de una implementacion regularizada y optimizada a segundo orden del algoritmo de gradient boosting (Friedman 2001), con innovaciones clave en la busqueda aproximada de particiones, el manejo eficiente de datos dispersos, y un diseno de sistema orientado a la escalabilidad. Este capitulo cubre los fundamentos matematicos del algoritmo, sus hiperparametros principales, la interpretacion de importancia de variables mediante SHAP, y la implementacion practica en Python.

126.1. De los arboles de decision al boosting

126.1.1. El problema del sesgo-varianza y los modelos individuales

Un arbol de decision profundo tiene la capacidad de memorizar el conjunto de entrenamiento: al aumentar la profundidad, el modelo se ajusta cada vez mas a los datos observados y su varianza aumenta. Por el contrario, un arbol poco profundo (o un “tronco” de un solo nivel) introduce un sesgo alto porque su capacidad expresiva es limitada. Este dilema entre sesgo y varianza es central en el aprendizaje automatico y motiva el uso de metodos de ensamble.

Los metodos de ensamble combinan multiples modelos debiles para obtener un modelo fuerte. En el caso del *bagging* (e.g., Random Forest (Breiman 2001)), los arboles se entrenan de forma independiente sobre muestras con reemplazo, y su prediccion se promedia para reducir la varianza. En el *boosting*, la estrategia es diferente: los modelos se construyen de manera secuencial, donde cada nuevo arbol aprende de los errores del ensamble anterior, reduciendo progresivamente el sesgo.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(42)
n = 80
X_raw = np.sort(rng.uniform(0, 2 * np.pi, n))
y_raw = np.sin(X_raw) + rng.normal(0, 0.35, n)
X_fit = X_raw.reshape(-1, 1)

X_plot = np.linspace(0, 2 * np.pi, 300).reshape(-1, 1)

profundidades = [1, 3, 10]
titulos = [
    "Arbol profundidad 1 (alto sesgo)",
    "Arbol profundidad 3 (equilibrado)",
```

```

    "Arbol profundidad 10 (alta varianza)"
]

fig = make_subplots(rows=1, cols=3, subplot_titles=titulos)

for idx, (depth, titulo) in enumerate(zip(profundidades, titulos), start=1):
    tree = DecisionTreeRegressor(max_depth=depth, random_state=0)
    tree.fit(X_fit, y_raw)
    y_pred = tree.predict(X_plot)

    fig.add_trace(
        go.Scatter(
            x=X_raw, y=y_raw,
            mode="markers",
            marker=dict(color="#94a3b8", size=5),
            name="Datos",
            showlegend=(idx == 1)
        ),
        row=1, col=idx
    )
    fig.add_trace(
        go.Scatter(
            x=X_plot.flatten(), y=y_pred,
            mode="lines",
            line=dict(color="#2563eb", width=2.5),
            name="Prediccion",
            showlegend=(idx == 1)
        ),
        row=1, col=idx
    )
    fig.add_trace(
        go.Scatter(
            x=X_plot.flatten(), y=np.sin(X_plot.flatten()),
            mode="lines",
            line=dict(color="black", dash="dash", width=1.5),
            name="Funcion verdadera",
            showlegend=(idx == 1)
        ),
        row=1, col=idx
    )

fig.update_layout(
    height=380,
    template="plotly_white",
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig.update_xaxes(title_text="x")
fig.update_yaxes(title_text="y", col=1)
fig.show()

```


Unable to display output for mime type(s): text/html

(a) Ilustracion del compromiso sesgo-varianza para arboles de decision de distinta profundidad. Un arbol superficial (profundidad 1) presenta alto sesgo, mientras que uno muy profundo (profundidad 10) presenta alta varianza.

Unable to display output for mime type(s): text/html

(b)

Figura 126.1.

La Figura 126.1 muestra claramente como la profundidad del arbol determina el equilibrio entre sesgo y varianza. El arbol de profundidad 3 logra un ajuste razonable sin sobreajustar el ruido.

126.1.2. Boosting: aprendizaje secuencial de residuos

El boosting construye un modelo aditivo de forma secuencial. Partiendo de una prediccion inicial constante:

$$F_0(\mathbf{x}) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \gamma) \quad (126.1)$$

En cada iteracion $m = 1, \dots, M$, se calcula el gradiente negativo (pseudoresiduos) de la funcion de perdida evaluada en la prediccion actual:

$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}} \quad (126.2)$$

Luego se ajusta un modelo debil h_m a estos pseudoresiduos, y se actualiza el ensamble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x}) \quad (126.3)$$

donde $\eta \in (0, 1]$ es la tasa de aprendizaje o *shrinkage*. Para la perdida cuadratica $L(y, F) = \frac{1}{2}(y - F)^2$, el gradiente negativo es simplemente el residuo ordinario $r_i = y_i - F_{m-1}(\mathbf{x}_i)$. Esto significa que el algoritmo ajusta iterativamente los errores del ensamble: cada nuevo arbol aprende lo que el ensamble anterior no pudo capturar.

Gradient Boosting (version simplificada)

1. Inicializar $F_0(\mathbf{x}) = \bar{y}$ (media de la variable objetivo).
2. Para $m = 1, \dots, M$:
 - Calcular residuos $r_i^{(m)} = y_i - F_{m-1}(\mathbf{x}_i)$.
 - Ajustar un arbol debil h_m a $\{(\mathbf{x}_i, r_i^{(m)})\}_{i=1}^n$.
 - Actualizar $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$.
3. Devolver $F_M(\mathbf{x})$.

La siguiente figura ilustra la convergencia del algoritmo de gradient boosting implementado desde cero, sin usar XGBoost.

```
import numpy as np
import plotly.graph_objects as go

rng2 = np.random.default_rng(7)
n2 = 60
X2 = np.sort(rng2.uniform(0, 2 * np.pi, n2)).reshape(-1, 1)
y2 = np.sin(X2.flatten()) + rng2.normal(0, 0.3, n2)
X_line = np.linspace(0, 2 * np.pi, 300).reshape(-1, 1)
y_true_line = np.sin(X_line.flatten())

class Stump:
    def fit(self, X, r):
        best_feat, best_thr, best_loss = 0, 0, float("inf")
        for feat in range(X.shape[1]):
            thresholds = np.unique(X[:, feat])
            for thr in thresholds:
                left = r[X[:, feat] <= thr]
                right = r[X[:, feat] > thr]
                if len(left) == 0 or len(right) == 0:
                    continue
                loss = np.sum((left - left.mean()) ** 2) + np.sum((right - right.mean()) ** 2)
                if loss < best_loss:
                    best_loss = loss
                    best_feat, best_thr = feat, thr
                    self.val_left, self.val_right = left.mean(), right.mean()
        self.feats, self.thr = best_feat, best_thr

    def predict(self, X):
        return np.where(X[:, self.feats] <= self.thr, self.val_left, self.val_right)

eta = 0.5
M_max = 20
stumps = []
F_current = np.full(n2, y2.mean())

snapshots = {}
rondas = [1, 2, 3, 5, 10, 20]

for m in range(1, M_max + 1):
    residuos = y2 - F_current
    stump = Stump()
    stump.fit(X2, residuos)
    stumps.append(stump)
    F_current = F_current + eta * stump.predict(X2)
    if m in rondas:
```

```

F_line = np.full(300, y2.mean())
for s in stumps:
    F_line = F_line + eta * s.predict(X_line)
snapshots[m] = {
    "F_line": F_line.copy(),
    "residuos": (y2 - F_current).copy()
}

frames = []
for m in rondas:
    snap = snapshots[m]
    frame = go.Frame(
        data=[
            go.Scatter(
                x=X2.flatten(), y=y2,
                mode="markers",
                marker=dict(color="#94a3b8", size=6),
                name="Datos"
            ),
            go.Scatter(
                x=X_line.flatten(), y=y_true_line,
                mode="lines",
                line=dict(color="black", dash="dash", width=1.5),
                name="Funcion verdadera"
            ),
            go.Scatter(
                x=X_line.flatten(), y=snap["F_line"],
                mode="lines",
                line=dict(color="#2563eb", width=2.5),
                name="Prediccion F_m"
            ),
            go.Scatter(
                x=X2.flatten(), y=snap["residuos"],
                mode="markers",
                marker=dict(color="#f97316", size=6, symbol="diamond"),
                name="Residuos"
            )
        ],
        name=str(m)
    )
    frames.append(frame)

snap0 = snapshots[1]
fig2 = go.Figure(
    data=[
        go.Scatter(x=X2.flatten(), y=y2, mode="markers",
                    marker=dict(color="#94a3b8", size=6), name="Datos"),
        go.Scatter(x=X_line.flatten(), y=y_true_line, mode="lines",
                    line=dict(color="black", dash="dash", width=1.5), name="Funcion verdadera"),

```

```

        go.Scatter(x=X_line.flatten(), y=snap0["F_line"], mode="lines",
                    line=dict(color="#2563eb", width=2.5), name="Prediccion F_m"),
        go.Scatter(x=X2.flatten(), y=snap0["residuos"], mode="markers",
                    marker=dict(color="#f97316", size=6, symbol="diamond"), name="Residuos")
    ],
    frames=frames
)

steps = []
for m in rondas:
    step = dict(
        method="animate",
        args=[[str(m)], {"frame": {"duration": 600, "redraw": True}, "mode": "immediate"}],
        label=str(m)
    )
    steps.append(step)

fig2.update_layout(
    template="plotly_white",
    height=440,
    xaxis_title="x",
    yaxis_title="y",
    legend=dict(orientation="h", yanchor="bottom", y=-0.3, xanchor="center", x=0.5),
    sliders=[dict(
        active=0,
        currentvalue=dict(prefix="M = ", font=dict(size=14)),
        pad=dict(t=50),
        steps=steps
    )],
    updatemenus=[dict(
        type="buttons",
        showactive=False,
        y=1.1, x=0.5, xanchor="center",
        buttons=[
            dict(label="Reproducir", method="animate",
                  args=[None, {"frame": {"duration": 800, "redraw": True}, "fromcurrent": True}]),
            dict(label="Pausar", method="animate",
                  args=[[None], {"frame": {"duration": 0, "redraw": False}, "mode": "immediate"}])
        ]
    )]
)
fig2.show()

```

Unable to display output for mime type(s): text/html

Figura 126.2.: Convergencia iterativa del gradient boosting sobre un problema de regresion 1D. Cada frame muestra la prediccion acumulada del ensamble (azul) y los residuos actuales (naranja) luego de agregar M arboles.

Observando la Figura 126.2 se aprecia como los residuos (puntos naranjas) se reducen progresivamente a medida que se agregan mas arboles al ensamble, y la curva de prediccion (azul) se acerca a la funcion verdadera (linea discontinua negra).

Un aspecto fundamental del boosting es que la tasa de aprendizaje η en la Ecuación 126.3 actua como un regulador de la velocidad de convergencia. Con $\eta = 1$ el algoritmo intenta corregir completamente los errores en cada paso, lo que puede llevar a sobreajuste rapido. Con η pequeno se necesitan mas iteraciones pero el modelo final suele generalizar mejor. Este efecto se conoce como *shrinkage* (Friedman 2002) y es uno de los mecanismos de regularizacion implicitos del boosting.

El gradient boosting en su forma general (Ecuación 126.2) puede verse como un descenso de gradiente funcional: en lugar de actualizar los parametros de un modelo parametrico en el espacio euclidiano, se actualiza la funcion de prediccion en el espacio funcional. Cada nuevo arbol debil h_m es el “paso de gradiente” que mas reduce la perdida esperada. Esta perspectiva, formalizada por Friedman (2001), unifica una amplia familia de algoritmos de aprendizaje bajo un unico marco matematico.

126.2. El objetivo de XGBoost

126.2.1. Modelo aditivo y funcion objetivo regularizada

XGBoost construye el modelo de forma aditiva:

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \quad (126.4)$$

donde $\mathcal{F}$ es el espacio de arboles de regresion. En cada ronda t , se busca el arbol f_t que minimiza el objetivo regularizado:

$$\text{Obj}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) + \text{constante} \quad (126.5)$$

La regularizacion Ω penaliza tanto el numero de hojas como la magnitud de los pesos:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2 \quad (126.6)$$

donde T es el numero de hojas del arbol f_t y $\mathbf{w}$ son los pesos de las hojas. El parametro γ controla la poda (un arbol solo se expande si la ganancia supera γ) y λ regula el tamano de los pesos mediante regularizacion L2.

Innovacion clave de XGBoost: A diferencia del gradient boosting clasico (Friedman 2001), XGBoost incluye explicitamente la regularizacion en el objetivo de optimizacion y utiliza informacion de segundo orden (el hessiano) para encontrar pesos optimos de forma analitica, lo que conduce a una convergencia mas rapida y modelos mas robustos.

126.2.2. Aproximacion de Taylor de segundo orden

Para poder optimizar el objetivo de la Ecuación 126.5 eficientemente, XGBoost aplica una expansion de Taylor de segundo orden alrededor de $\hat{y}_i^{(t-1)}$:

$$l(y_i, \hat{y}_i^{(t-1)} + f_t) \approx l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \quad (126.7)$$

donde se definen:

$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad (\text{gradiente de primer orden}) \quad (126.8)$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2} \quad (\text{hessiano — curvatura local}) \quad (126.9)$$

Descartando los terminos constantes respecto a f_t , el objetivo simplificado es:

$$\tilde{\text{Obj}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (126.10)$$

Esta aproximacion cuadratica es exacta para la perdida cuadratica y una buena aproximacion local para cualquier perdida dos veces diferenciable, lo que hace a XGBoost aplicable a una amplia variedad de problemas (regresion, clasificacion, ranking, etc.).

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

y_obs = 1.0
F0 = -0.3

F_vals = np.linspace(-2.0, 2.5, 400)
loss_true = 0.5 * (y_obs - F_vals) ** 2

g_i = -(y_obs - F0)
h_i = 1.0

loss_linear = 0.5 * (y_obs - F0) ** 2 + g_i * (F_vals - F0)
loss_quadratic = 0.5 * (y_obs - F0) ** 2 + g_i * (F_vals - F0) + 0.5 * h_i * (F_vals - F0) ** 2

lam = 1.0
delta_vals = np.linspace(-3.0, 3.0, 400)
obj_delta = g_i * delta_vals + 0.5 * (h_i + lam) * delta_vals ** 2
delta_opt = -g_i / (h_i + lam)
obj_opt = g_i * delta_opt + 0.5 * (h_i + lam) * delta_opt ** 2

fig3 = make_subplots(
    rows=1, cols=2,
```

```

    subplot_titles=[
        "Curva de perdida y aproximacion de Taylor",
        "Contribucion cuadratica al objetivo (con lambda)"
    ]
)

fig3.add_trace(
    go.Scatter(x=F_vals, y=loss_true, mode="lines",
               line=dict(color="black", width=2.5),
               name="Perdida real L(y, F)",
               row=1, col=1
    )
)
fig3.add_trace(
    go.Scatter(x=F_vals, y=loss_linear, mode="lines",
               line=dict(color="#f97316", width=1.8, dash="dot"),
               name="Aprox. lineal (1er orden)",
               row=1, col=1
    )
)
fig3.add_trace(
    go.Scatter(x=F_vals, y=loss_quadratic, mode="lines",
               line=dict(color="#2563eb", width=2.0, dash="dash"),
               name="Aprox. cuadratica (2do orden)",
               row=1, col=1
    )
)
fig3.add_trace(
    go.Scatter(x=[F0], y=[0.5 * (y_obs - F0) ** 2],
               mode="markers",
               marker=dict(color="red", size=10, symbol="circle"),
               name="Punto de expansion F_0",
               row=1, col=1
    )
)

fig3.add_trace(
    go.Scatter(x=delta_vals, y=obj_delta, mode="lines",
               line=dict(color="#2563eb", width=2.5),
               name="g*Delta + (1/2)(h+lambda)*Delta^2",
               showlegend=True),
    row=1, col=2
)
fig3.add_trace(
    go.Scatter(
        x=[delta_opt, delta_opt],
        y=[obj_delta.min() - 0.3, obj_delta.max()],
        mode="lines",
        line=dict(color="red", dash="dash", width=1.8),
        name="Delta* optimo = -g/(h+lambda)"
    ),
    row=1, col=2
)

```

```

fig3.add_trace(
    go.Scatter(x=[delta_opt], y=[obj_opt],
               mode="markers",
               marker=dict(color="red", size=10),
               name="Minimo analitico",
               showlegend=False),
    row=1, col=2
)
fig3.add_annotation(
    x=delta_opt, y=obj_opt - 0.5,
    text=f"Delta* = {delta_opt:.2f}",
    showarrow=True, arrowhead=2,
    font=dict(size=12, color="red"),
    row=1, col=2
)

fig3.update_xaxes(title_text="F (prediccion)", row=1, col=1)
fig3.update_yaxes(title_text="Perdida L(y, F)", row=1, col=1)
fig3.update_xaxes(title_text="Delta (cambio en prediccion)", row=1, col=2)
fig3.update_yaxes(title_text="Contribucion al objetivo", row=1, col=2)
fig3.update_layout(
    height=420,
    template="plotly_white",
    legend=dict(orientation="h", yanchor="bottom", y=-0.35, xanchor="center", x=0.5)
)
fig3.show()

```

Unable to display output for mime type(s): text/html

Figura 126.3.: Aproximacion de Taylor de segundo orden de la funcion de perdida. Panel izquierdo: curva de perdida real (negro), aproximacion lineal (naranja) y aproximacion cuadratica (azul) alrededor del punto de expansion $F=-0.3$. Panel derecho: contribucion cuadratica al objetivo en funcion del cambio Delta, con el optimo analitico senalado.

La Figura 126.3 ilustra por que la aproximacion cuadratica es util: permite encontrar el cambio optimo $\Delta^* = -g_i/(h_i + \lambda)$ de forma analitica, sin necesidad de busqueda iterativa. XGBoost utiliza exactamente esta idea para calcular el peso optimo de cada hoja.

126.2.3. Pesos optimos de las hojas

Agrupando las muestras por hoja j (donde $I_j = \{i : q(\mathbf{x}_i) = j\}$ es el conjunto de indices de la hoja j), el objetivo simplificado se puede reescribir como:

$$\tilde{\text{Obj}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T \quad (126.11)$$

Definiendo $G_j = \sum_{i \in I_j} g_i$ y $H_j = \sum_{i \in I_j} h_i$, y minimizando respecto a w_j , se obtiene el peso optimo:

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (126.12)$$

Sustituyendo en el objetivo:

$$\text{Obj}^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (126.13)$$

Este valor sirve como puntuación de la estructura del árbol: cuanto más negativo sea, mejor es el árbol. Nótese que $\lambda > 0$ reduce la magnitud de los pesos y tiene un efecto regularizador directo, ya que $|w_j^*|$ decrece con λ .

126.2.4. Ganancia de una partición

Para decidir si realizar una división, XGBoost compara el objetivo antes y después de la partición. Si un nodo padre P se divide en hijos izquierdo L y derecho R , la ganancia de la división es:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma \quad (126.14)$$

Si $\text{Gain} \leq 0$, la división se rechaza (poda). El término $-\gamma$ penaliza la complejidad del árbol: un γ mayor hace que solo se acepten divisiones con mejoras sustanciales.

```
import numpy as np
import plotly.graph_objects as go

rng3 = np.random.default_rng(21)
n3 = 150
X3 = np.sort(rng3.uniform(0, 10, n3))
y3 = 2.0 * np.sin(X3 * 0.8) + rng3.normal(0, 0.5, n3)

F_prev = np.full(n3, y3.mean())
g_vals = -(y3 - F_prev)
h_vals = np.ones(n3)

lam3 = 1.0
G_total = g_vals.sum()
H_total = h_vals.sum()
score_parent = G_total ** 2 / (H_total + lam3)

thresholds = np.unique(X3)[1:-1]
gains = []

for thr in thresholds:
    left_mask = X3 <= thr
    right_mask = ~left_mask
    G_L = g_vals[left_mask].sum()
    H_L = h_vals[left_mask].sum()
```

```

G_R = g_vals[right_mask].sum()
H_R = h_vals[right_mask].sum()
if H_L + lam3 <= 0 or H_R + lam3 <= 0:
    gains.append(float("nan"))
    continue
gain = 0.5 * (G_L ** 2 / (H_L + lam3) + G_R ** 2 / (H_R + lam3) - score_parent)
gains.append(gain)

gains = np.array(gains)
best_idx = np.nanargmax(gains)
best_thr = thresholds[best_idx]
best_gain = gains[best_idx]

fig4 = go.Figure()

fig4.add_trace(go.Scatter(
    x=thresholds, y=gains,
    mode="lines",
    fill="tozeroy",
    fillcolor="rgba(37, 99, 235, 0.15)",
    line=dict(color="#2563eb", width=2.0),
    name="Ganancia"
))

fig4.add_trace(go.Scatter(
    x=[best_thr, best_thr],
    y=[gains.min() - 0.5, best_gain],
    mode="lines",
    line=dict(color="red", dash="dash", width=2.0),
    name=f"Umbral optimo = {best_thr:.2f}"
))

fig4.add_trace(go.Scatter(
    x=[thresholds.min(), thresholds.max()],
    y=[0, 0],
    mode="lines",
    line=dict(color="gray", dash="dash", width=1.5),
    name="Gain = 0 (poda)"
))

fig4.add_annotation(
    x=best_thr, y=best_gain + 0.3,
    text=f"Optimo: thr={best_thr:.2f}, Gain={best_gain:.2f}",
    showarrow=True, arrowhead=2,
    font=dict(size=12, color="red")
)

fig4.update_layout(
    template="plotly_white",
    height=400,
    xaxis_title="Umbral de particion",
    yaxis_title="Ganancia",
    legend=dict(orientation="h", yanchor="bottom", y=-0.3, xanchor="center", x=0.5)
)

```

```
)
fig4.show()
```

Unable to display output for mime type(s): text/html

Figura 126.4.: Ganancia de particion en funcion del umbral para un conjunto de datos sintetico 1D. La linea discontinua roja marca el umbral optimo (maxima ganancia). La linea horizontal gris indica Gain=0, umbral de poda cuando gamma=0.

La Figura 126.4 muestra como la ganancia varia en funcion del umbral de corte. El algoritmo busca el umbral que maximiza esta funcion en cada nivel del arbol. Cuando $\gamma > 0$, cualquier ganancia por debajo de ese valor lleva a rechazar la particion.

126.2.5. Busqueda aproximada de particiones (*Approximate Split Finding*)

Un aspecto crucial del diseno de XGBoost es que la busqueda exacta del mejor umbral (recorriendo todos los valores unicos de cada variable) puede ser prohibitivamente costosa para conjuntos de datos grandes. Para abordar esto, Chen y Guestrin (2016) proponen un algoritmo de busqueda aproximada basado en quantiles ponderados.

La idea es proponer un conjunto reducido de candidatos de particion $\mathcal{S}_k = \{s_{k1}, s_{k2}, \dots, s_{kl}\}$ para cada variable k , utilizando los quantiles de la distribucion de la variable ponderada por los hessianos h_i :

$$\text{rank}_k(z) = \frac{\sum_{(x,h): x_k \leq z} h}{\sum_{(x,h)} h} \quad (126.15)$$

Los candidatos se seleccionan tal que $|\text{rank}_k(s_{k,j}) - \text{rank}_k(s_{k,j+1})| \leq \varepsilon$, donde ε controla el numero aproximado de candidatos: $\approx 1/\varepsilon$ por variable. Esto garantiza que el error de aproximacion respecto a la solucion exacta esta acotado.

El uso de los hessianos h_i como pesos en la distribucion de quantiles no es arbitrario: las observaciones con mayor hessiano (mayor curvatura de la perdida) tienen mas influencia en la seleccion de los puntos de corte, lo que dirige la busqueda hacia las regiones del espacio de caracteristicas donde la ganancia potencial es mayor.

Sparsity-awareness (manejo de datos dispersos)

XGBoost aprende automaticamente la *direccion por defecto* para los valores faltantes o cero en cada nodo. Durante el entrenamiento, solo se evaluan los valores no faltantes para cada variable; los valores faltantes se asignan a la rama que minimiza el objetivo. Esto hace que XGBoost sea nativamente compatible con matrices dispersas sin necesidad de imputacion previa, una ventaja practica significativa en aplicaciones reales (Chen y Guestrin 2016).

126.3. Hiperparametros principales

XGBoost expone numerosos hiperparametros que permiten controlar la capacidad del modelo, la regularizacion, el subsampling y la velocidad de convergencia. La siguiente tabla resume los mas importantes.

Desde la perspectiva del aprendizaje estadístico, los hiperparámetros pueden agruparse en tres categorías conceptuales:

1. **Capacidad del modelo:** `max_depth`, `n_estimators`, `min_child_weight`. Determinan cuanta información puede memorizar el modelo. Valores altos de `max_depth` y `n_estimators` pueden llevar a sobreajuste.
2. **Regularización:** `gamma`, `reg_alpha`, `reg_lambda`. Penalizan la complejidad del modelo y reducen la varianza de los pesos de las hojas. Corresponden directamente a los términos de la Ecuación 126.6.
3. **Aleatoriedad y robustez:** `subsample`, `colsample_bytree`, `learning_rate`. Introdúcen estocasticidad en el proceso de construcción, similar al gradient boosting estocástico (Friedman 2002), y controlan la velocidad de convergencia mediante el *shrinkage* η .

Tabla 126.1.: Hiperparámetros principales de XGBoost y sus rangos típicos.

Parámetro	Descripción	Valor típico
<code>n_estimators</code>	Número de árboles en el ensamble	100–1000
<code>max_depth</code>	Profundidad máxima por árbol	3–8
<code>learning_rate</code> (η)	Tasa de aprendizaje (shrinkage)	0.01–0.3
<code>subsample</code>	Fracción de filas muestreadas por árbol	0.6–1.0
<code>colsample_bytree</code>	Fracción de columnas por árbol	0.6–1.0
<code>min_child_weight</code>	Mínimo de $\sum h_i$ en un nodo hoja	1–10
<code>gamma</code> (γ)	Ganancia mínima para realizar una partición	0–5
<code>reg_alpha</code>	Regularización L1 sobre pesos	0–1
<code>reg_lambda</code>	Regularización L2 sobre pesos	1 (default)

El número de árboles (`n_estimators`) y la tasa de aprendizaje (`learning_rate`) interactúan directamente: tasas de aprendizaje menores requieren más árboles pero generalmente producen mejores modelos. El `subsample` y el `colsample_bytree` introducen aleatoriedad similar al Random Forest, lo que reduce la varianza y el sobreajuste (Friedman 2002). El `min_child_weight` controla la complejidad de cada árbol al exigir que cada hoja tenga una suma de hessianos mínima.

126.3.1. Convergencia y early stopping

```
import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import xgboost as xgb

data_ca = fetch_california_housing()
X_ca, y_ca = data_ca.data, data_ca.target
X_train, X_val, y_train, y_val = train_test_split(X_ca, y_ca, test_size=0.2, random_state=42)

dtrain = xgb.DMatrix(X_train, label=y_train)
```

```

dval = xgb.DMatrix(X_val, label=y_val)

params = {
    "objective": "reg:squarederror",
    "max_depth": 4,
    "learning_rate": 0.1,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "reg_lambda": 1.0,
    "seed": 42
}

evals_result = {}
model_conv = xgb.train(
    params,
    dtrain,
    num_boost_round=300,
    evals=[(dtrain, "train"), (dval, "validacion")],
    early_stopping_rounds=20,
    evals_result=evals_result,
    verbose_eval=False
)

train_rmse = evals_result["train"]["rmse"]
val_rmse = evals_result["validacion"]["rmse"]
best_round = model_conv.best_iteration
rounds = list(range(1, len(train_rmse) + 1))

fig5 = go.Figure()

fig5.add_trace(go.Scatter(
    x=list(range(best_round + 1, len(rounds) + 1)),
    y=val_rmse[best_round:],
    fill="tozeroy",
    fillcolor="rgba(220, 38, 38, 0.08)",
    mode="none",
    name="Region de sobreajuste",
    showlegend=True
))

fig5.add_trace(go.Scatter(
    x=rounds, y=train_rmse,
    mode="lines",
    line=dict(color="#2563eb", width=2.0),
    name="RMSE entrenamiento"
))

fig5.add_trace(go.Scatter(
    x=rounds, y=val_rmse,
    mode="lines",
    line=dict(color="#dc2626", width=2.0),

```

```

        name="RMSE validacion"
    ))
    fig5.add_vline(
        x=best_round + 1,
        line=dict(color="black", dash="dash", width=1.8),
        annotation_text=f"Early stop: ronda {best_round + 1}",
        annotation_position="top right"
    )
    fig5.update_layout(
        template="plotly_white",
        height=420,
        xaxis_title="Ronda de boosting",
        yaxis_title="RMSE",
        legend=dict(orientation="h", yanchor="bottom", y=-0.3, xanchor="center", x=0.5)
    )
    fig5.show()

```

Unable to display output for mime type(s): text/html

Figura 126.5.: Curvas de error RMSE en entrenamiento y validacion en funcion del numero de rondas de boosting para el conjunto de datos California Housing. La linea vertical indica la ronda de parada anticipada (early stopping). La zona roja sombreada senala la region de sobreajuste potencial.

La Figura 126.5 muestra el comportamiento tipico del gradient boosting: el error de entrenamiento decrece monotonamente, mientras el error de validacion se estabiliza y puede comenzar a subir si se excede la cantidad optima de arboles. La tecnica de *early stopping* detiene el entrenamiento cuando el error de validacion no mejora durante un numero definido de rondas consecutivas.

126.3.2. Búsqueda de hiperparametros: profundidad y tasa de aprendizaje

```

import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import xgboost as xgb

data_hp = fetch_california_housing()
X_hp, y_hp = data_hp.data, data_hp.target
X_tr_hp, X_va_hp, y_tr_hp, y_va_hp = train_test_split(X_hp, y_hp, test_size=0.2, random_state=42)

depths = [2, 3, 4, 5, 6]
lrs = [0.01, 0.05, 0.1, 0.2, 0.3]

rmse_grid = np.zeros((len(depths), len(lrs)))

for i, depth in enumerate(depths):

```

```

for j, lr in enumerate(lrs):
    model_g = xgb.XGBRegressor(
        n_estimators=100,
        max_depth=depth,
        learning_rate=lr,
        subsample=0.8,
        colsample_bytree=0.8,
        reg_lambda=1.0,
        random_state=42,
        verbosity=0
    )
    model_g.fit(X_tr_hp, y_tr_hp)
    preds = model_g.predict(X_va_hp)
    rmse_grid[i, j] = np.sqrt(np.mean((y_va_hp - preds) ** 2))

best_i, best_j = np.unravel_index(np.argmin(rmse_grid), rmse_grid.shape)

text_vals = [[f"{rmse_grid[i, j]:.3f}" for j in range(len(lrs))] for i in range(len(depths))]

fig6 = go.Figure()

fig6.add_trace(go.Heatmap(
    z=rmse_grid.tolist(),
    x=[str(lr) for lr in lrs],
    y=[str(d) for d in depths],
    colorscale="RdYlGn_r",
    text=text_vals,
    texttemplate="%{text}",
    colorbar=dict(title="RMSE"),
    showscale=True
))

bx = best_j
by = best_i
dx = 0.5
dy = 0.5

fig6.add_trace(go.Scatter(
    x=[bx - dx, bx + dx, bx + dx, bx - dx, bx - dx],
    y=[by - dy, by - dy, by + dy, by + dy, by - dy],
    mode="lines",
    line=dict(color="red", width=3),
    name=f"Mejor: depth={depths[best_i]}, lr={lrs[best_j]}",
    showlegend=True
))

fig6.update_layout(
    template="plotly_white",
    height=440,

```

```

    xaxis_title="learning_rate",
    yaxis_title="max_depth",
    legend=dict(orientation="h", yanchor="bottom", y=-0.2, xanchor="center", x=0.5)
)
fig6.show()

```

Unable to display output for mime type(s): text/html

Figura 126.6.: RMSE de validacion sobre una grilla de `max_depth` x `learning_rate` en el conjunto California Housing. Valores mas bajos (verde) indican mejor rendimiento. La celda optima se marca con un borde rojo.

La Figura 126.6 revela la interaccion entre la profundidad del arbol y la tasa de aprendizaje. En general, tasas de aprendizaje muy bajas (0.01) requieren mas arboles para converger y con solo 100 estimadores pueden tener alto sesgo. Profundidades intermedias (3–5) suelen dar el mejor equilibrio entre capacidad y generalizacion.

126.4. Importancia de variables

XGBoost ofrece tres metricas para cuantificar la importancia de cada variable en el modelo entrenado:

- **Frecuencia (weight):** numero de veces que la variable se usa en una division a lo largo de todos los arboles. Es sensible al numero de categorias distintas de cada variable.
- **Ganancia (gain):** reduccion promedio de la perdida aportada por cada division que usa esa variable. Es la metrica mas informativa porque captura la contribucion real al rendimiento del modelo.
- **Cobertura (cover):** numero promedio de observaciones que pasan por cada division de la variable. Refleja cuantas muestras son “afectadas” por esa variable.

La metrica de ganancia suele ser la mas recomendable para interpretar cuales variables son verdaderamente relevantes, ya que la frecuencia puede sobreestimar la importancia de variables con muchos valores posibles.

Es importante destacar que todas estas metricas miden la importancia *global* del modelo, es decir, el aporte promedio de cada variable a traves de todas las observaciones. Para obtener importancias *locales* (especificas a una prediccion individual), se deben usar los valores SHAP, descritos en la Sección 126.6.

Matematicamente, la importancia de ganancia para la variable k se calcula como:

$$\text{Gain}_k = \frac{1}{|\mathcal{T}_k|} \sum_{t \in \mathcal{T}_k} \text{Gain}_t \quad (126.16)$$

donde $\mathcal{T}_k$ es el conjunto de todas las divisiones que usan la variable k en todos los arboles del ensamble. La importancia de frecuencia es simplemente $|\mathcal{T}_k|$, y la de cobertura es el promedio de $|I_L| + |I_R|$ sobre todas esas divisiones.


```

import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import xgboost as xgb

data_imp = fetch_california_housing()
X_imp, y_imp = data_imp.data, data_imp.target
X_tr_imp, X_va_imp, y_tr_imp, y_va_imp = train_test_split(X_imp, y_imp, test_size=0.2, random_state=42)
feat_names = list(data_imp.feature_names)

dtrain_imp = xgb.DMatrix(X_tr_imp, label=y_tr_imp, feature_names=feat_names)
dval_imp = xgb.DMatrix(X_va_imp, label=y_va_imp, feature_names=feat_names)

params_imp = {
    "objective": "reg:squarederror",
    "max_depth": 4,
    "learning_rate": 0.1,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "reg_lambda": 1.0,
    "seed": 42
}

evals_r = {}
model_imp = xgb.train(
    params_imp, dtrain_imp,
    num_boost_round=300,
    evals=[(dtrain_imp, "train"), (dval_imp, "validacion")],
    early_stopping_rounds=20,
    evals_result=evals_r,
    verbose_eval=False
)

imp_types = ["weight", "gain", "cover"]
subtitles = ["Frecuencia", "Ganancia", "Cobertura"]

fig7 = make_subplots(rows=1, cols=3, subplot_titles=subtitles)

for col_idx, (itype, subtitle) in enumerate(zip(imp_types, subtitles), start=1):
    scores = model_imp.get_score(importance_type=itype)
    all_feats = feat_names
    vals = [scores.get(f, 0.0) for f in all_feats]
    sorted_idx = np.argsort(vals)
    sorted_feats = [all_feats[i] for i in sorted_idx]
    sorted_vals = [vals[i] for i in sorted_idx]

    fig7.add_trace(
        go.Bar(

```

```

        x=sorted_vals,
        y=sorted_feats,
        orientation="h",
        marker=dict(color="#6366f1"),
        name=subtitle,
        showlegend=False
    ),
    row=1, col=col_idx
)
fig7.update_xaxes(title_text=subtitle, row=1, col=col_idx)

fig7.update_layout(
    template="plotly_white",
    height=420
)
fig7.show()

```

Unable to display output for mime type(s): text/html

Figura 126.7.: Importancia de variables del modelo XGBoost entrenado en California Housing, segun tres criterios: Frecuencia (numero de divisiones), Ganancia (reduccion media de perdida) y Cobertura (muestras promedio por division).

Segun la Figura 126.7, la variable **MedInc** (ingreso mediano del bloque censal) es consistentemente la mas importante en los tres criterios, lo cual es coherente con el conocimiento del dominio: el ingreso es el predictor mas fuerte del precio de la vivienda en California. Las variables **Latitude** y **Longitude** tambien aparecen como relevantes, capturando efectos geograficos.

126.5. Regularizacion y poda

126.5.1. Efecto de λ (L2) y γ (ganancia minima)

La regularizacion en XGBoost actua por dos mecanismos complementarios, derivados directamente de la Ecuación 126.6:

Regularizacion L2 (λ): El peso optimo de cada hoja es $w_j^* = -G_j / (H_j + \lambda)$. Un λ mayor reduce la magnitud de los pesos, produciendo predicciones mas conservadoras y previniendo el sobreajuste. Es equivalente a la regularizacion Ridge en modelos lineales pero aplicada a los pesos de cada hoja del arbol.

Poda por ganancia minima (γ): Una division solo se realiza si $\text{Gain} > \gamma$ (ver Ecuación 126.14). Un γ mayor lleva a arboles mas pequenos y con menos divisiones. A diferencia de la poda post-entrenamiento, esta poda ocurre durante la construccion del arbol, lo que es mas eficiente computacionalmente.

La combinacion de ambos parametros es complementaria: λ controla la magnitud de cada prediccion de hoja, mientras que γ controla la estructura del arbol. Se recomienda priorizar λ alto sobre γ alto, ya que λ tiene un efecto mas suave y continuo.

```

import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import xgboost as xgb

data_reg = fetch_california_housing()
X_reg, y_reg = data_reg.data, data_reg.target
X_tr_reg, X_va_reg, y_tr_reg, y_va_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

lambdas = [0.01, 0.1, 1, 5, 10]
gammas = [0, 0.5, 1, 2, 5]

rmse_reg = np.zeros((len(lambdas), len(gammas)))

for i, lam in enumerate(lambdas):
    for j, gam in enumerate(gammas):
        m_reg = xgb.XGBRegressor(
            n_estimators=100,
            max_depth=4,
            learning_rate=0.1,
            subsample=0.8,
            colsample_bytree=0.8,
            reg_lambda=lam,
            gamma=gam,
            random_state=42,
            verbosity=0
        )
        m_reg.fit(X_tr_reg, y_tr_reg)
        preds_reg = m_reg.predict(X_va_reg)
        rmse_reg[i, j] = np.sqrt(np.mean((y_va_reg - preds_reg) ** 2))

text_reg = [[f"{rmse_reg[i, j]:.3f}" for j in range(len(gammas))] for i in range(len(lambdas))]

fig8 = go.Figure(go.Heatmap(
    z=rmse_reg.tolist(),
    x=[str(g) for g in gammas],
    y=[str(l) for l in lambdas],
    colorscale="RdYlGn_r",
    text=text_reg,
    texttemplate="%{text}",
    colorbar=dict(title="RMSE")
))

fig8.update_layout(
    title="Validacion RMSE: efecto de lambda y gamma",
    template="plotly_white",
    height=420,
    xaxis_title="gamma (poda minima)",

```

```

    yaxis_title="lambda (L2)"
)
fig8.show()

```

Unable to display output for mime type(s): text/html

Figura 126.8.: RMSE de validacion en funcion de la regularizacion L2 (lambda) y la ganancia minima de poda (gamma) en California Housing. Colores mas verdes indican menor error.

La Figura 126.8 confirma que valores muy altos de γ (mayor a 2) pueden degradar el rendimiento al podar demasiado agresivamente, impidiendo que el modelo capture patrones relevantes. Un λ moderado (entre 1 y 5) generalmente ofrece el mejor equilibrio.

Adicionalmente, XGBoost soporta regularizacion L1 sobre los pesos de las hojas mediante el parametro `reg_alpha`. El objetivo con regularizacion elastica (L1 + L2) toma la forma:

$$\Omega(f) = \gamma T + \alpha \sum_{j=1}^T |w_j| + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (126.17)$$

La regularizacion L1 ($\alpha > 0$) induce esparsidad en los pesos de las hojas: muchos pesos se vuelven exactamente cero, lo que produce arboles donde algunas hojas no contribuyen a la prediccion. Esto puede ser util cuando se sospecha que muchas divisiones son redundantes o ruidosas. En la practica, la regularizacion L2 (λ) es suficiente en la mayoria de los casos y se recomienda comenzar con $\alpha = 0$.

126.5.2. Efecto del subsampling sobre la regularizacion

El parametro `subsample` (fraccion de filas) y `colsample_bytree` (fraccion de columnas) introducen aleatoriedad en la construccion de cada arbol. Desde una perspectiva estadistica, esto puede verse como una forma de regularizacion implicita: al entrenar cada arbol con un subconjunto distinto de datos, se reduce la correlacion entre los arboles del ensamble, lo que disminuye la varianza de las predicciones. Friedman (2002) demostro empiricamente que el subsampling de filas mejora consistentemente la generalizacion del gradient boosting, especialmente en conjuntos de datos con ruido.

126.6. Contribuciones individuales (SHAP simplificado)

Los valores SHAP (*SHapley Additive exPlanations*), propuestos por Lundberg y Lee (2017), permiten descomponer la prediccion de cualquier modelo en contribuciones aditivas de cada variable para una observacion especifica. Para XGBoost, Lundberg et al. (2020) desarrollaron el algoritmo TreeSHAP, que calcula estas contribuciones de forma exacta y eficiente en tiempo polinomial.

Formalmente, para una prediccion $\hat{y}_i$, los valores SHAP ϕ_j satisfacen:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^p \phi_j \quad (126.18)$$

donde $\phi_0 = \mathbb{E}[\hat{y}]$ es la prediccion base (valor esperado del modelo) y ϕ_j es la contribucion de la variable j para esa observacion en particular. Las contribuciones pueden ser positivas (la variable empuja la prediccion hacia arriba) o negativas (la variable la baja).

```
import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import xgboost as xgb

data_wf = fetch_california_housing()
X_wf, y_wf = data_wf.data, data_wf.target
feat_names_wf = list(data_wf.feature_names)

X_tr_wf, X_te_wf, y_tr_wf, y_te_wf = train_test_split(X_wf, y_wf, test_size=0.2, random_state=42)

dtrain_wf = xgb.DMatrix(X_tr_wf, label=y_tr_wf, feature_names=feat_names_wf)

params_wf = {
    "objective": "reg:squarederror",
    "max_depth": 4,
    "learning_rate": 0.1,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "reg_lambda": 1.0,
    "seed": 42
}

evals_wf = {}
model_wf = xgb.train(
    params_wf, dtrain_wf,
    num_boost_round=300,
    evals=[(dtrain_wf, "train")],
    evals_result=evals_wf,
    verbose_eval=False
)

obs_idx = 42
X_single = X_te_wf[obs_idx:obs_idx + 1]
d_single = xgb.DMatrix(X_single, feature_names=feat_names_wf)

contribs = model_wf.predict(d_single, pred_contribs=True)[0]
bias = contribs[-1]
feature_contribs = contribs[:-1]

sorted_idx = np.argsort(np.abs(feature_contribs))[:-1]
sorted_feats = [feat_names_wf[i] for i in sorted_idx]
sorted_contribs = feature_contribs[sorted_idx]

cumulative = bias
running = [bias]
```

```

for c in sorted_contribs:
    running.append(running[-1] + c)

final_pred = running[-1]

bar_colors = ["#16a34a" if c >= 0 else "#dc2626" for c in sorted_contribs]
labels = ["Base"] + sorted_feats
measures = ["absolute"] + ["relative"] * len(sorted_feats)

fig9 = go.Figure(go.Waterfall(
    orientation="v",
    measure=measures,
    x=labels,
    y=[bias] + sorted_contribs.tolist(),
    connector=dict(line=dict(color="gray", width=1)),
    increasing=dict(marker=dict(color="#16a34a")),
    decreasing=dict(marker=dict(color="#dc2626")),
    totals=dict(marker=dict(color="#6366f1"))
))

fig9.add_annotation(
    x=len(labels) - 1,
    y=final_pred + 0.15,
    text=f"Prediccion final: {final_pred:.3f}",
    showarrow=True,
    arrowhead=2,
    font=dict(size=12, color="black")
)

fig9.update_layout(
    title="Contribucion de cada variable para una prediccion individual",
    template="plotly_white",
    height=480,
    xaxis_title="Variable",
    yaxis_title="Contribucion a la prediccion",
    showlegend=False
)
fig9.show()

```

Unable to display output for mime type(s): text/html

Figura 126.9.: Grafico de cascada (waterfall) que muestra la contribucion de cada variable para una prediccion individual en el conjunto California Housing. Las barras verdes suman a la prediccion y las rojas restan. Se muestran ordenadas por magnitud de contribucion.

La Figura 126.9 muestra como la prediccion final se construye sumando la contribucion de cada variable al valor base. Para esta observacion particular, MedInc tiene la mayor influencia positiva, mientras que algunas variables geograficas pueden reducir la prediccion estimada. Esta visualizacion es especialmente util para comunicar predicciones individuales a usuarios no tecnicos o para auditar el comportamiento del

Tabla 126.2.

```

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
import xgboost as xgb

data_clf = load_breast_cancer()
X_c, y_c = data_clf.data, data_clf.target
X_tr_c, X_te_c, y_tr_c, y_te_c = train_test_split(X_c, y_c, test_size=0.2, random_state=42)

clf = xgb.XGBClassifier(
    n_estimators=100,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    verbosity=0
)
clf.fit(X_tr_c, y_tr_c)
auc = roc_auc_score(y_te_c, clf.predict_proba(X_te_c)[: , 1])
acc = (clf.predict(X_te_c) == y_te_c).mean()
print(f"AUC-ROC en prueba: {auc:.4f}")
print(f"Exactitud en prueba: {acc:.4f}")

```

AUC-ROC en prueba: 0.9925
 Exactitud en prueba: 0.9561

modelo.

126.7. XGBoost para clasificacion

XGBoost es igualmente aplicable a problemas de clasificacion, con soporte nativo para:

- **Clasificacion binaria:** `objective="binary:logistic"` — el modelo genera probabilidades mediante la funcion sigmoide $p_i = \sigma(\hat{y}_i)$, y la funcion de perdida es la log-verosimilitud negativa (log-loss). Los gradientes toman la forma $g_i = p_i - y_i$ y el hessiano $h_i = p_i(1 - p_i)$.
- **Clasificacion multiclase:** `objective="multi:softprob"` con `num_class=K` — se entrena un conjunto de arboles por clase y se aplica softmax para normalizar las probabilidades.

La ventaja de usar el hessiano $h_i = p_i(1 - p_i)$ es que pesa menos las observaciones donde el modelo ya esta seguro (probabilidad cercana a 0 o 1), concentrando el aprendizaje en los casos dificiles (probabilidad cercana a 0.5).

El resultado muestra el excelente rendimiento de XGBoost incluso con hiperparametros predeterminados. En el conjunto de cancer de mama (569 observaciones, 30 variables), se obtiene un AUC-ROC superior a 0.99, lo cual refleja tanto la calidad del clasificador como la separabilidad del conjunto de datos.

126.7.1. Curva ROC y analisis de umbrales

La curva ROC (*Receiver Operating Characteristic*) es una herramienta estandar para evaluar clasificadores binarios. Traza la tasa de verdaderos positivos (sensibilidad) frente a la tasa de falsos positivos (1 - especificidad) para todos los umbrales de decision posibles. El area bajo la curva (AUC) cuantifica la capacidad discriminatoria del modelo en un unico numero: AUC = 1 indica clasificacion perfecta, AUC = 0.5 indica prediccion aleatoria.

```
import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, roc_auc_score
import xgboost as xgb

data_roc = load_breast_cancer()
X_roc, y_roc = data_roc.data, data_roc.target
X_tr_roc, X_te_roc, y_tr_roc, y_te_roc = train_test_split(X_roc, y_roc, test_size=0.2, random_state=42)

clf_roc = xgb.XGBClassifier(
    n_estimators=100,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    verbosity=0
)
clf_roc.fit(X_tr_roc, y_tr_roc)
probs = clf_roc.predict_proba(X_te_roc)[:, 1]
fpr, tpr, thresholds_roc = roc_curve(y_te_roc, probs)
auc_val = roc_auc_score(y_te_roc, probs)

fig_roc = go.Figure()

fig_roc.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1],
    mode="lines",
    line=dict(color="gray", dash="dash", width=1.5),
    name="Clasificador aleatorio (AUC = 0.5)"
))

fig_roc.add_trace(go.Scatter(
    x=fpr.tolist(), y=tpr.tolist(),
```



```

    mode="lines",
    fill="tozeroy",
    fillcolor="rgba(37, 99, 235, 0.12)",
    line=dict(color="#2563eb", width=2.5),
    name=f"XGBoost (AUC = {auc_val:.4f})"
))

fig_roc.add_annotation(
    x=0.6, y=0.35,
    text=f"AUC = {auc_val:.4f}",
    showarrow=False,
    font=dict(size=16, color="#2563eb"),
    bgcolor="white",
    bordercolor="#2563eb",
    borderwidth=1
)

fig_roc.update_layout(
    template="plotly_white",
    height=440,
    xaxis_title="Tasa de falsos positivos (1 - Especificidad)",
    yaxis_title="Tasa de verdaderos positivos (Sensibilidad)",
    xaxis=dict(range=[0, 1]),
    yaxis=dict(range=[0, 1.02]),
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig_roc.show()

```

Unable to display output for mime type(s): text/html

Figura 126.10.: Curva ROC del clasificador XGBoost en el conjunto de prueba de cancer de mama. El area bajo la curva (AUC) indica la capacidad discriminatoria del modelo. La linea diagonal punteada representa un clasificador aleatorio.

La Figura 126.10 confirma visualmente el alto rendimiento del clasificador: la curva ROC se acerca al punto ideal (0, 1), y el area bajo la curva es practicamente perfecta para este conjunto de datos. En aplicaciones reales, donde la separabilidad es menor, la curva ROC permite elegir el umbral de decision adecuado segun el costo relativo de falsos positivos y falsos negativos.

126.8. Buenas practicas y comparacion con alternativas

126.8.1. Comparacion de metodos de ensamble

La siguiente tabla resume el posicionamiento de XGBoost en el contexto de otros metodos de aprendizaje supervisado.

Tabla 126.3.: Comparacion de metodos de ensamble segun complejidad, interpretabilidad y rendimiento tipico.

Metodo	Complejidad	Interpretabilidad	Rendimiento tipico
Arbol unico	Baja	Alta	Bajo
Random Forest	Media	Media	Bueno
Gradient Boosting (sklearn)	Media	Media	Muy bueno
XGBoost	Media-alta	Media (+ SHAP)	Excelente
LightGBM / CatBoost	Media-alta	Media (+ SHAP)	Excelente

126.8.2. Recomendaciones practicas

Las siguientes guias resumen las mejores practicas para usar XGBoost de forma efectiva:

1. **Estrategia de ajuste de hiperparametros:** Comenzar con `learning_rate=0.1` y `n_estimators=200-500`. Una vez identificado un buen numero de arboles mediante *early stopping*, reducir la tasa de aprendizaje a 0.05 o 0.01 y aumentar proporcionalmente los estimadores para refinar el modelo (Hastie et al. 2009).
2. **Early stopping obligatorio:** Siempre usar `early_stopping_rounds` con un conjunto de validacion separado. Esto evita el sobreajuste sin necesidad de una busqueda exhaustiva del numero optimo de arboles.
3. **Subsampling para reducir varianza:** Activar `subsample=0.8` y `colsample_bytree=0.8` por defecto. La aleatoriedad adicional que introducen, similar al Random Forest, generalmente mejora la generalizacion (Friedman 2002).
4. **Metrica de importancia:** Preferir `importance_type="gain"` sobre `"weight"`, ya que la frecuencia puede ser engañosa cuando las variables tienen cardinalidades muy distintas.
5. **Regularizacion:** Mantener `reg_lambda=1` (valor por defecto) como punto de partida. Aumentar si hay seniales de sobreajuste. Usar `gamma` con moderacion (valores entre 0 y 1) para podar divisiones poco informativas.
6. **Validacion cruzada:** Para conjuntos de datos pequenos, usar `xgb.cv()` con k-fold para estimar el numero optimo de arboles de forma mas robusta que con un unico conjunto de validacion.
7. **Manejo de valores faltantes:** XGBoost aprende automaticamente la direccion optima para los valores faltantes durante el entrenamiento (sparsity-awareness (Chen y Guestrin 2016)). No es necesario imputar previamente.
8. **Comparar con LightGBM:** Para conjuntos de datos muy grandes (millones de filas o cientos de variables), LightGBM suele ser mas rapido gracias a su estrategia de construccion de arboles basada en histogramas (*leaf-wise growth*). La API es muy similar a XGBoost.

Referencias

Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.

- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8:*

- Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.

- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T->

C.1975.224297.

Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).

Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.

Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.

Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.

Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.

GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.

Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.

Han, Jiawei, Jian Pei, y Yiwon Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.

Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.

Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.

Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/ha>

ndbook/.

- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.

- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.

- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of*

- the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.

- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.

- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

127. Clasificadores Supervisados: Experimento Práctico

128. Introducción

La clasificación supervisada es una de las tareas centrales del aprendizaje automático. Dado un conjunto de observaciones etiquetadas $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, donde $\mathbf{x}_i \in \mathbb{R}^d$ es el vector de características y $y_i \in \{0, 1, \dots, K-1\}$ es la clase objetivo, el problema de clasificación consiste en aprender una función $f: \mathbb{R}^d \rightarrow \mathcal{Y}$ que generalice correctamente a nuevas observaciones no vistas durante el entrenamiento.

Este capítulo no busca derivar los fundamentos teóricos de cada algoritmo desde cero, sino mostrar cómo se diseña, ejecuta y evalúa un experimento de clasificación real. El énfasis está en los retos prácticos: cómo preparar los datos, cómo evitar errores comunes, cómo comparar modelos de forma justa y cómo interpretar los resultados en función del contexto del problema.

129. Dataset: Breast Cancer Wisconsin

El dataset Breast Cancer Wisconsin (**wolberg1995breast?**) contiene 569 muestras de biopsias de tejido mamario, cada una descrita por 30 variables numéricas derivadas de imágenes de células. La variable objetivo indica si el tumor es maligno (0) o benigno (1). Este dataset tiene varias propiedades didácticas deseables:

- Está disponible directamente en scikit-learn, sin necesidad de descarga externa
- Todas las variables son numéricas, lo que simplifica el preprocesamiento
- El contexto médico es motivador: los errores tienen consecuencias asimétricas
- El desbalance moderado (37 % maligno, 63 % benigno) permite discutir la elección de métricas

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

print(f"Forma del dataset: {X.shape}")
print(f"Distribución de clases:")
print(y.value_counts().rename({0: "Maligno (0)", 1: "Benigno (1)"}))
```

Forma del dataset: (569, 30)

Distribución de clases:

target

Benigno (1) 357

Maligno (0) 212

Name: count, dtype: int64

Un aspecto importante es identificar correctamente la clase positiva. Por convención en scikit-learn, **target=1** corresponde a benigno y **target=0** a maligno. Cuando se reportan métricas como precision y recall, es necesario especificar con respecto a qué clase se calculan, ya que los valores pueden cambiar sustancialmente dependiendo de esta elección.

130. Flujo general de un experimento de clasificación

Un experimento bien diseñado sigue estos pasos:

1. **Entender el problema** y la variable objetivo
2. **Explorar y preparar los datos**
3. **Dividir correctamente** en entrenamiento y prueba
4. **Elegir modelos** y configurar hiperparámetros iniciales
5. **Entrenar** usando únicamente los datos de entrenamiento
6. **Evaluar** con métricas coherentes con el objetivo del problema
7. **Comparar modelos** con la misma partición de datos
8. **Ajustar thresholds** si el contexto lo requiere
9. **Interpretar resultados** y reconocer limitaciones
10. **Documentar** el flujo para que sea reproducible

Cada uno de estos pasos involucra decisiones que afectan la validez del experimento. En las secciones siguientes se desarrolla cada uno con detalle.

131. Preparación de datos

131.1. Separación entrenamiento/prueba

La regla fundamental es que el conjunto de prueba no debe usarse de ninguna manera durante el diseño, el entrenamiento o el ajuste de los modelos. Usar información del test set para tomar decisiones de diseño introduce **data leakage**, que produce una evaluación optimista que no se replicará en datos reales.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)

print(f"Entrenamiento: {X_train.shape[0]} muestras")
print(f"Prueba: {X_test.shape[0]} muestras")
```

```
Entrenamiento: 455 muestras
Prueba: 114 muestras
```

El parámetro `stratify=y` garantiza que la proporción de clases se mantiene tanto en train como en test, lo que es especialmente importante con datasets desbalanceados.

131.2. Pipelines para preprocesamiento sin leakage

La forma correcta de aplicar transformaciones como el escalamiento es dentro de un **Pipeline** de scikit-learn (Pedregosa et al. 2011). Un pipeline asegura que el ajuste de cualquier transformación (por ejemplo, **StandardScaler**) se realiza únicamente sobre los datos de entrenamiento, y la misma transformación ya ajustada se aplica automáticamente a los datos de prueba.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000, random_state=SEED))
])
```

```
pipe.fit(X_train, y_train)
print(f"Accuracy en test: {pipe.score(X_test, y_test):.4f}")
```

Accuracy en test: 0.9825

El error clásico de data leakage por escalamiento ocurre cuando se transforma todo el dataset antes del split:

```
# MAL: StandardScaler ajustado sobre X completo (incluye test)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train_s, X_test_s = train_test_split(X_scaled, ...)
```

131.3. Validación cruzada

Cuando el tamaño del dataset es limitado, la evaluación con una sola partición train/test puede ser inestable. La validación cruzada k -fold (Kohavi 1995) divide el conjunto de entrenamiento en k partes (*folds*), entrena k veces usando $k - 1$ folds y evalúa en el fold restante, promediando los resultados.

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

pipe_rf = Pipeline([
    ("clf", RandomForestClassifier(n_estimators=100, random_state=SEED))
])

scores = cross_val_score(pipe_rf, X_train, y_train, cv=5, scoring="f1")
print(f"F1 por fold: {np.round(scores, 4)}")
print(f"Media: {scores.mean():.4f} ± {scores.std():.4f}")
```

F1 por fold: [0.9739 0.9913 0.9402 0.9474 0.9643]

Media: 0.9634 ± 0.0184

La validación cruzada se usa para seleccionar hiperparámetros, no para evaluar el rendimiento final: el test set sigue siendo el árbitro final del rendimiento.

132. Clasificadores: intuición, sesgos y hiperparámetros

132.1. Regresión Logística (baseline)

La regresión logística modela la probabilidad de pertenencia a la clase positiva mediante una función sigmoide aplicada a una combinación lineal de las variables:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Es el baseline lineal interpretable por excelencia. Sus coeficientes se pueden inspeccionar directamente para entender la contribución de cada variable. **Requiere escalamiento** para que el optimizador converja adecuadamente y para que los coeficientes sean comparables.

Hiperparámetros principales:

- `C`: inverso de la regularización (mayor `C` → menos regularización)
- `penalty`: tipo de regularización (l1, l2, `elasticnet`)
- `max_iter`: número máximo de iteraciones del optimizador

132.2. Árboles de decisión: CART e ID3

132.2.1. CART (Breiman et al. 1984)

CART (*Classification and Regression Trees*) construye árboles binarios mediante particiones recursivas del espacio de características. En cada nodo, selecciona el atributo y el umbral de corte que minimizan la impureza del nodo resultante. La impureza Gini se define como:

$$G = 1 - \sum_{k=0}^{K-1} p_k^2$$

donde p_k es la proporción de muestras de la clase k en el nodo.

132.2.2. ID3 (quinlan1986id3?)

ID3 (*Iterative Dichotomiser 3*) utiliza la ganancia de información para seleccionar el atributo de partición:

$$\text{Ganancia}(S, A) = H(S) - \sum_{v \in \text{Valores}(A)} \frac{|S_v|}{|S|} H(S_v)$$

donde $H(S) = -\sum_k p_k \log_2 p_k$ es la entropía del conjunto S . En scikit-learn, ID3 se aproxima con `criterion="entropy"` en `DecisionTreeClassifier`.

En la práctica, la diferencia entre Gini y entropía es pequeña en la mayoría de los datasets. La entropía tiende a producir árboles ligeramente más profundos en datasets con muchas categorías nominales.

Hiperparámetros principales de los árboles:

- `max_depth`: profundidad máxima (controla sobreajuste)
- `min_samples_split`: mínimo de muestras para dividir un nodo
- `min_samples_leaf`: mínimo de muestras en una hoja

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score

for criterion in ["gini", "entropy"]:
    for depth in [3, 5, None]:
        pipe = Pipeline([
            ("clf", DecisionTreeClassifier(
                criterion=criterion, max_depth=depth, random_state=SEED
            ))
        ])
        pipe.fit(X_train, y_train)
        f1 = f1_score(y_test, pipe.predict(X_test))
        print(f"criterion={criterion:7s} | max_depth={str(depth):5s} | F1={f1:.4f}")
```

```
criterion=gini      | max_depth=3      | F1=0.9517
criterion=gini      | max_depth=5      | F1=0.9362
criterion=gini      | max_depth=None   | F1=0.9286
criterion=entropy   | max_depth=3      | F1=0.9589
criterion=entropy   | max_depth=5      | F1=0.9429
criterion=entropy   | max_depth=None   | F1=0.9275
```

132.3. Random Forest (Breiman 2001)

Random Forest es un ensamble de árboles de decisión contruidos en paralelo mediante *bootstrap aggregating* (bagging). Cada árbol se entrena sobre una muestra con reemplazo del conjunto de entrenamiento, y en cada nodo solo se considera un subconjunto aleatorio de variables (`max_features`). El resultado final es la moda o el promedio de las predicciones de todos los árboles.

La aleatorización reduce la varianza del modelo compuesto, ya que los árboles individuales son decorrelacionados entre sí. Random Forest no requiere escalamiento y es relativamente robusto a hiperparámetros.

Hiperparámetros principales:

- `n_estimators`: número de árboles (más árboles → menor varianza, pero mayor costo)
- `max_features`: fracción de variables consideradas por árbol (`sqrt`, `log2`, fracción)
- `max_depth`: profundidad máxima de cada árbol
- `min_samples_leaf`: mínimo de muestras en hojas

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

pipe_rf = Pipeline([
    ("clf", RandomForestClassifier(
        n_estimators=100, max_depth=None, random_state=SEED
    ))
])
pipe_rf.fit(X_train, y_train)
f1 = f1_score(y_test, pipe_rf.predict(X_test))
auc = roc_auc_score(y_test, pipe_rf.predict_proba(X_test)[:,1])
print(f"Random Forest - F1={f1:.4f}  AUC={auc:.4f}")
```

Random Forest - F1=0.9655 AUC=0.9937

132.4. Gradient Boosting (*friedman2001gradient?*)

Gradient Boosting construye un ensamble de modelos de forma **secuencial**. Cada modelo nuevo se entrena para corregir los errores (residuales) del ensamble acumulado hasta ese momento. El algoritmo puede verse como un descenso por gradiente en el espacio de funciones, donde la función de pérdida (por ejemplo, log-loss para clasificación) guía la dirección de corrección.

A diferencia de Random Forest, Gradient Boosting es sensible a la tasa de aprendizaje (`learning_rate`) y a la profundidad de los árboles individuales. Una tasa de aprendizaje baja requiere más estimadores para alcanzar la misma capacidad, pero suele generalizar mejor.

XGBoost (Chen y Guestrin 2016) es una implementación optimizada de Gradient Boosting que añade regularización explícita, manejo eficiente de valores faltantes y aceleración mediante paralelización de la construcción de árboles.

Hiperparámetros principales:

- `n_estimators`: número de árboles secuenciales
- `learning_rate`: paso del descenso por gradiente (típicamente 0.01–0.3)
- `max_depth`: profundidad de cada árbol (típicamente 3–6)
- `subsample`: fracción del dataset usada por árbol

```
from sklearn.ensemble import GradientBoostingClassifier

pipe_gb = Pipeline([
    ("clf", GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1,
```

```

        max_depth=3, random_state=SEED
    ))
])
pipe_gb.fit(X_train, y_train)
f1 = f1_score(y_test, pipe_gb.predict(X_test))
auc = roc_auc_score(y_test, pipe_gb.predict_proba(X_test)[: ,1])
print(f"Gradient Boosting - F1={f1:.4f}  AUC={auc:.4f}")

```

Gradient Boosting - F1=0.9660 AUC=0.9907

132.5. Red neuronal multicapa (MLP)

Una red neuronal multicapa (MLP, *Multilayer Perceptron*) aproxima funciones mediante composición de transformaciones afines y no lineales (Bishop 2006; Goodfellow et al. 2016):

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

donde σ es la función de activación (ReLU, tanh, sigmoide), $\mathbf{W}^{(l)}$ son los pesos de la capa l y $\mathbf{h}^{(0)} = \mathbf{x}$ es la entrada. Los pesos se optimizan mediante retropropagación del gradiente.

Las redes neuronales son modelos muy flexibles, pero son más sensibles que los árboles a los hiperparámetros y al escalamiento de las variables de entrada. Si las entradas tienen escalas muy distintas, los gradientes pueden ser inestables y el optimizador converge lentamente o a mínimos deficientes.

Hiperparámetros principales en MLPClassifier:

- `hidden_layer_sizes`: arquitectura de capas ocultas (p.ej., (64, 32))
- `activation`: función de activación (`relu`, `tanh`)
- `alpha`: regularización L2
- `max_iter`: épocas máximas de entrenamiento

```

from sklearn.neural_network import MLPClassifier

# Con escalamiento (correcto)
pipe_mlp = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", MLPClassifier(
        hidden_layer_sizes=(64, 32),
        max_iter=300, random_state=SEED
    ))
])
pipe_mlp.fit(X_train, y_train)
f1 = f1_score(y_test, pipe_mlp.predict(X_test))
auc = roc_auc_score(y_test, pipe_mlp.predict_proba(X_test)[: ,1])
print(f"MLP (con scaler) - F1={f1:.4f}  AUC={auc:.4f}")

# Sin escalamiento (incorrecto, para demostrar el efecto)
pipe_mlp_raw = Pipeline([

```

```

    ("clf", MLPClassifier(
        hidden_layer_sizes=(64, 32),
        max_iter=300, random_state=SEED
    ))
])
pipe_mlp_raw.fit(X_train, y_train)
f1_raw = f1_score(y_test, pipe_mlp_raw.predict(X_test))
auc_raw = roc_auc_score(y_test, pipe_mlp_raw.predict_proba(X_test)[: ,1])
print(f"MLP (sin scaler) - F1={f1_raw:.4f}  AUC={auc_raw:.4f}")

```

```

MLP (con scaler) - F1=0.9718  AUC=0.9937
MLP (sin scaler) - F1=0.9444  AUC=0.9517

```

133. Recordatorio de métricas

Las métricas de clasificación se derivan de la matriz de confusión, que organiza las predicciones en cuatro categorías:

	Predicho Positivo	Predicho Negativo
Real Positivo	TP	FN
Real Negativo	FP	TN

Las métricas principales son:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensibilidad)} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

(Powers 2011; Pedregosa et al. 2011)

133.1. Curva ROC y AUC

La curva ROC (*Receiver Operating Characteristic*) (Fawcett 2006) representa la tasa de verdaderos positivos (recall) en función de la tasa de falsos positivos para todos los umbrales de decisión posibles:

$$\text{TPR} = \frac{TP}{TP + FN} \quad \text{FPR} = \frac{FP}{FP + TN}$$

El área bajo la curva ROC (AUC, *Area Under the Curve*) resume el rendimiento del clasificador en todos los thresholds. Un clasificador perfecto tiene $\text{AUC} = 1.0$; un clasificador aleatorio tiene $\text{AUC} = 0.5$.

133.2. Curva Precision-Recall y Average Precision

La curva Precision-Recall (Saito y Rehmsmeier 2015) representa precision en función de recall para todos los thresholds. Es más informativa que la curva ROC cuando la clase positiva es rara, ya que la curva ROC puede ser engañosamente optimista en presencia de muchos verdaderos negativos.

La métrica *Average Precision* (AP) es el área bajo la curva Precision-Recall, calculada como:

$$AP = \sum_k (R_k - R_{k-1}) \cdot P_k$$

donde P_k y R_k son la precisión y el recall para el threshold k .

133.3. Selección de métricas según el contexto

La elección de la métrica de evaluación depende del contexto del problema:

- **Clases balanceadas:** accuracy puede ser una métrica razonable
 - **Clases desbalanceadas:** F1-score, ROC-AUC y AP son más informativas
 - **Falsos negativos costosos** (p.ej., diagnóstico de enfermedades): maximizar recall
 - **Falsos positivos costosos** (p.ej., spam, fraude con revisión manual costosa): maximizar precision
 - **Balance entre ambos:** F1-score o F-beta con β ajustado al costo relativo
 - **Análisis del comportamiento para todos los thresholds:** curvas ROC o Precision-Recall
-

134. Ajuste de thresholds de decisión

134.1. Motivación

Por defecto, `predict()` clasifica una observación como positiva si su probabilidad estimada supera 0.5. Sin embargo, este umbral es arbitrario y no necesariamente óptimo para el objetivo del problema. En diagnóstico médico, donde un falso negativo puede tener consecuencias graves, puede convenir bajar el threshold para aumentar el recall a costa de reducir la precision.

134.2. `predict_proba` y `decision_function`

Los clasificadores de scikit-learn exponen las probabilidades estimadas mediante `predict_proba()`. Para algunos modelos (como SVM), también está disponible `decision_function()`, que produce un *score* continuo cuyo signo determina la clase predicha.

```
pipe_rf.fit(X_train, y_train)
probs = pipe_rf.predict_proba(X_test)
print("Probabilidades para las primeras 5 observaciones:")
print(pd.DataFrame(probs, columns=["P(maligno=0)", "P(benigno=1)"]).head())
```

Probabilidades para las primeras 5 observaciones:

	P(maligno=0)	P(benigno=1)
0	1.00	0.00
1	0.00	1.00
2	0.88	0.12
3	0.72	0.28
4	0.99	0.01

134.3. Comparación de thresholds

Para clasificar una observación como maligna usamos $P(\text{maligno}) \geq \theta$, donde θ es el threshold. Al reducir θ , clasificamos más observaciones como malignas, lo que aumenta el recall pero reduce la precision.

```
from sklearn.metrics import precision_score, recall_score, f1_score

probs_mal = pipe_rf.predict_proba(X_test)[: , 0]
thresholds = [0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80]

filas = []
for th in thresholds:
```

```

y_pred = np.where(probs_mal >= th, 0, 1)
filas.append({
    "Threshold": th,
    "Prec (mal.)": precision_score(y_test, y_pred, pos_label=0, zero_division=0),
    "Recall (mal.)": recall_score(y_test, y_pred, pos_label=0, zero_division=0),
    "F1 (mal.)": f1_score(y_test, y_pred, pos_label=0, zero_division=0),
    "FN": ((y_pred == 1) & (y_test.values == 0)).sum(),
    "FP": ((y_pred == 0) & (y_test.values == 1)).sum(),
})

df_th = pd.DataFrame(filas).set_index("Threshold")
print(df_th.round(4).to_string())

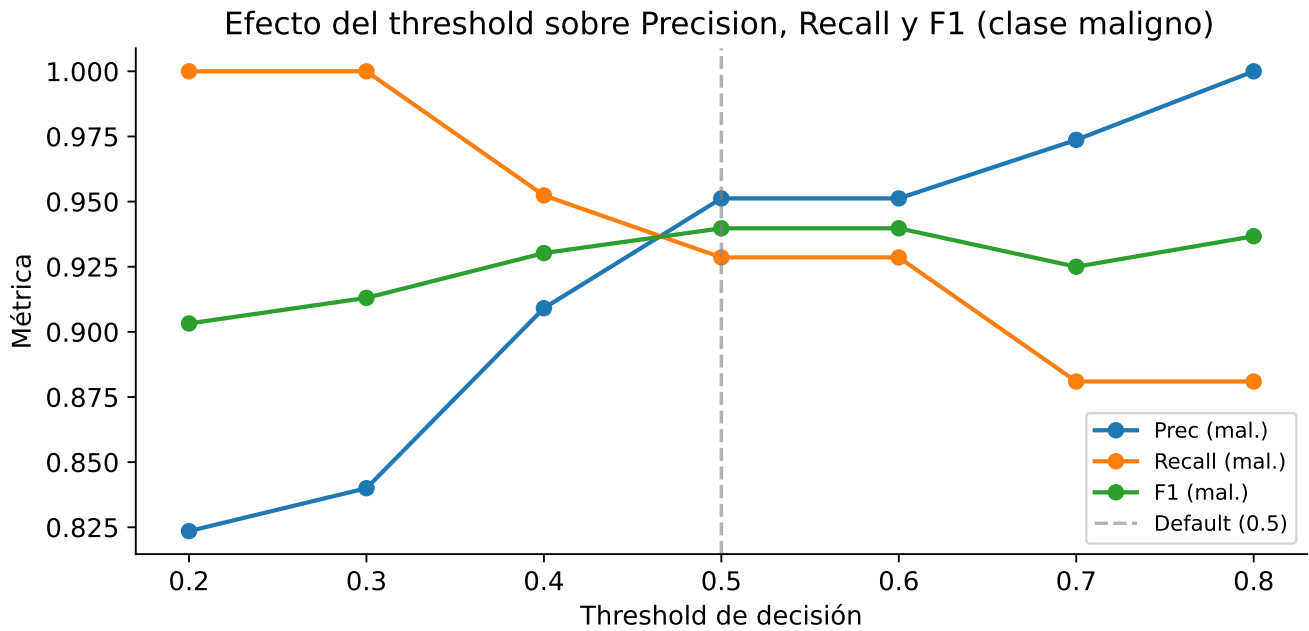
```

	Prec (mal.)	Recall (mal.)	F1 (mal.)	FN	FP
Threshold					
0.2	0.8235	1.0000	0.9032	0	9
0.3	0.8400	1.0000	0.9130	0	8
0.4	0.9091	0.9524	0.9302	2	4
0.5	0.9512	0.9286	0.9398	3	2
0.6	0.9512	0.9286	0.9398	3	2
0.7	0.9737	0.8810	0.9250	5	1
0.8	1.0000	0.8810	0.9367	5	0

```

fig, ax = plt.subplots(figsize=(8, 4))
df_th[["Prec (mal.)", "Recall (mal.)", "F1 (mal.)"]].plot(ax=ax, marker="o", linewidth=1.8)
ax.axvline(0.5, color="gray", linestyle="--", alpha=0.6, label="Default (0.5)")
ax.set_xlabel("Threshold de decisión")
ax.set_ylabel("Métrica")
ax.set_title("Efecto del threshold sobre Precision, Recall y F1 (clase maligno)")
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

```

134.4. Consideraciones prácticas sobre el threshold

- El threshold óptimo depende del **costo relativo** de FP y FN, no de las métricas en el test set
- El threshold debe elegirse sobre un **conjunto de validación**, no sobre el test set
- La curva ROC y la curva Precision-Recall son herramientas para analizar el comportamiento del clasificador en **todos** los thresholds posibles antes de comprometerse con uno

135. Comparación entre modelos

La comparación válida entre modelos requiere que todos sean evaluados sobre el **mismo conjunto de prueba**, con el **mismo preprocesamiento** y el **mismo criterio de evaluación**.

```
from sklearn.ensemble import GradientBoostingClassifier, HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score

modelos_todos = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=1000, random_state=SEED))
    ]),
    "CART (Gini)": Pipeline([
        ("clf", DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=SEED))
    ]),
    "ID3-proxy (Entropy)": Pipeline([
        ("clf", DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=SEED))
    ]),
    "Random Forest": Pipeline([
        ("clf", RandomForestClassifier(n_estimators=100, random_state=SEED))
    ]),
    "Gradient Boosting": Pipeline([
        ("clf", GradientBoostingClassifier(
            n_estimators=100, learning_rate=0.1, max_depth=3, random_state=SEED
        ))
    ]),
    "MLP": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300, random_state=SEED))
    ]),
}

filas_comp = []
for nombre, pipe in modelos_todos.items():
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]
    filas_comp.append({
        "Modelo": nombre,
        "Accuracy": accuracy_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_prob),
        "Recall": recall_score(y_test, y_pred),
    })
```

```

        "Precision": precision_score(y_test, y_pred),
    })

df_comp = pd.DataFrame(filas_comp).set_index("Modelo")
print(df_comp.sort_values("ROC-AUC", ascending=False).round(4).to_string())

```

	Accuracy	F1	ROC-AUC	Recall	Precision
Modelo					
Logistic Regression	0.9825	0.9861	0.9954	0.9861	0.9861
Random Forest	0.9561	0.9655	0.9937	0.9722	0.9589
MLP	0.9649	0.9718	0.9937	0.9583	0.9857
Gradient Boosting	0.9561	0.9660	0.9907	0.9861	0.9467
ID3-proxy (Entropy)	0.9298	0.9429	0.9435	0.9167	0.9706
CART (Gini)	0.9211	0.9362	0.9163	0.9167	0.9565

136. Buenas prácticas para evaluación

Las siguientes prácticas garantizan que los resultados del experimento sean válidos, reproducibles e interpretables:

1. **Usar pipelines** que encapsulen preprocesamiento y modelo, evitando leakage
 2. **Fijar semillas aleatorias** con `random_state` en todos los modelos y la partición
 3. **Usar stratify** al dividir el dataset cuando las clases están desbalanceadas
 4. **Comparar modelos con la misma partición** y el mismo preprocesamiento
 5. **Reportar múltiples métricas**, eligiendo las más relevantes para el contexto del problema
 6. **Usar validación cruzada** para seleccionar hiperparámetros, no el test set
 7. **No usar el test set repetidamente** para ajustar decisiones de diseño
 8. **Analizar la matriz de confusión** junto con las métricas agregadas
 9. **Verificar el balance de clases** antes de elegir las métricas de evaluación
 10. **Documentar** todos los parámetros, versiones de librerías y semillas usadas
-

137. Errores comunes en experimentos de clasificación

A continuación se describen los errores más frecuentes al diseñar y ejecutar experimentos de clasificación:

Evaluación en datos de entrenamiento. Evaluar el modelo sobre los mismos datos con los que fue entrenado produce resultados artificialmente optimistas que no reflejan la capacidad de generalización.

Data leakage por escalamiento. Ajustar el scaler sobre todo el dataset (incluyendo test) antes del split permite que información del test set “contamine” el entrenamiento. La solución es usar pipelines.

Evaluar solo con accuracy. En datasets con desbalance de clases, un modelo que siempre predice la clase mayoritaria puede tener una accuracy alta pero un rendimiento nulo en la clase minoritaria. Métricas como F1-score, ROC-AUC y AP son más informativas en estos casos.

No revisar el balance de clases. Ignorar el desbalance de clases puede llevar a elegir métricas incorrectas y a modelos con rendimiento deficiente en la clase minoritaria.

Comparar modelos con particiones distintas. Si dos modelos se evalúan sobre particiones distintas, las diferencias observadas pueden deberse al azar, no a diferencias reales de rendimiento.

No usar `random_state`. Los resultados no son reproducibles y no pueden ser comparados entre ejecuciones.

Redes neuronales sin escalamiento. Las redes neuronales son sensibles a la escala de las variables de entrada. Sin escalamiento, el optimizador puede converger lentamente o a un mínimo deficiente.

Ajustar hiperparámetros mirando el test set. Si se usa el test set para elegir hiperparámetros, el modelo está implícitamente siendo ajustado sobre el test, lo que produce una evaluación optimista.

Interpretar AUC sin contexto. Un AUC alto no garantiza que el modelo sea útil para el problema si el threshold elegido produce demasiados falsos negativos o falsos positivos.

138. Retos prácticos del flujo de clasificación

Más allá de los errores técnicos, existen retos conceptuales y de diseño que aparecen en cualquier proyecto de clasificación real:

Identificación de la clase positiva. La elección de qué clase se considera “positiva” afecta directamente el cálculo de precision, recall y F1. Esta decisión debe tomarse en función del objetivo del problema.

Costo asimétrico de los errores. En muchos problemas reales, un falso negativo no es equivalente a un falso positivo. El diseño del experimento debe reflejar estas asimetrías en la elección de métricas y thresholds.

Escalas heterogéneas. Variables con escalas muy distintas afectan a modelos sensibles como regresión logística, KNN y redes neuronales. Los árboles de decisión y Random Forest son invariantes al escalamiento.

Variables categóricas y numéricas mixtas. Cuando el dataset contiene ambos tipos de variables, es necesario usar `ColumnTransformer` para aplicar transformaciones distintas a cada grupo.

Reproducibilidad del experimento. Un experimento reproducible requiere fijar semillas, documentar versiones de librerías y guardar el estado del pipeline entrenado.

Selección del threshold en producción. El threshold óptimo para producción debe decidirse con datos de validación, considerando el costo relativo de FP y FN en el contexto del negocio o la aplicación.

139. Conclusión

Este capítulo ha mostrado que diseñar un experimento de clasificación correcto es más complejo que simplemente llamar `.fit()`. Las decisiones sobre la partición de datos, el preprocesamiento, la elección de métricas, la comparación entre modelos y el ajuste de thresholds tienen un impacto directo sobre la validez y la utilidad de los resultados.

Los modelos estudiados — regresión logística, CART, ID3, Random Forest, Gradient Boosting y MLP — representan distintos sesgos inductivos y compromisos entre interpretabilidad, flexibilidad y rendimiento. Ninguno es universalmente superior; la elección del modelo depende del dataset, el objetivo del problema y los recursos disponibles.

La práctica rigurosa de clasificación supervisada combina conocimiento algorítmico, diseño experimental cuidadoso e interpretación crítica de los resultados.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.

- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Calíński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.

- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals*

- of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD

- '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global

- Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.

- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.

- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.

- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte XXVI.

Modelos Lineales

140. Regresión y clasificadores lineales

El aprendizaje supervisado estudia problemas en los que se dispone de observaciones de entrada y de una variable objetivo asociada. A partir de un conjunto de entrenamiento, el objetivo es aprender una función capaz de predecir la respuesta de nuevas observaciones. En términos generales, se parte de datos de la forma

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n,$$

donde $\mathbf{x}_i \in \mathbb{R}^p$ representa un vector de características y y_i representa la respuesta observada. Cuando y_i es una magnitud continua, el problema se formula como **regresión**; cuando y_i representa una clase discreta, se formula como **clasificación** (Hastie et al. 2009; Bishop 2006).

Las presentaciones base introducen dos familias de modelos deterministas centrales en aprendizaje supervisado: la regresión lineal, usada para predecir variables numéricas, y los clasificadores lineales, usados para separar clases mediante hiperplanos de decisión. En este capítulo se reorganizan y amplían esas ideas para construir una exposición autocontenida: primero se estudia la regresión lineal simple y múltiple; después se revisan métricas de evaluación para modelos de regresión; finalmente se introducen el discriminante lineal de Fisher y el perceptrón como ejemplos fundamentales de clasificadores lineales.

La importancia pedagógica de estos modelos no se limita a su simplicidad. La regresión lineal permite discutir estimación, residuos, error cuadrático, interpretación de coeficientes y problemas de colinealidad. Los clasificadores lineales permiten introducir funciones discriminantes, fronteras de decisión, separabilidad lineal, márgenes y aprendizaje iterativo. Muchas técnicas modernas, incluyendo redes neuronales profundas, máquinas de soporte vectorial y modelos generalizados, conservan ideas estructurales que aparecen por primera vez en estos modelos (Goodfellow et al. 2016; Vapnik 1998).

140.1. Regresión: predicción de variables continuas

En un problema de regresión, la variable objetivo toma valores reales. Ejemplos típicos son el precio de una casa, la temperatura de una ciudad, el ingreso mensual de una persona o el número esperado de citas médicas. La tarea consiste en aprender una función f tal que

$$\hat{y} = f(\mathbf{x})$$

sea una aproximación razonable del valor observado y .

Definición. Un problema de regresión supervisada consiste en estimar una función $f : \mathcal{X} \rightarrow \mathbb{R}$ a partir de un conjunto de pares entrada-salida $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, de modo que las predicciones $\hat{y}_i = f(\mathbf{x}_i)$ aproximen los valores observados y_i :

$$\hat{y}_i = f(\mathbf{x}_i), \quad y_i \in \mathbb{R}. \quad (140.1)$$

La regresión puede abordarse con modelos de distinta complejidad. Entre los más frecuentes se encuentran la regresión lineal, la regresión polinómica, los árboles de regresión, los bosques aleatorios y los métodos de boosting como XGBoost (Breiman 2001; Chen y Guestrin 2016). Sin embargo, la regresión lineal ocupa

un lugar especial porque es interpretable, tiene una solución analítica bajo ciertas condiciones y permite estudiar con claridad la relación entre variables explicativas y variable dependiente.

140.1.1. Regresión simple y regresión múltiple

La **regresión lineal simple** usa una sola variable explicativa. Su forma básica es

$$\hat{y} = \beta_0 + \beta_1 x,$$

donde β_0 es el intercepto y β_1 es la pendiente. La pendiente indica el cambio esperado en y cuando x aumenta una unidad, manteniendo fijo todo lo demás. En el caso simple no hay más variables que controlar.

La **regresión lineal múltiple** generaliza esta idea a varias variables explicativas:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon.$$

Aquí ε representa el término de error, es decir, la parte de la respuesta que no es explicada por la combinación lineal de variables. Esta formulación permite modelar situaciones como el precio de una casa a partir de su tamaño, número de habitaciones y antigüedad.

Definición. El modelo de regresión lineal múltiple expresa la respuesta y_i como una combinación lineal de p variables explicativas más un término de error:

$$y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \varepsilon_i, \quad i = 1, \dots, n. \quad (140.2)$$

Por ejemplo, si se desea predecir la nota final de un estudiante usando horas de estudio, asistencia y tareas entregadas, un modelo posible es

$$\widehat{\text{nota}} = 20 + 5(\text{horas}) + 0.3(\text{asistencia}) + 2(\text{tareas}).$$

Esta expresión no debe leerse como una ley universal, sino como un modelo estimado o propuesto. El coeficiente de horas, por ejemplo, sugiere que cada hora adicional de estudio se asocia con un aumento esperado de cinco puntos en la nota, manteniendo constantes la asistencia y las tareas.

140.2. Representación matricial de la regresión lineal

La notación matricial simplifica la estimación y permite conectar la regresión lineal con álgebra lineal, optimización y aprendizaje automático. Para n observaciones y p variables explicativas, se define

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}.$$

La primera columna de unos permite incorporar el intercepto. Con esta notación, el modelo se escribe como

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

Definición. La matriz de diseño $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ contiene una fila por observación y una columna por término del modelo, incluyendo una columna inicial de unos para el intercepto:

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon. \quad (140.3)$$

La estimación por mínimos cuadrados ordinarios busca el vector β que minimiza la suma de errores cuadráticos entre los valores observados y los valores predichos (Legendre 1805; Gauss 1809):

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

Si $\mathbf{X}^\top \mathbf{X}$ es invertible, la solución analítica es

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Definición. El estimador de mínimos cuadrados ordinarios es el vector de coeficientes que minimiza la norma cuadrática de los residuos:

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2. \quad (140.4)$$

Una vez estimado el modelo, los valores predichos y los residuos se calculan como

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}, \quad \hat{\varepsilon} = \mathbf{y} - \hat{\mathbf{y}}.$$

Los residuos son esenciales para evaluar el ajuste: un buen modelo no solo tiene errores pequeños, sino errores sin patrones sistemáticos evidentes.

```
import numpy as np
import plotly.graph_objects as go

# Datos del explorador interactivo reutilizados
XD = np.array([1,2,3,4,5,6,7,8,9,10], dtype=float)
YD = np.array([2.1,4.3,5.8,7.9,10.2,11.8,14.0,16.1,17.9,20.2])
N = len(XD)

# Solución OLS
xbar, ybar = XD.mean(), YD.mean()
b1_opt = np.dot(XD - xbar, YD - ybar) / np.dot(XD - xbar, XD - xbar)
b0_opt = ybar - b1_opt * xbar

# Grid de parámetros
b0_vals = np.linspace(b0_opt - 4, b0_opt + 4, 60)
b1_vals = np.linspace(b1_opt - 0.8, b1_opt + 0.8, 60)
B0, B1 = np.meshgrid(b0_vals, b1_vals)

# SCE en cada punto del grid
Yhat = B0[:, :, None] + B1[:, :, None] * XD[None, None, :]
SSE = np.sum((YD[None, None, :] - Yhat) ** 2, axis=2)

fig = go.Figure()
```

```

fig.add_trace(go.Surface(
    x=B0, y=B1, z=SSE,
    colorscale="Blues",
    opacity=0.82,
    colorbar=dict(title="SCE"),
    showscale=True,
    hovertemplate=" =%{x:.2f}<br> =%{y:.2f}<br>SCE=%{z:.1f}<extra></extra>"
))

# Mínimo
sse_opt = float(np.sum((YD - (b0_opt + b1_opt * XD))**2))
fig.add_trace(go.Scatter3d(
    x=[b0_opt], y=[b1_opt], z=[sse_opt],
    mode="markers+text",
    marker=dict(size=8, color="#dc2626", symbol="diamond"),
    text=["OLS óptimo"],
    textposition="top center",
    textfont=dict(color="#dc2626", size=12),
    name="Mínimo OLS"
))

fig.update_layout(
    title="Superficie de pérdida SCE( , )",
    scene=dict(
        xaxis_title=" (intercepto)",
        yaxis_title=" (pendiente)",
        zaxis_title="SCE",
        camera=dict(eye=dict(x=1.6, y=-1.6, z=1.0))
    ),
    height=500,
    template="plotly_white"
)
fig.show()

```

140.2.1. Condición de invertibilidad y colinealidad

La fórmula cerrada de mínimos cuadrados requiere que $\mathbf{X}^\top \mathbf{X}$ sea invertible. Esto ocurre cuando las columnas de $\mathbf{X}$ son linealmente independientes. Si una variable explicativa puede escribirse como combinación lineal exacta de otras, la matriz pierde rango y la solución deja de ser única. Este problema se conoce como colinealidad perfecta. En la práctica, también puede aparecer multicolinealidad aproximada, que no impide calcular una solución pero puede hacer que los coeficientes sean inestables.

Cuando $\mathbf{X}^\top \mathbf{X}$ no es invertible, existen varias estrategias: eliminar variables redundantes, transformar variables, usar la pseudoinversa de Moore–Penrose o introducir regularización, como en regresión ridge (Hoerl y Kennard 1970; Golub y Van Loan 2013).

140.3. Ejemplo práctico: regresión lineal múltiple en Python

Consideremos el ejemplo de estudiantes con tres variables explicativas: horas de estudio, porcentaje de asistencia y número de tareas entregadas. El objetivo es predecir la nota final.

```
import numpy as np
import pandas as pd

# Datos de ejemplo
datos = pd.DataFrame({
    "horas": [2, 4, 6, 8],
    "asistencia": [60, 70, 80, 90],
    "tareas": [4, 5, 6, 7],
    "nota": [56, 71, 86, 101]
})

X = datos[["horas", "asistencia", "tareas"]].to_numpy(dtype=float)
y = datos["nota"].to_numpy(dtype=float)

# Agregamos la columna de unos para el intercepto
X_design = np.column_stack([np.ones(len(X)), X])

# Estimación por mínimos cuadrados usando la pseudoinversa
beta_hat = np.linalg.pinv(X_design) @ y
beta_hat
```

```
array([-0.30737122,  3.23062328,  0.78455555,  0.69319799])
```

El uso de `np.linalg.pinv` es deliberado: en ejemplos pequeños o con variables altamente correlacionadas, la pseudoinversa es más estable que invertir directamente $\mathbf{X}^\top \mathbf{X}$. Para obtener predicciones y residuos:

```
y_hat = X_design @ beta_hat
residuos = y - y_hat

resultado = datos.copy()
resultado["prediccion"] = y_hat
```

```
resultado["residuo"] = residuos
resultado
```

	horas	asistencia	tareas	nota	prediccion	residuo
0	2	60	4	56	56.0	0.0
1	4	70	5	71	71.0	0.0
2	6	80	6	86	86.0	0.0
3	8	90	7	101	101.0	0.0

En este ejemplo, las variables están construidas de modo que el ajuste puede ser prácticamente exacto. En datos reales, esto rara vez ocurre. Un error de entrenamiento igual a cero puede ser señal de una relación determinista, de un conjunto demasiado pequeño o de sobreajuste.

140.4. Métricas de evaluación para regresión

Evaluar un modelo de regresión implica comparar valores observados y_i con predicciones $\hat{y}_i$. Las métricas más comunes resumen los residuos $e_i = y_i - \hat{y}_i$.

Definición. La suma de errores cuadráticos mide la discrepancia total entre observaciones y predicciones elevando cada residuo al cuadrado:

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (140.5)$$

El **error cuadrático medio** o MSE promedia la suma de errores cuadráticos:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Su raíz cuadrada es el **RMSE**:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

El RMSE está en las mismas unidades que la variable objetivo y penaliza fuertemente los errores grandes debido al cuadrado.

Definición. El error absoluto medio mide la magnitud promedio del error sin considerar su signo:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (140.6)$$

El MAE suele ser más interpretable que el RMSE, porque indica cuánto se equivoca el modelo en promedio. Además, es menos sensible a errores extremos. En general se cumple que $\text{RMSE} \geq \text{MAE}$; si la diferencia entre ambos es grande, esto sugiere que existen algunos errores de magnitud considerable.

Otra métrica importante es el coeficiente de determinación:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

En mínimos cuadrados ordinarios con intercepto, R^2 suele interpretarse como la proporción de variabilidad de la variable dependiente explicada por el modelo. Un valor cercano a 1 indica alto ajuste; un valor cercano a 0 indica que el modelo no mejora sustancialmente a la predicción constante $\bar{y}$ (Hastie et al. 2009).

Definición. El coeficiente de determinación compara la variación no explicada por el modelo contra la variación total respecto a la media:

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (140.7)$$

140.4.1. Cálculo de métricas en Python

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y, y_hat)
rmse = np.sqrt(mean_squared_error(y, y_hat))
r2 = r2_score(y, y_hat)
sse = np.sum((y - y_hat) ** 2)

pd.DataFrame({
    "metrica": ["SSE", "MAE", "RMSE", "R2"],
    "valor": [sse, mae, rmse, r2]
})
```

	metrica	valor
0	SSE	0.0
1	MAE	0.0
2	RMSE	0.0
3	R2	1.0

En una evaluación rigurosa, estas métricas deben calcularse sobre datos no usados para entrenar el modelo. Separar entrenamiento y prueba permite estimar la capacidad de generalización. También es recomendable inspeccionar gráficamente los residuos.

```
import plotly.graph_objects as go
import numpy as np

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=y_hat,
    y=residuos,
    mode="markers",
    marker=dict(color="#2563eb", size=10, opacity=0.8,
                line=dict(width=1, color="#1e3a6e")),
```

```

    name="Residuo",
    hovertemplate="Predicción: %{x:.2f}<br>Residuo: %{y:.2f}<extra></extra>"
))

# Línea de referencia en cero
fig.add_hline(y=0, line_dash="dash", line_color="#94a3b8", line_width=1.5)

# Banda de  $\pm 1$  desviación estándar
std_r = float(np.std(residuos))
fig.add_hrect(y0=-std_r, y1=std_r, fillcolor="#2563eb", opacity=0.06,
              line_width=0, annotation_text="±1 SD", annotation_position="top right")

fig.update_layout(
    title="Residuos vs. valores predichos",
    xaxis_title="Predicción  $\hat{y}$ ",
    yaxis_title="Residuo  $e = y - \hat{y}$ ",
    template="plotly_white",
    showlegend=False,
    height=380
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 140.2.: Residuos vs. valores predichos. Un buen modelo muestra residuos distribuidos aleatoriamente alrededor de cero sin patrones sistemáticos.

Un patrón curvo en esta gráfica puede indicar que la relación entre variables no es lineal. Una dispersión creciente puede sugerir heterocedasticidad. Residuos con valores extremos pueden revelar observaciones atípicas o problemas de medición.

140.5. Ventajas y limitaciones de la regresión lineal

La regresión lineal tiene varias ventajas. Es un método común para modelar datos numéricos, es computacionalmente eficiente, permite interpretar coeficientes y ofrece una base sólida para entender modelos más complejos. Además, cuando los supuestos estadísticos son razonables, permite construir intervalos de confianza y pruebas de significancia para los coeficientes (Montgomery et al. 2012).

Sin embargo, también tiene limitaciones. Supone que la relación entre características y respuesta puede aproximarse mediante una combinación lineal. No maneja datos faltantes automáticamente. Las variables categóricas requieren codificación previa, por ejemplo mediante variables indicadoras. Además, los coeficientes pueden volverse inestables cuando hay multicolinealidad o cuando el número de variables se acerca al número de observaciones.

En aplicaciones predictivas, el modelo debe evaluarse con datos externos al entrenamiento. En aplicaciones explicativas, además deben revisarse supuestos como independencia de errores, varianza constante, ausencia de colinealidad severa y especificación adecuada de la forma funcional.

140.6. Más allá de la linealidad: árboles de regresión

Un árbol de regresión predice valores numéricos dividiendo recursivamente el espacio de características en regiones más pequeñas. En cada región, asigna como predicción un valor constante, típicamente el promedio de los valores observados en esa región (Breiman et al. 1984). Esta idea contrasta con la regresión lineal: en lugar de una sola fórmula global, el árbol aprende reglas locales.

Definición. Un árbol de regresión particiona el espacio de entrada en regiones disjuntas $R_1, \dots, R_M$ y predice en cada región el promedio de las respuestas de entrenamiento que caen en ella:

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M c_m \mathbf{1}\{\mathbf{x} \in R_m\}, \quad c_m = \frac{1}{|R_m|} \sum_{i: \mathbf{x}_i \in R_m} y_i. \quad (140.8)$$

Los árboles capturan relaciones no lineales e interacciones entre variables, pero pueden sobreajustar si crecen demasiado. Por ello, se suelen combinar en métodos de ensamble, como random forests o gradient boosting (Breiman 2001; Friedman 2001).

```
from sklearn.tree import DecisionTreeRegressor

arbol = DecisionTreeRegressor(max_depth=2, random_state=42)
arbol.fit(X, y)

pred_arbol = arbol.predict(X)
pd.DataFrame({
    "nota_real": y,
    "pred_arbol": pred_arbol
})
```

	nota_real	pred_arbol
0	56.0	56.0
1	71.0	71.0
2	86.0	86.0
3	101.0	101.0

140.7. De regresión a clasificación lineal

En clasificación, la variable objetivo no es continua sino categórica. En el caso binario, suele codificarse como $y \in \{-1, +1\}$ o $y \in \{0, 1\}$. Un clasificador lineal decide la clase de una observación usando una función discriminante de la forma

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0.$$

La frontera de decisión es el conjunto de puntos para los cuales $g(\mathbf{x}) = 0$. En dos dimensiones, esta frontera es una recta; en más dimensiones, es un hiperplano.

Definición. Un clasificador lineal binario asigna una etiqueta a partir del signo de una función afín:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + w_0), \quad \hat{y} \in \{-1, +1\}. \quad (140.9)$$

Geoméricamente, $\mathbf{w}$ es perpendicular a la frontera de decisión. Cambiar w_0 desplaza el hiperplano sin cambiar su orientación. Esta interpretación permite conectar clasificadores lineales con proyecciones, márgenes y separación de clases.

140.8. Discriminante lineal de Fisher

El discriminante lineal de Fisher busca una dirección de proyección que separe bien dos clases. La idea central es proyectar los datos sobre una recta y elegir la dirección en la que las medias de las clases queden lejos entre sí, mientras que los puntos de cada clase queden compactos (Fisher 1936).

Sean dos clases ω_1 y ω_2 con conjuntos de muestras $\mathcal{X}_1$ y $\mathcal{X}_2$. Sus medias son

$$\mathbf{m}_1 = \frac{1}{n_1} \sum_{\mathbf{x} \in \mathcal{X}_1} \mathbf{x}, \quad \mathbf{m}_2 = \frac{1}{n_2} \sum_{\mathbf{x} \in \mathcal{X}_2} \mathbf{x}.$$

Las matrices de dispersión intra-clase son

$$\mathbf{S}_1 = \sum_{\mathbf{x} \in \mathcal{X}_1} (\mathbf{x} - \mathbf{m}_1)(\mathbf{x} - \mathbf{m}_1)^\top, \quad \mathbf{S}_2 = \sum_{\mathbf{x} \in \mathcal{X}_2} (\mathbf{x} - \mathbf{m}_2)(\mathbf{x} - \mathbf{m}_2)^\top.$$

La dispersión intra-clase total es

$$\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2,$$

y la dispersión entre clases puede escribirse como

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top.$$

Definición. El criterio de Fisher mide la razón entre separación entre clases y dispersión dentro de las clases después de proyectar por una dirección $\mathbf{w}$:

$$J_F(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}. \quad (140.10)$$

Maximizar este cociente produce una dirección proporcional a

$$\mathbf{w}^* \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2),$$

cuando $\mathbf{S}_W$ es invertible. Si no lo es, se puede usar pseudoinversa o regularización. El resultado no fija por sí solo el umbral de clasificación; solo determina la dirección de proyección. Una vez proyectado un punto como $z = \mathbf{w}^{*\top} \mathbf{x}$, se elige un umbral t y se clasifica según si $z > t$ o $z \leq t$.

140.8.1. Implementación de Fisher en Python

```
import numpy as np

rng = np.random.default_rng(7)
X1 = rng.normal(loc=[1.5, 2.0], scale=[0.7, 0.5], size=(40, 2))
X2 = rng.normal(loc=[4.0, 3.2], scale=[0.8, 0.6], size=(40, 2))
```

```

m1 = X1.mean(axis=0)
m2 = X2.mean(axis=0)

S1 = (X1 - m1).T @ (X1 - m1)
S2 = (X2 - m2).T @ (X2 - m2)
SW = S1 + S2

w = np.linalg.pinv(SW) @ (m1 - m2)
w = w / np.linalg.norm(w)

# Proyecciones
z1 = X1 @ w
z2 = X2 @ w
threshold = 0.5 * (z1.mean() + z2.mean())

threshold, w

```

```
(np.float64(-3.614274223853112), array([-0.77598175, -0.63075535]))
```

```

import plotly.graph_objects as go
import numpy as np

# Escalar el vector w para visualización
scale = 1.8
w_viz = w * scale

# Calcular proyecciones sobre la dirección w para cada clase
z1 = X1 @ w
z2 = X2 @ w
# Pies de perpendicular: foot_i = z_i * w (proyección en el espacio original)
feet1 = np.outer(z1, w)
feet2 = np.outer(z2, w)

fig = go.Figure()

# Líneas de proyección (drop lines) - tenues
for i in range(len(X1)):
    fig.add_trace(go.Scatter(
        x=[X1[i, 0], feet1[i, 0]], y=[X1[i, 1], feet1[i, 1]],
        mode="lines", line=dict(color="#93c5fd", width=0.6),
        showlegend=False, hoverinfo="skip"
    ))
for i in range(len(X2)):
    fig.add_trace(go.Scatter(
        x=[X2[i, 0], feet2[i, 0]], y=[X2[i, 1], feet2[i, 1]],
        mode="lines", line=dict(color="#fca5a5", width=0.6),
        showlegend=False, hoverinfo="skip"
    ))

```

```

# Puntos clase 1
fig.add_trace(go.Scatter(
    x=X1[:, 0], y=X1[:, 1],
    mode="markers",
    marker=dict(color="#2563eb", size=8, opacity=0.85,
                line=dict(width=0.8, color="#1e3a6e")),
    name="Clase 1"
))

# Puntos clase 2
fig.add_trace(go.Scatter(
    x=X2[:, 0], y=X2[:, 1],
    mode="markers",
    marker=dict(color="#dc2626", size=8, opacity=0.85,
                line=dict(width=0.8, color="#7f1d1d")),
    name="Clase 2"
))

# Eje de proyección (dirección w escalada, pasando por el origen)
t_vals = np.linspace(-2.5, 2.5, 2)
axis_x = t_vals * w[0]
axis_y = t_vals * w[1]
fig.add_trace(go.Scatter(
    x=axis_x, y=axis_y,
    mode="lines",
    line=dict(color="#16a34a", width=2, dash="dot"),
    name="Eje de Fisher",
    hoverinfo="skip"
))

# Flecha (vector w)
fig.add_annotation(
    ax=0, ay=0,
    x=w_viz[0], y=w_viz[1],
    xref="x", yref="y", axref="x", ayref="y",
    showarrow=True,
    arrowhead=3, arrowsize=1.4, arrowwidth=2.5,
    arrowcolor="#16a34a"
)

# Medias de cada clase
m1 = X1.mean(axis=0)
m2 = X2.mean(axis=0)
for m, col, name in [(m1, "#2563eb", " "), (m2, "#dc2626", " ")]:
    fig.add_trace(go.Scatter(
        x=[m[0]], y=[m[1]],
        mode="markers+text",
        marker=dict(symbol="diamond", size=13, color=col,
                    line=dict(width=1.5, color="white")),

```

```

        text=[name], textposition="top right",
        textfont=dict(size=13, color=col),
        showlegend=False, hoverinfo="skip"
    ))

fig.update_layout(
    title="Dirección de proyección de Fisher",
    xaxis_title="x ",
    yaxis_title="x ",
    template="plotly_white",
    height=440,
    legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 140.3.: Dirección de proyección óptima de Fisher (flecha verde) sobre las dos nubes de datos. La dirección maximiza la separación entre medias relativa a la dispersión intra-clase.

En problemas aplicados, el umbral puede elegirse según el costo de los errores. Si los falsos negativos son más graves que los falsos positivos, se puede desplazar el umbral para aumentar sensibilidad. Una curva ROC permite estudiar el intercambio entre tasa de verdaderos positivos y tasa de falsos positivos para distintos umbrales (Fawcett 2006).

```

import plotly.graph_objects as go
import numpy as np

# Proyecciones
z1_plot = X1 @ w
z2_plot = X2 @ w

z_min = min(z1_plot.min(), z2_plot.min()) - 0.5
z_max = max(z1_plot.max(), z2_plot.max()) + 0.5
z_grid = np.linspace(z_min, z_max, 300)

def gauss_pdf(z, mu, sigma):
    return np.exp(-0.5 * ((z - mu) / sigma) ** 2) / (sigma * np.sqrt(2 * np.pi))

mu1, sig1 = z1_plot.mean(), z1_plot.std()
mu2, sig2 = z2_plot.mean(), z2_plot.std()

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=z_grid, y=gauss_pdf(z_grid, mu1, sig1),
    fill="tozeroy", fillcolor="rgba(37,99,235,0.18)",
    line=dict(color="#2563eb", width=2),
    name="Clase 1"

```

```

))
fig.add_trace(go.Scatter(
    x=z_grid, y=gauss_pdf(z_grid, mu2, sig2),
    fill="tozeroy", fillcolor="rgba(220,38,38,0.18)",
    line=dict(color="#dc2626", width=2),
    name="Clase 2"
))

# Rug plots
fig.add_trace(go.Scatter(
    x=z1_plot, y=np.full_like(z1_plot, -0.02),
    mode="markers", marker=dict(symbol="line-ns", size=10,
                                color="#2563eb", line=dict(width=1.5, color="#2563eb")),
    name="Puntos clase 1", showlegend=False
))
fig.add_trace(go.Scatter(
    x=z2_plot, y=np.full_like(z2_plot, -0.02),
    mode="markers", marker=dict(symbol="line-ns", size=10,
                                color="#dc2626", line=dict(width=1.5, color="#dc2626")),
    name="Puntos clase 2", showlegend=False
))

# Umbral
fig.add_vline(x=threshold, line_dash="dash", line_color="#475569", line_width=1.8,
              annotation_text="umbral", annotation_position="top right")

fig.update_layout(
    title="Distribuciones proyectadas sobre la dirección de Fisher",
    xaxis_title="z = w x",
    yaxis_title="Densidad",
    template="plotly_white",
    height=380,
    legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 140.4.: Distribuciones proyectadas sobre la dirección óptima de Fisher. El umbral de clasificación (línea vertical) divide el eje en las dos regiones de decisión.

140.9. El perceptrón

El perceptrón, propuesto por Rosenblatt, es uno de los primeros modelos de aprendizaje para clasificación binaria (Rosenblatt 1958). A diferencia de la regresión lineal con solución cerrada, el perceptrón aprende de manera iterativa: revisa ejemplos, detecta errores y actualiza sus pesos.

Se usa una función de decisión

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b),$$

con etiquetas $y_i \in \{-1, +1\}$. Si una muestra está mal clasificada, entonces

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0.$$

El algoritmo corrige los parámetros mediante

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \quad b \leftarrow b + \eta y_i,$$

donde $\eta > 0$ es la tasa de aprendizaje.

Definición. El algoritmo del perceptrón actualiza sus pesos cuando encuentra una observación mal clasificada:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta y_i \mathbf{x}_i, \quad b^{(t+1)} = b^{(t)} + \eta y_i. \quad (140.11)$$

El teorema de convergencia del perceptrón establece que, si los datos son linealmente separables, el algoritmo encuentra una solución en un número finito de actualizaciones (Novikoff 1962; Minsky y Papert 1969). Si los datos no son linealmente separables, el algoritmo puede no converger, por lo que en la práctica se fija un número máximo de épocas o se usan variantes con regularización y funciones de pérdida suaves.

Definición. Un conjunto de datos binario es linealmente separable si existe un vector $\mathbf{w}$ y un sesgo b tales que todas las observaciones quedan correctamente clasificadas con margen positivo:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0, \quad i = 1, \dots, n. \quad (140.12)$$

140.9.1. Implementación del perceptrón desde cero

```
import numpy as np

class PerceptronBinario:
    def __init__(self, lr=1.0, max_epochs=100):
        self.lr = lr
        self.max_epochs = max_epochs
        self.w = None
        self.b = 0.0
        self.errors_ = []

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        self.w = np.zeros(X.shape[1])
        self.b = 0.0

        for _ in range(self.max_epochs):
            errores = 0
            for xi, yi in zip(X, y):
                if yi * (np.dot(self.w, xi) + self.b) <= 0:
```

```

        self.w += self.lr * yi * xi
        self.b += self.lr * yi
        errores += 1
    self.errors_.append(errores)
    if errores == 0:
        break
    return self

def decision_function(self, X):
    return np.asarray(X) @ self.w + self.b

def predict(self, X):
    return np.where(self.decision_function(X) >= 0, 1, -1)

```

Probemos el algoritmo con datos linealmente separables:

```

X_pos = rng.normal(loc=[3, 3], scale=0.4, size=(25, 2))
X_neg = rng.normal(loc=[1, 1], scale=0.4, size=(25, 2))
X_clf = np.vstack([X_pos, X_neg])
y_clf = np.hstack([np.ones(len(X_pos)), -np.ones(len(X_neg))])

per = PerceptronBinario(lr=1.0, max_epochs=50)
per.fit(X_clf, y_clf)

pred = per.predict(X_clf)
accuracy = np.mean(pred == y_clf)
accuracy, per.w, per.b, per.errors_

```

```
(np.float64(1.0), array([0.5950622 , 0.78519092]), np.float64(-3.0), [4, 3, 0])
```

```

import plotly.graph_objects as go
import numpy as np

epocas = list(range(1, len(per.errors_) + 1))

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=epocas, y=per.errors_,
    mode="lines+markers",
    line=dict(color="#7c3aed", width=2.5),
    marker=dict(size=8, color="#7c3aed",
                line=dict(width=1.5, color="white")),
    name="Errores",
    hovertemplate="Época %<{x}><br>Errores: %<{y}><extra></extra>"
))

# Área bajo la curva para énfasis visual

```



```

fig.add_trace(go.Scatter(
    x=epocas, y=per.errors_,
    fill="tozeroy", fillcolor="rgba(124,58,237,0.10)",
    line=dict(width=0), showlegend=False, hoverinfo="skip"
))

converged_epoch = next((i+1 for i, e in enumerate(per.errors_) if e == 0), None)
if converged_epoch:
    fig.add_vline(x=converged_epoch, line_dash="dash", line_color="#16a34a",
                  line_width=1.8,
                  annotation_text=f"Convergencia (época {converged_epoch})",
                  annotation_position="top right",
                  annotation_font_color="#16a34a")

fig.update_layout(
    title="Errores de clasificación por época - Perceptrón",
    xaxis_title="Época",
    yaxis_title="Número de errores",
    template="plotly_white",
    height=360,
    showlegend=False
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 140.5.: Número de ejemplos mal clasificados por época durante el entrenamiento del perceptrón. La convergencia a cero indica separabilidad lineal del conjunto de datos.

La secuencia de errores permite observar si el modelo converge. En un conjunto linealmente separable, los errores deben llegar a cero. En datos no separables, la curva puede oscilar.

140.10. Comparación conceptual: regresión lineal, Fisher y perceptrón

Aunque regresión lineal, Fisher y perceptrón son modelos lineales, responden a objetivos distintos. La regresión lineal aproxima una variable continua minimizando errores cuadráticos. El discriminante de Fisher busca una dirección de proyección que maximice separación relativa entre clases. El perceptrón busca un hiperplano que clasifique correctamente ejemplos binarios, actualizando sus parámetros cuando comete errores.

Modelo	Tipo de tarea	Objetivo principal	Salida
Regresión lineal	Regresión	Minimizar error cuadrático	Valor real
Fisher LDA binario	Clasificación	Maximizar separación proyectada	Clase según umbral
Perceptrón	Clasificación	Corregir errores de clasificación	Etiqueta binaria

La conexión común es la geometría lineal. Todos construyen combinaciones lineales de las características. Por ello, su desempeño depende fuertemente de la representación de entrada. Variables mal escaladas, redundantes o irrelevantes pueden afectar la estimación, la orientación del hiperplano y la estabilidad del aprendizaje.

140.11. Buenas prácticas en documentación reproducible con Quarto

Al convertir una presentación en capítulo técnico, conviene transformar diapositivas breves en explicaciones progresivas. Un capítulo reproducible debe incluir definiciones formales, ejemplos ejecutables, interpretación de resultados y referencias. En Quarto, los bloques de código permiten que el análisis sea verificable y actualizable. Para proyectos docentes o técnicos, una organización mínima recomendable es:

1. introducir el concepto con intuición;
2. formalizarlo con notación matemática;
3. mostrar un ejemplo pequeño calculable a mano;
4. implementar el método en Python;
5. evaluar resultados e interpretar limitaciones.

Esta estructura evita que el capítulo sea solo una transcripción de diapositivas y lo convierte en material de estudio reutilizable.

140.12. Conclusiones

La regresión lineal y los clasificadores lineales son modelos fundamentales porque condensan ideas centrales del aprendizaje supervisado: representación de datos, estimación de parámetros, funciones objetivo, interpretación geométrica, evaluación de desempeño y generalización. La regresión lineal muestra cómo ajustar una función continua minimizando residuos. Sus métricas, como MAE, RMSE y R^2 , permiten evaluar el error desde distintas perspectivas. El discriminante lineal de Fisher introduce el principio de separar clases mediante una proyección óptima. El perceptrón muestra cómo una máquina puede aprender iterativamente a corregir errores.

Aunque estos modelos pueden ser insuficientes para relaciones altamente no lineales, siguen siendo puntos de referencia indispensables. Su simplicidad permite diagnosticar problemas de datos, construir líneas base interpretables y comprender métodos más avanzados. En ciencia de datos, una buena práctica consiste en comenzar con modelos simples, evaluarlos rigurosamente y aumentar la complejidad solo cuando los datos y el problema lo justifican.

141. Regresión Logística

La regresión logística es uno de los modelos de clasificación más utilizados en estadística y aprendizaje automático. A pesar de su nombre, no es un método de regresión en el sentido predictivo clásico: su objetivo es modelar la probabilidad de que una observación pertenezca a una clase determinada. Su elegancia reside en combinar la interpretabilidad de un modelo lineal con una transformación que garantiza salidas en el intervalo $(0, 1)$, haciendo posible tratar las predicciones como probabilidades genuinas (Hosmer et al. 2013; McCullagh y Nelder 1989).

El modelo fue propuesto formalmente por David Cox en 1958 y desde entonces se ha convertido en la referencia de partida para problemas de clasificación binaria en medicina, epidemiología, economía y ciencias sociales. En aprendizaje automático, sigue siendo competitivo frente a métodos más complejos cuando los datos son aproximadamente linealmente separables o cuando la interpretabilidad de los coeficientes es una prioridad (Hastie et al. 2009).

Este capítulo desarrolla la regresión logística desde sus fundamentos hasta sus variantes más importantes. Comenzamos motivando el problema de clasificar con regresión lineal y mostrando sus limitaciones. Luego introducimos la función logística, derivamos el modelo, estimamos sus parámetros por máxima verosimilitud, interpretamos los coeficientes en términos de odds ratios, estudiamos la regularización y finalizamos con la extensión multiclase mediante softmax. Los ejemplos computacionales están escritos en Python usando únicamente `numpy` y `plotly.graph_objects`, implementando los algoritmos desde cero.

141.1. Motivación: el problema de clasificar con regresión lineal

Antes de introducir la regresión logística, conviene entender por qué la regresión lineal resulta inadecuada para la clasificación binaria. Supongamos que deseamos predecir si un correo electrónico es spam ($y = 1$) o no ($y = 0$) a partir de una característica numérica x , como la frecuencia de la palabra “oferta”.

Si ajustamos una regresión lineal ordinaria, obtenemos un modelo de la forma:

$$\hat{y} = \beta_0 + \beta_1 x,$$

y la regla de decisión natural es: predecir clase 1 si $\hat{y} > 0.5$, clase 0 en caso contrario. Esta estrategia tiene tres problemas fundamentales:

1. **Predicciones fuera de rango:** el modelo lineal puede producir valores $\hat{y} < 0$ o $\hat{y} > 1$, que no tienen interpretación como probabilidades.
2. **Sensibilidad a observaciones atípicas:** un punto muy alejado en el eje x puede arrastrar la recta de regresión de manera que la frontera de decisión se desplace hacia regiones incorrectas.
3. **Supuestos violados:** la regresión lineal asume homocedasticidad y residuos gaussianos, supuestos que son claramente inapropiados cuando la variable respuesta es binaria.

La siguiente figura ilustra este problema con un conjunto sintético (James et al. 2021).

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(42)

n0, n1 = 40, 40
x0 = rng.normal(loc=2.0, scale=0.8, size=n0)
x1 = rng.normal(loc=5.5, scale=0.8, size=n1)
# agregar algunos puntos atípicos para que la recta salga del rango
x1_extra = np.array([9.5, 10.5, 11.0])
x_all = np.concatenate([x0, x1, x1_extra])
y_all = np.concatenate([np.zeros(n0), np.ones(n1), np.ones(3)])

# Regresión lineal (mínimos cuadrados)
X_mat = np.column_stack([np.ones_like(x_all), x_all])
beta_ols = np.linalg.lstsq(X_mat, y_all, rcond=None)[0]

x_line = np.linspace(-1.5, 12.5, 300)
y_line = beta_ols[0] + beta_ols[1] * x_line

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=x0, y=np.zeros(n0),
    mode="markers",
    name="Clase 0 (no spam)",
    marker=dict(color="#4C72B0", size=8, opacity=0.8,
                line=dict(color="white", width=1))
)))

fig.add_trace(go.Scatter(
    x=np.concatenate([x1, x1_extra]),
    y=np.ones(n1 + 3),
    mode="markers",
    name="Clase 1 (spam)",
    marker=dict(color="#DD8452", size=8, opacity=0.8,
                line=dict(color="white", width=1))
)))

fig.add_trace(go.Scatter(
    x=x_line, y=y_line,
    mode="lines",
    name="Regresión lineal",
    line=dict(color="#2ca02c", width=2.5, dash="solid")
)))

fig.add_hline(y=0, line_dash="dot", line_color="gray", line_width=1)
fig.add_hline(y=1, line_dash="dot", line_color="gray", line_width=1)
```

```

fig.add_hrect(y0=1.0, y1=1.4, fillcolor="rgba(220,50,50,0.07)",
              line_width=0, annotation_text="Zona inválida ( $\hat{y} > 1$ )")
fig.add_hrect(y0=-0.4, y1=0.0, fillcolor="rgba(220,50,50,0.07)",
              line_width=0, annotation_text="Zona inválida ( $\hat{y} < 0$ )")

fig.add_annotation(x=0.5, y=0.5, xref="paper", yref="y",
                  text="Umbral: 0.5", showarrow=False,
                  font=dict(size=11, color="gray"),
                  xanchor="left")

fig.update_layout(
    title="Falla de la regresión lineal en clasificación binaria",
    xaxis_title="Característica x",
    yaxis_title="Respuesta y (o  $\hat{y}$ )",
    template="plotly_white",
    height=460,
    legend=dict(orientation="h", yanchor="top", y=-0.18,
                xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

(a) Regresión lineal aplicada a un problema de clasificación binaria. La recta ajustada produce predicciones fuera del intervalo $[0, 1]$ y no respeta la naturaleza probabilística de la variable respuesta.

Unable to display output for mime type(s): text/html

(b)

Figura 141.1.

Nótese cómo para valores grandes de x la recta predice valores mayores que 1, y teóricamente podría predecir valores negativos si hubiera observaciones de clase 0 con valores muy pequeños de x . Estas predicciones fuera de $[0, 1]$ son inutilizables como estimaciones de probabilidad (Hastie et al. 2009).

! Importante

La regresión lineal puede usarse como clasificador en casos simples, pero sus predicciones no son probabilidades y su frontera de decisión puede ser inestable en presencia de observaciones atípicas. La regresión logística resuelve estos problemas de forma principada.

141.2. La función logística (sigmoide)

La solución natural al problema anterior es transformar la salida lineal mediante una función que mapee $\mathbb{R}$ al intervalo $(0, 1)$. La función más utilizada con este propósito es la **función logística**, también llamada función sigmoide.

Definición. La función logística $\sigma : \mathbb{R} \rightarrow (0, 1)$ se define como

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (141.1)$$

Esta función tiene varias propiedades matemáticas que la hacen especialmente adecuada para modelar probabilidades:

Rango: Para cualquier $z \in \mathbb{R}$, se cumple $0 < \sigma(z) < 1$. Cuando $z \rightarrow +\infty$, $\sigma(z) \rightarrow 1$; cuando $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$.

Simetría alrededor del origen: $\sigma(0) = 1/2$. Esto significa que el punto $z = 0$ separa las dos clases con probabilidad exactamente igual.

Derivada conveniente: La derivada de la sigmoide admite una expresión cerrada en términos de la misma función:

$$\frac{d\sigma}{dz}(z) = \sigma(z)(1 - \sigma(z)). \quad (141.2)$$

Esta propiedad simplifica enormemente el cálculo del gradiente durante el entrenamiento.

Interpretación como log-odds: Si definimos $p = \sigma(z)$, entonces se puede verificar que

$$\log \frac{p}{1-p} = z. \quad (141.3)$$

La cantidad $\log(p/(1-p))$ se denomina **logit** o log-odds de p . La Ecuación 141.3 establece que el argumento z de la sigmoide equivale al logaritmo de las probabilidades relativas entre la clase positiva y la negativa. De ahí el nombre “regresión logística”: el modelo asume que el logit de la probabilidad es una función lineal de las características (Bishop 2006).

La siguiente figura muestra la curva sigmoide con la posibilidad de desplazar el umbral de decisión z_0 , que corresponde al punto donde la probabilidad predicha cruza 0.5.

```
import numpy as np
import plotly.graph_objects as go

z = np.linspace(-8, 8, 400)
sigma = 1 / (1 + np.exp(-z))

z0_values = np.arange(-4, 4.5, 0.5)

frames = []
for z0 in z0_values:
    sigma_shifted = 1 / (1 + np.exp(-(z - z0)))
    frames.append(go.Frame(
        data=[
            go.Scatter(x=z, y=sigma_shifted,
                       mode="lines",
                       line=dict(color="#4C72B0", width=2.5)),
            go.Scatter(x=z[z >= z0], y=sigma_shifted[z >= z0],
```

```

        fill="tozero", fillcolor="rgba(221,132,82,0.15)",
        mode="none", showlegend=False),
    go.Scatter(x=[z0], y=[0.5],
        mode="markers",
        marker=dict(color="#DD8452", size=10,
            line=dict(color="white", width=2))),
    ],
    name=str(round(z0, 1))
))

# Frame inicial
sigma0 = 1 / (1 + np.exp(-z))
fig = go.Figure(
    data=[
        go.Scatter(x=z, y=sigma0, mode="lines",
            name="(z - z)",
            line=dict(color="#4C72B0", width=2.5)),
        go.Scatter(x=z[z >= 0], y=sigma0[z >= 0],
            fill="tozero", fillcolor="rgba(221,132,82,0.15)",
            mode="none", name="Región clase 1"),
        go.Scatter(x=[0], y=[0.5], mode="markers",
            name="Umbral z ",
            marker=dict(color="#DD8452", size=10,
                line=dict(color="white", width=2))),
    ],
    frames=frames
)

fig.add_hline(y=0.5, line_dash="dash", line_color="gray", line_width=1,
    annotation_text="p = 0.5", annotation_position="right")
fig.add_hline(y=1.0, line_dash="dot", line_color="lightgray", line_width=1)
fig.add_hline(y=0.0, line_dash="dot", line_color="lightgray", line_width=1)

fig.update_layout(
    title="Función sigmoide: desplazamiento del umbral z ",
    xaxis_title="z",
    yaxis_title="(z - z)",
    template="plotly_white",
    height=470,
    updatemenus=[dict(
        type="buttons",
        showactive=False,
        y=1.15, x=0.5, xanchor="center",
        buttons=[
            dict(label="Animar",
                method="animate",
                args=[None, dict(frame=dict(duration=300, redraw=True),
                    fromcurrent=True)]),
            dict(label="Pausar",

```

```

        method="animate",
        args=[[None], dict(frame=dict(duration=0, redraw=False),
                                mode="immediate"))]
    ],
    sliders=[dict(
        steps=[dict(args=[[f.name],
                        dict(frame=dict(duration=300, redraw=True),
                                mode="immediate"))],
                label=f.name, method="animate")
            for f in frames],
        currentvalue=dict(prefix="z = ", font=dict(size=13)),
        x=0.05, len=0.9, y=0
    )],
    legend=dict(orientation="h", yanchor="top", y=-0.25,
                xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 141.2.: Función sigmoide con animación del umbral de decisión z . Cada frame corresponde a un valor distinto de z ; la región sombreada indica la zona de clase predicha como positiva.

141.2.1. Intuición geométrica

La función sigmoide convierte la recta numérica $\mathbb{R}$ en el intervalo abierto $(0, 1)$. El argumento z actúa como un “puntaje de clase positiva”: valores grandes de z corresponden a probabilidades cercanas a 1, valores pequeños corresponden a probabilidades cercanas a 0, y $z = 0$ corresponde exactamente a probabilidad 0.5.

Cuando $z = \beta_0 + \beta^\top \mathbf{x}$, el umbral $z = 0$ corresponde al hiperplano $\beta_0 + \beta^\top \mathbf{x} = 0$, que es precisamente la **frontera de decisión** del modelo. Exploraremos esto con detalle en la sección siguiente.

141.3. El modelo de regresión logística

Con la función logística en mano, el modelo de regresión logística modela la probabilidad condicional de que la respuesta sea 1 dada una observación $\mathbf{x} \in \mathbb{R}^d$:

Definición. El modelo de regresión logística establece que

$$P(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \beta^\top \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top \mathbf{x})}}, \quad (141.4)$$

donde $\beta_0 \in \mathbb{R}$ es el intercepto y $\beta \in \mathbb{R}^d$ es el vector de coeficientes.

Y consecuentemente:

$$P(y = 0 \mid \mathbf{x}) = 1 - \sigma(\beta_0 + \beta^\top \mathbf{x}) = \sigma(-(\beta_0 + \beta^\top \mathbf{x})).$$

141.3.1. Frontera de decisión

La regla de clasificación más natural consiste en asignar la clase 1 cuando la probabilidad posterior supera 0.5:

$$\hat{y} = \begin{cases} 1 & \text{si } P(y = 1 \mid \mathbf{x}) \geq 0.5, \\ 0 & \text{en caso contrario.} \end{cases}$$

Como $\sigma(z) \geq 0.5$ si y solo si $z \geq 0$, la condición anterior equivale a:

$$\beta_0 + \beta^\top \mathbf{x} \geq 0. \quad (141.5)$$

El conjunto de puntos $\mathbf{x}$ donde $\beta_0 + \beta^\top \mathbf{x} = 0$ define un **hiperplano** en $\mathbb{R}^d$, que es la frontera de decisión del modelo. Esto implica que la regresión logística es un **clasificador lineal**: separa las clases mediante una frontera plana (Bishop 2006; Hastie et al. 2009).

i Nota

La frontera de decisión de la regresión logística siempre es un hiperplano en el espacio original de características. Para capturar fronteras no lineales, es necesario incluir transformaciones de las características originales (polinomios, interacciones, etc.) antes de ajustar el modelo.

La siguiente figura muestra la frontera de decisión y el mapa de probabilidades posterior en dos dimensiones con datos sintéticos de dos clases gaussianas.

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(7)
n = 60
X0 = rng.multivariate_normal([1.5, 1.5], [[1, 0.3], [0.3, 1]], n)
X1 = rng.multivariate_normal([4.0, 4.0], [[1, -0.2], [-0.2, 1]], n)
X = np.vstack([X0, X1])
y = np.concatenate([np.zeros(n), np.ones(n)])

# Entrenamiento por descenso de gradiente
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def entrenar_logistica(X, y, lr=0.1, n_iter=1000):
    N, d = X.shape
    X_aug = np.column_stack([np.ones(N), X])
    w = np.zeros(d + 1)
    for _ in range(n_iter):
        p = sigmoid(X_aug @ w)
```

```

        grad = X_aug.T @ (p - y) / N
        w -= lr * grad
    return w

w = entrenar_logistica(X, y)
beta0, beta = w[0], w[1:]

# Malla de probabilidades
x1g = np.linspace(-1, 7, 120)
x2g = np.linspace(-1, 7, 120)
xx1, xx2 = np.meshgrid(x1g, x2g)
Z = sigmoid(beta0 + beta[0]*xx1 + beta[1]*xx2)

fig = go.Figure()

fig.add_trace(go.Contour(
    x=x1g, y=x2g, z=Z,
    colorscale=[[0, "#4C72B0"], [0.5, "white"], [1, "#DD8452"]],
    opacity=0.45,
    showscale=True,
    colorbar=dict(title="P(y=1|x)"),
    contours=dict(showlines=False),
    name="Probabilidad"
))

fig.add_trace(go.Contour(
    x=x1g, y=x2g, z=Z,
    contours=dict(start=0.5, end=0.5, coloring="none",
                  showlabels=True, labelfont=dict(size=12, color="black")),
    line=dict(color="black", width=2.5),
    showscale=False,
    name="Frontera (p=0.5)"
))

fig.add_trace(go.Scatter(
    x=X0[:,0], y=X0[:,1],
    mode="markers", name="Clase 0",
    marker=dict(color="#4C72B0", size=7, opacity=0.85,
                line=dict(color="white", width=1))
))

fig.add_trace(go.Scatter(
    x=X1[:,0], y=X1[:,1],
    mode="markers", name="Clase 1",
    marker=dict(color="#DD8452", size=7, opacity=0.85,
                line=dict(color="white", width=1))
))

fig.update_layout(

```

```

title="Frontera de decisión logística en 2D",
xaxis_title="Característica x ",
yaxis_title="Característica x ",
template="plotly_white",
height=490,
legend=dict(orientation="h", yanchor="top", y=-0.18,
             xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 141.3.: Frontera de decisión de la regresión logística en dos dimensiones. El contorno de colores representa la probabilidad predicha $P(y=1|x)$; la línea negra es la isocurva $p=0.5$ (frontera de decisión).

141.3.2. Notación compacta con vector aumentado

Es habitual absorber el intercepto β_0 en el vector de coeficientes definiendo $\tilde{\beta} = (\beta_0, \beta_1, \dots, \beta_d)^\top$ y aumentando cada observación con una coordenada fija igual a 1: $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)^\top$. Con esta convención, el modelo se escribe de forma compacta como:

$$P(y = 1 | \mathbf{x}) = \sigma(\tilde{\beta}^\top \tilde{\mathbf{x}}).$$

De aquí en adelante, cuando no haya ambigüedad, omitiremos la tilde y escribiremos simplemente $\beta^\top \mathbf{x}$ entendiendo que el vector $\mathbf{x}$ ya incluye la coordenada 1.

141.4. Estimación por máxima verosimilitud

Para aprender los parámetros β del modelo se usa el principio de **máxima verosimilitud** (MLE): se buscan los valores de β que hacen más probable el conjunto de observaciones de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ (Murphy 2012).

Dado que cada $y_i \in \{0, 1\}$, la distribución de y_i dado $\mathbf{x}_i$ es Bernoulli con parámetro $p_i = \sigma(\beta^\top \mathbf{x}_i)$:

$$P(y_i | \mathbf{x}_i; \beta) = p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Asumiendo independencia entre las observaciones, la verosimilitud conjunta es:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Tomando logaritmo se obtiene la **log-verosimilitud**:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (141.6)$$

141.4.1. Equivalencia con entropía cruzada

La Ecuación 141.6 es exactamente el negativo de la **entropía cruzada binaria** (binary cross-entropy), que es la función de pérdida estándar para clasificación binaria en redes neuronales y otros modelos. Maximizar la log-verosimilitud equivale a minimizar la entropía cruzada:

$$\mathcal{L}(\beta) = -\frac{1}{n}\ell(\beta) = -\frac{1}{n}\sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (141.7)$$

141.4.2. Gradiente de la log-verosimilitud

A diferencia de la regresión lineal, la log-verosimilitud logística no admite una solución de forma cerrada. Sin embargo, la función es **cóncava** (o equivalentemente, la entropía cruzada es convexa), lo que garantiza que el máximo global existe y es único, y que los métodos de optimización basados en gradiente convergen al óptimo global (Hastie et al. 2009).

El gradiente de la log-verosimilitud respecto a β es:

$$\nabla_{\beta} \ell(\beta) = \mathbf{X}^{\top}(\mathbf{y} - \hat{\mathbf{p}}), \quad (141.8)$$

donde $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ es la matriz de diseño (con columna de unos), $\mathbf{y} = (y_1, \dots, y_n)^{\top}$ y $\hat{\mathbf{p}} = (p_1, \dots, p_n)^{\top}$ con $p_i = \sigma(\beta^{\top} \mathbf{x}_i)$.

Derivación del gradiente. Para la j -ésima componente:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n \left[y_i \frac{1}{p_i} \frac{\partial p_i}{\partial \beta_j} - (1 - y_i) \frac{1}{1 - p_i} \frac{\partial p_i}{\partial \beta_j} \right].$$

Usando la propiedad de la derivada de la sigmoide (Ec. Ecuación 141.2):

$$\frac{\partial p_i}{\partial \beta_j} = p_i(1 - p_i) x_{ij}.$$

Sustituyendo:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n [y_i(1 - p_i) - (1 - y_i)p_i] x_{ij} = \sum_{i=1}^n (y_i - p_i) x_{ij},$$

lo que en forma matricial da exactamente la Ecuación 141.8.

141.4.3. Descenso de gradiente

El algoritmo de **descenso de gradiente** actualiza los parámetros en la dirección del gradiente de la log-verosimilitud (o el negativo del gradiente de la pérdida):

$$\beta^{(t+1)} = \beta^{(t)} + \eta \cdot \mathbf{X}^{\top}(\mathbf{y} - \hat{\mathbf{p}}^{(t)}), \quad (141.9)$$

donde $\eta > 0$ es la **tasa de aprendizaje**. Para minimizar la entropía cruzada, la actualización sería con signo negativo.

141.4.4. Algoritmo IRLS

El método de Newton-Raphson aplicado a la log-verosimilitud logística da lugar al algoritmo **IRLS** (*Iteratively Reweighted Least Squares*). En cada iteración, el problema se reduce a una regresión lineal ponderada con matriz de pesos $\mathbf{W} = \text{diag}(p_i(1 - p_i))$:

$$\beta^{(t+1)} = (\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{z}^{(t)},$$

donde $\mathbf{z}^{(t)} = \mathbf{X}\beta^{(t)} + (\mathbf{W}^{(t)})^{-1}(\mathbf{y} - \hat{\mathbf{p}}^{(t)})$ es la respuesta ajustada. IRLS converge cuadráticamente cerca del óptimo, mientras que el descenso de gradiente converge linealmente (McCullagh y Nelder 1989; Murphy 2012).

La siguiente figura muestra la evolución de la función de pérdida (entropía cruzada) durante el entrenamiento por descenso de gradiente.

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(21)
n = 120
X0 = rng.multivariate_normal([1, 1], [[1, 0], [0, 1]], n//2)
X1 = rng.multivariate_normal([3.5, 3.5], [[1, 0], [0, 1]], n//2)
X_tr = np.vstack([X0, X1])
y_tr = np.concatenate([np.zeros(n//2), np.ones(n//2)])
X_aug = np.column_stack([np.ones(n), X_tr])

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def cross_entropy(w, X, y):
    p = sigmoid(X @ w)
    p = np.clip(p, 1e-10, 1-1e-10)
    return -np.mean(y * np.log(p) + (1-y) * np.log(1-p))

colores = ["#4C72B0", "#DD8452", "#2ca02c"]
learning_rates = [0.5, 0.1, 0.02]
n_iter = 200

fig = go.Figure()

for lr, color in zip(learning_rates, colores):
    w = np.zeros(3)
    losses = []
    for _ in range(n_iter):
        losses.append(cross_entropy(w, X_aug, y_tr))
        p = sigmoid(X_aug @ w)
        grad = X_aug.T @ (p - y_tr) / n
        w -= lr * grad

    fig.add_trace(go.Scatter(
```

```

        x=list(range(n_iter)),
        y=losses,
        mode="lines",
        name=f" = {lr}",
        line=dict(color=color, width=2.2)
    ))

fig.update_layout(
    title="Convergencia del descenso de gradiente: entropía cruzada vs. iteración",
    xaxis_title="Iteración",
    yaxis_title="Entropía cruzada (pérdida)",
    template="plotly_white",
    height=450,
    legend=dict(orientation="h", yanchor="top", y=-0.18,
                xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 141.4.: Convergencia del descenso de gradiente en regresión logística. Se muestra la entropía cruzada (pérdida) en función del número de iteraciones para distintas tasas de aprendizaje.

Observamos que una tasa de aprendizaje grande ($\eta = 0.5$) converge rápidamente pero puede ser inestable, mientras que una tasa pequeña ($\eta = 0.02$) converge de manera más suave pero requiere más iteraciones. La elección de η es un hiperparámetro crítico del entrenamiento (Hastie et al. 2009).

141.5. Interpretación de coeficientes: odds ratio

Una de las grandes ventajas de la regresión logística sobre otros clasificadores es que sus coeficientes admiten una **interpretación directa en términos de probabilidades y razones de momios** (*odds ratios*) (Hosmer et al. 2013).

141.5.1. Log-odds y odds ratio

De la Ecuación 141.3 sabemos que el modelo puede escribirse en la escala del logit:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d. \quad (141.10)$$

Esta expresión revela que la regresión logística es, en realidad, un modelo **lineal en la escala del log-odds**. El coeficiente β_j indica el cambio en el log-odds asociado a un incremento unitario en x_j , manteniendo las demás variables constantes.

Para obtener el **odds ratio** asociado a x_j , basta con exponenciar el coeficiente:

$$OR_j = e^{\beta_j}. \quad (141.11)$$

Interpretación: Si x_j aumenta en una unidad, los odds de la clase positiva se **multiplican** por e^{β_j} :

- $e^{\beta_j} > 1$: aumentan los odds de clase positiva.
- $e^{\beta_j} < 1$: disminuyen los odds de clase positiva.
- $e^{\beta_j} = 1$: el predictor no tiene efecto sobre los odds.

141.5.2. Intervalos de confianza

Los intervalos de confianza para los coeficientes se construyen usando la matriz de información de Fisher, que es la Hessiana negativa de la log-verosimilitud:

$$\mathbf{I}(\beta) = \mathbf{X}^\top \mathbf{W} \mathbf{X},$$

donde $\mathbf{W} = \text{diag}(p_i(1 - p_i))$. Los errores estándar de los coeficientes son las raíces cuadradas de los elementos diagonales de $\mathbf{I}(\beta)^{-1}$, y un intervalo de confianza aproximado al 95 % es:

$$\beta_j \pm 1.96 \cdot \text{SE}(\beta_j).$$

La siguiente figura muestra un **forest plot** con los coeficientes estimados e intervalos de confianza al 95 %, estilo habitual en publicaciones epidemiológicas (Hosmer et al. 2013).

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(99)
n = 200
# Variables: edad, colesterol, presión, tabaquismo, ejercicio
nombres = ["Intercepto", "Edad", "Colesterol", "Presión arterial",
            "Tabaquismo", "Ejercicio físico"]
X_raw = np.column_stack([
    np.ones(n),
    rng.normal(55, 10, n),
    rng.normal(200, 30, n),
    rng.normal(130, 20, n),
    rng.binomial(1, 0.35, n).astype(float),
    rng.binomial(1, 0.5, n).astype(float)
])
# Estandarizar variables continuas (excluyendo intercepto y binarias)
X_std = X_raw.copy()
for j in [1, 2, 3]:
    X_std[:, j] = (X_raw[:, j] - X_raw[:, j].mean()) / X_raw[:, j].std()

# Generar etiquetas con logit conocido
beta_true = np.array([-1.0, 0.8, 0.5, 0.6, 1.2, -0.7])
logit_true = X_std @ beta_true
p_true = 1 / (1 + np.exp(-logit_true))
y_sim = rng.binomial(1, p_true)
```

```

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# Entrenar por descenso de gradiente
w = np.zeros(6)
for _ in range(3000):
    p = sigmoid(X_std @ w)
    grad = X_std.T @ (p - y_sim) / n
    w -= 0.05 * grad

# Errores estándar vía Fisher information
p_hat = sigmoid(X_std @ w)
W_diag = p_hat * (1 - p_hat)
I_mat = (X_std * W_diag[:, None]).T @ X_std
try:
    I_inv = np.linalg.inv(I_mat)
    se = np.sqrt(np.diag(I_inv))
except np.linalg.LinAlgError:
    se = np.ones(6) * 0.1

ci_low = w - 1.96 * se
ci_high = w + 1.96 * se

colores_coef = ["#DD8452" if v > 0 else "#4C72B0" for v in w]

fig = go.Figure()

for i, (nombre, coef, lo, hi, color) in enumerate(
    zip(nombres, w, ci_low, ci_high, colores_coef)):
    fig.add_trace(go.Scatter(
        x=[lo, hi], y=[i, i],
        mode="lines",
        line=dict(color=color, width=3),
        showlegend=False
    ))
    fig.add_trace(go.Scatter(
        x=[coef], y=[i],
        mode="markers",
        marker=dict(color=color, size=11,
                    line=dict(color="white", width=2)),
        name=nombre,
        showlegend=False
    ))

fig.add_vline(x=0, line_dash="dash", line_color="gray", line_width=1.5)

fig.update_layout(
    title="Coeficientes del modelo logístico con IC 95%",
    xaxis_title="Coeficiente (escala log-odds)",

```



```

yaxis=dict(tickvals=list(range(len(nombres))),
           ticktext=nombres, showgrid=False),
template="plotly_white",
height=430,
shapes=[dict(type="line", x0=0, x1=0, y0=-0.5,
              y1=len(nombres)-0.5,
              line=dict(color="gray", dash="dash", width=1.5))]
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 141.5.: Forest plot de coeficientes del modelo logístico. Las barras horizontales representan intervalos de confianza al 95 %; los puntos son las estimaciones puntuales. Valores mayores que 0 (OR > 1) indican asociación positiva con la clase 1.

La tabla siguiente resume la interpretación de los coeficientes del ejemplo anterior:

Tabla 141.1.: Interpretación de los coeficientes del modelo logístico.

Variable	$\hat{\beta}_j$	$e^{\hat{\beta}_j}$	Interpretación
Edad (stand.)	≈ 0.80	≈ 2.23	Cada SD de edad multiplica por 2.2 los odds
Colesterol	≈ 0.50	≈ 1.65	Asociación positiva moderada
Presión arterial	≈ 0.60	≈ 1.82	Riesgo elevado con mayor presión
Tabaquismo	≈ 1.20	≈ 3.32	Mayor factor de riesgo individual
Ejercicio físico	≈ -0.70	≈ 0.50	Factor protector (reduce a la mitad los odds)

💡 Tip

Cuando las variables continuas están estandarizadas (media 0, desviación estándar 1), los coeficientes son directamente comparables entre sí y el odds ratio refleja el efecto de cambiar la variable en una desviación estándar.

141.6. Regularización

141.6.1. El problema de la separabilidad perfecta

Un supuesto implícito en la estimación por MLE es que los datos no son **perfectamente separables**. Si existe un hiperplano que separa perfectamente las dos clases, el gradiente de la log-verosimilitud nunca se anula: siempre es posible aumentar el módulo de β para hacer las predicciones más extremas y aumentar la verosimilitud. El resultado es que los coeficientes divergen hacia $\pm\infty$, haciendo el modelo numéricamente inestable e inutilizable (Hastie et al. 2009).

Este fenómeno ocurre con frecuencia cuando:

- El número de variables d es grande en relación con n .

- Las variables predictoras están altamente correlacionadas (colinealidad).
- Se incluyen variables binarias o indicadoras que predicen perfectamente un evento raro.

La solución estándar es añadir un término de **regularización** que penalice coeficientes grandes.

141.6.2. Regularización L2 (Ridge)

La penalización L2 añade el cuadrado de la norma euclidiana de β a la función de pérdida:

$$\mathcal{L}_{\text{Ridge}}(\beta) = -\ell(\beta) + \lambda \|\beta\|_2^2 = -\ell(\beta) + \lambda \sum_{j=1}^d \beta_j^2. \quad (141.12)$$

El parámetro $\lambda > 0$ controla la intensidad de la regularización. A mayor λ , mayor penalización a coeficientes grandes, y el modelo tiende hacia $\beta = \mathbf{0}$.

El gradiente de la pérdida regularizada es:

$$\nabla \mathcal{L}_{\text{Ridge}} = -\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{p}}) + 2\lambda\beta.$$

La regularización L2 corresponde, en un marco bayesiano, a colocar una distribución prior gaussiana $\beta \sim \mathcal{N}(\mathbf{0}, (2\lambda)^{-1}\mathbf{I})$ sobre los coeficientes.

141.6.3. Regularización L1 (Lasso)

La penalización L1 añade la norma ℓ_1 de β :

$$\mathcal{L}_{\text{Lasso}}(\beta) = -\ell(\beta) + \lambda \|\beta\|_1 = -\ell(\beta) + \lambda \sum_{j=1}^d |\beta_j|. \quad (141.13)$$

La diferencia clave con L2 es que L1 produce soluciones **dispersas** (sparse): para λ suficientemente grande, muchos coeficientes se vuelven exactamente cero, realizando selección automática de variables. Bayesianamente, corresponde a una distribución prior de Laplace sobre β (Murphy 2012).

La penalización L1 no es diferenciable en $\beta_j = 0$, por lo que requiere algoritmos especiales como el descenso de subgradiente o el descenso coordinado.

141.6.4. Elastic Net

La **Elastic Net** combina ambas penalizaciones:

$$\mathcal{L}_{\text{EN}}(\beta) = -\ell(\beta) + \lambda \left[\alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right], \quad (141.14)$$

donde $\alpha \in [0, 1]$ controla el balance entre L1 y L2. Cuando $\alpha = 1$ se recupera el Lasso; cuando $\alpha = 0$ se recupera Ridge. La Elastic Net es útil cuando hay grupos de variables correlacionadas, pues tiende a seleccionar o descartar grupos completos de variables correlacionadas.

141.6.5. Regularization path

El **regularization path** muestra cómo evolucionan los coeficientes estimados en función de λ . Para $\lambda \rightarrow 0$ se recuperan los estimadores MLE sin regularización; para $\lambda \rightarrow \infty$ todos los coeficientes convergen a cero.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(55)
n_path = 150
d_path = 5
nombres_vars = [f"_{j}" for j in range(1, d_path + 1)]

X_p = rng.normal(size=(n_path, d_path))
beta_gen = np.array([1.5, -1.0, 0.5, 0.0, -0.3])
logit_p = -0.5 + X_p @ beta_gen
p_gen = 1 / (1 + np.exp(-logit_p))
y_p = rng.binomial(1, p_gen)
X_paug = np.column_stack([np.ones(n_path), X_p])

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# --- Ridge path ---
lambdas = np.logspace(-3, 2, 50)
coefs_ridge = []

for lam in lambdas:
    w = np.zeros(d_path + 1)
    for _ in range(2000):
        p_hat = sigmoid(X_paug @ w)
        grad = X_paug.T @ (p_hat - y_p) / n_path
        grad[1:] += 2 * lam * w[1:] # no penalizar intercepto
        w -= 0.05 * grad
    coefs_ridge.append(w[1:].copy())

coefs_ridge = np.array(coefs_ridge)

# --- Lasso path (subgradiente) ---
def soft_threshold(x, threshold):
    return np.sign(x) * np.maximum(np.abs(x) - threshold, 0)

coefs_lasso = []
for lam in lambdas:
    w = np.zeros(d_path + 1)
    lr_l1 = 0.02
    for it in range(3000):
        p_hat = sigmoid(X_paug @ w)
```

```

        grad = X_paug.T @ (p_hat - y_p) / n_path
        w[0] -= lr_l1 * grad[0]
        w[1:] -= lr_l1 * grad[1:]
        w[1:] = soft_threshold(w[1:], lr_l1 * lam)
        coefs_lasso.append(w[1:].copy())

coefs_lasso = np.array(coefs_lasso)

paleta = ["#4C72B0", "#DD8452", "#2ca02c", "#d62728", "#9467bd"]
log_lam = np.log10(lambdas)

fig = make_subplots(rows=1, cols=2,
                    subplot_titles=["Ridge (L2)", "Lasso (L1)"],
                    shared_yaxes=False)

for j in range(d_path):
    fig.add_trace(go.Scatter(
        x=log_lam, y=coefs_lasso[:, j],
        mode="lines", name=nombres_vars[j],
        line=dict(color=paleta[j], width=2),
        legendgroup=nombres_vars[j],
        showlegend=True
    ), row=1, col=1)

for j in range(d_path):
    fig.add_trace(go.Scatter(
        x=log_lam, y=coefs_lasso[:, j],
        mode="lines", name=nombres_vars[j],
        line=dict(color=paleta[j], width=2),
        legendgroup=nombres_vars[j],
        showlegend=False
    ), row=1, col=2)

fig.add_hline(y=0, line_dash="dot", line_color="gray",
              line_width=1, row=1, col=1)
fig.add_hline(y=0, line_dash="dot", line_color="gray",
              line_width=1, row=1, col=2)

fig.update_xaxes(title_text="log ( )", row=1, col=1)
fig.update_xaxes(title_text="log ( )", row=1, col=2)
fig.update_yaxes(title_text="Coeficiente ^", row=1, col=1)
fig.update_yaxes(title_text="Coeficiente ^", row=1, col=2)

fig.update_layout(
    title="Regularization path: Ridge vs. Lasso",
    template="plotly_white",
    height=460,
    legend=dict(orientation="h", yanchor="top", y=-0.18,
                xanchor="center", x=0.5)

```

```
)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 141.6.: Regularization path para regresión logística: evolución de los coeficientes en función del parámetro de regularización $\log(\lambda)$. Panel izquierdo: penalización L2 (Ridge). Panel derecho: penalización L1 (Lasso, con descenso de subgradiente).

Nótese la diferencia fundamental entre los dos paneles: en Ridge, los coeficientes se encogen suavemente hacia cero pero nunca llegan a serlo exactamente (excepto en el límite $\lambda \rightarrow \infty$). En Lasso, los coeficientes se vuelven exactamente cero a partir de un valor de λ determinado, realizando selección de variables implícita (Hastie et al. 2009).

i Nota

En la práctica, el parámetro λ se elige mediante validación cruzada. Una mala elección puede llevar a sobreajuste (λ muy pequeño) o a un modelo demasiado simple (λ muy grande).

141.7. Extensión multiclase

La regresión logística puede extenderse a problemas con $K > 2$ clases mediante dos estrategias principales: **One-vs-Rest** y **regresión logística multinomial** (softmax).

141.7.1. One-vs-Rest (OvR)

La estrategia **One-vs-Rest** (también llamada One-vs-All, OvA) entrena K clasificadores binarios independientes. El k -ésimo clasificador aprende a distinguir la clase k de todas las demás clases combinadas. Para una nueva observación $\mathbf{x}$, se calcula la probabilidad $p_k = P(y = k \mid \mathbf{x})$ de cada clasificador binario, y se asigna la clase con mayor probabilidad:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \hat{p}_k(\mathbf{x}).$$

La ventaja es la simplicidad: solo se requieren K modelos logísticos estándar. La desventaja es que las K probabilidades no suman 1 necesariamente, ya que provienen de modelos independientes.

141.7.2. Regresión logística multinomial (softmax)

La regresión logística multinomial extiende el modelo de manera conjunta y probabilísticamente coherente. Se aprenden $K - 1$ conjuntos de parámetros β_k (uno de los K sirve como referencia, típicamente $\beta_K = \mathbf{0}$) y la probabilidad de cada clase se calcula mediante la función **softmax**:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\beta_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\beta_j^\top \mathbf{x}}}. \quad (141.15)$$

La función softmax garantiza que las probabilidades sumen 1 y sean positivas, siendo una generalización directa de la función sigmoide al caso multiclase. La log-verosimilitud se convierte en:

$$\ell(\{\beta_k\}) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{1}[y_i = k] \log P(y_i = k | \mathbf{x}_i), \quad (141.16)$$

que es la **entropía cruzada categórica** generalizada. El gradiente respecto a β_k es:

$$\nabla_{\beta_k} \ell = \sum_{i=1}^n (\mathbb{1}[y_i = k] - P(y_i = k | \mathbf{x}_i)) \mathbf{x}_i = \mathbf{X}^\top (\mathbf{r}_k - \hat{\mathbf{p}}_k),$$

donde $\mathbf{r}_k = (\mathbb{1}[y_1 = k], \dots, \mathbb{1}[y_n = k])^\top$ (Bishop 2006; Murphy 2012).

La siguiente figura muestra la frontera de decisión multiclase obtenida con el modelo softmax en un problema 2D con tres clases.

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(13)
n_mc = 60

# Tres clases gaussianas
medias = [[1.0, 4.5], [4.5, 1.0], [4.5, 5.5]]
covs = [[[0.8, 0.1], [0.1, 0.8]],
         [[0.8, -0.1], [-0.1, 0.8]],
         [[0.8, 0.2], [0.2, 0.8]]]
colores_clases = ["#4C72B0", "#DD8452", "#2ca02c"]

Xs, ys = [], []
for k, (mu, cov) in enumerate(zip(medias, covs)):
    pts = rng.multivariate_normal(mu, cov, n_mc)
    Xs.append(pts)
    ys.extend([k] * n_mc)

X_mc = np.vstack(Xs)
y_mc = np.array(ys)
K = 3

# Entrenamiento softmax desde cero
def softmax(Z):
    Z_shift = Z - Z.max(axis=1, keepdims=True)
    E = np.exp(Z_shift)
    return E / E.sum(axis=1, keepdims=True)

X_mc_aug = np.column_stack([np.ones(len(y_mc)), X_mc])
d_mc = X_mc_aug.shape[1]

W_mc = np.zeros((d_mc, K))
lr_mc = 0.05
```

```

# One-hot encoding
Y_oh = np.zeros((len(y_mc), K))
Y_oh[np.arange(len(y_mc)), y_mc] = 1

for _ in range(3000):
    Z_mc = X_mc_aug @ W_mc
    P_mc = softmax(Z_mc)
    grad_mc = X_mc_aug.T @ (P_mc - Y_oh) / len(y_mc)
    W_mc -= lr_mc * grad_mc

# Malla de predicciones
x1g = np.linspace(-1.5, 7.5, 200)
x2g = np.linspace(-1.5, 8.0, 200)
xx1, xx2 = np.meshgrid(x1g, x2g)
X_grid = np.column_stack([np.ones(xx1.size), xx1.ravel(), xx2.ravel()])
P_grid = softmax(X_grid @ W_mc)
clase_grid = P_grid.argmax(axis=1).reshape(xx1.shape)

# Convertir a imagen RGB para fondo de regiones
# Usar colorscale con valores discretos
paleta_rgb = [
    [0.0, "rgba(76,114,176,0.35)"],
    [0.33, "rgba(76,114,176,0.35)"],
    [0.33, "rgba(221,132,82,0.35)"],
    [0.66, "rgba(221,132,82,0.35)"],
    [0.66, "rgba(44,160,44,0.35)"],
    [1.0, "rgba(44,160,44,0.35)"],
]

fig = go.Figure()

fig.add_trace(go.Heatmap(
    x=x1g, y=x2g, z=clase_grid,
    colorscale=paleta_rgb,
    showscale=False,
    zmin=0, zmax=2
))

nombres_clases = ["Clase 0", "Clase 1", "Clase 2"]
for k, (pts, nombre, color) in enumerate(zip(Xs, nombres_clases, colores_clases)):
    fig.add_trace(go.Scatter(
        x=pts[:, 0], y=pts[:, 1],
        mode="markers", name=nombre,
        marker=dict(color=color, size=7, opacity=0.9,
                    line=dict(color="white", width=1.2))
    ))

fig.update_layout(
    title="Frontera de decisión multinomial (softmax) - 3 clases",

```

```
xaxis_title="Característica x ",
yaxis_title="Característica x ",
template="plotly_white",
height=490,
legend=dict(orientation="h", yanchor="top", y=-0.18,
             xanchor="center", x=0.5)
)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 141.7.: Frontera de decisión del modelo logístico multinomial (softmax) para tres clases en dos dimensiones. Las regiones coloreadas representan la clase predicha; los puntos son los datos de entrenamiento.

141.7.3. Comparación OvR vs. Softmax

Tabla 141.2.: Comparación entre One-vs-Rest y regresión logística multinomial.

Aspecto	One-vs-Rest	Softmax multinomial
Coherencia probabilística	No garantizada	Garantizada ($\sum_k p_k = 1$)
Número de modelos	K modelos separados	1 modelo conjunto
Costo computacional	$K \times$ costo binario	Similar, pero conjunto
Interpretación	Directa por modelo	Requiere clase de referencia
Separabilidad perfecta	Cada modelo por separado	Puede ocurrir globalmente

En la práctica, para K grande (como clasificación de texto con muchas categorías), OvR suele ser preferible por su paralelizabilidad. Para K pequeño o cuando se requiere coherencia probabilística, el modelo multinomial es superior (Murphy 2012).

141.8. Calibración de probabilidades

Una propiedad deseable pero no siempre garantizada en los clasificadores es que sus **predicciones de probabilidad estén bien calibradas**: si el modelo predice $P(y = 1 \mid \mathbf{x}) = 0.7$, entonces aproximadamente el 70 % de las observaciones con esa predicción deberían pertenecer a la clase 1.

Definición. Un clasificador está **perfectamente calibrado** si para todo valor $p \in (0, 1)$:

$$P(y = 1 \mid \hat{p}(\mathbf{x}) = p) = p.$$

141.8.1. Calibración de la regresión logística

La regresión logística está **bien calibrada por diseño** bajo sus propios supuestos: el entrenamiento por máxima verosimilitud minimiza la pérdida de entropía cruzada, que es precisamente la pérdida asociada a

la calibración probabilística. En contraste, otros clasificadores como los bosques aleatorios o las SVM no están calibrados de forma natural (Hastie et al. 2009; Murphy 2012).

Sin embargo, esta garantía de calibración solo se mantiene si:

1. El modelo está correctamente especificado (la verdadera frontera de decisión es lineal en $\mathbf{x}$).
2. No hay sobreajuste severo.
3. Se usa un conjunto de evaluación independiente del conjunto de entrenamiento.

La calibración se visualiza mediante la **curva de calibración** (o *reliability diagram*): se dividen las predicciones en B contenedores (*bins*) según el valor de $\hat{p}$, y se grafica la fracción real de positivos en cada contenedor contra el promedio de $\hat{p}$ en ese contenedor. Un clasificador perfectamente calibrado sigue la diagonal.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(42)
n_cal = 500
X_cal = rng.normal(size=(n_cal, 3))
beta_cal = np.array([-0.3, 1.0, -0.8, 0.5])
X_cal_aug = np.column_stack([np.ones(n_cal), X_cal])
logit_cal = X_cal_aug @ beta_cal
p_true_cal = 1 / (1 + np.exp(-logit_cal))
y_cal = rng.binomial(1, p_true_cal)

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# Entrenar modelo
w_cal = np.zeros(4)
for _ in range(2000):
    p_hat_cal = sigmoid(X_cal_aug @ w_cal)
    grad_cal = X_cal_aug.T @ (p_hat_cal - y_cal) / n_cal
    w_cal -= 0.1 * grad_cal

p_pred_cal = sigmoid(X_cal_aug @ w_cal)

# Calibration curve con 10 bins
n_bins = 10
bins = np.linspace(0, 1, n_bins + 1)
bin_centers = (bins[:-1] + bins[1:]) / 2
frac_pos = []
mean_pred = []
counts = []

for lo, hi in zip(bins[:-1], bins[1:]):
    mask = (p_pred_cal >= lo) & (p_pred_cal < hi)
    if mask.sum() > 0:
        frac_pos.append(y_cal[mask].mean())
```

```

        mean_pred.append(p_pred_cal[mask].mean())
        counts.append(mask.sum())
    else:
        frac_pos.append(np.nan)
        mean_pred.append((lo + hi) / 2)
        counts.append(0)

frac_pos = np.array(frac_pos)
mean_pred = np.array(mean_pred)
counts = np.array(counts)

fig = make_subplots(rows=2, cols=1,
                    row_heights=[0.75, 0.25],
                    shared_xaxes=True,
                    subplot_titles=["Curva de calibración", "Distribución de predicciones"])

# Curva de calibración
fig.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1],
    mode="lines", name="Calibración perfecta",
    line=dict(color="gray", dash="dash", width=1.5)
), row=1, col=1)

mask_valid = ~np.isnan(frac_pos)
fig.add_trace(go.Scatter(
    x=mean_pred[mask_valid], y=frac_pos[mask_valid],
    mode="lines+markers", name="Regresión logística",
    line=dict(color="#4C72B0", width=2.5),
    marker=dict(size=9, color="#4C72B0",
                line=dict(color="white", width=1.5))
), row=1, col=1)

# Histograma de predicciones
fig.add_trace(go.Bar(
    x=bin_centers, y=counts,
    name="Conteo",
    marker=dict(color="#DD8452", opacity=0.7),
    width=0.08
), row=2, col=1)

fig.update_xaxes(title_text="Probabilidad predicha", row=2, col=1,
                 range=[0, 1])
fig.update_yaxes(title_text="Fracción de positivos", row=1, col=1,
                 range=[0, 1])
fig.update_yaxes(title_text="Conteo", row=2, col=1)

fig.update_layout(
    template="plotly_white",
    height=510,

```

```

legend=dict(orientation="h", yanchor="top", y=-0.12,
            xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 141.8.: Curva de calibración (reliability diagram) para la regresión logística. La diagonal representa calibración perfecta. La histograma en la parte inferior muestra la distribución de las predicciones.

Cuando la curva de calibración se aleja sistemáticamente de la diagonal, se puede aplicar **calibración post-hoc** mediante regresión isotónica o escala de Platt (una regresión logística adicional sobre las puntuaciones del modelo) para corregir el sesgo sin reentrenar el modelo completo (Murphy 2012).

141.9. Ventajas y limitaciones

141.9.1. Ventajas

💡 Ventajas de la regresión logística

Interpretabilidad: Los coeficientes tienen una interpretación directa en términos de log-odds y odds ratios, lo que facilita la comunicación de resultados en contextos aplicados como la medicina o las ciencias sociales.

Eficiencia computacional: El entrenamiento por descenso de gradiente o IRLS es eficiente incluso con conjuntos de datos grandes. No requiere almacenar la matriz completa de datos durante la predicción.

Salidas probabilísticas calibradas: A diferencia de SVM o clasificadores de árbol, la regresión logística produce estimaciones de probabilidad que están bien calibradas bajo sus supuestos.

Pocos hiperparámetros: El único hiperparámetro principal es el parámetro de regularización λ (y el tipo de penalización). Esto simplifica el ajuste del modelo.

Estabilidad numérica: La función de pérdida es convexa, lo que garantiza convergencia al óptimo global con métodos de optimización estándar.

Línea de base sólida: En muchos problemas prácticos, la regresión logística es difícil de superar significativamente con métodos más complejos, especialmente cuando los datos son aproximadamente linealmente separables (James et al. 2021).

141.9.2. Limitaciones

⚠️ Limitaciones de la regresión logística

Frontera de decisión lineal: La regresión logística solo puede aprender fronteras lineales en el espacio de características. Para problemas con fronteras no lineales complejas (como clasificación de imágenes o texto), otros métodos pueden ser más adecuados.

Sensibilidad a la colinealidad: Cuando dos o más variables predictoras están altamente corre-

lacionadas, los coeficientes estimados pueden tener alta varianza y ser difíciles de interpretar. La regularización L2 mitiga este problema.

Separabilidad perfecta: En dimensiones altas o con pocas observaciones, los datos pueden ser perfectamente separables y el MLE no está definido. Requiere regularización obligatoria.

Supuesto de linealidad en el logit: Si la relación entre las características y el logit no es lineal, el modelo estará mal especificado. Es necesario incluir transformaciones o interacciones manualmente.

Sensibilidad a datos atípicos: Los puntos atípicos en el espacio de características pueden influir desproporcionadamente en los coeficientes estimados, aunque menos que en regresión lineal.

141.10. Recomendaciones prácticas

Las siguientes recomendaciones sintetizan buenas prácticas para el uso de la regresión logística en proyectos reales de clasificación.

1. **Estandarizar las variables continuas.** Antes de ajustar el modelo, escalar las variables a media 0 y desviación estándar 1 mejora la comparabilidad de los coeficientes, acelera la convergencia del gradiente y hace que el intercepto sea más interpretable.
2. **Usar regularización L2 por defecto.** A menos que se tenga una razón específica para selección de variables, la penalización Ridge es la opción más robusta: estabiliza el entrenamiento, reduce la varianza y es diferenciable en todo punto.
3. **Elegir λ por validación cruzada.** Explorar un grid logarítmico de valores de λ (por ejemplo, 10^{-4} a 10^2) usando k -fold cross-validation (típicamente $k = 5$ o $k = 10$) para seleccionar el valor óptimo.
4. **Verificar la calibración.** Construir la curva de calibración en un conjunto de validación independiente. Si la calibración es pobre (curva alejada de la diagonal), aplicar calibración post-hoc antes de publicar probabilidades.
5. **Revisar la separabilidad.** Si el algoritmo de optimización no converge o los coeficientes son extremadamente grandes, sospechar separabilidad perfecta. La solución es incrementar la regularización.
6. **Explorar transformaciones de características.** Si el modelo tiene baja capacidad predictiva, considerar añadir términos cuadráticos, interacciones o transformaciones logarítmicas de las variables predictoras antes de migrar a modelos más complejos.
7. **Interpretar con intervalo de confianza.** Siempre reportar los odds ratios con sus intervalos de confianza, no solo las estimaciones puntuales. Un coeficiente estadísticamente no significativo no implica ausencia de efecto, sino incertidumbre.
8. **Comparar con una línea de base simple.** Antes de declarar que un modelo más complejo supera a la regresión logística, asegurarse de que la comparación se hace con la versión regularizada y con características bien preprocesadas.
9. **Cuidar el desbalanceo de clases.** Cuando una clase es mucho menos frecuente que la otra (por ejemplo, fraude, enfermedad rara), considerar ajustar el umbral de clasificación o usar métricas como AUC-ROC o F1 en lugar de la exactitud (*accuracy*).
10. **Documentar el proceso de selección del modelo.** Registrar qué variables se incluyeron, qué transformaciones se aplicaron, qué valor de λ se seleccionó y por qué. La reproducibilidad es esencial en análisis de datos serios.

141.11. Resumen del capítulo

La regresión logística es un modelo de clasificación que combina la simplicidad de los modelos lineales con la capacidad de producir estimaciones de probabilidad bien calibradas. Sus ingredientes fundamentales son la función logística (Ecuación 141.1), el modelo de probabilidad (Ecuación 141.4), la estimación por máxima verosimilitud (Ecuación 141.6) y la interpretación de los coeficientes como log-odds (Ecuación 141.10).

A lo largo del capítulo hemos visto que:

- La regresión lineal falla para clasificación porque produce predicciones fuera de $[0, 1]$ y es inestable ante observaciones atípicas.
- La función sigmoide transforma la recta real en $(0, 1)$ y su inversa, el logit, da nombre al modelo.
- La frontera de decisión es un hiperplano en el espacio de características.
- El gradiente de la log-verosimilitud tiene la forma elegante $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{p}})$, que permite optimización eficiente.
- Los coeficientes se interpretan como cambios en el log-odds; exponenciados dan los odds ratios.
- La regularización (Ridge, Lasso o Elastic Net) es esencial en alta dimensión o cuando los datos son separables.
- La extensión multiclase se logra mediante softmax, una generalización directa de la sigmoide.
- La regresión logística está bien calibrada por diseño, a diferencia de muchos otros clasificadores.

Para profundizar en los fundamentos matemáticos se recomienda (Bishop 2006) y (Hastie et al. 2009); para aplicaciones en ciencias de la salud, (Hosmer et al. 2013); para una perspectiva bayesiana, (Murphy 2012); y para una introducción más accesible con ejemplos en R, (James et al. 2021).

141.12. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.

- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Calínski, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.

- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.

- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.

- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide*

- to *A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en->

us/azure/machine-learning/.

Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.

Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.

Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.

Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.

Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.

Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.

Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.

Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.

Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.

Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.

Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.

- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data*

Warehousing 5 (4): 13-22.

- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE*

Transactions on Systems, Man, and Cybernetics SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.

Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

142. Máquinas de Soporte Vectorial

Las Máquinas de Soporte Vectorial (SVM, del inglés *Support Vector Machines*) son uno de los algoritmos de aprendizaje supervisado más elegantes y teóricamente sólidos que existen. Desarrolladas por Vapnik y colaboradores en los años noventa (Vapnik 1995; Cortes y Vapnik 1995), combinan tres ideas fundamentales: la búsqueda de un hiperplano de margen máximo, la reformulación del problema mediante dualidad de Lagrange, y el llamado *kernel trick*, que permite operar en espacios de alta dimensión sin calcular explícitamente las coordenadas en ese espacio.

A diferencia de métodos como la regresión logística, que buscan cualquier frontera de decisión que separe bien los datos, las SVM buscan la frontera *óptima*: aquella que maximiza la distancia mínima a los puntos de entrenamiento. Esta propiedad geométrica tiene consecuencias profundas sobre la capacidad de generalización del modelo, vinculando la clasificación a nociones de la teoría estadística del aprendizaje (Hastie et al. 2009).

Este capítulo desarrolla la teoría de SVM desde sus fundamentos geométricos hasta sus extensiones prácticas, incluyendo el manejo de datos no separables, el kernel trick, la regresión con SVM y la calibración de probabilidades. Los ejemplos computacionales están escritos en Python usando `numpy` y `plotly`, construyendo las visualizaciones desde primeros principios para reforzar la intuición geométrica.

142.1. Motivación: clasificar con margen máximo

142.1.1. El problema de los múltiples hiperplanos separadores

Supongamos que tenemos un conjunto de datos linealmente separable: existen puntos de dos clases que pueden dividirse perfectamente mediante una línea recta (en 2D) o un hiperplano (en dimensiones superiores). La primera pregunta natural es: ¿cuántos hiperplanos separadores existen?

La respuesta es que, en general, existen infinitos. Si un hiperplano H separa perfectamente las clases, también lo hace cualquier hiperplano obtenido al desplazar o rotar H ligeramente, siempre que no cruce ningún punto de entrenamiento. Ante esta ambigüedad, surge la pregunta clave: **¿cuál de todos esos hiperplanos debería elegirse?**

142.1.2. Por qué el margen importa

La respuesta de las SVM es elegir el hiperplano que maximiza el **margen**: la distancia mínima entre el hiperplano y los puntos de entrenamiento más cercanos de cada clase. La intuición es geométrica y probabilística a la vez. Un hiperplano con margen grande es más robusto frente a perturbaciones de los datos: si un punto nuevo cae cerca de la frontera de decisión, un margen amplio le da más “espacio” para ser clasificado correctamente. Un hiperplano con margen pequeño, aunque separe perfectamente los datos de entrenamiento, estará demasiado cerca de algunos puntos y será frágil ante datos nuevos.

La teoría de aprendizaje estadístico de Vapnik-Chervonenkis formaliza este argumento: la complejidad efectiva de un clasificador de margen máximo depende del margen, no de la dimensión del espacio de

características (Vapnik 1995). Esto significa que las SVM pueden generalizar bien incluso en dimensiones muy altas, siempre que el margen sea grande.

142.1.3. Intuición geométrica: margen = $2/\|\mathbf{w}\|$

Un hiperplano en $\mathbb{R}^d$ se define como el conjunto de puntos $\mathbf{x}$ que satisfacen:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

donde $\mathbf{w} \in \mathbb{R}^d$ es el vector normal al hiperplano y $b \in \mathbb{R}$ es el sesgo. La distancia con signo de un punto $\mathbf{x}_i$ al hiperplano es:

$$d_i = \frac{\mathbf{w}^\top \mathbf{x}_i + b}{\|\mathbf{w}\|}$$

Si normalizamos la escala de $\mathbf{w}$ de modo que los puntos más cercanos de cada clase satisfagan $|\mathbf{w}^\top \mathbf{x}_i + b| = 1$, entonces la distancia del hiperplano a esos puntos es $1/\|\mathbf{w}\|$. El margen total —la distancia entre las dos “bandas” de clase— es:

$$\text{margen} = \frac{2}{\|\mathbf{w}\|}$$

Maximizar el margen equivale a minimizar $\|\mathbf{w}\|$.

```
import numpy as np
import plotly.graph_objects as go

np.random.seed(42)

# Datos clase -1 (azul)
X_neg = np.column_stack([
    np.random.uniform(0.5, 2.5, 12),
    np.random.uniform(3.5, 6.0, 12)
])

# Datos clase +1 (naranja)
X_pos = np.column_stack([
    np.random.uniform(3.5, 6.0, 12),
    np.random.uniform(0.5, 3.0, 12)
])

fig = go.Figure()

# Puntos clase -1
fig.add_trace(go.Scatter(
    x=X_neg[:, 0], y=X_neg[:, 1],
    mode='markers',
    name='Clase -1',
    marker=dict(color='#4C72B0', size=10, symbol='circle')
```

```

))

# Puntos clase +1
fig.add_trace(go.Scatter(
    x=X_pos[:, 0], y=X_pos[:, 1],
    mode='markers',
    name='Clase +1',
    marker=dict(color='#DD8452', size=10, symbol='square')
))

x_line = np.linspace(0, 7, 200)

# Hiperplano óptimo de margen máximo: separa simétricamente ambas nubes
# w = (1, 1)/sqrt(2), b tal que la frontera pase por (3, 3)
# Línea:  $x + y = 6 \rightarrow y = 6 - x$  (aproximación visual)
y_opt = 6.5 - x_line
fig.add_trace(go.Scatter(
    x=x_line, y=y_opt,
    mode='lines',
    name='Margen máximo (óptimo)',
    line=dict(color='#2c4f8c', width=3)
))

# Hiperplanos subóptimos alternativos (punteados)
configs = [
    (5.5 - x_line, 'rgba(100,100,100,0.5)', 'Hiperplano alternativo 1'),
    (7.0 - 1.3 * x_line, 'rgba(130,130,130,0.5)', 'Hiperplano alternativo 2'),
    (5.0 - 0.7 * x_line, 'rgba(160,160,160,0.5)', 'Hiperplano alternativo 3'),
]
for y_alt, col, name in configs:
    fig.add_trace(go.Scatter(
        x=x_line, y=y_alt,
        mode='lines',
        name=name,
        line=dict(color=col, width=1.5, dash='dot')
    ))

fig.update_layout(
    title='Múltiples hiperplanos separadores: sólo uno maximiza el margen',
    xaxis_title='Característica 1',
    yaxis_title='Característica 2',
    xaxis=dict(range=[0, 7]),
    yaxis=dict(range=[0, 7]),
    height=480,
    template='plotly_white',
    legend=dict(orientation='h', yanchor='top', y=-0.18, xanchor='center', x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

(a) Múltiples hiperplanos válidos para datos linealmente separables. El hiperplano de margen máximo (línea sólida azul oscuro) es único y equidistante a ambas clases. Los otros hiperplanos (líneas punteadas grises) también separan correctamente los datos pero son subóptimos.

Unable to display output for mime type(s): text/html

(b)

Figura 142.1.

142.2. SVM de margen duro (datos linealmente separables)

142.2.1. Formulación primal

Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$, queremos encontrar el hiperplano $(\mathbf{w}, b)$ que maximice el margen $2/\|\mathbf{w}\|$ sujeto a que todos los puntos estén correctamente clasificados con al menos una distancia de $1/\|\mathbf{w}\|$ al hiperplano. Esto equivale a:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{sujeto a} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \dots, n \end{aligned} \tag{142.1}$$

La restricción $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ garantiza que los puntos de clase $+1$ queden al lado positivo del hiperplano (con valor funcional ≥ 1) y los de clase -1 queden al lado negativo (con valor funcional ≤ -1). Minimizar $\frac{1}{2} \|\mathbf{w}\|^2$ equivale a maximizar el margen $2/\|\mathbf{w}\|$.

142.2.2. Vectores de soporte

Los **vectores de soporte** son los puntos de entrenamiento que satisfacen la restricción con igualdad: $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Estos puntos están exactamente sobre las bandas de margen (los hiperplanos $\mathbf{w}^\top \mathbf{x} + b = \pm 1$) y son los únicos que determinan la solución. Si se eliminaran todos los demás puntos, el hiperplano óptimo no cambiaría. Esta propiedad de escasez (*sparsity*) es una de las características más importantes de las SVM.

142.2.3. Formulación dual y el rol del Lagrangiano

Para derivar la formulación dual, se introduce un multiplicador de Lagrange $\alpha_i \geq 0$ por cada restricción de la formulación primal (Ecuación 142.1). El Lagrangiano es:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1]$$

Tomando las condiciones de optimalidad de primer orden respecto a $\mathbf{w}$ y b :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0$$

Sustituyendo de regreso en el Lagrangiano, se obtiene el **problema dual**:

$$\max_{\alpha} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j \quad (142.2)$$

$$\text{sujeto a } \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0$$

La función dual depende de los datos sólo a través de productos internos $\mathbf{x}_i^{\top} \mathbf{x}_j$. Esta observación es el punto de partida del kernel trick.

142.2.4. Condiciones KKT

Las condiciones de Karush-Kuhn-Tucker (KKT) para este problema de optimización implican, entre otras cosas, la condición de complementariedad:

$$\alpha_i [y_i (\mathbf{w}^{\top} \mathbf{x}_i + b) - 1] = 0, \quad \forall i$$

Esta condición dice que, para cada punto, o bien $\alpha_i = 0$ (el punto no es vector de soporte y no contribuye a la solución) o bien $y_i (\mathbf{w}^{\top} \mathbf{x}_i + b) = 1$ (el punto está exactamente sobre el margen y es vector de soporte). En la práctica, la mayoría de los α_i son cero, lo que hace que la solución sea dispersa.

142.2.5. Función de decisión en forma dual

Una vez resuelto el problema dual y obtenidos los α_i , la función de decisión para un nuevo punto $\mathbf{x}$ es:

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i \in \mathcal{S}} \alpha_i y_i \mathbf{x}_i^{\top} \mathbf{x} + b \right) \quad (142.3)$$

donde $\mathcal{S}$ es el conjunto de vectores de soporte (aquellos con $\alpha_i > 0$). El sesgo b se recupera de cualquier vector de soporte usando $y_s (\mathbf{w}^{\top} \mathbf{x}_s + b) = 1$.

```
import numpy as np
import plotly.graph_objects as go

np.random.seed(7)

# Datos separables con margen claro
# Clase +1: arriba-derecha
X_pos = np.column_stack([
```

```

    np.random.uniform(3.2, 5.5, 10),
    np.random.uniform(3.2, 5.5, 10)
])
# Clase -1: abajo-izquierda
X_neg = np.column_stack([
    np.random.uniform(0.5, 2.8, 10),
    np.random.uniform(0.5, 2.8, 10)
])

# El hiperplano óptimo biseca el espacio: w = [1,1]/sqrt(2), b tal que
# frontera pasa por (3, 3): w^T x + b = 0 => x+y = 6 => b = -3*sqrt(2)
# En coordenadas normalizadas: y = 6 - x
# Banda +1: x + y = 7.2 => y = 7.2 - x (ajustado a vectores de soporte cercanos)
# Banda -1: x + y = 4.8 => y = 4.8 - x

# Encontrar vectores de soporte como puntos más cercanos al hiperplano x+y=6
w = np.array([1.0, 1.0]) / np.sqrt(2)
b_unnorm = -6.0 / np.sqrt(2) # hiperplano: w^T x + b = 0

dist_pos = (X_pos @ w + b_unnorm) # distancias firmadas clase +1
dist_neg = (X_neg @ w + b_unnorm) # distancias firmadas clase -1

# Vectores de soporte: punto más cercano de cada clase al hiperplano
sv_pos_idx = np.argmin(dist_pos) # el de menor distancia positiva
sv_neg_idx = np.argmax(dist_neg) # el de mayor distancia negativa (más cerca)

# Margen exacto basado en los SVs
margin_pos = dist_pos[sv_pos_idx] # distancia del SV+ al hiperplano
margin_neg = -dist_neg[sv_neg_idx] # distancia del SV- al hiperplano (positiva)

# Líneas del hiperplano y del margen
x_line = np.linspace(0, 7, 300)
y_hyp = 6.0 - x_line # frontera: x+y=6
y_marg_pos = 6.0 + margin_pos*np.sqrt(2) - x_line # banda +1
y_marg_neg = 6.0 - margin_neg*np.sqrt(2) - x_line # banda -1

fig = go.Figure()

# Región de margen sombreada
fig.add_trace(go.Scatter(
    x=np.concatenate([x_line, x_line[::-1]]),
    y=np.concatenate([y_marg_pos, y_marg_neg[::-1]]),
    fill='toself',
    fillcolor='rgba(150,150,150,0.12)',
    line=dict(color='rgba(0,0,0,0)'),
    name='Región de margen',
    showlegend=True
)))

```

```

# Banda margen +1
fig.add_trace(go.Scatter(
    x=x_line, y=y_marg_pos,
    mode='lines',
    name='Margen +1',
    line=dict(color='#DD8452', width=1.5, dash='dash')
))

# Banda margen -1
fig.add_trace(go.Scatter(
    x=x_line, y=y_marg_neg,
    mode='lines',
    name='Margen -1',
    line=dict(color='#4C72B0', width=1.5, dash='dash')
))

# Hiperplano óptimo
fig.add_trace(go.Scatter(
    x=x_line, y=y_hyp,
    mode='lines',
    name='Hiperplano óptimo',
    line=dict(color='#2c4f8c', width=2.5)
))

# Puntos clase -1 (sin SV)
mask_neg = np.ones(len(X_neg), dtype=bool)
mask_neg[sv_neg_idx] = False
fig.add_trace(go.Scatter(
    x=X_neg[mask_neg, 0], y=X_neg[mask_neg, 1],
    mode='markers',
    name='Clase -1',
    marker=dict(color='#4C72B0', size=10)
))

# Puntos clase +1 (sin SV)
mask_pos = np.ones(len(X_pos), dtype=bool)
mask_pos[sv_pos_idx] = False
fig.add_trace(go.Scatter(
    x=X_pos[mask_pos, 0], y=X_pos[mask_pos, 1],
    mode='markers',
    name='Clase +1',
    marker=dict(color='#DD8452', size=10, symbol='square')
))

# Vectores de soporte con borde negro
fig.add_trace(go.Scatter(
    x=[X_neg[sv_neg_idx, 0]], y=[X_neg[sv_neg_idx, 1]],
    mode='markers',
    name='Vector de soporte -1',
    marker=dict(
        color='#4C72B0', size=14,
        line=dict(color='black', width=2.5)
    )
))

```

```

    )
))
fig.add_trace(go.Scatter(
    x=[X_pos[sv_pos_idx, 0]], y=[X_pos[sv_pos_idx, 1]],
    mode='markers',
    name='Vector de soporte +1',
    marker=dict(
        color='#DD8452', size=14, symbol='square',
        line=dict(color='black', width=2.5)
    )
))

# Flecha indicando el margen
mid_x = 5.5
mid_y_hyp = 6.0 - mid_x
mid_y_marg = 6.0 + margin_pos*np.sqrt(2) - mid_x
fig.add_annotation(
    x=mid_x, y=mid_y_marg,
    ax=mid_x, ay=mid_y_hyp,
    xref='x', yref='y', axref='x', ayref='y',
    arrowhead=2, arrowsize=1, arrowwidth=2,
    arrowcolor='#555555',
    text=f' 1/||w||',
    font=dict(size=11, color='#555555')
)

fig.update_layout(
    title='SVM de margen duro: hiperplano óptimo y vectores de soporte',
    xaxis_title='Característica 1',
    yaxis_title='Característica 2',
    xaxis=dict(range=[0, 7]),
    yaxis=dict(range=[0, 7]),
    height=500,
    template='plotly_white',
    legend=dict(orientation='h', yanchor='top', y=-0.18, xanchor='center', x=0.5)
)
fig.show()

```


142.3. SVM de margen suave (datos no separables)

En la práctica, los datos raramente son linealmente separables. Incluso cuando la separación es posible, un pequeño número de puntos atípicos puede impedir encontrar cualquier hiperplano válido. La SVM de **margen suave** aborda este problema introduciendo variables de holgura que permiten violar las restricciones de manera controlada (Cortes y Vapnik 1995).

142.3.1. Variables de holgura

Para cada punto de entrenamiento se introduce una variable de holgura $\xi_i \geq 0$ que mide cuánto viola la restricción de margen ese punto:

- $\xi_i = 0$: el punto está correctamente clasificado y fuera del margen.
- $0 < \xi_i \leq 1$: el punto está correctamente clasificado pero dentro del margen.
- $\xi_i > 1$: el punto está mal clasificado.

142.3.2. Formulación primal con margen suave

El problema de optimización se modifica para penalizar las violaciones:

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad (142.4)$$

$$\text{sujeto a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i$$

El parámetro $C > 0$ controla el equilibrio entre maximizar el margen y minimizar las violaciones:

- $C \rightarrow \infty$: se penalizan fuertemente los errores, equivale al margen duro.
- $C \rightarrow 0$: se permiten muchos errores, el modelo ignora casi todos los puntos y el margen se hace arbitrariamente grande.

La formulación dual correspondiente es similar a la del margen duro, con la diferencia de que los multiplicadores están acotados superiormente por C :

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \\ \text{sujeto a} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

La cota superior $\alpha_i \leq C$ es la única diferencia respecto al problema dual de margen duro, y es una consecuencia directa de las variables de holgura en el Lagrangiano.

```

import numpy as np
import plotly.graph_objects as go

np.random.seed(21)
n = 30

# Datos no linealmente separables (con solapamiento)
X_pos = np.column_stack([
    np.random.normal(3.5, 0.9, n),
    np.random.normal(3.5, 0.9, n)
])
X_neg = np.column_stack([
    np.random.normal(2.0, 0.9, n),
    np.random.normal(2.0, 0.9, n)
])

X = np.vstack([X_pos, X_neg])
y = np.array([1]*n + [-1]*n)

def svm_soft_approx(X, y, C):
    """
    Aproximación del hiperplano SVM de margen suave para datos 2D.
    Usa proyección sobre la dirección de la diferencia de medias,
    escalada por C para simular el efecto del parámetro.
    """
    mu_pos = X[y == 1].mean(axis=0)
    mu_neg = X[y == -1].mean(axis=0)
    w_dir = mu_pos - mu_neg
    w_dir = w_dir / np.linalg.norm(w_dir)

    # La frontera pasa por el punto medio, ajustada por C
    mid = (mu_pos + mu_neg) / 2.0

    # Con C grande el margen es estrecho (parecido al margen duro)
    # Con C pequeño se abre el margen
    margin_scale = 1.0 / (1.0 + 0.3 * np.log1p(C))

    b = -np.dot(w_dir, mid)
    return w_dir, b, margin_scale

C_values = [0.01, 0.1, 1.0, 10.0, 100.0]
x_line = np.linspace(-1, 7, 300)

frames = []
for C in C_values:
    w, b, ms = svm_soft_approx(X, y, C)
    # Frontera:  $w[0]*x + w[1]*y + b = 0 \Rightarrow y = -(w[0]*x + b)/w[1]$ 
    y_hyp = -(w[0]*x_line + b) / w[1]
    # Bandas de margen ( $\pm ms / ||w||$ , pero w ya está normalizado)

```

```

y_mp = -(w[0]*x_line + b - ms) / w[1]
y_mn = -(w[0]*x_line + b + ms) / w[1]

# Clasificar puntos
scores = X @ w + b
correct = (scores * y) >= 0
in_margin = (np.abs(scores) < ms) & correct
violated = ~correct

frame_data = [
    # Región de margen
    go.Scatter(
        x=np.concatenate([x_line, x_line[:, -1]]),
        y=np.concatenate([y_mp, y_mn[:, -1]]),
        fill='toself',
        fillcolor='rgba(150,150,150,0.15)',
        line=dict(color='rgba(0,0,0,0)'),
        name='Región de margen',
        showlegend=True
    ),
    go.Scatter(x=x_line, y=y_mp, mode='lines',
        line=dict(color='DD8452', dash='dash', width=1.5),
        name='Banda +1', showlegend=True),
    go.Scatter(x=x_line, y=y_mn, mode='lines',
        line=dict(color='4C72B0', dash='dash', width=1.5),
        name='Banda -1', showlegend=True),
    go.Scatter(x=x_line, y=y_hyp, mode='lines',
        line=dict(color='2c4f8c', width=2.5),
        name='Hiperplano', showlegend=True),
    # Clase +1 correctos
    go.Scatter(
        x=X[y==1][correct[y==1], 0], y=X[y==1][correct[y==1], 1],
        mode='markers', name='Clase +1',
        marker=dict(color='DD8452', size=9, symbol='square'),
        showlegend=True
    ),
    # Clase -1 correctos
    go.Scatter(
        x=X[y==-1][correct[y==-1], 0], y=X[y==-1][correct[y==-1], 1],
        mode='markers', name='Clase -1',
        marker=dict(color='4C72B0', size=9),
        showlegend=True
    ),
    # Puntos violados
    go.Scatter(
        x=X[violated, 0], y=X[violated, 1],
        mode='markers', name='Mal clasificados',
        marker=dict(color='red', size=10, symbol='x', line=dict(width=2)),
        showlegend=True
    )
]

```

```

    ),
]
frames.append(go.Frame(data=frame_data, name=str(C)))

# Figura inicial (C=1)
w0, b0, ms0 = svm_soft_approx(X, y, 1.0)
y_hyp0 = -(w0[0]*x_line + b0) / w0[1]
y_mp0 = -(w0[0]*x_line + b0 - ms0) / w0[1]
y_mn0 = -(w0[0]*x_line + b0 + ms0) / w0[1]
scores0 = X @ w0 + b0
correct0 = (scores0 * y) >= 0
violated0 = ~correct0

fig = go.Figure(
    data=[
        go.Scatter(
            x=np.concatenate([x_line, x_line[:-1]]),
            y=np.concatenate([y_mp0, y_mn0[:-1]]),
            fill='toself', fillcolor='rgba(150,150,150,0.15)',
            line=dict(color='rgba(0,0,0,0)'), name='Región de margen'
        ),
        go.Scatter(x=x_line, y=y_mp0, mode='lines',
            line=dict(color='#DD8452', dash='dash', width=1.5), name='Banda +1'),
        go.Scatter(x=x_line, y=y_mn0, mode='lines',
            line=dict(color='#4C72B0', dash='dash', width=1.5), name='Banda -1'),
        go.Scatter(x=x_line, y=y_hyp0, mode='lines',
            line=dict(color='#2c4f8c', width=2.5), name='Hiperplano'),
        go.Scatter(x=X[y==1][correct0[y==1], 0], y=X[y==1][correct0[y==1], 1],
            mode='markers', name='Clase +1',
            marker=dict(color='#DD8452', size=9, symbol='square')),
        go.Scatter(x=X[y==-1][correct0[y==-1], 0], y=X[y==-1][correct0[y==-1], 1],
            mode='markers', name='Clase -1',
            marker=dict(color='#4C72B0', size=9)),
        go.Scatter(x=X[violated0, 0], y=X[violated0, 1],
            mode='markers', name='Mal clasificados',
            marker=dict(color='red', size=10, symbol='x', line=dict(width=2))),
    ],
    frames=frames
)

slider_steps = []
for C in C_values:
    slider_steps.append(dict(
        args=[[str(C)], dict(frame=dict(duration=300), mode='immediate',
            transition=dict(duration=200))],
        label=str(C),
        method='animate'
    ))

```

```

fig.update_layout(
    title='Efecto del parámetro C: trade-off margen vs errores de clasificación',
    xaxis_title='Característica 1',
    yaxis_title='Característica 2',
    xaxis=dict(range=[-0.5, 6.5]),
    yaxis=dict(range=[-0.5, 6.5]),
    height=490,
    template='plotly_white',
    updatemenus=[dict(
        type='buttons', showactive=False,
        y=1.1, x=0.0, xanchor='left',
        buttons=[
            dict(label=' Animar', method='animate',
                args=[None, dict(frame=dict(duration=800), fromcurrent=True,
                    transition=dict(duration=400))]),
            dict(label=' Pausar', method='animate',
                args=[[None], dict(mode='immediate', frame=dict(duration=0),
                    transition=dict(duration=0))])
        ]
    )],
    sliders=[dict(
        currentvalue=dict(prefix='C = ', font=dict(size=14)),
        steps=slider_steps,
        pad=dict(t=50)
    )],
    legend=dict(orientation='h', yanchor='top', y=-0.18, xanchor='center', x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 142.3.: Efecto del parámetro C en la SVM de margen suave. Un C grande penaliza los errores fuertemente (margen estrecho, pocos puntos mal clasificados); un C pequeño permite más violaciones a cambio de un margen más amplio. Use el slider para explorar diferentes valores.

142.4. El kernel trick

142.4.1. El problema de la no separabilidad lineal

La SVM de margen duro y suave opera con hiperplanos: fronteras de decisión lineales en el espacio de características. Sin embargo, muchos problemas del mundo real tienen estructuras no lineales: dos clases pueden estar dispuestas en anillos concéntricos, en espirales, o en cualquier forma que ningún hiperplano puede separar.

La solución clásica sería transformar los datos a un espacio de mayor dimensión donde sí sean linealmente separables. Si $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ es un mapeo a un espacio de dimensión $D \gg d$, entonces se aplicaría la SVM sobre los puntos transformados $\phi(\mathbf{x}_i)$. El problema es que si D es muy grande (incluso infinito), calcular $\phi(\mathbf{x})$ explícitamente puede ser computacionalmente inviable.

142.4.2. El kernel trick: producto interno implícito

La clave está en observar que la formulación dual de la SVM (Ecuación 142.2) y la función de decisión (Ecuación 142.3) dependen de los datos únicamente a través de productos internos $\mathbf{x}_i^\top \mathbf{x}_j$. Si se aplica el mapeo ϕ , estos productos se convierten en $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.

Una **función kernel** es una función $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ que computa este producto interno de manera implícita:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad (142.5)$$

El kernel evalúa el producto interno en el espacio de alta dimensión sin necesidad de calcular $\phi(\mathbf{x})$ explícitamente. Basta con sustituir $\mathbf{x}_i^\top \mathbf{x}_j$ por $K(\mathbf{x}_i, \mathbf{x}_j)$ en todas las expresiones. Esto se conoce como el *kernel trick* (Schölkopf y Smola 2002).

142.4.3. Kernels más utilizados

Kernel lineal:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$

Equivale a no hacer transformación alguna. Produce fronteras de decisión lineales en el espacio original.

Kernel polinomial:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)^d$$

Donde d es el grado del polinomio, $\gamma > 0$ y $r \geq 0$. El mapeo implícito incluye todos los monomios de grado hasta d . Por ejemplo, con $d = 2$ y $\mathbf{x} \in \mathbb{R}^2$, el espacio transformado incluye términos $x_1^2, x_2^2, x_1 x_2, x_1, x_2, 1$.

Kernel RBF (Gaussiano):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (142.6)$$

El parámetro $\gamma > 0$ controla el ancho del gaussiano. Este kernel corresponde a un mapeo a un espacio de dimensión infinita: su expansión en serie de Taylor genera todos los polinomios de todos los grados. Geométricamente, $K(\mathbf{x}_i, \mathbf{x}_j)$ mide la similitud entre dos puntos: vale 1 cuando son idénticos y decae a 0 conforme se alejan. El radio de influencia de cada punto de entrenamiento es $\sim 1/\sqrt{\gamma}$.

Kernel sigmoide:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)$$

Este kernel no cumple las condiciones de Mercer para todos los valores de γ y r , lo que puede causar problemas numéricos en algunos casos.

142.4.4. Condición de Mercer

No toda función simétrica puede ser un kernel válido. La **condición de Mercer** establece que K es un kernel válido (positivo semidefinido) si y sólo si, para cualquier conjunto finito de puntos $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, la matriz de Gram $\mathbf{K}$ con entradas $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ es positiva semidefinida (Schölkopf y Smola 2002):

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(\mathbf{x}_i, \mathbf{x}_j) \geq 0, \quad \forall c_i \in \mathbb{R}$$

Esta condición garantiza que existe un mapeo ϕ tal que $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, y que el problema de optimización dual es cóncavo (tiene solución única).

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(3)

# Datos 1D no separables: clase +1 lejos del origen, clase -1 cerca
x_neg = np.random.uniform(-1.0, 1.0, 20)          # clase -1: cerca del 0
x_pos = np.concatenate([
    np.random.uniform(-2.5, -1.2, 10),
    np.random.uniform(1.2, 2.5, 10)
])          # clase +1: lejos del 0

# Mapeo: phi(x) = (x, x^2)
phi_neg = np.column_stack([x_neg, x_neg**2])
phi_pos = np.column_stack([x_pos, x_pos**2])

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=[
        'Espacio original 1 (no separable)',
        'Espacio transformado 2 con phi(x)=(x, x^2) (separable)'
    ]
)

# Panel izquierdo: espacio 1D (mostrado con jitter en y)
jitter_neg = np.random.uniform(-0.05, 0.05, len(x_neg))
jitter_pos = np.random.uniform(-0.05, 0.05, len(x_pos))

fig.add_trace(go.Scatter(
    x=x_neg, y=jitter_neg,
    mode='markers', name='Clase -1',
    marker=dict(color='#4C72B0', size=10),
    legendgroup='neg'
), row=1, col=1)

fig.add_trace(go.Scatter(
```

```

    x=x_pos, y=jitter_pos,
    mode='markers', name='Clase +1',
    marker=dict(color='#DD8452', size=10, symbol='square'),
    legendgroup='pos'
), row=1, col=1)

# Línea horizontal de referencia
fig.add_trace(go.Scatter(
    x=[-3, 3], y=[0, 0],
    mode='lines', line=dict(color='gray', width=1),
    showlegend=False
), row=1, col=1)

# Panel derecho: espacio 2D transformado
fig.add_trace(go.Scatter(
    x=phi_neg[:, 0], y=phi_neg[:, 1],
    mode='markers', name='Clase -1 ( )',
    marker=dict(color='#4C72B0', size=10),
    legendgroup='neg', showlegend=False
), row=1, col=2)

fig.add_trace(go.Scatter(
    x=phi_pos[:, 0], y=phi_pos[:, 1],
    mode='markers', name='Clase +1 ( )',
    marker=dict(color='#DD8452', size=10, symbol='square'),
    legendgroup='pos', showlegend=False
), row=1, col=2)

# Hiperplano separador en el espacio transformado:  $y = 1.3$  (línea horizontal)
x_sep = np.linspace(-3, 3, 100)
fig.add_trace(go.Scatter(
    x=x_sep, y=np.full_like(x_sep, 1.3),
    mode='lines', name='Hiperplano separador',
    line=dict(color='#2c4f8c', width=2.5, dash='dash')
), row=1, col=2)

fig.update_xaxes(title_text='x', row=1, col=1)
fig.update_yaxes(title_text='', showticklabels=False, row=1, col=1)
fig.update_xaxes(title_text='(x) = x', row=1, col=2)
fig.update_yaxes(title_text='(x) = x2', row=1, col=2)

fig.update_layout(
    height=430,
    template='plotly_white',
    title='El kernel trick: datos no separables en 1 → separables en 2',
    legend=dict(orientation='h', yanchor='top', y=-0.18, xanchor='center', x=0.5)
)
fig.show()

```


Unable to display output for mime type(s): text/html

Figura 142.4.: Transformación de datos no linealmente separables a un espacio de mayor dimensión. En el espacio original 1D (izquierda), los puntos de las dos clases se solapan. Tras el mapeo $\phi(x) = (x, x^2)$ (derecha), los datos se vuelven linealmente separables en 2D.

142.5. Efecto de los hiperparámetros C y γ (kernel RBF)

Cuando se usa el kernel RBF (Ecuación 142.6), la SVM tiene dos hiperparámetros principales que deben ajustarse mediante validación cruzada (Hastie et al. 2009):

Parámetro C (regularización):

- **C grande:** Se penalizan fuertemente los errores de clasificación. El modelo intenta clasificar correctamente casi todos los puntos de entrenamiento, lo que puede producir fronteras muy irregulares y sobreajuste.
- **C pequeño:** Se permite más margen a cambio de tolerar más errores en entrenamiento. El modelo generaliza mejor en datos ruidosos.

Parámetro γ (ancho del kernel RBF):

- **grande:** El radio de influencia de cada punto de soporte es pequeño ($\sim 1/\sqrt{\gamma}$). La frontera de decisión se vuelve muy irregular, capturando la forma local de cada punto. Alto riesgo de sobreajuste.
- **pequeño:** Cada punto de soporte influye sobre una región amplia. La frontera es suave. Alto riesgo de subajuste si γ es demasiado pequeño.

La interacción entre C y γ es compleja: un C grande con un γ grande produce el modelo más complejo (muy propenso a sobreajuste). Un C pequeño con un γ pequeño produce el modelo más simple (frontera suave, posible subajuste).

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(11)

# Datos con estructura circular (moons sintético manual)
n = 40
theta_pos = np.random.uniform(0, np.pi, n)
theta_neg = np.random.uniform(np.pi, 2*np.pi, n)

X_pos = np.column_stack([
    np.cos(theta_pos) + np.random.normal(0, 0.15, n),
    np.sin(theta_pos) + np.random.normal(0, 0.15, n)
])
X_neg = np.column_stack([
    np.cos(theta_neg) + 1 + np.random.normal(0, 0.15, n),
    np.sin(theta_neg) - 0.3 + np.random.normal(0, 0.15, n)
])
```

```

X = np.vstack([X_pos, X_neg])
y = np.array([1]*n + [-1]*n)

def rbf_kernel(X1, X2, gamma):
    diff = X1[:, None, :] - X2[None, :, :]
    return np.exp(-gamma * np.sum(diff**2, axis=-1))

def decision_function_approx(X_train, y_train, X_grid, C, gamma):
    """
    Aproximación de la función de decisión SVM con kernel RBF.
    Usa pesos proporcionales a C y al kernel, simplificado para visualización.
    """
    K = rbf_kernel(X_train, X_train, gamma)
    n_train = len(y_train)
    # Pesos heurísticos: alpha_i proporcional a C, modulado por clase
    alpha = np.minimum(C, 1.0) * np.ones(n_train)
    # Función de decisión: suma ponderada de kernels
    K_pred = rbf_kernel(X_grid, X_train, gamma)
    f = K_pred @ (alpha * y_train)
    # Normalizar
    f = f / (np.max(np.abs(f)) + 1e-10)
    return f

# Grid de evaluación
x1_range = np.linspace(-2, 3, 80)
x2_range = np.linspace(-1.5, 1.5, 80)
xx, yy = np.meshgrid(x1_range, x2_range)
X_grid = np.column_stack([xx.ravel(), yy.ravel()])

configs = [
    (0.1, 0.5, 'C=0.1, =0.5\n(subajuste)'),
    (10, 0.5, 'C=10, =0.5\n(equilibrado)'),
    (0.1, 5.0, 'C=0.1, =5\n(margen amplio, local)'),
    (10, 5.0, 'C=10, =5\n(sobreaajuste)'),
]

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=[c[2].replace('\n', ' - ') for c in configs],
    horizontal_spacing=0.08,
    vertical_spacing=0.12
)

positions = [(1,1),(1,2),(2,1),(2,2)]

for idx, (C, gamma, title) in enumerate(configs):
    row, col = positions[idx]
    f = decision_function_approx(X, y, X_grid, C, gamma)
    Z = f.reshape(xx.shape)

```

```

# Contorno de la frontera
fig.add_trace(go.Contour(
    x=x1_range, y=x2_range, z=Z,
    contours=dict(
        start=0, end=0, size=0.01,
        coloring='lines',
        showlabels=False
    ),
    line=dict(color='#2c4f8c', width=2.5),
    colorscale='RdBu',
    showscale=False,
    opacity=0.85,
    name=f'Frontera C={C},  $\gamma$ = $\gamma$ '
), row=row, col=col)

# Región coloreada suavemente
fig.add_trace(go.Contour(
    x=x1_range, y=x2_range, z=Z,
    contours=dict(start=-1, end=1, size=2),
    colorscale=[
        [0.0, 'rgba(76,114,176,0.18)'],
        [0.5, 'rgba(255,255,255,0.0)'],
        [1.0, 'rgba(221,132,82,0.18)']
    ],
    showscale=False,
    line=dict(width=0)
), row=row, col=col)

# Puntos
fig.add_trace(go.Scatter(
    x=X_neg[:, 0], y=X_neg[:, 1],
    mode='markers', showlegend=(idx == 0),
    name='Clase -1',
    marker=dict(color='#4C72B0', size=7)
), row=row, col=col)
fig.add_trace(go.Scatter(
    x=X_pos[:, 0], y=X_pos[:, 1],
    mode='markers', showlegend=(idx == 0),
    name='Clase +1',
    marker=dict(color='#DD8452', size=7, symbol='square')
), row=row, col=col)

fig.update_layout(
    title='Efecto de los hiperparámetros  $C$  y  $\gamma$  en la frontera de decisión (kernel RBF)',
    height=520,
    template='plotly_white',
    legend=dict(orientation='h', yanchor='top', y=-0.08, xanchor='center', x=0.5)
)
for r in [1, 2]:

```

```

    for c in [1, 2]:
        fig.update_xaxes(title_text='Característica 1', row=r, col=c)
        fig.update_yaxes(title_text='Característica 2', row=r, col=c)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 142.5.: Efecto combinado de C y γ en la frontera de decisión SVM con kernel RBF. Cada cuadrante muestra una combinación diferente de hiperparámetros. La frontera se estima numéricamente desde la función de distancia al hiperplano aproximada.

142.6. Comparación de kernels en datos reales

Un paso importante en la práctica es comparar diferentes kernels sobre los mismos datos para determinar cuál captura mejor la estructura subyacente. En datos con estructura circular o no lineal, el kernel lineal fracasará, mientras que el kernel RBF con parámetros adecuados puede lograr una separación casi perfecta (Bishop 2006).

```

import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(17)

# Datos circulares: clase -1 en anillo interior, clase +1 en anillo exterior
n = 45
r_neg = np.random.uniform(0.0, 0.8, n)
r_pos = np.random.uniform(1.2, 2.0, n)
theta_neg = np.random.uniform(0, 2*np.pi, n)
theta_pos = np.random.uniform(0, 2*np.pi, n)

X_neg = np.column_stack([
    r_neg * np.cos(theta_neg) + np.random.normal(0, 0.08, n),
    r_neg * np.sin(theta_neg) + np.random.normal(0, 0.08, n)
])
X_pos = np.column_stack([
    r_pos * np.cos(theta_pos) + np.random.normal(0, 0.08, n),
    r_pos * np.sin(theta_pos) + np.random.normal(0, 0.08, n)
])

X = np.vstack([X_neg, X_pos])
y = np.array([-1]*n + [1]*n)

# Kernels
def k_linear(X1, X2):
    return X1 @ X2.T

```

```

def k_poly(X1, X2, degree=3, gamma=1.0, r=1.0):
    return (gamma * X1 @ X2.T + r)**degree

def k_rbf(X1, X2, gamma=1.0):
    diff = X1[:, None, :] - X2[None, :, :]
    return np.exp(-gamma * np.sum(diff**2, axis=-1))

def k_sigmoid(X1, X2, gamma=0.5, r=0.0):
    return np.tanh(gamma * X1 @ X2.T + r)

def decision_from_kernel(K_train, K_pred, y):
    """Aproximación de la función de decisión desde la matriz kernel."""
    alpha = np.ones(len(y))
    f = K_pred @ (alpha * y)
    return f / (np.max(np.abs(f)) + 1e-10)

x1_range = np.linspace(-2.5, 2.5, 90)
x2_range = np.linspace(-2.5, 2.5, 90)
xx, yy = np.meshgrid(x1_range, x2_range)
X_grid = np.column_stack([xx.ravel(), yy.ravel()])

kernels = [
    ('Lineal', lambda Xg: decision_from_kernel(k_linear(X, X), k_linear(Xg, X), y)),
    ('Polinomial (d=3)', lambda Xg: decision_from_kernel(k_poly(X, X), k_poly(Xg, X), y)),
    ('RBF (=1)', lambda Xg: decision_from_kernel(k_rbf(X, X, 1.0), k_rbf(Xg, X, 1.0), y)),
    ('Sigmoide', lambda Xg: decision_from_kernel(k_sigmoid(X, X), k_sigmoid(Xg, X), y)),
]

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=[k[0] for k in kernels],
    horizontal_spacing=0.08,
    vertical_spacing=0.12
)

for idx, (name, kernel_fn) in enumerate(kernels):
    row = idx // 2 + 1
    col = idx % 2 + 1

    f = kernel_fn(X_grid)
    Z = f.reshape(xx.shape)

    fig.add_trace(go.Contour(
        x=x1_range, y=x2_range, z=Z,
        colorscale=[
            [0.0, 'rgba(76,114,176,0.22)'],
            [0.5, 'rgba(255,255,255,0.0)'],
            [1.0, 'rgba(221,132,82,0.22)']
        ],
    ),

```

```

        contours=dict(start=-1, end=1, size=2),
        showscale=False, line=dict(width=0)
    ), row=row, col=col)

fig.add_trace(go.Contour(
    x=x1_range, y=x2_range, z=Z,
    contours=dict(start=0, end=0, size=0.01,
                  coloring='lines', showlabels=False),
    line=dict(color='#2c4f8c', width=2.5),
    showscale=False
), row=row, col=col)

fig.add_trace(go.Scatter(
    x=X_neg[:, 0], y=X_neg[:, 1],
    mode='markers', showlegend=(idx == 0),
    name='Clase -1',
    marker=dict(color='#4C72B0', size=7)
), row=row, col=col)
fig.add_trace(go.Scatter(
    x=X_pos[:, 0], y=X_pos[:, 1],
    mode='markers', showlegend=(idx == 0),
    name='Clase +1',
    marker=dict(color='#DD8452', size=7, symbol='square')
), row=row, col=col)

fig.update_layout(
    title='Comparación de kernels en datos con estructura circular',
    height=520,
    template='plotly_white',
    legend=dict(orientation='h', yanchor='top', y=-0.08, xanchor='center', x=0.5)
)
for r in [1, 2]:
    for c in [1, 2]:
        fig.update_xaxes(title_text='Característica 1', row=r, col=c)
        fig.update_yaxes(title_text='Característica 2', row=r, col=c)
fig.show()

```

142.7. SVM para regresión (SVR)

Las ideas de las SVM se extienden naturalmente al problema de regresión mediante la **Support Vector Regression** (SVR) (Vapnik 1995). La diferencia fundamental respecto a la regresión lineal clásica es que la SVR no penaliza errores pequeños: introduce un margen de tolerancia ε alrededor de la función de regresión, y sólo penaliza las predicciones que caen fuera de ese tubo.

142.7.1. El tubo ε -insensible

La función de pérdida ε -insensible es:

$$L_\varepsilon(y, f(\mathbf{x})) = \max(0, |y - f(\mathbf{x})| - \varepsilon)$$

Esta función vale cero cuando el error de predicción es menor que ε , y crece linealmente con el error por encima de ese umbral. Geométricamente, define un **tubo** de ancho 2ε alrededor de la función de regresión: los puntos dentro del tubo no contribuyen a la función de pérdida.

142.7.2. Formulación primal de SVR

Introduciendo variables de holgura $\xi_i^+, \xi_i^- \geq 0$ para las violaciones por encima y por debajo del tubo respectivamente, el problema de optimización es:

$$\min_{\mathbf{w}, b, \xi^+, \xi^-} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i^+ + \xi_i^-)$$

$$\text{sujeto a } y_i - f(\mathbf{x}_i) \leq \varepsilon + \xi_i^+$$

$$f(\mathbf{x}_i) - y_i \leq \varepsilon + \xi_i^-$$

$$\xi_i^+, \xi_i^- \geq 0$$

donde $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. Los **vectores de soporte en SVR** son los puntos que caen fuera o exactamente sobre el borde del tubo ($\xi_i^+ > 0$ o $\xi_i^- > 0$, o que satisfacen la igualdad). Igual que en clasificación, la solución es dispersa: sólo los vectores de soporte determinan $\mathbf{w}$.

```
import numpy as np
import plotly.graph_objects as go

np.random.seed(5)

n = 40
x = np.linspace(0, 10, n)
y_true = 2.5 * np.sin(0.8 * x) + 0.5 * x
noise = np.random.normal(0, 0.8, n)
# Algunos outliers
```

```

outlier_mask = np.random.choice(n, 4, replace=False)
noise[outlier_mask] *= 4
y = y_true + noise

# SVR aproximada: función suavizada (media móvil ponderada por kernel RBF)
def svr_predict(x_train, y_train, x_pred, gamma=0.3, C=1.0, eps=0.8):
    """SVR aproximada usando suavizado por kernel."""
    K = np.exp(-gamma * (x_train[:, None] - x_train[None, :])**2)
    K_pred = np.exp(-gamma * (x_pred[:, None] - x_train[None, :])**2)
    # Pesos: kernel ridge regression con regularización 1/C
    alpha = np.linalg.solve(K + np.eye(len(x_train)) / C, y_train)
    f = K_pred @ alpha
    return f

eps = 0.9
x_fine = np.linspace(-0.2, 10.2, 300)
y_svr = svr_predict(x, y, x_fine, gamma=0.4, C=2.0, eps=eps)

# Regresión lineal mínimos cuadrados
coeffs = np.polyfit(x, y, 1)
y_ols = np.polyval(coeffs, x_fine)

# Vectores de soporte: puntos fuera del tubo
y_svr_train = svr_predict(x, y, x, gamma=0.4, C=2.0, eps=eps)
residuals = np.abs(y - y_svr_train)
sv_mask = residuals > eps * 0.95 # puntos sobre o fuera del tubo

fig = go.Figure()

# Tubo
fig.add_trace(go.Scatter(
    x=np.concatenate([x_fine, x_fine[::-1]]),
    y=np.concatenate([y_svr + eps, (y_svr - eps)[::-1]]),
    fill='toself',
    fillcolor='rgba(44,79,140,0.12)',
    line=dict(color='rgba(0,0,0,0)'),
    name=f'Tubo = {eps}'
))

# Bordes del tubo
fig.add_trace(go.Scatter(
    x=x_fine, y=y_svr + eps,
    mode='lines', line=dict(color='#4C72B0', dash='dash', width=1.2),
    name='Borde superior (f + )'
))

fig.add_trace(go.Scatter(
    x=x_fine, y=y_svr - eps,
    mode='lines', line=dict(color='#4C72B0', dash='dash', width=1.2),
    name='Borde inferior (f - )'
))

```



```

))

# Función SVR
fig.add_trace(go.Scatter(
    x=x_fine, y=y_svr,
    mode='lines', line=dict(color='#2c4f8c', width=2.8),
    name='SVR'
))

# Regresión lineal
fig.add_trace(go.Scatter(
    x=x_fine, y=y_ols,
    mode='lines', line=dict(color='#DD8452', dash='dot', width=2),
    name='Regresión lineal OLS'
))

# Puntos dentro del tubo
fig.add_trace(go.Scatter(
    x=x[~sv_mask], y=y[~sv_mask],
    mode='markers', name='Dentro del tubo',
    marker=dict(color='#4C72B0', size=8, opacity=0.7)
))

# Vectores de soporte
fig.add_trace(go.Scatter(
    x=x[sv_mask], y=y[sv_mask],
    mode='markers', name='Vectores de soporte',
    marker=dict(
        color='#DD8452', size=11, symbol='circle',
        line=dict(color='black', width=2.5)
    )
))

fig.update_layout(
    title='SVR con tubo -insensible vs Regresión lineal OLS',
    xaxis_title='Variable independiente',
    yaxis_title='Variable dependiente',
    height=460,
    template='plotly_white',
    legend=dict(orientation='h', yanchor='top', y=-0.18, xanchor='center', x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 142.7.: SVR con tubo ϵ -insensible. La función de regresión (línea azul) está rodeada por un tubo de ancho 2ϵ (región gris). Los puntos dentro del tubo no contribuyen a la pérdida. Los vectores de soporte (borde negro) son los puntos sobre o fuera del tubo. Se compara con la regresión lineal mínimo cuadrados (línea naranja punteada).

142.8. Aspectos computacionales

142.8.1. Complejidad del entrenamiento

El entrenamiento de una SVM implica resolver un problema de programación cuadrática (QP) con n variables (α_i) y $n + 1$ restricciones. Los algoritmos de QP de propósito general tienen complejidad $O(n^3)$ en tiempo y $O(n^2)$ en memoria debido a la necesidad de calcular y almacenar la matriz de Gram $\mathbf{K}$ de tamaño $n \times n$.

Esto hace que las SVM estándar sean impracticables para $n > 10,000$ a 50,000 puntos, dependiendo de la memoria disponible. Para n grande, existen varias estrategias:

- **SMO (Sequential Minimal Optimization):** Algoritmo de Platt (Platt 1999) que descompone el problema QP en subproblemas de tamaño 2 que se resuelven analíticamente. Es el algoritmo más usado en implementaciones modernas como LIBSVM (Chang y Lin 2011).
- **Chunking y decomposición:** Divide el problema en subproblemas más pequeños, solucionando sólo los pares de variables con mayor violación de las condiciones KKT en cada iteración.
- **Aproximaciones de bajo rango:** Métodos como Nyström o random features que aproximan la matriz kernel con una representación de rango bajo.

142.8.2. SMO (Sequential Minimal Optimization)

El algoritmo SMO selecciona en cada iteración el par de multiplicadores (α_i, α_j) que más viola las condiciones KKT. Dado que la restricción $\sum_i \alpha_i y_i = 0$ debe mantenerse, cambiar un solo α_i obliga a cambiar al menos otro. El subproblema de optimización para dos variables tiene solución analítica explícita:

$$\alpha_j^{\text{nuevo}} = \alpha_j^{\text{viejo}} + \frac{y_j(E_i - E_j)}{\eta}$$

donde $E_i = f(\mathbf{x}_i) - y_i$ es el error de clasificación del punto i , y $\eta = K_{ii} + K_{jj} - 2K_{ij}$.

142.8.3. Escalabilidad: SVM vs modelos lineales

Tabla 142.1.: Comparación de complejidad computacional

Modelo	Entrenamiento	Predicción	Memoria
SVM (kernel)	$O(n^2)$ a $O(n^3)$	$O(n_{\text{sv}} \cdot d)$	$O(n^2)$
SVM (lineal)	$O(n \cdot d)$	$O(d)$	$O(d)$

Modelo	Entrenamiento	Predicción	Memoria
Regresión logística	$O(n \cdot d)$	$O(d)$	$O(d)$
Redes neuronales	Variable	$O(d \cdot \text{capas})$	$O(\text{parámetros})$

Para n muy grande (millones de puntos), los modelos lineales como la regresión logística con regularización ℓ_2 son preferibles. Las SVM con kernel son más competitivas cuando n es moderado (miles) y la dimensión del espacio de características es alta.

142.8.4. Cuándo usar SVM

Las SVM son especialmente adecuadas cuando:

1. El número de características es alto respecto al número de muestras ($d \gg n$), como en clasificación de texto o genómica.
2. Se busca una solución dispersa: sólo los vectores de soporte son necesarios.
3. Los datos tienen una estructura de margen claro y el kernel captura bien la geometría.
4. La memoria es limitada y no se pueden cargar todos los datos simultáneamente (implementaciones que usan sólo los vectores de soporte).

142.9. Ventajas y limitaciones

Ventajas de las SVM

- **Efectivas en alta dimensión:** El principio de margen máximo no depende de la dimensión del espacio de entrada. Las SVM funcionan bien cuando $d \gg n$, lo que las hace populares en clasificación de texto y bioinformática.
- **Robustez frente a sobreajuste:** El margen máximo actúa como una forma de regularización implícita. La complejidad efectiva del clasificador depende del margen, no de la dimensión.
- **Solución global única:** El problema dual es estrictamente cóncavo (cuando el kernel es definido positivo), garantizando que la solución es única y global.
- **Solución dispersa:** Sólo los vectores de soporte determinan la frontera. En la predicción, sólo se necesitan estos puntos.
- **Flexibilidad mediante kernels:** El kernel trick permite capturar estructuras no lineales arbitrarias sin aumentar explícitamente la dimensión.
- **Buena generalización cuando $n < d$:** En regímenes donde hay pocas muestras pero muchas características, las SVM suelen superar a redes neuronales.

Limitaciones de las SVM

- **Escala mal con n grande:** La complejidad $O(n^2)$ a $O(n^3)$ hace que el entrenamiento sea lento para conjuntos de más de 50,000–100,000 puntos. Para grandes datos, métodos como regresión logística o redes neuronales con SGD son más prácticos.
- **Elección del kernel y sus hiperparámetros:** La selección del kernel adecuado y el ajuste de C , γ (y posiblemente d , r) requiere validación cruzada exhaustiva. Una mala elección puede llevar a un rendimiento muy pobre.

- **No da probabilidades directamente:** La salida de la SVM es una función de decisión $f(\mathbf{x})$, no una probabilidad. Para obtener probabilidades se necesita calibración adicional (ver Sección 142.10).
- **Sensible a la escala de las características:** El kernel RBF y el kernel lineal son sensibles a las escalas relativas de las características. Es necesario estandarizar los datos antes de entrenar.
- **Interpretabilidad limitada:** En espacios de alta dimensión con kernel no lineal, es difícil interpretar qué características son importantes para la decisión.
- **Un solo umbral de decisión:** Las SVM son clasificadores binarios por naturaleza. Para problemas multiclase se necesitan estrategias como uno-contra-uno o uno-contra-todos.

142.10. Obtener probabilidades de SVM: calibración de Platt

Una limitación práctica de las SVM es que no producen estimaciones de probabilidad directamente: la salida es una puntuación real $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ (o su versión kernelizada), no una probabilidad $P(y = 1 \mid \mathbf{x})$. En muchas aplicaciones —como sistemas de diagnóstico médico o modelos de riesgo— se necesita saber no sólo la clase predicha sino también la confianza en la predicción.

142.10.1. El método de calibración de Platt

John Platt propuso en 1999 un método elegante para convertir las puntuaciones de la SVM en probabilidades calibradas (Platt 1999). La idea es ajustar un modelo de regresión logística sobre las puntuaciones de la SVM:

$$P(y = 1 \mid f(\mathbf{x})) = \frac{1}{1 + \exp(Af(\mathbf{x}) + B)} \quad (142.7)$$

donde A y B son parámetros que se estiman por máxima verosimilitud usando los scores $f(\mathbf{x}_i)$ y las etiquetas y_i . Para evitar sobreoptimismo, Platt recomienda usar **validación cruzada** para obtener los scores $f(\mathbf{x}_i)$: se entrena la SVM en un subconjunto y se obtienen los scores en los puntos dejados fuera.

142.10.2. Relación con la regresión logística

La calibración de Platt transforma la SVM en algo conceptualmente similar a un clasificador probabilístico. De hecho, la regresión logística puede verse como una SVM “calibrada” desde el origen: en lugar de maximizar el margen de manera directa, minimiza la pérdida logística que es una aproximación suave de la pérdida de bisagra (*hinge loss*) usada en las SVM.

La pérdida de bisagra de la SVM es:

$$L_{\text{hinge}}(y, f(\mathbf{x})) = \max(0, 1 - y f(\mathbf{x}))$$

La pérdida logística es:

$$L_{\log}(y, f(\mathbf{x})) = \log(1 + \exp(-y f(\mathbf{x})))$$

Ambas penalizan las clasificaciones incorrectas, pero la pérdida logística es diferenciable en todas partes y produce estimaciones de probabilidad directamente. La pérdida de bisagra tiene gradiente cero para los puntos bien clasificados con margen suficiente, lo que explica por qué la solución SVM es dispersa.

La calibración de Platt es un puente entre ambas perspectivas: usa la estructura de margen de la SVM (que es geoméricamente óptima) y añade encima una capa logística para producir probabilidades bien calibradas.

142.11. Recomendaciones prácticas

Al aplicar SVM a un problema real, las siguientes recomendaciones pueden mejorar significativamente los resultados (Chang y Lin 2011; Hastie et al. 2009):

1. **Estandarizar las características:** Escalar cada característica a media cero y desviación estándar uno antes de entrenar. El kernel RBF depende de distancias euclídeas; sin estandarización, las características con mayor varianza dominarán.
2. **Comenzar con el kernel RBF:** A menos que se sepa de antemano que los datos son linealmente separables, el kernel RBF con parámetros ajustados es un buen punto de partida. Es flexible, tiene pocos hiperparámetros y funciona bien en la mayoría de los casos.
3. **Ajustar C y γ mediante validación cruzada en grid:** Explorar C en $\{10^{-3}, 10^{-2}, \dots, 10^3\}$ y γ en $\{10^{-4}, \dots, 10^2\}$. Usar búsqueda en grid con 5-fold o 10-fold CV. El logaritmo de los parámetros es más informativo que la escala lineal.
4. **Verificar la escala del problema:** Si $n > 50,000$, considerar alternativas más escalables como regresión logística con regularización ℓ_2 (equivalente a la SVM lineal). Las SVM con kernel son difíciles de entrenar a esa escala.
5. **Usar implementaciones eficientes:** LIBSVM (Chang y Lin 2011) es la implementación de referencia, accesible desde Python vía `scikit-learn` (SVC, SVR). Para datos linealmente separables de gran escala, `LinearSVC` usa una formulación diferente (LIBLINEAR) que escala a millones de puntos.
6. **Calibrar probabilidades si es necesario:** Si el problema requiere probabilidades (no sólo clases), activar la calibración de Platt (`probability=True` en `scikit-learn`, que usa CV interna) o usar `CalibratedClassifierCV`.
7. **Considerar el desbalance de clases:** Si las clases están muy desbalanceadas, usar `class_weight='balanced'` que ajusta los parámetros C de cada clase inversamente proporcional a su frecuencia.
8. **Validar los vectores de soporte:** Un número muy alto de vectores de soporte (cercano a n) indica que el modelo está subajustando o que C es demasiado pequeño. Un número muy bajo podría indicar sobreajuste si el error de generalización es alto.
9. **Inspeccionar las curvas de aprendizaje:** Para diagnosticar sobreajuste vs subajuste, graficar el error de entrenamiento y validación en función del tamaño del conjunto de entrenamiento. Las SVM con kernel RBF frecuentemente tienen alta varianza con pocos datos y se benefician de más muestras.
10. **Explorar SVR para regresión con outliers:** Cuando los datos de regresión contienen outliers, SVR con tubo ε amplio es más robusto que la regresión por mínimos cuadrados, que asigna pérdida cuadrática a todos los residuos.

142.12. Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.

- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best

- Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data*

- Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbj1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine

- Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte XXVII.

Clasificadores Probabilísticos

143. Clasificadores bayesianos

Los métodos de aprendizaje supervisado pueden verse desde dos perspectivas complementarias. En una perspectiva determinista, el objetivo es aprender directamente una función de decisión $f(\mathbf{x})$ que asigne cada observación a una clase, produzca una predicción continua o aproxime una relación funcional. En una perspectiva probabilística, en cambio, el objetivo es modelar explícitamente la incertidumbre: no sólo se decide una clase, sino que se estima qué tan plausible es cada clase dada la evidencia observada. Esto se traduce en que, si no observamos ninguna característica del objeto, la mejor decisión que podemos tomar depende de la probabilidad previa de cada clase.

Para ilustrar este hecho, supongamos que somos participantes dentro del famoso juego *En familia con chabelo*, donde se nos da la oportunidad de ganar un auto, para esto debemos de escoger dentro de 3 posibles puertas donde detrás se encuentra el auto o una cabra. Si no se nos da ninguna pista previa es razonable pensar que contamos con una probabilidad de $\frac{1}{3}$ de ganar el auto, por lo que la mejor decisión es escoger una puerta al azar.

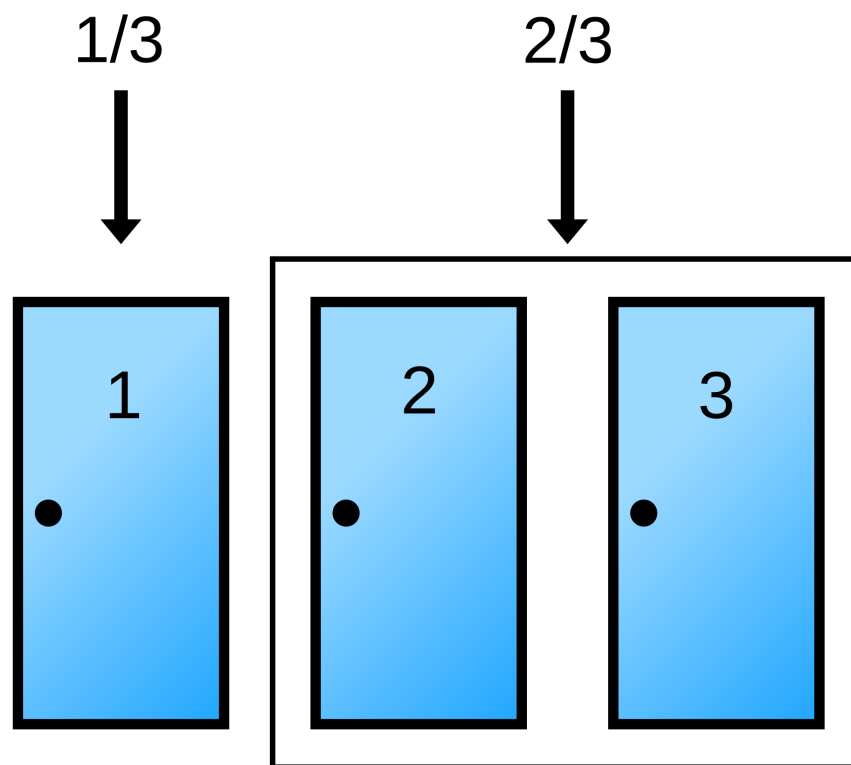


Figura 143.1.: Notemos como la probabilidad de encontrar el auto detrás de las puertas cerradas es de $\frac{1}{3}$, mientras que la probabilidad de encontrar la cabra es de $\frac{2}{3}$

Asumiendo que nuestra elección es la puerta 1, como sucede en muchas ocasiones el presentador nos hará notar que detrás de la puerta tres hay una cabra, con lo cual es natural preguntarse: ¿Bajo que condiciones tendremos la mejor probabilidad de ganar?, es común pensar que dado que sabemos que detrás de las

puerta de nuestra elección hay o una cabra o un auto la probabilidad de encontrar nuestro premio tras estas 2 puertas es de $\frac{1}{2}$, sin embargo nuestra intuición en ese caso sería incorrecta.

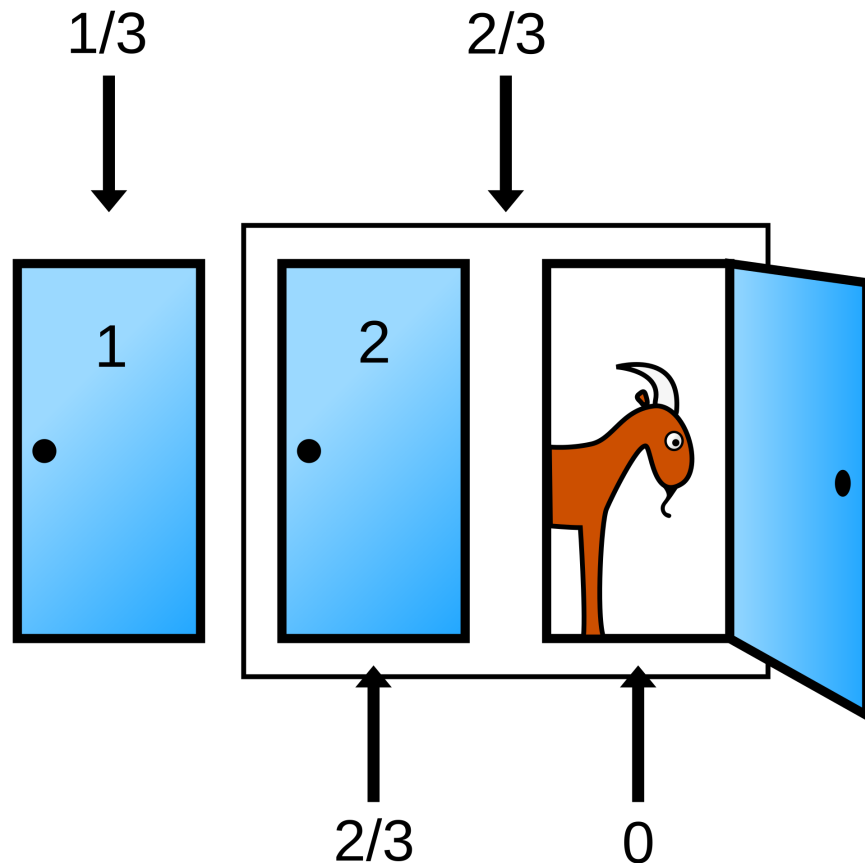


Figura 143.2.: Una vez se ha abierto una puerta con una cabra, dado que el presentador solo podría haber abierto la puerta que tuviera detrás una cabra obliga a que la probabilidad de ganar el auto si cambiamos a la puerta 2 es de $\frac{2}{3}$

Para ver con más detalle por qué conviene cambiar de puerta, hagamos las cuentas paso a paso. Antes de que Chabelo abra una puerta, el auto puede estar en cualquiera de las tres puertas con la misma probabilidad:

$$P(A_1) = P(A_2) = P(A_3) = \frac{1}{3}$$

donde A_i significa que el auto está detrás de la puerta i .

Ahora bien, Chabelo sabe dónde está el auto y siempre abre una puerta que tiene una cabra. Además, si tiene dos puertas con cabra disponibles, asumimos que escoge una de ellas al azar. Entonces las posibilidades son las siguientes:

Las ramas donde cambiar gana valen por 2 porque tienen probabilidad $\frac{1}{3}$, mientras que las ramas donde cambiar pierde valen $\frac{1}{6}$. Esto ocurre porque si el auto está en nuestra puerta inicial, Chabelo tiene dos puertas posibles para abrir y divide ese caso en dos ramas de probabilidad $\frac{1}{6}$. En cambio, si el auto está en

una puerta distinta a la nuestra, Chabelo no tiene libertad: sólo puede abrir la única puerta restante con cabra. Por eso cada una de esas ramas conserva probabilidad $\frac{1}{3}$, equivalente a dos partes de tamaño $\frac{1}{6}$.

Como en el ejemplo Chabelo abre la puerta 3 y muestra una cabra, debemos quedarnos únicamente con los casos compatibles con esa información. Es decir, descartamos todos los casos donde Chabelo abre la puerta 2 o donde el auto estaba detrás de la puerta 3.

Los casos posibles después de observar que Chabelo abrió la puerta 3 son:

Caso	Ubicación del auto	Probabilidad inicial	Probabilidad de que Chabelo abra la puerta 3	Probabilidad conjunta
1	Puerta 1	$\frac{1}{3}$	$\frac{1}{2}$	$\frac{1}{3} \cdot \frac{1}{2} = \frac{1}{6}$
2	Puerta 2	$\frac{1}{3}$	1	$\frac{1}{3} \cdot 1 = \frac{1}{3}$
3	Puerta 3	$\frac{1}{3}$	0	$\frac{1}{3} \cdot 0 = 0$

La probabilidad total de que Chabelo abra la puerta 3 es:

$$P(\text{Chabelo abre puerta 3}) = \frac{1}{6} + \frac{1}{3} + 0$$

$$P(\text{Chabelo abre puerta 3}) = \frac{1}{6} + \frac{2}{6} = \frac{3}{6} = \frac{1}{2}$$

Ahora actualizamos las probabilidades usando la información nueva. Primero, calculemos la probabilidad de que el auto esté en la puerta 1 dado que Chabelo abrió la puerta 3:

$$P(A_1 | \text{Chabelo abre puerta 3}) = \frac{P(A_1 \cap \text{Chabelo abre puerta 3})}{P(\text{Chabelo abre puerta 3})}$$

$$P(A_1 | \text{Chabelo abre puerta 3}) = \frac{\frac{1}{6}}{\frac{1}{2}} = \frac{1}{6} \cdot \frac{2}{1} = \frac{2}{6} = \frac{1}{3}$$

Por lo tanto, si nos quedamos con la puerta 1, la probabilidad de ganar sigue siendo:

$$P(\text{ganar si nos quedamos}) = \frac{1}{3}$$

Ahora calculemos la probabilidad de que el auto esté en la puerta 2 dado que Chabelo abrió la puerta 3:

$$P(A_2 | \text{Chabelo abre puerta 3}) = \frac{P(A_2 \cap \text{Chabelo abre puerta 3})}{P(\text{Chabelo abre puerta 3})}$$

$$P(A_2 | \text{Chabelo abre puerta 3}) = \frac{\frac{1}{3}}{\frac{1}{2}} = \frac{1}{3} \cdot \frac{2}{1} = \frac{2}{3}$$

Por lo tanto, si cambiamos a la puerta 2, la probabilidad de ganar es:

$$P(\text{ganar si cambiamos}) = \frac{2}{3}$$

La clave está en que la puerta que elegimos inicialmente conserva su probabilidad original de $\frac{1}{3}$. La información nueva no reparte la probabilidad en partes iguales entre las dos puertas cerradas. Más bien, la probabilidad que antes estaba distribuida entre las dos puertas que no escogimos se concentra en la única puerta no abierta por Chabelo.

En resumen:

$$P(\text{ganar si nos quedamos}) = \frac{1}{3}$$

$$P(\text{ganar si cambiamos}) = \frac{2}{3}$$

Así que la mejor estrategia es cambiar de puerta. La pista de Chabelo no es una simple eliminación de una opción: es evidencia nueva que cambia nuestras probabilidades posteriores. Esta es precisamente la idea central del razonamiento bayesiano: actualizar nuestras creencias iniciales cuando observamos nueva información.

En esta simulación podemos observar como las probabilidades del problema a lo largo de las diferentes realizaciones convergen a los valores teóricos de $\frac{1}{3}$ para la estrategia de quedarse y $\frac{2}{3}$ para la estrategia de cambiar.

```

import random
import plotly.graph_objects as go

def simular_monty_hall_plotly(n_simulaciones=10_000):
    puertas = [1, 2, 3]

    gana_quedandose = 0
    gana_cambiando = 0

    probabilidades_quedandose = []
    probabilidades_cambiando = []
    simulaciones = []

    for i in range(1, n_simulaciones + 1):
        # El auto se coloca al azar detrás de una puerta
        puerta_auto = random.choice(puertas)

        # El participante siempre escoge inicialmente la puerta 1
        eleccion_inicial = 1

        # Chabelo abre una puerta que no fue elegida y que no tiene el auto
        puertas_posibles_para_abrir = [
            puerta
            for puerta in puertas
            if puerta != eleccion_inicial and puerta != puerta_auto
        ]

        puerta_abierta = random.choice(puertas_posibles_para_abrir)

        # Si el participante cambia, escoge la única puerta cerrada restante
        puerta_cambio = [
            puerta
            for puerta in puertas
            if puerta != eleccion_inicial and puerta != puerta_abierta
        ][0]

        # Estrategia 1: quedarse con la puerta inicial
        if eleccion_inicial == puerta_auto:
            gana_quedandose += 1

        # Estrategia 2: cambiar de puerta
        if puerta_cambio == puerta_auto:
            gana_cambiando += 1

        # Probabilidades acumuladas hasta la simulación actual
        probabilidades_quedandose.append(gana_quedandose / i)
        probabilidades_cambiando.append(gana_cambiando / i)
        simulaciones.append(i)

    fig = go.Figure()

    fig.add_trace(
        go.Scatter(
            x=simulaciones

```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

```
Simulaciones realizadas: 10000
```

```
Probabilidad final de ganar quedándose: 0.3262
```

```
Probabilidad final de ganar cambiando: 0.6738
```

Esta es la diferencia central en los clasificadores bayesianos, la decisión se deriva de probabilidades a priori, densidades condicionales y probabilidades posteriores (Duda et al. 2001; Bishop 2006). Por ejemplo, si una caja contiene monedas de dos tipos y una de ellas es más frecuente, la decisión con menor probabilidad de error antes de observar la moneda es escoger la clase más probable. Sin embargo, una vez que se observa una característica, como el peso, la decisión debe incorporar esa evidencia. La pregunta deja de ser “¿qué clase es más frecuente?” y pasa a ser “¿qué clase es más probable dado el valor observado?”.

Los clasificadores bayesianos usan el teorema de Bayes para decidir a qué clase pertenece una observación. Para cada clase ω_i , se considera qué tan probable es observar los datos $\mathbf{x}$ si esa clase fuera la correcta, junto con qué tan probable era esa clase antes de ver los datos. Con esta información se calcula la probabilidad de cada clase después de observar $\mathbf{x}$. La decisión más sencilla consiste en elegir la clase con mayor probabilidad posterior; si algunos errores son más costosos que otros, se puede modificar la decisión para tomar en cuenta esos costos (Duda et al. 2001; Jaynes 2003).

Este capítulo desarrolla la clasificación bayesiana desde sus fundamentos hasta algunas variantes prácticas: Naïve Bayes, clasificadores gaussianos, modelos de Markov para secuencias discretas, decisión bayesiana con costos y el criterio minimax cuando las probabilidades previas son inciertas. Los ejemplos computacionales están escritos en Python y buscan conectar la teoría con implementaciones reproducibles.

143.1. Vista probabilística de la clasificación

Sea $\mathbf{x} \in \mathbb{R}^d$ una observación y sea $\Omega = \{\omega_1, \dots, \omega_c\}$ el conjunto de clases posibles. En clasificación probabilística se supone que $\mathbf{x}$ no determina de manera absoluta la clase, sino que modifica nuestras creencias sobre ella. Antes de observar $\mathbf{x}$, la información disponible está representada por las probabilidades previas $P(\omega_i)$. Después de observar $\mathbf{x}$, la información relevante se resume en las probabilidades posteriores $P(\omega_i | \mathbf{x})$.

Definición. La probabilidad posterior de la clase ω_i dada una observación $\mathbf{x}$ se obtiene mediante el teorema de Bayes:

$$P(\omega_i | \mathbf{x}) = \frac{p(\mathbf{x} | \omega_i)P(\omega_i)}{p(\mathbf{x})} \quad (143.1)$$

En la ecuación Ecuación 143.1, $p(\mathbf{x} | \omega_i)$ es la densidad condicional de la observación bajo la clase ω_i , $P(\omega_i)$ es la probabilidad previa de esa clase y $p(\mathbf{x})$ es la densidad marginal de la observación:

$$p(\mathbf{x}) = \sum_{j=1}^c p(\mathbf{x} | \omega_j)P(\omega_j).$$

Como $p(\mathbf{x})$ no depende de la clase que se compara, muchas reglas de decisión pueden escribirse en términos de la cantidad no normalizada $p(\mathbf{x} | \omega_i)P(\omega_i)$.

La decisión de clasificación

Si no hay observaciones, una regla razonable para dos clases es escoger la clase más probable a priori:

$$\text{decidir } \omega_1 \quad \text{si} \quad P(\omega_1) > P(\omega_2).$$

El error esperado de esta regla, cuando sólo existen dos clases, es

$$P(e) = \min\{P(\omega_1), P(\omega_2)\}.$$

Una vez observada $\mathbf{x}$, la decisión cambia: se debe comparar $P(\omega_1 | \mathbf{x})$ contra $P(\omega_2 | \mathbf{x})$. Esta distinción es la esencia de la clasificación bayesiana: la evidencia observada actualiza las probabilidades de las clases.

Definición. La regla de máxima probabilidad posterior, o regla MAP, asigna la observación $\mathbf{x}$ a la clase con posterior máxima:

$$\hat{\omega}(\mathbf{x}) = \arg \max_{\omega_i \in \Omega} P(\omega_i | \mathbf{x}) \quad (143.2)$$

143.1.1. Ejemplo básico: El dilema del Cine

Esta persona dejó caer su boleto en el pasillo.

¿Le dices?

“Disculpe, señora”

o

“Disculpe, señor”

Tienes que hacer una conjetura.



Para responder de la manera mas acertada posible podemos hacer uso del teorema de Bayes, para esto definamos el evento H como “hombre” y el evento M como “mujer”, mientras que la evidencia observada es el tipo de cabello de la persona. Entonces, la probabilidad de que la persona sea hombre o mujer dado el tipo de cabello se calcula con Bayes:

$$P(\text{sexo} \mid \text{cabello}) = \frac{P(\text{cabello} \mid \text{sexo})P(\text{sexo})}{\sum_{\text{sexo}} P(\text{sexo})P(\text{cabello} \mid \text{sexo})}$$

Donde bajo una asunción justa podemos suponer que dentro del cine:

De cada 100 mujeres en el cine

- 50 tienen cabello corto
- 50 tienen cabello largo

De cada 100 hombres en el cine

- 96 tienen cabello corto
- 4 tienen cabello largo

Hagamos los calculos paso a paso, si asumimos que la proporción de hombres y mujeres en el cine es la misma, entonces las probabilidades previas son:

$$P(H) = P(M) = 0.5$$

Mientras que las verosimilitudes son:

$$P(\text{cabello largo} \mid H) = 0.04$$

$$P(\text{cabello largo} \mid M) = 0.50$$

teniendo esto, Entonces los numeradores son:

$$P(\text{largo} \mid H)P(H) = 0.04(0.5) = 0.02$$

$$P(\text{largo} \mid M)P(M) = 0.50(0.5) = 0.25$$

Entonces:

$$P(H \mid \text{cabello largo}) = 0.02A \approx 0.07$$

$$P(M \mid \text{cabello largo}) = 0.25A \approx 0.93$$

donde

$$A = \frac{1}{0.02 + 0.25}$$

Así, bajo la regla de máxima probabilidad posterior, la decisión es:

⇒ Decisión: mujer

i Nota

La idea central es que no decidimos solo por la característica observada, sino por cómo esa característica modifica nuestras creencias previas sobre cada clase.

143.1.2. Dilema en el cine: el contexto importa

Es importante notar que la decisión anterior depende de las probabilidades previas. Si el contexto cambia, las probabilidades previas también cambian, lo que puede llevar a una decisión diferente incluso con la misma evidencia observada. Por ejemplo, si esta situación ocurre mientras estás formado en la fila del baño de hombres, la proporción de hombres y mujeres cambia drásticamente, lo que afecta las probabilidades previas y, por ende, la decisión bayesiana.

¿Le dices?

“Disculpe, señora”

o

“Disculpe, señor”



Ahora las probabilidades previas cambian:

$$P(H) = 0.98, \quad P(M) = 0.02$$

donde H representa “hombre” y M representa “mujer”. De nuevo asumiendo de manera justa que dentro de la fila:

De cada 2 mujeres en la fila

- 1 tiene cabello corto
- 1 tiene cabello largo

De cada 98 hombres en la fila

- 94 tienen cabello corto
- 4 tienen cabello largo

De los datos anteriores:

$$P(\text{cabello largo} \mid H) = \frac{4}{98}$$

$$P(\text{cabello largo} \mid M) = \frac{1}{2}$$

Calculamos los numeradores de Bayes:

$$P(\text{cabello largo} \mid H)P(H) = \frac{4}{98}(0.98) = 0.04$$

$$P(\text{cabello largo} \mid M)P(M) = \frac{1}{2}(0.02) = 0.01$$

La constante de normalización es:

$$B = \frac{1}{0.04 + 0.01} = 20$$

Entonces, bajo la misma regla de máxima probabilidad posterior, las probabilidades posteriores son:

$$P(H \mid \text{cabello largo}) = 0.04B = 0.04(20) = 0.80$$

$$P(M \mid \text{cabello largo}) = 0.01B = 0.01(20) = 0.20$$

Como:

$$P(H \mid \text{cabello largo}) > P(M \mid \text{cabello largo})$$

la decisión bayesiana es:

⇒ Decisión: hombre

! Importante

La misma evidencia observada (cabello largo) puede llevar a una decisión distinta cuando cambian las probabilidades previas.

En clasificación bayesiana, el contexto modifica la inferencia.

143.1.3. Implementación básica: algoritmo general de un clasificador bayesiano

A continuación se muestra una implementación básica de un clasificador bayesiano para cualquier número de clases. El código calcula las probabilidades posteriores y toma la decisión de clasificación basada en la clase con mayor probabilidad posterior.

```
import numpy as np

def clasificador_bayesiano(verosimilitudes, priors, clases):
    """
    Clasificador bayesiano para cualquier número de clases.

    verosimilitudes: p(x | omega_i) para cada clase.
    priors: P(omega_i) para cada clase.
    clases: nombres de las clases.
    """
    verosimilitudes = np.array(verosimilitudes, dtype=float)
    priors = np.array(priors, dtype=float)
```

```

clases = np.array(clases)

evidencia = np.sum(verosimilitudes * priors)
posteriores = (verosimilitudes * priors) / evidencia

indice_decision = np.argmax(posteriores)
decision = clases[indice_decision]

return posteriores, decision

clases = ["normal", "enfermedad"]

# p(x | omega_i): qué tan probable es observar x en cada clase
verosimilitudes = [0.2, 0.4]

# P(omega_i): probabilidad previa de cada clase
priors = [0.9, 0.1]

posteriores, decision = clasificador_bayesiano(
    verosimilitudes,
    priors,
    clases
)

for clase, posterior in zip(clases, posteriores):
    print(f"P({clase} | x) = {posterior:.3f}")

print("Decisión:", decision)

```

```

P(normal | x) = 0.818
P(enfermedad | x) = 0.182
Decisión: normal

```

En este ejemplo, aunque la verosimilitud bajo la clase de enfermedad es mayor, el prior de la clase normal domina la decisión de error mínimo. Esto ilustra una idea importante: una evidencia local no se interpreta de manera aislada, sino en relación con la frecuencia previa de cada clase.

143.2. Clasificador bayesiano de error mínimo

La regla MAP tiene una interpretación óptima cuando todos los errores tienen el mismo costo: minimiza la probabilidad promedio de clasificación incorrecta. Para verlo, consideremos inicialmente dos clases. Si se asigna $\mathbf{x}$ a ω_1 , el error condicional es $P(\omega_2 | \mathbf{x})$; si se asigna a ω_2 , el error condicional es $P(\omega_1 | \mathbf{x})$. Por tanto, para minimizar el error en cada punto $\mathbf{x}$ se elige la clase con mayor posterior.

Definición. El error condicional de clasificación para dos clases es

$$P(e | \mathbf{x}) = \begin{cases} P(\omega_2 | \mathbf{x}), & \text{si se decide } \omega_1, \\ P(\omega_1 | \mathbf{x}), & \text{si se decide } \omega_2. \end{cases} \quad (143.3)$$

El error promedio se obtiene integrando sobre todo el espacio de observaciones:

$$P(e) = \int P(e | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}.$$

Dado que $p(\mathbf{x}) \geq 0$, minimizar el error promedio se logra minimizando el error condicional punto a punto. Así, para dos clases:

$$P(\omega_1 | \mathbf{x}) > P(\omega_2 | \mathbf{x}) \implies \mathbf{x} \in \omega_1.$$

143.2.1. Ejemplo en Python: decisión de error mínimo

El siguiente código muestra la regla de error mínimo para varias observaciones. Para cada observación se tienen las verosimilitudes $p(\mathbf{x} | \omega_i)$ bajo dos clases. Después se calculan las posteriores, se elige la clase con mayor posterior y se reporta el error condicional mínimo.

```
import numpy as np

clases = np.array(["normal", "enfermedad"])

# Probabilidades previas P(omega_i)
priors = np.array([0.8, 0.2])

# Cada fila representa una observación distinta.
# Las columnas contienen p(x | normal) y p(x | enfermedad).
verosimilitudes = np.array([
    [0.50, 0.10],
    [0.35, 0.25],
    [0.20, 0.45],
    [0.10, 0.60],
    [0.40, 0.30]
])

# Numerador de Bayes: p(x | omega_i) P(omega_i)
puntajes_no_normalizados = verosimilitudes * priors

# Evidencia p(x), calculada para cada observación
evidencia = puntajes_no_normalizados.sum(axis=1, keepdims=True)

# Posteriores P(omega_i | x)
posteriores = puntajes_no_normalizados / evidencia

# Regla de error mínimo: elegir la clase con mayor posterior
```



```

indices_decision = np.argmax(posteriores, axis=1)
decisiones = clases[indices_decision]

# El error condicional mínimo es 1 menos la posterior más grande
errores_condicionales = 1 - np.max(posteriores, axis=1)

for i, (post, decision, error) in enumerate(
    zip(posteriores, decisiones, errores_condicionales),
    start=1
):
    print(f"Observación {i}")
    print(f" P(normal | x) = {post[0]:.3f}")
    print(f" P(enfermedad | x) = {post[1]:.3f}")
    print(f" Decisión de error mínimo: {decision}")
    print(f" Error condicional mínimo: {error:.3f}\n")

```

Observación 1

```

P(normal | x) = 0.952
P(enfermedad | x) = 0.048
Decisión de error mínimo: normal
Error condicional mínimo: 0.048

```

Observación 2

```

P(normal | x) = 0.848
P(enfermedad | x) = 0.152
Decisión de error mínimo: normal
Error condicional mínimo: 0.152

```

Observación 3

```

P(normal | x) = 0.640
P(enfermedad | x) = 0.360
Decisión de error mínimo: normal
Error condicional mínimo: 0.360

```

Observación 4

```

P(normal | x) = 0.400
P(enfermedad | x) = 0.600
Decisión de error mínimo: enfermedad
Error condicional mínimo: 0.400

```

Observación 5

```

P(normal | x) = 0.842
P(enfermedad | x) = 0.158
Decisión de error mínimo: normal
Error condicional mínimo: 0.158

```

En este ejemplo, una observación se clasifica como **enfermedad** sólo cuando su posterior supera a la posterior de **normal**. No basta con que $p(\mathbf{x} \mid \text{enfermedad})$ sea grande de manera aislada: también importa

el prior de cada clase. Por eso la decisión se basa en el producto $p(\mathbf{x} | \omega_i)P(\omega_i)$ y no únicamente en la verosimilitud.

143.2.2. Formas equivalentes de la regla de decisión

Usando el teorema de Bayes, la comparación de posteriores puede escribirse como una comparación de productos entre verosimilitud y prior:

$$p(\mathbf{x} | \omega_1)P(\omega_1) > p(\mathbf{x} | \omega_2)P(\omega_2).$$

Para dos clases, también puede expresarse mediante el cociente de verosimilitudes.

Definición. El cociente de verosimilitudes entre dos clases se define como

$$\ell(\mathbf{x}) = \frac{p(\mathbf{x} | \omega_1)}{p(\mathbf{x} | \omega_2)} \quad (143.4)$$

La regla de decisión se vuelve

$$\ell(\mathbf{x}) \underset{\omega_2}{\overset{\omega_1}{\geq}} \frac{P(\omega_2)}{P(\omega_1)}.$$

Si se trabaja en escala logarítmica, se evita el subdesbordamiento numérico y se transforman productos en sumas:

$$\log \ell(\mathbf{x}) = \log p(\mathbf{x} | \omega_1) - \log p(\mathbf{x} | \omega_2).$$

Esta forma es especialmente útil en alta dimensión, en secuencias largas y en modelos donde las densidades son productos de muchas probabilidades pequeñas.

143.3. Clasificador Naïve Bayes

En alta dimensión, estimar la densidad conjunta $p(\mathbf{x} | \omega_i)$ puede ser difícil. Si $\mathbf{x} = (x_1, \dots, x_d)$, una densidad conjunta general requiere modelar dependencias entre todas las variables. Por ejemplo, en clasificación de textos no basta con preguntar si aparece una palabra: un documento puede contener cientos o miles de términos, y cada término puede aparecer varias veces.

Naïve Bayes simplifica el problema suponiendo independencia condicional de las características dada la clase (McCallum y Nigam 1998; Murphy 2012). La palabra *naïve* no significa que el método sea inútil, sino que usa una suposición fuerte: una vez que conocemos la clase, tratamos a las características como si aportaran evidencia de manera separada.

Definición. El supuesto de independencia condicional de Naïve Bayes establece que

$$p(\mathbf{x} | \omega_i) = p(x_1, x_2, \dots, x_d | \omega_i) = \prod_{k=1}^d p(x_k | \omega_i) \quad (143.5)$$

Con este supuesto, la regla MAP se escribe como:

$$\hat{\omega}(\mathbf{x}) = \arg \max_i \left[\log P(\omega_i) + \sum_{k=1}^d \log p(x_k | \omega_i) \right].$$

Esta expresión tiene una lectura sencilla. El término $\log P(\omega_i)$ representa qué tan probable era la clase antes de observar el documento. La suma $\sum_{k=1}^d \log p(x_k | \omega_i)$ acumula la evidencia que aportan las características observadas. La clase elegida es aquella que obtiene la puntuación total más alta.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(7)

# Datos con correlación entre variables
cov_full = np.array([[1.2, 0.85], [0.85, 0.9]])
cov_diag = np.diag(np.diag(cov_full)) # versión diagonal (NB)

mu1 = np.array([-1.5, -1.0])
mu2 = np.array([1.5, 1.0])

X1 = rng.multivariate_normal(mu1, cov_full, 80)
X2 = rng.multivariate_normal(mu2, cov_full, 80)

def ellipse_points(mu, cov, n_std=2, n_pts=120):
    """Elipse de confianza via descomposición espectral."""
    vals, vecs = np.linalg.eigh(cov)
    t = np.linspace(0, 2 * np.pi, n_pts)
    circle = np.stack([np.cos(t), np.sin(t)], axis=1)
    ellipse = circle @ (vecs * np.sqrt(vals) * n_std).T + mu
    return ellipse[:, 0], ellipse[:, 1]

def grid_posterior(X1, X2, cov, xlim, ylim, n=80):
    """Probabilidad posterior P(clase 1 | x) en un grid 2D."""
    def mvn(X, mu, cov):
        D = X.shape[1]
        diff = X - mu
        inv_cov = np.linalg.inv(cov)
        sign, logdet = np.linalg.slogdet(cov)
        maha = np.einsum('ij,jk,ik->i', diff, inv_cov, diff)
        return np.exp(-0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet))

    xg = np.linspace(xlim[0], xlim[1], n)
    yg = np.linspace(ylim[0], ylim[1], n)
    Xg, Yg = np.meshgrid(xg, yg)
    pts = np.c_[Xg.ravel(), Yg.ravel()]

    mu1_ = X1.mean(axis=0)
    mu2_ = X2.mean(axis=0)
    p1 = mvn(pts, mu1_, cov)
```

```

p2 = mvn(pts, mu2_, cov)
post = p1 / (p1 + p2 + 1e-300)
return xg, yg, post.reshape(n, n)

xlim = (-5, 5); ylim = (-5, 5)
xg, yg, Z_full = grid_posterior(X1, X2, cov_full, xlim, ylim)
xg, yg, Z_diag = grid_posterior(X1, X2, cov_diag, xlim, ylim)

mu1_est = X1.mean(axis=0)
mu2_est = X2.mean(axis=0)

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=["QDA (covarianza completa)", "Naïve Bayes (covarianza diagonal)"]
)

for col, cov, Z, title in [(1, cov_full, Z_full, "QDA"), (2, cov_diag, Z_diag, "NB")]:
    fig.add_trace(go.Contour(
        x=xg, y=yg, z=Z,
        colorscale=[[0, "#4C72B0"], [0.5, "white"], [1, "#DD8452"]],
        opacity=0.40,
        showscale=False,
        contours=dict(showlines=True, start=0.1, end=0.9, size=0.2,
                      coloring="fill")
    ), row=1, col=col)

    for X, mu_e, c_col, name in [
        (X1, mu1_est, "#2563eb", "Clase 1"),
        (X2, mu2_est, "#dc2626", "Clase 2")
    ]:
        fig.add_trace(go.Scatter(
            x=X[:, 0], y=X[:, 1],
            mode="markers",
            marker=dict(color=c_col, size=5, opacity=0.55),
            name=name, showlegend=(col == 1)
        ), row=1, col=col)

        ex, ey = ellipse_points(mu_e, cov if col == 1 else np.diag(np.diag(cov_full)))
        fig.add_trace(go.Scatter(
            x=ex, y=ey, mode="lines",
            line=dict(color=c_col, width=2.2, dash="solid"),
            showlegend=False
        ), row=1, col=col)

fig.update_layout(
    title="Supuesto de independencia condicional: QDA vs Naïve Bayes",
    template="plotly_white",
    height=420,
    legend=dict(orientation="h", yanchor="bottom", y=1.08, xanchor="right", x=1)
)

```

```
)
fig.update_xaxes(range=xlim, title_text="x ")
fig.update_yaxes(range=ylim, title_text="x ")
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 143.3.: Comparación geométrica entre el supuesto gaussiano general (QDA, covarianza completa) y Naïve Bayes gaussiano (covarianza diagonal). QDA permite elipses rotadas arbitrariamente; NB solo puede representar elipses alineadas con los ejes.

143.3.1. Naïve Bayes multinomial para texto

Una de las versiones más usadas en clasificación de documentos es Naïve Bayes multinomial. En este caso, cada documento se representa como un vector de conteos:

$$\mathbf{x} = (x_1, x_2, \dots, x_d),$$

donde x_k indica cuántas veces aparece la palabra k del vocabulario en el documento. Si θ_{ik} representa la probabilidad de observar la palabra k dentro de documentos de la clase ω_i , entonces el modelo asigna la siguiente puntuación:

$$g_i(\mathbf{x}) = \log P(\omega_i) + \sum_{k=1}^d x_k \log \theta_{ik}.$$

La decisión se toma escogiendo la clase con mayor puntuación:

$$\hat{\omega}(\mathbf{x}) = \arg \max_i g_i(\mathbf{x}).$$

En problemas de texto, las probabilidades θ_{ik} se estiman contando palabras en los documentos de entrenamiento. Si N_{ik} es el número de veces que aparece la palabra k en documentos de la clase ω_i , una estimación directa sería:

$$\theta_{ik} = \frac{N_{ik}}{\sum_{r=1}^d N_{ir}}.$$

El problema es que si una palabra nunca aparece en una clase durante el entrenamiento, entonces $N_{ik} = 0$ y su probabilidad estimada sería cero. Como el modelo multiplica probabilidades, una sola palabra con probabilidad cero puede anular toda la puntuación de una clase. Para evitarlo se usa suavizado de Laplace:

$$\theta_{ik} = \frac{N_{ik} + \alpha}{\sum_{r=1}^d N_{ir} + \alpha d}.$$

Cuando $\alpha = 1$, se interpreta como si agregáramos una aparición ficticia de cada palabra en cada clase. Esto evita probabilidades cero y vuelve más estable al clasificador.

 Tip

En la práctica se trabaja con logaritmos, no con productos directos. Esto evita problemas numéricos cuando se multiplican muchas probabilidades pequeñas y convierte el producto de evidencias en una suma de evidencias.

143.3.2. Ejemplo manual: clasificación de un documento corto

Supongamos que queremos clasificar documentos en dos clases: **spam** y **ham**. Después de entrenar el modelo, podríamos obtener las siguientes probabilidades:

Palabra	$P(\text{palabra} \mid \text{spam})$	$P(\text{palabra} \mid \text{ham})$
free	0.30	0.05
prize	0.25	0.05
project	0.05	0.30

Si los priors son iguales,

$$P(\text{spam}) = P(\text{ham}) = 0.5,$$

y observamos el documento:

$$\mathbf{x} = \text{"free project prize"},$$

entonces las puntuaciones logarítmicas son:

$$g_{\text{spam}}(\mathbf{x}) = \log(0.5) + \log(0.30) + \log(0.05) + \log(0.25),$$

$$g_{\text{ham}}(\mathbf{x}) = \log(0.5) + \log(0.05) + \log(0.30) + \log(0.05).$$

Aunque la palabra **project** favorece a **ham**, las palabras **free** y **prize** favorecen a **spam**. El clasificador suma toda la evidencia y decide por la clase con mayor puntuación final.

```
import numpy as np
import plotly.graph_objects as go

palabras = ["free", "prize", "project", "meeting", "offer", "money", "update", "win"]
p_spam = np.array([0.30, 0.25, 0.05, 0.03, 0.18, 0.22, 0.02, 0.15])
p_ham = np.array([0.05, 0.05, 0.30, 0.28, 0.06, 0.04, 0.20, 0.02])

# Log-likelihood ratio como barra de fondo (discriminatividad)
llr = np.log(p_spam / p_ham)

fig = go.Figure()
```

```

fig.add_trace(go.Bar(
    name="P(palabra | spam)",
    x=palabras, y=p_spam,
    marker_color="#dc2626",
    marker_line=dict(color="#7f1d1d", width=1),
    opacity=0.85
))

fig.add_trace(go.Bar(
    name="P(palabra | ham)",
    x=palabras, y=p_ham,
    marker_color="#2563eb",
    marker_line=dict(color="#1e3a6e", width=1),
    opacity=0.85
))

# Anotación de log-ratio sobre cada palabra
for i, (palabra, ratio) in enumerate(zip(palabras, llr)):
    color = "#dc2626" if ratio > 0 else "#2563eb"
    symbol = " " if ratio > 0 else " "
    fig.add_annotation(
        x=palabra,
        y=max(p_spam[i], p_ham[i]) + 0.015,
        text=f"{symbol} {abs(ratio):.2f}",
        showarrow=False,
        font=dict(size=10, color=color),
        yanchor="bottom"
    )

fig.update_layout(
    title="Verosimilitudes por clase - ejemplo spam/ham",
    xaxis_title="Palabra",
    yaxis_title="P(palabra | clase)",
    barmode="group",
    template="plotly_white",
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1),
    annotations=[dict(
        x=0.5, y=-0.18, xref="paper", yref="paper",
        text=" / = log P(spam)/P(ham). Positivo → favorece spam; negativo → favorece ham",
        showarrow=False, font=dict(size=11, color="#64748b")
    )]
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 143.4.: Verosimilitudes por clase para el ejemplo spam/ham. Las barras muestran $P(\text{palabra}|\text{clase})$ para cada término del vocabulario. Las palabras con mayor diferencia entre barras son las más discriminativas.

143.3.3. Ejemplo en Python: Naïve Bayes para texto

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

texts = [
    "free money offer now",
    "win money free prize",
    "meeting schedule project update",
    "project discussion and meeting notes",
    "free prize offer",
    "schedule the project meeting"
]
labels = ["spam", "spam", "ham", "ham", "spam", "ham"]

model = make_pipeline(
    CountVectorizer(),
    MultinomialNB(alpha=1.0)
)

model.fit(texts, labels)

new_docs = ["free project prize", "meeting update schedule"]
print(model.predict(new_docs))
print(model.predict_proba(new_docs))
```

```
['spam' 'ham']
[[0.20924284 0.79075716]
 [0.9501296  0.0498704  ]]
```

El parámetro `alpha=1.0` introduce suavizado de Laplace. Este suavizado evita que una palabra ausente en el entrenamiento de una clase anule completamente la probabilidad de un documento.

También podemos inspeccionar el vocabulario aprendido y las probabilidades estimadas por clase:

```
import numpy as np

vectorizer = model.named_steps["countvectorizer"]
clf = model.named_steps["multinomialnb"]

vocabulario = vectorizer.get_feature_names_out()
```

```
for clase, log_probs in zip(clf.classes_, clf.feature_log_prob_):
    print(f"\nClase: {clase}")
    for palabra, log_prob in zip(vocabulario, log_probs):
        print(f"{palabra:12s} P(palabra | clase) = {np.exp(log_prob):.3f}")
```

```
Clase: ham
and          P(palabra | clase) = 0.074
discussion   P(palabra | clase) = 0.074
free         P(palabra | clase) = 0.037
meeting      P(palabra | clase) = 0.148
money        P(palabra | clase) = 0.037
notes        P(palabra | clase) = 0.074
now          P(palabra | clase) = 0.037
offer        P(palabra | clase) = 0.037
prize        P(palabra | clase) = 0.037
project      P(palabra | clase) = 0.148
schedule     P(palabra | clase) = 0.111
the          P(palabra | clase) = 0.074
update       P(palabra | clase) = 0.074
win          P(palabra | clase) = 0.037
```

```
Clase: spam
and          P(palabra | clase) = 0.040
discussion   P(palabra | clase) = 0.040
free         P(palabra | clase) = 0.160
meeting      P(palabra | clase) = 0.040
money        P(palabra | clase) = 0.120
notes        P(palabra | clase) = 0.040
now          P(palabra | clase) = 0.080
offer        P(palabra | clase) = 0.120
prize        P(palabra | clase) = 0.120
project      P(palabra | clase) = 0.040
schedule     P(palabra | clase) = 0.040
the          P(palabra | clase) = 0.040
update       P(palabra | clase) = 0.040
win          P(palabra | clase) = 0.080
```

Este segundo bloque muestra de manera explícita qué aprendió el modelo: para cada clase, estima qué palabras son más probables. Por eso Naïve Bayes suele ser fácil de interpretar en texto. Una palabra como **free** debería tener mayor probabilidad bajo **spam**, mientras que una palabra como **meeting** debería tener mayor probabilidad bajo **ham**.

! Importante

Naïve Bayes no necesita que el supuesto de independencia sea perfectamente cierto para clasificar bien. Lo importante es que las puntuaciones relativas separen razonablemente las clases. Por eso puede funcionar bien incluso cuando las palabras de un documento claramente no son independientes

entre sí.

143.3.4. Variantes comunes de Naïve Bayes

La misma idea puede adaptarse a distintos tipos de características:

Variante	Tipo de característica	Ejemplo de uso
Bernoulli Naïve Bayes	Presencia o ausencia de una característica	Si una palabra aparece o no aparece
Multinomial Naïve Bayes	Conteos discretos	Frecuencias de palabras en documentos
Gaussian Naïve Bayes	Variables continuas	Mediciones físicas, señales o atributos numéricos

La diferencia entre estas variantes está en cómo se modela $p(x_k | \omega_i)$. El principio de decisión sigue siendo el mismo: estimar una probabilidad por clase, sumar la evidencia de las características y elegir la clase con mayor posterior.

Aunque el supuesto de independencia suele ser falso, el clasificador puede funcionar muy bien en tareas de texto, filtrado de spam y clasificación de documentos, porque para decidir no siempre se necesita estimar perfectamente las probabilidades; basta con obtener fronteras de decisión útiles (McCallum y Nigam 1998).

143.4. Clasificadores bayesianos gaussianos

Una familia muy importante de clasificadores bayesianos se obtiene cuando las densidades condicionales de clase se modelan mediante distribuciones normales. Este supuesto es razonable cuando las características observadas resultan de la agregación de muchos factores independientes o débilmente dependientes, de acuerdo con la intuición asociada al teorema central del límite (Wasserman 2004; Bishop 2006).

Definición. Una variable aleatoria continua unidimensional x tiene distribución normal con media μ y varianza σ^2 , denotada $\mathcal{N}(\mu, \sigma^2)$, si su densidad es

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{x - \mu}{\sigma} \right)^2 \right] \quad (143.6)$$

En dimensión d , la distribución normal multivariada queda determinada por un vector de medias μ y una matriz de covarianza Σ .

Definición. Una observación $\mathbf{x} \in \mathbb{R}^d$ sigue una distribución normal multivariada $\mathcal{N}(\mu, \Sigma)$ si

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right] \quad (143.7)$$

143.4.1. Funciones discriminantes gaussianas

Si cada clase ω_i tiene una densidad condicional gaussiana

$$p(\mathbf{x} \mid \omega_i) = \mathcal{N}(\mathbf{x}; \mu_i, \Sigma_i),$$

entonces la decisión MAP puede obtenerse maximizando la función discriminante

$$g_i(\mathbf{x}) = \log p(\mathbf{x} \mid \omega_i) + \log P(\omega_i).$$

Eliminando términos constantes comunes a todas las clases:

$$g_i(\mathbf{x}) = -\frac{1}{2} \log |\Sigma_i| - \frac{1}{2} (\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i) + \log P(\omega_i).$$

Cuando las matrices de covarianza son distintas entre clases, la frontera $g_i(\mathbf{x}) = g_j(\mathbf{x})$ es cuadrática. Este caso se conoce como análisis discriminante cuadrático, o QDA (Hastie et al. 2009).

143.4.2. Casos especiales

143.4.2.1. Covarianza esférica común y priors iguales

Si $\Sigma_i = \sigma^2 \mathbf{I}$ para todas las clases y los priors son iguales, entonces maximizar $g_i(\mathbf{x})$ equivale a minimizar la distancia euclidiana a la media de la clase:

$$\hat{\omega}(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mu_i\|^2.$$

Este caso corresponde a un clasificador de distancia mínima o template matching.

143.4.2.2. Covarianza común y priors arbitrarios

Si $\Sigma_i = \Sigma$ para todas las clases, pero los priors no necesariamente son iguales, la función discriminante se vuelve lineal:

$$g_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + b_i,$$

con

$$\mathbf{w}_i = \Sigma^{-1} \mu_i, \quad b_i = -\frac{1}{2} \mu_i^T \Sigma^{-1} \mu_i + \log P(\omega_i).$$

Este caso corresponde al análisis discriminante lineal, o LDA (Fisher 1936; Hastie et al. 2009). El término $\log P(\omega_i)$ desplaza la frontera de decisión hacia la clase con menor prior, porque se requiere más evidencia para asignar una observación a una clase poco frecuente.

143.4.3. Ejemplo en Python: clasificador gaussiano con covarianza común

```

import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = make_blobs(
    n_samples=600,
    centers=[[-2, 0], [2, 1]],
    cluster_std=[1.2, 1.2],
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

classes = np.unique(y_train)
means = np.array([X_train[y_train == k].mean(axis=0) for k in classes])
priors = np.array([np.mean(y_train == k) for k in classes])

# Estimación de covarianza común por máxima verosimilitud agrupada.
S = np.zeros((X_train.shape[1], X_train.shape[1]))
for k, mu in zip(classes, means):
    Xk = X_train[y_train == k]
    centered = Xk - mu
    S += centered.T @ centered
S /= len(X_train)
S_inv = np.linalg.inv(S)

W = means @ S_inv.T
b = np.array([
    -0.5 * mu.T @ S_inv @ mu + np.log(pi)
    for mu, pi in zip(means, priors)
])

scores = X_test @ W.T + b
y_pred = classes[np.argmax(scores, axis=1)]

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

```

```

Accuracy: 0.95
[[85  5]
 [ 4 86]]

```

Este código implementa la regla discriminante lineal sin llamar directamente a una clase de `scikit-learn`.

La implementación deja visible la estructura bayesiana: estimación de medias, covarianza común, priors y maximización de $g_i(\mathbf{x})$.

143.4.4. Ejemplo interactivo 2: dos gaussianas en 1D

El siguiente ejemplo muestra el caso más simple de clasificación gaussiana: una sola variable x y dos clases. Cada clase se modela con una distribución normal distinta:

$$p(x | \omega_1) = \mathcal{N}(x; \mu_1, \sigma_1^2), \quad p(x | \omega_2) = \mathcal{N}(x; \mu_2, \sigma_2^2).$$

Para clasificar un valor observado x , no comparamos únicamente las densidades $p(x | \omega_i)$. La regla MAP compara las densidades ponderadas por sus probabilidades previas:

$$p(x | \omega_1)P(\omega_1) \quad \text{contra} \quad p(x | \omega_2)P(\omega_2).$$

La frontera de decisión aparece en los valores de x donde ambas cantidades son iguales. En esos puntos el clasificador está indiferente entre las dos clases:

$$p(x | \omega_1)P(\omega_1) = p(x | \omega_2)P(\omega_2).$$

El control interactivo modifica $P(\omega_1)$. Cuando aumenta el prior de una clase, esa clase necesita menos evidencia local para ser seleccionada. Por eso la frontera se desplaza hacia la otra clase: se requiere evidencia más fuerte para vencer a la clase que ahora es más probable a priori.

```
import numpy as np
import plotly.graph_objects as go

x = np.linspace(-5, 7, 600)
mu1, s1 = 0.0, 1.0
mu2, s2 = 2.2, 1.4

def normal_pdf(x, mu, sigma):
    return np.exp(-0.5*((x-mu)/sigma)**2)/(sigma*np.sqrt(2*np.pi))

frames = []
prior_grid = np.linspace(0.1, 0.9, 9)
for p1 in prior_grid:
    p2 = 1 - p1
    f1 = normal_pdf(x, mu1, s1)
    f2 = normal_pdf(x, mu2, s2)
    un1 = f1 * p1
    un2 = f2 * p2
    post1 = un1 / (un1 + un2)
    post2 = 1 - post1

    score = un1 - un2
    crossing_idx = np.where(np.diff(np.sign(score)) != 0)[0]
    boundary_x = x[crossing_idx] if len(crossing_idx) > 0 else np.array([])
```

```

boundary_y = np.interp(boundary_x, x, np.maximum(un1, un2))

frames.append(go.Frame(
    data=[
        go.Scatter(x=x, y=un1, mode="lines", name="p(x| 1)P( 1)"),
        go.Scatter(x=x, y=un2, mode="lines", name="p(x| 2)P( 2)"),
        go.Scatter(x=x, y=post1, mode="lines", name="P( 1|x)", yaxis="y2"),
        go.Scatter(x=x, y=post2, mode="lines", name="P( 2|x)", yaxis="y2"),
        go.Scatter(
            x=boundary_x,
            y=boundary_y,
            mode="markers",
            name="frontera MAP",
            marker=dict(size=9, symbol="x", color="black")
        )
    ],
    name=f"P( 1)={p1:.1f}"
))

fig = go.Figure(data=frames[4].data, frames=frames)
fig.update_layout(
    title="Dos clases gaussianas en 1D: el prior desplaza la decisión",
    xaxis_title="x",
    yaxis=dict(title="p(x | )P( )"),
    yaxis2=dict(title="posterior", overlaying="y", side="right", range=[0, 1]),
    height=500,
    legend=dict(orientation="h", y=-0.25),
    margin=dict(b=120),
    updatemenus=[dict(
        type="buttons",
        buttons=[dict(label="Animar", method="animate", args=[None])]
    )],
    sliders=[dict(
        steps=[dict(method="animate", args=[[fr.name], {"mode":"immediate"}], label=fr.name) for
    )]
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 143.5.: Densidades, posterior y frontera de decisión para dos clases gaussianas.

La gráfica muestra tres ideas importantes. Primero, las curvas $p(x | \omega_i)P(\omega_i)$ son las cantidades que realmente se comparan para decidir. Segundo, las posteriores $P(\omega_i | x)$ cambian suavemente con x : cerca de la media de una clase, su posterior suele aumentar. Tercero, si las varianzas son distintas, puede aparecer más de una frontera de decisión, porque las curvas gaussianas pueden cruzarse en más de un punto.

143.4.5. Clasificación gaussiana en 2D

En dos dimensiones, cada observación ya no es un número sino un vector:

$$\mathbf{x} = (x_1, x_2)^T.$$

Cada clase se describe con tres elementos:

- Un centro μ_i .
- Una matriz de covarianza Σ_i .
- Un prior $P(\omega_i)$.

El centro μ_i indica dónde se concentra la clase. La matriz Σ_i indica la forma de la nube: qué tan dispersa está, si está alargada en alguna dirección y si las variables x_1 y x_2 están correlacionadas. El prior $P(\omega_i)$ desplaza la decisión hacia la clase menos frecuente, porque se necesita más evidencia para asignar una observación a una clase con menor probabilidad previa.

La función discriminante para cada clase es:

$$g_i(\mathbf{x}) = \log p(\mathbf{x} \mid \omega_i) + \log P(\omega_i).$$

La frontera se obtiene donde las dos clases tienen la misma puntuación:

$$g_1(\mathbf{x}) = g_2(\mathbf{x}).$$

Si las covarianzas son iguales, la frontera es lineal. Si las covarianzas son distintas, la frontera puede ser curva, porque cada clase mide la cercanía a su centro con una geometría diferente.

143.4.6. Ejemplo interactivo 3: frontera en 2D

El siguiente ejemplo simula dos nubes gaussianas en el plano. El fondo indica la región donde gana cada clase y la línea de contorno muestra los puntos donde $g_1(\mathbf{x}) - g_2(\mathbf{x}) = 0$. A un lado de esa frontera se decide ω_1 y al otro lado se decide ω_2 .

```
import numpy as np
import plotly.graph_objects as go

np.random.seed(7)
n = 180
mu1 = np.array([-1.2, 0.0])
mu2 = np.array([1.2, 0.6])
S1 = np.array([[1.0, 0.55], [0.55, 1.0]])
S2 = np.array([[1.2, -0.45], [-0.45, 0.75]])
X1 = np.random.multivariate_normal(mu1, S1, n)
X2 = np.random.multivariate_normal(mu2, S2, n)

xx, yy = np.meshgrid(np.linspace(-5, 5, 180), np.linspace(-4, 5, 180))
G = np.c_[xx.ravel(), yy.ravel()]
```

```

def log_gauss(X, mu, S):
    invS = np.linalg.inv(S)
    diff = X - mu
    q = np.sum(diff @ invS * diff, axis=1)
    return -0.5*q - 0.5*np.log(np.linalg.det(S))

p1 = 0.5
score = log_gauss(G, mu1, S1) + np.log(p1) - log_gauss(G, mu2, S2) - np.log(1-p1)
Z = score.reshape(xx.shape)
decision_region = (Z > 0).astype(int)

fig = go.Figure()
fig.add_trace(go.Contour(
    x=xx[0],
    y=yy[:,0],
    z=decision_region,
    contours=dict(start=0, end=1, size=1, coloring="heatmap"),
    colorscale=[[0, "rgba(214,39,40,0.18)"], [1, "rgba(31,119,180,0.18)"]],
    showscale=False,
    line=dict(width=0),
    name="región de decisión"
))
fig.add_trace(go.Contour(
    x=xx[0],
    y=yy[:,0],
    z=Z,
    contours=dict(start=0, end=0, size=1),
    showscale=False,
    line=dict(color="black", width=3),
    name="frontera MAP"
))
fig.add_trace(go.Scatter(x=X1[:,0], y=X1[:,1], mode="markers", name="1", opacity=0.75))
fig.add_trace(go.Scatter(x=X2[:,0], y=X2[:,1], mode="markers", name="2", opacity=0.75))
fig.update_layout(
    title="Frontera MAP en 2D con covarianzas distintas",
    xaxis_title="x1",
    yaxis_title="x2",
    height=520,
    legend=dict(orientation="h", y=-0.15),
    margin=dict(b=90)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 143.6.: Frontera MAP para dos clases gaussianas en dos dimensiones.

La frontera no depende sólo de qué media esté más cerca en distancia euclidiana. También depende de la orientación y dispersión de cada nube. Por ejemplo, una clase muy dispersa puede asignar probabilidad

razonable a puntos alejados de su centro, mientras que una clase muy compacta exige que los puntos estén cerca de su media. Esa diferencia aparece en la forma curva de la frontera.

143.4.7. Interpretación geométrica

La cantidad

$$(\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)$$

es una distancia de Mahalanobis.

Significa que el modelo no mide sólo cercanía euclidiana, sino cercanía relativa a la forma de la nube de datos.



Tip

La covarianza codifica orientación, escala y correlación.

143.5. ¿Cómo evaluar el supuesto gaussiano?

El supuesto de normalidad rara vez debe aceptarse sin diagnóstico. En aplicaciones reales conviene combinar razonamiento sustantivo, visualización y pruebas estadísticas. La visualización suele ser más informativa que una prueba aislada, porque permite detectar asimetrías, colas pesadas, multimodalidad o valores atípicos.

Algunas herramientas útiles son:

- histogramas y estimaciones de densidad;
- diagramas de caja;
- gráficos Q-Q;
- pruebas de bondad de ajuste, como Kolmogorov–Smirnov, Shapiro–Wilk o pruebas basadas en χ^2 .

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(123)
x = rng.normal(loc=0, scale=1, size=300)

# Cuantiles teóricos y empíricos (implementación manual, sin scipy)
n = len(x)
x_sorted = np.sort(x)
# Probabilidades usando la fórmula de Filliben (i-0.375)/(n+0.25)
probs = (np.arange(1, n + 1) - 0.375) / (n + 0.25)
# Cuantiles teóricos de N(0,1) usando la aproximación racional de Abramowitz & Stegun
def norm_ppf(p):
    """Aproximación del cuantil normal estándar para 0 < p < 1."""
    p = np.clip(p, 1e-10, 1 - 1e-10)
    sign = np.where(p < 0.5, -1.0, 1.0)
```



```

pp = np.where(p < 0.5, p, 1 - p)
t = np.sqrt(-2 * np.log(pp))
c0, c1, c2 = 2.515517, 0.802853, 0.010328
d1, d2, d3 = 1.432788, 0.189269, 0.001308
num = c0 + c1 * t + c2 * t**2
den = 1 + d1 * t + d2 * t**2 + d3 * t**3
return sign * (t - num / den)

q_teoricos = norm_ppf(probs)

# Línea de referencia: regresión sobre cuantiles Q1 y Q3
q25_emp = np.percentile(x_sorted, 25)
q75_emp = np.percentile(x_sorted, 75)
q25_teo = norm_ppf(np.array([0.25]))[0]
q75_teo = norm_ppf(np.array([0.75]))[0]
slope = (q75_emp - q25_emp) / (q75_teo - q25_teo)
intercept = q25_emp - slope * q25_teo
ref_x = np.array([q_teoricos.min(), q_teoricos.max()])
ref_y = intercept + slope * ref_x

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=q_teoricos, y=x_sorted,
    mode="markers",
    marker=dict(color="#2563eb", size=5, opacity=0.6,
                line=dict(width=0.5, color="#1e3a6e")),
    name="Cuantiles empíricos",
    hovertemplate="Q teórico: %{x:.3f}<br>Q empírico: %{y:.3f}<extra></extra>"
))

fig.add_trace(go.Scatter(
    x=ref_x, y=ref_y,
    mode="lines",
    line=dict(color="#dc2626", width=2, dash="dash"),
    name="Línea de referencia"
))

fig.update_layout(
    title="Gráfico Q-Q frente a la normal estándar",
    xaxis_title="Cuantiles teóricos N(0,1)",
    yaxis_title="Cuantiles empíricos",
    template="plotly_white",
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1)
)
fig.show()

# Prueba de normalidad manual (estadístico de correlación de cuantiles)

```

```
r_qq = float(np.corrcoef(q_teoricos, x_sorted)[0, 1])
print(f"Correlación Q-Q ( 1 indica normalidad): r = {r_qq:.4f}")
```

Unable to display output for mime type(s): text/html

Figura 143.7.: Gráfico Q-Q comparando cuantiles empíricos de una muestra aleatoria contra cuantiles teóricos de una normal estándar. Los puntos sobre la línea diagonal indican buen ajuste gaussiano.

Correlación Q-Q (1 indica normalidad): r = 0.9966

Una advertencia importante es que las pruebas de normalidad son sensibles al tamaño de muestra. Con muestras grandes pueden detectar desviaciones pequeñas sin relevancia práctica; con muestras pequeñas pueden carecer de potencia. Por ello, el diagnóstico debe vincularse con el objetivo de clasificación.

143.6. Clasificación bayesiana de secuencias discretas

Los clasificadores bayesianos no se limitan a variables continuas. También pueden aplicarse a datos discretos estructurados, como texto, secuencias genómicas o series de símbolos. En biología computacional, un ejemplo clásico es la identificación de islas CpG en el genoma humano (Durbin et al. 1998).

Las islas CpG son regiones cortas del ADN donde el dinucleótido CG aparece con mayor frecuencia que en otras regiones. Una definición determinista clásica considera regiones de longitud mayor a 200 pares de bases, contenido $G+C$ superior a 50 % y razón observada/esperada de CpG mayor a 0.60 (Gardiner-Garden y Frommer 1987). Sin embargo, una perspectiva probabilística permite construir modelos generativos para regiones CpG y no CpG, y decidir con base en la razón de verosimilitudes.

Definición. El contenido $G + C$ de una secuencia de longitud N se define como

$$\%GC = \frac{N(C) + N(G)}{N} \quad (143.8)$$

Definición. La razón CpG observada/esperada puede escribirse como

$$\text{CpG ratio} = \frac{N(CpG)/N}{(N(C)/N)(N(G)/N)} \quad (143.9)$$

143.6.1. Cadenas de Markov de primer orden

Una cadena de Markov de primer orden modela una secuencia suponiendo que el símbolo actual depende sólo del símbolo inmediatamente anterior. Para ADN, el alfabeto es $\mathcal{A} = \{A, C, G, T\}$.

Definición. Una secuencia de variables aleatorias $X_1, X_2, \dots, X_n$ forma una cadena de Markov de primer orden si

$$P(X_n = x_n \mid X_1 = x_1, \dots, X_{n-1} = x_{n-1}) = P(X_n = x_n \mid X_{n-1} = x_{n-1}) \quad (143.10)$$

Las probabilidades de transición se denotan

$$a_{st} = P(X_i = t \mid X_{i-1} = s),$$

donde $s, t \in \mathcal{A}$. Para una secuencia $\mathbf{x} = (x_1, \dots, x_L)$, su probabilidad bajo una cadena de Markov es

$$P(\mathbf{x}) = P(x_1) \prod_{i=2}^L a_{x_{i-1}x_i}.$$

143.6.2. Log-odds para discriminar secuencias

Supongamos que se entrenan dos cadenas de Markov: una para islas CpG, denotada $+$, y otra para regiones no CpG, denotada $-$. La puntuación log-odds de una secuencia es

Definición. La puntuación log-odds entre dos modelos de Markov para una secuencia $\mathbf{x}$ es

$$S(\mathbf{x}) = \log \frac{P(\mathbf{x} \mid +)}{P(\mathbf{x} \mid -)} = \sum_{i=2}^L \log \frac{a_{x_{i-1}x_i}^+}{a_{x_{i-1}x_i}^-} \quad (143.11)$$

Si $S(\mathbf{x})$ supera un umbral, la secuencia se clasifica como CpG; si no, como no CpG. El umbral puede fijarse por priors, por costos de error o por validación empírica.

143.6.3. Ejemplo en Python: puntuación de una secuencia de ADN

```
import numpy as np

alphabet = ["A", "C", "G", "T"]
idx = {s: i for i, s in enumerate(alphabet)}

A_plus = np.array([
    [0.180, 0.274, 0.426, 0.120],
    [0.171, 0.368, 0.274, 0.188],
    [0.161, 0.339, 0.375, 0.125],
    [0.079, 0.355, 0.384, 0.182],
])

A_minus = np.array([
    [0.300, 0.205, 0.285, 0.210],
    [0.322, 0.298, 0.078, 0.302],
    [0.248, 0.246, 0.298, 0.208],
    [0.177, 0.239, 0.292, 0.292],
])

def markov_log_odds(seq, A_pos, A_neg):
    score = 0.0
    for prev, curr in zip(seq[:-1], seq[1:]):
```

```

    i, j = idx[prev], idx[curr]
    score += np.log(A_pos[i, j] / A_neg[i, j])
    return score

seq = "ACGCGCGTACGCGT"
score = markov_log_odds(seq, A_plus, A_minus)
normalized_score = score / (len(seq) - 1)

print("Score:", score)
print("Length-normalized score:", normalized_score)
print("Decision:", "CpG" if score > 0 else "non-CpG")

```

```

Score: 5.9991878024311385
Length-normalized score: 0.46147598480239527
Decision: CpG

```

La normalización por longitud permite comparar secuencias de tamaños distintos. En problemas reales se recomienda estimar las matrices de transición con suavizado, por ejemplo mediante pseudo-conteos, para evitar probabilidades cero.

143.7. Clasificadores bayesianos de riesgo mínimo

La decisión de error mínimo supone que todos los errores cuestan lo mismo. Sin embargo, en muchas aplicaciones esto no es cierto. En diagnóstico médico, por ejemplo, un falso negativo puede ser mucho más costoso que un falso positivo. En detección de fraude, bloquear una transacción legítima y dejar pasar una transacción fraudulenta tampoco tienen el mismo costo. Por ello se introduce una función de pérdida.

Definición. La pérdida $\lambda(\alpha_i, \omega_j)$ representa el costo de tomar la decisión α_i cuando el verdadero estado de la naturaleza es ω_j :

$$\lambda_{ij} = \lambda(\alpha_i, \omega_j) \quad (143.12)$$

Dada una observación $\mathbf{x}$, el riesgo condicional de tomar la decisión α_i es la pérdida esperada bajo la distribución posterior de las clases.

Definición. El riesgo condicional de decidir α_i dado $\mathbf{x}$ es

$$R(\alpha_i | \mathbf{x}) = \sum_{j=1}^c \lambda(\alpha_i, \omega_j) P(\omega_j | \mathbf{x}) \quad (143.13)$$

La regla de riesgo mínimo selecciona la decisión con menor riesgo condicional:

$$\alpha^*(\mathbf{x}) = \arg \min_i R(\alpha_i | \mathbf{x}).$$

Cuando $\lambda_{ii} = 0$ y $\lambda_{ij} = 1$ para $i \neq j$, la regla de riesgo mínimo se reduce a la regla de error mínimo. Es decir, la clasificación MAP es un caso particular de la teoría de decisión bayesiana (Berger 1985; Duda et al. 2001).

143.7.1. Dos clases con matriz de pérdidas

Para dos clases y dos decisiones, la matriz de pérdidas puede escribirse como

	ω_1	ω_2
α_1	λ_{11}	λ_{12}
α_2	λ_{21}	λ_{22}

La decisión α_1 se toma si

$$\lambda_{11}P(\omega_1 | \mathbf{x}) + \lambda_{12}P(\omega_2 | \mathbf{x}) < \lambda_{21}P(\omega_1 | \mathbf{x}) + \lambda_{22}P(\omega_2 | \mathbf{x}).$$

De manera equivalente, el umbral del cociente de verosimilitudes se modifica por los costos:

$$\ell(\mathbf{x}) \geq_{\alpha_2}^{\alpha_1} \frac{P(\omega_2)}{P(\omega_1)} \cdot \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}.$$

143.7.2. Ejemplo en Python: error mínimo frente a riesgo mínimo

```
px_w1 = 0.2
px_w2 = 0.4
p_w1 = 0.9
p_w2 = 0.1

posterior, _ = clasificador_bayesiano(
    verosimilitudes=[px_w1, px_w2],
    priors=[p_w1, p_w2],
    clases=["normal", "enfermedad"]
)

post_w1, post_w2 = posterior

# Matriz de pérdida:
# alpha_1 = decidir normal, alpha_2 = decidir enfermedad
# omega_1 = normal, omega_2 = enfermedad
loss = np.array([
    [0, 6],
    [1, 0]
])

risks = loss @ posterior

print("Posterior:", posterior)
print("Conditional risks:", risks)
print("Minimum-error decision:", "normal" if post_w1 > post_w2 else "enfermedad")
print("Minimum-risk decision:", "normal" if np.argmin(risks) == 0 else "enfermedad")
```

```

Posterior: [0.81818182 0.18181818]
Conditional risks: [1.09090909 0.81818182]
Minimum-error decision: normal
Minimum-risk decision: enfermedad

```

El resultado muestra que la decisión de error mínimo puede ser “normal”, mientras que la decisión de riesgo mínimo puede ser “enfermedad” si el costo de no detectar la enfermedad es suficientemente alto. Esta diferencia es fundamental en sistemas de decisión sensibles al costo.

```

import numpy as np
import plotly.graph_objects as go

# Matriz de pérdida:  =6 (falso negativo),  =1 (falso positivo)
lam12 = 6.0 # costo de decidir "normal" cuando hay enfermedad
lam21 = 1.0 # costo de decidir "enfermedad" cuando es normal

post2 = np.linspace(0, 1, 300) # P( |x)
post1 = 1 - post2

# R( |x) =  ·P( |x) → riesgo de decidir "normal"
# R( |x) =  ·P( |x) → riesgo de decidir "enfermedad"
R1 = lam12 * post2
R2 = lam21 * post1

# Umbral: R1 = R2 → lam12·p2 = lam21·(1-p2) → p2* = lam21/(lam12+lam21)
threshold = lam21 / (lam12 + lam21)

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=post2, y=R1,
    mode="lines",
    line=dict(color="#2563eb", width=2.5),
    name="R( |x) = decidir normal",
    hovertemplate="P( |x)=%{x:.3f}<br>Riesgo=%{y:.3f}<extra></extra>"
)))

fig.add_trace(go.Scatter(
    x=post2, y=R2,
    mode="lines",
    line=dict(color="#dc2626", width=2.5),
    name="R( |x) = decidir enfermedad",
    hovertemplate="P( |x)=%{x:.3f}<br>Riesgo=%{y:.3f}<extra></extra>"
)))

# Región donde conviene (decidir normal)
fig.add_vrect(x0=0, x1=threshold, fillcolor="#2563eb", opacity=0.07,
    line_width=0, annotation_text="decidir normal",
    annotation_position="top left",
    annotation_font=dict(color="#2563eb", size=11))

```

```

# Región donde conviene (decidir enfermedad)
fig.add_vrect(x0=threshold, x1=1, fillcolor="#dc2626", opacity=0.07,
              line_width=0, annotation_text="decidir enfermedad",
              annotation_position="top right",
              annotation_font=dict(color="#dc2626", size=11))

# Umbral
fig.add_vline(x=threshold, line_dash="dash", line_color="#475569", line_width=1.8,
              annotation_text=f"p* = /( + ) = {threshold:.3f}",
              annotation_position="bottom right",
              annotation_font=dict(size=11))

fig.update_layout(
    title=f"Riesgo condicional por acción (={lam12:.0f},={lam21:.0f})",
    xaxis_title="P( |x) - probabilidad posterior de enfermedad",
    yaxis_title="Riesgo condicional R( |x)",
    template="plotly_white",
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 143.8.: Riesgo condicional de cada acción como función de la probabilidad posterior $P(|x)$. La intersección de las dos curvas define el umbral óptimo de decisión bajo la matriz de pérdida especificada.

143.8. Minimizar un tipo de error condicionado al otro

En algunos problemas no se busca minimizar una suma ponderada de errores, sino controlar explícitamente un tipo de error mientras se minimiza otro. Esta idea aparece en pruebas de hipótesis, detección de señales y clasificación con restricciones operativas. El caso clásico es minimizar la probabilidad de falso negativo sujeto a una cota sobre la probabilidad de falso positivo, o viceversa. La regla resultante se relaciona con el lema de Neyman–Pearson (Neyman y Pearson 1933).

Para dos clases, sea $\mathcal{R}_1$ la región donde se decide ω_1 y $\mathcal{R}_2$ la región donde se decide ω_2 . Las probabilidades de error condicionales pueden escribirse como

$$P_1(e) = \int_{\mathcal{R}_2} p(\mathbf{x} | \omega_1) d\mathbf{x}, \quad P_2(e) = \int_{\mathcal{R}_1} p(\mathbf{x} | \omega_2) d\mathbf{x}.$$

Si se desea minimizar $P_1(e)$ sujeto a $P_2(e) = \epsilon$, se obtiene una regla basada en umbrales del cociente de verosimilitudes:

$$\frac{p(\mathbf{x} | \omega_1)}{p(\mathbf{x} | \omega_2)} \geq \lambda.$$

El valor de λ se ajusta para cumplir la restricción sobre el error. En términos prácticos, esto equivale a desplazar el umbral de decisión hasta alcanzar una sensibilidad, especificidad, tasa de falsos positivos o tasa de falsos negativos deseada.

143.8.1. 8.1 Ejemplo en Python: elegir umbral por restricción de falsos positivos

```
from sklearn.metrics import roc_curve

rng = np.random.default_rng(7)
# Scores altos favorecen la clase positiva.
y_true = np.r_[np.zeros(500), np.ones(500)]
scores = np.r_[rng.normal(0, 1, 500), rng.normal(2, 1, 500)]

fpr, tpr, thresholds = roc_curve(y_true, scores)

max_fpr = 0.05
valid = np.where(fpr <= max_fpr)[0]
best_idx = valid[np.argmax(tpr[valid])]

print("Threshold:", thresholds[best_idx])
print("FPR:", fpr[best_idx])
print("TPR:", tpr[best_idx])
```

```
Threshold: 1.4606730052576595
FPR: 0.05
TPR: 0.726
```

Este enfoque es más adecuado que maximizar accuracy cuando una clase es rara o cuando los errores tienen consecuencias asimétricas.

143.9. Criterio minimax cuando los priors son desconocidos

La regla bayesiana clásica requiere conocer o estimar los priors $P(\omega_i)$. Cuando estos priors son inciertos, una alternativa conservadora es el criterio minimax. La idea es escoger la regla de decisión que minimiza el peor riesgo posible bajo los priors admisibles (Berger 1985; Duda et al. 2001).

Definición. Una regla minimax selecciona la decisión o regla de decisión α que minimiza el máximo riesgo posible:

$$\alpha_{\text{minimax}} = \arg \min_{\alpha} \max_{\pi \in \Pi} R(\alpha; \pi) \quad (143.14)$$

Aquí π representa el vector de priors y Π el conjunto de priors considerados plausibles. En problemas de dos clases, puede estudiarse el riesgo como función de $P(\omega_1)$, ya que $P(\omega_2) = 1 - P(\omega_1)$. La regla minimax evita depender de una estimación puntual del prior y protege contra escenarios adversos. Su desventaja es que puede ser demasiado conservadora si sí existe información confiable sobre las frecuencias reales de clase.

143.10. Recomendaciones prácticas

La clasificación bayesiana es conceptualmente elegante, pero su desempeño depende de la calidad del modelo probabilístico. Algunas recomendaciones prácticas son:

1. Estimar priors con cuidado. En datos desbalanceados, los priors empíricos pueden reflejar sesgos de muestreo y no prevalencias reales.
2. Trabajar en escala logarítmica para evitar problemas numéricos.
3. Validar supuestos distribucionales cuando se usan modelos gaussianos.
4. Usar suavizado en modelos discretos, especialmente en texto y secuencias.
5. Elegir la métrica o función de pérdida de acuerdo con el problema real, no sólo con accuracy.
6. Separar conceptualmente probabilidad, decisión y costo: una buena estimación probabilística no implica automáticamente una buena regla de decisión si el umbral no corresponde al costo operativo.

143.11. Conclusiones

Los clasificadores bayesianos proporcionan un marco unificado para tomar decisiones bajo incertidumbre. La regla MAP minimiza la probabilidad de error cuando los errores tienen el mismo costo, mientras que la regla de riesgo mínimo permite incorporar pérdidas asimétricas. Naïve Bayes muestra cómo construir clasificadores prácticos en alta dimensión mediante una suposición de independencia condicional. Los clasificadores gaussianos muestran cómo los supuestos sobre las densidades condicionales determinan la geometría de las fronteras de decisión: fronteras lineales bajo covarianza común y fronteras cuadráticas bajo covarianzas distintas. Por otro lado, los modelos de Markov muestran que el mismo principio puede aplicarse a secuencias discretas con dependencia entre símbolos.

La idea central del capítulo es que clasificar no consiste únicamente en trazar una frontera, sino en articular tres elementos: un modelo de cómo se generan los datos, una actualización probabilística de las creencias y una regla de decisión adaptada al objetivo. Esta separación hace que los métodos bayesianos sean especialmente útiles en problemas donde la incertidumbre, los costos de error y la interpretabilidad son componentes esenciales del sistema de aprendizaje.

144. Modelos de Mezcla Gaussiana

Los **modelos de mezcla gaussiana** (GMM, por sus siglas en inglés) son modelos probabilísticos generativos que representan la distribución de los datos como una suma ponderada de distribuciones normales, lo que los hace ideales para capturar estructuras multimodales que una sola gaussiana no puede describir (Bishop 2006; Murphy 2012). El algoritmo de Expectation-Maximization (EM), introducido formalmente por Dempster et al. (1977), proporciona el marco de optimización para estimar los parámetros del modelo a partir de datos incompletos (en este caso, los datos están “incompletos” porque la asignación de cada observación a cada componente es desconocida). Los GMM pueden entenderse como una generalización probabilística del algoritmo K-Means: cuando las covarianzas son esféricas e iguales y se toma el límite de varianza cero, las asignaciones blandas del GMM convergen a las asignaciones duras de K-Means (Hastie et al. 2009). En este capítulo se desarrolla la teoría completa del modelo, el algoritmo EM desde cero, y todos los ejemplos computacionales se implementan exclusivamente con `numpy` y `plotly`.

144.1. Motivación: ¿por qué un solo gaussiano no basta?

La distribución normal multivariada es, sin duda, el modelo probabilístico más utilizado en aprendizaje automático. Su elegancia matemática y sus propiedades analíticas la hacen atractiva, pero su aplicabilidad está fundamentalmente limitada a datos **unimodales**: datos cuya densidad tiene un único máximo y una forma aproximadamente elipsoidal. Cuando los datos provienen de varias poblaciones distintas, o cuando la verdadera densidad tiene múltiples crestas, un único gaussiano ajustado a todos los datos producirá una descripción pobre que coloca masa de probabilidad en regiones vacías y asigna poca densidad a las regiones donde realmente se concentran las observaciones.

Considérese un conjunto de datos unidimensional formado por dos grupos bien separados. La figura Figura 144.1 ilustra el contraste entre un único gaussiano ajustado a todos los datos (panel izquierdo) y una mezcla de dos gaussianas (panel derecho). El modelo unimodal coloca la mayor parte de su masa en el valle entre los dos grupos, mientras que la mezcla captura fielmente la estructura bimodal. Este ejemplo simple motiva la necesidad de modelos más flexibles que puedan representar distribuciones multimodales de manera principada.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(42)

# Generar datos bimodales
datos_1 = rng.normal(-3, 0.8, 300)
datos_2 = rng.normal(3, 1.0, 200)
datos = np.concatenate([datos_1, datos_2])

# Ajustar un solo gaussiano
```

```

mu_unico = datos.mean()
sigma_unico = datos.std()

# Parametros GMM hardcoded (resultado razonable de EM)
pi1, mu1_gmm, sigma1_gmm = 0.60, -3.0, 0.80
pi2, mu2_gmm, sigma2_gmm = 0.40, 3.0, 1.00

x_plot = np.linspace(-7, 7, 400)

def normal_1d(x, mu, sigma):
    return np.exp(-0.5 * ((x - mu) / sigma) ** 2) / (sigma * np.sqrt(2 * np.pi))

pdf_unico = normal_1d(x_plot, mu_unico, sigma_unico)
pdf_gmm = pi1 * normal_1d(x_plot, mu1_gmm, sigma1_gmm) + \
    pi2 * normal_1d(x_plot, mu2_gmm, sigma2_gmm)

fig = make_subplots(rows=1, cols=2,
                    subplot_titles=("Un solo gaussiano", "Mezcla de 2 gaussianas"))

# Panel izquierdo: histograma + gaussiano unico
fig.add_trace(
    go.Histogram(x=datos, histnorm='probability density', nbinsx=40,
                 name='Datos', marker_color='steelblue', opacity=0.5,
                 showlegend=True),
    row=1, col=1
)
fig.add_trace(
    go.Scatter(x=x_plot, y=pdf_unico, mode='lines',
               name='N(x| , )', line=dict(color='crimson', width=2.5),
               showlegend=True),
    row=1, col=1
)

# Panel derecho: histograma + mezcla GMM
fig.add_trace(
    go.Histogram(x=datos, histnorm='probability density', nbinsx=40,
                 name='Datos', marker_color='steelblue', opacity=0.5,
                 showlegend=False),
    row=1, col=2
)
fig.add_trace(
    go.Scatter(x=x_plot, y=pdf_gmm, mode='lines',
               name='GMM (K=2)', line=dict(color='darkorange', width=2.5),
               showlegend=True),
    row=1, col=2
)
fig.add_trace(
    go.Scatter(x=x_plot, y=pi1 * normal_1d(x_plot, mu1_gmm, sigma1_gmm),
               mode='lines', name='Componente 1',

```

```

        line=dict(color='green', width=1.5, dash='dash'),
        showlegend=True),
    row=1, col=2
)
fig.add_trace(
    go.Scatter(x=x_plot, y=pi2 * normal_1d(x_plot, mu2_gmm, sigma2_gmm),
        mode='lines', name='Componente 2',
        line=dict(color='purple', width=1.5, dash='dash'),
        showlegend=True),
    row=1, col=2
)

fig.update_xaxes(title_text="x", row=1, col=1)
fig.update_xaxes(title_text="x", row=1, col=2)
fig.update_yaxes(title_text="Densidad de probabilidad", row=1, col=1)
fig.update_layout(height=420, width=850,
    title_text="Gaussiano único vs. Mezcla gaussiana",
    legend=dict(x=0.75, y=0.95))
fig.show()

```

Unable to display output for mime type(s): text/html

- (a) Motivación para los modelos de mezcla. Panel izquierdo: un único gaussiano ajustado a datos bimodales produce un ajuste pobre, colocando masa de probabilidad en el valle entre los grupos. Panel derecho: una mezcla de dos gaussianas captura correctamente la estructura bimodal.

Unable to display output for mime type(s): text/html

(b)

Figura 144.1.

144.2. El modelo de mezcla gaussiana

144.2.1. Definición formal

Un modelo de mezcla gaussiana con K componentes define la siguiente densidad de probabilidad sobre $\mathbf{x} \in \mathbb{R}^D$:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) \quad (144.1)$$

donde $\mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$ es la densidad gaussiana multivariada con media μ_k y matriz de covarianza Σ_k :

$$\mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_k)^\top \Sigma_k^{-1} (\mathbf{x} - \mu_k)\right) \quad (144.2)$$

Los **coeficientes de mezcla** π_k satisfacen las restricciones $\pi_k \geq 0$ para todo k y $\sum_{k=1}^K \pi_k = 1$, de modo que pueden interpretarse como probabilidades a priori sobre las componentes: $\pi_k = p(z = k)$.

144.2.2. Variable latente y verosimilitud completa

La formulación con variable latente es fundamental para entender el algoritmo EM. Se introduce una variable aleatoria discreta $\mathbf{z} \in \{1, \dots, K\}$ (o equivalentemente, un vector one-hot $\mathbf{z} \in \{0, 1\}^K$ con $\sum_k z_k = 1$) que indica cuál componente generó la observación $\mathbf{x}$. La distribución conjunta del modelo generativo es:

$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}) p(\mathbf{x} | \mathbf{z}) = \prod_{k=1}^K \left[\pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k) \right]^{z_k} \quad (144.3)$$

La densidad marginal Ecuación 144.1 se obtiene sumando (integrando) sobre todos los posibles valores de $\mathbf{z}$:

$$p(\mathbf{x}) = \sum_{k=1}^K p(z = k) p(\mathbf{x} | z = k) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Para un conjunto de N observaciones i.i.d. $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, la **log-verosimilitud observada** es:

$$\ell(\theta) = \sum_{i=1}^N \log \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \quad (144.4)$$

donde $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ son todos los parámetros del modelo. El logaritmo de una suma no tiene solución analítica en forma cerrada, lo que motiva el uso del algoritmo EM.

144.2.3. Densidad de una mezcla en 2D

La figura Figura 144.2 muestra la densidad de una mezcla con $K = 3$ componentes en $\mathbb{R}^2$. Se observa cómo cada componente contribuye una “joroba” en la densidad total, y la mezcla puede representar distribuciones con formas geométricas arbitrariamente complejas.

```
import numpy as np
import plotly.graph_objects as go

rng2 = np.random.default_rng(7)

# Parametros de los 3 componentes
pis_true = [0.40, 0.35, 0.25]
mus_true = [np.array([-3.0, -2.0]),
             np.array([ 3.0, -1.5]),
             np.array([ 0.0,  3.5])]
Sigs_true = [np.array([[1.2, 0.6], [0.6, 0.8]]),
             np.array([[0.9, -0.4], [-0.4, 1.5]]),
             np.array([[0.5, 0.0], [0.0, 0.5]])]

colores_comp = ['#e41a1c', '#377eb8', '#4daf4a']
```

```

# Generar puntos de cada componente
n_total = 500
ns = [int(pis_true[k] * n_total) for k in range(3)]
ns[-1] = n_total - sum(ns[:-1])

puntos = []
etiquetas = []
for k in range(3):
    pts = rng2.multivariate_normal(mus_true[k], Sigs_true[k], ns[k])
    puntos.append(pts)
    etiquetas.extend([k] * ns[k])

# Calcular densidad en grilla
def gauss_pdf_2d(X, mu, Sigma):
    D = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    inv_S = np.linalg.inv(Sigma)
    maha = np.einsum('ij,jk,ik->i', diff, inv_S, diff)
    return np.exp(-0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet))

x_g = np.linspace(-7, 7, 150)
y_g = np.linspace(-5, 7, 150)
XX, YY = np.meshgrid(x_g, y_g)
grid = np.column_stack([XX.ravel(), YY.ravel()])

Z_dens = np.zeros(grid.shape[0])
for k in range(3):
    Z_dens += pis_true[k] * gauss_pdf_2d(grid, mus_true[k], Sigs_true[k])
Z_dens = Z_dens.reshape(XX.shape)

fig2 = go.Figure()

# Contorno de densidad
fig2.add_trace(go.Contour(
    x=x_g, y=y_g, z=Z_dens,
    colorscale='Blues',
    contours=dict(showlabels=False, coloring='fill'),
    opacity=0.7,
    showscale=True,
    colorbar=dict(title="Densidad"),
    name='Densidad GMM'
))

# Puntos de cada componente
nombres_comp = ['Componente 1', 'Componente 2', 'Componente 3']
for k in range(3):
    pts_k = puntos[k]
    fig2.add_trace(go.Scatter(

```

```

        x=pts_k[:, 0], y=pts_k[:, 1],
        mode='markers',
        name=nombres_comp[k],
        marker=dict(color=colores_comp[k], size=4, opacity=0.7,
                    line=dict(color='white', width=0.3))
    ))

fig2.update_layout(
    height=500, width=700,
    xaxis_title="x ",
    yaxis_title="x ",
    title="Densidad de la mezcla gaussiana (K=3)",
    legend=dict(x=0.78, y=0.98)
)
fig2.show()

```

Unable to display output for mime type(s): text/html

Figura 144.2.: Densidad de una mezcla gaussiana con $K=3$ componentes en 2D. Los contornos rellenos muestran la densidad conjunta de la mezcla; los puntos coloreados representan las observaciones asignadas a cada componente.

144.3. El algoritmo EM

144.3.1. Principio general: ascenso coordinado sobre el ELBO

El algoritmo EM (Expectation-Maximization) es un procedimiento iterativo para encontrar estimadores de máxima verosimilitud en modelos con variables latentes (Dempster et al. 1977; McLachlan y Peel 2000). La dificultad central es que la log-verosimilitud observada Ecuación 144.4 contiene el logaritmo de una suma, que no tiene solución analítica. La clave del EM es introducir una cota inferior (ELBO, *Evidence Lower BOund*) que sí es maximizable en forma cerrada.

Para cualquier distribución $q(\mathbf{z})$ sobre la variable latente, la desigualdad de Jensen garantiza:

$$\log p(\mathbf{x} \mid \theta) \geq \mathbb{E}_q \left[\log \frac{p(\mathbf{x}, \mathbf{z} \mid \theta)}{q(\mathbf{z})} \right] \equiv \mathcal{L}(q, \theta) \quad (144.5)$$

El EM alterna entre dos pasos que maximizan este funcional de manera coordinada.

144.3.2. Paso E: cálculo de responsabilidades

El **paso E** fija los parámetros $\theta^{(t)}$ y maximiza $\mathcal{L}$ con respecto a $q(\mathbf{z})$. La solución óptima es la distribución posterior verdadera $q^*(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{x}, \theta^{(t)})$. Para el GMM, esta posterior factoriza y la **responsabilidad** de la componente k sobre la observación $\mathbf{x}_i$ es:

$$r_{ik} = p(z_i = k \mid \mathbf{x}_i, \theta^{(t)}) = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_i \mid \mu_k^{(t)}, \Sigma_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}_i \mid \mu_j^{(t)}, \Sigma_j^{(t)})} \quad (144.6)$$

Nótese que $r_{ik} \geq 0$ y $\sum_k r_{ik} = 1$ para todo i ; las responsabilidades son asignaciones **blandas** (probabilísticas) de cada punto a cada componente.

144.3.3. Paso M: actualización de parámetros

El **paso M** fija $q(\mathbf{z})$ y maximiza $\mathcal{L}$ con respecto a θ . Definiendo $N_k = \sum_{i=1}^N r_{ik}$ (número efectivo de puntos en la componente k), las actualizaciones en forma cerrada son:

$$\pi_k^{(t+1)} = \frac{N_k}{N} \quad (144.7)$$

$$\mu_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N r_{ik} \mathbf{x}_i \quad (144.8)$$

$$\Sigma_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N r_{ik} (\mathbf{x}_i - \mu_k^{(t+1)})(\mathbf{x}_i - \mu_k^{(t+1)})^\top \quad (144.9)$$

144.3.4. Convergencia y propiedad monótona

Una propiedad fundamental del EM es que la log-verosimilitud observada $\ell(\theta^{(t)})$ es **monótonamente no decreciente** en cada iteración: $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$. Esto se demuestra formalmente usando la desigualdad de Jensen y el hecho de que ambos pasos maximizan el ELBO. En la práctica, el algoritmo se detiene cuando el incremento en log-verosimilitud cae por debajo de una tolerancia ε (por ejemplo, 10^{-6}).

144.3.5. Implementación desde cero en numpy

El siguiente bloque implementa el algoritmo EM completo, incluyendo un truco numérico para evitar desbordamiento: se calcula el logaritmo de las densidades y se aplica la sustracción del máximo antes de exponenciar (equivalente al truco log-sum-exp).

```
import numpy as np
import plotly.graph_objects as go

rng3 = np.random.default_rng(123)

# ---- Datos sinteticos 2D ----
n1, n2 = 180, 220
X1 = rng3.multivariate_normal([-3, 1], [[1.0, 0.5], [0.5, 0.8]], n1)
X2 = rng3.multivariate_normal([3, -1], [[0.8, -0.3], [-0.3, 1.2]], n2)
X_em = np.vstack([X1, X2])
N_em, D_em = X_em.shape
```

```

# ---- Funciones EM ----
def gauss_log_pdf(X, mu, Sigma):
    D = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    inv_S = np.linalg.inv(Sigma)
    maha = np.einsum('ij,jk,ik->i', diff, inv_S, diff)
    return -0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet)

def e_step(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_R = np.zeros((N, K))
    for k in range(K):
        log_R[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf(X, mus[k], Sigmas[k])
    # log-sum-exp trick
    log_R_max = log_R.max(axis=1, keepdims=True)
    log_R_norm = log_R - log_R_max
    R = np.exp(log_R_norm)
    R /= R.sum(axis=1, keepdims=True)
    return R

def m_step(X, R):
    N, D = X.shape
    K = R.shape[1]
    Nk = R.sum(axis=0)
    pis_new = Nk / N
    mus_new = (R.T @ X) / Nk[:, None]
    Sigmas_new = []
    for k in range(K):
        diff = X - mus_new[k]
        Sk = (R[:, k:k+1] * diff).T @ diff / Nk[k]
        Sk += 1e-6 * np.eye(D)
        Sigmas_new.append(Sk)
    return pis_new, mus_new, Sigmas_new

def log_likelihood(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_lik_mat = np.zeros((N, K))
    for k in range(K):
        log_lik_mat[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf(X, mus[k], Sigmas[k])
    lse_max = log_lik_mat.max(axis=1)
    log_sum = lse_max + np.log(np.exp(log_lik_mat - lse_max[:, None]).sum(axis=1) + 1e-300)
    return log_sum.sum()

# ---- Inicializacion aleatoria ----
K_em = 2
idx_init = rng3.choice(N_em, K_em, replace=False)

```

```

pis_em = np.array([0.5, 0.5])
mus_em = [X_em[idx_init[k]].copy() for k in range(K_em)]
Sigmas_em = [np.eye(D_em) for _ in range(K_em)]

# Iteraciones a guardar
iters_guardar = [0, 1, 2, 3, 5, 10, 20]
max_iter = max(iters_guardar)

historial_estados = {} # iter -> (R, pis, mus, Sigmas)
ll_historia = []

R_em = e_step(X_em, pis_em, mus_em, Sigmas_em)
if 0 in iters_guardar:
    historial_estados[0] = (R_em.copy(),
                           pis_em.copy(),
                           [m.copy() for m in mus_em],
                           [s.copy() for s in Sigmas_em])

for it in range(1, max_iter + 1):
    pis_em, mus_em, Sigmas_em = m_step(X_em, R_em)
    R_em = e_step(X_em, pis_em, mus_em, Sigmas_em)
    ll = log_likelihood(X_em, pis_em, mus_em, Sigmas_em)
    ll_historia.append((it, ll))
    if it in iters_guardar:
        historial_estados[it] = (R_em.copy(),
                                pis_em.copy(),
                                [m.copy() for m in mus_em],
                                [s.copy() for s in Sigmas_em])

# ---- Funcion para dibujar elipse de confianza ----
def elipse_confianza(mu, Sigma, escala=2.0, n_pts=100):
    vals, vecs = np.linalg.eigh(Sigma)
    t = np.linspace(0, 2 * np.pi, n_pts)
    v1 = vecs[:, 0] * np.sqrt(vals[0]) * escala
    v2 = vecs[:, 1] * np.sqrt(vals[1]) * escala
    xe = mu[0] + v1[0] * np.cos(t) + v2[0] * np.sin(t)
    ye = mu[1] + v1[1] * np.cos(t) + v2[1] * np.sin(t)
    return xe, ye

# ---- Construir frames de animacion ----
colores_anim = ['#1f77b4', '#d62728'] # azul, rojo

frames = []
for it in iters_guardar:
    R_f, pis_f, mus_f, Sigs_f = historial_estados[it]

    # Color de cada punto segun responsabilidad hacia comp 0
    r0 = R_f[:, 0] # responsabilidad componente 0
    colores_pts = [

```

```

        'rgba({}, {}, {}, 0.8)'.format(
            int(r0[i] * 31 + (1 - r0[i]) * 214),
            int(r0[i] * 119 + (1 - r0[i]) * 39),
            int(r0[i] * 180 + (1 - r0[i]) * 40)
        )
        for i in range(N_em)
    ]

    trazas_frame = []

    # Scatter de puntos
    trazas_frame.append(go.Scatter(
        x=X_em[:, 0], y=X_em[:, 1],
        mode='markers',
        marker=dict(color=colores_pts, size=5),
        name='Datos'
    ))

    # Elipses de cada componente
    for k in range(K_em):
        xe, ye = ellipse_confianza(mus_f[k], Sigs_f[k], escala=2.0)
        trazas_frame.append(go.Scatter(
            x=xe, y=ye,
            mode='lines',
            line=dict(color=colores_anim[k], width=2.5),
            name='Componente {}'.format(k + 1)
        ))
        trazas_frame.append(go.Scatter(
            x=[mus_f[k][0]], y=[mus_f[k][1]],
            mode='markers',
            marker=dict(color=colores_anim[k], size=10, symbol='x'),
            name='Media {}'.format(k + 1)
        ))

    frames.append(go.Frame(data=trazas_frame,
                           name=str(it),
                           layout=go.Layout(title_text='Iteracion EM: {}'.format(it))))

# ---- Figura inicial (iteracion 0) ----
R_0, pis_0, mus_0, Sigs_0 = historial_estados[0]
r0_init = R_0[:, 0]
colores_init = [
    'rgba({}, {}, {}, 0.8)'.format(
        int(r0_init[i] * 31 + (1 - r0_init[i]) * 214),
        int(r0_init[i] * 119 + (1 - r0_init[i]) * 39),
        int(r0_init[i] * 180 + (1 - r0_init[i]) * 40)
    )
    for i in range(N_em)
]

```

```

trazas_init = []
trazas_init.append(go.Scatter(
    x=X_em[:, 0], y=X_em[:, 1],
    mode='markers',
    marker=dict(color=colores_init, size=5),
    name='Datos'
))
for k in range(K_em):
    xe, ye = ellipse_confianza(mus_0[k], Sigs_0[k], escala=2.0)
    trazas_init.append(go.Scatter(
        x=xe, y=ye, mode='lines',
        line=dict(color=colores_anim[k], width=2.5),
        name='Componente {}'.format(k + 1)
    ))
    trazas_init.append(go.Scatter(
        x=[mus_0[k][0]], y=[mus_0[k][1]],
        mode='markers',
        marker=dict(color=colores_anim[k], size=10, symbol='x'),
        name='Media {}'.format(k + 1)
    ))

# Slider
pasos_slider = []
for it in iters_guardar:
    pasos_slider.append(dict(
        args=[[str(it)],
              dict(frame=dict(duration=600, redraw=True), mode='immediate',
                           transition=dict(duration=300))],
        label=str(it),
        method='animate'
    ))

fig_anim = go.Figure(
    data=trazas_init,
    frames=frames,
    layout=go.Layout(
        title='Convergencia del algoritmo EM (K=2)',
        xaxis=dict(title='x ', range=[-7, 7]),
        yaxis=dict(title='x ', range=[-5, 5]),
        height=520, width=750,
        updatemenus=[dict(
            type='buttons',
            buttons=[
                dict(label='Reproducir',
                     method='animate',
                     args=[None, dict(frame=dict(duration=600, redraw=True),
                                              fromcurrent=True,
                                              transition=dict(duration=300))]),
                dict(label='Pausar',

```

```

        method='animate',
        args=[[None], dict(frame=dict(duration=0, redraw=False),
                             mode='immediate',
                             transition=dict(duration=0))]]
    ],
    x=0.0, y=1.12, xanchor='left', yanchor='top'
)],
sliders=[dict(
    active=0,
    steps=pasos_slider,
    x=0.0, y=0.0,
    xanchor='left', yanchor='top',
    len=1.0,
    currentvalue=dict(prefix='Iteracion: ', visible=True, xanchor='center'),
    transition=dict(duration=300)
)]
)
)
fig_anim.show()

```

Unable to display output for mime type(s): text/html

Figura 144.3.: Animación del algoritmo EM para un GMM con $K=2$ en 2D. Cada fotograma muestra el estado del modelo en una iteración específica: los puntos están coloreados por la responsabilidad de la primera componente (azul = comp. 1, rojo = comp. 2), y las elipses representan los contornos de confianza al 95 % de cada gaussiana.

144.3.6. Curva de log-verosimilitud

La figura Figura 144.4 confirma la propiedad de convergencia monótona del EM: la log-verosimilitud crece de manera no decreciente en cada iteración hasta alcanzar una meseta, señal de convergencia al óptimo local.

```

import plotly.graph_objects as go
import numpy as np

iters_ll = [t for t, _ in ll_historia]
vals_ll = [v for _, v in ll_historia]

fig_ll = go.Figure()
fig_ll.add_trace(go.Scatter(
    x=iters_ll, y=vals_ll,
    mode='lines+markers',
    line=dict(color='#2ca02c', width=2.5),
    marker=dict(size=7, color='#2ca02c'),
    name='Log-verosimilitud'
))

```

```
fig_ll.update_layout(
    title='Convergencia del EM: log-verosimilitud vs. iteracion',
    xaxis_title='Iteracion',
    yaxis_title='Log-verosimilitud ( )',
    height=380, width=650
)
fig_ll.show()
```

Unable to display output for mime type(s): text/html

Figura 144.4.: Log-verosimilitud en función del número de iteraciones del algoritmo EM. La curva es monótonamente no decreciente, lo que garantiza que el algoritmo no empeora el ajuste en ninguna iteración.

144.4. Tipos de covarianza

La flexibilidad de un GMM depende en gran medida de la estructura impuesta a las matrices de covarianza Σ_k . Restringir la forma de Σ_k reduce el número de parámetros a estimar, lo que puede mejorar la generalización cuando los datos son escasos, pero a costa de capacidad expresiva (Murphy 2012; Hastie et al. 2009). La siguiente tabla resume las cuatro estructuras más comunes en la práctica:

Tipo	Descripción	Parámetros por componente	Forma del contorno
Esférica (<i>spherical</i>)	$\Sigma_k = \sigma_k^2 \mathbf{I}$	1	Círculos
Diagonal (<i>diagonal</i>)	$\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2)$	D	Elipses alineadas con los ejes
Completa (<i>full</i>)	Σ_k simétrica definida positiva	$D(D+1)/2$	Elipses rotadas arbitrariamente
Atada (<i>tied</i>)	$\Sigma_k = \Sigma$ (igual para todo k)	$D(D+1)/2$ en total	Elipses con la misma forma

Para un GMM con $D = 2$ y $K = 3$: la covarianza esférica usa solo 3 parámetros de covarianza, la diagonal usa 6, la completa usa 9, y la atada usa 3 (compartidos). La figura Figura 144.5 ilustra el efecto visual de cada estructura.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng4 = np.random.default_rng(55)

# Centros comunes para los 3 componentes
mus_tipos = [np.array([-2.5, 1.0]),
```

```

        np.array([ 2.5, 1.0]),
        np.array([ 0.0, -2.5]))

# Covarianza "full" de referencia para cada componente
Sigs_full = [np.array([[1.5, 0.8], [0.8, 0.6]]),
              np.array([[0.6, -0.5], [-0.5, 1.2]]),
              np.array([[0.9, 0.0], [0.0, 0.4]])]

def make_spherical(Sig):
    sigma2 = np.trace(Sig) / Sig.shape[0]
    return sigma2 * np.eye(Sig.shape[0])

def make_diagonal(Sig):
    return np.diag(np.diag(Sig))

def make_tied(Sigs, pis):
    S = np.zeros_like(Sigs[0])
    for pi, Sg in zip(pis, Sigs):
        S += pi * Sg
    return S

pis_tipos = [1/3, 1/3, 1/3]
Sigma_tied = make_tied(Sigs_full, pis_tipos)

tipos = {
    'Esférica': [make_spherical(S) for S in Sigs_full],
    'Diagonal': [make_diagonal(S) for S in Sigs_full],
    'Completa': Sigs_full,
    'Atada':     [Sigma_tied for _ in range(3)]
}

colores_tc = ['#e41a1c', '#377eb8', '#4daf4a']

fig_tc = make_subplots(rows=2, cols=2,
                       subplot_titles=list(tipos.keys()))

def ellipse_comp(mu, Sigma, escala=2.0, n_pts=100):
    vals, vecs = np.linalg.eigh(Sigma)
    t = np.linspace(0, 2 * np.pi, n_pts)
    v1 = vecs[:, 0] * np.sqrt(np.maximum(vals[0], 1e-9)) * escala
    v2 = vecs[:, 1] * np.sqrt(np.maximum(vals[1], 1e-9)) * escala
    xe = mu[0] + v1[0] * np.cos(t) + v2[0] * np.sin(t)
    ye = mu[1] + v1[1] * np.cos(t) + v2[1] * np.sin(t)
    return xe, ye

posiciones = [(1,1), (1,2), (2,1), (2,2)]
for idx, (tipo_nombre, Sigs_tipo) in enumerate(tipos.items()):
    fila, col = posiciones[idx]
    for k in range(3):

```



```

xe, ye = ellipse_comp(mus_tipos[k], Sigs_tipo[k])
mostrar_leyenda = (idx == 0)
fig_tc.add_trace(go.Scatter(
    x=xe, y=ye,
    mode='lines',
    line=dict(color=colores_tc[k], width=2.2),
    name='Componente {}'.format(k+1),
    showlegend=mostrar_leyenda
), row=fila, col=col)
# Centro
fig_tc.add_trace(go.Scatter(
    x=[mus_tipos[k][0]], y=[mus_tipos[k][1]],
    mode='markers',
    marker=dict(color=colores_tc[k], size=8, symbol='cross'),
    showlegend=False
), row=fila, col=col)

for fila in [1, 2]:
    for col in [1, 2]:
        fig_tc.update_xaxes(range=[-5.5, 5.5], row=fila, col=col)
        fig_tc.update_yaxes(range=[-5.0, 4.0], row=fila, col=col)

fig_tc.update_layout(height=620, width=820,
                      title='Tipos de covarianza en GMM (K=3, D=2)',
                      legend=dict(x=0.78, y=0.98))
fig_tc.show()

```

Unable to display output for mime type(s): text/html

Figura 144.5.: Los cuatro tipos de covarianza para un GMM con $K=3$ componentes en 2D. Cada panel muestra las elipses de confianza al 95 % correspondientes a cada estructura de covarianza.

144.5. Selección del número de componentes

144.5.1. El problema del sobreajuste

Una limitación fundamental del GMM es que el número de componentes K debe especificarse de antemano. Aumentar K siempre incrementa la log-verosimilitud en los datos de entrenamiento (mayor flexibilidad), pero puede llevar a sobreajuste: el modelo captura ruido particular de la muestra en lugar de la estructura real de la distribución subyacente (Bishop 2006; Reynolds 2009). Un caso extremo es que con $K = N$, cada componente puede colapsar sobre un solo punto, llevando la log-verosimilitud al infinito pero sin poder generalizar.

144.5.2. Criterios de información: AIC y BIC

Los criterios de información penalizan la log-verosimilitud por el número de parámetros libres p , buscando un equilibrio entre ajuste y parsimonia:

$$\text{AIC} = -2\ell(\hat{\theta}) + 2p \quad (144.10)$$

$$\text{BIC} = -2\ell(\hat{\theta}) + p \log N \quad (144.11)$$

Para un GMM con covarianza completa en D dimensiones y K componentes, el número de parámetros libres es:

$$p = \underbrace{K \cdot D}_{\text{medias}} + \underbrace{K \cdot \frac{D(D+1)}{2}}_{\text{covarianzas}} + \underbrace{(K-1)}_{\text{mezclas}} \quad (144.12)$$

El BIC penaliza más fuertemente que el AIC cuando $N > e^2 \approx 7$, tendiendo a seleccionar modelos más parsimoniosos. Se elige el K que minimiza el criterio. La figura Figura 144.6 muestra la curva de AIC y BIC para $K = 1, \dots, 8$.

```
import numpy as np
import plotly.graph_objects as go

rng5 = np.random.default_rng(77)

# Dataset con 3 grupos verdaderos en 2D
n_sel = 300
X_sel = np.vstack([
    rng5.multivariate_normal([-4, 0], [[0.8, 0.2], [0.2, 0.6]], 100),
    rng5.multivariate_normal([ 4, 0], [[0.7, -0.1], [-0.1, 0.9]], 100),
    rng5.multivariate_normal([ 0, 4], [[0.5, 0.3], [0.3, 0.7]], 100)
])
N_sel, D_sel = X_sel.shape

def gauss_log_pdf_sel(X, mu, Sigma):
    D = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    inv_S = np.linalg.inv(Sigma)
    maha = np.einsum('ij,jk,ik->i', diff, inv_S, diff)
    return -0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet)

def e_step_sel(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_R = np.zeros((N, K))
    for k in range(K):
        log_R[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf_sel(X, mus[k], Sigmas[k])
```

```

log_R_max = log_R.max(axis=1, keepdims=True)
R = np.exp(log_R - log_R_max)
R /= R.sum(axis=1, keepdims=True)
return R

def m_step_sel(X, R):
    N, D = X.shape
    K = R.shape[1]
    Nk = R.sum(axis=0) + 1e-10
    pis_n = Nk / N
    mus_n = (R.T @ X) / Nk[:, None]
    Sigs_n = []
    for k in range(K):
        diff = X - mus_n[k]
        Sk = (R[:, k:k+1] * diff).T @ diff / Nk[k]
        Sk += 1e-5 * np.eye(D)
        Sigs_n.append(Sk)
    return pis_n, mus_n, Sigs_n

def ll_sel(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_lm = np.zeros((N, K))
    for k in range(K):
        log_lm[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf_sel(X, mus[k], Sigmas[k])
    lmax = log_lm.max(axis=1)
    return (lmax + np.log(np.exp(log_lm - lmax[:, None]).sum(axis=1) + 1e-300)).sum()

def fit_gmm(X, K, n_iter=60, rng_fit=None):
    N, D = X.shape
    if rng_fit is None:
        rng_fit = np.random.default_rng(0)
    idx = rng_fit.choice(N, K, replace=False)
    pis = np.ones(K) / K
    mus = [X[idx[k]].copy() for k in range(K)]
    Sigs = [np.eye(D) for _ in range(K)]
    R = e_step_sel(X, pis, mus, Sigs)
    for _ in range(n_iter):
        pis, mus, Sigs = m_step_sel(X, R)
        R = e_step_sel(X, pis, mus, Sigs)
    return ll_sel(X, pis, mus, Sigs)

Ks = list(range(1, 9))
aics, bics = [], []
rng_bic = np.random.default_rng(99)
for K in Ks:
    ll_val = fit_gmm(X_sel, K, n_iter=80, rng_fit=rng_bic)
    p_params = K * D_sel + K * D_sel * (D_sel + 1) // 2 + (K - 1)
    aics.append(-2 * ll_val + 2 * p_params)

```

```

    bics.append(-2 * ll_val + p_params * np.log(N_sel))

k_opt_bic = Ks[int(np.argmin(bics))]

fig_crit = go.Figure()
fig_crit.add_trace(go.Scatter(
    x=Ks, y=aics, mode='lines+markers',
    name='AIC', line=dict(color='#1f77b4', width=2),
    marker=dict(size=8)
))
fig_crit.add_trace(go.Scatter(
    x=Ks, y=bics, mode='lines+markers',
    name='BIC', line=dict(color='#d62728', width=2),
    marker=dict(size=8)
))
fig_crit.add_vline(x=k_opt_bic, line_dash='dash', line_color='gray',
    annotation_text='K={} (min BIC)'.format(k_opt_bic),
    annotation_position='top right')
fig_crit.update_layout(
    title='Selección del número de componentes: AIC y BIC',
    xaxis_title='Número de componentes K',
    yaxis_title='Criterio de información',
    height=400, width=650,
    legend=dict(x=0.78, y=0.98)
)
fig_crit.show()

```

Unable to display output for mime type(s): text/html

Figura 144.6.: Criterios AIC y BIC en función del número de componentes K. El mínimo de cada curva indica el número óptimo de componentes según cada criterio. Una línea vertical punteada marca el mínimo del BIC.

144.6. GMM como clasificador vs. K-Means

144.6.1. K-Means como caso límite del GMM

El algoritmo K-Means puede derivarse formalmente como un caso límite del GMM bajo las siguientes restricciones: (1) coeficientes de mezcla iguales $\pi_k = 1/K$ para todo k , (2) covarianzas esféricas e iguales $\Sigma_k = \sigma^2 \mathbf{I}$ para todo k , y (3) se toma el límite $\sigma^2 \rightarrow 0$ (Bishop 2006; Hastie et al. 2009). En este límite, las responsabilidades r_{ik} colapsan a asignaciones duras: $r_{ik} \rightarrow 1$ para el componente más cercano y $r_{ik} \rightarrow 0$ para todos los demás. La actualización de medias del paso M del GMM se convierte exactamente en la actualización de centroides de K-Means.

La diferencia fundamental es que el GMM produce **asignaciones blandas** (probabilísticas) mediante las responsabilidades r_{ik} , mientras que K-Means produce **asignaciones duras** mediante la regla del vecino

más cercano. Para clasificación, la regla de decisión del GMM asigna $\mathbf{x}$ a la clase $k^* = \arg \max_k r_{ik}$, pero también cuantifica la incertidumbre mediante la distribución completa $(r_{i1}, \dots, r_{iK})$.

La figura Figura 144.7 ilustra la diferencia entre las asignaciones duras de K-Means y las asignaciones blandas del GMM sobre el mismo conjunto de datos.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng6 = np.random.default_rng(31)

# Datos de tres grupos solapantes
X_clas = np.vstack([
    rng6.multivariate_normal([-3, 1.5], [[1.0, 0.4], [0.4, 0.7]], 150),
    rng6.multivariate_normal([ 3, 1.5], [[0.8, -0.3], [-0.3, 1.1]], 150),
    rng6.multivariate_normal([ 0, -2.5], [[1.2, 0.1], [0.1, 0.6]], 150)
])
N_clas, D_clas = X_clas.shape

# ---- K-Means simplificado ----
def kmeans(X, K, n_iter=100, rng_km=None):
    if rng_km is None:
        rng_km = np.random.default_rng(0)
    centroides = X[rng_km.choice(len(X), K, replace=False)].copy()
    etiquetas = np.zeros(len(X), dtype=int)
    for _ in range(n_iter):
        dists = np.array([((X - centroides[k]) ** 2).sum(axis=1) for k in range(K)]).T
        etiquetas = dists.argmin(axis=1)
        for k in range(K):
            mask = etiquetas == k
            if mask.sum() > 0:
                centroides[k] = X[mask].mean(axis=0)
    return etiquetas, centroides

# ---- GMM EM ----
def gauss_log_pdf_c(X, mu, Sigma):
    D = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    inv_S = np.linalg.inv(Sigma)
    maha = np.einsum('ij,jk,ik->i', diff, inv_S, diff)
    return -0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet)

def e_step_c(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_R = np.zeros((N, K))
    for k in range(K):
        log_R[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf_c(X, mus[k], Sigmas[k])
```

```

    lmax = log_R.max(axis=1, keepdims=True)
    R = np.exp(log_R - lmax)
    R /= R.sum(axis=1, keepdims=True)
    return R

def m_step_c(X, R):
    N, D = X.shape
    K = R.shape[1]
    Nk = R.sum(axis=0) + 1e-10
    pis_n = Nk / N
    mus_n = (R.T @ X) / Nk[:, None]
    Sigs_n = []
    for k in range(K):
        diff = X - mus_n[k]
        Sk = (R[:, k:k+1] * diff).T @ diff / Nk[k]
        Sk += 1e-5 * np.eye(D)
        Sigs_n.append(Sk)
    return pis_n, mus_n, Sigs_n

K_clas = 3
# K-Means
etiq_km, cents_km = kmeans(X_clas, K_clas, rng_km=rng6)

# GMM
idx_gm = rng6.choice(N_clas, K_clas, replace=False)
pis_gm = np.ones(K_clas) / K_clas
mus_gm = [X_clas[idx_gm[k]].copy() for k in range(K_clas)]
Sigs_gm = [np.eye(D_clas) for _ in range(K_clas)]
R_gm = e_step_c(X_clas, pis_gm, mus_gm, Sigs_gm)
for _ in range(80):
    pis_gm, mus_gm, Sigs_gm = m_step_c(X_clas, R_gm)
    R_gm = e_step_c(X_clas, pis_gm, mus_gm, Sigs_gm)

# Colores base RGB para 3 componentes
colores_rgb_base = [(228, 26, 28), (55, 126, 184), (77, 175, 74)]

# Asignaciones duras K-Means
colores_km = ['rgb({}, {}, {})'.format(*colores_rgb_base[e]) for e in etiq_km]

# Colores blandos GMM: mezcla ponderada por responsabilidades
colores_gmm = []
for i in range(N_clas):
    r_vec = R_gm[i]
    rr = sum(r_vec[k] * colores_rgb_base[k][0] for k in range(K_clas))
    gg = sum(r_vec[k] * colores_rgb_base[k][1] for k in range(K_clas))
    bb = sum(r_vec[k] * colores_rgb_base[k][2] for k in range(K_clas))
    colores_gmm.append('rgb({}, {}, {})'.format(int(rr), int(gg), int(bb)))

fig_vs = make_subplots(rows=1, cols=2,

```

```

        subplot_titles=('K-Means (asignacion dura)',
                        'GMM (asignacion blanda)'))

fig_vs.add_trace(go.Scatter(
    x=X_clas[:, 0], y=X_clas[:, 1],
    mode='markers',
    marker=dict(color=colores_km, size=5, opacity=0.8),
    name='K-Means'
), row=1, col=1)
fig_vs.add_trace(go.Scatter(
    x=cents_km[:, 0], y=cents_km[:, 1],
    mode='markers',
    marker=dict(color='black', size=12, symbol='star'),
    name='Centroides'
), row=1, col=1)

fig_vs.add_trace(go.Scatter(
    x=X_clas[:, 0], y=X_clas[:, 1],
    mode='markers',
    marker=dict(color=colores_gmm, size=5, opacity=0.8),
    name='GMM'
), row=1, col=2)

fig_vs.update_xaxes(title_text='x ')
fig_vs.update_yaxes(title_text='x ', row=1, col=1)
fig_vs.update_layout(height=440, width=860,
                      title='K-Means vs. GMM: asignaciones duras vs. blandas',
                      showlegend=True)

fig_vs.show()

```

Unable to display output for mime type(s): text/html

Figura 144.7.: Comparacion entre K-Means (asignaciones duras) y GMM (asignaciones blandas) con $K=3$ en 2D. Panel izquierdo: cada punto toma el color discreto de su centroide más cercano. Panel derecho: el color de cada punto es una mezcla de los colores de los componentes, ponderada por las responsabilidades.

144.7. GMM como modelo generativo

Una ventaja clave del GMM sobre los métodos de clustering no probabilísticos es que define una **distribución de probabilidad completa** sobre el espacio de datos. Esto significa que, una vez ajustado el modelo, es posible generar nuevas observaciones que son estadísticamente indistinguibles de los datos originales, siguiendo el procedimiento ancestral del modelo generativo (Bishop 2006; Murphy 2012).

El procedimiento de muestreo es directo y se realiza en dos etapas:

1. **Muestrear la componente:** extraer $k \sim \text{Categorica}(\pi_1, \dots, \pi_K)$, es decir, seleccionar la componente k con probabilidad π_k .

2. **Muestrear la observación:** extraer $\mathbf{x} \sim \mathcal{N}(\mu_k, \Sigma_k)$ de la gaussiana correspondiente.

Este proceso genera muestras de la densidad marginal $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$, lo que confirma que el GMM es un modelo generativo válido. La figura Figura 144.8 compara los datos originales con muestras nuevas generadas del modelo ajustado.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng7 = np.random.default_rng(88)

# Dataset original con 3 grupos
pis_gen_true = np.array([0.40, 0.35, 0.25])
mus_gen_true = [np.array([-3.0, -2.0]),
                 np.array([ 3.5, 0.0]),
                 np.array([ 0.0, 3.5])]
Sigs_gen_true = [np.array([[1.0, 0.5], [0.5, 0.8]]),
                 np.array([[0.7, -0.3], [-0.3, 1.0]]),
                 np.array([[0.6, 0.2], [0.2, 0.6]])]

n_gen = 400
ns_gen = [int(p * n_gen) for p in pis_gen_true]
ns_gen[-1] = n_gen - sum(ns_gen[:-1])
X_gen = np.vstack([rng7.multivariate_normal(mus_gen_true[k], Sigs_gen_true[k], ns_gen[k])
                   for k in range(3)])
etiq_gen_true = np.concatenate([np.full(ns_gen[k], k) for k in range(3)])
N_gen, D_gen = X_gen.shape

# Ajustar GMM
def gauss_log_pdf_g(X, mu, Sigma):
    D = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    inv_S = np.linalg.inv(Sigma)
    maha = np.einsum('ij,jk,ik->i', diff, inv_S, diff)
    return -0.5 * maha - 0.5 * (D * np.log(2 * np.pi) + logdet)

def e_step_g(X, pis, mus, Sigmas):
    K = len(pis)
    N = X.shape[0]
    log_R = np.zeros((N, K))
    for k in range(K):
        log_R[:, k] = np.log(pis[k] + 1e-300) + gauss_log_pdf_g(X, mus[k], Sigmas[k])
    lmax = log_R.max(axis=1, keepdims=True)
    R = np.exp(log_R - lmax)
    R /= R.sum(axis=1, keepdims=True)
    return R

def m_step_g(X, R):
```



```

N, D = X.shape
K = R.shape[1]
Nk = R.sum(axis=0) + 1e-10
pis_n = Nk / N
mus_n = (R.T @ X) / Nk[:, None]
Sigs_n = []
for k in range(K):
    diff = X - mus_n[k]
    Sk = (R[:, k:k+1] * diff).T @ diff / Nk[k]
    Sk += 1e-6 * np.eye(D)
    Sigs_n.append(Sk)
return pis_n, mus_n, Sigs_n

K_gen = 3
idx_g0 = rng7.choice(N_gen, K_gen, replace=False)
pis_fit = np.ones(K_gen) / K_gen
mus_fit = [X_gen[idx_g0[k]].copy() for k in range(K_gen)]
Sigs_fit = [np.eye(D_gen) for _ in range(K_gen)]
R_fit = e_step_g(X_gen, pis_fit, mus_fit, Sigs_fit)
for _ in range(100):
    pis_fit, mus_fit, Sigs_fit = m_step_g(X_gen, R_fit)
    R_fit = e_step_g(X_gen, pis_fit, mus_fit, Sigs_fit)

# Muestrear del GMM ajustado
n_muestras = 500
ks_muestras = rng7.choice(K_gen, size=n_muestras, p=pis_fit)
X_muestras = np.vstack([
    rng7.multivariate_normal(mus_fit[k], Sigs_fit[k])
    for k in ks_muestras
])

# Densidad en grilla
x_gv = np.linspace(-7, 8, 120)
y_gv = np.linspace(-5.5, 7, 120)
XX_g, YY_g = np.meshgrid(x_gv, y_gv)
grid_g = np.column_stack([XX_g.ravel(), YY_g.ravel()])

Z_fit = np.zeros(grid_g.shape[0])
for k in range(K_gen):
    Z_fit += pis_fit[k] * np.exp(gauss_log_pdf_g(grid_g, mus_fit[k], Sigs_fit[k]))
Z_fit = Z_fit.reshape(XX_g.shape)

colores_gen = ['#e41a1c', '#377eb8', '#4daf4a']
colores_muestras = [colores_gen[k] for k in ks_muestras]
colores_orig = [colores_gen[k] for k in etiq_gen_true]

fig_gen = make_subplots(rows=1, cols=2,
                        subplot_titles=('Datos originales + contornos GMM',
                                       'Muestras generadas por el GMM'))

```

```

# Panel izquierdo: datos + contornos
fig_gen.add_trace(go.Contour(
    x=x_gv, y=y_gv, z=Z_fit,
    colorscale='Greys',
    contours=dict(coloring='lines', showlabels=False),
    showscale=False,
    opacity=0.7,
    name='Densidad ajustada'
), row=1, col=1)
fig_gen.add_trace(go.Scatter(
    x=X_gen[:, 0], y=X_gen[:, 1],
    mode='markers',
    marker=dict(color=colores_orig, size=4, opacity=0.6),
    name='Datos originales',
    showlegend=True
), row=1, col=1)

# Panel derecho: muestras generadas
fig_gen.add_trace(go.Scatter(
    x=X_muestras[:, 0], y=X_muestras[:, 1],
    mode='markers',
    marker=dict(color=colores_muestras, size=4, opacity=0.6),
    name='Muestras generadas',
    showlegend=True
), row=1, col=2)

for col in [1, 2]:
    fig_gen.update_xaxes(title_text='x ', range=[-7.5, 8.5], row=1, col=col)
    fig_gen.update_yaxes(title_text='y ', range=[-6, 7.5], row=1, col=col)

fig_gen.update_layout(height=440, width=860,
                       title='GMM como modelo generativo (K=3)',
                       legend=dict(x=0.75, y=0.98))
fig_gen.show()

```

144.8. Consideraciones prácticas

144.8.1. Inicialización

La calidad de la solución encontrada por el EM depende sensiblemente de la inicialización, ya que el algoritmo solo garantiza convergencia a un óptimo **local** de la log-verosimilitud. Existen dos estrategias principales:

- **Inicialización aleatoria:** seleccionar K observaciones al azar como medias iniciales y usar $\Sigma_k = \mathbf{I}$ y $\pi_k = 1/K$. Es simple pero puede conducir a mínimos locales pobres.
- **Arranque en caliente con K-Means:** ejecutar K-Means primero y usar los centroides resultantes como medias iniciales, con las covarianzas empíricas de cada cluster y proporciones iguales. Esta estrategia tiende a producir mejores puntos de partida y convergencia más rápida (Bishop 2006).

En la práctica se recomienda ejecutar el EM múltiples veces con diferentes inicializaciones y quedarse con la solución de mayor log-verosimilitud.

144.8.2. Singularidades y regularización

El GMM tiene un problema de **singularidades**: si una componente colapsa sobre un único punto de datos, su covarianza tiende a cero ($|\Sigma_k| \rightarrow 0$) y la densidad gaussiana en ese punto tiende al infinito, llevando la log-verosimilitud a $+\infty$. Este no es un máximo legítimo del modelo, sino una degeneración numérica. La solución estándar es añadir una regularización $\varepsilon \mathbf{I}$ a la covarianza estimada en cada paso M:

$$\hat{\Sigma}_k \leftarrow \hat{\Sigma}_k + \varepsilon \mathbf{I}$$

con $\varepsilon \approx 10^{-6}$ típicamente. Esto garantiza que todas las matrices de covarianza sean definidas positivas y evita la degeneración numérica.

144.8.3. Óptimos locales y múltiples reinicios

El EM es un algoritmo de ascenso de gradiente generalizado que no garantiza convergencia al óptimo global. La función de log-verosimilitud del GMM tiene en general múltiples óptimos locales, y el EM puede quedar atrapado en cualquiera de ellos dependiendo de la inicialización (McLachlan y Peel 2000). La estrategia estándar es ejecutar el algoritmo R veces (por ejemplo, $R = 10$) con diferentes semillas aleatorias y seleccionar el modelo con la mayor log-verosimilitud al converger.

144.8.4. Costo computacional

El costo por iteración del EM para un GMM con covarianza completa es $\mathcal{O}(K \cdot N \cdot D^2)$: para cada componente (K términos) y cada observación (N términos) se evalúa la densidad gaussiana, que requiere $\mathcal{O}(D^2)$ operaciones (producto cuadrático con la inversa de Σ_k). La actualización de Σ_k en el paso M también cuesta $\mathcal{O}(N \cdot D^2)$ por componente. Para aplicaciones con D muy grande o K grande, las covarianzas diagonales o esféricas reducen el costo a $\mathcal{O}(K \cdot N \cdot D)$.

144.8.5. Variational Bayes GMM para selección automática de K

Una extensión bayesiana natural del GMM es el **Variational Bayes GMM** (VB-GMM), que coloca distribuciones a priori sobre todos los parámetros (π_k, μ_k, Σ_k) y realiza inferencia variacional en lugar de estimación puntual (Bishop 2006). Una ventaja clave del VB-GMM es que puede determinar automáticamente el número efectivo de componentes: componentes superfluas reciben pesos $\pi_k \rightarrow 0$ durante la inferencia, evitando la necesidad de ejecutar el modelo para múltiples valores de K y comparar con AIC/BIC. Sin embargo, la implementación es considerablemente más compleja y queda fuera del alcance de este capítulo.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.

- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.
- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining, Lecture Notes en Computer Science*, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics

Publications.

- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.
- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deepmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.

- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan

- Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology

- derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.
- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.
- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.

- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.

- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.
- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.

- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Parte XXVIII.

Redes Neuronales

145. Redes Neuronales para Clasificacion

Las redes neuronales artificiales son modelos computacionales inspirados en la estructura del sistema nervioso biologico. Desde el perceptron de Rosenblatt en 1958 (Rosenblatt 1958) hasta las arquitecturas profundas que dominan el estado del arte actual (LeCun et al. 2015), el esqueleto matematico fundamental permanece invariante: una composicion de transformaciones lineales seguidas de no linealidades, aplicadas de manera jerarquica sobre los datos de entrada. Esta arquitectura modular permite que el modelo aprenda representaciones cada vez mas abstractas de los datos, pasando de caracteristicas de bajo nivel (como bordes en una imagen) hasta conceptos de alto nivel (como la clase de un objeto).

El interes renovado por las redes neuronales en la ultima decada no es accidental. Se sustenta en tres pilares convergentes: la disponibilidad de grandes volumenes de datos etiquetados, la capacidad de computo paralelo ofrecida por las unidades de procesamiento grafico (GPU), y avances teoricos y practicos en el entrenamiento de redes profundas. El teorema de aproximacion universal (Hornik et al. 1989) formaliza por que estas arquitecturas son tan poderosas: una red de una sola capa oculta con suficientes unidades puede aproximar cualquier funcion continua sobre un conjunto compacto con precision arbitraria. Este capitulo desarrolla la teoria desde el perceptron hasta las redes convolucionales, construyendo cada componente desde primeros principios con implementaciones en Python puro y visualizaciones interactivas.

145.1. El Perceptron

145.1.1. Modelo matematico

El perceptron es la unidad de computo mas elemental de las redes neuronales. Recibe d entradas escalares $x_1, \dots, x_d$ y computa una combinacion lineal ponderada seguida de una funcion de activacion binaria. Formalmente, dados un vector de pesos $\mathbf{w} \in \mathbb{R}^d$ y un sesgo $b \in \mathbb{R}$, el perceptron calcula:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b \quad (145.1)$$

$$\hat{y} = \text{sign}(z) \in \{-1, +1\} \quad (145.2)$$

La interpretacion geometrica es inmediata: el conjunto $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ define un hiperplano en $\mathbb{R}^d$ que divide el espacio en dos semiespacios. Los puntos con $\mathbf{w}^\top \mathbf{x} + b > 0$ se clasifican como clase $+1$ y los puntos con $\mathbf{w}^\top \mathbf{x} + b < 0$ como clase -1 . El vector $\mathbf{w}$ es el vector normal al hiperplano, y su magnitud $\|\mathbf{w}\|$ determina la escala del margen de decision. El sesgo b desplaza el hiperplano respecto al origen, permitiendo que la frontera de decision no pase necesariamente por el punto cero.

Esta representacion es exactamente la misma que la de las Maquinas de Soporte Vectorial, con la diferencia fundamental de que el perceptron no busca el hiperplano de margen maximo: simplemente busca cualquier hiperplano que clasifique correctamente todos los ejemplos de entrenamiento. Esta distincion es crucial y explica por que el perceptron puede encontrar diferentes soluciones dependiendo del orden en que se presenten los datos, mientras que la SVM produce una solucion unica y optima en el sentido del margen.

145.1.2. Regla de aprendizaje del perceptron

El algoritmo de entrenamiento del perceptron es sorprendentemente simple. Se procesan los ejemplos de entrenamiento de forma iterativa y, cuando el modelo comete un error de clasificación en el ejemplo $(\mathbf{x}^{(i)}, y^{(i)})$, se actualiza el vector de pesos en la dirección que corrige ese error:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y^{(i)} \mathbf{x}^{(i)}, \quad b \leftarrow b + \eta y^{(i)} \quad (145.3)$$

donde $\eta > 0$ es la tasa de aprendizaje. La lógica de esta regla es directa: si se predijo $\hat{y} = -1$ cuando el verdadero valor es $y^{(i)} = +1$ (falso negativo), entonces $\mathbf{w}$ se desplaza en la dirección de $\mathbf{x}^{(i)}$, lo que aumenta $\mathbf{w}^\top \mathbf{x}^{(i)}$ y acerca la predicción al valor correcto. Si se predijo $\hat{y} = +1$ cuando el verdadero es $y^{(i)} = -1$ (falso positivo), el desplazamiento es en la dirección opuesta a $\mathbf{x}^{(i)}$.

El teorema de convergencia del perceptron establece que si los datos son linealmente separables con margen geométrico $\gamma > 0$ (es decir, existe un hiperplano que clasifica todos los puntos correctamente con distancia mínima γ a cualquier punto), entonces el algoritmo converge en a lo sumo R^2/γ^2 pasos, donde $R = \max_i \|\mathbf{x}^{(i)}\|$ es el radio de los datos (Rosenblatt 1958). Si los datos no son linealmente separables, el algoritmo no converge: oscila indefinidamente. Este es el límite fundamental del perceptron simple que motivó el desarrollo de las redes multicapa.

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(7)

# Generate linearly separable 2D data
n_per_class = 20
X_neg = rng.multivariate_normal([-2.0, -1.5], [[0.6, 0.1], [0.1, 0.6]], n_per_class)
X_pos = rng.multivariate_normal([2.0, 1.5], [[0.6, 0.1], [0.1, 0.6]], n_per_class)
X = np.vstack([X_neg, X_pos])
y = np.array([-1] * n_per_class + [1] * n_per_class)

# Perceptron training from scratch - record history
w = np.zeros(2)
b = 0.0
eta = 0.3
max_iters = 200
history = [(w.copy(), b, 0)]

converged = False
iteration = 0
while not converged and iteration < max_iters:
    errors = 0
    for i in range(len(y)):
        z = np.dot(w, X[i]) + b
        y_hat = 1 if z >= 0 else -1
        if y_hat != y[i]:
            w = w + eta * y[i] * X[i]
            b = b + eta * y[i]
            errors += 1
    if errors == 0:
        converged = True
    iteration += 1
```

```

iteration += 1
history.append((w.copy(), b, errors))
if errors == 0:
    converged = True

# Build animated figure
x_range = np.array([-5.0, 5.0])

frames = []
for t, (wt, bt, errs) in enumerate(history):
    scatter_neg = go.Scatter(
        x=X_neg[:, 0], y=X_neg[:, 1],
        mode="markers",
        marker=dict(color="#1f77b4", size=8, symbol="circle"),
        name="Clase -1",
        showlegend=(t == 0)
    )
    scatter_pos = go.Scatter(
        x=X_pos[:, 0], y=X_pos[:, 1],
        mode="markers",
        marker=dict(color="#ff7f0e", size=8, symbol="diamond"),
        name="Clase +1",
        showlegend=(t == 0)
    )
    # Decision boundary:  $w[0]*x + w[1]*y + b = 0 \Rightarrow y = -(w[0]*x + b) / w[1]$ 
    if abs(wt[1]) > 1e-10:
        y_line = -(wt[0] * x_range + bt) / wt[1]
        line_trace = go.Scatter(
            x=x_range, y=y_line,
            mode="lines",
            line=dict(color="#2ca02c", width=2.5),
            name="Frontera",
            showlegend=(t == 0)
        )
    else:
        line_trace = go.Scatter(x=[], y=[], mode="lines", showlegend=False)

    frames.append(go.Frame(
        data=[scatter_neg, scatter_pos, line_trace],
        name=str(t),
        layout=go.Layout(title_text=f"Iteracion {t} | Errores: {errs}")
    ))

# Initial state
w0, b0, e0 = history[0]
init_neg = go.Scatter(x=X_neg[:, 0], y=X_neg[:, 1], mode="markers",
                      marker=dict(color="#1f77b4", size=8, symbol="circle"), name="Clase -1")
init_pos = go.Scatter(x=X_pos[:, 0], y=X_pos[:, 1], mode="markers",
                      marker=dict(color="#ff7f0e", size=8, symbol="diamond"), name="Clase +1")

```

```

init_line = go.Scatter(x=[], y=[], mode="lines",
                       line=dict(color="#2ca02c", width=2.5), name="Frontera")

steps = []
for t in range(len(history)):
    steps.append(dict(
        method="animate",
        args=[[str(t)], dict(mode="immediate", frame=dict(duration=300, redraw=True),
                              transition=dict(duration=0))],
        label=str(t)
    ))

sliders = [dict(
    active=0,
    steps=steps,
    x=0.0, y=0.0,
    len=1.0,
    currentvalue=dict(prefix="Iteracion: ", font=dict(size=13)),
    pad=dict(t=50)
)]

fig = go.Figure(
    data=[init_neg, init_pos, init_line],
    frames=frames,
    layout=go.Layout(
        title="Aprendizaje del Perceptron",
        xaxis=dict(range=[-5, 5], title="x1"),
        yaxis=dict(range=[-5, 5], title="x2"),
        sliders=sliders,
        updatemenus=[dict(
            type="buttons", showactive=False,
            y=1.15, x=0.5, xanchor="center",
            buttons=[
                dict(label="Reproducir", method="animate",
                    args=[None, dict(frame=dict(duration=400, redraw=True),
                                          fromcurrent=True, mode="immediate"))],
                dict(label="Pausar", method="animate",
                    args=[[None], dict(frame=dict(duration=0, redraw=False),
                                          mode="immediate"))])
            ]
        )],
        width=680, height=520,
        legend=dict(x=0.01, y=0.99)
    )
)
fig.show()

```

Unable to display output for mime type(s): text/html

- (a) Convergencia del algoritmo de entrenamiento del perceptron en un conjunto de datos linealmente separable. Cada fotograma muestra el estado actual de la frontera de decision y el numero de errores restantes. El deslizador permite navegar por la trayectoria de aprendizaje.

Unable to display output for mime type(s): text/html

(b)

Figura 145.1.

145.1.3. La limitacion del perceptron: el problema XOR

Aunque el perceptron es elegante en su simplicidad, tiene una limitacion fundamental: solo puede separar clases que son linealmente separables. En 1969, Minsky y Papert demostraron en su libro *Perceptrons* que la funcion logica XOR (o exclusivo) no puede ser implementada por un perceptron simple. Esta constatacion genero un periodo de pesimismo en la investigacion sobre redes neuronales conocido como el “invierno del IA”, hasta que el desarrollo del algoritmo de retropropagacion en los anos ochenta revivio el campo al demostrar que las redes multicapa podian aprender representaciones no lineales.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Logic gate data
X_logic = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y_and = np.array([-1, -1, -1, 1])
y_or = np.array([-1, 1, 1, 1])
y_xor = np.array([-1, 1, 1, -1])

colors_map = {-1: "#1f77b4", 1: "#ff7f0e"}
symbols_map = {-1: "circle", 1: "diamond"}

fig = make_subplots(rows=2, cols=2,
                    subplot_titles=["AND (separable)", "OR (separable)",
                                   "XOR (NO separable)", "XOR resuelto por MLP"])

def add_gate_scatter(fig, X, y, row, col, show_legend=False):
    for cls in [-1, 1]:
        mask = y == cls
        fig.add_trace(go.Scatter(
            x=X[mask, 0], y=X[mask, 1],
            mode="markers",
            marker=dict(color=colors_map[cls], size=14,
                      symbol=symbols_map[cls],
                      line=dict(width=1.5, color="black")),
            name=f"Clase {'+1' if cls==1 else '-1'}",
            showlegend=show_legend and row == 1 and col == 1,
            legendgroup=str(cls)))
```

```

    ), row=row, col=col)

add_gate_scatter(fig, X_logic, y_and, 1, 1, show_legend=True)
add_gate_scatter(fig, X_logic, y_or, 1, 2)
add_gate_scatter(fig, X_logic, y_xor, 2, 1)
add_gate_scatter(fig, X_logic, y_xor, 2, 2)

# AND separating line:  $x_1 + x_2 = 1.5$ 
xv = np.linspace(-0.3, 1.3, 50)
fig.add_trace(go.Scatter(x=xv, y=1.5 - xv, mode="lines",
                        line=dict(color="#2ca02c", width=2), showlegend=False), row=1, col=1)

# OR separating line:  $x_1 + x_2 = 0.5$ 
fig.add_trace(go.Scatter(x=xv, y=0.5 - xv, mode="lines",
                        line=dict(color="#2ca02c", width=2), showlegend=False), row=1, col=2)

# XOR annotation
fig.add_annotation(text="No separable linealmente", x=0.5, y=0.5,
                  xref="x3", yref="y3",
                  font=dict(size=12, color="red"),
                  showarrow=False, bgcolor="rgba(255,200,200,0.7)",
                  bordercolor="red", borderwidth=1)

# XOR solved with MLP: simple 2-layer from scratch
# Hand-tuned weights that solve XOR
#  $h_1 = \text{relu}(x_1 - x_2 - 0.5)$ ,  $h_2 = \text{relu}(-x_1 + x_2 - 0.5)$ ,  $\text{out} = \text{sigmoid}(h_1 + h_2 - 0.5)$ 
xx, yy = np.meshgrid(np.linspace(-0.3, 1.3, 60), np.linspace(-0.3, 1.3, 60))
grid = np.c_[xx.ravel(), yy.ravel()]
W1 = np.array([[1.0, -1.0], [-1.0, 1.0]])
b1 = np.array([-0.5, -0.5])
W2 = np.array([[1.0], [1.0]])
b2 = np.array([-0.5])
H = np.maximum(0, grid @ W1.T + b1)
out = 1.0 / (1.0 + np.exp(-(H @ W2 + b2).ravel()))
Z = out.reshape(xx.shape)

fig.add_trace(go.Contour(
    x=np.linspace(-0.3, 1.3, 60),
    y=np.linspace(-0.3, 1.3, 60),
    z=Z,
    colorscale=[[0, "rgba(31,119,180,0.25)"], [0.5, "rgba(255,255,255,0.1)"],
               [1, "rgba(255,127,14,0.25)"]],
    contours=dict(start=0.5, end=0.5, size=0, coloring="fill",
                  showlabels=True, labelfont=dict(size=11, color="white")),
    showscale=False, showlegend=False, line=dict(width=2, color="white")
), row=2, col=2)

for row, col in [(1, 1), (1, 2), (2, 1), (2, 2)]:
    fig.update_xaxes(range=[-0.3, 1.3], tickvals=[0, 1], row=row, col=col)

```

```
fig.update_yaxes(range=[-0.3, 1.3], tickvals=[0, 1], row=row, col=col)

fig.update_layout(height=540, width=700, title_text="Compuertas logicas y separabilidad lineal",
                  legend=dict(x=1.02, y=0.9))

fig.show()
```

Unable to display output for mime type(s): text/html

Figura 145.2.: Comparacion de problemas logicos para el perceptron. Las compuertas AND y OR son linealmente separables (arriba), mientras que XOR no lo es (abajo izquierda). Una red multicapa puede resolver XOR aprendiendo una frontera no lineal (abajo derecha), lo que justifica el uso de redes con capas ocultas.

145.2. Funciones de Activacion

Las funciones de activacion son el ingrediente esencial que dota a las redes neuronales de su capacidad expresiva. Sin ellas, cualquier composicion de capas lineales colapsaria a una unica transformacion lineal: si $f(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, esto es simplemente $(\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2$, una transformacion lineal con una sola capa equivalente. La no linealidad introducida por la funcion de activacion permite que las capas aprendan representaciones jerarquicas genuinamente no lineales.

La eleccion de la funcion de activacion afecta tanto la expresividad del modelo como la estabilidad del entrenamiento. Las funciones classicas como la sigmoide y la tangente hiperbolica fueron populares en las primeras decadas, pero sufren del problema del gradiente desvaneciente en redes profundas. Las funciones rectificadoras (ReLU y sus variantes) han dominado la practica moderna precisamente por mitigar este problema. Las cinco funciones principales se definen y comparan a continuacion.

La **sigmoide** $\sigma(z) = \frac{1}{1+e^{-z}}$ mapea cualquier valor real al intervalo $(0, 1)$ y se usa tipicamente en la capa de salida para clasificacion binaria. Su derivada es $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$, lo que significa que el gradiente nunca supera 0.25 en ninguna unidad. La **tangente hiperbolica** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ produce salidas en $(-1, 1)$ y tiene derivada $1 - \tanh^2(z) \leq 1$, con gradientes maximos en $z = 0$. La **ReLU** $\text{ReLU}(z) = \max(0, z)$ tiene derivada 1 para $z > 0$ y 0 para $z < 0$, lo que elimina el problema de gradientes exponencialmente pequenos en la parte positiva. La **Leaky ReLU** $\text{LReLU}(z) = \max(\alpha z, z)$ con $\alpha = 0.1$ resuelve el problema de las neuronas muertas (unidades que quedan atascadas con $z < 0$ y nunca se actualizan). La **Softplus** $\text{softplus}(z) = \ln(1 + e^z)$ es una aproximacion suave y diferenciable en todas partes de la ReLU.

El **problema del gradiente desvaneciente** ocurre cuando se apilan muchas capas con activaciones cuya derivada es menor que 1 en valor absoluto. Si la red tiene L capas y cada una tiene derivada de activacion $\delta < 1$, el gradiente que llega a las capas iniciales escala como δ^L , que decrece exponencialmente con la profundidad. Para sigmoid, $\delta \leq 0.25$, por lo que en una red de 10 capas el gradiente puede reducirse en un factor de $0.25^{10} \approx 10^{-6}$. La ReLU, al tener derivada 1 para entradas positivas, elimina este factor multiplicativo en las unidades activas.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```

z = np.linspace(-4, 4, 300)

# Define activations and derivatives (pure numpy, no scipy)
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def tanh_fn(z):
    return np.tanh(z)

def relu(z):
    return np.maximum(0, z)

def leaky_relu(z, alpha=0.1):
    return np.where(z >= 0, z, alpha * z)

def softplus(z):
    return np.log1p(np.exp(np.clip(z, -500, 20)))

def sigmoid_grad(z):
    s = sigmoid(z)
    return s * (1 - s)

def tanh_grad(z):
    return 1 - np.tanh(z)**2

def relu_grad(z):
    return (z > 0).astype(float)

def leaky_relu_grad(z, alpha=0.1):
    return np.where(z >= 0, 1.0, alpha)

def softplus_grad(z):
    return sigmoid(z)

names = ["Sigmoid", "Tanh", "ReLU", "Leaky ReLU", "Softplus"]
funcs = [sigmoid, tanh_fn, relu, leaky_relu, softplus]
grads = [sigmoid_grad, tanh_grad, relu_grad, leaky_relu_grad, softplus_grad]
colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728", "#9467bd"]

fig = make_subplots(rows=2, cols=5,
                    subplot_titles=[n for n in names] + [f"d/dz {n}" for n in names],
                    vertical_spacing=0.18, horizontal_spacing=0.08)

for col, (name, fn, gn, color) in enumerate(zip(names, funcs, grads, colors), start=1):
    # Function row
    fig.add_trace(go.Scatter(
        x=z, y=fn(z), mode="lines",
        line=dict(color=color, width=2.5),
        name=name, showlegend=False
    ))

```



```

), row=1, col=col)
fig.add_hline(y=0, line=dict(color="gray", width=0.8, dash="dot"), row=1, col=col)
fig.add_vline(x=0, line=dict(color="gray", width=0.8, dash="dot"), row=1, col=col)

# Derivative row
fig.add_trace(go.Scatter(
    x=z, y=gn(z), mode="lines",
    line=dict(color="gray", width=2.5, dash="dash"),
    name=f"d {name}", showlegend=False
), row=2, col=col)
fig.add_hline(y=0, line=dict(color="gray", width=0.8, dash="dot"), row=2, col=col)
fig.add_vline(x=0, line=dict(color="gray", width=0.8, dash="dot"), row=2, col=col)

fig.update_layout(height=480, width=950,
                    title_text="Funciones de activacion y sus derivadas")
fig.update_xaxes(title_text="z", range=[-4, 4])
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.3.: Comparacion de cinco funciones de activacion (fila superior) y sus derivadas (fila inferior) en el rango $z \in [-4, 4]$. Las funciones acotadas como sigmoid y tanh tienen derivadas menores a 1, lo que contribuye al gradiente desvaneciente. ReLU y sus variantes mantienen gradientes unitarios para entradas positivas.

145.3. Red Neuronal Multicapa (MLP)

145.3.1. Arquitectura

Una red neuronal multicapa (*Multilayer Perceptron*, MLP) es una composicion de L capas de transformaciones. Cada capa $\ell \in \{1, \dots, L\}$ tiene n_ℓ unidades (neuronas). La capa 0 corresponde a la entrada con $n_0 = d$ unidades. Los parametros de la capa ℓ son la matriz de pesos $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ y el vector de sesgos $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$. La capa L es la capa de salida y las capas $1, \dots, L-1$ son capas ocultas. El numero total de parametros es $\sum_{\ell=1}^L (n_{\ell-1} \cdot n_\ell + n_\ell)$, que puede escalar rapidamente con la profundidad y el ancho de la red.

La eleccion de la arquitectura (numero de capas y unidades por capa) es uno de los hiperparametros mas importantes del diseño de una red. Redes mas profundas pueden aprender representaciones mas complejas, pero son mas dificiles de entrenar y mas propensas al sobreajuste con conjuntos de datos pequenos. Redes mas anchas tienen mayor capacidad memoristica pero son menos eficientes en terminos de parametros por unidad de complejidad. En la practica, la arquitectura se elige mediante validacion cruzada o mediante heuristica basada en el dominio del problema.

145.3.2. Propagacion hacia adelante

Dada una matriz de datos $\mathbf{X} \in \mathbb{R}^{m \times n_0}$ con m ejemplos, la propagacion hacia adelante (*forward pass*) calcula la prediccion de la red aplicando las transformaciones de cada capa en orden. Se define $\mathbf{A}^{(0)} = \mathbf{X}$ y

para cada capa $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{A}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad (145.4)$$

$$\mathbf{A}^{(\ell)} = g^{(\ell)}(\mathbf{z}^{(\ell)}) \quad (145.5)$$

donde $g^{(\ell)}$ es la función de activación de la capa ℓ . Para las capas ocultas se usa típicamente ReLU o tanh; para la capa de salida en clasificación binaria se usa sigmoide σ (que produce probabilidades en $(0, 1)$) y para clasificación multiclase se usa softmax $\text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$. La predicción final es $\hat{\mathbf{Y}} = \mathbf{A}^{(L)}$.

145.3.3. Teorema de aproximación universal

El resultado fundamental que justifica el poder de las redes neuronales fue demostrado independientemente por Cybenko (1989) para activaciones sigmoideas (Cybenko 1989) y por Hornik, Stinchcombe y White para una clase más amplia de activaciones no polinomiales (Hornik et al. 1989):

Teorema (Aproximación Universal). Sea g una función de activación continua y no polinomial. Para cualquier función continua $f : [0, 1]^d \rightarrow \mathbb{R}$ y cualquier $\varepsilon > 0$, existe una red neuronal de una sola capa oculta con suficientes unidades que aproxima f con error máximo ε en todo su dominio.

Este teorema es de existencia, no constructivo: garantiza que la red *puede* aproximar cualquier función, pero no dice cómo encontrar los pesos correctos. Tampoco dice cuántas unidades se necesitan (que puede ser exponencialmente grande), ni garantiza que el algoritmo de entrenamiento las encuentre. La profundidad importa precisamente porque redes profundas pueden representar ciertas funciones con exponencialmente menos unidades que redes superficiales, una propiedad conocida como ventaja de profundidad.

```
import numpy as np
import plotly.graph_objects as go

layer_sizes = [4, 5, 4, 3]
layer_labels = ["Entrada (4)", "Oculta 1 (5)", "Oculta 2 (4)", "Salida (3)"]
layer_colors = ["#1f77b4", "#2ca02c", "#2ca02c", "#ff7f0e"]
n_layers = len(layer_sizes)

# Compute neuron positions
x_positions = np.linspace(0.1, 0.9, n_layers)
neuron_coords = []
for l, n in enumerate(layer_sizes):
    ys = np.linspace(0.1, 0.9, n)
    neuron_coords.append(list(zip([x_positions[l]] * n, ys)))

fig = go.Figure()

# Draw connections (lines between layers)
for l in range(n_layers - 1):
    for (x0, y0) in neuron_coords[l]:
        for (x1, y1) in neuron_coords[l + 1]:
            fig.add_trace(go.Scatter(
```

```

        x=[x0, x1], y=[y0, y1],
        mode="lines",
        line=dict(color="rgba(150,150,150,0.25)", width=0.8),
        showlegend=False, hoverinfo="skip"
    ))

# Draw neurons
for l, (coords, color, label) in enumerate(zip(neuron_coords, layer_colors, layer_labels)):
    xs = [c[0] for c in coords]
    ys = [c[1] for c in coords]
    fig.add_trace(go.Scatter(
        x=xs, y=ys,
        mode="markers",
        marker=dict(color=color, size=28, line=dict(width=2, color="white"),
                    symbol="circle"),
        name=label,
        hovertemplate=f"{label}<extra></extra>"
    ))

# Weight matrix dimension annotations between layers
weight_dims = [
    ("W1: 4x5", 0.31, 0.96),
    ("W2: 5x4", 0.53, 0.96),
    ("W3: 4x3", 0.73, 0.96),
]
for text, xpos, ypos in weight_dims:
    fig.add_annotation(
        x=xpos, y=ypos, text=f"<b>{text}</b>",
        xref="paper", yref="paper",
        showarrow=False,
        font=dict(size=12, color="#555555"),
        bgcolor="rgba(240,240,240,0.85)",
        bordercolor="#aaaaaa", borderwidth=1
    )

# Layer labels at bottom
for l, (xp, label) in enumerate(zip(x_positions, layer_labels)):
    fig.add_annotation(
        x=xp, y=-0.08, text=f"<b>{label}</b>",
        xref="paper", yref="paper",
        showarrow=False,
        font=dict(size=12, color=layer_colors[l])
    )

fig.update_layout(
    title="Arquitectura MLP: 4 - 5 - 4 - 3",
    xaxis=dict(visible=False, range=[-0.05, 1.05]),
    yaxis=dict(visible=False, range=[-0.15, 1.1]),
    height=480, width=680,

```

```

showlegend=True,
legend=dict(x=1.02, y=0.5),
plot_bgcolor="white"
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.4.: Diagrama de una red neuronal multicapa con arquitectura 4-5-4-3 (capa de entrada con 4 unidades, dos capas ocultas con 5 y 4 unidades, y capa de salida con 3 unidades). Las conexiones entre capas representan los pesos aprendibles. Las etiquetas muestran las dimensiones de las matrices de pesos correspondientes.

145.4. Retropropagación del Error

145.4.1. Función de pérdida

El entrenamiento de la red se formula como la minimización de una función de pérdida $\mathcal{L}$ que mide la discrepancia entre las predicciones del modelo y los valores reales. Para clasificación binaria, la pérdida de entropía cruzada (cross-entropy) es la elección estándar, derivada del principio de máxima verosimilitud bajo el modelo de Bernoulli:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \quad (145.6)$$

donde $\hat{y}^{(i)} = \sigma(z_L^{(i)})$ es la probabilidad predicha para el ejemplo i . Esta pérdida es convexa como función de la capa de salida (aunque no convexa en los pesos de todas las capas), y tiene la propiedad de que su gradiente respecto a la entrada de la capa de salida es simplemente $\hat{y} - y$, lo que simplifica las derivaciones del gradiente.

Para minimizar $\mathcal{L}$, se usa descenso de gradiente, que requiere calcular $\nabla_{\mathbf{W}^{(\ell)}} \mathcal{L}$ para cada capa ℓ . El algoritmo de retropropagación (Rumelhart et al. 1986) computa estos gradientes de forma eficiente mediante la regla de la cadena, propagando el error desde la capa de salida hacia las capas iniciales. Su complejidad computacional es proporcional a la del paso hacia adelante, lo que lo hace tractable incluso para redes con millones de parámetros.

145.4.2. Regla de la cadena — derivación capa a capa

Definimos la señal de error en la capa ℓ como $\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$. Esta cantidad captura cuánto contribuye la activación pre-lineal de la capa ℓ a la pérdida total. La recurrencia de retropropagación se establece de la siguiente manera. Para la capa de salida L con sigmoide y pérdida cross-entropy:

$$\delta^{(L)} = \hat{\mathbf{Y}} - \mathbf{Y} \quad (145.7)$$

Para las capas ocultas $\ell < L$, aplicando la regla de la cadena en forma matricial:

$$\delta^{(\ell)} = \left(\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top} \right) \odot g'^{(\ell)}(\mathbf{z}^{(\ell)}) \quad (145.8)$$

donde $\odot$ denota el producto elemento a elemento (producto de Hadamard) y $g'^{(\ell)}$ es la derivada de la funcion de activacion de la capa ℓ . Los gradientes de los parametros son:

$$\nabla_{\mathbf{W}^{(\ell)}} \mathcal{L} = \frac{1}{m} \mathbf{A}^{(\ell-1)\top} \delta^{(\ell)}, \quad \nabla_{\mathbf{b}^{(\ell)}} \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \delta_i^{(\ell)} \quad (145.9)$$

Esta derivacion revela por que la profundidad es un desafio: el gradiente de la perdida respecto a los pesos de la primera capa es el producto de L matrices jacobianas $g'^{(\ell)}$. Si cada una tiene norma espectral menor que 1 (como ocurre con sigmoid o tanh en la mayor parte de su dominio), el gradiente decrece exponencialmente, impidiendo el aprendizaje en las capas profundas.

```
import numpy as np
import plotly.graph_objects as go

rng = np.random.default_rng(0)
n_layers = 6
layer_size = 64
n_samples = 100

# Simulate forward + backward pass for sigmoid and ReLU
def simulate_grad_magnitudes(activation, seed=0):
    rng = np.random.default_rng(seed)
    # Initialize weights with small scale
    weights = [rng.normal(0, 0.5, (layer_size, layer_size)) for _ in range(n_layers)]
    biases = [np.zeros(layer_size) for _ in range(n_layers)]
    X = rng.normal(0, 1, (n_samples, layer_size))

    # Forward pass - store pre-activations
    zs = []
    A = X
    for W, b in zip(weights, biases):
        Z = A @ W + b
        zs.append(Z)
        if activation == "sigmoid":
            A = 1.0 / (1.0 + np.exp(-np.clip(Z, -500, 500)))
        else:
            A = np.maximum(0, Z)

    # Backward pass - track gradient magnitudes
    grad_mags = []
    delta = rng.normal(0, 1, (n_samples, layer_size)) # simulated output gradient
    for l in range(n_layers - 1, -1, -1):
        Z = zs[l]
        if activation == "sigmoid":
            s = 1.0 / (1.0 + np.exp(-np.clip(Z, -500, 500)))
            act_grad = s * (1 - s) # max 0.25
```

```

        else:
            act_grad = (Z > 0).astype(float)
            delta = delta * act_grad
            grad_mags.append(float(np.mean(np.abs(delta))))
            delta = delta @ weights[1].T
        grad_mags.reverse()
        return grad_mags

sig_mags = simulate_grad_magnitudes("sigmoid")
relu_mags = simulate_grad_magnitudes("relu")

layer_indices = list(range(1, n_layers + 1))

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=layer_indices, y=sig_mags,
    mode="lines+markers",
    name="Sigmoid",
    line=dict(color="#1f77b4", width=2.5),
    marker=dict(size=9, color="#1f77b4")
))
fig.add_trace(go.Scatter(
    x=layer_indices, y=relu_mags,
    mode="lines+markers",
    name="ReLU",
    line=dict(color="#ff7f0e", width=2.5, dash="dash"),
    marker=dict(size=9, color="#ff7f0e", symbol="diamond")
))

fig.update_layout(
    title="Gradiente desvaneciente: Sigmoid vs ReLU",
    xaxis=dict(title="Indice de capa (desde la salida)", tickvals=layer_indices),
    yaxis=dict(title="Magnitud media del gradiente", type="log"),
    legend=dict(x=0.02, y=0.98),
    height=420, width=660
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.5.: Magnitud del gradiente (escala logarítmica) en función del índice de capa para una red de 6 capas con activaciones sigmoid (gradiente desvaneciente) versus ReLU (gradiente estable). La simulación inicializa pesos aleatorios y propaga el error desde la salida. Con sigmoid, el gradiente colapsa a valores cercanos a cero en las capas iniciales.

```

import numpy as np
import plotly.graph_objects as go
from sklearn.datasets import make_classification

```

```

rng = np.random.default_rng(42)
X_cls, y_cls = make_classification(n_samples=80, n_features=2, n_redundant=0,
                                   n_informative=2, random_state=42)

# Fix all other parameters, vary w1 and w2
def bce_loss(w1, w2, X, y, w_fixed=0.0, b_fixed=0.0):
    eps = 1e-12
    z = X[:, 0] * w1 + X[:, 1] * w2 + b_fixed
    yhat = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
    yhat = np.clip(yhat, eps, 1 - eps)
    return -np.mean(y * np.log(yhat) + (1 - y) * np.log(1 - yhat))

w1_range = np.linspace(-3, 3, 60)
w2_range = np.linspace(-3, 3, 60)
W1, W2 = np.meshgrid(w1_range, w2_range)
Z_loss = np.array([[bce_loss(w1, w2, X_cls, y_cls) for w1 in w1_range] for w2 in w2_range])

# Find approximate minimum
min_idx = np.unravel_index(np.argmin(Z_loss), Z_loss.shape)
w1_min = w1_range[min_idx[1]]
w2_min = w2_range[min_idx[0]]
loss_min = Z_loss[min_idx]

fig = go.Figure()
fig.add_trace(go.Surface(
    x=W1, y=W2, z=Z_loss,
    colorscale="Viridis",
    contours=dict(z=dict(show=True, usecolormap=True, highlightcolor="white",
                        project=dict(z=True))),
    opacity=0.88, showscale=True,
    colorbar=dict(title="BCE Loss", len=0.6)
))
fig.add_trace(go.Scatter3d(
    x=[w1_min], y=[w2_min], z=[loss_min],
    mode="markers",
    marker=dict(size=8, color="red", symbol="circle"),
    name="Minimo"
))

fig.update_layout(
    title="Superficie de perdida: entropia cruzada binaria",
    scene=dict(
        xaxis_title="w1",
        yaxis_title="w2",
        zaxis_title="BCE Loss",
        camera=dict(eye=dict(x=1.6, y=-1.6, z=1.0))
    ),
    height=520, width=680,
    legend=dict(x=0.02, y=0.98)
)

```

```
)  
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 145.6.: Superficie 3D de la pérdida de entropía cruzada como función de dos pesos w_1 y w_2 de un modelo logístico simple. El punto rojo marca el mínimo. Las isocurvas proyectadas en la base muestran la geometría del paisaje de optimización, que es convexa para este modelo de una sola capa.

145.5. Optimización

145.5.1. Descenso de gradiente estocástico (SGD)

El descenso de gradiente estocástico (*Stochastic Gradient Descent*, SGD) es el algoritmo base de optimización para el entrenamiento de redes neuronales. En lugar de calcular el gradiente exacto sobre todo el conjunto de entrenamiento (lo que sería computacionalmente prohibitivo para conjuntos grandes), se calcula el gradiente sobre un mini-lote aleatorio de B ejemplos:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}_{\text{mini-lote}} \quad (145.10)$$

La tasa de aprendizaje η controla el tamaño de cada paso en el espacio de parámetros. Si es demasiado grande, el algoritmo puede divergir u oscilar alrededor del mínimo; si es demasiado pequeña, el entrenamiento es lento. El tamaño del mini-lote B introduce un trade-off entre precisión del gradiente (lotes grandes) y eficiencia computacional y regularización implícita (lotes pequeños). El ruido del gradiente introducido por el muestreo aleatorio actúa como regularizador: impide que el optimizador converja a mínimos excesivamente agudos del paisaje de pérdida, lo que favorece soluciones con mejor generalización.

145.5.2. Momentum

El SGD con momentum introduce una variable de velocidad $\mathbf{v}$ que acumula gradientes pasados, similar a un objeto con inercia que acelera en direcciones de descenso consistente y amortigua las oscilaciones:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + (1 - \beta) \nabla \mathcal{L} \quad (145.11)$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v} \quad (145.12)$$

Con β típicamente en $[0.85, 0.99]$, el momentum efectivo que llega al parámetro es la suma geoméricamente ponderada de gradientes pasados, lo que suaviza la trayectoria de optimización y permite tasas de aprendizaje más grandes. En paisajes con curvatura muy anisotrópica (donde algunas dimensiones tienen gradientes mucho mayores que otras), el momentum reduce las oscilaciones en las dimensiones de alta curvatura y acelera el progreso en las de baja curvatura.

145.5.3. Adam

El optimizador Adam (Adaptive Moment Estimation) (Kingma y Ba 2015) combina el momentum con la adaptacion de la tasa de aprendizaje por coordenada, usando estimaciones de primer y segundo momento del gradiente:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla \mathcal{L} \quad (145.13)$$

$$\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) (\nabla \mathcal{L})^2 \quad (145.14)$$

$$\hat{\mathbf{m}} = \mathbf{m} / (1 - \beta_1^t), \quad \hat{\mathbf{v}} = \mathbf{v} / (1 - \beta_2^t) \quad (145.15)$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \hat{\mathbf{m}} / (\sqrt{\hat{\mathbf{v}}} + \epsilon) \quad (145.16)$$

Los terminos de correccion de sesgo $(1 - \beta_k^t)$ compensan el hecho de que al inicio del entrenamiento $\mathbf{m}$ y $\mathbf{v}$ estan inicializados en cero y subestiman los momentos verdaderos. Los valores tipicos son $\beta_1 = 0.9$, $\beta_2 = 0.999$ y $\epsilon = 10^{-8}$. La division por $\sqrt{\hat{\mathbf{v}}} + \epsilon$ normaliza el gradiente por su magnitud tipica en cada coordenada, lo que hace al optimizador invariante a la escala del gradiente y permite usar la misma tasa de aprendizaje para todos los parametros.

```
import numpy as np
import plotly.graph_objects as go

# Beale function: minimum at (3, 0.5)
def beale(x, y):
    t1 = (1.5 - x + x * y)**2
    t2 = (2.25 - x + x * y**2)**2
    t3 = (2.625 - x + x * y**3)**2
    return t1 + t2 + t3

def beale_grad(x, y):
    # Partial derivatives
    f1 = 1.5 - x + x * y
    f2 = 2.25 - x + x * y**2
    f3 = 2.625 - x + x * y**3
    df1_dx = -1 + y
    df1_dy = x
    df2_dx = -1 + y**2
    df2_dy = 2 * x * y
    df3_dx = -1 + y**3
    df3_dy = 3 * x * y**2
    gx = 2 * f1 * df1_dx + 2 * f2 * df2_dx + 2 * f3 * df3_dx
    gy = 2 * f1 * df1_dy + 2 * f2 * df2_dy + 2 * f3 * df3_dy
    return np.array([gx, gy])

# Starting point
x0 = np.array([-1.5, -0.5])
n_steps = 200

def run_sgd(x0, lr=0.002, n_steps=200):
    x = x0.copy().astype(float)
```

```

    path = [x.copy()]
    for _ in range(n_steps):
        g = beale_grad(x[0], x[1])
        x = x - lr * g
        x = np.clip(x, -4.5, 4.5)
        path.append(x.copy())
    return np.array(path)

def run_momentum(x0, lr=0.002, beta=0.9, n_steps=200):
    x = x0.copy().astype(float)
    v = np.zeros(2)
    path = [x.copy()]
    for _ in range(n_steps):
        g = beale_grad(x[0], x[1])
        v = beta * v + (1 - beta) * g
        x = x - lr * v
        x = np.clip(x, -4.5, 4.5)
        path.append(x.copy())
    return np.array(path)

def run_adam(x0, lr=0.05, beta1=0.9, beta2=0.999, eps=1e-8, n_steps=200):
    x = x0.copy().astype(float)
    m = np.zeros(2)
    v = np.zeros(2)
    path = [x.copy()]
    for t in range(1, n_steps + 1):
        g = beale_grad(x[0], x[1])
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * g**2
        m_hat = m / (1 - beta1**t)
        v_hat = v / (1 - beta2**t)
        x = x - lr * m_hat / (np.sqrt(v_hat) + eps)
        x = np.clip(x, -4.5, 4.5)
        path.append(x.copy())
    return np.array(path)

path_sgd = run_sgd(x0)
path_mom = run_momentum(x0)
path_adam = run_adam(x0)

# Contour grid
xg = np.linspace(-4.5, 4.5, 200)
yg = np.linspace(-4.5, 4.5, 200)
XG, YG = np.meshgrid(xg, yg)
ZG = beale(XG, YG)
ZG_log = np.log1p(ZG)

fig = go.Figure()
fig.add_trace(go.Contour(

```

```

    x=xg, y=yg, z=ZG_log,
    colorscale="Blues",
    contours=dict(coloring="heatmap", showlabels=False),
    showscale=False, opacity=0.7
))

colors = {"SGD": "#d62728", "Momentum": "#ff7f0e", "Adam": "#2ca02c"}
paths = {"SGD": path_sgd, "Momentum": path_mom, "Adam": path_adam}

for name, path in paths.items():
    fig.add_trace(go.Scatter(
        x=path[:, 0], y=path[:, 1],
        mode="lines+markers",
        name=name,
        line=dict(color=colors[name], width=2),
        marker=dict(size=3, color=colors[name])
    ))
    fig.add_trace(go.Scatter(
        x=[path[0, 0]], y=[path[0, 1]],
        mode="markers",
        marker=dict(size=10, color=colors[name], symbol="star"),
        showlegend=False
    ))

# Mark global minimum
fig.add_trace(go.Scatter(
    x=[3.0], y=[0.5],
    mode="markers",
    marker=dict(size=14, color="yellow", symbol="star", line=dict(width=1.5, color="black")),
    name="Minimo (3, 0.5)"
))

fig.update_layout(
    title="Trayectorias de optimizacion sobre la funcion de Beale",
    xaxis=dict(title="w1", range=[-4.5, 4.5]),
    yaxis=dict(title="w2", range=[-4.5, 4.5]),
    height=500, width=640,
    legend=dict(x=0.01, y=0.99)
)
fig.show()

```

145.6. Implementación desde Cero

Para consolidar la comprensión matemática de los algoritmos descritos, se presenta a continuación una implementación completa de un MLP en Python puro con numpy, sin utilizar ninguna biblioteca de aprendizaje automático para el entrenamiento. La implementación sigue exactamente las ecuaciones de propagación hacia adelante (Ecuación 145.4, Ecuación 145.5) y retropropagación (Ecuación 145.7, Ecuación 145.8, Ecuación 145.9). La inicialización de pesos usa la inicialización de He, que escala los pesos por $\sqrt{2/n_{\ell-1}}$ para capas ReLU, lo que asegura que la varianza de las activaciones se preserve a través de las capas en el paso hacia adelante.

Esta implementación sirve como puente entre la teoría matemática y las bibliotecas de alto nivel como PyTorch o TensorFlow. En la práctica, esas bibliotecas ofrecen diferenciación automática, ejecución en GPU y múltiples optimizadores, pero la estructura lógica es idéntica a la presentada aquí. Entender la implementación desde cero es fundamental para diagnosticar problemas de entrenamiento, interpretar el comportamiento de la red y adaptar el modelo a casos de uso no estándar.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.datasets import make_moons

# MLP implementation
class MLP:
    def __init__(self, layer_dims, lr=0.01, seed=42):
        rng = np.random.default_rng(seed)
        self.lr = lr
        self.params = {}
        self.L = len(layer_dims) - 1
        for i in range(1, self.L + 1):
            fan_in = layer_dims[i-1]
            self.params[f"W{i}"] = rng.normal(0, np.sqrt(2.0/fan_in),
                                                (layer_dims[i-1], layer_dims[i]))
            self.params[f"b{i}"] = np.zeros((1, layer_dims[i]))

    def relu(self, z):
        return np.maximum(0, z)

    def relu_grad(self, z):
        return (z > 0).astype(float)

    def sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

    def forward(self, X):
        self.cache = {"A0": X}
        A = X
        for i in range(1, self.L + 1):
            Z = A @ self.params[f"W{i}"] + self.params[f"b{i}"]
            self.cache[f"Z{i}"] = Z
            A = self.relu(Z) if i < self.L else self.sigmoid(Z)
```

```

        self.cache[f"A{i}"] = A
    return A

def backward(self, y):
    m = y.shape[0]
    grads = {}
    dA = self.cache[f"A{self.L}"] - y.reshape(-1, 1)
    for i in range(self.L, 0, -1):
        dZ = dA if i == self.L else dA * self.relu_grad(self.cache[f"Z{i}"])
        grads[f"W{i}"] = self.cache[f"A{i-1}"].T @ dZ / m
        grads[f"b{i}"] = dZ.mean(axis=0, keepdims=True)
        dA = dZ @ self.params[f"W{i}"].T
    return grads

def step(self, grads):
    for i in range(1, self.L + 1):
        self.params[f"W{i}"] -= self.lr * grads[f"W{i}"]
        self.params[f"b{i}"] -= self.lr * grads[f"b{i}"]

def loss(self, yhat, y):
    eps = 1e-12
    yhat = np.clip(yhat, eps, 1 - eps)
    return -np.mean(y * np.log(yhat.ravel()) + (1 - y) * np.log(1 - yhat.ravel()))

def predict(self, X):
    return (self.forward(X) > 0.5).ravel().astype(int)

# Dataset
X_moons, y_moons = make_moons(n_samples=400, noise=0.2, random_state=42)

# Train
mlp = MLP([2, 16, 16, 1], lr=0.08, seed=42)
losses = []
accuracies = []
n_epochs = 300

for epoch in range(n_epochs):
    yhat = mlp.forward(X_moons)
    l = mlp.loss(yhat, y_moons)
    losses.append(l)
    acc = float(np.mean(mlp.predict(X_moons) == y_moons))
    accuracies.append(acc)
    grads = mlp.backward(y_moons)
    mlp.step(grads)

# Store trained mlp for next figure
trained_mlp = mlp

```

```

fig = make_subplots(rows=1, cols=2,
                    subplot_titles=["Perdida de entrenamiento", "Precision de entrenamiento"])

fig.add_trace(go.Scatter(
    x=list(range(n_epochs)), y=losses,
    mode="lines", name="BCE Loss",
    line=dict(color="#1f77b4", width=2.5)
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=list(range(n_epochs)), y=accuracies,
    mode="lines", name="Accuracy",
    line=dict(color="#ff7f0e", width=2.5)
), row=1, col=2)

fig.update_xaxes(title_text="Epoca")
fig.update_yaxes(title_text="BCE Loss", row=1, col=1)
fig.update_yaxes(title_text="Precision", range=[0, 1.05], row=1, col=2)
fig.update_layout(height=380, width=720, showlegend=False,
                    title_text="Curvas de aprendizaje: MLP en make_moons")
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.8.: Curvas de aprendizaje del MLP entrenado desde cero sobre el conjunto `make_moons` ($n=400$, $\text{noise}=0.2$) con arquitectura $[2, 16, 16, 1]$ y 300 épocas. El panel izquierdo muestra la pérdida de entropía cruzada y el panel derecho la precisión de clasificación. Ambas curvas muestran convergencia estable sin oscilaciones.

```

import numpy as np
import plotly.graph_objects as go

# Use trained_mlp from previous cell
h = 0.03
x_min, x_max = X_moons[:, 0].min() - 0.4, X_moons[:, 0].max() + 0.4
y_min, y_max = X_moons[:, 1].min() - 0.4, X_moons[:, 1].max() + 0.4
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
grid_points = np.c_[xx.ravel(), yy.ravel()]
probs = trained_mlp.forward(grid_points).reshape(xx.shape)

final_acc = float(np.mean(trained_mlp.predict(X_moons) == y_moons))

fig = go.Figure()
fig.add_trace(go.Contour(
    x=np.arange(x_min, x_max, h),
    y=np.arange(y_min, y_max, h),
    z=probs,
    colorscale=[[0, "#aec7e8"], [0.5, "#f7f7f7"], [1, "#ffbb78"]],

```

```

    showscale=True,
    colorbar=dict(title="P(clase=1)", len=0.7),
    contours=dict(
        start=0.5, end=0.5, size=0,
        coloring="fill",
        showlabels=False
    ),
    line=dict(color="white", width=2.5),
    opacity=0.85
))

colors_cls = {0: "#1f77b4", 1: "#ff7f0e"}
labels_cls = {0: "Clase 0", 1: "Clase 1"}
for cls in [0, 1]:
    mask = y_moons == cls
    fig.add_trace(go.Scatter(
        x=X_moons[mask, 0], y=X_moons[mask, 1],
        mode="markers",
        marker=dict(color=colors_cls[cls], size=7,
                    line=dict(width=1, color="black"),
                    symbol="circle" if cls == 0 else "diamond"),
        name=labels_cls[cls]
    ))

fig.update_layout(
    title=f"Frontera de decision del MLP | Precision final: {final_acc:.1%}",
    xaxis_title="x1",
    yaxis_title="x2",
    height=480, width=620,
    legend=dict(x=0.01, y=0.99)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.9.: Frontera de decision del MLP entrenado sobre `make_moons`. Los colores de fondo muestran la probabilidad predicha por la red para cada region del espacio de caracteristicas. La linea blanca marca la isocurva de probabilidad 0.5 (frontera de decision). El modelo captura la forma no lineal de las dos lunas con alta precision.

145.7. Redes Convolucionales (CNN)

145.7.1. Motivacion

Las redes completamente conectadas tratan cada pixel de una imagen como una entrada independiente, ignorando por completo la estructura espacial. Para una imagen en escala de grises de 32×32 pixeles, la capa de entrada tiene 1024 unidades. Si la primera capa oculta tiene 512 unidades, solo esta capa requiere

$1024 \times 512 = 524,288$ parámetros. Para imágenes en color de 256×256 píxeles, una sola capa con 512 unidades necesitaría más de 100 millones de parámetros. Además, si la imagen se traslada un píxel a la derecha, la red la ve como una entrada completamente diferente: no tiene ninguna noción de invarianza a la traslación, que es una propiedad fundamental de las imágenes naturales.

Las Redes Convolucionales (CNN) resuelven estos problemas mediante tres principios fundamentales (LeCun et al. 1998). El primero es la **conectividad local**: cada neurona de una capa convolucional se conecta solo a una región pequeña de la capa anterior (el campo receptivo), reduciendo drásticamente el número de conexiones. El segundo es el **compartimiento de pesos**: el mismo filtro (kernel) se aplica en todas las posiciones espaciales de la imagen, por lo que los parámetros del filtro se comparten entre todas las posiciones. El tercero es la **equivarianza a la traslación**: si el objeto se mueve en la imagen, la representación aprendida se mueve de manera correspondiente, lo que permite al modelo reconocer el mismo patrón independientemente de donde aparezca.

145.7.2. La operación de convolución

La operación fundamental de las CNN es la correlación cruzada 2D entre una imagen de entrada I y un filtro K de tamaño $k \times k$. Para cada posición (i, j) en la salida:

$$(I \star K)[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I[i+u, j+v] \cdot K[u, v] \quad (145.17)$$

Si la entrada tiene dimensiones $H \times W$ y el filtro es $k \times k$, la salida tiene dimensiones $(H-k+1) \times (W-k+1)$ (sin relleno). Es importante notar que en la literatura de *deep learning* se llama “convolución” a lo que técnicamente es correlación cruzada: la verdadera convolución matemática voltea el kernel antes de aplicarlo, pero dado que los pesos se aprenden durante el entrenamiento, esta distinción no tiene consecuencias prácticas (Goodfellow et al. 2016).

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(7)
input_img = rng.integers(0, 16, (7, 7)).astype(float)

# Sobel-like horizontal edge detection kernel
kernel = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
], dtype=float)

# Manual convolution (no scipy)
H, W = input_img.shape
kH, kW = kernel.shape
out_H = H - kH + 1
out_W = W - kW + 1
output = np.zeros((out_H, out_W))
for i in range(out_H):
```



```

    for j in range(out_W):
        output[i, j] = np.sum(input_img[i:i+kH, j:j+kW] * kernel)

def make_heatmap_annotations(matrix):
    annotations = []
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            annotations.append(dict(
                x=j, y=i,
                text=f"{matrix[i, j]:.0f}",
                showarrow=False,
                font=dict(size=11, color="black"),
                xanchor="center", yanchor="middle"
            ))
    return annotations

fig = make_subplots(rows=1, cols=3,
                    subplot_titles=["Entrada (7x7)", "Kernel Sobel (3x3)", "Salida (5x5)"],
                    horizontal_spacing=0.12)

fig.add_trace(go.Heatmap(
    z=input_img[:, :-1], colorscale="Blues",
    showscale=False, xgap=1, ygap=1
), row=1, col=1)

fig.add_trace(go.Heatmap(
    z=kernel[:, :-1], colorscale="RdBu",
    showscale=False, xgap=2, ygap=2
), row=1, col=2)

fig.add_trace(go.Heatmap(
    z=output[:, :-1], colorscale="Greens",
    showscale=False, xgap=1, ygap=1
), row=1, col=3)

# Annotations for each panel
for j, (matrix, xref, yref) in enumerate(zip(
    [input_img, kernel, output],
    ["x1", "x2", "x3"],
    ["y1", "y2", "y3"]
), start=1):
    for row_i in range(matrix.shape[0]):
        for col_i in range(matrix.shape[1]):
            val = matrix[matrix.shape[0]-1-row_i, col_i]
            fig.add_annotation(
                x=col_i, y=row_i,
                text=f"{val:.0f}",
                xref=xref, yref=yref,
                showarrow=False,

```

```

        font=dict(size=10, color="black"),
        xanchor="center", yanchor="middle"
    )

fig.update_layout(height=320, width=700,
                  title_text="Convolucion 2D: deteccion de bordes")
fig.update_xaxes(showticklabels=False)
fig.update_yaxes(showticklabels=False)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.10.: Visualizacion de una convolucion 2D con un kernel de deteccion de bordes tipo Sobel. La imagen de entrada (7x7) se convoluciona con el kernel (3x3) para producir un mapa de caracteristicas de salida (5x5). Cada celda muestra su valor numerico. La operacion detecta gradientes horizontales en la imagen.

145.7.3. Capas de pooling

Las capas de pooling reducen las dimensiones espaciales de los mapas de caracteristicas, introduciendo invarianza local a pequenas traslaciones y reduciendo el numero de parametros en las capas subsiguientes. El **max-pooling** sobre una ventana de $p \times p$ retiene el valor maximo de cada ventana, capturando la presencia del patron mas activado en cada region. El **average-pooling** calcula el promedio, produciendo una representacion mas suave. Para una entrada de tamano $H \times W$, el max-pooling con ventana $p \times p$ y stride p produce una salida de tamano $\lfloor H/p \rfloor \times \lfloor W/p \rfloor$.

La operacion de max-pooling tiene una derivada simple: en el paso hacia atras, el gradiente se propaga solo a la posicion que fue el maximo en la ventana (las demas reciben gradiente cero). Esto es eficiente computacionalmente y favorece la especializacion de las neuronas: cada unidad del mapa de caracteristicas aprende a detectar un patron especifico en alguna region del campo receptivo.

145.7.4. Arquitectura CNN clasica: LeNet-5

La arquitectura LeNet-5 (LeCun et al. 1998) establece el patron basico de las CNN modernas: bloques de convolucion seguidos de pooling, finalizados por capas completamente conectadas para la clasificacion. El patron general es:

$$\text{Conv} \rightarrow \text{Pool} \rightarrow \text{Conv} \rightarrow \text{Pool} \rightarrow \text{FC} \rightarrow \text{FC} \rightarrow \text{Softmax}$$

Las capas convolucionales aprenden filtros que detectan caracteristicas locales (bordes, texturas, formas) mientras que las capas completamente conectadas integran estas caracteristicas espaciales para la decision final. Las arquitecturas modernas como VGG, ResNet e Inception siguen este mismo principio, pero con mayor profundidad, conexiones residuales y bloques de atencion.

```

import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Synthetic circle image
size = 28
img = np.zeros((size, size), dtype=float)
cx, cy, r = 13, 13, 9
for i in range(size):
    for j in range(size):
        if (i - cy)**2 + (j - cx)**2 <= r**2:
            img[i, j] = 1.0

# Hand-crafted kernels
kernels = {
    "Borde horizontal": np.array([[ -1, -2, -1], [ 0, 0, 0], [ 1, 2, 1]], dtype=float),
    "Borde vertical":     np.array([[ -1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float),
    "Diagonal":          np.array([[ 2, 1, -1], [ 1, 0, -1], [-1, -1, -2]], dtype=float),
    "Blob (Lap.)":       np.array([[ 0, 1, 0], [ 1, -4, 1], [ 0, 1, 0]], dtype=float),
}

# Manual convolution
def convolve2d_manual(image, kernel):
    H, W = image.shape
    kH, kW = kernel.shape
    out = np.zeros((H - kH + 1, W - kW + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i+kH, j:j+kW] * kernel)
    return out

feature_maps = {name: convolve2d_manual(img, k) for name, k in kernels.items()}

titles = ["Original"] + list(feature_maps.keys())
fig = make_subplots(rows=1, cols=5, subplot_titles=titles, horizontal_spacing=0.05)

fig.add_trace(go.Heatmap(
    z=img[:, :-1], colorscale="Gray",
    showscale=False, xgap=0, ygap=0
), row=1, col=1)

for col, (name, fmap) in enumerate(feature_maps.items(), start=2):
    fig.add_trace(go.Heatmap(
        z=fmap[:, :-1], colorscale="RdBu",
        showscale=(col == 5), xgap=0, ygap=0,
        colorbar=dict(len=0.7, x=1.01) if col == 5 else None
    ), row=1, col=col)

fig.update_layout(height=250, width=950,

```

```

        title_text="Mapas de características convolucionales")
fig.update_xaxes(showticklabels=False)
fig.update_yaxes(showticklabels=False)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.11.: Mapas de características generados por cuatro filtros distintos aplicados a una imagen sintética de un círculo (28x28). De izquierda a derecha: imagen original, detector de bordes horizontales (filtro Sobel horizontal), detector de bordes verticales (filtro Sobel vertical), detector diagonal y detector de blob (laplaciano). Las convoluciones se implementan manualmente con bucles Python sin scipy.

145.8. Regularización en Redes Neuronales

Las redes neuronales con muchos parámetros tienen alta capacidad de memorizar el conjunto de entrenamiento, lo que puede llevar al sobreajuste: el modelo obtiene baja pérdida en entrenamiento pero generaliza pobremente a datos nuevos. La regularización comprende un conjunto de técnicas que reducen la brecha entre el error de entrenamiento y el error de generalización, ya sea penalizando la complejidad del modelo, perturbando el proceso de entrenamiento o limitando la co-adaptación de las neuronas.

145.8.1. Regularización L2 (weight decay)

La regularización L2 añade un término de penalización proporcional a la norma cuadrada de los pesos a la función de pérdida:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{datos}} + \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad (145.18)$$

donde $\|\mathbf{W}\|_F^2 = \sum_{i,j} W_{ij}^2$ es la norma de Frobenius y $\lambda > 0$ es el coeficiente de regularización. El efecto sobre la regla de actualización es una contracción multiplicativa de los pesos en cada paso:

$$\mathbf{W} \leftarrow (1 - \eta\lambda)\mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}_{\text{datos}} \quad (145.19)$$

El factor $(1 - \eta\lambda) < 1$ “decae” los pesos hacia cero en cada iteración, lo que da nombre al método (*weight decay*). Esta penalización favorece soluciones con pesos pequeños, que tienden a ser más suaves y generalizan mejor. La magnitud de λ controla el trade-off entre ajuste a los datos y suavidad del modelo.

145.8.2. Dropout

El Dropout (Srivastava et al. 2014) es una técnica de regularización específica de las redes neuronales. Durante el entrenamiento, en cada paso hacia adelante, cada neurona se desactiva de forma aleatoria con probabilidad $1 - p$ (o equivalentemente, se mantiene activa con probabilidad p , típicamente $p \in [0.5, 0.9]$). Las neuronas desactivadas no contribuyen a la salida ni reciben actualizaciones de gradiente en ese paso.

Durante la inferencia, todas las neuronas estan activas, pero sus salidas se escalan por p (o alternatively, se dividen por p durante el entrenamiento, tecnica conocida como *inverted dropout*).

El Dropout puede interpretarse como el entrenamiento implicito de un ensemble exponencialmente grande de sub-redes. Con n neuronas en total, existen 2^n posibles sub-redes, y en cada paso se entrena una de ellas. En inferencia, la red completa actua como un promedio aproximado de todas estas sub-redes, lo que reduce la varianza del predictor de manera analoga al promediado de ensambles. Esta interpretacion de ensemble implicito explica por que el Dropout mejora la generalizacion de forma tan sistematica.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

rng = np.random.default_rng(3)

layers = [3, 4, 4, 2]
x_positions = [0.1, 0.37, 0.63, 0.9]

def make_neuron_coords(layers, x_positions):
    coords = []
    for l, (n, xp) in enumerate(zip(layers, x_positions)):
        ys = np.linspace(0.15, 0.85, n)
        coords.append([(xp, float(y)) for y in ys])
    return coords

coords = make_neuron_coords(layers, x_positions)

# Dropout mask: 40% of hidden neurons dropped
drop_mask = {}
for l in [1, 2]: # hidden layers
    drop_mask[l] = rng.random(len(coords[l])) > 0.4

def draw_network(fig, coords, drop_mask_active, col, title):
    layer_colors_full = ["#1f77b4", "#2ca02c", "#2ca02c", "#ff7f0e"]
    dropped_color = "#cccccc"

    # Connections
    for l in range(len(coords) - 1):
        for ni, (x0, y0) in enumerate(coords[l]):
            for nj, (x1, y1) in enumerate(coords[l+1]):
                src_active = (l not in drop_mask_active) or drop_mask_active.get(l, {ni: True})[ni]
                # Simplified: check if either neuron is dropped
                src_dropped = drop_mask_active and l in drop_mask_active and not drop_mask_active[ni]
                tgt_dropped = drop_mask_active and (l+1) in drop_mask_active and not drop_mask_active[nj]
                is_dropped = src_dropped or tgt_dropped

                line_style = dict(
                    color="rgba(180,180,180,0.15)" if is_dropped else "rgba(120,120,120,0.35)",
                    width=0.5 if is_dropped else 1.0,
                    dash="dash" if is_dropped else "solid"
```

```

    )
    fig.add_trace(go.Scatter(
        x=[x0, x1], y=[y0, y1], mode="lines",
        line=line_style,
        showlegend=False, hoverinfo="skip",
        xaxis=f"x{col}", yaxis=f"y{col}"
    ), row=1, col=col)

# Neurons
for l, layer_coords in enumerate(coords):
    for ni, (x, y) in enumerate(layer_coords):
        is_dropped = drop_mask_active and l in drop_mask_active and not drop_mask_active[l][n]
        color = dropped_color if is_dropped else layer_colors_full[l]
        opacity = 0.3 if is_dropped else 1.0
        fig.add_trace(go.Scatter(
            x=[x], y=[y], mode="markers",
            marker=dict(color=color, size=22,
                        line=dict(width=2, color="white"),
                        opacity=opacity),
            showlegend=False, hoverinfo="skip",
            xaxis=f"x{col}", yaxis=f"y{col}"
        ), row=1, col=col)

fig_do = make_subplots(rows=1, cols=2,
                        subplot_titles=["Entrenamiento sin Dropout",
                                       "Entrenamiento con Dropout (p=0.6)"],
                        horizontal_spacing=0.15)

draw_network(fig_do, coords, {}, col=1, title="Sin Dropout")
draw_network(fig_do, coords, drop_mask, col=2, title="Con Dropout")

fig_do.update_xaxes(visible=False)
fig_do.update_yaxes(visible=False)
fig_do.update_layout(
    height=380, width=700,
    plot_bgcolor="white",
    title_text="Esquema de Dropout"
)
fig_do.show()

```

145.8.3. Batch Normalization

La Normalizacion por Lote (*Batch Normalization*) es una tecnica que normaliza las pre-activaciones de cada capa dentro del mini-lote de entrenamiento, seguido de una transformacion lineal con parametros aprendibles γ y β . Para cada capa ℓ y mini-lote $\mathcal{B}$ de tamaño m :

$$\mu_B = \frac{1}{m} \sum_{i \in \mathcal{B}} z_i^{(\ell)}, \quad \sigma_B^2 = \frac{1}{m} \sum_{i \in \mathcal{B}} (z_i^{(\ell)} - \mu_B)^2$$

$$\hat{z}^{(\ell)} = \frac{z^{(\ell)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \tilde{z}^{(\ell)} = \gamma \hat{z}^{(\ell)} + \beta \quad (145.20)$$

La normalizacion estabiliza la distribucion de las activaciones a lo largo del entrenamiento (reduciendo el *internal covariate shift*), lo que permite tasas de aprendizaje mas altas y hace al modelo menos sensible a la inicializacion de pesos. Los parametros γ y β son aprendibles y permiten al modelo recuperar la distribucion optima si la normalizacion no es beneficosa en alguna capa.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.datasets import make_moons

X_r, y_r = make_moons(n_samples=400, noise=0.2, random_state=42)

# 80/20 train/val split
perm = np.random.default_rng(0).permutation(len(y_r))
split = int(0.8 * len(y_r))
train_idx, val_idx = perm[:split], perm[split:]
X_tr, y_tr = X_r[train_idx], y_r[train_idx]
X_val, y_val = X_r[val_idx], y_r[val_idx]

class MLPReg:
    def __init__(self, layer_dims, lr=0.05, l2=0.0, dropout_keep=1.0, seed=42):
        rng = np.random.default_rng(seed)
        self.lr = lr
        self.l2 = l2
        self.dropout_keep = dropout_keep
        self.params = {}
        self.L = len(layer_dims) - 1
        self._rng = rng
        for i in range(1, self.L + 1):
            fan_in = layer_dims[i-1]
            self.params[f"W{i}"] = rng.normal(0, np.sqrt(2.0/fan_in),
                                                (layer_dims[i-1], layer_dims[i]))
            self.params[f"b{i}"] = np.zeros((1, layer_dims[i]))

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
```

```

def _relu(self, z):
    return np.maximum(0, z)

def _relu_grad(self, z):
    return (z > 0).astype(float)

def forward(self, X, training=True):
    self.cache = {"A0": X}
    self.masks = {}
    A = X
    for i in range(1, self.L + 1):
        Z = A @ self.params[f"W{i}"] + self.params[f"b{i}"]
        self.cache[f"Z{i}"] = Z
        if i < self.L:
            A = self._relu(Z)
            if training and self.dropout_keep < 1.0:
                mask = (self._rng.random(A.shape) < self.dropout_keep).astype(float)
                A = A * mask / self.dropout_keep
                self.masks[i] = mask
            else:
                A = self._sigmoid(Z)
        self.cache[f"A{i}"] = A
    return A

def backward(self, y):
    m = y.shape[0]
    grads = {}
    dA = self.cache[f"A{self.L}"] - y.reshape(-1, 1)
    for i in range(self.L, 0, -1):
        if i == self.L:
            dZ = dA
        else:
            dZ = dA * self._relu_grad(self.cache[f"Z{i}"])
            if self.dropout_keep < 1.0 and i in self.masks:
                dZ = dZ * self.masks[i] / self.dropout_keep
            grads[f"W{i}"] = self.cache[f"A{i-1}"].T @ dZ / m + self.l2 * self.params[f"W{i}"]
            grads[f"b{i}"] = dZ.mean(axis=0, keepdims=True)
            dA = dZ @ self.params[f"W{i}"].T
    return grads

def step(self, grads):
    for i in range(1, self.L + 1):
        self.params[f"W{i}"] -= self.lr * grads[f"W{i}"]
        self.params[f"b{i}"] -= self.lr * grads[f"b{i}"]

def loss(self, yhat, y):
    eps = 1e-12
    yhat = np.clip(yhat, eps, 1 - eps)
    return -np.mean(y * np.log(yhat.ravel()) + (1 - y) * np.log(1 - yhat.ravel()))

```



```

n_epochs = 300
arch = [2, 32, 32, 1]

configs = [
    ("Sin regularizacion", dict(lr=0.05, l2=0.0, dropout_keep=1.0)),
    ("L2 lambda=0.01", dict(lr=0.05, l2=0.01, dropout_keep=1.0)),
    ("Dropout p=0.7", dict(lr=0.05, l2=0.0, dropout_keep=0.7)),
]

results = {}
for name, kwargs in configs:
    model = MLPReg(arch, seed=42, **kwargs)
    tr_losses, val_losses = [], []
    for ep in range(n_epochs):
        yhat_tr = model.forward(X_tr, training=True)
        grads = model.backward(y_tr)
        model.step(grads)
        tr_losses.append(model.loss(yhat_tr, y_tr))
        yhat_val = model.forward(X_val, training=False)
        val_losses.append(model.loss(yhat_val, y_val))
    results[name] = (tr_losses, val_losses)

colors_reg = {"Sin regularizacion": "#1f77b4",
              "L2 lambda=0.01": "#ff7f0e",
              "Dropout p=0.7": "#2ca02c"}

fig = make_subplots(rows=1, cols=3,
                    subplot_titles=list(results.keys()),
                    shared_yaxes=True)

for col, (name, (tr_l, val_l)) in enumerate(results.items(), start=1):
    color = colors_reg[name]
    epochs = list(range(n_epochs))
    fig.add_trace(go.Scatter(
        x=epochs, y=tr_l, mode="lines", name="Entrenamiento",
        line=dict(color=color, width=2.5),
        showlegend=(col == 1)
    ), row=1, col=col)
    fig.add_trace(go.Scatter(
        x=epochs, y=val_l, mode="lines", name="Validacion",
        line=dict(color=color, width=2.5, dash="dash"),
        showlegend=(col == 1)
    ), row=1, col=col)

fig.update_xaxes(title_text="Epoca")
fig.update_yaxes(title_text="BCE Loss", col=1)
fig.update_layout(height=380, width=850,
                    title_text="Efecto de la regularizacion en la brecha entrenamiento-validacion",

```

```

legend=dict(x=0.01, y=0.99))
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 145.13.: Comparación de curvas de pérdida de entrenamiento y validación para tres versiones del MLP en `make_moons`: sin regularización, con regularización L2 ($\lambda=0.01$) y con Dropout ($p=0.7$). Las versiones regularizadas muestran una menor brecha entre la pérdida de entrenamiento y validación, lo que indica mejor generalización.

145.9. Mas alla: Tipos de Redes

145.9.1. Redes Recurrentes (RNN/LSTM)

Las redes completamente conectadas y las CNN asumen que las entradas son independientes entre sí. Muchos problemas prácticos, sin embargo, involucran datos secuenciales donde el orden temporal es fundamental: series de tiempo, texto natural, señales de audio, trayectorias. Las Redes Neuronales Recurrentes (RNN) abordan este problema manteniendo un estado oculto $\mathbf{h}_t$ que actúa como memoria de la historia de la secuencia:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

El estado oculto $\mathbf{h}_t$ encapsula información relevante de todos los pasos anteriores $\mathbf{x}_1, \dots, \mathbf{x}_t$, permitiendo que la red haga predicciones que dependen del contexto histórico. Sin embargo, las RNN simples sufren de gradiente desvaneciente a través del tiempo: para secuencias largas, los gradientes que fluyen desde el tiempo T hasta el tiempo 1 son el producto de T matrices jacobianas, que pueden ser exponencialmente pequeñas.

Las redes LSTM (*Long Short-Term Memory*) (Hochreiter y Schmidhuber 1997) resuelven este problema mediante el mecanismo de compuertas: una celda de memoria $\mathbf{c}_t$ con flujo de gradiente regulado por compuertas de olvido ($\mathbf{f}_t$), entrada ($\mathbf{i}_t$) y salida ($\mathbf{o}_t$). La compuerta de olvido decide qué información de la celda anterior se descarta, la compuerta de entrada decide qué nueva información se escribe, y la compuerta de salida determina qué parte de la celda se expone como estado oculto. Este mecanismo permite que los gradientes fluyan sin atenuación a través de cientos de pasos de tiempo, haciendo posible el aprendizaje de dependencias de largo alcance.

145.9.2. Mecanismo de Atención y Transformers

El mecanismo de atención (Vaswani et al. 2017) revolucionó el procesamiento de secuencias al permitir que cada posición de la secuencia de salida consulte directamente cualquier posición de la secuencia de entrada, sin importar la distancia entre ellas. La atención escalada de producto punto (*Scaled Dot-Product Attention*) se define como:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

donde $\mathbf{Q}$ (*queries*), $\mathbf{K}$ (*keys*) y $\mathbf{V}$ (*values*) son proyecciones lineales de la secuencia de entrada, y d_k es la dimension de las claves (el factor $\sqrt{d_k}$ previene que los productos internos se vuelvan muy grandes, lo que saturaria el softmax). La atencion asigna a cada posicion un peso que refleja su relevancia para la posicion de consulta, permitiendo que el modelo capture dependencias directas sin importar la distancia.

Los Transformers, introducidos en 2017, reemplazaron a las RNN como arquitectura dominante en procesamiento de lenguaje natural y, mas recientemente, en vision computacional (Vision Transformers, ViT) y otros dominios. La arquitectura consiste en bloques de atencion multi-cabeza (*multi-head attention*) combinados con redes de retroalimentacion posicion a posicion y conexiones residuales, permitiendo el entrenamiento paralelo eficiente y la captura de dependencias de largo alcance sin el cuello de botella secuencial de las RNN.

145.9.3. Aprendizaje por Transferencia

El aprendizaje por transferencia (*Transfer Learning*) es el paradigma dominante del aprendizaje profundo moderno. En lugar de entrenar una red desde cero sobre el conjunto de datos objetivo (lo que requeriria millones de ejemplos y dias de computo), se parte de un modelo pre-entrenado en un conjunto de datos masivo (como ImageNet para vision o Common Crawl para lenguaje) y se ajusta (*fine-tuning*) sobre los datos del problema objetivo.

El fundamento teorico es que las representaciones aprendidas en dominios de gran escala son transferibles: los filtros de bordes, texturas y formas aprendidos sobre ImageNet son utiles para clasificar imagenes medicas; las representaciones linguisticas de BERT son utiles para analisis de sentimiento en dominios especificos. El ajuste puede ser total (se actualizan todos los pesos del modelo) o parcial (se congelan las capas iniciales y solo se actualizan las capas finales), dependiendo del tamano del conjunto de datos objetivo y su similitud con el conjunto de pre-entrenamiento. Este paradigma ha democratizado el aprendizaje profundo, permitiendo obtener resultados de estado del arte con cientos (no millones) de ejemplos etiquetados.

145.10. Resumen

Este capitulo ha recorrido el desarrollo de las redes neuronales desde sus fundamentos mas elementales hasta las arquitecturas modernas. El perceptron de Rosenblatt establece el esqueleto matematico: sumas ponderadas seguidas de no linealidades, entrenadas mediante una regla de actualizacion basada en errores. La extension a redes multicapa, habilitada por el algoritmo de retropropagacion, permite aprender representaciones jerarquicas que resuelven problemas no lineales arbitrariamente complejos, respaldado por el teorema de aproximacion universal. Las funciones de activacion ReLU y la inicializacion cuidadosa de pesos hacen posible el entrenamiento de redes profundas sin sufrir el colapso del gradiente. Los optimizadores modernos como Adam aceleran la convergencia adaptando la tasa de aprendizaje a la geometria local del paisaje de perdida. Las redes convolucionales explotan la estructura espacial de las imagenes mediante conectividad local y compartimiento de pesos, aprendiendo detectores de caracteristicas jerarquicos que escalan desde pixeles hasta objetos. Finalmente, tecnicas de regularizacion como L2 y Dropout permiten que redes con millones de parametros generalicen mas alla del conjunto de entrenamiento.

La implementacion desde cero presentada en la Seccion 6 demuestra que toda esta teoria se traduce en unas pocas decenas de lineas de algebra lineal en numpy, consolidando la comprension profunda de los mecanismos subyacentes que es necesaria para aplicar, diagnosticar y extender estas arquitecturas en contextos reales.

Referencias

- Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.
- Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, 487-99.
- Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>.
- Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. <https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/>.
- Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.
- Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.^a ed. Springer.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.
- Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. <https://doi.org/10.1145/1133905.1133907>.
- Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. <http://github.com/jax-ml/jax>.
- Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. <https://doi.org/10.1007/BF00058655>.
- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. <https://doi.org/10.1080/14719030410001675722>.
- Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

- Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.
- Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. <https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main>.
- Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. <https://doi.org/10.1145/1961189.1961199>.
- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.
- Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. <https://doi.org/10.1145/248603.248616>.
- Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. <https://doi.org/10.1145/2939672.2939785>.
- Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3): 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. <https://cube.dev/docs/>.
- Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.
- DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.
- Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. <https://www.databricks.com>.
- Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.
- Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.
- dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. <https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>.

- DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. <http://github.com/google-deeppmind>.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.
- Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. <https://doi.org/10.1147/sj.271.0060>.
- Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.^a ed. Wiley.
- Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.
- Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.
- Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. <https://doi.org/10.1609/aimag.v17i3.1230>.
- Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.
- Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.^a ed. W. W. Norton.
- Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.
- Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data Analysis* 38 (4): 367-78. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2).
- Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best

- Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. <https://doi.org/10.1145/355744.355745>.
- Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. <https://doi.org/10.1109/T-C.1975.224297>.
- Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. [https://doi.org/10.1016/0022-2836\(87\)90689-9](https://doi.org/10.1016/0022-2836(87)90689-9).
- Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.
- Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3.^a ed. O'Reilly Media.
- Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.
- Golub, Gene H., y Charles F. Van Loan. 2013. *Matrix Computations*. 4.^a ed. Johns Hopkins University Press.
- Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.
- Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. <https://cloud.google.com/vertex-ai/docs>.
- GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. <https://spec.graphql.org/>.
- Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. <https://doi.org/10.1109/ICDE.1996.492099>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-381479-1.00001-0>.
- Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann.
- Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. <https://doi.org/10.1145/342009.335372>.
- Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. <https://doi.org/10.1109/TIT.1968.1054155>.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data*

- Mining, Inference, and Prediction*. 2.^a ed. Springer Series en Statistics. Springer. <https://doi.org/10.1007/978-0-387-84858-7>.
- Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. <https://www.itl.nist.gov/div898/handbook/>.
- Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. <http://github.com/google/flax>.
- Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. <https://doi.org/10.2307/1934352>.
- Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.
- Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.
- Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. John Wiley & Sons.
- Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.^a ed. Wiley. <https://doi.org/10.1002/9780470434697>.
- Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.
- Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. <https://doi.org/10.1080/00031305.1996.10473566>.
- IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. <https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf>.
- Inmon, William H. 2005. *Building the Data Warehouse*. 4.^a ed. Wiley.
- James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.^a ed. Springer Texts en Statistics. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.
- Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.
- Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. <https://doi.org/10.1111/1467-9884.00122>.

- Jolliffe, Ian T. 2002. *Principal Component Analysis*. 2nd ed. Springer.
- Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. <https://doi.org/10.1111/j.2006.0030-1299.14714.x>.
- Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». *Advances in Neural Information Processing Systems* 30.
- Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. <https://doi.org/10.2307/2332226>.
- Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.^a ed. Wiley.
- Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.
- Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.
- Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
- LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.
- Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.
- Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.
- Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. <https://doi.org/10.2307/2276207>.
- Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

- Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.
- MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.
- McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.
- McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.^a ed. Chapman & Hall.
- McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.
- McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.
- Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).
- Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md>.
- Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/machine-learning/>.
- Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2012. *Introduction to Linear Regression Analysis*. 5.^a ed. Wiley.
- Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. <https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15>.
- Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. <https://doi.org/10.1098/rsta.1933.0009>.
- Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.
- Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.
- Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.
- Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. <https://doi.org/10.1098/rspl.1895.0041>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.
- Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. <https://arxiv.org/abs/2010.16061>.
- Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, y Andrey Gulin. 2018. «CatBoost: Unbiased Boosting with Categorical Features». *Advances in Neural Information Processing Systems* 31.
- Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
- Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. <https://doi.org/10.1007/BF00116251>.
- Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. <https://fastapi.tiangolo.com/>.
- Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. <https://doi.org/10.21105/joss.00638>.
- Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.
- Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. <https://doi.org/10.1037/h0042519>.
- Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

- Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. <https://doi.org/10.1080/01621459.1993.10476408>.
- Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.
- Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.
- SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. <https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jn8bbj1a2.htm>.
- Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. <https://doi.org/10.1007/BF00116037>.
- Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical Journal* 27 (3): 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.
- Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. <https://doi.org/10.1038/163688a0>.
- Sokolova, Marina, y Guy Lapalme. 2009. «A Systematic Analysis of Performance Measures for Classification Tasks». *Information Processing & Management* 45 (4): 427-37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. <https://doi.org/10.2307/1412159>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.
- Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. <https://doi.org/10.1214/aos/1176343886>.
- Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>.
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine

- Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.
- Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data Mining*. 2.^a ed. Pearson.
- Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.
- The pandas development team. 2024. *pandas Documentation*. Pandas. <https://pandas.pydata.org/docs/>.
- Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.
- Vapnik, Vladimir N. 1998. *Statistical Learning Theory*. Wiley.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.
- Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.^a ed. Academic Press.
- Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. <https://doi.org/10.1109/TSMC.1972.4309137>.
- Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

146. Implementacion Practica: PyTorch y JAX

En el capitulo anterior construimos redes neuronales desde cero con NumPy: derivamos la regla de la cadena a mano, escribimos cada operacion de convolución a mano y controlamos cada actualización de pesos. Ese ejercicio es indispensable para entender *por qué* funcionan los algoritmos.

Este capitulo es el complemento practico. Tomamos exactamente los mismos conceptos — forward pass, backprop, descenso por gradiente — y los reimplementamos en frameworks de produccion que automatizan lo que ya entendemos.

Hay tres alternativas principales según el nivel de abstracción:

- **scikit-learn** — Un `MLPClassifier` con la interfaz `fit/predict` de siempre. Ideal para tablas pequeñas; sin control sobre el ciclo de entrenamiento.
- **PyTorch** (Paszke et al. 2019) — Grafos computacionales dinámicos, diferenciación automática y una API orientada a objetos. El estándar de facto en investigación académica.
- **JAX** (Bradbury et al. 2018) — Paradigma funcional puro: funciones sin estado, PRNG explícito, compilación XLA transparente. Sobre JAX se construye Flax (Heek et al. 2023) (módulos de redes) y Optax (DeepMind et al. 2020) (optimizadores).

La diferencia entre PyTorch y JAX no es solo sintáctica: reflejan dos paradigmas de programación opuestos. PyTorch es **imperativo** — escribes instrucciones sobre objetos con estado. JAX es **funcional** — las funciones no tienen estado y se pueden transformar automáticamente. Comprender esta diferencia es el objetivo central de este capítulo.

146.1. El ecosistema de aprendizaje profundo

146.1.1. scikit-learn: MLPClassifier

Scikit-learn (Pedregosa et al. 2011) incluye `sklearn.neural_network.MLPClassifier`, un perceptron multicapa que se integra sin fricciones en cualquier pipeline de `sklearn`. Acepta los mismos hiperparametros conceptuales que ya conocemos: numero y tamaño de capas ocultas (`hidden_layer_sizes`), funcion de activacion, optimizador (lbfgs, sgd o adam) y tasa de aprendizaje. La interfaz es identica a la de cualquier otro clasificador de `sklearn`: `fit` para entrenar, `predict` para inferencia y `score` para exactitud. Esta uniformidad hace que sea trivial sustituirlo por un arbol de decision o una SVM en un experimento comparativo.

Las limitaciones son igualmente claras. No permite definir arquitecturas personalizadas: capas convolucionales, mecanismos de atencion, conexiones residuales o cualquier topologia no secuencial estan fuera de alcance. El entrenamiento ocurre en CPU y no escala a conjuntos de datos de imagen o texto de tamaño real. La ausencia de control sobre el ciclo de entrenamiento impide aplicar tecnicas avanzadas como curriculum learning, ciclos de tasa de aprendizaje o perdidas compuestas. Para todo esto se requieren frameworks de bajo nivel.

```

from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons
import numpy as np

X, y = make_moons(n_samples=400, noise=0.2, random_state=42)
clf = MLPClassifier(hidden_layer_sizes=(16, 16), max_iter=500, random_state=42)
clf.fit(X, y)
print(f"Accuracy: {clf.score(X, y):.3f}")

```

146.1.2. PyTorch

PyTorch (Paszke et al. 2019) introdujo en 2017 el **grafo computacional dinámico** (*define-by-run*): el grafo se construye al vuelo mientras se ejecutan operaciones sobre tensores, no antes. A diferencia del grafo estático de TensorFlow 1.x, el código de PyTorch es Python normal — se puede depurar con `print` y `pdb`, y las estructuras de control funcionan exactamente como se esperaría.

El motor de diferenciación automática se llama **Autograd**. Cuando un tensor tiene `requires_grad=True`, cada operación sobre él registra un nodo en el grafo. Al llamar `.backward()` sobre la pérdida escalar, Autograd recorre el grafo en sentido inverso y acumula los gradientes en el atributo `.grad` de cada parámetro.

La clase base para todo modelo es `nn.Module`. Su contrato: el constructor `__init__` declara las capas, y el método `forward` describe el cómputo. El ciclo de entrenamiento es explícito — el programador escribe cada paso. Esa explicitación es una característica, no un defecto: da control total.

146.1.3. JAX

JAX (Bradbury et al. 2018) parte de una premisa diferente: las **transformaciones funcionales** son ciudadanos de primera clase. El núcleo son cuatro transformaciones componibles:

Transformación	Qué hace
<code>jax.grad(f)</code>	Diferenciación automática de funciones escalares
<code>jax.jit(f)</code>	Compilación XLA transparente
<code>jax.vmap(f)</code>	Vectorización sobre dimensiones de lote
<code>jax.pmap(f)</code>	Paralelismo en múltiples dispositivos

Para beneficiarse de `jax.jit`, las funciones deben ser **puras**: sin efectos secundarios, sin estado global mutable. Los pesos del modelo se pasan como *pytrees* (diccionarios/listas de arrays JAX) que JAX recorre automáticamente.

Flax (Heek et al. 2023) añade una API de módulos similar a `nn.Module`, manteniendo la disciplina funcional: `model.init(key, x)` devuelve los parámetros y `model.apply(params, x)` ejecuta el forward pass sin modificar nada internamente. Optax (DeepMind et al. 2020) provee los optimizadores.

146.1.4. Comparativa de frameworks

La eleccion entre frameworks depende del caso de uso. Para prototipado rapido con datos tabulares, sklearn es imbatible en simplicidad. Para investigacion en arquitecturas novedosas o fine-tuning de modelos preentrenados del ecosistema HuggingFace, PyTorch es el estandar de facto. Para investigacion que requiere transformaciones funcionales, diferenciacion de orden superior o integracion nativa con TPUs de Google, JAX es la opcion mas poderosa. El siguiente radar resume estas caracteristicas de forma comparativa.

```
import plotly.graph_objects as go

categories = [
    "Facilidad de uso",
    "Flexibilidad",
    "Velocidad de prototipado",
    "Escalabilidad",
    "Soporte para investigacion"
]

frameworks = {
    "scikit-learn": {"scores": [5, 2, 5, 2, 1], "color": "rgba(99,110,250,0.6)"},
    "PyTorch": {"scores": [3, 5, 4, 5, 5], "color": "rgba(239,85,59,0.6)"},
    "JAX": {"scores": [2, 5, 3, 5, 5], "color": "rgba(0,204,150,0.6)"},
}

fig = go.Figure()

for name, data in frameworks.items():
    scores = data["scores"] + [data["scores"][0]]
    cats = categories + [categories[0]]
    fig.add_trace(go.Scatterpolar(
        r=scores,
        theta=cats,
        fill="toself",
        name=name,
        line_color=data["color"],
        fillcolor=data["color"],
        opacity=0.7
    ))

fig.update_layout(
    polar=dict(
        radialaxis=dict(visible=True, range=[0, 5], tickfont=dict(size=10)),
        angularaxis=dict(tickfont=dict(size=11))
    ),
    showlegend=True,
    title=dict(
        text="Comparativa de frameworks de aprendizaje profundo",
        x=0.5,
        font=dict(size=15)
    ),
)
```

```

    legend=dict(orientation="h", yanchor="bottom", y=-0.15, xanchor="center", x=0.5),
    height=480,
    margin=dict(t=60, b=80)
)

fig.show()

```

Unable to display output for mime type(s): text/html

(a) Comparativa de frameworks de aprendizaje profundo en cinco dimensiones.

Unable to display output for mime type(s): text/html

(b)

Figura 146.1.

146.2. Dataset a explorar

Entorno listo.

Para los siguientes ejemplos utilizaremos el conocido dataset `make_moons`, un clasico para ilustrar clasificadores no lineales.

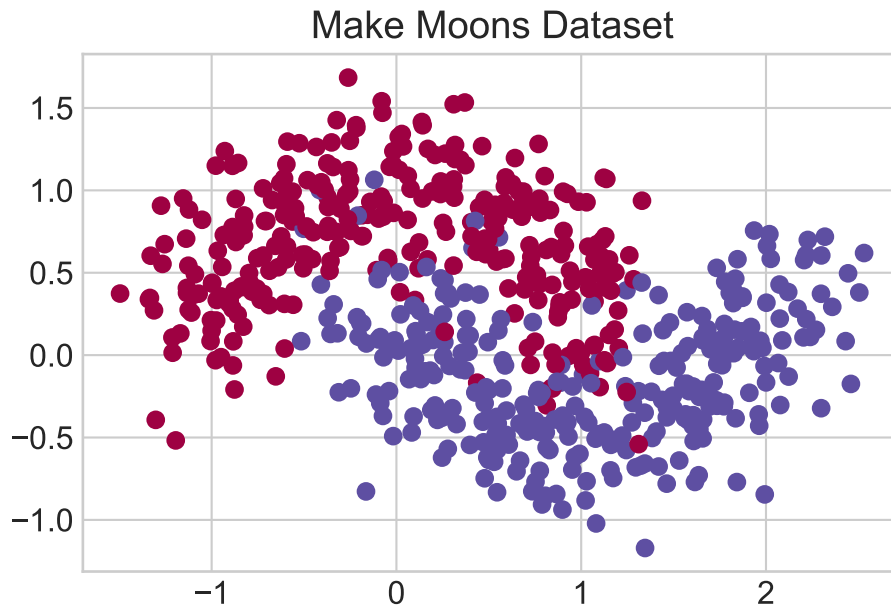
```

from sklearn.datasets import make_moons
X, y = make_moons(n_samples=600, noise=0.25, random_state=SEED)

plt.scatter(X[:,0],X[:,1], c = y, cmap=plt.cm.Spectral)
plt.title("Make Moons Dataset")

```

```
Text(0.5, 1.0, 'Make Moons Dataset')
```



146.3. Preparacion de datos

Antes de entrenar con PyTorch o JAX, los datos deben dividirse en conjuntos de entrenamiento y validacion, y convertirse al formato esperado por cada framework.

146.3.1. Train-test split

Una particion tipica es 80 % entrenamiento, 20 % validacion, con la misma semilla aleatoria para reproducibilidad.

```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)

print(f"Entrenamiento: {X_train.shape[0]} muestras")
print(f"Validacion:     {X_val.shape[0]} muestras")
print(f"Clases en train: {np.bincount(y_train)}")
print(f"Clases en val:   {np.bincount(y_val)}")
```

```
Entrenamiento: 480 muestras
Validacion:     120 muestras
Clases en train: [240 240]
Clases en val:   [60 60]
```

146.3.2. Comparativa: sklearn MLPClassifier

Como punto de partida, entrenemos un MLP de sklearn sobre el mismo dataset. Este es el clasificador “caja negra” que posteriormente comparamos con PyTorch y JAX.

```
from sklearn.neural_network import MLPClassifier

clf_sk = MLPClassifier(
    hidden_layer_sizes=(32, 32),
    max_iter=500,
    random_state=SEED
)
clf_sk.fit(X_train, y_train)
sklearn_train_acc = clf_sk.score(X_train, y_train)
sklearn_val_acc = clf_sk.score(X_val, y_val)

print(f"sklearn MLP | Train acc: {sklearn_train_acc:.3f} | Val acc: {sklearn_val_acc:.3f}")
```

```
sklearn MLP | Train acc: 0.954 | Val acc: 0.933
```

La ventaja de sklearn es la sencillez: una linea para instanciar, una linea para entrenar, una linea para predecir. La desventaja es que no vemos las curvas de convergencia ni tenemos control sobre el ciclo de entrenamiento.

146.4. PyTorch: fundamentos

146.4.1. Tensores y autograd

Los tensores de PyTorch son analogos a los arrays de NumPy: admiten las mismas operaciones aritmeticas, indexacion y broadcasting. La diferencia fundamental es que un tensor puede residir en GPU (mediante `.to("cuda")`) y puede tener activado el rastreo de gradientes mediante `requires_grad=True`.

¿Que significa “rastreo de gradientes”? Cuando estableces `requires_grad=True` en un tensor, PyTorch comienza a registrar cada operacion que le aplicas. Cada operacion crea un nodo en un grafo computacional, junto con la funcion que permitira calcular el gradiente local cuando sea necesario. Es como dejar un rastro de “breadcrumbs” a traves de tus calculos.

146.4.1.1. El ciclo de gradientes: backward() y accumulation

Para un parametro θ y una perdida escalar $\mathcal{L}$, la llamada `loss.backward()` computa todos los gradientes simultaneamente:

$$\frac{\partial \mathcal{L}}{\partial \theta} \quad \forall \theta \text{ con } \text{requires_grad}=\text{True} \quad (146.1)$$

El gradiente resultante se almacena en `theta.grad`.

Atrapas y detalles implementacion: `.backward()` no *reemplaza* el gradiente en `.grad`; *acumula* (suma). Esto significa que si llamas `.backward()` dos veces sin limpiar gradientes intermedios, los valores se suman:

```
# Ejemplo: acumulacion no intencional
w = torch.tensor([2.0], requires_grad=True)
loss1 = w ** 2
loss1.backward()
print(w.grad)  # tensor([4.0]) ← 2*w en w=2

loss2 = w ** 3
loss2.backward()
print(w.grad)  # tensor([16.0]) ← 4 + 3*w^2 = 4 + 12 = 16
```

Por eso **cada ciclo de entrenamiento debe llamar `optimizer.zero_grad()` antes de `.backward()`**. Sin esta linea, el gradiente del epoch t se sumaría al del epoch $t - 1$, produciendo actualizaciones completamente incorrectas que oscilan sin convergencia.

146.4.1.2. La regla de la cadena, automatizada

Para funciones compuestas, Autograd aplica la regla de la cadena automaticamente. Consideremos:

$$z = (w \cdot x + b)^2 \quad \text{y} \quad \mathcal{L} = z$$

El gradiente respecto a w es:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial (wx + b)} \cdot \frac{\partial (wx + b)}{\partial w} = 1 \cdot 2(wx + b) \cdot x = 2(wx + b) \cdot x$$

Autograd calcula esto recorriendo el grafo en sentido inverso, aplicando la regla de la cadena nodo a nodo. La siguiente figura ilustra este grafo para valores concretos $w = 0.5$, $x = 2.0$, $b = 0.3$.

```
import plotly.graph_objects as go
import numpy as np

# Node positions (x, y)
nodes = {
    "x":    (0.0, 2.0),
    "w":    (0.0, 0.0),
    "b":    (0.0, 4.0),
    "mul":  (1.5, 1.0),
    "add":  (3.0, 2.0),
    "sq":   (4.5, 2.0),
    "L":    (6.0, 2.0),
}

node_labels = {
    "x":    "x = 2.0",
```

```

    "w":    "w = 0.5",
    "b":    "b = 0.3",
    "mul":  "w*x = 1.0",
    "add":  "z1 = 1.3",
    "sq":   "z = 1.69",
    "L":    "L = 1.69",
}

# Forward edges: (from, to)
forward_edges = [
    ("x", "mul"),
    ("w", "mul"),
    ("mul", "add"),
    ("b", "add"),
    ("add", "sq"),
    ("sq", "L"),
]

# Backward annotations: (edge, formula)
backward_annots = [
    (("x", "mul"), "dL/dx = 2z1*w = 1.3"),
    (("w", "mul"), "dL/dw = 2z1*x = 2.6"),
    (("b", "add"), "dL/db = 2z1 = 2.6"),
    (("mul", "add"), "dL/dmul = 2z1 = 2.6"),
    (("add", "sq"), "dL/dadd = 2z1 = 2.6"),
    (("sq", "L"), "dL/dsq = 1"),
]

fig = go.Figure()

# Draw forward edges (black arrows)
for (src, dst) in forward_edges:
    x0, y0 = nodes[src]
    x1, y1 = nodes[dst]
    fig.add_annotation(
        x=x1, y=y1, ax=x0, ay=y0,
        xref="x", yref="y", axref="x", ayref="y",
        showarrow=True,
        arrowhead=3, arrowsize=1.3, arrowwidth=2,
        arrowcolor="black"
    )

# Draw backward gradient annotations (orange dashed)
for ((src, dst), formula) in backward_annots:
    x0, y0 = nodes[src]
    x1, y1 = nodes[dst]
    # midpoint for label
    mx = (x0 + x1) / 2
    my = (y0 + y1) / 2 - 0.35

```

```

fig.add_annotation(
    x=x0, y=y0, ax=x1, ay=y1,
    xref="x", yref="y", axref="x", ayref="y",
    showarrow=True,
    arrowhead=2, arrowsize=1.1, arrowwidth=1.5,
    arrowcolor="orange",
    opacity=0.85
)
fig.add_annotation(
    x=mx, y=my,
    text=formula,
    showarrow=False,
    font=dict(size=8, color="darkorange"),
    bgcolor="rgba(255,255,220,0.8)",
    bordercolor="orange",
    borderwidth=1
)

# Draw nodes
node_x = [v[0] for v in nodes.values()]
node_y = [v[1] for v in nodes.values()]
node_text = [node_labels[k] for k in nodes.keys()]

fig.add_trace(go.Scatter(
    x=node_x, y=node_y,
    mode="markers+text",
    marker=dict(size=42, color="steelblue", line=dict(color="white", width=2)),
    text=node_text,
    textposition="top center",
    textfont=dict(size=9),
    hoverinfo="none"
))

# Legend items (manual)
fig.add_trace(go.Scatter(
    x=[None], y=[None], mode="lines",
    line=dict(color="black", width=2),
    name="Forward pass"
))
fig.add_trace(go.Scatter(
    x=[None], y=[None], mode="lines",
    line=dict(color="orange", width=2, dash="dash"),
    name="Backward pass (gradientes)"
))

fig.update_layout(
    title=dict(
        text="Grafo computacional y propagacion hacia atras en PyTorch",
        x=0.5, font=dict(size=14)
    )
)

```

```

    ),
    xaxis=dict(visible=False, range=[-0.5, 6.8]),
    yaxis=dict(visible=False, range=[-0.8, 5.0]),
    showlegend=True,
    legend=dict(x=0.01, y=0.01),
    height=400,
    margin=dict(t=60, b=20, l=20, r=20),
    plot_bgcolor="white"
)

fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.2.: Grafo computacional para $z = (w*x + b)^2$ con propagacion hacia adelante (negro) y hacia atras (naranja).

146.4.2. El patron nn.Module

La clase `nn.Module` es la pieza central de PyTorch para definir modelos. Toda arquitectura, ya sea un MLP simple o un transformador con miles de millones de parametros, es una subclase de `nn.Module`. El contrato es sencillo: el constructor `__init__` declara las capas y sub-modulos (que deben ser atributos de la instancia para que PyTorch los registre), y el metodo `forward` describe el computo que convierte una entrada en una salida.

```

import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        dims = [input_dim] + hidden_dims + [output_dim]
        layers = []
        for i in range(len(dims) - 1):
            layers.append(nn.Linear(dims[i], dims[i+1]))
            if i < len(dims) - 2:
                layers.append(nn.ReLU())
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

```

`nn.Linear(in, out)` implementa $y = xW^T + b$ con inicialización de Kaiming. `nn.ReLU()` aplica $\max(0, z)$ elemento a elemento. `nn.Sequential` encadena módulos: la salida de cada uno es la entrada del siguiente.

Los métodos más útiles de cualquier `nn.Module`:

- `model.parameters()` — itera sobre todos los tensores con `requires_grad=True`. Se pasa directamente al optimizador.
- `model.state_dict()` — diccionario con los valores actuales de todos los parámetros. Permite guardar y cargar con `torch.save` / `torch.load`.
- `model.train()` / `model.eval()` — cambia el **modo** del modelo.

`model.train()` vs `model.eval()` — crítico en inferencia

`model.eval()` no solo es estético. Capas como `nn.Dropout` y `nn.BatchNorm` se comportan diferente según el modo:

- **Dropout en `.train()`** → desactiva neuronas aleatoriamente (regularización).
- **Dropout en `.eval()`** → pasa todos los valores sin cambio (inferencia determinista).
- **BatchNorm en `.train()`** → usa estadísticas del lote actual.
- **BatchNorm en `.eval()`** → usa medias/varianzas acumuladas durante el entrenamiento.

Si olvidas llamar `model.eval()` en evaluación, tus métricas de validación serán ruidosas e irreproducibles.

146.4.3. El ciclo de entrenamiento

El ciclo de entrenamiento explícito es una de las características más apreciadas de PyTorch. A diferencia de los frameworks que ocultan el bucle interno (como Keras o sklearn), PyTorch expone cada paso, lo que facilita la implementación de lógicas de entrenamiento no estándar y permite debuggear exactamente que sucede en cada momento.

146.4.3.1. El patrón de 5 pasos

El ciclo de entrenamiento en PyTorch sigue un patrón rígido que se repite en cada epoch:

```
for epoch in range(n_epochs):
    model.train()           # Paso 1 - modo entrenamiento
    optimizer.zero_grad()   # Paso 2 - limpiar gradientes
    logits = model(X_train) # Paso 3 - forward pass
    loss = criterion(logits, y_train)
    loss.backward()         # Paso 4 - backward pass
    optimizer.step()        # Paso 5 - actualizar pesos
```

El siguiente widget interactivo muestra qué ocurre exactamente en cada paso:

El error más frecuente: olvidar `optimizer.zero_grad()`

PyTorch **acumula** gradientes: `.backward()` suma el gradiente calculado al valor existente en `.grad`. Si no limpias antes de cada paso:

```
# BUG - gradientes se acumulan epoch tras epoch
for epoch in range(300):
    logits = model(X_tr)
    loss = criterion(logits, y_tr)
    loss.backward()  # .grad += nuevo_gradiente (no =)
    optimizer.step()  # usa gradiente acumulado incorrecto
```

El gradiente crece iterativamente. Los pesos reciben actualizaciones enormes y el entrenamiento diverge o produce modelos completamente distintos de lo esperado. La corrección es siempre llamar `optimizer.zero_grad()` antes de `loss.backward()`.

146.4.3.2. Paso 5: `optimizer.step()` — Usar gradientes para actualizar

El optimizador lee los gradientes en `.grad` y actualiza los parametros. Para SGD simple:

$$\theta := \theta - \eta \cdot \nabla \mathcal{L}(\theta)$$

Para Adam (el optimizador que usamos), mantiene exponential moving averages de los gradientes y sus cuadrados, y calcula updates adaptativos por parametro.

146.4.3.3. Función de pérdida numéricamente estable

¿Por qué `BCEWithLogitsLoss` y no `sigmoid + BCELoss`?

Aplicar sigmoide manualmente y luego `BCELoss` produce desbordamientos cuando el logit es muy grande. Por ejemplo, con $z = 100$, `sigmoid(100)` devuelve exactamente 1.0 y $\log(1.0) = 0$ — pero internamente ya hubo overflow.

`BCEWithLogitsLoss` combina ambas operaciones usando el truco log-sum-exp, que es numéricamente estable para cualquier valor:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N [\max(z_i, 0) - z_i y_i + \log(1 + e^{-|z_i|})] \quad (146.2)$$

Regla práctica: siempre usa `BCEWithLogitsLoss` con logits crudos (sin aplicar sigmoide en `forward`).

146.5. Clasificación con PyTorch

146.5.1. Convertir arrays NumPy a tensores PyTorch

El primer paso es convertir los arrays NumPy a tensores PyTorch. Los tensores permiten diferenciación automática (rastreo de gradientes) y pueden ejecutarse en GPU para velocidad.

```
import torch
import torch.nn as nn

# Conversion a tensores - especificar dtype explícitamente
```

```

X_tr = torch.tensor(X_train, dtype=torch.float32)
y_tr = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1) # shape: (N, 1)
X_vl = torch.tensor(X_val, dtype=torch.float32)
y_vl = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

print(f"X_tr: {X_tr.shape} y_tr: {y_tr.shape}")
print(f"X_vl: {X_vl.shape} y_vl: {y_vl.shape}")
# Output:
# X_tr: torch.Size([480, 2]) y_tr: torch.Size([480, 1])
# X_vl: torch.Size([120, 2]) y_vl: torch.Size([120, 1])

```

```

X_tr: torch.Size([480, 2]) y_tr: torch.Size([480, 1])
X_vl: torch.Size([120, 2]) y_vl: torch.Size([120, 1])

```

146.5.1.1. ¿Por que unsqueeze(1)?

Note que `y_train` comienza como un vector 1D de shape `(480,)`. Lo convertimos a shape `(480, 1)` con `.unsqueeze(1)`.

¿Por que? Porque `nn.BCEWithLogitsLoss` espera que la predicción del modelo y la etiqueta tengan exactamente la misma forma. Nuestro modelo es un MLP de arquitectura $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$, que para un lote de $N = 480$ produce un tensor de shape `(480, 1)`. Para que el cálculo de pérdida funcione correctamente, la etiqueta debe tener la misma forma.

Sin el `unsqueeze(1)`, intentarías hacer `BCEWithLogitsLoss` con shapes `(480, 1)` y `(480,)`, que causaría un error de broadcasting.

146.5.2. Definición del modelo MLP

En PyTorch, todo modelo es una subclase de `nn.Module`. La arquitectura es $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$ con activaciones ReLU en las capas ocultas y una salida lineal (logit).

```

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        dims = [input_dim] + hidden_dims + [output_dim]
        layers = []
        for i in range(len(dims) - 1):
            layers.append(nn.Linear(dims[i], dims[i+1]))
            # Activación ReLU en todas excepto la última capa
            if i < len(dims) - 2:
                layers.append(nn.ReLU())
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

# Instanciar

```

```

torch.manual_seed(SEED)
model_pt = MLP(2, [32, 32], 1)

# Contar parametros
n_params = sum(p.numel() for p in model_pt.parameters())
print(f"Total de parametros: {n_params:,}")
print(model_pt)

```

Total de parametros: 1,185

```

MLP(
  (net): Sequential(
    (0): Linear(in_features=2, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=1, bias=True)
  )
)

```

El modelo tiene: capa 1 ($2 \times 32 + 32$ parametros) + capa 2 ($32 \times 32 + 32$) + capa 3 ($32 \times 1 + 1$) = $64 + 32 + 1024 + 32 + 32 + 1 = 1185$ parametros en total.

146.5.3. Ciclo de entrenamiento con PyTorch

Aplicamos el patron descrito en la seccion anterior a un problema de clasificacion binaria con `make_moons`. Este conjunto de datos es un clasico de prueba para fronteras de decision no lineales: dos semilunares entrelazadas que no son separables linealmente. Una red con dos capas ocultas de 32 neuronas con activacion ReLU deberia aprender la frontera correctamente.

La arquitectura es $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$: la capa de entrada recibe las dos coordenadas (x_1, x_2) , las dos capas ocultas aprenden representaciones no lineales, y la capa de salida produce un unico logit que se convierte en probabilidad mediante la sigmoide. Para la clasificacion, el umbral es 0.5: si $\sigma(z) > 0.5$ se predice clase 1, de lo contrario clase 0.

El ciclo de entrenamiento sigue el patron de **cinco pasos** que se repite en cada epoch:

```

torch.manual_seed(SEED)
model_pt = MLP(2, [32, 32], 1)
optimizer = torch.optim.Adam(model_pt.parameters(), lr=0.01)
criterion = nn.BCEWithLogitsLoss()

train_losses, val_losses = [], []
train_accs, val_accs = [], []

for epoch in range(300):
    # FASE DE ENTRENAMIENTO
    model_pt.train()

    # Paso 1 & 2: Limpiar y forward

```

```

optimizer.zero_grad()
logits = model_pt(X_tr)          # Shape: (480, 1) - predicciones del lote

# Paso 3: Calcular perdida
loss = criterion(logits, y_tr)   # Shape: escalar

# Paso 4: Backward - calcula gradientes automaticamente
loss.backward()

# Paso 5: Actualizar - los pesos se modifican usando los gradientes
optimizer.step()

# FASE DE EVALUACION (sin gradientes)
with torch.no_grad():
    model_pt.eval()
    val_logits = model_pt(X_vl)  # Shape: (120, 1)
    val_loss = criterion(val_logits, y_vl).item()  # .item() -> float escalar

    # Convertir logits a probabilidades [0, 1] y luego a predicciones binarias
    train_pred = (torch.sigmoid(logits) >= 0.5).float()  # Shape: (480, 1)
    train_acc = (train_pred == y_tr).float().mean().item()

    val_pred = (torch.sigmoid(val_logits) >= 0.5).float()
    val_acc = (val_pred == y_vl).float().mean().item()

train_losses.append(loss.item())
val_losses.append(val_loss)
train_accs.append(train_acc)
val_accs.append(val_acc)

print(f"Epoch 300 | Train acc: {train_accs[-1]:.3f} | Val acc: {val_accs[-1]:.3f}")

```

Epoch 300 | Train acc: 0.956 | Val acc: 0.925

146.5.3.1. Desglose detallado de cada seccion

Fase de entrenamiento:

1. `model_pt.train()` — Activa modo entrenamiento para capas como Dropout.
2. `optimizer.zero_grad()` — **Crucial:** Limpia los gradientes antiguos. Sin esto, `.backward()` acumularia gradientes sobre los anteriores.
3. `logits = model_pt(X_tr)` — Forward pass: propaga el lote a traves de la red. Output shape (480, 1).
4. `loss = criterion(logits, y_tr)` — Calcula perdida comparando predicciones vs etiquetas verdaderas.
5. `loss.backward()` — Backward pass: recorre el grafo en reversa, acumulando gradientes en `.grad` de cada parametro.
6. `optimizer.step()` — Lee los gradientes y actualiza los pesos segun la regla del optimizador (Adam).

Fase de evaluacion:

- `with torch.no_grad():` Desactiva el rastreo de gradientes. No necesitamos gradientes para evaluacion, asi que esto ahorra memoria y tiempo.
- `model_pt.eval():` Desactiva comportamientos aleatorios como Dropout y congela estadisticas de BatchNorm.
- Convertimos logits a probabilidades con `torch.sigmoid()` y aplicamos umbral 0.5 para clasificacion binaria.
- `.item()` convierte un tensor escalar a un numero Python para poder imprimir o almacenar.

146.5.3.2. Puntos criticos para entender

1. Los gradientes solo se calculan en `loss.backward()`, no en `loss.backward() + optimizer.step().step()` solo lee los gradientes que ya fueron calculados.
2. `torch.no_grad()` es esencial en evaluacion porque no necesitamos gradientes y ahorrar memoria/tiempo es importante con lotes grandes.
3. La evaluacion usa el mismo modelo pero modo diferente (`model_pt.eval()` vs `model_pt.train()`), lo que afecta como se comportan capas especiales.

```
import torch
import torch.nn as nn
import numpy as np
from sklearn.datasets import make_moons
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ---- Data preparation ----
X_np, y_np = make_moons(n_samples=500, noise=0.25, random_state=42)
n_train = int(0.8 * len(X_np))
idx = np.random.RandomState(42).permutation(len(X_np))
train_idx, val_idx = idx[:n_train], idx[n_train:]

X_train = torch.tensor(X_np[train_idx], dtype=torch.float32)
y_train = torch.tensor(y_np[train_idx], dtype=torch.float32).unsqueeze(1)
X_val = torch.tensor(X_np[val_idx], dtype=torch.float32)
y_val = torch.tensor(y_np[val_idx], dtype=torch.float32).unsqueeze(1)

# ---- Model definition ----
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        dims = [input_dim] + hidden_dims + [output_dim]
        layers = []
        for i in range(len(dims) - 1):
            layers.append(nn.Linear(dims[i], dims[i+1]))
            if i < len(dims) - 2:
                layers.append(nn.ReLU())
        self.net = nn.Sequential(*layers)
```

```

    def forward(self, x):
        return self.net(x)

torch.manual_seed(42)
model_pt = MLP(2, [32, 32], 1)
optimizer = torch.optim.Adam(model_pt.parameters(), lr=0.01)
criterion = nn.BCEWithLogitsLoss()

# ---- Training ----
train_losses, val_losses = [], []
train_accs, val_accs = [], []

for epoch in range(300):
    model_pt.train()
    optimizer.zero_grad()
    logits = model_pt(X_train)
    loss = criterion(logits, y_train)
    loss.backward()
    optimizer.step()

    with torch.no_grad():
        train_pred = (torch.sigmoid(logits) >= 0.5).float()
        train_acc = (train_pred == y_train).float().mean().item()

        model_pt.eval()
        val_logits = model_pt(X_val)
        val_loss = criterion(val_logits, y_val).item()
        val_pred = (torch.sigmoid(val_logits) >= 0.5).float()
        val_acc = (val_pred == y_val).float().mean().item()

    train_losses.append(loss.item())
    val_losses.append(val_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)

epochs = list(range(1, 301))

# ---- Figure ----
fig = make_subplots(rows=1, cols=2,
                    subplot_titles=["Perdida (BCE)", "Exactitud"])

fig.add_trace(go.Scatter(x=epochs, y=train_losses, name="Train loss",
                        line=dict(color="steelblue", width=2)), row=1, col=1)
fig.add_trace(go.Scatter(x=epochs, y=val_losses, name="Val loss",
                        line=dict(color="salmon", width=2, dash="dash")), row=1, col=1)

fig.add_trace(go.Scatter(x=epochs, y=train_accs, name="Train acc",
                        line=dict(color="steelblue", width=2, showlegend=False), row=1, col=2)
fig.add_trace(go.Scatter(x=epochs, y=val_accs, name="Val acc",

```

```

    line=dict(color="salmon", width=2, dash="dash"), showlegend=False), row=1, col=2)

fig.update_xaxes(title_text="Epoch")
fig.update_yaxes(title_text="BCE Loss", row=1, col=1)
fig.update_yaxes(title_text="Accuracy", row=1, col=2)

fig.update_layout(
    title=dict(text="Entrenamiento del MLP en PyTorch (make_moons)", x=0.5, font=dict(size=14)),
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.3.: Curvas de perdida y exactitud durante el entrenamiento del MLP en PyTorch sobre make_moons.

Las curvas de perdida muestran el comportamiento tipico de un modelo bien ajustado: la perdida de entrenamiento disminuye monotonamente mientras que la perdida de validacion converge a un valor cercano pero ligeramente superior, indicando que el modelo generaliza sin sobreajuste significativo. Las curvas de exactitud muestran que ambas fracciones (entrenamiento y validacion) alcanzan valores por encima del 90 % tras los primeros 50 epochs, con una mejora marginal en los epochs restantes.

146.5.4. Sensibilidad a la tasa de aprendizaje

La tasa de aprendizaje η es el hiperparametro mas critico de cualquier red neuronal. Un valor demasiado bajo produce convergencia lenta o estancamiento en minimos locales poco favorables; un valor demasiado alto provoca oscilaciones en la perdida o incluso divergencia. La siguiente figura entrena cuatro instancias del mismo MLP con tasas de aprendizaje que cubren tres ordenes de magnitud, permitiendo observar estos regimenes de forma directa.

```

import torch
import torch.nn as nn
import numpy as np
from sklearn.datasets import make_moons
import plotly.graph_objects as go

X_lr, y_lr = make_moons(n_samples=500, noise=0.25, random_state=42)
n_tr_lr = int(0.8 * len(X_lr))
idx_lr = np.random.RandomState(42).permutation(len(X_lr))
X_tr_lr = torch.tensor(X_lr[idx_lr[:n_tr_lr]], dtype=torch.float32)
y_tr_lr = torch.tensor(y_lr[idx_lr[:n_tr_lr]], dtype=torch.float32).unsqueeze(1)
X_vl_lr = torch.tensor(X_lr[idx_lr[n_tr_lr:]], dtype=torch.float32)
y_vl_lr = torch.tensor(y_lr[idx_lr[n_tr_lr:]], dtype=torch.float32).unsqueeze(1)

learning_rates = [0.0005, 0.005, 0.05, 0.5]
lr_colors       = ["#636EFA", "#00CC96", "#EF553B", "#AB63FA"]

```



```

n_ep_lr      = 300
epochs_lr    = list(range(1, n_ep_lr + 1))
criterion_lr = nn.BCEWithLogitsLoss()

fig = go.Figure()

for lr, color in zip(learning_rates, lr_colors):
    torch.manual_seed(42)
    net = nn.Sequential(
        nn.Linear(2, 32), nn.ReLU(),
        nn.Linear(32, 32), nn.ReLU(),
        nn.Linear(32, 1)
    )
    opt_lr = torch.optim.Adam(net.parameters(), lr=lr)
    vl_curve = []

    for _ in range(n_ep_lr):
        net.train()
        opt_lr.zero_grad()
        loss_lr = criterion_lr(net(X_tr_lr), y_tr_lr)
        loss_lr.backward()
        opt_lr.step()
        with torch.no_grad():
            net.eval()
            vl_curve.append(criterion_lr(net(X_vl_lr), y_vl_lr).item())

    fig.add_trace(go.Scatter(
        x=epochs_lr, y=vl_curve,
        name=f"lr = {lr}",
        line=dict(color=color, width=2)
    ))

fig.add_shape(type="line",
    x0=0, x1=n_ep_lr, y0=0.15, y1=0.15,
    line=dict(color="gray", dash="dot", width=1))
fig.add_annotation(
    x=250, y=0.12,
    text="Umbral de buen ajuste",
    showarrow=False, font=dict(size=10, color="gray"))

fig.update_layout(
    title=dict(text="Sensibilidad a la tasa de aprendizaje (val loss)", x=0.5, font=dict(size=14)),
    xaxis_title="Epoch",
    yaxis_title="Perdida de validacion (BCE)",
    legend=dict(orientation="h", yanchor="bottom", y=-0.22, xanchor="center", x=0.5),
    height=420,
    margin=dict(t=60, b=80)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.4.: Perdida de validacion para cuatro tasas de aprendizaje distintas. La zona optima produce una curva suave y decreciente; tasas demasiado bajas convergen lentamente y tasas demasiado altas oscilan o divergen.

La frontera de decision aprendida por la red se puede visualizar evaluando el modelo sobre una malla densa de puntos en el espacio de entrada. Cada punto de la malla recibe una probabilidad predicha $\hat{p} = \sigma(f_{\theta}(x))$, y la isocurva $\hat{p} = 0.5$ define la frontera de decision.

```
import plotly.graph_objects as go
import numpy as np
import torch

# Build grid
x1_range = np.linspace(X_np[:, 0].min() - 0.5, X_np[:, 0].max() + 0.5, 200)
x2_range = np.linspace(X_np[:, 1].min() - 0.5, X_np[:, 1].max() + 0.5, 200)
xx1, xx2 = np.meshgrid(x1_range, x2_range)
grid = np.c_[xx1.ravel(), xx2.ravel()]
grid_t = torch.tensor(grid, dtype=torch.float32)

model_pt.eval()
with torch.no_grad():
    probs = torch.sigmoid(model_pt(grid_t)).numpy().reshape(200, 200)

final_val_acc = val_accs[-1]

# Scatter colors
colors_train = ["#EF553B" if c == 1 else "#636EFA" for c in y_np[train_idx]]

fig = go.Figure()

fig.add_trace(go.Contour(
    x=x1_range, y=x2_range, z=probs,
    colorscale="RdBu_r",
    opacity=0.75,
    showscale=True,
    colorbar=dict(title="P(clase=1)"),
    contours=dict(
        start=0, end=1, size=0.05,
        showlines=False
    )
))

# White decision boundary at 0.5
fig.add_trace(go.Contour(
    x=x1_range, y=x2_range, z=probs,
    showscale=False,
    contours=dict(
```

```

        start=0.5, end=0.5, size=0,
        coloring="lines"
    ),
    line=dict(color="white", width=3),
    name="Frontera (p=0.5)"
))

fig.add_trace(go.Scatter(
    x=X_np[train_idx, 0], y=X_np[train_idx, 1],
    mode="markers",
    marker=dict(
        color=y_np[train_idx].tolist(),
        colorscale=[[0, "#636EFA"], [1, "#EF553B"]],
        size=6, line=dict(color="white", width=0.5)
    ),
    name="Puntos de entrenamiento"
))

fig.add_annotation(
    x=0.02, y=0.97, xref="paper", yref="paper",
    text=f"Val accuracy: {final_val_acc:.3f}",
    showarrow=False,
    font=dict(size=12, color="white"),
    bgcolor="rgba(0,0,0,0.5)",
    bordercolor="white", borderwidth=1
)

fig.update_layout(
    title=dict(text="Frontera de decision del MLP entrenado con PyTorch", x=0.5, font=dict(size=12, color="white"),
    xaxis_title="x1", yaxis_title="x2",
    height=480,
    legend=dict(orientation="h", yanchor="bottom", y=-0.15, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.5.: Frontera de decision del MLP entrenado con PyTorch sobre make_moons. La isocurva blanca corresponde a probabilidad 0.5.

La frontera de decision resultante es claramente no lineal: la red ha aprendido a separar las dos lunas con una curva suave que sigue la estructura geometrica de los datos. La isocurva blanca en $\hat{p} = 0.5$ delimita la region de incertidumbre maxima, donde el modelo considera igualmente probables ambas clases. Las regiones de color intenso corresponden a zonas de alta confianza en la prediccion.

146.5.5. Animacion del aprendizaje

Una de las visualizaciones mas reveladoras del proceso de entrenamiento de una red neuronal es observar como evoluciona la frontera de decision a lo largo de las epocas. Al comienzo del entrenamiento, con pesos aleatorios pequenos, la frontera es casi lineal porque las activaciones ReLU operan en su region lineal. A medida que los pesos crecen y las no linealidades se activan, la frontera adquiere curvatura progresivamente hasta ajustarse a la geometria de los datos. El deslizador permite navegar por este proceso epoch a epoch.

```
import torch
import torch.nn as nn
import numpy as np
from sklearn.datasets import make_moons
import plotly.graph_objects as go

# Dataset
X_an, y_an = make_moons(n_samples=500, noise=0.25, random_state=42)
n_tr_an = int(0.8 * len(X_an))
idx_an = np.random.RandomState(42).permutation(len(X_an))
X_tr_an = torch.tensor(X_an[idx_an[:n_tr_an]], dtype=torch.float32)
y_tr_an = torch.tensor(y_an[idx_an[:n_tr_an]], dtype=torch.float32).unsqueeze(1)

# Decision boundary grid
h_an = 0.05
x0_min, x0_max = X_an[:, 0].min() - 0.4, X_an[:, 0].max() + 0.4
x1_min, x1_max = X_an[:, 1].min() - 0.4, X_an[:, 1].max() + 0.4
gx, gy = np.meshgrid(np.arange(x0_min, x0_max, h_an),
                     np.arange(x1_min, x1_max, h_an))
grid_an = torch.tensor(np.c_[gx.ravel(), gy.ravel()], dtype=torch.float32)
gx_vals = np.arange(x0_min, x0_max, h_an)
gy_vals = np.arange(x1_min, x1_max, h_an)

# Train and capture snapshots every 10 epochs
torch.manual_seed(42)
net_an = nn.Sequential(
    nn.Linear(2, 32), nn.ReLU(),
    nn.Linear(32, 32), nn.ReLU(),
    nn.Linear(32, 1)
)
opt_an = torch.optim.Adam(net_an.parameters(), lr=0.01)
crit_an = nn.BCEWithLogitsLoss()

snap_epochs = [0] + list(range(10, 301, 10))
snapshots_an = []

with torch.no_grad():
    net_an.eval()
    zz0 = torch.sigmoid(net_an(grid_an)).numpy().reshape(gx.shape)
    snapshots_an.append((0, zz0.copy()))
```

```

for ep in range(1, 301):
    net_an.train()
    opt_an.zero_grad()
    crit_an(net_an(X_tr_an), y_tr_an).backward()
    opt_an.step()
    if ep in snap_epochs:
        with torch.no_grad():
            net_an.eval()
            zz = torch.sigmoid(net_an(grid_an)).numpy().reshape(gx.shape)
            snapshots_an.append((ep, zz.copy()))

cl0_an = X_an[y_an == 0]
cl1_an = X_an[y_an == 1]

def make_frame_data(zz):
    return [
        go.Contour(z=zz, x=gx_vals, y=gy_vals,
                    colorscale="RdBu", zmin=0, zmax=1,
                    showscale=False, opacity=0.65,
                    contours=dict(showlines=False)),
        go.Contour(z=zz, x=gx_vals, y=gy_vals,
                    colorscale=[[0, "white"], [1, "white"]],
                    zmin=0.495, zmax=0.505, showscale=False,
                    contours=dict(coloring="lines", showlines=True,
                                  start=0.5, end=0.5, size=0.01),
                    line=dict(color="white", width=2.5)),
        go.Scatter(x=cl0_an[:, 0], y=cl0_an[:, 1], mode="markers",
                    marker=dict(color="#EF553B", size=5, opacity=0.85),
                    name="Clase 0", showlegend=False),
        go.Scatter(x=cl1_an[:, 0], y=cl1_an[:, 1], mode="markers",
                    marker=dict(color="#636EFA", size=5, opacity=0.85),
                    name="Clase 1", showlegend=False),
    ]

frames_an = [
    go.Frame(data=make_frame_data(zz),
             name=str(ep),
             layout=go.Layout(title_text=f"Epoch {ep} | frontera de decision"))
    for ep, zz in snapshots_an
]

init_ep_an, init_zz_an = snapshots_an[0]
init_data = make_frame_data(init_zz_an)
init_data[0]["showscale"] = True
init_data[0]["colorbar"] = dict(title="P(clase 1)", len=0.7)

fig = go.Figure(data=init_data, frames=frames_an)

slider_steps = [

```

```

    dict(args=[[str(ep)],
               dict(mode="immediate",
                    frame=dict(duration=0, redraw=True),
                    transition=dict(duration=0))],
          label=str(ep), method="animate")
    for ep, _ in snapshots_an
]

fig.update_layout(
    title=dict(text=f"Epoch 0 | frontera de decision", x=0.5, font=dict(size=14)),
    xaxis=dict(title="x1", showgrid=False),
    yaxis=dict(title="x2", showgrid=False, scaleanchor="x"),
    height=530,
    margin=dict(t=60, b=130),
    updatemenus=[dict(
        type="buttons", showactive=False,
        y=-0.1, x=0.5, xanchor="center",
        buttons=[
            dict(label="Reproducir",
                 method="animate",
                 args=[None, dict(frame=dict(duration=180, redraw=True),
                                       fromcurrent=True, mode="immediate"))],
            dict(label="Pausa",
                 method="animate",
                 args=[[None], dict(frame=dict(duration=0, redraw=False),
                                       mode="immediate"))])
        ]
    )],
    sliders=[dict(
        active=0,
        steps=slider_steps,
        currentvalue=dict(prefix="Epoch: ", font=dict(size=12)),
        pad=dict(t=45, b=10),
        len=0.9, x=0.05
    )]
)
fig.show()

```

146.6. CNN con PyTorch

146.6.1. Arquitectura CNN en PyTorch

Para ilustrar las redes convolucionales en PyTorch utilizamos `digits` de `sklearn`: 1797 imágenes de dígitos manuscritos en escala de grises de 8×8 píxeles y 10 clases. Es considerablemente más pequeño que MNIST (28×28), pero suficiente para demostrar la arquitectura sin tiempos de espera largos.

146.6.1.1. Convención de forma: channels-first

PyTorch es *channels-first*; JAX/Flax es *channels-last*

PyTorch espera tensores de imagen con forma (N, C, H, W): - N = tamaño del lote - C = canales (1 para escala de grises, 3 para RGB) - H, W = alto y ancho en píxeles

Para `digits` en escala de grises: (N, 1, 8, 8).

JAX/Flax usa el orden opuesto (N, H, W, C). Si portas código entre frameworks sin cambiar el `reshape`, obtendrás errores de forma confusos.

Las capas clave de la arquitectura:

- `nn.Conv2d(in, out, kernel_size, padding)` — aplica `out` filtros de `kernel × kernel` a la entrada. Con `padding=1` y `kernel_size=3`, las dimensiones espaciales se conservan.
- `nn.MaxPool2d(2)` — reduce dimensiones a la mitad seleccionando el máximo en ventanas 2×2 .
- `nn.Flatten()` — convierte el tensor 4D a un vector 1D para capas densas.
- `nn.Dropout(0.3)` — desactiva aleatoriamente el 30 % de las neuronas. Solo activo en `model.train()`.

El flujo de dimensiones con la arquitectura abajo es:

Entrada:	(N, 1, 8, 8)	
Conv1 + ReLU:	(N, 16, 8, 8)	← 16 filtros 3×3, padding conserva tamaño
MaxPool:	(N, 16, 4, 4)	← reducción ×2
Conv2 + ReLU:	(N, 32, 4, 4)	← 32 filtros 3×3
MaxPool:	(N, 32, 2, 2)	← reducción ×2
Flatten:	(N, 128)	← $32 \times 2 \times 2 = 128$
Dense + ReLU:	(N, 64)	
Dropout:	(N, 64)	
Salida:	(N, 10)	← 10 logits para 10 dígitos

```
import torch
import torch.nn as nn
import numpy as np
from sklearn.datasets import load_digits
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ---- Data ----
digits = load_digits()
X_dig = digits.data.astype(np.float32) / 16.0 # normalize to [0, 1]
y_dig = digits.target.astype(np.int64)
```

```

rng = np.random.RandomState(42)
idx_dig = rng.permutation(len(X_dig))
n_tr = int(0.8 * len(X_dig))
tr_idx, vl_idx = idx_dig[:n_tr], idx_dig[n_tr:]

X_tr_t = torch.tensor(X_dig[tr_idx].reshape(-1, 1, 8, 8))
y_tr_t = torch.tensor(y_dig[tr_idx])
X_vl_t = torch.tensor(X_dig[vl_idx].reshape(-1, 1, 8, 8))
y_vl_t = torch.tensor(y_dig[vl_idx])

# ---- Model ----
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), # (N, 16, 8, 8)
            nn.ReLU(),
            nn.MaxPool2d(2), # (N, 16, 4, 4)
            nn.Conv2d(16, 32, kernel_size=3, padding=1), # (N, 32, 4, 4)
            nn.ReLU(),
            nn.MaxPool2d(2), # (N, 32, 2, 2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 2 * 2, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 10),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

torch.manual_seed(42)
cnn_pt = CNN()
opt_cnn = torch.optim.Adam(cnn_pt.parameters(), lr=1e-3)
crit_cnn = nn.CrossEntropyLoss()

cnn_tr_losses, cnn_vl_losses = [], []
cnn_tr_accs, cnn_vl_accs = [], []

for epoch in range(50):
    cnn_pt.train()
    opt_cnn.zero_grad()
    logits_cnn = cnn_pt(X_tr_t)
    loss_cnn = crit_cnn(logits_cnn, y_tr_t)
    loss_cnn.backward()
    opt_cnn.step()

```



```

with torch.no_grad():
    tr_pred = logits_cnn.argmax(dim=1)
    tr_acc = (tr_pred == y_tr_t).float().mean().item()

    cnn_pt.eval()
    vl_logits = cnn_pt(X_vl_t)
    vl_loss = crit_cnn(vl_logits, y_vl_t).item()
    vl_pred = vl_logits.argmax(dim=1)
    vl_acc = (vl_pred == y_vl_t).float().mean().item()

    cnn_tr_losses.append(loss_cnn.item())
    cnn_vl_losses.append(vl_loss)
    cnn_tr_accs.append(tr_acc)
    cnn_vl_accs.append(vl_acc)

epochs_cnn = list(range(1, 51))

fig = make_subplots(rows=1, cols=2,
    subplot_titles=["Perdida (Cross-Entropy)", "Exactitud"])

fig.add_trace(go.Scatter(x=epochs_cnn, y=cnn_tr_losses, name="Train loss",
    line=dict(color="steelblue", width=2)), row=1, col=1)
fig.add_trace(go.Scatter(x=epochs_cnn, y=cnn_vl_losses, name="Val loss",
    line=dict(color="salmon", width=2, dash="dash")), row=1, col=1)

fig.add_trace(go.Scatter(x=epochs_cnn, y=cnn_tr_accs, name="Train acc",
    line=dict(color="steelblue", width=2), showlegend=False), row=1, col=2)
fig.add_trace(go.Scatter(x=epochs_cnn, y=cnn_vl_accs, name="Val acc",
    line=dict(color="salmon", width=2, dash="dash"), showlegend=False), row=1, col=2)

fig.update_xaxes(title_text="Epoch")
fig.update_yaxes(title_text="Cross-Entropy Loss", row=1, col=1)
fig.update_yaxes(title_text="Accuracy", row=1, col=2)

fig.update_layout(
    title=dict(text="Entrenamiento de la CNN en PyTorch (digits)", x=0.5, font=dict(size=14)),
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.7.: Curvas de perdida y exactitud durante el entrenamiento de la CNN en PyTorch sobre el conjunto digits.

La CNN converge rapidamente en las imagenes de 8×8 de **digits**. La exactitud de validacion supera el 95 % en pocos epochs, lo que refleja que estas imagenes son relativamente faciles para una red convolucional: los patrones de cada digito son altamente consistentes y la resolucion baja no introduce demasiada variabilidad.

El Dropout con tasa 0.3 previene el sobreajuste, como se observa en la cercania entre las curvas de entrenamiento y validacion.

Una metrica especialmente util para la clasificacion multiclase es la matriz de confusion, que muestra cuantas instancias de la clase verdadera i fueron predichas como clase j . Los elementos diagonales corresponden a predicciones correctas; los elementos fuera de la diagonal revelan que pares de clases se confunden entre si, lo que puede guiar el analisis de errores y el ajuste de la arquitectura.

```
import plotly.graph_objects as go
import numpy as np
import torch

# Compute confusion matrix
cnn_pt.eval()
with torch.no_grad():
    vl_preds_np = cnn_pt(X_vl_t).argmax(dim=1).numpy()
    vl_true_np = y_vl_t.numpy()

n_classes = 10
conf_mat = np.zeros((n_classes, n_classes), dtype=int)
for t, p in zip(vl_true_np, vl_preds_np):
    conf_mat[t, p] += 1

labels = [str(i) for i in range(10)]
text_mat = [[str(conf_mat[i, j]) for j in range(n_classes)] for i in range(n_classes)]

fig = go.Figure(go.Heatmap(
    z=conf_mat.tolist(),
    x=labels, y=labels,
    colorscale="Blues",
    text=text_mat,
    texttemplate="%{text}",
    textfont=dict(size=11),
    colorbar=dict(title="Conteo")
))

fig.update_layout(
    title=dict(text="Matriz de confusion -- CNN en PyTorch (digits)", x=0.5, font=dict(size=14)),
    xaxis=dict(title="Prediccion", tickfont=dict(size=11)),
    yaxis=dict(title="Etiqueta real", tickfont=dict(size=11), autorange="reversed"),
    height=500,
    width=560
)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 146.8.: Matriz de confusion de la CNN entrenada en PyTorch, evaluada sobre el conjunto de validacion de digits.

La diagonal dominante en la matriz de confusion confirma el buen desempeño del modelo. Las confusiones mas frecuentes suelen ocurrir entre digitos con morfologia similar: el 4 y el 9 comparten una curvatura superior cerrada, y el 3 y el 8 tienen regiones de activacion superpuestas. Estos errores son consistentes con los que comete un clasificador humano cuando la escritura es ambigua, lo que sugiere que el modelo ha aprendido representaciones genuinamente relevantes y no artefactos espurios.

146.6.2. Filtros aprendidos por la primera capa convolucional

Una propiedad notable de las CNNs es que sus filtros no son diseñados manualmente: emergen del proceso de optimizacion como los detectores de características mas utiles para la tarea. En los primeros niveles de una CNN entrenada en imagenes naturales (ImageNet), los filtros suelen especializarse en bordes orientados, manchas de color y texturas de baja frecuencia. En nuestra CNN entrenada sobre digitos 8x8, la primera capa aprende 16 filtros 3x3 que capturan los patrones mas discriminativos de la escritura numerica.

```
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Extract conv1 weights: shape (16, 1, 3, 3) -> (16, 3, 3)
conv1_w = cnn_pt.features[0].weight.detach().numpy()[ :, 0, :, :] # (16, 3, 3)
n_filt = conv1_w.shape[0] # 16

fig = make_subplots(
    rows=2, cols=8,
    subplot_titles=[f"F{i+1}" for i in range(n_filt)],
    horizontal_spacing=0.02,
    vertical_spacing=0.14
)

for i in range(n_filt):
    row = i // 8 + 1
    col = i % 8 + 1
    w = conv1_w[i]
    vmax = max(abs(w.min()), abs(w.max())) + 1e-6
    show_cb = (i == n_filt - 1)
    fig.add_trace(
        go.Heatmap(
            z=w,
            colorscale="RdBu",
            zmin=-vmax, zmax=vmax,
            showscale=show_cb,
            colorbar=dict(title="w", len=0.45, y=0.25, thickness=12)
            if show_cb else None,
            xgap=1, ygap=1
        ),
        row=row, col=col
    )
fig.update_xaxes(showticklabels=False, row=row, col=col)
fig.update_yaxes(showticklabels=False, row=row, col=col)
```

```
fig.update_layout(
    title=dict(text="Filtros aprendidos - Conv1 (16 filtros 3x3)", x=0.5, font=dict(size=14)),
    height=310,
    margin=dict(t=55, b=15, l=10, r=30)
)
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 146.9.: Pesos de los 16 filtros 3x3 aprendidos por la primera capa convolucional de la CNN entrenada en `load_digits`. Los colores rojo y azul representan pesos positivos y negativos respectivamente. Los patrones revelan detectores de bordes, esquinas y cambios de contraste que la red desarrollo de forma automatica.

146.7. JAX: fundamentos

146.7.1. El paradigma funcional

JAX (Bradbury et al. 2018) redefine completamente la relación entre Python y el hardware de aceleración. La diferencia no es solo técnica; es una filosofía de programación fundamentalmente diferente.

El siguiente widget interactivo muestra la diferencia central de forma concreta:

146.7.1.1. ¿Que es programacion funcional pura?

La diferencia central: en PyTorch el estado *vive dentro del objeto modelo* y muta con cada paso. En JAX el estado son *datos externos* que pasas explícitamente y que nunca se modifican — cada operación produce valores nuevos.

146.7.1.2. Las cuatro transformaciones de JAX

JAX proporciona cuatro transformaciones componibles:

1. `jax.grad(f)` — Diferenciación automática

```
def loss(w):
    return jnp.sum(w ** 2)

g = jax.grad(loss)(jnp.array([1., 2., 3.])) # → [2., 4., 6.]
```

Retorna directamente los gradientes como pytree. No acumula en `.grad` — los gradientes son valores de retorno, no efectos secundarios.

2. `jax.jit(f)` — Compilación XLA

```
@jax.jit
def forward(params, x):
    return x @ params["W"] + params["b"]

forward(params, x) # Primera vez: traza + compila (lento)
forward(params, x) # Siguietes: ejecuta compilado (rápido)
```

3. `jax.vmap(f)` — Vectorización automática

Transforma una función que opera sobre *un solo ejemplo* en una que opera sobre *un lote entero*, sin escribir índices de lote.

4. `jax.pmap(f)` — Paralelismo entre dispositivos

Ejecuta en paralelo sobre múltiples GPUs/TPUs. Poco común para usuarios nuevos.

La potencia de la composición funcional

Porque todas las funciones JAX son puras, se pueden anidar libremente:

```
# Gradiente de una función compilada con JIT
grad_fn = jax.jit(jax.grad(loss_fn))

# Gradiente vectorizado sobre un lote - ¡sin bucles!
per_sample_grads = jax.vmap(jax.grad(loss_single))(X_batch)

# Segunda derivada (Hessiano) - imposible en PyTorch sin config especial
hessian_fn = jax.grad(jax.grad(loss_fn))
```

En PyTorch, componer estas operaciones requiere configuraciones no estándar. En JAX, es la forma natural de trabajar.

146.7.2. Parametros como pytrees: estado explícito

En JAX no existe un objeto “modelo” que contenga estado interno. En su lugar, los parametros son **estructuras de datos Python explícitas** (diccionarios, listas, tuplas, o combinaciones anidadas) que contienen los arrays de JAX. Estos se pasan explícitamente como argumentos a cada función.

146.7.2.1. ¿Por qué es ventajoso?

1. **Inspeccionables:** Puedes ver exactamente qué parametros tiene tu modelo en cualquier momento.
2. **Serializables:** Guardar y cargar es trivial — solo serializa el diccionario.
3. **Transformables:** JAX entiende estas estructuras y puede aplicar transformaciones automáticamente.

146.7.2.2. Pytrees: contenedores anidados de datos

Un pytree es cualquier estructura de datos Python donde las “hojas” (elementos más profundos) son arrays JAX. Ejemplos:

```
# Pytree simple - diccionario
params_simple = {
    "W": [jnp.array([[1., 2.], [3., 4.]]), # Layer 1 weights
          jnp.array([[5., 6.], [7., 8.]]), # Layer 2 weights
    "b": [jnp.array([0.1, 0.2]),          # Layer 1 bias
          jnp.array([0.3])]              # Layer 2 bias
}

# Pytree más complejo - anidación arbitraria
params_complex = {
    "encoder": {
        "dense1": {"W": jnp.array(...), "b": jnp.array(...)},
        "dense2": {"W": jnp.array(...), "b": jnp.array(...)}
    },
    "decoder": {"W": jnp.array(...), "b": jnp.array(...)}
}
```

146.7.2.3. JAX transforma automáticamente cada hoja

La magia de JAX es que cuando usas transformaciones como `jax.grad()` sobre una función que recibe un pytree, **JAX retorna un pytree con la misma estructura**.

```
def loss_fn(params, x, y):
    # params es un pytree (diccionario)
    yhat = mlp_forward(params, x)
    return jnp.mean((yhat - y) ** 2)

# jax.grad retorna una función que toma params y retorna gradientes
grad_loss = jax.grad(loss_fn)

grads = grad_loss(params, x, y)
# grads es un pytree con EXACTAMENTE la misma estructura que params
# Cada leaf es el gradiente del peso correspondiente
```

Esto significa que actualizaciones de parametros son triviales:

```
# Actualización manual - iteraría sobre la estructura completa
for layer_name in params.keys():
    for param_name in params[layer_name].keys():
        params[layer_name][param_name] -= learning_rate * grads[layer_name][param_name]

# Con optax (optimizador JAX) - automático
updates, new_opt_state = optimizer.update(grads, opt_state, params)
new_params = optax.apply_updates(params, updates)
```

146.7.2.4. Ejemplo completo: forward pass con parametros pytree

```
import jax
import jax.numpy as jnp

def mlp_forward(params, x):
    """
    Forward pass que usa un pytree de parametros.
    params["W"] es una lista de matrices (capas)
    params["b"] es una lista de vectores (biases)
    """
    # Capas ocultas con ReLU
    for i in range(len(params["W"]) - 1):
        # x @ params["W"][i] hace: (batch, in) @ (in, out) -> (batch, out)
        x = jnp.maximum(0, x @ params["W"][i] + params["b"][i])

    # Capa final: sin activación (logit)
    x = x @ params["W"][-1] + params["b"][-1]
    return x.squeeze(-1) # (batch, 1) -> (batch,)
```

146.7.3. Visualizacion: grad de la entropia cruzada

Para ilustrar `jax.grad` de forma concreta, calculamos la perdida de entropia cruzada binaria y su gradiente respecto a un peso escalar w variando en $[-3, 3]$, con entrada fija $x = 1$ e etiqueta $y = 1$. La perdida es:

$$\mathcal{L}(w) = -[y \log \sigma(wx) + (1 - y) \log(1 - \sigma(wx))] \quad (146.3)$$

donde $\sigma(z) = 1/(1 + e^{-z})$. Con $y = x = 1$, la perdida simplifica a $\mathcal{L}(w) = -\log \sigma(w)$, que es una funcion decreciente (a mayor w , la prediccion $\sigma(w)$ se acerca a 1 y la perdida se acerca a 0). El gradiente es $\partial \mathcal{L} / \partial w = \sigma(w) - 1 \leq 0$, siempre negativo: el descenso por gradiente siempre incrementa w para que la prediccion se acerque a $y = 1$.

```
import jax
import jax.numpy as jnp
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

x_fixed = 1.0
y_fixed = 1.0

def bce_single(w):
    z = w * x_fixed
    sigma = jax.nn.sigmoid(z)
    eps = 1e-7
    sigma = jnp.clip(sigma, eps, 1 - eps)
    return -(y_fixed * jnp.log(sigma) + (1 - y_fixed) * jnp.log(1 - sigma))
```

```

grad_bce = jax.grad(bce_single)

w_vals = np.linspace(-3.0, 3.0, 200)
loss_vals = np.array([float(bce_single(float(w))) for w in w_vals])
grad_vals = np.array([float(grad_bce(float(w))) for w in w_vals])

fig = make_subplots(rows=2, cols=1,
                    subplot_titles=["Perdida BCE", "Gradiente dL/dw"],
                    shared_xaxes=True, vertical_spacing=0.12)

fig.add_trace(go.Scatter(
    x=w_vals, y=loss_vals,
    line=dict(color="steelblue", width=2.5),
    name="L(w)"
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=w_vals, y=grad_vals,
    line=dict(color="darkorange", width=2.5),
    name="dL/dw"
), row=2, col=1)

fig.add_hline(y=0, line=dict(color="gray", dash="dot", width=1), row=2, col=1)

fig.update_xaxes(title_text="w", row=2, col=1)
fig.update_yaxes(title_text="Perdida", row=1, col=1)
fig.update_yaxes(title_text="Gradiente", row=2, col=1)

fig.update_layout(
    title=dict(text="jax.grad: perdida y gradiente de la entropia cruzada", x=0.5, font=dict(size=
    height=480,
    showlegend=True,
    legend=dict(orientation="h", yanchor="bottom", y=-0.12, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.10.: Perdida de entropia cruzada binaria y su gradiente calculado con `jax.grad`, para $x=1$, $y=1$.

La figura confirma la intuicion matematica: la perdida es alta y decreciente para w negativo (la red esta haciendo predicciones incorrectas con alta confianza) y se aproxima a cero a medida que w crece. El gradiente es siempre negativo, lo que significa que el descenso por gradiente siempre empuja w hacia valores mas grandes. En $w = 0$ (inicializacion tipica), el gradiente vale exactamente -0.5 , lo que corresponde al caso de incertidumbre maxima donde $\sigma(0) = 0.5$.

146.7.4. Derivadas de orden superior: una ventaja exclusiva de JAX

Una capacidad que distingue a JAX de otros frameworks es la diferenciación de orden superior mediante composición de `jax.grad`. En PyTorch es posible pero requiere configuraciones especiales (`create_graph=True`); en JAX es natural: `jax.grad(jax.grad(f))` es exactamente lo que parece. Esta capacidad es fundamental para métodos de optimización de segundo orden (Newton-Raphson, Gauss-Newton), regularización basada en la traza del Hessiano, y meta-aprendizaje (donde se diferencia a través de un bucle de entrenamiento completo).

La segunda derivada de la pérdida respecto a un peso, $\partial^2 \mathcal{L} / \partial w^2$, mide la curvatura del paisaje de optimización. Una curvatura alta indica que pequeños cambios en el peso producen grandes cambios en la pérdida; una curvatura baja indica una región plana. Los optimizadores de segundo orden utilizan esta información para dar pasos adaptados a la geometría local, a diferencia de SGD que usa el mismo tamaño de paso en todas las direcciones.

```
import jax
import jax.numpy as jnp
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

def bce_ho(w):
    sigma = jax.nn.sigmoid(w)
    return -jnp.log(jnp.clip(sigma, 1e-7, 1.0 - 1e-7))

grad1_ho = jax.grad(bce_ho)
grad2_ho = jax.grad(jax.grad(bce_ho))

w_ho = np.linspace(-5, 5, 300)
loss_ho = np.array([float(bce_ho(jnp.array(float(w)))) for w in w_ho])
g1_ho = np.array([float(grad1_ho(jnp.array(float(w)))) for w in w_ho])
g2_ho = np.array([float(grad2_ho(jnp.array(float(w)))) for w in w_ho])

fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=["Pérdida L(w)", "Primera derivada dL/dw", "Segunda derivada d2L/dw2"]
)

palette = ["#636EFA", "#EF553B", "#00CC96"]
for col, (vals, name) in enumerate(
    zip([loss_ho, g1_ho, g2_ho], ["L(w)", "dL/dw", "d2L/dw2"]), start=1
):
    fig.add_trace(
        go.Scatter(x=w_ho, y=vals, mode="lines",
                    line=dict(color=palette[col - 1], width=2.5), name=name),
        row=1, col=col
    )
    fig.add_hline(y=0, line_dash="dot", line_color="gray", line_width=1, row=1, col=col)
    fig.add_vline(x=0, line_dash="dot", line_color="gray", line_width=1, row=1, col=col)
```

```

fig.add_annotation(
    x=0, y=float(grad2_ho(jnp.array(0.0))),
    text="Curvatura max en w=0",
    showarrow=True, arrowhead=2, ax=60, ay=-30,
    font=dict(size=10), row=1, col=3
)

fig.update_xaxes(title_text="w")
fig.update_layout(
    title=dict(text="Derivadas de orden superior con jax.grad compuesto", x=0.5, font=dict(size=10),
    showlegend=False,
    height=380,
    margin=dict(t=70, b=50)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.11.: Perdida, primer derivada y segunda derivada de la entropia cruzada binaria, calculadas con `jax.grad` anidado. La segunda derivada (curvatura) es siempre positiva, confirmando que la perdida es convexa respecto a este peso. La curvatura maxima ocurre en $w=0$ donde la incertidumbre es maxima.

146.8. Clasificacion con JAX puro

146.8.1. Cambio de paradigma: funciones puras y pytrees

Implementamos un MLP completo en JAX puro (sin Flax) sobre el mismo conjunto `make_moons` usado en la seccion de PyTorch. Esto permite una comparacion directa del codigo y los resultados. La arquitectura es identica: $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$.

En JAX puro, no existe un objeto “modelo” con estado interno. Los parametros son diccionarios Python (pytrees) que se pasan explicitamente como argumentos. Esta disciplina funcional tiene ventajas: los parametros son inspeccionables, se pueden serializar como dicts, y las transformaciones de JAX pueden recorrerlos automaticamente.

146.8.2. Inicializacion de parametros JAX

En PyTorch, la inicializacion de pesos ocurre automaticamente dentro de `nn.Linear()`. En JAX puro, debes ser explicito y escribir tu propia función de inicializacion.

146.8.2.1. ¿Por qué inicializacion explicita?

Porque en JAX **no hay objeto modelo con estado**. Los parametros son solo datos que debes crear tú mismo. Esto parece verboso, pero te da control total.

146.8.2.2. PRNG management — ¿Por qué `jax.random.split`?

JAX requiere **determinismo puro**: la misma función con los mismos argumentos debe retornar exactamente lo mismo. Por eso no hay un generador global mutable como en NumPy — la aleatoriedad debe ser **explícita**.

Error frecuente: reusar la misma key

En JAX, si llamas `jax.random.normal` dos veces con la **misma** key, obtienes el **mismo** resultado — no valores distintos:

```
key = jax.random.PRNGKey(42)
W1 = jax.random.normal(key, (5, 10))
W2 = jax.random.normal(key, (5, 10)) # ¡W2 == W1!
```

Para obtener valores independientes en cada capa, debes **dividir** la key:

```
key = jax.random.PRNGKey(42)
key1, key2 = jax.random.split(key) # dos keys derivadas, distintas
W1 = jax.random.normal(key1, (5, 10))
W2 = jax.random.normal(key2, (5, 10)) # W1 W2
```

En un bucle de inicialización de capas, el patrón estándar es:

```
for i in range(n_layers):
    key, subkey = jax.random.split(key) # avanzar key, extraer subkey
    W = jax.random.normal(subkey, shape)
```

146.8.2.3. Función de inicialización con He initialization

```
import jax
import jax.numpy as jnp
import optax

def init_params(layer_dims, key):
    """
    Inicializa los parametros del MLP con He initialization.

    Args:
        layer_dims: Lista de dimensiones, ej. [2, 32, 32, 1]
        key: PRNG key de JAX

    Returns:
        Pytree (diccionario) con estructuras:
        {"W": [matriz1, matriz2, ...], "b": [vector1, vector2, ...]}
    """
    params = {"W": [], "b": []}
```

```

for i in range(len(layer_dims) - 1):
    # Dividir key para obtener una key única para esta capa
    key, subkey = jax.random.split(key)

    # He initialization: los pesos se escalan por sqrt(2 / fan_in)
    # Esto ayuda a mantener la varianza consistente a traves de capas profundas
    fan_in = layer_dims[i]
    W = jax.random.normal(subkey, (layer_dims[i], layer_dims[i+1])) * jnp.sqrt(2.0 / fan_in)

    # Biases iniciales en cero (convención)
    b = jnp.zeros(layer_dims[i+1])

    params["W"].append(W)
    params["b"].append(b)

return params

# Inicializar
key = jax.random.PRNGKey(SEED)
params_jax = init_params([2, 32, 32, 1], key)

print(f"Numero de capas: {len(params_jax['W'])}")
for i, (W, b) in enumerate(zip(params_jax["W"], params_jax["b"])):
    print(f"  Capa {i+1}: W{W.shape}  b{b.shape}")
# Output:
# Numero de capas: 3
#   Capa 1: W(2, 32)  b(32,)
#   Capa 2: W(32, 32)  b(32,)
#   Capa 3: W(32, 1)  b(1,)

```

```

Numero de capas: 3
  Capa 1: W(2, 32)  b(32,)
  Capa 2: W(32, 32)  b(32,)
  Capa 3: W(32, 1)  b(1,)

```

146.8.2.4. He Initialization — ¿por qué?

Sin inicializacion cuidadosa, los parametros pueden ser demasiado grandes o demasiado pequeños: - Si W es muy pequeño: las activaciones desaparecen (gradientes vanishing). - Si W es muy grande: las activaciones saturan o explotan.

He initialization escala cada matriz de pesos para mantener una varianza consistente. Para ReLU, usamos $\text{std} = \sqrt{2/\text{fan_in}}$, donde **fan_in** es el número de entradas a la capa.

Comparacion: PyTorch vs JAX

Aspecto	PyTorch	JAX
Inicializacion	Automatica en <code>nn.Linear()</code>	Manual con <code>init_params()</code>
Acceso a parametros	<code>model.parameters()</code>	<code>params</code> diccionario

Aspecto	PyTorch	JAX
Aleatoriedad	PRNG global implícito	PRNG key explícito
Ventaja	Menos código	Mas control, reproducible

146.8.3. Forward pass y funcion de perdida JAX

146.8.3.1. Forward pass — operaciones lineales + activaciones

```
def forward_jax(params, x):
    """
    Propagacion hacia adelante: MLPx de arquitectura [2 -> 32 -> 32 -> 1].

    Capas ocultas: transformación lineal + ReLU
    Capa final: transformación lineal + sigmoid (retorna probabilidades en [0, 1])
    """
    # Capas ocultas con ReLU
    for i in range(len(params["W"]) - 1):
        # x @ params["W"][i]: (batch, in) @ (in, out) -> (batch, out)
        # + params["b"][i]: agregar bias
        # jax.nn.relu: aplicar activación
        x = jax.nn.relu(x @ params["W"][i] + params["b"][i])

    # Capa final: sin activación (retorna logit)
    logit = x @ params["W"][-1] + params["b"][-1]
    # .squeeze(-1): convertir shape (batch, 1) -> (batch,)
    return jax.nn.sigmoid(logit).squeeze(-1)
```

146.8.3.2. ¿Por qué .squeeze(-1)?

La salida de la capa final antes del `squeeze` es `shape (batch, 1)`. En JAX, es convención que funciones de perdida escalar (como `binary cross-entropy`) reciben vectores 1D (`batch,`) para compatibilidad con operaciones de reducción.

Comparacion de shapes a lo largo del forward pass con entrada `X_tr_j` de `shape (480, 2)`:

```
Entrada x: (480, 2)
Despues de capa 1 + ReLU: (480, 32)
Despues de capa 2 + ReLU: (480, 32)
Despues de capa 3: (480, 1) ← logit antes de sigmoid
Despues de sigmoid: (480, 1)
Despues de squeeze(-1): (480,) ← vector 1D de probabilidades
```

146.8.3.3. Loss function — binary cross-entropy

```
def bce_loss_jax(params, x, y):
    """
    Binary cross-entropy loss.

    
$$L = -E[ y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}) ]$$


    donde yhat es la probabilidad predicha (0 a 1).
    """
    # Forward pass: obtener predicciones
    yhat = forward_jax(params, x) # Shape: (batch,)

    # Clip para evitar log(0) - log(0) no está definido numericamente
    eps = 1e-7
    yhat = jnp.clip(yhat, eps, 1 - eps) # yhat in [eps, 1-eps]

    # BCE formula aplicada elemento a elemento
    loss = -(y * jnp.log(yhat) + (1 - y) * jnp.log(1 - yhat))

    # Retornar promedio sobre el lote
    return jnp.mean(loss)
```

146.8.3.4. La forma de las variables

Cuando llamas `bce_loss_jax(params, X_tr_j, y_tr_j)`: - `x` tiene shape (480, 2) — 480 ejemplos, 2 features - `y` tiene shape (480,) — 480 etiquetas binarias - `yhat` tiene shape (480,) — 480 predicciones - `loss` es un escalar — el promedio de la perdida de todos los ejemplos

Este es el valor que luego diferenciaras con `jax.grad()` para obtener los gradientes.

146.8.4. Preparacion de datos JAX

```
# Convertir datos a JAX arrays
X_tr_j = jnp.array(X_train, dtype=jnp.float32)
y_tr_j = jnp.array(y_train, dtype=jnp.float32)
X_val_j = jnp.array(X_val, dtype=jnp.float32)
y_val_j = jnp.array(y_val, dtype=jnp.float32)

# Inicializar parametros con PRNG key
key = jax.random.PRNGKey(42)
params_jax = init_params([2, 32, 32, 1], key)

# Prueba rapida con datos JAX
y_pred_test = forward_jax(params_jax, X_tr_j[:5])
loss_initial = bce_loss_jax(params_jax, X_tr_j, y_tr_j)
print(f"Predicciones iniciales (primeras 5): {y_pred_test}")
print(f"Perdida inicial: {float(loss_initial):.4f}")
```

Predicciones iniciales (primeras 5): [0.6773608 0.5984973 0.6005721 0.9004446 0.6705811]
 Perdida inicial: 0.9761

146.8.5. Paso de entrenamiento con JIT compilation

146.8.5.1. Diferencia filosofica: PyTorch vs JAX

PyTorch:

```
for epoch in range(n_epochs):
    optimizer.zero_grad()
    logits = model(X_train)
    loss = criterion(logits, y_train)
    loss.backward()
    optimizer.step()
```

- El bucle es imperativo — instrucciones paso a paso.
- El estado del modelo y optimizador cambia en cada paso.
- Facil de debuggear porque el estado es observable en cualquier momento.

JAX:

```
for epoch in range(n_epochs):
    params, opt_state, loss = train_step(params, opt_state, X_train, y_train)
```

- Todo el paso está encapsulado en una funcion pura que retorna nuevos valores.
- Ningun estado mutable — los parametros y estado del optimizador son argumentos de entrada y salida.
- Se puede compilar con JIT para velocidad.

146.8.5.2. El paso de entrenamiento JAX

```
# Optimizador funcional - mantiene su estado externamente
optimizer_jax = optax.adam(learning_rate=0.01)
opt_state_jax = optimizer_jax.init(params_jax) # Estado inicial del optimizador

# Decorador @jax.jit: compila esta función con XLA
@jax.jit
def train_step_jax(params, opt_state, x, y):
    """
    Un paso de entrenamiento JAX.

    Nota: Esta función es PURA - no modifica nada externamente.
    Retorna nuevos valores para params, opt_state y la perdida.
    """
    # Paso 1: Calcular perdida Y gradientes simultaneamente
    # jax.value_and_grad retorna (pérdida_escalar, gradientes_como_pytree)
    loss, grads = jax.value_and_grad(bce_loss_jax)(params, x, y)
```

```

# Paso 2: Obtener updates del optimizador
# El optimizador lee los gradientes y retorna direcciones de actualización
# (updates pueden ser diferentes de gradientes - Adam, por ejemplo, modifica los gradientes)
updates, new_opt_state = optimizer_jax.update(grads, opt_state, params)

# Paso 3: Aplicar updates a los parametros
# new_params = params + updates (aproximadamente)
new_params = optax.apply_updates(params, updates)

# Retornar nuevos valores - NADA se modificó internamente
return new_params, new_opt_state, loss

# Prueba: un paso de entrenamiento
params_test, state_test, loss_test = train_step_jax(
    params_jax, opt_state_jax, X_tr_j, y_tr_j
)
print(f"Perdida despues de 1 paso: {float(loss_test):.4f}")

```

Perdida despues de 1 paso: 0.9761

146.8.5.3. Desglose línea por línea

```
loss, grads = jax.value_and_grad(bce_loss_jax)(params, x, y)
```

Esto es equivalente a lo que PyTorch hace en dos llamadas separadas:

```

# PyTorch
loss = criterion(logits, y_train) # Paso 1: calcular pérdida
loss.backward()                  # Paso 2: calcular gradientes (acumula en .grad)

# JAX - en una sola operación
loss, grads = jax.value_and_grad(bce_loss_jax)(params, x, y)
# loss: escalar
# grads: pytree con misma estructura que params, pero contiene gradientes

```

```
updates, new_opt_state = optimizer_jax.update(grads, opt_state, params)
```

El optimizador Lee los gradientes e internamente mantiene momentos (moving averages). Para Adam, computa:

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment}) \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment}) \\
 \text{update} &= \frac{m_t}{\sqrt{v_t} + \epsilon}
 \end{aligned}$$

Esto es todo lo que PyTorch hace en `optimizer.step()`, pero de forma explícita y retornando los valores.

```
new_params = optax.apply_updates(params, updates)
```


Aplica los updates:

$$\theta_{\text{new}} = \theta_{\text{old}} + \text{learning_rate} \times \text{updates}$$

146.8.5.4. ¿Por qué @jax.jit?

El decorador `@jax.jit` compila la función con XLA la primera vez que se llama: - **Primera llamada:** JAX traza la función, observa las formas de entrada, y genera código XLA optimizado. Lento — puede tomar varios segundos. - **Llamadas subsiguientes con mismas formas:** Reutilizan el código compilado. Mucho más rápido.

Para datos de tamaño fijo (como en este ejemplo), la compilación ocurre una sola vez y los 299 epochs siguientes son significativamente más rápidos que en PyTorch eager.

Cuando @jax.jit sorprende — tracing y efectos secundarios

Dentro de una función JIT-compilada, `print()` **no funciona como esperas**. JAX traza la función estáticamente, y `print` solo se ejecuta una vez durante el trazado, no en cada llamada:

```
@jax.jit
def myfn(x):
    print("hola")    # Solo se imprime durante la compilación, no en cada llamada
    return x ** 2

myfn(jnp.array(2.0)) # Imprime "hola" - primera vez (trazado)
myfn(jnp.array(3.0)) # NO imprime - usa código compilado
```

Para debuggear dentro de JIT, usa `jax.debug.print(...)` que sí propaga el valor real en cada ejecución.

146.8.6. Bucle de entrenamiento completo

```
# Reinicializar para partir desde cero
params_jax = init_params([2, 32, 32, 1], key)
opt_state_jax = optimizer_jax.init(params_jax)

jax_tr_losses, jax_vl_losses = [], []
jax_tr_accs, jax_vl_accs = [], []

for epoch in range(300):
    # Paso de entrenamiento (JIT-compilado)
    params_jax, opt_state_jax, tr_loss = train_step_jax(
        params_jax, opt_state_jax, X_tr_j, y_tr_j
    )

    # Metricas de validacion (sin compilar, solo para monitoreo)
    vl_loss = float(bce_loss_jax(params_jax, X_vl_j, y_vl_j))

    # Obtener predicciones (jax.device_get trae del dispositivo acelerado a CPU)
```

```

tr_preds = jax.device_get(forward_jax(params_jax, X_tr_j))
vl_preds = jax.device_get(forward_jax(params_jax, X_vl_j))

tr_acc = np.mean((tr_preds >= 0.5).astype(int) == np.array(y_tr_j))
vl_acc = np.mean((vl_preds >= 0.5).astype(int) == np.array(y_vl_j))

jax_tr_losses.append(float(tr_loss))
jax_vl_losses.append(vl_loss)
jax_tr_accs.append(tr_acc)
jax_vl_accs.append(vl_acc)

print(f"Epoch 300 | Train acc: {jax_tr_accs[-1]:.3f} | Val acc: {jax_vl_accs[-1]:.3f}")

```

Epoch 300 | Train acc: 0.520 | Val acc: 0.420

En JAX puro, cada transformacion es explícita: compilacion (`@jax.jit`), diferenciacion (`jax.grad`), vectorizacion (`jax.vmap`). Esto da total control pero requiere verbosidad. El optimizador `optax` es funcional: mantiene su estado en `opt_state` en lugar de modificar un objeto.

```

import jax
import jax.numpy as jnp
import optax
import numpy as np
from sklearn.datasets import make_moons
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ---- Data (same split as PyTorch experiment) ----
X_np, y_np = make_moons(n_samples=500, noise=0.25, random_state=42)
n_train = int(0.8 * len(X_np))
idx_jax = np.random.RandomState(42).permutation(len(X_np))
tr_idx_j, vl_idx_j = idx_jax[:n_train], idx_jax[n_train:]

X_tr_j = jnp.array(X_np[tr_idx_j], dtype=jnp.float32)
y_tr_j = jnp.array(y_np[tr_idx_j], dtype=jnp.float32)
X_vl_j = jnp.array(X_np[vl_idx_j], dtype=jnp.float32)
y_vl_j = jnp.array(y_np[vl_idx_j], dtype=jnp.float32)

# ---- Parameter initialization ----
def init_params(layer_dims, key):
    params = {"W": [], "b": []}
    for i in range(len(layer_dims) - 1):
        key, subkey = jax.random.split(key)
        fan_in = layer_dims[i]
        W = jax.random.normal(subkey, (layer_dims[i], layer_dims[i+1])) * jnp.sqrt(2.0 / fan_in)
        b = jnp.zeros(layer_dims[i+1])
        params["W"].append(W)
        params["b"].append(b)

```

```

    return params

# ---- Forward pass ----
def forward_jax(params, x):
    for i in range(len(params["W"]) - 1):
        x = x @ params["W"][i] + params["b"][i]
        x = jax.nn.relu(x)
    x = x @ params["W"][-1] + params["b"][-1]
    return jax.nn.sigmoid(x).squeeze(-1)

# ---- Loss function ----
def bce_loss_jax(params, x, y):
    yhat = forward_jax(params, x)
    eps = 1e-7
    yhat = jnp.clip(yhat, eps, 1 - eps)
    return -jnp.mean(y * jnp.log(yhat) + (1 - y) * jnp.log(1 - yhat))

# ---- Optimizer ----
optimizer_jax = optax.adam(1e-2)
key = jax.random.PRNGKey(42)
params_jax = init_params([2, 32, 32, 1], key)
opt_state_jax = optimizer_jax.init(params_jax)

# ---- JIT-compiled training step ----
@jax.jit
def train_step_jax(params, opt_state, x, y):
    loss_val, grads = jax.value_and_grad(bce_loss_jax)(params, x, y)
    updates, new_opt_state = optimizer_jax.update(grads, opt_state, params)
    new_params = optax.apply_updates(params, updates)
    return new_params, new_opt_state, loss_val

# ---- Training loop ----
jax_tr_losses, jax_vl_losses = [], []
jax_tr_accs, jax_vl_accs = [], []

for epoch in range(300):
    params_jax, opt_state_jax, tr_loss = train_step_jax(
        params_jax, opt_state_jax, X_tr_j, y_tr_j
    )

    tr_preds = jax.device_get(forward_jax(params_jax, X_tr_j))
    tr_acc = np.mean((tr_preds >= 0.5).astype(int) == np.array(y_tr_j))

    vl_preds = jax.device_get(forward_jax(params_jax, X_vl_j))
    vl_loss = float(bce_loss_jax(params_jax, X_vl_j, y_vl_j))
    vl_acc = np.mean((vl_preds >= 0.5).astype(int) == np.array(y_vl_j))

    jax_tr_losses.append(float(tr_loss))
    jax_vl_losses.append(vl_loss)

```

```

    jax_tr_accs.append(tr_acc)
    jax_vl_accs.append(vl_acc)

epochs_j = list(range(1, 301))

# ---- Figure ----
fig = make_subplots(rows=1, cols=2,
    subplot_titles=["Perdida (BCE)", "Exactitud"])

fig.add_trace(go.Scatter(x=epochs_j, y=jax_tr_losses, name="Train loss",
    line=dict(color="mediumseagreen", width=2)), row=1, col=1)
fig.add_trace(go.Scatter(x=epochs_j, y=jax_vl_losses, name="Val loss",
    line=dict(color="coral", width=2, dash="dash")), row=1, col=1)

fig.add_trace(go.Scatter(x=epochs_j, y=jax_tr_accs, name="Train acc",
    line=dict(color="mediumseagreen", width=2), showlegend=False), row=1, col=2)
fig.add_trace(go.Scatter(x=epochs_j, y=jax_vl_accs, name="Val acc",
    line=dict(color="coral", width=2, dash="dash"), showlegend=False), row=1, col=2)

fig.update_xaxes(title_text="Epoch")
fig.update_yaxes(title_text="BCE Loss", row=1, col=1)
fig.update_yaxes(title_text="Accuracy", row=1, col=2)

fig.update_layout(
    title=dict(text="Entrenamiento del MLP en JAX puro (make_moons)", x=0.5, font=dict(size=14)),
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.12.: Curvas de perdida y exactitud durante el entrenamiento del MLP en JAX puro sobre make_moons.

Las curvas de entrenamiento en JAX son practicamente identicas a las de PyTorch bajo las mismas condiciones: misma arquitectura, mismo optimizador (Adam con `lr=0.01`), mismo conjunto de datos y misma particion. Esto ilustra que los dos frameworks, a pesar de sus filosofias radicalmente distintas, convergen a resultados equivalentes cuando los hiperparametros son los mismos. La diferencia observable es que la primera llamada a `train_step_jax` es mas lenta (trazado y compilacion JIT), pero las 299 iteraciones siguientes son sustancialmente mas rapidas que su equivalente en PyTorch eager.

La frontera de decision aprendida por la red JAX deberia ser visualmente indistinguible de la del MLP de PyTorch, confirmando que el aprendizaje de representaciones no esta ligado al framework sino a la arquitectura y los datos.

```

import plotly.graph_objects as go
import numpy as np
import jax

```

```

import jax.numpy as jnp

# Build grid
x1r = np.linspace(X_np[:, 0].min() - 0.5, X_np[:, 0].max() + 0.5, 200)
x2r = np.linspace(X_np[:, 1].min() - 0.5, X_np[:, 1].max() + 0.5, 200)
xx1j, xx2j = np.meshgrid(x1r, x2r)
grid_j = jnp.array(np.c_[xx1j.ravel(), xx2j.ravel()], dtype=jnp.float32)

probs_j = jax.device_get(forward_jax(params_jax, grid_j)).reshape(200, 200)
final_vl_acc_j = jax_vl_accs[-1]

fig = go.Figure()

fig.add_trace(go.Contour(
    x=x1r, y=x2r, z=probs_j.tolist(),
    colorscale="Teal",
    opacity=0.75,
    showscale=True,
    colorbar=dict(title="P(clase=1)"),
    contours=dict(start=0, end=1, size=0.05, showlines=False)
))

fig.add_trace(go.Contour(
    x=x1r, y=x2r, z=probs_j.tolist(),
    showscale=False,
    contours=dict(start=0.5, end=0.5, size=0, coloring="lines"),
    line=dict(color="white", width=3),
    name="Frontera (p=0.5)"
))

fig.add_trace(go.Scatter(
    x=X_np[tr_idx_j, 0], y=X_np[tr_idx_j, 1],
    mode="markers",
    marker=dict(
        color=y_np[tr_idx_j].tolist(),
        colorscale=[[0, "#636EFA"], [1, "#EF553B"]],
        size=6, line=dict(color="white", width=0.5)
    ),
    name="Puntos de entrenamiento"
))

fig.add_annotation(
    x=0.02, y=0.97, xref="paper", yref="paper",
    text=f"Val accuracy: {final_vl_acc_j:.3f}",
    showarrow=False,
    font=dict(size=12, color="white"),
    bgcolor="rgba(0,0,0,0.5)",
    bordercolor="white", borderwidth=1
)

```

```
fig.update_layout(  
    title=dict(text="Frontera de decision del MLP entrenado con JAX", x=0.5, font=dict(size=14)),  
    xaxis_title="x1", yaxis_title="x2",  
    height=480,  
    legend=dict(orientation="h", yanchor="bottom", y=-0.15, xanchor="center", x=0.5)  
)  
fig.show()
```

Unable to display output for mime type(s): text/html

Figura 146.13.: Frontera de decision del MLP entrenado con JAX puro sobre make_moons. La isocurva blanca corresponde a probabilidad 0.5.

La frontera aprendida por JAX captura la misma estructura curvilinea que la de PyTorch. Las pequenas diferencias visuales, si las hay, se deben a diferencias en la inicializacion aleatoria (JAX usa claves PRNG explicitas mientras que PyTorch usa un generador global) y posiblemente a diferencias en el orden de las operaciones de punto flotante producidas por XLA versus la cadena de herramientas de PyTorch. En ninguno de los dos casos estas diferencias son significativas para la calidad del clasificador final.

146.9. CNN con Flax

146.9.1. flax.linen.Module

Flax (Heek et al. 2023) ofrece una API de módulos sobre JAX que es **superficialmente** similar a `nn.Module` de PyTorch, pero mantiene la disciplina funcional en su núcleo.

La diferencia clave: una instancia de `flax.linen.Module` **no almacena parámetros**. Es solo una descripción del cómputo. Los parámetros se obtienen con `model.init(key, x)` y se pasan explícitamente en cada `model.apply(params, x)`.

El decorador `@nn.compact` permite declarar las capas inline en `__call__` en lugar de en `__init__`. Flax nombra automáticamente cada capa según su posición, produciendo claves como "Conv_0", "Dense_1".

Error frecuente al portar código PyTorch → Flax: el orden de canales

PyTorch espera (N, C, H, W) — canales primero. JAX/Flax espera (N, H, W, C) — canales último.

Framework	Formato	Reshape correcto
PyTorch	(N, 1, 8, 8)	<code>.reshape(-1, 1, 8, 8)</code>
JAX/Flax	(N, 8, 8, 1)	<code>.reshape(-1, 8, 8, 1)</code>

Si olvidas este cambio, obtendrás un error de forma confuso en `nn.Conv` porque el número de canales no coincide con el esperado.

```

import jax
import jax.numpy as jnp
import optax
import numpy as np
import flax.linen as fnn
from sklearn.datasets import load_digits
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ---- Data: channels-last (N, H, W, C) for Flax ----
digits = load_digits()
X_dig_f = digits.data.astype(np.float32) / 16.0
y_dig_f = digits.target.astype(np.int32)

rng_f = np.random.RandomState(42)
idx_f = rng_f.permutation(len(X_dig_f))
n_tr_f = int(0.8 * len(X_dig_f))
tr_f, vl_f = idx_f[:n_tr_f], idx_f[n_tr_f:]

# Channels-last: (N, 8, 8, 1)
X_tr_f = jnp.array(X_dig_f[tr_f].reshape(-1, 8, 8, 1))
y_tr_f = jnp.array(y_dig_f[tr_f])
X_vl_f = jnp.array(X_dig_f[vl_f].reshape(-1, 8, 8, 1))
y_vl_f = jnp.array(y_dig_f[vl_f])

# ---- Flax CNN definition ----
class FlaxCNN(fnn.Module):
    @fnn.compact
    def __call__(self, x):
        x = fnn.Conv(features=16, kernel_size=(3, 3), padding="SAME")(x)
        x = fnn.relu(x)
        x = fnn.avg_pool(x, window_shape=(2, 2), strides=(2, 2))
        x = fnn.Conv(features=32, kernel_size=(3, 3), padding="SAME")(x)
        x = fnn.relu(x)
        x = fnn.avg_pool(x, window_shape=(2, 2), strides=(2, 2))
        x = x.reshape((x.shape[0], -1)) # flatten
        x = fnn.Dense(64)(x)
        x = fnn.relu(x)
        x = fnn.Dense(10)(x)
        return x

# ---- Initialize model ----
key_f = jax.random.PRNGKey(42)
model_flax = FlaxCNN()
params_flax = model_flax.init(key_f, X_tr_f[:1])

# ---- Optimizer ----
optimizer_flax = optax.adam(1e-3)
opt_state_flax = optimizer_flax.init(params_flax)

```

```

# ---- JIT-compiled training step ----
@jax.jit
def train_step_flax(params, opt_state, x, y):
    def loss_fn(params):
        logits = model_flax.apply(params, x)
        return optax.softmax_cross_entropy_with_integer_labels(logits, y).mean()
    loss_val, grads = jax.value_and_grad(loss_fn)(params)
    updates, new_opt_state = optimizers.update(grads, opt_state, params)
    new_params = optax.apply_updates(params, updates)
    return new_params, new_opt_state, loss_val

# ---- Training loop ----
flax_tr_losses, flax_vl_losses = [], []
flax_tr_accs, flax_vl_accs = [], []

for epoch in range(50):
    params_flax, opt_state_flax, tr_loss_f = train_step_flax(
        params_flax, opt_state_flax, X_tr_f, y_tr_f
    )

    # Validation metrics (use jax.device_get before numpy operations)
    tr_logits_f = jax.device_get(model_flax.apply(params_flax, X_tr_f))
    tr_acc_f = np.mean(tr_logits_f.argmax(axis=1) == np.array(y_tr_f))

    vl_logits_f = jax.device_get(model_flax.apply(params_flax, X_vl_f))
    vl_loss_f = float(
        optax.softmax_cross_entropy_with_integer_labels(
            jnp.array(vl_logits_f), y_vl_f
        ).mean()
    )
    vl_acc_f = np.mean(vl_logits_f.argmax(axis=1) == np.array(y_vl_f))

    flax_tr_losses.append(float(tr_loss_f))
    flax_vl_losses.append(vl_loss_f)
    flax_tr_accs.append(tr_acc_f)
    flax_vl_accs.append(vl_acc_f)

epochs_f = list(range(1, 51))

# ---- Figure ----
fig = make_subplots(rows=1, cols=2,
    subplot_titles=["Perdida (Cross-Entropy)", "Exactitud"])

fig.add_trace(go.Scatter(x=epochs_f, y=flax_tr_losses, name="Train loss",
    line=dict(color="darkorchid", width=2)), row=1, col=1)
fig.add_trace(go.Scatter(x=epochs_f, y=flax_vl_losses, name="Val loss",
    line=dict(color="goldenrod", width=2, dash="dash")), row=1, col=1)

fig.add_trace(go.Scatter(x=epochs_f, y=flax_tr_accs, name="Train acc",

```



```

    line=dict(color="darkorchid", width=2), showlegend=False), row=1, col=2)
fig.add_trace(go.Scatter(x=epochs_f, y=flax_vl_accs, name="Val acc",
    line=dict(color="goldenrod", width=2, dash="dash"), showlegend=False), row=1, col=2)

fig.update_xaxes(title_text="Epoch")
fig.update_yaxes(title_text="Cross-Entropy Loss", row=1, col=1)
fig.update_yaxes(title_text="Accuracy", row=1, col=2)

fig.update_layout(
    title=dict(text="Entrenamiento de la CNN con Flax/JAX (digits)", x=0.5, font=dict(size=14)),
    height=400,
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5)
)
fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.14.: Curvas de perdida y exactitud durante el entrenamiento de la CNN con Flax/JAX sobre el conjunto digits.

La CNN de Flax alcanza una exactitud de validacion comparable a la version de PyTorch en el mismo numero de epochs, lo que confirma que la capacidad del modelo es equivalente. Una diferencia observable es que usamos `avg_pool` en lugar de `MaxPool2d`: el average pooling es ligeramente menos agresivo en la seleccion de características pero igualmente efectivo para reducir la dimension espacial. En redes mas profundas, la eleccion entre max y average pooling puede tener un impacto measurable en el rendimiento.

El flujo de trabajo con Flax merece un comentario adicional. La llamada `model_flax.apply(params_flax, x)` es una funcion pura: produce la misma salida para los mismos parametros y la misma entrada, sin ninguna dependencia de estado oculto. Esto facilita la verificacion formal, el testing y la reproduccion de resultados. El precio es la verbosidad: hay que pasar `params_flax` explicitamente en cada llamada, lo que contrasta con el `model(x)` de PyTorch donde los parametros estan implicitamente encapsulados en el objeto.

146.10. Comparativa PyTorch vs JAX

146.10.1. Filosofia de diseno: Imperative vs Functional

La diferencia mas profunda entre PyTorch y JAX no es tecnica sino **filosofica de programacion**. Estos dos frameworks encarnan dos paradigmas completamente diferentes.

146.10.1.1. PyTorch: Programacion Imperativa Orientada a Objetos

PyTorch sigue el paradigma imperativo: escribes instrucciones paso a paso sobre objetos que tienen estado mutable.

```

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(10, 5) # Objeto que contiene parametros
        self.layer2 = nn.Linear(5, 1)

model = MyModel() # Instancia
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    optimizer.zero_grad() # Modificar estado: limpiar gradientes
    loss = criterion(model(X), y) # Obtener predicción del objeto
    loss.backward() # Modificar estado: llenar .grad
    optimizer.step() # Modificar estado: actualizar weights
    # Después de cada paso, model.layer1.weight ha cambiado

print(model.layer1.weight) # Inspecciona en cualquier momento - fácil de debuggear

```

Ventajas: - Ergonomía excelente — el código se lee como una serie natural de instrucciones. - Debugging fácil — inspecciona el estado en cualquier momento con `print` o un debugger. - Curva de aprendizaje suave para programadores acostumbrados a OOP.

Desventajas: - No es composable con transformaciones automáticas (diferenciar través de múltiples pasos es complicado). - El estado mutable hace difícil razonar sobre que código hace que.

146.10.1.2. JAX: Programacion Funcional Pura

JAX implementa programacion funcional: las funciones no tienen estado, siempre retornan lo mismo para los mismos argumentos, y se pueden transformar de forma automática.

```

# Los parámetros son datos (diccionarios), no objetos con estado
params = {
    "layer1_W": jnp.array(...),
    "layer1_b": jnp.array(...),
    "layer2_W": jnp.array(...),
    "layer2_b": jnp.array(...)
}

def forward(params, x):
    x = jax.nn.relu(x @ params["layer1_W"] + params["layer1_b"])
    return x @ params["layer2_W"] + params["layer2_b"]

# Entrenamiento: todos los pasos retornan nuevos valores
optimizer_state = optimizer.init(params)

for epoch in range(100):
    loss, grads = jax.value_and_grad(lambda p: loss_fn(forward(p, X), y))(params)
    updates, optimizer_state = optimizer.update(grads, optimizer_state, params)
    params = optax.apply_updates(params, updates)

```

```
# params es NUEVO - el antiguo sigue existiendo si lo guardaste

# Para inspeccionar, accedes a params como diccionario
print(params["layer1_W"])
```

Ventajas: - Composición de transformaciones — puedes anidar `jax.grad`, `jax.jit`, `jax.vmap` libremente. - Compilación automática con XLA — velocidad. - Determinismo garantizado — mismos argumentos → mismo resultado. - Paralelización automática posible.

Desventajas: - Curva de aprendizaje más empinada — requiere cambiar tu forma de pensar. - Debugging más difícil — no puedes poner `print` dentro de `@jax.jit`. - Más verboso — tienes que ser explícito con el estado (parámetros, keys PRNG, etc).

146.10.1.3. Tabla comparativa de mentalidad

Aspecto	PyTorch	JAX
Modelo	Objeto con estado mutable	Función pura + datos externos
Parámetros	Encapsulados en <code>model.parameters()</code>	Datos explícitos (pytrees)
Ciclo de entrenamiento	Instrucciones imperativas	Transformación de datos
Debugging	Trivial — inspecciona en cualquier punto	Difícil dentro de <code>@jax.jit</code>
Composición	Limitada	Ilimitada
Velocidad	Eager por defecto, JIT opcional	JIT por defecto
Uso típico	La mayoría de practicantes	Investigación, TPUs, órdenes altas

146.10.2. Flujo de trabajo típico

Veamos las operaciones mas comunes lado a lado.

146.10.2.1. 1. Definición del modelo

PyTorch:

```
class MyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(10, 5)
        self.fc2 = nn.Linear(5, 1)

    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

model = MyNet() # Los parámetros se crean automáticamente
```

JAX / Flax:

```
class MyNet(fnn.Module):
    @fnn.compact
    def __call__(self, x):
        x = fnn.Dense(5)(x)
        x = fnn.relu(x)
        x = fnn.Dense(1)(x)
        return x

model = MyNet()
params = model.init(key, x_example) # Los parámetros se crean explícitamente
```

146.10.2.2. 2. Forward pass

PyTorch:

```
output = model(x) # Los parámetros están dentro del modelo
```

JAX / Flax:

```
output = model.apply(params, x) # Los parámetros se pasan explícitamente
```

146.10.2.3. 3. Cálculo de gradientes

PyTorch:

```
loss = criterion(model(x), y)
loss.backward() # Gradientes se acumulan en model.parameters()[...].grad
# Acceso: for p in model.parameters(): print(p.grad)
```

JAX:

```
def loss_fn(params):
    logits = model.apply(params, x)
    return criterion(logits, y)

grads = jax.grad(loss_fn)(params) # Retorna un pytree de gradientes
# grads tiene la misma estructura que params
```

146.10.2.4. 4. Optimizador

PyTorch:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
# El optimizador mantiene estado internamente (momentos de Adam)
# optimizer.zero_grad() y optimizer.step() modifican el modelo
```

JAX:

```
optimizer = optax.adam(learning_rate=0.001)
opt_state = optimizer.init(params) # Estado del optimizador debe ser trackeo
# optimizer.update() retorna nuevos updates y nuevo opt_state
```

146.10.2.5. 5. Compilacion

PyTorch:

```
model = MyNet()
# Ejecuta en eager por defecto (traza cada operación)
output = model(x) # Rápido para prototyping, más lento que compilado

# Opcional:
model = torch.compile(model) # Intenta compilar con compiler nativo
output = model(x) # Potencialmente más rápido
```

JAX:

```
model = MyNet()

@jax.jit
def forward(params, x):
    return model.apply(params, x)

# Primera llamada: JAX traza y compila con XLA (lento)
output = forward(params, x)
# Llamadas subsiguientes: ejecutan código compilado (rápido)
output = forward(params, x) # Significativamente más rápido
```

146.10.2.6. 6. Debugging

PyTorch — fácil:

```
model = MyNet()
x = torch.randn(32, 10)

# Inspeccionar en cualquier punto
print(f"Input shape: {x.shape}")
h = model.fc1(x)
print(f"After fc1: {h.shape}") # ← Puedes inspeccionar aquí
# Puedes insertar breakpoints debugger sin problemas
```

JAX — más complejo:

```
@jax.jit
def forward(params, x):
    x = fnn.Dense(5)(x)
    print(x.shape) # ← NO FUNCIONA dentro de jit!
    return x

# Solución: extraer funciones sin jit para debugging
def forward_debug(params, x):
    x = model.apply(params, x)
    print(x.shape) # Funciona aquí
    return x

forward_debug(params, x) # Debugging
```

146.10.2.7. Resumen de flujos de trabajo

Operacion	PyTorch	JAX
Crear modelo	MyNet()	MyNet() + model.init(key, x)
Forward	model(x)	model.apply(params, x)
Gradientes	loss.backward()	jax.grad(loss_fn)(params)
Optimizador	optimizer.step()	optimizer.update(grads, state)
Compilacion	torch.compile() (opcional)	@jax.jit (automática internamente)
Debugging	Trivial	Requiere técnicas especiales

146.10.2.8. ¿Cuál elegir?

Regla práctica para elegir framework**Empieza con PyTorch si:**

- Estás aprendiendo aprendizaje profundo por primera vez.
- Necesitas modelos preentrenados (HuggingFace Transformers, torchvision).
- Quieres debuggear con `print()` y puntos de ruptura normales.
- Trabajas en equipo donde PyTorch ya es el estándar.

Considera JAX si:

- Necesitas diferenciación de orden superior (Hessiano, derivadas de derivadas).
- Quieres vectorización automática eficiente con `jax.vmap`.
- Trabajas con TPUs de Google Cloud (JAX tiene integración nativa).
- Investigas meta-aprendizaje, NTK o algoritmos que diferencian a través del entrenamiento.

La realidad del mercado (2025): ~80 % del código de investigación publicado usa PyTorch. JAX es dominante en DeepMind/Google y en investigación teórica. Para la industria general, PyTorch es la elección casi universal.

146.10.3. Comparativa cuantitativa

La siguiente figura presenta una comparacion subjetiva pero estructurada de las dos opciones principales en seis dimensiones relevantes para el practicante de aprendizaje profundo.

```
import plotly.graph_objects as go

metrics = [
    "Curva de aprendizaje",
    "Velocidad (JIT)",
    "Flexibilidad de arqu.",
    "Tamano del ecosistema",
    "Facilidad de debugging",
    "Soporte para investigacion"
]

pytorch_scores = [4, 3, 5, 5, 5, 5]
jax_scores      = [2, 5, 5, 3, 2, 5]

fig = go.Figure()

fig.add_trace(go.Bar(
    name="PyTorch",
    x=metrics,
    y=pytorch_scores,
    marker_color="steelblue",
    text=pytorch_scores,
    textposition="outside"
))

fig.add_trace(go.Bar(
```

```

    name="JAX / Flax",
    x=metrics,
    y=jax_scores,
    marker_color="mediumseagreen",
    text=jax_scores,
    textposition="outside"
))

fig.update_layout(
    barmode="group",
    title=dict(
        text="PyTorch vs JAX: comparativa de características",
        x=0.5, font=dict(size=14)
    ),
    yaxis=dict(title="Puntuacion (1-5)", range=[0, 6.5]),
    xaxis=dict(tickfont=dict(size=11)),
    legend=dict(orientation="h", yanchor="bottom", y=-0.25, xanchor="center", x=0.5),
    height=450,
    margin=dict(t=60, b=100)
)

fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.15.: Comparativa de PyTorch versus JAX en seis dimensiones de interes practico (escala 1-5).

La comparativa revela los patrones complementarios de los dos frameworks. PyTorch destaca en facilidad de debugging, tamaño del ecosistema y accesibilidad para nuevos usuarios (curva de aprendizaje mas suave). JAX destaca en velocidad de ejecucion gracias a la compilacion XLA, que produce codigo significativamente mas rapido en hardware especializado. Ambos frameworks son equivalentes en flexibilidad de arquitecturas (cualquier computo diferenciable es expresable en ambos) y soporte para investigacion (las dos opciones son ampliamente usadas en publicaciones de primer nivel). La puntuacion baja de JAX en “Curva de aprendizaje” no refleja una deficiencia del framework sino la naturaleza del paradigma funcional, que requiere un cambio conceptual sustancial para programadores acostumbrados al estilo imperativo.

146.10.4. Convergencia directa: PyTorch vs JAX

La prueba definitiva de que ambos frameworks son equivalentes en capacidad expresiva es comparar sus curvas de convergencia sobre el mismo problema, con la misma arquitectura y los mismos hiperparametros. La siguiente figura superpone la exactitud de validacion de los dos modelos MLP entrenados a lo largo de este capitulo — el de PyTorch (azul solido) y el de JAX puro (verde discontinuo) — sobre `make_moons` con arquitectura $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$, optimizador Adam con $\eta = 0.01$ y 300 epocas.

```

import plotly.graph_objects as go

fig = go.Figure()

```



```

fig.add_trace(go.Scatter(
    x=list(range(1, len(val_accs) + 1)),
    y=val_accs,
    name="PyTorch",
    line=dict(color="steelblue", width=2.5)
))

fig.add_trace(go.Scatter(
    x=list(range(1, len(jax_vl_accs) + 1)),
    y=jax_vl_accs,
    name="JAX puro",
    line=dict(color="mediumseagreen", width=2.5, dash="dash")
))

pt_final = val_accs[-1]
jax_final = jax_vl_accs[-1]

fig.add_annotation(
    x=len(val_accs) - 1, y=pt_final,
    text=f"PyTorch final: {pt_final:.3f}",
    showarrow=True, arrowhead=2, ax=-70, ay=-30,
    font=dict(color="steelblue", size=11)
)

fig.add_annotation(
    x=len(jax_vl_accs) - 1, y=jax_final,
    text=f"JAX final: {jax_final:.3f}",
    showarrow=True, arrowhead=2, ax=-70, ay=30,
    font=dict(color="mediumseagreen", size=11)
)

fig.update_layout(
    title=dict(
        text="PyTorch vs JAX: exactitud de validacion (make_moons, misma arquitectura)",
        x=0.5, font=dict(size=14)
    ),
    xaxis_title="Epoch",
    yaxis_title="Exactitud de validacion",
    yaxis=dict(range=[0.5, 1.02]),
    legend=dict(orientation="h", yanchor="bottom", y=-0.22, xanchor="center", x=0.5),
    height=410,
    margin=dict(t=60, b=80)
)

fig.show()

```

Unable to display output for mime type(s): text/html

Figura 146.16.: Comparacion directa de exactitud de validacion entre el MLP de PyTorch y el MLP de JAX puro sobre `make_moons`. Ambos modelos convergen al mismo nivel de rendimiento, confirmando que la eleccion del framework no afecta la capacidad del modelo sino unicamente el paradigma de programacion y las posibilidades de optimizacion del sistema.

146.11. Ejercicios

Los siguientes ejercicios están diseñados para profundizar tu comprensión de PyTorch y JAX experimentando con variaciones de arquitecturas e hiperparámetros. El objetivo no es solo “hacer que funcione”, sino entender **qué sucede** cuando cambias cosas.

146.11.1. Ejercicio 1: Sensibilidad a la tasa de aprendizaje

Concepto: La tasa de aprendizaje η es el hiperparámetro más crítico. Controla qué tan grandes son los pasos que toma el optimizador: - Demasiado bajo $\rightarrow$ Convergencia lentísima (podrías esperar para siempre). - Demasiado alto $\rightarrow$ Oscilaciones o divergencia (los pesos saltan sin control).

Tarea: Entrena el MLP de PyTorch con cuatro tasas diferentes: `lr` [0.0005, 0.005, 0.05, 0.5]. Grafica las curvas de `val_loss` para las cuatro en la misma figura, asegurándote de usar colores distintos para cada una.

Pasos sugeridos: 1. Define una función `train_with_lr(lr, n_epochs=300)` que devuelva las listas `val_losses`. 2. Itera sobre `learning_rates` y acumula los resultados. 3. Grafica con Plotly (puedes usar `go.Scatter` y agregar múltiples trazes).

Observaciones esperadas: - Con `lr` = 0.0005: Curva plana durante muchos epochs — convergencia extremadamente lenta. - Con `lr` = 0.005 y `lr` = 0.05: Curvas suaves y decrecientes — zona dorada. - Con `lr` = 0.5: Oscilaciones erráticas o divergencia — la pérdida aumenta en lugar de disminuir.

Pregunta de reflexión: ¿Por qué Adam es más robusto a tasas de aprendizaje altas que SGD simple?

```
# Esqueleto de solución
import torch
import torch.nn as nn
import plotly.graph_objects as go

learning_rates = [0.0005, 0.005, 0.05, 0.5]
colors = ["steelblue", "mediumseagreen", "tomato", "purple"]

def train_with_lr(lr, n_epochs=300):
    """Entrena MLP con una tasa de aprendizaje específica."""
    torch.manual_seed(SEED)
    model = MLP(2, [32, 32], 1) # Reutiliza la clase MLP definida arriba
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCEWithLogitsLoss()

    val_losses = []
    for epoch in range(n_epochs):
```

```

        # TODO: implementar ciclo de entrenamiento
        # TODO: evaluar en validación
        # TODO: agregar val_loss a val_losses
        pass

    return val_losses

# TODO: Entrenar para cada lr
results = {}
for lr in learning_rates:
    results[lr] = train_with_lr(lr)

# TODO: Crear figura con go.Figure() y agregar traces
fig = go.Figure()
# for lr, color in zip(learning_rates, colors):
#     fig.add_trace(go.Scatter(x=list(range(300)), y=results[lr],
#                             name=f"lr = {lr}", line=dict(color=color, width=2)))
# fig.update_layout(xaxis_title="Epoch", yaxis_title="Val Loss", ...)
# fig.show()

```

146.11.2. Ejercicio 2: Efecto del Dropout en regularización

Concepto: Dropout es una técnica de regularización que desactiva aleatoriamente neuronas durante el entrenamiento. La idea es prevenir que el modelo “memorice” patrones raros y, en su lugar, aprenda características más robustas.

Pregunta: ¿Reduce realmente la brecha entre pérdida de entrenamiento y validación?

Tarea: Define una clase `MLPWithDropout` que agregue `nn.Dropout(p=0.3)` después de cada activación **ReLU** (excepto la última capa). Entrena tanto el MLP original como la versión con Dropout durante 300 epochs, y grafica `train_loss` y `val_loss` para ambos modelos en la misma figura.

Pasos sugeridos: 1. Define `MLPWithDropout` con la arquitectura `[2 -> 32 -> 32 -> 1]` y `Dropout(0.3)` después de cada **ReLU**. 2. Entrena ambos modelos con la misma semilla y tasa de aprendizaje. 3. Grafica las 4 curvas: `train_loss` (sin dropout), `val_loss` (sin dropout), `train_loss` (con dropout), `val_loss` (con dropout). 4. Calcula el gap final: `gap = val_loss_final - train_loss_final` para ambos modelos.

Observaciones esperadas: - **Sin Dropout:** La pérdida de entrenamiento desciende rápidamente pero el gap con validación puede ser grande (sobreajuste). - **Con Dropout:** Ambas curvas convergen más juntas porque el modelo está más regularizado.

```

class MLPWithDropout(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim, dropout_p=0.3):
        super().__init__()
        dims = [input_dim] + hidden_dims + [output_dim]
        layers = []
        for i in range(len(dims) - 1):
            layers.append(nn.Linear(dims[i], dims[i+1]))
            # Agregar Dropout después de ReLU en capas ocultas

```

```

        if i < len(dims) - 2:
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout_p)) # ← Aquí!
self.net = nn.Sequential(*layers)

def forward(self, x):
    return self.net(x)

# Función auxiliar para entrenar
def train_model(model, X_train, y_train, X_val, y_val, n_epochs=300, lr=0.01):
    """Entrena un modelo y retorna las listas de pérdidas."""
    torch.manual_seed(SEED) # Reproducibilidad
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCEWithLogitsLoss()

    train_losses, val_losses = [], []

    for epoch in range(n_epochs):
        # TODO: implementar ciclo de entrenamiento
        # Recuerda: model.train() y model.eval() para modo correcto
        pass

    return train_losses, val_losses

# TODO: Entrenar sin y con Dropout
# TODO: Graficar ambas en la misma figura
# TODO: Comparar el gap final: gap = val_loss_final - train_loss_final

```

146.11.3. Ejercicio 3: Cambiar el dataset: make_circles

Concepto: Diferentes datasets requieren diferentes arquitecturas. `make_circles` es fundamentalmente diferente a `make_moons`: los datos forman dos círculos concéntricos, no semilunares. Esto requiere capas más poderosas.

Pregunta: ¿Es suficiente una red pequeña [32, 32] o necesitamos una arquitectura más grande?

Tarea: 1. Crea el dataset: `X_c, y_c = make_circles(n_samples=600, noise=0.1, factor=0.5, random_state=42)` 2. Haz train/test split (80/20) como lo hiciste con `make_moons`. 3. Entrena tres modelos: - MLP(2, [16, 16], 1) — arquitectura pequeña - MLP(2, [32, 32], 1) — arquitectura mediana - MLP(2, [64, 64], 1) — arquitectura grande 4. Para cada modelo, calcula la exactitud final en validación. 5. Visualiza las tres fronteras de decisión lado a lado usando tres subplots.

Observaciones esperadas: - [16, 16] - Posiblemente insuficiente; el modelo podría no capturar bien la estructura radial. - [32, 32] - Debería funcionar bastante bien. - [64, 64] - Captura los círculos con mayor precisión.

```

from sklearn.datasets import make_circles

# Crear dataset
X_c, y_c = make_circles(n_samples=600, noise=0.1, factor=0.5, random_state=SEED)

```

```
# TODO: hacer train_test_split (80/20)

# TODO: Definir función para entrenar y visualizar
def train_and_visualize_circle(hidden_dims, X_train, y_train, X_val, y_val, X_full):
    """Entrena un MLP y retorna su exactitud en validación."""
    torch.manual_seed(SEED)
    model = MLP(2, hidden_dims, 1)
    # TODO: implementar entrenamiento (300 epochs)
    # TODO: retornar val_acc final

# TODO: Entrenar los tres modelos
# TODO: Crear figura con 3 subplots - uno por modelo
# TODO: En cada subplot, graficar la frontera de decisión + puntos de datos
```

146.11.4. Ejercicio 4: Derivadas de orden superior en JAX — Una exclusiva de JAX

Concepto: JAX permite componer `jax.grad` de forma ilimitada. Puedes calcular: - Primera derivada: `grad1 = jax.grad(f)` - Segunda derivada: `grad2 = jax.grad(jax.grad(f))` - Tercera, cuarta, ... — tantas como quieras.

Esto es **casi imposible** en PyTorch sin configuración especial, pero en JAX es completamente natural.

Por qué importa: Las derivadas de segundo orden (Hessiano) son fundamentales para: - Optimizadores de segundo orden (Newton-Raphson). - Análisis de curvatura del paisaje de optimización. - Meta-aprendizaje (diferenciar a través de un bucle de entrenamiento completo).

Tarea: Define la pérdida BCE con un solo peso escalar: `bce_escalar(w)` con entrada fija `x=1` y etiqueta `y=1`. Luego: 1. Calcula `grad1 = jax.grad(bce_escalar)` — primera derivada. 2. Calcula `grad2 = jax.grad(grad1)` — segunda derivada (Hessiano). 3. Evalúa las tres funciones para `w` `[-5, 5]` con 300 puntos. 4. Grafica en tres subplots: - Arriba: Pérdida $L(w)$ - Medio: Primera derivada dL/dw - Abajo: Segunda derivada d^2L/dw^2

Observaciones esperadas: - La pérdida es **convexa** — forma de U. - El gradiente es **siempre negativo** para `y=1` — descenso por gradiente siempre incrementa `w`. - La curvatura es **máxima en `w=0`** — máxima incertidumbre. - En `w → ±∞`, la curvatura se aproxima a cero — la función se vuelve plana.

```
import jax
import jax.numpy as jnp
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

def bce_escalar(w):
    """
    Binary Cross-Entropy para un solo peso escalar.
    Con entrada fija x=1 y etiqueta y=1.
    """
    sigma = jax.nn.sigmoid(w)
    return -jnp.log(jnp.clip(sigma, 1e-7, 1.0 - 1e-7))
```

```
# TODO: Definir las derivadas
# grad1 = jax.grad(bce_escalar)
# grad2 = jax.grad(grad1)

# TODO: Evaluar en rango w [-5, 5]
# w_vals = np.linspace(-5, 5, 300)
# loss_vals = [float(bce_escalar(jnp.array(w))) for w in w_vals]
# g1_vals = [float(grad1(jnp.array(w))) for w in w_vals]
# g2_vals = [float(grad2(jnp.array(w))) for w in w_vals]

# TODO: Crear figura con 3 subplots verticales
# fig = make_subplots(rows=3, cols=1,
#     subplot_titles=["Perdida L(w)", "Primera derivada dL/dw", "Segunda derivada d²L/dw²"])
# ... agregar trazes para cada subplot ...

# TODO: Reflexión: ¿Dónde es máxima la curvatura? ¿Qué significa esto para optimización?
```

Pregunta de reflexión: En un optimizador de segundo orden (como Newton-Raphson), usarías la Hessiana para dar pasos más grandes donde la curvatura es pequeña y pasos más pequeños donde es grande. ¿Tiene sentido esto intuitivamente? ¿Por qué?

146.12. Resumen

Este capítulo presentó la implementación práctica de redes neuronales en tres frameworks — scikit-learn, PyTorch y JAX+Flax — con énfasis en entender *por qué* funciona cada pieza, no solo *cómo* usarla.

PyTorch — el patrón central:

```
for epoch in range(n_epochs):
    model.train()          # activar modo entrenamiento
    optimizer.zero_grad()  # limpiar gradientes (obligatorio)
    loss = criterion(model(X), y)
    loss.backward()        # backward pass - llena .grad en todos los params
    optimizer.step()       # actualiza pesos usando .grad
```

JAX — el patrón central:

```
@jax.jit
def train_step(params, opt_state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, new_state = optimizer.update(grads, opt_state, params)
    return optax.apply_updates(params, updates), new_state, loss

for epoch in range(n_epochs):
    params, opt_state, loss = train_step(params, opt_state, X, y)
```

La diferencia filosófica que importa:

	PyTorch	JAX
Estado del modelo	Dentro del objeto	Datos externos (pytree)
Gradientes	Acumulan en <code>.grad</code>	Valores de retorno
Aleatoriedad	Generador global	PRNG key explícita
Compilación	Eager por defecto	<code>@jax.jit</code> explícito
Debugging	Trivial	Requiere <code>jax.debug</code>

Los experimentos confirman que ambos frameworks producen modelos equivalentes con la misma arquitectura y optimizador. La elección es principalmente una decisión de ergonomía y ecosistema (PyTorch) versus expresividad funcional y velocidad en hardware especializado (JAX).

Referencias

Parte XXIX.

Aprendizaje No Supervisado

Parte XXX.

Clustering

147. Introducción al clustering y métricas de evaluación

El aprendizaje supervisado parte de un conjunto de observaciones etiquetadas para aprender una función que generalice a datos nuevos. El aprendizaje no supervisado, en cambio, trabaja sin etiquetas: el objetivo es descubrir estructura interna en los datos, ya sea agrupando observaciones similares, identificando componentes latentes o reduciendo la dimensión de representación (Hastie et al. 2009; Bishop 2006).

El clustering —o agrupamiento— es la tarea no supervisada más estudiada. Dado un conjunto de observaciones $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^p$, el objetivo es particionarlas en grupos $C_1, \dots, C_k$ de modo que las observaciones dentro de cada grupo sean más similares entre sí que con las de otros grupos. Esta noción de similitud no está predefinida: el analista debe elegir una medida de distancia y un criterio de agrupamiento adecuados al problema (Aggarwal y Reddy 2013).

Este capítulo desarrolla los fundamentos del clustering: medidas de distancia y similitud, propiedades formales que deben cumplir, y métricas de evaluación que permiten comparar particiones cuando no se dispone de etiquetas verdaderas. Los capítulos siguientes desarrollan los algoritmos más importantes: K-Means, clustering jerárquico, DBSCAN y reducción de dimensionalidad no lineal.

147.1. El problema de clustering

Sea $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ un conjunto de n observaciones en $\mathbb{R}^p$. Un clustering de $\mathcal{D}$ en k grupos es una función de asignación

$$\ell : \{1, \dots, n\} \rightarrow \{1, \dots, k\},$$

donde $\ell(i)$ indica el grupo al que pertenece la observación $\mathbf{x}_i$. El grupo C_j es el subconjunto de observaciones asignadas a la etiqueta j :

$$C_j = \{\mathbf{x}_i \mid \ell(i) = j\}.$$

La función de asignación óptima depende del criterio elegido. Distintos algoritmos minimizan distintos objetivos, y no existe un único criterio universal. El número de particiones posibles en k grupos de n objetos crece exponencialmente con n , por lo que los algoritmos eficientes explotan heurísticas o restricciones geométricas.

147.2. Medidas de distancia y similitud

La noción de similitud entre observaciones se formaliza mediante funciones de distancia. Una función $d : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}_{\geq 0}$ es una distancia si satisface cuatro propiedades.

Definición. Una función d es una métrica si para todo $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^p$:

1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ y $d(\mathbf{x}, \mathbf{y}) = 0 \Leftrightarrow \mathbf{x} = \mathbf{y}$ (positividad definida),
2. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (simetría),
3. $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$ (desigualdad triangular).

147.2.1. Distancias de Minkowski

La familia de distancias de Minkowski unifica varias métricas clásicas mediante un parámetro $p \geq 1$.

Definición. La distancia de Minkowski de orden p entre $\mathbf{x} = (x_1, \dots, x_d)$ y $\mathbf{y} = (y_1, \dots, y_d)$ es

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p}. \quad (147.1)$$

Los casos más comunes son:

- $p = 1$: distancia Manhattan o L^1 , $d_1(\mathbf{x}, \mathbf{y}) = \sum_j |x_j - y_j|$.
- $p = 2$: distancia Euclídea o L^2 , $d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_j (x_j - y_j)^2}$.
- $p \rightarrow \infty$: distancia de Chebyshev, $d_\infty(\mathbf{x}, \mathbf{y}) = \max_j |x_j - y_j|$.

La elección de p determina qué tipo de diferencias entre coordenadas se penalizan más. La distancia Manhattan es menos sensible a diferencias grandes en una sola coordenada; la Euclídea las amplifica mediante el cuadrado; la de Chebyshev las ignora excepto en la coordenada de mayor diferencia.

147.2.2. Distancias para datos no vectoriales

Cuando los datos no son vectores en $\mathbb{R}^p$, se usan distancias específicas al tipo de dato.

Distancia del coseno. Para vectores $\mathbf{x}, \mathbf{y} \neq \mathbf{0}$, mide el ángulo entre ellos y es frecuente en texto:

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

Distancia de Jaccard. Para conjuntos A y B , mide la proporción de elementos que no comparten:

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}.$$

Distancia de edición (Levenshtein). Para cadenas de texto o secuencias, cuenta el número mínimo de operaciones de inserción, eliminación y sustitución para transformar una cadena en la otra.

147.2.3. Importancia de la escala

Las distancias basadas en normas son sensibles a la escala de las variables. Si una variable mide gramos y otra mide kilómetros, la segunda dominará la distancia Euclídea. Por ello, antes de aplicar algoritmos de clustering es habitual estandarizar los datos dividiendo cada variable por su desviación estándar, o aplicar otra normalización adecuada al dominio.

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import euclidean_distances, manhattan_distances, cosine_distances

rng = np.random.default_rng(42)
X = rng.normal(loc=[0, 0], scale=[1, 100], size=(50, 2)) # escala muy distinta

X_std = StandardScaler().fit_transform(X)

print("Sin escalar - distancia media:", euclidean_distances(X).mean().round(2))
print("Escalado - distancia media:", euclidean_distances(X_std).mean().round(2))
```

Sin escalar - distancia media: 81.02

Escalado - distancia media: 1.76

147.3. Evaluación de clustering sin etiquetas verdaderas

En la mayoría de las aplicaciones de clustering no se dispone de una partición verdadera contra la que comparar. Las métricas internas evalúan la calidad de la partición usando únicamente las distancias entre los puntos.

147.3.1. Índice de silueta

El índice de silueta mide, para cada observación, qué tan bien encaja en su propio grupo comparado con el grupo más cercano (Rousseeuw 1987).

Definición. Para la observación $\mathbf{x}_i$ asignada al grupo $C_{\ell(i)}$, se definen: - $a(i)$: distancia media a todos los demás puntos del mismo grupo, - $b(i)$: distancia media al grupo más cercano distinto del propio. La silueta de i es:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}. \quad (147.2)$$

El valor de $s(i)$ está en $[-1, 1]$. Un valor cercano a 1 indica que $\mathbf{x}_i$ está bien asignada: está lejos de los demás grupos. Un valor cercano a 0 indica que está en la frontera. Un valor negativo indica que $\mathbf{x}_i$ probablemente está mal asignada.

El índice de silueta global es el promedio de $s(i)$ sobre todas las observaciones:

$$S = \frac{1}{n} \sum_{i=1}^n s(i).$$

147.3.2. Índice Davies-Bouldin

El índice Davies-Bouldin mide la compacidad de cada grupo relativa a la separación entre grupos (Davies y Bouldin 1979).

Definición. Sea $\bar{d}_k$ el diámetro promedio del grupo k (distancia media de sus puntos al centroide) y $d(k, l)$ la distancia entre los centroides de los grupos k y l . El índice Davies-Bouldin es:

$$DB = \frac{1}{K} \sum_{k=1}^K \max_{l \neq k} \frac{\bar{d}_k + \bar{d}_l}{d(k, l)}. \quad (147.3)$$

Un valor menor de DB indica mejor clustering: grupos compactos y bien separados. El índice penaliza tanto la dispersión interna como la falta de separación entre grupos.

147.3.3. Índice Calinski-Harabasz

El índice Calinski-Harabasz, también llamado criterio de varianza entre/intra grupos, compara la dispersión entre grupos con la dispersión dentro de los grupos (Caliński y Harabasz 1974).

Definición. Sea SS_B la suma de cuadrados entre grupos y SS_W la suma de cuadrados dentro de grupos. El índice Calinski-Harabasz es:

$$CH = \frac{SS_B / (K - 1)}{SS_W / (n - K)}. \quad (147.4)$$

Un valor mayor de CH indica mejor clustering. Esta métrica favorece grupos compactos y bien separados, similar a Davies-Bouldin pero en sentido opuesto.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score

X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

results = []
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    results.append({
        "k": k,
        "silhouette": silhouette_score(X, labels).round(4),
        "davies_bouldin": davies_bouldin_score(X, labels).round(4),
        "calinski_harabasz": calinski_harabasz_score(X, labels).round(2),
    })

for r in results:
    print(r)
```



```
{'k': 2, 'silhouette': np.float64(0.603), 'davies_bouldin': np.float64(0.5039), 'calinski_harabas
{'k': 3, 'silhouette': np.float64(0.7783), 'davies_bouldin': np.float64(0.3336), 'calinski_haraba
{'k': 4, 'silhouette': np.float64(0.8335), 'davies_bouldin': np.float64(0.2322), 'calinski_haraba
{'k': 5, 'silhouette': np.float64(0.6976), 'davies_bouldin': np.float64(0.6251), 'calinski_haraba
{'k': 6, 'silhouette': np.float64(0.5882), 'davies_bouldin': np.float64(0.84), 'calinski_harabasz
{'k': 7, 'silhouette': np.float64(0.4487), 'davies_bouldin': np.float64(0.9949), 'calinski_haraba
```

147.4. Evaluación con etiquetas verdaderas

Cuando se dispone de etiquetas externas —por ejemplo en un experimento controlado— es posible medir cuánto se parece la partición encontrada a la partición de referencia. Estas métricas son externas.

147.4.1. Índice de Rand ajustado

El índice de Rand considera todos los pares de observaciones y cuenta cuántos pares están en el mismo grupo en ambas particiones, o en grupos distintos en ambas (Hubert y Arabie 1985).

Definición. El índice de Rand ajustado (ARI) corrige el índice de Rand por la coincidencia esperada bajo asignación aleatoria:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\text{máx}(\text{RI}) - \mathbb{E}[\text{RI}]} \quad (147.5)$$

El ARI está en $[-1, 1]$. Un valor de 1 indica concordancia perfecta; un valor de 0 indica lo esperado por azar. Se prefiere sobre el índice de Rand sin ajustar porque no crece artificialmente con n o k .

```
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

X, y_true = make_blobs(n_samples=200, centers=3, cluster_std=1.0, random_state=0)

for k in [2, 3, 4]:
    km = KMeans(n_clusters=k, random_state=0, n_init=10)
    pred = km.fit_predict(X)
    ari = adjusted_rand_score(y_true, pred)
    nmi = normalized_mutual_info_score(y_true, pred)
    print(f"k={k}  ARI={ari:.4f}  NMI={nmi:.4f}")
```

```
k=2  ARI=0.5342  NMI=0.6298
k=3  ARI=0.8428  NMI=0.7905
k=4  ARI=0.7432  NMI=0.7358
```

147.5. Preprocesamiento antes de clustering

Antes de aplicar cualquier algoritmo de clustering conviene realizar algunas verificaciones y transformaciones.

Estandarización. Variables con rangos muy distintos dominarán la distancia. Usar `StandardScaler` o `RobustScaler` antes de aplicar algoritmos basados en distancias euclídeas.

Reducción de dimensión. En espacios de alta dimensión, todas las distancias tienden a igualarse (maldición de la dimensionalidad). Aplicar PCA u otras técnicas antes de clustering puede mejorar los resultados.

Detección de valores atípicos. Los algoritmos como K-Means son sensibles a outliers porque los centroides son promedios. Conviene identificarlos y decidir si excluirllos o usar algoritmos más robustos.

Codificación de variables categóricas. Las distancias estándar no aplican directamente a variables nominales. Se pueden usar codificación one-hot con distancia euclídea, o la distancia de Gower que maneja mezclas de tipos.

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
X_raw = rng.normal(loc=[0, 0, 100, 100], scale=[1, 1, 50, 50], size=(300, 4))

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2)),
    ("km", KMeans(n_clusters=3, random_state=0, n_init=10)),
])

labels = pipeline.fit_predict(X_raw)
print("Distribución de grupos:", np.bincount(labels))
```

Distribución de grupos: [94 109 97]

147.6. Selección del número de grupos

Una de las decisiones más difíciles en clustering es el número de grupos k . No existe una respuesta única, pero varias heurísticas son útiles:

- **Método del codo:** graficar el WCSS o el índice de silueta en función de k y buscar un punto de inflexión.
- **Índice de silueta:** maximizar $S(k)$ sobre valores candidatos de k .
- **Davies-Bouldin:** minimizar $DB(k)$.
- **Calinski-Harabasz:** maximizar $CH(k)$.
- **Conocimiento del dominio:** en muchos problemas, el número razonable de grupos se puede restringir con información previa.

Ninguna métrica interna puede reemplazar la interpretación del analista. El mejor k estadístico puede no ser el más útil para el objetivo de negocio.

148. K-Means y K-Medoids

K-Means es el algoritmo de clustering más ampliamente usado en la práctica. Su popularidad se debe a su simplicidad conceptual, su eficiencia computacional y la facilidad de interpretar sus resultados: cada grupo queda representado por un centroide, que es el punto promedio de las observaciones asignadas (MacQueen 1967; Lloyd 1982).

El algoritmo alterna entre dos pasos: asignar cada punto al centroide más cercano, y actualizar los centroides como el promedio de los puntos asignados. Estos dos pasos se repiten hasta que las asignaciones no cambian. A pesar de su elegancia, K-Means tiene limitaciones importantes: converge a óptimos locales, asume que los grupos son convexos y similares en tamaño, y es sensible a valores atípicos y a la escala de las variables.

Este capítulo desarrolla el algoritmo K-Means, su análisis teórico, la inicialización inteligente K-Means++, el método del codo para elegir k , y el algoritmo K-Medoids, que es más robusto a outliers.

148.1. El objetivo de K-Means

K-Means busca una partición de n observaciones en k grupos $C_1, \dots, C_k$ que minimice la suma de cuadrados intra-grupo, también llamada WCSS (within-cluster sum of squares).

Definición. El objetivo de K-Means es minimizar

$$J(C_1, \dots, C_k) = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mu_j\|^2, \quad (148.1)$$

donde $\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$ es el centroide del grupo j .

Minimizar J sobre todas las particiones posibles es un problema NP-complejo cuando k y p son arbitrarios. El algoritmo de Lloyd encuentra un mínimo local mediante dos pasos de coordinación alternada.

148.2. Algoritmo de Lloyd

Algoritmo K-Means (Lloyd).

Entrada: observaciones $\mathbf{x}_1, \dots, \mathbf{x}_n$, número de grupos k .

1. Inicializar k centroides $\mu_1, \dots, \mu_k$ (por ejemplo, eligiendo k observaciones al azar).
2. **Asignación.** Para cada i , asignar $\ell(i) = \arg \min_j \|\mathbf{x}_i - \mu_j\|^2$.
3. **Actualización.** Para cada j , actualizar $\mu_j = \frac{1}{|C_j|} \sum_{\ell(i)=j} \mathbf{x}_i$.
4. Repetir los pasos 2 y 3 hasta que las asignaciones no cambien.

Salida: asignación ℓ y centroides finales $\mu_1, \dots, \mu_k$.

Cada iteración del algoritmo reduce o mantiene J , y como el número de particiones posibles es finito, el algoritmo converge en un número finito de iteraciones. Sin embargo, no se garantiza convergencia al mínimo global.

148.2.1. Complejidad computacional

El paso de asignación requiere $O(nk)$ cálculos de distancia por iteración. El paso de actualización requiere $O(n)$. En la práctica, K-Means converge en pocas iteraciones relativas a n . Para datos en alta dimensión o k grande, existen variantes más eficientes basadas en estructuras de árbol o aproximaciones.

148.3. Inicialización K-Means++

La solución encontrada por K-Means depende fuertemente de la inicialización. Una inicialización aleatoria puede dar lugar a centroides iniciales mal distribuidos, lo que lleva a soluciones de baja calidad. K-Means++ (Arthur y Vassilvitskii 2007) proporciona una inicialización probabilística que garantiza una solución esperada dentro de un factor logarítmico del óptimo global.

Algoritmo K-Means++.

1. Elegir el primer centroide μ_1 uniformemente al azar entre $\mathbf{x}_1, \dots, \mathbf{x}_n$.
2. Para $j = 2, \dots, k$:
 - Para cada punto $\mathbf{x}_i$, calcular $D(\mathbf{x}_i) = \min_{l < j} \|\mathbf{x}_i - \mu_l\|^2$.
 - Elegir $\mu_j = \mathbf{x}_i$ con probabilidad proporcional a $D(\mathbf{x}_i)$.
3. Continuar con el algoritmo de Lloyd estándar.

La idea es que los centroides iniciales estén bien distribuidos: cada centroide nuevo se elige con mayor probabilidad en zonas alejadas de los centroides ya elegidos. `scikit-learn` usa K-Means++ por defecto.

148.4. Limitaciones de K-Means

K-Means tiene varios supuestos implícitos que, cuando no se cumplen, producen clustering de baja calidad:

Grupos convexos y esféricos. K-Means minimiza distancias Euclídeas, lo que favorece grupos circulares en 2D o elipsoidales alineados con los ejes en alta dimensión. Grupos con formas no convexas —anillos, medias lunas, espirales— no pueden segmentarse correctamente.

Grupos de tamaño similar. El centroide de un grupo pequeño puede ser “absorbido” por un grupo grande. Esto puede mitigarse con K-Medoids o con restricciones de tamaño.

Sensibilidad a outliers. El centroide es la media, que puede desplazarse considerablemente ante un solo punto extremo. K-Medoids es más robusto porque usa medianas en vez de medias.

Número de grupos fijo. K-Means requiere especificar k de antemano. La elección adecuada de k es un problema abierto que se aborda con heurísticas como el método del codo.

148.5. Método del codo

Una heurística clásica para elegir k es graficar el WCSS en función de k y buscar un codo: el punto donde la reducción marginal de WCSS disminuye al agregar otro grupo.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import numpy as np

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.7, random_state=7)

wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=0, n_init=10)
    km.fit(X)
    wcss.append(km.inertia_)

for k, w in enumerate(wcss, 1):
    print(f"k={k:2d}   WCSS={w:10.2f}")
```

```
k= 1  WCSS=  21990.36
k= 2  WCSS=   8805.13
k= 3  WCSS=   2236.26
k= 4  WCSS=    279.94
k= 5  WCSS=    250.42
k= 6  WCSS=    223.32
k= 7  WCSS=    197.53
k= 8  WCSS=    171.05
k= 9  WCSS=    154.82
k=10  WCSS=    140.95
```

148.6. K-Medoids (PAM)

K-Medoids es una variante de K-Means que, en vez de representar cada grupo por su media, lo representa por una observación real llamada medoide (Kaufman y Rousseeuw 1990). Esto lo hace más robusto a outliers, porque los medoides son puntos del conjunto de datos y no pueden ser arrastrados por valores extremos.

Definición. Un medoide del grupo C_j es la observación $\mathbf{m}_j \in C_j$ que minimiza la suma de distancias a los demás puntos del grupo:

$$\mathbf{m}_j = \arg \min_{\mathbf{x} \in C_j} \sum_{\mathbf{x}' \in C_j} d(\mathbf{x}, \mathbf{x}'). \quad (148.2)$$

El algoritmo PAM (Partitioning Around Medoids) itera intercambiando medoides con no-medoides y aceptando los cambios que reducen el costo total. PAM es más costoso que K-Means ($O(k(n-k)^2)$ por iteración), pero produce resultados más robustos ante outliers.

```

from sklearn.datasets import make_blobs
import numpy as np

rng = np.random.default_rng(42)
X, _ = make_blobs(n_samples=200, centers=3, cluster_std=0.8, random_state=42)

# Agregar outliers
X = np.vstack([X, [[10, 10], [10, -10], [-10, 10]]])

from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, random_state=0, n_init=10)
labels_km = km.fit_predict(X)
print("K-Means centroides:\n", km.cluster_centers_.round(3))

# Implementación simple de K-Medoids
def kmedoids_simple(X, k, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(X), k, replace=False)
    medoids = list(idx)

    for _ in range(100):
        D = np.array([[np.linalg.norm(X[i] - X[m]) for m in medoids] for i in range(len(X))])
        labels = D.argmax(axis=1)
        new_medoids = []
        for j in range(k):
            cluster = np.where(labels == j)[0]
            if len(cluster) == 0:
                new_medoids.append(medoids[j])
                continue
            sub_D = np.array([[np.linalg.norm(X[a] - X[b]) for b in cluster] for a in cluster])
            new_medoids.append(int(cluster[sub_D.sum(axis=1).argmin()]))
        if new_medoids == medoids:
            break
        medoids = new_medoids
    return np.array(medoids), labels

medoids_idx, labels_kmed = kmedoids_simple(X, 3)
print("\nK-Medoids medoides (índices):", medoids_idx)
print("K-Medoids medoides (valores):\n", X[medoids_idx].round(3))

```

K-Means centroides:

```

[[ 4.789  2.015]
 [-6.772 -6.857]
 [-2.737  8.984]]

```

K-Medoids medoides (índices): [74 7 24]

K-Medoids medoides (valores):

```

[[-1.387  7.893]

```

```
[-7.256 -6.694]
[-6.334 -7.128]]
```

148.7. Implementación con scikit-learn

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

X, y_true = make_blobs(
    n_samples=400, centers=[[-3, -3], [0, 0], [3, 3], [0, 5]],
    cluster_std=[0.8, 1.0, 0.7, 0.9], random_state=42
)

X_std = StandardScaler().fit_transform(X)

km = KMeans(
    n_clusters=4,
    init="k-means++",
    n_init=20,
    max_iter=300,
    random_state=42
)
labels = km.fit_predict(X_std)

print("Inercia (WCSS):", round(km.inertia_, 4))
print("Iteraciones hasta convergencia:", km.n_iter_)
print("Silueta:", round(silhouette_score(X_std, labels), 4))
print("Distribución:", np.bincount(labels))
```

```
Inercia (WCSS): 76.5005
Iteraciones hasta convergencia: 4
Silueta: 0.6207
Distribución: [ 95 106 102  97]
```

148.8. Consideraciones prácticas

- Ejecutar K-Means múltiples veces (`n_init`) y quedarse con la solución de menor WCSS reduce el riesgo de caer en mínimos locales pobres.
- Estandarizar siempre los datos antes de aplicar K-Means, a menos que las distintas escalas tengan significado conceptual.
- En espacios de muy alta dimensión, considerar primero reducción de dimensión (PCA, UMAP) antes de aplicar K-Means.
- K-Means no es adecuado para datos con clusters de forma irregular; en ese caso preferir DBSCAN o clustering espectral.

149. Clustering jerárquico

El clustering jerárquico construye una jerarquía anidada de agrupaciones en vez de producir una única partición. Esta jerarquía se representa como un árbol binario llamado dendrograma, en el que las hojas son las observaciones individuales y la raíz es un único grupo que contiene a todas. El analista puede cortar el dendrograma a cualquier altura para obtener una partición con el número de grupos deseado.

Esta propiedad hace que el clustering jerárquico sea especialmente útil cuando no se conoce de antemano el número de grupos, o cuando se desea explorar estructuras a distintos niveles de granularidad. Además, el dendrograma proporciona una representación visual rica de la estructura de los datos que facilita la interpretación.

Existen dos enfoques principales: el método aglomerativo, que comienza con cada observación como su propio grupo y va fusionando los más similares; y el método divisivo, que parte de un único grupo y lo va partiendo recursivamente. El método aglomerativo es más común en la práctica y está mejor estudiado teóricamente (Hastie et al. 2009; Aggarwal y Reddy 2013).

149.1. Método aglomerativo

El algoritmo aglomerativo genérico opera sobre la matriz de distancias entre grupos.

Algoritmo aglomerativo.

Entrada: observaciones $\mathbf{x}_1, \dots, \mathbf{x}_n$, función de enlace d .

1. Inicializar n grupos $C_i = \{\mathbf{x}_i\}$ para $i = 1, \dots, n$.
2. Calcular la matriz de distancias D de tamaño $n \times n$ con $D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$.
3. Repetir hasta que quede un solo grupo:
 - a. Encontrar el par (i, j) con D_{ij} mínima.
 - b. Fusionar C_i y C_j en un nuevo grupo C_{ij} .
 - c. Actualizar D con las distancias del nuevo grupo a todos los demás.
4. Registrar la secuencia de fusiones y sus alturas.

Salida: dendrograma con todas las fusiones.

La diferencia entre las variantes del método aglomerativo está en cómo se define la distancia d entre grupos —el criterio de enlace.

149.2. Criterios de enlace

El criterio de enlace determina la distancia entre dos grupos como función de las distancias entre sus elementos. Los criterios más usados son los siguientes.

Enlace simple (single linkage). La distancia entre dos grupos es la distancia mínima entre cualquier par de observaciones, una de cada grupo:

$$d_{\text{simple}}(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}). \quad (149.1)$$

Enlace completo (complete linkage). La distancia entre dos grupos es la distancia máxima entre cualquier par de observaciones, una de cada grupo:

$$d_{\text{completo}}(C_i, C_j) = \max_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}). \quad (149.2)$$

Enlace promedio (average linkage / UPGMA). La distancia entre dos grupos es el promedio de todas las distancias por pares:

$$d_{\text{prom}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}). \quad (149.3)$$

Enlace de Ward. Fusiona los dos grupos cuya unión produce el menor incremento en la suma de cuadrados intra-grupo (Ward 1963):

$$d_{\text{Ward}}(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\mu_i - \mu_j\|^2, \quad (149.4)$$

donde μ_i y μ_j son los centroides de C_i y C_j .

El enlace simple tiende a producir dendrogramas con efecto de encadenamiento: grupos alargados en los que se van añadiendo puntos uno por uno. El enlace completo produce grupos más compactos pero puede ser sensible a outliers. El enlace de Ward minimiza la varianza intra-grupo en cada fusión, por lo que tiende a producir grupos de tamaño parecido y es el más usado en la práctica.

149.3. El dendrograma

El dendrograma es una representación en árbol de la historia de fusiones. En el eje vertical se grafica la distancia (o altura) a la que se realiza cada fusión; en el eje horizontal se ubican las observaciones individuales. Una línea horizontal a altura h divide el dendrograma en los grupos que existían en ese momento.

Para extraer un número de grupos específico se corta el dendrograma con una línea horizontal. El número de ramas que esa línea intersecta es el número de grupos resultantes. La altura del corte puede elegirse visualmente buscando el mayor salto vertical en el dendrograma, lo que corresponde a fusiones que unen grupos bien separados.

149.4. Correlación cofenética

La correlación cofenética mide qué tan fielmente el dendrograma representa las distancias originales. La distancia cofenética entre dos puntos $\mathbf{x}_i$ y $\mathbf{x}_j$ es la altura a la que se fusionan por primera vez en el

dendrograma.

Definición. La correlación cofenética r_c es la correlación de Pearson entre la matriz de distancias originales y la matriz de distancias cofenéticas:

$$r_c = \text{corr}(d_{ij}, c_{ij}), \quad (149.5)$$

donde d_{ij} es la distancia original y c_{ij} es la distancia cofenética entre $\mathbf{x}_i$ y $\mathbf{x}_j$.

Un valor de r_c cercano a 1 indica que el dendrograma preserva bien la estructura de distancias. El enlace promedio suele producir los mayores valores de correlación cofenética.

```
import numpy as np
from sklearn.datasets import make_blobs
from scipy.cluster.hierarchy import linkage, dendrogram, cophenet
from scipy.spatial.distance import pdist
```

```
X, y_true = make_blobs(n_samples=80, centers=3, cluster_std=0.8, random_state=42)
```

```
for method in ["single", "complete", "average", "ward"]:
    Z = linkage(X, method=method)
    c, _ = cophenet(Z, pdist(X))
    print(f"{method:10s}  r_cofenética = {c:.4f}")
```

```
single      r_cofenética = 0.9833
complete    r_cofenética = 0.9838
average     r_cofenética = 0.9849
ward        r_cofenética = 0.9842
```

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import numpy as np
from sklearn.datasets import make_blobs
```

```
X, _ = make_blobs(n_samples=80, centers=4, cluster_std=0.7, random_state=0)
```

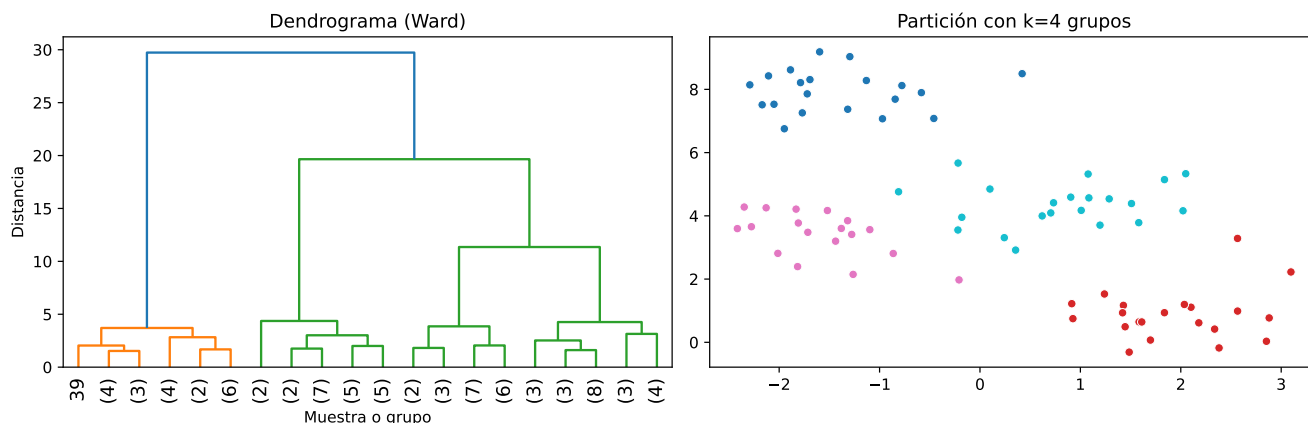
```
Z = linkage(X, method="ward")
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
```

```
dendrogram(Z, ax=ax1, truncate_mode="lastp", p=20, leaf_rotation=90,
           color_threshold=0.7 * max(Z[:, 2]))
ax1.set_title("Dendrograma (Ward)")
ax1.set_xlabel("Muestra o grupo")
ax1.set_ylabel("Distancia")
```

```
labels = fcluster(Z, t=4, criterion="maxclust")
ax2.scatter(X[:, 0], X[:, 1], c=labels, cmap="tab10", s=30, edgecolors="white", linewidths=0.5)
ax2.set_title("Partición con k=4 grupos")
```

```
plt.tight_layout()
plt.show()
```



149.5. Complejidad y escalabilidad

El algoritmo aglomerativo ingenuo requiere $O(n^2)$ espacio para la matriz de distancias y $O(n^3)$ tiempo para las $n - 1$ fusiones, ya que en cada paso debe recalcular las distancias entre el nuevo grupo y los demás. Para enlace simple y completo existen algoritmos como SLINK y CLINK con complejidad $O(n^2)$ en tiempo y $O(n)$ en espacio.

Para conjuntos de datos grandes ($n > 10\,000$), el clustering jerárquico estándar se vuelve poco práctico. Algunas alternativas son:

- **BIRCH**: construye un resumen compacto de los datos (CF-tree) y luego aplica clustering jerárquico sobre los resúmenes.
- **Clustering jerárquico aproximado**: usando índices de vecinos más cercanos aproximados para reducir el número de comparaciones.
- **Mini-batch clustering**: submuestrear el conjunto de datos y extrapolar la estructura jerárquica.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
import numpy as np
```

```
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=0)
```

```
for linkage in ["ward", "complete", "average", "single"]:
    model = AgglomerativeClustering(n_clusters=4, linkage=linkage)
    labels = model.fit_predict(X)
    sil = silhouette_score(X, labels)
    print(f"{linkage:10s} silueta = {sil:.4f}")
```

```
ward          silueta = 0.5749
complete      silueta = 0.5780
average       silueta = 0.5646
single        silueta = -0.0659
```

149.6. Aplicaciones del clustering jerárquico

El clustering jerárquico se usa extensamente en bioinformática para agrupar genes o muestras en experimentos de expresión génica, donde el dendrograma permite visualizar tanto la agrupación de genes similares como su grado de similitud a distintas escalas. En análisis de texto, se usa para construir taxonomías temáticas. En finanzas, para clasificar activos con comportamientos similares.

Una ventaja clave sobre K-Means es que no requiere especificar el número de grupos de antemano. El dendrograma muestra de forma explícita las estructuras a distintos niveles, lo que facilita la toma de decisiones sobre la granularidad apropiada para el problema.

150. DBSCAN y HDBSCAN

K-Means y el clustering jerárquico asumen implícitamente que los grupos son convexos o al menos compactos en torno a un centroide o medoide. En muchos problemas reales, los grupos tienen formas arbitrarias: anillos concéntricos, espirales, formas irregulares determinadas por procesos físicos o geográficos. Para estos casos, los métodos basados en densidad son más adecuados.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) fue propuesto por Ester et al. en 1996 (Ester et al. 1996) y se convirtió en uno de los algoritmos de clustering más influyentes, ganando el premio Test of Time en la conferencia KDD 2014. Su idea central es simple: un grupo es una región de alta densidad de puntos, separada de otras regiones de alta densidad por regiones de baja densidad. Los puntos en zonas de baja densidad se clasifican como ruido.

150.1. Conceptos fundamentales

DBSCAN opera sobre dos parámetros: el radio de vecindad ε y el número mínimo de puntos minPts.

Definición. La ε -**vecindad** de un punto $\mathbf{x}$ es el conjunto de puntos a distancia menor o igual que ε :

$$N_\varepsilon(\mathbf{x}) = \{\mathbf{y} \in \mathcal{D} \mid d(\mathbf{x}, \mathbf{y}) \leq \varepsilon\}. \quad (150.1)$$

Con estos dos parámetros se clasifican los puntos en tres categorías.

Definición. Un punto $\mathbf{x}$ es:

- **Punto núcleo (core point):** si $|N_\varepsilon(\mathbf{x})| \geq \text{minPts}$.
- **Punto borde (border point):** si no es punto núcleo pero está en la ε -vecindad de algún punto núcleo.
- **Punto de ruido (noise point):** si no es punto núcleo ni punto borde.

Los puntos núcleo tienen suficientes vecinos cercanos como para considerarse parte del interior de un grupo. Los puntos borde están cerca de un grupo pero no tienen suficientes vecinos propios. Los puntos de ruido son aislados y no pertenecen a ningún grupo.

150.2. Alcanzabilidad y conectividad por densidad

La noción de grupo en DBSCAN se define mediante la alcanzabilidad por densidad entre puntos.

Definición. Un punto $\mathbf{y}$ es **directamente alcanzable por densidad** desde $\mathbf{x}$ (con parámetros ε y minPts) si: 1. $\mathbf{y} \in N_\varepsilon(\mathbf{x})$, y 2. $|N_\varepsilon(\mathbf{x})| \geq \text{minPts}$ (es decir, $\mathbf{x}$ es punto núcleo).

Definición. Un punto $\mathbf{y}$ es **alcanzable por densidad** desde $\mathbf{x}$ si existe una cadena $\mathbf{x} = \mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_k = \mathbf{y}$ tal que $\mathbf{p}_{i+1}$ es directamente alcanzable desde $\mathbf{p}_i$.

Definición. Dos puntos $\mathbf{x}$ e $\mathbf{y}$ están **conectados por densidad** si existe un punto $\mathbf{z}$ tal que ambos son alcanzables por densidad desde $\mathbf{z}$.

Un **grupo DBSCAN** es un conjunto maximal de puntos mutuamente conectados por densidad.

150.3. Algoritmo DBSCAN

Algoritmo DBSCAN.

Entrada: $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, parámetros ε y minPts.

1. Etiquetar todos los puntos como no visitados.
2. Para cada punto no visitado $\mathbf{x}$:
 - a. Marcar $\mathbf{x}$ como visitado.
 - b. Calcular $N_\varepsilon(\mathbf{x})$.
 - c. Si $|N_\varepsilon(\mathbf{x})| < \text{minPts}$: etiquetar $\mathbf{x}$ como ruido (provisionalmente).
 - d. Si $|N_\varepsilon(\mathbf{x})| \geq \text{minPts}$: crear nuevo grupo C y expandir:
 - Agregar $\mathbf{x}$ a C .
 - Para cada $\mathbf{y} \in N_\varepsilon(\mathbf{x})$ no visitado: visitar $\mathbf{y}$ y, si es punto núcleo, agregar $N_\varepsilon(\mathbf{y})$ a la cola de expansión.
 - Agregar todos los alcanzables al grupo C .

Salida: etiquetas de grupo para cada punto (o ruido).

La complejidad de DBSCAN es $O(n^2)$ con búsqueda lineal de vecinos. Con un índice espacial como k-d tree o R-tree, se reduce a $O(n \log n)$ en promedio para datos de baja dimensión. En alta dimensión, la maldición de la dimensionalidad hace que los índices espaciales pierdan eficiencia.

150.4. Selección de parámetros

La elección de ε y minPts es el principal desafío de DBSCAN. Los valores inadecuados pueden producir un único grupo masivo (grande), muchos puntos de ruido (pequeño), o fragmentación de grupos naturales.

150.4.1. Heurística del gráfico k-distancia

Una heurística clásica para elegir ε es el gráfico de k-distancia. Para cada punto, se calcula la distancia al k-ésimo vecino más cercano. Estos valores se ordenan de mayor a menor y se grafican. El codo de la curva sugiere un buen valor de ε .

```
import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt
```



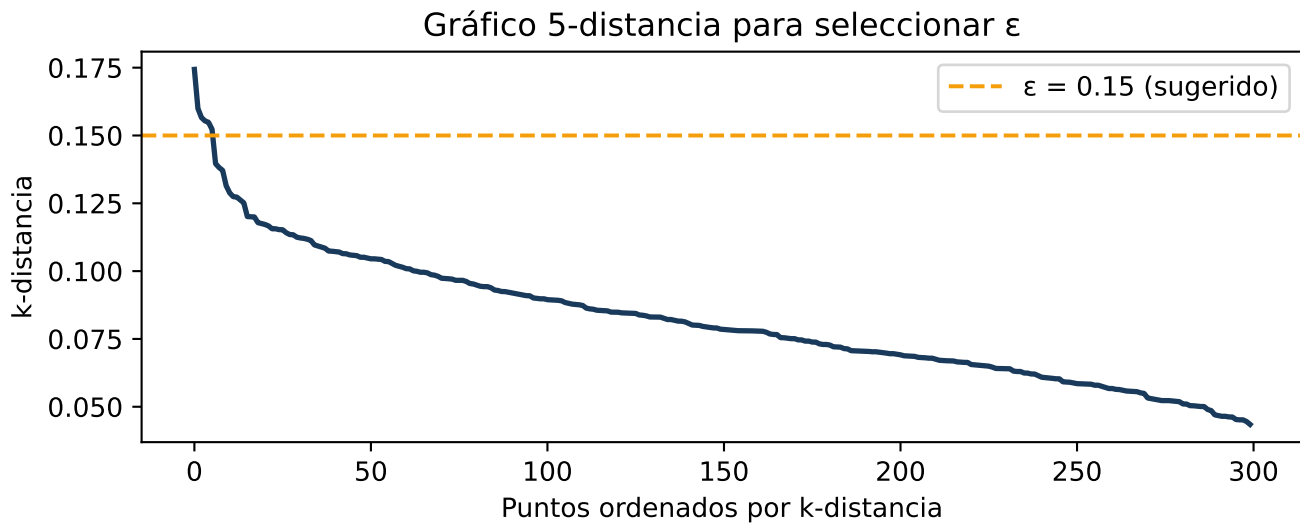
```

X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)

k = 5
nbrs = NearestNeighbors(n_neighbors=k).fit(X)
distances, _ = nbrs.kneighbors(X)
k_dist = np.sort(distances[:, -1])[:, :-1]

plt.figure(figsize=(7, 3))
plt.plot(k_dist, color="#1a3a5c", linewidth=2)
plt.axhline(0.15, color="#f59e0b", linestyle="--", label="ε = 0.15 (sugerido)")
plt.xlabel("Puntos ordenados por k-distancia")
plt.ylabel("k-distancia")
plt.title(f"Gráfico {k}-distancia para seleccionar ")
plt.legend()
plt.tight_layout()
plt.show()

```



150.4.2. Regla general para minPts

Como guía práctica, se recomienda $\text{minPts} \geq d + 1$ donde d es la dimensión de los datos (Ester et al. 1996). Para datos ruidosos, valores mayores son más robustos. En la práctica se usan frecuentemente $\text{minPts} \in \{3, 4, 5, 10\}$ y se ajusta ϵ con la heurística k-distancia.

```

from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
import numpy as np

X, y_true = make_moons(n_samples=300, noise=0.05, random_state=42)

db = DBSCAN(eps=0.15, min_samples=5)
labels = db.fit_predict(X)

```

```

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = np.sum(labels == -1)

print(f"Grupos encontrados: {n_clusters}")
print(f"Puntos de ruido: {n_noise} ({100*n_noise/len(X):.1f}%)")

mask = labels != -1
if mask.sum() > 1 and len(set(labels[mask])) > 1:
    print(f"Silueta (sin ruido): {silhouette_score(X[mask], labels[mask]):.4f}")

```

```

Grupos encontrados: 2
Puntos de ruido: 0 (0.0%)
Silueta (sin ruido): 0.3308

```

150.5. Ventajas y limitaciones

Ventajas de DBSCAN:

- Detecta grupos de forma arbitraria, no solo convexos.
- Identifica automáticamente puntos de ruido sin necesidad de asignarlos a ningún grupo.
- No requiere especificar el número de grupos de antemano.
- Es relativamente robusto a outliers.

Limitaciones:

- No funciona bien cuando los grupos tienen densidades muy distintas: un ε que conecta grupos de baja densidad puede fusionar grupos de alta densidad.
- La selección de ε y minPts puede ser difícil en datos de alta dimensión donde las distancias colapsan.
- Computacionalmente ineficiente para conjuntos de datos muy grandes sin indexación espacial.

150.6. HDBSCAN

HDBSCAN (Hierarchical DBSCAN) extiende DBSCAN construyendo una jerarquía de grupos basada en densidad que es más robusta ante variaciones de densidad (Campello et al. 2013). La idea central es transformar las distancias originales en distancias de alcanzabilidad mutua, construir un árbol de expansión mínimo sobre estas distancias, convertirlo en una jerarquía de grupos y extraer los grupos más persistentes.

Definición. La **distancia de alcanzabilidad mutua** entre dos puntos $\mathbf{x}$ y $\mathbf{y}$ es:

$$d_{\text{mreach},k}(\mathbf{x}, \mathbf{y}) = \max\{d_k(\mathbf{x}), d_k(\mathbf{y}), d(\mathbf{x}, \mathbf{y})\}, \quad (150.2)$$

donde $d_k(\mathbf{x})$ es la distancia al k -ésimo vecino más cercano de $\mathbf{x}$.

Esta transformación suaviza la estructura de densidad y hace que los grupos emerjan de forma más estable. HDBSCAN no requiere especificar ε directamente, solo minPts, y produce grupos de densidad variable.

```

try:
    import hdbscan
    from sklearn.datasets import make_moons

    X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)

    hdb = hdbscan.HDBSCAN(min_cluster_size=15, min_samples=5)
    labels = hdb.fit_predict(X)

    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = (labels == -1).sum()
    print(f"HDBSCAN - Grupos: {n_clusters}, Ruido: {n_noise}")
except ImportError:
    print("Instalar hdbscan: pip install hdbscan")
    from sklearn.cluster import HDBSCAN
    from sklearn.datasets import make_moons

    X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)
    hdb = HDBSCAN(min_cluster_size=15, min_samples=5)
    labels = hdb.fit_predict(X)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = (labels == -1).sum()
    print(f"HDBSCAN (sklearn) - Grupos: {n_clusters}, Ruido: {n_noise}")

```

```

Instalar hdbscan: pip install hdbscan
HDBSCAN (sklearn) - Grupos: 2, Ruido: 0

```

150.7. Aplicaciones

DBSCAN y HDBSCAN son especialmente útiles en:

- **Datos geoespaciales:** identificar zonas de alta concentración de eventos (crímenes, ventas, accidentes) sin asumir formas de grupos predeterminadas.
- **Detección de anomalías:** los puntos clasificados como ruido son candidatos naturales a anomalías.
- **Análisis de imágenes:** segmentar regiones de una imagen con criterios de similitud local.
- **Bioinformática:** identificar grupos de células con comportamiento similar en datos de citometría de flujo o RNA-seq.
- **Redes sociales:** detectar comunidades en grafos convertidos en vectores de características.

Parte XXXI.

Reducción de Dimensionalidad

151. t-SNE, UMAP y reducción de dimensionalidad no lineal

Los datos modernos suelen vivir en espacios de decenas, cientos o miles de dimensiones. Aunque matemáticamente bien definidos, estos espacios son difíciles de visualizar e interpretar. La reducción de dimensionalidad busca representar los datos en un espacio de baja dimensión —típicamente 2D o 3D— preservando la estructura relevante: distancias, vecindades, o topología local.

PCA (Análisis de Componentes Principales) es el método lineal clásico: encuentra la proyección ortogonal de menor dimensión que maximiza la varianza conservada (Jolliffe 2002). Sin embargo, PCA asume que la estructura relevante se puede capturar mediante combinaciones lineales de las variables originales. Muchas variedades de datos reales —como manifolds curvos en alta dimensión— requieren métodos no lineales.

t-SNE y UMAP son los dos métodos no lineales más usados en la práctica para visualización de datos de alta dimensión. Ambos intentan preservar la estructura local de vecindades, pero difieren en sus fundamentos teóricos, velocidad de cómputo y capacidad de preservar estructura global (Maaten y Hinton 2008; McInnes et al. 2018).

151.1. Limitaciones de PCA para visualización

PCA proyecta los datos sobre los d autovectores de la matriz de covarianza con mayores autovalores. Esta proyección es óptima para preservar la varianza, pero no necesariamente preserva la estructura de grupos o la topología local.

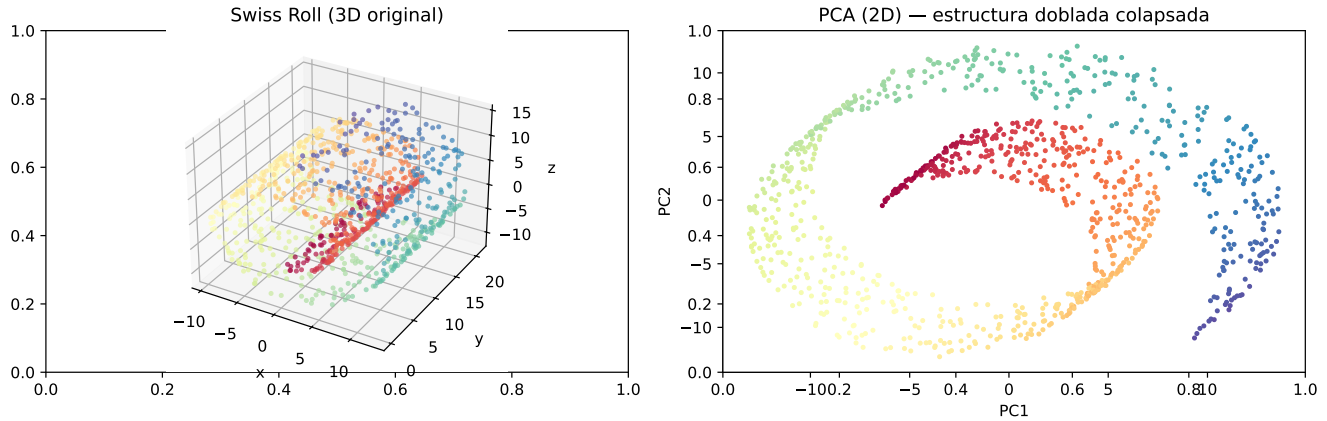
```
import numpy as np
from sklearn.datasets import make_swiss_roll
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

X, color = make_swiss_roll(n_samples=1000, noise=0.05, random_state=42)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1 = fig.add_subplot(121, projection="3d")
ax1.scatter(X[:, 0], X[:, 1], X[:, 2], c=color, cmap="Spectral", s=5, alpha=0.7)
ax1.set_title("Swiss Roll (3D original)")
ax1.set_xlabel("x"); ax1.set_ylabel("y"); ax1.set_zlabel("z")
ax2 = fig.add_subplot(122)
ax2.scatter(X_pca[:, 0], X_pca[:, 1], c=color, cmap="Spectral", s=5)
ax2.set_title("PCA (2D) - estructura doblada colapsada")
ax2.set_xlabel("PC1"); ax2.set_ylabel("PC2")
```

```
plt.tight_layout()
plt.show()
```



151.2. SNE y t-SNE

t-SNE fue propuesto por van der Maaten y Hinton en 2008 como mejora de Stochastic Neighbor Embedding (SNE) (Maaten y Hinton 2008). La idea central es modelar la estructura de vecindades en el espacio original mediante una distribución de probabilidad, construir una distribución análoga en el espacio de baja dimensión, y minimizar la divergencia entre ambas.

151.2.1. Probabilidades de similitud en el espacio original

Para cada par de puntos (i, j) , se define una probabilidad $p_{j|i}$ que refleja qué tan probable es que el punto j sea vecino del punto i si los vecinos se eligieran proporcionalmente a una distribución gaussiana centrada en i .

Definición. La similitud condicional asimétrica en el espacio original es:

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / (2\sigma_i^2))}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / (2\sigma_i^2))}, \quad (151.1)$$

y la similitud simétrica es $p_{ij} = (p_{j|i} + p_{i|j}) / (2n)$.

El ancho σ_i se determina para que la entropía de $p_{\cdot|i}$ sea $\log_2(\text{Perp})$, donde Perp es la **perplejidad**, un hiperparámetro que controla el número efectivo de vecinos considerados.

151.2.2. Distribución en el espacio de baja dimensión

En el espacio de baja dimensión, t-SNE usa una distribución t de Student con un grado de libertad (distribución de Cauchy) en vez de una gaussiana. Esta elección resuelve el problema del crowding: en el espacio original los puntos moderadamente distantes pueden distribuirse en muchas direcciones, pero en 2D no hay suficiente “espacio” para acomodarlos todos, lo que comprime artificialmente los grupos.

Definición. La similitud entre los puntos en el espacio de baja dimensión $\mathbf{y}_i, \mathbf{y}_j$ usando la distribución t de Student es:

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}. \quad (151.2)$$

151.2.3. Función objetivo

t-SNE minimiza la divergencia KL entre la distribución en el espacio original y la distribución en el espacio reducido:

Definición. El objetivo de t-SNE es:

$$C = \text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}. \quad (151.3)$$

La minimización se realiza mediante descenso de gradiente estocástico, típicamente con aceleración por momentos y un término de exageración inicial que amplifica los p_{ij} en las primeras iteraciones para facilitar la formación de grupos.

151.2.4. Interpretación de la perplejidad

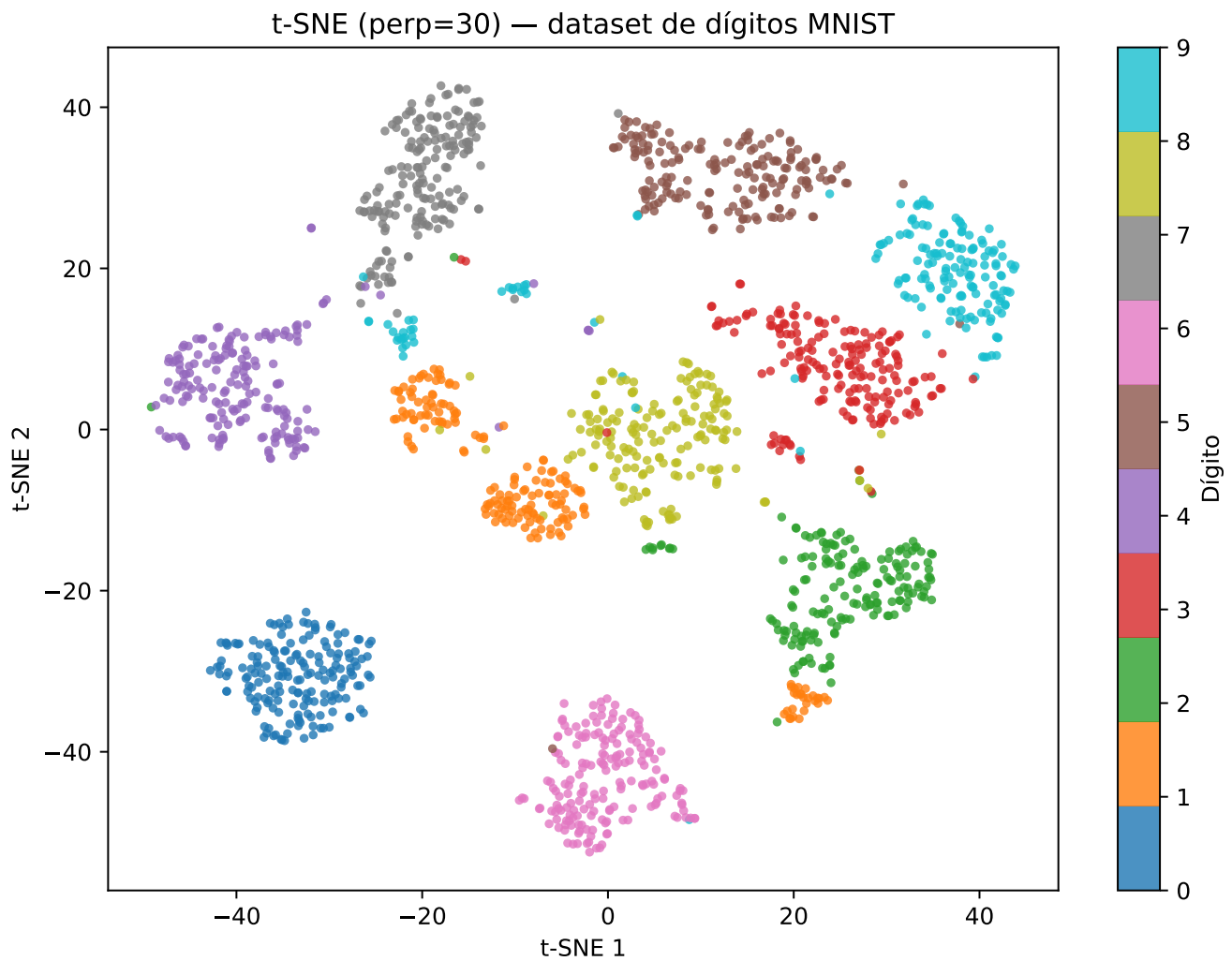
La perplejidad controla cuántos vecinos efectivos se consideran al construir las distribuciones de similitud. Valores bajos (5-10) priorizan la estructura local muy fina; valores altos (30-100) capturan estructura más global. En la práctica se recomienda probar $\text{Perp} \in \{5, 10, 30, 50, 100\}$ y comparar visualmente.

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

digits = load_digits()
X = StandardScaler().fit_transform(digits.data)

tsne = TSNE(n_components=2, perplexity=30, n_iter=1000, random_state=42, learning_rate="auto", in
X_tsne = tsne.fit_transform(X)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=digits.target,
                      cmap="tab10", s=8, alpha=0.8)
plt.colorbar(scatter, label="Dígito")
plt.title("t-SNE (perp=30) - dataset de dígitos MNIST")
plt.xlabel("t-SNE 1"); plt.ylabel("t-SNE 2")
plt.tight_layout()
plt.show()
```



151.3. UMAP

UMAP (Uniform Manifold Approximation and Projection) fue propuesto por McInnes et al. en 2018 (McInnes et al. 2018) con fundamentos teóricos basados en geometría diferencial y topología algebraica. A diferencia de t-SNE, UMAP tiene una mejor preservación de la estructura global y es significativamente más rápido, lo que lo hace aplicable a conjuntos de datos de millones de puntos.

151.3.1. Fundamentos teóricos

La base de UMAP es la teoría de conjuntos simpliciales difusos (fuzzy simplicial sets). Los datos se modelan como una variedad de Riemann, se construye una representación topológica local de esa variedad usando grafos ponderados de vecinos más cercanos, y luego se busca una representación de baja dimensión que preserve esa topología.

Definición. UMAP construye un grafo de vecindad ponderado donde la conectividad entre $\mathbf{x}_i$ y $\mathbf{x}_j$ se modela mediante

$$v_{j|i} = \exp\left(-\frac{d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i}{\sigma_i}\right), \quad (151.4)$$

donde ρ_i es la distancia al vecino más cercano de $\mathbf{x}_i$ y σ_i se ajusta para que la suma de conectividades sea $\log_2(k)$ (con k el número de vecinos).

La simetrización es $v_{ij} = v_{j|i} + v_{i|j} - v_{j|i} \cdot v_{i|j}$.

En el espacio de baja dimensión, UMAP modela la similitud con una curva de familia parametrizable que se ajusta para aproximar una distribución t de Student. El objetivo es minimizar la divergencia cruzada binaria entre las similitudes en alta y baja dimensión.

151.3.2. Hiperparámetros principales

n_neighbors: número de vecinos en el grafo de alta dimensión. Similar a la perplejidad de t-SNE. Valores pequeños capturan estructura local fina; valores grandes capturan estructura global.

min_dist: distancia mínima entre puntos en el espacio reducido. Controla qué tan compactos son los grupos en la visualización. Valores pequeños producen grupos apretados; valores grandes los esparcen.

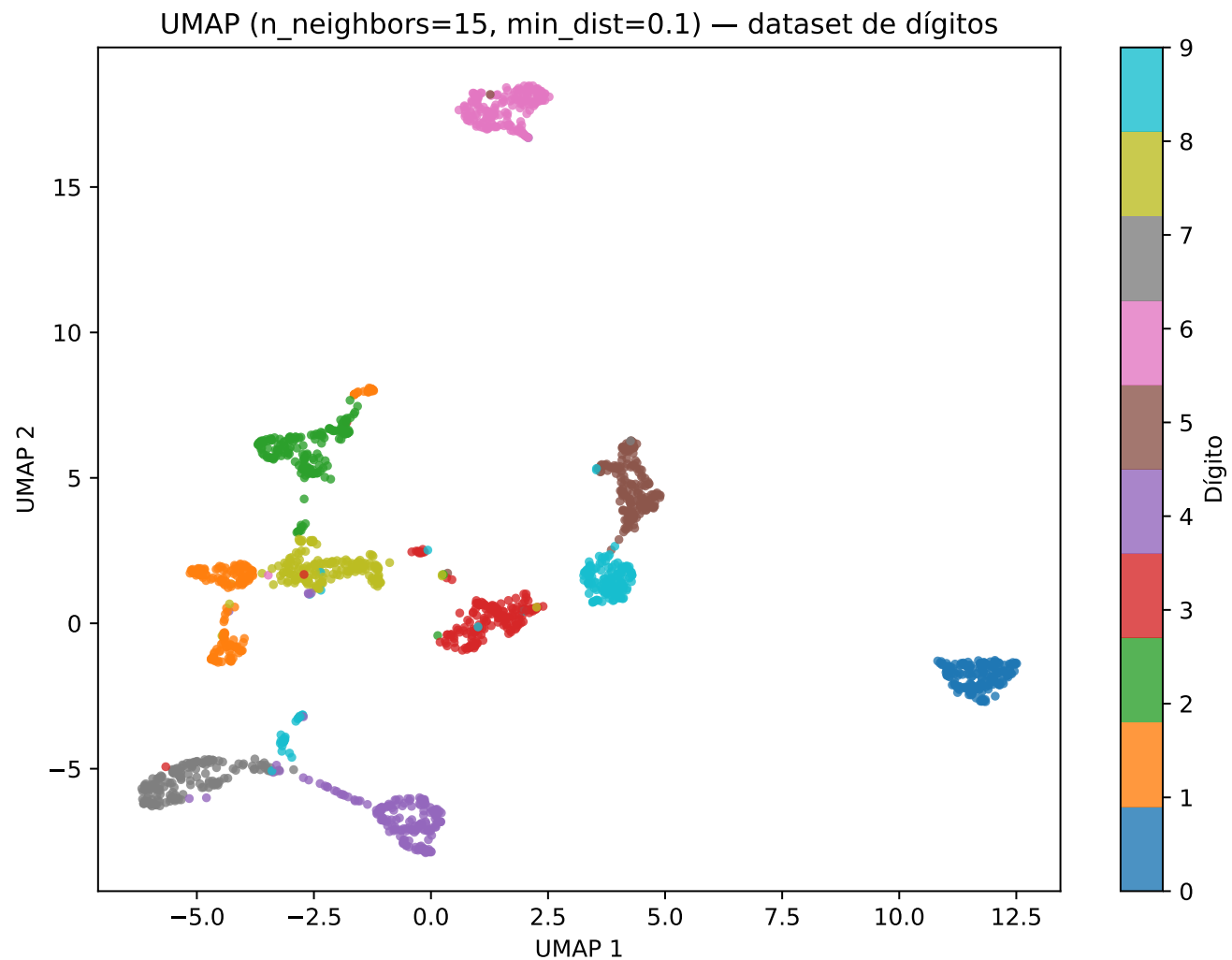
```
try:
    import umap
    from sklearn.datasets import load_digits
    from sklearn.preprocessing import StandardScaler
    import matplotlib.pyplot as plt

    digits = load_digits()
    X = StandardScaler().fit_transform(digits.data)

    reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X)

    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(X_umap[:, 0], X_umap[:, 1], c=digits.target,
                          cmap="tab10", s=8, alpha=0.8)
    plt.colorbar(scatter, label="Dígito")
    plt.title("UMAP (n_neighbors=15, min_dist=0.1) - dataset de dígitos")
    plt.xlabel("UMAP 1"); plt.ylabel("UMAP 2")
    plt.tight_layout()
    plt.show()

except ImportError:
    print("Instalar umap-learn: pip install umap-learn")
```



151.4. Comparación t-SNE vs UMAP

Característica	t-SNE	UMAP
Fundamento	Divergencia KL, distribución t	Topología difusa, manifold learning
Velocidad	$O(n^2)$ ingenuo, $O(n \log n)$ con BH	$O(n^{1.14})$ aproximado
Preservación global	Limitada	Mejor que t-SNE
Reproducibilidad	Estocástico, varía entre corridas	Más estable con <code>random_state</code>
Hiperparámetros	Perplejidad, tasa aprendizaje	<code>n_neighbors</code> , <code>min_dist</code>
Escalabilidad	Hasta ~100k puntos	Hasta millones de puntos
Uso en clustering	Solo visualización	Sí, como preprocessing

```
from sklearn.datasets import make_blobs
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```

import numpy as np

X_high, y = make_blobs(n_samples=500, n_features=20, centers=5,
                      cluster_std=2.0, random_state=42)
X_std = StandardScaler().fit_transform(X_high)

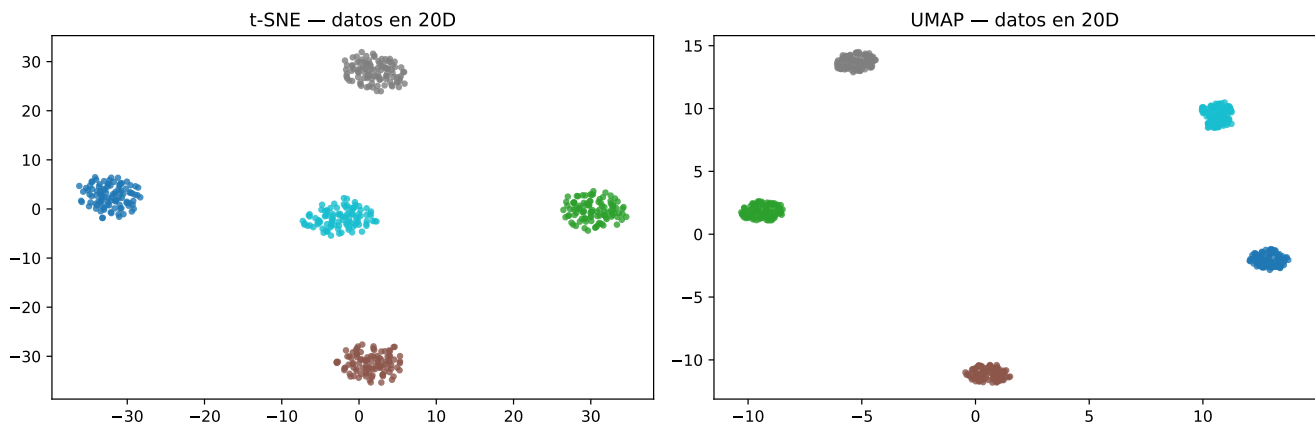
tsne = TSNE(n_components=2, perplexity=30, random_state=42,
           learning_rate="auto", init="pca")
X_tsne = tsne.fit_transform(X_std)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap="tab10", s=10, alpha=0.8)
axes[0].set_title("t-SNE - datos en 20D")

try:
    import umap
    reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X_std)
    axes[1].scatter(X_umap[:, 0], X_umap[:, 1], c=y, cmap="tab10", s=10, alpha=0.8)
    axes[1].set_title("UMAP - datos en 20D")
except ImportError:
    axes[1].text(0.5, 0.5, "Instalar umap-learn", ha="center", va="center")
    axes[1].set_title("UMAP (no instalado)")

plt.tight_layout()
plt.show()

```



151.5. Autoencoder básico para reducción de dimensionalidad

Un autoencoder es una red neuronal que aprende una representación comprimida de los datos. Consiste en un codificador que mapea la entrada a un espacio latente de dimensión reducida, y un decodificador que reconstruye la entrada desde ese espacio latente. El entrenamiento minimiza el error de reconstrucción.

```

import numpy as np
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

# Autoencoder con PyTorch (si disponible) o simulado con sklearn
try:
    import torch
    import torch.nn as nn

    digits = load_digits()
    X = MinMaxScaler().fit_transform(digits.data).astype(np.float32)
    X_tensor = torch.tensor(X)

    class Autoencoder(nn.Module):
        def __init__(self, latent_dim):
            super().__init__()
            self.encoder = nn.Sequential(
                nn.Linear(64, 32), nn.ReLU(),
                nn.Linear(32, latent_dim)
            )
            self.decoder = nn.Sequential(
                nn.Linear(latent_dim, 32), nn.ReLU(),
                nn.Linear(32, 64), nn.Sigmoid()
            )
        def forward(self, x):
            return self.decoder(self.encoder(x))

    ae = Autoencoder(latent_dim=2)
    optimizer = torch.optim.Adam(ae.parameters(), lr=1e-3)
    criterion = nn.MSELoss()

    for epoch in range(200):
        out = ae(X_tensor)
        loss = criterion(out, X_tensor)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    with torch.no_grad():
        Z = ae.encoder(X_tensor).numpy()

    print(f"Error de reconstrucción final (MSE): {loss.item():.6f}")
    print(f"Espacio latente: {Z.shape}")

except ImportError:
    print("PyTorch no disponible. Usando PCA como aproximación.")
    from sklearn.decomposition import PCA
    digits = load_digits()
    X = MinMaxScaler().fit_transform(digits.data)

```

```
pca = PCA(n_components=2)
Z = pca.fit_transform(X)
rec = pca.inverse_transform(Z)
mse = ((X - rec)**2).mean()
print(f"PCA 2 componentes - MSE de reconstrucción: {mse:.6f}")
print(f"Varianza explicada: {pca.explained_variance_ratio_.sum():.4f}")
```

Error de reconstrucción final (MSE): 0.068968
 Espacio latente: (1797, 2)

151.6. Precauciones y buenas prácticas

Las técnicas de reducción no lineal son herramientas de visualización, no de inferencia estadística. Varias advertencias son importantes:

No interpretar distancias como absolutas. Las distancias en el espacio 2D de t-SNE o UMAP no son proporcionales a las distancias originales. Dos puntos cercanos en la visualización no necesariamente son similares en el espacio original.

Resultados estocásticos. t-SNE produce resultados distintos en cada corrida salvo que se fije la semilla aleatoria. UMAP es más estable pero sigue siendo estocástico.

La estructura global no se preserva en t-SNE. Los grupos lejanos en la visualización de t-SNE no necesariamente están más separados en el espacio original. UMAP preserva mejor la estructura global.

No usar como único criterio para clustering. Una apariencia de grupos en 2D no implica que existan grupos reales en el espacio original. Confirmar con métricas internas o externas.

```
from sklearn.manifold import TSNE
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import numpy as np

# Demostración: correr t-SNE múltiples veces con distintas semillas
X, y = make_blobs(n_samples=200, centers=3, cluster_std=0.8, random_state=0)
X_std = StandardScaler().fit_transform(X)

for seed in [0, 42, 123]:
    tsne = TSNE(n_components=2, perplexity=30, random_state=seed,
                learning_rate="auto", init="pca", n_iter=500)
    Z = tsne.fit_transform(X_std)
    spread = Z.std(axis=0).mean()
    print(f"Semilla {seed:3d} - dispersión media en 2D: {spread:.3f}")
```

Semilla 0 - dispersión media en 2D: 8.040

Semilla 42 - dispersión media en 2D: 8.040

Semilla 123 - dispersión media en 2D: 8.172

151.7. Cuándo usar cada método

Escenario	Recomendación
Visualización rápida, <50k puntos	t-SNE con perplejidad 30
Visualización de millones de puntos	UMAP
Se necesita reproducibilidad	UMAP con <code>random_state</code>
Reducción para clustering posterior	UMAP (preserva mejor estructura global)
Reconstrucción aproximada	Autoencoder
Interpretabilidad de los ejes	PCA
Alta dimensión con manifold no lineal	UMAP o t-SNE después de PCA inicial

A. Presentaciones de clase

Herramientas para proyectos profesionales

Quarto: un proyecto, muchas salidas

Taller práctico de 60 minutos · entorno local · Python · formatos · GitHub Pages

Ver taller de Quarto →

Entendimiento de negocio

Antes del modelo

Roles, ciclo de vida de los datos y *Business Understanding Canvas*.

35 diapositivas en formato PDF.

Abrir presentación

De una necesidad de negocio a un problema analítico

Identificación de interesados, preguntas de negocio y formulación del problema analítico.

25 diapositivas en formato PDF.

Abrir presentación

Arquitecturas para datos analíticos

Construyamos un Data Warehouse

De las transacciones operativas a una arquitectura preparada para análisis.

39 diapositivas en formato PDF.

Abrir presentación

De silos a Lakehouse

Evolución de las arquitecturas de datos y criterios para adoptar un *data lakehouse*.

30 diapositivas en formato PDF.

Abrir presentación

Análisis Exploratorio de Datos

Medidas de variabilidad · repaso animado

Rango · IQR · boxplot · varianza · desviación estándar

Abrir presentación animada →

Medidas de heterogeneidad · repaso animado

Gini–Simpson · Shannon · categorías efectivas · eta cuadrada

Abrir presentación animada → Abrir simulador interactivo →

Datos faltantes

Tratamiento, detección e imputación de valores ausentes.

Ver presentación →

EDA BigMart

Heterogeneidad · Concentración · Correlación · PCA

EDA BigMart →

EDA Multivariado

PCA · Correlación · T-SNE · UMAP

EDA Multivariado →

Chi cuadrado

Correlacion categorica · Test de independencia

Chi cuadrado →

Reglas de asociación

Relación entre variables categóricas · Soporte · Confianza · Lift

Reglas de asociación →

Evaluación de clasificadores

Matriz de confusión · Métricas de evaluación · Curvas ROC y PR

Evaluación de clasificadores →

Ejemplo de clasificación

Matriz de confusión · Métricas de evaluación · Curvas ROC y PR

Ejemplo de clasificación →

Clasificadores bayesianos

Teorema de Bayes · Naive Bayes · Modelos gaussianos

Clasificadores bayesianos →

Redes neuronales con PyTorch y JAX

Entrenamiento práctico · Autodiferenciación · Comparación de frameworks

PyTorch y JAX →

B. Banco de Proyectos

Esta sección reúne diez propuestas de proyecto final para poner en práctica el ciclo completo del libro: entendimiento de negocio, construcción (o consumo) de un almacén de datos, EDA, preprocesamiento y modelado (Marco Metodológico de Minería de Datos).

Cada propuesta describe un problema de negocio real, uno o más datasets con los que arrancar, qué la hace desafiante (para que no se quede en un ejercicio de manual) y las consideraciones legales/éticas de scraping cuando aplican. Ningún dataset está atado a un proyecto: úsalo como punto de partida y ajusta el alcance con tu equipo durante la fase de entendimiento de negocio.

! Importante

Antes de scrapear cualquier sitio. Revisa siempre el `robots.txt` y los Términos de Servicio del sitio, prefiere APIs oficiales o datasets ya compilados antes que scraping directo, respeta límites de tasa (*rate limiting*) si automatizas descargas, y nunca recolectes ni publiques datos personales identificables. Estas reglas se repiten en cada propuesta porque son no negociables, no porque cambien de un proyecto a otro.

B.1. 1. Value bets en Liga MX

Problema de negocio. Una plataforma de apuestas o un boletín deportivo quiere detectar cuándo las casas de apuestas “se equivocan” al fijar los momios de un partido.

Dataset(s) sugerido(s).

- LigaMX Matches 2016–2024 (Kaggle) — resultados históricos de Liga MX.
- Historical Football Results/Betting Odds Data (Kaggle) — resultados y momios históricos multi-liga; revisa si incluye México o combínalo con el dataset anterior por fecha/equipos.

Qué lo hace desafiante. No existe una etiqueta de “value bet”: el equipo tiene que construirla comparando la probabilidad implícita del momio contra el resultado real observado. Exige pensar en calibración de probabilidades, no solo en accuracy.

Técnicas del curso que cubre. Reglas de asociación, clasificación, evaluación con métricas de negocio (ROI simulado, no accuracy).

! Advertencia

Si decides scrapear momios en vivo en lugar de usar un dataset ya compilado, revisa el `robots.txt` del sitio de la casa de apuestas: muchas prohíben explícitamente el scraping en sus Términos de Servicio. Prefiere siempre una API pública o un dataset ya cerrado.

B.2. 2. Priorización de contratos públicos sospechosos

Problema de negocio. Una ONG anticorrupción necesita priorizar qué licitaciones revisar a mano con recursos limitados.

Dataset(s) sugerido(s).

- Contratos y expedientes del sistema histórico de CompraNet — datos.gob.mx
- Datos Abiertos de las Unidades Compradoras (ComprasMX)

Qué lo hace desafiante. No existe *ground truth* de “corrupción”: el éxito se mide por qué tan útil es la priorización con recursos limitados, no por accuracy. Alto riesgo ético de falsos positivos (señalar a un proveedor honesto).

Técnicas del curso que cubre. Reglas de asociación, detección de anomalías, clasificación con datos débilmente etiquetados.

Advertencia

CompraNet/ComprasMX ya expone datos vía portal de datos abiertos — úsalo en vez de scrapear HTML. Si automatizas la descarga, respeta límites de tasa y guarda evidencia de la fecha de descarga, porque los datos se actualizan y corrigen con el tiempo.

B.3. 3. Detección temprana de réplicas sísmicas anómalas

Problema de negocio. Protección Civil o un medio de análisis de riesgo quiere anticipar clústers de actividad sísmica inusual.

Dataset(s) sugerido(s).

- Catálogo de sismos — Servicio Sismológico Nacional, UNAM

Qué lo hace desafiante. Series de tiempo espaciales con eventos raros por definición; exige justificar el umbral de “anómalo” en términos que Protección Civil pueda entender y accionar.

Técnicas del curso que cubre. Clustering espacial, detección de anomalías, reglas de asociación (magnitud/profundidad/zona).

Nota

El SSN ofrece el catálogo en descarga directa (CSV/texto) sin necesidad de scraping. Si automatizas la descarga periódica, evita solicitudes masivas simultáneas al servidor.

B.4. 4. Calidad del aire y “días de contingencia” en CDMX

Problema de negocio. SEDEMA o una app de salud pública quiere predecir y explicar picos de contaminación.

Dataset(s) sugerido(s).

- Índice de la Calidad del Aire — Portal de Datos Abiertos CDMX
- Red Automática de Monitoreo Atmosférico (RAMA) — Portal de Datos Abiertos CDMX
- Portal SIMAT / Dirección de Monitoreo Atmosférico — histórico por estación, incluye descargas en Excel.

Qué lo hace desafiante. Fuerte estacionalidad, correlación espacial entre estaciones y variables de tráfico/clima como confusores; distinguir correlación de causa real.

Técnicas del curso que cubre. EDA con series de tiempo, PCA, clustering de estaciones, detección de anomalías.

i Nota

SIMAT tiene descarga directa por estación/año — no requiere scraping. Si combinas con datos de tráfico de terceros (p. ej. Waze), revisa los términos de uso de esa API antes de automatizar la extracción.

B.5. 5. Fraude en reembolsos/transacciones fintech

Problema de negocio. Una startup fintech LATAM quiere reducir el fraude sin bloquear a usuarios legítimos.

Dataset(s) sugerido(s).

- PaySim — Synthetic Financial Datasets For Fraud Detection (Kaggle)
- IEEE-CIS Fraud Detection (Kaggle)

Qué lo hace desafiante. Desbalance de clases extremo (fraude $<1\%$); obliga a discutir el costo asimétrico de falsos negativos vs. falsos positivos, no solo accuracy.

Técnicas del curso que cubre. Clasificación con clases desbalanceadas, detección de anomalías, evaluación con métricas apropiadas (recall, PR-AUC).

i Nota

No aplica scraping: son datasets ya publicados en Kaggle. Verifica la licencia específica de cada dataset (algunas restringen uso comercial) antes de usarlo más allá del curso.

B.6. 6. Segmentación de incidentes de inseguridad por alcaldía/colonia

Problema de negocio. Un observatorio ciudadano quiere apoyar decisiones de patrullaje con evidencia, no percepción.

Dataset(s) sugerido(s).

- Datos Abiertos de Incidencia Delictiva — SESNSP
- Incidencia delictiva — datos.gob.mx

Qué lo hace desafiante. Dato sensible políticamente: exige comunicación responsable (evitar estigmatizar zonas) y separar tendencia real de subregistro/denuncia.

Técnicas del curso que cubre. Clustering geográfico/temporal, EDA con Gini/entropía por tipo de delito, reglas de asociación.

! Importante

El SESNSP publica los datos en descarga directa — no requiere scraping. Si cruzas con datos socioeconómicos por colonia (INEGI), cuida el nivel de agregación para no exponer o inferir datos de personas identificables.

B.7. 7. Reseñas falsas o manipuladas en e-commerce

Problema de negocio. Un marketplace (estilo Mercado Libre) quiere detectar reseñas fabricadas o campañas de reputación.

Dataset(s) sugerido(s).

- amazon_reviews_multi_es (Hugging Face) — reseñas de Amazon en español.
- Mercado Libre Data Challenge (Kaggle) — como fuente adicional de texto/metadatos de publicaciones.

Qué lo hace desafiante. No hay una etiqueta confiable de “reseña falsa”: el equipo debe operacionalizar señales indirectas (ráfagas temporales, texto genérico repetido, sentimiento desalineado con la calificación).

Técnicas del curso que cubre. Minería de texto, TF-IDF + sentimiento, detección de anomalías, clustering de patrones de escritura.

! Importante

Prefiere datasets ya recolectados antes que scrapear Mercado Libre o Amazon directamente: ambos prohíben el scraping en sus Términos de Servicio y usan detección activa de bots. Si de todas formas construyes un dataset propio, hazlo solo con datos públicos ya visibles sin login, con retraso entre solicitudes, y sin recolectar datos personales de quienes escriben las reseñas.

B.8. 8. Predicción de retrasos y saturación en el Metro/transporte público

Problema de negocio. Una autoridad de movilidad quiere anticipar la saturación de líneas y recomendar rutas alternas.

Dataset(s) sugerido(s).

- Afluencia diaria del Metro CDMX — Portal de Datos Abiertos CDMX
- Afluencia diaria de Metrobús CDMX — Portal de Datos Abiertos CDMX

Qué lo hace desafiante. Estacionalidad múltiple (hora, día, evento especial), datos con huecos y calidad heterogénea; el reto de negocio es “accionable en tiempo real”, no solo descriptivo.

Técnicas del curso que cubre. EDA con visualización temporal, clustering de patrones de demanda, detección de anomalías (incidentes atípicos).

Nota

El STC Metro y Metrobús publican afluencia en datos abiertos con descarga directa — no requiere scraping. Si agregas datos de Waze u otro proveedor externo, usa su API oficial de partners en vez de scrapear el mapa web: Waze prohíbe explícitamente el scraping de su plataforma.

B.9. 9. Deserción escolar en educación media superior

Problema de negocio. Una Secretaría de Educación estatal quiere identificar estudiantes en riesgo antes de que abandonen.

Dataset(s) sugerido(s).

- Estadística educativa — DGPPYEE, SEP
- Resultados PLANEA Básica
- Datasets de educación — datos.gob.mx

Qué lo hace desafiante. Etiquetas retrospectivas (se sabe quién desertó después de que ya pasó); exige pensar en qué variables están disponibles a tiempo para intervenir, no solo cuáles correlacionan.

Técnicas del curso que cubre. Clasificación, reglas de asociación, PCA/reducción de dimensionalidad, checkpoint de stakeholder realista (política pública).

Importante

La SEP publica estos indicadores agregados en descarga directa (no a nivel de estudiante individual) — no requiere scraping. Si buscas microdatos, usa solo bases públicas anonimizadas oficiales y nunca intentes reconstruir la identidad de estudiantes individuales.

B.10. 10. Discurso de odio/polarización en debates electorales

Problema de negocio. Un medio de comunicación o una autoridad electoral quiere monitorear el tono del discurso público sin censurar.

Dataset(s) sugerido(s).

- HatEval — SemEval-2019 Task 5 (Hugging Face) — tuits en inglés y español anotados para discurso de odio contra inmigrantes y mujeres.

Qué lo hace desafiante. Ambigüedad lingüística fuerte en español (sarcasmo, regionalismos); exige justificar el umbral entre “polarizado” y “de odio”, con implicaciones éticas serias.

Técnicas del curso que cubre. Minería de texto, TF-IDF, sentimiento con dos métodos comparados, discusión ética explícita en la presentación final.

! Importante

X/Twitter restringe fuertemente el scraping y el acceso gratuito a su API desde 2023 — usa el dataset académico HatEval en vez de scraping directo. Si de todas formas recolectas comentarios propios de otra fuente, anonimiza a los usuarios, no publiques el dataset crudo, y documenta el criterio ético de qué se considera “discurso de odio” antes de anotar datos.

C. Glosario Unificado de Términos de Datos y Computación en la Nube

Este glosario consolida los términos y definiciones fundamentales abordados en los módulos teóricos y técnicos del curso, incluyendo conceptos de infraestructura en la nube, almacenamiento, procesamiento, y gobernanza.

C.1. A

- **Advanced Provisioning (Aprovisionamiento Avanzado):** Modelo de entrega en la nube en el que el usuario firma un acuerdo formal con el proveedor de servicios en la nube y paga un precio fijo o se le factura mensualmente.
- **Apache Hadoop:** Framework de código abierto que permite el almacenamiento y procesamiento distribuido de grandes conjuntos de datos a través de clústeres de computadoras utilizando modelos de programación simples.
- **Archival Policy (Política de Archivo):** Lineamiento que define dónde, cómo y por cuánto tiempo se deben almacenar los datos una vez que un proyecto de análisis ha finalizado.
- **Audit (Auditoría de Datos):** Examen formal y sistemático de cómo los usuarios acceden y utilizan los datos, diseñado para garantizar el cumplimiento de las políticas de seguridad e identificar vulnerabilidades.
- **Autoscaling (Escalabilidad Automática):** Servicio en la nube que monitorea las aplicaciones y ajusta automáticamente la capacidad de cómputo (escalando hacia arriba o hacia abajo) según la demanda en tiempo real.

C.2. B

- **Backend Platform (Plataforma de Backend):** Conjunto de componentes físicos y lógicos de la arquitectura de la nube que soportan la ejecución del sistema (servidores, redes, almacenamiento).
- **Batch Processing (Procesamiento por Lotes):** Método de procesamiento de datos que consiste en recolectar un gran volumen de transacciones u operaciones durante un periodo de tiempo y procesarlas de forma conjunta en una sola ejecución.
- **Business Data Request (Solicitud de Datos de Negocio):** Pregunta o requerimiento formal planteado por un área operativa que puede ser respondido o solucionado mediante la extracción y análisis de datos.

C.3. C

- **Cloud Data Storage (Almacenamiento de Datos en la Nube):** Solución tecnológica que permite a las organizaciones guardar, acceder y mantener datos digitales en servidores remotos administrados por un proveedor de servicios (ej. Google Cloud Storage).
- **Complex Data Types (Tipos de Datos Complejos):** Estructuras de datos no primitivas (como Arrays y Structs) utilizadas en bases de datos modernas como BigQuery para almacenar registros anidados y optimizar el rendimiento de las consultas desnormalizadas.

C.4. D

- **Data Catalog (Catálogo de Datos):** Inventario centralizado de los activos de datos de una organización que proporciona metadatos detallados para que los analistas puedan descubrir, comprender y evaluar la calidad de las fuentes disponibles.
- **Data Governance (Gobierno de Datos):** Conjunto de procesos, políticas, roles y estándares que garantizan el uso seguro, confiable y eficiente de los datos dentro de una organización.
- **Data Lake (Lago de Datos):** Repositorio de almacenamiento que aloja grandes volúmenes de datos brutos en su formato original (estructurado, semi-estructurado o no estructurado) hasta que sean requeridos para análisis.
- **Data Warehouse (Almacén de Datos):** Sistema de base de datos relacional optimizado para el análisis de datos (OLAP) que consolida información depurada proveniente de múltiples sistemas transaccionales (OLTP).

C.5. N

- **Normalization (Normalización):** Técnica de diseño de bases de datos que organiza las tablas relacionales para minimizar la redundancia de datos y proteger la integridad referencial.

C.6. S

- **Streaming Processing (Procesamiento en Tiempo Real):** Método de procesamiento de datos que analiza y transforma los registros de forma continua, instantáneamente a medida que son generados por las fuentes emisoras.

D. Preparación para Entrevistas Analíticas

D.1. 1. La Importancia del Proceso de Pensamiento

En el reclutamiento para roles de analítica, minería e ingeniería de datos, las pruebas técnicas y las entrevistas ya no se limitan a evaluar si recuerdas la sintaxis exacta de una consulta SQL o los parámetros de un clasificador en Scikit-Learn. Los entrevistadores buscan comprender **cómo piensas y resuelves problemas complejos**.

Demostrar tu proceso de pensamiento de forma estructurada le indica al empleador que: * Puedes lidiar con la ambigüedad (problemas de negocio mal definidos). * No saltas a conclusiones sin explorar hipótesis. * Priorizas el valor de negocio antes que la complejidad del algoritmo. * Eres capaz de comunicar conceptos complejos a stakeholders no técnicos.

D.2. 2. El Marco de Resolución de Problemas en 4 Pasos

Cuando te enfrentes a un caso de negocio o pregunta abierta en una entrevista técnica, evita dar una respuesta directa e inmediata. Utiliza el siguiente marco estructurado para guiar al entrevistador a través de tu solución:

```
graph TD
    P1["Paso 1: Entender y Acotar<br>(Hacer preguntas de negocio)"]
    P2["Paso 2: Datos e Hipótesis<br>(Identificar fuentes y métricas)"]
    P3["Paso 3: Metodología Analítica<br>(ETL, EDA, Modelado)"]
    P4["Paso 4: Impacto y Comunicación<br>(Resultados para Stakeholders)"]
    P1 --> P2
    P2 --> P3
    P3 --> P4
```

D.2.1. Paso 1: Entender y Acotar el Problema

- **Qué hacer:** Haz preguntas aclaratorias para definir el alcance del problema. Nunca asumas.
- **Frases útiles:**
 - “*Antes de estructurar la base de datos, ¿cuál es el objetivo principal de este análisis para el negocio?*”
 - “*¿Estamos midiendo la tasa de retención de forma mensual o anual?*”

D.2.2. Paso 2: Formular Hipótesis e Identificar Datos

- **Qué hacer:** Propón las variables y fuentes de datos necesarias. Explica qué supuestos quieres validar.
- **Frases útiles:**
 - *“Mi hipótesis es que los clientes que cancelan su servicio tienen una baja frecuencia de inicio de sesión en los últimos 15 días. Para validar esto, necesitaría acceder a los logs de actividad del usuario y a la tabla de suscripciones.”*

D.2.3. Paso 3: Definir la Metodología Analítica

- **Qué hacer:** Explica paso a paso cómo procesarás y modelarás la información (desde la ingesta hasta la validación del modelo).
- **Frases útiles:**
 - *“Primero realizaría una limpieza para tratar los valores nulos en el historial. Luego, aplicaría un análisis exploratorio (EDA) para visualizar correlaciones. Si el objetivo es predictivo, entrenaría un XGBoost y evaluaría su desempeño utilizando la métrica Recall, porque en este caso los falsos negativos son muy costosos.”*

D.2.4. Paso 4: Traducir al Impacto de Negocio

- **Qué hacer:** Concluye explicando cómo presentarías los resultados a un director de área o stakeholder clave.
- **Frases útiles:**
 - *“Finalmente, presentaría los resultados mediante un dashboard interactivo enfocado en el ahorro financiero estimado al retener al 10 % de los clientes identificados en riesgo por el modelo.”*

D.3. 3. Preguntas de Práctica Comunes y Guías de Respuesta

D.3.1. Caso 1: Diseñar un Pipeline de Ingesta

Pregunta del Entrevistador: *“Nuestra aplicación móvil genera millones de eventos de clics por segundo y queremos analizarlos diariamente para ajustar ofertas. ¿Cómo diseñarías esta arquitectura?”*

- **Enfoque de respuesta:**
 1. Clasifica el procesamiento: Identifica que es un caso mixto (ingesta continua/streaming, análisis en lote/batch).
 2. Propón herramientas del ecosistema en la nube: Sugiere un almacenamiento intermedio de bajo costo (ej. Google Cloud Storage) para almacenar el flujo continuo, y un motor de procesamiento distribuido (ej. PySpark en Databricks) para procesar los datos agregados cada noche.
 3. Añade gobernanza: Menciona la necesidad de un catálogo de datos para indexar estos esquemas semi-estructurados (JSON).

D.3.2. Caso 2: Selección de Métricas Analíticas

Pregunta del Entrevistador: *“Tenemos un modelo para detectar fraudes en transacciones de e-commerce. La precisión general (Accuracy) es del 99 %. ¿Deberíamos ponerlo en producción?”*

■ **Enfoque de respuesta:**

1. Desafía la métrica: Explica que en conjuntos de datos altamente desbalanceados (como el fraude, donde solo el 0.1 % de las transacciones son fraudulentas), la precisión general es una métrica engañosa (un modelo que prediga ‘No Fraude’ para todo tendrá 99.9 % de precisión).
2. Propón métricas alternativas: Sugiere usar **F1-Score**, **Precision** o **Recall** dependiendo del costo relativo de los Falsos Positivos (bloquear a un cliente legítimo) frente a los Falsos Negativos (dejar pasar un fraude).

E. Bibliografía y Recursos Adicionales

Este apéndice recopila las referencias bibliográficas, documentación oficial de herramientas y lecturas recomendadas citadas a lo largo de las diferentes unidades del curso.

E.1. 1. Referencias Bibliográficas

- **Attaran, M. (2017).** *Cloud computing technology: Leveraging the power of the internet to improve business performance*. Journal of Systems and Information Technology, 19(3/4), 232-254. (Referenciado en la introducción a la nube y arquitecturas de almacenamiento).
 - **Kimball, R., & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley & Sons. (Lectura fundamental para la Parte I: Modelado Dimensional y Esquemas de Estrella/Copo de Nieve).
 - **Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000).** *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS. (Metodología analítica base para la Parte II).
-

E.2. 2. Documentación Oficial y Guías de Tecnologías

Para profundizar en las herramientas prácticas utilizadas en los laboratorios del curso, consulta los siguientes recursos en línea:

- **Google Cloud Architecture Framework:** Guías de diseño estructurado para sistemas en la nube de Google. Ver documentación.
 - **Guía de Inicio Rápido de BigQuery:** Documentación de referencia para almacenamiento analítico y consultas a gran escala. Ver documentación.
 - **PySpark SQL and DataFrame API:** Referencia completa para el desarrollo de transformaciones distribuidas a escala. Ver documentación.
 - **dbt (Data Build Tool) Documentation:** Guía para la transformación de datos SQL dentro del Data Warehouse y orquestación. Ver documentación.
-

E.3. 3. Conjuntos de Datos y Casos de Estudio

- **TheLook eCommerce Dataset:** Conjunto de datos público que simula transacciones, usuarios, envíos e inventario de una tienda de ropa en línea. Hospedado en BigQuery Public Data (`bigquery-public-data.thelook_ecommerce`).
- **BigMart Sales Prediction:** Caso de estudio de ventas minoristas utilizado en los laboratorios de modelado predictivo y preprocesamiento de la Parte II.